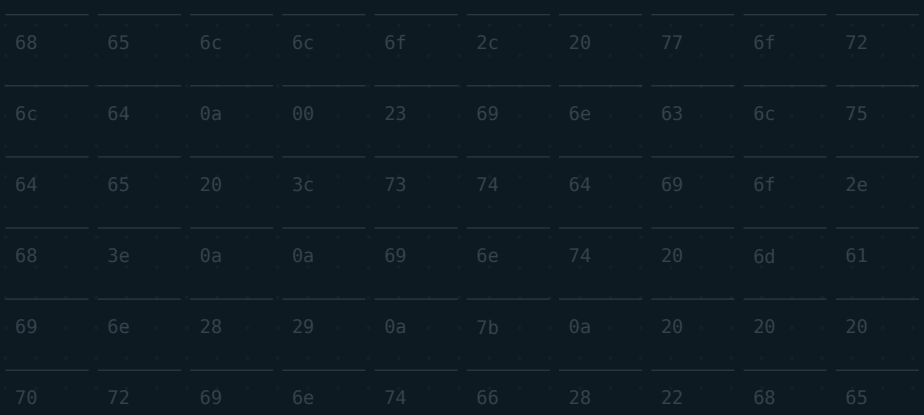
COMPUTER SYSTEMS · THIRD EDITION

深入理解计算机系统

程序员视角下的系统构造

Computer Systems: A Programmer's Perspective, Third Edition

Randal E. Bryant David R. O'Hallaron



第一卷·程序与数据 2026

深入理解计算机系统

程序员视角下的系统构造

第三版

Randal E. Bryant

David R. O'Hallaron

Computer Systems: A Programmer's Perspective

Third Edition

中文重排本

原著：Randal E. Bryant, David R. O'Hallaron,
Computer Systems: A Programmer's Perspective, Third Edition,
Pearson, 2016.

本版为中文重排本：正文、图、表、公式与代码均重新排印，
图由 LaTeX/TikZ 重绘，正文术语与原著第三版一致。
排版使用 XeLaTeX 与 ElegantBook，全部字体嵌入。

2026 年

目录

第 1 章 导论:计算机系统漫游 1

1.1 信息就是位加上下文 2
1.2 程序被其他程序翻译成不同的形式 3
1.3 理解编译系统的工作方式大有裨益 5
1.4 处理器读并解释存储在内存中的指令 6
1.5 高速缓存至关重要 10
1.6 存储设备形成层次结构 12
1.7 操作系统管理硬件 13
1.8 系统之间利用网络通信 18
1.9 重要主题 20
1.10 小结 25
1.11 参考文献说明 26
1.12 练习答案 26

第一部分 程序结构与执行 27

第 2 章 信息的表示和处理 29

2.1 信息存储 31
2.2 整数表示 52
2.3 整数运算 75
2.4 浮点数 96
2.5 小结 111
2.6 参考文献说明 112
2.7 家庭作业 113
2.8 练习答案 127

第 3 章 程序的机器级表示 143

3.1 历史观点 145
3.2 程序编码 148
3.3 机器级代码 149

3.4 代码示例 151
3.5 关于格式的注解 154
3.6 数据格式 155
3.7 访问信息 157
3.8 算术和逻辑操作 166
3.9 控制 174
3.10 过程 208
3.11 数组分配和访问 224
3.12 异质的数据结构 233
3.13 在机器级程序中把控制与数据结合起来 243
3.14 浮点代码 259
3.15 小结 273
3.16 参考文献说明 274
3.17 家庭作业 274
3.18 练习答案 287

第 4 章 处理器体系结构 310
4.1 Y86-64 指令集体系结构 313
4.2 程序员可见的状态 313
4.3 Y86-64 指令 314
4.4 指令编码 316
4.5 Y86-64 异常 321
4.6 Y86-64 程序 322
4.7 逻辑设计和硬件控制语言 HCL 329
4.8 Y86-64 的顺序实现 341
4.9 SEQ 的硬件结构 352
4.10 SEQ 的时序 356
4.11 SEQ 各阶段的实现 358
4.12 流水线的通用原理 366
4.13 计算流水线 366
4.14 流水线操作的详细考察 368
4.15 流水线的局限 370
4.16 带有反馈的系统的流水线化 373
4.17 Y86-64 的流水线实现 374
4.18 SEQ+：重新排列计算阶段 374
4.19 插入流水线寄存器 377

4.20 重新排列和标号信号 379
4.21 预测下一个 PC 381
4.22 流水线冒险 382
4.23 异常处理 397
4.24 PIPE 各阶段的实现 399
4.25 流水线控制逻辑 405
4.26 性能分析 413
4.27 尚未完成的工作 417
4.28 小结 419
4.29 参考文献说明 421
4.30 家庭作业 421
4.31 练习答案 427

第 5 章 优化程序性能 440
5.1 优化编译器的能力和局限性 442
5.2 表示程序性能 445
5.3 程序示例 447
5.4 消除循环的低效率 451
5.5 减少过程调用 454
5.6 消除不必要的存储器引用 456
5.7 理解现代处理器 459
5.8 循环展开 471
5.9 增强并行性 475
5.10 合并代码优化的结果小结 488
5.11 一些限制因素 489
5.12 理解存储器性能 494
5.13 现实世界中的情况:性能改进技术 501
5.14 识别并消除性能瓶颈 502
5.15 小结 508
5.16 参考文献说明 509
5.17 家庭作业 509
5.18 练习题答案 512

第 6 章 存储器层次结构 517
6.1 存储技术 518
6.2 局部性 539

6.3 存储器层次结构 545
6.4 高速缓存存储器 550
6.5 编写高速缓存友好的代码 570
6.6 综合:高速缓存对程序性能的影响 575
6.7 小结 585
6.8 参考文献说明 586
6.9 家庭作业 587
6.10 练习答案 598

第二部分 在系统上运行程序 605

第 7 章 链接 607
7.1 编译器驱动程序 609
7.2 静态链接 610
7.3 目标文件 611
7.4 可重定位目标文件 611
7.5 符号和符号表 613
7.6 符号解析 617
7.7 重定位 627
7.8 可执行目标文件 632
7.9 加载可执行目标文件 634
7.10 动态链接共享库 635
7.11 从应用程序中加载和链接共享库 638
7.12 位置无关代码(PIC) 641
7.13 库打桩机制 644
7.14 处理目标文件的工具 649
7.15 小结 650
7.16 参考文献说明 650
7.17 家庭作业 651
7.18 练习答案 654

第 8 章 异常控制流 656
8.1 异常 657
8.2 进程 667
8.3 系统调用错误处理 673
8.4 进程控制 674

8.5 信号 690
8.6 非局部跳转 713
8.7 操作进程的工具 717
8.8 小结 718
8.9 参考文献说明 718
8.10 家庭作业 719
8.11 练习答案 725

第 9 章 虚拟内存 731
9.1 物理和虚拟寻址 732
9.2 地址空间 733
9.3 虚拟内存作为缓存的工具 734
9.4 虚拟内存作为内存管理的工具 740
9.5 虚拟内存作为内存保护的工具 742
9.6 地址翻译 743
9.7 案例研究：Intel Core i7/Linux 内存系统 756
9.8 内存映射 764
9.9 动态内存分配 770
9.10 垃圾收集 795
9.11 C 程序中常见的与内存有关的错误 800
9.12 小结 805
9.13 参考文献说明 805
9.14 家庭作业 806
9.15 练习答案 810

第三部分 程序间的交互和通信 814

第 10 章系统级 I/O 816
10.1 Unix I/O 817
10.2 文件 818
10.3 打开和关闭文件 819
10.4 读和写文件 821
10.5 用 Rio 包健壮地读写 822
10.6 读取文件元数据 828
10.7 读取目录内容 830
10.8 共享文件 831

10.9 I/O 重定向 834
10.10 标准 I/O 835
10.11 综合：应该使用哪些 I/O 函数？ 836
10.12 小结 838
10.13 参考文献说明 838
10.14 家庭作业 838
10.15 练习答案 839

第 11 章网络编程 841

11.1 客户端-服务器编程模型 841
11.2 网络 842
11.3 全球 IP 因特网 847
11.4 套接字接口 854
11.5 Web 服务器 869
11.6 综合：Tiny Web 服务器 877
11.7 小结 884
11.8 参考文献说明 885
11.9 家庭作业 885
11.10 练习答案 886

第 12 章并发编程 889

12.1 基于进程的并发编程 890
12.2 基于 I/O 多路复用的并发编程 895
12.3 基于线程的并发编程 903
12.4 多线程程序中的共享变量 910
12.5 用信号量同步线程 913
12.6 利用线程实现并行 929
12.7 其他并发问题 935
12.8 小结 944
12.9 参考文献说明 944
12.10 家庭作业 945
12.11 练习答案 949

1

导论：计算机系统漫游

计算机系统（computer system）由硬件和系统软件组成，两者协同工作，运行应用程序（application program）。系统的具体实现随时间不断变化，但内在的概念是不变的。所有计算机系统都有相似的硬件和软件组件，执行相似的功能。本书是为这样一类程序员写的：他们想把自己的技艺再提高一层，为此要理解这些组件如何工作，又如何影响程序的正确性和性能。

你即将踏上一段激动人心的旅程。如果你潜心学习本书中的概念，就会走上成为少见的“强力程序员”（power programmer）之路：透彻理解底层的计算机系统，看它如何影响你的应用程序。

你将学到一些实用技能：如何避免由计算机表示数的方式引起的各种奇怪的数值错误；如何利用现代处理器和存储器系统的设计，用一些巧妙的技巧优化你的 C 代码；编译器（compiler）如何实现过程调用，以及如何利用这些知识避开缓冲区溢出（buffer overflow）漏洞造成的安全缺陷，这类漏洞正困扰着网络和因特网软件；如何识别并避开链接过程中让普通程序员头疼的讨厌错误。你还将学会编写自己的 Unix shell、自己的动态存储分配包，甚至自己的 Web 服务器；并发（concurrency）的希望与陷阱你也会了解，随着多个处理器核心集成到单一芯片上，这个话题正变得日益重要。

在他们关于 C 程序设计语言的经典著作 [61] 中，Kernighan 和 Ritchie 用图 1.1 所示的 `hello` 程序把读者领进 C 的大门。`hello` 是一个非常简单的程序，但要让它完整地跑起来，系统的主要部件都必须协同工作。从某种意义上说，本书的目标就是帮助你理解：在你的系统上运行 `hello` 时发生了什么，以及为什么会如此。

我们对系统的学习，从追踪 `hello` 程序的一生开始：从程序员创建它讲起，直到它在系统上运行，打印出一条简单的消息，然后终止。沿着这个程序一生的轨迹，我们会简要介绍其间登场的关键概念、术语和组件，后面的章节会把这些内容逐一展开。

```
1  #include <stdio.h>
```

```
2
3  int main()
4  {
5      printf("hello, world\n");
6      return 0;
7  }
```

图 1.1 hello 程序(`code/intro/hello.c`)。(来源:[60])

#	i	n	c	l	u	d	e	*SP*	<	s	t	d	i	o	.
35	105	110	99	108	117	100	101	32	60	115	116	100	105	111	46
h	>	\n	\n	i	n	t	*SP*	m	a	i	n	(	)	\n	{
104	62	10	10	105	110	116	32	109	97	105	110	40	41	10	123
\n	*SP*	*SP*	*SP*	*SP*	p	r	i	n	t	f	(	"	h	e	l
10	32	32	32	32	112	114	105	110	116	102	40	34	104	101	108
l	o	,	*SP*	w	o	r	l	d	\	n	"	)	;	\n	*SP*
108	111	44	32	119	111	114	108	100	92	110	34	41	59	10	32
SP	*SP*	*SP*	r	e	t	u	r	n	*SP*	0	;	\n	}	\n	
32	32	32	114	101	116	117	114	110	32	48	59	10	125	10	

图 1.2 (示意图)

1.1 信息就是位加上下文

我们的 `hello` 程序从一段源程序(source program,或称源文件)开始它的生命周期。程序员用编辑器写下这段程序,把它保存为一个叫作 `hello.c` 的文本文件(text file)。源程序由一串位(bit)组成,每个位的取值非 0 即 1,它们按 8 个一组构成字节(byte)。每个字节表示程序中的某个文本字符。

多数计算机系统用 ASCII 标准表示文本字符:每个字符对应一个唯一的、长度为一个字节的整数值。[1]例如,图 1.2 给出了 `hello.c` 程序的 ASCII 表示。

`hello.c` 程序以字节序列的形式存放在文件中,每个字节都是一个整数值,对应某个字符。例如,第一个字节的整数值是 35,对应字符“#”;第二个字节的整数值是 105,对应字符“i”,其余依此类推。注意,每一行文本都以不可见的换行字符(newline)“`\n`”结尾,它的整数值是 10。像 `hello.c` 这样完全由 ASCII 字符构成的文件称为文本文件,其余的文件则称为二进制文件(binary file)。

`hello.c` 的表现形式揭示了一个基本思想:系统中的所有信息(包括磁盘文件、存放在内存中的程序、存放在内存中的用户数据,以及经由网络传输的数据)都是用一串位表示的。区分不同数据对象的唯一依据,是我们看待这些位的方式,也就是上下文(context)。例如,放在不同的上下文里,同一串字节可以

[1]非英语语言的文本使用另外的编码方法,相关讨论见 2.1 节中的边栏。

表示一个整数、一个浮点数(floating point)、一个字符串,也可以表示一条机器指令(machine instruction)。

作为程序员,我们必须理解数字的机器表示,因为这种表示与数学意义上的整数和实数并不相同。它们只是有限近似值,行为可能出乎意料。这一基本思想将在第 2 章中详细探讨。

边栏 C 程序设计语言的起源

C 语言由贝尔实验室的 Dennis Ritchie 在 1969 年至 1973 年间开发。美国国家标准学会(ANSI)于 1989 年批准了 ANSI C 标准,此后这项标准化工作转由国际标准化组织(ISO)负责。这些标准定义了 C 语言本身,以及一组称为 C 标准库(C standard library)的库函数。Kernighan 和 Ritchie 在他们的经典著作中描述了 ANSI C,这本书被亲切地称为“K&R”[61]。用 Ritchie 自己的话 [92] 说,C“古怪、有缺陷,但取得了巨大的成功”。那么,成功从何而来?

- **C 与 Unix 操作系统(operating system)紧密相连。** C 从一开始就是作为 Unix 的系统级程序设计语言开发的。Unix 内核(操作系统的核心部分)的大部分代码,连同其全部配套工具和库,都用 C 写成。20 世纪 70 年代末到 80 年代初,Unix 在各大学流行起来,许多人由此接触到 C,并发现自己喜欢上了它。既然 Unix 几乎完全用 C 写成,把它移植到新机器上就很容易,这让 C 和 Unix 都赢得了更广泛的用户。
- **C 是一门小而简单的语言。** 它的设计由一个人主导,而不是出自委员会,因此设计干净、一致,几乎没有包袱。K&R 的书仅用 261 页,连同大量实例和练习,就完整描述了整个语言和标准库。C 的简单使它相对容易学习,也容易移植到不同的计算机上。
- **C 是为实际目的而设计的。** 设计 C 是为了实现 Unix 操作系统。后来,人们发现用它写自己想要的程序时,语言本身并不碍事。

C 是系统级程序设计的首选语言,在应用程序层面也有庞大的存量程序。但 C 并不能让所有程序员在所有场合都满意:C 的指针是引起混乱和编程错误的常见根源;C 也缺少对类、对象、异常这类有用抽象的显式支持。C++ 和 Java 等较新的语言,正是在应用程序层面回应了这些问题。

1.2 程序被其他程序翻译成不同的形式

`hello` 程序最初以高级 C 程序的形式面世,因为人能够直接阅读和理解这种形式。不过,要在系统上运行 `hello.c`,其中一条条 C 语句都必须经由其他程序翻译,变成一系列低级的机器语言指令。这些指令随后被打包成一种叫作可执行目标程序(executable object program)的形式,并以二进制磁盘文件的形式存放。目标程序也叫可执行目标文件(executable object file)。

在 Unix 系统上,从源文件到目标文件的翻译由编译器驱动程序(compiler driver)完成:

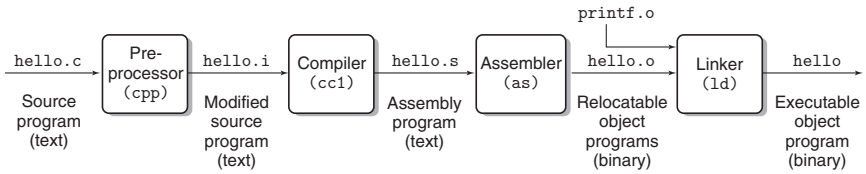


图 1.3 （示意图）

```
linux> gcc -o hello hello.c
```

这里，gcc 编译器驱动程序读取源文件 hello.c，把它翻译成可执行目标文件 hello，翻译过程按图 1.3 所示的四个阶段依次进行。执行这四个阶段的程序（预处理器（preprocessor）、编译器、汇编器（assembler）、链接器（linker））合起来称为编译系统（compilation system）。

- **预处理阶段**。 预处理器（cpp）根据以字符“#”开头的命令（directive）修改原始的 C 程序。例如，hello.c 第 1 行的 #include <stdio.h> 命令告诉预处理器：读入系统头文件 stdio.h 的内容，并直接插入程序文本。结果得到另一个 C 程序，通常以 .i 作为文件扩展名。
- **编译阶段**。 编译器（cc1）把文本文件 hello.i 翻译成文本文件 hello.s，其中包含一个汇编语言（assembly language）程序。这个程序含有对 main 函数的如下定义：

```
1  main:
2    subq    $8, %rsp
3    movl    $.LC0, %edi
4    call    puts
5    movl    $0, %eax
6    addq    $8, %rsp
7    ret
```

这个定义中的第 2 ~ 7 行，每行都以文本形式描述一条低级机器语言指令。汇编语言之所以有用，是因为对不同高级语言的各种编译器来说，它提供了一种共同的输出语言。例如，C 编译器和 Fortran 编译器生成的输出文件，用的都是同一种汇编语言。

- **汇编阶段**。 接下来，汇编器（as）把 hello.s 翻译成机器语言指令，把它们打包成一种叫作可重定位目标程序（relocatable object program）的形式，并把结果保存在目标文件 hello.o 中。这个文件是一个二进制文件，包含 17 个字节，用来对 main 函数的指令进行编码。如果用文本编辑器打开 hello.o，看到的就是一片乱码。
- **链接阶段**。 注意，我们的 hello 程序调用了 printf 函数，它是每个 C 编译器都会提供的标准 C 库的一部分。printf 函数存放在一个单独的预编

译目标文件 printf.o 中,而这个文件必须以某种方式与我们的 hello.o 程序合并。链接器(ld)负责完成这项合并。结果就是 hello 文件,它是一个可执行目标文件(或简称可执行文件),随时可以加载到内存中,由系统执行。

1.3 理解编译系统的工作方式大有裨益

对于 hello.c 这样简单的程序,我们可以依赖编译系统生成正确且高效的机器码(machine code)。不过,有几条重要的理由说明,程序员为什么需要理解编译系统是如何工作的:

- **优化程序性能。** 现代编译器都是成熟的工具,通常能生成不错的代码。作为程序员,我们不必了解编译器的内部工作机制,也能写出高效的代码。但是,要想在 C 程序中做出好的编码决策,确实需要对机器级代码(machine-level code)以及编译器如何把不同的 C 语句翻译成机器代码有基本的理解。例如:switch 语句是否总比一连串 if-else 语句更高效?一次函数调用会带来多少开销?while 循环是否比 for 循环更高效?指针引用是否比数组下标更高效?把求和结果写进一个局部变量、而不是写进一个按引用传递的参数,为什么循环会快得多?只是重新排列一下算术表达式中的括号,函数为什么就能跑得更快?

  在第 3 章中,我们将介绍 x86-64——近几代 Linux、Macintosh 和 Windows 计算机所使用的机器语言,并说明编译器如何把不同的 C 语言构造(construct)翻译成这种语言。第 5 章将教你通过对 C 代码做一些简单的变换来优化 C 程序的性能,这些变换能帮助编译器更好地完成工作。第 6 章将介绍存储器系统的层次结构本质,C 编译器如何在内存中存放数据数组,以及你的 C 程序如何利用这些知识来更高效地运行。

- **理解链接时错误。** 依我们的经验,一些最令人困惑的程序设计错误都和链接器的操作有关,尤其是当你试图构建大型软件系统时。例如,当链接器报告无法解析某个引用时,这是什么意思?静态变量和全局变量有什么区别?如果在两个不同的 C 文件里定义了同名的全局变量,会发生什么?静态库和动态库有什么区别?为什么在命令行上列出库的顺序很重要?最吓人的是,为什么有些与链接器相关的错误直到运行时才会出现?第 7 章会讲清楚这类问题的答案。

- **避免安全漏洞。** 多年来,缓冲区溢出漏洞造成了网络和因特网服务器上的许多安全漏洞。这类漏洞之所以存在,是因为很少有程序员认识到,必须谨慎限制从不可信来源接受的数据的数量和形式。学习安全程序设计的第一

边栏　GNU 项目

gcc 是 GNU 项目开发的许多有用工具之一。GNU(“GNU's Not Unix”的缩写)项目由 Richard Stallman 于 1984 年发起,是一个免税的慈善机构,其目标雄心勃勃:开发一个完整的类 Unix 系统,允许任何人自由地修改和分发其源代码。GNU 项目开发出的环境包含了 Unix 操作系统的所有主要组件,唯独缺内核——内核由 Linux 项目单独开发。这个 GNU 环境包括 emacs 编辑器、gcc 编译器、gdb 调试器、汇编器、链接器、处理二进制文件的工具程序,以及其他一些组件。gcc 编译器发展到今天,已支持许多不同的语言,并能为许多不同的机器生成代码,支持的语言包括 C、C++、Fortran、Java、Pascal、Objective-C 和 Ada。GNU 项目是一项了不起的成就,却常被忽视。现代开源运动(人们通常把它与 Linux 联系起来)在思想渊源上源自 GNU 项目的自由软件(free software)理念(“自由”指言论自由的那种自由,不是免费啤酒的那种免费)。此外,Linux 之所以广受欢迎,很大程度上要归功于 GNU 工具,正是这些工具为 Linux 内核提供了环境。

步,是理解数据和控制信息存放在程序栈上的方式会带来什么后果。第 3 章将在讲汇编语言时讨论栈的规则(stack discipline)和缓冲区溢出漏洞。我们还会学到程序员、编译器和操作系统可以用来降低攻击威胁的方法。

1.4 处理器读并解释存储在内存中的指令

此刻,我们的 `hello.c` 源程序已经被编译系统翻译成一个叫作 `hello` 的可执行目标文件,存放在磁盘上。要在 Unix 系统上运行这个可执行文件,我们在一个叫作 shell 的应用程序里输入它的名字:

```
linux> ./hello
hello, world
linux>
```

shell 是一个命令行解释器(command-line interpreter),它打印出一个提示符,等待你输入一个命令行,然后执行这条命令。如果命令行的第一个词不是内置的 shell 命令,shell 就会认为这是一个应当加载并运行的可执行文件的名字。于是在这个例子里,shell 加载并运行 `hello` 程序,然后等待它终止。`hello` 程序在屏幕上打印出它的消息,然后终止。接着,shell 打印出提示符,等待下一条输入的命令行。

1.4.1 系统的硬件组织

要理解 `hello` 程序在运行时发生了什么,就需要理解一个典型系统的硬件组织,如图 1.4 所示。这幅图以近几代 Intel 系统家族为模型,但所有系统的外观和感受都是相似的。现在不必担心这幅图的复杂性,随着本书学习的展开,我们会分阶段讲到它的各种细节。

总线

贯穿整个系统的是一组称为总线(bus)的电子管道,它们在部件之间来回传送字节信息。总线通常被设计成传送固定大小的字节块,这样的字节块称为字(word)。字中的字节数称为字长(word size),它是一个基本的系统参数,因系统而异。如今大多数机器的字长要么是 4 个字节(32 位),要么是 8 个字节(64 位)。本书不对字长做任何固定的假设。在需要明确定义“字”的地方,我们会具体说明这个词的含义。

I/O 设备

输入/输出(I/O)设备是系统与外部世界的联系。我们的示例系统有四个 I/O 设备:供用户输入用的键盘和鼠标、供用户输出用的显示器,以及用于长期存

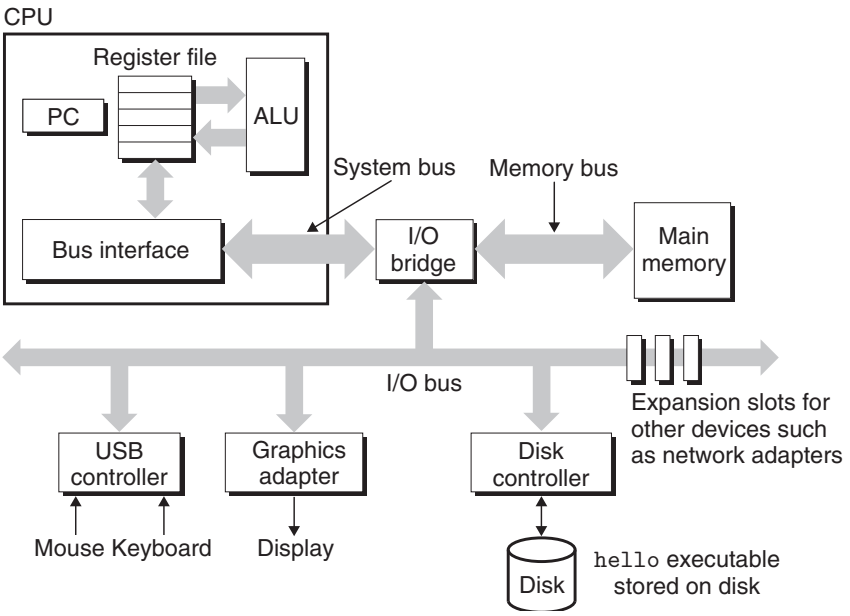


图 1.4 （示意图）

放数据和程序的磁盘驱动器(disk drive,简称磁盘)。最初,可执行的 `hello` 程序就存放在磁盘上。

每个 I/O 设备都通过一个控制器(controller)或适配器(adapter)与 I/O 总线相连。两者的区别主要在于封装方式:控制器是设备本身或系统主印刷电路板(通常称为主板)上的芯片组,而适配器则是插在主板插槽上的一块卡。不管是哪一种,其目的都是在 I/O 总线和 I/O 设备之间来回传送信息。

第 6 章会更详细地介绍磁盘这类 I/O 设备的工作原理。第 10 章将教你如何在应用程序中使用 Unix I/O 接口来访问设备。我们会重点讨论网络这一类特别有趣的设备,不过这些技术同样适用于其他类型的设备。

主存

主存是一个临时存储设备,在处理器执行程序时,用来存放程序本身及其处理的数据。从物理上看,主存由一组动态随机存取存储器(DRAM)芯片组成。从逻辑上看,存储器是一个线性的字节数组,每个字节都有自己唯一的地址(数组下标),下标从 0 开始。一般说来,构成程序的每条机器指令都可以由不同数量的字节组成。与 C 程序变量相对应的数据项的大小随类型而异。例如,在运行 Linux 的 x86-64 机器上,`short` 类型的数据需要 2 个字节,`int` 和 `float` 类型需要 4 个字节,`long` 和 `double` 类型需要 8 个字节。

第 6 章会更详细地介绍 DRAM 芯片这类存储器技术的工作原理,以及它们如何组合起来构成主存。

处理器

中央处理单元（CPU），简称处理器，是解释（或执行）存放在主存中的指令的引擎。处理器的核心是一个字大小的存储设备（或寄存器），称为程序计数器（PC）。在任何时刻，PC 都指向主存中的某条机器语言指令（也就是说，PC 的内容是该指令的地址）。[2]

从系统通电的那一刻起到断电为止，处理器不断重复两件事：执行程序计数器指向的指令，并更新程序计数器，使其指向下一条指令。处理器看起来是按照一个非常简单的指令执行模型来操作的，这个模型由它的指令集体系结构（ISA）定义。在这个模型中，指令严格按照先后顺序执行，而执行一条指令又包括一系列步骤。处理器从内存中读出 PC 所指向的指令，解释指令中的位，执行该指令所要求的某个简单操作，然后更新 PC，使其指向下一条指令——这条指令在内存中不一定紧挨着刚执行过的那条指令。

这样的简单操作为数不多，它们都围绕着主存、寄存器堆（register file）和算术/逻辑单元（ALU）进行。寄存器堆是一个很小的存储设备，由若干个字大小的寄存器组成，每个寄存器都有自己唯一的名字。ALU 负责计算新的数据和地址值。下面举例说明 CPU 应某条指令的要求可能执行哪些简单操作：

- **加载**：从主存复制一个字节或者一个字到寄存器，覆盖寄存器原来的内容。
- **存储**：从寄存器复制一个字节或者一个字到主存的某个位置，覆盖该位置原来的内容。
- **运算**：把两个寄存器的内容复制到 ALU，对这两个字执行一次算术运算，并把结果存入一个寄存器，覆盖该寄存器原来的内容。
- **跳转**：从指令本身抽取一个字，并把这个字复制到 PC 中，覆盖 PC 原来的值。

我们说处理器看起来是它的指令集体系结构的简单实现，但事实上，现代处理器使用了复杂得多的机制来加速程序的执行。因此，我们可以把处理器的指令集体系结构——描述每条机器码指令的效果——和它的微体系结构（microarchitecture，描述处理器实际上是如何实现的）区分开。第 3 章学习机器码时，我们将着重于机器的指令集体系结构所提供的抽象。第 4 章会更详细地介绍处理器实际上是如何实现的。第 5 章描述了一个现代处理器工作方式的模型，利用它可以预测并优化机器语言程序的性能。

[2] PC 也是“personal computer（个人计算机）”这一说法常用的缩写，不过二者的区别从上下文应当能够看清楚。

1.4.2 运行 hello 程序

有了对系统硬件组织和操作的这样一种简单认识,我们可以开始理解运行示例程序时会发生什么了。这里有很多细节必须省略,留待后面补充,但眼下我们只关心整体图景。

开始时,shell 程序正在执行它的指令,等待我们输入一条命令。当我们在键盘上键入字符`./hello`时,shell 程序把这些字符逐个读入寄存器,再逐个存放到内存中,如图 1.5 所示。

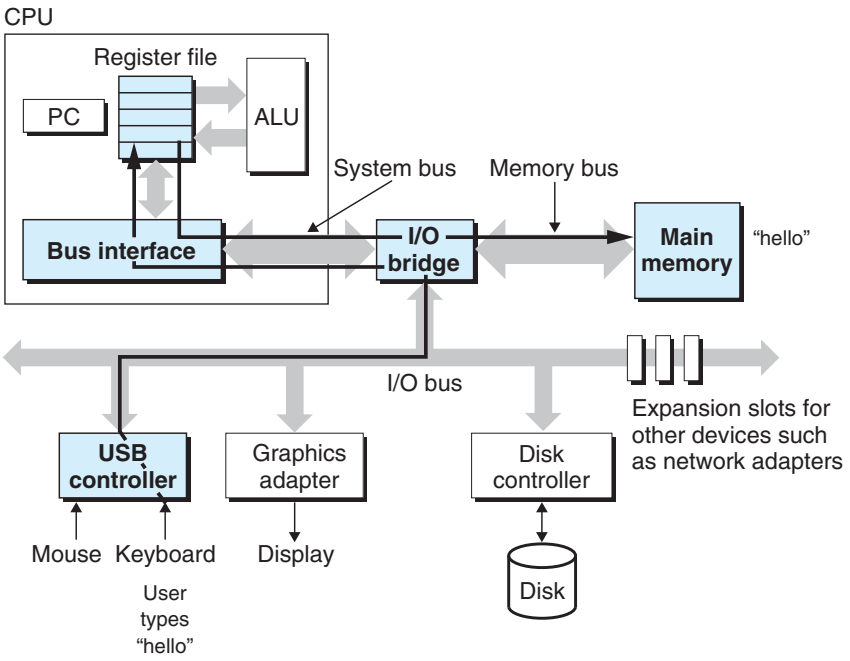


图 1.5 （示意图）

当我们在键盘上按下回车键时,shell 就知道我们已经输入完了命令。然后,shell 通过执行一系列指令来加载可执行文件`hello`,这些指令把`hello`目标文件中的代码和数据从磁盘复制到主存。数据中就包括最终要被打印出来的字符串`hello, world\n`。

利用一种叫作直接内存访问(DMA,将在第 6 章讨论)的技术,数据可以不经过处理器,直接从磁盘传送到主存。这一步骤如图 1.6 所示。

`hello`目标文件中的代码和数据被加载到内存之后,处理器就开始执行`hello`程序的`main`例程中的机器语言指令。这些指令把`hello, world\n`字符串中的字节从内存复制到寄存器堆,再从寄存器堆复制到显示设备,最终显示在屏幕上。这一步骤如图 1.7 所示。

1.5 高速缓存至关重要

这个简单的例子揭示了一条重要经验:系统要花大量时间把信息从一个地方搬到另一个地方。hello 程序中的机器指令最初存放在磁盘上。程序被加载时,这些指令被复制到主存。处理器运行程序时,指令又会从主存被复制到处理器中。类似地,数据串 hello,world\n 最初也在磁盘上,先被复制到主存,然后又从主存复制到显示设备。从程序员的角度看,这些复制操作大部分都是开销,会拖慢程序"真正的工作"。因此,系统设计者的一个主要目标,就是让这些复制操作尽可能快地完成。

由于物理定律,容量较大的存储设备总是比容量较小的存储设备慢。而且,速度更快的设备,其制造成本也高于速度较慢的同类设备。

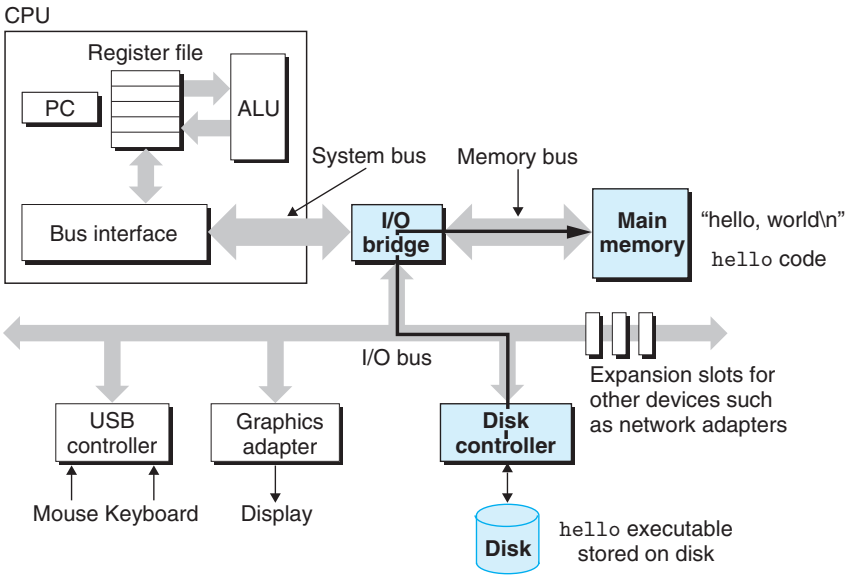


图 1.6 (示意图)

例如,典型系统中的磁盘驱动器可以比主存大 1,000 倍,但处理器从磁盘上读出一个字所花的时间,却可能是从主存读出同一字时的 10,000,000 倍。

同样,典型的寄存器堆只能存放几百字节的信息,而主存却能存放数十亿字节。不过,处理器从寄存器堆读取数据的速度,几乎比从主存读取快 100 倍。更麻烦的是,随着半导体技术这些年的进步,处理器与主存之间的差距还在不断扩大。让处理器跑得更快,比让主存跑得更快更容易,成本也更低。

为了弥合处理器与主存之间的差距,系统设计者加入了更小、更快的存储设备,称为高速缓存存储器(cache memory,简称高速缓存),用来作为处理器近期可能要用到的信息的临时中转站。图 1.8 给出了典型系统中的高速缓存存储

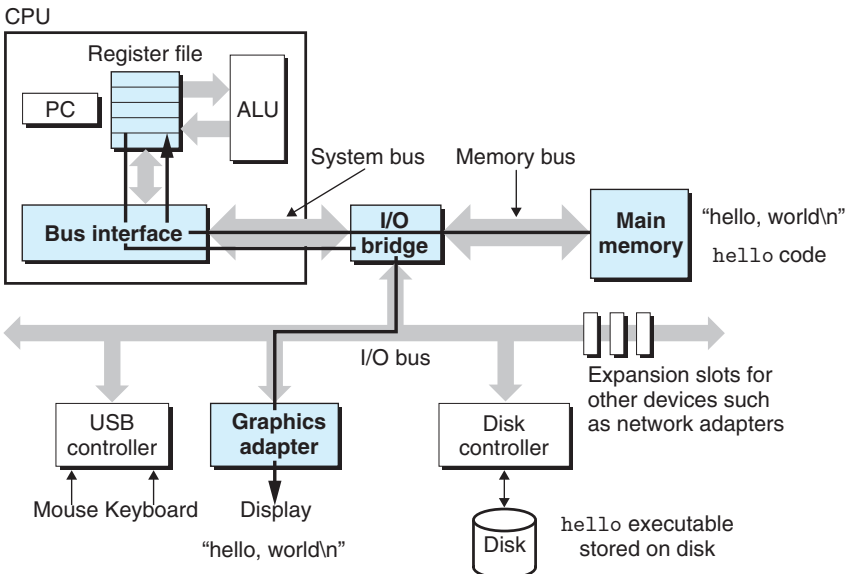


图 1.7 （示意图）

器。处理器芯片上的 L1 高速缓存可以存放数万字节,访问速度接近寄存器堆。更大的 L2 高速缓存可以存放数十万到数百万字节,经由一条专用总线与处理器相连。处理器访问 L2 高速缓存也许要花 5 倍于 L1 的时间,但仍比访问主存快 5 到 10 倍。L1 和 L2 高速缓存用一种称为静态随机访问存储器(SRAM)的硬件技术实现。更新、更强大的系统甚至有三层高速缓存:L1、L2 和 L3。缓存背后的思想是:程序倾向于访问集中在局部区域的数据和代码,这种特性称为局部性(locality);利用局部性,系统就能同时获得大容量存储器与高速存储器两方面的效果。通过让高速缓存存放很可能被频繁访问的数据,大多数存储器操作都可以由这些快速的高速缓存来完成。

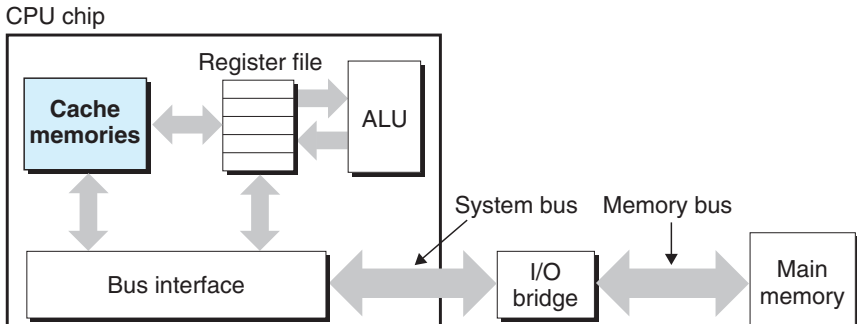


图 1.8 （示意图）

本书最重要的经验之一：懂得高速缓存的程序员能够利用它，把自己的程序性能提高一个数量级。第 6 章会进一步介绍这些重要设备，以及如何利用它们。

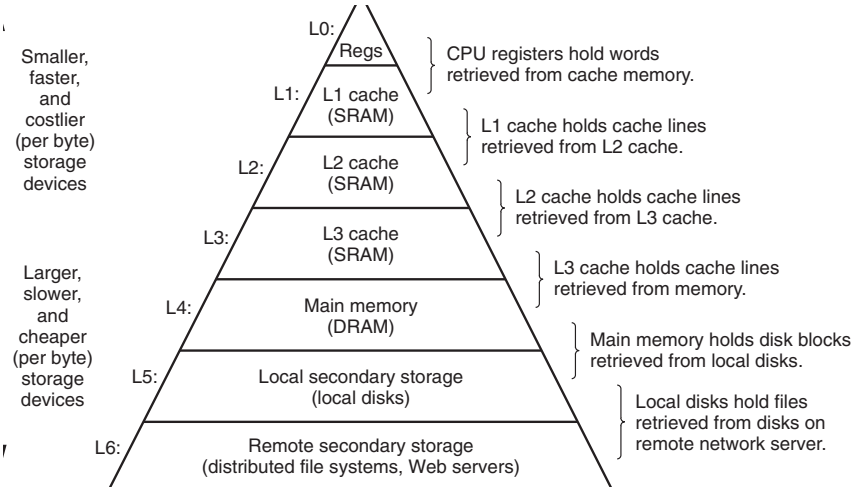


图 1.9 （示意图）

1.6 存储设备形成层次结构

在处理器与更大、更慢的设备（例如主存）之间插入更小、更快的存储设备（例如高速缓存存储器），事实证明是一个具有普遍性的想法。实际上，每台计算机系统中的存储设备都组织成存储器层次结构（memory hierarchy），与图 1.9 类似。从层次结构顶端走向底部，设备越来越慢、越来越大，每字节的成本也越来越低。寄存器堆占据层次结构的顶层，称为第 0 层，即 L0。我们给出 L1 到 L3 三层高速缓存，占据存储器层次结构的第 1 到第 3 层。主存占据第 4 层，依此类推。

存储器层次结构的主要思想是：某一层上的存储器为下一层存储器充当高速缓存。于是，寄存器堆是 L1 高速缓存的高速缓存，L1 和 L2 高速缓存分别是 L2 和 L3 的高速缓存，L3 高速缓存是主存的高速缓存，而主存又是磁盘的高速缓存。在某些带有分布式文件系统（distributed file system）的联网系统中，本地磁盘就为存放在其他系统磁盘上的数据充当高速缓存。

程序员既能利用对不同高速缓存的了解来提高性能，也能利用对整个存储器层次结构的理解来提高性能。第 6 章会就这一点展开更多讨论。

1.7 操作系统管理硬件

回到我们的 hello 例子。当 shell 加载并运行 hello 程序、hello 程序打印出消息时，没有哪个程序直接访问键盘、显示器、磁盘或主存，它们依靠的都是操作系统提供的服务。可以把操作系统看成夹在应用程序和硬件之间的一层软件，如图 1.10 所示。应用程序想操作硬件，任何尝试都必须经过操作系统。

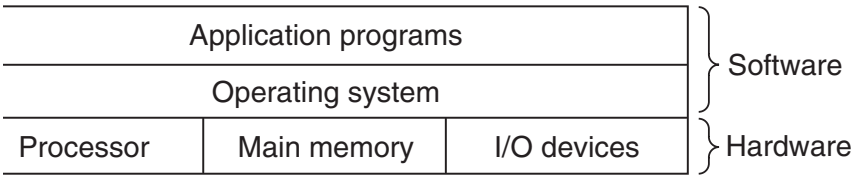


图 1.10 （示意图）

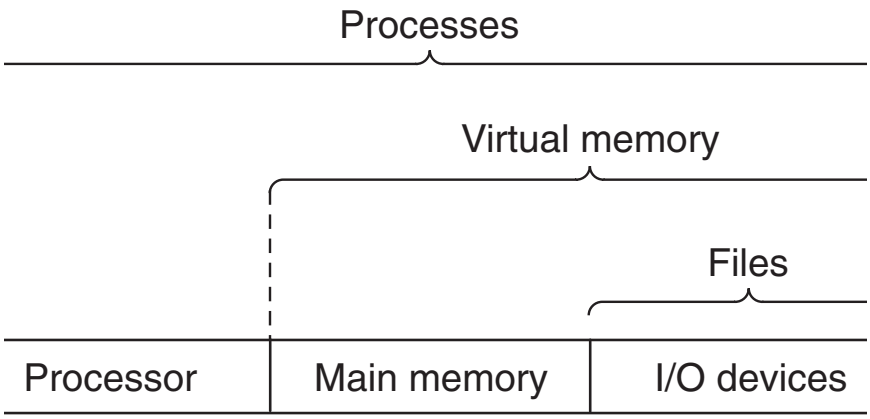


图 1.11 （示意图）

操作系统有两个首要目的：(1)保护硬件，不让失控的应用程序滥用它；(2)为应用程序提供简单而一致的机制，去操作那些复杂而且往往千差万别的底层硬件设备。操作系统通过图 1.11 所示的基本抽象来实现这两个目标：进程(process)、虚拟内存(virtual memory)和文件(file)。如图所示，文件是对 I/O 设备的抽象，虚拟内存既是对主存的抽象也是对磁盘 I/O 设备的抽象，而进程则是对处理器、主存和 I/O 设备的抽象。下面逐一讨论。

1.7.1 进程

当像 hello 这样的程序在现代系统上运行时，操作系统营造出一种假象，好像系统中只有它在运行：程序似乎独占着处理器、主存和 I/O 设备；处理器似乎在连续不断地依次执行程序中的指令；程序中的代码和数据似乎也是系统存

储器中仅有的对象。这些假象都来自进程这个概念——计算机科学中最重要、最成功的思想之一。

进程是操作系统对一个正在运行的程序所建立的抽象。同一系统上可以有多个进程并发地运行,每个进程看起来都独占着硬件。所谓并发(concurrent),是指一个进程的指令与另一个进程的指令交错执行。在大多数系统中,需要运行的进程比能运行它们的处理器还要多。

边栏 Unix、Posix 和标准 Unix 规范

20 世纪 60 年代是庞大而复杂的操作系统的年代,例如 IBM 的 OS/360 和霍尼韦尔(Honeywell)公司的 Multics 系统。OS/360 是历史上最成功的软件项目之一,Multics 拖了多年却始终没有获得广泛应用。贝尔实验室本是 Multics 项目最初的合作方之一,由于担心项目过于复杂、进展缓慢,于 1969 年退出。也许正是 Multics 项目留下的这段不愉快经历,促使贝尔实验室的一组研究人员——Ken Thompson、Dennis Ritchie、Doug McIlroy 与 Joe Ossanna——于 1969 年开始为数字设备公司(Digital Equipment Corporation)的 PDP-7 计算机编写一个更简单的操作系统,完全用机器语言写成。新系统的许多思想,比如层次式文件系统以及把 `shell` 作为用户级进程的概念,都借自 Multics,只是被实现成一个更小、更简单的系统。1970 年,Brian Kernighan 把这个新系统命名为 Unix,作为对 Multics 复杂性的一个双关。内核于 1973 年用 C 语言重写,Unix 则于 1974 年公之于世 [93]。

由于贝尔实验室以优厚的条件向大学提供源代码,Unix 在大学里赢得了大批追随者。最有影响的工作是 20 世纪 70 年代末与 80 年代初在加州大学伯克利分校完成的,伯克利的研究者在一系列称为 Unix 4.xBSD(Berkeley Software Distribution,伯克利软件发布版)的版本中加入了虚拟内存和因特网协议。同一时期,贝尔实验室也在陆续发布自己的版本,后来被称为 System V Unix。其他厂商的版本,例如 Sun Microsystems 的 Solaris 系统,则从最初的 BSD 和 System V 版本衍生而来。

麻烦在 20 世纪 80 年代中期出现了:各 Unix 厂商纷纷通过增加新的、常常互不兼容的特性来标新立异。为了遏制这种趋势,IEEE(电气与电子工程师学会,Institute of Electrical and Electronics Engineers)发起了一项将 Unix 标准化的工作,后来由 Richard Stallman 命名为 Posix。其成果是一族称为 Posix 标准的标准,覆盖 Unix 系统调用的 C 语言接口、`shell` 程序与实用工具、线程以及网络编程等方面。近来,又有一项独立的标准工作——标准 Unix 规范(Standard Unix Specification)——与 Posix 联手,为 Unix 系统建立了单一的统一标准。在这些标准化努力之后,Unix 各版本之间的差异已基本消失。

传统系统一次只能执行一个程序,而较新的多核处理器(multi-core processor)可以同时执行几个程序。无论哪种情形,单个 CPU 都可以通过在各进程之间来回切换,呈现出并发执行多个进程的外观。操作系统用一个称为上下文切换(context switching)的机制来完成这种交错。为简化后文的讨论,我们只考虑包含单个 CPU 的单处理器(uniprocessor)系统。关于多处理器系统的讨论将在 1.9.2 节继续。

操作系统会记录进程运行所需的全部状态信息。这些状态称为上下文,包括 PC 的当前值、寄存器堆的当前值和主存的内容等信息。在任一时刻,单处理器系统都只能执行单个进程的代码。当操作系统决定把控制权从当前进程转交给某个新进程时,就会执行一次上下文切换:保存当前进程的上下文,恢复新进程的上下文,然后

随后把控制权交给新进程。新进程正好从上次离开的地方继续执行。图 1.12 展示了 `hello` 示例场景的基本思想。

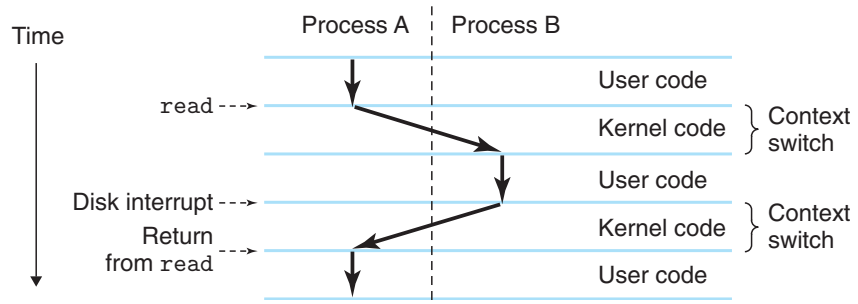


图 1.12 （示意图）

在我们的示例场景中,有两个并发执行的进程:shell 进程和 `hello` 进程。起初,shell 进程独自运行,在命令行上等待输入。当我们要求它运行 `hello` 程序时,shell 通过调用一个特殊的函数来完成这一请求,这个函数称为系统调用(system call),它把控制权交给操作系统。操作系统保存 shell 的上下文,创建新的 `hello` 进程及其上下文,然后把控制权交给新创建的 `hello` 进程。`hello` 终止之后,操作系统恢复 shell 进程的上下文,并把控制权交回给 shell,由它继续等待下一条命令行输入。

如图 1.12 所示,从一个进程到另一个进程的转换由操作系统内核管理。内核是操作系统代码中常驻主存的那一部分。当应用程序需要操作系统替它执行某个操作(比如读文件或写文件)时,它会执行一条特殊的系统调用指令,把控制权交给内核。内核随后执行所请求的操作,再返回到应用程序。注意,内核并不是一个单独的进程。相反,它是系统用来管理所有进程的一组代码和数据结构的集合。

实现进程抽象需要底层硬件与操作系统软件之间的紧密配合。第 8 章将探讨这种配合如何实现,以及应用程序如何创建和控制自己的进程。

1.7.2 线程

尽管我们通常把进程看作一条单独的控制流,但在现代系统中,一个进程实际上可以由多个执行单元组成,这些执行单元称为线程(thread)。每个线程

都运行在进程的上下文之中,共享相同的代码和全局数据。线程之所以正在成为越来越重要的编程模型,是因为网络服务器对并发的需求,因为多个线程之间共享数据比多个进程之间更容易,也因为线程通常比进程更高效。在多个处理器可用的场合,多线程也是让程序运行得更快的一种途径,这一点我们将在 1.9.2 节讨论。你会在第 12 章学到并发的基本概念,包括如何编写多线程程序。

1.7.3 虚拟内存

虚拟内存是一个抽象,它让每个进程都觉得自己在独占地使用主存。每个进程看到的都是同样一致的存储器视图,这一视图称为它的虚拟地址空间(virtual address space)。Linux 进程的虚拟地址空间如图 1.13 所示(其他 Unix 系统的布局与此类似)。在 Linux 中,地址空间最顶端的区域保留给操作系统中为所有进程所共有的代码和数据;地址空间的低端区域存放用户进程定义的代码和数据。注意,图中的地址自下而上递增。

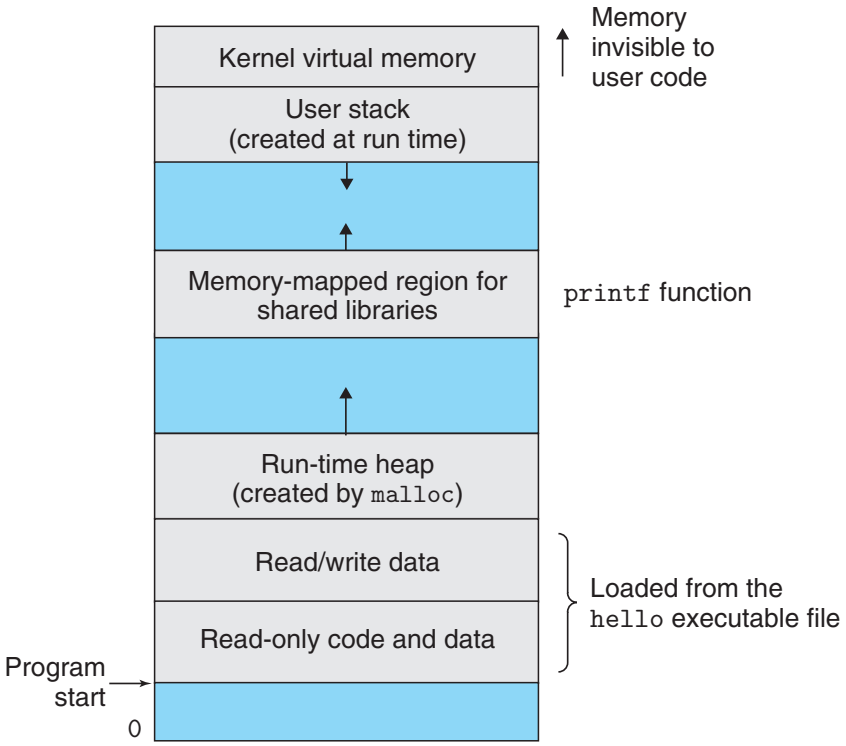


图 1.13 (示意图)

每个进程看到的虚拟地址空间由若干界线分明、用途各异的区域组成。本书后面会进一步讨论这些区域,而从最低地址开始,自下而上把它们逐一简要看一遍是很有帮助的:

- **程序代码和数据**。 对所有进程来说,代码都从同一个固定地址开始,接着是保存 C 全局变量的数据位置。代码和数据区域直接由可执行目标文件的内容初始化——在我们的例子里,就是 `hello` 可执行文件。第 7 章学习链接和加载时,你会更深入地了解地址空间的这一部分。
- **堆**。代码和数据区域之后紧接着是运行时堆(run-time heap)。与代码和数据区域不同,堆的大小在进程开始运行之后并不固定:随着程序调用 `malloc`、`free` 这类 C 标准库例程,堆在运行时动态地扩张和收缩。学习第 9 章中管理虚拟内存的内容时,我们会详细研究堆。
- **共享库**。 地址空间中部附近有一块区域,存放共享库(shared library)的代码和数据,例如 C 标准库和数学库。共享库是一个功能强大但有些难以掌握的概念。第 7 章学习动态链接时,你会了解它们的工作原理。
- **栈**。 用户虚拟地址空间的顶部是用户栈(user stack),编译器用它来实现函数调用。与堆一样,用户栈在程序执行期间动态地扩张和收缩。具体地说,我们每调用一个函数,栈就随之增长;每从一个函数返回,栈就随之收缩。第 3 章会讲解编译器如何使用栈。
- **内核虚拟内存**。 地址空间最顶端的区域保留给内核。应用程序不允许读写这个区域的内容,也不允许直接调用内核代码中定义的函数;要执行这些操作,它们必须请求内核代劳。

虚拟内存要运转起来,硬件与操作系统软件之间必须进行复杂的协作,其中包括对处理器生成的每一个地址做硬件转换。基本思想是:把进程虚拟内存的内容存放在磁盘上,再用主存作为磁盘的高速缓存。第 9 章会解释这一切如何运作,以及它对现代系统的运行为何如此重要。

1.7.4 文件

文件就是字节序列,仅此而已。每一个 I/O 设备——包括磁盘、键盘、显示器,甚至网络——都被建模为文件。系统中所有的输入输出都是通过读写文件完成的,所用的是一组称为 Unix I/O 的系统调用。

文件这个简单而优雅的概念却非常强大,因为它向应用程序提供了关于系统中各种各样 I/O 设备的统一视图。例如,处理磁盘文件内容的应用程序员完全不必了解具体的磁盘技术。而且,同一个程序还能运行在使用不同磁盘技术的其他系统上。第 10 章将介绍 Unix I/O。

1.8 系统之间利用网络通信

至此,在我们的系统漫游中,一直把系统当作一个孤立的硬件与软件集合。实际上,现代系统常常通过网络与其他系统相连。从一个单独系统的角度看,网络不过是又一个 I/O 设备,如图 1.14 所示。当系统把一串字节从主存复制到网络适配器(network adapter)时,数据会经过网络流向另一台机器,而不是流向本地磁盘驱动器这类设备。类似地,系统可以读取其他机器发送来的数据,并把这些数据复制到自己的主存中。

边栏 Linux 项目
1991 年 8 月,芬兰一位名叫 Linus Torvalds 的研究生谦逊地宣布了一个新的类 Unix 操作系统内核:

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Date: 25 Aug 91 20:57:08 GMT

Hello everybody out there using minix -
I'm doing a (free) operating system (just a hobby, won't be big and
professional like gnu) for 386(486) AT clones. This has been brewing
since April, and is starting to get ready. I'd like any feedback on
things people like/dislike in minix, as my OS resembles it somewhat
(same physical layout of the file-system (due to practical reasons)
among other things).

I've currently ported bash(1.08) and gcc(1.40), and things seem to
work.
This implies that I'll get something practical within a few months,
and
I'd like to know what features most people would want. Any suggestions
are welcome, but I won't promise I'll implement them :-)

Linus (torvalds@kruuna.helsinki.fi)
```

正如 Torvalds 所说,他着手创建 Linux 的起点是 Minix——Andrew S. Tanenbaum 为教学目的开发的一个操作系统 [113]。接下来的事情,正如人们常说的那样,都成了历史。Linux 已经发展成一种技术现象和文化现象。通过与 GNU 项目联手,Linux 项目开发出了一个完整的、符合 Posix 标准的 Unix 操作系统版本,其中包括内核和全部配套的基础设施。从手持设备到大型计算机,Linux 可以运行在种类繁多的计算机上。IBM 的一个小组甚至把 Linux 移植到了一块手表上!

随着因特网这类全球网络的出现,把信息从一台机器复制到另一台机器,已经成为计算机系统最重要的用途之一。电子邮件、即时消息、万维网、FTP 和 `telnet` 之类的应用程序,全都建立在通过网络复制信息的能力之上。

回到我们的 `hello` 程序:可以用大家熟悉的 telnet 应用程序在一台远程机器上运行 `hello`。假设本地机器上运行着一个 telnet 客户端,连接到远程机器上的一个 telnet 服务器。登录到远程机器并运行一个 shell 之后,远程 shell 就

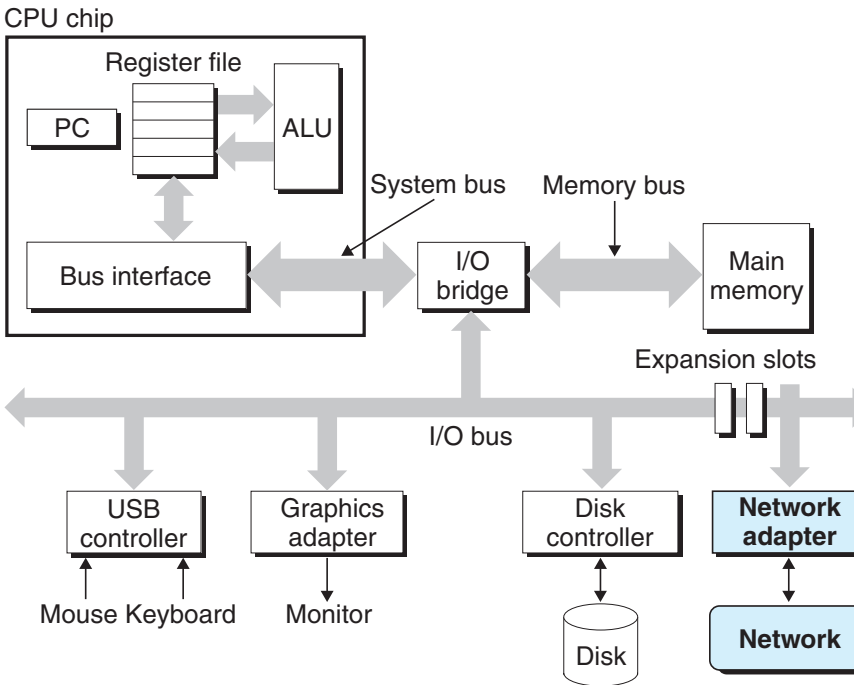


图 1.14 （示意图）

在等待接收输入的命令。从这一刻起，远程运行 `hello` 程序要经过图 1.15 所示的五个基本步骤。

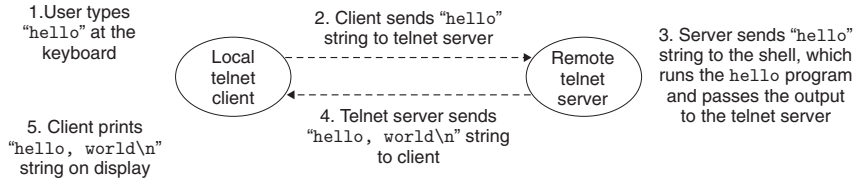


图 1.15 （示意图）

在 telnet 客户端中键入 `hello` 字符串并敲下回车键之后，客户端就会把这个字符串发送给 telnet 服务器。telnet 服务器从网络上收到该字符串后，会把它传递给远程 shell 程序。接下来，远程 shell 运行 `hello` 程序，并把输出行传回 telnet 服务器。最后，telnet 服务器把这个输出字符串经网络转发给 telnet 客户端，由客户端把输出字符串显示在我们的本地终端上。

这种客户端与服务器之间的交换方式，是所有网络应用程序的典型形态。第 11 章将要讲述如何构建网络应用程序，并运用这些知识来构建一个简单的 Web 服务器。

1.9 重要主题

至此,我们对系统走马观花式的初步漫游就告一段落。从这次漫游中要带走的一个重要思想是:系统不仅仅是硬件,它还是由硬件和系统软件相互交织而成的一个集合体,必须依靠两者的协同工作,才能达成运行应用程序这一最终目标。本书余下的篇幅将补充硬件和软件方面的一些细节,并展示如何凭借这些细节写出更快、更可靠、更安全的程序。

作为本章的收尾,我们提要几个贯穿计算机系统各个方面的重要概念。本书将在多处讨论这些概念的重要性。

1.9.1 阿姆达尔定律

对于改善系统某一部分性能的有效性,计算领域的早期先驱之一 Gene Amdahl 作出了一个简单而富有洞见的观察。这一观察后来被称为阿姆达尔定律(Amdahl's law)。它的主要思想是:当我们加速系统的一个部分时,这个部分对系统整体性能的影响取决于两点——它原本有多重要,以及被加速的幅度有多大。考虑这样一个系统:执行某个应用程序需要时间 T_{old}。设系统的某个部分占这段时间的比例为 α,并且我们把它的性能提高了 k 倍。也就是说,该部件原来需要时间 αT_{old},现在需要时间 $(\alpha T_{old})/k$。于是,总的执行时间就是

$$T_{new} = (1-\alpha)T_{old} + (\alpha T_{old})/k = T_{old}\left[(1-\alpha) + \alpha/k\right]$$

由此可以计算出加速比 $S = T_{old}/T_{new}$:

$$S = \frac{1}{(1-\alpha) + \alpha/k} \tag{1.1}$$

举个例子。设系统某一部分最初占用 60% 的时间 $(\alpha = 0.6)$,把它加速 3 倍 $(k = 3)$,得到的加速比是 $1/[0.4 + 0.6/3] = 1.67\times$。尽管我们对系统的一个主要部分作了重大改进,总的加速比仍明显低于这一部分自身提速的幅度。这就是阿姆达尔定律的主要洞见——要想显著地加速整个系统,就必须提高整个系统中非常大一部分的速度。

练习 1.1 假设你是一名卡车司机,受雇把一车土豆从爱达荷州的博伊西运到明尼苏达州的明尼阿波利斯,全程共 2,500 千米。你估计在遵守限速的前提下平均时速可达 100 km/hr,整个行程需要 25 小时。

A. 你听到新闻说,蒙大拿州刚刚取消了限速,该州路段占了这次行程的 1,500 千米,你的卡车可以以 150 km/hr 的速度行驶。这次行程的加速比将是多少?

边栏 相对性能的表达

表达性能提升的最好方式是 T_{old}/T_{new} 这种比值形式,其中 T_{old} 是原始版本所需的时间,T_{new} 是修改后的版本所需的时间。只要确实有了改进,这个数就大于 1.0。我们用后缀"×"表示这类比值,"2.2×"口头上说成"2.2 倍"。

用百分比表示相对变化是更传统的做法,在变化较小时还算好用,但它的定义并不明确:应该是 $100\cdot(T_{old}-T_{new})/T_{new}$,还是 $100\cdot(T_{old}-T_{new})/T_{old}$,或是别的什么?此外,对很大的变化,百分比也没有那么直观。要理解"性能提高了 120%",比直接说"性能提高了 2.2×"要困难得多。

B. 你可以在 www.fasttrucks.com 为卡车购买一个新的涡轮增压器,那里备有各种型号,但你想要的速度越快,花的钱也越多。要让整个行程获得 1.67× 的加速比,你在蒙大拿州境内必须以多快的速度行驶?

练习 1.2 一家汽车制造公司向客户承诺,即将发布的新发动机将有 4× 的性能提升。你被指派去兑现这个承诺。经你测定,发动机中只有 90% 的部分可以改进。要把这个部分改进多少倍(即 k 取什么值),才能达到发动机的整体性能目标?

阿姆达尔定律有一个有趣的特例:考虑令 k 取 ∞ 的效果。也就是说,我们可以把系统的某个部分加速到所需时间可以忽略不计的程度。这时得到

$$S_{\infty} = \frac{1}{(1-\alpha)} \tag{1.2}$$

举例来说,如果我们可以把系统中 60% 的部分加速到几乎不占用任何时间,总的加速比仍然只有 $1/0.4 = 2.5\times$。

阿姆达尔定律描述的是改进任何过程的一般原理。除了用于加速计算机系统之外,它还可以指导一家试图降低刀片生产成本的公司,或者一个想提高平均绩点的学生。在计算机的世界里,这一定律或许体现得最为充分:在这里,把性能提高 2 倍或更多是家常便饭,而这样高的倍数,只有通过优化系统中的很大一部分才能达到。

1.9.2 并发与并行

纵观数字计算机的历史,有两项需求一直是推动系统改进的不竭动力:我们希望计算机做更多的事,也希望它跑得更快。当处理器同时做更多事情时,这两个方面都会得到改善。并发(concurrency)指的是一个系统内存在多个同时进行的活动这样的一般概念;并行(parallelism)指的是利用并发来使系统运行得更快。并行可以在计算机系统的多个抽象层次上加以利用。这里我们从系统层次结构的最高层到最低层,依次介绍其中三个层次。

线程级并发

在进程抽象的基础上,我们可以设计出让多个程序同时执行的系统,这就带来了并发。借助线程(thread),我们甚至可以让单个进程内同时执行多个控制流。自 20 世纪 60 年代初出现分时系统(time-sharing)以来,计算机系统就一直支持并发执行。传统上,这种并发执行只是模拟出来的:让一台计算机在它正在执行的多个进程之间快速切换,就像杂耍艺人让多个球同时在空中飞舞一样。这种形式的并发使多个用户可以同时与一个系统交互,比如许多人都想从同一个 Web 服务器获取页面;它也使单个用户可以同时从事多项任务,比如

在一个窗口中打开 Web 浏览器,在另一个窗口中打开文字处理器,同时还播放流媒体音乐。直到不久之前,实际的计算工作都是由单个处理器完成的,即使这个处理器不得不在多个任务之间来回切换。这种配置称为单处理器系统(uniprocessor system)。

当我们构建一个由多个处理器组成、且统一受单一操作系统内核控制的系统时,就得到了多处理器系统(multiprocessor system)。这类系统从 20 世纪 80 年代起就已经用于大规模计算,但随着多核处理器(multi-core processor)和超线程(hyperthreading)的出现,近来它们已经变得司空见惯。图 1.16 给出了这些不同处理器类型的分类。

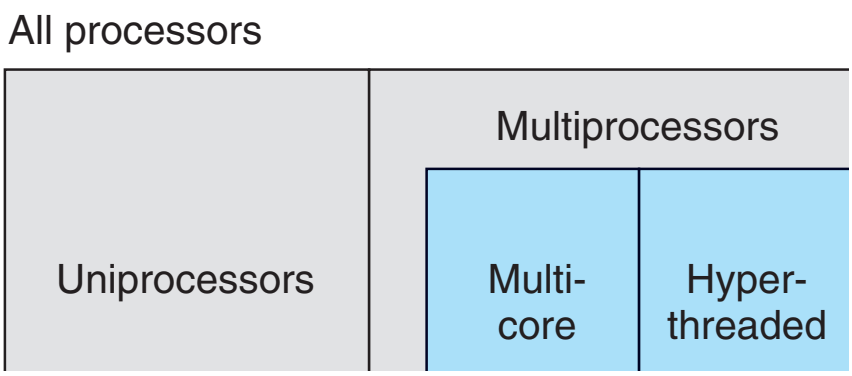


图 1.16 (示意图)

多核处理器把几个 CPU(称为“核”,core)集成到同一块集成电路芯片上。图 1.17 描述了一个典型多核处理器的组织结构:芯片上有 4 个 CPU 核,每个核都有自己的 L1 和 L2 高速缓存,而且每个 L1 高速缓存还分成两个部分——一个存放最近取出的指令,一个存放数据。各个核共享更高层次的高速缓存,也共享通往主存的接口。业界专家预计,单芯片上最终将能够集成几十个乃至上百个核。

超线程(hyperthreading),有时也称为同时多线程(simultaneous multi-threading),是一种让单个 CPU 执行多个控制流的技术。它把 CPU 硬件的某些部件(如程序计数器和寄存器堆)复制出多份副本,而其他部件(如执行浮点算术运算的部件)则只有一个副本。常规处理器需要大约 20,000 个时钟周期才能在不同线程之间转换,而超线程处理器则是逐周期地决定要执行哪个线程。这使 CPU 能够更好地利用自己的处理资源。例如,当某个线程必须等待某些数据装入高速缓存时,CPU 就可以接着执行另一个线程。又如,Intel Core i7 处理器可以让每个核执行两个线程,于是一个 4 核系统实际上能够并行执行 8 个线程。

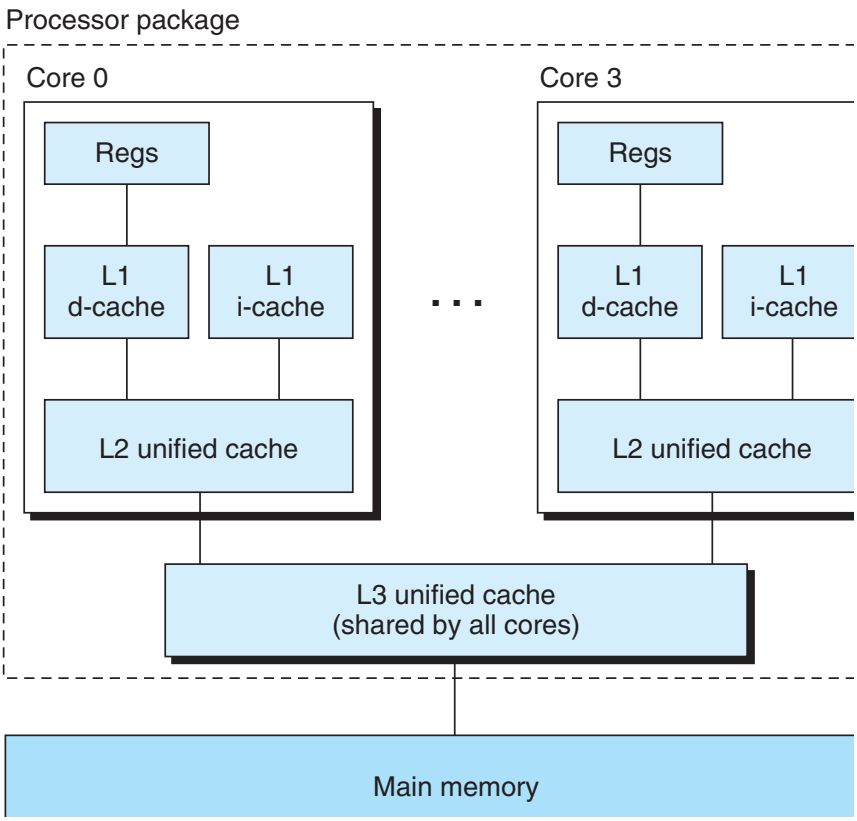


图 1.17 （示意图）

使用多处理可以在两个方面提高系统性能。第一,它减少了在执行多项任务时模拟并发的需要。如前所述,即使是一台只有单人使用的个人计算机,也被期望同时进行许多活动。第二,它可以让单个应用程序运行得更快,但前提是该程序被表述成多个能够有效并行执行的线程。因此,尽管并发的原理从提出和研究至今已有 50 多年,多核和超线程系统的出现还是极大地增强了人们的愿望:找到一些方法,写出能够利用硬件所提供的线程级并行的应用程序。第 12 章将更深入地探讨并发,以及如何利用并发来实现处理资源的共享,并让程序执行获得更大的并行性。

指令级并行

在低得多的抽象层次上,现代处理器可以同时执行多条指令,这种性质称为指令级并行(instruction-level parallelism)。例如,早期的微处理器,如 1978 年的 Intel 8086,执行一条指令需要多个(通常为 3 ~ 10 个)时钟周期。而较新的处理器能够保持每个时钟周期 2 ~ 4 条指令的执行速度。任何一条指令从开始到完成所需的时间都要长得多,也许要 20 个周期甚至更多,但处理器利用许多

巧妙的手段,可以一次处理多达 100 条指令。在第 4 章,我们将探讨流水线化(pipelining)的使用:把执行一条指令所需的动作划分成不同的步骤,并把处理器硬件组织成一系列阶段(stage),每个阶段完成其中一个步骤。各阶段可以并行工作,分别处理不同指令的不同部分。我们会看到,相当简单的硬件设计就能保持接近每个时钟周期 1 条指令的执行速度。

能够保持高于每周期一条指令的执行速度的处理器称为超标量(superscalar)处理器。大多数现代处理器都支持超标量操作。在第 5 章,我们将描述这类处理器的一个高层模型。我们会看到,应用程序员可以利用这个模型来理解自己程序的性能,进而编写程序,使生成的代码获得更高程度的指令级并行,从而运行得更快。

单指令多数据(SIMD)并行

在最低的层次上,许多现代处理器配有专门的硬件,允许一条指令就引起多项操作的并行执行,这种模式称为单指令多数据(SIMD)并行(single-instruction, multiple-data parallelism)。例如,最近几代的 Intel 和 AMD 处理器都有一些指令,可以并行地对 8 对单精度浮点数(C 数据类型 `float`)求和。

提供这些 SIMD 指令,主要是为了加速处理图像、声音和视频数据的应用程序。虽然有些编译器会尝试从 C 程序中自动提取 SIMD 并行性,但更可靠的方法是使用 gcc 这类编译器支持的特殊向量数据类型来编写程序。我们将在网络边栏 `opt:simd` 中描述这种编程风格,作为第 5 章更一般的程序优化论述的补充。

1.9.3 抽象在计算机系统中的重要性

抽象的使用是计算机科学中最重要的概念之一。例如,良好编程实践的一个方面,就是为一组函数制定一个简单的应用程序接口(API),让程序员不必深入代码的内部工作原理就能使用这些代码。不同的程序设计语言为抽象提供不同形式和层次的支持,例如 Java 的类声明和 C 的函数原型。

我们已经见过图 1.18 所示的计算机系统中的几个抽象。在处理器方面,指令集体系结构提供了对实际处理器硬件的抽象。有了这个抽象,机器码程序的行为就仿佛是在一台每次只执行一条指令的处理器上执行一样。底层硬件实际要精巧得多,它能并行地执行多条指令,但总是与这个简单的顺序执行模型保持一致。只要保持相同的执行模型,不同的处理器实现就可以执行同样的机器码,同时提供不同层次的成本与性能。

在操作系统方面,我们已经介绍过三个抽象:文件是对 I/O 设备的抽象,虚拟内存是对程序内存的抽象,进程是对一个正在运行的程序的抽象。在这些抽

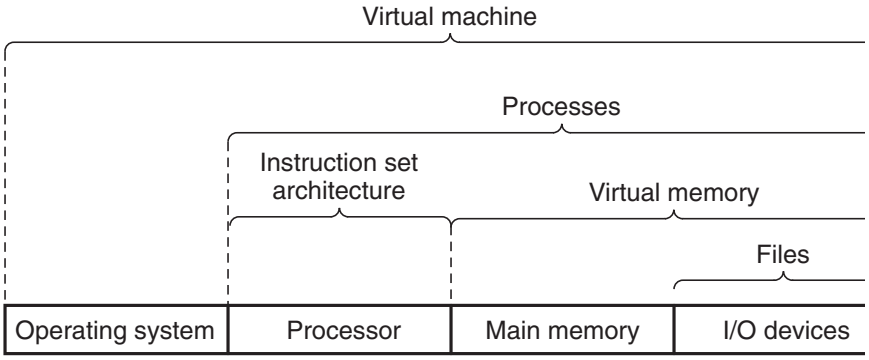


图 1.18 （示意图）

象之外再增加一个新的抽象：虚拟机（virtual machine），它是对包括操作系统、处理器和程序在内的整台计算机的抽象。虚拟机的思想由 IBM 在 20 世纪 60 年代提出，但近来变得愈发重要，它成为管理如下计算机的一种方式：这类计算机必须能够运行为多种操作系统（如 Microsoft Windows、Mac OS X 和 Linux）或同一操作系统的不同版本所设计的程序。

本书后面的各节还会回过头来讨论这些抽象。

1.10 小结

计算机系统由硬件和系统软件组成，两者协同工作来运行应用程序。计算机内的信息表示为一组组位，根据上下文的不同以不同的方式加以解释。程序由其他程序翻译成不同的形式：最初是 ASCII 文本，随后由编译器和链接器翻译成二进制可执行文件。

处理器读取并解释存放在主存中的二进制指令。由于计算机把大部分时间都花在存储器、I/O 设备和 CPU 寄存器之间来回复制数据上，系统中的存储设备被排列成一个层次结构：顶层是 CPU 寄存器，接着是多级硬件高速缓存存储器、DRAM 主存和磁盘存储。层次结构中位置较高的存储设备比较低层的设备速度更快，每位的成本也更高；位置较高的存储设备充当较低层设备的高速缓存。程序员通过理解并利用存储器层次结构，可以优化自己 C 程序的性能。

操作系统内核是应用程序与硬件之间的媒介。它提供三个基本抽象：(1) 文件是对 I/O 设备的抽象。(2) 虚拟内存是对主存和磁盘的抽象。(3) 进程是对处理器、主存和 I/O 设备的抽象。

最后，网络为计算机系统之间的相互通信提供了途径。从一个特定系统的视角来看，网络不过是又一个 I/O 设备。

1.11 参考文献说明

Ritchie[91, 92] 曾写过关于 C 和 Unix 早期岁月的有趣的第一手记述。Ritchie 和 Thompson[93] 发表了第一篇公开发表的关于 Unix 的记述。Silberschatz、Galvin 和 Gagne[102] 对 Unix 各种不同流派的历史作了全面的梳理。GNU(www.gnu.org)和 Linux(www.linux.org)的网页上有大量当今和历史的资料。Posix 标准可在网上获取(www.unix.org)。

1.12 练习答案

答案 1.1 这道题说明,阿姆达尔定律的适用范围并不只限于计算机系统。

A. 根据式 (1.1),有 $\alpha = 0.6$、$k = 1.5$。更直接地算:驶过蒙大拿州的 1,500 千米需要 10 小时,其余的路程也需要 10 小时,这样加速比就是 $25/(10+10) = 1.25\times$。

B. 根据式 (1.1),有 $\alpha = 0.6$,并要求 $S = 1.67$,由此可以解出 k。更直接地看:要让这次行程加速 $1.67\times$,必须把总时间缩短到 15 小时。蒙大拿州以外的路段仍然需要 10 小时,所以必须在 5 小时内驶过蒙大拿州,这就要求车速达到 300 km/hr——这对一辆卡车来说可真够快的!

答案 1.2 只有通过解答实例,才能真正理解阿姆达尔定律。这道题要求你从一个不同寻常的角度来看式 (1.1),是对该公式的一个简单应用。已知 $S = 4$ 和 $\alpha = 0.9$,解出 k:

$$\begin{aligned} 4 &= \frac{1}{(1-0.9)+0.9/k} \\ 0.4 + 3.6/k &= 1.0 \\ k &= 6.0 \end{aligned}$$

第一部分

程序结构与执行

我们对计算机系统的探索，从研究计算机本身开始：先看硬件和系统软件如何为应用程序搭建舞台，再看 C 程序如何被翻译成处理器能够执行的机器码。在此基础上，我们会依次讨论处理器体系结构、程序性能优化、存储器层次结构以及链接这几条主线。

学完这一部分，你将对应用程序在系统上运行时究竟发生了什么有深入的理解：代码如何被翻译、数据如何在存储器层次结构中流动、程序的性能由哪些因素决定。

2

信息的表示和处理

现代计算机存储和处理的信息用二值信号表示。这些不起眼的二进制数字，或者叫位（bit），构成了数字革命的基石。人们熟悉的十进制表示法，即以10为基数的表示法，已经沿用了一千多年：它起源于印度，12世纪经阿拉伯数学家改进，13世纪由意大利数学家Leonardo Pisano（约1170—约1250）传入西方，这位数学家更为人熟知的名字是Fibonacci。对长着10根手指的人类来说，用十进制记数是自然的选择，但在构造存储和处理信息的机器时，二进制值工作得更好。二值信号表示、存储和传输起来都很方便，比如可以表示成穿孔卡片上有孔或无孔、导线上的高电压或低电压、顺时针方向或逆时针方向取向的磁畴。用来存储并对二值信号执行计算的电子电路非常简单可靠，这使得制造商能够把数百万甚至数十亿个这样的电路集成到一块硅芯片上。

单个的位没有多大用处。但是，若把位组合起来并施加某种解释，赋予不同的位模式以含义，我们就能表示任何有限集合的元素。例如，利用二进制数系统，可以用一组一组位来编码非负数；利用标准字符码，可以编码文档中的字母和符号。本章将讲述这两种编码，还将讲述表示负数的编码以及近似表示实数的编码。

我们研究三种最重要的数字表示。无符号（unsigned）编码基于传统的二进制记法，表示大于等于0的数。补码（two's-complement）编码是表示有符号整数（signed integer）——既可为正也可为负的数——的最常见方式。浮点数（floating-point）编码是以2为基数的科学记数法的一个版本，用来表示实数。计算机用这些不同的表示来实现加法、乘法这类算术运算，其方式与对整数和实数进行的相应运算类似。

计算机的表示法用有限的位数来编码一个数，因此当计算结果大到无法表示时，某些运算就会溢出（overflow）。这会产生一些令人吃惊的结果。例如，在当今的大多数计算机上（即对数据类型 `int` 采用32位表示的机器），计算表达式

```
200 * 300 * 400 * 500
```

得到的却是 −884,901,888。这违背了整数运算的性质——对一组正数求乘积,结果竟是一个负数。

另一方面，计算机的整数运算满足真实整数运算的许多熟悉性质。例如，乘法满足结合律和交换律，所以计算下面任何一个 C 表达式都会得到 −884,901,888：

```
(500 * 400) * (300 * 200)
((500 * 400) * 300) * 200
((200 * 500) * 300) * 400
400   * (200 * (300 * 500))
```

计算机也许得不到期望的结果,但至少结果是前后一致的!

浮点算术具有完全不同的数学性质。一组正数的乘积总是正的，虽然溢出会产生特殊值 +∞。由于表示的精度有限，浮点算术不满足结合律。例如，在大多数机器上，C 表达式 `(3.14+1e20)-1e20` 的求值结果是 0.0，而 `3.14+(1e20-1e20)` 的求值结果是 3.14。整数算术与浮点算术的数学性质之所以不同,源于二者处理表示有限性的方式不同——整数表示能编码的取值范围相对较小,但编码得很精确;浮点表示能编码的范围很大,但只能近似编码。

通过研究实际的数字表示,我们可以理解可表示的取值范围以及各种算术运算的性质。要写出能在完整取值范围内正确工作、且能在机器、操作系统和编译器的各种组合之间移植的程序,这种理解至关重要。正如我们将要讲述的,许多计算机安全漏洞正是由于计算机运算的某些微妙之处而出现的。在早先的年代,程序 bug 只有碰巧被触发时才会给人带来不便;而今天,大批黑客时刻准备着利用他们能发现的任何 bug,来获取对他人系统的未授权访问。这就把更高的义务压到程序员身上:不仅要理解自己的程序是如何工作的,还要理解它们会怎样被诱发出种种不良行为。

计算机使用几种不同的二进制表示来编码数值。进入第 3 章的机器级编程之后,你需要熟悉这些表示。本章将描述这些编码,并告诉你如何对数字表示进行推理。

我们直接操作数的位级表示,由此推导出若干执行算术运算的方法。理解这些技术，对理解编译器为优化算术表达式求值性能而生成的机器级代码很重要。

我们对这部分内容的讲解以一组核心数学原理为基础。我们先给出编码的基本定义,再推导各种性质,比如可表示数的取值范围、它们的位级表示,以及算术运算的性质。我们相信,从这种抽象的观点审视这部分内容很重要,因为程序员需要清楚地理解计算机运算与更熟悉的整数运算和实数运算之间的关系。

C++ 程序设计语言建立在 C 的基础上，使用的数字表示和运算与 C 完全相同。本章关于 C 的一切论断对 C++ 同样成立。而 Java 的语言定义为数字表示和运算另立了一套标准。C 标准的制定允许实现有广泛的选择余地，Java 标准则对数据的格式和编码规定得相当具体。本章将在若干处着重指出 Java 支持的表示和运算。

2.1 信息存储

大多数计算机并不访问存储器中单独的位，而是以 8 位的块，即字节，作为存储器最小的可寻址单位。机器级程序把存储器看作一个非常大的字节数组，称为虚拟内存（virtual memory）。存储器的每个字节都由一个唯一数字标识，这个数字称为它的地址（address），所有可能地址的集合称为虚拟地址空间（virtual address space）。顾名思义，这个虚拟地址空间只是呈现给机器级程序的一个概念性映像。实际的实现（将在第 9 章讲述）综合运用动态随机访问存储器（DRAM）、闪存、磁盘存储、专用硬件和操作系统软件，为程序提供看起来浑然一体的一个字节数组。

在后续几章中，我们将讲述编译器和运行时系统如何把这个存储器空间划分成更便于管理的单元，以存储不同的程序对象（program object），也就是程序数据、指令和控制信息。程序不同部分的存储由各种机制来分配和管理，而这些管理都是在虚拟地址空间内进行的。例如，C 中指针（pointer）的值——无论它指向一个整数、一个结构还是其他某个程序对象——都是某块存储的第一个字节的虚拟地址。C 编译器还给每个指针关联类型信息，从而能根据那个值的类型，生成不同的机器级代码来访问由指针指定的位置上存储的值。尽管 C 编译器维护着这些类型信息，它生成的机器级程序却不包含关于数据类型的任何信息，只是把每个程序对象当作一个字节块，把程序本身当作一个字节序列来对待。

边栏　C 语言的演进

正如第 1 章中介绍 C 语言起源的边栏所述，C 程序设计语言最初由贝尔实验室的 Dennis Ritchie 开发，供 Unix 操作系统（同样诞生于贝尔实验室）使用。那个年代，多数系统程序（比如操作系统）为了能访问不同数据类型的低层表示，大部分都得用汇编代码来写。例如，用那个时代的其他高级语言写一个内存分配器——就像 `malloc` 库函数提供的那种——是行不通的。

最初的贝尔实验室版 C 记载于 Brian Kernighan 和 Dennis Ritchie 那本书的第一版 [60]。此后，C 在若干标准化小组的努力下不断演进。对最初的 Bell Labs C 的第一次重大修订，促成了 1989 年的 ANSI C 标准，它由美国国家标准学会（ANSI）领导下的一个工作组制定。ANSI C 与 Bell Labs C 差别很大，函数的声明方式尤其如此。

边栏　如何阅读本章

在本章中，我们考察计算机如何表示数字和其他形式的数据，以及计算机对这些数据执行的运算所具有的基本性质。这要求我们深入数学的语言：书写公式和方程，并展示重要性质的推导过程。

为了帮助你阅读这些论述，我们把内容的组织方式定为：先用数学记号把一条性质表述为一条原理，然后用例子和非正式的讨论来说明这条原理。我们建议你在原理的表述与例子、讨论之间来回对照，直到对所说的内容及该性质的重要之处形成直观认识。对比较复杂的性质，我们还给出了推导，其结构很像数学证明。你应当努力最终理解这些推导，但初读时可以先跳过它们。

我们还鼓励你随着讲解的推进去做练习题。练习题让你主动学习，帮助你把想法付诸行动。有了这些练习做铺垫，你回头再看那些推导就容易多了。另外可以放心：理解这部分内容所需的数学技能，只要扎实掌握高中代数，就完全能够达到。

ANSI C 在 Kernighan 和 Ritchie 那本书的第二版 [61] 中加以描述，该书至今仍被视为最好的 C 参考书之一。

国际标准化组织（ISO）随后接管了 C 语言标准化的职责，在 1990 年采纳了一个与 ANSI C 实质相同的版本，因此称为“ISO C90”。

还是这个组织，又于 1999 年发起了一次语言更新，产生了“ISO C99”。除其他内容外，这个版本引入了一些新的数据类型，并为要求使用英语中找不到的字符的文本字符串提供了支持。更新的标准于 2011 年获得批准，名为“ISO C11”，再次增加了更多的数据类型和特性。这些新近的增补大多向后兼容（backward compatible），也就是说，按较早标准（至少早至 ISO C90）编写的程序，用较新的标准编译时行为不变。

GNU 编译器套件（GNU Compiler Collection，`gcc`）可以根据不同的命令行选项，按若干不同 C 语言版本的约定来编译程序，如图 2.1 所示。例如，要按 ISO C11 编译程序 `prog.c`，可以给出如下命令行：

```
linux> gcc -std=c11 prog.c
```

选项 `-ansi` 与 `-std=c89` 的效果相同——代码按 ANSI 或 ISO C90 标准编译。（C90 有时也称“C89”，因为其标准化工作始于 1989 年。）选项 `-std=c99` 让编译器遵循 ISO C99 的约定。

到撰写本书时为止，如果不指定选项，程序将按一个以 ISO C90 为基础的 C 版本编译，但其中包含 C99 的某些特性、C11 的某些特性、C++ 的某些特性，还有 gcc 特有的一些内容。GNU 项目正在开发一个版本，它以 ISO C11 加上其他特性为基础，可用命令行选项 `-std=gnu11` 来指定。（目前这个实现还不完整。）它将成为默认版本。

图 2.1 用 gcc 指定不同版本的 C 语言。

C 版本	gcc 命令行选项
GNU 89	无，`-std=gnu89`
ANSI、ISO C90	`-ansi`、`-std=c89`
ISO C99	`-std=c99`
ISO C11	`-std=c11`

给 C 语言初学者 C 语言中指针的作用

2.1.1 十六进制表示法

一个字节由 8 位组成。用二进制记法，它的取值范围为 00000000_2 到 11111111_2；若把它看作一个十进制整数，取值范围为 0_{10} 到 255_{10}。这两种记法对描述位模式（bit pattern）来说都不很方便：二进制记法太冗长，而十进制记法与位模式之间的双向换算又很麻烦。于是，我们把位模式写成以 16 为基数的数，即十六进制（hexadecimal）数。十六进制（或简称“hex”）使用数字‘0’到‘9’以及字符‘A’到‘F’来表示 16 个可能的值。图 2.2 给出了与这 16 个十六进制数字对应的十进制值和二进制值。用十六进制写出，单个字节的取值范围为 00_{16} 到 FF_{16}。

在 C 中，以 0x 或 0X 开头的数字常量被解释为十六进制值。字符‘A’到‘F’既可以写成大写，也可以写成小写。例如，可以把数 $\texttt{FA1D37B}_{16}$ 写作 0xFA1D37B、0xfa1d37b，甚至大小写混用（比如 0xFa1D37b）。本书表示十六进制值时采用 C 语言记法。

做机器级程序时，一项常见任务是在位模式的十进制、二进制和十六进制表示之间手工转换。二进制与十六进制之间的转换很直截了当，因为可以一次转换一位十六进制数字。各位数字的转换可以参照如图 2.2 那样的表。要做心算转换，一个简单的技巧是记住十六进制数字 A、C 和 F 的十进制等值。

图 2.2 十六进制表示法。每个十六进制数字编码 16 个可能值中的一个。

十六进制数字	0	1	2	3	4	5	6	7
十进制值	0	1	2	3	4	5	6	7
二进制值	0000	0001	0010	0011	0100	0101	0110	0111
十六进制数字	8	9	A	B	C	D	E	F
十进制值	8	9	10	11	12	13	14	15
二进制值	1000	1001	1010	1011	1100	1101	1110	1111

十六进制值 B、D 和 E 的十进制值，只要计算它们相对于前三个值的大小就能求出。

举例来说，给定数 0x173A4C，只要把每一位十六进制数字展开，就能把它转换成二进制格式：

十六进制	1	7	3	A	4	C
二进制	0001	0111	0011	1010	0100	1100

由此得到二进制表示 000101110011101001001100。

反过来，给定二进制数 111100101011011011 0011，要把它转换成十六进制，可以先把它分成每组 4 位。不过要注意，如果总位数不是 4 的倍数，应该让**最左边**一组是不足 4 位的那一组，这相当于在数的前面补上若干个 0。然后把每一组位翻译成对应的十六进制数字：

二进制	11	1100	1010	1101	1011	0011
十六进制	3	C	A	D	B	3

练习 2.1 完成下面这些数制转换：

A. 把 0x25B9D2 转换成二进制

B. 把二进制 `1010111001001001` 转换成十六进制

C. 把 `0xA8B3D` 转换成二进制

D. 把二进制 `11001000101101100101l0` 转换成十六进制

当 x 是 2 的幂，即 $x = 2^n$（n 是某个非负整数）时，容易直接写出 x 的十六进制形式，只要记住 x 的二进制表示就是 1 后面跟 n 个 0。一位十六进制的 0 表示 4 个二进制的 0。所以，把 n 写成 $i + 4j$ 的形式（$0 \le i \le 3$）之后，写 x 时可以取前导十六进制数字 $1(i = 0)$、$2(i = 1)$、$4(i = 2)$ 或 $8(i = 3)$，后面再跟 j 个十六进制 0。例如，对 $x = 2{,}048 = 2^{11}$，有 $n = 11 = 3 + 4 \cdot 2$，得到十六进制表示 `0x800`。

练习 2.2 补全下表中的空项，给出不同的 2 的幂的十进制表示和十六进制表示：

n	2^n（十进制）	2^n（十六进制）
5	32	`0x20`
23	_____	_____
__	32,768	_____
__	_____	`0x2000`
12	_____	_____
__	64	_____
__	_____	`0x100`

十进制表示和十六进制表示之间的相互转换，在一般情形下要用乘法或除法来处理。要把十进制数 x 转换成十六进制，可以反复用 16 去除 x，得到商 q 和余数 r，使得 $x = q \cdot 16 + r$。然后把表示 r 的那位十六进制数字用作最低有效数字，再对 q 重复这一过程，生成其余各位。以十进制数 314,156 的转换为例：

$$314{,}156 = 19{,}634 \cdot 16 + 12 \quad \text{(C)}$$
$$19{,}634 = 1{,}227 \cdot 16 + 2 \quad \text{(2)}$$
$$1{,}227 = 76 \cdot 16 + 11 \quad \text{(B)}$$
$$76 = 4 \cdot 16 + 12 \quad \text{(C)}$$
$$4 = 0 \cdot 16 + 4 \quad \text{(4)}$$

由此可以直接读出十六进制表示 `0x4CB2C`。

反过来，要把十六进制数转换成十进制，可以把每一位十六进制数字乘上恰当的 16 的幂。例如，给定数 `0x7AF`，计算它的十进制等值：$7 \cdot 16^2 + 10 \cdot 16 + 15 = 7 \cdot 256 + 10 \cdot 16 + 15 = 1{,}792 + 160 + 15 = 1{,}967$。

练习 2.3 一个字节可以用 2 位十六进制数字表示。补全下表中缺失的项，给出不同字节模式的十进制、二进制和十六进制值：

十进制	二进制	十六进制
0	0000 0000	0x00
158	_____	_____
76	_____	_____
145	_____	_____
__	1010 1110	_____
__	0011 1100	_____
__	1111 0001	_____
__	_____	0x75
__	_____	0xBD
__	_____	0xF5

练习 2.4 不把数转换成十进制或二进制，试着解出下面这些算术问题，答案用十六进制给出。提示：只需把平时做十进制加减法的方法改成以 16 为基数。

A. `0x605c + 0x5` = _____

B. `0x605c - 0x20` = _____

C. `0x605c + 32` = _____

D. `0x60fa - 0x605c` = _____

边栏 十进制与十六进制的相互转换

要在十进制和十六进制之间转换较大的数值，最好让计算机或计算器来做。能做这件事的工具很多。一种简单的办法是用任意一个标准搜索引擎，输入如下形式的查询：

```
Convert 0xabcd to decimal
```

或

```
123 in hex
```

2.1.2 字数据大小

每台计算机都有一个字长（word size），指明指针数据的标称大小。由于虚拟地址就用这样一个字来编码，字长所决定的最重要的系统参数就是虚拟地址空间的最大大小。也就是说，对一个字长为 w 位的机器，虚拟地址的范围在 0 到 $2^w - 1$ 之间，程序最多能访问 2^w 个字节。

近年来，出现了一场从 32 位字长机器向 64 位字长机器的普遍转变。这一转变最初发生在为大规模科学计算和数据库应用设计的高端机器上，随后蔓延到台式机和笔记本电脑，最近则轮到智能手机中的处理器。32 位字长把虚拟地址空间限制在 4 千兆字节（写作 4 GB）以内，也就是刚好比 4×10^9 字节略多一点。把字长提升到 64 位，虚拟地址空间则达到 16 艾字节（exabyte），约 1.84×10^{19} 字节。

大多数 64 位机器也可以运行为 32 位机器编译的程序，这是一种向后兼容（backward compatibility）的形式。例如，当程序 `prog.c` 用下面这条指令编译时

```
linux> gcc -m32 prog.c
```

那么这个程序既能在 32 位机器上正确运行，也能在 64 位机器上正确运行。反之，用下面这条指令编译的程序

```
linux> gcc -m64 prog.c
```

只能在 64 位机器上运行。因此，我们把程序称为“32 位程序”或“64 位程序”，因为这种区分取决于程序如何编译，而不取决于它运行在哪种类型的机器上。

计算机和编译器支持多种数据格式，用不同的方式来编码数据，比如整数和浮点数，也支持不同的长度。例如，许多机器既有处理单个字节的指令，也有处理以 2 字节、4 字节和 8 字节表示的整数的指令；它们还支持以 4 字节和 8 字节表示的浮点数。

C 语言为整数和浮点数数据都支持多种数据格式。图 2.3 给出为不同的 C 数据类型通常分配的字节数。(C 标准所保证的与典型情况之间的关系，我们在 2.2 节讨论。)某些数据类型的准确字节数取决于程序如何编译。我们给出的是典型的 32 位程序和 64 位程序的大小。整数数据可以是有符号的(signed)，能够表示负数、零和正数，也可以是无符号的(unsigned)，只允许非负值。数据类型 `char` 表示单个字节。`char` 这个名字源于这样一个事实：它用来在一个文本串中存放单个字符，但它也可以用来存放整数值。数据类型 `short`、`int` 和 `long` 的本意是提供一定范围的大小。即使编译为 64 位系统，数据类型 `int` 通常也只有 4 个字节。数据类型 `long` 在 32 位程序中通常为 4 字节，在 64 位程序中通常为 8 字节。

图 2.3　C 语言基本数据类型的典型大小(以字节计)。分配的字节数随程序编译方式而变化，本表给出 32 位和 64 位程序的典型值。

有符号	无符号	32 位	64 位
`[signed] char`	`unsigned char`	1	1
`short`	`unsigned short`	2	2
`int`	`unsigned`	4	4
`long`	`unsigned long`	4	8
`int32_t`	`uint32_t`	4	4
`int64_t`	`uint64_t`	8	8
`char *`		4	8
`float`		4	4
`double`		8	8

给 C 初学者的提示　声明指针

对于任何数据类型 T，声明

```
T *p;
```

表示 p 是一个指针变量，指向一个类型为 T 的对象。例如，

```
char *p;
```

就是指向一个 `char` 类型对象的指针的声明。

为了避免依赖“典型”大小和各种编译器设置所导致的不确定性，ISO C99 引入了一类数据类型，其数据大小不随编译器和机器设置而变。其中就有数据类型 `int32_t` 和 `int64_t`，分别恰好是 4 字节和 8 字节。使用固定大小的整数类型，是程序员对数据表示进行精细控制的最好办法。

大多数数据类型编码的都是有符号值，除非加上关键字 `unsigned` 作为前缀，或者对固定大小的数据类型使用专门的无符号声明。这一规则的例外是数据类型 `char`。虽然大多数编译器和机器都把 `char` 当作有符号数据处理，但 C

标准对此不作保证。所以，正如方括号所提示的，程序员应该用 `signed char` 声明来保证得到 1 字节的有符号值。不过在很多情形下，程序的行为对数据类型 `char` 是有符号还是无符号并不敏感。

C 语言允许用多种次序来排列这些关键字，也允许包含或省略可选的关键字。举例来说，下面这些声明具有完全相同的意义：

```
unsigned long
unsigned long int
long unsigned
long unsigned int
```

本书统一采用图 2.3 中出现的形式。

图 2.3 还表明，指针（例如声明为类型 `char *` 的变量）会占满整个程序的字长。大多数机器还支持两种不同的浮点格式：单精度（single precision），在 C 中用 `float` 声明；双精度（double precision），在 C 中用 `double` 声明。这两种格式分别使用 4 字节和 8 字节。

程序员应该力求让自己编写的程序能在不同的机器和编译器之间移植。可移植性的一个方面，就是让程序对各种数据类型的确切大小不敏感。C 标准为不同数据类型的数值范围设定了下界（后面会讲到），但没有设定上界（固定大小的类型除外）。大约从 1980 年到 2010 年，占主导地位的组合是 32 位机器配 32 位程序，因此许多程序都是按照图 2.3 中为 32 位程序列出的分配来写的。随着向 64 位机器过渡，许多隐藏的对字长的依赖，在把这些程序迁移到新机器时以 bug 的形式暴露出来。例如，许多程序员向来假定，声明为 `int` 类型的对象可以用来存放一个指针。这对大多数 32 位程序都行得通，但对 64 位程序就会带来问题。

2.1.3 寻址和字节顺序

对于跨越多个字节的对象，必须建立两个约定：这个对象的地址是什么，以及在存储器中如何排列这些字节。在几乎所有的机器上，多字节对象都存储为一个连续的字节序列，对象的地址为所用字节中最小的那个地址。例如，假设一个类型为 `int` 的变量 x 的地址为 `0x100`，也就是说，地址表达式 `&x` 的值为 `0x100`。那么（假设数据类型 `int` 采用 32 位表示）x 的 4 个字节将存放在存储器位置 `0x100`、`0x101`、`0x102` 和 `0x103` 上。

要对表示一个对象的字节规定顺序，有两种常见的约定。考虑一个 w 位整数，其位表示为 $[x_{w-1}, x_{w-2}, \ldots, x_1, x_0]$，其中 x_{w-1} 是最高有效位，x_0 是最低有效位。假设 w 是 8 的倍数，这些位就可以按字节分组：最高有效字节包含位 $[x_{w-1}, x_{w-2}, \ldots, x_{w-8}]$，最低有效字节包含位 $[x_7, x_6, \ldots, x_0]$，其余的字节则包

含中间各位。有些机器选择在存储器中按从最低有效字节到最高有效字节的顺序存放对象，另一些机器则按从最高有效字节到最低有效字节的顺序存放。前一种约定（最低有效字节在前）称为小端法（little endian）。后一种约定（最高有效字节在前）称为大端法（big endian）。

假设类型为 `int`、地址为 `0x100` 的变量 x 的十六进制值为 `0x01234567`。在地址范围 `0x100` 至 `0x103` 内，这些字节的排列顺序取决于机器的类型：

		0x100	0x101	0x102	0x103	
大端法	…	01	23	45	67	…
小端法	…	67	45	23	01	…

注意，在字 `0x01234567` 中，最高有效字节的十六进制值为 `0x01`，而最低有效字节的值为 `0x67`。

大多数与 Intel 兼容的机器只用小端模式。另一方面，IBM 和 Oracle 的大多数机器（Oracle 的现有服务器产品线源于它在 2010 年对 Sun Microsystems 的收购）用大端模式。注意，我们说的是“大多数”。这些约定并不严格按照企业边界划分。例如，IBM 和 Oracle 都生产使用 Intel 兼容处理器的机器，因而它们是小端的。许多新近的微处理器芯片是双端（bi-endian）的，这意味着它们可以被配置成小端或大端机器来运行。不过在实践中，一旦选定了某个操作系统，字节顺序就固定下来。例如，许多手机使用的 ARM 微处理器，其硬件既可按小端模式也可按大端模式运行，但用于这些芯片的两种最常见操作系统——Android（来自 Google）和 IOS（来自 Apple）——只按小端模式运行。

> **边栏 “endian”一词的来历**
>
> 下面是乔纳森·斯威夫特（Jonathan Swift）在 1726 年对大端与小端之争的历史所作的描述：
>
> ……利立浦特（Lilliput）和布莱夫斯库（Blefuscu）……正如我本来要告诉你的，两国之间已经打了一场非常顽固的战争，至今已有三十六个月。战事起于这样一个缘由：大家都承认，吃鸡蛋之前的原始做法是从较大的一端打破鸡蛋；可是当今皇上的祖父在小时候按古老的做法吃鸡蛋、磕鸡蛋时，恰好割破了一个手指。于是他的父亲、当时的皇帝就颁布了一道敕令，命令全体臣民务必从小端打破鸡蛋，违者严惩。人民对这项法令深恶痛绝，据我们的史书记载，由此一共发生过六次叛乱，其中一次皇帝丢了性命，另一次丢了王冠。这些内乱不断受到布莱夫斯库的君主们的煽动；每次动乱平息之后，流亡者总是逃到那个帝国去寻求庇护。据说先后已有一万一千人宁肯送命，也不肯把鸡蛋的小端打破。围绕这场争论出版了数百卷大部头的著作：但大端派的书早遭查禁，这一派的人也被法律规定不得担任任何职务。（乔纳森·斯威夫特。《格列佛游记》（Gulliver’s Travels），Benjamin Motte，1726 年。）

> 在斯威夫特生活的时代，他是借这个故事讽刺英格兰（利立浦特）与法兰西（布莱夫斯库）之间接连不断的冲突。网络协议的早期开拓者 Danny Cohen 最先把这两个词用来指称字节顺序 [24]，这一术语从此得到广泛采用。

对于哪种字节顺序才是“正确的”，人们的情绪反应出奇地强烈。事实上，“小端法”和“大端法”这两个术语出自乔纳森·斯威夫特的《格列佛游记》，书中交战的两个派系无法就一个半熟鸡蛋应该由小端还是大端打开达成一致。就像那个鸡蛋问题一样，选择哪一种字节顺序约定并没有任何技术上的理由，于是这类争论就退化成对社会政治问题的争吵。只要选定一种约定并始终坚持，这个选择就是随意的。

对大多数应用程序员来说，他们的机器采用哪种字节顺序是完全不可见的；为两类机器编译的程序给出完全相同的结果。然而，字节顺序有时会成为一个问题。第一种情况是，二进制数据在不同机器之间通过网络通信。常见的一个问题是：小端机器产生的数据要发送给大端机器，或者反过来，导致接收方程序中字内的字节顺序正好颠倒。为了避免这类问题，为网络应用编写的代码必须遵循已确立的字节顺序约定，确保发送方机器把自己的内部表示转换成网络标准，而接收方机器把网络标准转换成自己的内部表示。我们将在第 11 章看到这些转换的例子。

字节顺序变得重要的第二种情况，是在查看表示整数数据的字节序列时。检查机器级程序时经常会遇到这种情况。例如，下面这一行出自一个文件，该文件给出 Intel x86-64 处理器机器级代码的文本表示：

```
4004d3:    01 05 43 0b 20 00           add     %eax,0x200b43(%rip)
```

这一行是由反汇编器（disassembler）生成的，反汇编器是一种工具，用来确定可执行程序文件所表示的指令序列。我们将在第 3 章进一步学习反汇编器，以及如何解读这样的行。眼下只需注意，这一行说明，十六进制字节序列 `01 05 43 0b 20 00` 是一条指令的字节级表示，该指令把一个字的数据与存放在某个地址处的值相加，而这个地址是用 `0x200b43` 加上程序计数器的当前值求得的；程序计数器存放的是下一条要执行的指令的地址。如果取该序列的最后 4 个字节 `43 0b 20 00`，把它们反序写出，就得到 `00 20 0b 43`。去掉前导的 0，得到值 `0x200b43`，正是右边写出的数值。阅读像这一台机器这样为小端机器生成的机器级程序表示时，字节以反序出现是常见现象。书写字节序列的自然方式，是让编号最低的字节在左、编号最高的在右，但这与书写数字的常规方式正好相反——常规方式是最高有效数字在左、最低的在右。

字节顺序显现出来的第三种情况，是编写的程序绕开了正常的类型系统。在 C 语言中，可以用强制类型转换（cast）或联合（union）来让一个对象按照与创

建它时不同的数据类型被引用。对大多数应用编程来说,强烈不建议使用这类编码技巧,但对系统级编程来说,它们相当有用,甚至是必要的。

图 2.4 给出了一份 C 代码, 它使用强制类型转换来访问和打印不同程序对象的字节表示。我们用 `typedef` 定义数据类型 `byte_pointer`, 作为指向 `unsigned char` 类型对象的指针。这样的字节指针引用一个字节序列, 其中每个字节都被当作一个非负整数。第一个例程 `show_bytes` 收到一个字节指针所指示的字节序列的地址和一个字节数。字节数的类型规定为 `size_t`, 这是表达数据结构大小的首选数据类型。例程以十六进制打印各个字节。C 格式化指令`%.2x` 表示整数应以至少 2 位数字的十六进制打印。

```
1  #include <stdio.h>
2
3  typedef unsigned char *byte_pointer;
4
5  void show_bytes(byte_pointer start, size_t len) {
6      int i;
7      for (i = 0; i < len; i++)
8          printf(" %.2x", start[i]);
9      printf("\n");
10 }
11
12 void show_int(int x) {
13     show_bytes((byte_pointer) &x, sizeof(int));
14 }
15
16 void show_float(float x) {
17     show_bytes((byte_pointer) &x, sizeof(float));
18 }
19
20 void show_pointer(void *x) {
21     show_bytes((byte_pointer) &x, sizeof(void *));
22 }
```

图 2.4　打印程序对象字节表示的代码。这段代码使用强制类型转换绕过类型系统。为其他数据类型定义类似的函数也很容易。

过程 `show_int`、 `show_float` 和 `show_pointer` 演示了如何用过程 `show_bytes` 打印 int、float 和 void * 类型的 C 程序对象的字节表示。可以看到,它们只是把指向自己参数 x 的指针 `&x` 传给 `show_bytes`,并把指针转换成 `unsigned char *` 类型。这一转换告诉编译器,程序应当把这个指针看作指向一个字节序列,而不是指向原始数据类型的对象。这样,这个指针就指向该对象所占用的最低字节地址。

这些过程用 C 的 `sizeof` 运算符来确定对象占用的字节数。一般地,表达式 `sizeof(`T`)` 返回存储一个 T 类型对象所需的字节数。与使用固定数值相比,使用 `sizeof` 是朝着编写能在不同类型机器之间移植的代码迈出的一步。

我们在几台不同的机器上运行了图 2.5 所示的代码，得到图 2.6 所示的结果。使用的机器如下：

Linux 32 运行 Linux 的 Intel IA32 处理器。

Windows 运行 Windows 的 Intel IA32 处理器。

Sun 运行 Solaris 的 Sun Microsystems SPARC 处理器。(这些机器现在由 Oracle 生产。)

Linux 64 运行 Linux 的 Intel x86-64 处理器。

```
code/data/show-bytes.c
1  void test_show_bytes(int val) {
2      int ival = val;
3      float fval = (float) ival;
4      int *pval = &ival;
5      show_int(ival);
6      show_float(fval);
7      show_pointer(pval);
8  }
code/data/show-bytes.c
```

图 2.5 字节表示的示例。这段代码打印样本数据对象的字节表示。

图 2.6 不同数据值的字节表示。除字节顺序外，int 和 float 的结果相同。指针值与机器有关。

机器	值	类型	字节(十六进制)
Linux 32	12,345	int	39 30 00 00
Windows	12,345	int	39 30 00 00
Sun	12,345	int	00 00 30 39
Linux 64	12,345	int	39 30 00 00
Linux 32	12,345.0	float	00 e4 40 46
Windows	12,345.0	float	00 e4 40 46
Sun	12,345.0	float	46 40 e4 00
Linux 64	12,345.0	float	00 e4 40 46
Linux 32	&ival	int *	e4 f9 ff bf
Windows	&ival	int *	b4 cc 22 00
Sun	&ival	int *	ef ff fa 0c
Linux 64	&ival	int *	b8 11 e5 ff ff 7f 00 00

参数 12,345 的十六进制表示是 0x00003039。对于 int 数据，除字节顺序之外，所有机器的结果完全相同。特别地，可以看到最低有效字节值 0x39 在 Linux 32、Windows 和 Linux 64 上最先打印出来，说明它们是小端法机器；在 Sun 上最后打印，说明它是大端法机器。类似地，float 数据的字节也是除字节顺序之外完全相同。另一方面，指针值完全不同。不同的机器/操作系统配置采

用不同的存储分配约定。需要注意的一点是,Linux 32、Windows 和 Sun 机器使用 4 字节地址,而 Linux 64 机器使用 8 字节地址。

C 语言新知　用 typedef 命名数据类型

C 中的 typedef 声明为给数据类型起名字提供了一种途径。这对改善代码可读性很有帮助,因为嵌套很深的类型声明很难解读。

typedef 的语法与声明一个变量的写法完全一样,只是用类型名代替了变量名。因此,图 2.4 中 `byte_pointer` 的声明与 unsigned char * 类型变量的声明具有相同的形式。

例如,声明

```
typedef int *int_pointer;
int_pointer ip;
```

把类型 `int_pointer` 定义为指向 int 的指针,并声明一个该类型的变量 ip。另一种做法是直接声明这个变量:

```
int *ip;
```

C 语言新知　用 printf 格式化打印

printf 函数(连同它的几个近亲 fprintf 和 sprintf)提供了一种打印信息的方法,对格式细节有很强的控制能力。第一个参数是格式串(format string),其余参数是要打印的值。在格式串中,每个以' 指定定长数据类型(例如 `int_32t`)的格式要复杂一些,2.2 节的一条边栏对此有描述。

C 语言新知　指针与数组

在函数 `show_bytes`(图 2.4)中,我们可以看到指针与数组之间的紧密联系,3.8 节将详细讨论这一点。这个函数有一个类型为 `byte_pointer` 的参数 start(它已被定义为指向 unsigned char 的指针),而在第 8 行却出现了数组引用 start[i]。在 C 中,既可以用数组记号来解引用指针,也可以用指针记号来引用数组元素。在这个例子中,引用 start[i] 表示要读取 start 所指位置之后第 i 个位置上的那个字节。

可以看到,虽然浮点数据和整数数据编码的都是数值 12,345,但它们的字节模式截然不同:整数是 0x00003039,浮点数是 0x4640E400。一般而言,这两种格式使用不同的编码方案。把这些十六进制模式展开成二进制形式并适当移动之后,会发现 13 个匹配的位,用一列星号标示如下:

```
  0   0   0   0   3   0   3   9
00000000000000000011000000111001
            *************
      4   6   4   0   E   4   0   0
  01000110010000001110010000000000
```

这并非巧合。在学习浮点格式时,我们还会回到这个例子。

练习 2.5　考虑下面三次对 `show_bytes` 的调用:

```
int a = 0x12345678;
byte_pointer ap = (byte_pointer) &a;
show_bytes(ap, 1); /* A. */
show_bytes(ap, 2); /* B. */
show_bytes(ap, 3); /* C. */
```

指出每次调用在小端法机器上和大端法机器上会打印出什么值:

A. 小端法: 大端法:

B. 小端法: 大端法:

C. 小端法: 大端法:

练习 2.6　利用 `show_int` 和 `show_float`,我们确定整数 2607352 的十六进制表示是 0x0027C8F8,而浮点数 3510593.0 的十六进制表示是 0x4A1F23E0。

A. 写出这两个十六进制值的二进制表示。

B. 把这两个位串相对移动,使匹配的位数达到最多。有多少位匹配?

C. 位串的哪些部分不匹配?

2.1.4 表示字符串

C 中的字符串被编码成一个以空字符(其值为 0)结尾的字符数组。每个字符都用某种标准编码表示,最常用的是 ASCII 字符编码。因此,用参数"12345"和 6(把结尾字符也包括进来)运行我们的例程 show_bytes,会得到结果 31 32 33 34 35 00。注意,十进制数字 x 的 ASCII 码恰好是 0x3x,而结尾字节的十六进制表示为 0x00。在任何以 ASCII 作为字符编码的系统上都会得到同样的结果,与字节顺序和字长的约定无关。因此,与二进制数据相比,文本数据具有更强的平台无关性。

练习 2.7 下面对 show_bytes 的调用会打印出什么结果?

```
const char *m = "mnopqr";
show_bytes((byte_pointer) m, strlen(m));
```

注意,字母'a' 到'z' 的 ASCII 码是从 0x61 到 0x7A。

2.1.5 表示代码

考虑下面这个 C 函数:

```
1  int sum(int x, int y) {
2      return x + y;
3  }
```

在我们的样例机器上编译时,会生成具有如下字节表示的机器码:

Linux 32 55 89 e5 8b 45 0c 03 45 08 c9 c3

Windows 55 89 e5 8b 45 0c 03 45 08 5d c3

Sun 81 c3 e0 08 90 02 00 09

Linux 64 55 48 89 e5 89 7d fc 89 75 f8 03 45 fc c9 c3

我们发现,指令的编码是不同的。不同类型的机器使用不同且互不兼容的指令与编码。即使在相同的处理器上运行不同的操作系统,两者的编码约定也有差异,因此它们二进制不兼容。二进制代码很少能在机器与操作系统的不同组合之间移植。

计算机系统的一个基本概念是:从机器的视角看,程序不过是字节序列。机器对原始的源程序一无所知,或许只有为帮助调试而维护的一些辅助表是例外。在第 3 章学习机器级编程时,这一点会看得更清楚。

C 语言新知 指针的创建与解引用

在图 2.4 的第 13 行、第 17 行和第 21 行,我们看到了两种运算的用法,正是这两种运算造就了 C(并因此包括 C++)的独特风格。C 的取地址(address of)运算符 & 创建一个指针。在这三行中,表达式 &x 创建一个指针,指向保存变量 x 所指对象的那个位置。这个指针的类型取决于 x 的类型,因此这三个指针的类型分别是 int *、float * 和 void **。(void * 是一种特殊的指针,不附带任何类型信息。)

强制类型转换运算符把一个数据类型转换为另一个数据类型。因此,转换 (byte_pointer) &x 表明,无论指针 &x 之前是什么类型,程序现在都将把对数据的引用当作指向 unsigned char 类型数据的指针。这里给出的转换并不改变实际的指针;它们只是告诉编译器,按照新的数据类型来解释被指向的数据。

边栏 生成 ASCII 表

执行命令 man ascii,就会显示一个 ASCII 字符编码表。

2.1.6 布尔代数简介

二进制值是计算机编码、存储和操作信息的核心，围绕着对 0 和 1 这两个值的研究，已经发展出了一个丰富的数学知识体系。它始于 George Boole（1815—1864）在 1850 年前后的工作，因此被称为布尔代数（Boolean algebra）。Boole 观察到，把逻辑值真（true）和假（false）编码为二进制值 1 和 0，就可以建立一种能够刻画逻辑推理基本原理的代数。

最简单的布尔代数定义在两元素集合 $\{0, 1\}$ 上。图 2.7 定义了这种代数中的几种运算。我们用来表示这些运算的符号，特意选得与 C 语言位级运算所用的符号相同，

图 2.7 布尔代数的运算。二进制值 1 和 0 编码逻辑值“真”和“假”，而运算 ~、&、| 和 ^ 分别编码逻辑运算非、与、或和异或

~		&	0	1			0	1	^	0	1
	0	0	0	0		0	0	1		0	1
	1	0	0	1		1	1	1		1	0

正如将在后文讨论的那样。布尔运算 ~ 对应逻辑运算非（not），用符号 $\neg$ 表示。也就是说，当 P 不为真时，我们就说 $\neg P$ 为真，反之亦然。相应地，~p 在 p 等于 0 时等于 1，反之亦然。布尔运算 & 对应逻辑运算与（and），用符号 $\wedge$ 表示。当 P 为真且 Q 为真时，我们说 $P \wedge Q$ 成立。相应地，只有当 $p = 1$ 且 $q = 1$ 时，p & q 才等于 1。布尔运算 | 对应逻辑运算或（or），用符号 $\vee$ 表示。当 P 为真或 Q 为真时，我们说 $P \vee Q$ 成立。相应地，当 $p = 1$ 或 $q = 1$ 时，p | q 等于 1。布尔运算 ^ 对应逻辑运算异或（exclusive-or），用符号 $\oplus$ 表示。当 P 和 Q 中恰好有一个为真时，我们说 $P \oplus Q$ 成立。相应地，当 $p = 1$ 且 $q = 0$，或者 $p = 0$ 且 $q = 1$ 时，p ^ q 等于 1。

Claude Shannon（1916—2001）后来创立了信息论这一领域。他最早把布尔代数和数字逻辑联系在了一起。在 1937 年的硕士论文中，他证明了布尔代数可以应用于机电继电器网络的设计与分析。尽管此后计算机技术已经取得了长足的进步，但布尔代数仍然在数字系统的设计与分析中扮演着核心角色。

边栏 用于文本编码的 Unicode 标准

ASCII 字符集适合对英文文档编码，但它几乎没有什么特殊字符，例如法语中的’ç’。用它对希腊语、俄语和中文等语言写的文档编码是完全不合适的。多年来，人们为不同语言的文本编码开发出了各种各样的方法。Unicode 协会（Unicode Consortium）制定的文本编码标准是目前最全面、接受程度最广的。当前的 Unicode 标准（7.0 版）收录的字符超过 100,000 个，支持的语言范围很广，包括古埃及语和古巴比伦语。难能可贵的是，Unicode 技术委员会否决了一项提案，该提案要求为克林贡语（Klingon）——电视连续剧《星际迷航》中虚构文明的语言——制定一种标准写法。

基本编码称为 Unicode 的“通用字符集”（Universal Character Set），用 32 位表示字符。这看起来似乎要求每段文本的每个字符都占 4 个字节。不过，也可以采用其他编码：常用的字符只需 1 个或 2 个字节，不常用的字符则占用更多字节。特别是，UTF-8 表示把每个字符编码成字节序列，标准 ASCII 字符使用的单字节编码与它们在 ASCII 中的编码完全相同，这蕴含着：所有 ASCII 字节序列在 UTF-8 中的含义与在 ASCII 中相同。

Java 程序设计语言在字符串表示中使用了 Unicode。C 也有支持 Unicode 的程序库。

我们还可以把四种布尔运算扩展到位向量(bit vector)——长度 w 固定、由若干 0 和 1 组成的串——上来使用。我们定义位向量上的运算的方式,是把运算应用到操作数中相互匹配的元素上。设 a 和 b 分别表示位向量 $[a_{w-1}, a_{w-2}, \ldots, a_0]$ 和 $[b_{w-1}, b_{w-2}, \ldots, b_0]$。我们定义 a & b 也是一个长度为 w 的位向量,其中第 i 个元素等于 a_i & b_i(对 $0 \le i < w$)。运算 |、^ 和 ~ 也以类似的方式扩展到位向量上。

作为例子,考虑 $w = 4$ 的情形,参数为 $a =$ `[0110]` 和 $b =$ `[1100]`。那么四个运算 a & b、a | b、a ^ b 和 ~b 给出

```
   0110      0110      0110
&  1100   |  1100   ^  1100   ~  1100
   ----      ----      ----      ----
   0100      1110      1010      0011
```

练习 2.8 填写下表,给出对位向量进行各种布尔运算求值的结果。

网络边栏 DATA:BOOL 布尔代数与布尔环的更多内容

布尔运算 |、& 和 ~ 作用在长度为 w 的位向量上,就构成了一个布尔代数(Boolean algebra),其中 w 可以是任意大于 0 的整数。最简单的是 $w = 1$ 的情形,这时只有两个元素;在更一般的情形中,长度为 w 的位向量共有 2^w 个。布尔代数的许多性质与整数的算术相同。例如,正如乘法对加法有分配律,写成 $a \cdot (b+c) = (a \cdot b) + (a \cdot c)$,布尔运算 & 对 | 也有分配律,写成 a & $(b$ | $c) = (a$ & $b)$ | $(a$ & $c)$。除此之外,布尔运算 | 对 & 也满足分配律,于是我们可以写 a | $(b$ & $c) = (a$ | $b)$ & $(a$ | $c)$,却不能说 $a + (b \cdot c) = (a + b) \cdot (a + c)$ 对所有整数都成立。

当我们考虑 ^、& 和 ~ 这些运算作用在长度为 w 的位向量上时,就得到另一种数学形式,称为布尔环(Boolean ring)。布尔环与整数算术有许多共同的性质。例如,整数算术的一个性质是:每个值 x 都有一个加法逆元 $-x$,使得 $x + (-x) = 0$。布尔环也有类似的性质,其中 ^ 是这里的"加法"运算,但此时每个元素都是它自己的加法逆元。也就是说,对任意值 a 都有 a ^ $a = 0$,这里的 0 表示全零的位向量。对单个位可以验证这一点,因为 0 ^ 0 = 1 ^ 1 = 0,而且这一性质同样可以扩展到位向量。即使把各项重新排列、按不同的顺序合并,这个性质依然成立,所以 $(a$ ^ $b)$ ^ $a = b$。正如我们将在练习 2.10 中探索的那样,这个性质会带来一些有趣的结果和巧妙的技巧。

位向量的一个有用的应用是表示有限集合。用位向量 $[a_{w-1}, \ldots, a_1, a_0]$ 可以编码集合 $\{0, 1, \ldots, w-1\}$ 的任意子集 A,其中 $a_i = 1$ 当且仅当 $i \in A$。例如,回忆一下我们约定的写法:a_{w-1} 在左、a_0 在右,位向量 $a =$ `[01101001]` 编码集合 $A = \{0, 3, 5, 6\}$,而位向量 $b =$ `[01010101]` 编码集合 $B = \{0, 2, 4, 6\}$。按照这种集合编码方式,布尔运算 | 和 & 分别对应集合并和集合交,~ 对应集合补。继续前面的例子,运算 a & b 得到位向量 `[01000001]`,而 $A \cap B = \{0, 6\}$。

我们将在许多实际应用中看到用位向量编码集合的做法。例如在第 8 章中,我们将看到有许多种不同的信号可以中断程序的执行。通过指定位向量掩

运算	结果
a	[01001110]
b	[11100001]
~a	
~b	
a &b	
a | b	
a ^ b	

码(bit-vector mask),我们可以有选择地允许或禁止不同的信号:位位置 i 上的 1 表示信号 i 被允许,而 0 表示被禁止。这样,掩码就表示了被允许的信号的集合。

练习 2.9 计算机通过混合三种不同颜色的光——红、绿、蓝——在视频屏幕或液晶显示器上生成彩色图像。设想一个简单的方案:三盏不同的灯,每一盏都可以打开或关闭,把光投射到一块玻璃屏上:

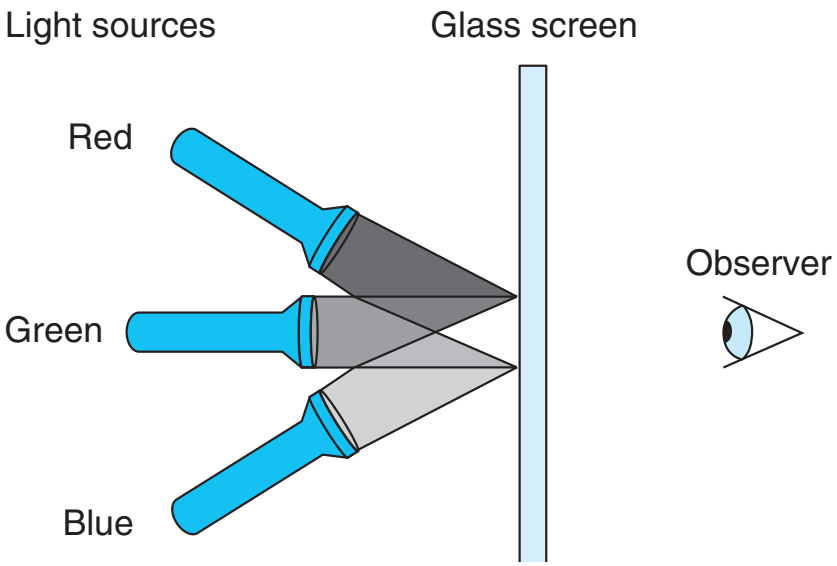


图 PP2.9 (示意图)

然后,根据光源 R、G 和 B 的关闭(0)或打开(1),可以创造出八种不同的颜色:

这八种颜色中的每一种都可以表示成长度为 3 的位向量,于是我们可以对它们应用布尔运算。

A. 一种颜色的补色(complement)就是把原来开着的灯关掉、把原来关着的灯打开而得到的颜色。上面列出的八种颜色中,每一种的补色是什么?

R	G	B	颜色
0	0	0	黑色
0	0	1	蓝色
0	1	0	绿色
0	1	1	青色
1	0	0	红色
1	0	1	品红色
1	1	0	黄色
1	1	1	白色

B. 描述对下列颜色应用布尔运算所产生的效果：

```
蓝色 | 绿色 = _________
黄色 & 青色 = _________
红色 ^ 品红色 = _________
```

2.1.7 C 语言中的位级运算

C 语言的一个很有用的特性是支持位级布尔运算。事实上，我们前面用来表示布尔运算的符号，正是 C 语言所使用的符号：| 表示 or，& 表示 and，~ 表示 not，^ 表示 exclusive-or。这些运算可以应用于任何“整型”（integral）数据类型，包括图 2.3 列出的所有类型。下面是对数据类型 `char` 进行表达式求值的一些例子：

C 表达式	二进制表示	二进制结果	十六进制结果
`~0x41`	`~[0100 0001]`	`[1011 1110]`	`0xBE`
`~0x00`	`~[0000 0000]`	`[1111 1111]`	`0xFF`
`0x69 &0x55`	`[0110 1001] &[0101 0101]`	`[0100 0001]`	`0x41`
`0x69 | 0x55`	`[0110 1001] | [0101 0101]`	`[0111 1101]`	`0x7D`

从这些例子可以看出，要确定一个位级表达式的效果，最好的办法是：先把十六进制的参数展开成二进制表示，在二进制下执行运算，然后再转换回十六进制。

练习 2.10 作为 $a \hat{} a = 0$（对任何位向量 a 都成立）这一性质的一个应用，考虑下面的程序：

```
1  void inplace_swap(int *x, int *y) {
2      *y = *x ^ *y;  /* 第 1 步 */
3      *x = *x ^ *y;  /* 第 2 步 */
4      *y = *x ^ *y;  /* 第 3 步 */
5  }
```

顾名思义，我们声称这个过程的效果是交换指针变量 `x` 和 `y` 所指位置上存储的值。注意，与通常交换两个值的技术不同，我们不需要第三个位置来临时

存放其中一个值。这种交换方式在性能上没有任何好处，它只是一项供人玩味的智力游戏。

设 x 和 y 所指位置上的初始值分别为 a 和 b。请填写下面的表格，给出该过程每一步执行之后这两个位置上存储的值。请利用 ^ 的性质证明确实达到了期望的效果。回忆一下，每个元素都是它自身的加法逆元（也就是说，a ^ $a = 0$）。

步骤	*x	*y
初始	a	b
第 1 步		
第 2 步		
第 3 步		

练习 2.11 有了练习 2.10 的函数 inplace_swap 之后，你决定编写一段代码，通过从数组两端向中间逐步交换元素来倒转数组中的元素。

于是你写出了下面这个函数：

```
1  void reverse_array(int a[], int cnt) {
2      int first, last;
3      for (first = 0, last = cnt-1;
4          first <= last;
5          first++,last--)
6          inplace_swap(&a[first], &a[last]);
7  }
```

当你把这个函数用在一个包含元素 1、2、3、4 的数组上时，发现数组不出所料地变成了 4、3、2、1。可是，当你在一个包含 1、2、3、4、5 的数组上试它时，却惊讶地发现数组变成了 5、4、0、2、1。事实上你会发现，这段代码对偶数长度的数组总是能得到正确结果，但只要数组长度是奇数，它就会把中间那个元素设置为 0。

A. 对于一个长度为奇数 cnt $= 2k + 1$ 的数组，在函数 reverse_array 的最后一次迭代中，变量 first 和 last 的值各是多少？

B. 为什么这一次对函数 inplace_swap 的调用会把数组元素设置为 0？

C. 对 reverse_array 的代码做怎样的简单修改，就能消除这个问题？

位级运算的一个常见用途是实现掩码运算（masking operation）。在一个字中，掩码（mask）是一个位模式，它指出了一个被选中的位的集合。举例来说，掩码 0xFF（最低 8 位为 1）表示一个字的低位字节。位级运算 x & 0xFF 生成一个值，由 x 的最低有效字节组成，其余所有字节都设置为 0。例如，当 $x =$ 0x89ABCDEF 时，该表达式的结果为 0x000000EF。表达式 ~0 会生成一个

全为 1 的掩码，与数据表示的大小无关。当数据类型 `int` 为 32 位时，同一个掩码也可以写成 `0xFFFFFFFF`，但那样可移植性就要差一些。

练习 2.12 针对下面的值，写出用变量 `x` 表示的 C 表达式。你的代码应该对任何字长 $w \geq 8$ 都适用。作为参考，我们给出了当 $x =$ `0x87654321`、$w = 32$ 时这些表达式求值的结果：

A. x 的最低有效字节，其余所有位都设置为 0。`[0x00000021]`

B. x 中除最低有效字节以外的所有字节都取反，最低有效字节保持不变。`[0x789ABC21]`

C. 最低有效字节设置为全 1，x 的其余所有字节保持不变。`[0x876543FF]`

练习 2.13 数字设备公司（Digital Equipment）的 VAX 计算机从 20 世纪 70 年代末到 80 年代末都是一款非常流行的机器。它没有实现布尔运算 and 和 or 的指令，取而代之的是指令 `bis`（位设置，bit set）和 `bic`（位清除，bit clear）。这两条指令都接受一个数据字 x 和一个掩码字 m。它们生成一个结果 z，z 由 x 的各个位组成，并按照 m 的各个位加以修改。使用 `bis`，这种修改是在 m 为 1 的每个位位置上把 z 设置为 1。使用 `bic`，这种修改是在 m 为 1 的每个位位置上把 z 设置为 0。

要弄清这些运算与 C 位级运算之间的关系，假设我们已有实现位设置和位清除运算的函数 `bis` 和 `bic`，希望只用这两个函数来实现计算位级运算 `|` 和 `^` 的函数，而不使用任何其他 C 运算。请在下面的代码中填上空缺的部分。提示：用 C 表达式写出操作 bis 和 bic。

```
/* 声明实现操作 bis 和 bic 的函数 */
int bis(int x, int m);
int bic(int x, int m);

/* 只用对函数 bis 和 bic 的调用来计算 x|y */
int bool_or(int x, int y) {
    int result = ________;
    return result;
}

/* 只用对函数 bis 和 bic 的调用来计算 x^y */
int bool_xor(int x, int y) {
    int result = ________;
    return result;
}
```

2.1.8 C 语言中的逻辑运算

C 还提供了一组逻辑运算符 `||`、`&&` 和 `!`，对应于逻辑中的或、与和非运算。逻辑运算很容易与位级运算相混淆，但它们的行为截然不同。逻辑运算把任何

非零参数都当作代表真,把参数 0 当作代表假。它们返回 1 或 0,分别表示结果为真或为假。下面是对表达式求值的一些例子:

表达式	结果
`!0x41`	`0x00`
`!0x00`	`0x01`
`!!0x41`	`0x01`
`0x69 &&0x55`	`0x01`
`0x69 || 0x55`	`0x01`

可以看到,只有在参数只限于 0 或 1 这一特殊情况下,位级运算的行为才会与对应的逻辑运算相一致。

逻辑运算符 && 和 || 与它们对应的位级运算 & 和 | 之间的第二个重要区别是:如果只对第一个参数求值就能确定表达式的结果,逻辑运算符就不会对第二个参数求值。因此,举例来说,表达式 `a && 5/a` 绝不会造成除以零,而表达式 `p && *p++` 也绝不会造成对空指针的解引用。

练习 2.14 假设 a 和 b 的字节值分别为 `0x55` 和 `0x46`。请填写下表,指出下列各个 C 表达式的字节值:

表达式	值	表达式	值
`a &b`		`a &&b`	
`a | b`		`a || b`	
`~a | ~b`		`!a || !b`	
`a &!b`		`a &&~b`	

练习 2.15 只使用位级运算和逻辑运算,写出一个等价于 `x == y` 的 C 表达式。也就是说,当 x 和 y 相等时它返回 1,否则返回 0。

2.1.9 C 语言中的移位运算

C 还提供了一组移位运算,用于把位模式向左和向右移动。对于一个位表示为 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 的操作数 x,C 表达式 `x << k` 生成一个值,其位表示为 $[x_{w-k-1}, x_{w-k-2}, \ldots, x_0, 0, \ldots, 0]$。也就是说,$x$ 向左移动 k 位,丢弃最高的 k 位,并在右端补上 k 个 0。移位量应该是一个介于 0 和 $w-1$ 之间的值。移位运算从左往右结合,所以 `x << j << k` 等价于 `(x << j) << k`。

与之对应,还有一种右移运算,在 C 中写作 `x >> k`,但它的行为有些微妙。一般而言,机器支持两种形式的右移:

逻辑 逻辑右移在左端补 k 个 0,得到结果 $[0, \ldots, 0, x_{w-1}, x_{w-2}, \ldots x_k]$。

算术 算术右移在左端补上 k 个最高有效位的副本，得到结果 $[x_{w-1}, \cdots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots x_k]$。这种约定看上去有些特殊，但是我们会看到，它对操作有符号整数数据很有用。

作为例子，下表给出了对 8 位参数 x 的两个不同值应用各种移位运算的效果：

运算	值 1	值 2
参数 x	[01100011]	[10010101]
x << 4	[0011*0000*]	[0101*0000*]
x >> 4(逻辑)	[*0000*0110]	[*0000*1001]
x >> 4(算术)	[*0000*0110]	[*1111*1001]

表中斜体的数字表示填充在右端(左移时)或左端(右移时)的值。可以看到，除一个条目外，其余都是用 0 来填充。唯一的例外是把 [10010101] 算术右移的情况：它的最高有效位是 1，所以用 1 作为填充值。

C 标准并未精确定义对有符号数应当使用哪种类型的右移——算术右移和逻辑右移都可以使用。遗憾的是，这意味着任何假定其中某种形式的代码都可能遇到可移植性问题。不过在实践中，几乎所有编译器/机器组合对有符号数据都使用算术右移，而许多程序员也都假定为这种情况。与之相反，对无符号数据来说，右移就必须使用逻辑右移。

与 C 不同，Java 对右移应当如何执行有明确的定义。表达式 x >> k 把 x 算术右移 k 位，而 x >>> k 把它逻辑右移。

练习 2.16 填写下表，说明各种移位运算对单字节数值的影响。思考移位运算的最好办法是使用二进制表示：先把初始值转换为二进制，执行移位，然后再转换回十六进制。每个答案都应当是 8 位二进制数字或 2 位十六进制数字。

				逻辑		算术	
a		a << 2		a >> 3		a >> 3	
十六进制	二进制	二进制	十六进制	二进制	十六进制	二进制	十六进制
0xD4							
0x64							
0x72							
0x44							

边栏　对较大的 k 值进行移位

对于一个由 w 个位组成的数据类型，当移位量为某个 $k \geq w$ 的值时，移动应当产生什么效果？例如，假设数据类型 `int` 的 $w = 32$，计算下面几个表达式会发生什么：

```
int      lval = 0xFEDCBA98 << 32;
int      aval = 0xFEDCBA98 >> 36;
unsigned uval = 0xFEDCBA98u >> 40;
```

C 标准有意不对这种情况下应当如何处理作出规定。在许多机器上，对一个 w 位的值进行移位时，移位指令只考虑移位量的低 $\log_2 w$ 位，因此移位量按 $k \bmod w$ 计算。例如取 $w = 32$，上面三条移位指令实际上分别按移位量 0、4、8 来计算，得到的结果为

```
lval      0xFEDCBA98
aval      0xFFEDCBA9
uval      0x00FEDCBA
```

不过，C 程序并不保证一定得到这样的行为，因此移位量应当始终保持小于字长。另一方面，Java 则明确要求移位量必须按我们前面所示的取模方式计算。

边栏　移位运算的优先级问题

写出 `1<<2 + 3<<4` 这种表达式，本意大概是 $(1 << 2) + (3 << 4)$。然而在 C 中，头一种写法等价于 `1 << (2+3) << 4`，因为加法（以及减法）的优先级高于移位。左结合规则随后使这个表达式被加上括号，成为 `(1 << (2+3)) << 4`，得到的值是 512，而不是想要的 52。

在 C 表达式中弄错优先级是程序错误的常见来源，而且这类错误往往很难靠肉眼检查发现。拿不准时，就加上括号！

2.2 整数表示

本节介绍用位对整数进行编码的两种不同方式——一种只能表示非负数，另一种则能表示负数、零和正数。我们将会看到，它们无论在数学性质上还是在机器级实现上都紧密相关。我们还考察把一个已编码的整数扩展或收缩以适配不同长度的表示时会产生什么影响。

图 2.8 列出了我们引入的一组数学术语，它们用来精确定义和刻画计算机如何对整数数据进行编码和运算。这些术语将随着讲解逐步引入。此处先列出该表，供随时查阅。

2.2.1 整型数据类型

C 支持多种整型数据类型（integral data type）——即表示整数的有限取值范围的数据类型。图 2.9 和图 2.10 给出了这些类型，以及它们在“典型的”32 位和 64 位程序中所能取的范围。每种类型都可以用关键字 `char`、`short`、`long` 指定大小，还可以指明所表示的数是否全部非负（声明为 `unsigned`），或者有可能为负（默认情况）。如图 2.3 所示，为不同大小分配的字节数取决于程序是按 32 位还是按 64 位编译。根据字节分配的不同，各大小允许表示的取值范围也不同。表中唯一与机器相关的取值范围属于大小指示符 `long`。大多数 64 位程序使用 8 字节表示，其取值范围比 32 位程序使用的 4 字节表示宽得多。

图 2.8　整数数据与运算的术语。下标 w 表示数据表示的位数。"页码"栏给出了定义相应内容的页。

符号	含义	页码
$\mathrm{B2T}_w$	函数:二进制转补码	100
$\mathrm{B2U}_w$	函数:二进制转无符号数	98
$\mathrm{U2B}_w$	函数:无符号数转二进制	100
$\mathrm{U2T}_w$	函数:无符号数转补码	107
$\mathrm{T2B}_w$	函数:补码转二进制	101
$\mathrm{T2U}_w$	函数:补码转无符号数	107
TMin_w	常量:补码的最小值	101
TMax_w	常量:补码的最大值	101
UMax_w	常量:无符号数的最大值	99
$+_w^t$	运算:补码加法	126
$+_w^u$	运算:无符号数加法	121
$*_w^t$	运算:补码乘法	133
$*_w^u$	运算:无符号数乘法	132
$-_w^t$	运算:补码求负	131
$-_w^u$	运算:无符号数求负	125

关于图 2.9 和图 2.10, 需要注意的一个重要特征是: 取值范围不对称——负数部分的取值范围比正数部分多一个值。等到考察负数是如何表示的时候, 我们就会明白其中的原因。

图 2.9　32 位程序中 C 整型数据类型的典型取值范围

C 数据类型	最小值	最大值
`[signed] char`	−128	127
`unsigned char`	0	255
`short`	−32,768	32,767
`unsigned short`	0	65,535
`int`	−2,147,483,648	2,147,483,647
`unsigned`	0	4,294,967,295
`long`	−2,147,483,648	2,147,483,647
`unsigned long`	0	4,294,967,295
`int32_t`	−2,147,483,648	2,147,483,647
`uint32_t`	0	4,294,967,295
`int64_t`	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
`uint64_t`	0	18,446,744,073,709,551,615

C 标准为每种数据类型定义了它必须能够表示的最小取值范围。如图 2.11 所示, 这些范围与图 2.9 和图 2.10 所示的典型实现相同或更小。特别地, 除定

图 2.10 64 位程序中 C 整型数据类型的典型取值范围

C 数据类型	最小值	最大值
[signed] char	−128	127
unsigned char	0	255
short	−32,768	32,767
unsigned short	0	65,535
int	−2,147,483,648	2,147,483,647
unsigned	0	4,294,967,295
long	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	0	18,446,744,073,709,551,615
int32_t	−2,147,483,648	2,147,483,647
uint32_t	0	4,294,967,295
int64_t	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint64_t	0	18,446,744,073,709,551,615

长数据类型之外，我们看出标准只要求正数与负数的取值范围对称。我们还看到，int 类型可以用 2 字节数字来实现，尽管这主要是 16 位机器时代的历史遗留。我们还看到，long 可以用 4 字节数字实现，对 32 位程序而言通常也确实如此。定长数据类型保证其取值范围恰好是图 2.9 中典型数值所给出的范围，包括正负数之间的不对称。

C 语言新知 C、C++ 和 Java 中的有符号数与无符号数

C 和 C++ 都既支持有符号数（默认）又支持无符号数。Java 只支持有符号数。

图 2.11 C 整型数据类型的保证取值范围。C 标准要求这些数据类型的取值范围至少达到本表给出的值。

C 数据类型	最小值	最大值
[signed] char	−127	127
unsigned char	0	255
short	−32,767	32,767
unsigned short	0	65,535
int	−32,767	32,767
unsigned	0	65,535
long	−2,147,483,647	2,147,483,647
unsigned long	0	4,294,967,295
int32_t	−2,147,483,648	2,147,483,647
uint32_t	0	4,294,967,295
int64_t	−9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint64_t	0	18,446,744,073,709,551,615

2.2.2 无符号数的编码

设想一个 w 位的整数数据类型。我们把位向量写作 $\vec{x}$，表示整个向量；或写作 $[x_{w-1}, x_{w-2}, \ldots, x_0]$，表示向量中的各个位。把 $\vec{x}$ 看作一个按二进制记法写出的数，就得到 $\vec{x}$ 的无符号（unsigned）解释。在这种编码中，每个位 x_i 取值 0 或 1；当取 1 时，表示应把数值 2^i 计入整个数值。我们可以把这个解释表达为一个函数 $\mathrm{B2U}_w$（binary to unsigned，“二进制到无符号数”，长度为 w）：

图 2.12 $w = 4$ 时的无符号数示例。当二进制表示中的位 i 取值为 1 时，它对数值的贡献为 2^i。

位向量	数值
`[0001]`	1
`[0101]`	5
`[1011]`	11
`[1111]`	15

原理：无符号数编码的定义

对于向量 $\vec{x} = [x_{w-1}, x_{w-2}, \ldots, x_0]$：

$$\mathrm{B2U}_w(\vec{x}) \doteq \sum_{i=0}^{w-1} x_i\, 2^i \tag{2.1}$$

在这个等式中，记号 $\doteq$ 表示左端被定义为等于右端。函数 $\mathrm{B2U}_w$ 把长度为 w 的 0 和 1 的串映射为非负整数。作为例子，图 2.12 给出了 B2U 对下述各情形的从位向量到整数的映射：

$$\begin{aligned}
&\mathrm{B2U}_4([\texttt{0001}]) = 0\cdot 2^3 + 0\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = 0+0+0+1 = 1\\
&\mathrm{B2U}_4([\texttt{0101}]) = 0\cdot 2^3 + 1\cdot 2^2 + 0\cdot 2^1 + 1\cdot 2^0 = 0+4+0+1 = 5\\
&\mathrm{B2U}_4([\texttt{1011}]) = 1\cdot 2^3 + 0\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0 = 8+0+2+1 = 11\\
&\mathrm{B2U}_4([\texttt{1111}]) = 1\cdot 2^3 + 1\cdot 2^2 + 1\cdot 2^1 + 1\cdot 2^0 = 8+4+2+1 = 15
\end{aligned} \tag{2.2}$$

在图中，每个位位置 i 用一条长度为 2^i、指向右边的蓝色条表示。位向量所对应的数值，就等于取值为 1 的那些位所对应的条的长度之和。

考虑用 w 位可以表示的取值范围。最小值由位向量 $[00\ldots0]$ 给出，其整数值为 0；最大值由位向量 $[11\ldots1]$ 给出，其整数值为 UMax_w：

$$\mathrm{UMax}_w \doteq \sum_{i=0}^{w-1} 2^i = 2^w - 1$$

以 4 位的情形为例，我们有 $\text{UMax}_4 = \text{B2U}_4([\texttt{1111}]) = 2^4 - 1 = 15$。因此，函数 B2U_w 可以定义为一个映射 $\text{B2U}_w : \{0,1\}^w \to \{0, \ldots, \text{UMax}_w\}$。

无符号二进制表示有一个重要性质：0 到 $2^w - 1$ 之间的每个数都有一个唯一的 w 位值编码。例如，十进制数 11 作为无符号 4 位数只有一种表示，即 `[1011]`。我们把这一点当作一条数学原理突出地提出来，先陈述这条原理，再加以解释。

原理：无符号数编码的唯一性

函数 B2U_w 是一个双射。

数学术语双射（bijection）指一个双向起作用的函数 f：它把值 x 映射为值 y，其中 $y = f(x)$；它也能反向运算，因为对每个 y，都存在唯一的 x 使 $f(x) = y$。这由反函数 f^{-1} 给出，在我们的例子里就是 $x = f^{-1}(y)$。函数 B2U_w 把每个长度为 w 的位向量映射为 0 到 $2^w - 1$ 之间一个唯一的数，并且它有逆函数——我们称之为 U2B_w（unsigned to binary，“无符号数到二进制”），它把 0 到 $2^w - 1$ 范围内的每个数映射为一个唯一的 w 位模式。

2.2.3 补码编码

许多应用还要求表示负值。有符号数在计算机中最常见的表示称为补码（two's-complement）形式。补码的定义方式是把字的最高有效位解释为具有负的权。我们可以把这个解释表达为一个函数 B2T_w（binary to two's complement，“二进制到补码”，长度为 w）：

原理：补码编码的定义

对于向量 $\vec{x} = [x_{w-1}, x_{w-2}, \ldots, x_0]$：

$$\text{B2T}_w(\vec{x}) \doteq -x_{w-1}\, 2^{w-1} + \sum_{i=0}^{w-2} x_i\, 2^i \tag{2.3}$$

最高有效位 x_{w-1} 也称为符号位（sign bit）。它的“权”是 -2^{w-1}，即在无符号表示中该位权重的相反数。当符号位置为 1 时，所表示的值为负；置为 0 时，值为非负。作为例子，图 2.13 给出了 B2T 对下述各情形的从位向量到整数的映射：

$$\begin{aligned}
\text{B2T}_4([\texttt{0001}]) &= -0 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 0 + 0 + 1 = 1 \\
\text{B2T}_4([\texttt{0101}]) &= -0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 0 + 4 + 0 + 1 = 5 \\
\text{B2T}_4([\texttt{1011}]) &= -1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 0 + 2 + 1 = -5 \\
\text{B2T}_4([\texttt{1111}]) &= -1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = -8 + 4 + 2 + 1 = -1
\end{aligned} \tag{2.4}$$

在图中，我们用一个指向左边的灰色条来表明符号位具有负权。于是，位向量所对应的数值，就由可能存在的指向左边的灰色条和那些指向右边的蓝色条组合而成。

图 2.13　$w = 4$ 时的补码数示例。位 3 充当符号位；当它置为 1 时，对数值的贡献为 $-2^3 = -8$。这一权重用一条指向左边的灰色条表示。

位向量	数值
[0001]	1
[0101]	5
[1011]	−5
[1111]	−1

我们看到，图 2.12 和图 2.13 的位模式相同（公式 2.2 与公式 2.4 也是如此），但是当最高有效位为 1 时，两者得到的值不同：一边它的权是 $+8$，另一边是 -8。

考虑一个 w 位补码数所能表示的取值范围。最小的可表示值由位向量 $[10\ldots0]$ 给出（把具有负权的位设置为 1，其余各位清 0），其整数值为 $\text{TMin}_w \doteq -2^{w-1}$。最大值由位向量 $[01\ldots1]$ 给出（把具有负权的位清 0，其余各位置 1），其整数值为 TMax_w：

$$\text{TMax}_w \doteq \sum_{i=0}^{w-2} 2^i = 2^{w-1} - 1$$

以 4 位的情形为例，我们有 $\text{TMin}_4 = \text{B2T}_4([\texttt{1000}]) = -2^3 = -8$，以及 $\text{TMax}_4 = \text{B2T}_4([\texttt{0111}]) = 2^2 + 2^1 + 2^0 = 4 + 2 + 1 = 7$。

可以看出，B2T_w 是长度为 w 的位模式到 TMin_w 与 TMax_w 之间各数的一个映射，记作 $\text{B2T}_w : \{0,1\}^w \to \{\text{TMin}_w, \ldots, \text{TMax}_w\}$。与无符号表示一样，可表示范围内的每个数都有一个唯一的 w 位补码编码。由此得到关于补码数的一条原理，与无符号数的那条类似：

原理：补码编码的唯一性

函数 B2T_w 是一个双射。

我们定义函数 T2B_w（two's complement to binary，“补码到二进制”）为 B2T_w 的逆。也就是说，对一个满足 $\text{TMin}_w \le x \le \text{TMax}_w$ 的数 x，$\text{T2B}_w(x)$ 就是编码 x 的那个（唯一的）w 位模式。

练习 2.17　设 $w = 4$，对每个可能的十六进制数字，无论按无符号解释还是按补码解释，都可以赋予它一个数值。请按这两种解释，写出公式 2.1 和公式 2.3 的求和式中非零的 2 的幂，以完成下表：

十六进制	位向量	$B2U_4(\vec{x})$	$B2T_4(\vec{x})$
0xA	[1010]	$2^3 + 2^1 = 10$	$-2^3 + 2^1 = -6$
0x1	_____	_____	_____
0xB	_____	_____	_____
0x2	_____	_____	_____
0x7	_____	_____	_____
0xC	_____	_____	_____

图 2.14 给出了不同字长下几个重要数字的位模式和数值。前三行分别用 $UMax_w$、$TMin_w$ 和 $TMax_w$ 的值刻画了可表示整数的取值范围。在接下来的讨论中，我们会经常提到这三个特殊值。当字长 w 可以从上下文推断出来，或者对该讨论无关紧要时，我们就略去下标 w，把这几个值记作 UMax、TMin 和 TMax。

关于这些数字，有几点值得强调。第一，如图 2.9 和图 2.10 所示，补码的取值范围是不对称的：$|TMin| = |TMax| + 1$，也就是说，TMin 没有与之对应的正数。我们将看到，这会给补码运算带来一些特殊的性质，并且可能成为隐蔽程序错误的来源。这种不对称性源于：有一半的位模式（符号位为 1 的那些）表示负数，另一半（符号位为 0 的那些）表示非负数。0 属于非负数，因此非负数能够表示的正数比负数少一个。第二，无符号数的最大值刚好超过补码最大值的两倍：$UMax = 2TMax + 1$。所有在补码记法中表示负数的位模式，在无符号表示中都变成了正值。

图 2.14 重要数字。同时给出了数值及其十六进制表示。

值	$w = 8$	$w = 16$	$w = 32$	$w = 64$
$UMax_w$	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
	255	65,535	4,294,967,295	18,446,744,073,709,551,615
$TMin_w$	0x80	0x8000	0x80000000	0x8000000000000000
	−128	−32,768	−2,147,483,648	−9,223,372,036,854,775,808
$TMax_w$	0x7F	0x7FFF	0x7FFFFFFF	0x7FFFFFFFFFFFFFFF
	127	32,767	2,147,483,647	9,223,372,036,854,775,807
−1	0xFF	0xFFFF	0xFFFFFFFF	0xFFFFFFFFFFFFFFFF
0	0x00	0x0000	0x00000000	0x0000000000000000

边栏 关于定长整数类型的更多内容

对某些程序来说，数据类型必须用特定大小的表示来编码，这一点至关重要。例如，若要编写程序使一台机器按照某种标准协议在因特网上通信，数据类型与协议所规

定的类型相容就很重要。我们已经看到，某些 C 数据类型（尤其是 `long`）在不同机器上的取值范围不同；而且实际上，C 标准对任何数据类型都只规定了最小的取值范围，而不是确切的取值范围。虽然我们可以选择与大多数机器上的标准表示相容的数据类型，但这并不能保证可移植性。

前面已经遇到过定长整数类型的 32 位和 64 位版本（图 2.3）；它们是一大类数据类型中的一部分。ISO C99 标准在文件 `stdint.h` 中引入了这类整数类型。该文件定义了一组数据类型，声明形式为 `intN_t` 和 `uintN_t`，分别指定 N 位有符号整数和 N 位无符号整数，N 可以取多种值。N 的确切取值随实现而异，不过大多数编译器允许取 8、16、32 和 64。于是，把一个变量的类型声明为 `uint16_t`，就毫无歧义地声明了一个 16 位无符号变量；声明为 `int32_t`，就是一个 32 位有符号变量。

与这些数据类型配套的还有一组宏，对 N 的每个取值定义了最小值和最大值，名字的形式为 `INTN_MIN`、`INTN_MAX` 和 `UINTN_MAX`。

用定长数据类型进行格式化打印，需要使用一组宏，它们以与系统相关的方式展开成格式串。例如，打印类型分别为 `int32_t` 和 `uint64_t` 的变量 x 与 y 的值，可以像下面这样调用 `printf`：

```
printf("x = %" PRId32 ", y = %" PRIu64 "\n", x, y);
```

当按 64 位程序编译时，宏 `PRId32` 展开成字符串 `"d"`，而 `PRIu64` 展开成一对字符串 `"l" "u"`。当 C 预处理器遇到一串只用空格（或其他空白字符）分隔的字符串常量时，会把它们拼接在一起。于是，上面的 `printf` 调用变成

```
printf("x = %d, y = %lu\n", x, y);
```

使用这些宏可以保证，无论代码如何编译，都会生成正确的格式串。

图 2.14 还给出了常量 -1 和 0 的表示。注意，-1 的位表示与 UMax 相同——是一串全 1。数值 0 在两种表示中都用一串全 0 表示。

C 标准并不要求有符号整数必须以补码形式表示，但几乎所有的机器都这样做。如果程序员希望程序在所有可能的机器上达到最大的可移植性，那么除图 2.11 所给出的取值范围之外，不应假设可表示数值的任何特定取值范围，也不应假设有符号数的任何特定表示。另一方面，许多程序是按照“有符号数采用补码表示、并具有图 2.9 和图 2.10 所示的“典型”取值范围”这一假设编写的，这些程序可以移植到广泛的机器和编译器上。C 库中的文件 `<limits.h>` 定义了一组常量，界定了编译器所运行的那台机器上各种整数数据类型的取值范围。例如，它定义了常量 `INT_MAX`、`INT_MIN` 和 `UINT_MAX`，描述有符号整数和无符号整数的取值范围。对一台数据类型 `int` 为 w 位的补码机器来说，这些常量对应于 TMax_w、TMin_w 和 UMax_w 的值。

边栏　有符号数的其他表示方法

有符号数还有另外两种标准表示方法。

反码 (ones' complement)。它与补码相同，只是最高有效位的权是 $-(2^{w-1}-1)$，而不是 -2^{w-1}：$\mathrm{B2O}_w(\vec{x}) \doteq -x_{w-1}\,(2^{w-1}-1) + \sum_{i=0}^{w-2} x_i\,2^i$ **原码 (sign magnitude)**。最高有效位是符号位，它决定其余各位被赋以负权还是正权：$\mathrm{B2S}_w(\vec{x}) \doteq (-1)^{x_{w-1}} \sum_{i=0}^{w-2} x_i\,2^i$ 这两种表示都有一个奇怪的性质：数 0 有两种不同的编码。对这两种表示来说，[00 ... 0] 都被解释为 +0。值 −0 用原码可以表示为 [10 ... 0]，用反码可以表示为 [11 ... 1]。虽然基于反码表示的机器在过去确实建造过，但几乎所有现代机器都使用补码。我们将会看到，原码编码用在浮点数上。

注意撇号位置的不同：two's complement 与 ones' complement。"two's complement"（补码）一词源于这样一个事实：对非负的 x，$-x$ 的 w 位表示可以按 $2^w - x$ 计算出来（用到单个 two）。"ones' complement"（反码）一词的来历则是：按这种记法，$-x$ 可以用 $[111 \ldots 1] - x$ 计算出来（用到多个 ones）。

Java 标准对整数数据类型的取值范围和表示规定得非常明确。它要求补码表示，并且取值范围与图 2.10 中 64 位情形给出的完全相同。在 Java 中，单字节数据类型称为 `byte` 而不是 `char`。这样细致的要求，是为了使 Java 程序无论运行在什么机器或操作系统上，行为都完全相同。

为了更好地理解补码表示，考虑下面这个代码示例：

```
1  short x = 12345;
2  short mx = -x;
3
4  show_bytes((byte_pointer) &x, sizeof(short));
5  show_bytes((byte_pointer) &mx, sizeof(short));
```

在大端法机器上运行时，这段代码打印 `30 39` 和 `cf c7`，表明 x 的十六进制表示是 `0x3039`，而 mx 的十六进制表示是 `0xCFC7`。把它们展开成二进制，得到 x 的位模式 `[0011000000111001]` 和 mx 的位模式 `[1100111111000111]`。如图 2.15 所示，公式 2.3 对这两个位模式给出的值分别是 12,345 和 −12,345。

练习 2.18 在第 3 章中，我们将会查看由反汇编器生成的清单。反汇编器是把一个可执行程序文件转换回更易读的 ASCII 形式的程序。这类文件中包含许多十六进制数，它们通常以补码形式表示数值。能够认出这些数并理解其含义（例如是负数还是正数）是一项重要的技能。

对下面清单中标注 A ~ I 的各行（见右侧的标号），把指令名称（sub、mov 和 add）右边给出的十六进制数值（32 位补码形式）转换成相应的十进制数：

```
4004d0:    48 81 ec e0 02 00 00    sub    $0x2e0,%rsp
 ↪  A.
4004d7:    48 8b 44 24 a8          mov    -0x58(%rsp),%rax
 ↪  B.
4004dc:    48 03 47 28             add    0x28(%rdi),%rax
 ↪  C.
4004e0:    48 89 44 24 d0          mov    %rax,-0x30(%rsp)
 ↪  D.
```

图 2.15 12,345 与 $-12{,}345$ 的补码表示，以及 53,191 的无符号表示。注意后两者具有相同的位表示。

	12,345		$-12{,}345$		53,191	
权	位	值	位	值	位	值
1	1	1	1	1	1	1
2	0	0	1	2	1	2
4	0	0	1	4	1	4
8	1	8	0	0	0	0
16	1	16	0	0	0	0
32	1	32	0	0	0	0
64	0	0	1	64	1	64
128	0	0	1	128	1	128
256	0	0	1	256	1	256
512	0	0	1	512	1	512
1,024	0	0	1	1,024	1	1,024
2,048	0	0	1	2,048	1	2,048
4,096	1	4,096	0	0	0	0
8,192	1	8,192	0	0	0	0
16,384	0	0	1	16,384	1	16,384
$\pm 32{,}768$	0	0	1	$-32{,}768$	1	32,768
总计		12,345		$-12{,}345$		53,191

```
4004e5:    48 8b 44 24 78          mov    0x78(%rsp),%rax
 ↪  E.
4004ea:    48 89 87 88 00 00 00    mov    %rax,0x88(%rdi)
 ↪  F.
4004f1:    48 8b 84 24 f8 01 00    mov    0x1f8(%rsp),%rax
 ↪  G.
4004f8:    00
4004f9:    48 03 44 24 08          add    0x8(%rsp),%rax
4004fe:    48 89 84 24 c0 00 00    mov    %rax,0xc0(%rsp)
 ↪  H.
400505:    00
400506:    48 8b 44 d4 b8          mov    -0x48(%rsp,%rdx,8),%rax
 ↪  I.
```

2.2.4 有符号数和无符号数之间的转换

C 允许在不同数值数据类型之间进行强制类型转换。例如，设变量 x 声明为 `int` 类型，u 声明为 `unsigned` 类型。表达式 `(unsigned)` x 把 x 的值转换成一个无符号值，而 `(int)` u 把 u 的值转换成一个有符号整数。把有符号值转换成无符号值，或者反过来，应当有怎样的效果？从数学的角度可以设想几种不同的约定。显然，我们希望保住任何在两种形式下都能表示的值。另一方面，把一个负值转换成无符号值，可能会得到 0；把一个太大以致补码形式无法表

示的无符号值转换过来，可能会得到 TMax。不过，对大多数 C 实现来说，这个问题的答案基于位级视角，而不是数值视角。

例如，考虑下面的代码：

```
1  short        int v  = -12345;
2  unsigned short   uv  = (unsigned short) v;
3  printf("v = %d, uv = %u\n", v, uv);
```

在补码机器上运行时，这段代码产生如下的输出：

```
v = -12345, uv = 53191
```

这里我们看到，强制类型转换的效果是保持各个位的值不变，只改变对这些位的解释方式。在图 2.15 中我们看到，−12,345 的 16 位补码表示与 53,191 的 16 位无符号表示是完全相同的。从 `short` 转换成 `unsigned short` 改变了数值，但没有改变位表示。

类似地，考虑下面的代码：

```
1  unsigned u = 4294967295u;    /* UMax */
2  int      tu = (int) u;
3  printf("u = %u, tu = %d\n", u, tu);
```

在补码机器上运行时，这段代码产生如下的输出：

```
u = 4294967295, tu = -1
```

从图 2.14 可以看出，当字长为 32 位时，无符号形式下表示 4,294,967,295 (UMax_{32})的位模式，与补码形式下表示 -1 的位模式完全相同。把 unsigned 转换成 int 时，底层的位表示保持不变。

这就是大多数 C 实现处理相同字长的有符号数与无符号数之间转换的一般规则——数值可能会改变，但位模式不会改变。我们把这个想法写成更数学化的形式。前面定义的函数 U2B_w 和 T2B_w，分别把数映射为无符号或补码形式的位表示。也就是说，对取值范围 $0 \le x < \text{UMax}_w$ 之内的整数 x，函数 $\text{U2B}_w(x)$ 给出 x 唯一的 w 位无符号表示；类似地，当 x 处于 $\text{TMin}_w \le x \le \text{TMax}_w$ 范围内时，函数 $\text{T2B}_w(x)$ 给出 x 唯一的 w 位补码表示。

现在定义函数 T2U_w：$\text{T2U}_w(x) \doteq \text{B2U}_w(\text{T2B}_w(x))$。这个函数把一个介于 TMin_w 与 TMax_w 之间的数变成一个介于 0 与 UMax_w 之间的数，两者的位表示相同，唯一的区别是参数的表示是补码，而结果是无符号的。类似地，对于 0 到 UMax_w 之间的 x，函数 U2T_w 定义为 $\text{U2T}_w(x) \doteq \text{B2T}_w(\text{U2B}_w(x))$，它给出的数以 x 的无符号表示作为其补码表示。

回到前面的例子，从图 2.15 可以看到 $\text{T2U}_{16}(-12{,}345) = 53{,}191$，以及 $\text{U2T}_{16}(53{,}191) = -12{,}345$。也就是说，十六进制写作 `0xCFC7` 的 16 位位模式，既是 $-12{,}345$ 的补码表示，也是 53,191 的无符号表示。同时还注意到 $12{,}345 + 53{,}191 = 65{,}536 = 2^{16}$。这个性质可以推广成同一个位模式的两个数值（补码与无符号）之间的关系。类似地，从图 2.14 可以看到 $\text{T2U}_{32}(-1) = 4{,}294{,}967{,}295$，以及 $\text{U2T}_{32}(4{,}294{,}967{,}295) = -1$。也就是说，UMax 的无符号位表示与 -1 的补码位表示相同。我们还能看到这两个数之间的关系：$1 + \text{UMax}_w = 2^w$。

于是我们看到，函数 T2U 描述了把补码数转换成其对应无符号数的过程，而 U2T 则做相反方向的转换。这两个函数描述了大多数 C 实现中这些数据类型之间强制类型转换的效果。

练习 2.19 利用你在解答练习 2.17 时填写的那张表，填写下表，描述函数 T2U_4：

x	$\text{T2U}_4(x)$
-1	_____
-5	_____
-6	_____
-4	_____
1	_____
8	_____

我们已经通过几个例题看到了给定一个位模式时补码值与无符号值之间的关系。这个关系可以表述为函数 T2U 的一条性质：

原理：从补码到无符号数的转换

对于满足 $\text{TMin}_w \le x \le \text{TMax}_w$ 的 x：

$$\text{T2U}_w(x) = \begin{cases} x + 2^w, & x < 0 \\ x, & x \ge 0 \end{cases} \tag{2.5}$$

例如，我们已经看到 $\text{T2U}_{16}(-12{,}345) = -12{,}345 + 2^{16} = 53{,}191$，还看到 $\text{T2U}_w(-1) = -1 + 2^w = \text{UMax}_w$。

这条性质可以通过比较公式 2.1 与公式 2.3 推导出来。

推导：从补码到无符号数的转换

比较公式 2.1 与公式 2.3 可以看出，对位模式 $\vec{x}$，如果计算差 $\text{B2U}_w(\vec{x}) - \text{B2T}_w(\vec{x})$，那么从位 0 到 $w-2$ 的加权和会彼此抵消，只剩下 $\text{B2U}_w(\vec{x}) -$

$\mathrm{B2T}_w(\vec{x}) = x_{w-1}\left(2^{w-1} - (-2^{w-1})\right) = x_{w-1}2^w$。这就给出关系 $\mathrm{B2U}_w(\vec{x}) = \mathrm{B2T}_w(\vec{x}) + x_{w-1}2^w$。于是有

$$\mathrm{B2U}_w(\mathrm{T2B}_w(x)) = \mathrm{T2U}_w(x) = x + x_{w-1}2^w \tag{2.6}$$

在 x 的补码表示中，位 x_{w-1} 决定了 x 是否为负，由此得到公式 2.5 的两种情形。

作为一个例子，图 2.16 就 $w = 4$ 的情形比较了函数 B2U 与 B2T 给各位模式赋予的值。对补码的情形，最高有效位充当符号位，我们把它画成一个指向左边的灰色条。对无符号的情形，这一位的权是正的，我们用一个指向右边的黑色条来表示。从补码转换到无符号时，最高有效位的权从 -8 变为 $+8$。结果是，补码表示中取负值的那些值，在无符号表示中增加了 $2^4 = 16$。于是 -5 变成 $+11$，-1 变成 $+15$。

图 2.16 $w = 4$ 时无符号表示与补码表示的比较。最高有效位的权在补码中为 -8，在无符号中为 $+8$，两者相差 16。

位向量	无符号值	补码值
[0000]	0	0
[0001]	1	1
[0010]	2	2
[0011]	3	3
[0100]	4	4
[0101]	5	5
[0110]	6	6
[0111]	7	7
[1000]	8	−8
[1001]	9	−7
[1010]	10	−6
[1011]	11	−5
[1100]	12	−4
[1101]	13	−3
[1110]	14	−2
[1111]	15	−1

图 2.17 说明了函数 T2U 的一般行为。如图所示，把一个有符号数映射为它的无符号对应数时，负数被转换成很大的正数，而非负数保持不变。

练习 2.20 解释公式 2.5 如何应用于你在解答练习 2.19 时生成的表中的各个条目。

在另一个方向上，可以把无符号数 u 与它的有符号对应数 $\mathrm{U2T}_w(u)$ 之间的关系陈述为：

原理：从无符号数到补码的转换

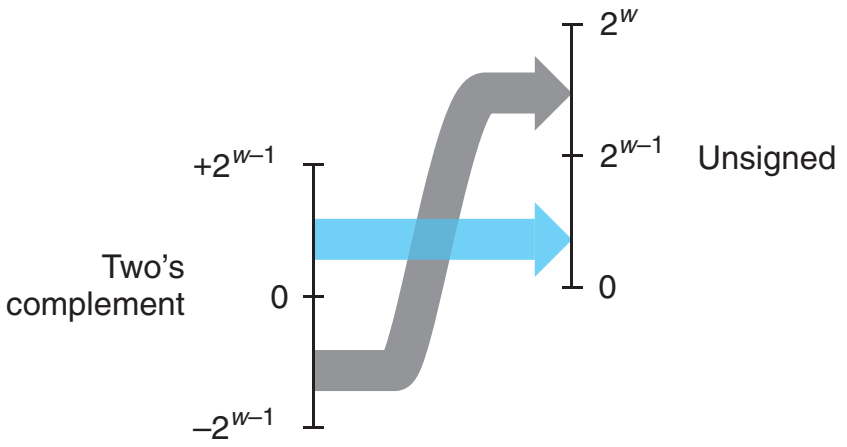


图 2.17 （示意图）

对于满足 $0 \le u \le \text{UMax}_w$ 的 u：

$$\text{U2T}_w(u) = \begin{cases} u, & u \le \text{TMax}_w \\ u - 2^w, & u > \text{TMax}_w \end{cases} \tag{2.7}$$

这条原则可以证明如下：

推导：从无符号数到补码的转换

令 $\vec{u} = \text{U2B}_w(u)$。这个位向量同时也就是 $\text{U2T}_w(u)$ 的补码表示。把公式 2.1 与公式 2.3 合并起来，可得

$$\text{U2T}_w(u) = -u_{w-1}2^w + u \tag{2.8}$$

在 u 的无符号表示中，位 u_{w-1} 决定了 u 是否大于 $\text{TMax}_w = 2^{w-1} - 1$，由此得到公式 2.7 的两种情形。

函数 U2T 的行为如图 2.18 所示。对较小的（$\le \text{TMax}_w$）数，从无符号数到有符号数的转换保持数值不变。较大的（$> \text{TMax}_w$）数则被转换成负值。

总结一下，我们考察了在无符号表示与补码表示之间进行双向转换的效果。对取值范围 $0 \le x \le \text{TMax}_w$ 之内的 x，有 $\text{T2U}_w(x) = x$ 且 $\text{U2T}_w(x) = x$，也就是说，这个范围内的数具有相同的无符号表示和补码表示。对这个范围之外的值，转换要么加上 2^w，要么减去 2^w。例如，$\text{T2U}_w(-1) = -1 + 2^w = \text{UMax}_w$——最接近零的负数映射为最大的无符号数。在另一个极端，可以看到 $\text{T2U}_w(\text{TMin}_w) = -2^{w-1} + 2^w = 2^{w-1} = \text{TMax}_w + 1$——最小的负数映射为一个恰好略微超出正的补码数范围的无符号数。用图 2.15 的例子来验证：$\text{T2U}_{16}(-12{,}345) = 65{,}536 + (-12{,}345) = 53{,}191$。

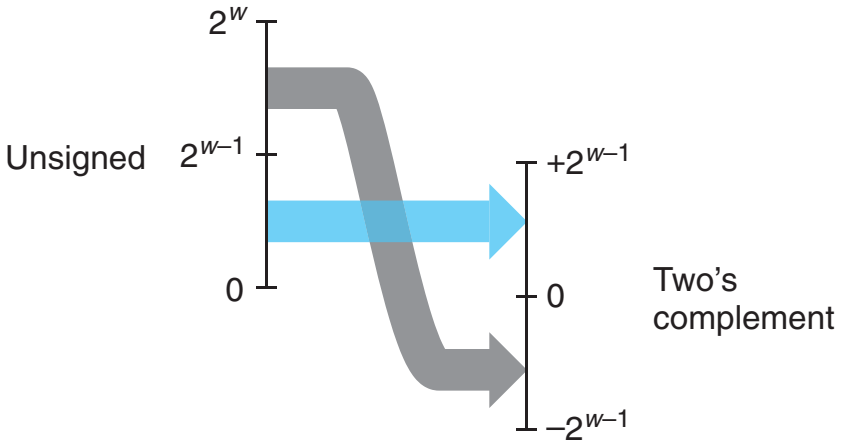


图 2.18 （示意图）

2.2.5 C 语言中的有符号数与无符号数

如图 2.9 和图 2.10 所示，C 对其所有整数数据类型既支持有符号运算，也支持无符号运算。虽然 C 标准没有规定有符号数采用哪种特定表示，但几乎所有机器都使用补码。一般而言，大多数数默认都是有符号的。例如，声明一个诸如 12345 或 `0x1A2B` 这样的常量时，这个值就被当作有符号数。在后面加上后缀字符 U 或 u，就创建一个无符号常量，例如 `12345U` 或 `0x1A2Bu`。

C 语言允许在有符号数与无符号数之间转换。虽然 C 标准没有精确规定这种转换应该如何完成，但大多数系统都遵循一条规则：底层的位表示保持不变。这条规则意味着，从无符号数转换到有符号数时应用函数 $\mathrm{U2T}_w$，从有符号数转换到无符号数时应用函数 $\mathrm{T2U}_w$，其中 w 是该数据类型的位数。

转换可能由显式的强制类型转换(cast)引起，比如在下面的代码中：

```
1  int tx, ty;
2  unsigned ux, uy;
3
4  tx = (int) ux;
5  uy = (unsigned) ty;
```

另外，当一个类型的表达式被赋给另一个类型的变量时，转换也可能隐式地发生，如下面的代码所示：

```
1  int tx, ty;
2  unsigned ux, uy;
3
4  tx = ux; /* 转换为有符号 */
5  uy = ty; /* 转换为无符号 */
```

用 printf 打印数值时，%d、%u 和%x 分别用来以有符号十进制、无符号十进制和十六进制的格式打印一个数。注意，printf 并不利用任何类型信息，因此完全可以用%u 打印一个 int 类型的值，也可以用%d 打印一个 unsigned 类型的值。例如，考虑下面的代码：

```
1  int x = -1;
2  unsigned u = 2147483648; /* 2 的 31 次方 */
3
4  printf("x = %u = %d\n", x, x);
5  printf("u = %u = %d\n", u, u);
```

按 32 位程序编译时，这段代码打印出下面的内容：

```
x = 4294967295 = -1
u = 2147483648 = -2147483648
```

在这两种情况下，printf 先把这个字当作表示一个无符号数来打印一次，然后又把它当作表示一个有符号数来打印一次。从中可以看到类型转换的过程在起作用：$\mathrm{T2U}_{32}(-1) = \mathrm{UMax}_{32} = 2^{32} - 1$，而 $\mathrm{U2T}_{32}(2^{31}) = 2^{31} - 2^{32} = -2^{31} = \mathrm{TMin}_{32}$。

C 语言对同时含有有符号数和无符号数的表达式的处理方式，会产生一些可能非直观的行为。当一次运算中两个操作数一个是有符号的、另一个是无符号的时候，C 会隐式地把有符号的那个参数转换为无符号数，然后执行运算，并假定两个数都非负。我们将看到，这一约定对标准算术运算几乎没有什么影响，但对于诸如 < 和 > 这样的关系运算符，它却会导致一些非直观的结果。图 2.19 给出了一些关系表达式及其求值结果，其中假定数据类型 int 采用 32 位补码表示。考虑比较 -1 < 0U：由于第二个操作数是无符号的，第一个操作数会被隐式地转换为无符号数，于是这个表达式等价于比较 4294967295U < 0U（回忆一下，$\mathrm{T2U}_w(-1) = \mathrm{UMax}_w$），结果当然为假。其余情况可以用类似的分析来理解。

练习 2.21 假设各个表达式在采用补码运算的机器上以 32 位程序的方式执行时求值，请仿照图 2.19 的样式填写下表，说明强制类型转换和关系运算的效果：

2.2.6 扩展一个数字的位表示

一个常见的运算是：保持数值不变，而在字长不同的整数之间进行转换。当然，如果目标数据类型太小，不足以表示想要的值，这种转换可能无法做到。不过，从较小的数据类型转换为较大的数据类型则总是可能的。

图 2.19 C 语言类型提升规则的影响。非直观的情形用“*”标出。当比较运算的任一操作数是无符号数时，另一个操作数会被隐式地转换为无符号数。关于为什么把 TMin_{32} 写成 `-2,147,483,647-1`，请见网络边栏 data:tmin。

表达式	类型	求值
`0 == 0U`	无符号	1
`-1 < 0`	有符号	1
`-1 < 0U`	无符号	0*
`2147483647 > -2147483647-1`	有符号	1
`2147483647U > -2147483647-1`	无符号	0*
`2147483647 > (int) 2147483648U`	有符号	1*
`-1 > -2`	有符号	1
`(unsigned) -1 > -2`	无符号	1

表达式	类型	求值
`-2147483647-1 == 2147483648U`		
`-2147483647-1 < 2147483647`		
`-2147483647-1U < 2147483647`		
`-2147483647-1 < -2147483647`		
`-2147483647-1U < -2147483647`		

网络边栏 DATA:TMIN 在 C 中书写 TMin

在图 2.19 和练习 2.21 中，我们小心地把 TMin_{32} 的值写成了 `-2,147,483,647-1`。为什么不干脆把它写成 `-2,147,483,648` 或 `0x80000000` 呢？看一下 C 的头文件 `limits.h`，你会发现其中用一种与我们的写法相似的方法来书写 TMin_{32} 和 TMax_{32}：

```
/* 一个 'signed int' 类型所能表示的最小值和最大值   */
#define INT_MAX    2147483647
#define INT_MIN    (-INT_MAX - 1)
```

遗憾的是，补码表示的非对称性与 C 类型转换规则之间一个奇特的相互作用，迫使我们不得不用这种不寻常的方式来书写 TMin_{32}。要理解这个问题，需要深入钻研 C 语言标准中一个比较晦涩的角落；不过，这有助于我们体会整数数据类型及其表示的诸多微妙之处。

要把一个无符号数转换成更大的数据类型，我们只需在表示的前面补上几个 0 就行了。这种运算称为零扩展（zero extension），用下面的原理来表达：

原理：无符号数的零扩展

设宽度为 w 的位向量 $\vec{u} = [u_{w-1}, u_{w-2}, \ldots, u_0]$ 和宽度为 w' 的位向量 $\vec{u}' = [0, \ldots, 0, u_{w-1}, u_{w-2}, \ldots, u_0]$，其中 $w' > w$。那么 $\mathrm{B2U}_w(\vec{u}) = \mathrm{B2U}_{w'}(\vec{u}')$。

从公式 2.1 给出的无符号编码定义出发，可以直接看出这条原理成立。

把一个补码数转换成更大的数据类型，规则则是执行一次符号扩展（sign extension），即在表示的前面加上最高有效位的多个副本。用下面的原理来表

达。为了突出符号位 x_{w-1} 在符号扩展中所起的作用，我们把符号位标成了蓝色。

原理：补码数的符号扩展

设宽度为 w 的位向量 $\vec{x} = [x_{w-1}, x_{w-2}, \ldots, x_0]$ 和宽度为 w' 的位向量 $\vec{x}' = [x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_0]$，其中 $w' > w$。那么 $\mathrm{B2T}_w(\vec{x}) = \mathrm{B2T}_{w'}(\vec{x}')$。

举例来说，考虑下面的代码：

```
1   short sx = -12345;                /* -12345 */
2   unsigned short usx = sx;          /* 53191 */
3   int x = sx;                       /* -12345 */
4   unsigned ux = usx;                /* 53191 */
5
6   printf("sx = %d:\t", sx);
7   show_bytes((byte_pointer) &sx, sizeof(short));
8   printf("usx = %u:\t", usx);
9   show_bytes((byte_pointer) &usx, sizeof(unsigned short));
10  printf("x   = %d:\t", x);
11  show_bytes((byte_pointer) &x, sizeof(int));
12  printf("ux = %u:\t", ux);
13  show_bytes((byte_pointer) &ux, sizeof(unsigned));
```

在一台采用补码表示的大端机器上按 32 位程序运行这段代码，它会打印出如下输出：

```
sx = -12345:      cf c7
usx = 53191:      cf c7
x   = -12345:     ff ff cf c7
ux = 53191:       00 00 cf c7
```

我们看到，在 16 位字长下，−12,345 的补码表示与 53,191 的无符号表示完全相同，但到了 32 位字长，两者就不一样了。具体来说，−12,345 的十六进制表示是 `0xFFFFCFC7`，而 53,191 的十六进制表示是 `0x0000CFC7`。前者经过了符号扩展——作为前导位，加上了最高有效位 1 的 16 个副本，它们合起来就是十六进制的 `0xFFFF`；后者则是用 16 个前导 0 扩展的，即十六进制的 `0x0000`。

举个例子，图 2.20 展示了按符号扩展从字长 $w = 3$ 扩展到 $w = 4$ 的结果。位向量 `[101]` 表示的值是 $-4 + 1 = -3$。应用符号扩展就得到位向量 `[1101]`，它表示的值是 $-8 + 4 + 1 = -3$。我们可以看到，当 $w = 4$ 时，两个最高有效位的合成值 $-8 + 4 = -4$，与 $w = 3$ 时符号位的值相同。类似地，位向量 `[111]` 和 `[1111]` 表示的值都是 -1。

有了这个直观认识，我们现在就可以证明符号扩展能够保持补码数的数值不变。

推导：补码数的符号扩展

图 2.20 从 $w=3$ 到 $w=4$ 的符号扩展示例。当 $w=4$ 时，最高 2 位的合成权值是 $-8+4=-4$，与 $w=3$ 时符号位的权值相同。

位向量	各位的权值	值
`[101]`	$-2^2+2^0 \quad (w=3)$	-3
`[1101]`	$-2^3+2^2+2^0 \quad (w=4)$	-3
`[111]`	$-2^2+2^1+2^0 \quad (w=3)$	-1
`[1111]`	$-2^3+2^2+2^1+2^0 \quad (w=4)$	-1

令 $w'=w+k$。我们要证明的是

$$\mathrm{B2T}_{w+k}([\underbrace{x_{w-1},\ldots,x_{w-1}}_{k\text{ 次}},x_{w-1},x_{w-2},\ldots,x_0])=\mathrm{B2T}_w([x_{w-1},x_{w-2},\ldots,x_0])$$

证明用对 k 归纳的方法。也就是说，只要能证明按 1 位做符号扩展时数值保持不变，那么在按任意位数做符号扩展时这个性质也成立。于是，任务就归结为证明

$$\mathrm{B2T}_{w+1}([x_{w-1},x_{w-1},x_{w-2},\ldots,x_0])=\mathrm{B2T}_w([x_{w-1},x_{w-2},\ldots,x_0])$$

用公式 2.3 展开等式的左端，得到：

$$\begin{aligned}
\mathrm{B2T}_{w+1}([x_{w-1},x_{w-1},x_{w-2},\ldots,x_0]) &= -x_{w-1}\,2^w+\sum_{i=0}^{w-1}x_i\,2^i \\
&= -x_{w-1}\,2^w+x_{w-1}\,2^{w-1}+\sum_{i=0}^{w-2}x_i\,2^i \\
&= -x_{w-1}(2^w-2^{w-1})+\sum_{i=0}^{w-2}x_i\,2^i \\
&= -x_{w-1}\,2^{w-1}+\sum_{i=0}^{w-2}x_i\,2^i \\
&= \mathrm{B2T}_w([x_{w-1},x_{w-2},\ldots,x_0])
\end{aligned}$$

用到的关键性质是 $2^w-2^{w-1}=2^{w-1}$。因此，加上一个权值为 -2^w 的位，并把那个权值为 -2^{w-1} 的位变成权值为 2^{w-1} 的位，两步合起来的效果就是保持原来的数值不变。

练习 2.22 应用公式 2.3，证明下面的每一个位向量都是 -4 的补码表示：

A. `[1100]`

B. `[11100]`

C. `[111100]`

请注意,第二个和第三个位向量可以从第一个通过符号扩展得到。

有一点值得强调: 数据大小之间的转换与有符号数和无符号数之间的转换,这两类转换的先后次序会影响程序的行为。考虑下面的代码:

```
1  short sx = -12345;    /* -12345 */
2  unsigned uy = sx;     /* 谜题!  */
3
4  printf("uy = %u:\t", uy);
5  show_bytes((byte_pointer) &uy, sizeof(unsigned));
```

在一台大端机器上运行这段代码,会打印出如下输出:

```
uy = 4294954951:        ff ff cf c7
```

这表明, 从 short 转换到 unsigned 时, 程序先改变大小, 然后才改变符号类型。也就是说, (unsigned) sx 等价于 (unsigned) (int) sx, 求值得到 4,294,954,951, 而不是 (unsigned) (unsigned short) sx——后者求值得到 53,191。事实上,C 标准要求的就是这种约定。

练习 2.23 考虑下面这两个 C 函数:

```
int fun1(unsigned word) {
    return (int) ((word << 24) >> 24);
}

int fun2(unsigned word) {
    return ((int) word << 24) >> 24;
}
```

假设这两个函数在采用补码运算的机器上按 32 位程序执行, 并且假设对有符号值做的右移是算术移位,对无符号值做的右移是逻辑移位。

A. 填写下表,说明这两个函数在若干示例参数下的效果。用十六进制表示处理会更方便;请记住,十六进制数字 8 到 F 的最高有效位都等于 1。

w	fun1(w)	fun2(w)
0x00000076		
0x87654321		
0x000000C9		
0xEDCBA987		

B. 用文字描述这两个函数各自完成的有用运算。

2.2.7 截断数字

现在假设我们不扩展一个值,而是缩减表示一个数所用的位数。这种情况会出现在比如下面的代码中:

```
1  int x = 53191;
2  short sx = (short) x; /* -12345 */
3  int y = sx;           /* -12345 */
```

把 x 强制转换为 short，会把一个 32 位的 int 截断成一个 16 位的 short。正如前面看到的，这个 16 位的位模式就是 −12,345 的补码表示。当把这个数转回 int 时，符号扩展会把高序 16 位置为 1，于是又得到了 −12,345 的 32 位补码表示。

把一个 w 位的数 $\vec{x} = [x_{w-1}, x_{w-2}, \ldots, x_0]$ 截断成一个 k 位的数时，我们丢弃高序的 $w-k$ 个位，得到位向量 $\vec{x}' = [x_{k-1}, x_{k-2}, \ldots, x_0]$。截断会改变数值——这是溢出的一种形式。对一个无符号数，我们可以直接刻画截断后所得的数值。

原理：无符号数的截断

设 $\vec{x}$ 为位向量 $[x_{w-1}, x_{w-2}, \ldots, x_0]$，$\vec{x}'$ 为把它截断到 k 位的结果：$\vec{x}' = [x_{k-1}, x_{k-2}, \ldots, x_0]$。令 $x = \mathrm{B2U}_w(\vec{x}), x' = \mathrm{B2U}_k(\vec{x}')$。那么 $x' = x \bmod 2^k$。

这个原理背后的直观理解很简单：所有被截断的位，其权值都具有 2^i（其中 $i \geq k$）的形式，因此在取模运算下每个这样的权值都化为零。这一结论可以形式化为下面的推导：

推导：无符号数的截断

对公式 2.1 应用取模运算，得到

$$\begin{aligned}
\mathrm{B2U}_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k &= \left(\sum_{i=0}^{w-1} x_i\, 2^i\right) \bmod 2^k \\
&= \left(\sum_{i=0}^{k-1} x_i\, 2^i\right) \bmod 2^k \\
&= \sum_{i=0}^{k-1} x_i\, 2^i \\
&= \mathrm{B2U}_k([x_{k-1}, x_{k-2}, \ldots, x_0])
\end{aligned}$$

在这个推导中，我们利用了这样的性质：对任意的 $i \geq k$，有 $2^i \bmod 2^k = 0$。

补码数的截断也有类似的性质，区别在于这种运算同时把最高有效位变成了符号位：

原理：补码数的截断

设 $\vec{x}$ 为位向量 $[x_{w-1}, x_{w-2}, \ldots, x_0]$，$\vec{x}'$ 为把它截断到 k 位的结果：$\vec{x}' = [x_{k-1}, x_{k-2}, \ldots, x_0]$。令 $x = \mathrm{B2T}_w(\vec{x}), x' = \mathrm{B2T}_k(\vec{x}')$。那么 $x' = \mathrm{U2T}_k(x \bmod 2^k)$。

在这个公式中，$x \bmod 2^k$ 将是一个 0 到 $2^k - 1$ 之间的数。把函数 $\mathrm{U2T}_k$ 应用到它上面，效果是把最高有效位 x_{k-1} 的权值从 2^{k-1} 转换成 -2^{k-1}。以把值 $x = 53{,}191$ 从 `int` 转换到 `short` 为例，就能看到这一点。因为 $2^{16} = 65{,}536 \geq x$，我们得到 $x \bmod 2^{16} = x$。但是当把这个数转换成 16 位补码数时，就得到 $x' = 53{,}191 - 65{,}536 = -12{,}345$。

推导：补码数的截断

用与无符号数截断类似的论证可以证明

$$\mathrm{B2T}_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k = \mathrm{B2U}_k([x_{k-1}, x_{k-2}, \ldots, x_0])$$

也就是说，$x \bmod 2^k$ 可以用一个位级表示为 $[x_{k-1}, x_{k-2}, \ldots, x_0]$ 的无符号数来表示。把这个数转换成补码数，便得到 $x' = \mathrm{U2T}_k(x \bmod 2^k)$。

概括起来，截断对无符号数的效果是：

$$\mathrm{B2U}_k([x_{k-1}, x_{k-2}, \ldots, x_0]) = \mathrm{B2U}_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k \tag{2.9}$$

而对补码数的效果则是：

$$\mathrm{B2T}_k([x_{k-1}, x_{k-2}, \ldots, x_0]) = \mathrm{U2T}_k(\mathrm{B2U}_w([x_{w-1}, x_{w-2}, \ldots, x_0]) \bmod 2^k) \tag{2.10}$$

练习 2.24 假设我们把一个 4 位数值（用十六进制数字 0 到 F 表示）截断成一个 3 位数值（用十六进制数字 0 到 7 表示）。填写下表，按照这些位模式的无符号解释和补码解释，说明这种截断对以下几个情况的效果：

十六进制		无符号		补码	
原始值	截断值	原始值	截断值	原始值	截断值
1	1	1		1	
3	3	3		3	
5	5	5		5	
C	4	12		−4	
E	6	14		−2	

请说明公式 2.9 和公式 2.10 如何适用于这些情况。

2.2.8 关于有符号数与无符号数的建议

正如我们已经看到的，从有符号数到无符号数的隐式转换会导致一些非直观的行为。非直观的特性常常导致程序错误，而涉及隐式转换细微之处的错误尤其难以察觉。这种强制类型转换发生时，代码中看不到任何明显的提示，因此程序员常常忽略它的影响。

下面的两个练习说明了隐式转换和无符号数据类型可能带来的一些微妙错误。

练习 2.25 下面这段代码试图对数组 a 的各元素求和,元素的个数由参数 length 给出:

```
1  /* WARNING:这是有缺陷的代码 */
2  float sum_elements(float a[], unsigned length) {
3      int i;
4      float result = 0;
5
6      for (i = 0; i <= length-1; i++)
7          result += a[i];
8      return result;
9  }
```

当参数 length 等于 0 时,这段代码本应返回 0.0,但实际上它碰到了一个存储器错误。请解释为什么会出现这种情况,并说明如何改正这段代码。

练习 2.26 你的任务是编写一个函数,判断一个字符串是否比另一个字符串长。你决定利用字符串库函数 strlen,它有如下声明:

```
/* Prototype for library function strlen */
size_t strlen(const char *s);
```

下面是你最初的尝试:

```
/* Determine whether string s is longer than string t */
/* WARNING: This function is buggy */
int strlonger(char *s, char *t) {
    return strlen(s) - strlen(t) > 0;
}
```

当你在一些样本数据上测试这段代码时,它似乎不能正常工作。你进一步调查后发现,把这段代码编译成 32 位程序时,在头文件 stdio.h 中通过 typedef 定义的数据类型 size_t 被定义为 unsigned。

A. 在哪些情况下,这个函数会产生不正确的结果?

B. 解释这个不正确的结果是如何产生的。

C. 说明如何修改代码,使它能够可靠地工作。

我们已经看到,无符号算术的种种微妙特性,特别是从有符号数到无符号数的隐式转换,会以多种方式造成错误或安全漏洞。避免这类 bug 的一种办法是永远不使用无符号数。事实上,除 C 以外很少有语言支持无符号整数。看来,这些语言的设计者认为无符号整数带来的麻烦超过了它的好处。例如,Java 只支持有符号整数,并要求用补码运算来实现它们。普通的右移运算符 >> 保证执行算术移位,而专门的运算符 >>> 则被定义为执行逻辑右移。

当我们想把一个字仅仅看作位的集合而不作任何数值解释时，无符号值非常有用；例如，把描述各种布尔条件的标志打包进一个字里时就是如此。地址天然就是无符号的，因此系统程序员发现无符号类型很有用。在实现模运算和多精度运算的数学软件包时，无符号值也很有用，因为在多精度运算中，一个数用多个字的数组来表示。

2.3 整数运算

许多初学者都会惊讶地发现，两个正数相加可能得到一个负的结果，而且比较 $x < y$ 得到的结果可能与比较 $x - y < 0$ 不同。这些现象都是计算机运算有限性的产物。理解计算机运算的这些微妙之处，可以帮助程序员写出更可靠的代码。

2.3.1 无符号加法

考虑两个非负整数 x 和 y，满足 $0 \leq x, y < 2^w$。这两个值都可以表示成一个 w 位的无符号数。然而，如果计算它们的和，其取值范围是 $0 \leq x+y \leq 2^{w+1}-2$。要表示这个和，可能需要 $w+1$ 个位。例如，图 2.21 画出了 x 和 y 都取 4 位表示时函数 $x+y$ 的图形。参数（标在横轴上）的范围是 0 到 15，但和的范围是 0 到 30。函数的形状是一个倾斜的平面（该函数在两个维度上都是线性的）。如果我们把这个和保存为一个 $(w+1)$ 位的数，再把它加到另一个值上，就可能需要 $w+2$ 个位，如此继续下去。这种不断持续的“字长

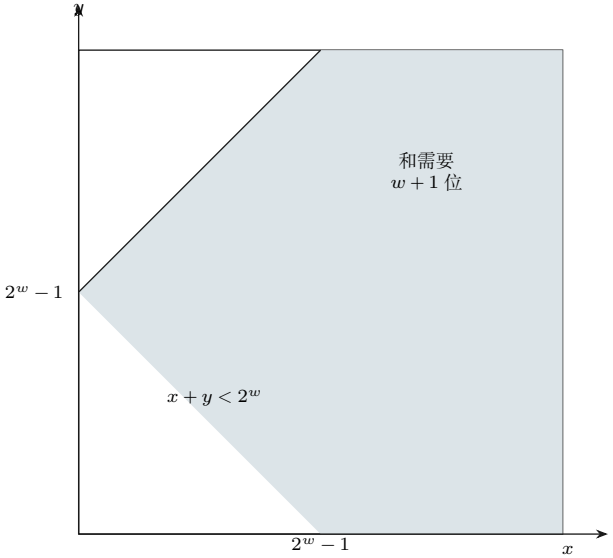


图 2.21　整数加法。字长为 w 时，和最大可能需要 $w+1$ 位，阴影区内的结果已超出 w 位能表示的范围。

膨胀”意味着,要把算术运算的结果完整地表示出来,所需的字长可以是任意大的,我们无法给出任何界限。有些编程语言(如 Lisp)实际支持任意大小(arbitrary size)的算术运算,从而允许任意大小的整数(当然,要在计算机内存允许的范围之内)。更常见的做法是,编程语言支持固定大小的算术运算,因此“加法”“乘法”这类运算与它们在整数上的相应运算有所不同。

设运算 $+_w^u$ 定义在参数 x 和 y(满足 $0 \leq x, y < 2^w$)上,它把整数和 $x+y$ 截断为 w 位长,再把结果看成一个无符号数,$+_w^u$ 的值就是截断后的结果。这可以刻画为模运算的一种形式:在 $x+y$ 的位级表示中,直接丢弃所有权大于 2^{w-1} 的位,就是按模 2^w 计算和。例如,考虑一个 4 位的数表示,取 $x=9$、$y=12$,二者的位表示分别为 [1001] 和 [1100]。和为 21,其 5 位表示为 [10101]。如果丢弃最高位,就得到 [0101],即十进制值 5。这与 21 mod 16 = 5 相符。

边栏 getpeername 中的安全漏洞

2002 年,参与 FreeBSD 开源操作系统项目的程序员发现,他们对库函数 getpeername 的实现存在一个安全漏洞。其代码的简化版本大致如下:

```
1  /*
2   * 代码漏洞示意, 其代码与 FreeBSD 的
3   * getpeername() 实现中发现的问题类似
4   */
5
6  /* 库函数 memcpy 的声明 */
7  void *memcpy(void *dest, void *src, size_t n);
8
9  /* 保存用户可访问数据的内核内存区域 */
10 #define KSIZE 1024
11 char kbuf[KSIZE];
12
13 /* 最多把 maxlen 字节从内核区域复制到用户缓冲区 */
14 int copy_from_kernel(void *user_dest, int maxlen) {
15     /* 字节数 len 取缓冲区大小与 maxlen 的较小值 */
16     int len = KSIZE < maxlen ? KSIZE : maxlen;
17     memcpy(user_dest, kbuf, len);
18     return len;
19 }
```

在这段代码中,我们给出了库函数 memcpy 的原型(第 7 行),它的设计用途是把指定数目的 n 个字节从内存的一个区域复制到另一个区域。

函数 copy_from_kernel 从第 14 行开始,其设计用途是把操作系统内核维护的一部分数据复制到指定的、用户可访问的内存区域。内核维护的大多数数据结构都不应该让用户读取,因为其中可能包含关于其他用户和系统上正在运行的其他作业的敏感信息,但标作 kbuf 的那个区域原本是打算让用户能够读取的。参数 maxlen 的预期含义是由用户分配、由参数 user_dest 指示的那个缓冲区的长度。这样,第 16 行的计算就能确保被复制的字节数不超过源缓冲区和目的缓冲区中任何一个的可用数量。

然而,假设某个恶意程序员编写代码,用一个负的 maxlen 值调用 copy_from_kernel。那么第 16 行的取最小值计算会把 len 算成这个值,接着

> 它作为参数 n 传给 memcpy。但请注意，参数 n 被声明为数据类型 size_t。这个数据类型在库文件 stdio.h 中声明（通过 typedef）。通常它被定义为 32 位程序的 unsigned、64 位程序的 unsigned long。由于参数 n 是无符号的，memcpy 会把它当成一个非常大的正数，并试图从内核区域向用户的缓冲区复制这么多字节。复制这么多字节（至少 2^{31} 个）实际上不会成功，因为程序在此过程中会遇到无效地址，但程序可能读取到它无权访问的内核内存区域。
> 可以看出，这个问题的起因是数据类型不匹配：在一处长度参数是有符号的，在另一处却是无符号的。这类不匹配可能是缺陷的来源，而且正如这个例子所表明的，甚至可能导致安全漏洞。所幸没有收到过程序员利用 FreeBSD 这个漏洞的报告。他们发布了安全公告“FreeBSD-SA-02:38.signed-error”，提醒系统管理员如何应用补丁来清除这一漏洞。把 copy_from_kernel 的参数 maxlen 声明为类型 size_t，使其与 memcpy 的参数 n 保持一致，就能修复这个缺陷。我们还应当把局部变量 len 和返回值也声明为类型 size_t。

于是可以把运算 $+_w^u$ 刻画如下：

原理：无符号加法

对满足 $0 \le x, y < 2^w$ 的 x 和 y：

$$x +_w^u y = \begin{cases} x + y, & x + y < 2^w \quad \text{正常} \\ x + y - 2^w, & 2^w \le x + y < 2^{w+1} \quad \text{溢出} \end{cases} \tag{2.11}$$

公式 2.11 的两种情形见图 2.22：左边的和 $x + y$ 映射到右边的无符号 w 位和 $x +_w^u y$。正常情形保持 $x + y$ 的值不变，而溢出情形的效果则是把这个和减去 2^w。

推导：无符号加法

一般地，如果 $x + y < 2^w$，那么该和的 $(w+1)$ 位表示的首位将为 0，因此把它丢弃不会改变数值。另一方面，如果 $2^w \le x + y < 2^{w+1}$，那么该和的 $(w+1)$ 位表示的首位将为 1，于是把它丢弃等价于从和中减去 2^w。

算术运算的完整整数结果若不能放进该数据类型的字长限制之内，就称为溢出（overflow）。正如公式 2.11 所表明的，当两个操作数之和达到 2^w 或更多时，就会发生溢出。图 2.23 画出了字长 $w = 4$ 时无符号加法函数的图像。和按模 $2^4 = 16$ 计算。当 $x + y < 16$ 时没有溢出，$x +_4^u y$ 就是 $x + y$，这对应图中标注为“正常”的那块倾斜平面。当 $x + y \ge 16$ 时，加法溢出，效果是把和减去 16，这对应图中标注为“溢出”的那块倾斜平面。

执行 C 程序时，溢出并不作为错误报出来。不过，有时我们可能想知道溢出到底发生了没有。

原理：检测无符号加法的溢出

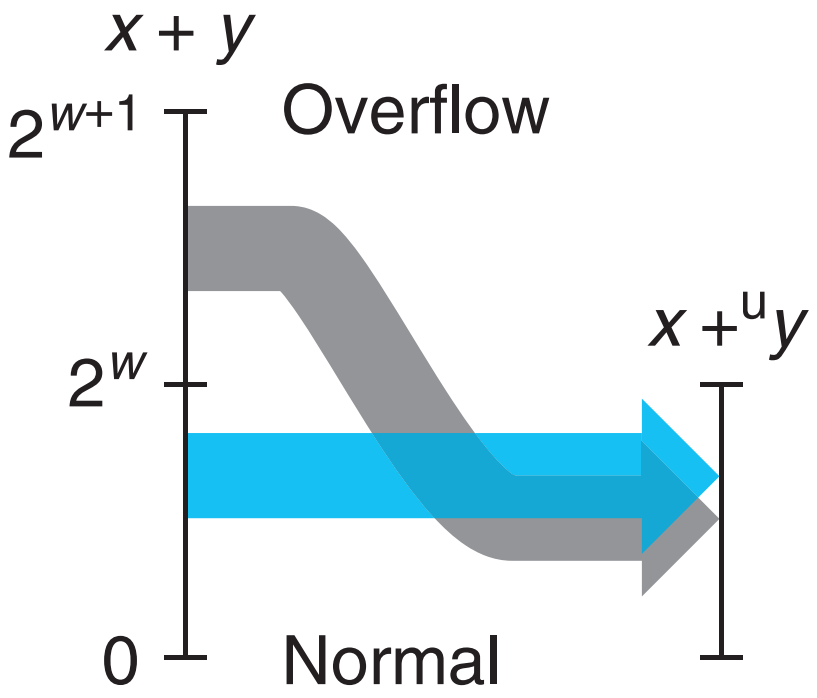


图 2.22 （示意图）

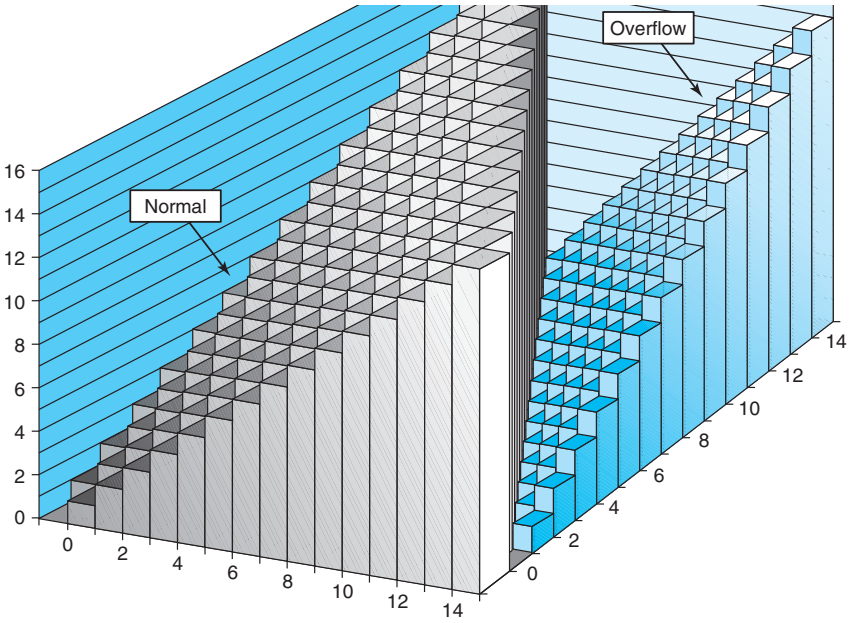


图 2.23 （示意图）

对处在 $0 \le x, y \le \text{UMax}_w$ 范围内的 x 和 y，令 $s \doteq x +_w^u y$。当且仅当 $s < x$（等价地，$s < y$）时，s 的计算发生了溢出。

作为一个示例，在前面的例子中我们算得 $9 +_4^u 12 = 5$。可以看出发生了溢出，因为 $5 < 9$。

推导：检测无符号加法的溢出

注意到 $x + y \ge x$，因此如果 s 没有溢出，那么肯定有 $s \ge x$。另一方面，如果 s 溢出了，就有 $s = x + y - 2^w$。由于 $y < 2^w$，即 $y - 2^w < 0$，所以 $s = x + (y - 2^w) < x$。

练习 2.27 写一个具有以下原型的函数：

```
/* 判断参数能否相加而不溢出 */
int uadd_ok(unsigned x, unsigned y);
```

如果参数 x 和 y 能够相加而不引起溢出，该函数就返回 1。

模加法构成一种叫作阿贝尔群（abelian group）的数学结构，得名于挪威数学家 Niels Henrik Abel（1802—1829）。也就是说，这种运算是可交换的（“阿贝尔”之名即由此而来）、可结合的；它有单位元 0，并且每个元素都有加法逆元。考虑全体 w 位无符号数的集合，运算为 $+_w^u$。对每个值 x，必定存在某个值 $-_w^u x$，使得 $-_w^u x +_w^u x = 0$。这个加法逆元运算可以刻画如下：

原理：无符号非

对任意满足 $0 \le x < 2^w$ 的数 x，其 w 位无符号非 $-_w^u x$ 由下式给出：

$$-_w^u x = \begin{cases} x, & x = 0 \\ 2^w - x, & x > 0 \end{cases} \tag{2.12}$$

这个结果可以用分情况分析较快地导出：

推导：无符号非

当 $x = 0$ 时，加法逆元显然是 0。对 $x > 0$，考虑值 $2^w - x$。注意到这个数在范围 $0 < 2^w - x < 2^w$ 之内。还可以看出 $(x + 2^w - x) \bmod 2^w = 2^w \bmod 2^w = 0$。因此它是 x 在 $+_w^u$ 下的逆元。

练习 2.28 长度为 $w = 4$ 的位模式可以用一个十六进制数字表示。对这些数字作无符号解释，请用公式 2.12 填写下表，给出所列各数字的无符号加法逆元的值和位表示（十六进制）：

2.3.2 补码加法

对于补码加法，当结果太大（正数）或太小（负数）而无法表示时，必须决定如何处理。设 x、y 是处于 $-2^{w-1} \le x, y \le 2^{w-1} - 1$ 范围内的整数值，则它们

x(十六进制)	x(十进制)	$-_4^u x$(十进制)	$-_4^u x$(十六进制)
1			
4			
7			
A			
E			

的和位于 $-2^w \le x+y \le 2^w-2$ 范围内,要精确表示可能需要 $w+1$ 个位。与前面一样,我们通过把表示截断到 w 位来避免数据规模的不断膨胀。不过,这样得到的结果在数学上不如模加法那样为人熟知。我们定义 $x +_w^t y$:把整数和 $x+y$ 截断为 w 位长,再把结果看成一个补码数,所得到的值就是 $x +_w^t y$。

原理:补码加法

对处于 $-2^{w-1} \le x, y \le 2^{w-1}-1$ 范围内的整数值 x 和 y:

$$x +_w^t y = \begin{cases} x+y-2^w, & 2^{w-1} \le x+y \quad \text{正溢出} \\ x+y, & -2^{w-1} \le x+y < 2^{w-1} \quad \text{正常} \\ x+y+2^w, & x+y < -2^{w-1} \quad \text{负溢出} \end{cases} \tag{2.13}$$

这条原理见图 2.24:左边的和 $x+y$ 取值在 $-2^w \le x+y \le 2^w-2$ 范围内,右边是该和截断为 w 位补码数以后的结果。(图中的标注“情形 1”到“情形 4”是为这条原理的形式化推导中的分情况分析而设的。)当和 $x+y$ 超过 TMax_w(情形 4)时,我们说发生了正溢出(positive overflow)。此时截断的效果是把和减去 2^w。当和 $x+y$ 小于 TMin_w(情形 1)时,我们说发生了负溢出(negative overflow)。此时截断的效果是给和加上 2^w。

两个数的 w 位补码和,与它们的无符号和具有完全相同的位级表示。实际上,大多数计算机都用同一条机器指令来执行无符号加法或有符号加法。

推导:补码加法

由于补码加法与无符号加法具有完全相同的位级表示,我们可以把运算 $+_w^t$ 刻画成:先把两个参数转换成无符号数,执行无符号加法,然后再转换回补码:

$$x +_w^t y = \text{U2T}_w(\text{T2U}_w(x) +_w^u \text{T2U}_w(y)) \tag{2.14}$$

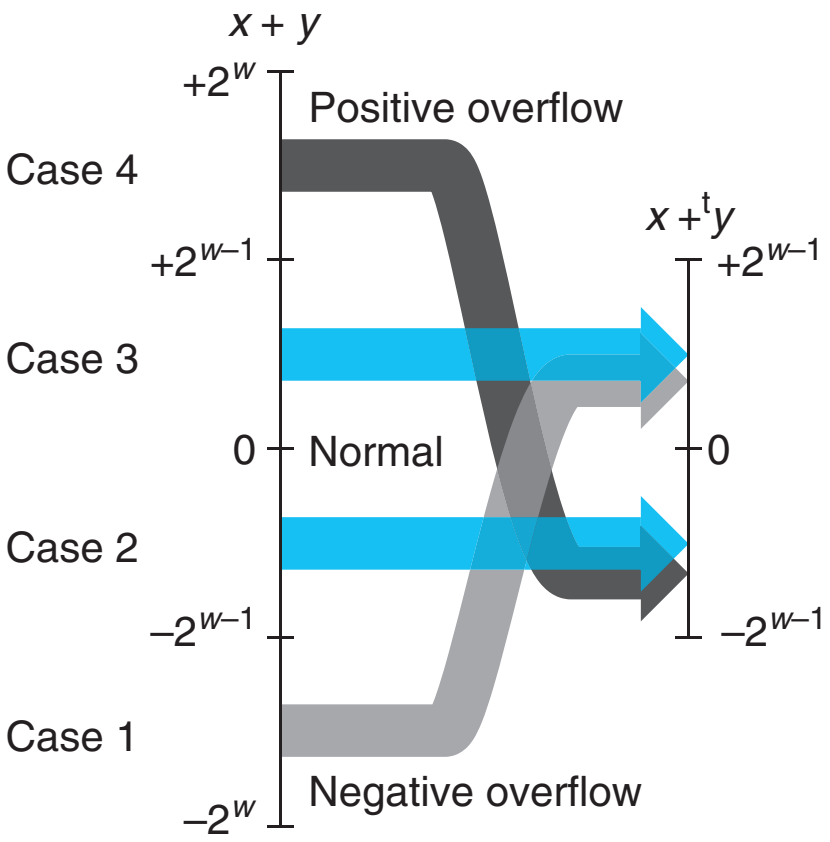


图 2.24 （示意图）

由公式 2.6 可以把 $\mathrm{T2U}_w(x)$ 写成 $x_{w-1}2^w + x$，把 $\mathrm{T2U}_w(y)$ 写成 $y_{w-1}2^w + y$。利用 $+_w^u$ 就是按模 2^w 的加法这一性质，再加上模加法的各条性质，我们便得到

$$\begin{aligned} x +_w^t y &= \mathrm{U2T}_w(\mathrm{T2U}_w(x) +_w^u \mathrm{T2U}_w(y)) \\ &= \mathrm{U2T}_w[(x_{w-1}2^w + x + y_{w-1}2^w + y) \bmod 2^w] \\ &= \mathrm{U2T}_w[(x + y) \bmod 2^w] \end{aligned}$$

项 $x_{w-1}2^w$ 和 $y_{w-1}2^w$ 被消掉了，因为它们按模 2^w 等于 0。

为了更好地理解这个量，我们定义 z 为整数和 $z \doteq x + y$，z' 为 $z' \doteq z \bmod 2^w$，z'' 为 $z'' \doteq \mathrm{U2T}_w(z')$。值 z'' 就等于 $x +_w^t y$。如图 2.24 所示，我们可以把分析分成四种情形：

1. $-2^w \le z < -2^{w-1}$。此时将有 $z' = z + 2^w$，给出 $0 \le z' < -2^{w-1} + 2^w = 2^{w-1}$。对照公式 2.7 可以看到，$z'$ 所处的范围满足 $z'' = z'$。这是负溢出

的情形。我们把两个负数 x 和 y 加在一起(只有这样才能使 $z < -2^{w-1}$),并得到一个非负的结果 $z'' = x + y + 2^w$。

2. $-2^{w-1} \le z < 0$。此时将再次有 $z' = z + 2^w$,给出 $-2^{w-1} + 2^w = 2^{w-1} \le z' < 2^w$。对照公式 2.7 可以看到,$z'$ 所处的范围满足 $z'' = z' - 2^w$,因此 $z'' = z' - 2^w = z + 2^w - 2^w = z$。也就是说,补码和 z'' 等于整数和 $x + y$。
3. $0 \le z < 2^{w-1}$。此时将有 $z' = z$,给出 $0 \le z' < 2^{w-1}$,因此 $z'' = z' = z$。同样,补码和 z'' 等于整数和 $x + y$。
4. $2^{w-1} \le z < 2^w$。此时将再次有 $z' = z$,给出 $2^{w-1} \le z' < 2^w$。在这一范围内有 $z'' = z' - 2^w$,给出 $z'' = x + y - 2^w$。这是正溢出的情形。我们把两个正数 x 和 y 加在一起(只有这样才能使 $z \ge 2^{w-1}$),并得到一个负的结果 $z'' = x + y - 2^w$。

图 2.25 补码加法示例。4 位补码和的位级表示,可以通过把两个操作数按位相加再把结果截断到 4 位来获得。

x	y	$x+y$	$x +_4^t y$	情形
−8	−5	−13	3	1
[1000]	[1011]	[10011]	[0011]	
−8	−8	−16	0	1
[1000]	[1000]	[10000]	[0000]	
−8	5	−3	−3	2
[1000]	[0101]	[11101]	[1101]	
2	5	7	7	3
[0010]	[0101]	[00111]	[0111]	
5	5	10	−6	4
[0101]	[0101]	[01010]	[1010]	

作为补码加法的示例,图 2.25 给出了 $w = 4$ 时的一些例子。每个例子都标注了它在公式 2.13 推导中对应的情形。注意到 $2^4 = 16$,因此负溢出的结果比整数和多 16,而正溢出的结果比整数和少 16。表中给出了操作数和结果的位级表示。可以看出,把两个操作数按位相加再把结果截断到 4 位,就能得到这个结果。

图 2.26 画出了字长 $w = 4$ 时补码加法的图像。操作数的取值范围为 −8 到 7。当 $x + y < -8$ 时,补码加法发生负溢出,使和增加 16。当 $-8 \le x + y < 8$ 时,加法结果就是 $x + y$。当 $x + y \ge 8$ 时,加法发生正溢出,使和减少 16。这三种取值范围在图中各自形成一块倾斜的平面。

公式 2.13 还让我们能够识别发生溢出的情形:

原理：检测补码加法的溢出

对处于 $\text{TMin}_w \le x, y \le \text{TMax}_w$ 范围内的 x 和 y，令 $s \doteq x +_w^t y$。当且仅当 $x > 0$、$y > 0$ 而 $s \le 0$ 时，s 的计算发生了正溢出。当且仅当 $x < 0$、$y < 0$ 而 $s \ge 0$ 时，s 的计算发生了负溢出。

图 2.25 展示了这条原理在 $w = 4$ 时的几个例子。第一行是负溢出的情形：两个负数相加得到一个正数。最后一行是正溢出的情形：两个正数相加得到一个负数。

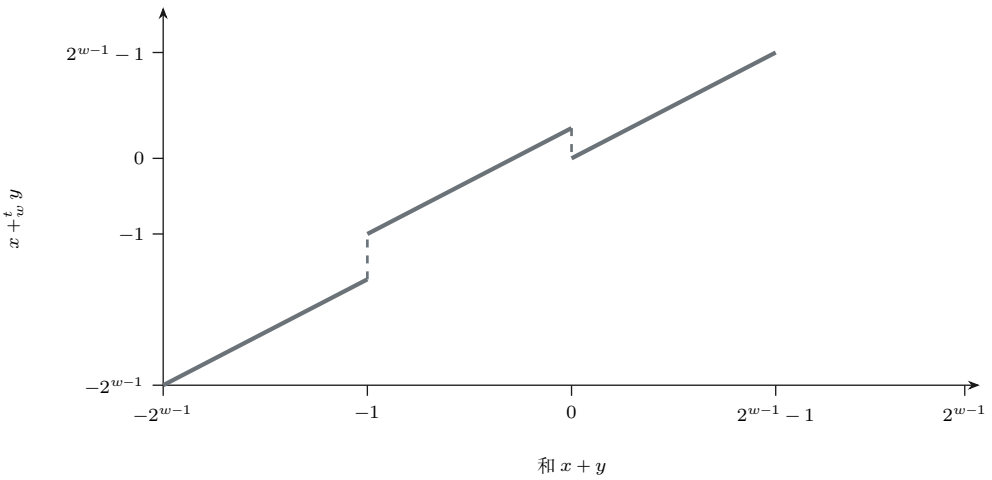


图 2.26　补码加法。位模式相同，但结果按补码解释，超出范围时正负回绕。

推导：检测补码加法的溢出

我们先分析正溢出。如果 $x > 0$、$y > 0$ 而 $s \le 0$，那么显然发生了正溢出。反过来，发生正溢出要求 (1) $x > 0$ 且 $y > 0$（否则 $x + y < \text{TMax}_w$），并且 (2) $s \le 0$（由公式 2.13）。对负溢出可以作一套类似的论证。

练习 2.29　按图 2.25 的样式填写下表。给出 5 位参数的整数值、它们的整数和与补码和的值、补码和的位级表示，以及公式 2.13 推导中的情形编号。

x	y	$x+y$	$x +_5^t y$	情形
[10100]	[10001]			
[11000]	[11000]			
[10111]	[01000]			
[00010]	[00101]			
[01100]	[00100]			

练习 2.30　写一个具有以下原型的函数：

```
/* 判断参数能否相加而不溢出 */
int tadd_ok(int x, int y);
```

如果参数 x 和 y 能够相加而不引起溢出,该函数就返回 1。

练习 2.31 你的一位同事对补码加法溢出条件的这样一番分析感到不耐烦了,交给你下面这个 `tadd_ok` 的实现:

```
/* 判断参数能否相加而不溢出 */
/* 警告:这段代码有缺陷。 */
int tadd_ok(int x, int y) {
    int sum = x+y;
    return (sum-x == y) && (sum-y == x);
}
```

你看了看这段代码,笑了起来。请解释为什么。

练习 2.32 给你的任务是为函数 `tsub_ok` 编写代码,参数为 x 和 y,当计算 $x-y$ 不引起溢出时返回 1。你刚写完练习 2.30 的代码,于是写下如下代码:

```
/* 判断参数能否相减而不溢出 */
/* 警告:这段代码有缺陷。 */
int tsub_ok(int x, int y) {
    return tadd_ok(x, -y);
}
```

对什么样的 x 和 y 值,这个函数会给出不正确的结果?把这个函数的正确版本留作练习(家庭作业 2.74)。

2.3.3 补码的非

我们看到,$\text{TMin}_w \le x \le \text{TMax}_w$ 范围内的每个数 x 在 $+_w^t$ 下都有加法逆元,我们把它记作 $-_w^t x$,如下:

原理:补码的非

对处于 $\text{TMin}_w \le x \le \text{TMax}_w$ 范围内的 x,其补码的非 $-_w^t x$ 由下式给出:

$$-_w^t x = \begin{cases} \text{TMin}_w, & x = \text{TMin}_w \\ -x, & x > \text{TMin}_w \end{cases} \tag{2.15}$$

也就是说,对 w 位补码加法而言,TMin_w 的加法逆元就是它自己,而任何其他值 x 的加法逆元都是 $-x$。

推导:补码的非

注意到 $\text{TMin}_w + \text{TMin}_w = -2^{w-1} + -2^{w-1} = -2^w$。这会引起负溢出,因此 $\text{TMin}_w +_w^t \text{TMin}_w = -2^w + 2^w = 0$。对满足 $x > \text{TMin}_w$ 的那些 x,值 $-x$ 也可以表示成一个 w 位补码数,而它们的和将是 $-x + x = 0$。

练习 2.33 长度为 $w = 4$ 的位模式可以用一个十六进制数字表示。对这些数字作补码解释,请填写下表求出所列各数字的加法逆元:

x(十六进制)	x(十进制)	$-_4^t x$(十进制)	$-_4^t x$(十六进制)
2			
3			
9			
B			
C			

关于补码求非(与练习 2.28 的无符号求非相比)所产生的位模式,你观察到什么?

网络边栏 DATA:TNEG 补码非运算的位级表示

在位级上求一个值的补码的非,几种巧妙的方法都行得通。下面两种技术都很有用(例如调试程序时遇到值 0xfffffffa 的时候),而且能让我们更深入地理解补码表示的本质。

在位级上执行补码的非的一种技术是:先对各位求反(取补),再把结果加 1。在 C 中可以对任意整数值 x 说:表达式 `-x` 与 `~x + 1` 会给出相同的结果。以下是字长为 4 位时的一些例子:

$\vec{x}$		$\sim \vec{x}$		incr($\sim \vec{x}$)	
[0101]	5	[1010]	-6	[1011]	-5
[0111]	7	[1000]	-8	[1001]	-7
[1100]	-4	[0011]	3	[0100]	4
[0000]	0	[1111]	-1	[0000]	0
[1000]	-8	[0111]	7	[1000]	-8

回到我们之前的例子:0xf 的补是 0x0,0xa 的补是 0x5,因此 0xfffffffa 是 -6 的补码表示。

对一个数 x 执行补码的非的第二种方法,是把位向量拆成两个部分。设 k 是最右边那个 1 的位置,于是 x 的位级表示形如 $[x_{w-1}, x_{w-2}, \ldots, x_{k+1}, 1, 0, \ldots, 0]$(只要 $x \neq 0$,这就总是可行的)。这个非就可以写成二进制形式 $[\sim x_{w-1}, \sim x_{w-2}, \ldots, \sim x_{k+1}, 1, 0, \ldots, 0]$。也就是说,我们把位 k 左边的每一位都取反。

我们用几个 4 位数来说明这个想法,其中最右边那个 $1, 0, \ldots, 0$ 模式用斜体标出:

$\vec{x}$		$-\vec{x}$	
[1*100*]	-4	[0*100*]	4
[*1000*]	-8	[*1000*]	-8
[0101]	5	[1011]	-5
[0111]	7	[1001]	-7

2.3.4 无符号乘法

设 x、y 是 $0 \le x, y \le 2^w - 1$ 范围内的整数，它们都能表示成 w 位无符号数，但乘积 $x \cdot y$ 的取值范围却是 0 到 $(2^w - 1)^2 = 2^{2w} - 2^{w+1} + 1$，表示起来可能需要多达 $2w$ 个位。于是，C 中的无符号乘法定义为产生 $2w$ 位整数乘积的低 w 位所构成的 w 位值。我们把这个值记作 $x *_w^u y$。把一个无符号数截断到 w 位，等价于计算它按模 2^w 的值，由此得到：

原理：无符号乘法

对满足 $0 \le x, y \le \text{UMax}_w$ 的 x 和 y：

$$x *_w^u y = (x \cdot y) \bmod 2^w \tag{2.16}$$

2.3.5 补码乘法

设 x、y 是 $-2^{w-1} \le x, y \le 2^{w-1} - 1$ 范围内的整数，它们都能表示成 w 位补码数，但乘积 $x \cdot y$ 的取值范围介于 $-2^{w-1} \cdot (2^{w-1} - 1) = -2^{2w-2} + 2^{w-1}$ 与 $-2^{w-1} \cdot -2^{w-1} = 2^{2w-2}$ 之间。要用补码形式表示它，可能需要多达 $2w$ 个位。于是，C 中的有符号乘法一般是通过把 $2w$ 位乘积截断到 w 位来执行的。我们把这个值记作 $x *_w^t y$。把一个补码数截断到 w 位，等价于先计算它按模 2^w 的值，再把结果从无符号数转换成补码数，由此得到：

原理：补码乘法

对满足 $\text{TMin}_w \le x, y \le \text{TMax}_w$ 的 x 和 y：

$$x *_w^t y = \text{U2T}_w((x \cdot y) \bmod 2^w) \tag{2.17}$$

我们断言，乘积运算的位级表示对无符号乘法和补码乘法是完全相同的，如下面的原理所述：**原理：无符号乘法与补码乘法的位级等价**

设 $\vec{x}$ 和 $\vec{y}$ 是长度为 w 的位向量。定义整数 x 和 y 为这些位按补码形式表示的值：$x = \text{B2T}_w(\vec{x})$，$y = \text{B2T}_w(\vec{y})$。定义非负整数 x' 和 y' 为这些位按无符号形式表示的值：$x' = \text{B2U}_w(\vec{x})$，$y' = \text{B2U}_w(\vec{y})$。那么

$$\text{T2B}_w(x *_w^t y) = \text{U2B}_w(x' *_w^u y')$$

作为说明，图 2.27 给出了把不同的 3 位数相乘的结果。对每一对位级操作数，我们同时执行无符号乘法和补码乘法，得到 6 位的乘积，然后把这些乘积截断到 3 位。截断后的无符号乘积总是等于 $x \cdot y \bmod 8$。两个截断乘积的位级表示，无论对无符号乘法还是对补码乘法都是相同的，尽管完整的 6 位表示各不相同。

图 2.27 3 位无符号数与补码乘法的例子。完整乘积的位级表示可能不同，截断乘积的位级表示却是相同的。

方式	x	y	$x \cdot y$	截断的 $x \cdot y$
无符号	5 [101]	3 [011]	15 [001111]	7 [111]
补码	−3 [101]	3 [011]	−9 [110111]	−1 [111]
无符号	4 [100]	7 [111]	28 [011100]	4 [100]
补码	−4 [100]	−1 [111]	4 [000100]	−4 [100]
无符号	3 [011]	3 [011]	9 [001001]	1 [001]
补码	3 [011]	3 [011]	9 [001001]	1 [001]

推导：无符号乘法与补码乘法的位级等价性

由公式 2.6 可得 $x' = x + x_{w-1}2^w$ 和 $y' = y + y_{w-1}2^w$。对这两个值的乘积按 2^w 取模，得到：

$$\begin{aligned}(x' \cdot y') \bmod 2^w &= [(x + x_{w-1}2^w) \cdot (y + y_{w-1}2^w)] \bmod 2^w \\ &= [x \cdot y + (x_{w-1}y + y_{w-1}x)2^w + x_{w-1}y_{w-1}2^{2w}] \bmod 2^w \\ &= (x \cdot y) \bmod 2^w\end{aligned} \tag{2.18}$$

权为 2^w 和 2^{2w} 的各项会因取模运算而消去。由公式 2.17 可得 $x *_w^{\mathrm{t}} y = \mathrm{U2T}_w((x \cdot y) \bmod 2^w)$。对等式两边应用 $\mathrm{T2U}_w$，得到

$$\mathrm{T2U}_w(x *_w^{\mathrm{t}} y) = \mathrm{T2U}_w(\mathrm{U2T}_w((x \cdot y) \bmod 2^w)) = (x \cdot y) \bmod 2^w$$

把这个结果与公式 2.16 和公式 2.18 合并起来，可得 $\mathrm{T2U}_w(x *_w^{\mathrm{t}} y) = (x' \cdot y') \bmod 2^w = x' *_w^{\mathrm{u}} y'$。再对两边应用 $\mathrm{U2B}_w$：

$$\mathrm{U2B}_w(\mathrm{T2U}_w(x *_w^{\mathrm{t}} y)) = \mathrm{T2B}_w(x *_w^{\mathrm{t}} y) = \mathrm{U2B}_w(x' *_w^{\mathrm{u}} y')$$

练习 2.34 请按照图 2.27 的样式，在下表中填入不同的 3 位数相乘的结果：

方式	x	y	$x \cdot y$	截断的 $x \cdot y$
无符号	[100]	[101]		
补码	[100]	[101]		
无符号	[010]	[111]		
补码	[010]	[111]		
无符号	[110]	[110]		
补码	[110]	[110]		

练习 2.35 你接到一项任务：为函数 `tmult_ok` 编写代码，判定两个参数相乘会不会导致溢出。下面是你的方案：

```
/* 判断两个参数能否相乘而不溢出 */
int tmult_ok(int x, int y) {
    int p = x*y;
    /* 要么 x 为零,要么用 x 除 p 的商等于 y */
    return !x || p/x == y;
}
```

你用多组 x 和 y 的值对这段代码做了测试，它看起来工作正常。同事却向你发难道：“既然不能（见练习 2.31）用减法来检查加法是否溢出，那你怎么能用除法来检查乘法是否溢出呢？”

请为你的做法构造一个数学论证，思路如下。首先，论证 $x = 0$ 的情形得到了正确的处理。否则，考虑 w 位数 $x(x \neq 0)$、y、p 和 q，其中 p 是对 x 和 y 执行补码乘法的结果，q 是 p 除以 x 的结果。

1. 证明 x 和 y 的整数乘积 $x \cdot y$ 可以写成 $x \cdot y = p + t2^w$ 的形式，这里 $t \neq 0$ 当且仅当 p 的计算溢出。
2. 证明 p 可以写成 $p = x \cdot q + r$ 的形式，其中 $|r| < |x|$。
3. 证明 $q = y$ 当且仅当 $r = t = 0$。

练习 2.36 就数据类型 `int` 为 32 位的情形，设计一个 `tmult_ok`（练习 2.35）的新版本，利用数据类型 `int64_t` 的 64 位精度，并且不使用除法。

练习 2.37 你的任务是修补 XDR 代码（见 2.3.5 节边栏）中的安全漏洞，条件是数据类型 `int` 和 `size_t` 都为 32 位。你决定改用数据类型 `uint64_t` 来计算要分配的字节数，从而消除乘法溢出的可能性。你把原来对 `malloc` 的调用（第 9 行）替换为如下代码：

边栏 XDR 库中的安全漏洞

2002 年，人们发现 Sun Microsystems 提供的、用于实现 XDR 库（在程序之间共享数据结构的一种广泛使用的设施）的代码存在一个安全漏洞，其根源在于：乘法可能溢出，而程序得不到任何通知。
下面这段代码与含有该漏洞的代码类似：

```
1  /* Illustration of code vulnerability similar to that found in
2    * Sun's XDR library.
3    */
4  void* copy_elements(void *ele_src[], int ele_cnt,
                       size_t ele_size) {
5      /*
6       * Allocate buffer for ele_cnt objects, each of ele_size bytes
7       * and copy from locations designated by ele_src
8       */
9      void *result = malloc(ele_cnt * ele_size);
10     if (result == NULL)
11         /* malloc failed */
```

```
12          return NULL;
13      void *next = result;
14      int i;
15      for (i = 0; i < ele_cnt; i++) {
16          /* Copy object i to destination */
17          memcpy(next, ele_src[i], ele_size);
18          /* Move pointer to next memory region */
19          next += ele_size;
20      }
21      return result;
22  }
```

函数 `copy_elements` 的设计用途，是把 `ele_cnt` 个数据结构复制到由第 9 行调用分配的缓冲区中，每个数据结构由 `ele_size` 个字节组成。需要的字节数按 `ele_cnt * ele_size` 计算。

然而可以设想，恶意程序员在程序按 32 位编译的情况下，用 `ele_cnt` 为 1,048,577 $(2^{20}+1)$、`ele_size` 为 4,096(2^{12})来调用这个函数。这时第 9 行的乘法会溢出，实际只分配了 4,096 字节，而不是容纳这些数据所需的 4,294,971,392 字节。从第 15 行开始的循环会试图把这些字节全部复制过去，越出已分配缓冲区的末端，从而破坏其他数据结构。这可能导致程序崩溃，或产生其他不当行为。

Sun 的代码几乎被每个操作系统使用，Internet Explorer 和 Kerberos 认证系统这类广泛使用的程序也不例外。计算机应急响应组（Computer Emergency Response Team，CERT）——由卡内基梅隆大学软件工程研究所运作、负责跟踪安全漏洞和安全事件的组织——发布了公告“CA-2002-25”，许多公司急忙修补了自己的代码。所幸没有报道过由这个漏洞造成的安全事件。

类似的漏洞也存在于 `calloc` 库函数的许多实现中，后来都修补好了。遗憾的是，许多程序员用算术表达式作实参来调用 `malloc` 之类的分配函数，却不检查这些表达式是否溢出。编写一个可靠的 `calloc` 版本留作练习（练习 2.76）。

```
uint64_t asize =
    ele_cnt * (uint64_t) ele_size;
void *result = malloc(asize);
```

回忆一下，`malloc` 的参数类型是 `size_t`。

A. 相比原来的调用，你的代码有什么改进吗？

B. 你会怎样修改代码来消除这个漏洞？

2.3.6 乘以常数

早些年，许多机器上的整数乘法指令相当慢，需要 10 个甚至更多时钟周期，而加法、减法、位级运算和移位等其他整数运算只需 1 个时钟周期。就连在我们选作参考机器的 Intel Core i7 Haswell 上，整数乘法也需要 3 个时钟周期。因此，编译器的一项重要优化是尝试用移位和加法运算的组合来替代乘以常数因子的乘法。我们先考虑乘以 2 的幂的情形，再把它推广到任意常数。

原理：乘以 2 的幂

设位模式 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 表示无符号整数 x。那么对任意 $k \geq 0$，$x2^k$ 的 $w+k$ 位无符号表示为 $[x_{w-1}, x_{w-2}, \ldots, x_0, 0, \ldots, 0]$，也就是在右边添上 k 个 0。

例如，$w = 4$ 时，11 可以表示为 `[1011]`。把它左移 $k = 2$ 位得到 6 位向量 `[101100]`，编码的无符号数是 $11 \cdot 4 = 44$。

推导：乘以 2 的幂

利用公式 2.1 可以推导出这一性质：

$$\begin{aligned} \mathrm{B2U}_{w+k}([x_{w-1}, x_{w-2}, \ldots, x_0, 0, \ldots, 0]) &= \sum_{i=0}^{w-1} x_i \, 2^{i+k} \\ &= \sum_{i=0}^{w-1} x_i \, 2^i \cdot 2^k \\ &= x2^k \end{aligned}$$

对固定字长左移 k 位时，最高 k 位会被丢弃，得到

$$[x_{w-k-1}, x_{w-k-2}, \ldots, x_0, 0, \ldots, 0]$$

不过对固定字长的字执行乘法时同样会出现这种情况。因此可以看出，把一个值左移等价于执行乘以 2 的幂的无符号乘法：

原理：乘以 2 的幂的无符号乘法

设 C 变量 `x` 和 `k` 具有无符号值 x 和 k，且 $0 \leq k < w$，则 C 表达式 `x << k` 产生值 $x *_w^{\mathrm{u}} 2^k$。

由于固定字长的补码算术在位级运算上与无符号算术等价，对左移与乘以 2 的幂之间的关系，同样可以就补码算术做出类似的陈述：

原理：乘以 2 的幂的补码乘法

设 C 变量 `x` 的补码值为 x、`k` 的无符号值为 k，且 $0 \leq k < w$，则 C 表达式 `x << k` 产生值 $x *_w^{\mathrm{t}} 2^k$。

注意，无论无符号算术还是补码算术，乘以 2 的幂都可能引起溢出。而我们的结论表明，即便发生溢出，用移位也会得到同样的效果。回到前面的例子：把 4 位模式 `[1011]`（数值 11）左移两位得到 `[101100]`（数值 44），把它截断为 4 位得 `[1100]`（数值 $12 = 44 \bmod 16$）。

既然整数乘法比移位和加法的开销大，许多 C 编译器就设法用移位、加法和减法的组合来消除整数乘以常数的许多情形。例如，假设程序中含有表达式 `x*14`。识别出 $14 = 2^3 + 2^2 + 2^1$ 后，编译器可以把这个乘法改写为 `(x<<3) +`

`(x<<2) + (x<<1)`，用三次移位和两次加法替代一次乘法。无论 `x` 是无符号数还是补码数，即便乘法会导致溢出，两种计算都会给出相同的结果。更进一步，编译器还可以利用性质 $14 = 2^4 - 2^1$，把乘法改写为 `(x<<4) - (x<<1)`，只需要两次移位和一次减法。

练习 2.38 3 章会看到，`lea` 指令可以完成形如 `(a<<k) + b` 的计算，其中 k 取 0、1、2 或 3，b 为 0 或某个程序值。编译器经常用这条指令来完成乘以常数因子的乘法。例如，$3 \cdot a$ 可以按 `(a<<1) + a` 来计算。

考虑 b 为 0 或等于 a 的各种情形，以及 k 的所有可能取值，一条 `lea` 指令能算出 a 的哪些倍数？

从上面的例子推广开，考虑为表达式 $x \cdot K$ 生成代码的任务，其中 K 是某个常数。编译器可以把 K 的二进制表示写成由 0 和 1 交替组成的序列：

$$[(0 \ldots 0)\,(1 \ldots 1)\,(0 \ldots 0)\ \ldots\ (1 \ldots 1)]$$

例如，14 可以写成 $[(0 \ldots 0)(111)(0)]$。考虑从位 n 下延到位 $m (n \geq m)$ 的一段连续的 1（对 14 而言，$n = 3, m = 1$）。这些位对乘积的贡献可以按以下两种形式中的任何一种来计算：

形式 A： `(x<<n) + (x<<(n-1)) + …+ (x<<m)`

形式 B： `(x<<(n+1)) - (x<<m)`

把每段连续 1 的结果加在一起，就能不经任何乘法算出 $x \cdot K$。当然，是用移位、加法、减法的组合还是用一条乘法指令，这种权衡取决于这些指令的相对速度，而相对速度可能高度依赖机器。大多数编译器只在所需的移位、加法和减法次数很少时才做这种优化。

练习 2.39 当位 n 是最高有效位时，应当如何修改形式 B 的表达式？

练习 2.40 对下面给出的每个 K 值，设法用指明的操作次数把 $x \cdot K$ 表示出来，这里认为加法和减法的开销相当。你可能需要用到简单的形式 A 和形式 B 规则之外的一些技巧。

K	移位	加/减	表达式
7	1	1	
30	4	3	
28	2	1	
55	2	2	

练习 2.41 对从位 n 下延到位 $m(n \geq m)$的一段连续的 1,我们已经看到可以生成 A 和 B 两种形式的代码。编译器应当如何决定使用哪种形式?

2.3.7 除以 2 的幂

在大多数机器上,整数除法甚至比整数乘法还要慢——需要 30 个甚至更多时钟周期。除以 2 的幂同样可以只用移位运算来实现,但这次用的是右移而不是左移。两种不同的右移——逻辑右移和算术右移——分别服务于无符号数和补码数。

图 2.28 无符号数除以 2 的幂。这些例子说明,执行逻辑右移 k 位与先除以 2^k、再向零舍入具有相同的效果。

k	x >> k(二进制)	十进制	$12{,}340/2^k$
0	0011000000110100	12,340	12,340.0
1	0001100000011010	6,170	6,170.0
4	000011000000011	771	771.25
8	000000000110000	48	48.203125

整数除法总是向零舍入(round toward zero)。为了准确定义,我们引入一些记号。对任意实数 a,定义 $\lfloor a \rfloor$ 为满足 $a' \leq a < a'+1$ 的唯一整数 a'。例如,$\lfloor 3.14 \rfloor = 3, \lfloor -3.14 \rfloor = -4, \lfloor 3 \rfloor = 3$。类似地,定义 $\lceil a \rceil$ 为满足 $a'-1 < a \leq a'$ 的唯一整数 a'。例如,$\lceil 3.14 \rceil = 4, \lceil -3.14 \rceil = -3, \lceil 3 \rceil = 3$。当 $x \geq 0$ 且 $y > 0$ 时,整数除法应给出 $\lfloor x/y \rfloor$;当 $x < 0$ 且 $y > 0$ 时,应给出 $\lceil x/y \rceil$。也就是说,正的结果向下舍入,负的结果向上舍入。

把移位用于无符号算术的情形很直接,部分原因在于对无符号值而言,右移保证按逻辑移位执行。

原理:除以 2 的幂的无符号除法

设 C 变量 x 和 k 具有无符号值 x 和 k,且 $0 \leq k < w$,则 C 表达式 x >> k 产生值 $\lfloor x/2^k \rfloor$。

作为例子,图 2.28 展示了对 16 位表示的 12,340 执行逻辑右移、实现除以 1、2、16 和 256 的效果。从左边移入的零用斜体表示,我们还列出了用实数算术做这些除法会得到的结果。这些例子表明,移位的结果一致地向零舍入,这正是整数除法的约定。

推导:除以 2 的幂的无符号除法

设位模式 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 表示无符号整数 x,k 取值在 $0 \leq k < w$ 范围内。设 x' 是 $w-k$ 位表示 $[x_{w-1}, x_{w-2}, \ldots, x_k]$ 的无符号数,x'' 是 k 位表示 $[x_{k-1}, \ldots, x_0]$ 的无符号数。可以看出 $x = 2^k x' + x''$,且 $0 \leq x'' < 2^k$,于是 $\lfloor x/2^k \rfloor = x'$。

对位向量 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 执行 k 位逻辑右移，得到位向量

$$[0, \ldots, 0, x_{w-1}, x_{w-2}, \ldots, x_k]$$

这个位向量的数值就是 x'。我们已经看到，这正是计算表达式 x >> k 得到的值。

就补码算术而言，除以 2 的幂的情形要稍微复杂一些。首先，移位应当用算术右移来完成，以保证负值仍然是负数。我们来考察这样的右移会产生什么值。

原理：除以 2 的幂的补码除法，向下舍入

设 C 变量 x 和 k 分别具有补码值 x 和无符号值 k，且 $0 \leq k < w$。当移位按算术方式执行时，C 表达式 x >> k 产生值 $\lfloor x/2^k \rfloor$。

当 $x \geq 0$ 时，变量 x 的最高有效位是 0，算术移位的效果与逻辑右移相同。于是，对非负数而言，算术右移 k 位等同于除以 2^k。举一个负数的例子：图 2.29 展示了对 16 位表示的 −12,340 施加不同移位量的算术右移。在不需要舍入的情形 $(k = 1)$，结果就是 $x/2^k$；在需要舍入的情形，移位会使结果向下舍入。例如，右移四位的效果是把 −771.25 向下舍入为 −772。要处理 x 为负值的除法，就必须调整策略。

图 2.29 应用算术右移。这些例子说明，算术右移类似于除以 2 的幂，只是它向下舍入，而不是向零舍入。

k	x >> k（二进制）	十进制	$-12{,}340/2^k$
0	1100111111001100	−12,340	−12,340.0
1	1110011111100110	−6,170	−6,170.0
4	1111110011111100	−772	−771.25
8	1111111111001111	−49	−48.203125

推导：除以 2 的幂的补码除法，向下舍入

设位模式 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 表示补码整数 x，k 取值在 $0 \leq k < w$ 范围内。设 x' 是 $w-k$ 位 $[x_{w-1}, x_{w-2}, \ldots, x_k]$ 表示的补码数，x'' 是低 k 位 $[x_{k-1}, \ldots, x_0]$ 表示的无符号数。通过与无符号情形类似的分析，可得 $x = 2^k x' + x''$ 且 $0 \leq x'' < 2^k$，于是 $x' = \lfloor x/2^k \rfloor$。再注意，把位向量 $[x_{w-1}, x_{w-2}, \ldots, x_0]$ 算术右移 k 位，得到位向量

$$[x_{w-1}, \ldots, x_{w-1}, x_{w-1}, x_{w-2}, \ldots, x_k]$$

它是 $[x_{w-1}, x_{w-2}, \ldots, x_k]$ 从 $w-k$ 位到 w 位的符号扩展。因此，这个移位后的位向量就是 $\lfloor x/2^k \rfloor$ 的补码表示。

负数右移时出现的不恰当舍入，可以在移位之前先对值“加偏置”(bias)来修正。

原理：除以 2 的幂的补码除法，向上舍入

设 C 变量 x 和 k 分别具有补码值 x 和无符号值 k，且 $0 \le k < w$。当移位按算术方式执行时，C 表达式 `(x + (1 << k) - 1) >> k` 产生值 $\lceil x/2^k \rceil$。

图 2.30 演示了在执行算术右移之前加上适当的偏置如何使结果得到正确的舍入。第三列给出把偏置值加到 −12,340 上的结果，其中低 k 位(右移时将被移出的那几位)用斜体表示。可以看到，这些位左边的各位可能加 1，也可能不加。在不需要舍入的情形($k = 1$)，加偏置只影响将被移出的位。在需要舍入的情形，加偏置会使高位加 1，从而使结果向零舍入。

图 2.30 补码数除以 2 的幂。右移之前加上偏置，结果便向零舍入。

k	偏置	−12,340 加偏置(二进制)	x >> k(二进制)	十进制	$-12{,}340/2^k$
0	0	1100111111001100	1100111111001100	−12,340	−12,340.0
1	1	110011111100110*1*	1110011111100110	−6,170	−6,170.0
4	15	11001111110*11011*	1111110011111101	−771	−771.25
8	255	11010000*11001011*	1111111111010000	−48	−48.203125

加偏置技术利用了如下性质：对整数 x 和满足 $y > 0$ 的 y，有 $\lceil x/y \rceil = \lfloor (x + y - 1)/y \rfloor$。举例来说，当 $x = -30$、$y = 4$ 时，$x + y - 1 = -27$，且 $\lceil -30/4 \rceil = -7 = \lfloor -27/4 \rfloor$；当 $x = -32$、$y = 4$ 时，$x + y - 1 = -29$，且 $\lceil -32/4 \rceil = -8 = \lfloor -29/4 \rfloor$。

推导：除以 2 的幂的补码除法，向上舍入

为看出 $\lceil x/y \rceil = \lfloor (x + y - 1)/y \rfloor$，设 $x = qy + r$，其中 $0 \le r < y$，于是 $(x + y - 1)/y = q + (r + y - 1)/y$，从而 $\lfloor (x + y - 1)/y \rfloor = q + \lfloor (r + y - 1)/y \rfloor$。当 $r = 0$ 时，后一项等于 0；当 $r > 0$ 时，它等于 1。也就是说，先给 x 加上偏置 $y - 1$，再把除法向下舍入，则 y 整除 x 时得到 q，否则得到 $q + 1$。

回到 $y = 2^k$ 的情形：C 表达式 `x + (1 << k) - 1` 产生值 $x + 2^k - 1$，把它算术右移 k 位，于是得到 $\lceil x/2^k \rceil$。

这些分析表明，在使用算术右移的补码机器上，C 表达式

```
(x<0 ? x+(1<<k)-1 : x) >> k
```

将计算出 $x/2^k$ 的值。

练习 2.42 为整数参数 x 编写函数 `div16`，返回 $x/16$ 的值。函数不得使用除法、模运算、乘法、任何条件语句(`if` 或`?:`)、任何比较运算符(如 `<`、`>` 或

==),也不得使用任何循环。可以假设数据类型 int 为 32 位长、采用补码表示,并且右移按算术移位执行。

现在我们看到,除以 2 的幂既可以用逻辑右移实现,也可以用算术右移实现。这正是大多数机器提供这两种右移的原因。遗憾的是,这种做法不能推广到除以任意常数。与乘法不同,除以任意常数 K 无法用除以 2 的幂来表达。

练习 2.43 在下面的代码中,我们略去了常数 M 和 N 的定义:

```
#define M      /* 神秘数 1 */
#define N      /* 神秘数 2 */
int arith(int x, int y) {
    int result = 0;
    result = x*M + y/N; /* M 和 N 是神秘数。 */
    return result;
}
```

我们用特定的 M 和 N 值编译了这段代码。编译器用我们讨论过的方法优化了其中的乘法和除法。以下是把生成的机器代码反译回 C 的结果:

```
/* arith 汇编代码的对照翻译 */
int optarith(int x, int y) {
    int t = x;
    x <<= 5;
    x -= t;
    if (y < 0) y += 7;
    y >>= 3; /* 算术移位 */
    return x+y;
}
```

M 和 N 的值各是什么?

2.3.8 关于整数运算的最后思考

正如我们所见,计算机执行的“整数”运算实际上是一种模运算。用来表示数的有限字长限制了可能的取值范围,因而相应的运算可能溢出。我们还看到,补码表示提供了一种巧妙的方法,能够用与无符号算术相同的位级实现来同时表示正值和负值——无论操作数是无符号形式还是补码形式,加法、减法、乘法乃至除法的位级行为要么完全相同,要么非常相似。

我们还看到,C 语言的一些约定会产生令人吃惊的结果,它们可能是难以识别、难以理解的错误之源。尤其是无符号数据类型,概念上虽然直观直接,却可能导致连经验丰富的程序员都预料不到的行为。我们还看到,这种数据类型会以意想不到的方式出现——例如在书写整数常量时,以及在调用库例程时。

练习 2.44 假设数据类型 int 为 32 位长,对有符号值采用补码表示。有符号值的右移按算术移位执行,无符号值的右移按逻辑移位执行。变量按如下方式声明并初始化:

```
int x = foo();      /* 任意值 */
int y = bar();      /* 任意值 */

unsigned ux = x;
unsigned uy = y;
```

对下面的每个 C 表达式，或者(1)论证它对 x 和 y 的所有取值都为真(求值为 1)，或者(2)给出使它为假(求值为 0)的 x 和 y 的取值：

A. `(x > 0) || (x-1 < 0)`

B. `(x & 7) != 7 || (x<<29 < 0)`

C. `(x * x) >= 0`

D. `x < 0 || -x <= 0`

E. `x > 0 || -x >= 0`

F. `x+y == uy+ux`

G. `x*~y + uy*ux == -x`

2.4 浮点数

浮点表示(floating-point representation)对形如 $V = x \times 2^y$ 的有理数进行编码。它对执行涉及非常大的数($|V| \gg 0$)和非常接近 0 的数($|V| \ll 1$)的计算很有用，更一般地说，也可作为实数运算的一种近似。

边栏 IEEE

电气与电子工程师学会(Institute of Electrical and Electronics Engineers，IEEE——读作“eye-triple-ee”)是一个专业学会，涵盖整个电子与计算机技术领域。它出版期刊、主办会议，并设立委员会来制定标准，主题从输电技术到软件工程不等。IEEE 标准的另一个例子是无线网络的 802.11 标准。

20 世纪 80 年代以前，每家计算机制造商都各自设计了约定，用来规定浮点数如何表示、对其执行的运算细节如何。此外，厂商往往不太在意运算的精度，认为运算速度和实现的容易程度比数值精度更为要紧。

1985 年前后，这一切随着 IEEE 标准 754 的出现而改变。这个标准经过精心设计，用于表示浮点数以及对它们执行的运算。这项工作始于 1976 年，在 Intel 的赞助下展开，来自为 8086 处理器提供浮点支持的 8087 芯片的设计。Intel 聘请加州大学伯克利分校(University of California, Berkeley)教授 William Kahan 担任顾问，帮助为其未来的处理器设计一个浮点标准。Intel 允许 Kahan 加入一个制定全行业标准的委员会，该委员会由电气与电子工程师学会(IEEE)主持。委员会最终采纳的标准与 Kahan 为 Intel 设计的标准非常接近。如今，几乎所有的计算机都支持人们熟知的 IEEE 浮点。这大大提高了科学应用程序跨不同机器的可移植性。

本节将看到数在 IEEE 浮点格式下是如何表示的。我们还将考察舍入(rounding)的问题——当一个数无法用这种格式精确表示时，必须向上或向下调整。随后我们将探讨加法、乘法和比较运算符的数学性质。许多程序员认

为浮点数顶多乏味无趣,往坏了说则玄奥难懂。但我们将看到,由于 IEEE 格式建立在一小组一致的原则之上,它其实相当优雅,也完全可以理解。

2.4.1 二进制小数

理解浮点数的第一步,是考虑取小数值的二进制数。先来看更熟悉的十进制记法。十进制记法采用的表示形式为

$$d_m\, d_{m-1} \ldots d_1\, d_0 \,.\, d_{-1}\, d_{-2} \ldots d_{-n}$$

其中每个十进制数字 d_i 的取值范围是 0 到 9。这种记法表示一个数值 d,定义为

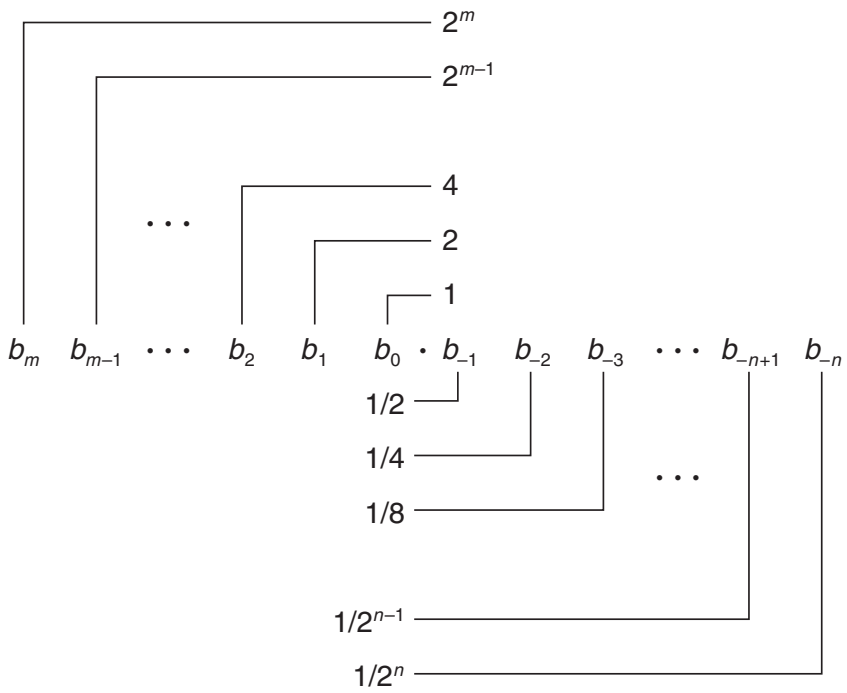


图 2.31 （示意图）

$$d = \sum_{i=-n}^{m} 10^i \times d_i$$

数字的权相对于小数点符号(“.”)来定义:小数点左边的数字用 10 的非负次幂加权,给出整数值;右边的数字用 10 的负次幂加权,给出小数值。例如,12.34_{10} 表示数 $1 \times 10^1 + 2 \times 10^0 + 3 \times 10^{-1} + 4 \times 10^{-2} = 12\,\frac{34}{100}$。

仿照这个思路，考虑形如

$$b_m\, b_{m-1} \ldots b_1\, b_0\, .\, b_{-1}\, b_{-2} \ldots b_{-n+1}\, b_{-n}$$

其中每个二进制数字 b_i（也称为位）的取值范围是 0 和 1，如图 2.31 所示。这种记法表示一个数 b，定义为

$$b = \sum_{i=-n}^{m} 2^i \times b_i \tag{2.19}$$

这时符号“.”便成为二进制小数点（binary point）：小数点左边的位用 2 的非负次幂加权，右边的位用 2 的负次幂加权。例如，101.11_2 表示数 $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 4 + 0 + 1 + \frac{1}{2} + \frac{1}{4} = 5\,\frac{3}{4}$。

从公式 2.19 可以直接看出，把二进制小数点向左移一位，相当于把这个数除以 2。例如，101.11_2 表示数 $5\,\frac{3}{4}$，而 10.111_2 表示数 $2 + 0 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = 2\,\frac{7}{8}$。反过来，把二进制小数点向右移一位，相当于把这个数乘以 2：例如 1011.1_2 表示数 $8 + 0 + 2 + 1 + \frac{1}{2} = 11\,\frac{1}{2}$。

注意，形如 $0.11 \ldots 1_2$ 的数表示的是略小于 1 的数。例如 0.111111_2 表示 $\frac{63}{64}$。后面我们会用 $1.0 - \epsilon$ 这种简写来表示这样的小数。

假设只考虑有限长度的编码，那么十进制记法无法精确表示像 $\frac{1}{3}$ 和 $\frac{5}{7}$ 这样的数。类似地，二进制小数记法只能表示那些可以写成 $x \times 2^y$ 形式的数，其他值只能近似。例如，数 $\frac{1}{5}$ 可以精确地表示为十进制小数 0.20；但当作二进制小数时，我们就无法精确表示它，只能通过加长二进制表示来不断提高近似的精度：

表示	数值	十进制
0.0_2	$\frac{0}{2}$	0.0_{10}
0.01_2	$\frac{1}{4}$	0.25_{10}
0.010_2	$\frac{2}{8}$	0.25_{10}
0.0011_2	$\frac{3}{16}$	0.1875_{10}
0.00110_2	$\frac{6}{32}$	0.1875_{10}
0.001101_2	$\frac{13}{64}$	0.203125_{10}
0.0011010_2	$\frac{26}{128}$	0.203125_{10}
0.00110011_2	$\frac{51}{256}$	0.19921875_{10}

练习 2.45 请填写下面表格中缺失的信息：

练习 2.46 浮点算术的不精确性可能带来灾难性的后果。1991 年 2 月 25 日，海湾战争期间，驻扎在沙特阿拉伯达兰（Dharan）的一个美军爱国者导弹连未能拦截来袭的一枚伊拉克“飞毛腿”导弹。这枚导弹击中一座美军营房，炸死

小数值	二进制表示	十进制表示
$\frac{1}{8}$	0.001	0.125
$\frac{3}{4}$		
$\frac{5}{16}$		
	10.1011	
	1.001	
		5.875
		3.1875

28 名士兵。美国总审计局(General Accounting Office,GAO)对这次失败作了详细分析 [76],认定其根本原因是一次数值计算不精确。本练习将让你重现 GAO 分析的一部分。

爱国者系统内有一个时钟,实现为一个每 0.1 秒递增一次的计数器。为了得到以秒为单位的时间,程序要把这个计数器的值乘以一个 24 位的量,它相当于 $\frac{1}{10}$ 的二进制小数近似。具体地说,$\frac{1}{10}$ 的二进制表示是不终止的序列 $0.000110011[0011]\ldots_2$,其中括号部分无限重复。程序用一个值 x 来近似 0.1,只考虑二进制小数点右侧序列的前 23 位:$x = 0.00011001100110011001100_2$。(关于他们本来可以怎样更精确地近似 0.1,见家庭作业 2.51 中的讨论。)

A. $0.1 - x$ 的二进制表示是什么?

B. $0.1 - x$ 的近似十进制值是多少?

C. 系统第一次上电时,时钟从 0 开始计数,此后一直递增。本例中系统已经运行了约 100 小时,软件计算出的时间与实际时间相差多少?

D. 系统根据来袭导弹的速度和上一次雷达探测的时间,预测导弹将出现的位置。已知"飞毛腿"导弹的速度约为 2,000 米/秒,预测会偏差多远?

通常,时钟读数在绝对时间上的一点小误差并不影响跟踪计算,因为跟踪计算依据的应当是两次相邻读数之间的相对时间。问题在于,爱国者软件已升级为使用更精确的读时函数,但并非所有对该函数的调用都换成了新代码。结果,跟踪软件在一次读数中使用了精确时间,而在另一次读数中使用了不精确的时间 [103]。

2.4.2 IEEE 浮点表示

上一节讨论的位置记法在表示非常大的数时效率不高。例如,5×2^{100} 的表示就是位模式 101 后面跟 100 个零。我们其实希望以 $x \times 2^y$ 的形式表示数,同时给出 x 和 y 的值。

IEEE 浮点标准用 $V = (-1)^s \times M \times 2^E$ 的形式表示一个数:

- 符号 s 决定这个数是负数($s = 1$)还是正数($s = 0$),至于数值 0 的符号位作何解释,则作为特殊情况处理。
- 尾数(significand)M 是一个二进制小数,取值范围要么在 1 与 $2 - \epsilon$ 之间,要么在 0 与 $1 - \epsilon$ 之间。
- 阶码(exponent)E 对值加权,权值是 2 的(可能为负的)幂。

浮点数的位表示划分为三个字段,分别对这些值进行编码:

- 单独一个符号位(sign bit)s,它直接编码符号 s。
- k 位的阶码字段 `exp`= $e_{k-1} \ldots e_1 e_0$,它编码阶码 E。
- n 位的小数域(fraction field)`frac`= $f_{n-1} \ldots f_1 f_0$,它编码尾数 M,不过编码出的值还要取决于阶码字段是否等于 0。

图 2.32 给出了这两种最常见格式把这三个字段打包进字中的方式。在单精度(single precision)浮点格式(C 语言中的 `float`)中,s、`exp` 和 `frac` 字段依次为 1 位、$k = 8$ 位和 $n = 23$ 位,共同构成一个 32 位的表示。在双精度(double precision)浮点格式(C 语言中的 `double`)中,s、`exp` 和 `frac` 字段依次为 1 位、$k = 11$ 位和 $n = 52$ 位,共同构成一个 64 位的表示。

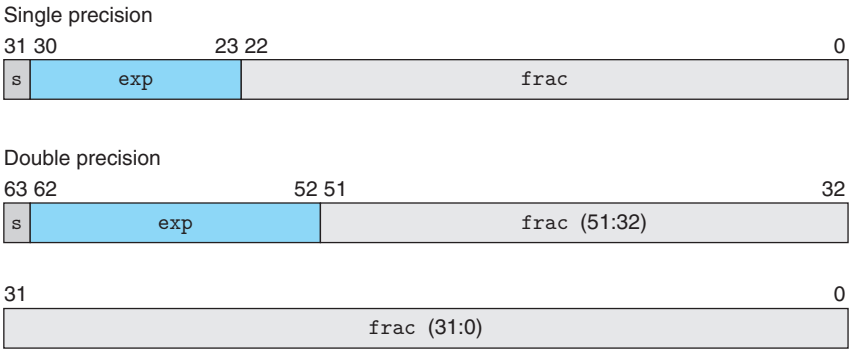


图 2.32 (示意图)

一个给定位表示编码出的值,可以按照 `exp` 字段的值分为三种不同的情况(最后一种情况又有两个变体)。图 2.33 以单精度格式为例说明了这三种情况。

情况 1:规格化的值

这是最常见的情况。当 `exp` 的位模式既不全为 0(数值 0)也不全为 1(单精度时这个数值为 255,双精度时为 2047)时,就属于这种情况。此时,阶码字段被解释为以偏置形式表示的一个有符号整数。也就是说,阶码值为 $E = e - \text{Bias}$,其中 e 是位模式 $e_{k-1} \ldots e_1 e_0$ 对应的无符号数,而 Bias 是一个偏置(bias)值,等

于 $2^{k-1}-1$（单精度为 127，双精度为 1023）。这样，单精度得到的阶码范围是 -126 到 $+127$，双精度是 -1022 到 $+1023$。

小数域 `frac` 被解释为描述小数值 f，其取值范围是 $0 \le f < 1$，二进制表示为 $0.f_{n-1}\cdots f_1 f_0$，也就是说，二进制小数点位于最高有效位的左边。尾数定义为 $M = 1 + f$。这种表示有时称为隐含前导 1（implied leading 1）表示，因为我们可以把 M 看作二进制表示为 $1.f_{n-1}f_{n-2}\cdots f_0$ 的数。这种表示是免费多得一位精度的技巧：只要假设没有溢出，我们总能调整阶码 E，使尾数 M 落在范围 $1 \le M < 2$ 内，因此无须显式表示前导位，它恒等于 1。

图 2.33　单精度浮点值的分类。阶码的值决定这个数是(1)规格化的、(2)非规格化的，还是(3)特殊值。

类别	符号位	阶码 `exp`	小数域 `frac`
1. 规格化	s	$\neq 0$ 且 $\neq 255$	f
2. 非规格化	s	全 0	f
3a. 无穷	s	全 1	全 0
3b. NaN	s	全 1	$\neq 0$

边栏　非规格化值的偏置为何这样设置？

阶码值取 $1 - \text{Bias}$ 而不是简单地取 $-\text{Bias}$，看上去也许违反直觉。我们很快就会看到，这样安排使从非规格化值到规格化值的过渡非常平滑。

情况 2：非规格化的值

当阶码字段全为 0 时，所表示的数是非规格化（denormalized）形式。此时，阶码值为 $E = 1 - \text{Bias}$，尾数值为 $M = f$，也就是小数域的值，不包含隐含的前导 1。

非规格化数有两个用途。第一，它们提供了一种表示数值 0 的方法，因为规格化数必须始终满足 $M \ge 1$，所以无法利用规格化数表示 0。事实上，+0.0 的浮点表示就是全 0 的位模式：符号位为 0，阶码字段全为 0（表明这是一个非规格化值），小数域也全为 0，得到 $M = f = 0$。有意思的是，当符号位为 1 而其他字段全为 0 时，得到的是值 -0.0。IEEE 浮点格式把 -0.0 和 $+0.0$ 看作在某些方面不同、在另一些方面相同的两个值。

非规格化数的第二个用途是表示非常接近 0.0 的数。它们提供了一种称为逐渐下溢（gradual underflow）的性质：在 0.0 附近，可能的数值分布得很均匀。

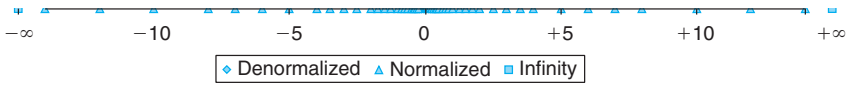
情况 3：特殊值

最后一类值在阶码字段全为 1 时出现。当小数域全为 0 时，得到的值表示无穷：$s = 0$ 时是 $+\infty$，$s = 1$ 时是 $-\infty$。无穷可以用来表示溢出的结果，比如两个非常大的数相乘，或者除以零。当小数域非零时，得到的值称为 NaN（not a number，“非数”）。这样的值是一些运算的结果无法表示为实数或无穷时的返

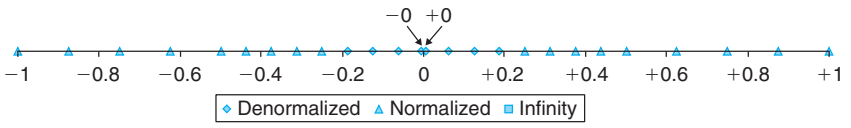
回值，比如计算 $\sqrt{-1}$ 或 $\infty - \infty$ 时。在某些应用中，NaN 还可以用来表示未初始化的数据。

2.4.3 示例数字

图 2.34 给出了一个假想的 6 位格式所能表示的值的集合，该格式有 $k = 3$ 个阶码位和 $n = 2$ 个小数位。偏置值为 $2^{3-1} - 1 = 3$。图的(a)部分画出了所有可表示的值（NaN 除外）。两个无穷值位于最两端。绝对值最大的规格化数是 ± 14。非规格化数集中在 0 附近。这一点在图的(b)部分看得更清楚，那里只画出了 -1.0 到 $+1.0$ 之间的数。两个 0 都是非规格化数的特殊情形。请注意，可表示的数不是均匀分布的——越靠近原点分布越密。



(a) Complete range



(b) Values between −1.0 and +1.0

图 2.34 （示意图）

图 2.35 给出了一个假想的 8 位浮点格式的一些示例值，该格式有 $k = 4$ 个阶码位和 $n = 3$ 个小数位。偏置值为 $2^{4-1} - 1 = 7$。图分为三个区域，表示三类数。不同的列表明阶码字段如何编码阶码 E，小数域如何编码尾数 M，两者合起来构成表示的值 $V = 2^E \times M$。最接近 0 的是非规格化数，从 0 本身开始。该格式中非规格化数的 $E = 1 - 7 = -6$，给出权值 $2^E = \frac{1}{64}$。小数 f 和尾数 M 的取值范围是 $0、\frac{1}{8}、\ldots、\frac{7}{8}$，给出 V 在 0 到 $\frac{1}{64} \times \frac{7}{8} = \frac{7}{512}$ 范围内的数。

该格式中最小的规格化数同样有 $E = 1 - 7 = -6$，小数取值范围同样是 $0、\frac{1}{8}、\ldots、\frac{7}{8}$。不过，尾数此时的取值范围是 $1 + 0 = 1$ 到 $1 + \frac{7}{8} = \frac{15}{8}$，给出 V 在 $\frac{8}{512} = \frac{1}{64}$ 到 $\frac{15}{512}$ 范围内的数。

请注意，最大非规格化数 $\frac{7}{512}$ 与最小规格化数 $\frac{8}{512}$ 之间的过渡是平滑的。这种平滑性正是由我们对非规格化值的 E 的定义带来的。把 E 定为 $1 - \text{Bias}$ 而不是 $-\text{Bias}$，我们就补偿了非规格化数的尾数没有隐含前导 1 这件事。

随着阶码的增大，我们依次得到越来越大的规格化值，中途经过 1.0，一直到达最大的规格化数。这个数的阶码 $E = 7$，给出权 $2^E = 128$；小数值为 $7/8$，给出尾数 $M = 15/8$。于是，数值为 $V = 240$。超出这个值就会溢出为 $+\infty$。

图 2.35　8 位浮点格式的非负示例值。有 $k=4$ 个阶码位和 $n=3$ 个小数位。偏置值为 7。

说明	位表示	阶码			尾数		值		
		e	E	2^E	f	M	$2^E \times M$	V	十进制
零	0 0000 000	0	-6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{0}{8}$	$\frac{0}{512}$	0	0.0
最小的正数	0 0000 001	0	-6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{512}$	$\frac{1}{512}$	0.001953
	0 0000 010	0	-6	$\frac{1}{64}$	$\frac{2}{8}$	$\frac{2}{8}$	$\frac{2}{512}$	$\frac{1}{256}$	0.003906
	0 0000 011	0	-6	$\frac{1}{64}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{3}{512}$	$\frac{3}{512}$	0.005859
	⋮								
最大非规格化数	0 0000 111	0	-6	$\frac{1}{64}$	$\frac{7}{8}$	$\frac{7}{8}$	$\frac{7}{512}$	$\frac{7}{512}$	0.013672
最小规格化数	0 0001 000	1	-6	$\frac{1}{64}$	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{512}$	$\frac{1}{64}$	0.015625
	0 0001 001	1	-6	$\frac{1}{64}$	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{512}$	$\frac{9}{512}$	0.017578
	⋮								
	0 0110 110	6	-1	$\frac{1}{2}$	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{14}{16}$	$\frac{7}{8}$	0.875
	0 0110 111	6	-1	$\frac{1}{2}$	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{15}{16}$	$\frac{15}{16}$	0.9375
1	0 0111 000	7	0	1	$\frac{0}{8}$	$\frac{8}{8}$	$\frac{8}{8}$	1	1.0
	0 0111 001	7	0	1	$\frac{1}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	$\frac{9}{8}$	1.125
	0 0111 010	7	0	1	$\frac{2}{8}$	$\frac{10}{8}$	$\frac{10}{8}$	$\frac{5}{4}$	1.25
	⋮								
	0 1110 110	14	7	128	$\frac{6}{8}$	$\frac{14}{8}$	$\frac{1792}{8}$	224	224.0
最大规格化数	0 1110 111	14	7	128	$\frac{7}{8}$	$\frac{15}{8}$	$\frac{1920}{8}$	240	240.0
无穷	0 1111 000	——	——	——	——	——	——	∞	——

这种表示有一个有趣的性质：如果把图 2.35 中各值的位表示解读为无符号整数，那么它们按升序排列，而这些位表示作为浮点数所代表的值也按同样的顺序排列。这并非巧合——IEEE 格式在设计时就使得浮点数可以借助整数排序例程来排序。处理负数时会有一个小麻烦：负数的位表示以 1 开头，并且按降序出现。不过，不用浮点运算也能进行比较，这个困难可以克服（见家庭作业 2.84）。

练习 2.47　考虑一种基于 IEEE 浮点格式的 5 位浮点表示，它有 1 个符号位、2 个阶码位（$k=2$）和 2 个小数位（$n=2$）。阶码偏置为 $2^{2-1}-1=1$。

接着的那张表列出了这种 5 位浮点表示的整个非负范围。请按下面的说明填写表中的空项：

e：把阶码域看作无符号整数时所表示的值

E：偏置之后的阶码值

2^E：阶码的数值权

f：小数值

M：尾数值

$2^E \times M$：该数的(未经约简的)小数值

V：该数的(约简后的)小数值

Decimal：该数的十进制表示

2^E、f、M、$2^E \times M$ 和 V 的值,能写成整数就写成整数,否则写成 x/y 形式的分数,其中 y 是 2 的幂。标有“—”的项不必填写。

位模式	e	E	2^E	f	M	$2^E \times M$	V	十进制
0 00 00								
0 00 01								
0 00 10								
0 00 11								
0 01 00								
0 01 01	1	0	1	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	1.25
0 01 10								
0 01 11								
0 10 00								
0 10 01								
0 10 10								
0 10 11								
0 11 00	—	—	—	—	—	—		—
0 11 01	—	—	—	—	—	—		—
0 11 10	—	—	—	—	—	—		—
0 11 11	—	—	—	—	—	—		—

图 2.36 给出了一些重要的单精度和双精度浮点数的表示和数值。与图 2.35 所示的 8 位格式一样,对于一个阶码为 k 位、小数为 n 位的浮点表示,我们可以观察到一些一般性质:

- 值 +0.0 的位表示总是全零。

- 最小的正非规格化数，其位表示在最低有效位上为 1，其余各位全为零。它的小数（也是尾数）值为 $M = f = 2^{-n}$，阶码值为 $E = -2^{k-1} + 2$。因此，它的数值为 $V = 2^{-n-2^{k-1}+2}$。
- 最大的非规格化数，其位表示的阶码域全为零、小数域全为一。它的小数（也是尾数）值为 $M = f = 1 - 2^{-n}$（我们记作 $1 - \epsilon$），阶码值为 $E = -2^{k-1} + 2$。因此，它的数值为 $V = (1 - 2^{-n}) \times 2^{-2^{k-1}+2}$，只比最小的规格化值还要小一点。
- 最小的正规格化数，其位表示的阶码域最低有效位为 1，其余各位全为零。它的尾数值为 $M = 1$，阶码值为 $E = -2^{k-1} + 2$。因此，它的数值为 $V = 2^{-2^{k-1}+2}$。
- 值 1.0 的位表示中，阶码域除最高有效位外全部为 1，其余各位全为零。它的尾数值为 $M = 1$，阶码值为 $E = 0$。
- 最大的规格化数，其位表示为：符号位为 0，阶码的最低有效位为 0，其余各位全为 1。它的小数值为 $f = 1 - 2^{-n}$，给出尾数 $M = 2 - 2^{-n}$（我们记作 $2 - \epsilon$）。它的阶码值为 $E = 2^{k-1} - 1$，给出数值 $V = (2 - 2^{-n}) \times 2^{2^{k-1}-1} = (1 - 2^{-n-1}) \times 2^{2^{k-1}}$。

图 2.36 非负浮点数的示例

说明	exp	frac	单精度数值	单精度十进制	双精度数值	双精度十进制
零	00...00	0...00	0	0.0	0	0.0
最小非规格化数	00...00	0...01	$2^{-23} \times 2^{-126}$	1.4×10^{-45}	$2^{-52} \times 2^{-1022}$	4.9×10^{-324}
最大非规格化数	00...00	1...11	$(1 - \epsilon) \times 2^{-126}$	1.2×10^{-38}	$(1 - \epsilon) \times 2^{-1022}$	2.2×10^{-308}
最小规格化数	00...01	0...00	1×2^{-126}	1.2×10^{-38}	1×2^{-1022}	2.2×10^{-308}
1	01...11	0...00	1×2^{0}	1.0	1×2^{0}	1.0
最大规格化数	11...10	1...11	$(2 - \epsilon) \times 2^{127}$	3.4×10^{38}	$(2 - \epsilon) \times 2^{1023}$	1.8×10^{308}

理解浮点表示的一个有用的练习，是把整数取一些样本值转换成浮点形式。例如，在图 2.15 中我们看到 12,345 的二进制表示为 `[1100000 0111001]`。把它规格化，相对二进制小数点向右移 13 位，得到 $12{,}345 = 1.1000000111001_2 \times 2^{13}$。用 IEEE 单精度格式对它编码时，我们舍去前导的 1，并在末尾补上 10 个 0，由此构造出小数域，得到二进制表示 `[10000001110010000000000]`。为了构造阶码域，给 13 加上偏置 127，得到 140，其二进制表示为 `[10001100]`。把它与符号位 0 组合起来，就得到该数的浮点二进制表示 `[01000110010000001110010000000000]`。回想 2.1.3 节，我们曾在整数值 12345（`0x3039`）与单精度浮点值 12345.0（`0x4640E400`）的位级表示之间观察到下面这种关联：

```
  0   0   0   0   3   0   3   9
00000000000000000011000000111001
                  *************
           4   6   4   0   E   4   0   0
01000110010000001110010000000000
```

现在可以看出，这个对应区域对应的是整数的低位部分，正好停在最高有效位 1 的前面（这一位构成了隐含的前导 1），并与浮点表示中小数部分的高位相匹配。

练习 2.48 正如家庭作业 2.6 中提到的，整数 3,510,593 的十六进制表示为 `0x00359141`，而单精度浮点数 3,510,593.0 的十六进制表示为 `0x4A564504`。请推导出这个浮点表示，并解释整数表示与浮点表示的各位之间的关联。

练习 2.49

A. 对于一种有 n 位小数的浮点格式，给出不能被精确表示的最小正整数（因为它需要 $n+1$ 位小数才能精确表示）的公式。假定阶码域的大小 k 足够大，可表示阶码的范围不会成为本题的限制。

B. 对于单精度格式（$n = 23$），这个整数的数值是多少？

2.4.4 舍入

浮点运算只能近似于实数运算，因为这种表示的取值范围和精度都是有限的。因此，对于一个值 x，我们通常想要一种系统的方法，找出一个能够用目标浮点格式表示的“最接近”的值 x'。这就是舍入（rounding）运算的任务。关键问题之一是：当一个值正好位于两个可能结果的中间时，应当确定往哪个方向舍入。例如，我有 $1.50，想把它舍入到最接近的整数美元，结果应该是 $1 还是 $2？另一种做法是给实际数量维护一个下界和一个上界。例如，我们可以确定两个可表示的值 $x\hat{}-$ 和 $x\hat{}+$，使 x 保证落在它们之间：$x\hat{}- \leq x \leq x\hat{}+$。IEEE 浮点格式定义了四种不同的舍入方式。默认的方式是寻找最接近的匹配，另外三种方式则可以用来计算上界和下界。

图 2.37 展示了把金额舍入到最接近的整数美元时四种舍入方式的应用。向偶数舍入（round-to-even，也称为 round-to-nearest 就近舍入）是默认方式，它试图找到一个最接近的匹配。因此，它把 $1.40 舍入到 $1，把 $1.60 舍入到 $2，因为它们是最接近的整数美元值。唯一需要设计的是：当一个值正好位于两个可能结果的中间时，如何确定舍入的效果。向偶数舍入方式采用的约定是：向上或向下舍入，使得结果的最低有效数字是偶数。因此，它把 $1.50 和 $2.50 都舍入到 $2。

另外三种方式对实际值产生有保证的界，在某些数值应用中会很有用。向零舍入方式把正数向下舍入、把负数向上舍入，得到一个值 $\hat{x}$，满足 $|\hat{x}| \leq |x|$。

Mode	$1.40	$1.60	$1.50	$2.50	$–1.50
Round-to-even	$1	$2	$2	$2	$–2
Round-toward-zero	$1	$1	$1	$2	$–1
Round-down	$1	$1	$1	$2	$–2
Round-up	$2	$2	$2	$3	$–1

图 2.37 （示意图）

向下舍入方式把正数和负数都向下舍入，得到一个值 x^-，满足 $x^- \leq x$。向上舍入方式把正数和负数都向上舍入，得到一个值 x^+，满足 $x \leq x^+$。

向偶数舍入初看起来目标相当随意——有什么理由偏好偶数呢？为什么不把位于两个可表示值中间的值始终向上舍入？这种约定的问题在于，很容易想象出这样的场景：按这种方式对一组数据值进行舍入，会在计算这组值的平均值时引入统计偏差。按这种方式舍入得到的一组数的平均值，会比这组数本身的平均值略高。反过来，如果我们总是把位于中间的数向下舍入，那么舍入后的这组数的平均值就会比原平均值略低。向偶数舍入在大多数现实情形中避免了这种统计偏差，它大约有 50% 的时间向上舍入，50% 的时间向下舍入。

即使不是舍入到整数，向偶数舍入也可以应用。我们只需考虑最低有效数字是偶数还是奇数。例如，假设我们要把十进制数舍入到最接近的百分位。无论采用哪种舍入方式，我们都会把 1.2349999 舍入为 1.23，把 1.2350001 舍入为 1.24，因为它们并不在 1.23 和 1.24 的中间。另一方面，1.2350000 和 1.2450000 我们都会舍入为 1.24，因为 4 是偶数。

同样，向偶数舍入也可以应用到二进制小数上。我们规定最低有效位的值为 0 时算作偶数，为 1 时算作奇数。一般来说，只有当位模式具有 `XX...X.YY...Y100...` 的形式时，舍入方式才起作用，其中 `X` 和 `Y` 表示任意的位值，最右边的 `Y` 是我们希望舍入到的位置。只有这种形式的位模式所表示的值才位于两个可能结果的中间。举例来说，考虑把值舍入到最接近的四分之一（即二进制小数点右边 2 位）。我们会把 $10.00011_2 (2\frac{3}{32})$ 向下舍入为 10.00_2 (2)，把 $10.00110_2 (2\frac{3}{16})$ 向上舍入为 $10.01_2 (2\frac{1}{4})$，因为这些值不在两个可能值的中间。我们会把 $10.11100_2 (2\frac{7}{8})$ 向上舍入为 $11.00_2 (3)$，把 $10.10100_2 (2\frac{5}{8})$ 向下舍入为 $10.10_2 (2\frac{1}{2})$，因为这些值正好位于两个可能结果的中间，而我们偏要让最低有效位等于 0。

练习 2.50 按照向偶数舍入的规则，说明下面这些二进制小数值如何舍入到最接近的半（二进制小数点右边 1 位）。每种情况都要给出舍入之前和之后的数值。

A. 10.111_2

B. 11.010_2

C. 11.000_2

D. 10.110_2

练习 2.51 我们在家用作业 2.6 中看到，爱国者导弹软件把 0.1 近似为 $x = 0.0001100110011001100110011_2$。假设他们改用 IEEE 向偶数舍入方式来确定 0.1 的近似值 x'，二进制小数点右边保留 23 位。

A. x' 的二进制表示是什么？

B. $x' - 0.1$ 的近似十进制值是多少？

C. 运行 100 小时之后，计算出的时钟会相差多少？

D. 程序对“飞毛腿”导弹位置做出的预测会相差多少？

练习 2.52 考虑下面两种基于 IEEE 浮点格式的 7 位浮点表示。两者都没有符号位——它们只能表示非负数。

1. **格式 A**

   有 $k = 3$ 个阶码位，阶码偏置为 3。有 $n = 4$ 个小数位。

2. **格式 B**

   有 $k = 4$ 个阶码位，阶码偏置为 7。有 $n = 3$ 个小数位。

下面以格式 A 给出了一些位模式，任务是把它们转换成格式 B 中最接近的值。必要时应当应用向偶数舍入规则。此外，还要给出格式 A 和格式 B 的位模式所表示的数值。这些值写成整数（例如 17）或分数（例如 17/64）的形式。

格式 A		格式 B	
位	值	位	值
011 0000	1	0111 000	1
101 1110			
010 1001			
110 1111			
000 0001			

2.4.5 浮点运算

IEEE 标准为确定加法、乘法这类算术运算的结果指定了一条简单的规则。把浮点值 x 和 y 看作实数，$\odot$ 是定义在实数上的某个运算，那么计算结果应当是 Round$(x \odot y)$，也就是把舍入应用于该实数运算的精确结果之后所得到的值。在实际实现中，浮点部件的设计者会使用一些巧妙的技巧来避免做这个精确计算，因为计算只需足够精确，就可以保证舍入后的结果是对的。当其中一个参

数是特殊值，比如 -0、∞ 或 NaN 时，标准指定了一些力图合理的约定。例如，定义 $1/-0$ 得到 $-\infty$，而 $1/+0$ 得到 $+\infty$。

IEEE 标准指定浮点运算行为的方法有一个优点：它独立于任何特定的硬件或软件实现。这样，我们就可以在不考虑实际如何实现的情况下，考察它的抽象数学性质。

我们之前看到，整数加法，无论无符号的还是补码的，都构成一个阿贝尔群（abelian group）。实数上的加法也构成一个阿贝尔群，但是我们必须考虑舍入对这些性质有何影响。定义 $x +^f y$ 为 Round$(x + y)$。这个运算对 x 和 y 的所有值都有定义，不过即使 x 和 y 都是实数，由于溢出，它也可能产生无穷大。这个运算满足交换律，对 x 和 y 的所有值都有 $x +^f y = y +^f x$。但这个运算不满足结合律。例如，对于单精度浮点数，表达式 `(3.14+1e10)-1e10` 求值结果为 0.0——由于舍入，值 3.14 丢失了。另一方面，表达式 `3.14+(1e10-1e10)` 的求值结果为 3.14。与阿贝尔群一样，大多数值在浮点加法下都有逆元，即 $x +^f -x = 0$。例外是无穷大（因为 $+\infty - \infty =$ NaN）和 NaN，因为对任意 x 都有 NaN $+^f x =$ NaN。

浮点加法不满足结合律，这是最重要的缺失的群性质。它对科学计算程序员和编译器编写者都有重要影响。例如，假设给编译器下面这段代码：

```
x = a + b + c;
y = b + c + d;
```

编译器可能想省去一次浮点加法，生成下面的代码：

```
t = b + c;
x = a + t;
y = t + d;
```

然而，这个计算所得到的 x 的值可能与原来的不同，因为它使用了不同的加法结合方式。在大多数应用中，这种差异会小到无关紧要。遗憾的是，编译器无法知道用户愿意在效率与忠实于原程序的精确行为之间做怎样的权衡。因此，编译器往往非常保守，避免任何可能对功能产生哪怕最细微影响的优化。

另一方面，浮点加法满足下面的单调性（monotonicity）性质：如果 $a \geq b$，那么对除 NaN 之外的任何 a、b 和 x 的值，都有 $x +^f a \geq x +^f b$。实数（以及整数）加法具有这一性质，而无符号加法和补码加法都不遵守。

浮点乘法也遵守人们通常与乘法相联系的许多性质。定义 $x *^f y$ 为 Round$(x \times y)$。这个运算对乘法是封闭的（尽管可能产生无穷大或 NaN），它满足交换律，并且以 1.0 为乘法单位元。另一方面，由于有可能溢出，或者舍入造成精度损失，它不满足结合律。例如，对于单精度浮点数，表达式

`(1e20*1e20)*1e-20` 求值结果为 $+\infty$，而 `1e20*(1e20*1e-20)` 求值结果为 `1e20`。另外，浮点乘法对加法不满足分配律。例如，对于单精度浮点数，表达式 `1e20*(1e20-1e20)` 求值结果为 0.0，而 `1e20*1e20-1e20*1e20` 求值结果为 NaN。另一方面，对于除 NaN 之外的任何 a、b 和 c 的值，浮点乘法满足下面的单调性性质：

$$a \geq b \quad \text{且} \quad c \geq 0 \quad \Rightarrow \quad a *^f c \geq b *^f c$$
$$a \geq b \quad \text{且} \quad c \leq 0 \quad \Rightarrow \quad a *^f c \leq b *^f c$$

另外，只要 $a \neq \text{NaN}$，我们还可以保证 $a *^f a \geq 0$。正如我们之前看到的，这些单调性性质没有一条对无符号乘法或补码乘法成立。

缺少结合律和分配律，对科学计算程序员和编译器编写者来说是一件非常严重的事。就连编写代码判断三维空间中的两条直线是否相交这样看似简单的任务，都可能成为一项重大挑战。

2.4.6 C 语言中的浮点数

C 的所有版本都提供两种不同的浮点数据类型：`float` 和 `double`。在支持 IEEE 浮点的机器上，这两种数据类型分别对应单精度和双精度浮点数。另外，这些机器使用向偶数舍入方式。遗憾的是，由于 C 标准不要求机器使用 IEEE 浮点，所以没有标准的方法可以改变舍入方式，或者得到像 -0、$+\infty$、$-\infty$ 或 NaN 这样的特殊值。大多数系统会提供一组头文件（`.h` 文件）和过程库来提供对这些功能的访问，但具体细节因系统而异。例如，当程序文件中出现下面这两行时，GNU 编译器 `gcc` 会定义程序常量 `INFINITY`（代表 $+\infty$）和 `NAN`（代表 NaN）：

```
#define _GNU_SOURCE 1
#include <math.h>
```

练习 2.53 填写下面的宏定义，使它们生成双精度值 $+\infty$、$-\infty$ 和 -0：

```
#define POS_INFINITY
#define NEG_INFINITY
#define NEG_ZERO
```

不允许使用任何头文件（比如 `math.h`），但可以利用这样一个事实：用双精度所能表示的最大有限数大约是 1.8×10^{308}。

在 `int`、`float` 和 `double` 格式之间对值进行强制类型转换时，程序会按下面的方式改变数值和位表示（假设数据类型 `int` 为 32 位）：

- 从 `int` 转换到 `float`，数值不会溢出，但可能被舍入。

- 从 int 或 float 转换到 double，因为 double 既有更大的取值范围（即数值范围），又有更高的精度（即有效位数更多），所以精确的数值可以被保留下来。
- 从 double 转换到 float，因为范围变小，值可能溢出为 $+\infty$ 或 $-\infty$。此外，因为精度变小，还可能被舍入。
- 从 float 或 double 转换到 int，值将被向零舍入。例如，1.999 将被转换成 1，而 -1.999 将被转换成 -1。而且，值还可能溢出。C 标准对这种情况没有规定固定的结果。Intel 兼容的微处理器把位模式 [10...00]（字长为 w 时的 TMin_w）指定为整数不定值（integer indefinite）。任何不能给出合理整数近似的浮点数到整数的转换，都会产生这个值。因此，表达式 (int) +1e10 产生 -21483648，从一个正数得到一个负值。

练习 2.54 假设变量 x、f 和 d 的类型分别是 int、float 和 double。它们的取值任意，只是 f 和 d 都不等于 $+\infty$、$-\infty$ 或 NaN。对于下面每个 C 表达式，要么论证它总为真（即求值为 1），要么给变量找一组值，使它不为真（即求值为 0）。

A. `x == (int)(double) x`

B. `x == (int)(float) x`

C. `d == (double)(float) d`

D. `f == (float)(double) f`

E. `f == -(-f)`

F. `1.0/2 == 1/2.0`

G. `d*d >= 0.0`

H. `(f+d)-f == d`

2.5 小结

计算机把信息编码成位，位一般组织成字节序列。表示整数、实数和字符串时使用不同的编码。不同型号的计算机在数的编码方式以及多字节数据中字节的排列顺序上使用不同的约定。

C 语言在设计时要适应字长和数字编码方面各种各样的实现。64 位字长的机器日益普及，取代了在市场上占主导地位约 30 年的 32 位机器。由于 64 位机器也能运行专为 32 位机器编译的程序，我们一直关注的是 32 位与 64 位程序之间的区别，而不是机器之间的区别。64 位程序的优点在于，它能够突破 32 位程序 4 GB 的地址限制。

大多数机器使用补码表示来编码有符号数，并按照 IEEE 754 标准编码浮点数。要在位的层面上理解这些编码，同时理解算术运算的数学特性，这对于编写能在全部数值取值范围内正确运行的程序十分重要。

在同样位长的有符号整数与无符号整数之间转换时，大多数 C 实现都遵循这样一个约定：底层的位模式不变。在补码机器上，对 w 位的值来说，这种行为可以用 $\mathrm{T2U}_w$ 和 $\mathrm{U2T}_w$ 这两个函数来刻画。C 的隐式转换会给出许多程序员没有预料到的结果，往往导致程序错误。

由于编码长度有限，计算机算术的性质与传统意义上的整数算术和实数算术大不相同。当数值超出表示的取值范围时，有限长度会导致溢出。当浮点值非常接近 0.0 以致被改成零时，还会发生下溢。

C 以及大多数其他编程语言实现的有限整数算术，与真正的整数算术相比有一些奇特的性质。例如，由于溢出，表达式 `x*x` 的求值结果可能是一个负数。不过，无符号算术和补码算术仍然满足整数算术的许多其他性质，包括结合律、交换律和分配律。这使得编译器可以做很多优化。例如，用 `(x<<3)-x` 替换表达式 `7*x` 时，我们用到了结合、交换和分配性质，再加上移位与乘以 2 的幂之间的关系。

我们已经看到几种把位级运算和算术运算组合起来加以利用的巧妙方法。例如，我们看到了在补码算术中 `x+1` 与 `-x` 等价。再举一个例子，假设我们想要一个位

……形式的位模式 $[0, \ldots, 0, 1, \ldots, 1]$，由 $w-k$ 个 0 后跟 k 个 1 组成。这样的位模式对掩码运算很有用。这种模式可以用 C 表达式 `(1<<k)-1` 生成，它利用了该位模式数值为 $2^k - 1$ 这一性质。例如，表达式 `(1<<8)-1` 会生成位模式 `0xFF`。

浮点表示通过编码 $x \times 2^y$ 形式的数来近似实数。IEEE 标准 754 提供多种不同的精度，最常见的是单精度（32 位）和双精度（64 位）。IEEE 浮点对表示正无穷大和负无穷大的特殊值，以及非数（NaN），也有相应的表示。

必须非常小心地使用浮点运算，因为它的表示范围和精度有限，而且它不满足结合性这类常见的数学性质。

边栏　阿丽亚娜 5（Ariane 5）：浮点溢出的高昂代价

把过大的浮点数转换为整数是编程错误的一个常见来源。1996 年 6 月 4 日，这样一个错误给阿丽亚娜 5 号火箭的首航带来了灾难性后果。升空仅 37 秒后，火箭便偏离了飞行轨道，解体爆炸。火箭上搭载着价值 5 亿美元的通信卫星。事后的调查 [73, 33] 表明，控制惯性导航系统的计算机向控制发动机喷嘴的计算机发送了无效数据。它发送的不是飞行控制信息，而是一个诊断位模式，表明在把一个 64 位浮点数转换成 16 位有符号整数的过程中发生了溢出。溢出的值度量的是火箭的水平速度，它可能比早期阿丽亚娜 4 号火箭所达到的速度高出 5 倍以上。在设计阿丽亚娜 4 号的软件时，工程师们仔细分析了这些数值，确定水平速度决不会使 16 位数溢出。不幸的是，他们直接把软件的这一部分用到了阿丽亚娜 5 号上，却没有检验它所依据的假设。

2.6 参考文献说明

有关 C 的参考图书 [45, 61] 讨论了各种不同数据类型和运算的性质。这两本书中，只有 Steele 和 Harbison 的著作 [45] 讲到了 ISO C99 中的较新特性。目前似乎还没有讲到 ISO C11 新特性的书。C 标准没有规定精确的字长或数值编码这类细节。这些细节是特意略去的，以便能把 C 实现到各种不同的机器

上。有几本书为 C 程序员提供建议 [59, 74],告诫读者注意溢出、隐式转换成无符号数,以及本章讨论过的其他一些陷阱。这些书还就变量命名、编码风格和代码测试给出了有益的建议。Seacord 关于 C 和 C++ 程序安全问题的著作 [97] 把关于 C 程序、程序如何编译和执行,以及漏洞如何产生的信息结合在了一起。Java 方面的书(我们推荐由 Java 语言的创始人 James Gosling 参与合著的那一本 [5])介绍了 Java 支持的数据格式和算术运算。

大多数逻辑设计方面的书 [58, 116] 都有一节讲述编码和算术运算。这类书介绍了实现算术电路的各种方法。Overton 关于 IEEE 浮点数的书 [82] 从数值应用程序员的角度,详细描述了这种格式及其性质。

2.7 家庭作业

家庭作业 2.55 ▲ 编译并运行使用 `show_bytes` 的示例代码(文件 `show-bytes.c`),在你能够访问的不同机器上运行。确定这些机器所使用的字节顺序。

家庭作业 2.56 ▲ 换几个不同的示例值,试试 `show_bytes` 的代码。

家庭作业 2.57 ▲ 编写 `show_short`、`show_long` 和 `show_double` 三个函数,分别打印 C 中 `short`、`long` 和 `double` 类型对象的字节表示。在几台不同的机器上试一试它们。

家庭作业 2.58 ▲▲ 编写函数 `is_little_endian`,当它编译并运行在小端法机器上时返回 1,编译并运行在大端法机器上时返回 0。这个程序应当能在任何机器上运行,无论其字长是多少。

家庭作业 2.59 ▲▲ 编写一个 C 表达式,使它生成一个由 x 的最低有效字节和 y 的其余字节组成的字。对于 $x = $ `0x89ABCDEF` 和 $y = $ `0x76543210`,结果应为 `0x765432EF`。

家庭作业 2.60 ▲▲ 假设我们给一个 w 位字中的字节编号,从 0(最低有效)到 $w/8 - 1$(最高有效)。为下面的 C 函数编写代码,该函数返回一个无符号值,其中参数 x 的第 i 个字节已被替换为字节 b:

```
unsigned replace_byte (unsigned x, int i, unsigned char b);
```

下面给出几个示例,说明这个函数应当如何工作:

```
replace_byte(0x12345678, 2, 0xAB) --> 0x12AB5678
replace_byte(0x12345678, 0, 0xAB) --> 0x123456AB
```

2.7.1 位级整数编码规则

在下面的几个问题中，我们会人为限制你能使用的编程结构，以帮助你更好地理解 C 的位级、逻辑和算术运算。在回答这些问题时，你的代码必须遵循这些规则：

- **假设**
  - 整数以补码形式表示。
  - 有符号数据的右移按算术移位执行。
  - 数据类型 `int` 的字长是 w 位。对其中一些问题，会给定 w 的具体值；否则，只要 w 是 8 的倍数，你的代码就应当能正常工作。你可以用表达式 `sizeof(int)<<3` 来计算 w。
- **禁止使用**
  - 条件语句（`if` 或 `?:`）、循环、`switch` 语句、函数调用和宏调用。
  - 除法、取模和乘法。
  - 相对比较运算符（`<`、`>`、`<=` 和 `>=`）。
- **允许的运算**
  - 所有位级运算和逻辑运算。
  - 左移和右移，但移位量只能在 0 和 $w-1$ 之间。
  - 加法和减法。
  - 相等（`==`）测试和不等（`!=`）测试。（有些问题不允许这些。）
  - 整型常量 `INT_MIN` 和 `INT_MAX`。
  - 在数据类型 `int` 和 `unsigned` 之间进行强制类型转换，显式或隐式皆可。

即使有这些规则，你也应当让代码保持可读：选用有描述性的变量名，并加注释说明答案背后的逻辑。作为一个例子，下面的代码从整数参数 x 中提取出最高有效字节：

```
/* 从 x 中取出最高有效字节 */
int get_msb(int x) {
    /* 移位 w-8 */
    int shift_val = (sizeof(int)-1)<<3;
    /* 算术移位 */
    int xright = x >> shift_val;
    /* 除 LSB 外全部清零 */
    return xright & 0xFF;
}
```

家庭作业 2.61 ▲▲ 编写 C 表达式，使它在下列条件成立时求值为 1，不成立时求值为 0。假设 x 的类型是 `int`。

A. x 的任意一位等于 1。

B. x 的任意一位等于 0。

C. x 的最低有效字节中任意一位等于 1。

D. x 的最高有效字节中任意一位等于 0。

你的代码应当遵循位级整数编码规则（见“位级整数编码规则”），另外还有一条限制：不能使用相等（==）测试和不等（!=）测试。

家庭作业 2.62 ▲▲▲ 编写函数 `int_shifts_are_arithmetic()`，当它运行在对 `int` 类型数据采用算术右移的机器上时产生 1，否则产生 0。你的代码应当能在任何字长的机器上工作。在几台不同的机器上测试你的代码。

家庭作业 2.63 ▲▲▲ 为下面的 C 函数填空。函数 `srl` 用一次算术右移（得到的值为 `xsra`）执行逻辑右移，随后再进行其他不包括右移和除法的运算。函数 `sra` 用一次逻辑右移（得到的值为 `xsrl`）执行算术右移，随后再进行其他不包括右移和除法的运算。你可以用计算 `8*sizeof(int)` 来确定 w，即 `int` 类型的位数。移位量 k 的取值范围是 0 到 $w-1$。

```
unsigned srl(unsigned x, int k) {
    /* 用算术方式移位 */
    unsigned xsra = (int) x >> k;
    .
    .
    .
    .
    .
    .
}

int sra(int x, int k) {
    /* 用逻辑方式移位 */
    int xsrl = (unsigned) x >> k;
    .
    .
    .
    .
    .
    .
}
```

家庭作业 2.64 ▲ 编写代码实现下面的函数：

```
/* 当 x 的任意奇数位等于 1 时返回 1;否则返回 0。
   假设 w=32 */
int any_odd_one(unsigned x);
```

你的函数应当遵循位级整数编码规则(见“位级整数编码规则”),另外可以假设数据类型 int 是 $w = 32$ 位。

家庭作业 2.65 ▲▲▲▲ 编写代码实现下面的函数:

```
/* 当 x 中包含奇数个 1 时返回 1;否则返回 0。
   假设 w=32 */
int odd_ones(unsigned x);
```

你的函数应当遵循位级整数编码规则(见“位级整数编码规则”),另外可以假设数据类型 int 是 $w = 32$ 位。

你的代码总共最多只应包含 12 个算术、位级和逻辑运算。

家庭作业 2.66 ▲▲▲ 编写代码实现下面的函数:

```
/*
 * 生成掩码,指示 x 中最左边的 1。假设 w=32。
 * 例如,0xFF00 -> 0x8000,而 0x6600 --> 0x4000。
 * 如果 x = 0,则返回 0。
 */
int leftmost_one(unsigned x);
```

你的函数应当遵循位级整数编码规则(见“位级整数编码规则”),另外可以假设数据类型 int 是 $w = 32$ 位。

你的代码总共最多只应包含 15 个算术、位级和逻辑运算。

提示:先把 x 变换成 $[0 \ldots 011 \ldots 1]$ 形式的位向量。

家庭作业 2.67 ▲▲ 你接到一项任务,要编写过程 int_size_is_32(),当它运行在 int 为 32 位的机器上时产生 1,否则产生 0。不允许使用 sizeof 运算符。下面是最初的一次尝试:

```
1  /* 下面的代码在某些机器上不能正确运行 */
2  int bad_int_size_is_32() {
3      /* 设置 32 位机器的最高有效位 */
4      int set_msb = 1 << 31;
5      /* 移出 32 位字的 msb */
6      int beyond_msb = 1 << 32;
7
8      /* 字长 >= 32 时 set_msb 非零
9         字长 <= 32 时 beyond_msb 为零          */
10     return set_msb && !beyond_msb;
11 }
```

然而,当这段代码在 32 位的 SUN SPARC 上编译并运行时,它却返回 0。下面的编译器消息给我们指出了问题所在:

```
warning: left shift count >= width of type
```

A. 我们的代码在哪些方面不符合 C 标准?

B. 修改这段代码，使它能正确运行在 int 数据类型至少为 32 位的任何机器上。

C. 修改这段代码，使它能正确运行在 int 数据类型至少为 16 位的任何机器上。

家庭作业 2.68 ▲▲ 为具有如下原型的函数编写代码：

```
/*
 * 掩码:最低 n 位置为 1
 * 示例:n = 6 --> 0x3F,n = 17 --> 0x1FFFF
 * 假设 1 <= n <= w
 */
int lower_one_mask(int n);
```

你的函数应当遵循位级整数编码规则（见“位级整数编码规则”）。要当心 $n = w$ 的情况。

家庭作业 2.69 ▲▲▲ 为具有如下原型的函数编写代码：

```
/*
 * 执行循环左移。假设 0 <= n < w
 * 当 x = 0x12345678 且 w = 32 时:
 *    n=4 -> 0x23456781,n=20 -> 0x67812345
 */
unsigned rotate_left(unsigned x, int n);
```

你的函数应当遵循位级整数编码规则（见“位级整数编码规则”）。要当心 $n = 0$ 的情况。

家庭作业 2.70 ▲▲ 为具有如下原型的函数编写代码：

```
/*
 * 当 x 能表示成一个 n 位补码数时返回 1;否则返回 0
 * 假设 1 <= n <= w
 */
int fits_bits(int x, int n);
```

你的函数应当遵循位级整数编码规则（见“位级整数编码规则”）。

家庭作业 2.71 ▲ 你刚开始在一家公司工作，这家公司要实现一组过程，处理一个把 4 个有符号字节打包进一个 32 位无符号数的数据结构。字中的字节从 0（最低有效）到 3（最高有效）编号。你分配到的任务，是为一台采用补码运算和算术右移的机器实现具有如下原型的函数：

```
/* 数据类型声明:把 4 个字节打包进
   一个 unsigned */
typedef unsigned packed_t;

/* 从字中提取字节。以有符号整数的形式返回 */
int xbyte(packed_t word, int bytenum);
```

也就是说，这个函数会提取指定的字节，并把它符号扩展为一个 32 位的 int。

你的前任（因为无能而被解雇了）写了下面这段代码：

```
/* xbyte 的失败尝试 */
int xbyte(packed_t word, int bytenum)
{
    return (word >> (bytenum << 3)) & 0xFF;
}
```

A. 这段代码有什么问题？

B. 给出该函数的一个正确实现，它只使用左移和右移，外加一次减法。

家庭作业 2.72 ▲▲ 你接到一项任务，要编写一个函数，把整数 val 复制到缓冲区 buf，但只有在缓冲区有足够可用空间时才这么做。下面是你写的代码：

```
/* 如果有空间,就把整数复制到缓冲区 */
/* 警告:下面的代码有缺陷 */
void copy_int(int val, void *buf, int maxbytes) {
    if (maxbytes-sizeof(val) >= 0)
            memcpy(buf, (void *) &val, sizeof(val));
}
```

这段代码使用了库函数 memcpy。虽然在这里用它有点牵强——我们只不过想复制一个 int 而已——但它说明了一种复制更大数据结构时常用的方法。

你仔细测试这段代码后发现，即使 maxbytes 太小，它也总是把值复制到缓冲区。

A. 解释为什么代码中的条件测试总是成功。提示：sizeof 运算符返回 size_t 类型的值。

B. 说明如何改写条件测试，使它能正确工作。

家庭作业 2.73 ▲▲ 为具有如下原型的函数编写代码：

```
/* 加法,饱和到 TMin 或 TMax */
int saturating_add(int x, int y);
```

与其像普通补码加法那样发生溢出，饱和加法（saturating addition）在会发生正溢出时返回 *TMax*，在会发生负溢出时返回 *TMin*。饱和运算在执行数字信号处理的程序中很常用。

你的函数应当遵循位级整数编码规则（见“位级整数编码规则”）。

家庭作业 2.74 ▲▲ 编写具有如下原型的函数：

```
/* 判断参数是否可以相减而不溢出 */
int tsub_ok(int x, int y);
```

如果计算 $x - y$ 不会溢出，这个函数就应当返回 1。

家庭作业 2.75 ▲▲▲ 假设我们想计算 $x \cdot y$ 的完整 $2w$ 位表示，其中 x 和 y 都是无符号的，而机器上 `unsigned` 数据类型是 w 位。乘积的低 w 位可以用表达式 `x*y` 算出，所以我们只需要一个具有如下原型的过程：

```
unsigned unsigned_high_prod(unsigned x, unsigned y);
```

它对无符号变量计算 $x \cdot y$ 的高 w 位。

我们有一个具有如下原型的库函数：

```
int signed_high_prod(int x, int y);
```

它对 x 和 y 为补码形式的情形计算 $x \cdot y$ 的高 w 位。请编写代码调用这个过程来实现无符号参数版本的函数，并论证你答案的正确性。

提示：看看公式 2.18 的推导中有符号乘积 $x \cdot y$ 与无符号乘积 $x' \cdot y'$ 之间的关系。

家庭作业 2.76 ▲ 库函数 `calloc` 具有如下声明：

```
void *calloc(size_t nmemb, size_t size);
```

按照库文档的说明，“`calloc` 函数为一个数组分配内存，数组有 `nmemb` 个元素，每个元素 `size` 字节。内存被置为 0。如果 `nmemb` 或 `size` 为零，则 `calloc` 返回 `NULL`。”

编写一个 `calloc` 的实现，它通过一次对 `malloc` 的调用完成分配，并用 `memset` 把内存清零。你的代码不应有任何因算术溢出造成的漏洞，而且无论用多少位来表示 `size_t` 类型的数据都应当能正确工作。

作为参考，函数 `malloc` 和 `memset` 具有如下声明：

```
void *malloc(size_t size);
void *memset(void *s, int c, size_t n);
```

家庭作业 2.77 ▲▲ 假设我们的任务是生成代码，把整型变量 x 乘以各种不同的常数因子 K。为了效率，我们想只用 +、- 和 « 这几种运算。对下面的每个 K 值，写出执行乘法的 C 表达式，每个表达式至多使用三次运算。

A. $K = 17$

B. $K = -7$

C. $K = 60$

D. $K = -112$

家庭作业 2.78 ▲▲ 为具有如下原型的函数编写代码：

```
/* 除以 2 的幂。假设 0 <= k < w-1 */
int divide_power2(int x, int k);
```

这个函数应当计算 $x/2^k$，并且要正确舍入，还应当遵循位级整数编码规则（见“位级整数编码规则”）。

家庭作业 2.79 ▲▲ 为函数 `mul3div4` 编写代码，对整数参数 x 计算出 $3*x/4$，但要遵循位级整数编码规则（见“位级整数编码规则”）。你的代码应当复现计算 $3*x$ 可能造成溢出这一事实。

家庭作业 2.80 ▲▲▲ 为函数 `threefourths` 编写代码，对整数参数 x 计算出 $\frac{3}{4}x$ 的值，向零舍入。它不应溢出。你的函数应当遵循位级整数编码规则（见“位级整数编码规则”）。

家庭作业 2.81 ▲▲ 编写 C 表达式，生成下列位模式，其中 a^k 表示符号 a 重复 k 次。假设是 w 位的数据类型。你的代码可以引用参数 j 和 k，它们分别表示 j 和 k 的值，但不能引用表示 w 的参数。

A. $1^{w-k}0^k$

B. $0^{w-k-j}1^k0^j$

家庭作业 2.82 ▲ 我们运行的程序中 `int` 类型的值是 32 位的。它们用补码表示，右移按算术右移执行。`unsigned` 类型的值也是 32 位。

我们生成任意的值 x 和 y，并把它们转换成无符号值，方法如下：

```
/* 创建一些任意值 */
int x = random();
int y = random();
/* 转换成 unsigned */
unsigned ux = (unsigned) x;
unsigned uy = (unsigned) y;
```

对下面每个 C 表达式，你要判断表达式是否总是产生 1。如果总是产生 1，请描述其背后的数学原理。否则，举出让它产生 0 的参数例子。

A. `(x<y) == (-x>-y)`

B. `((x+y)<<4) + y-x == 17*y+15*x`

C. `~x+~y+1 == ~(x+y)`

D. `(ux-uy) == -(unsigned)(y-x)`

E. `((x >> 2) << 2) <= x`

家庭作业 2.83 ▲▲ 考虑二进制表示是由形如 $0.yyyyyy\ldots$ 的无限串组成的数，其中 y 是一个 k 位的序列。例如，$\frac{1}{3}$ 的二进制表示是 $0.01010101\ldots(y=01)$，而 $\frac{1}{5}$ 的表示是 $0.001100110011\ldots(y=0011)$。

A. 令 $Y = \mathrm{B2U}_k(y)$，也就是二进制表示为 y 的那个数。给出一个用 Y 和 k 表示的公式，求出这个无限串所表示的值。提示：考虑把二进制小数点向右移 k 位的效果。

B. 对下面的 y 值，这个串的数值是多少？

- (a)`101`
- (b)`0110`
- (c)`010011`

家庭作业 2.84 ▲ 为下面的过程填写返回值。这个过程检测它的第一个参数是否小于等于第二个参数。假设函数 `f2u` 返回一个无符号 32 位数，其位表示与它的浮点参数相同。你可以假设两个参数都不是 NaN。两种零($+0$ 和 -0)被认为是相等的。

```
int float_le(float x, float y) {
    unsigned ux = f2u(x);
    unsigned uy = f2u(y);

    /* 取符号位 */
    unsigned sx = ux >> 31;
    unsigned sy = uy >> 31;

    /* 给出只使用 ux、uy、sx 和 sy 的表达式 */
    return        ;
}
```

家庭作业 2.85 ▲ 给定一种浮点格式，阶码为 k 位、小数为 n 位，为下列各个量写出阶码 E、尾数 M、小数 f 和值 V 的公式。另外，描述它的位表示。

A. 数 7.0

B. 能被精确表示的最大奇整数

C. 最小正规格化值的倒数

家庭作业 2.86 ▲ 与 Intel 兼容的处理器还支持一种“扩展精度”浮点格式，它是一个 80 位的字，划分为一个符号位、$k = 15$ 个阶码位、一个整数位(integer bit)和 $n = 63$ 个小数位。整数位是 IEEE 浮点表示中隐含位的显式副本。也就是说，对规格化值它等于 1，对非规格化值它等于 0。填写下面的表，给出这种格式中一些“有趣的”数的近似值：

把数据声明为 `long double` 类型，为 Intel 兼容机器编译的 C 程序就可以使用这种格式。不过，这会迫使编译器基于陈旧的 8087 浮点指令生成代码。得到的程序多半会比 `float` 或 `double` 类型的情况慢得多。

家庭作业 2.87 ▲ IEEE 浮点标准的 2008 年版本，名为 IEEE 754-2008，包含一种 16 位的“半精度”浮点格式。它最初是由计算机图形公司设计出来的，

家庭作业 2.86 扩展精度格式中一些"有趣的"数的近似值

描述	值	十进制
最小的正非规格化数		
最小的正规格化数		
最大的规格化数		

用于存储那些要求动态范围比 16 位整数所能达到的更高的数据。这种格式有 1 个符号位、5 个阶码位 ($k = 5$) 和 10 个小数位 ($n = 10$)。阶码偏置值为 $2^{5-1} - 1 = 15$。为给出的每个数填写随后的表,每一列的填法规定如下:

- Hex:描述编码形式的四位十六进制数字。
- M:尾数的值。它应当是一个形如 x 或 x/y 的数,其中 x 是整数,y 是 2 的整数次幂。例子有 0、$\frac{67}{64}$ 和 $\frac{1}{256}$。
- E:阶码的整数值。
- V:所表示的数值。使用 x 或 $x \times 2^z$ 的记法,其中 x 和 z 是整数。
- D:(可能是近似的)数值,就像用 `printf` 的`%f` 格式说明打印出来的那样。

作为一个例子,要表示数 $\frac{7}{8}$,我们会取 $s = 0$、$M = \frac{7}{4}$、$E = -1$。于是这个数的阶码域就是 01110_2(十进制值 $15 - 1 = 14$),尾数域是 1100000000_2,十六进制表示为 `3B00`。数值为 0.875。

标有—的条目不必填写。

家庭作业 2.87 IEEE 754-2008 半精度格式中给定各数的属性

描述	Hex	M	E	V	D
-0				-0	-0.0
比 2 大的最小值					
512				512	512.0
最大的非规格化数					
$-\infty$		—	—	$-\infty$	$-\infty$
十六进制表示为 `3BB0` 的数	`3BB0`				

家庭作业 2.88 ▲▲ 考虑下面两种基于 IEEE 浮点格式的 9 位浮点表示。

1. **格式 A**
   - 有 1 个符号位。
   - 有 $k = 5$ 个阶码位。阶码偏置值为 15。
   - 有 $n = 3$ 个小数位。
2. **格式 B**

- 有 1 个符号位。
- 有 $k = 4$ 个阶码位。阶码偏置值为 7。
- 有 $n = 4$ 个小数位。

下面的表中给出了一些格式 A 的位模式，你的任务是把它们转换成格式 B 中最接近的值。如果需要舍入，应当**向 $+\infty$ 舍入**。另外，要给出格式 A 和格式 B 的位模式所表示的数值。这些值既可以用整数（例如 17）表示，也可以用分数（例如 17/64 或 $17/2^6$）表示。

家庭作业 2.88　把格式 A 的位模式转换成最接近的格式 B 值

格式 A 位模式	格式 A 值	格式 B 位模式	格式 B 值
1 01111 001	$-\frac{9}{8}$	1 0111 0010	$-\frac{9}{8}$
0 10110 011			
1 00111 010			
0 00000 111			
1 11100 000			
0 10111 100			

家庭作业 2.89 ▲　我们在这样一台机器上运行程序：`int` 类型的值是 32 位补码表示，`float` 类型使用 32 位 IEEE 格式，`double` 类型使用 64 位 IEEE 格式。

我们生成任意的整数值 x、y 和 z，并把它们转换成 `double` 类型的值，方法如下：

```
/* 创建一些任意值 */
int x = random();
int y = random();
int z = random();
/* 转换成 double */
double   dx = (double) x;
double   dy = (double) y;
double   dz = (double) z;
```

对下面每个 C 表达式，你要判断表达式是否总是产生 1。如果总是产生 1，请描述其背后的数学原理。否则，举出让它产生 0 的参数例子。注意，不能用运行 gcc 的 IA32 机器来检验你的答案，因为它对 `float` 和 `double` 都会用 80 位扩展精度表示。

A. `(float) x == (float) dx`

B. `dx - dy == (double) (x-y)`

C. `(dx + dy) + dz == dx + (dy + dz)`

D. `(dx * dy) * dz == dx * (dy * dz)`

E. `dx / dx == dz / dz`

家庭作业 2.90 ▲ 要求你编写一个 C 函数来计算 2^x 的浮点表示。你决定最好的办法是直接构造结果的 IEEE 单精度表示。当 x 太小时，你的程序返回 0.0；当 x 太大时，返回 $+\infty$。填写后面代码中空出的部分，使之计算出正确的结果。假设函数 `u2f` 返回一个浮点值，其位表示与它的无符号参数相同。

```
float fpwr2(int x)
{
    /* 结果的阶码和小数域 */
    unsigned exp, frac;
    unsigned u;

    if (x <          ) {
        /* 太小。返回 0.0 */
        exp =          ;
        frac =          ;
    } else if (x <          ) {
        /* 非规格化结果 */
        exp =          ;
        frac =          ;
    } else if (x <          ) {
        /* 规格化结果。 */
        exp =          ;
        frac =          ;
    } else {
        /* 太大。返回 +oo */
        exp =          ;
        frac =          ;
    }

    /* 把 exp 和 frac 打包成 32 位 */
    u = exp << 23 | frac;
    /* 作为 float 返回 */
    return u2f(u);
}
```

家庭作业 2.91 ▲ 大约在公元前 250 年，希腊数学家阿基米德(Archimedes)证明了 $\frac{223}{71} < \pi < \frac{22}{7}$。如果他那时能用上计算机和标准库 `<math.h>`，他就能确定 π 的单精度浮点近似值的十六进制表示是 `0x40490FDB`。当然，这些都只是近似值，因为 π 不是有理数。

A. 这个浮点值所表示的二进制小数是多少?

B. $\frac{22}{7}$ 的二进制小数表示是什么? 提示:参见问题 2.83。

C. π 的这两个近似值从(相对于二进制小数点的)哪一位开始不同?

2.7.2 位级浮点数编码规则

在下面的问题中，你要编写代码来实现浮点函数，直接对浮点数的位级表示进行操作。你的代码应当完全复现 IEEE 浮点运算的各项约定，包括在需要舍入时采用向偶数舍入模式。

为此，我们定义数据类型 `float_bits`，它等价于 `unsigned`：

```
/* 访问浮点数的位级表示 */
typedef unsigned float_bits;
```

在代码中你将用 `float_bits`，而非数据类型 `float`。你可以使用 `int` 和 `unsigned` 两种数据类型，包括它们的无符号常量、整型常量和相关运算。不能使用任何联合(union)、结构(struct)或数组。最重要的是，不能使用任何浮点数据类型、运算或常量。你的代码应当通过位操作来实现指定的浮点运算。

下面的函数说明了这些编码规则的用法。对参数 f，如果它是非规格化的(保留 f 的符号)，它返回 ± 0；否则返回 f。

```
/* 如果 f 是非规格化数，返回 0；否则返回 f */
float_bits float_denorm_zero(float_bits f) {
    /* 把位表示分解成几个部分 */
    unsigned sign = f>>31;
    unsigned exp = f>>23 & 0xFF;
    unsigned frac = f      & 0x7FFFFF;
    if (exp == 0) {
        /* 非规格化。把小数域置为 0 */
        frac = 0;
    }
    /* 重新拼装位 */
    return (sign << 31) | (exp << 23) | frac;
}
```

家庭作业 2.92 ▲▲ 按照位级浮点数编码规则，实现具有如下原型的函数：

```
/* 计算 -f。如果 f 是 NaN，则返回 f。 */
float_bits float_negate(float_bits f);
```

对浮点数 f，这个函数计算 $-f$。如果 f 是 NaN，你的函数应当直接返回 f。

通过对参数 f 的全部 2^{32} 个值求值来测试你的函数，并把结果与用机器的浮点运算所得到的结果进行比较。

家庭作业 2.93 ▲▲ 按照位级浮点数编码规则，实现具有如下原型的函数：

```
/* 计算 |f|。如果 f 是 NaN，则返回 f。 */
float_bits float_absval(float_bits f);
```

对浮点数 f，这个函数计算 $|f|$。如果 f 是 NaN，你的函数应当直接返回 f。

通过对参数 f 的全部 2^{32} 个值求值来测试你的函数，并把结果与用机器的浮点运算所得到的结果进行比较。

家庭作业 2.94 ▲▲▲ 按照位级浮点数编码规则，实现具有如下原型的函数：

```
/* 计算 2*f。如果 f 是 NaN,则返回 f。 */
float_bits float_twice(float_bits f);
```

对浮点数 f，这个函数计算 $2.0 \cdot f$。如果 f 是 NaN，你的函数应当直接返回 f。

通过对参数 f 的全部 2^{32} 个值求值来测试你的函数，并把结果与用机器的浮点运算所得到的结果进行比较。

家庭作业 2.95 ▲▲▲ 按照位级浮点数编码规则，实现具有如下原型的函数：

```
/* 计算 0.5*f。如果 f 是 NaN,则返回 f。 */
float_bits float_half(float_bits f);
```

对浮点数 f，这个函数计算 $0.5 \cdot f$。如果 f 是 NaN，你的函数应当直接返回 f。

通过对参数 f 的全部 2^{32} 个值求值来测试你的函数，并把结果与用机器的浮点运算所得到的结果进行比较。

家庭作业 2.96 ▲▲▲▲ 按照位级浮点数编码规则，实现具有如下原型的函数：

```
/*
 * 计算 (int) f。
 * 如果转换造成溢出或 f 是 NaN,返回 0x80000000
 */
int float_f2i(float_bits f);
```

对浮点数 f，该函数计算 `(int)` f。你的函数应当向零舍入。如果 f 不能表示成一个整数（例如它超出取值范围，或者它是 NaN），那么该函数应当返回 `0x80000000`。

通过对全部 2^{32} 个可能的参数 f 的取值逐一求值，并把结果与用你的机器上的浮点运算得到的结果相比较，以此测试你的函数。

练习 2.97 按照位级浮点编码规则，实现具有如下原型的函数：

```
/* 计算 (float) i */
float_bits float_i2f(int i);
```

对参数 i，该函数计算 `(float)` i 的位级表示。

通过对全部 2^{32} 个可能的参数 f 的取值逐一求值，并把结果与用你的机器上的浮点运算得到的结果相比较，以此测试你的函数。

2.8 练习答案

答案 2.1 一旦开始考察机器级程序，理解十六进制格式与二进制格式之间的关系就变得十分重要。做这些转换的方法正文中已经给出，但要熟练掌握还需要一点练习。

A. 把 `0x25B9D2` 转换成二进制：

十六进制	2	5	B	9	D	2
二进制	0010	0101	1101	1001	1101	0010

B. 把二进制数 `1100 1001 0111 1011` 转换成十六进制：

二进制	1100	1001	0111	1011
十六进制	C	9	7	B

C. 把 `0xA8B3D` 转换成二进制：

十六进制	A	8	B	3	D
二进制	1010	1000	1011	0011	1101

D. 把二进制数 `11 0010 0010 1101 1001 0110` 转换成十六进制：

二进制	11	0010	0010	1101	1001	0110
十六进制	3	2	2	D	9	6

答案 2.2 这个问题给你一个思考 2 的幂及其十六进制表示的机会。

n	2^n（十进制）	2^n（十六进制）
5	32	`0x20`
23	8,388,608	`0x800000`
15	32,768	`0x8000`
13	8,192	`0x2000`
12	4,096	`0x1000`
6	64	`0x40`
8	256	`0x100`

答案 2.3 这个问题让你尝试在较小数的十六进制表示和十进制表示之间进行转换。对于较大的数，使用一个计算器或转换程序会方便可靠得多。

十进制	二进制	十六进制
0	0000 0000	`0x00`
$158 = 16 \cdot 9 + 14$	1001 1110	`0x9E`
$76 = 16 \cdot 4 + 12$	0100 1100	`0x4C`
$145 = 16 \cdot 9 + 1$	1001 0001	`0x91`
$16 \cdot 10 + 14 = 174$	1010 1110	`0xAE`
$16 \cdot 3 + 12 = 60$	0011 1100	`0x3C`
$16 \cdot 15 + 1 = 241$	1111 0001	`0xF1`
$16 \cdot 7 + 5 = 117$	0111 0101	`0x75`
$16 \cdot 11 + 13 = 189$	1011 1101	`0xBD`
$16 \cdot 15 + 5 = 245$	1111 0101	`0xF5`

答案 2.4 当你开始调试机器级程序时，会发现许多场合做一些简单的十六进制算术很有用。你固然可以把数转换成十进制、执行算术运算、再转换回去，但能够直接用十六进制来工作会更有效率，也更能说明问题。

A. `0x605C + 0x5 = 0x6061`。给十六进制的 C 加上 5 得到 1，并向前进位 1。

B. `0x605C - 0x20 = 0x603C`。在第二位上把 2 从 5 中减去，不需要向第三位借位，这就给出 3。

C. `0x605C + 32 = 0x607C`。十进制的 $32(2^5)$ 等于十六进制的 `0x20`。

D. `0x60FA - 0x605C = 0x9E`。为了从十六进制的 A（十进制 10）中减去 C（十进制 12），我们向第二位借 16，得到十六进制的 F（十进制 15）。在第二位上，我们再从十六进制的 E（十进制 14）中减去 5，得到十进制 9。

答案 2.5 这个问题检验你对数据的字节表示以及两种不同字节顺序的理解。

A. 小端法：`78` 大端法：`12` **B.** 小端法：`78 56` 大端法：`12 34`

C. 小端法：`78 56 34` 大端法：`12 34 56`

请记住，`show_bytes` 从地址最低的字节开始、朝着地址最高的字节的方向逐一枚举这些字节。在小端法机器上，它按从最低有效字节到最高有效字节的顺序列出字节；在大端法机器上，它按从最高有效字节到最低有效字节的顺序列出字节。

答案 2.6 这个问题再次给你练习十六进制到二进制转换的机会，同时让你思考整数表示和浮点数表示。我们将在本章后面更详细地考察这两种表示。

A. 按照正文中示例所用的记法，我们把这两个位串写成如下形式：

```
  0   0   2   7   C   8   F   8
00000000001001111100100011111000
         **********************
  4   A   1   F   2   3   E   0
01001010000111110010001111100000
```

B. 把第二个字相对第一个字向右移动两个位置之后，我们找到一个有 21 位匹配的序列。

C. 我们发现，浮点数中嵌入了整数的所有位，只有值为 0 的最高有效位除外。正文中的示例也是这样。此外，浮点数还有一些非零的高位与整数的对应位不匹配。

答案 2.7 它打印出 `6D 6E 6F 70 71 72`。还要记住，库例程 `strlen` 不把结尾的空字符计入其中，所以 `show_bytes` 只打印到字符 `r`。

答案 2.8 这个问题是一个帮助你更加熟悉布尔运算的小练习。

运算	结果
a	[01001110]
b	[11100001]
~a	[10110001]
~b	[00011110]
a & b	[01000000]
a | b	[11101111]
a ^ b	[10101111]

答案 2.9 这个问题展示了如何用布尔代数来描述现实世界中的系统并对它进行推理。可以看出,这种颜色代数与长度为 3 的位向量上的布尔代数完全相同。

A. 对一种颜色求补,就是把 R、G、B 的值分别求补。由此可以看出:白色是黑色的补色,黄色是蓝色的补色,品红色是绿色的补色,青色是红色的补色。

B. 我们基于颜色的位向量表示来执行布尔运算,得到下列结果:

```
蓝(001) | 绿(010)   = 青(011)
黄(110) & 青(011)   = 绿(010)
红(100) ^ 品红(101) = 蓝(001)
```

答案 2.10 这个过程依赖于如下事实:异或满足交换律和结合律,并且对任意 a 都有 `a ^ a` $= 0$。

步骤	*x	*y
初始时	a	b
第 1 步	a	a ^ b
第 2 步	a ^ (a ^ b) = (a ^ a) ^ b = b	a ^ b
第 3 步	b	b ^ (a ^ b) = (b ^ b) ^ a = a

练习 2.11 给出了一个该函数将会失败的情形。

答案 2.11 这个问题说明我们的 `inplace_swap` 例程有一个微妙而有趣的特性。

A. `first` 和 `last` 的值都是 k,因此我们是要把中间这个元素与它自身交换。

B. 在这种情况下,传给 `inplace_swap` 的参数 `x` 和 `y` 都指向同一个位置。计算 `*x ^ *y` 会得到 0。于是我们把 0 存进数组的中间位置,而后面的步骤又一直把这个元素设置为 0。可见,我们在练习 2.10 中的推理隐含地假设了 `x` 和 `y` 指向不同的位置。

C. 只需把 `reverse_array` 第 4 行中的测试改为 `first < last`,因为中间这个元素没有必要与它自身交换。

答案 2.12 下面是这些表达式:

A. `x & 0xFF`

B. `x ~0xFF`

C. `x | 0xFF`

这些表达式是执行低位位级运算时常见的一类写法。表达式 `~0xFF` 生成的掩码使 8 个最低有效位等于 0，其余所有位等于 1。注意，无论字长是多少，这样的掩码都会生成。相比之下，表达式 `0xFFFFFF00` 只有在数据类型 `int` 为 32 位时才有效。

答案 2.13 这些练习帮助你思考布尔运算与程序员应用掩码运算的典型方式之间的关系。下面是代码：

```
/* 声明实现 bis 和 bic 运算的函数 */
int bis(int x, int m);
int bic(int x, int m);

/* 只通过调用函数 bis 和 bic 来计算 x|y */
int bool_or(int x, int y) {
  int result = bis(x,y);
  return result;
}

/* 只通过调用函数 bis 和 bic 来计算 x^y */
int bool_xor(int x, int y) {
  int result = bis(bic(x,y), bic(y,x));
  return result;
}
```

`bis` 运算等价于布尔 `or`—只要某一位在 `x` 中为 1 或者在 `m` 中为 1，`z` 中的这一位就会被置为 1。另一方面，`bic(x, m)` 等价于 `x & ~m`：我们希望结果只在 `x` 的对应位为 1 而 `m` 的对应位为 0 时才等于 1。

已知这些，我们用一次对 `bis` 的调用就能实现 `|`。要实现 `^`，我们利用性质

```
x ^ y = (x & ~y) | (~x & y)
```

答案 2.14 这个问题凸显了位级布尔运算与 C 语言逻辑运算之间的关系。一个常见的编程错误是，本想用逻辑运算却用了位级运算，或者反过来。

表达式	值	表达式	值
`a & b`	`0x44`	`a && b`	`0x01`
`a | b`	`0x57`	`a || b`	`0x01`
`~a | ~b`	`0xBB`	`!a || !b`	`0x00`
`a & !b`	`0x00`	`a && ~b`	`0x01`

答案 2.15 这个表达式是`!(x ^ y)`。

也就是说，`x^y` 当且仅当 x 的每一位都与 y 的对应位相同时才为零。然后我们利用 `!` 判断一个字是否含有任何非零位的能力。

其实没有什么真正的理由要用这个表达式而不是简单地写 `x == y`，不过它确实展示了位级运算和逻辑运算的某些奥妙。

答案 2.16 这个问题是一个帮助你理解不同移位运算的小练习。

x		$a \ll 2$		逻辑 $a \gg 3$		算术 $a \gg 3$	
十六进制	二进制	二进制	十六进制	二进制	十六进制	二进制	十六进制
0xD4	[11010100]	[01010000]	0x50	[00011010]	0x1A	[11111010]	0xFA
0x64	[01100100]	[10010000]	0x90	[00001100]	0x0C	[11101100]	0xEC
0x72	[01110010]	[11001000]	0xC8	[00001110]	0x0E	[00001110]	0x0E
0x44	[01000100]	[00010000]	0x10	[00001000]	0x08	[11101000]	0xE9

答案 2.17 一般而言，对非常小的字长演算一些例子，是理解计算机算术的好方法。

无符号值与图 2.2 中的那些相同。对补码值而言，十六进制数字 0 到 7 的最高有效位为 0，产生非负值；而十六进制数字 8 到 F 的最高有效位为 1，产生一个负值。

$\vec{x}$		$B2U_4(\vec{x})$	$B2T_4(\vec{x})$
十六进制	二进制		
0xA	[1010]	$2^3 + 2^1 = 10$	$-2^3 + 2^2 = -6$
0x1	[0001]	$2^0 = 1$	$2^0 = 1$
0xB	[1011]	$2^3 + 2^1 + 2^0 = 11$	$-2^3 + 2^1 + 2^0 = -5$
0x2	[0010]	$2^1 = 2$	$2^1 = 2$
0x7	[0111]	$2^2 + 2^1 + 2^0 = 7$	$2^2 + 2^1 + 2^0 = 7$
0xC	[1100]	$2^3 + 2^2 = 12$	$-2^3 + 2^2 = -4$

答案 2.18 对一个 32 位字来说，任何由 8 个十六进制数字组成、第一个数字是数字 8 到 f 之一的值都表示一个负数。以一串 f 开头的数很常见，因为负数的前导位全是 1。不过你必须仔细地看。例如，数 `0x8048337` 只有 7 位数字。用前导零补足之后得到 `0x08048337`，它是一个正数。

```
4004d0:  48 81 ec e0 02 00 00     sub    $0x2e0,%rsp             A.
 ↪  736
4004d7:  48 8b 44 24 a8           mov    -0x58(%rsp),%rax        B.
 ↪  -88
4004dc:  48 03 47 28              add    0x28(%rdi),%rax         C. 40
4004e0:  48 89 44 24 d0           mov    %rax,-0x30(%rsp)        D.
 ↪  -48
4004e5:  48 8b 44 24 78           mov    0x78(%rsp),%rax         E.
 ↪  120
4004ea:  48 89 87 88 00 00 00     mov    %rax,0x88(%rdi)         F.
 ↪  136
4004f1:  48 8b 84 24 f8 01 00     mov    0x1f8(%rsp),%rax        G.
 ↪  504
4004f8:  00
4004f9:  48 03 44 24 08           add    0x8(%rsp),%rax
4004fe:  48 89 84 24 c0 00 00     mov    %rax,0xc0(%rsp)         H.
 ↪  192
400505:  00
```

```
400506:  48 8b 44 d4 b8          mov    -0x48(%rsp,%rdx,8),%rax  I.
↪  -72
```

答案 2.19 $T2U$ 和 $U2T$ 这两个函数从数学的角度来看非常特别。理解它们的行为很重要。

我们解决这个问题的办法是，把练习 2.17 答案中的各行按其补码值重新排序，然后列出无符号值作为函数运算的结果。我们同时给出十六进制值，使这个过程更加具体。

$\vec{x}$（十六进制）	x	$T2U_4(\vec{x})$
0xF	-1	15
0xB	-5	11
0xA	-6	10
0xC	-4	12
0x1	1	1
0x8	8	8

答案 2.20 这个练习检验你对公式 2.5 的理解。

对前四项来说，x 的值为负，且 $T2U_4(x) = x + 2^4$。对其余两项来说，x 的值为非负，且 $T2U_4(x) = x$。

答案 2.21 这个问题帮助你巩固对补码表示和无符号表示之间关系的理解，以及 C 语言类型提升规则的影响。回想一下，$TMin_{32}$ 是 $-2{,}147{,}483{,}648$，而把它强制转换成无符号数之后就变成 $2{,}147{,}483{,}648$。另外，只要有一个操作数是无符号的，另一个操作数在比较之前也会被强制转换成无符号数。

表达式	类型	求值
`-2147483647-1 == 2147483648U`	无符号	1
`-2147483647-1 < 2147483647`	有符号	1
`-2147483647-1U < 2147483647`	无符号	0
`-2147483647-1 < -2147483647`	有符号	1
`-2147483647-1U < -2147483647`	无符号	1

答案 2.22 这个练习具体演示了符号扩展如何保持补码表示的数值。

A. $[1100]$ $-2^3 + 2^2 = -8 + 4 = -4$

B. $[11100]$ $-2^4 + 2^3 + 2^2 = -16 + 8 + 4 = -4$

C. $[111100]$ $-2^5 + 2^4 + 2^3 + 2^2 = -32 + 16 + 8 + 4 = -4$

答案 2.23 这些函数中的表达式都是从打包了多个位域的字中提取值的常见程序惯用法。它们利用了不同移位运算的补零性质和符号扩展性质。请仔细注意强制类型转换与移位运算的先后顺序：在 `fun1` 中，移位是对无符号变量 `word` 执行的，因此是逻辑移位；在 `fun2` 中，移位是在把 `word` 强制转换成 `int` 之后执行的，因此是算术移位。

w	fun1(w)	fun2(w)
0x00000076	0x00000076	0x00000076
0x87654321	0x00000021	0x00000021
0x000000C9	0x000000C9	0xFFFFFFC9
0xEDCBA987	0x00000087	0xFFFFFF87

B. 函数 fun1 从参数的低 8 位中提取一个值，得到一个介于 0 到 255 之间的整数。函数 fun2 也从参数的低 8 位中提取一个值，但是它还执行了符号扩展，因此结果将是介于 -128 到 127 之间的一个数。

答案 2.24 对无符号数来说，截断的效果相当直观，但对补码数来说则不然。这个练习让你用非常小的字长来考察它的性质。

十六进制		无符号		补码	
原值	截断后	原值	截断后	原值	截断后
1	1	1	1	1	1
3	3	3	3	3	3
5	5	5	5	5	5
C	4	12	4	-4	4
E	6	14	6	-2	6

如公式 2.9 所述，对无符号值而言，这种截断的效果就是简单地求出它们对 8 的余数。截断对有符号值的影响要复杂一些。根据公式 2.10，我们先求出参数对 8 求余的余数。对参数 0 到 7，以及对参数 -8 到 -1，这都会给出 0 到 7 的值。然后我们对这些余数应用函数 $U2T_3$，得到 0 到 3 和 -4 到 -1 这两个序列各重复一遍。

答案 2.25 这个问题的设计意图是展示由有符号到无符号的隐式转换是多么容易引起缺陷。把参数 length 作为无符号数传递看起来很自然，因为没有人会想用负的长度。停止条件 i <= length-1 看起来也很自然。但是把这两者结合起来会产生意想不到的结果！

由于参数 length 是无符号数，计算 $0-1$ 用的是无符号算术，它等价于模运算加法。结果就变成 $UMax$。<= 比较也是用无符号比较执行的，而且任何数都小于等于 $UMax$，所以这个比较永远成立！于是，这段代码会试图访问数组 a 的无效元素。

把 length 声明为 int，或者把 for 循环的测试改为 i < length，这段代码都可以修复。

答案 2.26 这个例子展示了无符号算术的一个微妙特性，也展示了我们有时会在不知不觉中执行无符号算术的性质。这可能引起非常棘手的缺陷。

A. 该函数会在什么情况下产生不正确的结果？ 当 s 短于 t 时，函数将错误地返回 1。

B. 解释这种不正确的结果是如何产生的。 由于 `strlen` 被定义为返回无符号值，差值和比较都是用无符号算术来计算的。当 s 比 t 短时，差值 `strlen(s) - strlen(t)` 本应为负数，却变成了一个很大的无符号数，它大于 0。

C. 说明如何修改代码使它能可靠地工作。 把测试换成下面的语句：

```
return strlen(s) > strlen(t);
```

答案 2.27 这个函数是判断无符号加法是否溢出的规则的直接实现。

```
/* 判断参数相加是否不会产生溢出 */
int uadd_ok(unsigned x, unsigned y) {
    unsigned sum = x+y;
    return sum >= x;
}
```

答案 2.28 这个问题是对模 16 算术的一个简单演示。最简单的解法是先把十六进制模式转换成它的无符号十进制值。对 x 的非零值，必定有 $(-_4^u x) + x = 16$。然后我们把这个取补之后的值转换回十六进制。

x		$-_4^u x$	
十六进制	十进制	十进制	十六进制
1	1	15	F
4	4	12	C
7	7	9	9
A	10	6	6
E	14	2	2

答案 2.29 这个练习用来确认你理解补码加法。

x	y	$x+y$	$x +_5^t y$	情形
−12	−15	−27	5	1
[10100]	[10001]	[100101]	[00101]	
−8	−8	−16	−16	2
[11000]	[11000]	[110000]	[10000]	
−9	8	−1	−1	2
[10111]	[01000]	[111111]	[11111]	
2	5	7	7	3
[00010]	[00101]	[000111]	[00111]	
12	4	16	−16	4
[01100]	[00100]	[010000]	[10000]	

答案 2.30 这个函数是判断补码加法是否溢出的规则的直接实现。

```
/* 判断参数相加是否不会产生溢出 */
int tadd_ok(int x, int y) {
    int sum = x+y;
```

```
    int neg_over = x < 0 && y < 0 && sum >= 0;
    int pos_over = x >= 0 && y >= 0 && sum < 0;
    return !neg_over && !pos_over;
}
```

答案 2.31 你的同事本可以通过学习 2.3.2 节认识到，补码加法构成一个阿贝尔群。因此，无论加法是否溢出，表达式 `(x+y)-x` 都会求值为 `y`，而 `(x+y)-y` 总会求值为 `x`。

答案 2.32 这个函数给出的值是正确的，但当 y 为 $TMin$ 时除外。这时 $-y$ 也等于 $TMin$，于是调用 `tadd_ok` 函数会在 x 为负时指示溢出，而在 x 非负时不指示。事实恰恰相反：`tsub_ok(x, TMin)` 在 x 为负时应当产生 0，在 x 非负时应当产生 1。

这道练习能学到的一个教训是：在为函数编写任何测试过程时，都应当把 $TMin$ 作为一种情形包括在测试用例中。

答案 2.33 这个问题用一个很小的字长帮助你理解补码取负。

对 $w = 4$，有 $TMin_4 = -8$。因此 -8 是它自己的加法逆元，而其他值都用整数取负来求负。

x		$-_4^t x$	
十六进制	十进制	十进制	十六进制
2	2	−2	E
3	3	−3	D
9	−9	−9	7
B	−5	5	5
C	−4	4	4

这些位模式与无符号取负的位模式是相同的。

答案 2.34 这个练习用来确认你理解补码乘法。

模式	x		y		$x \cdot y$		截断后的 $x \cdot y$	
	值	位串	值	位串	值	位串	值	位串
无符号	4	[100]	5	[101]	20	[010100]	4	[100]
补码	−4	[100]	−3	[101]	12	[001100]	−4	[100]
无符号	2	[010]	7	[111]	14	[001110]	6	[110]
补码	2	[010]	−1	[111]	−2	[111110]	−2	[110]
无符号	6	[110]	6	[110]	36	[100100]	4	[100]
补码	−2	[110]	−2	[110]	4	[000100]	−4	[100]

答案 2.35 对 x 和 y 的所有可能取值来测试这个函数是不现实的。即使每秒能运行 100 亿次测试，当数据类型 `int` 为 32 位时，测完所有组合也要花

58 年以上。另一方面,把函数写成数据类型为 short 或 char 的版本再彻底地测试,则是可行的。

下面是一种更有原则的做法,按照如下的论证:

1. 我们知道 $x \cdot y$ 可以写成一个 $2w$ 位的补码数。设 u 表示由低 w 位所表示的无符号数,v 表示由高 w 位所表示的补码数。那么,根据公式 2.3,可以看出 $x \cdot y = v2^w + u$。

   我们还知道 $u = T2U_w(p)$,因为它们是由同一个位模式产生的无符号数和补码数,于是根据公式 2.6,可以写成 $u = p + p_{w-1}2^w$,其中 p_{w-1} 是 p 的最高有效位。令 $t = v + p_{w-1}$,我们有 $x \cdot y = p + t2^w$。

   当 $t = 0$ 时,$x \cdot y = p$,乘法不发生溢出;当 $t \neq 0$ 时,$x \cdot y \neq p$,乘法发生溢出。

2. 根据整数除法的定义,用非零的 x 去除 p 得到商 q 和余数 r,满足 $p = x \cdot q + r$,且 $|r| < |x|$。(这里使用绝对值,因为 x 和 r 的符号可能不同。例如,-7 除以 2 得到商 -3 余 -1。)

3. 假设 $q = y$。那么 $x \cdot y = x \cdot y + r + t2^w$。由此可见 $r + t2^w = 0$。但 $|r| < |x| \leq 2^w$,所以这个等式只有当 $t = 0$ 时才能成立,此时 $r = 0$。

   假设 $r = t = 0$。那么我们将有 $x \cdot y = x \cdot q$,这意味着 $y = q$。

当 x 等于 0 时,乘法不会溢出,因此我们看到,这段代码提供了一种可靠的方法来测试补码乘法是否引起溢出。

答案 2.36 有了 64 位,我们就可以在不溢出的情况下完成这个乘法。然后我们测试把乘积强制转换成 32 位是否会改变它的值:

```
1  /* 判断两个参数相乘时
2     是否不会发生溢出 */
3  int tmult_ok(int x, int y) {
4      /* 在不溢出的情况下计算乘积 */
5      int64_t pll = (int64_t) x*y;
6      /* 检查强制转换成 int 后值是否保持不变 */
7      return pll == (int) pll;
8  }
```

注意,第 5 行右端的强制类型转换至关重要。如果把这个语句写成

```
  int64_t pll = x*y;
```

那么乘积将按 32 位值来计算(可能会溢出),然后再符号扩展为 64 位。

答案 2.37 **A.** 这个改动没有丝毫帮助。尽管 asize 的计算结果是准确的,但对 malloc 的调用仍会把这个值转换成一个 32 位无符号数,因此同样会出现溢出的条件。

B. malloc 的参数是一个 32 位无符号数，它不可能分配超过 2^{32} 字节的块，因此试图分配或复制这么大的内存毫无意义。函数应当转而中止并返回 NULL。下面的代码将替换原来第 9 行对 malloc 的调用：

```
uint64_t required_size = ele_cnt * (uint64_t) ele_size;
size_t request_size = (size_t) required_size;
if (required_size != request_size)
    /* 必然发生了溢出,中止操作 */
    return NULL;
void *result = malloc(request_size);
if (result == NULL)
    /* malloc 失败 */
    return NULL;
```

答案 2.38 在第 3 章中，我们会看到许多 lea 指令发挥作用的例子。提供这条指令是为了支持指针算术运算，但 C 编译器常常把它用作乘以小常数的方法。

对每个 k 值，我们可以计算两个倍数：2^k（当 b 为 0 时）和 $2^k + 1$（当 b 为 a 时）。这样，我们就能计算 1、2、3、4、5、8 和 9 这些倍数。

答案 2.39 这个表达式就简单地变成 -(x<<m)。要看清这一点，设字长为 w，于是 $n = w - 1$。形式 B 规定我们应该计算 (x<<w) - (x<<m)，但是把 x 向左移动 w 位会得到值 0。

答案 2.40 这个问题要求你练习已经介绍的优化方法，同时也要求你发挥一点自己的独创性。

K	移位	加/减	表达式
7	1	1	(x<<3) - x
30	4	3	(x<<4) + (x<<3) + (x<<2) + (x<<1)
28	2	1	(x<<5) - (x<<2)
55	2	2	(x<<6) - (x<<3) - x

注意，第四种情形使用了形式 B 的一个修改版本。我们可以把位模式 [11011] 看作中间夹着一个 0 的连续 6 个 1，于是先应用形式 B 的规则，再减去与中间那个 0 位对应的项。

答案 2.41 假设加法和减法具有同样的性能，规则是：当 $n = m$ 时选择形式 A，当 $n = m + 1$ 时两种形式任选其一，当 $n > m + 1$ 时选择形式 B。

这一规则的论证如下。先假设 $m > 0$。当 $n = m$ 时，形式 A 只需要一次移位，而形式 B 需要两次移位和一次减法。当 $n = m + 1$ 时，两种形式都需要两次移位和一次加法或减法。当 $n > m + 1$ 时，形式 B 只需要两次移位和一次减法，而形式 A 需要 $n - m + 1 > 2$ 次移位和 $n - m > 1$ 次加法。对于 $m = 0$ 的情形，形式 A 和形式 B 都少一次移位，因此相同的选择规则仍然适用。

答案 2.42 这里唯一的挑战是,不用任何测试或条件运算就算出偏置。我们使用的技巧是:如果 x 为负,表达式 `x >> 31` 会生成一个全 1 的字,否则生成全 0。屏蔽掉适当的位之后,就得到想要的偏置值。

```
int div16(int x) {
    /* 计算偏置:x >= 0 时为 0,x < 0 时为 15 */
    int bias = (x >> 31) & 0xF;
    return (x + bias) >> 4;
}
```

答案 2.43 我们发现,当直接面对汇编代码做这道题时,人们会觉得有困难。把它写成 `optarith` 中所示的形式就清楚了。

可以看出 M 是 31:`x*M` 是按 `(x<<5)-x` 来计算的。

可以看出 N 是 8:当 y 为负时加上一个 7 的偏置值,而且右移了 3 位。

答案 2.44 这些"C 谜题"提供了清晰的例证,说明程序员必须理解计算机算术的性质:

A. `(x > 0) || (x-1 < 0)` **假**。令 x 为 $-2{,}147{,}483{,}648(TMin_{32})$,那么 `x-1` 将等于 $2{,}147{,}483{,}647(TMax_{32})$。

B. `(x & 7) != 7 || (x<<29 < 0)` **真**。如果 `(x & 7) != 7` 求值为 0,那么位 x_2 必定为 1。当左移 29 位之后,这一位将成为符号位。

C. `(x * x) >= 0` **假**。当 x 为 65,535 (`0xFFFF`) 时, `x*x` 为 $-131{,}071$ (`0xFFFE0001`)。

D. `x < 0 || -x <= 0` **真**。如果 x 非负,那么 `-x` 非正。

E. `x > 0 || -x >= 0` **假**。令 x 为 $-2{,}147{,}483{,}648(TMin_{32})$,那么 `x` 和 `-x` 都是负的。

F. `x+y == uy+ux` **真**。补码加法和无符号加法在位级的行为相同,而且它们满足交换律。

G. `x*~y + uy*ux == -x` **真**。`~y` 等于 `-y-1`。`uy*ux` 等于 `x*y`。于是,左端等价于 `x*-y-x+x*y`。

答案 2.45 理解小数二进制表示是理解浮点编码的重要一步。这个练习让你尝试一些简单的例子。

$\frac{1}{8}$	0.001	0.125
$\frac{3}{4}$	0.11	0.75
$\frac{25}{16}$	1.1001	1.5625
$\frac{43}{16}$	10.1011	2.6875
$\frac{9}{8}$	1.001	1.125
$\frac{47}{8}$	101.111	5.875
$\frac{51}{16}$	11.0011	3.1875

思考小数二进制表示的一种简单方法是把数表示成形如 $\frac{x}{2^k}$ 的分数。我们可以用 x 的二进制表示来写出这个二进制数,并在距右端 k 个位置处插入二进制小数点。举例来说,对 $\frac{25}{16}$,我们有 $25_{10} = 11001_2$。然后我们把二进制小数点放在距右端四个位置处,得到 1.1001_2。

答案 2.46 多数情况下,浮点数精度有限并不是一个大问题,因为计算的相对误差仍然相当低。然而在这个例子中,系统对绝对误差很敏感。

A. 可以看到 $0.1 - x$ 的二进制表示是

$$0.000000000000000000000001100[1100]\cdots_2$$

B. 把它与 $\frac{1}{10}$ 的二进制表示相比较,可以看出它就是 $2^{-20} \times \frac{1}{10}$,约为 9.54×10^{-8}。

C. $9.54 \times 10^{-8} \times 100 \times 60 \times 60 \times 10 \approx 0.343$ 秒。

D. $0.343 \times 2{,}000 \approx 687$ 米。

答案 2.47 对非常小的字长逐一推敲浮点表示,有助于弄清 IEEE 浮点数是如何工作的。尤其要注意非规格化值与规格化值之间的过渡。

位串	e	E	2^E	f	M	$2^E \times M$	V	十进制
0 00 00	0	0	1	$\frac{0}{4}$	$\frac{0}{4}$	$\frac{0}{4}$	$None$	0.0
0 00 01	0	0	1	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$	0.25
0 00 10	0	0	1	$\frac{2}{4}$	$\frac{2}{4}$	$\frac{2}{4}$	$\frac{1}{2}$	0.5
0 00 11	0	0	1	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	0.75
0 01 00	1	0	1	$\frac{0}{4}$	$\frac{4}{4}$	$\frac{4}{4}$	1	1.0
0 01 01	1	0	1	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	$\frac{5}{4}$	1.25
0 01 10	1	0	1	$\frac{2}{4}$	$\frac{6}{4}$	$\frac{6}{4}$	$\frac{3}{2}$	1.5
0 01 11	1	0	1	$\frac{3}{4}$	$\frac{7}{4}$	$\frac{7}{4}$	$\frac{7}{4}$	1.75
0 10 00	2	1	2	$\frac{0}{4}$	$\frac{4}{4}$	$\frac{8}{4}$	2	2.0
0 10 01	2	1	2	$\frac{1}{4}$	$\frac{5}{4}$	$\frac{10}{4}$	$\frac{5}{2}$	2.5
0 10 10	2	1	2	$\frac{2}{4}$	$\frac{6}{4}$	$\frac{12}{4}$	3	3.0
0 10 11	2	1	2	$\frac{3}{4}$	$\frac{7}{4}$	$\frac{14}{4}$	$\frac{7}{2}$	3.5
0 11 00	—	—	—	—	—	—	∞	—
0 11 01	—	—	—	—	—	—	NaN	—
0 11 10	—	—	—	—	—	—	NaN	—
0 11 11	—	—	—	—	—	—	NaN	—

答案 2.48 十六进制的 `0x359141` 等价于二进制的 `[11010110010001001000001]`。把它右移 21 位得到 $1.10101100100010100000{1}_2 \times 2^{21}$。我们通过去掉前导 1 并在后面添加两个 0 来构成小数域,得到

`[10101100100010100000100]`

阶码由 21 加上偏置 127 得到,即 148(二进制 `[10010100]`)。我们用一个为 0 的符号域与之组合,得到二进制表示

`[01001010010101100100010100000100]`

可以看到,这两个表示中匹配的那些位对应于整数的低位:从等于 1 的最高有效位开始,它们与小数域的高 21 位相匹配:

```
0   0   3   5   9   1   4   1
```

```
00000000001101011001000101000001
          **********************
  4   A   5   6   4   5   0   4
01001010010101100100010100000100
```

答案 2.49 这个练习帮助你思考哪些数不能被浮点数精确地表示。

A. 这个数的二进制表示为 1，后面跟 n 个 0，再后面是 1，其值为 $2^{n+1}+1$。

B. 当 $n=23$ 时，该值为 $2^{24}+1=16{,}777{,}217$。

答案 2.50 手工进行舍入有助于强化二进制数向偶数舍入的概念。

原值		舍入结果	
10.111_2	$2\frac{7}{8}$	11.0	3
11.010_2	$3\frac{1}{4}$	11.0	3
11.000_2	3	11.0	3
10.110_2	$2\frac{3}{4}$	11.0	3

答案 2.51 **A.** 观察 $\frac{1}{10}$ 的不终止序列，可以看到舍入位置右侧开始的 2 位是 1，因此把 x 加 1 得到 $x' = 0.00011001100110011001101_2$ 是对 $\frac{1}{10}$ 更好的近似，它比 0.1 要大。

B. 可以看到 $x' - 0.1$ 的二进制表示是

$$0.000000000000000000000000[1100]$$

把它与 $\frac{1}{10}$ 的二进制表示相比较，可以看出它就是 $2^{-22} \times \frac{1}{10}$，约为 2.38×10^{-8}。

C. $2.38 \times 10^{-8} \times 100 \times 60 \times 60 \times 10 \approx 0.086$ 秒，比爱国者系统中的误差小 4 倍。

D. $0.086 \times 2{,}000 \approx 171$ 米。

答案 2.52 这个问题检验关于浮点表示的许多概念，包括规格化值和非规格化值的编码，以及舍入。

格式 A		格式 B		
位	值	位	值	说明
011 0000	1	0111 000	1	
101 1110	$\frac{15}{2}$	1001 111	$\frac{15}{2}$	
010 1001	$\frac{25}{32}$	0110 100	$\frac{3}{4}$	向下舍入
110 1111	$\frac{31}{2}$	1011 000	16	向上舍入
000 0001	$\frac{1}{64}$	0001 000	$\frac{1}{64}$	非规格化 → 规格化

答案 2.53 一般而言，使用一个库宏比自己发明代码更好。不过，下面这段代码在各种机器上似乎都能工作。

我们假定值 `1e400` 会溢出为无穷大。

```
#define POS_INFINITY 1e400
#define NEG_INFINITY (-POS_INFINITY)
#define NEG_ZERO (-1.0/POS_INFINITY)
```

答案 2.54 像这样的练习有助于你培养从程序员的视角推理浮点运算的能力。务必确保理解每一个答案。

A. `x == (int)(double) x` 可以,因为 `double` 具有比 `int` 更高的精度和更大的取值范围。

B. `x == (int)(float) x` 不行。例如,当 x 是 $TMax$ 时。

C. `d == (double)(float) d` 不行。例如,当 d 为 `1e40` 时,右端会得到 $+\infty$。

D. `f == (float)(double) f` 可以,因为 `double` 具有比 `float` 更高的精度和更大的取值范围。

E. `f == -(-f)` 可以,因为对浮点数取负只是简单地翻转它的符号位。

F. `1.0/2 == 1/2.0` 可以,在做除法之前,分子和分母都会先被转换成浮点表示。

G. `d*d >= 0.0` 可以,尽管它可能溢出为 $+\infty$。

H. `(f+d)-f == d` 不行。例如,当 f 为 `1.0e20` 而 d 为 1.0 时,表达式 `f+d` 将被舍入为 `1.0e20`,于是左端求值为 0.0,而右端是 1.0。

3

程序的机器级表示

计算机执行机器码（machine code），即对低层操作进行编码的字节序列，这些操作用来处理数据、管理存储器、在存储设备上读写数据并通过网络进行通信。编译器通过一系列阶段生成机器码，所依据的是程序设计语言的规则、目标机器的指令集，以及操作系统遵循的约定。gcc C 编译器以汇编代码（assembly code）的形式生成输出，汇编代码是机器码的文本表示，逐条给出程序中的指令。gcc 再调用汇编器和链接器，由汇编代码生成可执行的机器码。本章中，我们将仔细考察机器码，以及它的人类可读表示——汇编代码。

用 C 这样的高级语言编程时，我们被屏蔽在程序机器级实现的细节之外；用 Java 编程时更是如此。相比之下，用汇编代码编写程序时（如计算技术早期年代的做法），程序员必须指明程序用来进行计算的各个低层指令。在大多数情况下，在高级语言提供的那层较高的抽象上工作是更有成效、也更可靠的。编译器提供的类型检查能帮助发现许多程序错误，并保证我们以一致的方式引用和操作数据。有了现代的优化编译器，生成的代码通常至少与熟练的汇编语言程序员手工写出的代码同样高效。最好的一点是，用高级语言写的程序可以编译后在许多不同的机器上执行，而汇编代码却与具体的机器高度相关。

既然如此，为什么还要花时间学习机器码呢？尽管生成汇编代码的大部分工作都由编译器完成，但能够阅读并理解汇编代码，对认真的程序员来说是一项重要技能。用适当的命令行参数调用编译器，编译器就会生成一个文件，以汇编代码的形式显示其输出。通过阅读这些代码，我们可以了解编译器的优化能力，并分析代码中潜在的效率低下之处。正如我们将在第 5 章体会到的，追求让关键代码段的性能达到最大的程序员，常常尝试源代码的各种不同变体，每次都编译并检视生成的汇编代码，以了解程序运行起来的效率如何。此外，高级语言提供的抽象层，有时会掩盖我们需要理解的程序运行时行为的信息。例如，在用线程包编写并发程序时（第 12 章会讲到），重要的是理解程序的数据是如何被不同线程共享或保持私有的，以及确切地在何处、以何种方式访问共享数据。这类信息在机器码这一层是可见的。再举一个例子，程序遭受攻击、

使恶意软件得以感染系统的许多途径，都涉及程序存储运行时控制信息的方式中的一些微妙之处。许多攻击利用系统程序中的弱点来改写信息，从而控制系统。要理解这些脆弱性如何产生、又如何防范，就需要了解程序的机器级表示。程序员学习机器码的必要性，多年来已经发生了转变：过去要能够直接用汇编代码编写程序，而现在要能够阅读和理解编译器生成的代码。

在本章中，我们将详细了解一种特定的汇编语言，并看看 C 程序是如何被编译成这种形式的机器码的。阅读编译器生成的汇编代码需要一套不同于手工编写汇编代码的技能。我们必须理解典型的编译器在把 C 的各种构造转换成机器码时所做的变换。相对于 C 代码表达的计算，优化编译器可以重新安排执行顺序，消除不需要的计算，用较快的操作替换较慢的操作，甚至把递归计算改变成迭代计算。理解源代码与生成的汇编代码之间的关系常常是一种挑战——这很像是拼一幅拼图，而这幅拼图的设计与盒子上的图画略有出入。这是一种逆向工程（reverse engineering）——试图通过研究系统、往回追溯的办法，来弄清系统是如何被造出来的。这里的系统是机器生成的汇编语言程序，而不是什么由人设计出来的东西。这其实反而简化了逆向工程的任务，因为生成的代码遵循相当规则的模式，而且我们可以做实验，让编译器为许多不同的程序生成代码。在我们的讲述中，我们给出了大量示例，并提供了许多练习，来说明汇编语言和编译器的种种侧面。这门学科的特点是，掌握细节是理解更深层的基本概念的前提。那些说“我理解一般原理，不想费心去学那些细节”的人是在自欺欺人。花时间研究这些例子、逐一做出这些练习，并对照给出的答案核对自己的解答，对你来说是至关重要的。

我们的讲述基于 x86-64，它是当今大多数便携式和台式机器中的处理器所用的机器语言，也是支撑超大规模数据中心和超级计算机的机器语言。这种语言经历了漫长的演化：从英特尔公司 1978 年的第一个 16 位处理器开始，经过向 32 位的扩展，最近又扩展到 64 位。在此过程中，人们不断加入一些特性，以便更好地利用现有的半导体技术，并满足市场的需求。这些发展主要由英特尔推动，但它的竞争对手超威半导体公司（Advanced Micro Devices，AMD）也做出了重要贡献。其结果是一个相当奇特的设计：只有从历史的角度看，它的许多特性才说得通。它还背负着大量提供向后兼容的特性，而现代的编译器和操作系统并不使用这些特性。我们将把注意力集中在 gcc 和 Linux 所使用的一个特性子集上。这样，我们可以避开 x86-64 的大部分复杂性和许多晦涩的特性。

我们的技术讲述从一次快速的浏览开始，展示 C、汇编代码和机器码之间的关系。然后进入 x86-64 的细节，从数据的表示与操作以及控制的实现开始。我们会看到 C 中的控制构造（如 `if`、`while` 和 `switch` 语句）是如何实现的。接着讲述过程的实现，包括程序如何维护一个运行时栈，以支持过程之间数据和

控制的传递,并为局部变量提供存储。之后,我们考察数组、结构和联合这类数据结构在机器级是如何实现的。有了机器级编程的这些背景,我们就可以考察越界内存引用的问题,以及系统面对缓冲区溢出攻击的脆弱性。这一部分讲述的最后,给出一些使用 gdb 调试器检查机器级程序运行时行为的技巧。本章最后讲述涉及浮点数据和运算的代码的机器程序表示。

> **网络边栏　ASM:IA32　IA32 编程**
> IA32 是 x86-64 的 32 位前身,由英特尔于 1985 年推出。几十年来,它一直是最受青睐的机器语言。如今出售的大多数 x86 微处理器,以及装在这些机器上的大多数操作系统,都是为运行 x86-64 而设计的,不过它们也能在后向兼容模式下执行 IA32 程序。因此,许多应用程序仍然以 IA32 为基础。此外,由于硬件或系统软件的限制,许多现有的系统不能执行 x86-64。IA32 仍然是一种重要的机器语言。你会发现,有了 x86-64 的背景,学习 IA32 机器语言会相当容易。

计算机工业最近完成了从 32 位机器向 64 位机器的过渡。一台 32 位机器只能使用大约 4 GB(2^{32} 字节)的随机访问存储器。存储器价格急剧下跌,而我们的计算需求和数据规模不断增大,突破这一限制在经济上已经可行,在技术上也顺理成章。当前的 64 位机器最长可以使用 256 TB(2^{48} 字节)的存储器,而且可以轻易扩展到最多 16 EB(2^{64} 字节)。虽然很难想象真有装了这么多存储器的机器,但请记住,当 32 位机器在 20 世纪 70 年代和 80 年代变得普遍的时候,4 GB 存储器看起来就像一个极端的数字。

我们的讲述聚焦于把面向现代操作系统的 C 及类似程序设计语言编译时所生成的那类机器级程序。因此,我们不去描述 x86-64 的许多特性;那些特性源自它对早期微处理器时代程序风格的遗留支持,那时许多代码都是手写的,程序员还不得不与 16 位机器所允许的有限地址范围苦苦周旋。

3.1 历史观点

英特尔处理器系列,俗称 x86,经历了漫长的演化发展。它最初只是最早的一批单芯片 16 位微处理器之一,当时受集成电路技术能力所限,不得不做出许多妥协。此后,它不断利用技术的进步来获得优势,同时满足对更高性能、对支持更先进的操作系统的需求。

下面的清单列出了英特尔处理器的一些型号以及它们的一些关键特性,特别是那些影响机器级编程的特性。我们用实现这些处理器所需的晶体管数量,来表示它们的复杂程度是如何演化的。在这个表中,“K”表示 1,000(10^3),“M”表示 1,000,000(10^6),“G”表示 1,000,000,000(10^9)。

- **8086（1978 年，29 K 晶体管）**。最早的单芯片 16 位微处理器之一。8088 是 8086 的一个变体，外部总线为 8 位，它是第一代 IBM 个人计算机的核心。IBM 与当时规模尚小的微软签约，开发 MS-DOS 操作系统。最初的机型配置了 32,768 字节的存储器和两个软盘驱动器（没有硬盘）。在体系结构上，这些机器被限制在一个 655,360 字节的地址空间内——地址只有 20 位长（可寻址 1,048,576 字节），而且操作系统为自己保留了 393,216 字节。1980 年，英特尔推出了 8087 浮点协处理器（45 K 晶体管），与 8086 或 8088 处理器相伴工作，执行浮点指令。8087 确立了 x86 系列的浮点模型，通常称为“x87”。
- **80286（1982 年，134 K 晶体管）**。增加了更多（如今已经过时的）寻址模式。它是 IBM PC-AT 个人计算机的基础，也是 MS Windows 最初的平台。
- **i386（1985 年，275 K 晶体管）**。把体系结构扩展到 32 位。加入了 Linux 以及 Windows 操作系统近期版本所采用的平坦寻址模型。这是这个系列中第一台能够完整支持 Unix 操作系统的机器。
- **i486（1989 年，1.2 M 晶体管）**。性能有所提高，并把浮点单元集成到处理器芯片上，但没有显著改变指令集。
- **Pentium（1993 年，3.1 M 晶体管）**。性能有所提高，但只对指令集做了少量扩展。
- **PentiumPro（1995 年，5.5 M 晶体管）**。引入了全新的处理器设计，内部称为 P6 微体系结构。向指令集加入了一类“条件传送”指令。
- **Pentium/MMX（1997 年，4.5 M 晶体管）**。为 Pentium 处理器加入了一类新的指令，用于处理整数向量。每个数据可以是 1、2 或 4 字节长，每个向量共 64 位。
- **Pentium II（1997 年，7 M 晶体管）**。延续 P6 微体系结构。
- **Pentium III（1999 年，8.2 M 晶体管）**。引入 SSE，这是一类处理整数或浮点数向量的指令。每个数据可以是 1、2 或 4 字节，打包成 128 位的向量。这个芯片的后续版本晶体管数达到了 24 M，因为把二级高速缓存集成到了芯片上。
- **Pentium 4（2000 年，42 M 晶体管）**。把 SSE 扩展为 SSE2，增加了新的数据类型（包括双精度浮点数），以及面向这些格式的 144 条新指令。有了这些扩展，编译器可以采用 SSE 指令（而不是 x87 指令）来编译浮点代码。
- **Pentium 4E（2004 年，125 M 晶体管）**。增加了超线程（hyperthreading），这是一种在单个处理器上同时运行两个程序的技术；还增加了 EM64T——

英特尔对由超威半导体公司(Advanced Micro Devices,AMD)开发的 IA32 之 64 位扩展的实现,我们称之为 x86-64。

- **Core 2(2006 年,291 M 晶体管)**。回归到一种与 P6 相似的微体系结构。这是第一款英特尔多核微处理器,把多个处理器实现在单个芯片上。不支持超线程。
- **Core i7,Nehalem(2008 年,781 M 晶体管)**。同时集成了超线程与多核,最初的版本支持在每个核上执行两个程序、每个芯片上最多四个核。
- **Core i7,Sandy Bridge(2011 年,1.17 G 晶体管)**。引入 AVX,这是对 SSE 的扩展,支持打包成 256 位向量的数据。
- **Core i7,Haswell(2013 年,1.4 G 晶体管)**。把 AVX 扩展为 AVX2,增加了更多的指令和指令格式。

每一代处理器都被设计成向后兼容的——能够运行为更早的任何版本编译的代码。我们将看到,由于这一演化的传承,指令集中留下了许多奇怪的痕迹。英特尔的处理器系列有过好几个名字,包括 IA32(意为“Intel Architecture 32-bit”),以及最近的 Intel64——IA32 的 64 位扩展,我们称之为 x86-64。至于整个系列,我们沿用最常用的俗称“x86”来称呼它,这个名字反映了直到 i486 为止的处理器命名惯例。

多年来,有几家公司生产出了与英特尔处理器兼容、能够运行完全相同的机器级程序的处理器。其中最主要的是超威半导体公司(Advanced Micro Devices,AMD)。多年来 AMD 在技术上稍稍落后于英特尔,这迫使它采取一种营销策略:生产价格更低、但性能也略逊一筹的处理器。大约在 2002 年,它变得更有竞争力了——它率先让市售微处理器突破 1 千兆赫兹的时钟频率大关,并且推出了 x86-64,这是对英特尔 IA32 的一种 64 位扩展,如今已获得广泛采用。虽然我们谈论的是英特尔处理器,但我们的讲述对英特尔竞争对手生产的兼容处理器同样适用。

对那些关心 gcc 编译器为 Linux 操作系统生成的程序的人来说,x86 的很大一部分复杂性是无须操心的。最早的 8086 提供的存储器模型以及它在 80286 中的扩展,到 i386 就废弃了。最初的 x87 浮点指令则随着 SSE2 的引入而废弃了。虽然我们在 x86-64 程序中还能看到 x86 历史演化的痕迹,但 x86 的许多最晦涩的特性并没有出现在其中。

> **边栏　摩尔定律**
> 如果把不同英特尔处理器的晶体管数量相对于其推出的年份画出来,纵轴采用对数刻度,就可以看到其增长非同寻常。用一条直线去拟合这些数据,晶体管的数量大

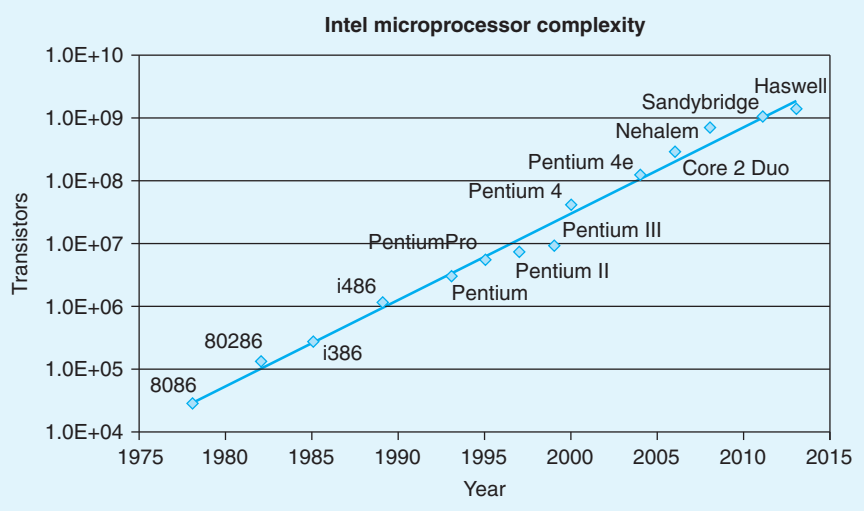


图 A-MOORE （示意图）

> 约以每年 37% 的速率增长，也就是说，大约每 26 个月翻一番。这种增长在 x86 微处理器几十年的历史中一直持续着。
> 1965 年，英特尔公司的创始人之一戈登·摩尔（Gordon Moore）根据当时的芯片技术做了一个外推（那时他们能在一块芯片上制作的电路大约有 64 个晶体管），并预测每块芯片上的晶体管数量在此后的 10 年里每年翻一番。这个预测后来被称为摩尔定律（Moore's Law）。事后看来，他的预测有一点乐观，但也有点目光短浅。在 50 多年里，半导体工业平均每 18 个月就能让晶体管数量翻一番。
> 计算机技术的其他方面也出现了类似的指数增长，包括磁盘和半导体存储器的存储容量。非凡的增长速度是计算机革命最主要的推动力。

3.2 程序编码

假设我们写了一个 C 程序，它由两个文件 `p1.c` 和 `p2.c` 组成。我们可以用一条 Unix 命令行来编译这段代码：

```
linux> gcc -Og -o p p1.c p2.c
```

命令中的 gcc 指的是 gcc C 编译器。由于 gcc 是 Linux 上的默认编译器，我们也可以直接用 cc 来调用它。命令行选项 `-Og`[1] 告诉编译器采用能达到与原始 C 代码整体结构一致的优化级别来生成机器码。使用更高的优化级别，生成的代码会变换得如此厉害，以致生成的机器码与原始源代码之间的关系很难弄懂。因此，我们把 `-Og` 优化作为一种学习工具来用，然后看看提高优化级别会

[1] 这个优化级别是在 gcc 4.8 版中引入的。更早版本的 gcc，以及非 GNU 的编译器，无法识别这个选项。对这些编译器来说，使用优化级别 1（用命令行标志 `-O1` 指定）很可能是生成遵循原程序结构的代码的最佳选择。

发生什么。在实践中,更高的优化级别(例如用选项 `-O1` 或 `-O2` 指定的)从结果程序的性能来看被认为是更好的选择。

gcc 命令调用一整套程序,把源代码变成可执行代码。首先,C 预处理器展开源代码,把用 `#include` 命令指定的文件并入其中,并展开用 `#define` 声明指定的宏。其次,编译器生成两个源文件的汇编代码版本,文件名为 `p1.s` 和 `p2.s`。接着,汇编器把汇编代码转换成二进制目标代码文件 `p1.o` 和 `p2.o`。目标代码是机器码的一种形式——它包含了所有指令的二进制表示,但还没有填入全局值的地址。最后,链接器把这两个目标代码文件与实现库函数的代码(例如 `printf`)合并在一起,生成最终的可执行代码文件 p(由命令行指示 `-o p` 指定)。可执行代码是我们要考虑的第二种形式的机器码——它正是处理器执行的那样一种代码形式。这些不同形式的机器码之间的关系以及链接过程,将在第 7 章更详细地描述。

3.3 机器级代码

如 1.9.3 节所述,计算机系统采用若干不同的抽象形式,利用更简单的抽象模型来隐藏实现的细节。其中有两个对机器级编程特别重要。第一,机器级程序的格式和行为是由指令集体系结构(instruction set architecture,ISA)定义的,它规定了处理器状态、指令的格式,以及每条指令对状态的影响。包括 x86-64 在内的大多数 ISA,把程序的行为描述成每条指令按顺序执行的样子,一条指令完成后下一条才开始。实际的处理器硬件要精巧得多,它并发地执行许多指令,但会用各种保障机制使总体行为同 ISA 规定的顺序操作相吻合。第二,机器级程序使用的存储器地址是虚拟地址(virtual address),它提供了一种看起来是一个非常大的字节数组的存储器模型。存储器系统的实际实现把多种硬件存储器和操作系统软件结合起来,这将在第 9 章描述。

在整条编译流程中,编译器做的是大部分工作:把用 C 提供的那个相当抽象的执行模型表示的程序,变换成处理器执行的非常基本的指令。汇编代码的表示与机器码非常接近,其主要特点是,与机器码的二进制格式相比,它采用的是更易读的文本格式。能够理解汇编代码以及它与原始 C 代码的关系,是理解计算机如何执行程序的关键一步。

对 x86-64 而言,机器码与原始的 C 代码大不相同。那些通常对 C 程序员隐藏的处理器状态,有一部分是可见的:

- 程序计数器(通常称为 PC,在 x86-64 中叫`%rip`)指示下一条要执行的指令在存储器中的地址。

- 整数寄存器堆包含 16 个命名的位置，每个位置存放 64 位的值。这些寄存器可以存放地址（对应于 C 的指针）或整数数据。有些寄存器用来记录程序状态的某些关键部分，另一些则用来存放临时数据，例如过程的参数和局部变量，以及函数要返回的值。
- 条件码寄存器保存最近执行的算术或逻辑指令的状态信息。它们用来实现控制流或数据流的条件变化，例如实现 `if` 和 `while` 语句所需的那些变化。
- 一组向量寄存器，每个都能存放一个或多个整数或浮点值。

在 C 中提供的是一种模型，其中可以把不同数据类型的对象在存储器中声明和分配；而机器码只是把存储器看成一个很大的、可按字节寻址的数组。C 中的聚合数据类型（如数组和结构）在机器码中表示为连续的字节集合。即使是标量数据类型，汇编代码也不区分有符号整数与无符号整数，不区分不同类型的指针，甚至不区分指针和整数。

程序存储器中包含程序的可执行机器码、操作系统所需的一些信息、一个用来管理过程调用和返回的运行时栈，以及用户分配的存储器块（例如通过调用 malloc 库函数）。如前所述，程序存储器用虚拟地址来寻址。在任意给定的时刻，只有虚拟地址中有限的部分被看作是有效的。例如，x86-64 的虚拟地址用 64 位的字表示。在目前这些机器的实现中，高 16 位必须置 0，因此一个地址可能指定的范围可达 2^{48}，即 64 TB。更常见的程序只会有几个 MB，或者也许几个 GB 的可用空间。操作系统负责管理这个虚拟地址空间，把虚拟地址翻译成处理器实际存储器中各值的物理地址。

一条单独的机器指令只完成一个非常基本的功能。例如，它可能把存放在寄存器中的两个数相加，在存储器和寄存器之间传送数据，或者有条件地跳转到新的指令地址。编译器必须生成这些指令的序列，来实现诸如算术表达式求值、循环、过程调用和返回这样一些程序构造。

> **边栏　生成代码的形式在不断变化**
> 在本书的讲述中，我们给出的代码是某个特定版本的 gcc 在特定命令行选项设置下生成的。如果你在自己的机器上编译代码，很可能用的是不同的编译器或不同版本的 gcc，从而会生成不同的代码。支持 gcc 的开源社区一直在修改代码生成器，力求按照微处理器制造商不断更新的代码编写准则生成更高效的代码。
> 我们研究本书所示示例的目的，是演示如何检视汇编代码，并把它映射回高级程序设计语言中能找到的那些构造。你需要把这些技巧因地制宜地用到你自己的编译器所生成的代码风格上。

3.4 代码示例

假设我们写了一个 C 代码文件 `mstore.c`,其中包含如下函数定义:

```
long mult2(long, long);

void multstore(long x, long y, long *dest) {
    long t = mult2(x, y);
    *dest = t;
}
```

要看看 C 编译器生成的汇编代码,可以在命令行中使用 `-S` 选项:

```
linux> gcc -Og -S mstore.c
```

这会使 gcc 运行编译器,生成汇编文件 `mstore.s` 后就此为止。(通常它此后还会调用汇编器生成目标代码文件。)

汇编代码文件包含各种声明,其中包括下面这些行:

```
multstore:
  pushq   %rbx
  movq    %rdx, %rbx
  call    mult2
  movq    %rax, (%rbx)
  popq    %rbx
  ret
```

边栏 如何显示程序的字节表示?
要显示一个程序(比如说 mstore)的二进制目标代码,我们使用一个反汇编器(下文将介绍)来确定这条过程的代码长 14 字节。然后在文件 `mstore.o` 上运行 GNU 调试工具 gdb,并给出命令

```
(gdb) x/14xb multstore
```

告诉它从函数 multstore 所在的地址开始,显示(缩写'x')14 个以十六进制格式(同样缩写'x')表示的字节('b')。你会发现 gdb 有许多分析机器级程序的有用特性,3.10.2 节会对此进行讨论。

代码中每一行缩进的行都对应于一条单独的机器指令。例如,指令 pushq 表示寄存器%rbx 的内容应当被压入程序栈。究竟用哪个局部变量名、哪种数据类型的信息通通都被去掉,只剩指令本身。

如果我们使用 `-c` 命令行选项,gcc 会既编译又汇编这段代码:

```
linux> gcc -Og -c mstore.c
```

这会生成目标代码文件 `mstore.o`,它是二进制格式的,因此不能直接查看。在文件 `mstore.o` 的 1,368 个字节中,嵌着一串长 14 字节、十六进制表示为

```
53 48 89 d3 e8 00 00 00 00 48 89 03 5b c3
```

这就是与前面列出的汇编指令相对应的目标代码。由此要学到的一个重要经验是：机器执行的程序只是一个字节序列，这些字节对一系列指令进行编码。机器几乎没有任何关于生成这些指令的源代码的信息。

要查看机器码文件的内容，有一类称为反汇编器（disassembler）的程序会是得力的工具。这些程序从机器码生成一种与汇编代码很相似的格式。在 Linux 系统上，用 objdump（意为“object dump”）程序并加上 `-d` 命令行标志，就可以承担这一角色：

```
linux> objdump -d mstore.o
```

结果如下（我们在左边加了行号，并用斜体文本给出注解）：

```
        二进制文件 mstore.o 中函数 sum 的反汇编
1  0000000000000000 <multstore>:
     偏移      字节                          等价的汇编语言
2      0:   53                         push   %rbx
3      1:   48 89 d3                   mov    %rdx,%rbx
4      4:   e8 00 00 00 00             callq  9 <multstore+0x9>
5      9:   48 89 03                   mov    %rax,(%rbx)
6      c:   5b                         pop    %rbx
7      d:   c3                         retq
```

左边我们看到的是 14 个十六进制字节值，按前面给出的字节序列列出，划分成每组 1 到 5 个字节。每一组就是一条指令，右边给出与之等价的汇编语言。

关于机器码及其反汇编表示，有几点值得注意：

- x86-64 的指令长度可以从 1 到 15 字节不等。指令编码的设计使得常用的指令以及操作数较少的指令，占用的字节数比不常用的指令或操作数较多的指令少。
- 指令格式的设计使得从任何给定的起始位置开始，把字节解码成机器指令只有一种解读。例如，只有指令 pushq %rbx 能以字节值 53 开头。
- 反汇编器只依据机器码文件中的字节序列来确定汇编代码。它并不需要访问程序的源代码或汇编代码版本。
- 反汇编器对指令采用的命名约定与 gcc 生成的汇编代码略有不同。在我们的例子中，它把许多指令的后尾缀“q”都省略了。这些后缀是表示大小的标记，在大多数情况下可以省略。相反，反汇编器却给 call 和 ret 指令加上了后缀“q”。同样，这些后缀可以安全地省去。

要生成实际的可执行代码，需要对一组目标代码文件运行链接器，而这组文件中必须有一个包含 main 函数。假设文件 `main.c` 中有如下函数：

```
#include <stdio.h>

void multstore(long, long, long *);

int main() {
    long d;
    multstore(2, 3, &d);
    printf("2 * 3 --> %ld\n", d);
    return 0;
}

long mult2(long a, long b) {
    long s = a * b;
    return s;
}
```

然后,我们可以生成可执行程序 prog:

```
linux> gcc -Og -o prog main.c mstore.c
```

文件 prog 长到了 8,655 字节,因为它不仅包含我们提供的那些过程的机器码,还包含用来启动和中止程序以及与操作系统交互的代码。

我们可以反汇编文件 prog:

```
linux> objdump -d prog
```

反汇编器会抽取出各种代码序列,其中包括如下内容:

```
  二进制文件 prog 中函数 sum 的反汇编
1  0000000000400540 <multstore>:
2    400540:   53                  push     %rbx
3    400541:   48 89 d3            mov      %rdx,%rbx
4    400544:   e8 42 00 00 00      callq    40058b <mult2>
5    400549:   48 89 03            mov      %rax,(%rbx)
6    40054c:   5b                  pop      %rbx
7    40054d:   c3                  retq
8    40054e:   90                  nop
9    40054f:   90                  nop
```

这段代码与对 `mstore.c` 反汇编时生成的代码几乎完全相同。一个重要的区别是,左边列出的地址不同——链接器把这行代码的位置挪到了另一个地址范围。第二个区别是,链接器把 callq 指令调用函数 mult2 时应使用的地址填上了(反汇编输出中第 4 行)。链接器的一项任务,就是把函数调用同这些函数可执行代码的位置匹配起来。最后一个区别是我们多看到了两行代码(第 8–9 行)。这些指令对程序不会有任何作用,因为它们出现在返回指令(第 7 行)之后。插入它们是为了把该函数的代码扩展到 16 字节,这样就能从存储系统性能的角度更好地安排下一个代码块的位置。

3.5 关于格式的注解

gcc 生成的汇编代码人很难读。一方面,它包含我们不必关心的一些信息;另一方面,它又没有提供关于程序或其工作方式的任何说明。例如,假设我们给出命令

```
linux> gcc -Og -S mstore.c
```

来生成文件 `mstore.s`。该文件的全部内容如下:

```
        .file   "010-mstore.c"
        .text
        .globl  multstore
        .type   multstore, @function
multstore:
        pushq   %rbx
        movq    %rdx, %rbx
        call    mult2
        movq    %rax, (%rbx)
        popq    %rbx
        ret
        .size   multstore, .-multstore
        .ident  "GCC: (Ubuntu 4.8.1-2ubuntu1~12.04) 4.8.1"
        .section          .note.GNU-stack,"",@progbits
```

所有以“.”开头的行都是指导汇编器和链接器的指示(directive)。一般可以忽略它们。另一方面,文件中没有任何解释性的话,说明这些指令做些什么,或者它们与源代码有什么关系。

为了给出更清楚的汇编代码,我们将采用一种省去大部分指示的形式来展示它,同时加上行号和解释性的注解。对我们的例子来说,加注之后的形式如下:

```
  void multstore(long x, long y, long *dest)
  x 在 %rdi,y 在 %rsi,dest 在 %rdx
1  multstore:
2       pushq   %rbx                    保存 %rbx
3       movq    %rdx, %rbx              把 dest 复制到 %rbx
4       call    mult2                   调用 mult2(x, y)
5       movq    %rax, (%rbx)            把结果存储在 *dest
6       popq    %rbx                    恢复 %rbx
7       ret                             返回
```

我们通常只展示与所讨论要点相关的代码行。每一行在左边标上行号供引用,右边加上简要的注解,说明该指令的作用以及它与原始 C 代码计算的关系。这是对汇编语言程序员排版代码方式的一种风格化呈现。

我们还提供网络边栏,涵盖一些面向机器语言爱好者的内容。一个网络边栏讲述 IA32 机器码。有了 x86-64 的背景,学习 IA32 就相当容易。另一个网络

边栏简要介绍把汇编代码并入 C 程序的方法。对某些应用,程序员必须下沉到汇编代码这一级,才能访问机器的低层特性。一种办法是用汇编代码编写整个函数,并在链接阶段把它们与 C 函数组合起来;另一种办法是利用 gcc 对把汇编代码直接嵌入 C 程序的支持。

边栏 ATT 格式与 Intel 格式的汇编代码
在本书的讲述中,汇编代码采用 ATT 格式(以多年经营贝尔实验室的 AT&T 公司命名),这是 gcc、objdump 以及我们将要考虑的其他工具的默认格式。其他的程序设计工具,包括微软的工具以及英特尔的文档,则以 Intel 格式给出汇编代码。这两种格式在许多方面不同。举个例子,用下面的命令行,gcc 可以按 Intel 格式为 sum 函数生成代码:

```
linux> gcc -Og -S -masm=intel mstore.c
```

这会给出如下的汇编代码:

```
multstore:
  push    rbx
  mov     rbx, rdx
  call    mult2
  mov     QWORD PTR [rbx], rax
  pop     rbx
  ret
```

我们看到,Intel 格式与 ATT 格式有如下差别:

- Intel 代码省略了表示大小的后缀。我们看到的是指令 push 和 mov,而不是 pushq 和 movq。
- Intel 代码省略了寄存器名前面的“%”字符,用 rbx 而不是%rbx。
- Intel 代码用另一种方式描述存储器中的位置——例如,用 `QWORD PTR [rbx]` 而不是 `(%rbx)`。
- 操作数有多个的指令,按相反的顺序列出各操作数。在两种格式之间切换时,这一点可能非常令人迷惑。

虽然我们在本书的讲述中不会使用 Intel 格式,但你会在英特尔和微软的文档中遇到它。

3.6 数据格式

由于英特尔起源于一个后来扩展成 32 位体系的 16 位体系结构,它用“字”(word)这个词来称呼 16 位的数据类型。在此基础上,把 32 位的量称为“双字”(double word),64 位的量称为“四字”(quad word)。图 3.1 给出的是表示 C 基本数据类型时使用的 x86-64 表示。标准的 int 值存放为双字(32 位)。指针(这里写作 `char *`)存放为 8 字节的四字,这是 64 位机器理应有的做法。使用

x86-64 时，数据类型 long 以 64 位实现，允许一个很宽的取值范围。本章大多数代码示例都使用指针和 long 数据类型，因此它们将对四字进行运算。x86-64 指令集也包括针对字节、字和双字的一整套指令。

网络边栏　ASM:EASM　把汇编代码与 C 程序结合起来

C 编译器在把程序表达的计算转换成机器码方面做得很好，但机器的有些特性是 C 程序访问不到的。例如，x86-64 处理器每次执行算术或逻辑运算时，都会设置一个名叫 PF（parity flag，即“奇偶标志”）的 1 位条件码标志，当运算结果的低 8 位中 1 的个数为偶数时把它设为 1，否则设为 0。在 C 中计算这个信息至少需要 7 次移位、掩码和异或运算（见练习 2.65）。尽管硬件在每次算术或逻辑运算中都执行这个计算，C 程序却没有任何办法确定 PF 条件码标志的值。只要在程序中嵌入少量汇编代码指令，这项任务就很容易完成。

把汇编代码并入 C 程序有两种方法。其一，把整个函数写成一个单独的汇编代码文件，让汇编器和链接器把它同我们用 C 写的代码合并。其二，利用 gcc 的内联汇编（inline assembly）特性，用 asm 指令把简短的汇编代码片段并入 C 程序。这种做法的优点是使与机器相关的代码量最少。

当然，在 C 程序中编入汇编代码会使代码只适用于某一类特定的机器（如 x86-64），所以只有当所需的特性只能这样访问时才应当使用。

图 3.1　x86-64 中 C 数据类型的大小（在 64 位机器上，指针为 8 字节长）

C 声明	Intel 数据类型	汇编代码后缀	大小（字节）
`char`	字节	`b`	1
`short`	字	`w`	2
`int`	双字	`l`	4
`long`	四字	`q`	8
`char *`	四字	`q`	8
`float`	单精度	`s`	4
`double`	双精度	`l`	8

浮点数有两种主要格式：单精度（4 字节）值，对应于 C 数据类型 float；双精度（8 字节）值，对应于 C 数据类型 double。x86 家族的微处理器沿用历史方式，用一种特殊的 80 位（10 字节）浮点格式实现所有的浮点运算（见练习 2.86）。在 C 程序中可以用声明 long double 来指定这种格式。不过我们建议不要使用这种格式。它不能移植到其他几类机器，而且通常

也未使用与单精度和双精度算术相同的高性能硬件来实现。

正如图 3.1 的表所表明的，gcc 生成的大多数汇编代码指令都带一个单字符后缀，用来表示操作数的大小。例如，数据传送指令有四个变体：`movb`（传送字节）、`movw`（传送字）、`movl`（传送双字）和 `movq`（传送四字）。后缀 l 用于双字，因为 32 位的量被视为“长字”。汇编代码用后缀 l 既表示 4 字节整数，也表示 8

字节双精度浮点数。这不会产生歧义,因为浮点代码使用的是一套完全不同的指令和寄存器。

3.7 访问信息

x86-64 的中央处理器(CPU)包含一组 16 个通用寄存器,用来存储 64 位的值。这些寄存器用来存放整数数据和指针。图 3.2 给出了这 16 个寄存器。它们的名字都以%r 开头,除此之外还遵循多种不同的命名习惯,这是指令集历史演变的结果。最初的 8086 有 8 个 16 位寄存器,即图 3.2 中的%ax 到%bp。每个寄存器都有特定用途,因此它们按各自的预期用法命名。随着扩展为 IA32,这些寄存器扩大到 32 位,记作%eax 到%ebp。扩展到 x86-64 时,最初的 8 个寄存器再扩大到 64 位,记作%rax 到%rbp。此外还新增了 8 个寄存器,按新的命名习惯记作%r8 到%r15。

正如图 3.2 中嵌套的方框所表明的,指令可以对存储在这 16 个寄存器低位字节中不同大小的数据进行操作。字节级操作可以访问最低有效字节,16 位操作可以访问低位的 2 个字节,32 位操作可以访问低位的 4 个字节,而 64 位操作可以访问整个寄存器。

在后面的章节中,我们会介绍许多用来复制和生成 1、2、4、8 字节值的指令。当这些指令以寄存器为目的时,对于生成的值不足 8 字节的指令,寄存器中剩余字节的处置有两条约定:生成 1 字节或 2 字节数量的指令保持剩余字节不变;生成 4 字节数量的指令把寄存器的高 4 个字节置 0。后一条约定是作为从 IA32 扩展到 x86-64 的一部分而采用的。

正如图 3.2 右侧的注释所表明的,在典型程序中,不同的寄存器扮演不同的角色。其中最独特的是栈指针(stack pointer)%rsp,用来指示运行时栈的末端位置。有些指令专门读写这个寄存器。其余 15 个寄存器的用法较为灵活,只有少数指令会特别使用某些寄存器。更重要的是,一套标准的编程约定规定了这些寄存器该如何用于管理栈、给函数传递参数、从函数返回值,以及存放局部数据和临时数据。我们会在讲述中介绍这些约定,尤其是在 3.7 节,那里将描述过程的实现。

3.7.1 操作数指示符

大多数指令具有一个或多个操作数(operand),指定执行某个操作时要使用的源值,以及放置结果的目标位置。x86-64 支持多种操作数格式(见图 3.3)。源值可以用常量给出,也可以从寄存器或存储器读出。结果可以存入寄存器,也可以存入存储器。因此,各种操作数的可能性可以分为三类。第一类是立即数(immediate),表示常量值。在 ATT 格式的汇编代码中,立即数写成一个 $ 符

图 3.2 整数寄存器。所有 16 个寄存器的低位部分都可以按字节、字(16 位)、双字(32 位)和四字(64 位)来访问。

63	31	15	7	用途
%rax	%eax	%ax	%al	返回值
%rbx	%ebx	%bx	%bl	被调用者保存(callee saved)
%rcx	%ecx	%cx	%cl	第 4 个参数
%rdx	%edx	%dx	%dl	第 3 个参数
%rsi	%esi	%si	%sil	第 2 个参数
%rdi	%edi	%di	%dil	第 1 个参数
%rbp	%ebp	%bp	%bpl	被调用者保存
%rsp	%esp	%sp	%spl	栈指针
%r8	%r8d	%r8w	%r8b	第 5 个参数
%r9	%r9d	%r9w	%r9b	第 6 个参数
%r10	%r10d	%r10w	%r10b	调用者保存(caller saved)
%r11	%r11d	%r11w	%r11b	调用者保存
%r12	%r12d	%r12w	%r12b	被调用者保存
%r13	%r13d	%r13w	%r13b	被调用者保存
%r14	%r14d	%r14w	%r14b	被调用者保存
%r15	%r15d	%r15w	%r15b	被调用者保存

号加上一个按标准 C 记法表达的整数——例如 `$-577` 或 `$0x1F`。不同的指令允许的立即数取值范围不一样；汇编器会自动选择编码一个值的最紧凑方式。第二类是寄存器(register)，表示寄存器的内容；对于 64、32、16、8 位的操作数，就是相应寄存器的 16 种 8、4、2、1 字节低位部分中的一种。在图 3.3 中，我们用记法 r_a 表示任意寄存器 a，并用引用 $R[r_a]$ 指示它的值，即把寄存器集合看作一个以寄存器标识符为下标的数组 R。

图 3.3 操作数格式。操作数可以表示立即数(常量)值、寄存器值或存储器中的值。比例因子 s 必须是 1、2、4 或 8。

类型	形式	操作数值	名称
立即数	$\$\text{Imm}$	Imm	立即数
寄存器	r_a	$R[r_a]$	寄存器
存储器	Imm	$M[\text{Imm}]$	绝对寻址(absolute)
存储器	(r_a)	$M[R[r_a]]$	间接寻址(indirect)
存储器	$\text{Imm}(r_b)$	$M[\text{Imm}+R[r_b]]$	基址加位移量(base + displacement)
存储器	(r_b, r_i)	$M[R[r_b]+R[r_i]]$	变址寻址(indexed)
存储器	$\text{Imm}(r_b, r_i)$	$M[\text{Imm}+R[r_b]+R[r_i]]$	变址寻址
存储器	$(, r_i, s)$	$M[R[r_i]\cdot s]$	比例变址寻址(scaled indexed)
存储器	$\text{Imm}(, r_i, s)$	$M[\text{Imm}+R[r_i]\cdot s]$	比例变址寻址
存储器	(r_b, r_i, s)	$M[R[r_b]+R[r_i]\cdot s]$	比例变址寻址
存储器	$\text{Imm}(r_b, r_i, s)$	$M[\text{Imm}+R[r_b]+R[r_i]\cdot s]$	比例变址寻址

第三类操作数是存储器引用（memory reference），即按照算出来的地址访问某个存储器位置，这个地址通常称为有效地址（effective address）。由于我们概念上把存储器看作一个大的字节数组，就用记法 $M_b[\text{Addr}]$ 表示对存储器中从地址 Addr 开始的 b 字节值的引用。为简单起见，我们通常省略下标 b。

如图 3.3 所示，有许多不同的寻址方式（addressing mode），允许不同形式的存储器引用。最一般的形式以表底的语法 $\text{Imm}(r_b, r_i, s)$ 给出。这种引用有四个组成部分：一个立即数偏移量 Imm、一个基址寄存器 r_b、一个变址寄存器 r_i 和一个比例因子 s，其中 s 必须是 1、2、4 或 8。基址寄存器和变址寄存器都必须是 64 位的。有效地址计算为 $\text{Imm} + R[r_b] + R[r_i] \cdot s$。引用数组元素时经常见到这种一般形式。其他形式只是一般形式省略了某些组成部分的特例。我们将看到，较复杂的寻址方式在引用数组和结构元素时很有用。

练习 3.1 假设下列值存放在指定的存储器地址和寄存器中：

地址	值	寄存器	值
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

填写下表，给出所列各操作数的值：

操作数	值
%rax	
0x104	
$0x108	
(%rax)	
4(%rax)	
9(%rax,%rdx)	
260(%rcx,%rdx)	
0xFC(,%rcx,4)	
(%rax,%rdx,4)	

3.7.2 数据传送指令

使用得最频繁的指令当中，有一类是把数据从一个位置复制到另一个位置的指令。操作数记法的通用性使得一条简单的数据传送指令可以表达出许多机器上需要好几条不同指令才能表达的功能。我们会介绍许多不同的数据传送指令，它们在源和目的的类型、执行的转换，以及可能具备的其他副作用等方面各不相同。在讲述中，我们把众多指令归组成若干指令类（instruction class），同一类中的指令执行相同的操作，只是操作数大小不同。

图 3.4 列出了最简单的数据传送指令——mov 类。这些指令把数据从源位置复制到目的位置，不做任何变换。这个类由四条指令组成：movb、movw、movl 和 movq。这四条指令的效果相似，主要区别在于它们操作的数据大小不同：分别为 1、2、4、8 字节。

图 3.4 简单数据传送指令。

指令	效果	描述
mov	S, D	传送
movb	–	传送字节
movw	–	传送字
movl	–	传送双字
movq	–	传送四字
movabsq	I, R	传送绝对四字

源操作数指示的值可以是立即数、存放在寄存器中，或存放在存储器中。目的操作数指示的位置可以是寄存器，也可以是存储器地址。x86-64 加上一条限制：move 指令的两个操作数不能都指向存储器位置。把值从一个存储器位置复制到另一个存储器位置需要两条指令——第一条把源值加载到一个寄存器中，第二条把这个寄存器值写到目的位置。参考图 3.2，这些指令的寄存器操作数可以是 16 个寄存器中任何一个带标号的部分，寄存器的大小必须与指令名最后一个字符（b、w、l 或 q）指定的大小相匹配。大多数情况下，mov 指令只会更新目的操作数指明的那几个寄存器字节或存储器位置。唯一的例外是，当 movl 以寄存器为目的时，它还会把该寄存器高位的 4 个字节置为 0。这个例外源于 x86-64 采用的一条约定：任何给寄存器生成 32 位值的指令都会把该寄存器的高位部分清零。

下面的 mov 指令示例展示了源和目的类型的五种可能组合。回忆一下，源操作数在先，目的操作数在后。

```
1  movl $0x4050,%eax            立即数--寄存器,4 字节
2  movw %bp,%sp                 寄存器--寄存器,2 字节
3  movb (%rdi,%rcx),%al         存储器--寄存器,1 字节
4  movb $-17,(%esp)             立即数--存储器,1 字节
5  movq %rax,-12(%rbp)          寄存器--存储器,8 字节
```

图 3.4 记录的最后一条指令用于处理 64 位立即数。常规的 movq 指令只能把可表示为 32 位补码数的立即数作为源操作数；这个值随后被符号扩展，为目的地产生 64 位的值。movabsq 指令的源操作数可以是任意的 64 位立即数，而它只能以寄存器为目的。

图 3.5 和图 3.6 记录了两类数据传送指令，用于把较小的源值复制到较大的目的地。所有这些指令都把数据从源（源可以是寄存器，也可以存放于存储

器中）复制到寄存器目的。movz 类中的指令用零填充目的中其余的字节，而 movs 类中的指令通过符号扩展来填充，即复制源操作数最高有效位的多个副本。注意到每条指令名字的最后两个字符是大小指示符——第一个指定源的大小，第二个指定目的的大小。可以看到，这两个类各有三条指令，覆盖了 1 或 2 字节源、2 或 4 字节目的的所有情况，当然只考虑目的大于源的情形。

图 3.5　零扩展数据传送指令。这些指令以寄存器或存储器位置为源，以寄存器为目的。

指令	效果	描述
movz	S, R	$R \leftarrow \text{ZeroExtend}(S)$，零扩展传送
movzbw	–	零扩展传送字节到字
movzbl	–	零扩展传送字节到双字
movzwl	–	零扩展传送字到双字
movzbq	–	零扩展传送字节到四字
movzwq	–	零扩展传送字到四字

图 3.5 中缺少一条把 4 字节源值零扩展到 8 字节目的地的显式指令。这样一条指令在逻辑上应命名为 movzlq，但它并不存在。作为替代，这种类型的数据传送可以用以寄存器为目的的 movl 指令实现。这一技巧利用了这样一条性质：给寄存器生成 4 字节值的指令会把该寄存器的高 4 字节填零。而且，对 64 位目的而言，符号扩展传送支持全部三种源类型，而零扩展传送只支持两种较小的源类型。

图 3.6 还记录了 cltq 指令。这条指令没有操作数——它总是用寄存器%eax 作源，用%rax 作符号扩展结果的目的。因此，它与指令 movslq %eax, %rax 的效果完全相同，只是它的编码更紧凑。

图 3.6　符号扩展数据传送指令。movs 系列指令以寄存器或存储器位置为源，以寄存器为目的。cltq 指令专门作用于寄存器%eax 和%rax。

指令	效果	描述
movs	S, R	$R \leftarrow \text{SignExtend}(S)$，符号扩展传送
movsbw	–	符号扩展传送字节到字
movsbl	–	符号扩展传送字节到双字
movswl	–	符号扩展传送字到双字
movsbq	–	符号扩展传送字节到四字
movswq	–	符号扩展传送字到四字
movslq	–	符号扩展传送双字到四字
cltq	–	%rax ← SignExtend(%eax)，把%eax 符号扩展到%rax

练习 3.2　对下面每行汇编语言代码，根据操作数确定恰当的指令后缀。（例如，mov 可以改写为 movb、movw、movl 或 movq。）

边栏　理解数据传送指令如何改变目的寄存器

如上所述，关于数据传送指令是否修改以及如何修改目的寄存器的高位字节，有两条不同的约定。下面的代码序列说明了这一区别：

```
1  movabsq
$0x0011223344556677,
%rax       %rax =
0011223344556677
2  movb $-1, %al
%rax = 00112233445566FF
3  movw $-1, %ax
%rax = 001122334455FFFF
4  movl $-1, %eax
%rax = 00000000FFFFFFFF
5  movq $-1, %rax
%rax = FFFFFFFFFFFFFFFF
```

在下面的讨论中，我们使用十六进制记法。示例中，第 1 行的指令把寄存器%rax 初始化为位模式 0011223344556677。其余指令以立即数 −1 为源值。回忆一下，−1 的十六进制表示形如 FF…F，其中 F 的个数是该表示中字节数的两倍。因此，movb 指令（第 2 行）把%rax 的低位字节设为 FF，而 movw 指令（第 3 行）把低 2 个字节设为 FFFF，其余字节保持不变。movl 指令（第 4 行）把低 4 个字节设为 FFFFFFFF，但同时把高 4 个字节置为 00000000。最后，movq 指令（第 5 行）把整个寄存器设为 FFFFFFFFFFFFFFFF。

```
mov         %eax, (%rsp)
mov         (%rax), %dx
mov         $0xFF, %bl
mov         (%rsp,%rdx,4), %dl
mov         (%rdx), %rax
mov         %dx, (%rax)
```

练习 3.3 下面每行代码在调用汇编器时都会产生一条错误信息。请解释每一行错在哪里。

```
movb $0xF, (%ebx)
movl %rax, (%rsp)
movw (%rax),4(%rsp)
movb %al,%sl
movq %rax,$0x123
movl %eax,%rdx
movb %si, 8(%rbp)
```

3.7.3 数据传送示例

作为一个使用数据传送指令的代码示例，考虑图 3.7 给出的数据交换例程，其中同时给出它的 C 代码和 gcc 生成的汇编代码。

如图 3.7(b) 所示，函数 exchange 只用三条指令实现：两条数据传送（movq）加上一条返回函数调用地点的指令（ret）。我们将在 3.7 节讨论函数调用和返回的细节。在那之前，只需知道参数是通过寄存器传给函数的就够了。我们带注释的汇编代码记录了这一点。函数把值存入寄存器%rax 或该寄存器的某个低位部分，从而返回值。

```
(a) C 代码
long exchange(long *xp, long y)
{
    long x = *xp;
    *xp = y;
    return x;
}
```

```
(b) 汇编代码
long exchange(long *xp, long y)
xp in %rdi, y in %rsi
1  exchange:
2    movq (%rdi), %rax    读出 xp 处的 x。作为返回值。
3    movq %rsi, (%rdi)    把 y 存到 xp 处。
4    ret                  返回。
```

图 3.7 exchange 例程的 C 代码和汇编代码。寄存器%rdi 和%rsi 分别保存参数 xp 和 y。

过程开始执行时，过程参数 xp 和 y 分别存放在寄存器%rdi 和%rsi 中。然后指令 2 从存储器读出 x，并把值存到寄存器%rax 中，这是 C 程序中操作 x =

边栏 比较字节的传送指令

下面的例子说明不同的数据传送指令如何改变（或不改变）目的的高位字节。可以看到，三条字节传送指令 movb、movsbq 和 movzbq 以微妙的方式相互区别。这里给出一个例子：

```
1  movabsq $0x0011223344556677, %rax     %rax = 0011223344556677
2  movb $0xAA, %dl                       %dl = AA
3  movb %dl,%al                          %rax = 00112233445566AA
4  movsbq %dl,%rax                       %rax = FFFFFFFFFFFFFFAA
5  movzbq %dl,%rax                       %rax = 00000000000000AA
```

在下面的讨论中，所有的值都用十六进制记法。代码的前两行把寄存器%rax 和%dl 分别初始化为 0011223344556677 和 AA。其余各条指令都是把%rdx 的低位字节复制到%rax 的低位字节。movb 指令（第 3 行）不改变其他字节。movsbq 指令（第 4 行）把其余 7 个字节全部置为 1 或全部置为 0，取决于源字节的最高位。由于十六进制的 A 表示二进制值 1010，符号扩展使各个高位字节都被置为 FF。movzbq 指令（第 5 行）总是把其余 7 个字节置为 0。

*xp 的直接实现。稍后,寄存器%rax 将用来从函数返回值,所以返回值就是 x。指令 3 把 y 写到寄存器%rdi 中 xp 指定的存储器位置,这是操作 *xp = y 的直接实现。这个例子说明了 mov 指令如何用来从存储器读到寄存器(第 2 行),以及从寄存器写到存储器(第 3 行)。

关于这段汇编代码,有两点值得注意。第一,我们看到,C 中所谓“指针”其实就是地址。解引用一个指针涉及把该指针复制到一个寄存器,然后在存储器引用中使用这个寄存器。第二,像 x 这样的局部变量经常保存在寄存器中,而不是存放在存储器位置上。寄存器访问比存储器访问快得多。

练习 3.4　假设变量 sp 和 dp 用如下类型声明:

```
src_t *sp;
dest_t *dp;
```

其中 src_t 和 dest_t 是用 typedef 声明的数据类型。我们要用恰当的一对数据传送指令来实现操作 *dp = (dest_t) *sp;。

假设 sp 和 dp 的值分别存放在寄存器%rdi 和%rsi 中。对表中的每一项,给出实现所指定数据传送的两条指令。序列中的第一条指令应当从存储器读出,执行恰当的转换,并设置寄存器%rax 的恰当部分。第二条指令应当把%rax 的恰当部分写到存储器。两种情况下,栏中的寄存器部分可以是%rax、%eax、%ax 或%al,并且它们可以互不相同。

回忆 2.2.6 节,当执行的强制类型转换既改变大小、又改变 C 语言中的“符号性”(signedness)时,操作应当先改变大小。

src_t	dest_t	指令
long	long	movq (%rdi), %rax movq %rax, (%rsi)
char	int	
unsigned char	long	
int	char	
unsigned	unsigned char	
char	short	

练习 3.5　给定如下信息。一个函数原型为:

```
void decode1(long *xp, long *yp, long *zp);
```

编译成汇编代码后得到如下代码:

```
void decode1(long *xp, long *yp, long *zp)
xp in %rdi, yp in %rsi, zp in %rdx
decode1:
  movq (%rdi), %r8
  movq (%rsi), %rcx
```

给 C 语言初学者　指针示例

函数 exchange(图 3.7(a))很好地说明了 C 中指针的用法。参数 xp 是一个指向 long 整数的指针,而 y 本身是一个 long 整数。语句

```
long x = *xp;
```

表示我们应该读出存储在 xp 所指位置上的值,把它作为名为 x 的局部变量保存。这个读操作称为指针解引用(pointer dereferencing)。C 运算符 * 执行指针解引用。语句

```
*xp = y;
```

做的是相反的事——它把参数 y 的值写到 xp 所指的位置上。这也是一种指针解引用(因此才有运算符 *),但它指示的是写操作,因为它出现在赋值的左边。

下面的例子展示了 exchange 在起作用:

```
long a = 4;
long b = exchange(&a, 3);
printf("a = %ld, b = %ld\n", a, b);
```

这段代码将打印出

```
a = 3, b = 4
```

C 运算符 &(称为“取地址”运算符)创建一个指针,这里指向保存局部变量 a 的那个位置。函数 exchange 把 a 中保存的值改写为 3,但把原来的值 4 作为函数值返回。请注意,通过给 exchange 传递一个指针,它就能修改存放在别处某些位置上的数据。

```
movq (%rdx), %rax
movq %r8, (%rsi)
movq %rcx, (%rdx)
movq %rax, (%rdi)
ret
```

参数 xp、yp 和 zp 分别存放在寄存器%rdi、%rsi 和%rdx 中。

为 decode1 编写 C 代码，使其效果与给出的汇编代码等价。

3.7.4 压入和弹出栈数据

最后两条数据传送操作用于把数据压入程序栈和从程序栈里弹出数据，如图 3.8 所示。我们将看到，栈在处理过程调用时起着至关重要的作用。先交代一点背景：栈是一种数据结构，可以对它添加或删除值，但只能按照“后进、先出”的原则进行。我们通过压入（push）操作把数据加入栈，通过弹出（pop）操作把数据从栈中移除，其性质是弹出的值总是最近压入且仍在栈中的那个值。栈可以用数组来实现，总是在数组的一端插入和删除元素。这一端称为栈顶。x86-64 中，程序栈存放在存储器的某个区域。如图 3.9 所示，栈向下增长，使栈顶元素的地址成为所有栈元素中最低的。（按照惯例，我们把栈画成倒置的，栈“顶”显示在图的底部。）栈指针%rsp 保存栈顶元素的地址。

图 3.8 压入和弹出指令。

指令	效果	描述
pushq S	$R[\texttt{\%rsp}] \leftarrow R[\texttt{\%rsp}] - 8; M[R[\texttt{\%rsp}]] \leftarrow S$	压入四字
popq D	$D \leftarrow M[R[\texttt{\%rsp}]]; R[\texttt{\%rsp}] \leftarrow R[\texttt{\%rsp}] + 8$	弹出四字

```
%rax
%rdx
%rsp
0x108
0
0x123
0x108
%rax
%rdx
%rsp
0x108
0x100
0
0x123
0x100
%rax
%rdx
%rsp
0x123
```

```
0x123
pushq %rax
popq %rdx
0x108
Initially
Stack “bottom”
Increasing
address
Stack “top”
Stack “bottom”
0x123
0x123
Stack “top”
Stack “top”
0x108
Stack “bottom”
Figure 3.9
Illustration of stack operation. By convention,
      we draw stacks upside down,
so that the “top” of the stack is shown at the bottom. With x86-64,
      stacks grow toward
lower addresses, so pushing involves decrementing the stack pointer
        (register %rsp) and
storing to memory, while popping involves reading from memory and
        incrementing the
stack pointer.
```

pushq 指令提供把数据压入栈的能力，popq 指令则弹出数据。这两条指令各取一个操作数——压入时是数据源，弹出时是数据目的。

把一个四字值压入栈，要先把栈指针减 8，再把该值写到新的栈顶地址。因此，指令 pushq %rbp 的行为等价于下面这对指令：

```
subq $8,%rsp          栈指针减 8
movq %rbp,(%rsp)      把 %rbp 存到栈上
```

只不过 pushq 指令在机器码中只编码成一个字节，而上面所示的那对指令总共需要 8 个字节。图 3.9 的前两列说明了当%rsp 为 0x108、%rax 为 0x123 时执行指令 pushq %rax 的效果：先把%rsp 减 8，得到 0x100，然后把 0x123 存到存储器地址 0x100 处。

弹出一个四字值，要先从栈顶位置读出数据，再把栈指针加 8。因此，指令 popq %rax 等价于下面这对指令：

```
movq (%rsp),%rax      从栈中读出 %rax
addq $8,%rsp          栈指针加 8
```

图 3.9 的第三列说明了紧跟在 pushq 之后执行指令 popq %edx 的效果。值 0x123 从存储器读出并写入寄存器%rdx。寄存器%rsp 被加回到 0x108。如

图所示，值 0x123 会一直留在存储器位置 0x104 上，直到它被覆盖（例如再执行一次 push 操作）。不过，栈顶始终被认为就是%rsp 指示的地址。

由于栈与程序代码及其他形式的程序数据存放在同一个存储器中，程序可以用标准的存储器寻址方法访问栈中的任意位置。例如，假设栈顶元素是一个四字，指令 movq 8(%rsp),%rdx 会把栈中的第二个四字复制到寄存器%rdx。

3.8 算术和逻辑操作

图 3.10 列出了 x86-64 的部分整数运算和逻辑运算。大多数运算以指令类的形式给出，因为它们可以有针对不同操作数大小的各种变体。（只有 leaq 没有其他大小变体。）例如，add 指令类由四条加法指令组成：addb、addw、addl 和 addq，分别对字节、字、双字和四字求和。的确，图中给出的每个指令类都有针对这四种不同数据大小的指令。这些运算分为四组：加载有效地址、一元操作、二元操作和移位。二元操作有两个操作数，一元操作有一个操作数。这些操作数用与 3.4 节所述相同的记法来指定。

3.8.1 加载有效地址

加载有效地址（load effective address）指令 leaq 实际上是 movq 指令的一个变体。它具有从存储器读入寄存器的指令形式，但实际上它根本不访问存储器。它的第一个操作数看上去像是一个存储器引用，但指令并不是从指定的位置读取数据，而是把有效地址复制到目的。我们在图 3.10 中用 C 取地址运算符 &S 表示这一计算。这条指令可用于为后面的存储器引用生成指针。此外，它还能用来紧凑地描述常见的算术运算。例如，若寄存器%rdx 中保存着值 x，则指令 leaq 7(%rdx,%rdx,4), %rax 会把寄存器%rax 设置为 $5x+7$。编译器经常发现 leaq 的巧妙用法，这些用法与有效地址的计算毫无关系。目的操作数必须是一个寄存器。

练习 3.6 假设寄存器%rbx 保存值 p，%rdx 保存值 q。填写下表，给出对每条汇编代码指令，寄存器%rax 将保存的值的公式：

指令	结果
leaq 9(%rdx), %rax	
leaq (%rdx,%rbx), %rax	
leaq (%rdx,%rbx,3), %rax	
leaq 2(%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

为了说明编译后的代码中 leaq 的用法，考虑下面的 C 程序：

```
long scale(long x, long y, long z) {
```

图 3.10 整数算术操作。加载有效地址(leaq)指令常用于执行简单的算术运算。其余都是更常规的一元或二元操作。我们用记法 $>>_A$ 和 $>>_L$ 分别表示算术右移和逻辑右移。注意 ATT 格式汇编代码中操作数的次序并不直观。

指令	效果	描述
leaq S, D	$D \leftarrow \&S$	加载有效地址
inc D	$D \leftarrow D + 1$	加 1
dec D	$D \leftarrow D - 1$	减 1
neg D	$D \leftarrow -D$	取负
not D	$D \leftarrow \sim D$	取反
add S, D	$D \leftarrow D + S$	加
sub S, D	$D \leftarrow D - S$	减
imul S, D	$D \leftarrow D * S$	乘
xor S, D	$D \leftarrow D \hat{} S$	异或
or S, D	$D \leftarrow D \mid S$	或
and S, D	$D \leftarrow D \& S$	与
sal k, D	$D \leftarrow D << k$	左移
shl k, D	$D \leftarrow D << k$	左移(同 sal)
sar k, D	$D \leftarrow D >>_A k$	算术右移
shr k, D	$D \leftarrow D >>_L k$	逻辑右移

```
    long t = x + 4 * y + 12 * z;
    return t;
}
```

编译时,函数的算术运算由三条 leaq 指令组成的序列来实现,右侧的注释对此作了说明:

```
long scale(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
scale:
  leaq (%rdi,%rsi,4), %rax      x + 4*y
  leaq (%rdx,%rdx,2), %rdx      z + 2*z = 3*z
  leaq (%rax,%rdx,4), %rax      (x+4*y) + 4*(3*z) = x + 4*y + 12*z
  ret
```

leaq 指令能够执行加法和有限形式的乘法,在编译本例这类简单的算术表达式时十分有用。

练习 3.7 考虑下面的代码,其中我们略去了要计算的表达式:

```
short scale3(short x, short y, short z) {
  short t =          ;
  return t;
}
```

用 gcc 编译实际的函数,得到如下汇编代码:

```
short scale3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
scale3:
  leaq (%rsi,%rsi,9), %rbx
  leaq (%rbx,%rdx), %rbx
  leaq (%rbx,%rdi,%rsi), %rbx
  ret
```

在 C 代码中补全缺失的表达式。

3.8.2 一元和二元操作

第二组操作是一元操作,唯一的操作数既是源又是目的。这个操作数可以是寄存器,也可以是存储器位置。例如,指令 `incq (%rsp)` 会把栈顶的 8 字节元素加 1。这种语法让人想起 C 的增量(`++`)和减量(`--`)运算符。

第三组由二元操作组成,其中第二个操作数既用作源又用作目的。这种语法让人想起 C 的赋值运算符,例如 `x -= y`。不过要注意,源操作数在先,目的在后。对不满足交换律的运算来说,这种次序看起来有些怪。例如,指令 `subq %rax,%rdx` 是把寄存器`%rdx` 按`%rax` 中的值减小。(把它读作 “从%rdx 中减去%rax”会有帮助。)第一个操作数可以是立即数、寄存器或存储器位置;第二个操作数可以是寄存器或存储器位置。与 `mov` 指令一样,两个操作数不能都是存储器位置。注意,当第二个操作数是存储器位置时,处理器必须先从存储器中读出值,执行操作,然后再把结果写回存储器。

练习 3.8 假设下列值存放在指定的存储器地址和寄存器中:

地址	值	寄存器	值
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

填写下表,说明下面各指令的效果,既要给出将被更新的寄存器或存储器位置,也要给出得到的值:

指令	目的	值
`addq %rcx,(%rax)`		
`subq %rdx,8(%rax)`		
`imulq $16,(%rax,%rdx,8)`		
`incq 16(%rax)`		
`decq %rcx`		
`subq %rdx,%rax`		

3.8.3 移位操作

最后一组是移位操作，移位量在先，被移位的值在后。算术右移和逻辑右移都是可能的。不同的移位指令可以用立即数指定移位量，也可以用单字节寄存器`%cl` 指定。（这些指令的特殊之处在于，只允许用这一个特定寄存器作为操作数。）原则上，使用 1 字节的移位量可以编码的移位量范围最大可达 $2^8 - 1 = 255$。在 x86-64 中，对 w 位长数据进行操作的移位指令，通过寄存器`%cl` 的低 m 位确定移位量，其中 $2^m = w$。高位被忽略。例如，当寄存器`%cl` 的十六进制值为 `0xFF` 时，指令 `salb` 的移位量为 7，`salw` 为 15，`sall` 为 31，而 `salq` 为 63。

如图 3.10 所示，左移指令有两个名字：`sal` 和 `shl`。二者效果相同，从右边补零。右移指令则有所不同：`sar` 执行算术移位（补入符号位的副本），而 `shr` 执行逻辑移位（补入零）。移位操作的目的操作数可以是寄存器，也可以是存储器位置。我们在图 3.10 中把两种右移记作 $>>_A$（算术）和 $>>_L$（逻辑）。

练习 3.9 假设我们要为下面的 C 函数生成汇编代码：

```
long shift_left4_rightn(long x, long n)
{
    x <<= 4;
    x >>= n;
    return x;
}
```

下面的代码是完成具体移位并把最终值保留在寄存器`%rax` 中的那部分汇编代码。其中省略了两条关键指令。参数 x 和 n 分别存放在寄存器`%rdi` 和`%rsi` 中。

```
long shift_left4_rightn(long x, long n)
x in %rdi, n in %rsi
shift_left4_rightn:
  movq %rdi, %rax          取出 x
                           x <<= 4
  movl %esi, %ecx          取出 n(4 字节)
                           x >>= n
```

按照右侧注释填入缺失的指令。右移应当按算术移位执行。

3.8.4 讨论

可以看到，图 3.10 中列出的大多数指令既可以用于无符号算术，也可以用于补码算术。只有右移需要用不同的指令来区分有符号数据与无符号数据。这也是使补码算术成为实现有符号整数运算首选方式的特性之一。

图 3.11 给出了一个执行算术运算的函数及其翻译成汇编代码的例子。参数 x、y 和 z 初始时分别保存在寄存器`%rdi`、`%rsi` 和`%rdx` 中。汇编代码中的指

令与 C 源代码的各行紧密对应。第 2 行计算 $x \hat{} y$ 的值。第 3 行和第 4 行用 `leaq` 指令与移位指令的组合计算表达式 `z*48`。第 5 行计算 `t1` 与 `0x0F0F0F0F` 的与。最后的减法由第 6 行计算。由于减法的目的寄存器是`%rax`,它将是该函数返回的值。

在图 3.11 的汇编代码中，寄存器`%rax` 中的值的序列对应于程序值 `3*z`、`z*48` 和 `t4`(作为返回值)。一般而言,编译器生成的代码会用同一个寄存器承载多个程序值,并在寄存器之间移动这些程序值。

```
(a) C 代码
long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}
```

```
(b) 汇编代码
        long arith(long x, long y, long z)
        x in %rdi, y in %rsi, z in %rdx
1  arith:
2  xorq  %rsi, %rdi             t1 = x ^ y
3  leaq  (%rdx,%rdx,2), %rax    3*z
4  salq  $4, %rax               t2 = 16 * (3*z) = 48*z
5  andl  $252645135, %edi       t3 = t1 & 0x0F0F0F0F
6  subq  %rdi, %rax             返回 t2 - t3
7  ret
```

图 3.11 `arith` 函数的 C 代码与汇编代码。

练习 3.10 考虑下面的代码,其中我们省略了要计算的表达式:

```
short arith3(short x, short y, short z)
{
    short p1 = ________;
    short p2 = ________;
    short p3 = ________;
    short p4 = ________;
    return p4;
}
```

生成的汇编代码中实现这些表达式的部分如下:

```
short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:
  orq   %rsi, %rdx
  sarq  $9, %rdx
  notq  %rdx
```

```
movq  %rdx, %bax
subq  %rsi, %rbx
ret
```

根据这段汇编代码,填写 C 代码中缺少的部分。

练习 3.11 在由 C 代码生成、而 C 里并不包含异或运算的代码中,经常会见到如下形式的汇编代码行:

```
xorq %rcx,%rcx
```

- **A.** 解释这条特定的异或指令的效果,并说明它实现的是一种什么有用的运算。
- **B.** 在汇编代码中表达这个运算,更直接的做法是什么?
- **C.** 比较对同一个运算的这三种不同实现中任意两种进行编码所需的字节数。

3.8.5 特殊的算术操作

正如我们在 2.3 节中看到的,两个 64 位有符号或无符号整数相乘,得到的乘积可能需要 128 位才能表示。x86-64 指令集对涉及 128 位(16 字节)数的运算提供了有限的支持。沿用“字”(2 字节)、“双字”(4 字节)和“四字”(8 字节)的命名惯例,Intel 把 16 字节的量称为八字(*oct word*)。图 3.12 描述了支持生成两个 64 位数的完整 128 位乘积的指令,以及支持整数除法的指令。

图 3.12 特殊算术操作。这些操作对有符号数和无符号数都提供完整的 128 位乘法与除法。寄存器对%rdx 和%rax 被看作构成一个单个的 128 位八字。

指令	效果	描述
`imulq` S	`R[%rdx]:R[%rax]` $\leftarrow S \times$ `R[%rax]`	有符号完整乘法
`mulq` S	`R[%rdx]:R[%rax]` $\leftarrow S \times$ `R[%rax]`	无符号完整乘法
`cqto`	`R[%rdx]:R[%rax]` $\leftarrow$ SignExtend(`R[%rax]`)	转换成八字
`idivq` S	`R[%rdx]` $\leftarrow$ `R[%rdx]:R[%rax]` mod S; `R[%rax]` $\leftarrow$ `R[%rdx]:R[%rax]` $\div S$	有符号除法
`divq` S	`R[%rdx]` $\leftarrow$ `R[%rdx]:R[%rax]` mod S; `R[%rax]` $\leftarrow$ `R[%rdx]:R[%rax]` $\div S$	无符号除法

`imulq` 指令有两种不同的形式。图 3.10 所示的一种形式是 `imul` 指令类的一员。在这种形式中,它充当一条“双操作数”乘法指令,由两个 64 位操作数生成一个 64 位的乘积。它实现的是 2.3.4 节和 2.3.5 节中描述的 $*^u_{64}$ 和 $*^t_{64}$ 运算。(回忆一下,把乘积截断到 64 位时,无符号乘法与补码乘法具有相同的位级行为。)

此外，x86-64 指令集还包括两条不同的“单操作数”乘法指令，用来计算两个 64 位值的完整 128 位乘积——一条用于无符号乘法（`mulq`），一条用于补码乘法（`imulq`）。对这两条指令来说，一个参数必须在寄存器`%rax` 中，另一个则作为指令的源操作数给出。乘积随后保存在寄存器`%rdx`（高 64 位）和`%rax`（低 64 位）中。虽然 `imulq` 这个名字用于两种不同的乘法运算，但汇编器可以通过数一数操作数的个数来判断用的是哪一个。

作为一个例子，下面的 C 代码演示了两个无符号 64 位数 x 和 y 的 128 位乘积的生成：

```
#include <inttypes.h>

typedef unsigned __int128 uint128_t;

void store_uprod(uint128_t *dest, uint64_t x, uint64_t y) {
    *dest = x * (uint128_t) y;
}
```

在这个程序中，我们利用文件 `inttypes.h` 中声明的定义（作为 C 标准的一个扩展的一部分），显式地把 x 和 y 声明为 64 位数。遗憾的是，该标准并没有为 128 位的值做出规定。因此我们转而依赖 `gcc` 对 128 位整数的支持，用名字 `__int128` 来声明。我们的代码使用一条 `typedef` 声明定义了数据类型 `uint128_t`，仿照 `inttypes.h` 中其他数据类型的命名模式。代码指定把得到的乘积存储在指针 `dest` 指定的 16 个字节中。

`gcc` 为这个函数生成的汇编代码如下：

```
void store_uprod(uint128_t *dest, uint64_t x, uint64_t y)
dest in %rdi, x in %rsi, y in %rdx
1  store_uprod:
2  movq  %rsi, %rax      把 x 复制到被乘数
3  mulq  %rdx            乘以 y
4  movq  %rax, (%rdi)    在 dest 处存储低 8 字节
5  movq  %rdx, 8(%rdi)   在 dest+8 处存储高 8 字节
6  ret
```

可以看到，存储这个乘积需要两条 `movq` 指令：一条用于低 8 字节（第 4 行），一条用于高 8 字节（第 5 行）。由于代码是为小端法机器生成的，高位字节存放在较高的地址，如地址说明 `8(%rdi)` 所示。

我们前面给出的算术运算表（图 3.10）没有列出任何除法或取模运算。这些运算由单操作数除法指令提供，与单操作数乘法指令类似。有符号除法指令 `idivl` 把寄存器`%rdx`（高 64 位）和`%rax`（低 64 位）中的 128 位数作为被除数。除数作为指令的操作数给出。该指令把商存储在寄存器`%rax` 中，把余数存储在寄存器`%rdx` 中。

对 64 位加法的大多数应用来说，被除数是以 64 位值的形式给出的。这个值应当存储在寄存器%rax 中。%rdx 的各位则应当设置为全 0(无符号算术)，或者设置为%rax 的符号位(有符号算术)。后一种运算可以用指令 cqto 来完成。[2]这条指令不取操作数——它隐式地从%rax 读出符号位，并将它复制到%rdx 的所有位。

为了说明用 x86-64 实现除法，下面的 C 函数计算两个 64 位有符号数的商和余数：

```
void remdiv(long x, long y,
            long *qp, long *rp) {
    long q = x/y;
    long r = x%y;
    *qp = q;
    *rp = r;
}
```

它编译成下面的汇编代码：

```
void remdiv(long x, long y, long *qp, long *rp)
x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1  remdiv:
2  movq  %rdx, %r8       复制 qp
3  movq  %rdi, %rax      把 x 移到被除数的低 8 字节
4  cqto                  符号扩展到被除数的高 8 字节
5  idivq %rsi            除以 y
6  movq  %rax, (%r8)     把商存储到 qp
7  movq  %rdx, (%rcx)    把余数存储到 rp
8  ret
```

在这段代码中，参数 rp 必须先保存到另一个寄存器(第 2 行)，因为进行除法运算需要用到参数寄存器%rdx。第 3 ~ 4 行随后通过复制并符号扩展 x 来准备被除数。除法之后，寄存器%rax 中的商被存储到 qp(第 6 行)，而寄存器%rdx 中的余数被存储到 rp(第 7 行)。

无符号除法使用 divq 指令。这时通常会把寄存器%rdx 预先设置为 0。

练习 3.12 考虑下面这个计算两个无符号 64 位数的商和余数的函数：

```
void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp) {
    unsigned long q = x/y;
    unsigned long r = x%y;
    *qp = q;
    *rp = r;
}
```

修改所示的有符号除法的汇编代码，来实现这个函数。

[2]这条指令在 Intel 文档中称为 cqo，这是少有的几种 ATT 格式的指令名与 Intel 名字不匹配的情况之一。

3.9 控制

到目前为止，我们只考虑过直线式代码（straight-line code）的行为，其中的指令一条接一条依次执行。C 语言中的一些结构，比如条件语句、循环和 `switch` 语句，要求有条件执行，此时要执行的操作序列取决于对数据应用测试的结果。机器代码提供了两种基本的低级机制来实现条件行为：测试数据值，然后根据测试的结果改变控制流或数据流。

依赖于数据的控制流是实现条件行为更普遍也更常用的方式，所以我们首先考察这种方式。通常，C 语句和机器代码指令都是顺序执行的，即按照它们在程序中出现的顺序执行。机器代码指令的执行顺序可以用跳转（*jump*）指令改变，表明控制应传递给程序的某个其他部分，且这种传递可能有赖于某个测试的结果。编译器必须围绕这一低级机制生成指令序列，来实现 C 的各种控制结构。

在下面的讲述中，我们先介绍实现条件操作的两种方式，然后再描述表示循环和 `switch` 语句的方法。

3.9.1 条件码

除了整数寄存器，CPU 还维护着一组单个位的条件码（*condition code*）寄存器，用来描述最近的算术或逻辑运算的属性。可以通过测试这些寄存器来执行条件分支。最有用的条件码是：

- `CF` 进位标志（carry flag）。最近的操作使最高有效位产生了进位。用于检测无符号运算的溢出。
- `ZF` 零标志（zero flag）。最近的操作得出的结果为 0。
- `SF` 符号标志（sign flag）。最近的操作得出的结果为负值。
- `OF` 溢出标志（overflow flag）。最近的操作导致了补码溢出——正溢出或负溢出。

举个例子，假设我们用某条 `add` 指令完成等价于 C 赋值 `t = a+b` 的运算，其中变量 a、b 和 t 都是整数。那么条件码就会按照下面的 C 表达式来设置：

```
CF  (unsigned) t < (unsigned) a              无符号溢出
ZF  (t == 0)                                 零
SF  (t < 0)                                  负
OF  (a < 0 == b < 0) && (t < 0 != a < 0)     有符号溢出
```

指令 `leaq` 不改变任何条件码，因为它是用来进行地址计算的。除此之外，图 3.10 中列出的所有指令都会设置条件码。对于逻辑运算，比如异或，进位标志和溢出标志会被设置为 0。对于移位运算，进位标志设置为最后被移出的位，

而溢出标志设置为 0。出于我们不去深究的原因，inc 和 dec 指令设置溢出标志和零标志，但保持进位标志不变。

除了图 3.10 中的指令会设置条件码之外，还有两类指令（有 8 位、16 位、32 位和 64 位的形式）只设置条件码而不改变任何其他寄存器；它们列在图 3.13 中。cmp 指令根据两个操作数之差来设置条件码。它们的行为与 sub 指令相同，只是它们设置条件码而不更新目的寄存器。在 ATT 格式中，操作数的列出顺序是反的，使代码难以阅读。当两个操作数相等时，这些指令设置零标志，其他标志则可以用来确定两个操作数之间的大小关系。test 指令的行为与 and 指令相同，只是它们设置条件码而不改变目的操作数。典型的情况是，两个操作数相同（例如 testq %rax,%rax，用来检查%rax 是负数、零还是正数），或者其中一个操作数是指明要检查哪些位的掩码。

图 3.13 比较指令与测试指令。这些指令设置条件码，但不更新任何其他寄存器。

指令	依据	描述
cmp S_1, S_2	$S_2 - S_1$	比较
		比较字节
		比较字
		比较双字
		比较四字
test S_1, S_2	$S_1 \& S_2$	测试
		测试字节
		测试字
		测试双字
		测试四字

3.9.2 访问条件码

除了直接读取条件码之外，使用条件码有三种常见的方式：(1) 可以根据条件码的某种组合把一个字节设置为 0 或 1；(2) 可以有条件地跳转到程序的某个其他部分；(3) 可以有条件地传送数据。对于第一种情况，图 3.14 中描述的指令根据条件码的某种组合把单个字节设置为 0 或 1。我们把这一整类指令称为 set（设置）类指令；它们彼此的区别在于所考虑的条件码组合不同，这由指令名中不同的后缀来表明。必须认识到，这些指令的后缀表示的是不同的条件，而不是不同的操作数大小。例如，指令 setl 和 setb 代表的是“set less”和“set below”，而不是“set long word”或“set byte”。

一条 set 类指令的目的操作数或者是低位单字节寄存器元素（图 3.2）之一，或者是一个单字节的内存位置，指令把这个字节设置为 0 或 1。要生成 32

位或 64 位的结果，我们还必须清除高位。计算 C 表达式 $a < b$（其中 a 和 b 的类型都是 `long`）的典型指令序列如下：

图 3.14 `set` 类指令。每条指令根据条件码的某种组合把一个单个字节设置为 0 或 1。有些指令有“同义词”，即同一机器指令的其他名字。

指令	同义词	效果	设置条件
`sete` D	`setz`	$D \leftarrow$ `ZF`	相等 / 零
`setne` D	`setnz`	$D \leftarrow$ `~ZF`	不相等 / 非零
`sets` D		$D \leftarrow$ `SF`	负
`setns` D		$D \leftarrow$ `~SF`	非负
`setg` D	`setnle`	$D \leftarrow$ `~(SF ^ OF) & ~ZF`	大于（有符号 >）
`setge` D	`setnl`	$D \leftarrow$ `~(SF ^ OF)`	大于等于（有符号 >=）
`setl` D	`setnge`	$D \leftarrow$ `SF ^ OF`	小于（有符号 <）
`setle` D	`setng`	$D \leftarrow$ `(SF ^ OF) | ZF`	小于等于（有符号 <=）
`seta` D	`setnbe`	$D \leftarrow$ `~CF & ~ZF`	高于（无符号 >）
`setae` D	`setnb`	$D \leftarrow$ `~CF`	高于等于（无符号 >=）
`setb` D	`setnae`	$D \leftarrow$ `CF`	低于（无符号 <）
`setbe` D	`setna`	$D \leftarrow$ `CF | ZF`	低于等于（无符号 <=）

```
int comp(data_t a, data_t b)
a in %rdi, b in %rsi
1  comp:
2  cmpq   %rsi, %rdi     比较 a:b
3  setl   %al            将 %eax 的低位字节置为 0 或 1
4  movzbl %al, %eax      清除 %eax 的其余部分(以及 %rax 的其余部分)
5  ret
```

注意 `cmpq` 指令的比较顺序（第 2 行）。虽然参数列出的顺序是`%rsi`（b），然后是`%rdi`（a），但真正的比较是在 a 与 b 之间进行的。还要回忆一下，如 3.4.2 节所讨论的，`movzbl` 指令（第 4 行）清除的不仅是`%eax` 的高 3 个字节，还有整个寄存器`%rax` 的高 4 个字节。

对某些底层机器指令来说，存在多个可能的名字，我们把它们列为“同义词”。例如，`setg`（“set greater”）和 `setnle`（“set not less or equal”）指的是同一条机器指令。编译器和反汇编器会随意选择用哪个名字。

虽然所有的算术和逻辑运算都会设置条件码，但各个 `set` 类指令的描述适用于已经执行过了某条比较指令、并根据计算 `t = a-b` 设置了条件码的情形。更具体地说，设 a、b 和 t 分别是由变量 `a`、`b` 和 `t` 以补码形式表示的整数，于是 $t = a - {}^{t}_{w}b$，这里 w 取决于与 a 和 b 相关联的大小。

考虑 `sete`（“相等时设置”）指令。当 $a = b$ 时，我们有 $t = 0$，因此零标志指示了相等。同样，考虑用 `setl`（“小于时设置”）指令测试有符号比较。当没

有溢出发生(由 OF 置为 0 指示)时,若 $a - {}_w^t b < 0$(由 SF 置为 1 指示),我们有 $a < b$;而若 $a - {}_w^t b \geq 0$(由 SF 置为 0 指示),我们有 $a \geq b$。另一方面,当溢出发生时,若 $a - {}_w^t b > 0$(负溢出),我们有 $a < b$;而若 $a - {}_w^t b < 0$(正溢出),我们有 $a > b$。当 $a = b$ 时不可能发生溢出。因此,当 OF 置为 1 时,当且仅当 SF 置为 0 时才有 $a < b$。综合这些情形,溢出位和符号位的异或提供了对"是否 $a < b$"的一个测试。其他的有符号比较测试则基于 `SF ^ OF` 和 `ZF` 的其他组合。

对无符号比较进行测试时,我们改设 a 和 b 就是变量 `a` 和 `b` 以无符号形式表示的整数。在执行计算 `t = a-b` 时,当 $a - b < 0$ 时 `cmp` 指令会设置进位标志,因此无符号比较使用进位标志和零标志的组合。

机器代码如何区分、又如何不区分有符号值与无符号值,这一点很重要。与 C 不同,它不给每个程序值关联一个数据类型。相反,它对两种情形大多使用相同的指令,因为对无符号算术和补码算术来说,许多算术运算的位级行为是相同的。有些情形则需要对有符号运算和无符号运算使用不同的指令,比如使用不同的右移版本、不同的除法和乘法指令,以及不同的条件码组合。

练习 3.13 考虑下面的 C 代码:

```
int comp(data_t a, data_t b) {
    return a COMP b;
}
```

它展示了参数 `a` 与 `b` 的一个一般性比较,其中参数的数据类型 `data_t`(通过 `typedef` 定义)是图 3.1 中列出的整数数据类型之一,可以是有符号的,也可以是无符号的。比较运算 COMP 通过 `#define` 定义。

假设 `a` 位于`%rdx` 的某一部分中,而 `b` 位于`%rsi` 的某一部分中。对下面的每个指令序列,确定哪些数据类型 `data_t` 和哪些比较 COMP 可能使编译器生成这段代码。(可能有多个正确答案;你应当把它们全部列出来。)

```
A.  cmpl  %esi, %edi
    setl  %al

B.  cmpw  %si, %di
    setge %al

C.  cmpb  %sil, %dil
    setbe %al

D.  cmpq  %rsi, %rdi
    setne %a
```

练习 3.14 考虑下面的 C 代码:

```
int test(data_t a) {
    return a TEST 0;
}
```

它展示了参数 a 与 0 之间的一般性比较，其中我们可以通过用 typedef 声明 data_t 来设定参数的数据类型，通过用 #define 声明 TEST 来设定比较的性质。下面的指令序列实现了这个比较，其中 a 保存在寄存器%rdi 的某一部分中。对每个序列，确定哪些数据类型 data_t 和哪些比较 TEST 可能使编译器生成这段代码。(可能有多个正确答案；请列出所有正确的。)

```
A.  testq %rdi, %rdi
    setge %al

B.  testw %di, %di
    sete  %al

C.  testb %dil, %dil
    seta  %al

D.  testl %edi, %edi
    setle %al
```

3.9.3 跳转指令

正常执行时，指令按照列出的顺序一条接一条地执行。跳转指令可以让执行切换到程序中一个完全不同的位置。在汇编代码中，这些跳转目的地一般用一个标号(*label*)来指示。考虑下面这个(非常人为的)汇编代码序列：

```
movq $0,%rax       设 %rax 为 0
jmp .L1            转到 .L1
movq (%rax),%rdx   空指针解引用(跳过)
.L1:
popq %rdx          跳转目标
```

指令 jmp .L1 会使程序跳过 movq 指令，改为从 popq 指令恢复执行。在生成目标代码文件时，汇编器会确定所有带标号指令的地址，并把跳转目标(目标指令的地址)编码为跳转指令的一部分。

图 3.15 给出了各种不同的跳转指令。jmp 指令无条件跳转。它可以是直接跳转 (*direct jump*)，跳转目标编码在指令之内；也可以是间接跳转 (*indirect jump*)，跳转目标从寄存器或内存位置读出。直接跳转在汇编代码中通过给出一个标号作为跳转目标来书写，例如所示代码中的标号.L1。间接跳转用“*”后跟一个操作数指示符来书写，操作数指示符使用图 3.3 中描述的某种内存操作数格式。例如，指令

```
jmp *%rax
```

用寄存器%rax 中的值作为跳转目标，而指令

```
jmp *(%rax)
```

图 3.15 跳转指令。当跳转条件成立时，这些指令跳转到带标号的目的地。有些指令有“同义词”，即同一机器指令的其他名字。

指令	同义词	跳转条件	描述
`jmp` *Label*		1	直接跳转
`jmp` **Operand*		1	间接跳转
`je` *Label*	`jz`	ZF	相等 / 零
`jne` *Label*	`jnz`	~ZF	不相等 / 非零
`js` *Label*		SF	负
`jns` *Label*		~SF	非负
`jg` *Label*	`jnle`	~(SF ^ OF) & ~ZF	大于(有符号 >)
`jge` *Label*	`jnl`	~(SF ^ OF)	大于等于(有符号 >=)
`jl` *Label*	`jnge`	SF ^ OF	小于(有符号 <)
`jle` *Label*	`jng`	(SF ^ OF) | ZF	小于等于(有符号 <=)
`ja` *Label*	`jnbe`	~CF & ~ZF	高于(无符号 >)
`jae` *Label*	`jnb`	~CF	高于等于(无符号 >=)
`jb` *Label*	`jnae`	CF	低于(无符号 <)
`jbe` *Label*	`jna`	CF | ZF	低于等于(无符号 <=)

从内存读取跳转目标，以`%rax` 中的值作为读取地址。表中其余的跳转指令是条件跳转——它们要么跳转，要么继续执行代码序列中的下一条指令，取决于条件码的某种组合。这些指令的名字以及它们执行跳转的条件与 `set` 类指令相同（见图 3.14）。与 `set` 类指令一样，某些底层机器指令也有多个名字。条件跳转只能是直接跳转。

3.9.4 跳转指令的编码

大多数时候，我们并不关心机器代码的详细格式。不过，理解跳转指令的目标是如何编码的，在我们学习第 7 章的链接时将变得很重要。此外，它还有助于解读反汇编器的输出。在汇编代码中，跳转目标用符号标号书写。汇编器（以及后来的链接器）会生成跳转目标的正确编码。跳转有几种不同的编码，但其中一些最常用的是 PC 相对的（*PC relative*）。也就是说，它们编码的是目标指令的地址与紧跟在跳转之后的那条指令的地址之差。这些偏移量可以用 1、2 或 4 个字节来编码。第二种编码方式是给出一个“绝对”地址，用 4 个字节直接给出目标。汇编器和链接器会为跳转目的地选择合适的编码。

作为 PC 相对寻址的一个例子，下面给出一个函数的汇编代码，它是由编译文件 `branch.c` 生成的。它包含两个跳转：第 2 行的 `jmp` 指令向前跳到一个较高的地址，而第 7 行的 `jg` 指令向后跳到一个较低的地址。

```
1  movq   %rdi, %rax
```

```
2  jmp    .L2
3  .L3:
4  sarq   %rax
5  .L2:
6  testq  %rax, %rax
7  jg     .L3
8  rep; ret
```

汇编器生成的.o 格式的反汇编版本如下：

```
1   0:   48 89 f8    mov    %rdi,%rax
2   3:   eb 03       jmp    8 <loop+0x8>
3   5:   48 d1 f8    sar    %rax
4   8:   48 85 c0    test   %rax,%rax
5   b:   7f f8       jg     5 <loop+0x5>
6   d:   f3 c3       repz retq
```

在反汇编器生成的右侧注释中，第 2 行的跳转指令的跳转目标标示为 0x8，第 5 行的跳转指令的跳转目标标示为 0x5（反汇编器列出的所有数字都是十六进制的）。不过，观察这些指令的字节编码可以发现，第一条跳转指令的目标编码（在第二个字节中）是 0x03。把它加上 0x5——紧跟在跳转之后的那条指令的地址——就得到跳转目标地址 0x8，即第 4 行指令的地址。

> **边栏　指令 rep 和 repz 是做什么用的？**
> 上一节汇编代码的第 8 行包含指令组合 rep; ret。它们在反汇编代码（第 6 行）中呈现为 repz retq。可以推断 repz 是 rep 的同义词，就像 retq 是 ret 的同义词。查阅 Intel 和 AMD 文档中关于 rep 指令的说明，我们发现它通常用来实现重复的字符串操作 [3, 51]，在这里似乎完全不合时宜。这个谜团的答案可以在 AMD 给编译器编写者的指南 [1] 中找到。他们建议使用 rep 后跟 ret 的组合，避免让 ret 指令成为条件跳转指令的目的地。如果没有 rep 指令，jg 指令（汇编代码第 7 行）在分支不发生时就会继续执行 ret 指令。按照 AMD 的说法，当 ret 指令是通过一条跳转指令到达时，他们的处理器无法正确预测 ret 指令的目的地。这里的 rep 指令起到一种空操作的作用，因此把它插入作为跳转目的地并不改变代码的行为，只会让代码在 AMD 处理器上运行得更快。对本书其余部分出现的 rep 或 repz 指令，我们都可以放心地忽略。

类似地，第二条跳转指令的目标编码是 0xf8（十进制 −8），用的是单字节补码表示。把它加上 0xd（十进制 13），即第 6 行指令的地址，就得到 0x5，即第 3 行指令的地址。

正如这些例子所说明的，执行 PC 相对寻址时，程序计数器的值是紧跟在跳转之后的那条指令的地址，而不是跳转指令本身的地址。这一惯例可以追溯到早期实现，那时处理器执行指令的第一步就是更新程序计数器。

下面是程序链接之后的反汇编版本：

```
1   4004d0:   48 89 f8    mov    %rdi,%rax
2   4004d3:   eb 03       jmp    4004d8 <loop+0x8>
3   4004d5:   48 d1 f8    sar    %rax
4   4004d8:   48 85 c0    test   %rax,%rax
5   4004db:   7f f8       jg     4004d5 <loop+0x5>
6   4004dd:   f3 c3       repz retq
```

这些指令被重定位到了不同的地址,但第 2 行和第 5 行中跳转目标的编码保持不变。通过对跳转目标使用 PC 相对编码,这些指令可以紧凑地编码(只需 2 字节),而且目标代码可以不加改变地移到内存中的不同位置。

练习 3.15 在下面从反汇编得到的二进制文件中摘取的片段里,一些信息已被 X 代替。请回答与这些指令有关的下列问题。

A. 下面这条 `je` 指令的目标是什么?(这里你不需要知道 `callq` 指令的任何细节。)

```
4003fa: 74 02    je      XXXXXX
4003fc: ff d0    callq   *%rax
```

B. 下面这条 `je` 指令的目标是什么?

```
40042f: 74 f4    je      XXXXXX
400431: 5d       pop     %rbp
```

C. `ja` 和 `pop` 指令的地址是什么?

```
XXXXXX: 77 02    ja      400547
XXXXXX: 5d       pop     %rbp
```

D. 在下面的代码中,跳转目标以 PC 相对形式编码为一个 4 字节的补码数。这些字节按从最低有效字节到最高有效字节的顺序列出,这反映了 x86-64 的小端法字节顺序。跳转目标的地址是什么?

```
4005e8: e9 73 ff ff ff    jmpq    XXXXXXX
4005ed: 90                nop
```

跳转指令为实现条件执行 (`if`) 以及若干种不同的循环结构提供了一种手段。

3.9.5 用条件控制来实现条件分支

把 C 中的条件表达式和语句翻译成机器代码, 最通用的方式是使用条件跳转和无条件跳转的组合。(另一种方式我们会看到, 如 3.6.6 节所述, 某些条件可以用条件传送数据而不是控制来实现。) 作为例子, 图 3.16(a) 给出了一个

函数的 C 代码，它计算两个数之差的绝对值。[3]该函数还有一个副作用，即递增两个计数器之一，这两个计数器编码为全局变量 `lt_cnt` 和 `ge_cnt`。`gcc` 生成的汇编代码如图 3.16(c) 所示。我们把机器代码改写成的 C 版本如函数 `gotodiff_se` 所示（图 3.16(b)）。它使用了 C 中的 `goto` 语句，类似于汇编代码中的无条件跳转。使用 `goto` 语句通常被认为是糟糕的编程风格，因为它们会使代码非常难以阅读和调试。在我们的讲述中，我们用它们来构造描述机器代码控制流的 C 程序。我们把这种风格的程序设计称为“*goto* 代码”。

(a) 原始的 C 代码

```
long lt_cnt = 0;
long ge_cnt = 0;

long absdiff_se(long x, long y)
{
    long result;
    if (x < y) {
        lt_cnt++;
        result = y - x;
    }
    else {
        ge_cnt++;
        result = x - y;
    }
    return result;
}
```

(b) 等价的 goto 版本

```
1   long gotodiff_se(long x, long y)
2   {
3       long result;
4       if (x >= y)
5           goto x_ge_y;
6       lt_cnt++;
7       result = y - x;
8       return result;
9     x_ge_y:
10      ge_cnt++;
11      result = x - y;
12      return result;
13  }
```

(c) 生成的汇编代码

```
        long absdiff_se(long x, long y)
        x in %rdi, y in %rsi
1   absdiff_se:
2     cmpq  %rsi, %rdi          比较 x:y
```

[3]实际上，如果某一次减法发生溢出，它可能返回一个负值。我们关注的是演示机器代码，而不是编写健壮的代码。

```
3   jge   .L2                 如果 >= 则转到 x_ge_y
4   addq  $1, lt_cnt(%rip)    lt_cnt++
5   movq  %rsi, %rax
6   subq  %rdi, %rax          result = y - x
7   ret                       返回
8  .L2:                       x_ge_y:
9   addq  $1, ge_cnt(%rip)    ge_cnt++
10  movq  %rdi, %rax
11  subq  %rsi, %rax          result = x - y
12  ret                       返回
```

图 3.16 条件语句的编译。(a) C 过程 `absdiff_se` 包含一条 if-else 语句。生成的汇编代码如 (c) 所示，同时给出了 (b) 一个模仿汇编代码控制流的 C 过程 `gotodiff_se`。

在 *goto* 代码（图 3.16(b)）中，第 5 行的语句 `goto x_ge_y` 导致跳转到第 9 行的标号 `x_ge_y`（因为它在 $x \geq y$ 时出现）。从这个点继续执行，它会完成 `absdiff_se` 函数的 else 部分所指定的计算并返回。另一方面，如果测试 `x >= y` 失败，程序过程会执行 `absdiff_se` 的 if 部分所指定的步骤并返回。

边栏 用 C 代码描述机器代码

图 3.16 展示了我们如何演示把 C 语言控制结构翻译成机器代码的一个例子。该图包含一个示例 C 函数 (a)，以及 `gcc` 生成的汇编代码的一个带注释的版本 (c)。它还包含一个与汇编代码结构非常接近的 C 版本 (b)。虽然这些版本是按照 (a)、(c)、(b) 的顺序生成的，但我们建议按 (a)、(b)，再 (c) 的顺序来阅读它们。也就是说，机器代码的 C 版本会帮助你理解要点，从而引导你去理解实际的汇编代码。

汇编代码实现（图 3.16(c)）首先比较两个操作数（第 2 行），设置条件码。如果比较结果指示 x 大于或等于 y，它就跳转到从第 8 行开始的代码块，该代码块递增全局变量 `ge_cnt`，计算 `x-y` 作为返回值，然后返回。否则，它继续执行从第 4 行开始的代码，该代码递增全局变量 `lt_cnt`，计算 `y-x` 作为返回值，然后返回。由此可以看出，为 `absdiff_se` 生成的汇编代码的控制流与 `gotodiff_se` 的 *goto* 代码非常贴近。

C 中 if-else 语句的一般形式由下面的模板给出：

```
if (test-expr)
    then-statement
else
    else-statement
```

其中 *test-expr* 是一个整数表达式，求值的要么是 0（解释为“假”），要么是一个非零值（解释为“真”）。两个分支语句（*then-statement* 或 *else-statement*）只有其中一个会被执行。

对这种一般形式，汇编实现通常遵循下面的形式，其中我们用 C 语法来描述控制流：

```
t = test-expr;
if (!t)
    goto false;
then-statement
goto done;
false:
else-statement
done:
```

也就是说，编译器为 *then* 语句和 *else* 语句分别生成独立的代码块，并插入条件分支和无条件分支，确保执行正确的代码块。

练习 3.16 给定如下 C 代码：

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

gcc 生成的汇编代码如下：

```
  void cond(short a, short *p)
  a 在 %rdi 中,p 在 %rsi 中
cond:
  testq    %rdi, %rdi
  je       .L1
  cmpq     %rsi, (%rdi)
  jle      .L1
  movq     %rdi, (%rsi)
.L1:
  rep; ret
```

A. 按照图 3.16(b) 的风格，写出一个完成同样计算、并模仿该汇编代码的控制流的 C 语言 goto 版本。你会发现，先像我们的例子中那样给汇编代码加上注释，是很有帮助的。

B. 解释为什么 C 代码只有一条 if 语句，汇编代码却包含两个条件分支。

练习 3.17 把 if 语句翻译成 goto 代码的另一条规则如下：

```
    t = test-expr;
    if (t)
        goto true;
    else-statement
    goto done;
true:
    then-statement
done:
```

A. 按这条替代规则改写 `absdiff_se` 的 goto 版本。

B. 你能想出选择一种规则而不是另一种的理由吗?

练习 3.18 从如下形式的 C 代码开始:

```
short test(short x, short y, short z) {
    short val = ________;
    if (________) {
        if (________)
            val = ________;
        else
            val = ________;
    } else if (________)
        val = ________;
    return val;
}
```

gcc 生成的汇编代码如下:

```
  short test(short x, short y, short z)
  x 在 %rdi 中,y 在 %rsi 中,z 在 %rdx 中
test:
  leaq     (%rdx,%rsi), %rax
  subq     %rdi, %rax
  cmpq     $5, %rdx
  jle      .L2
  cmpq     $2, %rsi
  jle      .L3
  movq     %rdi, %rax
  idivq    %rdx, %rax
  ret
.L3:
  movq     %rdi, %rax
  idivq    %rsi, %rax
  ret
.L2:
  cmpq     $3, %rdx
  jge      .L4
  movq     %rdx, %rax
  idivq    %rsi, %rax
.L4:
  rep; ret
```

填写 C 代码中缺失的表达式。

3.9.6 用条件传送实现条件分支

实现条件操作的传统方法是控制的条件传送(conditional transfer of control):条件成立时,程序沿一条执行路径进行;条件不成立时,则沿另一条路径进行。这种机制简单而通用,但在现代处理器上可能非常低效。

另一种策略是数据的条件传送(conditional transfer of data)。这种方法把条件操作的两个结果都计算出来,然后根据条件是否成立从中选取一个。这种策

略只在受限的情形下才有意义，但一旦适用，它可以用一条简单的**条件传送指令**（conditional move instruction）来实现，而这类指令与现代处理器的性能特性配合得好得多。本小节考察这一策略及其在 x86-64 上的实现。

图 3.17(a) 给出了一个可以用条件传送来编译的代码示例。这个函数同我们先前的示例（图 3.16）一样，计算参数 x 与 y 的绝对值。先前的示例在分支中含有副作用，会修改 `lt_cnt` 或 `ge_cnt` 的值；而这个版本只是计算函数要返回的值。

（a）原始的 C 代码

```
long absdiff(long x, long y)
{
    long result;
    if (x < y)
        result = y - x;
    else
        result = x - y;
    return result;
}
```

（b）使用条件赋值的实现

```
1  long cmovdiff(long x, long y)
2  {
3      long rval = y-x;
4      long eval = x-y;
5      long ntest = x >= y;
6      /* 下面这一行必须
7         编译成单条指令: */
8      if (ntest) rval = eval;
9      return rval;
10 }
```

（c）生成的汇编代码

```
   long absdiff(long x, long y)
   x 在 %rdi 中,y 在 %rsi 中
1  absdiff:
2    movq    %rsi, %rax
3    subq    %rdi, %rax              rval = y-x
4    movq    %rdi, %rdx
5    subq    %rsi, %rdx              eval = x-y
6    cmpq    %rsi, %rdi              比较 x:y
7    cmovge  %rdx, %rax              若 >=,rval = eval
8    ret                             返回 tval
```

图 3.17 使用条件赋值编译条件语句。C 函数 `absdiff`（(a)）包含一个条件表达式。汇编代码如 (c) 所示，同时给出的还有模仿汇编代码操作的 C 函数 `cmovdiff`（(b)）。

对这个函数，gcc 生成的汇编代码如图 3.17(c) 所示，其近似形式就是图 3.17(b) 中的 C 函数 `cmovdiff`。研究这个 C 版本可以看出：它把 `y-x` 和 `x-y` 都计算出来，分别命名为 `rval` 和 `eval`；接着测试 x 是否大于等于 y，如果是，就在返回 `rval` 之前把 `eval` 复制到 `rval`。图 3.17(c) 中的汇编代码遵循同样的逻辑。关键在于：汇编代码中单独一条 `cmovge` 指令（第 7 行）实现了 `cmovdiff` 的条件赋值（第 8 行）；只有当第 6 行的 `cmpq` 指令表明一个值大于等于另一个值时（后缀 `ge` 即此含意），它才把数据从源寄存器传送到目的寄存器。

要理解为什么基于条件数据传送的代码能胜过基于条件控制传送的代码（如图 3.16），我们得先了解一些现代处理器的工作方式。正如第 4 章和第 5 章将要讲到的，处理器通过流水线化（pipelining）来获得高性能：一条指令的处理要经过一系列阶段，每个阶段完成所需操作中的一小部分（例如，从存储器取出指令、确定指令类型、从存储器读数、执行算术运算、向存储器写数，以及更新程序计数器）。这种方法靠相邻指令的步骤重叠来获得高性能，比如在为一条指令执行算术运算的同时去取下一条指令。要做到这一点，就必须能够相当提前地确定即将执行的指令序列，让流水线中始终装满待执行的指令。当机器遇到一个条件跳转（称为“分支”）时，它先要求出分支条件的值，才能确定分支会走向哪边。处理器用复杂的分支预测（branch prediction）逻辑来猜测每条跳转指令是否会被执行。只要能可靠地猜对（现代微处理器设计力求达到约 90% 的成功率），指令流水线就会一直装满指令。相反，一旦对跳转的预测出错，处理器就得丢弃已经对后续指令做完的大部分工作，再开始从正确的位置取指令来重新填满流水线。我们会看到，一次预测错误会带来严重的损失，例如浪费 15 ~ 30 个时钟周期的努力，导致程序性能严重下降。

作为一个例子，我们在 Intel Haswell 处理器上对 `absdiff` 函数计时，比较实现条件操作的两种方法。在典型的应用中，测试 `x < y` 的结果高度不可预测，因此即使是最精巧的分支预测硬件，猜对的概率也只有大约 50%。此外，两种代码序列所做的计算各自只需要一个时钟周期。于是，分支预测错误的损失支配着这个函数的性能。对于使用条件跳转的 x86-64 代码，我们测得：分支模式容易预测时，每次调用约需 8 个时钟周期；分支模式随机时，每次调用约需 17.50 个时钟周期。由此可以推断，预测错误的损失约为 19 个时钟周期。这意味着，视分支是否被正确预测而定，该函数所需时间在约 8 个周期到 27 个周期之间。

> **边栏　这个损失是怎么得出的？**
> 假设预测错误的概率为 p，无误预测时执行代码所需的时间为 T_{OK}，预测错误的损失为 T_{MP}。那么，执行代码的平均时间作为 p 的函数为 $T_{\mathrm{avg}}(p) = (1-p)T_{\mathrm{OK}} +$

> $p(T_{OK}+T_{MP})=T_{OK}+pT_{MP}$。已知 T_{OK} 和 T_{ran}（即 $p=0.5$ 时的平均时间），要求出 T_{MP}。代入公式得 $T_{ran}=T_{avg}(0.5)=T_{OK}+0.5T_{MP}$，因此 $T_{MP}=2(T_{ran}-T_{OK})$。取 $T_{OK}=8$、$T_{ran}=17.5$，得到 $T_{MP}=19$。

另一方面，用条件传送编译的代码无论被测试的数据是什么都只需要约 8 个时钟周期。控制流不依赖于数据，这使处理器更容易保持流水线充满。

练习 3.19 在一种新的处理器型号上运行，我们的代码在分支模式随机时约需 45 个周期，分支模式高度可预测时约需 25 个周期。

A. 预测错误损失大约是多少？

B. 当分支被错误预测时，该函数需要多少个周期？

图 3.18 列出了 x86-64 提供的一些条件传送指令。每条指令有两个操作数：源寄存器或存储器位置 S，以及目的寄存器 R。与不同的 set 指令（3.6.2 节）和跳转指令（3.6.3 节）一样，这些指令的执行结果取决于条件码的值。源值从存储器或源寄存器读出，但只有当指定的条件成立时才把它复制到目的寄存器。

源值和目的值的长度可以是 16、32 或 64 位；单字节的条件传送不受支持。无条件传送指令把操作数长度明确地编码在指令名里（例如 `movw` 和 `movl`）；条件传送指令则不同，汇编器可以根据目的寄存器的名字推断出操作数长度，因此同一条指令名可以用于所有操作数长度。

与条件跳转不同，处理器执行条件传送指令时无须预测测试的结果。处理器只需读出源值（可能来自存储器），检查条件码，然后或者更新目的寄存器，或者让它保持不变。我们将在第 4 章考察条件传送的实现。

要理解条件操作如何能通过条件数据传送来实现，考虑条件表达式和赋值的一种一般形式：

```
v = test-expr ? then-expr : else-expr;
```

用条件控制传送来编译这个表达式的标准方式具有如下形式：

```
    if (!test-expr)
        goto false;
    v = then-expr;
    goto done;
false:
    v = else-expr;
done:
```

这段代码包含两个代码序列，一个计算 *then-expr*，一个计算 *else-expr*，并且用条件跳转和无条件跳转的组合来保证恰好只计算其中一个序列。

对于基于条件传送的代码，*then-expr* 和 *else-expr* 都会被计算出来，最终的值根据 *test-expr* 的求值结果来选取。这可以用下面的抽象代码来描述：

图 3.18 条件传送指令。当传送条件成立时，这些指令把源值 S 复制到目的 R。有些指令带有“同义名”，即同一条机器指令的别名。

指令	同义名	传送条件	说明
`cmove` S, R	`cmovz`	`ZF`	相等 / 零
`cmovne` S, R	`cmovnz`	`~ZF`	不相等 / 非零
`cmovs` S, R		`SF`	负
`cmovns` S, R		`~SF`	非负
`cmovg` S, R	`cmovnle`	`~(SF ^ OF) & ~ZF`	大于(有符号 $>$)
`cmovge` S, R	`cmovnl`	`~(SF ^ OF)`	大于或等于(有符号 $\geq$)
`cmovl` S, R	`cmovnge`	`SF ^ OF`	小于(有符号 $<$)
`cmovle` S, R	`cmovng`	`(SF ^ OF) | ZF`	小于或等于(有符号 $\leq$)
`cmova` S, R	`cmovnbe`	`~CF & ~ZF`	高于(无符号 $>$)
`cmovae` S, R	`cmovnb`	`~CF`	高于或等于(无符号 $\geq$)
`cmovb` S, R	`cmovnae`	`CF`	低于(无符号 $<$)
`cmovbe` S, R	`cmovna`	`CF | ZF`	低于或等于(无符号 $\leq$)

```
v  = then-expr;
ve = else-expr;
t  = test-expr;
if (!t) v = ve;
```

这个序列的最后一条语句用条件传送来实现：只有当测试条件 *t* 不成立时，值 *ve* 才被复制到 *v*。

并非所有条件表达式都能用条件传送来编译。最主要的一点是：我们给出的抽象代码无论测试结果如何都会对 *then-expr* 和 *else-expr* 求值。如果这两个表达式中有可能产生错误条件或副作用，就可能导致无效的行为。前面那个例子（图 3.16）正是如此。事实上，我们把副作用放进那个例子，就是特意要迫使 gcc 用条件传送来实现该函数。

作为第二个说明性的例子，考虑下面的 C 函数：

```
long cread(long *xp) {
    return (xp ? *xp : 0);
}
```

初看起来，这似乎是一个适合用条件传送来编译的好例子：当指针为空时把结果置为 0，如下面的汇编代码所示：

```
   long cread(long *xp)
   函数 cread 的无效实现
   xp 在寄存器 %rdi 中
1  cread:
2    movq    (%rdi), %rax            v = *xp
3    testq   %rdi, %rdi              测试 x
```

```
4    movl    $0, %edx                设置 ve = 0
5    cmove   %rdx, %rax              若 x==0,v = ve
6    ret                             返回 v
```

然而这个实现是无效的，因为第 2 行的 `movq` 指令对 `xp` 的解引用即使测试失败也会发生，从而造成对空指针解引用的错误。因此，这段代码必须用分支代码来编译。

使用条件传送也不总能提高代码效率。例如，如果 *then-expr* 或 *else-expr* 的求值需要大量的计算，那么当相应的条件不成立时，这些计算就白费了。编译器必须权衡浪费的计算与分支预测错误可能造成的性能损失孰轻孰重。实际上，它们并没有足够的信息来可靠地做出这一决定；例如，它们并不知道分支会在多大程度上呈现可预测的模式。我们用 gcc 做的实验表明，只有当两个表达式都能非常容易地计算出来时（例如各用一条加法指令），gcc 才使用条件传送。按照我们的经验，即使在分支预测错误的代价超过更复杂计算的许多情形中，gcc 仍然使用条件控制传送。

总的来说，我们看到：条件数据传送为实现条件操作提供了条件控制传送之外的另一种策略。它只能用在受限的情形中，但这些情形相当常见，而且与现代处理器的运行方式配合得好得多。

练习 3.20 在下面的 C 函数中，我们有目的地把运算 OP 的定义留作未完成：

```
#define OP              /* 未知运算符 */

short arith(short x) {
    return x OP 16;
}
```

编译该函数时，gcc 生成的汇编代码如下：

```
  short arith(short x)
  x 在 %rdi 中
arith:
  leaq     15(%rdi), %rbx
  testq    %rdi, %rdi
  cmovns   %rdi, %rbx
  sarq     $4, %rbx
  ret
```

A. OP 是什么运算？

B. 给代码加上注释，解释它是如何工作的。

练习 3.21 从如下形式的 C 代码开始：

```
short test(short x, short y) {
```

```
    short val = ________;
    if (________) {
        if (________)
            val = ________;
        else
            val = ________;
    } else if (________)
        val = ________;
    return val;
}
```

gcc 生成的汇编代码如下：

```
  short test(short x, short y)
  x 在 %rdi 中,y 在 %rsi 中
test:
  leaq    12(%rsi), %rbx
  testq   %rdi, %rdi
  jge     .L2
  movq    %rdi, %rbx
  imulq   %rsi, %rbx
  movq    %rdi, %rdx
  orq     %rsi, %rdx
  cmpq    %rsi, %rdi
  cmovge  %rdx, %rbx
  ret
.L2:
  idivq   %rsi, %rdi
  cmpq    $10, %rsi
  cmovge  %rdi, %rbx
  ret
```

填写 C 代码中缺失的表达式。

3.9.7 循环

C 提供了几种循环结构，即 do-while、while 和 for。机器代码中没有与之对应的指令。作为替代，是用条件测试与跳转的组合来实现循环的效果的。gcc 与其他编译器根据两种基本的循环模式来生成循环代码。我们将循序渐进地考察循环的翻译：从 do-while 出发，逐步过渡到实现更为复杂的循环，从而把这两种模式都覆盖到。

do-while 循环

do-while 语句的一般形式如下：

```
do
    body-statement
while (test-expr);
```

该循环的作用是：反复执行 *body-statement*，然后对 *test-expr* 求值，若求值结果非零则继续循环。可以看到，*body-statement* 至少要执行一次。

这种一般形式可以翻译成条件语句和 goto 语句,如下所示:

```
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
```

这就是说,程序在每次迭代中先执行循环体语句,再对测试表达式求值;如果测试成功,程序就转回去进行下一次迭代。

(a)C 代码

```
long fact_do(long n)
{
    long result = 1;
    do {
        result *= n;
        n = n-1;
    } while (n > 1);
    return result;
}
```

(b)等价的 goto 版本

```
long fact_do_goto(long n)
{
    long result = 1;
loop:
    result *= n;
    n = n-1;
    if (n > 1)
        goto loop;
    return result;
}
```

(c)对应的汇编语言代码

```
   long fact_do(long n)
   n 在 %rdi 中
1  fact_do:
2    movl    $1, %eax              设置 result = 1
3  .L2:                           loop:
4    imulq   %rdi, %rax            计算 result *= n
5    subq    $1, %rdi              n 减 1
6    cmpq    $1, %rdi              比较 n:1
7    jg      .L2                   若 >,goto loop
8    rep; ret                      返回
```

图 3.19 阶乘程序 do-while 版本的代码。条件跳转使程序得以循环。

作为一个例子, 图 3.19(a) 给出了一个用 do-while 循环计算其参数的阶乘(记作 $n!$)的例程。只有当 $n > 0$ 时,这个函数才能计算出正确的值。

练习 3.22

A. 试用 32 位的 `int` 计算 14!。验证一下 14! 的计算是否溢出。

B. 如果改用 64 位的 `long int` 来计算会怎样?

图 3.19(b) 中的 goto 代码展示了循环如何被变换成更低级的测试与条件跳转的组合。初始化 *result* 之后,程序开始循环。它先执行循环体,这里由对变量 *result* 和 *n* 的更新组成;然后测试 $n > 1$ 是否成立,若成立,就跳回循环的开头。图 3.19(c) 给出了生成该 goto 代码的汇编代码。条件跳转指令 `jg`(第 7 行)是实现循环的关键指令:它决定是继续迭代还是退出循环。

> **边栏 对循环进行逆向工程(reverse engineering)**
> 要理解生成的汇编代码与原始源代码之间的对应关系,关键是找到程序值与寄存器之间的映射。对图 3.19 的循环来说,这个任务还算简单;但对更复杂的程序而言,它可能困难得多。C 编译器常常会重新安排计算,从而使得 C 代码中的某些变量在机器代码中没有对应物,同时机器代码中又引入了一些源代码中不存在的新值。此外,编译器还常常试图通过把多个程序值映射到同一个寄存器来减少寄存器的使用。
> 我们对 `fact_do` 所描述的过程可以推广为对循环进行逆向工程的一般策略:考察寄存器在循环之前如何被初始化、在循环内部如何被更新和测试、在循环之后如何被使用。每一点都提供了一条线索,把这些线索组合起来,就可以解开谜题。请做好心理准备:你将会遇到一些令人惊奇的变换;其中有些显然是编译器能够优化代码的情形,另一些则很难解释编译器为何选择那个特定的策略。

对图 3.19(c) 这样的汇编代码进行逆向工程,需要弄清哪些寄存器保存哪些程序值。本例中的映射相当容易确定:我们知道 n 会通过寄存器`%rdi` 传给函数;可以看到寄存器`%rax` 被初始化为 1(第 2 行)。(回想一下,虽然这条指令的目的寄存器是`%eax`,它同时也会把`%rax` 的高 4 字节置为 0。)我们还可以看到,这个寄存器在第 4 行又被乘法更新。此外,由于`%rax` 用来返回函数值,它常常被选中保存要返回的程序值。因此我们断定:`%rax` 对应于程序值 *result*。

练习 3.23 对于下面的 C 代码:

```
short dw_loop(short x) {
    short y = x/9;
    short *p = &x;
    short n = 4*x;
    do {
        x += y;
        (*p) += 5;
        n -= 2;
    } while (n > 0);
    return x;
}
```

gcc 生成的汇编代码如下:

```
   short dw_loop(short x)
   x 初始时在 %rdi 中
1  dw_loop:
2    movq    %rdi, %rbx
3    movq    %rdi, %rcx
4    idivq   $9, %rcx
5    leaq    (,%rdi,4), %rdx
6  .L2:
7    leaq    5(%rbx,%rcx), %rcx
8    subq    $1, %rdx
9    testq   %rdx, %rdx
10   jg      .L2
11   rep; ret
```

A. 哪些寄存器被用来保存程序值 x、y 和 n?

B. 编译器是怎样消除对指针变量 p 的需要，以及表达式 (*p)+=5 所隐含的指针解引用的?

C. 给这段汇编代码加上注释，描述程序的操作，与图 3.19(c) 中显示的那些注释类似。

while 循环

while 语句的一般形式如下：

```
while (test-expr)
    body-statement
```

它与 do-while 的不同之处在于：*test-expr* 在第一次执行 *body-statement* 之前就被求值，循环可能就此结束。把 while 循环翻译成机器代码的方法有多种，gcc 生成的代码采用了其中两种。两者都使用我们在 do-while 循环中看到的同样的循环结构，区别在于如何实现初始测试。

第一种翻译方法称为“跳转到中间”(jump to middle)，它通过一条无条件跳转、跳到位于循环末尾的测试处来执行初始测试。可以用下面的模板来表示从 while 循环的一般形式到 goto 代码的翻译：

```
    goto test;
loop:
    body-statement
test:
    t = test-expr;
    if (t)
        goto loop;
```

作为一个例子，图 3.20(a) 给出了一个用 while 循环实现的阶乘函数，这个函数能正确地计算出 $0! = 1$。紧随其后的……

```
(a) C 代码
long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}

(b) 等价的 goto 版本
long fact_while_jm_goto(long n)
{
    long result = 1;
    goto test;
  loop:
    result *= n;
    n = n-1;
  test:
    if (n > 1)
        goto loop;
    return result;
}

(c) 对应的汇编语言代码
  long fact_while(long n)
  n in %rdi
fact_while:
  movl     $1, %eax           设置 result = 1
  jmp      .L5                跳转到 test
.L6:                        loop:
  imulq    %rdi, %rax         计算 result *= n
  subq     $1, %rdi           n 递减
.L5:                        test:
  cmpq     $1, %rdi           比较 n:1
  jg       .L6                如果 >,跳转到 loop
  rep; ret                    返回
```

图 3.20　采用跳转到中间（jump-to-middle）翻译的阶乘函数 while 版本的 C 代码与汇编代码。C 函数 `fact_while_jm_goto` 展示了汇编代码版本的行为。

函数 `fact_while_jm_goto`（图 3.20(b)）是 gcc 在用命令行选项 `-Og` 指定优化时所生成汇编代码的一种 C 表述。把为 `fact_while` 生成的 goto 代码（图 3.20(b)）与为 `fact_do` 生成的 goto 代码（图 3.19(b)）进行比较，两者非常相似，唯一的区别是循环之前的 `goto test` 语句使程序在修改 `result` 或 `n` 的值之前先对 `n` 进行测试。图的下半部分（图 3.20(c)）给出了实际生成的汇编代码。

练习 3.24　对于具有如下一般形式的 C 代码：

```
short loop_while(short a, short b)
```

```
{
    short result = __________;
    while (__________) {
        result = __________;
        a = __________;
    }
    return result;
}
```

在命令行选项 `-Og` 下运行的 gcc 产生如下代码：

```
 short loop_while(short a, short b)
   a in %rdi, b in %rsi
1  loop_while:
2    movl     $0, %eax
3    jmp      .L2
4  .L3:
5    leaq     (,%rsi,%rdi), %rdx
6    addq     %rdx, %rax
7    subq     $1, %rdi
8  .L2:
9    cmpq     %rsi, %rdi
10   jg       .L3
11   rep; ret
```

可以看出，编译器使用的是跳转到中间的翻译：第 3 行的 `jmp` 指令跳转到以标号 `.L2` 开始的测试部分。请填写 C 代码中缺失的部分。

第二种翻译方法称为守护 do（guarded do），它先用一条条件分支在初始测试失败时跳过整个循环，从而把代码变换成 do-while 循环。gcc 在更高的优化级别（例如命令行选项 `-O1`）下编译时采用这一策略。从一般的 while 循环形式到 do-while 循环的变换由如下模板给出：

```
t = test-expr;
if (!t)
    goto done;
do
    body-statement
    while (test-expr);
done:
```

随后，这一模板又可以变换成下列 goto 代码：

```
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    t = test-expr;
    if (t)
        goto loop;
done:
```

使用这一实现策略，编译器通常还能优化初始测试，例如断定测试条件总是成立。

作为一个例子，图 3.21 给出了与图 3.20 相同的阶乘函数 C 代码，展示了给 gcc 命令行选项 `-O1` 时的编译结果。图 3.21(c) 给出实际生成的汇编代码，图 3.21(b) 则用一种更易读的 C 表示来呈现这段汇编代码。对照这段 goto 代码可以看出，对 n 的初始值而言，只要 $n \leq 1$，循环就会被跳过。循环本身与该函数 do-while 版本（图 3.19）生成的代码具有相同的总体结构。不过有一个有趣的特征：循环测试（汇编代码第 9 行）从原始 C 代码中的 $n > 1$ 改成了 $n \neq 1$。编译器已经判定，只有 $n > 1$ 时循环才会进入，而 n 递减之后要么 $n > 1$，要么 $n = 1$。因此，测试 $n \neq 1$ 等价于测试 $n \leq 1$。

```
(a) C 代码
long fact_while(long n)
{
    long result = 1;
    while (n > 1) {
        result *= n;
        n = n-1;
    }
    return result;
}

(b) 等价的 goto 版本
long fact_while_gd_goto(long n)
{
    long result = 1;
    if (n <= 1)
        goto done;
 loop:
    result *= n;
    n = n-1;
    if (n != 1)
        goto loop;
 done:
    return result;
}

(c) 对应的汇编语言代码
  long fact_while(long n)
  n in %rdi
 1  fact_while:
 2    cmpq     $1, %rdi           比较 n:1
 3    jle      .L7                如果 <=,跳转到 done
 4    movl     $1, %eax           设置 result = 1
 5  .L6:                        loop:
 6    imulq    %rdi, %rax         计算 result *= n
 7    subq     $1, %rdi           n 递减
 8    cmpq     $1, %rdi           比较 n:1
 9    jne      .L6                如果 !=,跳转到 loop
10    rep; ret                    返回
```

```
11  .L7:                          done:
12    movl     $1, %eax           计算 result = 1
13    ret                         返回
```

图 3.21　采用守护 do 翻译的阶乘函数 while 版本的 C 代码与汇编代码。函数 `fact_while_gd_goto` 展示了汇编代码版本的行为。

练习 3.25　对于具有如下一般形式的 C 代码:

```
long loop_while2(long a, long b)
{
    long result = __________;
    while (__________) {
        result = __________;
        b = __________;
    }
    return result;
}
```

在命令行选项 `-O1` 下运行的 gcc 产生如下代码:

```
    a in %rdi, b in %rsi
 1  loop_while2:
 2    testq   %rsi, %rsi
 3    jle     .L8
 4    movq    %rsi, %rax
 5  .L7:
 6    imulq   %rdi, %rax
 7    subq    %rdi, %rsi
 8    testq   %rsi, %rsi
 9    jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

可以看出,编译器使用的是守护 do 翻译:第 3 行的 `jle` 指令在初始测试失败时跳过循环代码。请填写 C 代码中缺失的部分。注意,汇编代码中的控制结构与按我们的翻译规则对 C 代码直接翻译得到的结果并不完全一致——它有两条不同的 `ret` 指令(第 10 行和第 13 行)。不过,你可以把 C 代码中缺失的部分填写成某种形式,使其行为与这段汇编代码等价。

练习 3.26　函数 `test_one` 具有如下总体结构:

```
short test_one(unsigned short x) {
    short val = 1;
    while ( ... ) {
        .
        .
        .
    }
    return ...;
}
```

gcc 生成如下汇编代码：

```
 short test_one(unsigned short x)
   x in %rdi
1  test_one:
2    movl    $1, %eax
3    jmp     .L5
4  .L6:
5    xorq    %rdi, %rax
6    shrq    %rdi               右移 1 位
7  .L5:
8    testq   %rdi, %rdi
9    jne     .L6
10   andl    $0, %eax
11   ret
```

对这段代码的操作进行逆向工程，然后完成下列各题：

A. 判定使用了哪种循环翻译方法。

B. 利用汇编代码版本填写 C 代码中缺失的部分。

C. 用英语描述这个函数计算的是什么。

for 循环

for 循环的一般形式如下：

```
for (init-expr; test-expr; update-expr)
    body-statement
```

C 语言标准规定（练习 3.29 强调了其中一个例外），这种循环的行为与下面使用 while 循环的代码完全相同：

```
init-expr;
while (test-expr) {
    body-statement
    update-expr;
}
```

程序先求出初始化表达式 `init-expr`，然后进入一个循环：先求测试条件 `test-expr`，测试失败就退出循环；否则执行循环体 `body-statement`，最后求更新表达式 `update-expr`。

gcc 为 for 循环生成的代码同样遵循我们对 while 循环采用的两种翻译策略之一，具体取决于优化级别。也就是说，跳转到中间策略会给出如下 goto 代码：

```
init-expr;
goto test;
loop:
```

```
    body-statement
    update-expr;
test:
    t = test-expr;
    if (t)
        goto loop;
```

而守护 do 策略会给出：

```
init-expr;
t = test-expr;
if (!t)
    goto done;
loop:
    body-statement
    update-expr;
    t = test-expr;
    if (t)
        goto loop;
done:
```

作为一个例子，考虑用 for 循环写出的阶乘函数：

```
long fact_for(long n)
{
    long i;
    long result = 1;
    for (i = 2; i <= n; i++)
        result *= i;
    return result;
}
```

从这段代码可以看出，用 for 循环写阶乘函数的自然方式是把从 2 到 n 的各个因子依次相乘，因此这个函数与前面用 while 或 do-while 写出的代码很不一样。

这段代码中 for 循环的各个组成部分如下：

```
init-expr          i = 2
test-expr          i <= n
update-expr        i++
body-statement     result *= i;
```

把这些组成部分代回前面给出的、把 for 循环变换成 while 循环的模板，得到：

```
long fact_for_while(long n)
{
    long i = 2;
    long result = 1;
    while (i <= n) {
```

```
        result *= i;
        i++;
    }
    return result;
}
```

对 while 循环应用跳转到中间变换,就得到下列 goto 代码版本:

```
long fact_for_jm_goto(long n)
{
     long i = 2;
     long result = 1;
     goto test;
  loop:
     result *= i;
     i++;
  test:
     if (i <= n)
          goto loop;
     return result;
}
```

确实,仔细考察 gcc 用命令行选项 `-Og` 生成的汇编代码,会发现它与这个模板高度一致:

```
  long fact_for(long n)
  n in %rdi
fact_for:
  movl     $1, %eax           设置 result = 1
  movl     $2, %edx           设置 i = 2
  jmp      .L8                跳转到 test
.L9:                        loop:
  imulq    %rdx, %rax         计算 result *= i
  addq     $1, %rdx           i 递增
.L8:                        test:
  cmpq     %rdi, %rdx         比较 i:n
  jle      .L9                如果 <=,跳转到 loop
  rep; ret                    返回
```

练习 3.27 为名为 `fibonacci` 的函数编写 goto 代码,它用 while 循环打印斐波那契数。请应用守护 do 变换。

从上面的讨论可以看出,C 的三种循环形式——do-while、while 和 for——都可以用一种简单的策略来翻译,生成的代码包含一个或多个条件分支。有条件的控制转移为把循环翻译成机器码提供了基本机制。

练习 3.28 函数 `test_two` 具有如下总体结构:

```
short test_two(unsigned short x) {
    short val = 0;
    short i;
    for ( ... ; ... ; ... ) {
```

```
        .
        .
        .
    }
    return val;
}
```

gcc 生成如下汇编代码：

```
 test fun_b(unsigned test x)
   x in %rdi
1  test_two:
2    movl    $1, %edx
3    movl    $65, %eax
4  .L10:
5    movq    %rdi, %rcx
6    andl    $1, %ecx
7    addq    %rax, %rax
8    orq     %rcx, %rax
9    shrq    %rdi                右移 1 位
10   addq    $1, %rdx
11   jne     .L10
12   rep; ret
```

对这段代码的操作进行逆向工程，然后完成下列各题：

A. 利用汇编代码版本填写 C 代码中缺失的部分。

B. 解释为什么循环之前既没有初始测试，也没有一开始就跳转到循环测试部分的跳转。

C. 用英语描述这个函数计算的是什么。

练习 3.29 在 C 中执行 `continue` 语句会使程序跳到当前循环本轮迭代的末尾。处理 `continue` 语句时，前面给出的把 for 循环翻译成 while 循环的规则需要稍加修正。例如，考虑下面的代码：

```
/* 包含 continue 语句的 for 循环示例 */
/* 求 0 到 9 之间的偶数之和 */
long sum = 0;
long i;
for (i = 0; i < 10; i++) {
    if (i & 1)
        continue;
    sum += i;
}
```

A. 如果直接套用我们把 for 循环翻译成 while 循环的规则，会得到什么结果？这段代码有什么问题？

B. 如何把 `continue` 语句替换成 `goto` 语句，才能保证 while 循环正确地复制 for 循环的行为？

3.9.8 switch 语句

switch 语句根据一个整型索引值提供多路分支的能力。当测试可能产生大量不同的结果时，它特别有用。switch 语句不仅使 C 代码更易读，而且可以借助一种称为跳转表(jump table)的数据结构高效地实现。跳转表是一个数组，其第 i 个表项是一个代码段的地址，该代码段实现当 switch 索引等于 i 时程序应采取的动作。代码对跳转表执行一次数组引用，用 switch 索引来确定跳转指令的目标。与一长串 if-else 语句相比，使用跳转表的优点是：执行 switch 所需的时间与 switch 分支的个数无关。gcc 根据 switch 分支的个数和 case 值的稀疏程度来选择翻译 switch 语句的方法。当分支个数较多(例如 4 个或更多)并且它们跨越一个较小的取值范围时，就使用跳转表。

图 3.22(a) 给出了 switch 语句的一个例子。这个例子有许多有趣的特性：case 标号跨越的取值不连续(没有对应于 101 和 105 的标号)，有的 case 带有多个标号(104 和 106)，有的 case 会贯穿(fall through)到其他 case(102)，因为该 case 的代码不是以 `break` 语句结尾的。

图 3.23 给出了编译 `switch_eg` 时生成的汇编代码。这段代码的行为在 C 中表现为图 3.22(b) 中的过程 `switch_eg_impl`。这段代码利用了 gcc 作为对 C 语言的扩展而提供的跳转表支持。数组 `jt` 有 7 个表项，每个表项都是一段代码的地址。这些位置由代码中的标号定义，在 `jt` 的表项中用代码指针表示，即在标号前面加上 &&。(回忆一下，运算符“&”为数据值创建指针。在实现这一扩展时，gcc 的作者创建了一个新的运算符“&&”，用来为代码位置创建指针。)我们建议你仔细研究 C 过程 `switch_eg_impl`，以及它与汇编代码版本的对应关系。

原来的 C 代码有对应于值 100、102 ~ 104 和 106 的 case，但 switch 变量 n 可以是任意的整数。编译器先把 n 减去 100，把范围平移到 0 ~ 6 之间，在我们的 C 版本中把这个新的程序变量称为 `index`。编译器进而把 `index` 当作无符号值处理来简化分支的可能性，其依据是：补码表示中的负数会映射成无符号表示中的大正数。因此，要判断 `index` 是否落在 0 ~ 6 的范围之外，只需测试它是否大于 6。在 C 和汇编代码中，根据 `index` 的值，有 5 个不同的跳转目标：`loc_A`(在汇编代码中标识为 `.L3`)、`loc_B`(`.L5`)、`loc_C`(`.L6`)、`loc_D`(`.L7`)和 `loc_def`(`.L8`)，其中最后一个是 default 分支的目标。这些标号各自标识一段实现某个 case 分支的代码。在 C 和汇编代码中，程序都把 `index` 与 6 比较，如果大于 6 就跳转到 default 分支的代码。

执行 switch 语句的关键步骤是通过跳转表访问一个代码位置。这在 C 代码中体现为第 16 行的 `goto` 语句，它引用跳转表 `jt`。gcc 支持这种计算 goto 作

为对 C 语言的扩展。在汇编代码版本中，类似的操作出现在第 5 行，jmp 指令的操作数以“*”为前缀，表明这是一条间接跳转，操作数指定一个存储器位置，它由保存 index 值的寄存器%eax 来索引。(3.8 节会看到数组引用是如何翻译成机器码的。)

```
(a) switch 语句
void switch_eg(long x, long n,
               long *dest)
{
    long val = x;
    switch (n) {
        case 100:
            val *= 13;
            break;

        case 102:
            val += 10;
            /* 贯穿 */

        case 103:
            val += 11;
            break;

        case 104:
        case 106:
            val *= val;
            break;

        default:
            val = 0;
    }
    *dest = val;
}

(b) 翻译成扩展的 C
 1  void switch_eg_impl(long x, long n,
 2                      long *dest)
 3  {
 4      /* 代码指针表 */
 5      static void *jt[7] = {
 6          &&loc_A, &&loc_def, &&loc_B,
 7          &&loc_C, &&loc_D, &&loc_def,
 8          &&loc_D
 9      };
10      unsigned long index = n - 100;
11      long val;
12
13      if (index > 6)
14          goto loc_def;
15      /* 多路分支 */
16      goto *jt[index];
17
18   loc_A:    /* case 100 */
19      val = x * 13;
```

```
20      goto done;
21   loc_B:    /* case 102 */
22      x = x + 10;
23      /* 贯穿 */
24   loc_C:    /* case 103 */
25      val = x + 11;
26      goto done;
27   loc_D:    /* case 104, 106 */
28      val = x * x;
29      goto done;
30   loc_def:  /* default 分支 */
31      val = 0;
32   done:
33      *dest = val;
34   }
```

图 3.22　switch 语句的例子及其到扩展 C 的翻译。译文给出了跳转表 `jt` 的结构及其访问方式。gcc 以对 C 语言的扩展形式支持这种表。

我们的 C 代码把跳转表声明为一个有 7 个元素的数组，每个元素都是指向某个代码位置的指针。这些元素覆盖 `index` 的 0 ~ 6 的取值范围，对应 n 的 100 ~ 106 的取值。可以看到，跳转表通过给表项 4 和表项 6 使用同一个代码标号(`loc_D`)来处理重复的 case，并通过给表项 1 和表项 5 使用 default 分支的标号(`loc_def`)来处理缺失的 case。

```
 void switch_eg(long x, long n, long *dest)
 x in %rdi, n in %rsi, dest in %rdx
 1  switch_eg:
 2    subq    $100, %rsi             计算 index = n-100
 3    cmpq    $6, %rsi               比较 index:6
 4    ja      .L8                    如果 >,跳转到 loc_def
 5    jmp     *.L4(,%rsi,8)          跳转到 *jg[index]
 6  .L3:                             loc_A:
 7    leaq    (%rdi,%rdi,2), %rax    计算 3*x
 8    leaq    (%rdi,%rax,4), %rdi    计算 val = 13*x
 9    jmp     .L2                    跳转到 done
10  .L5:                             loc_B:
11    addq    $10, %rdi              计算 x = x + 10
12  .L6:                             loc_C:
13    addq    $11, %rdi              计算 val = x + 11
14    jmp     .L2                    跳转到 done
15  .L7:                             loc_D:
16    imulq   %rdi, %rdi             计算 val = x * x
17    jmp     .L2                    跳转到 done
18  .L8:                             loc_def:
19    movl    $0, %edi               计算 val = 0
20  .L2:                             done:
21    movq    %rdi, (%rdx)           计算 *dest = val
22    ret                            返回
```

图 3.23　图 3.22 中 switch 语句示例的汇编代码。

在汇编代码中，跳转表由下面的声明表示，我们对它们加了注释：

```
 1    .section            .rodata
 2    .align 8            将地址对齐到 8 的倍数
 3  .L4:
 4    .quad    .L3        case 100: loc_A
 5    .quad    .L8        case 101: loc_def
 6    .quad    .L5        case 102: loc_B
 7    .quad    .L6        case 103: loc_C
 8    .quad    .L7        case 104: loc_D
 9    .quad    .L8        case 105: loc_def
10    .quad    .L7        case 106: loc_D
```

这些声明说明,在目标代码文件中名为.rodata(“只读数据”)的段中,应当有一串 7 个“四字”(8 字节)字,每个字的值由与相应汇编代码标号(例如.L3)相关联的指令地址给出。标号.L4 标记这一存储分配的起点,与这个标号相关联的地址作为间接跳转(第 5 行)的基地址。

各个代码块(C 标号 loc_A 到 loc_D 以及 loc_def)实现 switch 语句的不同分支。其中大多数只是为 val 计算出一个值,然后跳到函数末尾。类似地,汇编代码块为寄存器%rdi 计算出一个值,然后跳到函数末尾处标号.L2 指示的位置。只有 case 标号 102 的代码不符合这个模式,这是为了照顾原始 C 代码中该 case 的代码贯穿到标号为 103 的代码块的方式。从标号.L5 开始的汇编代码块通过省略块末尾的 jmp 指令来处理这种情形,让代码继续执行下一个块。类似地,C 版本 switch_eg_impl 中以 loc_B 开始的块末尾没有 goto 语句。

研读所有这些代码需要下一些功夫,但关键是要看到:使用跳转表为实现多路分支提供了一种非常高效的方法。在我们的例子里,程序通过一次对跳转表的引用就能分支到 5 个不同的位置。即使 switch 语句有上百个 case,一次对跳转表的访问也能处理它们。

练习 3.30 在下面的 C 函数中,我们省略了 switch 语句的主体。在 C 代码中,case 标号覆盖的取值不连续,而且有些 case 有多个标号。

```
void switch2(short x, short *dest) {
    short val = 0;
    switch (x) {
        .
        . switch 语句的主体省略
        .
    }
    *dest = val;
}
```

编译这个函数时,gcc 为过程的开头部分生成如下汇编代码,变量 x 保存在寄存器%rdi 中:

```
 void switch2(short x, short *dest)
   x in %rdi
```

```
1  switch2:
2    addq      $2, %rdi
3    cmpq      $8, %rdi
4    ja        .L2
5    jmp       *.L4(,%rdi,8)
```

它为跳转表生成如下代码：

```
1  .L4:
2    .quad    .L9
3    .quad    .L5
4    .quad    .L6
5    .quad    .L7
6    .quad    .L2
7    .quad    .L7
8    .quad    .L8
9    .quad    .L2
10   .quad    .L5
```

根据这些信息，回答下列问题：

A. switch 语句中 case 标号的值是多少？

B. C 代码中哪些 case 有多个标号？

练习 3.31 对于具有如下总体结构的 C 函数 `switcher`：

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case ________:        /* case A */
         c = ________;
         /* 贯穿 */
    case ________:        /* case B */
         val = ________;
         break;
    case ________:        /* case C */
    case ________:        /* case D */
         val = ________;
         break;
    case ________:        /* case E */
         val = ________;
         break;
    default:
         val = ________;
    }
    *dest = val;
}
```

gcc 生成图 3.24 所示的汇编代码和跳转表。

请填写 C 代码中缺失的部分。除了 case 标号 C 和 D 的先后顺序之外，把这些不同的 case 放入模板的方式只有一种。

```
(a) 代码
  void switcher(long a, long b, long c, long *dest)
    a in %rsi, b in %rdi, c in %rdx, d in %rcx
 1  switcher:
 2    cmpq     $7, %rdi
 3    ja       .L2
 4    jmp      *.L4(,%rdi,8)
 5    .section          .rodata
 6  .L7:
 7    xorq     $15, %rsi
 8    movq     %rsi, %rdx
 9  .L3:
10    leaq     112(%rdx), %rdi
11    jmp      .L6
12  .L5:
13    leaq     (%rdx,%rsi), %rdi
14    salq     $2, %rdi
15    jmp      .L6
16  .L2:
17    movq     %rsi, %rdi
18  .L6:
19    movq     %rdi, (%rcx)
20    ret
(b) 跳转表
 1  .L4:
 2    .quad     .L3
 3    .quad     .L2
 4    .quad     .L5
 5    .quad     .L2
 6    .quad     .L6
 7    .quad     .L7
 8    .quad     .L2
 9    .quad     .L5
```

图 3.24　练习 3.31 的汇编代码和跳转表。

3.10 过程

过程(procedure)是软件中的一个关键抽象。它们提供了一种方式,把实现某种功能的代码封装起来,配以一组指定的参数和一个可选的返回值。这一功能随后可以在程序的不同地点被调用。设计良好的软件把过程用作一种抽象机制:一方面隐藏某个动作的详细实现,另一方面提供清晰简洁的接口定义,说明将计算出哪些值、过程会对程序状态产生哪些影响。在不同的编程语言中,过程以多种面目出现——函数(function)、方法(method)、子例程(subroutine)、处理例程(handler)等等——但它们都享有一组共同的特性。

在为过程提供机器级支持时,有许多不同的属性要处理。为了便于讨论,假设过程 P 调用过程 Q,Q 执行完毕后返回到 P。这些动作涉及下述机制中的一种或多种:

- **传递控制**。程序计数器在进入时必须设置为 Q 的代码的起始地址，在返回时必须设置为 P 中紧随对 Q 的调用之后的指令。
- **传递数据**。P 必须能够向 Q 提供一个或多个参数，Q 必须能够向 P 返回一个值。
- **分配和释放内存**。Q 在开始时可能需要为局部变量分配空间，而在返回之前释放这部分存储空间。

x86-64 对过程的实现是特殊指令与一组约定的组合，后者规定如何使用寄存器、程序存储器这类机器资源。为了把调用过程的开销降到最低，人们付出了大量的努力。因此，它遵循一种可以视为极简主义的策略：对每个具体的过程，只实现上述机制集合中实际需要的部分。在接下来的叙述中，我们将一步一步地建立这些不同的机制：先讨论控制传递，然后讨论数据传递，最后讨论内存管理。

3.10.1 运行时栈

C 以及大多数其他语言的调用过程机制的一个关键特性，是它能利用栈数据结构所提供的后进先出的内存管理规则。仍以过程 P 调用过程 Q 的例子来说，Q 执行期间，P 以及调用链上直到 P 为止的所有过程都被暂时挂起。Q 运行时，只有它才需要有为局部变量分配新存储或者建立对另一个过程的调用的能力。另一方面，当 Q 返回时，它分配的所有局部存储都可以释放。因此，程序可以用栈来管理其过程所需的存储空间：栈和程序寄存器存放着传递控制与数据所需的信息，以及分配内存所需的信息。当 P 调用 Q 时，控制和数据信息被加到栈的末端；P 返回时，这些信息被释放。

如 3.4.4 节所述，x86-64 的栈向低地址方向增长，栈指针%rsp 指向栈顶元素。可以用指令 `pushq` 和 `popq` 把数据入栈和出栈。没有指定初始值的数据所需的空间可以这样在栈上分配：把栈指针减去适当的量即可。类似地，把栈指针加上适当的量即可释放空间。

当 x86-64 过程需要的存储超出了寄存器所能容纳的数量时，它就会在栈上分配空间。这个区域称为过程的栈帧（stack frame）。图 3.25 给出了运行时栈的总体结构，包括它划分成栈帧时的最一般形式。当前正在执行的过程的帧总是位于栈顶。当过程 P 调用过程 Q 时，P 会把返回地址压入栈中，指明 Q 返回后应从 P 中的哪个位置恢复执行。我们把返回地址视为 P 的栈帧的一部分，因为它存放的是与 P 有关的状态。Q 的代码通过扩展当前栈的边界来分配其栈帧所需的空间，在这个空间里，它可以保存寄存器的值、为局部变量分配空间，并为它所调用的过程建立参数。大多数过程的栈帧是定长的，在过程开始时分

配。不过有些过程需要变长的帧,这个问题在 3.10.5 节讨论。过程 P 最多可以在栈上传递 6 个整数值(即指针和整数),但如果 Q 需要的参数更多,P 可以在调用之前把这些参数存放在自己的栈帧里。

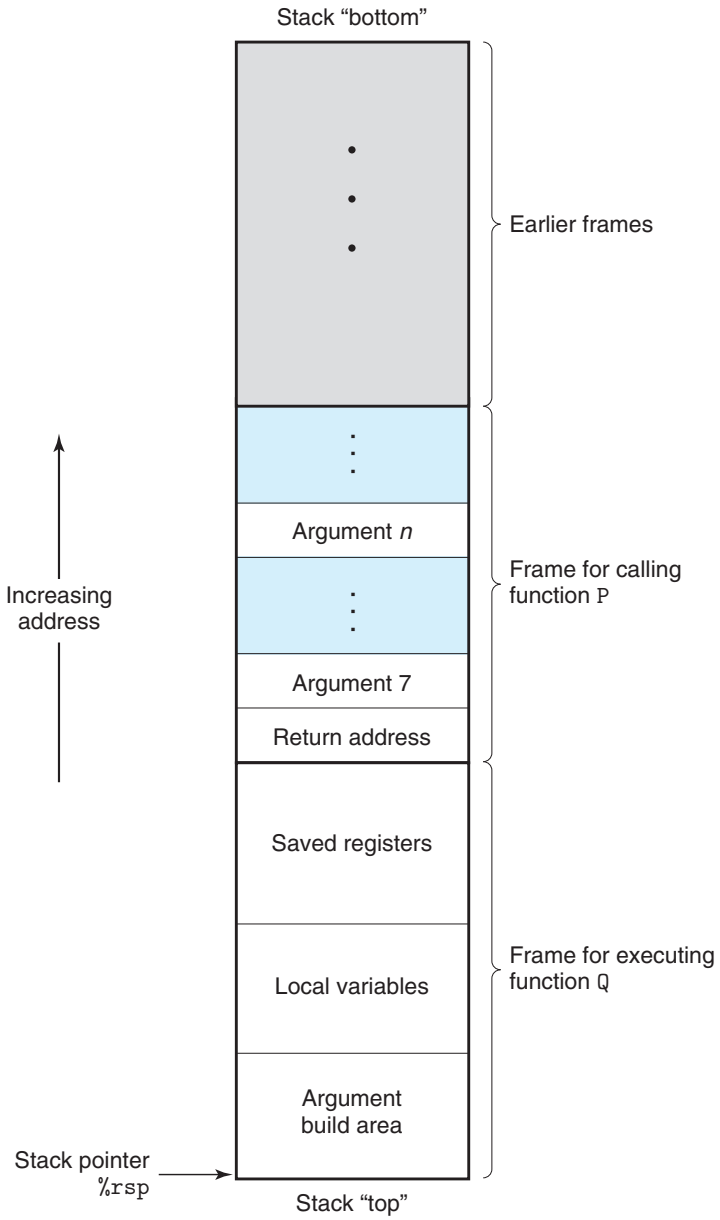


图 3.25 （示意图）

出于对空间和时间效率的考虑, x86-64 的过程只分配它们的栈帧中实际需要的部分。例如,许多过程有 6 个或更少的参数,因而它们的全部参数都可

以用寄存器传递。于是，图 3.25 中画出的栈帧的某些部分可以省略。实际上，许多函数甚至根本不需要栈帧。这种情况出现在所有局部变量都能保存在寄存器中、并且该函数不调用任何其他函数的时候（这样的过程有时称为叶过程（leaf procedure），得名于过程调用所构成的树结构）。例如，到目前为止我们考察过的所有函数都不需要栈帧。

3.10.2 控制转移

把控制从函数 P 传递到函数 Q，只须把程序计数器（PC）设置为 Q 的代码的起始地址即可。不过，等到 Q 后来要返回时，处理器必须留有某种记录，指明应该在哪个代码位置恢复 P 的执行。在 x86-64 机器上，这条信息是通过用指令 `call Q` 调用过程 Q 来记录的。该指令把一个地址 A 压入栈，并把 PC 设置为 Q 的开头。压入的地址 A 称为返回地址（return address），它被计算为紧随 `call` 指令之后那条指令的地址。与之对应的指令 `ret` 从栈中弹出地址 A，并把 PC 设置为 A。

`call` 和 `ret` 两条指令的一般形式如下：

```
指令                描述
call Label          过程调用
call *Operand       过程调用
ret                 从调用返回
```

（这些指令在程序 `objdump` 生成的反汇编输出中被称为 `callq` 和 `retq`。加上后缀 “q” 只是为了强调这两个指令是 `call` 指令和返回指令的 x86-64 版本，而不是 IA32 版本。在 x86-64 汇编代码中，两种形式可以互换使用。）

`call` 指令有一个目标操作数，指明被调用过程的第一条指令的地址。与跳转一样，`call` 既可以是直接的，也可以是间接的。在汇编代码中，直接调用的目标以标号给出，而间接调用的目标由“*”后跟一个操作数描述符给出，其格式采用图 3.3 中描述的几种形式之一。

```
%rip
%rsp
0x400563
0x7fffffffe840
(a) Executing call
%rip
%rsp
0x400540
0x7fffffffe838
0x400568
(b) After call
%rip
%rsp
0x400568
```

```
0x7fffffffe840
(c) After ret
Figure 3.26
Illustration of call and ret functions. The call instruction transfers
        control to the start of a
function, while the ret instruction returns back to the instruction
        following the call.
```

图 3.26 展示了 3.2.2 节介绍的 multstore 和 main 函数执行 call 和 ret 指令的情况。下面是这两个函数的反汇编代码摘录:

```
     函数 multstore 的开始
1  0000000000400540 <multstore>:
2    400540: 53                    push     %rbx
3    400541: 48 89 d3              mov      %rdx,%rbx
     . . .
     函数 multstore 的返回
4    40054d:     c3                retq
     . . .
     从 main 中调用 multstore
5    400563:     e8 d8 ff ff ff    callq    400540 <multstore>
6    400568:     48 8b 54 24 08    mov      0x8(%rsp),%rdx
```

在这段代码中可以看到, main 中地址为 0x400563 的 call 指令调用函数 multstore。这一状态如图 3.26(a) 所示, 图中标出了栈指针%rsp 和程序计数器%rip 的值。这条指令的效果是把返回地址 0x400568 压入栈, 并跳转到函数 multstore 的第一条指令, 其地址为 0x0400540 (图 3.26(b))。函数 multstore 持续执行,直到遇到地址 0x40054d 处的 ret 指令。这条指令从栈中弹出值 0x400568 并跳转到这个地址, 从 call 指令之后恢复 main 的执行 (图 3.26(c))。

作为向过程传递和从过程传回控制的一个更详细的例子, 图 3.27(a) 给出了两个函数 top 和 leaf 的反汇编代码, 以及函数 main 中调用 top 的那部分代码。每条指令都用标号标识: leaf 中为 L1 ~ L2, top 中为 T1 ~ T4, main 中为 M1 ~ M2。图的 (b) 部分给出了代码执行的详细追踪, 其中 main 调用 top(100),使 top 调用 leaf(95)。函数 leaf 向 top 返回 97,

(a) 用于演示过程调用与返回的反汇编代码

```
leaf(long y) 的反汇编
y 放在 %rdi 中
 1  0000000000400540 <leaf>:
 2    400540:  48 8d 47 02     lea     0x2(%rdi),%rax    L1: z+2
 3    400544:  c3             retq                           L2: 返回

 4  0000000000400545 <top>:
    top(long x) 的反汇编
    x 放在 %rdi 中
```

```
 5    400545:  48 83 ef 05    sub     $0x5,%rdi            T1: x-5
 6    400549:  e8 f2 ff ff ff  callq  400540 <leaf>        T2: 调用
        leaf(x-5)
 7    40054e:  48 01 c0       add     %rax,%rax            T3: 结果加倍
 8    400551:  c3             retq                          T4: 返回

          . . .
    从函数 main 调用 top
 9    40055b:  e8 e5 ff ff ff  callq  400545 <top>         M1: 调用
        top(100)
10    400560:  48 89 c2       mov     %rax,%rdx            M2: 恢复执行
```

(b) 示例代码的执行跟踪

图 3.27 涉及过程调用与返回的程序的详细执行。把返回地址压入栈中来存放，使得过程能够返回到正确的位置。

标号	PC	指令	%rdi	%rax	%rsp	*%rsp	说明
M1	0x40055b	callq	100	—	0x7fffffffe820	—	调用 top(100)
T1	0x400545	sub	100	—	0x7fffffffe818	0x400560	进入 top
T2	0x400549	callq	95	—	0x7fffffffe818	0x400560	调用 leaf(95)
L1	0x400540	lea	95	—	0x7fffffffe810	0x40054e	进入 leaf
L2	0x400544	retq	—	97	0x7fffffffe810	0x40054e	从 leaf 返回 97
T3	0x40054e	add	—	97	0x7fffffffe818	0x400560	恢复 top
T4	0x400551	retq	—	194	0x7fffffffe818	0x400560	从 top 返回 194
M2	0x400560	mov	—	194	0x7fffffffe820	—	恢复 main

随后向 main 返回 194。表中前三列描述正在执行的指令，包括指令标号、地址和指令类型。接着的四列给出指令执行之前的程序状态，包括寄存器%rdi、%rax 和%rsp 的内容，以及栈顶的值。这张表的内容应当仔细研究，因为它们展现了运行时栈在管理支持过程调用与返回所需的存储方面所起的重要作用。

leaf 的指令 L1 把%rax 设置为 97，即要返回的值。指令 L2 随后执行返回。它从栈中弹出 0x400054e。弹出的这个值装入 PC 后，控制就返回到 top 的指令 T3。程序成功地完成了对 leaf 的调用，回到 top。

指令 T3 把%rax 设置为 194，即要从 top 返回的值。指令 T4 随后执行返回。它从栈中弹出 0x4000560，从而把 PC 设置为 main 的指令 M2。程序成功地完成了对 top 的调用，回到 main。我们还看到，栈指针也被恢复为 0x7fffffffe820，即调用 top 之前的值。

这种把返回地址压入栈的简单机制，使函数日后能够返回到程序中正确的位置。C（以及大多数编程语言）的标准调用/返回机制，与栈提供的后进先出存储管理规则恰好相合。

练习 3.32 两个函数 first 和 last 的反汇编代码如下所示，同时给出了函数 main 调用 first 的代码：

```
last(long u, long v) 的反汇编
u 放在 %rdi 中,v 放在 %rsi 中
 1  0000000000400540 <last>:
 2    400540:  48 89 f8        mov   %rdi,%rax         L1: u
 3    400543:  48 0f af c6     imul  %rsi,%rax         L2: u*v
 4    400547:  c3              retq                    L3: 返回

    first(long x) 的反汇编
    x 放在 %rdi 中
 5  0000000000400548 <first>:
 6    400548:  48 8d 77 01     lea   0x1(%rdi),%rsi    F1: x+1
 7    40054c:  48 83 ef 01     sub   $0x1,%rdi         F2: x-1
 8    400550:  e8 eb ff ff ff  callq 400540 <last>     F3: 调用
        last(x-1,x+1)
 9    400555:  f3 c3           repz retq               F4: 返回
     .
     .
     .
10    400560:  e8 e3 ff ff ff  callq 400548 <first>    M1: 调用
↪   first(10)
11    400565:  48 89 c2        mov   %rax,%rdx         M2: 恢复执行
```

这些指令的每一条都被赋予一个标号，与图 3.27(a) 中的标号类似。从 main 调用 first(10) 开始，填写下表来跟踪指令的执行，直到程序返回到 main 为止。

标号	PC	指令	%rdi	%rsi	%rax	%rsp	*%rsp	说明
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	调用 first(10)
F1								
F2								
F3								
L1								
L2								
L3								
F4								
M2								

3.10.3 数据传递

过程调用除了在调用时把控制交给过程、在过程返回时再把控制交回来之外，还可能涉及把数据当作参数传入，从过程返回时也可能带回一个返回值。在 x86-64 中，这些进出过程的数据大部分都通过寄存器传递。例如，我们已经见过许多这样的函数：参数通过寄存器`%rdi`、`%rsi` 等传入，返回值通过寄存器`%rax` 返回。当过程 P 调用过程 Q 时，P 的代码必须先把参数复制到相应的寄存器中。类似地，当 Q 返回 P 时，P 的代码就能访问寄存器`%rax` 中的返回值。本节更详细地探讨这些约定。

在 x86-64 中，最多有 6 个整型（即整数或指针）参数可以通过寄存器传递。这些寄存器按指定的顺序使用，所用寄存器的名字取决于被传递数据类型的大小，如图 3.28 所示。参数按照它们在参数列表中的次序分配到这些寄存器上。小于 64 位的参数，可以通过 64 位寄存器中相应的低位部分来访问。例如，如果第一个参数是 32 位的，它就可以用`%edi` 来访问。

图 3.28 传递函数参数所用寄存器。这些寄存器按指定的顺序使用，并按参数的大小来命名。

操作数大小（位）	1	2	3	4	5	6
64	%rdi	%rsi	%rdx	%rcx	%r8	%r9
32	%edi	%esi	%edx	%ecx	%r8d	%r9d
16	%di	%si	%dx	%cx	%r8w	%r9w
8	%dil	%sil	%dl	%cl	%r8b	%r9b

当一个函数的整型参数多于 6 个时，其余的参数就要放在栈上传送。设过程 P 调用过程 Q 时带有 n 个整型参数，且 $n > 6$。那么 P 的代码必须分配一个栈帧，给第 7 ~ n 个参数留出足够的存储空间，如图 3.25 所示。它把参数 1 ~ 6 复制到相应的寄存器中，把参数 7 ~ n 放到栈上，参数 7 位于栈顶。通过栈传递参数时，所有数据大小都要向上取整到 8 的倍数。参数就位后，程序就执行 call 指令，把控制传给过程 Q。过程 Q 通过寄存器访问自己的参数，也可能从栈中访问。如果 Q 又要调用一个参数多于 6 个的函数，它可以在自己的栈帧内为这些参数分配空间，如图 3.25 中标注“参数构造区”（Argument build area）的区域所示。

作为参数传递的一个例子，考虑图 3.29(a) 中的 C 函数 proc。这个函数有 8 个参数，包括字节数不同（8、4、2、1）的整数，还包括不同类型的指针，每个指针占 8 个字节。

为 proc 生成的汇编代码如图 3.29(b) 所示。前 6 个参数通过寄存器传递，最后两个通过栈传递，图 3.30 的示意图说明了这一点。这张图给出了执行 proc

期间的栈的状态。可以看到,返回地址是作为过程调用的一部分被压入栈中的。因此,这两个参数距栈指针的位置分别是 8 和 16。从代码中可以看出,根据操作数大小的不同,用到了 add 指令的不同版本:a1(long)用 addq,a2(int)用 addl,a3(short)用 addw,a4(char)用 addb。注意,第 6 行的 movl 从内存读出 4 个字节,随后的 addb 只用到其中的低位字节。

(a) C 代码

```
void proc(long  a1, long  *a1p,
          int   a2, int   *a2p,
          short a3, short *a3p,
          char  a4, char  *a4p)
{
    *a1p += a1;
    *a2p += a2;
    *a3p += a3;
    *a4p += a4;
}
```

(b) 生成的汇编代码

```
  void proc(a1, a1p, a2, a2p, a3, a3p, a4, a4p)
  参数传递方式如下:
    a1  放在 %rdi 中      (64 位)
    a1p 放在 %rsi 中      (64 位)
    a2  放在 %edx 中      (32 位)
    a2p 放在 %rcx 中      (64 位)
    a3  放在 %r8w 中      (16 位)
    a3p 放在 %r9 中       (64 位)
    a4  位于 %rsp+8       (8 位)
    a4p 位于 %rsp+16      (64 位)
1  proc:
2    movq    16(%rsp), %rax      取出 a4p        (64 位)
3    addq    %rdi, (%rsi)        *a1p += a1      (64 位)
4    addl    %edx, (%rcx)        *a2p += a2      (32 位)
5    addw    %r8w, (%r9)         *a3p += a3      (16 位)
6    movl    8(%rsp), %edx       取出 a4         (8 位)
7    addb    %dl, (%rax)         *a4p += a4      (8 位)
8    ret                         返回
```

图 3.29 具有多个不同类型参数的函数示例。参数 1 ~ 6 通过寄存器传递,参数 7 ~ 8 通过栈传递。

练习 3.33 C 函数 `procprob` 有四个参数 u、a、v 和 b。每个参数或者是一个有符号数,或者是一个指向有符号数的指针,这些数的大小各不相同。该函数的函数体如下:

```
    *u += a;
    *v += b;
    return sizeof(a) + sizeof(b);
```

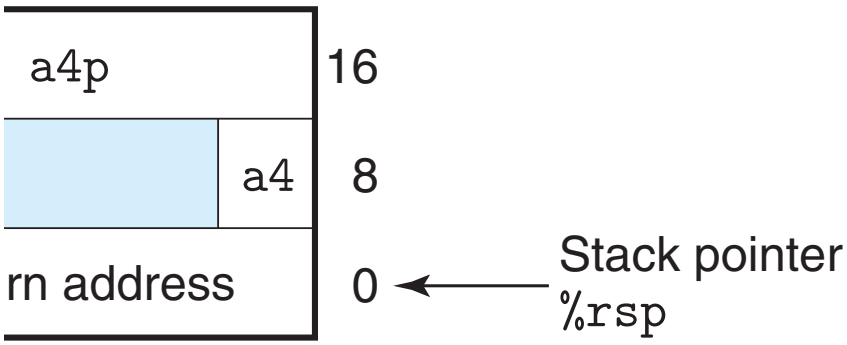


图 3.30 （示意图）

它编译成下面的 x86-64 代码：

```
1  procprob:
2    movslq  %edi, %rdi
3    addq    %rdi, (%rdx)
4    addb    %sil, (%rcx)
5    movl    $6, %eax
6    ret
```

确定这四个参数的有效顺序和类型。存在两种正确答案。

3.10.4 栈上的局部存储

到目前为止，我们见到的大多数过程例子，所需的局部存储都没有超过寄存器能保存的数量。不过，局部数据有时必须存放在内存中。常见的情况有：

- 可用的寄存器不足以保存全部局部数据。
- 对局部变量使用了取地址运算符“&”，因而必须为它生成一个地址。
- 有些局部变量是数组或结构，因此必须通过数组或结构的引用来访问。这种情形留到讨论数组和结构的分配时再展开。

过程通常通过减小栈指针来在栈帧中分配空间。这样得到的就是图 3.25 中标注“局部变量”（Local variables）的那部分栈帧。

作为处理取地址运算符的一个例子，考虑图 3.31(a) 中的两个函数。函数 `swap_add` 交换指针 xp 和 yp 所指向的两个值，同时返回这两个值的和。函数 `caller` 创建指向局部变量 arg1 和 arg2 的指针，并把它们传给 `swap_add`。图 3.31(b) 展示了 `caller` 如何用一个栈帧来实现这两个局部变量。`caller` 的代码先把栈指针减 16，这实际上是在栈上分配了 16 个字节。令 S 表示栈指针的值，可以看到，代码在第 5 行把 &arg2 算成 $S+8$，在第 6 行把 &arg1 算成 S。因此可以推断，局部变量 arg1 和 arg2 存放在栈帧中、距栈指针偏移量为 0 和 8 的地方。对 `swap_add` 的调用完成后，`caller` 的代码从栈中取出这两个值（第

8 ~ 9 行)，计算它们的差，再乘上 swap_add 在寄存器%rax 中返回的值(第 10 行)。最后，该函数把栈指针加 16 来释放栈帧(第 11 行)。从这个例子可以看到，运行时栈提供了一种简单的机制：需要时分配局部存储，函数完成时释放它。

作为一个更复杂的例子，图 3.32 所示的函数 call_proc 体现了 x86-64 栈规范(stack discipline)的许多方面。这个例子虽然长，却值得仔细研究。它展示了一个函数，既要为局部变量在栈上分配存储，又要把值传给有 8 个参数的函数 proc(图 3.29)。该函数创建了一个栈帧，如图 3.33 所示。

查看 call_proc 的汇编代码(图 3.32(b))可以发现，代码中有很大一部分(第 2 ~ 15 行)是在为调用函数

(a) swap_add 与调用函数的代码

```
long swap_add(long *xp, long *yp)
{
    long x = *xp;
    long y = *yp;
    *xp = y;
    *yp = x;
    return x + y;
}

long caller()
{
    long arg1 = 534;
    long arg2 = 1057;
    long sum = swap_add(&arg1, &arg2);
    long diff = arg1 - arg2;
    return sum * diff;
}
```

(b) 为调用函数生成的汇编代码

```
  long caller()
 1  caller:
 2    subq    $16, %rsp          为栈帧分配 16 个字节
 3    movq    $534, (%rsp)       把 534 存入 arg1
 4    movq    $1057, 8(%rsp)     把 1057 存入 arg2
 5    leaq    8(%rsp), %rsi      计算 &arg2,作为第二个参数
 6    movq    %rsp, %rdi         计算 &arg1,作为第一个参数
 7    call    swap_add           调用 swap_add(&arg1, &arg2)
 8    movq    (%rsp), %rdx       取出 arg1
 9    subq    8(%rsp), %rdx      计算 diff = arg1 - arg2
10    imulq   %rdx, %rax         计算 sum * diff
11    addq    $16, %rsp          释放栈帧
12    ret                        返回
```

图 3.31 过程定义与调用示例。由于取地址运算符的存在，调用代码必须分配一个栈帧。

proc 做准备。这包括为局部变量和函数参数设置栈帧，以及把函数参数装载到寄存器中。如图 3.33 所示，局部变量 x1 ~ x4 在栈上分配，大小各不相同。把它们的位置表示成相对栈指针的偏移量，它们占据字节 24 ~ 31（x1）、20 ~ 23（x2）、18 ~ 19（x3）和 17（s3）。指向这些位置的指针由 leaq 指令生成（第 7、10、12、14 行）。参数 7（值为 4）和参数 8（指向 x4 位置的指针）存放在栈上、距栈指针偏移量为 0 和 8 的位置。

(a) 调用函数的 C 代码

```
long call_proc()
{
    long x1 = 1; int x2 = 2;
    short x3 = 3; char x4 = 4;
    proc(x1, &x1, x2, &x2, x3, &x3, x4, &x4);
    return (x1+x2)*(x3-x4);
}
```

(b) 生成的汇编代码

```
  long call_proc()
 1  call_proc:
        设置 proc 的参数
 2    subq    $32, %rsp         为栈帧分配 32 个字节
 3    movq    $1, 24(%rsp)      把 1 存入 &x1
 4    movl    $2, 20(%rsp)      把 2 存入 &x2
 5    movw    $3, 18(%rsp)      把 3 存入 &x3
 6    movb    $4, 17(%rsp)      把 4 存入 &x4
 7    leaq    17(%rsp), %rax    生成 &x4
 8    movq    %rax, 8(%rsp)     把 &x4 作为参数 8 存入
 9    movl    $4, (%rsp)        把 4 作为参数 7 存入
10    leaq    18(%rsp), %r9     传递 &x3 作为参数 6
11    movl    $3, %r8d          传递 3 作为参数 5
12    leaq    20(%rsp), %rcx    传递 &x2 作为参数 4
13    movl    $2, %edx          传递 2 作为参数 3
14    leaq    24(%rsp), %rsi    传递 &x1 作为参数 2
15    movl    $1, %edi          传递 1 作为参数 1
        调用 proc
16    call    proc
        取回内存中的改动
17    movslq  20(%rsp), %rdx    取出 x2 并转换成 long
18    addq    24(%rsp), %rdx    计算 x1+x2
19    movswl  18(%rsp), %eax    取出 x3 并转换成 int
20    movsbl  17(%rsp), %ecx    取出 x4 并转换成 int
21    subl    %ecx, %eax        计算 x3-x4
22    cltq                      转换成 long
23    imulq   %rdx, %rax        计算 (x1+x2) * (x3-x4)
24    addq    $32, %rsp         释放栈帧
25    ret                       返回
```

图 3.32 调用图 3.29 中定义的函数 proc 的代码示例。这段代码创建一个栈帧。

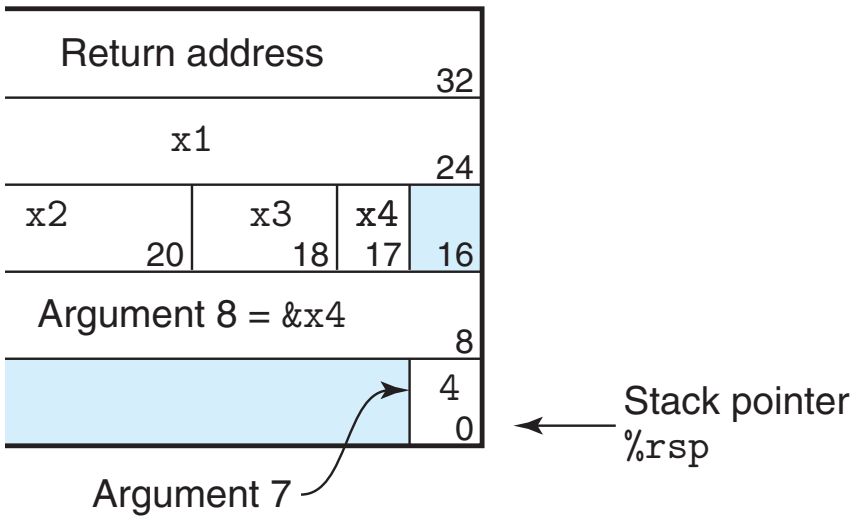


图 3.33 （示意图）

过程 proc 被调用时，程序将开始执行图 3.29(b) 所示的代码。如图 3.30 所示，此时参数 7 和 8 相对栈指针的偏移量是 8 和 16，因为返回地址已经压入了栈。

程序返回到 `call_proc` 后，代码取出四个局部变量的值（第 17 ~ 20 行），并完成最后的计算。最后把栈指针加 32，释放这个栈帧。

3.10.5 寄存器中的局部存储

程序寄存器组是作为单一资源被所有过程共享的。虽然在给定时刻只能有一个过程处于活动状态，但我们必须保证，当一个过程（调用者，caller）调用另一个过程（被调用者，callee）时，被调用者不会覆盖掉调用者稍后还要使用的某个寄存器的值。为此，x86-64 采用了一组统一的寄存器使用约定，所有过程都必须遵守，包括程序库中的过程。

按照约定，寄存器`%rbx`、`%rbp` 和`%r12` ~ `%r15` 划分为被调用者保存寄存器（callee-saved registers）。当过程 P 调用过程 Q 时，Q 必须**保存**这些寄存器的值，保证 Q 返回 P 时，这些寄存器与被调用时的值相同。过程 Q 若要保存某个寄存器的值，要么完全不改它，要么先把这个寄存器的原值压入栈中，改动它，然后在返回之前把旧值从栈中弹出。把寄存器值入栈，其效果就是创建了图 3.25 中标注“保存的寄存器”（Saved registers）的那部分栈帧。有了这一约定，P 的代码就可以放心地把一个值存入某个被调用者保存寄存器（当然，得先把原来的值保存到栈上），调用 Q，然后再使用这个寄存器中的值，而不必担心它已被破坏。

除栈指针`%rsp` 以外的所有其他寄存器都划分为调用者保存寄存器（caller-saved registers）。这就是说，任何函数都可以改写它们。“调用者保存”这个名字

可以这样理解：设过程 P 在某个这类寄存器中存放着局部数据，又调用过程 Q。由于 Q 可以随意改写这个寄存器，所以 P（调用者）在调用之前必须先把数据保存起来。

作为例子，考虑图 3.34(a) 中的函数 P。它调用 Q 两次。第一次调用期间，它必须保留 x 的值以备后用。类似地，第二次调用期间，它必须保留为 Q(y) 算出的值。在图 3.34(b) 中，

(a) 调用函数

```
long P(long x, long y)
{
    long u = Q(y);
    long v = Q(x);
    return u + v;
}
```

(b) 为调用函数生成的汇编代码

```
  long P(long x, long y)
  x 放在 %rdi 中,y 放在 %rsi 中
 1  P:
 2    pushq   %rbp           保存 %rbp
 3    pushq   %rbx           保存 %rbx
 4    subq    $8, %rsp       栈帧对齐
 5    movq    %rdi, %rbp     保存 x
 6    movq    %rsi, %rdi     把 y 复制到第一个参数
 7    call    Q              调用 Q(y)
 8    movq    %rax, %rbx     保存结果
 9    movq    %rbp, %rdi     把 x 复制到第一个参数
10    call    Q              调用 Q(x)
11    addq    %rbx, %rax     把保存的 Q(y) 加到 Q(x) 上
12    addq    $8, %rsp       释放栈的最后部分
13    popq    %rbx           恢复 %rbx
14    popq    %rbp           恢复 %rbp
15    ret
```

图 3.34 演示被调用者保存寄存器用法的代码。x 必须在第一次调用期间加以保留，Q(y) 必须在第二次调用期间加以保留。

可以看出，gcc 生成的代码用了两个被调用者保存寄存器：`%rbp` 保存 x，`%rbx` 保存算出的 Q(y) 值。函数开头先把这两个寄存器的值保存在栈上（第 2 ~ 3 行）。第一次调用 Q 之前，把参数 x 复制到`%rbp`（第 5 行）。第二次调用 Q 之前，把这次调用的结果复制到`%rbx`（第 8 行）。函数结尾（第 13 ~ 14 行）把这两个被调用者保存寄存器的值从栈中弹出恢复。注意，弹出顺序与压入顺序正好相反，这是为了符合栈的后进先出规则。

练习 3.34 考虑一个函数 P，它会生成一些局部值，命名为 a0 ~ a8。然后它用这些生成的值作为参数调用函数 Q。gcc 为 P 的第一部分生成如下代码：

```
  long P(long x)
  x 放在 %rdi 中
 1  P:
 2    pushq   %r15
 3    pushq   %r14
 4    pushq   %r13
 5    pushq   %r12
 6    pushq   %rbp
 7    pushq   %rbx
 8    subq    $24, %rsp
 9    movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
        . . .
```

A. 指出哪些局部值被保存在被调用者保存寄存器中。

B. 指出哪些局部值被保存在栈上。

C. 解释为什么这个程序无法把所有局部值都存放在被调用者保存寄存器中。

3.10.6 递归过程

前面讨论的寄存器和栈使用约定，使 x86-64 过程能够调用自己，也就是递归调用。每次过程调用在栈上都有自己私有的空间，所以多个尚未完成的调用的局部变量互不干扰。而且，栈规范本来就提供了恰当的做法：在过程被调用时分配合适的局部存储，在返回之前释放它。

图 3.35 同时给出一个递归阶乘函数的 C 代码和生成的汇编代码。可以看到，汇编代码用寄存器`%rbx` 保存参数 n：先把该寄存器原有的值保存到栈上（第 2 行），返回之前再恢复该值（第 11 行）。由于栈规范和寄存器保存约定的存在，我们可以确信，当对 rfact(n-1) 的递归调用返回时（第 9 行），(1) 调用的结果将保存在寄存器

(a) C 代码

```
long rfact(long n)
{
    long result;
    if (n <= 1)
         result = 1;
    else
```

```
        result = n * rfact(n-1);
    return result;
}
```

(b) 生成的汇编代码

```
  long rfact(long n)
  n 放在 %rdi 中
 1  rfact:
 2    pushq    %rbx             保存 %rbx
 3    movq     %rdi, %rbx       把 n 存入被调用者保存寄存器
 4    movl     $1, %eax         设置返回值 = 1
 5    cmpq     $1, %rdi         比较 n:1
 6    jle      .L35             若 <=,跳转到 done
 7    leaq     -1(%rdi), %rdi   计算 n-1
 8    call     rfact            调用 rfact(n-1)
 9    imulq    %rbx, %rax       结果乘以 n
10  .L35:                       done:
11    popq     %rbx             恢复 %rbx
12    ret                       返回
```

图 3.35 递归阶乘程序的代码。标准的过程处理机制足以实现递归函数。

%rax 中,(2) 参数 n 的值将保存在寄存器%rbx 中。把这两个值相乘就得到想要的结果。

从这个例子可以看出,递归调用一个函数,与其他任何函数调用并无不同。我们的栈规范提供了一种机制:函数的每次调用都有自己私有的存储,用来保存状态信息(保存的返回位置和被调用者保存寄存器的值)。需要时,它还能为局部变量提供存储。这种分配与释放的栈规范,与函数的调用/返回次序自然吻合。这种实现函数调用与返回的方法,甚至适用于更复杂的模式,包括相互递归(例如,过程 P 调用 Q,而 Q 又调用 P)。

练习 3.35 对于一个具有如下一般结构的 C 函数

```
long rfun(unsigned long x) {
    if (           )
        return         ;
    unsigned long nx =                ;
    long rv = rfun(nx);
    return           ;
}
```

gcc 生成如下的汇编代码:

```
  long rfun(unsigned long x)
  x 放在 %rdi 中
 1  rfun:
 2    pushq   %rbx
 3    movq    %rdi, %rbx
 4    movl    $0, %eax
```

```
 5    testq   %rdi, %rdi
 6    je      .L2
 7    shrq    $2, %rdi
 8    call    rfun
 9    addq    %rbx, %rax
10  .L2:
11    popq    %rbx
12    ret
```

A. rfun 把什么值存放在被调用者保存寄存器%rbx 中?

B. 补全上面的 C 代码中缺失的表达式。

3.11 数组分配和访问

在 C 语言中,把标量数据聚合成更大的数据类型的方法之一就是数组。C 对数组的实现特别简单,因此翻译成机器代码也相当直接。C 的一个不同寻常之处是能够生成指向数组内部元素的指针,并对这种指针进行运算。这些指针会被翻译成机器代码中的地址计算。

优化编译器特别擅长简化数组索引所使用的地址计算。这会使 C 代码与其机器码翻译之间的对应关系变得有点难以辨认。

3.11.1 基本原则

设 T 为数据类型,N 为整数常量,考虑如下形式的声明:

```
T A[N];
```

把数组的起始位置记作 x_A。这个声明有两个作用。第一,它在内存中分配一个 $L \cdot N$ 字节的连续区域,其中 L 是数据类型 T 的大小(以字节为单位)。第二,它引入标识符 A,A 可以用作指向数组开头的指针,这个指针的值就是 x_A。数组元素可以用 0 到 $N-1$ 之间的整数索引来访问。数组元素 i 将存放在地址 $x_A + L \cdot i$ 处。

作为例子,考虑下面的声明:

```
char    A[12];
char   *B[8];
int     C[6];
double *D[5];
```

这些声明生成的数组具有如下参数:

数组	元素大小	总大小	起始地址	元素 i
A	1	12	x_A	$x_A + i$
B	8	64	x_B	$x_B + 8i$
C	4	24	x_C	$x_C + 4i$
D	8	40	x_D	$x_D + 8i$

数组 A 由 12 个单字节(char)元素组成。数组 C 由 6 个整数组成,每个整数需要 4 个字节。B 和 D 都是指针数组,因此数组元素各为 8 字节。

x86-64 的内存引用指令为简化数组访问而设计。例如,设 E 是一个元素类型为 int 的数组,我们希望对 E[i] 求值,数组 E 的地址存放在寄存器%rdx 中,索引 i 存放在寄存器%rcx 中。那么指令

```
movl (%rdx,%rcx,4),%eax
```

将执行地址计算 $x_E + 4i$,读出该内存位置的内容,并把结果复制到寄存器%eax 中。允许的比例因子 1、2、4、8 覆盖了常见原始数据类型的大小。

练习 3.36 考虑如下声明:

```
int    P[5];
short  Q[2];
int   **R[9];
double *S[10];
short  *T[2];
```

填写下表,说明这些数组各自的元素大小、总大小和元素 i 的地址。

数组	元素大小	总大小	起始地址	元素 i
P			x_P	
Q			x_Q	
R			x_R	
S			x_S	
T			x_T	

3.11.2 指针运算

C 允许对指针做算术运算,计算出的值会按指针所引用数据类型的大小缩放。也就是说,如果 p 是指向类型 T 数据的指针,p 的值为 x_p,那么表达式 p+i 的值是 $x_p + L \cdot i$,其中 L 是数据类型 T 的大小。

一元运算符 & 和 * 允许生成指针和对指针解引用。也就是说,对表示某个对象的表达式 Expr,&Expr 是一个给出该对象地址的指针。对表示一个地址的表达式 AExpr, *AExpr 给出该地址上的值。因此, 表达式 Expr 与 *&Expr 是等价的。数组下标运算既可以用于数组, 也可以用于指针: 数组引用 A[i] 与表达式 *(A+i) 完全相同,它先算出第 i 个数组元素的地址,然后访问这个内存位置。

接着前面的例子,设整数数组 E 的起始地址和整数索引 i 分别存放在寄存器%rdx 和%rcx 中。下面是一些涉及 E 的表达式。我们还为每个表达式给出一种汇编代码实现,结果存放在寄存器%eax(对数据)或寄存器%rax(对指针)中。

表达式	类型	值	汇编代码
E	int *	x_E	movl %rdx,%rax
E[0]	int	$M[x_E]$	movl (%rdx),%eax
E[i]	int	$M[x_E + 4i]$	movl (%rdx,%rcx,4),%eax
&E[2]	int *	$x_E + 8$	leaq 8(%rdx),%rax
E+i-1	int *	$x_E + 4i - 4$	leaq -4(%rdx,%rcx,4),%rax
*(E+i-3)	int	$M[x_E + 4i - 12]$	movl -12(%rdx,%rcx,4),%eax
&E[i]-E	long	i	movq %rcx,%rax

在这些例子中可以看到，返回数组值的运算的类型是 int，因此对应的是 4 字节操作（如 movl）和寄存器（如%eax）。返回指针的运算的类型是 int *，因此对应 8 字节操作（如 leaq）和寄存器（如%rax）。最后一个例子表明，可以计算同一个数据结构内两个指针的差，得到的数据类型是 long，值等于两个地址的差除以数据类型的大小。

练习 3.37 设 short 整数数组 P 的地址 x_P 和长整数索引 i 分别存放在寄存器%rdx 和%rcx 中。对下面每个表达式，给出它的类型、它的值的一个公式和一种汇编代码实现。结果若是指针，应存放在寄存器%rax 中；若数据类型是 short，应存放在元素寄存器%ax 中。

表达式	类型	值	汇编代码
P[1]			
P+3+i			
P[i * 6 - 5]			
P[2]			
&P[i + 2]			

3.11.3 嵌套数组

数组分配与引用的一般原则，在你创建数组的数组时同样成立。例如，声明

```
int A[5][3];
```

等价于声明

```
typedef int row3_t[3];
row3_t A[5];
```

数据类型 row3_t 定义为三个整数组成的数组。数组 A 包含 5 个这样的元素，每个元素需要 12 个字节来存放这三个整数。整个数组的大小就是 $4 \cdot 5 \cdot 3 = 60$ 字节。

数组 A 也可以看作是一个有 5 行 3 列的二维数组，用 A[0][0] 到 A[4][2] 来引用。数组元素在内存中按行优先（row-major）顺序排列，意思是：先是

第 0 行的所有元素(可写成 A[0]),接着是第 1 行(A[1])的所有元素,以此类推。如图 3.36 所示。

这种排列顺序是嵌套声明的结果。把 A 看作是一个含 5 个元素的数组,而它的每个元素又是含 3 个 int 的数组,那么先是 A[0],然后是 A[1],以此类推。

要访问多维数组的元素,编译器生成的代码要计算出目标元素的位移量,然后使用某一条 mov 指令,以数组的起始地址作为基址,以这个(可能经过缩放的)位移量作为索引。一般地,对于声明为

```
T D[R][C];
```

的数组,数组元素 D[i][j] 所在的内存地址为

$$\&D[i][j] = x_D + L \cdot (C \cdot i + j) \tag{3.1}$$

图 3.36　按行优先顺序排列的数组元素

行	元素	地址
A[0]	A[0][0]	x_A
	A[0][1]	$x_A + 4$
	A[0][2]	$x_A + 8$
A[1]	A[1][0]	$x_A + 12$
	A[1][1]	$x_A + 16$
	A[1][2]	$x_A + 20$
A[2]	A[2][0]	$x_A + 24$
	A[2][1]	$x_A + 28$
	A[2][2]	$x_A + 32$
A[3]	A[3][0]	$x_A + 36$
	A[3][1]	$x_A + 40$
	A[4][2]	$x_A + 44$
A[4]	A[4][0]	$x_A + 48$
	A[4][1]	$x_A + 52$
	A[4][2]	$x_A + 56$

其中 L 是数据类型 T 的大小(以字节计)。作为例子,考虑前面定义的 5×3 整数数组 A。设 x_A、i 和 j 分别存放在寄存器%rdi、%rsi 和%rdx 中,那么数组元素 A[i][j] 可以用下面的代码复制到寄存器%eax 中:

```
  A 放在 %rdi 中,i 放在 %rsi 中,j 放在 %rdx 中
1    leaq    (%rsi,%rsi,2), %rax     计算 3i
2    leaq    (%rdi,%rax,4), %rax     计算 xA + 12i
3    movl    (%rax,%rdx,4), %eax     从 M[xA + 12i + 4] 读出
```

可以看出,这段代码利用 x86-64 地址运算的缩放与加法能力,把元素的地址计算为 $x_A + 12i + 4j = x_A + 4(3i + j)$。

练习 3.38 考虑下面的源码,其中 M 和 N 是用 `#define` 声明的常量:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

编译这个程序时,gcc 生成如下的汇编代码:

```
  long sum_element(long i, long j)
  i 放在 %rdi 中,j 放在 %rsi 中
1  sum_element:
2    leaq    0(,%rdi,8), %rdx
3    subq    %rdi, %rdx
4    addq    %rsi, %rdx
5    leaq    (%rsi,%rsi,4), %rax
6    addq    %rax, %rdi
7    movq    Q(,%rdi,8), %rax
8    addq    P(,%rdx,8), %rax
9    ret
```

用你的逆向工程技巧,根据这段汇编代码确定 M 和 N 的值。

3.11.4 定长数组

对操作固定大小多维数组的代码,C 编译器能够做出许多优化。这里我们演示 gcc 在用 -O1 选项设置优化级别时所做的一些优化。假设用如下方式声明数据类型 `fix_matrix`,表示 16×16 的整数数组:

```
#define N 16
typedef int fix_matrix[N][N];
```

(这个例子说明了一种良好的编码做法。程序中凡是把某个常量用作数组维度或缓冲区大小的地方,最好都用 `#define` 声明为它起一个名字,然后一致地使用这个名字,而不要直接用数值。这样,一旦将来需要改变这个值,只需修改 `#define` 声明即可。)图 3.37(a) 中的代码计算数组 A 和 B 的乘积的第 i、k 个元素,也就是 A 的第 i 行与 B 的第 k 列的内积。这个乘积由下式给出:

$$\sum_{0 \le j < N} a_{i,j} \cdot b_{j,k}$$

gcc 生成的代码随后被我们改写成 C,即图 3.37(b) 中的函数 `fix_prod_ele_opt`。这段代码包含了许多聪明的优化。它去掉了整数

索引 j，把所有数组引用都转换成指针解引用。这包括：(1) 生成一个指针（我们把它命名为 Aptr），指向 A 第 i 行中相继的元素；(2) 生成一个指针（命名为 Bptr），指向 B 第 k 列中相继的元素；(3) 生成一个指针（命名为 Bend），它的值就是循环该终止时 Bptr 将具有的值。Aptr 的初值是 A 第 i 行第一个元素的地址，用 C 表达式 `&A[i][0]` 给出。Bptr 的初值是 B 第 k 列第一个元素的地址，用 C 表达式 `&B[0][k]` 给出。Bend 的值是 B 第 j 列中第 $(n+1)$ 个元素的下标，用 C 表达式 `&B[N][k]` 给出。

(a) 原始 C 代码

```
/* 计算定长矩阵乘积的第 i,k 个元素 */
int fix_prod_ele (fix_matrix A, fix_matrix B, long i, long k) {
    long j;
    int result = 0;

    for (j = 0; j < N; j++)
        result += A[i][j] * B[j][k];

    return result;
}
```

(b) 优化后的 C 代码

```
 1  /* 计算定长矩阵乘积的第 i,k 个元素 */
 2  int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k) {
 3      int *Aptr = &A[i][0];       /* 指向 A 第 i 行的元素 */
 4      int *Bptr = &B[0][k];       /* 指向 B 第 k 列的元素 */
 5      int *Bend = &B[N][k];       /* 标出 Bptr 的停止点 */
 6      int result = 0;
 7      do {                        /* 无需初始测试 */
 8          result += *Aptr * *Bptr;  /* 把下一个乘积加进和值 */
 9          Aptr ++;                  /* 把 Aptr 移到下一列 */
10          Bptr += N;                /* 把 Bptr 移到下一行 */
11      } while (Bptr != Bend);     /* 测试是否到达停止点 */
12      return result;
13  }
```

图 3.37 为定长数组计算矩阵乘积的第 i、k 个元素的原始代码与优化后代码。编译器自动完成这些优化。

下面是 gcc 为函数 `fix_prod_ele` 实际生成的汇编代码。可以看到，四个寄存器的分工如下：`%eax` 保存 result，`%rdi` 保存 Aptr，`%rcx` 保存 Bptr，`%rsi` 保存 Bend。

```
  int fix_prod_ele_opt(fix_matrix A, fix_matrix B, long i, long k)
  A 放在 %rdi 中,B 放在 %rsi 中,i 放在 %rdx 中,k 放在 %rcx 中
1  fix_prod_ele:
2    salq    $6, %rdx               计算 64 * i
3    addq    %rdx, %rdi             计算 Aptr = xA + 64i = &A[i][0]
```

```
 4    leaq    (%rsi,%rcx,4), %rcx    计算 Bptr = xB + 4k = &B[0][k]
 5    leaq    1024(%rcx), %rsi       计算 Bend = xB + 4k + 1024 =
 ↪  &B[N][k]
 6    movl    $0, %eax               设置 result = 0
 7  .L7:                           loop:
 8    movl    (%rdi), %edx           读出 *Aptr
 9    imull   (%rcx), %edx           乘以 *Bptr
10    addl    %edx, %eax             加到 result 上
11    addq    $4, %rdi               Aptr ++
12    addq    $64, %rcx              Bptr += N
13    cmpq    %rsi, %rcx             比较 Bptr:Bend
14    jne     .L7                    若不等于则跳转到 loop
15    rep; ret                       返回
```

练习 3.39 利用公式 3.1 解释：图 3.37(b) 的 C 代码（第 3 ~ 5 行）对 Aptr、Bptr 和 Bend 初值的计算，为什么能够正确描述为函数 `fix_prod_ele` 生成的汇编代码（第 3 ~ 5 行）中对它们的计算。

练习 3.40 下面的 C 代码把我们的一个定长数组的对角线元素设置为 val：

```
/* 把所有对角线元素都设置为 val */
void fix_set_diag(fix_matrix A, int val) {
    long i;
    for (i = 0; i < N; i++)
        A[i][i] = val;
}
```

用优化级别 -O1 编译时，gcc 生成如下的汇编代码：

```
1  fix_set_diag:
       void fix_set_diag(fix_matrix A, int val)
       A 放在 %rdi 中,val 放在 %rsi 中
2    movl    $0, %eax
3  .L13:
4    movl    %esi, (%rdi,%rax)
5    addq    $68, %rax
6    cmpq    $1088, %rax
7    jne     .L13
8    rep; ret
```

编写一个 C 代码程序 `fix_set_diag_opt`，它采用与这段汇编代码类似的优化，写成与图 3.37(b) 中代码相同的风格。要用含参数 N 的表达式，而不是整数常量，这样即使 N 被重新定义，你的代码也能正确工作。

3.11.5 变长数组

过去，C 语言只支持那些大小（可能除第一维之外）在编译时就能确定的多维数组。

以前,需要变长数组的程序员必须用 malloc 或 calloc 之类的函数为这些数组分配存储空间,还必须按照公式 3.1 所表达的方式,通过行主序索引显式地把多维数组到一维数组的映射编码出来。ISO C99 引入了这样一项能力:数组的维数表达式可以在分配数组的同时才进行计算。

在 C 的变长数组版本中,我们可以把数组

```
int A[expr1][expr2]
```

声明成一个局部变量,或者一个函数参数;在遇到这个声明时,通过对表达式 `expr1` 和 `expr2` 求值即可确定数组的维数。例如,我们可以按如下方式编写一个函数,访问 $n \times n$ 数组中的元素 i, j:

```
int var_ele(long n, int A[n][n], long i, long j) {
    return A[i][j];
}
```

参数 n 必须出现在参数 `A[n][n]` 之前,这样在遇到该参数时,函数才能计算出数组的维数。

gcc 为这个引用元素的函数生成的代码如下:

```
 int var_ele(long n, int A[n][n], long i, long j)
 n 在 %rdi 中,A 在 %rsi 中,i 在 %rdx 中,j 在 %rcx 中
1  var_ele:
2    imulq  %rdx, %rdi                计算 n·i
3    leaq   (%rsi,%rdi,4), %rax       计算 xA + 4(n·i)
4    movl   (%rax,%rcx,4), %eax       从 M[xA + 4(n·i) + 4j] 读出
5    ret
```

正如注释所示,这段代码把元素 i, j 的地址计算为 $x_{\mathrm{A}} + 4(n \cdot i) + 4j = x_{\mathrm{A}} + 4(n \cdot i + j)$。这个地址计算与定长数组的地址计算(3.8.3 节)相似,区别只有两点:(1) 由于新增了参数 n,寄存器的用法有所改变;(2) 用一条乘法指令(第 2 行)来计算 $n \cdot i$,而不是用 leaq 指令计算 $3i$。由此可见,引用变长数组只需要对定长数组的做法作很小的推广:动态版本必须用一条乘法指令把 i 缩放 n 倍,而不是用一串移位和加法指令。在某些处理器上,这种乘法会带来相当大的性能损失,不过在这里这是不可避免的。

在循环中引用变长数组时,编译器常常可以利用访问模式的规律性来优化下标计算。例如,图 3.38(a) 给出计算 $n \times n$ 数组 A 和 B 的乘积中元素 i, k 的 C 代码。gcc 生成汇编代码之后,我们把它重新表述成 C 代码(图 3.38(b))。这段代码与定长数组的优化代码(图 3.37)风格不同,但这更多是编译器所做选择的结果,而不是这两种不同函数的本质要求。图 3.38(b) 的代码保留了循环变量 j,它既用来检测何

图 3.38 计算变长数组矩阵乘积中元素 i,k 的原始 C 代码 (a) 与优化后的 C 代码 (b)。这些优化是编译器自动完成的。

```
1  /* 计算变长矩阵乘积的元素 i,k */
2  int var_prod_ele(long n, int A[n][n], int B[n][n], long i, long k) {
3      long j;
4      int result = 0;
5
6      for (j = 0; j < n; j++)
7          result += A[i][j] * B[j][k];
8
9      return result;
10 }
```

```
/* 计算变长矩阵乘积的元素 i,k */
int var_prod_ele_opt(long n, int A[n][n], int B[n][n], long i, long k)
↪  {
    int *Arow = A[i];
    int *Bptr = &B[0][k];
    int result = 0;
    long j;
    for (j = 0; j < n; j++) {
        result += Arow[j] * *Bptr;
        Bptr += n;
    }
    return result;
}
```

时循环终止,也用来在由 A 的第 i 行元素构成的数组中作下标检索。

下面是 `var_prod_ele` 的循环的汇编代码:

```
 n 在 %rdi 中,Arow 在 %rsi 中,Bptr 在 %rcx 中
 4n 在 %r9 中,result 在 %eax 中,j 在 %edx 中
1  .L24:                                  loop:
2    movl   (%rsi,%rdx,4), %r8d           读出 Arow[j]
3    imull  (%rcx), %r8d                  乘以 *Bptr
4    addl   %r8d, %eax                    加到 result 上
5    addq   $1, %rdx                      j++
6    addq   %r9, %rcx                     Bptr += n
7    cmpq   %rdi, %rdx                    比较 j:n
8    jne    .L24                          若 !=,跳转到 loop
```

我们看到,程序既用了缩放过的值 $4n$(寄存器`%r9`)来递增 `Bptr`,又用了 n 的值(寄存器`%rdi`)来检查循环

的上下界。由于指针运算自带缩放,需要用到两个值这件事在 C 代码中并不会显现出来。

我们已经看到, 启用优化时, gcc 能够识别出程序步进遍历多维数组元素时所产生的模式,进而生成避免直接应用公式 3.1 就会产生的那次乘法的代码。无论它生成的是基于指针的代码(图 3.37(b))还是基于数组的代码(图 3.38(b)),这些优化都会显著提高程序的性能。

3.12 异质的数据结构

C 提供了两种把不同类型的对象组合起来创建数据类型的机制：结构(structure)用关键字 `struct` 声明，它把多个对象聚合到一个单独的单元之中；联合(union)用关键字 `union` 声明，它允许一个对象按照几种不同的类型来引用。

3.12.1 结构

C 的 `struct` 声明创建了一种数据类型，把可能属于不同类型的对象组合成一个对象。结构的各个组成部分用名字来引用。结构的实现与数组类似：结构的所有组成部分都存放在内存中一片连续的区域里，而指向结构的指针就是它第一个字节的地址。编译器为每个结构类型维护一份信息，指出每个字段的字节偏移量。生成对结构元素的引用时，编译器就把这些偏移量用作存储器引用指令中的位移。

作为一个例子，考虑下面的结构声明：

```
struct rec {
    int i;
    int j;
    int a[2];
    int *p;
};
```

这个结构包含四个字段：两个 4 字节的 `int` 类型的值、一个 `int` 类型的二元素数组，以及一个 8 字节的整数指针，总共 24 字节：

偏移	0	4	8		16	24
内容	i	j	a[0]	a[1]	p	

注意，数组 a 就嵌在结构之中。表顶部一行的数字给出的是每个字段相对于结构起始处的字节偏移量。

为了访问结构的字段，编译器生成的代码会把适当的偏移量加到结构的地址上。例如，假设变量 r

> **给 C 语言初学者　把对象表示为 struct**
>
> `struct` 数据类型构造符是 C 提供的、最接近 C++ 和 Java 中对象的东西。它允许程序员把某个实体的信息保存在一个数据结构中，并用名字来引用这些信息。
> 例如，一个图形程序可以用一个结构来表示矩形：
>
> ```
> struct rect {
>     long llx;             /* 左下角顶点的 X 坐标 */
>     long lly;             /* 左下角顶点的 Y 坐标 */
>     unsigned long width;  /* 宽度（像素） */
>     unsigned long height; /* 高度（像素） */
>     unsigned color;       /* 颜色编码 */
> ```

```
};
```

我们可以声明一个 `struct rect` 类型的变量 r,并按如下方式设置各个字段的值:

```
struct rect r;
r.llx = r.lly = 0;
r.color = 0xFF00FF;
r.width = 10;
r.height = 20;
```

其中表达式 `r.llx` 选取结构 r 的字段 `llx`。
或者,我们也可以用一条语句同时完成变量的声明和字段的初始化:

```
struct rect r = { 0, 0, 0xFF00FF, 10, 20 };
```

常见的做法是把指向结构的指针从一处传递到另一处,而不是复制结构本身。例如,下面的函数计算矩形的面积,参数是指向矩形结构的指针:

```
long area(struct rect *rp) {
    return (*rp).width * (*rp).height;
}
```

表达式 `(*rp).width` 先对指针解引用,再从得到的结构中选取 `width` 字段。这里的括号是必需的,否则编译器会把表达式 `*rp.width` 解释成 `*(rp.width)`,这是无效的。这种“解引用并选取字段”的组合非常常见,因此 C 提供了 `->` 记法作为替代。也就是说,`rp->width` 等价于表达式 `(*rp).width`。例如,我们可以编写一个把矩形逆时针旋转 90 度的函数:

```
void rotate_left(struct rect *rp) {
    /* 交换宽度和高度 */
    long t = rp->height;
    rp->height = rp->width;
    rp->width = t;
    /* 平移到新的左下角位置 */
    rp->llx   -= t;
}
```

(类型为 `struct rec *`)保存在寄存器`%rdi` 中。那么下面的代码会把元素 `r->i` 复制到元素 `r->j`:

```
  r 在 %rdi 中
1  movl  (%rdi), %eax        获取 r->i
2  movl  %eax, 4(%rdi)      写到 r->j
```

字段 i 的偏移量为 0,因此这个字段的地址就是 r 的值。要把数据存到字段 j 里,代码就在 r 的地址上加上偏移量 4。

要生成指向结构内部对象的指针,我们只需把字段的偏移量加到结构的地址上。例如,加上偏移量 $8 + 4 \cdot 1 = 12$ 就可以生成指针 `&(r->a[1])`。对于保存在寄存器`%rdi` 中的指针 r 以及保存在寄存器`%rsi` 中的长整型变量 i, 我们只用一条指令就能生成指针值 `&(r->a[i])`:

```
  r 在 %rdi 中,i 在 %rsi 中
1  leaq  8(%rdi,%rsi,4), %rax    把 %rax 设为 &r->a[i]
```

作为最后一个例子,下面的代码实现了语句

```
r->p = &r->a[r->i + r->j];
```

以保存在寄存器`%rdi` 中的 r 为起点:

```
  r 在 %rdi 中
1  movl  4(%rdi), %eax           获取 r->j
2  addl  (%rdi), %eax            加上 r->i
3  cltq                          扩展到 8 字节
4  leaq  8(%rdi,%rax,4), %rax    计算 &r->a[r->i + r->j]
5  movq  %rax, 16(%rdi)          存到 r->p
```

正如这些例子所展示的,选取结构的各个字段完全是在编译时处理的。机器代码中不包含关于字段声明的任何信息,也不包含字段的名字。

给 C 语言初学者　把对象表示为 struct(续)

C++ 和 Java 的对象比 C 的结构更精致:它们还把一组方法与对象关联起来,这些方法可以被调用来执行计算。在 C 中,我们干脆把它们写成普通函数,比如前面给出的 `area` 和 `rotate_left` 函数就是如此。

练习 3.41　考虑下面的结构声明:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

这个声明说明一个结构可以嵌在另一个结构之中,就像数组可以嵌在结构里、数组可以嵌套在数组里一样。

下面的过程(其中省略了一些表达式)对这个结构进行操作:

```
void st_init(struct test *st) {
    st->s.y = ________;
    st->p   = ________;
    st->next = ________;
}
```

A. 下列字段的偏移量(以字节计)各是多少?

`p`

`s.x`

`s.y`

`next`

B. 这个结构总共需要多少字节?

C. 编译器为 `st_init` 生成了下面的汇编代码:

```
 void st_init(struct test *st)
 st 在 %rdi 中
1  st_init:
2    movl  8(%rdi), %eax
3    movl  %eax, 10(%rdi)
4    leaq  10(%rdi), %rax
5    movq  %rax, (%rdi)
6    movq  %rdi, 12(%rdi)
7    ret
```

根据这些信息,请填出 `st_init` 代码中缺失的表达式。

练习 3.42 下面的代码给出了类型 `ACE` 的结构声明以及函数 `test` 的原型:

```
struct ACE {
    short      v;
    struct ACE *p;
};

short test(struct ACE *ptr);
```

当对 fun 的代码进行编译时,gcc 生成了下面的汇编代码:

```
 short test(struct ACE *ptr)
 ptr 在 %rdi 中
1  test:
2    movl   $1, %eax
3    jmp    .L2
4  .L3:
5    imulq  (%rdi), %rax
6    movq   2(%rdi), %rdi
7  .L2:
8    testq  %rdi, %rdi
9    jne    .L3
10   rep; ret
```

A. 施展你的逆向工程本领,写出 `test` 的 C 代码。

B. 描述这个结构所实现的数据结构,以及 `test` 所执行的操作。

3.12.2 联合

联合提供了一种绕开 C 的类型系统的途径，允许一个对象按照多种不同的类型被引用。联合声明的句法与结构完全相同，但它的语义却大不相同。与让不同的字段引用不同的存储块相反，这些字段全部引用同一个存储块。

考虑下面的声明：

```
struct S3 {
    char c;
    int i[2];
    double v;
};

union U3 {
    char c;
    int i[2];
    double v;
};
```

在一台 x86-64 Linux 机器上编译时，字段的偏移量以及数据类型 `S3` 和 `U3` 的总大小如下表所示：

类型	c	i	v	大小
S3	0	4	16	24
U3	0	0	0	8

（我们很快就会看到，为什么在 `S3` 中 i 的偏移量是 4 而不是 1，而 v 的偏移量是 16 而不是 9 或 12。）对于 `union U3 *` 类型的指针 p，引用 `p->c`、`p->i[0]` 和 `p->v` 引用的都是这个数据结构的开头。还可以看到，联合的总大小等于它的字段中最大字段的大小。

联合在好几种场景中都会有用。不过，它们也可能带来令人讨厌的错误，因为它们绕过了 C 类型系统提供的安全保障。有一种应用场景是：我们事先知道，数据结构中两个不同字段的使用是互斥的。这时，把这两个字段声明为联合的组成部分而不是结构的组成部分，就可以减少所分配的总空间。

例如，假设我们想实现一个二叉树数据结构，让每个叶节点带两个 `double` 数据值，让每个内部节点带两个指向子女的指针但不带数据。如果把它声明为

```
struct node_s {
    struct node_s *left;
    struct node_s *right;
    double data[2];
};
```

那么每个节点都需要 32 字节，而每种类型的节点都会浪费掉一半的字节。反过来，如果按下面的方式声明节点：

```
union node_u {
    struct {
        union node_u *left;
        union node_u *right;
    } internal;
    double data[2];
};
```

那么每个节点就只需要 16 字节。若设 n 是指向 `union node_u *` 类型节点的指针，我们会用 `n->data[0]` 和 `n->data[1]` 引用叶节点的数据，而用 `n->internal.left` 和 `n->internal.right` 引用内部节点的子女。

不过，采用这种编码时，我们就没有办法确定给定的一个节点究竟是叶节点还是内部节点了。一种常用方法是引入一个枚举类型，定义出联合各个可能的选择，然后创建一个同时包含一个标签字段和这个联合的结构：

```
typedef enum { N_LEAF, N_INTERNAL } nodetype_t;

struct node_t {
    nodetype_t type;
    union {
        struct {
            struct node_t *left;
            struct node_t *right;
        } internal;
        double data[2];
    } info;
};
```

这个结构总共需要 24 字节：4 字节给 type，或者 `info.internal.left` 和 `info.internal.right` 各 8 字节，或者 `info.data` 占 16 字节。我们很快就会讨论到，在 type 字段与联合的各个成员之间还需要 4 字节的额外填充，使结构的总大小达到 $4 + 4 + 16 = 24$。就本例而言，使用联合带来的节省，与由此产生的代码的别扭程度相比，是很小的。对于字段更多的数据结构来说，这样的节省才能算得上诱人。

联合还可以用来访问不同数据类型的位模式。例如，假设我们用一个简单的强制类型转换，把 `double` 类型的值 d 转换成 `unsigned long` 类型的值 u：

```
unsigned long u = (unsigned long) d;
```

值 u 将是 d 的一个整数表示。除了 d 为 0.0 的情形之外，u 的位表示与 d 的位表示会有天壤之别。现在考虑下面这段代码，它从一个 `double` 生成一个 `unsigned long` 类型的值：

```
unsigned long double2bits(double d) {
    union {
```

```
        double d;
        unsigned long u;
    } temp;
    temp.d = d;
    return temp.u;
};
```

在这段代码中，我们用一种数据类型把实参存进联合，再用另一种数据类型去访问它。这样得到的结果是：u 与 d 有相同的位级表示，其中像符号位、阶码与尾数这些字段，正如

3.11 节所描述的那样。u 的数值与 d 的数值没有关系，d 为 0.0 的情形除外。

把不同大小的数据类型用联合组合在一起时，字节顺序问题就可能变得非常重要。例如，假设我们编写一个过程，用两个 4 字节无符号值所给出的位模式来创建一个 8 字节的 `double`：

```
double uu2double(unsigned word0, unsigned word1)
{
    union {
        double d;
        unsigned u[2];
    } temp;

    temp.u[0] = word0;
    temp.u[1] = word1;
    return temp.d;
}
```

在一台小端法机器（比如 x86-64 处理器）上，参数 `word0` 会成为 d 低端的 4 个字节，而 `word1` 会成为高端的 4 个字节。在大端法机器上，这两个参数的角色正好互换。

练习 3.43 假设分配给你的工作是检查一个 C 编译器是否为结构和联合的访问生成正确的代码。你写出了下面的结构声明：

```
typedef union {
    struct {
        long    u;
        short   v;
        char    w;
    } t1;
    struct {
        int     a[2];
        char    *p;
    } t2;
} u_type;
```

接着你编写了一组形如下式的函数：

```
void get(u_type *up, type *dest) {
    *dest = expr;
}
```

它们分别使用不同的访问表达式 expr，而且目的数据类型 type 依 expr 相关联的类型而定。然后你检查编译这些函数所生成的代码，看看它们是否与你的预期一致。

假设在这些函数中，`up` 和 `dest` 分别被加载到寄存器`%rdi` 和`%rsi` 中。请填写下面这张表，给出数据类型 type 以及一到三条指令组成的序列，用来计算该表达式并把结果存到 `dest`。

expr	type	Code
`up->t1.u`	`long`	`movq (%rdi), %rax  movq %rax, (%rsi)`
`up->t1.v`		
`&up->t1.w`		
`up->t2.a`		
`up->t2.a[up->t1.u]`		
`*up->t2.p`		

3.12.3 数据对齐

许多计算机系统对基本数据类型允许使用的地址作出了限制，要求某些对象的地址必须是某个值 K（通常是 2、4 或 8）的倍数。这种对齐限制简化了构成处理器与存储器系统之间接口的硬件的设计。例如，假设一个处理器每次总是从存储器取出 8 字节，且取数的地址必须是 8 的倍数。如果我们能保证任何 `double` 的地址都是 8 的倍数，那么就可以用一次存储器操作读出或写入这个值；否则可能需要执行两次存储器访问，因为该对象可能被拆放在两个 8 字节的存储器块里。

无论数据是否对齐，x86-64 硬件都能正确工作。不过 Intel 还是建议对数据进行对齐，以提高存储器系统的性能。他们的对齐规则基于这样一个原则：任何 K 字节的基本对象的地址都必须是 K 的倍数。可以看出，这条规则会引出下表所列的对齐要求：

对齐是这样得到实施的：确保每个数据类型都以某种方式加以组织和分配，使该类型内的每个对象都满足各自的对齐限制。编译器会在汇编代码中放置伪指令（directive），指出对全局数据所要求的对齐。例如，3.6.8 节那个跳转表的汇编代码声明在第 2 行包含下面这条伪指令：

K	类型
1	char
2	short
4	int、float
8	long、double、char *

```
.align 8
```

这条伪指令保证它后面的数据(在这里就是跳转表的起始处)将从一个 8 的倍数地址开始。由于每个表项都是 8 字节长,相继的元素就都会满足 8 字节对齐限制。

对于涉及结构的代码,编译器可能需要在字段的分配中插入间隙,使结构的每个元素都满足其对齐要求。于是,这个结构对其起始地址就有了一定的对齐要求。

例如,考虑下面的结构声明:

```
struct S1 {
    int i;
    char c;
    int j;
};
```

假设编译器使用最小化的 9 字节分配,布局示意如下:

偏移 0 4 5 9

内容 i c j

这样就不可能同时满足字段 i(偏移量 0)和字段 j(偏移量 5)这两种 4 字节对齐要求了。于是,编译器在 c 和 j 之间插入一个 3 字节的间隙(下表中空出的那格):

偏移 0 4 5 8 12

内容 i c j

这样 j 的偏移量就是 8,而整个结构的大小是 12 字节。另外,编译器还必须确保类型为 `struct S1 *` 的任何指针 p 满足 4 字节对齐。按照我们前面的记法,设指针 p 的值为 x_p,那么 x_p 必须是 4 的倍数。这就保证了 `p->i`(地址 x_p)和 `p->j`(地址 $x_p + 8$)都满足各自的 4 字节对齐要求。

此外,编译器可能还需要给结构的末尾加上填充,使得结构数组的每个元素都满足其各自的对齐要求。例如,考虑下面的结构声明:

```
struct S2 {
    int i;
```

```
    int j;
    char c;
};
```

如果我们把这个结构打包进 9 字节中，只要确保结构的起始地址满足 4 字节对齐要求，仍然可以让字段 i 和 j 满足对齐要求。不过，考虑下面的声明：

```
struct S2 d[4];
```

按 9 字节分配时，不可能让 d 的每个元素都满足对齐要求，因为这些元素的地址将是 x_d、$x_d + 9$、$x_d + 18$ 和 $x_d + 27$。因此，编译器为结构 `S2` 分配 12 字节，最后的 3 个字节成为浪费的空间：

偏移	0	4	8	9	12
内容	`i`	`j`	`c`		

这样一来，d 的元素的地址就是 x_d、$x_d + 12$、$x_d + 24$ 和 $x_d + 36$。只要 x_d 是 4 的倍数，全部对齐限制就都得到满足了。

边栏　强制对齐的一个实例

对大多数 x86-64 指令来说，数据保持对齐可以提高效率，但并不影响程序行为。不过，某些型号的 Intel 和 AMD 处理器在用一部分实现多媒体运算的 SSE 指令处理非对齐数据时，无法正确执行。这些指令作用于 16 字节的数据块，而在 SSE 单元和存储器之间传送数据的指令要求存储器地址是 16 的倍数。任何以不满足这种对齐的地址去访问内存的企图都会导致异常（见 8.1 节），缺省行为是程序终止。

结果，x86-64 处理器的任何编译器和运行时系统都必须确保：一切为保存某个可能被读入 SSE 寄存器或从 SSE 寄存器写出的数据结构而分配的内存，都必须满足 16 字节对齐。这条要求带来下面两条后果：

- 由内存分配函数（`alloca`、`malloc`、`calloc` 或 `realloc`）生成的任何存储块的起始地址都必须是 16 的倍数。
- 大多数函数的栈帧都必须按 16 字节的边界对齐。（这条要求有不少例外。）

较新版本的 x86-64 处理器实现了 AVX 多媒体指令。除了提供 SSE 指令的一个超集之外，支持 AVX 的处理器也没有强制对齐要求。

练习 3.44　对下面的每一个结构声明，确定每个字段的偏移量、结构的总大小，以及它们在 x86-64 上的对齐要求：

```
A. struct P1 { short i; int c; int *j; short *d; };
B. struct P2 { int i[2]; char c[8]; short s[4]; long *j; };
C. struct P3 { long w[2]; int *c[2] };
D. struct P4 { char w[16]; char *c[2] };
E. struct P5 { struct P4 a[2]; struct P1 t };
```

练习 3.45　对下面的结构声明，回答这些问题：

```
struct {
  int    *a;
  float  b;
  char   c;
  short  d;
  long   e;
  double f;
  int    g;
  char   *h;
} rec;
```

A. 给出结构中所有字段的字节偏移量。

B. 结构的总大小是多少?

C. 重新排列结构中的字段,使浪费的空间减到最少,然后给出重排后结构的字节偏移量和总大小。

3.13 在机器级程序中把控制与数据结合起来

到目前为止,我们已经分别考察了机器级代码如何实现程序的控制方面,以及它如何实现不同的数据结构。在本节中,我们考察数据与控制彼此之间的相互作用。我们首先深入研究指针:它是 C 程序设计语言中最重要的概念之一,但对它许多程序员只有很浅的理解。接下来我们会评述符号调试器 gdb 的用法,用它来考察机器级程序的详细操作过程。然后我们看到,理解机器级程序使我们能够研究缓冲区溢出(buffer overflow),这是许多现实世界系统中的一个重要安全漏洞。最后,我们来考察

机器级程序如何实现这样的情形:一个函数所需的栈存储量随一次执行与另一次执行而变化。

3.13.1 理解指针

指针是 C 程序设计语言的一个核心特性。它们以一种统一的方式来生成对各种数据结构内部元素的引用。对初学编程的程序员来说,指针是造成困惑的一个根源,但其底层概念其实相当简单。我们在这里突出指针的若干关键原理,以及它们到机器代码的映射。

- **每个指针都关联着一个类型。** 这个类型表明该指针所指向的是哪一类对象。用下面的指针声明作为示例:

```
int *ip;
char **cpp;
```

变量 ip 是一个指向 `int` 类型对象的指针,而 cpp 是一个指向某对象的指针,这个对象本身又是指向一个 `char` 类型对象的指针。一般而言,如果对象

具有类型 T，那么指针的类型就是 $*T$。特殊的 `void *` 类型表示通用指针。例如，malloc 函数就返回一个通用指针，可以通过显式强制类型转换，或者通过赋值运算的隐式转换，把它转换为有类型的指针。指针类型并不是机器代码的一部分；它们是 C 提供的一种抽象，帮助程序员避免寻址错误。

- **每个指针都有一个值。** 这个值是某个指定类型对象的地址。特殊的 NULL (0)值表示该指针不指向任何地方。
- **用 & 运算符创建指针。** 这个运算符可以应用于任何表达式，只要这个表达式是所谓的左值 (lvalue)——也就是能够出现在赋值号左边的表达式。这样的例子包括变量以及结构、联合和数组的元素。我们已经看到，& 运算符的机器代码实现常常用 leaq 指令来计算表达式的值，因为这条指令就是设计用来计算存储器引用的地址的。
- **用 * 运算符对指针解引用。** 结果是具有与该指针相关联类型的值。解引用由一次存储器引用实现，或者向指定的地址存放数据，或者从中读取数据。
- **数组和指针紧密相关。** 数组的名字可以当作指针变量来引用(但不能更新它)。对数组的引用(例如 `a[3]`)与指针运算加解引用(例如 `*(a+3)`)具有完全相同的效果。对数组的引用和指针运算都需要按对象大小对偏移量进行缩放。当我们对值为 p 的指针 p 写表达式 $p+i$ 时，结果的地址按 $p+L\cdot i$ 计算，其中 L 是与指针 p 相关联的数据类型的大小。
- **把指针从一种类型强制类型转换为另一种类型，改变的是它的类型，而不是它的值。** 强制类型转换的一个效果就是改变指针运算的缩放细节。所以，如果 p 是一个值为 p 的 `char *` 类型指针，那么表达式 `(int *) p+7` 计算的是 $p+28$，而 `(int *) (p+7)` 计算的是 $p+7$。(回忆一下，强制类型转换的优先级高于加法。)
- **指针也可以指向函数。** 这为保存并传递代码的引用提供了一种很有力的手段，这样的代码可以在程序的其他部分被调用。例如，如果有一个按原型

```
int fun(int x, int *p);
```

定义的函数，那么我们就可以用下面的代码序列来声明指向这个函数的指针 fp 并对它赋值：

```
int (*fp)(int, int *);
fp = fun;
```

然后我们就可以用这个指针调用该函数：

```
int y = 1;
int result = fp(3, &y);
```

函数指针的值就是该函数的机器代码表示中第一条指令的地址。

> **给 C 语言初学者　函数指针**
> 声明函数指针的句法对初学编程的程序员来说尤其难懂。对于形如
>
> ```
> int (*f)(int*);
> ```
>
> 的声明，从里向外读会有所帮助（也就是说，从“f”开始向外读）。这样我们可以看出，f 是一个指针，这一点由 `(*f)` 表明。它是一个指向函数的指针，这个函数以单个 `int *` 作为参数，这一点由 `(*f)(int*)` 表明。最后我们看到，它是一个指向函数的指针，该函数以一个 `int *` 作为参数并返回 `int`。
> `*f` 两边的括号是必需的，因为没有它们，声明
>
> ```
> int *f(int*);
> ```
>
> 就会被读作
>
> ```
> (int *) f(int*);
> ```
>
> 也就是说，它会被解释为一个函数原型，声明一个函数 f：它以 `int *` 为参数并返回 `int *`。
> Kernighan 和 Ritchie [61，5.12 节] 给出了一个阅读 C 声明的有益教程。

3.13.2 现实中的情况：使用 gdb 调试器

GNU 调试器 gdb 提供了许多有用的特性，支持对机器级程序的运行时求值和分析。在本书的示例和练习中，我们试图做到仅仅通过阅读代码来推断程序的行为。而使用 gdb，就可能在观察程序运行的同时研究它的行为，并对它的执行有相当大的控制权。

图 3.39 给出了一些 gdb 命令的例子，在处理机器级 x86-64 程序时它们对你有帮助。先运行 `objdump` 得到程序的反汇编版本，是很有帮助的。我们的示例基于对文件 prog 运行 gdb，这个文件在 3.2.2 节中生成并反汇编。我们用下面的命令行启动 gdb：

```
linux> gdb prog
```

总的做法是在程序中感兴趣的点附近设置断点。断点可以设置在刚进入某个函数之后，也可以设置在某个程序地址处。程序执行过程中遇到了一个断点时，它会停下，把控制交还给用户。在断点处，我们可以以各种格式检查不同的寄存器和存储器位置。我们还可以对程序单步执行，一次只运行几条指令，或者继续执行到下一个断点。

图 3.39　gdb 命令示例。这些例子说明了 gdb 支持调试机器级程序的一些方法。

命令	效果
启动和停止	
`quit`	退出 gdb
`run`	运行你的程序(在这里给出命令行参数)
`kill`	终止你的程序
断点	
`break multstore`	在函数 multstore 的入口处设置断点
`break *0x400540`	在地址 `0x400540` 处设置断点
`delete 1`	删除断点 1
`delete`	删除所有断点
执行	
`stepi`	执行一条指令
`stepi 4`	执行四条指令
`nexti`	与 stepi 类似,但继续步进通过函数调用
`continue`	恢复执行
`finish`	运行直到当前函数返回
检查代码	
`disas`	反汇编当前函数
`disas multstore`	反汇编函数 multstore
`disas 0x400544`	反汇编地址 `0x400544` 附近的代码
`disas 0x400540, 0x40054d`	反汇编指定地址范围内的代码
`print /x $rip`	以十六进制打印程序计数器
检查数据	
`print $rax`	以十进制打印`%rax` 的内容
`print /x $rax`	以十六进制打印`%rax` 的内容
`print /t $rax`	以二进制打印`%rax` 的内容
`print 0x100`	打印 `0x100` 的十进制表示
`print /x 555`	打印 555 的十六进制表示
`print /x ($rsp+8)`	以十六进制打印`%rsp` 的内容加 8
`print *(long *) 0x7fffffffe818`	打印地址 `0x7fffffffe818` 处的长整数
`print *(long *) ($rsp+8)`	打印地址`%rsp+8` 处的长整数
`x/2g 0x7fffffffe818`	检查从地址 `0x7fffffffe818` 开始的两个(8 字节)字
`x/20b multstore`	检查函数 multstore 的前 20 字节
有用的信息	
`info frame`	关于当前栈帧的信息
`info registers`	所有寄存器的值
`help`	获取 gdb 的信息

从我们的例子可以看出，gdb 的命令语法晦涩难懂，但在线帮助信息（在 gdb 内用 `help` 命令调出）弥补了这个缺点。许多程序员不喜欢使用 gdb 的命令行界面，而是更愿意使用 ddd——gdb 的一个扩展，它提供了图形用户界面。

3.13.3 越界内存引用和缓冲区溢出

我们已经看到，C 对数组引用不执行任何边界检查，而且局部变量和诸如保存的寄存器值及返回地址这样的状态信息一起存放在栈上。这两件事结合起来可能导致严重的程序错误：向越界的数组元素写入，会破坏保存在栈上的状态。接下来，当程序试图重新加载寄存器或者带着被破坏的状态执行 `ret` 指令时，就会出严重的问题。

状态破坏的一种特别常见的来源叫作缓冲区溢出（buffer overflow）。典型的情况是，在栈上分配某个字符数组来保存一个字符串，但字符串的大小超过了为数组分配的空间。下面的程序示例演示了这一点：

```
/* 库函数 gets() 的实现 */
char *gets(char *s)
{
    int c;
    char *dest = s;

    while ((c = getchar()) != '\n' && c != EOF)
        *dest++ = c;
    if (c == EOF && dest == s)
        /* 未读到任何字符 */
        return NULL;
    *dest++ = '\0'; /* 结束字符串 */
    return s;
}

/* 读入一行输入并把它写回 */
void echo()
{
    char buf[8]; /* 实在太小了！ */
    gets(buf);
    puts(buf);
}
```

前面这段代码给出了库函数 gets 的一个实现，用它来演示这个函数的一个严重问题。它从标准输入读入一行，在遇到终止换行字符或者其他错误条件时停止。它把这个字符串复制到参数 s 指定的位置，并在字符串末端加上空字符作为结束。我们在函数 `echo` 中展示 gets 的用法，这个函数只是从标准输入读取一行，并把它回显到标准输出。

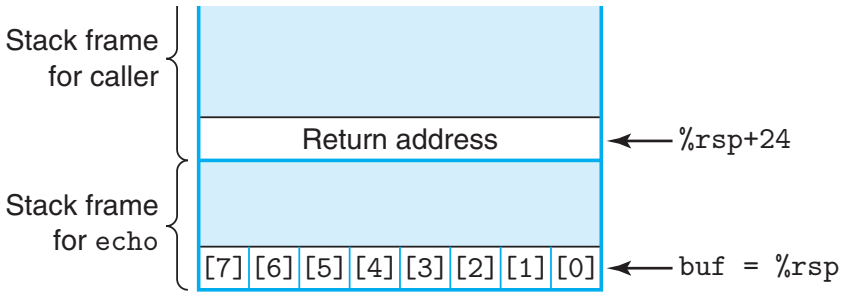


图 3.40 （示意图）

gets 的问题在于它没有办法判断是否已经为保存整个字符串分配了足够的空间。在我们的 echo 示例中，我们故意把缓冲区做得非常小——只有 8 个字符长。任何超过 7 个字符的字符串都会导致一次越界写。

通过检查 gcc 为 echo 生成的汇编代码，我们可以推断出栈的组织方式：

```
  void echo()
1 echo:
2    subq   $24, %rsp              在栈上分配 24 字节
3    movq   %rsp, %rdi             把 buf 计算为 %rsp
4    call   gets                   调用 gets
5    movq   %rsp, %rdi             把 buf 计算为 %rsp
6    call   puts                   调用 puts
7    addq   $24, %rsp              释放栈空间
8    ret                           返回
```

图 3.40 说明了 echo 执行期间的栈组织。程序用栈指针减去 24 的办法在栈上分配 24 字节(第 2 行)。字符数组 buf 位于栈的顶端——

输入的字符数	被破坏的附加状态
0–7	无
9–23	未使用的栈空间
24–31	返回地址
32+	调用者中保存的状态

长度不超过 23 个字符的字符串不会造成严重后果，但超过这个长度之后，返回指针的值、还可能加上其他一些已保存的状态，就会遭到破坏。如果保存的返回地址的值被破坏了，那么 `ret` 指令(第 8 行)就会让程序跳转到一个完全出乎意料的位置。仅仅根据 C 代码，这些行为没有一个看上去是可能的。像 gets 这样对存储器越界写的函数所带来的影响，只有通过在机器码一级研究程序才能理解。

我们的 echo 代码简单但草率。更好的版本是使用函数 `fgets`，它带有一个参数，指明最多可以读取的字节数。家庭作业 3.71 要求你编写一个能够处

理任意长度输入字符串的 echo 函数。一般说来，使用 gets 或者任何可能使存储溢出的函数都被认为是坏的编程习惯。遗憾的是，许多常用的库函数，包括 `strcpy`、`strcat` 和 `sprintf`，都具有这样的性质：它们可以生成一个字节序列，却不被告知目的缓冲区的大小 [97]。这类情况会带来缓冲区溢出漏洞。

练习 3.46　图 3.41 给出了一个函数的（低质量）实现，该函数从标准输入读入一行，把字符串复制到新分配的存储区，并返回指向结果的指针。

考虑下面的场景。`get_line` 过程在被调用时，返回地址等于 `0x400776`，寄存器`%rbx` 等于 `0x0123456789ABCDEF`。你键入下面的字符串：

```
0123456789012345678901234
```

```
(a) C 代码

/* 这段代码的质量非常低。
   它旨在说明糟糕的编程习惯做法。
   参见练习 3.46。 */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}

(b) 到调用 gets 为止的反汇编代码

    char *get_line()
1  0000000000400720 <get_line>:
2    400720:  53              push   %rbx
3    400721:  48 83 ec 10     sub    $0x10,%rsp
       在此处绘制此点的栈图
4    400725:  48 89 e7        mov    %rsp,%rdi
5    400728:  e8 73 ff ff ff  callq  4006a0 <gets>
       修改图以显示此时栈的内容
```

图 3.41　练习 3.46 的 C 代码和反汇编代码。

程序以段错误终止。你运行 gdb，确定错误发生在 `get_line` 的 ret 指令执行过程中。

- **A.** 填写下面给出的图，尽可能多地指出反汇编第 3 行的指令刚执行完之后关于栈的信息。把存储在栈中的量（例如“返回地址”）标注在右侧，把它们已知的十六进制值标注在框内，每个框代表 8 字节。指出`%rsp` 的位置。回忆一下，字符 0–9 的 ASCII 码是 `0x30`–`0x39`。
- **B.** 修改你的图，展示出调用 gets（第 5 行）的效果。

00 00 00 00 00 40 00 76	返回地址

- **C.** 程序试图返回到哪个地址?
- **D.** 当 `get_line` 返回时,哪些寄存器带有被破坏的值?
- **E.** 除了缓冲区溢出的潜在危险之外,`get_line` 的代码还有哪两处错误?

缓冲区溢出的一种更加阴险的用法,是让程序去执行一个它本来不会去执行的功能。这是通过网络攻击计算机系统安全的最常用方法之一。通常的做法是给程序喂入一个字符串,它既包含某段可执行代码的字节编码(称为攻击代码,exploit code),又包含一些额外的字节,用一个指向攻击代码的指针来覆写返回地址。这样,执行 `ret` 指令的结果就是跳转到攻击代码。

在一种攻击形式中,攻击代码随后用一次系统调用启动一个 shell 程序,给攻击者提供一系列操作系统的功能。在另一种攻击形式中,攻击代码完成某项在平时未被授权的任务,修复对栈的破坏,然后再执行一次 ret,造成一次(看起来)正常地返回到调用者。

举个例子,1988 年 11 月著名的因特网蠕虫用了四种不同的方法来获得对因特网上大量计算机的访问权。其中一种方法是对 finger 守护进程 fingerd 的缓冲区溢出攻击,该守护进程为 finger 命令的请求提供服务。通过带上适当的字符串来调用 finger,蠕虫可以让远程站点上的守护进程发生缓冲区溢出,并执行一段代码,把对远程系统的访问权交给蠕虫。蠕虫一旦获得了一个系统的访问权,它就会自我复制,几乎耗尽机器的全部计算资源。结果是,数以百计的机器实际上瘫痪了,直到安全专家研究出如何清除蠕虫为止。蠕虫的作者被抓获并受到起诉,被判处 3 年缓刑、400 小时的社区服务,并处罚金 10,500 美元。不过时至今日,人们仍然不断在系统中发现使系统容易遭受缓冲区溢出攻击的安全漏洞。这凸显了小心编程的必要性:与外部环境的任何接口都应当做到“防弹”,使任何外部行为体的举动都不能造成系统行为异常。

3.13.4 阻止缓冲区溢出攻击

缓冲区溢出攻击已经变得如此普遍,给计算机系统造成了这么多问题,以致现代的编译器和操作系统都实现了相应的机制,使发动这类攻击更加困难,并限制入侵者通过缓冲区溢出攻击控制系统的途径。本节将介绍 Linux 上最新版本的 gcc 提供的机制。

栈随机化

> **边栏　蠕虫和病毒**
> 蠕虫和病毒都是一些试图在计算机之间传播自己的代码。正如 Spafford [105] 所描述的，蠕虫是一个可以自行运行的程序，它能够把一个功能完全正常的自身版本传播到其他的机器上。病毒则是一段把自己加到其他程序（包括操作系统）中的代码，它不能独立运行。在大众媒体中，“病毒”一词被用来指各种在系统之间传播攻击代码的策略，所以你会听到人们把本应更准确地称为“蠕虫”的东西说成“病毒”。

为了把攻击代码插入一个系统，攻击者必须把代码本身和一个指向这段代码的指针都作为攻击字符串的一部分注入进来。生成这个指针需要知道字符串将位于哪个栈地址。历史上，一个程序的栈地址是高度可预测的：对运行相同程序和操作系统版本组合的所有系统来说，栈的位置在许多机器上都相当稳定。因此，举例来说，如果攻击者能够确定一个常见的 Web 服务器所使用的栈地址，它就可以设计出一个对很多机器都能奏效的攻击。以传染病作类比，这就等于许多系统都容易感染同一种病毒，这种现象常被称为安全单一化（security monoculture）[96]。

栈随机化的思想是让栈的位置在程序的一次运行与另一次运行之间有所变化。这样，即使许多机器运行的是完全相同的代码，它们也都会使用不同的栈地址。这可以在程序开始时在栈上分配一个范围在 0 到 n 字节之间的随机大小的空间来实现，例如使用分配函数 `alloca`，它在栈上为指定数量的字节分配空间。程序并不使用被分配的这段空间，但它会让后续所有的栈位置在程序的一次执行与另一次执行之间发生改变。分配范围 n 需要大到足以使栈地址产生足够的变化，同时又必须小到不浪费太多程序空间。

下面的代码给出了一种确定“典型”栈地址的简单方法：

```
int main() {
    long local;
    printf("local at %p\n", &local);
    return 0;
}
```

这段代码只是打印 main 函数中一个局部变量的地址。在一台 32 位模式的 Linux 机器上，这段代码运行 10,000 次，地址的范围从 `0xff7fc59c` 到 `0xffffd09c`，范围大约是 2^{23}。在较新的机器上以 64 位模式运行时，地址的范围从 `0x7fff0001b698` 到 `0x7ffffffaa4a8`，范围接近 2^{32}。

栈随机化已经成为 Linux 系统中的标准做法。它属于一类更庞大的技术，称为地址空间布局随机化（address-space layout randomization，ASLR）[99]。使用 ASLR，程序的各个不同的部分，包括程序代码、库代码、栈、全局变量和堆数

据，都会在程序每次运行时被加载到存储器的不同区域。这意味着在一台机器上运行的一个程序，与在另一台机器上运行的同一个程序，会有非常不同的地址映射。这能够阻止某些形式的攻击。

不过总体说来，一个执着的攻击者可以通过蛮力克服随机化，用不同的地址反复尝试攻击。一种常见的技巧是在实际的攻击代码之前加入很长的一串 nop（读音为“no op”，是“no operation”的简写）指令。执行这条指令除了把程序计数器递增到下一条指令之外没有任何效果。只要攻击者能够猜中这个序列中间的某个地址，程序就会一路滑过这个序列，然后碰到攻击代码。这类序列的通用术语是“nop 空橇（nop sled）”，[97] 表达的是程序“滑过”这个序列的思想。如果我们建立一个 256 字节的 nop 空橇，那么通过枚举 $2^{15} = 32,768$ 个起始地址，就可以破解对 $n = 2^{23}$ 的随机化，这对一个决心坚定的攻击者来说完全可行。对 64 位的情况，试图枚举 $2^{24} = 16,777,216$ 个地址就有点令人生畏了。可以看出，栈随机化和 ASLR 的其他方面会增加成功攻击一个系统所需的工作量，从而大大降低病毒或蠕虫的传播速度，但它不能提供完全的保护。

练习 3.47　在一台运行 Linux 版本 2.6.16 的系统上，栈检查代码运行了 10,000 次，得到的地址范围从最小的 `0xffffb754` 到最大的 `0xffffd754`。

A. 地址的近似范围有多大？

B. 如果我们尝试用 128 字节的 nop 空橇来进行缓冲区越界攻击，测试所有可能的起始地址大约需要多少次尝试？

栈破坏检测

第二道防线是能够检测到栈何时被破坏了。我们在 echo 函数（图 3.40）的例子中已经看到，破坏通常发生在程序越过了局部缓冲区的边界时。在 C 中，没有一种可靠的方法能够防止对数组边界之外的写入。作为替代，程序可以尝试在这种写产生任何有害影响之前就检测出它已经发生了。

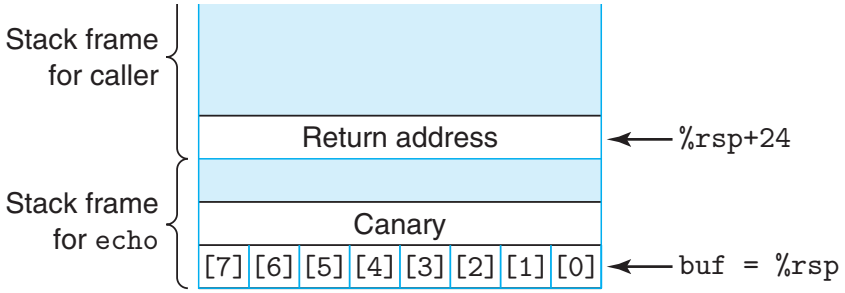


图 3.42 （示意图）

最新版本的 gcc 在生成的代码中加入了一种称为栈保护者(stack protector)的机制,用来检测缓冲区越界。其思想是在任何局部缓冲区与栈状态的其余部分之间存放一个特殊的金丝雀值(canary value[4]),如图 3.42 所示 [26, 97]。这个金丝雀值也称为保护值(guard value),每次程序运行时都会随机生成,因此攻击者没有容易的办法来确定它的值。在恢复寄存器状态并从函数返回之前,程序会检查这个金丝雀值是否已被本函数或者它调用的某个函数的操作改动。如果是,程序就报错退出。

最新版本的 gcc 会尝试判断一个函数是否容易遭受栈溢出的攻击,然后在必要时自动插入这种类型的溢出检测代码。实际上,在我们前面演示栈溢出时,我们必须给出命令行选项 `-fno-stack-protector`,才能阻止 gcc 插入这段代码。不带这个选项来编译 echo 函数(也就是启用栈保护者),会得到如下的汇编代码:

```
  void echo()
1  echo:
2     subq   $24, %rsp              在栈上分配 24 字节
3     movq   %fs:40, %rax           获取金丝雀值
4     movq   %rax, 8(%rsp)          存到栈上
5     xorl   %eax, %eax             将寄存器清零
6     movq   %rsp, %rdi             把 buf 计算为 %rsp
7     call   gets                   调用 gets
8     movq   %rsp, %rdi             把 buf 计算为 %rsp
9     call   puts                   调用 puts
10    movq   8(%rsp), %rax          获取金丝雀值
11    xorq   %fs:40, %rax           与存储的值比较
12    je     .L9                    如果相等,跳转到 ok
13    call   __stack_chk_fail       栈被破坏了!
14 .L9:                             ok:
15    addq   $24, %rsp              释放栈空间
16    ret
```

我们看到,这个版本的函数会从存储器中取出一个值(第 3 行),并把它存放在栈上距`%rsp` 偏移量为 8 的位置,刚好在为 buf 分配的区域之外。指令参数`%fs:40` 表明,金丝雀值是用段寻址(segmented addressing)从存储器中读取的——这是一种可以追溯到 80286 的寻址机制,在如今系统上运行的程序中已经很少见到了。把金丝雀值存放在一个特殊的段中,可以把该段标记为“只读”,这样攻击者就无法覆写存储的金丝雀值。在恢复寄存器状态并返回之前,函数会把栈位置上存储的值与金丝雀值进行比较(通过第 11 行的 xorq 指令)。如果两个值相同,xorq 指令会得到 0,函数就按通常的方式完成。一个非零值表明栈上的金丝雀值被修改过,于是代码会调用一个错误例程。

[4]“canary”这个名称源于历史上用这种鸟来检测煤矿中是否存在危险气体。

栈保护对防止缓冲区溢出攻击破坏存储在程序栈上的状态有很好的效果。它带来的性能损失也很小，尤其是因为 gcc 只在函数内含有 `char` 类型的局部缓冲区时才插入这项检查。当然，还存在其他破坏正在执行的程序的状态的方法，不过降低栈的脆弱性就能挫败许多常见的攻击策略。

练习 3.48 函数 intlen、len 和 iptoa 提供了一种非常迂回的计算表示一个整数所需的十进制数字个数的方法。我们用它来研究 gcc 栈保护者机制的一些方面。

```
int len(char *s) {
    return strlen(s);
}

void iptoa(char *s, long *p) {
    long val = *p;
    sprintf(s, "%ld", val);
}

int intlen(long x) {
    long v;
    char buf[12];
    v = x;
    iptoa(buf, &v);
    return len(buf);
}
```

下面给出的是 intlen 代码的片段，既有不带栈保护者编译的，也有带栈保护者编译的：

```
(a) 不带保护

  int intlen(long x)
  x 在 %rdi
1  intlen:
2    subq    $40, %rsp
3    movq    %rdi, 24(%rsp)
4    leaq    24(%rsp), %rsi
5    movq    %rsp, %rdi
6    call    iptoa
```

```
(b) 带保护

  int intlen(long x)
  x 在 %rdi
1  intlen:
2    subq    $56, %rsp
3    movq    %fs:40, %rax
4    movq    %rax, 40(%rsp)
5    xorl    %eax, %eax
6    movq    %rdi, 8(%rsp)
```

```
7    leaq    8(%rsp), %rsi
8    leaq    16(%rsp), %rdi
9    call    iptoa
```

A. 对两个版本分别回答：buf、v，以及（如果存在的）金丝雀值在栈帧中的位置各是什么？

B. 带保护的代码中对局部变量重新排列了顺序，这种新的顺序如何为抵御缓冲区越界攻击提供更好的安全保障？

限制可执行代码区域

最后一步是消除攻击者将可执行代码插入系统的能力。一种方法是限制哪些存储器区域能够容纳可执行代码。在典型的程序中，只有存放编译器所生成代码的那部分存储器才需要是可执行的；其余部分可以限制为只允许读和写。如我们在第 9 章将会看到的，虚拟内存空间在逻辑上被划分为页，每页通常为 2,048 或 4,096 字节。硬件支持各种形式的存储保护，用来指明允许用户程序和操作系统内核进行的访问形式。许多系统允许对三种访问形式进行控制：读（从存储器读出数据）、写（把数据存入存储器）和执行（把存储器的内容当作机器级代码）。历史上，x86 体系结构把读和执行的访问控制合并成一个 1 位的标志，因此任何被标记为可读的页同时也是可执行的。栈必须既可读又可写，于是栈上的字节也就是可执行的。人们实现过各种方案，想要把一些页限制为可读但不可执行，不过这些方案通常会带来显著的低效率。

最近，AMD 在它的 64 位处理器的存储保护中引入了 NX（“no-execute”，不可执行）位，把读和执行这两种访问模式区分开了，Intel 也随之效仿。有了这个特性，栈就可以被标记为可读、可写但不可执行，而一页是否可执行的检查在硬件中完成，不带来效率损失。

有些类型的程序需要具备动态生成并执行代码的能力。例如，“即时”（just-in-time）编译技术会为用 Java 这样的解释型语言编写的程序动态生成代码，以提高执行性能。至于运行时系统是否能够把可执行代码限制为只是编译器在创建原始程序时生成的那部分代码，则取决于语言和操作系统。

我们已经概述的这些技术——随机化、栈保护和限制哪些部分存储器能够容纳可执行代码——是最常用的把程序对缓冲区溢出攻击的脆弱性减到最小的三种机制。它们都具有这样的性质：不需要程序员付出特别的努力，而且只带来极小的或者没有性能损失。每一种单独使用都能降低脆弱程度，而组合起来使用效果就更强。遗憾的是，仍然有办法攻击计算机 [85, 97]，所以蠕虫和病毒还在继续破坏许多机器的完整性。

3.13.5 支持可变大小的栈帧

到目前为止，我们已经考察了多种函数的机器级代码，但它们都有一个共同性质：编译器可以预先确定必须为它们的栈帧分配多少空间。不过，有些函数需要数量可变的局部存储。这种情况可能发生，例如，函数调用了 `alloca`——一个可以在栈上分配任意数量字节存储空间的标准库函数；当代码声明了一个长度可变的局部数组时，也会有这种情况。

虽然本节呈现的信息理应被当作过程实现的一个方面，但是我们还是把这些内容推迟到了这里，因为它们需要对数组以及对齐的理解。

图 3.43(a) 的代码给出了一个包含可变大小数组的函数示例。该函数声明了局部数组 p，这是一个由 n 个指针组成的数组，n 由第一个参数给出。这就需要在栈上分配 $8n$ 字节的空间，而 n 的值可能随函数的每次调用而变化，因此编译器无法确定必须为该函数的栈帧分配多少空间。此外，程序生成了一个对局部变量 i 的地址的引用，所以这个变量也必须存放在栈上。执行过程中，程序既要能访问局部变量 i，又要能访问数组 p 的元素。返回时，函数必须释放栈帧，并把栈指针置于保存的返回地址所在的位置。

为了管理可变大小的栈帧，x86-64 代码用寄存器`%rbp` 作为帧指针（frame pointer），有时也称为基指针（base pointer）——这正是`%rbp` 中字母 `bp` 的由来。使用帧指针时，栈帧的组织如图 3.44 给出的函数 vframe 的情况所示。我们看到，代码必须先在栈上保存`%rbp` 的前一个值，因为它是一个被调用者保存的寄存器；然后在整个函数执行期间让`%rbp` 一直指向这个位置，并以相对于`%rbp` 的偏移量来引用固定长度的局部变量，例如 i。

```
(a) C 代码

long vframe(long n, long idx, long *q) {
    long i;
    long *p[n];
    p[0] = &i;
    for (i = 1; i < n; i++)
        p[i] = q;
    return *p[idx];
}

(b) 生成的汇编代码片段

    long vframe(long n, long idx, long *q)
    n 在 %rdi 中,idx 在 %rsi 中,q 在 %rdx 中
    只给出部分代码
1  vframe:
2    pushq   %rbp              保存原来的 %rbp
3    movq    %rsp, %rbp        设置帧指针
4    subq    $16, %rsp         为 i 分配空间(%rsp = s1)
5    leaq    22(%rdi,8), %rax
```

```
6    andq    $-16, %rax
7    subq    %rax, %rsp          为数组 p 分配空间(%rsp = s2)
8    leaq    7(%rsp), %rax
9    shrq    $3, %rax
10   leaq    0(,%rax,8), %r8     把 %r8 设为 &p[0]
11   movq    %r8, %rcx           把 %rcx 设为 &p[0](%rcx = p)
        . . .
     初始化循环的代码
     i 在 %rax 中且在栈上,n 在 %rdi 中,p 在 %rcx 中,q 在 %rdx 中
12   .L3:                    loop:
13   movq    %rdx, (%rcx,%rax,8)     把 p[i] 设为 q
14   addq    $1, %rax                i 加 1
15   movq    %rax, -8(%rbp)          存到栈上
16   .L2:
17   movq    -8(%rbp), %rax          从栈中取出 i
18   cmpq    %rdi, %rax              比较 i:n
19   jl      .L3                     若 <,转到 loop
        . . .
     函数退出部分的代码
20   leave                           恢复 %rbp 和 %rsp
21   ret                             返回
```

图 3.43　需要使用帧指针的函数。可变大小的数组意味着栈帧的大小无法在编译时确定。

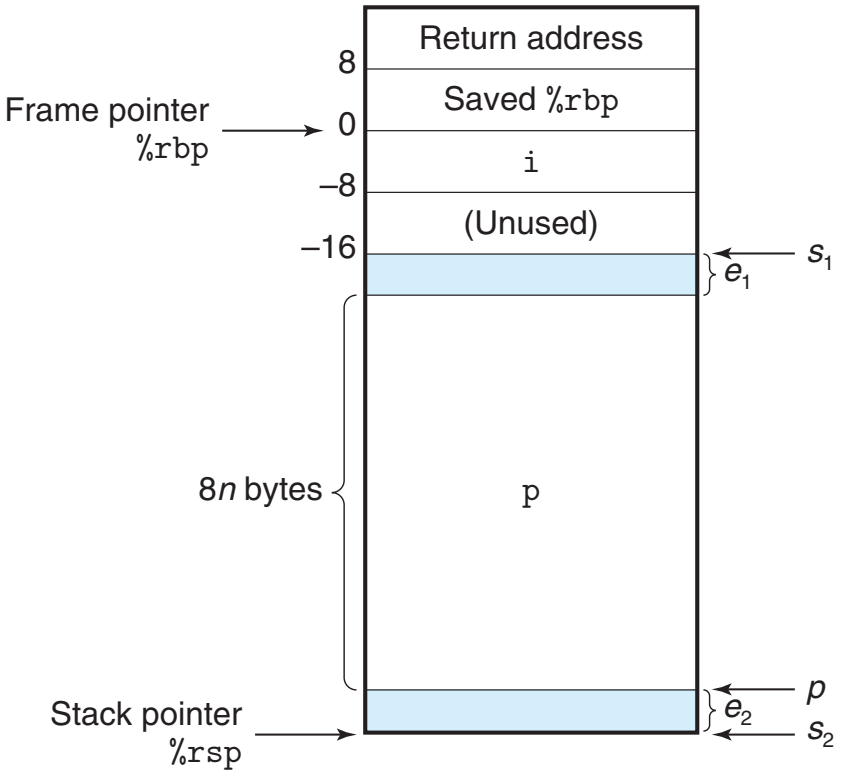


图 3.44 （示意图）

图 3.43(b) 给出了 gcc 为函数 vframe 生成的代码的片段。在函数的开头，我们看到建立栈帧和分配数组 p 的空间的那部分代码。代码首先把%rbp 的当前值压入栈，并把%rbp 设置为指向这个栈位置（第 2–3 行）。接下来，它在栈上分配 16 字节，前 8 字节用来存储局部变量 i，后 8 字节没有使用。然后它为数组 p 分配空间（第 5–11 行）。至于它分配了多少空间，以及它把 p 放在这个空间的什么位置，这些细节在练习 3.49 中探讨。简单地说，当程序进行到第 11 行时，它已经（1）在栈上分配了至少 $8n$ 字节，（2）把数组 p 放在该已分配区域内，使得至少有 $8n$ 字节可供它使用。

初始化循环的代码展示了引用局部变量 i 和 p 的例子。第 13 行把数组元素 p[i] 设置为 q，这条指令用寄存器%rcx 中的值作为数组 p 的起始地址。我们可以看到更新（第 15 行）和读取（第 17 行）局部变量 i 的情形。i 的地址由引用 -8(%rbp) 给出——也就是相对帧指针偏移量为 −8 的位置。

在函数的结尾部分，帧指针通过一条 leave 指令（第 20 行）恢复为它以前的值。这条指令不带任何参数。它等价于执行下面这两条指令：

```
movq %rbp, %rsp      把栈指针设置为帧的开始
popq %rbp            恢复保存的 %rbp,并把栈指针设置到调用者帧的末尾
```

也就是说，栈指针首先被设置为保存的%rbp 值所在的位置，然后这个值从栈中弹出进入%rbp。这条指令组合的效果是释放整个栈帧。

在早期版本的 x86 代码中，每次函数调用都使用帧指针。而在 x86-64 代码中，只有当栈帧可能是变长的时候才使用帧指针，函数 vframe 的情况正是如此。从历史上看，大多数编译器在生成 IA32 代码时都使用帧指针，而最近版本的 gcc 放弃了这个惯例。可以观察到，把使用帧指针的代码与不使用帧指针的代码混合在一起是可以的，只要所有函数都把%rbp 当作被调用者保存的寄存器来对待。

练习 3.49　在本题中，我们将探讨图 3.43(b) 中第 5–11 行代码的逻辑，这条代码为可变数组 p 分配空间。正如代码的注解所说明的，令 s_1 表示在执行第 4 行的 subq 指令之后栈指针的地址；这条指令为局部变量 i 分配空间。令 s_2 表示在执行第 7 行的 subq 指令之后栈指针的值；这条指令为局部数组 p 分配存储空间。最后，令 p 表示在第 10–11 行的指令中赋给寄存器%r8 和%rcx 的值；这两个寄存器都用来引用数组 p。

图 3.44 的右侧用图形描绘了 s_1、s_2 和 p 所指示的各位置。它还表明 s_1 和 p 的值之间可能存在 e_2 字节的偏移量，这部分空间不会被使用。s_1 指示的位置和数组 p 末端之间还可能存在 e_1 字节的偏移量。

A. 用数学术语解释第 5–7 行计算 s_2 的逻辑。提示:想一想 −16 的位级表示,以及它对第 6 行 `andq` 指令的影响。

B. 用数学术语解释第 8–10 行计算 p 的逻辑。提示:你可能需要参考 2.3.7 节中关于除以 2 的幂的讨论。

■ **C.** 对于下面给出的 n 和 s_1 的取值,跟踪代码的执行过程,确定 s_2、p、e_1 和 e_2 的结果值:

n	s_1	s_2	p	e_1	e_2
5	2,065				
6	2,064				

■ **D.** 这段代码对 s_2 和 p 的值保证了什么对齐性质?

3.14 浮点代码

处理器的浮点体系结构包括了影响在浮点数据上进行运算的程序如何映射到机器的各个方面,包括:

- 浮点值如何存取。这通常是通过某种形式的寄存器来完成的。
- 对浮点数据进行操作的指令。
- 作为参数把浮点值传递给函数以及作为结果返回它们时所采用的约定。
- 在函数调用期间如何保存寄存器的约定——例如,一些寄存器被指定为调用者保存,另一些为被调用者保存。

要理解 x86-64 的浮点体系结构,从历史的角度稍作观察是有帮助的。自 1997 年 Pentium/MMX 引入以来,Intel 和 AMD 都在不断地加入相继的几代媒体指令,用来支持图形和图像处理。这些指令最初的着眼点是,允许把多个操作放在一种称为单指令多数据(single instruction, multiple data, SIMD,读音为 “sim-dee”)的并行模式下执行。在这种模式下,同一个操作对许多不同的数据值并行进行。多年来,这类扩展经历了一系列重大修订,名称也从 MMX 改为 SSE(“streaming SIMD extensions”,流式 SIMD 扩展),最近又改为 AVX(“advanced vector extensions”,高级向量扩展)。每一代内部还有不同的版本。每种扩展都用一组寄存器来管理数据,分别被称为 MMX 的 “MM” 寄存器、SSE 的“XMM”寄存器和 AVX 的“YMM”寄存器,其大小从 MM 寄存器的 64 位,到 XMM 的 128 位,再到 YMM 的 256 位。例如,每个 YMM 寄存器可以存放 8 个 32 位的值,或者 4 个 64 位的值,这些值既可以是整数,也可以是浮点数。

从 2000 年 Pentium 4 引入的 SSE2 开始，媒体指令就包含了对标量浮点数据进行操作的指令，这类指令使用 XMM 或 YMM 寄存器低 32 位或低 64 位中的单个值。这种标量模式提供了一组寄存器和指令，更加接近于其他处理器支持浮点的方式。所有能够执行 x86-64 代码的处理器都支持 SSE2 或更高版本，因此 x86-64 的浮点方式以 SSE 或 AVX 为基础，包括了传递过程参数和返回值的约定 [77]。

我们的讲述以 AVX2 为基础，它是 AVX 的第二个版本，随 2013 年的 Core i7 Haswell 处理器引入。在命令行中给出参数 `-mavx2` 时，gcc 会生成 AVX2 代码。基于 SSE 各个不同版本以及 AVX 第一个版本的代码在概念上与它是相似的，尽管指令的名称和格式有所不同。我们只介绍在用 gcc 编译浮点程序时出现的那些指令。这些指令大多数是标量 AVX 指令，不过我们也会注明那些适合对整个数据向量做运算的指令出现的场合。关于如何利用 SSE 和 AVX 的 SIMD 能力，网络边栏 OPT:SIMD 给出了更完整的介绍。读者可以参考 AMD 和 Intel 的文档来了解各条指令的细节 [4, 51]。与整数操作一样，请注意，我们讲述中使用的 ATT 格式与这些文档使用的 Intel 格式不同，这两个版本列出指令操作数的顺序是不一样的。

如图 3.45 所示，AVX 浮点体系结构允许把数据存放在 16 个 YMM 寄存器中，这些寄存器的名字是`%ymm0`–`%ymm15`。每个 YMM 寄存器长 256 位（32 字节）。对标量数据进行操作时，这些寄存器只保存浮点数据，并且只用到低 32 位（float）或低 64 位（double）。汇编代码用它们的 SSE XMM 寄存器名`%xmm0`–`%xmm15` 来引用这些寄存器，其中每个 XMM 寄存器是对应的 YMM 寄存器低 128 位（16 字节）。

图 3.46 浮点传送指令。这些操作在存储器和寄存器之间以及成对的寄存器之间传送值。（X：XMM 寄存器（例如`%xmm3`）；M32：32 位存储器区域；M64：64 位存储器区域）

指令	源	目的	描述
`vmovss`	M32	X	传送单精度
`vmovss`	X	M32	传送单精度
`vmovsd`	M64	X	传送双精度
`vmovsd`	X	M64	传送双精度
`vmovaps`	X	X	传送对齐的打包单精度
`vmovapd`	X	X	传送对齐的打包双精度

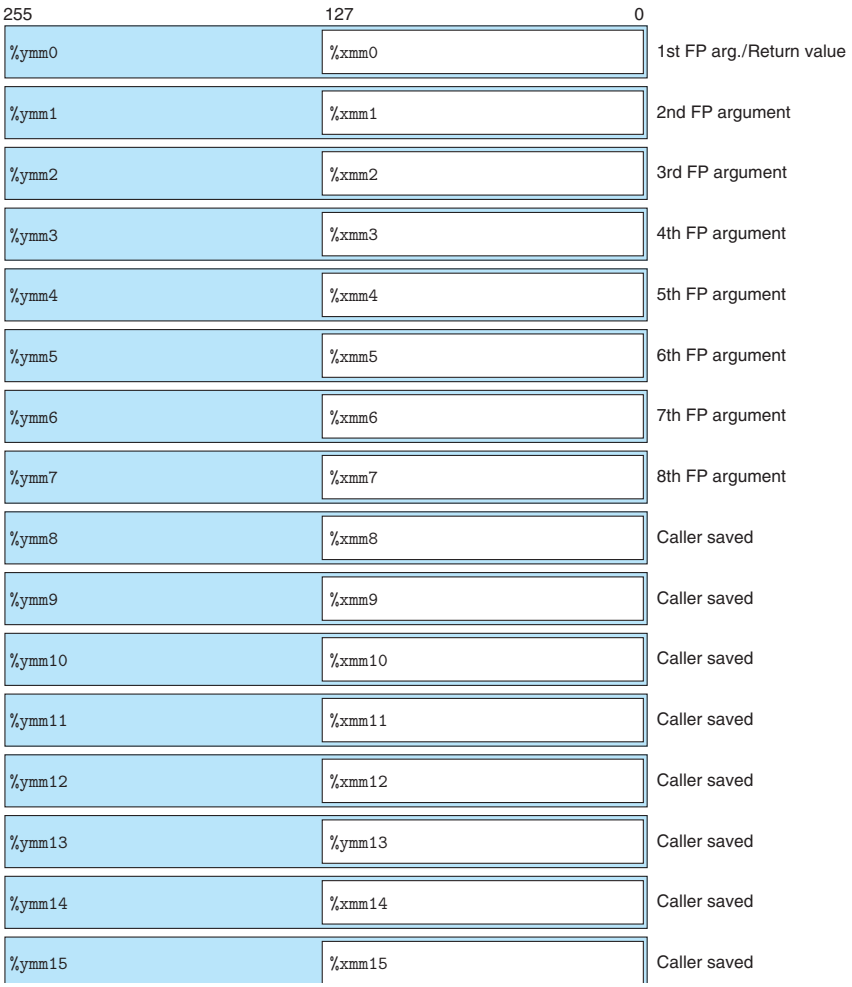


图 3.45 （示意图）

3.14.1 浮点传送和转换操作

图 3.46 给出了一组指令，用于在存储器和 XMM 寄存器之间传送浮点数据，以及从一个 XMM 寄存器到另一个 XMM 寄存器进行不带任何转换的传送。其中那些访问存储器的指令是标量指令，意思是它们对单个数据值而不是打包的数据值进行操作。数据要么保存在存储器中（在表中用 M32 和 M64 表示），要么保存在 XMM 寄存器中（在表中用 X 表示）。无论数据的对齐方式如何，这些指令都能正确工作，不过代码优化准则建议 32 位的存储器数据满足 4 字节对齐、64 位的数据满足 8 字节对齐。存储器引用的指定方式与整数 `mov` 指令相同，可以使用偏移量、基址寄存器、变址寄存器和缩放因子的所有可能的组合。

gcc 只用标量传送操作在存储器和 XMM 寄存器之间传送数据。要在两个 XMM 寄存器之间传送数据，它使用两条不同指令中的一条来把一个 XMM 寄存器的完整内容复制到另一个——对单精度值用 `vmovaps`，对双精度值用 `vmovapd`。对这两种情形而言，程序复制整个寄存器还是仅复制其低位，既不影响程序的功能，也不影响执行速度，因此使用这些指令而不是专用于标量数据的指令并不会有什么实际差别。这些指令名中的字母“a”代表“aligned”（对齐）。当用它们读写存储器时，如果地址不满足 16 字节对齐，就会引起一个异常。在两个寄存器之间传送时，不存在对齐出错的可能。

作为不同的浮点传送操作的一个示例，考虑 C 函数

```
float float_mov(float v1, float *src, float *dst) {
    float v2 = *src;
    *dst = v1;
    return v2;
}
```

图 3.47 双操作数浮点转换指令。这些指令把浮点数据转换成整数。（X：XMM 寄存器（如%xmm3）；R32：32 位通用寄存器（如%eax）；R64：64 位通用寄存器（如%rax）；M32：32 位内存范围；M64：64 位内存范围）

指令	源	目的	描述
vcvttss2si	X/M32	R32	用截断把单精度数转换成整数
vcvttsd2si	X/M64	R32	用截断把双精度数转换成整数
vcvttss2siq	X/M32	R64	用截断把单精度数转换成四字整数
vcvttsd2siq	X/M64	R64	用截断把双精度数转换成四字整数

图 3.48 三操作数浮点转换指令。这些指令把第一个源操作数的数据类型转换成目的操作数的数据类型。第二个源操作数的值不影响结果的低位字节。（X：XMM 寄存器（如%xmm3）；M32：32 位内存范围；M64：64 位内存范围）

指令	源 1	源 2	目的	描述
vcvtsi2ss	M32/R32	X	X	把整数转换成单精度数
vcvtsi2sd	M32/R32	X	X	把整数转换成双精度数
vcvtsi2ssq	M64/R64	X	X	把四字整数转换成单精度数
vcvtsi2sdq	M64/R64	X	X	把四字整数转换成双精度数

以及与之对应的 x86-64 汇编代码

```
float float_mov(float v1, float *src, float *dst)
v1 在 %xmm0 中,src 在 %rdi 中,dst 在 %rsi 中
 1  float_mov:
 2    vmovaps %xmm0, %xmm1          把 v1 复制一份
 3    vmovss  (%rdi), %xmm0         从 src 读出 v2
 4    vmovss  %xmm1, (%rsi)         把 v1 写到 dst
 5    ret                           在 %xmm0 中返回 v2
```

在这个例子中，我们看到了用 vmovaps 指令把数据从一个寄存器复制到另一个寄存器，以及用 vmovss 指令把数据从存储器复制到 XMM 寄存器、再从 XMM 寄存器复制到存储器的用法。

图 3.47 和图 3.48 给出了在浮点数与整数数据类型之间、以及在不同浮点格式之间进行转换的指令集。这些都是对单个数据值进行操作的标量指令。图 3.47 中的指令从 XMM 寄存器或存储器读出浮点值，并把结果写进通用寄存器（如%rax、%ebx 等）。把浮点值转换成整数时，它们执行截断，向零舍入，这正是 C 和大多数其他程序设计语言所要求的。

图 3.48 中的指令把整数转换成浮点数。它们使用一种不寻常的三操作数格式，即两个源操作数和一个目的操作数。第一个操作数从存储器或通用寄存器读出。就我们的目的而言，可以忽略第二个操作数，因为它的值只影响结果的高位字节。目的操作数必须是 XMM 寄存器。在常见的用法中，第二个源操作数和目的操作数是同一个，比如指令

```
vcvtsi2sdq %rax, %xmm1, %xmm1
```

这条指令从寄存器%rax 读出一个 long 类型整数，把它转换成 double 类型，并把结果存到 XMM 寄存器%xmm1 的低位字节中。

最后，在两种不同的浮点格式之间转换时，目前版本的 gcc 生成的代码需要单独加以说明。假设%xmm0 的低 4 个字节存放着一个单精度值；那么看起来最直接的做法就是用这样的指令

```
vcvtss2sd %xmm0, %xmm0, %xmm0
```

把它转换成一个双精度值，并把结果存到寄存器%xmm0 的低 8 个字节中。而 gcc 生成的却是下面的代码：

```
从单精度到双精度的转换
 1  vunpcklps  %xmm0, %xmm0, %xmm0      复制第一个向量元素
 2  vcvtps2pd  %xmm0, %xmm0             把两个向量元素转换成 double
```

vunpcklps 指令的常规用途是把两个 XMM 寄存器中的值交错组合，存到第三个寄存器中。也就是说，如果一个源寄存器包含字 $[s_3, s_2, s_1, s_0]$，另一个包含字 $[d_3, d_2, d_1, d_0]$，那么目的寄存器中的值将是 $[s_1, d_1, s_0, d_0]$。在上面的代码中，三个操作数用的是同一个寄存器，因此如果原来的寄存器保存着值 $[x_3, x_2, x_1, x_0]$，这条指令就会把它更新为 $[x_1, x_1, x_0, x_0]$。vcvtps2pd 指令把源 XMM 寄存器中的两个低位单精度值扩展成目的 XMM 寄存器中的两个双精度值。把它应用于前面 vunpcklps 指令的结果，得到的值将是 $[dx_0, dx_0]$，其中 dx_0 是把 x_0 转换成双精度的结果。也就是说，这两条指令的净效果，是把%xmm0 低 4

个字节中的原始单精度值转换成双精度，并在%xmm0 中存放两份。gcc 为什么会生成这样的代码并不清楚。让这个值在 XMM 寄存器中重复存放，既没有好处，也没有必要。

在从双精度转换到单精度时，gcc 也会生成类似的代码：

```
从双精度到单精度的转换
 1  vmovddup   %xmm0, %xmm0            复制第一个向量元素
 2  vcvtpd2psx %xmm0, %xmm0            把两个向量元素转换成单精度数
```

假设这两条指令开始执行时寄存器%xmm0 中保存着两个双精度值 $[x_1, x_0]$。vmovddup 指令会把它设置成 $[x_0, x_0]$。然后 vcvtpd2psx 指令把这些值转换成单精度，装进寄存器的低半部分，并把高半部分设置为 0，得到结果 $[0.0, 0.0, x_0, x_0]$（回忆一下，浮点值 0.0 的位模式是全零）。同样，与其用这种方式、不如直接用下面这条指令来完成精度之间的转换，前者看不出有什么好处：

```
vcvtsd2ss %xmm0, %xmm0, %xmm0
```

作为这几种不同浮点转换操作的一个例子，考虑下面的 C 函数

```
double fcvt(int i, float *fp, double *dp, long *lp)
{
    float f = *fp; double d = *dp; long l = *lp;
    *lp = (long)    d;
    *fp = (float)   i;
    *dp = (double) l;
    return (double) f;
}
```

以及与之对应的 x86-64 汇编代码

```
double fcvt(int i, float *fp, double *dp, long *lp)
i 在 %edi 中,fp 在 %rsi 中,dp 在 %rdx 中,lp 在 %rcx 中
 1  fcvt:
 2    vmovss (%rsi), %xmm0              读取 f = *fp
 3    movq    (%rcx), %rax              读取 l = *lp
 4    vcvttsd2siq (%rdx), %r8           读取 d = *dp 并转换成 long
 5    movq    %r8, (%rcx)               存到 lp
 6    vcvtsi2ss  %edi, %xmm1, %xmm1     把 i 转换成 float
 7    vmovss  %xmm1, (%rsi)             存到 fp
 8    vcvtsi2sdq  %rax, %xmm1, %xmm1    把 l 转换成 double
 9    vmovsd  %xmm1, (%rdx)             存到 dp
            下面两条指令把 f 转换成 double
10    vunpcklps  %xmm0, %xmm0, %xmm0
11    vcvtps2pd  %xmm0, %xmm0
12    ret                               返回 f
```

fcvt 的所有参数都通过通用寄存器来传递，因为它们不是整数就是指针。结果在寄存器%xmm0 中返回。如图 3.45 所示，这是专门为 float 或 double 值指

定的返回寄存器。在这段代码中，我们看到图 3.46–3.48 中许多传送和转换指令的用法，也看到了 gcc 偏好的从单精度转换到双精度的方法。

练习 3.50 对于下面的 C 代码，表达式 `val1`–`val4` 都对应到程序值 `i`、`f`、`d` 和 `l`：

```
double fcvt2(int *ip, float *fp, double *dp, long l)
{
    int i = *ip; float f = *fp; double d = *dp;
    *ip = (int)     val1;
    *fp = (float)   val2;
    *dp = (double)  val3;
    return (double) val4;
}
```

请根据下面该函数的 x86-64 汇编代码，确定这种对应关系：

```
double fcvt2(int *ip, float *fp, double *dp, long l)
ip 在 %rdi 中,fp 在 %rsi 中,dp 在 %rdx 中,l 在 %rcx 中
结果在 %xmm0 中返回
 1  fcvt2:
 2    movl    (%rdi), %eax
 3    vmovss  (%rsi), %xmm0
 4    vcvttsd2si  (%rdx), %r8d
 5    movl    %r8d, (%rdi)
 6    vcvtsi2ss   %eax, %xmm1, %xmm1
 7    vmovss  %xmm1, (%rsi)
 8    vcvtsi2sdq  %rcx, %xmm1, %xmm1
 9    vmovsd  %xmm1, (%rdx)
10    vunpcklps   %xmm0, %xmm0, %xmm0
11    vcvtps2pd   %xmm0, %xmm0
12    ret
```

练习 3.51 下面的 C 函数把一个 `src_t` 类型的参数转换成 `dst_t` 类型的返回值，这两个类型都是用 typedef 定义的：

```
dest_t cvt(src_t x)
{
    dest_t y = (dest_t) x;
    return y;
}
```

在 x86-64 上执行时，假设参数 x 或者在`%xmm0` 中，或者在寄存器`%rdi` 相应命名的那部分中（即`%rdi` 或`%edi`）。要求用一条或两条指令完成类型转换，并把值复制到寄存器`%rax`（整数结果）或`%xmm0`（浮点结果）相应命名的那部分中。请写出这些指令，包括源寄存器和目的寄存器：

3.14.2 过程中的浮点代码

在 x86-64 上，XMM 寄存器用来把浮点参数传给函数，以及从函数返回浮点值。如图 3.45 所示，这里要遵守以下几条约定：

T_x	T_y	指令
long	double	vcvtsi2sdq %rdi, %xmm0
double	int	
double	float	
long	float	
float	long	

- 最多八个浮点参数可以通过 XMM 寄存器%xmm0–%xmm7 传递。这些寄存器按参数列出的顺序使用。更多的浮点参数可以通过栈来传递。
- 返回浮点值的函数在寄存器%xmm0 中返回该值。
- 所有 XMM 寄存器都是调用者保存的。被调用者可以不经保存就覆盖这些寄存器中的任何一个。

当一个函数同时含有指针、整数和浮点参数时，指针和整数通过通用寄存器传递，而浮点值通过 XMM 寄存器传递。这意味着参数到寄存器的映射既取决于参数的类型，也取决于它们的排列顺序。下面举几个例子：

```
double f1(int x, double y, long z);
```

这个函数会把 x 放在%edi 中，y 放在%xmm0 中，z 放在%rsi 中。

```
double f2(double y, int x, long z);
```

这个函数的寄存器分配情况与函数 f1 相同。

```
double f1(float x, double *y, long *z);
```

这个函数会把 x 放在%xmm0 中，y 放在%rdi 中，z 放在%rsi 中。

对下面给出的每个函数声明，确定存放各个参数的寄存器：

A. `double g1(double a, long b, float c, int d);`

B. `double g2(int a, double *b, float *c, long d);`

C. `double g3(double *a, double b, int c, float d);`

D. `double g4(float a, int *b, float c, double d);`

3.14.3 浮点算术运算

图 3.49 记录了一组执行算术运算的标量 AVX2 浮点指令。每条指令都带有一个（S_1）或两个（S_1、S_2）源操作数和一个目的操作数 D。第一个源操作数 S_1 可以是 XMM 寄存器，也可以是存储器位置。第二个源操作数和目的操作数必须是 XMM 寄存器。每种运算都有一条单精度指令和一条双精度指令。结果存放在目的寄存器中。

作为一个例子，考虑下面的浮点函数：

```
double funct(double a, float x, double b, int i)
{
    return a*x - b/i;
}
```

该函数的 x86-64 汇编代码如下:

```
double funct(double a, float x, double b, int i)
a 在 %xmm0 中,x 在 %xmm1 中,b 在 %xmm2 中,i 在 %edi 中
 1  funct:
        下面两条指令把 x 转换成 double
 2    vunpcklps  %xmm1, %xmm1, %xmm1
 3    vcvtps2pd  %xmm1, %xmm1
 4    vmulsd  %xmm0, %xmm1, %xmm0          计算 a 乘以 x
 5    vcvtsi2sd   %edi, %xmm1, %xmm1       把 i 转换成 double
 6    vdivsd  %xmm1, %xmm2, %xmm2          计算 b/i
 7    vsubsd  %xmm2, %xmm0, %xmm0          从 a*x 中减去
 8    ret                                  返回
```

图 3.49 标量浮点算术运算。这些指令有一个或两个源操作数和一个目的操作数。

单精度	双精度	效果	描述
vaddss	vaddsd	$D \leftarrow S_2 + S_1$	浮点加
vsubss	vsubsd	$D \leftarrow S_2 - S_1$	浮点减
vmulss	vmulsd	$D \leftarrow S_2 \times S_1$	浮点乘
vdivss	vdivsd	$D \leftarrow S_2 / S_1$	浮点除
vmaxss	vmaxsd	$D \leftarrow \max(S_2, S_1)$	浮点最大值
vminss	vminsd	$D \leftarrow \min(S_2, S_1)$	浮点最小值
sqrtss	sqrtsd	$D \leftarrow \sqrt{S_1}$	浮点平方根

三个浮点参数 a、x 和 b 通过 XMM 寄存器`%xmm0`–`%xmm2` 传递,而整数参数 i 通过寄存器`%edi` 传递。把参数 x 转换成 double 采用的是标准的两条指令序列(第 2–3 行)。把参数 i 转换成 double 还需要另一条转换指令(第 5 行)。函数值在寄存器`%xmm0` 中返回。

练习 3.53 对于下面的 C 函数,四个参数的类型由 typedef 定义:

```
double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
{
    return p/(q+r) - s;
}
```

gcc 在编译这个函数时生成了下面的代码:

```
double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
 1  funct1:
 2    vcvtsi2ssq  %rsi, %xmm2, %xmm2
 3    vaddss  %xmm0, %xmm2, %xmm0
```

```
 4    vcvtsi2ss   %edi, %xmm2, %xmm2
 5    vdivss  %xmm0, %xmm2, %xmm0
 6    vunpcklps   %xmm0, %xmm0, %xmm0
 7    vcvtps2pd   %xmm0, %xmm0
 8    vsubsd  %xmm1, %xmm0, %xmm0
 9    ret
```

请判定这四个参数类型的可能组合(可能不止一种)。

练习 3.54 函数 funct2 具有如下原型:

```
double funct2(double w, int x, float y, long z);
```

gcc 为该函数生成了下面的代码:

```
double funct2(double w, int x, float y, long z)
w 在 %xmm0 中,x 在 %edi 中,y 在 %xmm1 中,z 在 %rsi 中
 1  funct2:
 2    vcvtsi2ss   %edi, %xmm2, %xmm2
 3    vmulss  %xmm1, %xmm2, %xmm1
 4    vunpcklps   %xmm1, %xmm1, %xmm1
 5    vcvtps2pd   %xmm1, %xmm2
 6    vcvtsi2sdq  %rsi, %xmm1, %xmm1
 7    vdivsd  %xmm1, %xmm0, %xmm0
 8    vsubsd  %xmm0, %xmm2, %xmm0
 9    ret
```

请写出 funct2 的一个 C 版本。

3.14.4 浮点常量的定义和使用

与整数算术运算不同,AVX 浮点运算不能用立即数作操作数。相反,编译器必须为所有的常数值分配并初始化存储,之后代码再从存储器读取这些值。下面的把摄氏温度转换成华氏温度的函数说明了这一点:

```
double cel2fahr(double temp)
{
    return 1.8 * temp + 32.0;
}
```

这个函数的 x86-64 汇编代码的相关部分如下:

```
double cel2fahr(double temp)
temp 在 %xmm0 中
 1  cel2fahr:
 2    vmulsd  .LC2(%rip), %xmm0, %xmm0       乘以 1.8
 3    vaddsd  .LC3(%rip), %xmm0, %xmm0       加上 32.0
 4    ret
 5  .LC2:
 6    .long   3435973837                     1.8 的低 4 个字节
 7    .long   1073532108                     1.8 的高 4 个字节
 8  .LC3:
 9    .long   0                              32.0 的低 4 个字节
10    .long   1077936128                     32.0 的高 4 个字节
```

我们看到,函数从标号.LC2 所指的存储器位置读出值 1.8,从标号.LC3 所指的位置读出值 32.0。查看与这些标号相关联的值,可以看到每个值都是用一对.long 声明指定的,数值以十进制给出。这些值应当如何解释成浮点数呢?看标号为.LC2 的声明,两个值是 3435973837(0xcccccccd)和 1073532108(0x3ffccccc)。因为机器采用小端法字节顺序,第一个值给出低 4 个字节,第二个值给出高 4 个字节。从高位字节中可以提取出 0x3ff(1023)的阶码域,减去偏置 1023,得到阶码 0。把两个值的小数位拼接起来,得到小数域 0xcccccccccccccd,可以证明,这正是 0.8 的二进制小数表示,再加上隐含的前导 1,就得到 1.8。

图 3.50 对打包数据的位级运算。这些指令对 XMM 寄存器中的全部 128 位执行布尔运算。

单精度	双精度	效果	描述
vxorps	xorpd	$D \leftarrow S_2 \hat{} S_1$	位级异或
vandps	andpd	$D \leftarrow S_2 \,\&\, S_1$	位级与

练习 3.55 请说明标号.LC3 处声明的那些数是如何把数 32.0 编码出来的。

3.14.5 在浮点代码中使用位级运算

有时我们会发现 gcc 生成对 XMM 寄存器执行位级运算的代码,用它们来产生有用的浮点结果。图 3.50 给出了一些相关指令,它们与操作通用寄存器的那些对应指令类似。这些运算都作用于打包数据,意思是它们会更新整个目的 XMM 寄存器,把位级运算应用到两个源寄存器中的所有数据上。再说一次,对标量数据而言,我们唯一关心的是这些指令对目的寄存器低 4 或 8 个字节产生的作用。这些运算往往是操纵浮点值既简单又便利的方法,下面的问题会对此进行探索。

练习 3.56 考虑下面的 C 函数,其中 EXPR 是用 #define 定义的宏:

```
double simplefun(double x) {
    return EXPR(x);
}
```

下面我们给出针对 EXPR 不同定义所生成的 AVX2 代码,其中值 x 保存在%xmm0 中。它们都对应于对浮点值所做的某种有用的运算。请指出这些运算分别是什么。要给出答案,你需要理解从存储器取出的那些常数字的位模式。

```
A.   1  vmovsd  .LC1(%rip), %xmm1
     2  vandpd  %xmm1, %xmm0, %xmm0
```

```
    3  .LC1:
    4  .long    4294967295
    5  .long    2147483647
    6  .long    0
    7  .long    0
```

```
B.  1  vxorpd  %xmm0, %xmm0, %xmm0
```

```
C.  1  vmovsd  .LC2(%rip), %xmm1
    2  vxorpd  %xmm1, %xmm0, %xmm0
    3  .LC2:
    4  .long    0
    5  .long    -2147483648
    6  .long    0
    7  .long    0
```

3.14.6 浮点比较操作

AVX2 提供了两条用于比较浮点值的指令：

指令	基于	描述
`ucomiss` S_1, S_2	$S_2 - S_1$	比较单精度数
`ucomisd` S_1, S_2	$S_2 - S_1$	比较双精度数

这些指令与 `cmp` 指令（见 3.6 节）类似：它们比较操作数 S_1 和 S_2（但顺序与预期的相反），并设置条件码来指明这两个值的相对大小。与 `cmpq` 一样，它们遵循 ATT 格式把操作数按相反顺序列出的约定。参数 S_2 必须放在 XMM 寄存器中，而 S_1 可以放在 XMM 寄存器中，也可以放在存储器中。

浮点比较指令设置三个条件码：零标志 ZF、进位标志 CF 和奇偶标志 PF。3.6.1 节没有介绍奇偶标志，因为 gcc 生成的 x86 代码中很少用到它。对整数运算来说，当最近一次算术或逻辑运算产生的值的最低有效字节具有偶校验（即字节中 1 的个数为偶数）时，该标志置位。而对浮点比较来说，只要任一操作数是 NaN，该标志就会置位。按照惯例，当参数中有一个是 NaN 时，C 中的任何比较都被认为失败，这个标志就是用来检测这种情况的。例如，甚至 `x == x` 这样的比较，在 x 为 NaN 时也会产生 0。

条件码按如下方式设置：

无序的情况发生在任一操作数为 NaN 时，这种情况可以用奇偶标志检测出来。通常用 `jp`（“jump on parity”，按奇偶性跳转）指令在浮点比较产生无序结果时进行条件跳转。除此之外，进位标志和零标志的值与无符号数的比较相同：两个操作数相等时 ZF 置位，而 $S_2 < S_1$ 时 CF 置位。像 `ja` 和 `jb` 这样的指令，用于根据这些标志的不同组合进行条件跳转。

序关系 $S_2 : S_1$	CF	ZF	PF
无序	1	1	1
$S_2 < S_1$	1	0	0
$S_2 = S_1$	0	1	0
$S_2 > S_1$	0	0	0

```
(a) C 代码
typedef enum {NEG, ZERO, POS, OTHER} range_t;

range_t find_range(float x)
{
    int result;
    if (x < 0)
         result = NEG;
    else if (x == 0)
         result = ZERO;
    else if (x > 0)
         result = POS;
    else
         result = OTHER;
    return result;
}
```

```
(b) 生成的汇编代码
range_t find_range(float x)
x 在 %xmm0 中
 1  find_range:
 2    vxorps   %xmm1, %xmm1, %xmm1          把 %xmm1 设为 0
 3    vucomiss %xmm0, %xmm1                 比较 0:x
 4    ja       .L5                          如果 >,转到 neg
 5    vucomiss %xmm1, %xmm0                 比较 x:0
 6    jp       .L8                          如果为 NaN,转到 posornan
 7    movl     $1, %eax                     result = ZERO
 8    je       .L3                          如果 =,转到 done
 9  .L8:                                    posornan:
10    vucomiss .LC0(%rip), %xmm0            比较 x:0
11    setbe    %al                          设置 result = NaN ? 1 : 0
12    movzbl   %al, %eax                    零扩展
13    addl     $2, %eax                     result += 2(> 0 时为 POS,NaN
        时为 OTHER)
14    ret                                   返回
15  .L5:                                    neg:
16    movl     $0, %eax                     result = NEG
17  .L3:                                    done:
18    rep; ret                              返回
```

图 3.51　浮点代码中的条件分支示例。

作为浮点比较的一个例子, 图 3.51(a) 中的 C 函数按参数 x 与 0.0 的大小关系对它进行分类,返回一个枚举类型作为结果。C 中的枚举类型编码成整数,

因此可能的函数值是:0(NEG)、1(ZERO)、2(POS)和 3(OTHER)。最后这个结果只在 x 的值为 NaN 时出现。

gcc 为 `find_range` 生成了图 3.51(b) 所示的代码。这段代码效率不高——它把 x 与 0.0 比较了三次,虽然一次比较就能得到所需的信息。它还生成了两次浮点常数 0.0——一次是用 `vxorps` 生成的,一次是从存储器读出的。下面对四种可能的比较结果逐一跟踪函数的执行流程:

- $x < 0.0$:第 4 行的 `ja` 分支会被选中,直接跳到末尾,返回值为 0。
- $x = 0.0$:`ja` 分支(第 4 行)和 `jp` 分支(第 6 行)都不成立,但 `je` 分支(第 8 行)会成立,返回时`%eax` 为 1。
- $x > 0.0$:三个分支都不成立。第 11 行的 `setbe` 将得到 0,随后第 13 行的 `addl` 指令再加上 2,返回值为 2。
- $x =$ NaN:`jp` 分支(第 6 行)会被选中。第三条 `vucomiss` 指令(第 10 行)会同时把进位标志和零标志置位,于是第 11 行的 `setbe` 指令及其后面那条指令会把`%eax` 设置为 1,再由第 13 行的 `addl` 指令加上 2,返回值为 3。

家庭作业 3.73 和 3.74 要求你挑战手工生成 `find_range` 的更高效实现。

练习 3.57 函数 funct3 具有如下原型:

```
double funct3(int *ap, double b, long c, float *dp);
```

对这个函数,gcc 生成了下面的代码:

```
double funct3(int *ap, double b, long c, float *dp)
ap 在 %rdi 中,b 在 %xmm0 中,c 在 %rsi 中,dp 在 %rdx 中
 1  funct3:
 2    vmovss  (%rdx), %xmm1
 3    vcvtsi2sd   (%rdi), %xmm2, %xmm2
 4    vucomisd  %xmm2, %xmm0
 5    jbe     .L8
 6    vcvtsi2ssq  %rsi, %xmm0, %xmm0
 7    vmulss  %xmm1, %xmm0, %xmm1
 8    vunpcklps   %xmm1, %xmm1, %xmm1
 9    vcvtps2pd   %xmm1, %xmm0
10    ret
11  .L8:
12    vaddss  %xmm1, %xmm1, %xmm1
13    vcvtsi2ssq  %rsi, %xmm0, %xmm0
14    vaddss  %xmm1, %xmm0, %xmm0
15    vunpcklps   %xmm0, %xmm0, %xmm0
16    vcvtps2pd   %xmm0, %xmm0
17    ret
```

请写出 funct3 的一个 C 版本。

3.14.7 关于浮点代码的观察

我们看到，用 AVX2 操作浮点数据所生成的机器代码，其总体风格与前面看到的整数数据操作类似。两者都使用一组寄存器来保存和操作值，并用这些寄存器传递函数参数。

当然，处理不同的数据类型以及求值含有多种数据类型的表达式时的各种规则，带来了许多复杂性。而且 AVX2 代码涉及的指令和格式种类，也远比通常只见于纯整数算术函数的多得多。

AVX2 还具有通过对打包数据执行并行操作来加速计算的潜力。编译器开发人员正在努力实现从标量代码到并行代码的自动转换，不过就目前而言，想通过并行性获得更高的性能，最可靠的方法还是使用 gcc 支持的 C 语言扩展来操作向量数据。具体做法可以参见网络边栏 `opt:simd`。

3.15 小结

在这一章里，我们透过 C 语言提供的抽象层，窥见了机器级程序设计的内幕。让编译器生成机器级程序的汇编代码表示，我们对编译器及其优化能力，以及机器、它的数据类型和指令集，都获得了深入的认识。在第 5 章中我们将会看到，了解编译器的特性有助于编写在机器上有高效映射的程序。我们还对程序把数据存储在不同存储器区域的方式有了更完整的认识。在第 12 章中，我们会看到许多这样的例子：应用程序员需要知道一个程序变量是在运行时栈上、在动态分配的数据结构中，还是全局程序数据的一部分。理解程序到机器的映射，有助于理解这几类存储之间的差别。

机器级程序及其汇编代码表示与 C 程序有许多不同之处。不同数据类型之间的区别很少。程序被表示成一个指令序列，每条指令完成一个单独的操作。程序状态的一部分，比如寄存器和运行时栈，对程序员是直接可见的。系统只提供低级操作来支持数据操纵和程序控制。编译器必须用多条指令来生成和操作不同的数据结构，并实现条件、循环和过程这样的控制结构。我们讨论了 C 的许多方面以及它是如何被编译的。我们看到，C 缺少边界检查，这使得许多程序容易遭受缓冲区溢出攻击。许多系统曾因此容易受到恶意入侵者的攻击，不过运行时系统和编译器最近提供的防护措施有助于提高程序的安全性。

我们只考察了 C 到 x86-64 的映射，但这里讲到的许多内容，对其他的语言与机器组合也以类似的方式处理。例如，编译 C++ 与编译 C 非常相似。实际上，C++ 的早期实现首先做的是从 C++ 到 C 的源到源转换，然后对转换结果运行一个 C 编译器来生成目标代码。C++ 的对象用结构来表示，类似于 C 的 struct；方法则用指向实现这些方法的代码的指针表示。相比之下，Java 的实现方式完

全不同。Java 的目标代码是一种称为 Java 字节码（Java byte code）的特殊二进制表示。这种代码可以看作一台虚拟机的机器级程序。顾名思义，这台机器不是直接用硬件实现的。相反，是由软件解释器来处理字节码，模拟虚拟机的行为。另一种称为即时编译（just-in-time compilation）的方法动态地把字节码序列翻译成机器指令。当代码（比如循环中的代码）被多次执行时，这种方式能带来更快的执行速度。用字节码作为程序低级表示的优点是，同一段代码可以在许多不同的机器上“执行”，而我们考察的这种机器码只能在 x86-64 机器上运行。

3.16 参考文献说明

Intel 和 AMD 都提供了关于其处理器的详尽文档。这些文档既包括从汇编语言程序员视角对硬件的总体描述 [2, 50]，也包括关于各条指令的详细参考 [3, 51]。阅读这些指令描述相当费劲，原因有三：（1）所有文档都基于 Intel 汇编代码格式；（2）由于寻址方式和执行模式的不同，每条指令都有许多变体；（3）文档中没有说明性的例子。尽管如此，它们仍然是关于每条指令行为的最权威的参考资料。

组织 x86-64.org 负责为运行在 Linux 系统上的 x86-64 代码定义应用二进制接口（ABI）[77]。这个接口描述了过程链接、二进制代码文件以及机器码程序正确执行所要求的许多其他特性的细节。

正如我们已经讨论的，gcc 使用的 ATT 格式与 Intel 文档和其他编译器（包括 Microsoft 编译器）使用的 Intel 格式差别非常大。

Muchnick 关于编译器设计的著作 [80] 被认为是关于代码优化技术最全面的参考资料。它涵盖了这里讨论的许多技术，比如寄存器使用惯例。

关于利用缓冲区溢出攻击因特网上的系统，已有大量论述。对 1988 年因特网蠕虫，Spafford [105] 以及帮助遏制其蔓延的 MIT 团队的成员们 [35] 都发表过详细分析。此后，一批论文和项目同时给出了实施缓冲区溢出攻击和防范这类攻击的办法。Seacord 的著作 [97] 提供了大量关于缓冲区溢出以及针对 C 编译器生成的代码的其他攻击的信息。

3.17 家庭作业

家庭作业 3.58 ▲ 对于一个具有如下原型的函数：

```
long decode2(long x, long y, long z);
```

gcc 生成了下面的汇编代码：

```
1  decode2:
```

```
2    subq    %rdx, %rsi
3    imulq   %rsi, %rdi
4    movq    %rsi, %rax
5    salq    $63, %rax
6    sarq    $63, %rax
7    xorq    %rdi, %rax
8    ret
```

参数 x、y 和 z 通过寄存器`%rdi`、`%rsi` 和`%rdx` 传递。代码把返回值保存在寄存器`%rax` 中。

请为 decode2 编写 C 代码，使它的效果与上面的汇编代码等价。

家庭作业 3.59 ▲▲ 下面的代码计算两个 64 位有符号值 x 和 y 的 128 位乘积，并把结果存到存储器中：

```
1  typedef __int128 int128_t;
2
3  void store_prod(int128_t *dest, int64_t x, int64_t y) {
4      *dest = x * (int128_t) y;
5  }
```

gcc 生成实现这个计算的汇编代码如下：

```
1  store_prod:
2    movq    %rdx, %rax
3    cqto
4    movq    %rsi, %rcx
5    sarq    $63, %rcx
6    imulq   %rax, %rcx
7    imulq   %rsi, %rdx
8    addq    %rdx, %rcx
9    mulq    %rsi
10   addq    %rcx, %rdx
11   movq    %rax, (%rdi)
12   movq    %rdx, 8(%rdi)
13   ret
```

这段代码使用三次乘法，完成在一台 64 位机器上实现 128 位算术所需的多精度算术。请描述计算乘积所用的算法，并给汇编代码加上注解，说明它如何实现你的算法。提示：把 x 和 y 的参数扩展成 128 位时，可以把它们改写成 $x = 2^{64} \cdot x_h + x_l$ 和 $y = 2^{64} \cdot y_h + y_l$，其中 x_h、x_l、y_h 和 y_l 是 64 位的值。类似地，128 位乘积可以写成 $p = 2^{64} \cdot p_h + p_l$，其中 p_h 和 p_l 是 64 位的值。请说明代码如何用 x_h、x_l、y_h 和 y_l 计算出 p_h 和 p_l 的值。

家庭作业 3.60 ▲▲ 考虑下面的汇编代码：

```
long loop(long x, int n)
x 在 %rdi 中,n 在 %esi 中
 1  loop:
```

```
 2    movl    %esi, %ecx
 3    movl    $1, %edx
 4    movl    $0, %eax
 5    jmp     .L2
 6  .L3:
 7    movq    %rdi, %r8
 8    andq    %rdx, %r8
 9    orq     %r8, %rax
10    salq    %cl, %rdx
11  .L2:
12    testq   %rdx, %rdx
13    jne     .L3
14    rep; ret
```

上面的代码是编译具有如下整体形式的 C 代码得到的:

```
1  long loop(long x, long n)
2  {
3      long result = ________;
4      long mask;
5      for (mask = ________; mask ________________; mask = ________) {
6          result |= ________;
7      }
8      return result;
9  }
```

你的任务是填上 C 代码中缺失的部分,得到一个与生成的汇编代码等价的程序。回忆一下,函数的结果是在寄存器`%rax` 中返回的。你会发现,在循环之前、循环执行过程中和循环之后分别考察汇编代码,建立起寄存器和程序变量之间一致的对应关系,会很有帮助。

A. 哪些寄存器保存着程序值 x、n、result 和 mask?

B. result 和 mask 的初始值是什么?

C. mask 的测试条件是什么?

D. mask 如何得到更新?

E. result 如何得到更新?

F. 填上 C 代码中所有缺失的部分。

家庭作业 3.61 ▲▲ 在 3.6.6 节中,我们把下面这段代码作为使用条件数据传送的候选情况考察过:

```
long cread(long *xp) {
    return (xp ? *xp : 0);
}
```

我们给出了一个用条件传送指令实现的试验性版本,但论证了它是不可行的,因为它可能会试图从空地址读出。

请编写一个 C 函数 `cread_alt`，它的行为与 cread 相同，但可以被编译成使用条件数据传送。编译时，生成的代码应当使用一条条件传送指令，而不是某条跳转指令。

家庭作业 3.62 ▲▲ 下面的代码展示了在 switch 语句中根据一个枚举类型值进行分支的例子。回忆一下，C 中的枚举类型只是引入一组具有关联整数值的名字的一种方式。默认情况下，赋给这些名字的值从零开始向上计数。在我们的代码中，与不同 case 标号相关联的动作都被省略了。

```
1  /* 枚举类型建立了一组从 0 开始向上编号的常量 */
2  typedef enum {MODE_A, MODE_B, MODE_C, MODE_D, MODE_E} mode_t;
3
4  long switch3(long *p1, long *p2, mode_t action)
5  {
6      long result = 0;
7      switch(action) {
8      case MODE_A:
9
10     case MODE_B:
11
12     case MODE_C:
13
14     case MODE_D:
15
16     case MODE_E:
17
18     default:
19
20     }
21     return result;
22 }
```

生成的汇编代码中实现各个动作的那一部分如图 3.52 所示。注解标出了参数存放的位置、寄存器的值，以及各个跳转目标所对应的 case 标签。

把 C 代码中缺失的部分填上。其中有一个 case 会直落到另一个 case——试着把这个结构复原出来。

家庭作业 3.63 ▲▲ 这个问题让你有机会从反汇编得到的机器码中逆向重构一个 switch 语句。在下面的过程中，switch 语句的主体被省略了：

```
1  long switch_prob(long x, long n) {
2      long result = x;
3      switch(n) {
4          /* 在这里填写代码 */
5
6      }
7      return result;
8  }
```

```
p1 in %rdi, p2 in %rsi, action in %edx
1   .L8:                                MODE_E(情形 E)
2     movl    $27, %eax
3     ret
4   .L3:                                MODE_A(情形 A)
5     movq    (%rsi), %rax
6     movq    (%rdi), %rdx
7     movq    %rdx, (%rsi)
8     ret
9   .L5:                                MODE_B(情形 B)
10    movq    (%rdi), %rax
11    addq    (%rsi), %rax
12    movq    %rax, (%rdi)
13    ret
14  .L6:                                MODE_C(情形 C)
15    movq    $59, (%rdi)
16    movq    (%rsi), %rax
17    ret
18  .L7:                                MODE_D(情形 D)
19    movq    (%rsi), %rax
20    movq    %rax, (%rdi)
21    movl    $27, %eax
22    ret
23  .L9:                                default(默认情况)
24    movl    $12, %eax
25    ret
```

图 3.52 问题 3.62 的汇编代码。这一代码实现了 switch 语句的各个不同分支。

图 3.53 给出了这个过程反汇编之后的机器码。

跳转表位于内存中的另一个区域。从第 5 行的间接跳转可以看出,跳转表从地址 `0x4006f8` 开始。使用 gdb 调试器,我们可以用命令 `x/6gx 0x4006f8` 来检查组成跳转表的 6 个 8 字节字。gdb 打印出:

```
(gdb) x/6gx 0x4006f8
0x4006f8:    0x00000000004005a1        0x00000000004005c3
0x400708:    0x00000000004005a1        0x00000000004005aa
0x400718:    0x00000000004005b2        0x00000000004005bf
```

用 C 代码填出 switch 语句的主体,使它具有与机器码相同的行为。

```
long switch_prob(long x, long n)
x in %rdi, n in %rsi
1  0000000000400590 <switch_prob>:
2    400590: 48 83 ee 3c              sub     $0x3c,%rsi
3    400594: 48 83 fe 05              cmp     $0x5,%rsi
4    400598: 77 29                    ja      4005c3 <switch_prob+0x33>
5    40059a: ff 24 f5 f8 06 40 00     jmpq    *0x4006f8(,%rsi,8)
6    4005a1: 48 8d 04 fd 00 00 00 00  lea     0x0(,%rdi,8),%rax
7    4005a8: 00
8    4005a9: c3                       retq
```

```
9    4005aa: 48 89 f8                mov     %rdi,%rax
10   4005ad: 48 c1 f8 03             sar     $0x3,%rax
11   4005b1: c3                      retq
12   4005b2: 48 89 f8                mov     %rdi,%rax
13   4005b5: 48 c1 e0 04             shl     $0x4,%rax
14   4005b9: 48 29 f8                sub     %rdi,%rax
15   4005bc: 48 89 c7                mov     %rax,%rdi
16   4005bf: 48 0f af ff             imul    %rdi,%rdi
17   4005c3: 48 8d 47 4b             lea     0x4b(%rdi),%rax
18   4005c7: c3                      retq
```

图 3.53 问题 3.63 的反汇编代码。

家庭作业 3.64 ▲▲▲ 考虑下面的源代码，其中 R、S 和 T 是用 `#define` 声明定义的常量：

```
1  long A[R][S][T];
2
3  long store_ele(long i, long j, long k, long *dest)
4  {
5      *dest = A[i][j][k];
6      return sizeof(A);
7  }
```

编译这个程序时，gcc 生成如下汇编代码：

```
long store_ele(long i, long j, long k, long *dest)
i in %rdi, j in %rsi, k in %rdx, dest in %rcx
1   store_ele:
2     leaq    (%rsi,%rsi,2), %rax
3     leaq    (%rsi,%rax,4), %rax
4     movq    %rdi, %rsi
5     salq    $6, %rsi
6     addq    %rsi, %rdi
7     addq    %rax, %rdi
8     addq    %rdi, %rdx
9     movq    A(,%rdx,8), %rax
10    movq    %rax, (%rcx)
11    movl    $3640, %eax
12    ret
```

A. 把公式 3.1 从两个维度扩展到三个维度，给出表示数组元素 `A[i][j][k]` 位置的公式。

B. 运用你的逆向工程技巧，根据汇编代码确定 R、S 和 T 的值。

家庭作业 3.65 ▲ 下面的代码对一个 $M \times M$ 数组的元素做转置，其中 M 是用 `#define` 定义的常量：

```
1  void transpose(long A[M][M]) {
2      long i, j;
3      for (i = 0; i < M; i++)
4          for (j = 0; j < i; j++) {
```

```
5              long t = A[i][j];
6              A[i][j] = A[j][i];
7              A[j][i] = t;
8          }
9  }
```

用优化级别 `-O1` 编译时,gcc 为该函数的内层循环生成如下代码:

```
1  .L6:
2    movq    (%rdx), %rcx
3    movq    (%rax), %rsi
4    movq    %rsi, (%rdx)
5    movq    %rcx, (%rax)
6    addq    $8, %rdx
7    addq    $120, %rax
8    cmpq    %rdi, %rax
9    jne     .L6
```

可以看到 gcc 已经把数组下标运算转换成了指针代码。

A. 哪个寄存器保存着指向数组元素 `A[i][j]` 的指针?

B. 哪个寄存器保存着指向数组元素 `A[j][i]` 的指针?

C. M 的值是多少?

家庭作业 3.66 ▲ 考虑下面的源代码,其中 `NR` 和 `NC` 是用 `#define` 声明的宏表达式,它们根据参数 n 来计算数组 `A` 的维度。这段代码计算数组第 j 列的元素之和。

```
1  long sum_col(long n, long A[NR(n)][NC(n)], long j) {
2      long i;
3      long result = 0;
4      for (i = 0; i < NR(n); i++)
5          result += A[i][j];
6      return result;
7  }
```

编译这个程序时,gcc 生成如下汇编代码:

```
long sum_col(long n, long A[NR(n)][NC(n)], long j)
n in %rdi, A in %rsi, j in %rdx
1   sum_col:
2     leaq    1(,%rdi,4), %r8
3     leaq    (%rdi,%rdi,2), %rax
4     movq    %rax, %rdi
5     testq   %rax, %rax
6     jle     .L4
7     salq    $3, %r8
8     leaq    (%rsi,%rdx,8), %rcx
9     movl    $0, %eax
10    movl    $0, %edx
11  .L3:
12    addq    (%rcx), %rax
```

```
13    addq    $1, %rdx
14    addq    %r8, %rcx
15    cmpq    %rdi, %rdx
16    jne     .L3
17    rep; ret
18  .L4:
19    movl    $0, %eax
20    ret
```

运用你的逆向工程技巧,确定 NR 和 NC 的定义。

家庭作业 3.67 ▲▲ 在这个练习中,我们将考察 gcc 为参数和返回值都是结构体的函数生成的代码,并由此看出这些语言特性通常是如何实现的。

下面的 C 代码包含一个以结构体作为参数和返回值的函数 process,以及一个调用 process 的函数 eval:

```
1   typedef struct {
2       long a[2];
3       long *p;
4   } strA;
5
6   typedef struct {
7       long u[2];
8       long q;
9   } strB;
10
11  strB process(strA s) {
12      strB r;
13      r.u[0] = s.a[1];
14      r.u[1] = s.a[0];
15      r.q =     *s.p;
16      return r;
17  }
18
19  long eval(long x, long y, long z) {
20      strA s;
21      s.a[0] = x;
22      s.a[1] = y;
23      s.p = &z;
24      strB r = process(s);
25      return r.u[0] + r.u[1] + r.q;
26  }
```

gcc 为这两个函数生成如下代码:

```
strB process(strA s)
1   process:
2     movq    %rdi, %rax
3     movq    24(%rsp), %rdx
4     movq    (%rdx), %rdx
5     movq    16(%rsp), %rcx
6     movq    %rcx, (%rdi)
```

```
7     movq    8(%rsp), %rcx
8     movq    %rcx, 8(%rdi)
9     movq    %rdx, 16(%rdi)
10    ret
```

```
long eval(long x, long y, long z)
x in %rdi, y in %rsi, z in %rdx
1   eval:
2     subq    $104, %rsp
3     movq    %rdx, 24(%rsp)
4     leaq    24(%rsp), %rax
5     movq    %rdi, (%rsp)
6     movq    %rsi, 8(%rsp)
7     movq    %rax, 16(%rsp)
8     leaq    64(%rsp), %rdi
9     call    process
10    movq    72(%rsp), %rax
11    addq    64(%rsp), %rax
12    addq    80(%rsp), %rax
13    addq    $104, %rsp
14    ret
```

A. 从函数 `eval` 的第 2 行可以看出，它在栈上分配了 104 个字节。画出 `eval` 的栈帧图，标出它在调用 process 之前存储到栈上的各个值。

B. `eval` 在调用 process 时传递的是什么值?

C. process 的代码是如何访问结构体参数 s 的各个元素的?

D. process 的代码是如何设置结果结构体 r 的各个字段的?

E. 完成你的 `eval` 栈帧图，标出 `eval` 在 process 返回之后是如何访问结构体 r 的各个元素的。

F. 关于结构体值如何作为函数参数传递以及如何作为函数结果返回，你能从中看出哪些一般原则?

家庭作业 3.68 ▲▲▲ 在下面的代码中，A 和 B 是用 `#define` 定义的常量:

```
1   typedef struct {
2       int x[A][B];  /* 未知的常量 A 和 B */
3       long y;
4   } str1;
5
6   typedef struct {
7       char array[B];
8       int t;
9       short s[A];
10      long u;
11  } str2;
12
13  void setVal(str1 *p, str2 *q) {
14      long v1 = q->t;
15      long v2 = q->u;
```

```
16      p->y = v1+v2;
17  }
```

gcc 为 setVal 生成如下代码:

```
void setVal(str1 *p, str2 *q)
p in %rdi, q in %rsi
1   setVal:
2     movslq    8(%rsi), %rax
3     addq      32(%rsi), %rax
4     movq      %rax, 184(%rdi)
5     ret
```

A 和 B 的值各是多少?(解是唯一的。)

家庭作业 3.69 ▲▲▲ 你负责维护一个大型的 C 程序，并碰到了下面这段代码:

```
1   typedef struct {
2       int first;
3       a_struct a[CNT];
4       int last;
5   } b_struct;
6
7   void test(long i, b_struct *bp)
8   {
9       int n = bp->first + bp->last;
10      a_struct *ap = &bp->a[i];
11      ap->x[ap->idx] = n;
12  }
```

编译期常量 CNT 和结构体 a_struct 的声明存放在一个你没有相应访问权限的文件里。幸运的是，你有一份这份代码的.o 版本的副本，可以用 objdump 程序对它进行反汇编,得到如下反汇编结果:

```
void test(long i, b_struct *bp)
i in %rdi, bp in %rsi
1   0000000000000000 <test>:
2      0:   8b 8e 20 01 00 00       mov     0x120(%rsi),%ecx
3      6:   03 0e                   add     (%rsi),%ecx
4      8:   48 8d 04 bf             lea     (%rdi,%rdi,4),%rax
5      c:   48 8d 04 c6             lea     (%rsi,%rax,8),%rax
6     10:   48 8b 50 08             mov     0x8(%rax),%rdx
7     14:   48 63 c9                movslq  %ecx,%rcx
8     17:   48 89 4c d0 10          mov     %rcx,0x10(%rax,%rdx,8)
9     1c:   c3                      retq
```

运用你的逆向工程技巧,推断出以下内容:

A. CNT 的值。

B. 结构体 a_struct 的完整声明。假设这个结构体只有 idx 和 x 两个字段,并且这两个字段中包含的都是有符号值。

家庭作业 3.70 ▲▲▲ 考虑下面这个联合声明:

```
union ele {
    struct {
        long *p;
        long y;
    } e1;
    struct {
        long x;
        union ele *next;
    } e2;
};
```

这个声明说明结构体可以嵌套在联合中。

下面的函数(省略了几个表达式)处理一个以这些联合作为表元素的链表:

```
void proc (union ele *up) {
    up->          = *(              ) -             ;
}
```

A. 下面这些字段的偏移量(以字节为单位)分别是多少:

- e1.p
- e1.y
- e2.x
- e2.next

B. 这个结构体总共需要多少个字节?

C. 编译器为 proc 生成如下汇编代码:

```
void proc (union ele *up)
up in %rdi
proc:
    movq    8(%rdi), %rax
    movq    (%rax), %rdx
    movq    (%rdx), %rdx
    subq    8(%rax), %rdx
    movq    %rdx, (%rdi)
    ret
```

根据这些信息,把 proc 代码中缺少的表达式填上。提示:某些联合引用可以有含混不清的解读方式。当你沿着这些引用看下去时,这些歧义就会自行消解。只有一种答案既不进行任何强制类型转换,也不违反任何类型约束。

家庭作业 3.71 ▲ 编写一个函数 good_echo,它从标准输入读一行,并把它写到标准输出。你的实现应当对任意长度的输入行都能正常工作。你可以使用库函数 fgets,但必须保证即使输入一行所需的空间超过你为缓冲区分配的空间,你的函数也能正确工作。你的代码还应当检查错误条件,并在遇到错误时返回。请参考标准 I/O 函数的定义文档 [45, 61]。

家庭作业 3.72 ▲▲ 图 3.54(a) 展示了一个与函数 vframe(图 3.43(a))类似的函数的代码。我们曾用 vframe 来说明在管理可变大小的栈帧时帧指针的用法。新函数 aframe 调用库函数 alloca 来为局部数组 p 分配空间。这个函数与更常用的 malloc 很相似,区别在于它是在运行时栈上分配空间的。当执行的过程返回时,这些空间会被自动释放。

```
(a) C 代码
1  #include <alloca.h>
2
3  long aframe(long n, long idx, long *q) {
4      long i;
5      long **p = alloca(n * sizeof(long *));
6      p[0] = &i;
7      for (i = 1; i < n; i++)
8          p[i] = q;
9      return *p[idx];
10 }

(b) 生成的部分汇编代码
   long aframe(long n, long idx, long *q)
   n in %rdi, idx in %rsi, q in %rdx
1  aframe:
2    pushq   %rbp
3    movq    %rsp, %rbp
4    subq    $16, %rsp                为 i 分配空间(%rsp = s1)
5    leaq    30(,%rdi,8), %rax
6    andq    $-16, %rax
7    subq    %rax, %rsp               为数组 p 分配空间(%rsp = s2)
8    leaq    15(%rsp), %r8
9    andq    $-16, %r8                将 %r8 设为 &p[0]
     .
     .
```

图 3.54 问题 3.72 的代码。这一函数与图 3.43 中的函数类似。

图 3.54(b) 展示了汇编代码中设置帧指针并为局部变量 i 和 p 分配空间的那一部分。它与 vframe 的相应代码非常相似。我们沿用练习 3.49 中的记法:栈指针在第 4 行被设为值 s_1,在第 7 行被设为值 s_2。数组 p 的起始地址在第 9 行被设为值 p。多余空间 e_2 可能出现在 s_2 和 p 之间,多余空间 e_1 可能出现在数组 p 的末端和 s_1 之间。

A. 用数学的语言解释计算 s_2 的逻辑。

B. 用数学的语言解释计算 p 的逻辑。

C. 找出使 e_1 得到最小值和最大值的 n 与 s_1 的值。

D. 这段代码对 s_2 和 p 的值保证了哪些对齐属性?

家庭作业 3.73 ▲ 编写一段汇编代码,使其行为与图 3.51 中的函数 find_range 相同。你的代码应当只包含一条浮点比较指令,然后利用条件

分支来产生正确的结果。请在全部 2^{32} 个可能的参数值上测试你的代码。网络边栏 asm:easm 描述了如何把用汇编语言编写的函数并入 C 程序。

家庭作业 3.74 ▲▲ 编写一段汇编代码，使其行为与图 3.51 中的函数 find_range 相同。你的代码应当只包含一条浮点比较指令，然后利用条件传送来产生正确的结果。你可能会想用到指令 cmovp（奇偶校验为偶时传送）。请在全部 2^{32} 个可能的参数值上测试你的代码。网络边栏 asm:easm 描述了如何把用汇编语言编写的函数并入 C 程序。

家庭作业 3.75 ▲ ISO C99 包括了支持复数的扩展。任何浮点类型都可以用关键字 complex 来修饰。下面是一些对复数数据进行操作、并调用若干相关库函数的示例函数：

```
1   #include <complex.h>
2
3   double c_imag(double complex x) {
4       return cimag(x);
5   }
6
7   double c_real(double complex x) {
8       return creal(x);
9   }
10
11  double complex c_sub(double complex x, double complex y) {
12      return x - y;
13  }
```

编译时，gcc 为这些函数生成如下汇编代码：

```
double c_imag(double complex x)
1  c_imag:
2    movapd   %xmm1, %xmm0
3    ret

double c_real(double complex x)
4  c_real:
5    rep; ret

double complex c_sub(double complex x, double complex y)
6  c_sub:
7    subsd    %xmm2, %xmm0
8    subsd    %xmm3, %xmm1
9    ret
```

根据这些示例，确定下面的问题：

A. 复数参数是如何传递给函数的？

B. 复数值是如何从函数返回的？

3.18 练习答案

答案 3.1 这个练习帮助你练习使用各种不同的操作数形式。

操作数	值	说明
`%rax`	`0x100`	寄存器
`0x104`	`0xAB`	绝对地址
`$0x108`	`0x108`	立即数
`(%rax)`	`0xFF`	地址 `0x100`
`4(%rax)`	`0xAB`	地址 `0x104`
`9(%rax,%rdx)`	`0x11`	地址 `0x10C`
`260(%rcx,%rdx)`	`0x13`	地址 `0x108`
`0xFC(,%rcx,4)`	`0xFF`	地址 `0x100`
`(%rax,%rdx,4)`	`0x11`	地址 `0x10C`

答案 3.2 正如我们已经看到的，gcc 生成的汇编代码在指令上带有后缀，而反汇编器不带。在这两种形式之间切换是一项需要掌握的重要技能。一个重要特征是：x86-64 中的内存引用总是用四字寄存器给出，比如`%rax`，即使操作数是字节、单字或双字。

下面是带后缀形式的代码：

```
movl     %eax, (%rsp)
movw     (%rax), %dx
movb     $0xFF, %bl
movb     (%rsp,%rdx,4), %dl
movq     (%rdx), %rax
movw     %dx, (%rax)
```

答案 3.3 既然我们会依赖 gcc 来生成大部分汇编代码，能写出正确的汇编代码并不是一项关键技能。尽管如此，这个练习会帮助你更加熟悉各种不同的指令类型和操作数类型。

下面是代码以及对错误的解释：

`movb $0xF, (%ebx)`	不能把`%ebx` 用作地址寄存器
`movl %rax, (%rsp)`	指令后缀与寄存器标识符不匹配
`movw (%rax),4(%rsp)`	源和目的不能都是内存引用
`movb %al,%sl`	没有名为`%sl` 的寄存器
`movl %eax,$0x123`	目的操作数不能是立即数
`movl %eax,%dx`	目的操作数的大小不正确
`movb %si, 8(%rbp)`	指令后缀与寄存器标识符不匹配

答案 3.4 这个练习让你更多地练习各种数据传送指令，并思考它们与 C 语言数据类型和转换规则之间的关系。转换中有符号性与大小两方面的细节问题，再加上类型提升，都给这个问题增加了难度。

src_t	dest_t	指令	说明
long	long	movq (%rdi), %rax	读取 8 个字节
		movq %rax, (%rsi)	存储 8 个字节
char	int	movsbl (%rdi), %eax	将 char 转换为 int
		movl %eax, (%rsi)	存储 4 个字节
char	unsigned	movsbl (%rdi), %eax	将 char 转换为 int
		movl %eax, (%rsi)	存储 4 个字节
unsigned char	long	movzbl (%rdi), %eax	读取字节并零扩展
		movq %rax, (%rsi)	存储 8 个字节
int	char	movl (%rdi), %eax	读取 4 个字节
		movb %al, (%rsi)	存储低位字节
unsigned	unsigned char	movl (%rdi), %eax	读取 4 个字节
		movb %al, (%rsi)	存储低位字节
char	short	movsbw (%rdi), %ax	读取字节并符号扩展
		movw %ax, (%rsi)	存储 2 个字节

答案 3.5 逆向工程是理解系统的一种好方法。在这个例子中，我们要反过来推演 C 编译器的作用，找出产生这段汇编代码的是什么样的 C 代码。最好的方法是做一次“仿真”：先让 x、y 和 z 分别位于由指针 xp、yp 和 zp 所指定的位置上，再去运行这段代码。这样我们就会得到如下的行为：

```
void decode1(long *xp, long *yp, long *zp)
xp in %rdi, yp in %rsi, zp in %rdx
decode1:
  movq    (%rdi), %r8       读取 x = *xp
  movq    (%rsi), %rcx      读取 y = *yp
  movq    (%rdx), %rax      读取 z = *zp
  movq    %r8, (%rsi)       把 x 存入 yp
  movq    %rcx, (%rdx)      把 y 存入 zp
  movq    %rax, (%rdi)      把 z 存入 xp
  ret
```

由此可以生成如下的 C 代码：

```
void decode1(long *xp, long *yp, long *zp)
{
    long x = *xp;
    long y = *yp;
    long z = *zp;

    *yp = x;
    *zp = y;
    *xp = z;
}
```

答案 3.6 这个练习展示了 leaq 指令的多面性，也让你进一步练习解读各种不同的操作数形式。尽管这些操作数形式在图 3.3 中被归为“内存”类，但实际上并没有发生内存访问。

指令	结果
`leaq 9(%rdx), %rax`	$9+q$
`leaq (%rdx,%rbx), %rax`	$q+p$
`leaq (%rdx,%rbx,3), %rax`	$q+3p$
`leaq 2(%rbx,%rbx,7), %rax`	$2+8p$
`leaq 0xE(,%rdx,3), %rax`	$14+3q$
`leaq 6(%rbx,%rdx,7), %rdx`	$6+p+7q$

答案 3.7 再一次，事实证明逆向工程是学习 C 代码与所生成的汇编代码之间关系的一种有用方法。

解决这类问题的最好方法，是给汇编代码的各行加上注解，标明正在执行的操作。下面是一个示例：

```
short scale3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
scale3:
  leaq    (%rsi,%rsi,9), %rbx         10 * y
  leaq    (%rbx,%rdx), %rbx           10 * y + z
  leaq    (%rbx,%rdi,%rsi), %rbx      10 * y + z + y * x
  ret
```

从这里出发，很容易写出缺失的表达式：

```
short t = 10 * y + z + y * x;
```

答案 3.8 这个问题让你有机会检验自己对操作数和算术指令的理解。这组指令序列经过专门设计，每条指令的结果都不会影响后面各条指令的行为。

指令	目的操作数	值
`addq %rcx,(%rax)`	`0x100`	`0x100`
`subq %rdx,8(%rax)`	`0x108`	`0xA8`
`imulq $16,(%rax,%rdx,8)`	`0x118`	`0x110`
`incq 16(%rax)`	`0x110`	`0x14`
`decq %rcx`	`%rcx`	`0x0`
`subq %rdx,%rax`	`%rax`	`0xFD`

答案 3.9 这个练习给你机会写一点汇编代码。答案中的代码是由 gcc 生成的。gcc 把参数 n 加载到寄存器`%ecx` 中，这样它随后就可以用字节寄存器`%cl` 来指明 `sarq` 指令的移位量。考虑到 n 有 8 个字节长，用一条 `movl` 指令也许看起来有些奇怪，但请记住，指定位移量只需要最低有效字节。

```
long shift_left4_rightn(long x, long n)
x in %rdi, n in %rsi
shift_left4_rightn:
  movq    %rdi, %rax        读取 x
  salq    $4, %rax          x <<= 4
  movl    %esi, %ecx        读取 n(4 字节)
  sarq    %cl, %rax         x >>= n
```

答案 3.10 这个问题相当直接,因为汇编代码的结构与 C 代码的结构非常贴近。

```
short p1 = y | z;
short p2 = p1 >> 9;
short p3 = ~p2;
short p4 = y - p3;
```

答案 3.11

A. 这条指令用来把寄存器%rcx 置为 0,它利用了这样的性质:对任意 x 都有 `x ^ x = 0`。它对应于 C 语句 `x = 0`。

B. 把寄存器%rcx 置为 0,更直接的方法是使用指令 `movq $0,%rcx`。

C. 不过,把这段代码汇编后再反汇编,我们发现带 `xorq` 的版本只需要 3 个字节,而带 `movq` 的版本需要 7 个字节。把%rcx 置为 0 的其他方式利用了这样一条性质:任何更新低 4 字节的指令都会使高位字节被置为 0。因此,我们既可以选用 `xorl %ecx,%ecx`(2 个字节),也可以选用 `movl $0,%ecx`(5 个字节)。

答案 3.12 我们可以简单地把 `cqto` 指令换成一条把寄存器%rdx 置为 0 的指令,并用 `divq` 而不是 `idivq` 来做除法,得到如下代码:

```
void uremdiv(unsigned long x, unsigned long y,
             unsigned long *qp, unsigned long *rp)
x in %rdi, y in %rsi, qp in %rdx, rp in %rcx
1   uremdiv:
2      movq    %rdx, %r8         拷贝 qp
3      movq    %rdi, %rax        将 x 移到被除数的低 8 字节
4      movl    $0, %edx          将被除数的高 8 字节置为 0
5      divq    %rsi              除以 y
6      movq    %rax, (%r8)       将商存储到 qp
7      movq    %rdx, (%rcx)      将余数存储到 rp
8      ret
```

答案 3.13 很有必要理解,汇编代码并不记录程序值的类型。相反,是由各种不同的指令来决定操作数的大小,以及操作数是无符号数还是有符号数。当从指令序列往回映射到 C 代码时,我们必须做一点侦探工作,来推断程序值的数据类型。

A. 后缀 `l` 和寄存器标识符说明操作数是 32 位的,而比较执行的是补码的小于(<)。我们可以推断 `data_t` 必须是 `int`。

B. 后缀 `w` 和寄存器标识符说明操作数是 16 位的,而比较执行的是补码的大于等于(≥)。我们可以推断 `data_t` 必须是 `short`。

C. 后缀 `b` 和寄存器标识符说明操作数是 8 位的,而比较执行的是无符号数的小于等于(≤)。我们可以推断 `data_t` 必须是 `unsigned char`。

D. 后缀 q 和寄存器标识符说明操作数是 64 位的，而比较执行的是不等(≠)测试——无论参数是有符号数、无符号数还是指针，其行为都完全一样。我们可以推断 data_t 可以是 long、unsigned long，也可以是某种形式的指针。

答案 3.14 这个问题与练习 3.13 类似，不同之处在于它使用的是 test 指令，而不是 cmp 指令。

A. 后缀 q 和寄存器标识符说明操作数是 64 位的，而比较执行的是大于等于(≥)，它必须是有符号数的比较。我们可以推断 data_t 必须是 long。

B. 后缀 w 和寄存器标识符说明操作数是 16 位的，而比较执行的是相等(=)。对有符号数和无符号数，这种行为都一样。我们可以推断 data_t 必须是 short 或 unsigned short。

C. 后缀 b 和寄存器标识符说明操作数是 8 位的，而比较执行的是无符号数的大于(>)。我们可以推断 data_t 必须是 unsigned char。

D. 后缀 l 和寄存器标识符说明操作数是 32 位的，而比较执行的是小于(<)。我们可以推断 data_t 必须是 int。

答案 3.15 这个练习要求你详细考察反汇编代码，并对跳转目标的编码进行推理。它还让你练习十六进制算术。

A. je 指令的目标是 0x4003fc + 0x02。正如原来的反汇编代码所示，这就是 0x4003fe：

```
4003fa: 74 02         je      4003fe
4003fc: ff d0         callq   *%rax
```

B. je 指令的目标是 0x400431−12(因为 0xf4 是 −12 的 1 字节补码表示)。正如原来的反汇编代码所示，这就是 0x400425：

```
40042f: 74 f4         je      400425
400431: 5d            pop     %rbp
```

C. 根据反汇编器产生的注解，跳转目标位于绝对地址 0x400547。而根据字节编码，目标必然位于比 pop 指令的地址多出 0x2 个字节的地方。两者相减，得到地址 0x400545。由于 ja 指令的编码需要 2 个字节，它必定位于地址 0x400543。检查原来的反汇编代码即可证实这些结论：

```
400543: 77 02         ja      400547
400545: 5d            pop     %rbp
```

D. 反过来读这些字节，我们可以看出目标偏移量是 0xffffff73，即十进制的 −141。把它与 0x4005ed(nop 指令的地址)相加，得到地址 0x400560：

```
4005e8: e9 73 ff ff ff      jmpq    400560
4005ed: 90                  nop
```

答案 3.16 给汇编代码加注解，并编写模仿其控制流的 C 代码，是理解汇编语言程序的很好起点。这个问题用一个具有简单控制流的例子让你做练习。它也让你有机会考察逻辑运算的实现。

A. 下面是 C 代码：

```
void goto_cond(short a, short *p) {
    if (a == 0)
        goto done;
    if (a >= *p)
        goto done;
    *p = a;
  done:
    return;
}
```

B. 第一个条件分支是 && 表达式实现的一部分。如果对 a 是否非零的测试失败，代码就会跳过对 `a >= *p` 的测试。

答案 3.17 这个练习帮助你思考一般性翻译规则的思想，以及如何应用它。

A. 转换成这种替换形式只需要把几行代码对调一下位置：

```
long gotodiff_se_alt(long x, long y) {
    long result;
    if (x < y)
         goto x_lt_y;
    ge_cnt++;
    result = x - y;
    return result;
  x_lt_y:
    lt_cnt++;
    result = y - x;
    return result;
}
```

B. 在大多数方面，这种选择是随意的。不过，对于没有 else 语句的常见情形，原来的规则效果更好。对于这种情况，我们可以简单地把翻译规则修改成如下形式：

```
t = test-expr;
if (!t)
     goto done;
then-statement
done:
```

而基于替换规则的翻译则更加繁琐。

答案 3.18 这个问题要求你攻克一个嵌套的分支结构，你会从中看到翻译 if 语句的规则是如何应用的。总的来说，机器码是 C 代码的一次非常直接的翻译。

```
short test(short x, short y, short z) {
    short val = z+y-x;
    if (z > 5) {
        if (y > 2)
             val = x/z;
        else
             val = x/y;
    } else if (z < 3)
        val = z/y;
    return val;
}
```

答案 3.19 这个问题巩固了我们计算预测错误的惩罚（misprediction penalty）的方法。

A. 我们可以直接应用公式，得到 $TMP = 2(45 - 25) = 40$。

B. 发生分支预测错误时，该函数大约需要 $25 + 40 = 65$ 个周期。

答案 3.20 这道题给你一个练习条件传送指令用法的机会。

A. 运算符是‘/’。可以看出，这是通过右移实现除以 4 的幂的一个例子（见 2.3.7 节）。在移位 $k = 4$ 位之前，当被除数为负时，必须加上偏置 $2^k - 1 = 15$。

B. 下面是带注释的汇编代码：

```
short arith(short x)
x 在 %rdi 中
arith:
    leaq   15(%rdi), %rbx        temp = x+15
    testq  %rdi, %rdi            测试 x
    cmovns %rdi, %rbx            若 x >= 0,temp = x
    sarq   $4, %rbx              result = temp >> 4 (= x/16)
    ret
```

程序预先创建了一个等于 $x + 15$ 的临时值，以备 x 为负而需要加偏置的情况。当 $x \geq 0$ 时，`cmovns` 指令有条件地把这个值改成 x，然后它被移位 4 位，生成 $x/16$。

答案 3.21 这道题与 3.18 类似，只是其中一些条件判断是通过条件数据传送来实现的。虽然把这段代码放进原来的 C 代码框架似乎让人望而生畏，但你会发现它相当贴合那些翻译规则。

```
short test(short x, short y) {
    short val = y + 12;
    if (x < 0) {
```

```
        if (x < y)
            val = x * y;
        else
            val = x | y;
    } else if (y > 10)
        val = x / y;
    return val;
}
```

答案 3.22 **A.** 用 32 位 `int` 计算 14! 会溢出。正如在练习 2.35 中学到的,当我们尝试计算 $n!$ 而得到值 x 时,可以计算 x/n 并看它是否等于 $(n-1)!$,以此来测试是否溢出(前提是我们已经确保 $(n-1)!$ 的计算没有溢出)。这里我们得到 1,278,945,280/14 = 91353234.286。作为第二个测试,可以看到 10! 之后的任何阶乘都必须是 100 的倍数,因而最后两位数字是 0。14! 的正确值是 87,178,291,200。

另外,我们还可以列出用数据类型 `int` 计算到 14! 为止的阶乘表,如下所示:

n	$n!$	是否正确
1	1	Y
2	2	Y
3	6	Y
4	24	Y
5	120	Y
6	720	Y
7	5,040	Y
8	40,320	Y
9	362,880	Y
10	3,628,800	Y
11	39,916,800	Y
12	479,001,600	Y
13	1,932,053,504	N
14	1,278,945,280	N

B. 用数据类型 `long` 来做这个计算可以一直算到 20!,因此 14! 的计算不会溢出。

答案 3.23 编译循环生成的代码分析起来会很棘手,因为编译器会对循环代码进行许多不同的优化,而且要把程序变量和寄存器对应起来也可能很困难。这个特定的例子展示了汇编代码并非只是 C 代码的直接翻译的几个地方。

A. 虽然参数 x 是通过寄存器`%rdi` 传给函数的,但可以看到,一旦进入循环,这个寄存器就再没有被引用过。反而可以看到,寄存器`%rbx`、`%rcx` 和`%rdx` 在第 2 ~ 5 行被初始化为 x、$x/9$ 和 $4*x$。因此可以得出结论:这些寄存器保存的是程序变量。

B. 编译器判定指针 p 总是指向 x，因此表达式 `(*p)+=5` 只是对 x 加 5。它通过第 7 行的 `leaq` 指令，把这个加 5 与 y 的递增合并在一起。

C. 带注释的代码如下：

```
short dw_loop(short x)
x 初始在 %rdi 中
1  dw_loop:
2    movq   %rdi, %rbx            将 x 复制到 %rbx
3    movq   %rdi, %rcx
4    idivq  $9, %rcx              计算 y = x/9
5    leaq   (,%rdi,4), %rdx       计算 n = 4*x
6    .L2:                         loop:
7    leaq   5(%rbx,%rcx), %rcx    计算 y += x + 5
8    subq   $2, %rdx              将 n 递减 2
9    testq  %rdx, %rdx            测试 n
10   jg     .L2                   若 > 0,转到 loop
11   rep; ret                     返回
```

答案 3.24 这段汇编代码是对使用跳转到中间方法翻译的循环的一个相当直接的翻译。完整的 C 代码如下：

```
short loop_while(short a, short b)
{
    short result = 0;
    while (a > b) {
        result = result + (a*b);
        a = a-1;
    }
    return result;
}
```

答案 3.25 虽然生成的代码没有完全遵循 guarded-do 翻译的标准模式，但可以看到它等价于下面这段 C 代码：

```
long loop_while2(long a, long b)
{
    long result = b;
    while (b > 0) {
        result = result * a;
        b = b-a;
    }
    return result;
}
```

我们会经常遇到这种情况，尤其是在用更高的优化级别编译时，`gcc` 会在它生成的代码的确切形式上作一些小小的改动，同时保持所需的功能。

答案 3.26 能够从汇编代码反向推导回 C 代码，是逆向工程（reverse engineering）的一个典型例子。

A. 可以看到，代码使用的是跳转到中间的翻译，这体现在第 3 行的 `jmp` 指令上。

B. 下面是原来的 C 代码：

```
short test_one(unsigned short x) {
    short val = 1;
    while (x) {
        val ^= x;
        x >>= 1;
    }
    return val & 0;
}
```

C. 这段代码计算参数 x 的奇偶性(parity)。也就是说，如果 x 中 1 的个数是奇数，它返回 1；如果是偶数，返回 0。

答案 3.27 这道练习旨在强化你对循环是如何实现的理解。

```
long fibonacci_gd_goto(long n)
{
    long i = 2;
    long next, first = 0, second = 1;
    if (n <= 1)
        goto done;
loop:
        next = first + second;
        first = second; second = next;
        i++;
        if (i <= n)
            goto loop;
done:
        return n;
}
```

答案 3.28 这道题比 3.26 更棘手，因为循环内的代码更复杂，整体操作也更不熟悉。

A. 下面是原来的 C 代码：

```
long fun_b(unsigned long x) {
    long val = 0;
    long i;
    for (i = 64; i != 0; i--) {
        val = (val << 1) | (x & 0x1);
        x >>= 1;
    }
    return val;
}
```

B. 代码是用 guarded-do 变换生成的，但编译器检测到，由于 i 被初始化为 64，它必然满足测试 $i \neq 0$，因此不需要初始测试。

C. 这段代码把 x 中各个位反转,产生一个镜像。它把 x 的各位从左向右逐个移出,然后在 `val` 从右向左移位的过程中把这些位填进去。

答案 3.29 我们给出的把 `for` 循环翻译成 `while` 循环的规则有一点过于简单了——这正是唯一需要特殊考虑的地方。

A. 应用我们的翻译规则会得到如下代码:

```
/* 把 for 循环朴素地翻译成 while 循环 */
/* 警告:这段代码有缺陷 */
long sum = 0;
long i = 0;
while (i < 10) {
    if (i & 1)
        /* 这会导致死循环 */
        continue;
    sum += i;
    i++;
}
```

这段代码会出现死循环,因为 `continue` 语句会阻止索引变量 i 被更新。

B. 一般的解决办法是用一条 `goto` 语句代替 `continue` 语句,它跳过循环体的剩余部分,直接转到更新部分:

```
/* 把 for 循环正确地翻译成 while 循环 */
long sum = 0;
long i = 0;
while (i < 10) {
    if (i & 1)
        goto update;
    sum += i;
update:
    i++;
}
```

答案 3.30 这道题让你有机会对 `switch` 语句的控制流进行推理。回答这些问题需要把汇编代码中几处地方的信息结合起来。

- 汇编代码的第 2 行将 x 加上 2,把各个情形的下界变成 0。这意味着最小的 case 标签是 -2。
- 第 3 行和第 4 行在调整后的 case 值大于 8 时,让程序跳到 default 情形。这意味着最大的 case 标签是 $-2+8=6$。
- 在跳转表中,可以看到第 6 行(case 值 2)和第 9 行(case 值 5)的条目与第 4 行跳转指令的目标 `.L2` 相同,表示 default 情形的行为。因此,`switch` 语句体中缺少 case 标签 2 和 5。
- 在跳转表中,可以看到第 3 行和第 10 行的条目具有相同的目标。它们对应于 case -1 和 6。

- 在跳转表中,可以看到第 5 行和第 7 行的条目具有相同的目标。它们对应于 case 1 和 3。

由此推理,我们得出如下结论:

A. `switch` 语句体中的 case 标签的值为 −2、−1、0、1、3、4 和 6。

B. 目标为`.L5` 的 case 拥有标签 −1 和 6。

C. 目标为`.L7` 的 case 拥有标签 1 和 3。

答案 3.31 逆向工程编译出来的 `switch` 语句的关键,是把来自汇编代码和跳转表的信息结合起来,理清各个不同的情形。从 `ja` 指令(第 3 行)可以看到,default 情形的代码的标签是`.L2`。可以看到跳转表中唯一重复的其他标签是`.L5`,所以这必定是 case C 和 D 的代码。可以看到代码在第 8 行贯穿(fall through),因此标签`.L7` 必定对应 case A,标签`.L3` 必定对应 case B。这样只剩下标签`.L6` 对应 case E。

原来的 C 代码如下:

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case 5:
        c = b ^ 15;
        /* 贯穿执行 */
    case 0:
        val = c + 112;
        break;
    case 2:
    case 7:
        val = (c + b) << 2;
        break;
    case 4:
        val = a;
        break;
    default:
        val = b;
    }
    *dest = val;
}
```

答案 3.32 在这一层细节上跟踪程序执行,能够巩固关于过程调用和返回的许多方面的知识。我们可以清楚地看到,函数被调用时控制是如何传给它的,返回时调用函数又是如何继续执行的。我们还可以看到参数是如何通过寄存器`%rdi` 和`%rsi` 传递的,而结果又是如何通过寄存器`%rax` 返回的。

指令			状态值(执行开始时)					
标号	PC	指令	%rdi	%rsi	%rax	%rsp	*%rsp	说明
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	调用 first(10)
F1	0x400548	lea	10	—	—	0x7fffffffe818	0x400565	进入 first
F2	0x40054c	sub	10	11	—	0x7fffffffe818	0x400565	
F3	0x400550	callq	9	11	—	0x7fffffffe818	0x400565	调用 last(9, 11)
L1	0x400540	mov	9	11	—	0x7fffffffe810	0x400555	进入 last
L2	0x400543	imul	9	11	9	0x7fffffffe810	0x400555	
L3	0x400547	retq	9	11	99	0x7fffffffe810	0x400555	从 last 返回 99
F4	0x400555	repz repq	9	11	99	0x7fffffffe818	0x400565	从 first 返回 99
M2	0x400565	mov	9	11	99	0x7fffffffe820	—	恢复 main

答案 3.33 由于不同数据大小的混合,这道题有点棘手。

我们先描述一个答案,然后再解释第二种可能性。如果我们假设第一次加法(第 3 行)实现的是 `*u += a`,而第二次加法(第 4 行)实现的是 `v += b`,那么可以看到,a 是作为第一个参数在`%edi` 中传递的,并且在把它加到`%rdx` 指向的 8 个字节之前,先从 4 字节转换成了 8 字节。这意味着 a 必然是 `int` 类型,而 u 必然是 `long *` 类型。我们还可以看到,参数 b 的低位字节被加到了`%rcx` 指向的字节上。这意味着 v 必然是 `char *` 类型,但 b 的类型是不确定的——它可能是 1、2、4 或 8 字节长。

注意到返回值 6——它计算的是 a 和 b 的长度之和——就可以消解这个不确定性。既然我们知道 a 是 4 字节长,就可以推出 b 必然是 2。

这个函数的一个带注释的版本解释了这些细节:

```
int procprobl(int a, short b, long *u, char *v)
a 在 %edi 中,b 在 %si 中,u 在 %rdx 中,v 在 %rcx 中
1  procprob:
2     movslq %edi, %rdi            将 a 转换成 long
3     addq   %rdi, (%rdx)          加到 *u(long)上
4     addb   %sil, (%rcx)          把 b 的低位字节加到 *v 上
5     movl   $6, %eax              返回 4+2
6     ret
```

另外还可以看到,如果这两次求和在汇编代码中是按照与 C 代码中相反的顺序计算的,那么同一段汇编代码也是有效的。这需要把参数 a 和 b 互换、参数 u 和 v 互换,得到如下原型:

```
int procprob(int b, short a, long *v, char *u);
```

答案 3.34 这个例子演示了被调用者保存寄存器的使用,以及用栈来保存局部数据。

A. 可以看到,第 9 ~ 14 行把局部值 `a0` ~ `a5` 分别保存到被调用者保存寄存器`%rbx`、`%r15`、`%r14`、`%r13`、`%r12` 和`%rbp` 中。

B. 局部值 `a6` 和 `a7` 保存到栈上,偏移分别为相对于栈指针的 0 和 8(第 16 行和第 18 行)。

C. 在保存了六个局部变量之后,程序已经用完了被调用者保存寄存器的储备。它把剩余的两个局部值保存到栈上。

答案 3.35 这道题给你一个考察递归函数代码的机会。一个重要的经验是:递归代码具有与我们看过的其他函数完全相同的结构。栈和寄存器保存规则就足以使递归函数正确运行。

A. 寄存器%rbx 保存参数 x 的值,以便在计算结果表达式时使用。

B. 这段汇编代码是由下面的 C 代码生成的:

```
long rfun(unsigned long x) {
    if (x == 0)
        return 0;
    unsigned long nx = x>>2;
    long rv = rfun(nx);
    return x + rv;
}
```

答案 3.36 这道练习考察你对数据大小和数组下标的理解。注意,任何类型的指针的长度都是 8 字节。数据类型 short 需要 2 字节,而 int 需要 4 字节。

数组	元素大小	总大小	起始地址	元素 i
P	4	20	x_P	$x_P + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R	8	72	x_R	$x_R + 8i$
S	8	80	x_S	$x_S + 8i$
T	8	16	x_T	$x_T + 8i$

答案 3.37 这道题是针对整数数组 E 给出的那道题的变体。理解指针与被指向的对象之间的区别很重要。由于数据类型 short 需要 2 字节,所有的数组下标都要乘以因子 2。不是像以前那样使用 movl,我们现在使用 movw。

表达式	类型	值	汇编代码
P[1]	short	$\mathrm{M}[x_P + 2]$	movw 2(%rdx),%ax
P+3+i	short *	$x_P + 6 + 2i$	leaq 6(%rdx,%rcx,2),%rax
P[i*6-5]	short	$\mathrm{M}[x_P + 12i - 10]$	movw -10(%rdx,%rcx,12),%ax
P[2]	short	$\mathrm{M}[x_P + 4]$	movw 4(%rdx),%ax
&P[i+2]	short *	$x_P + 2i + 4$	leaq 4(%rdx,%rcx,2),%rax

答案 3.38 这道题要求你逐步做比例运算,确定地址的计算,并应用公式 (3.1) 进行按行主序的下标计算。第一步是给汇编代码加注,确定地址引用是如何计算的:

```
long sum_element(long i, long j)
i 在 %rdi 中,j 在 %rsi 中
1  sum_element:
2    leaq   0(,%rdi,8), %rdx      计算 8i
3    subq   %rdi, %rdx            计算 7i
4    addq   %rsi, %rdx            计算 7i + j
```

```
5    leaq   (%rsi,%rsi,4), %rax   计算 5j
6    addq   %rax, %rdi            计算 i + 5j
7    movq   Q(,%rdi,8), %rax      读取 M[xQ + 8(5j + i)]
8    addq   P(,%rdx,8), %rax      加上 M[xP + 8(7i + j)]
9    ret
```

我们可以看到，对矩阵 P 的引用位于字节偏移 $8 \cdot (7i + j)$ 处，而对矩阵 Q 的引用位于字节偏移 $8 \cdot (5j + i)$ 处。由此可以确定 P 有 7 列，而 Q 有 5 列，因此 $M = 5$、$N = 7$。

答案 3.39 这些计算都是公式 (3.1) 的直接应用：

- 当 $L = 4$、$C = 16$ 且 $j = 0$ 时，指针 `Aptr` 计算为 $x_A + 4 \cdot (16i + 0) = x_A + 64i$。
- 当 $L = 4$、$C = 16$、$i = 0$ 且 $j = k$ 时，`Bptr` 计算为 $x_B + 4 \cdot (16 \cdot 0 + k) = x_B + 4k$。
- 当 $L = 4$、$C = 16$、$i = 16$ 且 $j = k$ 时，`Bend` 计算为 $x_B + 4 \cdot (16 \cdot 16 + k) = x_B + 1{,}024 + 4k$。

答案 3.40 这道练习要求你能够通过研究编译器生成的汇编代码，弄清都做了哪些优化。在这个例子中，编译器的优化是非常聪明的。

我们先来研究下面的 C 代码，然后再看它是如何从为原始函数生成的汇编代码推导出来的。

```
/* 把所有对角线元素都设为 val */
void fix_set_diag_opt(fix_matrix A, int val) {
    int *Abase = &A[0][0];
    long i = 0;
    long iend = N*(N+1);
    do {
        Abase[i] = val;
        i += (N+1);
    } while (i != iend);
}
```

这个函数引入了一个类型为 `int *` 的变量 `Abase`，它指向数组 A 的开头。这个指针指定了一个由数组 A 的元素按行主序排列而成的 4 字节整数序列。我们引入一个整数变量 `index` 来遍历 A 的对角线元素，其性质是：对角线元素 i 与 $i + 1$ 在这个序列中相距 $N + 1$ 个元素，而一旦到达对角线元素 N（`index` 值为 $N(N + 1)$），就已经越过了末尾。

实际的汇编代码遵循这个一般形式，但现在指针增量必须乘以因子 4。我们把寄存器`%rax` 标为保存值 `index4`，它等于我们 C 版本中的 `index`，但已经乘以因子 4。对于 $N = 16$，可以看到 `index4` 的停止点将是 $4 \cdot 16(16 + 1) = 1{,}088$。

```
1  fix_set_diag:
      void fix_set_diag(fix_matrix A, int val)
      A 在 %rdi 中,val 在 %rsi 中
2    movl   $0, %eax                设置 index4 = 0
3    .L13:                          loop:
4    movl   %esi, (%rdi,%rax)       把 Abase[index4/4] 设为 val
5    addq   $68, %rax               增量 index4 += 4(N+1)
6    cmpq   $1088, %rax             比较 index4: 4N(N+1)
7    jne    .L13                    若 !=,转到 loop
8    rep; ret                       返回
```

答案 3.41 这道题让你思考结构的布局以及访问结构字段的代码。这个结构声明是正文中给出的例子的一个变体。它表明,嵌套结构是通过把内层结构嵌入外层结构来分配空间的。

A. 该结构的布局如下:

偏移	0	8	10	12	20
内容	p	s.x	s.y	next	

B. 它使用 20 字节。

C. 和往常一样,我们从给汇编代码加注开始:

```
void st_init(struct test *st)
st 在 %rdi 中
1  st_init:
2    movl   8(%rdi), %eax          读取 st->s.x
3    movl   %eax, 10(%rdi)         保存到 st->s.y
4    leaq   10(%rdi), %rax         计算 &(st->s.y)
5    movq   %rax, (%rdi)           存入 st->p
6    movq   %rdi, 12(%rdi)         把 st 存入 st->next
7    ret
```

由此,我们可以生成如下 C 代码:

```
void st_init(struct test *st)
{
    st->s.y   = st->s.x;
    st->p     = &(st->s.y);
    st->next = st;
}
```

答案 3.42 这道题演示了一种非常常见的数据结构以及在它上面的操作是如何在机器码中实现的。我们通过先给汇编代码加注来求解这道题,识别出结构的两个字段位于偏移 0(v)和 2(p)处。

```
short test(struct ACE *ptr)
ptr 在 %rdi 中
1  test:
2    movl   $1, %eax               result = 1
3    jmp    .L2                    转到 middle
```

```
4    .L3:                          loop:
5    imulq  (%rdi), %rax           result *= ptr->v
6    movq   2(%rdi), %rdi          ptr = ptr->p
7    .L2:                          middle:
8    testq  %rdi, %rdi             测试 ptr
9    jne    .L3                    若 != NULL,转到 loop
10   rep; ret
```

A. 根据带注释的代码,我们可以生成一个 C 版本:

```
short test(struct ACE *ptr) {
    short val = 1;
    while (ptr) {
        val *= ptr->v;
        ptr = ptr->p;
    }
    return val;
}
```

B. 可以看到,每个结构都是单向链表中的一个元素,其中字段 v 是元素的值,p 是指向下一个元素的指针。函数 fun 计算链表中各元素值之和。

答案 3.43 结构和联合涉及的概念集合很简单,但是要熟练掌握各种不同的引用模式及其实现,还需要练习。

表达式	类型	代码
up->t1.u	long	movq (%rdi), %rax movq %rax, (%rsi)
up->t1.v	short	movw 8(%rdi), %ax movw %rax, (%rsi)
&up->t1.w	char *	addq $10, %rdi movq %rdi, (%rsi)
up->t2.a	int *	movq %rdi, (%rsi)
up->t2.a[up->t1.u]	int	movq (%rdi), %rax movl (%rdi,%rax,4), %eax movl %eax, (%rsi)
*up->t2.p	char	movq 8(%rdi), %rax movb (%rax), %al movb %al, (%rsi)

答案 3.44 理解结构的布局和对齐,对于理解不同的数据结构需要多少存储空间,以及理解编译器为访问结构而生成的代码,都非常重要。这道题让你算出一些示例结构的具体细节。

A. `struct P1 { short i; int c; int *j; short *d; };`

i	c	j	d	总大小	对齐
0	2	6	14	16	8

B. `struct P2 { int i[2]; char c[8]; short [4]; long *j; };`

i	c	d	j	总大小	对齐
0	8	16	24	32	8

C. `struct P3 { long w[2]; int *c[2] };`

w	c	总大小	对齐
0	16	32	8

D. `struct P4 { char w[16]; char *c[2] };`

w	c	总大小	对齐
0	16	32	8

E. `struct P5 { struct P4 a[2]; struct P1 t };`

a	t	总大小	对齐
0	24	40	8

答案 3.45 这是一道理解结构布局和对齐的练习题。

A. 下面是各对象的大小和字节偏移:

字段	a	b	c	d	e	f	g	h
大小	8	4	1	2	8	8	4	8
偏移	0	8	12	16	24	32	40	48

B. 这个结构总长度为 56 字节。结构的末尾不需要填充就能满足 8 字节对齐的要求。

C. 当所有数据元素的长度都是 2 的幂时,一个有效的策略是按大小递减的顺序排列结构元素。由此得到如下声明:

```
struct {
    int    *a;
    char   *h;
    double f;
    long   e;
    float b;
    int    g;
    short d;
    char   c;
```

```
} rec;
```

各字段的偏移如下：

字段	a	h	f	e	b	g	d	c
大小	8	8	8	8	4	4	2	1
偏移	0	8	16	24	32	36	40	42

该结构必须填充 5 字节才能满足 8 字节对齐的要求，总共 48 字节。

答案 3.46 这道题覆盖了广泛的主题，如栈帧、字符串表示、ASCII 码和字节顺序。它演示了越界内存引用的危险性以及缓冲区溢出背后的基本思想。

A. 第 3 行之后的栈：

```
00 00 00 00 00 40 00 76 返回地址
01 23 45 67 89 AB CD EF 被保存的%rbx
                        ← buf = %rsp
```

B. 第 5 行之后的栈：

```
00 00 00 00 00 40 00 34 返回地址
33 32 31 30 39 38 37 36 被保存的%rbx
35 34 33 32 31 30 39 38
37 36 35 34 33 32 31 30 ← buf = %rsp
```

C. 程序试图返回到地址 `0x040034`。低位的 2 个字节被字符 '4' 的代码和结尾的空字符覆盖了。

D. 寄存器`%rbx` 的被保存值被设为 `0x3332313039383736`。这个值会在 `get_line` 返回之前被加载到寄存器中。

E. 调用 `malloc` 时本应使用 `strlen(buf)+1` 作为参数，而且代码还应该检查返回值不等于 `NULL`。

答案 3.47 **A.** 这对应于大约 2^{13} 个地址的取值范围。

B. 一个 128 字节的 nop 雪橇在每次试验中可以覆盖 2^7 个地址，因此我们只需要大约 $2^6 = 64$ 次尝试。

这个例子清楚地表明，这个版本的 Linux 的随机化程度只能对溢出攻击提供很小的阻碍。

答案 3.48 这道题让你再一次看到 x86-64 代码是如何管理栈的，同时也帮助你更好地理解如何防御缓冲区溢出攻击。

A. 对于未受保护的代码，可以看到第 4 行和第 5 行计算出 `v` 和 `buf` 的位置分别在相对于`%rsp` 的偏移 24 和 0 处。在受保护的代码中，金丝雀值存储在偏移 40 处(第 4 行)，而 `v` 和 `buf` 在偏移 8 和 16 处(第 7 行和第 8 行)。

B. 在受保护的代码中，局部变量 `v` 的位置比 `buf` 更靠近栈顶，因此 `buf` 越界不会破坏 `v` 的值。

答案 3.49 这段代码把我们在位级算术运算中见过的许多技巧结合在了一起。要弄明白它，需要仔细研究。

A. 第 5 行的 `leaq` 指令计算出 $8n + 22$，然后第 6 行的 `andq` 指令把它向下舍入到 16 的最近倍数。当 n 为奇数时得到的值是 $8n + 8$，当 n 为偶数时是 $8n + 16$，再从 s_1 中减去这个值，得到 s_2。

B. 这个序列中的三条指令把 s_2 向上舍入到 8 的最近倍数。它们使用的是我们在 2.3.7 节中看到的为除以 2 的幂而用的偏置与移位相结合的方法。

C. 这两个例子可以看作是使 e_1 和 e_2 的值达到最小和最大的情形。

n	s_1	s_2	p	e_1	e_2
5	2,065	2,017	2,024	1	7
6	2,064	2,000	2,000	16	0

D. 可以看到，s_2 的计算方式保持了 s_1 相对于 16 的最近倍数的偏移。我们还可以看到，p 会按 8 的倍数对齐，这也正是对 8 字节元素的数组所推荐的做法。

答案 3.50 这道练习要求你逐步执行这段代码，仔细注意使用了哪些转换和数据传送指令。我们可以看到各值被取出并转换的过程如下：

- `dp` 处的值被取出，转换成 `int`(第 4 行)，然后存储到 `ip`。因此可以推断 `val1` 是 `d`。
- `ip` 处的值被取出，转换成 `float`(第 6 行)，然后存储到 `fp`。因此可以推断 `val2` 是 `i`。
- `l` 的值被转换成 `double`(第 8 行)并存储到 `dp`。因此可以推断 `val3` 是 `l`。
- `fp` 处的值在第 3 行被取出。第 10 ~ 11 行的两条指令把它转换成双精度，作为寄存器`%xmm0` 中返回的值。因此可以推断 `val4` 是 `f`。

答案 3.51 这些情形可以这样处理：从图 3.47 和图 3.48 的表中选取相应的条目，或者使用在浮点数格式之间进行转换的代码序列之一。

T_x	T_y	指令
long	double	vcvtsi2sdq %rdi, %xmm0, %xmm0
double	int	vcvttsd2si %xmm0, %eax
float	double	vunpcklpd %xmm0, %xmm0, %xmm0 vcvtpd2ps %xmm0, %xmm0
long	float	vcvtsi2ssq %rdi, %xmm0, %xmm0
float	long	vcvttss2siq %xmm0, %rax

答案 3.52 把参数映射到寄存器的基本规则相当简单(尽管当参数更多、类型更复杂时,这些规则会变得复杂得多 [77])。

A. `double g1(double a, long b, float c, int d);` 寄存器: a 在`%xmm0` 中,b 在`%rdi` 中,c 在`%xmm1` 中,d 在`%esi` 中。

B. `double g2(int a, double *b, float *c, long d);` 寄存器:a 在`%edi` 中,b 在`%rsi` 中,c 在`%rdx` 中,d 在`%rcx` 中。

C. `double g3(double *a, double b, int c, float d);` 寄存器: a 在`%rdi` 中,b 在`%xmm0` 中,c 在`%esi` 中,d 在`%xmm1` 中。

D. `double g4(float a, int *b, float c, double d);` 寄存器:a 在`%xmm0` 中,b 在`%rdi` 中,c 在`%xmm1` 中,d 在`%xmm2` 中。

答案 3.53 从汇编代码可以看到,有两个整数参数,通过寄存器`%rdi` 和`%rsi` 传递,我们命名为 `i1` 和 `i2`。类似地,有两个浮点参数,通过寄存器`%xmm0` 和`%xmm1` 传递,我们命名为 `f1` 和 `f2`。

然后我们可以给汇编代码加注:

```
参数记为 i1 (%rdi)、i2 (%esi)、
          f1 (%xmm0) 和 f2 (%xmm1)

double funct1(arg1_t p, arg2_t q, arg3_t r, arg4_t s)
1  funct1:
2    vcvtsi2ssq    %rsi, %xmm2, %xmm2     读取 i2 并从 long 转换成 float
3    vaddss        %xmm0, %xmm2, %xmm0    加上 f1(类型 float)
4    vcvtsi2ss     %edi, %xmm2, %xmm2     读取 i1 并从 int 转换成 float
5    vdivss        %xmm0, %xmm2, %xmm0    计算 i1 / (i2 + f1)
6    vunpcklps     %xmm0, %xmm0, %xmm0
7    vcvtps2pd     %xmm0, %xmm0           转换成 double
8    vsubsd        %xmm1, %xmm0, %xmm0    计算 i1 / (i2 + f1) - f2
 ↪ (double)
9    ret
```

由此我们看到,这段代码计算的值是 `i1/(i2+f1)-f2`。我们还可以看到,`i1` 的类型是 `int`,`i2` 的类型是 `long`,`f1` 的类型是 `float`,而 `f2` 的类型是 `double`。参数与这些名称相匹配的唯一歧义来自乘法的可交换性——这会产生两种可能的结果:

```
double funct1a(int p, float q, long r, double s);
```

```
double funct1b(int p, long q, float r, double s);
```

答案 3.54 这道题可以轻易地解决：逐条执行汇编代码，确定每一步上计算的是什么，如下面的注释所示：

```
double funct2(double w, int x, float y, long z)
w 在 %xmm0 中,x 在 %edi 中,y 在 %xmm1 中,z 在 %rsi 中
1  funct2:
2    vcvtsi2ss    %edi, %xmm2, %xmm2      把 x 转换成 float
3    vmulss       %xmm1, %xmm2, %xmm1     乘以 y
4    vunpcklps    %xmm1, %xmm1, %xmm1
5    vcvtps2pd    %xmm1, %xmm2            把 x*y 转换成 double
6    vcvtsi2sdq   %rsi, %xmm1, %xmm1      把 z 转换成 double
7    vdivsd       %xmm1, %xmm0, %xmm0     计算 w/z
8    vsubsd       %xmm0, %xmm2, %xmm0     从 x*y 中减去
9    ret                                  返回
```

从这个分析我们可以得出结论：该函数计算的是 $y * x - w/z$。

答案 3.55 这道题涉及的推理与确定标号 .LC2 处声明的数字编码为 1.8 时所需的推理相同，但例子更简单。

我们看到这两个值是 0 和 1077936128（0x40400000）。从高位字节中，我们可以提取出阶码字段 0x404（1028），从中减去偏置 1023，得到阶码 5。把这两个值的小数位拼接起来，得到小数域为 0，但隐含的前导位使其值为 1.0。因此这个常量是 $1.0 \times 2^5 = 32.0$。

答案 3.56 **A.** 我们看到，从地址 .LC1 开始的 16 字节构成一个掩码，其中低序 8 个字节全为 1，只有最高有效位除外——而那一位是双精度值的符号位。当我们计算这个掩码与 %xmm0 的 AND 时，它会清除 x 的符号位，从而得到绝对值。实际上，我们是通过把 EXPR(x) 定义为 fabs(x) 来生成这段代码的，其中 fabs 定义在 <math.h> 中。

B. 我们看到 vxorpd 指令把整个寄存器设置为 0，因此这是一种生成浮点常量 0.0 的方式。

C. 我们看到，从地址 .LC2 开始的 16 字节构成一个掩码，它只有单个 1 位，位于 XMM 寄存器中低序值的符号位的位置上。当我们计算这个掩码与 %xmm0 的异或时，就会改变 x 的符号，计算出表达式 $-x$。

答案 3.57 我们再来给代码加注，包括处理条件分支：

```
double funct3(int *ap, double b, long c, float *dp)
ap 在 %rdi 中,b 在 %xmm0 中,c 在 %rsi 中,dp 在 %rdx 中
1  funct3:
2    vmovss     (%rdx), %xmm1               读取 d = *dp
3    vcvtsi2sd  (%rdi), %xmm2, %xmm2        读取 a = *ap 并转换成 double
4    vucomisd   %xmm2, %xmm0                比较 b : a
5    jbe        .L8                         若 <=,转到 lesseq
```

```
6    vcvtsi2ssq %rsi, %xmm0, %xmm0          把 c 转换成 float
7    vmulss     %xmm1, %xmm0, %xmm1         乘以 d
8    vunpcklps  %xmm1, %xmm1, %xmm1
9    vcvtps2pd  %xmm1, %xmm0                转换成 double
10   ret                                    返回
11   .L8:                                   lesseq:
12   vaddss     %xmm1, %xmm1, %xmm1         计算 d+d = 2.0 * d
13   vcvtsi2ssq %rsi, %xmm0, %xmm0          把 c 转换成 float
14   vaddss     %xmm1, %xmm0, %xmm0         计算 c + 2*d
15   vunpcklps  %xmm0, %xmm0, %xmm0
16   vcvtps2pd  %xmm0, %xmm0                转换成 double
17   ret                                    返回
```

由此,我们可以为 funct3 写出如下代码:

```
double funct3(int *ap, double b, long c, float *dp) {
    int a = *ap;
    float d = *dp;
    if (a < b)
        return c*d;
    else
        return c+2*d;
}
```

4

处理器体系结构

现代微处理器属于有史以来人类创造的最复杂的系统之列。一小块指甲大小的硅片，可以容纳多个高性能处理器、容量相当大的高速缓存，以及把这些处理器与外部设备相连所需的逻辑电路。就性能而言，如今集成在单块芯片上的处理器，已经远远超过 20 年前售价超过 1,000 万美元、要占据整个房间的超级计算机。就连手机、导航系统和可编程恒温器这类日常电器中的嵌入式处理器，其威力也大大超出了计算机早期开发者所能设想的程度。

到目前为止，我们都是以机器级程序为底限来观察计算机系统的。我们已经看到，处理器必须执行一个指令序列，其中每条指令完成某种基本操作，例如把两个数相加。一条指令以二进制形式编码成一个或多个字节的序列。某个特定处理器所支持的指令及其字节级编码，称为它的指令集体系结构（instruction set architecture，ISA）。不同的处理器“家族”，例如 Intel IA32 和 x86-64、IBM/Freescale Power，以及 ARM 处理器家族，有着不同的 ISA。为一种机器编译的程序不能在另一种机器上运行。另一方面，同一处理器家族内存在许多不同的型号。每个制造商生产的处理器性能和复杂度都在不断提高，但不同的型号在 ISA 层面保持兼容。x86-64 这样广为流行的家族，其处理器由多家制造商供货。这样，ISA 就在编译器编写者与处理器设计者之间提供了一个概念上的抽象层：编译器编写者只需要知道哪些指令是允许使用的以及它们如何编码，而处理器设计者则必须造出能执行这些指令的机器。

在本章中，我们将简要考察处理器硬件的设计。我们研究一个硬件系统执行某个特定 ISA 指令的方式。这种视角能让你更好地理解计算机的工作方式，以及计算机制造商所面临的技术挑战。一个重要的概念是：现代处理器实际的运转方式，可能与 ISA 所隐含的计算模型很不一样。ISA 模型看上去暗示的是顺序指令执行，即每条指令都先取出、执行完毕，然后才开始下一条指令。通过同时执行多条指令的不同部分，处理器可以取得比一次只执行一条指令更高的性能。处理器用一些特殊的机制来保证自己算出的结果与顺序执行时相同。用巧妙的技巧来提高性能、同时保持一个更简单更抽象的模型所具有的功能，这

种思想在计算机科学中很有名。例子包括 Web 浏览器中高速缓存的使用，以及平衡二叉树、哈希表这类信息检索数据结构。

你很可能永远不会去设计自己的处理器。这是只有全世界不到 100 家公司里的专家才从事的任务。那么，为什么还要学习处理器设计呢？

- 学习它在智力上有趣而且重要。了解事物如何运转本身就有一种内在的价值。弄清一个系统的内部机理尤其有意思：它已经与计算机科学家和工程师的日常生活如此密不可分，却对许多人来说依然是个谜。处理器设计浓缩了许多优秀的工程实践准则。它要求为执行一个复杂的任务而创造出简单而规则的结构。
- 理解处理器的工作方式，有助于理解整个计算机系统的工作方式。在第 6 章中，我们将考察存储器系统，以及用来营造出访问很快、容量很大的存储器假象的技术。看到处理器—存储器接口中处理器一侧的情形，会让这部分讲述更加完整。
- 虽然设计处理器的人很少，但设计包含处理器的硬件系统的人却很多。随着处理器被嵌入到汽车和家用电器这样的现实系统中，这种情况已经司空见惯。嵌入式系统设计者必须理解处理器如何工作，因为这类系统通常是在比桌面和服务器系统更低的抽象层次上设计和编程的。
- 你说不定哪天真的会去做处理器设计。生产微处理器的公司虽然不多，但负责这些处理器设计的工作团队规模已经很大，而且还在继续增长。一个大型的处理器设计，参与其各个方面工作的人可能超过 1,000 名。

边栏　计算机技术的进步

要感受过去 40 年间计算机技术进步到了何等地步，不妨考察下面这两款处理器。

第一台 Cray 1 超级计算机于 1976 年交付给洛斯阿拉莫斯国家实验室。它是当时世界上速度最快的计算机，每秒最多能进行 2.5 亿次算术运算。它配有 8 MB 的随机访问存储器，这是硬件所允许的最大配置。这台机器也非常庞大——重 5,000 kg，耗电 115 千瓦，售价 900 万美元。它总共只生产了大约 80 台。

苹果公司 2013 年为驱动 iPhone 5S 而推出的 ARM A7 微处理器芯片，包含两个 CPU，每个每秒能进行几十亿次算术运算，还配有 1 GB 的随机访问存储器。整部手机只重 112 克，功耗约 1 瓦，售价不到 800 美元。在它发布后的第一个周末就售出 900 多万部。它不仅是一台强大的计算机，还能用来拍照、打电话和提供驾驶导航——这些功能是 Cray 1 从来没有考虑过的。

这两个系统前后只相隔 37 年，展示了半导体技术的巨大进步。Cray 1 的 CPU 用大约 10 万枚半导体芯片构成，每枚芯片上的晶体管不到 20 个；而苹果 A7 的单枚芯片上集成了超过 10 亿个晶体管。Cray 1 的 8 MB 存储器需要 8,192 枚芯片，而 iPhone 的 1 GB 存储器就装在一枚芯片上。

在本章中，我们首先定义一个简单的指令集，用它作为处理器实现的运行示例。我们把它称为“Y86-64”指令集，因为它的灵感来自 x86-64 指令集。与 x86-64 相比，Y86-64 指令集的数据类型、指令和寻址方式都要少一些。它的字节级编码也很简单，这使得其机器码不如相应的 x86-64 代码紧凑，但 CPU 的译码逻辑也因此容易设计得多。Y86-64 指令集虽然非常简单，却足够完整，让我们得以编写处理整数数据的程序。设计一个实现 Y86-64 的处理器，需要面对处理器设计者所遇到的许多挑战。

然后我们介绍数字硬件设计的一些背景知识。我们描述处理器用到的基本构件，以及它们如何连接起来并如何操作。这部分内容建立在我们第 2 章讨论布尔代数和位级运算的基础之上。我们还引入一种简单的语言 HCL（hardware control language，硬件控制语言），用来描述硬件系统中的控制部分。此后我们将用这个语言来描述处理器设计。即使你已经有了一些逻辑设计方面的背景，也应当阅读这一节，以便了解我们所使用的特定记法。

设计处理器的第一步，是给出一个功能正确但眼下并不实用的 Y86-64 处理器，它基于顺序操作。这个处理器在每个时钟周期内完整地执行一条 Y86-64 指令。时钟必须运行得足够慢，才能让一整串动作在一个周期之内完成。这样的处理器是可以造出来的，但其性能会远低于这一规模的硬件本可以发挥的水平。

以顺序设计为基础，我们接着进行一系列变换，创建流水线化的处理器。这个处理器把每条指令的执行分成五个步骤，每个步骤由硬件的一个单独部分（称为流水级）来处理。指令在流水线的各级间不断前进，每个时钟周期都有一条指令进入流水线。这样一来，处理器就可以同时执行最多五条指令的不同步骤。要让这个处理器保持 Y86-64 ISA 的顺序行为，需要处理各种各样的冒险（hazard）情况，即一条指令的位置或操作数依赖于仍在流水线中的其他指令的那些情况。

我们设计了一整套工具，用来研究我们的处理器设计并对它们做实验。这些工具包括一个 Y86-64 的汇编器、一个能在你的机器上运行 Y86-64 程序的模拟器，以及针对两个顺序设计和一个流水线设计的模拟器。这些设计的控制逻辑用 HCL 记法的文件来描述。通过编辑这些文件并重新编译模拟器，你就可以修改和扩展模拟器的行为。我们提供了一些练习，涉及实现新指令以及改变机器处理指令的方式。还提供了测试代码来帮助你检验修改的正确性。这些练习会大大加深你对这部分内容的理解，并让你体会处理器设计者所面对的种种不同的设计选择。

网络边栏 arch:vlog 用 Verilog 硬件描述语言给出了我们流水线化 Y86-64 处理器的一种表示。这包括为基本硬件构件和整个处理器结构建立模块。我们

把控制逻辑的 HCL 描述自动翻译成 Verilog。先用我们的模拟器调试 HCL 描述,可以消除许多原本要到硬件设计中才会暴露出来的棘手缺陷。有了 Verilog 描述之后,就有商业的和开源的工具可以支持仿真和逻辑综合,生成微处理器的实际电路设计。所以,尽管我们在这里付出的许多努力是为了创建系统的图示和文字描述——就像人们编写软件时那样——但这些设计能够被自动综合这一点表明,我们实际创建的确实是一个能够用硬件实现的系统。

4.1 Y86-64 指令集体系结构

定义一个像 Y86-64 这样的 ISA,包括定义其状态的各个组成部分、指令的集合及其编码、一组程序设计约定,以及对异常事件的处理。

4.2 程序员可见的状态

如图 4.1 所示,Y86-64 程序中的每条指令都可以读取和修改处理器状态的某些部分。这些部分称为程序员可见的状态(programmer-visible state),这里的“程序员”可以是直接用汇编代码编写程序的人,也可以是生成机器级代码的编译器。在讲述处理器实现时我们将看到,只要能保证机器级程序看起来可以访问程序员可见的状态,我们就不必严格按照 ISA 所隐含的方式来表示和组织这套状态。

> Figure 4.1
> Y86-64 programmer-
> visible state. As with
> x86-64, programs for Y86-
> 64 access and modify
> the program registers,
> the condition codes, the
> program counter (PC), and
> the memory. The status
> code indicates whether
> the program is running
> normally or some special
> event has occurred.
> RF: Program registers
> Stat: Program status
> DMEM: Memory
> CC:
> Condition
> codes
> %rax
> %rcx
> %rdx
> %rbx
> %rsp
> %rbp

```
%rsi
%rdi
%r8
%r9
%r10
%r11
%r12
%r13
%r14
PC
ZF SF OF
```

Y86-64 的状态与 x86-64 类似。程序寄存器共有 15 个:%rax、%rcx、%rdx、%rbx、%rsp、%rbp、%rsi、%rdi,以及%r8 到%r14。(为了简化指令编码,我们省略了 x86-64 的寄存器%r15。)每个寄存器存放一个 64 位的字。寄存器%rsp 被 push、pop、call 和 return 指令用作栈指针。除此之外,这些寄存器没有固定的含义或值。还有三个单 bit 的条件码 ZF、SF 和 OF,存放着

最近执行的算术或逻辑指令的效果信息。程序计数器(PC)存放当前正在执行的指令的地址。

存储器在概念上是一个很大的字节数组,既存放程序也存放数据。Y86-64 程序用虚拟地址来引用存储器位置。硬件与操作系统软件相结合,把虚拟地址翻译成实际的、物理的地址,指明各个值在存储器中实际存放的位置。我们将在第 9 章更详细地研究虚拟内存。现在不妨这样看:虚拟内存系统为 Y86-64 程序提供了一个看起来是整块的字节数组的存储器假象。

程序状态的最后一个部分是状态码 Stat,它表示程序执行的总体状态。它要么表示正常运行,要么表示发生了某种类型的异常,例如某条指令试图读取一个无效的存储器地址。可能的状态码和异常的处理方法见 4.1.4 节。

4.3 Y86-64 指令

图 4.2 对 Y86-64 ISA 中的各条指令作了扼要的描述。我们用这个指令集作为处理器实现的目标。Y86-64 指令集在很大程度上是 x86-64 指令集的一个子集。它只包含 8 字节的整数运算,寻址方式较少,操作也较少。由于我们只用 8 字节的数据,因此可以毫无歧义地把它们称作“字”。图 4.2 中,左边给出指令的汇编代码表示,右边给出字节编码。图 4.3 进一步给出了其中一些指令的细节。汇编代码的格式与 x86-64 的 ATT 格式类似。

下面是关于 Y86-64 指令的一些细节:

- x86-64 的 movq 指令被拆分成四条不同的指令:irmovq、rrmovq、mrmovq 和 rmmovq,明确地表示出源和目的的形式。源可以是立即数(i)、寄存器(r)或存储器(m),由指令名的第一个字符指定。目的可以是寄存器(r)或

图 4.2 Y86-64 指令集。指令的编码长度在 1 到 10 字节之间。一条指令由一个 1 字节的指令说明符组成，可能再配一个 1 字节的寄存器说明符，还可能再配一个 8 字节的常数字。域 fn 指明某个特定的整数运算(OPq)、数据传送条件(cmovXX)或分支条件(jXX)。所有数值均以十六进制显示。

指令	字节								
	0	1	2	3	4	5	6	7	8	9
halt	0 0									
nop	1 0									
rrmovq rA, rB	2 0	rA rB								
irmovq V, rB	3 0	F rB				V				
rmmovq rA, D(rB)	4 0	rA rB				D				
mrmovq D(rB), rA	5 0	rA rB				D				
OPq rA, rB	6 fn	rA rB								
jXX Dest	7 fn			Dest						
cmovXX rA, rB	2 fn	rA rB								
call Dest	8 0			Dest						
ret	9 0									
pushq rA	A 0	rA F								
popq rA	B 0	rA F								

存储器(m)，由指令名的第二个字符指定。明确地区分出这四种数据传送类型，对我们决定如何实现它们会有帮助。

- 两条存储器传送指令的存储器引用采用简单的基址加位移量格式。在地址计算中，我们既不支持第二个变址寄存器，也不支持对寄存器的值进行任何缩放。
- 与 x86-64 一样，我们不允许从一个存储器位置直接传送到另一个存储器位置。此外，我们也不允许把立即数传送到存储器。

- 有四条整数运算指令，在图 4.2 中显示为 OPq。它们是 addq、subq、andq 和 xorq。它们只对寄存器数据进行操作，而 x86-64 还允许对存储器数据操作。这些指令设置三个条件码 ZF、SF 和 OF(零、符号和溢出)。

- 有七条跳转指令(在图 4.2 中显示为 jXX):jmp、jle、jl、je、jne、jge 和 jg。是否发生跳转,取决于分支类型和条件码的设置。分支条件与 x86-64 中的相同(图 3.15)。
- 有六条条件传送指令(在图 4.2 中显示为 cmovXX):cmovle、cmovl、cmove、cmovne、cmovge 和 cmovg。它们的格式与寄存器—寄存器传送指令 rrmovq 相同,但只有当条件码满足要求的约束时才会更新目的寄存器。
- call 指令把返回地址压入栈,然后跳转到目的地址。ret 指令从这样的调用中返回。
- pushq 和 popq 指令实现入栈和出栈操作,正如它们在 x86-64 中所做的那样。
- halt 指令停止指令的执行。x86-64 中有一条与之相当的指令,叫作 hlt。由于该指令会使整个系统停止运行,x86-64 应用程序不允许使用它。对 Y86-64 来说,执行 halt 指令会使处理器停止,同时状态码被置为 HLT。(见 4.1.4 节。)

4.4 指令编码

图 4.2 也给出了指令的字节级编码。根据需要哪些域,一条指令占 1 到 10 个字节。每条指令的第一个字节标明指令的类型。这个字节分成两个 4 位的部分:高阶的码(code)部分和低阶的函数(function)部分。如图 4.2 所示,码的取值范围是 0 到 0xB。只有在一组相关指令共用一个码的情况下,函数值才有意义。这些情形见图 4.3,图中给出了整数运算、分支和条件传送指令的具体编码。可以看到,rrmovq 与各条条件传送指令共用同一个指令码。它可以看作“无条件传送”,正如 jmp 是无条件跳转一样,两者的函数码都是 0。

如图 4.4 所示,15 个程序寄存器中的每一个都有一个相关联的寄存器标识符(ID),取值从 0 到 0xE。Y86-64 中寄存器的编号与 x86-64 中所用的一致。程序寄存器存放在 CPU 的一个寄存器堆中,这是一个小的随机访问存储器,寄存器 ID 就充当它的地址。当需要在指令编码以及我们的硬件设计中表示不访问任何寄存器时,就使用 ID 值 0xF。

有些指令只有 1 字节长,而需要操作数的指令编码更长。首先可以有一个附加的寄存器说明符字节,指明一个或两个寄存器。这些寄存器域在图 4.2 中称为 rA 和 rB。正如各条指令的汇编代码形式所示,取决于指令类型,它们可以指明数据源和目的使用的寄存器,也可以指明地址计算中使用的基址寄存器。没有寄存器操作数的指令(例如分支指令和 call)没有寄存器说明符字节。只

图 4.3 Y86-64 指令集的功能码。码指明某个特定的整数运算、分支条件或数据传送条件。这些指令在图 4.2 中显示为 OPq、jXX 和 cmovXX。

运算			分支			条件传送		
addq	6	0	jmp	7	0	jne	7	4
subq	6	1	jle	7	1	jge	7	5
andq	6	2	jl	7	2	jg	7	6
xorq	6	3	je	7	3	cmove	2	3

图 4.4 Y86-64 程序寄存器标识符。15 个程序寄存器中的每一个都有一个相关联的标识符(ID),取值从 0 到 0xE。指令的寄存器域中出现 ID 0xF,表示没有寄存器操作数。

编号	寄存器名	编号	寄存器名
0	%rax	8	%r8
1	%rcx	9	%r9
2	%rdx	A	%r10
3	%rbx	B	%r11
4	%rsp	C	%r12
5	%rbp	D	%r13
6	%rsi	E	%r14
7	%rdi	F	无寄存器

需要一个寄存器操作数的指令(irmovq、pushq 和 popq)就把另一个寄存器说明符置为 0xF。这一约定在我们的处理器实现中将很有用。

有些指令还需要一个附加的 8 字节常数字。这个字可以充当 irmovq 的立即数,rmmovq 和 mrmovq 地址说明符中的位移量,以及分支指令和 call 的目的地。注意,分支指令和 call 的目的地址用绝对地址给出,而不是像 x86-64 中那样使用 PC 相对寻址。处理器使用 PC 相对寻址,是为了让分支指令的编码更紧凑,并使代码能够在存储器中从一个位置移到另一个位置而不必更新所有分支目标地址。因为我们的讲述更看重简单性,所以采用绝对寻址。与 x86-64 一样,所有整数都采用小端法编码。以反汇编形式书写指令时,这些字节按相反的顺序出现。

举例来说，我们来生成指令 `rmmovq %rsp,0x123456789abcd(%rdx)` 的十六进制字节编码。从图 4.2 可以看到，`rmmovq` 的首字节是 40。还可以看到，源寄存器`%rsp` 应编码在 `rA` 域，而基址寄存器`%rdx` 应编码在 `rB` 域。按照图 4.4 中的寄存器编号，得到寄存器说明符字节为 42。最后，位移量编码在 8 字节常数字中。先把 0x123456789abcd 用前导零补足 8 个字节，得到字节序列 00 01 23 45 67 89 ab cd；写成字节反序便是 cd ab 89 67 45 23 01 00。合在一起，得到指令编码 4042cdab896745230100。

任何指令集的一个重要性质是其字节编码必须有唯一的解释。任意的字节序列要么编码了一个唯一的指令序列，要么就是非法的字节序列。Y86-64 满足这个性质，因为每条指令首字节中的码与函数的组合都是唯一的，而且给定首字节后，我们就能确定任何后续字节的长度与含义。这个性质保证了处理器执行目标代码程序时，对代码的理解不会产生歧义。即使代码嵌在程序的其他字节之间，只要从序列的第一个字节开始，我们也能顺利地确定整条指令序列。反过来，如果不知道代码序列的起始位置，我们就无法可靠地判断应当如何把序列切分成一条条指令。这给反汇编器以及那些试图直接从目标代码字节序列中提取机器级程序的工具带来了麻烦。

边栏　比较 x86-64 与 Y86-64 指令编码

与 x86-64 使用的指令编码相比，Y86-64 的编码要简单得多，但也不那么紧凑。寄存器域在所有的 Y86-64 指令中都只出现在固定的位置；而在 x86-64 的各条指令中，它们被挤进各种不同的位置。x86-64 指令可以用 1、2、4 或 8 字节来编码常数值，而 Y86-64 总是要求 8 字节。

边栏　RISC 与 CISC 指令集

x86-64 有时被贴上“复杂指令集计算机”（complex instruction set computer，CISC——“sisk”）的标签，与被归为“精简指令集计算机”（reduced instruction set computer，RISC——“risk”）的 ISA 相对。从历史上看，CISC 机器出现得更早，是从最早的计算机逐步演化而来的。到 20 世纪 80 年代初，大型机和小型机的指令集已经相当庞大，因为机器设计者不断加入新的指令来支持高级任务，例如操纵循环缓冲区、执行十进制算术、求多项式的值。最早的微处理器出现在 20 世纪 70 年代初期，当时的集成电路技术对单芯片上能实现的东西构成了严格的限制，所以指令集有限。微处理器演化得很快，到 80 年代初，它们沿着与大型机、小型机相同的路径，走上不断加大指令集复杂度之路。x86 家族走的正是这条路：先演化成 IA32，最近又演化成 x86-64。随着新应用的需求催生出新类别的指令，x86 这条产品线的演化至今仍在继续。

RISC 设计思想是 20 世纪 80 年代初作为对上述趋势的一种替代而发展起来的。IBM 的一批硬件和编译器专家深受 IBM 研究员 John Cocke 思想的影响，认识到他们可以为一种简单得多的指令集形式生成高效的代码。事实上，当时正在加入各指令集的许多高层指令，用编译器很难生成，而且很少被用到。一种更简单的指令集可以

> 用少得多的硬件来实现，还能组织成高效的流水线结构，就像本章稍后描述的那样。直到多年以后，IBM 才把这个思想商品化，开发出了 Power 和 PowerPC ISA。
> RISC 概念后来又由加州大学伯克利分校的 David Patterson 教授和斯坦福大学的 John Hennessy 教授进一步发展。Patterson 给这个新类别的机器起名 RISC，而把已有的那类机器称为 CISC，因为在此之前，对如此大行其道的指令集形式并没有设过专门的名称。
> 比较 CISC 与原始的 RISC 指令集，可以总结出以下一般特征：

CISC	早期 RISC
指令数量庞大。描述 Intel 完整指令集的文档 [51] 长达 1,200 多页。	指令少得多——通常不到 100 条。
有些指令执行时间很长。其中包括把一整块数据从存储器一处复制到另一处的指令，以及在存储器与寄存器之间成批搬运数据的指令。	没有执行时间长的指令。有些早期 RISC 机器甚至没有整数乘法指令，需要编译器用一连串加法来实现乘法。
变长编码。x86-64 指令的长度可以从 1 到 15 字节。	定长编码。通常所有指令都编码为 4 字节。
多种操作数指定格式。在 x86-64 中，一个存储器操作数说明符可以由位移量、基址和变址寄存器以及比例因子组合成许多不同的形式。	简单的寻址格式。通常只有基址加位移量寻址。
算术和逻辑运算既可以作用于存储器操作数，也可以作用于寄存器操作数。	算术和逻辑运算只使用寄存器操作数。对存储器的引用只允许通过 load 指令（把存储器内容读入寄存器）和 store 指令（把寄存器内容写入存储器）进行。这一约定称为 load/store 体系结构。
实现细节对机器级程序隐藏。ISA 在程序与指令的实际执行方式之间提供了一层干净的抽象。	实现细节向机器级程序暴露。有些 RISC 机器禁止特定的指令序列，还有的跳转指令要等下一条指令执行之后才生效。编译器的任务是在这些约束之内优化性能。
条件码。指令作为副作用设置一些特殊的标志，然后用于条件分支测试。	没有条件码。代之以显式的测试指令，把测试结果存入普通寄存器，供条件判定使用。
以栈为中心的过程链接。栈用来保存过程参数和返回地址。	以寄存器为中心的过程链接。寄存器用来保存过程参数和返回地址。这样，有些过程可以完全避免访存。通常处理器拥有多得多的（多达 32 个）寄存器。

Y86-64 指令集兼有 CISC 和 RISC 指令集的特征。属于 CISC 一面的，是它有条件码和变长的指令，并且用栈来保存返回地址。属于 RISC 一面的，是它采用 load/store 体系结构和规则的指令编码，并且通过寄存器传递过程参数。可以把它看作是取一个 CISC 指令集（x86）然后运用一些 RISC 原则加以简化的产物。

边栏 RISC 与 CISC 之争

整个 20 世纪 80 年代，计算机体系结构界围绕 RISC 与 CISC 指令集孰优孰劣展开了激烈的争论。RISC 的支持者声称，把精简的指令集设计、先进的编译器技术和流水线化的处理器实现结合起来，就能用同样数量的硬件获得更强的计算能力。CISC 的支持者则反驳说，完成一个给定任务所需的 CISC 指令更少，因此他们的机器可以取得更高的总体性能。

几家大公司先后推出了各自的 RISC 处理器产品线，包括 Sun Microsystems（SPARC）、IBM 与摩托罗拉（PowerPC）以及数字设备公司（Alpha）。英国的一家公司 Acorn Computers Ltd. 则开发出了自己的体系结构 ARM（最初是“Acorn RISC machine”的缩写），它在手机等嵌入式应用中获得了广泛使用。

到了 20 世纪 90 年代初，这场争论逐渐平息，因为人们已经清楚：最纯粹的 RISC 和最纯粹的 CISC，都不如兼取两者优长的设计。RISC 机器不断演化，引入了越来越多的指令，其中许多要用多个周期才能执行完。今天的 RISC 机器的指令库里有数百条指令，几乎配不上“精简指令集计算机”这个名字了。把实现细节暴露给机器级程序的想法事后看来是短视的。随着新的处理器型号以更先进的硬件结构开发出来，这些细节中的许多已经无关紧要，但它们仍然保留在指令集里。不过，RISC 设计的核心——适合在流水线机器上执行的指令集——经受住了时间的检验。

更晚近的 CISC 机器同样利用了高性能的流水线结构。正如 5.7 节将要讨论的，它们取出 CISC 指令，再动态地把它们翻译成更简单的、类 RISC 的操作序列。例如，一条把寄存器加进存储器的指令，会被翻译成三个操作：一个读出原来的存储器值，一个执行加法，第三个再把和写回存储器。由于动态翻译一般都能在实际执行指令之前从容完成，处理器可以维持很高的执行速率。

除了技术因素之外，营销因素对不同指令集的成败也起了很大作用。Intel 靠着与其现有处理器保持兼容，使得 x86 用户可以轻松地一代接一代地升级处理器。随着集成电路技术的进步，Intel 和其他 x86 处理器制造商克服了最初 8086 指令集设计所造成的低效之处，用 RISC 技术做出了与最好的 RISC 机器相当的处理器。正如 3.1 节所述，把 IA32 演进成 x86-64 提供了一个机会，把 RISC 的若干特性并入了 x86 家族。在台式机、笔记本和服务器计算领域，x86 已经接近完全统治地位。

RISC 处理器则在嵌入式处理器市场上大获成功，控制着蜂窝电话、汽车制动器和因特网家电这类系统。在这些应用中，节省成本和功耗比保持向后兼容更重要。就处理器售出的数量而言，这是一个非常大而且还在增长的市场。

4.5 Y86-64 异常

Y86-64 的程序员可见状态（图 4.1）包含一个状态码 Stat，描述正在执行的程序的总体状态。这个码可能的取值见图 4.5。

取值为 1 的码叫作 AOK，表示程序执行正常，而其他码表示出现了某种类型的异常。取值为 2 的码叫作 HLT，表示处理器执行了一条 `halt` 指令。取值为 3 的码叫作 ADR，表示处理器试图对无效存储器地址进行读或写——可能是在取指令时，也可能是在读写数据时。我们限制最大地址（确切的限制随实

图 4.5 Y86-64 状态码。在我们的设计中，对 AOK 以外的任何状态码，处理器都会停止。

取值	名称	含义
1	AOK	正常运行
2	HLT	遇到 halt 指令
3	ADR	遇到无效地址
4	INS	遇到无效指令

现而异），对超出这个限制的地址的任何访问都会触发 ADR 异常。取值为 4 的码叫作 INS，表示遇到了无效的指令码。

对 Y86-64 来说，只要遇到上述任何一种异常，我们就让处理器停止执行指令。在更完整的设计中，处理器通常会调用一个异常处理程序（exception handler），即被指定来处理所遇到的那类特定异常的过程。正如第 8 章将要描述的，异常处理程序可以配置成具有不同的效果，例如终止程序或者调用用户定义的信号处理程序。

4.6 Y86-64 程序

图 4.6 为下面的 C 函数给出了 x86-64 和 Y86-64 两种汇编代码：

```
1  long sum(long *start, long count)
2  {
3      long sum = 0;
4      while (count) {
5          sum += *start;
6          start++;
7          count--;
8      }
9      return sum;
10 }
```

x86-64 代码由 gcc 编译器生成。Y86-64 代码与之相似，但有下面几点差别：

- Y86-64 代码要把常数装入寄存器（第 2–3 行），因为它不能在算术指令中使用立即数。

```
x86-64 代码
  long sum(long *start, long count)
  start 在 %rdi 中,count 在 %rsi 中
1  sum:
2    movl   $0, %eax          sum = 0
3    jmp    .L2               转向 test
4  .L3:                      loop:
5    addq   (%rdi), %rax      把 *start 加到 sum
6    addq   $8, %rdi          start++
7    subq   $1, %rsi          count--
8  .L2:                      test:
```

```
 9    testq  %rsi, %rsi          测试 sum
10    jne    .L3                若不为 0,转向 loop
11    rep; ret                  返回

Y86-64 代码
  long sum(long *start, long count)
  start 在 %rdi 中,count 在 %rsi 中
 1  sum:
 2    irmovq $8,%r8             常数 8
 3    irmovq $1,%r9             常数 1
 4    xorq %rax,%rax            sum = 0
 5    andq %rsi,%rsi            设置 CC
 6    jmp    test               转向 test
 7  loop:
 8    mrmovq (%rdi),%r10        取 *start
 9    addq %r10,%rax            加到 sum
10    addq %r8,%rdi             start++
11    subq %r9,%rsi             count--.  设置 CC
12  test:
13    jne    loop               为 0 时停止
14    ret                       返回
```

图 4.6 Y86-64 汇编程序与 x86-64 汇编程序的对比。sum 函数计算一个整型数组各元素的和。Y86-64 代码与 x86-64 代码遵循同样的总体模式。

- Y86-64 代码需要两条指令(第 8–9 行)才能从存储器读出一个值并把它加到寄存器上,而 x86-64 代码用一条单独的 `addq` 指令(第 5 行)就能做到。
- 我们手工编写的 Y86-64 实现利用了这样一个性质:`subq` 指令(第 11 行)也会设置条件码,因此 gcc 生成的代码中的 `testq` 指令(第 9 行)就不再需要了。不过,要使这一点成立,Y86-64 代码必须在进入循环之前先用一条 `andq` 指令(第 5 行)设置条件码。

图 4.7 是一个用 Y86-64 汇编代码书写的完整程序文件的示例。这个程序既有数据也有指令。指示告诉汇编器把代码或数据放在哪里以及如何对齐。程序规定了栈位置的安排、数据初始化、程序初始化和程序终止这样的事宜。

在这个程序中,以“.”开头的词是汇编指示,告诉汇编器调整它生成代码所用地址,或者插入一些数据字。指示`.pos 0`(第 2 行)告诉汇编器从地址 0 开始生成代码。这是所有 Y86-64 程序的起始地址。下一条指令(第 3 行)对栈指针进行初始化。可以看到,标号 `stack` 是在程序末尾(第 40 行)声明的,用一个`.pos` 指示(第 39 行)表明地址 0x200。因此,我们的栈从这个地址开始,向低地址方向增长。我们必须保证栈不会涨得太厉害,以至于覆写了代码或其他程序数据。

程序的第 8–13 行声明了一个由 4 个字组成的数组,其值为

```
0x000d000d000d000d, 0x00c000c000c000c0,
0x0b000b000b000b00, 0xa000a000a000a000
```

标号 array 表示这个数组的开头，并且对齐到 8 字节边界（使用.align 指示）。第 16–19 行是一个“main”过程，它对由 4 个字组成的数组调用函数 sum，然后停机。

正如这个例子所表明的，既然我们创建 Y86-64 代码的唯一工具是汇编器，程序员就必须亲自完成一些我们平时交给编译器、链接器和运行时系统的任务。好在只有小程序才需要这样做，对这些程序来说，简单的机制就足够了。

图 4.8 给出了用我们称为 yas 的汇编器汇编图 4.7 所示代码的结果。汇编器的输出采用 ASCII 格式以便于阅读。在汇编文件中包含指令或数据的行上，目标代码中有一个地址，后面跟着 1 到 10 个字节的值。

```
 1  # 从地址 0 开始执行
 2      .pos 0
 3      irmovq stack, %rsp       # 设置栈指针
 4      call main                # 执行主程序
 5      halt                     # 终止程序
 6
 7  # 4 个元素的数组
 8      .align 8
 9  array:
10      .quad 0x000d000d000d
11      .quad 0x00c000c000c0
12      .quad 0x0b000b000b00
13      .quad 0xa000a000a000
14
15  main:
16      irmovq array,%rdi
17      irmovq $4,%rsi
18      call sum                 # sum(array, 4)
19      ret
20
21  # long sum(long *start, long count)
22  # start 在 %rdi 中,count 在 %rsi 中
23  sum:
24      irmovq $8,%r8            # 常数 8
25      irmovq $1,%r9            # 常数 1
26      xorq %rax,%rax           # sum = 0
27      andq %rsi,%rsi           # 设置 CC
28      jmp     test             # 转向 test
29  loop:
30      mrmovq (%rdi),%r10       # 取 *start
31      addq %r10,%rax           # 加到 sum
32      addq %r8,%rdi            # start++
33      subq %r9,%rsi            # count--.  设置 CC
34  test:
35      jne    loop              # 为 0 时停止
36      ret                      # 返回
37
38  # 栈从这里开始,向低地址方向增长
39      .pos 0x200
40  stack:
```

图 4.7 用 Y86-64 汇编代码写成的示例程序。列出对 sum 函数的调用用它来计算一个四元素数组的和。

```
                            |  # 从地址 0 开始执行
0x000:                      |    .pos 0
0x000: 30f40002000000000000 |    irmovq stack, %rsp       # 设置栈指针
0x00a: 8038000000000000     |    call main                # 执行主程序
0x013: 00                   |    halt                     # 终止程序
                            |
                            |  # 4 个元素的数组
0x018:                      |    .align 8
0x018:                      |  array:
0x018: 0d000d000d000000     |    .quad 0x000d000d000d
0x020: c000c000c0000000     |    .quad 0x00c000c000c0
0x028: 000b000b000b0000     |    .quad 0x0b000b000b00
0x030: 00a000a000a00000     |    .quad 0xa000a000a000
                            |
0x038:                      |  main:
0x038: 30f71800000000000000 |    irmovq array,%rdi
0x042: 30f60400000000000000 |    irmovq $4,%rsi
0x04c: 8056000000000000     |    call sum                 #
 ↪  sum(array,
      4)
0x055: 90                   |    ret
                            |
                            |  # long sum(long *start, long count)
                            |  # start 在 %rdi 中,count 在 %rsi 中
0x056:                      |  sum:
0x056: 30f80800000000000000 |    irmovq $8,%r8           # 常数 8
0x060: 30f90100000000000000 |    irmovq $1,%r9           # 常数 1
0x06a: 6300                 |    xorq %rax,%rax          # sum = 0
0x06c: 6266                 |    andq %rsi,%rsi          # 设置 CC
0x06e: 7087000000000000     |    jmp    test             # 转向 test
0x077:                      |  loop:
0x077: 50a70000000000000000 |    mrmovq (%rdi),%r10      # 取 *start
0x081: 60a0                 |    addq %r10,%rax          # 加到 sum
0x083: 6087                 |    addq %r8,%rdi           # start++
0x085: 6196                 |    subq %r9,%rsi           # count--.
        设置 CC
0x087:                      |  test:
0x087: 7477000000000000     |    jne   loop              # 为 0 时停止
0x090: 90                   |    ret                     # 返回
                            |
                            |  # 栈从这里开始,向低地址方向增长
0x200:                      |    .pos 0x200
0x200:                      |  stack:
```

图 4.8 yas 汇编器的输出。每一行包括一个十六进制地址和 1 到 10 个字节的目标代码。

我们实现了一个名为 yis 的指令集模拟器。它的目的是模拟一个 Y86-64 机器码程序的执行,而不试图对任何具体处理器实现的行为建模。这种形式的模拟,在实际硬件尚未就绪时可以用来调试程序,也可用于检查对硬件模拟的

结果，或者由硬件本身运行该程序的结果。对我们的示例目标代码运行 yis，产生如下输出：

```
Stopped in 34 steps at PC = 0x13. Status 'HLT', CC Z=1 S=0 O=0
Changes to registers:
%rax:   0x0000000000000000      0x0000abcdabcdabcd
%rsp:   0x0000000000000000      0x0000000000000200
%rdi:   0x0000000000000000      0x0000000000000038
%r8:    0x0000000000000000      0x0000000000000008
%r9:    0x0000000000000000      0x0000000000000001
%r10:   0x0000000000000000      0x0000a000a000a000

Changes to memory:
0x01f0: 0x0000000000000000      0x0000000000000055
0x01f8: 0x0000000000000000      0x0000000000000013
```

模拟输出的第一行概括了执行情况以及 PC 和程序状态最后得到的值。在打印寄存器和存储器的值时，它只打印在模拟期间发生改变的值，或者在寄存器、或者在存储器中。原来的值（在这个程序里都是零）显示在左边，最终的值显示在右边。从这份输出中可以看到，寄存器`%rax` 中放着 0xabcdabcdabcdabcd，即传给过程 `sum` 的那个 4 元素数组所有元素的和。此外还可以看到，从地址 0x200 开始向低地址方向增长的栈已经被用过，地址 0x1f0–0x1f8 处的存储器字因此发生了变化。可执行代码的最大地址是 0x090，所以在栈上压入和弹出值并没有破坏可执行代码。

练习 4.1 确定下面这条 Y86-64 指令序列的字节编码。行`.pos 0x100` 表示目标代码的起始地址应当是 `0x100`。

```
.pos 0x100    # 从地址 0x100 开始放置代码
    irmovq $15,%rbx
    rrmovq %rbx,%rcx
loop:
    rmmovq %rcx,-3(%rbx)
    addq   %rbx,%rcx
    jmp loop
```

练习 4.2 对下面列出的每个字节序列，确定它编码的 Y86-64 指令序列。如果序列中有无效字节，给出到该处为止的指令序列，并指出无效值出现的位置。对每个序列的书写方式是，先给出起始地址，接着是冒号，然后是字节序列。

```
A. 0x100: 30f3fcffffffffffffff4063000800000000000000
B. 0x200: a06f800c020000000000000030f30a00000000000000
C. 0x300: 505407000000000000000010f0b01f
D. 0x400: 611373000400000000000000
E. 0x500: 6362a0f0
```

练习 4.3 在机器级程序中，给一条寄存器加一个常数值是一种常见模式。用到目前为止介绍的 Y86-64 指令，这需要先用一条 `irmovq` 指令把常数装进

一个寄存器，再用一条 addq 指令把这个值加到目的寄存器上。假设我们想要增加一条具有如下格式的新指令 iaddq：

指令	字节									
	0	1	2	3	4	5	6	7	8	9
iaddq V, rB	C 0	F rB	V							

这条指令把常数值 V 加到寄存器 rB 上。改写图 4.6 中的 Y86-64 sum 函数，使之使用 iaddq 指令。在原来的版本中，我们让寄存器%r8 和%r9 专用来保存常数值。现在可以完全不用这两个寄存器了。

练习 4.4 请根据下面的 C 代码，编写 Y86-64 代码来实现递归的乘积函数 rproduct。

```
long rproduct(long *start, long count)
{
    if (count <= 1)
        return 1;
    return *start * rproduct(start+1, count-1);
}
```

参数传递和寄存器保存的约定与 x86-64 代码相同。你会发现，先在 x86-64 机器上编译这段 C 代码，再把得到的指令翻译成 Y86-64，是很有帮助的做法。

练习 4.5 修改 sum 函数（图 4.6）的 Y86-64 代码，实现一个计算数组元素绝对值之和的函数 absSum。在内层循环中使用一条条件跳转指令。

练习 4.6 修改 sum 函数（图 4.6）的 Y86-64 代码，实现一个计算数组元素绝对值之和的函数 absSum。在内层循环中使用一条条件传送指令。

4.6.1 Y86-64 指令的一些细节

大多数 Y86-64 指令都以直截了当的方式改变程序状态，因此要定义每条指令预期的作用并不困难。不过，有两种不常见的指令组合需要特别注意。

pushq 指令既要使栈指针减 8，又要往存储器里写一个寄存器的值。因此在执行指令 pushq %rsp 时，处理器应该做什么并不完全明确，因为被压栈的寄存器同时又被这条指令本身修改。有两种可能的约定：(1) 压入%rsp 的原始值；(2) 压入减量后的%rsp 的值。

对 Y86-64 处理器，我们采用与 x86-64 相同的约定，具体结论由下面的练习确定。

练习 4.7 我们来判定 x86-64 处理器执行指令 `pushq %rsp` 时的行为。可以去读英特尔关于这条指令的文档,但更简单的办法是直接在一台真实的机器上做实验。C 编译器一般不会生成这条指令,所以我们必须为这个任务手工生成汇编代码。下面是我们写的一个测试函数(网络边栏 `asm:easm` 描述了如何编写把 C 代码和手写汇编代码结合起来的程序):

```
1      .text
2    .globl pushtest
3    pushtest:
4      movq    %rsp, %rax        拷贝栈指针
5      pushq   %rsp              压入栈指针
6      popq    %rdx              把它弹回来
7      subq    %rdx, %rax        返回 0 或 4
8      ret
```

在这些实验中,我们发现函数 `pushtest` 总是返回 0。这对 x86-64 下指令 `pushq %rsp` 的行为意味着什么?

`popq %rsp` 指令也有类似的歧义:它可以把%rsp 设置为从存储器读出的值,也可以设置为加 1 之后的栈指针值。与练习 4.7 一样,我们做个实验来判定 x86-64 机器会怎样处理这条指令,然后让 Y86-64 机器遵循同样的约定。

练习 4.8 下面的汇编代码函数可以帮助我们判定 x86-64 下指令 `popq %rsp` 的行为:

```
1      .text
2    .globl poptest
3    poptest:
4      movq    %rsp, %rdi        保存栈指针
5      pushq   $0xabcd           压入测试值
6      popq    %rsp              弹出到栈指针
7      movq    %rsp, %rax        把弹出的值设为返回值
8      movq    %rdi, %rsp        恢复栈指针
9      ret
```

我们发现这个函数总是返回 0xabcd。这对 `popq %rsp` 的行为意味着什么?还有哪一条 Y86-64 指令的行为与它完全相同?

> **边栏 把细节做对:不同 x86 模型之间的不一致性**
> 练习题 4.7 和 4.8 的目的,是帮助我们为压入或弹出栈指针的指令确定一套前后一致的约定。似乎没有什么理由要去执行这些操作中的任何一个,于是很自然地会问:“为什么要关心这些吹毛求疵的细节?”
> 从下面摘自英特尔 push 指令文档 [51] 的说明文字中,我们能够得到几点关于一致性的启示:
>
> > 对于从 Intel 286 开始的 IA-32 处理器,`PUSH ESP` 指令压入的是指令执行之前 `ESP` 寄存器的值。(Intel 64 体系结构以及 IA-32 体系结构

> 的实地址模式和虚拟 8086 模式也是如此。）对于 Intel 8086 处理器，PUSH SP 指令压入的是 SP 寄存器的新值，也就是它减 2 之后的值。（PUSH 指令条目，Intel 公司，条目 50。）
>
> 尽管这条注记的细节可能不易看懂，但它告诉了我们一点：当被要求压入栈指针寄存器时，x86 处理器会因所操作的模式不同而作出不同的反应。有些模式压入原始值，另一些压入减量后的值。（有意思的是，向栈指针寄存器弹出并没有对应的歧义。）这种不一致性有两个缺点：
>
> - 它降低了代码的可移植性。程序的行为取决于处理器所处的模式。虽然这条指令本身并不常用，但哪怕只是存在不兼容的可能性，就会造成严重的后果。
> - 它使文档变得复杂。正如这里看到的，需要一条专门的注记来澄清这些差别。x86 的文档本来已经相当复杂，不需要再添这样的特例。
>
> 由此我们得出的结论是：把细节提前想清楚并力求完全一致，从长远看可以省去许多麻烦。

4.7 逻辑设计和硬件控制语言 HCL

在硬件设计中，人们用电子电路来对比特进行运算，并把比特存储到各种不同的存储单元里。当前大多数电路技术用信号导线上的高电平或低电平来区分不同的比特值。在现代工艺中，逻辑值 1 用 1.0 伏左右的高电压表示，逻辑值 0 用 0.0 伏左右的低电压表示。实现一个数字系统需要三类主要部件：对各个比特进行运算的组合逻辑（combinational logic）、存储比特的存储单元（memory element），以及用来控制存储单元何时更新的时钟信号。

本节简要介绍这些不同的部件，同时介绍 HCL（指 hardware control language，硬件控制语言）——我们用来描述各种处理器设计的控制逻辑的语言。这里只是非正式地描述 HCL，完整的参考资料见网络边栏 `arch:hcl`。

> **边栏　现代逻辑设计**
>
> 曾几何时，硬件设计人员通过绘制逻辑电路的原理图来创建电路设计（最初用纸和铅笔，后来用计算机图像终端）。如今，大多数设计都用硬件描述语言（hardware description language，HDL）来表达，这是一种文本记法，看起来类似程序设计语言，但描述的是硬件结构而不是程序行为。常用的语言有两条：一条是语法类似 C 的 Verilog，另一条是语法类似 Ada 语言的 VHDL。这些语言最初是为数字电路建立仿真模型而设计的。20 世纪 80 年代中期，研究人员开发出了能够从 HDL 描述生成高效电路的逻辑综合程序。现在已经有多种商用的综合程序，综合软件已成为生成数字电路的主要手段。这种从手工设计电路到用综合程序生成电路的转变，可以类比于从用汇编代码编写程序到用高级语言编写程序、由编译器产生机器码的转变。
>
> 我们的 HCL 语言只描述硬件设计中的控制部分，它只有有限的操作集合，也不支持模块化。不过可以看到，控制逻辑正是设计微处理器时最困难的部分。我们开发了

一些工具，能直接把 HCL 翻译成 Verilog，把生成的代码与基本硬件单元的 Verilog 代码组合起来，就能得到可以综合出实际能正常工作的微处理器的 HDL 描述。只要把控制逻辑分离出来，细心地设计和测试它，我们就能以合理的代价构建一个能正常工作的微处理器。网络边栏 `arch:vlog` 描述了如何生成 Y86-64 处理器的 Verilog 版本。

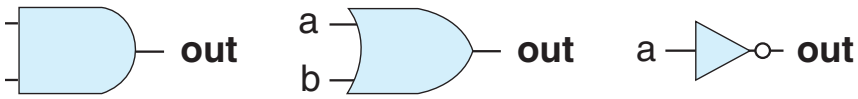


图 4.9　逻辑门类型。每个门生成的输出等于其输入的某个布尔函数。

4.7.1 逻辑门

逻辑门是数字电路的基本计算元件，它产生的输出等于输入比特位的某种布尔函数。图 4.9 给出了表征布尔函数与（and）、或（or）和非（not）的标准符号，每个门电路的下方标出了与之对应的 HCL 表达式，采用的是 C 语言（见 2.1.8 节）的运算符：`&&` 表示与、`||` 表示或、`!` 表示非。我们采用这些运算符，而不使用 C 的位级运算符 `&`、`|` 和 `~`，是因为逻辑门只处理单个的位，而不是整个字。

虽然图中画的只是两输入版本的与门和或门，但这些门也常被用作 $n > 2$ 的 n 路运算。不过在 HCL 中，这类运算我们仍然写成二元运算符；例如，对输入 `a`、`b` 和 `c`，三输入与门的运算就用 HCL 表达式 `a && b && c` 来描述。逻辑门总是活跃的：只要门电路的某个输入发生变化，经过一小段时间之后，它的输出就会随之改变。

4.7.2 组合电路和 HCL 布尔表达式

把许多逻辑门组装成一个网络，就能构造出一种称为组合电路（combinational circuits）的计算部件。构造这种网络有几条限制：

- **每个逻辑门的输入必须与下列情况之一相连**：（1）系统的某个输入（称为主输入，primary input）；（2）某个存储单元的输出连接；或（3）某个逻辑门的输出。
- **多个逻辑门的输出不能连接在一起**。否则，它们会试图把同一条导线驱动成不同的电压，可能出现无效电压，或电路功能失常。
- **网络必须是无环的**。也就是说，网络中不能有一条穿过一串门而构成回路的路径。这样的回路会导致网络所计算的函数存在歧义。

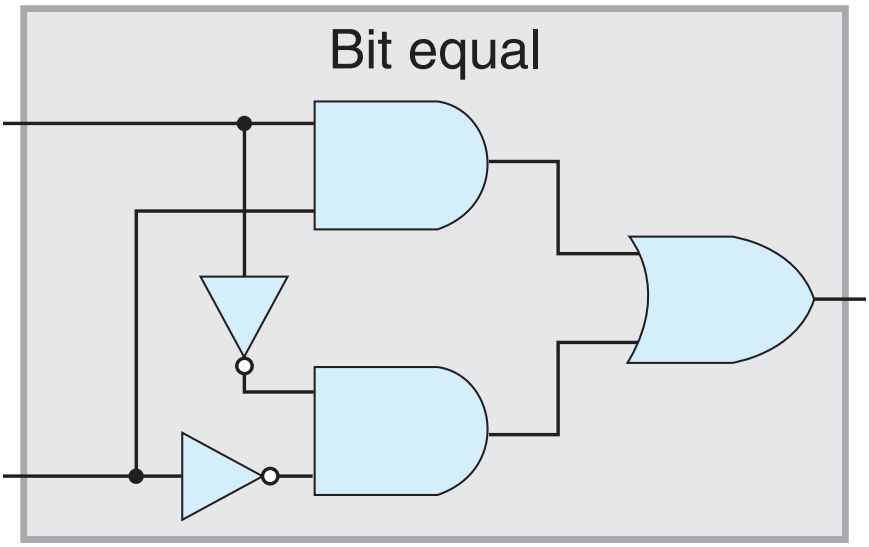


图 4.10　测试位相等的组合电路。当两个输入都为 0 或都为 1 时，输出等于 1。

图 4.10 给出了一个简单但很有用的组合电路的示例。它有两个输入 a 和 b，并生成单个输出 eq。当 a 与 b 都为 1（由上方的与门检出），或都为 0（由下方的与门检出）时，输出等于 1。这个网络的功能可以表示成 HCL：

```
bool eq = (a && b) || (!a && !b);
```

这段代码只是把位级信号（由数据类型 bool 指明）eq 定义为 a 和 b 的某个函数。从这个例子可以看出，HCL 采用 C 风格的记法："=" 把信号名与一个表达式关联起来。不过与 C 不同，我们并不把这件事看做要执行某次计算并把结果存入某个存储位置，它只是给表达式命名的一种方法。

练习 4.9　写出一个 HCL 表达式 "信号 xor 的值为输入 a 和 b 的异或"。xor 与前面定义的 eq 之间有什么关系？

图 4.11 给出了另一个简单而很有用的组合电路，称为多路复用器（multiplexor，常简写为 MUX）。

多路复用器根据某个控制输入信号从一组数据信号中选出一路。在这个单比特多路复用器里，两个数据信号是输入位 a 和 b，控制信号是输入位 s。当 s 为 1 时输出等于 a，当 s 为 0 时输出等于 b。从这个电路可以看出，两个与门分别决定是否把各自对应的数据输入传给或门：当 s 为 0 时，下方的与门把信号 b 传出去（因为该门电路的另一个输入是!s）；当 s 为 1 时，上方的与门把信号 a 传出去。同样，我们可以用出现在组合电路中的那些运算写出一个输出信号的 HCL 表达式：

```
bool out = (s && a) || (!s && b);
```

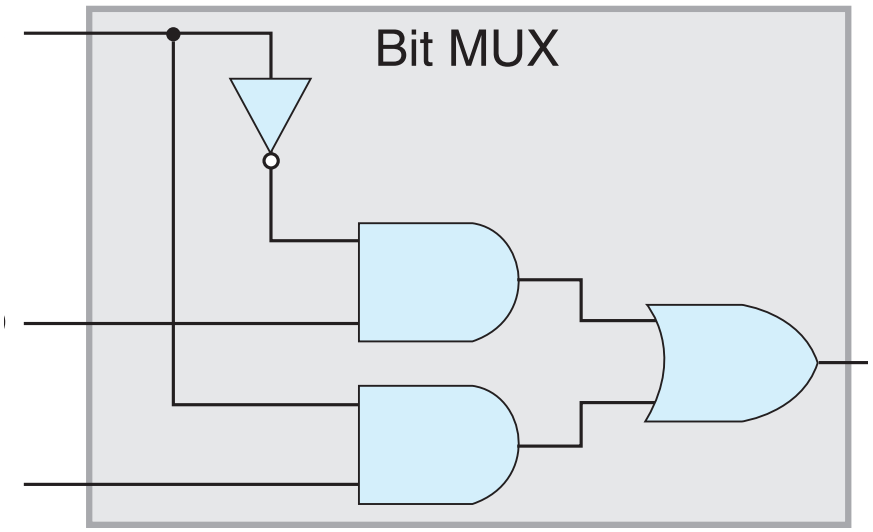


图 4.11 单位多路复用器电路。当控制信号 s 为 1 时,输出等于输入 a;当 s 为 0 时,输出等于输入 b。

我们的 HCL 表达式与组合逻辑电路及 C 的逻辑表达式并行。两者都用布尔运算来计算输入的函数。但这两种表达计算的方法之间存在几个值得注意的差别:

- **组合电路由一串逻辑门构成,它的输出会持续地响应输入的变化。** 只要电路的某个输入发生变化,那么经过一段时间之后,输出就会随之改变。相比之下,C 表达式只有在程序执行过程中遇到它时才会被求值。
- **C 中的逻辑表达式允许参数是任意整数。** 取值为 0 表示假,取任何其他值都表示真。相比之下,逻辑门只对位值 0 和 1 进行运算。
- **逻辑电路规则保证门电路的运算过程中不会跳过任何输入的求值。** 相反,C 的逻辑表达式可能只计算部分——如果只靠计算第一个参数就能得到 and 或 or 的结果,第二个参数根本不会被求值。例如,对于 C 表达式

```
(a && !a) && func(b,c)
```

函数 `func` 不会被调用,因为表达式 `(a && !a)` 的求值结果是 0。相比之下,逻辑电路没有部分求值的规则,门电路只会响应变化的输入。

4.7.3 字级的组合电路和 HCL 整数表达式

把大量的门电路组装成规模更大的网络,我们就能构造出能计算复杂得多的逻辑电路。通常,我们设计的电路操作的数据单位是一组位级信号,即用来

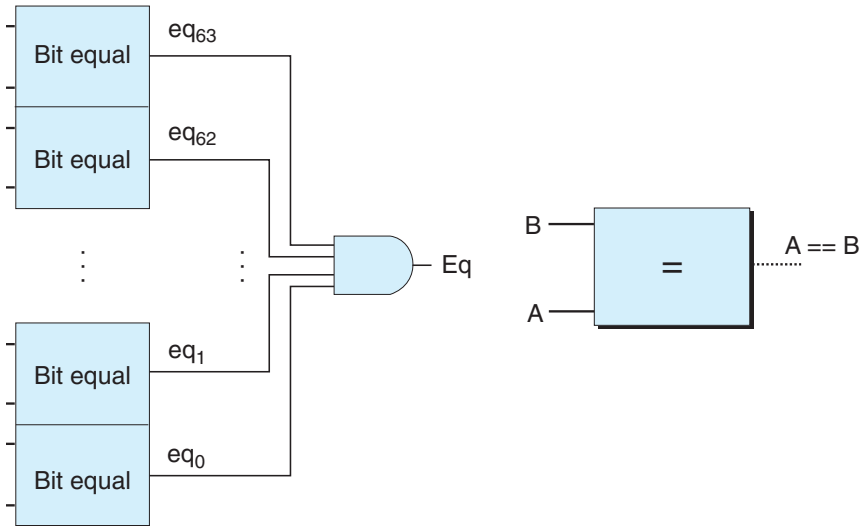


图 4.12 字级相等测试电路。当字 A 的每一位都等于字 B 中对应的位时，输出等于 1。字级相等是 HCL 中的一种运算。

表示一个整数或某种控制模式的字。例如，我们的处理器设计中包含许多字，字长在 4 到 64 位之间，表示整数、地址、指令代码和寄存器标识符。

执行字级运算的组合电路是这样构造的：利用门电路，根据输入字各个位的取值，计算输出字的每一位。例如，图 4.12 给出了一个组合电路，判定两个 64 位字 A 与 B 是否相同。也就是说，当且仅当 A 的每一位都等于 B 中对应的位时，输出才等于 1。这个电路是由 64 份图 4.10 中位级相等电路的拷贝组成。各个位级电路的输出再通过与门聚合起来，形成整个电路的输出。

在 HCL 中，我们将所有字级信号声明为 `int`，而不必指明其位长。这样做的目的以简单为准。在功能完备的硬件描述语言中，每个字都可以声明为某个特定的位长。HCL 允许对字进行比较是否相等，因而在字一级上，图 4.12 所示电路的功能可以表示为

```
bool Eq = (A == B);
```

这里参数 A 和 B 的类型是 `int`。注意，我们沿用了 C 语言的语法约定：“=”是赋值运算符，“==”是相等判断运算符。

图 4.12 的右半部分所示，画字级电路图时，我们用中粗实线表示承载字中各个位的导线集合，而用虚线表示单比特信号。

练习 4.10 假设希望用练习 4.9 的异或电路而不是位级相等电路，来构造一个字级相等电路。请设计 64 位字的电路，要求只使用 64 份位级异或电路拷贝和其他两个逻辑门。

图 4.13 给出了字级多路复用器的电路图。这个电路根据控制位 s 生成一个 64 位的字 Out，等于两个输入字 A、B 之一。整个电路由 64 份同构的子电路组成，每个子电路的结构都类似于图 4.11 中的位级多路复用器。不过，字级版本并不把位级多路复用器复制 64 份，而是只生成一次!s，然后在每一位位置反复使用它，从而减少反相器的数量。

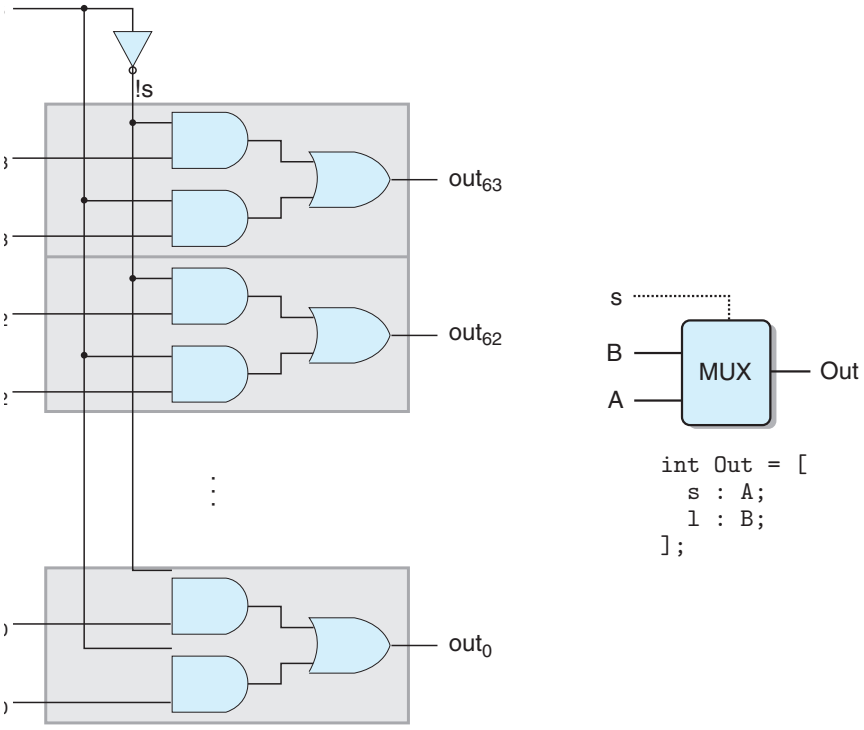


图 4.13　字级多路复用器电路。当控制信号 s 为 1 时，输出等于输入字 A，否则等于 B。在 HCL 中用 case 表达式来描述多路复用器。

我们的处理器设计中会用到各种各样的多路复用器，利用它们可根据某个控制条件从多个来源中选出一个字。多路复用功能在 HCL 中用 case 表达式（case expression）描述。case 表达式的一般形式如下：

```
[
    select1 : expr1;
    select2 : expr2;
    . . .
    selectk : exprk;
]
```

表达式包含一系列分支。每个分支 i 由两部分组成：一部分是布尔表达式 $select_i$，说明什么时候要选择这个分支；另一部分是整数表达式 $expr_i$，说明输出结果的值。

与 C 的 `switch` 语句不同，我们不要求各个选择表达式互斥。逻辑上，选择表达式按顺序求值，第一个等于 1 的分支即被选中。例如，图 4.13 的字级多路复用器可以描述成：

```
word Out = [
    s: A;
    1: B;
];
```

这段代码中第二个选择表达式就是 1，意思是没有哪个前面的分支被选中时，就选择这个分支。这是在 HCL 中指定默认分支的方法。差不多所有的 case 表达式都以这种方式结尾。

允许选择表达式不互斥，使得 HCL 代码更容易读懂。实际的硬件多路复用器必须由互斥的信号来控制哪个输入字能通过并输出到输出端，例如图 4.13 中的信号 `s` 与`!s`。要把一个 HCL 的 case 表达式翻译成硬件，逻辑综合程序需要分析所有选择表达式的集合，通过只让第一个匹配的分支被选中来化解所有可能的冲突。

选择表达式可以是任意的布尔表达式，分支的个数也可以是任意的。这样，case 表达式就能描述输入信号有多个可选值、而选择条件很复杂的电路。例如，考虑图 4.14 画出的四路多路复用器的示意图。该电路根据控制信号 `s1` 和 `s0` 从四个输入字 A、B、C 和 D 中选出一个，把控制信号当作 2 位二进制数来处理。我们可以用 HCL 来描述它，用布尔表达式描述控制位模式的所有组合：

```
word Out4 = [
    !s1 && !s0 : A; # 00
    !s1        : B; # 01
    !s0        : C; # 10
    1          : D; # 11
];
```

右边的注释（从 # 开始一直延伸到行尾的文本都是注释）说明 `s1` 和 `s0` 取什么组合时会选中这个分支。可以看到，由于只有第一个匹配的分支才被选中，选择表达式有时能化简。例如，第二个表达式可以写成`!s1`，而不必写成更完整的`!s1&& s0`，因为 `s1` 等于 0 时的另一种可能已经在第一个选择表达式中处理掉了。类似地，第三个表达式可以写成`!s0`，而第四个直接写 1。

最后一个例子：假设要设计一个逻辑电路，在一组字 A、B、C 中挑出最小的，示意图如下：

我们可以用一个 HCL case 表达式来表示它的功能：

```
word Min3 = [
    A <= B && A <= C : A;
```

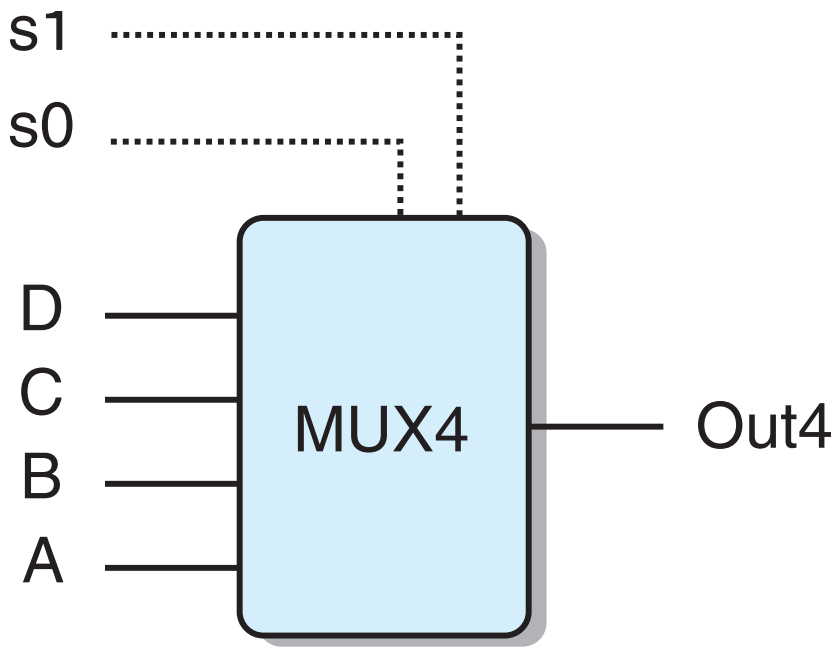


图 4.14 四路多路复用器。控制信号 s1 和 s0 的不同组合决定将哪个数据输入传送到输出。

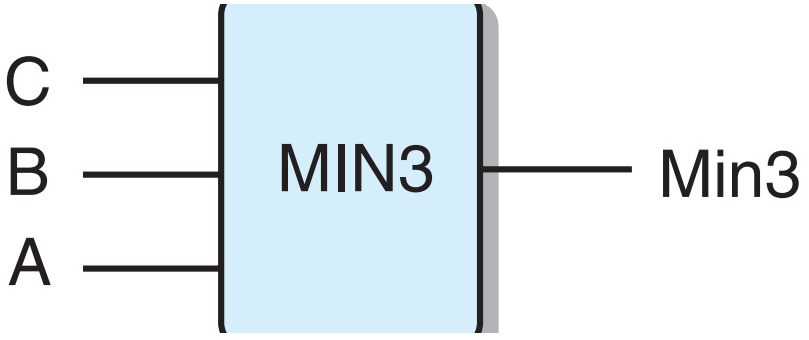


图 4.90 （示意图）

```
    B <= A && B <= C : B;
    1                : C;
];
```

练习 4.11 已给的用来计算三个字中最小值的 HCL 代码包含四个形如 $X \leq Y$ 的比较表达式。请重写这段代码，使得只用三次比较就能计算出同样的结果。

练习 4.12 请编写一个组合逻辑电路，其输入为字 A、B、C，输出是三个值的中间值。也就是说，输出等于三个输入中介于最大值和最小值之间的那个字。

组合逻辑电路可以设计成对字级数据执行许多种不同类型的运算。这些电路的详细设计超出了本书论述的范围。有一种很重要的组合电路，称为算

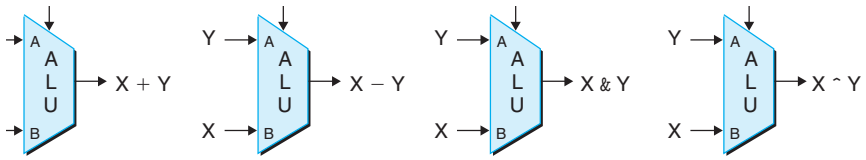


图 4.15　算术/逻辑单元(ALU)。根据功能输入的设置,该电路执行四种不同的算术或逻辑运算之一。

术/逻辑单元(ALU),图 4.15 以抽象层次画出了它的结构图。在我们的版本中,该电路有三个输入:两个分别标着 A 和 B 的数据输入,以及一个控制输入。根据控制输入的值的不同,电路会对数据输入执行不同的算术或逻辑运算。可以看到,图中 ALU 的四种运算正好与 Y86-64 指令集所支持的四整数运算相一致,而各控制值恰好与这些指令的功能代码相匹配(见图 4.3)。另外还要注意减法的操作数顺序:B 的输入会减去 A 的输入。这样安排是考虑到 `subq` 指令中参数的排列顺序。

4.7.4 集合关系

我们的处理器设计中有许多地方需要把一个信号与若干个可能匹配的信号比较,例如判断正在处理的某条指令的代码是否属于某类指令代码。举个简单的例子:假设我们想为图 4.14 的四路多路复用器生成信号 `s1` 和 `s0`,做法是从 2 位的信号 `code` 中取其高位和低位,示意如下:

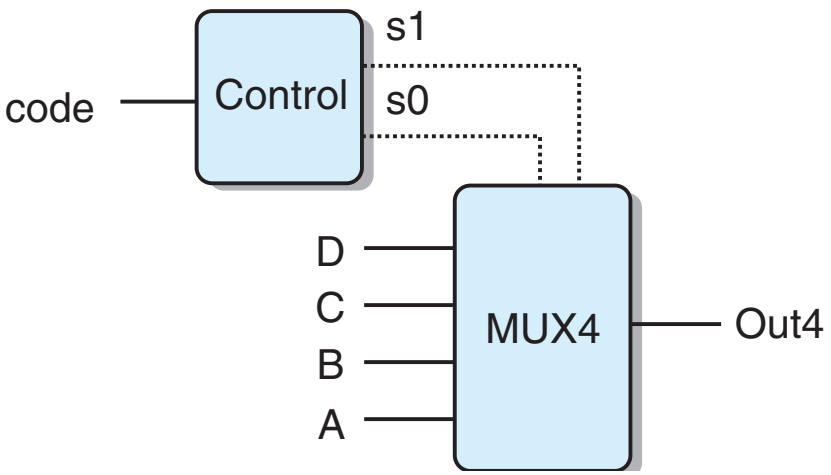


图 4.91　(示意图)

在这个电路中,2 位的信号 `code` 选择四个数据字 A、B、C 和 D 中的一个。我们可以用针对 `code` 的可能取值的相等测试来表示信号 `s1` 和 `s0` 的生成:

```
bool s1 = code == 2 || code == 3;
```

```
bool s0 = code == 1 || code == 3;
```

还有更精炼的写法可以直接说明这一性质：`s1` 在 `code` 属于集合 $\{2,3\}$ 时为 1，`s0` 在 `code` 属于集合 $\{1,3\}$ 时为 1：

```
bool s1 = code in { 2, 3 };
bool s0 = code in { 1, 3 };
```

集合关系测试的一般形式是

```
iexpr in {iexpr1, iexpr2, . . . , iexprk}
```

其中被测试的值 `iexpr` 和那些可能匹配的值 `iexpr1` 至 `iexprk` 都是整数表达式。

4.7.5 存储器和时钟

组合电路从本质上讲并不保存任何信息。它们只是对输入端的信号做出响应，生成输出值等于输入的某个函数的信号。要得到时序电路（sequential circuit），也就是既有状态、又对该状态进行运算的系统，就必须引入能保存以比特表示的信息的设备。我们的存储设备都由同一个时钟来控制，这是一个周期性信号，用来决定什么时候往设备里装入新值。我们考虑两类存储设备：

- **时钟寄存器（clocked register，或简称寄存器）**。存放单个的位或字，靠时钟信号控制把输入端上的值装入寄存器。
- **随机访问存储器（random access memory，或简称存储器）**。存放多个字，用地址来选择要读出或写入的字。随机访问存储器包括：(1) 处理器的虚拟内存系统——硬件与操作系统软件协同工作，使处理器看起来好像能访问一个大地址空间中的任意一个字；(2) 寄存器堆（register file），在这类设备中，寄存器的标识符就相当于地址。在 Y86-64 处理器中，寄存器堆保存着 15 个程序寄存器（`%rax` 到`%r14`）。

可以看到，说到硬件与机器级程序设计时，“寄存器”这个词的含义有点不同。从硬件的角度看，一个寄存器就是直接通过它的输入和输出线连接到电路其余部分的一个电路器件。从机器级程序设计角度看，寄存器代表 CPU 中提供的有限个可寻址的字，其地址由寄存器标识符构成。这些字一般保存在寄存器堆中，我们以后会看到，硬件有时会把一个字直接从一条指令传送到另一条指令，从而避免“先写寄存器堆、再读寄存器堆”造成的时延。为了避免混淆，必要时我们会把这两类寄存器分别称为“硬件寄存器”和“程序寄存器”。

图 4.16 更详细地画出了硬件寄存器及其工作方式。在大部分时间里，寄存器都保持在某个固定的状态（记作 x），产生的输出等于它当前的状态。信号穿

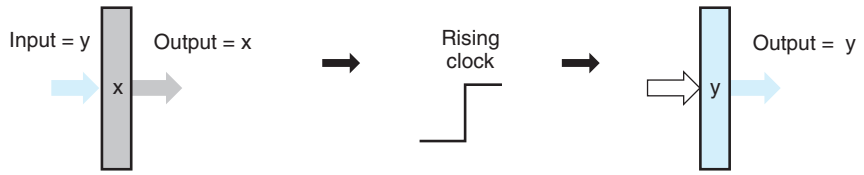


图 4.16　寄存器操作。寄存器输出保持当前的寄存器状态，直到时钟信号上升。当时钟上升时，寄存器输入端的值被捕获，成为新的寄存器状态。

过寄存器前面的组合逻辑，为寄存器的输入生成一个新值（记作 y），但只要时钟保持低电平，寄存器的输出就保持不变。当时钟上升时，输入信号被装入寄存器，成为它的下一个状态（y），并作为寄存器新的输出一直保持到下一个时钟上升沿。这里关键的一点是，寄存器充当电路中不同部分之间组合逻辑的屏障。每个时钟周期内，值只在时钟上升沿这一时刻从寄存器的输入传播到寄存器的输出。我们的 Y86-64 处理器将用时钟寄存器来保存程序计数器（PC）、条件码（CC）和程序状态（Stat）。

下面是典型的寄存器堆的示意图：

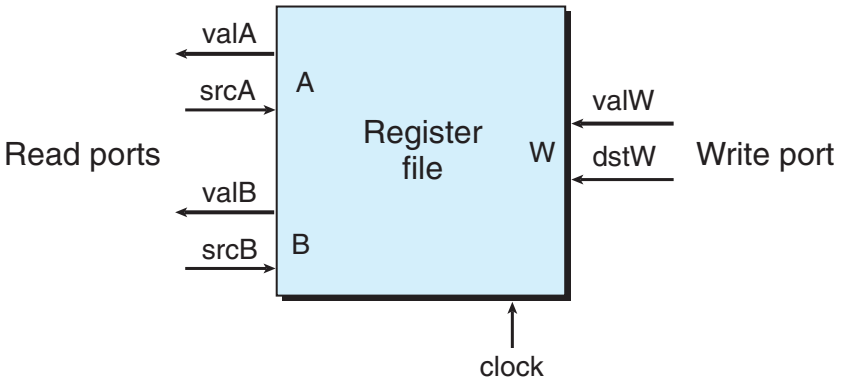


图 4.92　（示意图）

这个寄存器堆有两个读端口，分别称为 A 与 B，还有一个写端口称为 W。这种多端口的随机访问存储器允许同时进行多次读写操作。图中寄存器堆的电路可以读出两个程序寄存器的值并同时更新第三个寄存器的状态。每个端口都有一个地址输入，说明要选择哪个程序寄存器，并有一个数据输出或输入给出该程序寄存器的取值。地址是寄存器标识符，按图 4.4 的编码。两个读端口的地址输入的名称是 `srcA` 和 `srcB`（“源 A”与“源 B”的缩写），数据相应地输出为 `valA` 和 `valB`（“值 A”与“值 B”的缩写）。写端口的地址输入被称为 `dstW`（“目的 W”的缩写），数据输入为 `valW`（“值 W”的缩写）。

寄存器堆不是组合电路——它有内部存储单元。不过，我们实现的版本中，寄存器允许像对待一组合逻辑电路一样对它进行读操作：给出地址输入，输出

数据。只要 srcA 或 srcB 设置为某个寄存器的 ID，经过一段时间延迟后，相应寄存器中的值就会出现在 valA 或 valB 上。例如，将 srcA 设置为 3 将使程序寄存器%rbx 的值能被读出，随后这个值会出现在输出 valA 上。

往寄存器堆写字是由时钟信号控制的，方式与往时钟寄存器装入值类似。每次时钟上升时，输入 valW 上的值被写入由输入 dstW 上的寄存器 ID 指明的程序寄存器。当 dstW 被设置为特殊 ID 值 0xF 时，不写任何程序寄存器。寄存器堆既能读又能写，于是有个自然的问题是："如果电路试图同时读和写同一个寄存器，会发生什么？"答案很直接：如果读端口和写端口用的是同一个寄存器 ID，那么当时钟上升时，读端口的数据输出会从旧值过渡到新值。把寄存器堆集成到处理器设计中时，我们会注意考虑到这一性质。

我们的处理器中有一个用来保存程序数据的随机访问存储器，示意如下：

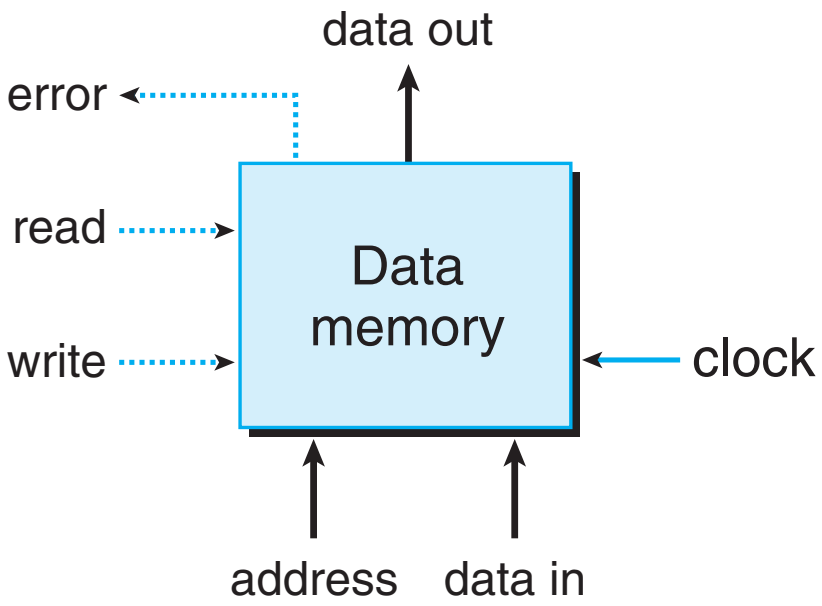


图 4.93 （示意图）

这个存储器有一个地址输入、一个用于写入的数据输入和一个用于读出的数据输出。与寄存器堆一样，从存储器中读数据的方式和组合逻辑类似：在地址输入端给出一个地址，把写控制信号设为 0，经过一段时间延迟之后，存储在那个地址上的值就会出现在数据输出 data out 上。如果地址超出了允许的范围，错误信号 error 会被置为 1，否则为 0。往存储器写字由时钟控制：把地址设成想要写的地址、把输入数据设成想要的值、把写信号设为 1。当时钟被触发时，只要地址有效，存储器中指定的位置就会被更新。与读操作类似，如果地

址无效,error 信号会被置 1。这个信号由组合逻辑生成,因为所需的边界检查仅仅是地址输入的纯函数,不涉及保存任何状态。

> **边栏 真实世界中的存储器设计**
> 一个真正的商用微处理器的存储器系统要比我们在设计中假设的简单存储器复杂得多。它由若干种形式的硬件存储器组成,包括几个随机访问存储器,加上非易失性存储器或磁盘,还包括管理这些设备的各种硬件和软件机制。我们将在第 6 章详细探讨存储器系统的设计和特性.
> 尽管如此,我们所设计的这个简单的存储器,对于较小的系统还是可以使用的,而且对更复杂的系统来说,它提供了处理器与存储器之间接口的一个抽象。

我们的处理器还包含一个只读存储器,用来读出指令。在大多数真实的系统中,这些存储器都被合并成一个有两端口的存储器:一个端口用于读指令,另一个端口用于读或写数据。

4.8 Y86-64 的顺序实现

现在,实现 Y86-64 处理器所需的各个部件都已经齐备了。第一步,我们来描述一个称为 extttSEQ 的处理器(SEQ 取自“sequential”,即“顺序”之意)。在每个时钟周期里,extttSEQ 要完成处理一条完整指令所需的全部步骤。但是这样需要的时钟周期就会非常长,因而时钟频率会低得无法接受。开发 SEQ 的目的,是为我们最终的目标——实现一个高效的流水线处理器——迈出第一步。

4.8.1 将处理组织成阶段

一般说来,处理一条指令包含许多操作。我们把它们按一个特定的阶段顺序组织起来,努力使所有指令都遵循统一的变化,即使各种指令的行为差别很大。每个步骤中的细节则由所执行的具体指令决定。这样设计框架将有助于我们设计出最充分地利用硬件的处理器。下面是各阶段及其中所执行操作的一个非正式描述:

- **取指(fetch)**。 取指阶段以程序计数器(PC)的值为存储器地址,读出一条指令中的字节。就指令本身而言,把指示符字节取出后的两个 4 位部分称为 icode(指令码)和 ifun(指令功能)。指令可能还会附带一个寄存器指示字节,给出一个或两个寄存器操作数指示符 rA 和 rB。它还可能附带一个 8 字节的常数字 valC。取指阶段还会计算 valP,也就是按顺序紧跟在当前这条指令后面的那条指令的地址。也就是说,valP 等于 PC 的值加上刚取出的指令的长度。

- **译码(decode)**。 译码阶段从寄存器堆读出最多两个操作数,得到值 valA 和 / 或 valB。通常情况下,这一阶段读取 rA 和 rB 字段对应的寄存器,但对某些指令来说,它会读的是寄存器%rsp。
- **执行(execute)**。 在执行阶段,算术/逻辑单元(ALU)或者执行由指令指定的运算(依据 ifun 的值),或者计算一次存储器引用的有效地址,或者对栈指针加 1 或减 1。这样计算出来的值称为 valE。条件也有可能被设置。对条件传送指令,这一阶段考察条件码和传送条件(由 ifun 给出),并且只有当条件成立时才允许更新目的寄存器。类似地,对跳转指令,这一阶段判定分支是否会被执行。
- **访存(memory)**。 访存阶段可以把数据写到存储器中,也可以从存储器读出数据。读出的值记作 valM。
- **写回(write back)**。 写回阶段把最多两个结果写回寄存器堆。
- **PC 更新(PC update)**。 将程序计数器设置成下一条指令的地址。

处理器会无限反复地执行这些阶段。在我们简化后的实现中,处理器在出现任何异常条件时就会停止——也就是说,当它执行 halt 指令或非法指令时,或者当它试图读或写非法地址时。在更完整的设计中,处理器会进入异常处理模式,开始执行由异常类型决定的特殊代码。

从上面的描述可以看出,执行一条指令所要求的计算量多得惊人。我们不仅要执行指令所声明的运算,还要计算地址、更新栈指针以及确定下一条指令的地址。好在每条指令的总体流程可以尽可能地做成相似。在硬件设计中,采用非常简单而统一的结构非常重要,因为我们想将需要的硬件总量减到最少,而且最终必须把它们映射到集成电路芯片的二维表面上去。为了减少复杂度,一种做法是让不同的指令尽量共享同一批硬件。比如,我们的每种处理器设计里都只有一个算术/逻辑单元,根据被执行指令的类型让它在不同场合以不同的方式工作。在硬件中复制逻辑电路块的成本比在软件中保存多份代码副本的成本要高得多;而且,与软件相比,在硬件系统中处理各种特例和细节也要难得多。

我们的挑战是:让执行每条不同指令所需的计算都能装进这个通用框架。我们将用图 4.17 给出的代码来说明不同 Y86-64 指令的处理过程。图 4.18 到图 4.21 中的表描述了各种 Y86-64 指令如何经过各个阶段。认真研究这些表是值得的。这些表的形式使其能够直接映射到硬件上。表中的每一行描述对某个信号或存储状态的一次赋值

```
1  0x000: 30f20900000000000000 |     irmovq $9, %rdx
2  0x00a: 30f31500000000000000 |     irmovq $21, %rbx
3  0x014: 6123                 |     subq %rdx,
```

```
      %rbx                          # 计算减法
4  0x016: 30f48000000000000000 |     irmovq $128,%rsp
        # 练习 4.13
5  0x020: 40436400000000000000 |     rmmovq %rsp,
      100(%rbx)                  # 存储
6  0x02a: a02f                 |     pushq %rdx
        # 压栈
7  0x02c: b00f                 |     popq %rax
        # 练习 4.14
8  0x02e: 734000000000000000   |     je done
        # 不发生跳转
9  0x037: 804100000000000000   |     call proc
        # 练习 4.18
10 0x040:                      | done:
11 0x040: 00                   |     halt
12 0x041:                      | proc:
13 0x041: 90                   |     ret
        # 返回
14                               |
```

图 4.17　样例 Y86-64 指令序列。我们要逐条跟踪这些指令在各阶段中的处理过程。

(赋值用符号“←”表示)。这些赋值应当看做是按从上到下的顺序一步步执行的。等以后把这些计算映射到硬件时,我们会发现,其实并不需要严格按照这个顺序。

图 4.18 给出了指令类型 OPq(整数运算和逻辑运算)、rrmovq(寄存器-寄存器传送)和 irmovq(立即数-寄存器传送)所需的处理。先来看整数运算。对照图 4.2 可以看到,我们精心选择了指令的编码,使得四个整数运算(addq、subq、andq 和 xorq)的 icode 值都相同。于是这四类指令都可以用同一组步骤来处理,只有 ALU 要执行的计算必须按照 ifun 编码的具体指令加以设置。

整数运算指令的处理遵循上面给出的一般模式。在取指阶段不需要常数字,所以 valP 按 PC + 2 计算。在译码阶段读出两个操作数。执行阶段把它们与功能指示符 ifun 一起供给 ALU,使 valE 成为指令的结果。这个计算写成表达式 *valB* OP *valA*,其中 OP 表示由 ifun 指定的运算。注意两个参数的顺序——它与 Y86-64(及 x86-64)的约定一致。例如,指令 subq %rax,%rdx 应当计算出 R[%rdx] − R[%rax]。访存阶段对这些指令不做任何事,但是 valE 会在写回阶段被写入寄存器 rB,而 PC 会被设置为 valP,指令的执行到此结束。

执行 rrmovq 指令与进行算术运算差不多。只是这时我们不需要去取第二个寄存器操作数,而是把第二个 ALU 输入端接常数 0,并把 0 加到第一个输入上,得 $valE = valA$,

图 4.18　Y86-64 指令 OPq、rrmovq 和 irmovq 顺序实现的计算。这些指令计算出一个值并把结果存入一个寄存器。记号 $icode:ifun$ 表示指令指示符字节的两个部分，$rA:rB$ 表示寄存器指示符字节的两个部分，$M_1[x]$ 表示对存储位置 x 上 1 个字节的访问(读或写)，$M_8[x]$ 表示对 8 个字节的访问。

阶段	OPq *rA*, *rB*	rrmovq *rA*, *rB*	irmovq *V*, *rB*
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valC \leftarrow M_8[\text{PC}+2]$
	$valP \leftarrow \text{PC}+2$	$valP \leftarrow \text{PC}+2$	$valP \leftarrow \text{PC}+10$
译码	$valA \leftarrow \text{R}[rA]$ $valB \leftarrow \text{R}[rB]$	$valA \leftarrow \text{R}[rA]$	
执行	$valE \leftarrow valB$ OP $valA$ 设置 CC	$valE \leftarrow 0 + valA$	$valE \leftarrow 0 + valC$
访存			
写回	$\text{R}[rB] \leftarrow valE$	$\text{R}[rB] \leftarrow valE$	$\text{R}[rB] \leftarrow valE$
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow valP$

然后把这个值写入寄存器堆。irmovq 的处理与此类似，只是用常数 valC 作为第一个 ALU 输入。另外，由于指令格式很长，irmovq 必须把程序计数器加上 10。这两条指令都保持条件码不变。

练习 4.13　填写下表右栏，描述图 4.17 目标代码第 4 行 irmovq 指令的处理：

阶段	通用 irmovq *V*, *rB*	具体 irmovq $128, %rsp
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valC \leftarrow M_8[\text{PC}+2]$ $valP \leftarrow \text{PC}+10$	
译码		
执行	$valE \leftarrow 0 + valC$	

边栏　对 subq 指令执行的追踪

我们来看图 4.17 目标代码第 3 行 subq 指令的处理过程，以此为例。可以看出，前面两条指令把寄存器%rdx 和%rbx 分别初始化为 9 和 21。还可以看出，这条指令位于地址 0x014，共 2 个字节，值为 0x61 和 0x23。各阶段的过程如下表所示：表左边列出处理 OPq 指令的一般规则(图 4.18)，右边是针对这一具体指令的计算。

阶段	OPq *rA*, *rB*	subq %rdx, %rbx
取指	$icode:ifun \leftarrow M_1[\text{PC}]$	$icode:ifun \leftarrow M_1[\texttt{0x014}] = \texttt{6:1}$
	$rA:rB \leftarrow M_1[\text{PC}+1]$	$rA:rB \leftarrow M_1[\texttt{0x015}] = \texttt{2:3}$
	$valP \leftarrow \text{PC}+2$	$valP \leftarrow \texttt{0x014} + 2 = \texttt{0x016}$
译码	$valA \leftarrow \text{R}[rA]$	$valA \leftarrow \text{R}[\texttt{\%rdx}] = 9$
	$valB \leftarrow \text{R}[rB]$	$valB \leftarrow \text{R}[\texttt{\%rbx}] = 21$
执行	$valE \leftarrow valB$ OP $valA$	$valE \leftarrow 21 - 9 = 12$
	设置 CC	$\text{ZF} \leftarrow 0, \text{SF} \leftarrow 0, \text{OF} \leftarrow 0$
访存		
写回	$\text{R}[rB] \leftarrow valE$	$\text{R}[\texttt{\%rbx}] \leftarrow valE = 12$
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow valP = \texttt{0x016}$

从这个追踪可以看出，这条指令的效果是：把寄存器%rbx 设置为 12，把三个条件码都设为 0，并把 PC 增加 2。

	通用	具体
阶段	irmovq *V*, *rB*	irmovq $128, %rsp
访存		
写回	$\text{R}[rB] \leftarrow valE$	
PC 更新	$\text{PC} \leftarrow valP$	

这条指令执行会如何修改寄存器和 PC?

图 4.19 给出了访存指令 rmmovq（存储器写指令）和 mrmovq（存储器读指令）所需的处理。我们看到与之前一样的基本流程，只不过此时使 ALU 把 valC 加到 valB 上，得到这次访存操作的有效地址（即位移量和基址寄存器的和）。在访存阶段，要么把寄存器值 valA 写入存储器，要么从存储器读出 valM。

图 4.20 给出了处理 pushq 和 popq 指令所需的步骤。这两条指令是 Y86-64 语言中最难实现的指令之一，因为它们既要访问存储器，又要对栈指针做加或减。这两条指令的流程虽然相似，但有几个重要的差别。

pushq 指令的开始与前面的指令很相似，不过译码阶段以%rsp 作为第二个寄存器操作数的标识符，使 valB 取到栈指针的值。在执行阶段，用 ALU 把栈指针减 8。这个减过之后的值既用作存储器写的地址，也在写回阶段存回%rsp。

图 4.19　Y86-64 指令 `rmmovq` 和 `mrmovq` 顺序实现的计算。这两条指令读或写存储器。

阶段	`rmmovq` $rA, D(rB)$	`mrmovq` $D(rB), rA$
取指	$icode : ifun \leftarrow M_1[\text{PC}]$ $rA : rB \leftarrow M_1[\text{PC}+1]$ $valC \leftarrow M_8[\text{PC}+2]$ $valP \leftarrow \text{PC}+10$	$icode : ifun \leftarrow M_1[\text{PC}]$ $rA : rB \leftarrow M_1[\text{PC}+1]$ $valC \leftarrow M_8[\text{PC}+2]$ $valP \leftarrow \text{PC}+10$
译码	$valA \leftarrow \text{R}[rA]$ $valB \leftarrow \text{R}[rB]$	$valB \leftarrow \text{R}[rB]$
执行	$valE \leftarrow valB + valC$	$valE \leftarrow valB + valC$
访存	$\text{M}_8[valE] \leftarrow valA$	$valM \leftarrow \text{M}_8[valE]$
写回		$\text{R}[rA] \leftarrow valM$
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow valP$

用 `valE` 作为写地址，正好遵循了 Y86-64（及 x86-64）的约定：`pushq` 应当先减栈指针再写，虽然栈指针的实际更新要等到访存操作完成之后才发生。

`popq` 指令的处理过程与 `pushq` 类似，不同的是在译码阶段会读出栈指针的两份拷贝。这看起来明显是冗余的，不过后面会看到，让 `valA` 和 `valB` 都取栈指针的值，会使后续的流程与其他指令更相似，从而提高整个设计的统一性。在执行阶段用 ALU 把栈指针加 8，而访存操作用的是没有加 8 之前的值。写回阶段既把加过之后的栈指针写回%rsp，也把从存储器读出的值更新到寄存器 `rA` 中。把未加过的栈指针值作为访存地址维持了 Y86-64（及 x86-64）的约定：`popq` 应当先读存储器再对栈指针加 8。

> **边栏　对 `rmmovq` 指令执行的追踪**
> 我们来追踪图 4.17 目标代码第 5 行 `rmmovq` 指令的处理过程。可以看到，前一条指令把寄存器%rsp 初始化为 128，而%rbx 中保存的仍是 `subq` 指令（第 3 行）算出的 12。还可以看到，这条指令位于地址 `0x020`，共 10 个字节：前 2 个字节为 `0x40` 和 `0x43`，后 8 个字节是数字 `0x0000000000000064`（十进制的 100）按字节反序的结果。各阶段的处理如下：

	通用	具体
阶段	`rmmovq` *rA*, *D*(*rB*)	`rmmovq %rsp, 100(%rbx)`
取指	$icode:ifun \leftarrow M_1[\text{PC}]$	$icode:ifun \leftarrow M_1[\texttt{0x020}] = \texttt{4:0}$
	$rA:rB \leftarrow M_1[\text{PC}+1]$	$rA:rB \leftarrow M_1[\texttt{0x021}] = \texttt{4:3}$
	$valC \leftarrow M_8[\text{PC}+2]$	$valC \leftarrow M_8[\texttt{0x022}] = 100$
	$valP \leftarrow \text{PC}+10$	$valP \leftarrow \texttt{0x020}+10 = \texttt{0x02a}$
译码	$valA \leftarrow \text{R}[rA]$	$valA \leftarrow \text{R}[\texttt{\%rsp}] = 128$
	$valB \leftarrow \text{R}[rB]$	$valB \leftarrow \text{R}[\texttt{\%rbx}] = 12$
执行	$valE \leftarrow valB + valC$	$valE \leftarrow 12+100 = 112$
访存	$\text{M}_8[valE] \leftarrow valA$	$\text{M}_8[112] \leftarrow 128$
写回		
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow \texttt{0x02a}$

从这个追踪可以看出，指令的效果是把 128 写到存储器的地址 112 处，并且 PC 增加 10。

（该约定同样适用于 x86-64）：`popq` 应当先读存储器，再对栈指针加 8。

练习 4.14　填写下表右栏，描述图 4.17 目标代码第 7 行 `popq` 指令的处理：

	通用	具体
阶段	`popq` *rA*	`popq %rax`
取指	$icode:ifun \leftarrow M_1[\text{PC}]$	
	$rA:rB \leftarrow M_1[\text{PC}+1]$	
	$valP \leftarrow \text{PC}+2$	

这条指令执行对寄存器和 PC 有什么影响？

练习 4.15　按图 4.20 列出的步骤，执行指令 `pushq %rsp` 会产生什么效果？这与练习 4.7 中为 Y86-64 确定的预期行为一致吗？

边栏　对 pushq 指令执行的追踪

我们来追踪图 4.17 目标代码第 6 行 `pushq` 指令的处理。此时寄存器%rdx 中是 9，寄存器%rsp 中是 128。还可以看到，这条指令位于地址 `0x02a`，共 2 个字节，值为 `0xa0` 和 `0x2f`。各阶段处理如下：

图 4.20　Y86-64 指令 `pushq` 和 `popq` 顺序实现的计算。这两条指令压栈和出栈。

阶段	`pushq` *rA*	`popq` *rA*
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valP \leftarrow \text{PC}+2$	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valP \leftarrow \text{PC}+2$
译码	$valA \leftarrow \text{R}[rA]$ $valB \leftarrow \text{R}[\texttt{\%rsp}]$	$valA \leftarrow \text{R}[\texttt{\%rsp}]$ $valB \leftarrow \text{R}[\texttt{\%rsp}]$
执行	$valE \leftarrow valB + (-8)$	$valE \leftarrow valB + 8$
访存	$\text{M}_8[valE] \leftarrow valA$	$valM \leftarrow \text{M}_8[valA]$
写回	$\text{R}[\texttt{\%rsp}] \leftarrow valE$	$\text{R}[\texttt{\%rsp}] \leftarrow valE$ $\text{R}[rA] \leftarrow valM$
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow valP$

阶段	通用 `popq` *rA*	具体 `popq %rax`
译码	$valA \leftarrow \text{R}[\texttt{\%rsp}]$ $valB \leftarrow \text{R}[\texttt{\%rsp}]$	
执行	$valE \leftarrow valB + 8$	
访存	$valM \leftarrow \text{M}_8[valA]$	
写回	$\text{R}[\texttt{\%rsp}] \leftarrow valE$ $\text{R}[rA] \leftarrow valM$	
PC 更新	$\text{PC} \leftarrow valP$	

阶段	通用 `pushq` *rA*	具体 `pushq %rdx`
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valP \leftarrow \text{PC}+2$	$icode:ifun \leftarrow M_1[\texttt{0x02a}] = \texttt{a:0}$ $rA:rB \leftarrow M_1[\texttt{0x02b}] = \texttt{2:f}$ $valP \leftarrow \texttt{0x02a} + 2 = \texttt{0x02c}$
译码	$valA \leftarrow \text{R}[rA]$ $valB \leftarrow \text{R}[\texttt{\%rsp}]$	$valA \leftarrow \text{R}[\texttt{\%rdx}] = 9$ $valB \leftarrow \text{R}[\texttt{\%rsp}] = 128$
执行	$valE \leftarrow valB + (-8)$	$valE \leftarrow 128 + (-8) = 120$
访存	$\text{M}_8[valE] \leftarrow valA$	$\text{M}_8[120] \leftarrow 9$
写回	$\text{R}[\texttt{\%rsp}] \leftarrow valE$	$\text{R}[\texttt{\%rsp}] \leftarrow 120$
PC 更新	$\text{PC} \leftarrow valP$	$\text{PC} \leftarrow \texttt{0x02c}$

从这个追踪可以看出,这条指令的效果是:把%rsp 设为 120,把 9 写入地址 120,并把 PC 增加 2。

练习 4.16 假设 popq 在写回阶段的两次寄存器写入按照图 4.20 中列出的顺序进行。执行 popq %rsp 会产生什么效果?这与练习 4.8 中确定的 Y86-64 的预期行为一致吗?

图 4.21 表明了三类控制传输指令的处理:各种跳转指令、call 和 ret。我们看到,这些指令用与前面指令相同的总体流程就能实现。

同整数运算一样,各类跳转指令可以用统一的方式处理,因为它们的差别只在判断是否执行分支上。跳转指令在取指和译码阶段与前面那些指令很像,只是没有寄存器指示字节。在执行阶段,检查条件码和跳转条件,判定是否发生分支,得到一位信号 Cnd。在 PC 更新阶段,根据这个标志:标志为 1 时把 PC 设为 valC(跳转目标),为 0 时设为 valP(其后那条指令的地址)。我们采用记号 $x\,?\,a:b$,它同 C 语言的条件表达式类似:x 为 1 时取 a,x 为 0 时取 b。

图 4.21 Y86-64 指令 jXX、call 和 ret 顺序实现的计算。这三条指令引起控制转移。

阶段	jXX *Dest*	call *Dest*	ret
取指	*icode*:*ifun* $\leftarrow M_1$[PC]	*icode*:*ifun* $\leftarrow M_1$[PC]	*icode*:*ifun* $\leftarrow M_1$[PC]
	valC $\leftarrow M_8$[PC + 1]	*valC* $\leftarrow M_8$[PC + 1]	
	valP $\leftarrow$ PC + 9	*valP* $\leftarrow$ PC + 9	*valP* $\leftarrow$ PC + 1
译码			*valA* $\leftarrow$ R[%rsp]
		valB $\leftarrow$ R[%rsp]	*valB* $\leftarrow$ R[%rsp]
执行		*valE* $\leftarrow$ *valB* + (−8)	*valE* $\leftarrow$ *valB* + 8
	Cnd $\leftarrow$ Cond(CC, *ifun*)		
访存		M_8[*valE*] $\leftarrow$ *valP*	*valM* $\leftarrow M_8$[*valA*]
写回		R[%rsp] $\leftarrow$ *valE*	R[%rsp] $\leftarrow$ *valE*
PC 更新	PC $\leftarrow$ *Cnd* ? *valC* : *valP*	PC $\leftarrow$ *valC*	PC $\leftarrow$ *valM*

练习 4.17 从指令编码(图 4.2 和图 4.3)可以看出,rrmovq 是一类更一般的指令(包含条件传送指令)的无条件版本。请说明怎样修改下面给出的 rrmovq 指令的处理步骤,使它也能处理那六条条件传送指令。参考图 4.21 中 jXX 指令实现条件行为的方式会对你有帮助。

边栏 对 je 指令执行的追踪

我们来追踪图 4.17 目标代码第 8 行 je 指令的处理。条件码被第 3 行的 subq 指令全置成了 0,所以分支不会发生。这条指令位于地址 0x02e,共 9 个字节:第一个字

阶段	cmovXX *rA*, *rB*
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $rA:rB \leftarrow M_1[\text{PC}+1]$ $valP \leftarrow \text{PC}+2$
译码	$valA \leftarrow \text{R}[rA]$
执行	$valE \leftarrow 0+valA$
访存	
写回	$\text{R}[rB] \leftarrow valE$
PC 更新	$\text{PC} \leftarrow valP$

节值为 `0x73`，其余 8 个字节是数字 `0x0000000000000040`（跳转目标）按字节反序的表示。各阶段处理如下：

	通用	具体
阶段	jXX *Dest*	`je 0x040`
取指	$icode:ifun \leftarrow M_1[\text{PC}]$ $valC \leftarrow M_8[\text{PC}+1]$ $valP \leftarrow \text{PC}+9$	$icode:ifun \leftarrow M_1[\texttt{0x02e}] = \texttt{7:3}$ $valC \leftarrow M_8[\texttt{0x02f}] = \texttt{0x040}$ $valP \leftarrow \texttt{0x02e}+9 = \texttt{0x037}$
译码		
执行	$Cnd \leftarrow \text{Cond}(\text{CC}, ifun)$	$Cnd \leftarrow \text{Cond}(\langle 0,0,0\rangle, 3) = 0$
访存		
写回		
PC 更新	$\text{PC} \leftarrow Cnd\ ?\ valC : valP$	$\text{PC} \leftarrow 0\ ?\ \texttt{0x040} : \texttt{0x037} = \texttt{0x037}$

从这个追踪可以看出，这条指令的效果是把 PC 增加 9。

`call` 与 `ret` 指令同 `pushq` 与 `popq` 指令有几分相似，但向量和弹出的是程序计数器。对 `call` 指令，压入的是 `valP`，即 `call` 后面那条指令的地址。在 PC 更新阶段，PC 被设为 `valC`，即调用的目标地址。对 `ret` 指令，在 PC 更新阶段把弹出的值 `valM` 赋给 PC。

练习 4.18　填写下表右栏，描述图 4.17 目标代码第 9 行 `call` 指令的处理：

阶段	通用 `call` *Dest*	具体 `call 0x041`
取指	*icode*:*ifun* $\leftarrow M_1$[PC] *valC* $\leftarrow M_8$[PC + 1] *valP* $\leftarrow$ PC + 9	

边栏　对 `ret` 指令执行的追踪

我们来追踪图 4.17 目标代码第 13 行 `ret` 指令的处理。这条指令位于地址 `0x041`，编码为单个字节 `0x90`。前面的 `call` 指令把%rsp 设为 120，并把返回地址 `0x040` 保存在存储器地址 120 处。各阶段处理如下：

阶段	通用 `ret`	具体 `ret`
取指	*icode*:*ifun* $\leftarrow M_1$[PC] *valP* $\leftarrow$ PC + 1	*icode*:*ifun* $\leftarrow M_1$[`0x041`] = `9:0` *valP* $\leftarrow$ `0x041` + 1 = `0x042`
译码	*valA* $\leftarrow$ R[`%rsp`] *valB* $\leftarrow$ R[`%rsp`]	*valA* $\leftarrow$ R[`%rsp`] = 120 *valB* $\leftarrow$ R[`%rsp`] = 120
执行	*valE* $\leftarrow$ *valB* + 8	*valE* $\leftarrow$ 120 + 8 = 128
访存	*valM* $\leftarrow$ M$_8$[*valA*]	*valM* $\leftarrow$ M$_8$[120] = `0x040`
写回	R[`%rsp`] $\leftarrow$ *valE*	R[`%rsp`] $\leftarrow$ 128
PC 更新	PC $\leftarrow$ *valM*	PC $\leftarrow$ `0x040`

从这个追踪可以看出，这条指令的效果是把 PC 设为 `0x040`，也就是 `halt` 指令的地址，并把%rsp 设为 128。

阶段	通用 `call` *Dest*	具体 `call 0x041`
译码	*valB* $\leftarrow$ R[`%rsp`]	
执行	*valE* $\leftarrow$ *valB* + (−8)	
访存	M$_8$[*valE*] $\leftarrow$ *valP*	
写回	R[`%rsp`] $\leftarrow$ *valE*	
PC 更新	PC $\leftarrow$ *valC*	

这条指令执行会对寄存器、PC 和存储器产生什么影响？

我们已经建立了一个统一的框架，能处理所有不同类型的 Y86-64 指令。虽然这些指令的行为千差万别，我们还是能把处理组织成六个阶段。现在的任务是构造实现这些阶段、并把它们连接起来的硬件。

4.9 SEQ 的硬件结构

实现全部 Y86-64 指令所需的计算可以组织成一个由六个基本阶段构成的序列：取指、译码、执行、访存、写回和更新 PC。图 4.22 展示了能够执行这些计算的一个硬件结构的抽象视图。程序计数器保存在一个寄存器中，位于图的左下方（标为“PC”）。信息沿着导线流动（图中把相关的导线画在一起，用一条粗灰线表示），先向上，然后绕向右边。处理工作由与各阶段相关联的硬件单元完成。从右侧折返下来的反馈路径中携带着要写入寄存器堆的更新值，以及更新后的程序计数器。在 SEQ 中，硬件单元的全部处理都发生在单个时钟周期之内，这一点将在 4.3.3 节中讨论。这张图省略了若干小块组合逻辑，以及驱动各个硬件单元、把合适的值路由给这些单元所需的全部控制逻辑。我们稍后会补上这些细节。我们把处理器画成信息自下而上流动的样式，这种画法与众不同。在开始设计流水线化的处理器时，我们会解释采用这一惯例的原因。

这些硬件单元与不同的处理阶段相关联：

取指。 以程序计数器寄存器中的值作为地址，指令存储器读出一条指令的字节。PC 增量器计算 `valP`，即增量后的程序计数器。

译码。 寄存器堆有两个读端口 A 和 B，寄存器值 `valA` 和 `valB` 同时经这两个端口读出。

执行。 执行阶段按照指令类型，用算术/逻辑单元（ALU）完成不同的事情。对整数运算指令，它执行指定的运算。对其他指令，它充当加法器，用来计算增大或减小后的栈指针、计算有效地址，或者干脆通过加零把自己的某个输入原样送出。

条件码寄存器（`CC`）保存三个条件码位。新的条件码值由 ALU 算出。执行条件传送指令时，是否更新目的寄存器的判定依据条件码和传送条件得出。类似地，执行跳转指令时，分支信号 `Cnd` 依据条件码和跳转类型计算。

访存。 执行访存指令时，数据存储器读出或写入一个存储器字。指令存储器和数据存储器访问同样的存储器位置，但目的不同。

写回。 寄存器堆有两个写端口。端口 E 用于写入 ALU 计算出的值，而端口 M 用于写入从数据存储器读出的值。

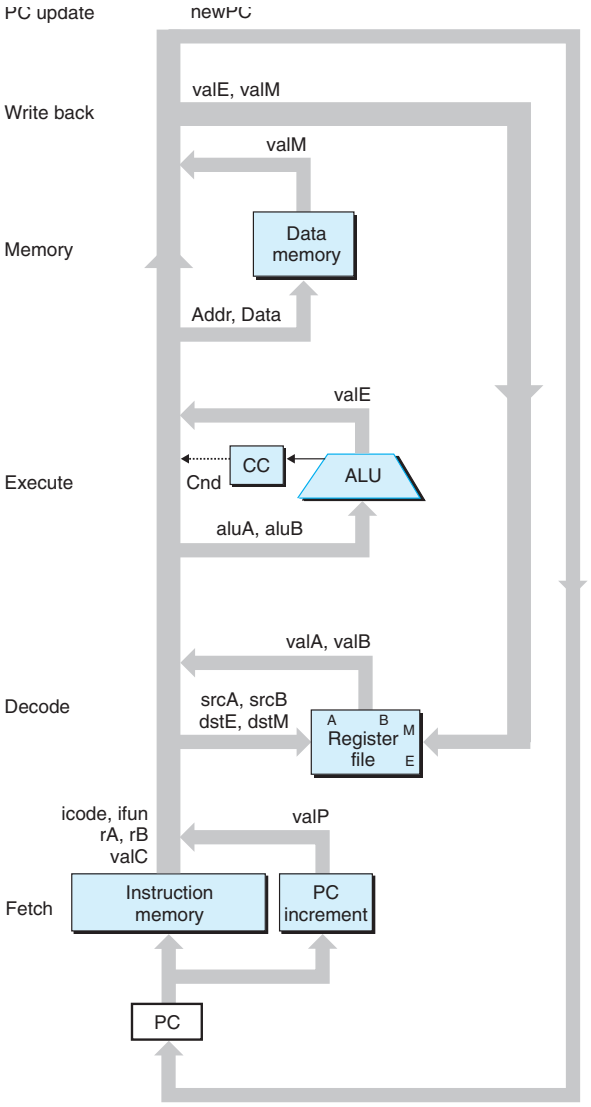


Figure 4.22 Abstract view of SEQ, a sequential implementation. The information processed during execution of an instruction follows a clockwise flow starting with an instruction fetch using the program counter (PC), shown in the lower left-hand corner of the figure.

图 4.22

更新 PC。 程序计数器的新值会在下面三者中选定：valP，即下一条指令的地址；valC，即 call 或跳转指令指明的目的地址；或者 valM，即从存储器读出的返回地址。

图 4.23 更为详细地展示了实现 SEQ 所需的硬件（要等到我们逐一考察各个阶段时，才会看到完整的细节）。我们看到与前面相同的一组硬件单元，只是现在把导线明确画了出来。在这张图以及我们的其他硬件图中，我们采用下列画图惯例：

- **时钟寄存器画成白色矩形**。程序计数器 PC 是 SEQ 中唯一的时钟寄存器（clocked register）。
- **硬件单元画成浅蓝色的方框**。其中包括存储器、ALU 等。我们的所有处理器实现都会用同一套基本单元。我们把这些单元当作“黑盒”看待，不去细究它们的内部设计。
- **控制逻辑块画成灰色圆角矩形**。这些块负责从一组信号源中进行选择，或者计算某个布尔函数。我们会完整、细致地考察这些块，包括编写它们的 HCL 描述。
- **导线名称用白色圆圈标示**。它们只是导线上的标签，并不是任何硬件元件。
- **字宽的数据连接用中等粗细的线表示**。每一条这样的线实际上代表一捆 64 根并行连接的导线，用来把一个字从硬件的一个部分传送到另一个部分。
- **字节宽及更窄的数据连接用细线表示**。每一条这样的线实际上代表一捆 4 根或 8 根导线，取决于导线上必须传送的是哪种类型的值。
- **单根位连接用虚线表示**。它们表示在芯片上各单元和块之间传递的控制值。

我们在图 4.18 至图 4.21 中给出的所有计算都有一个性质：每一条线要么代表对某个特定值的计算，例如 valP；要么代表对某个硬件单元的激活，例如存储器。这些计算和动作列在图 4.24 的第二列中。除了我们已经描述过的信号之外，这个清单还包括 4 个寄存器 ID 信号：srcA，即 valA 的来源；srcB，即 valB 的来源；dstE，即 valE 要写入的寄存器；以及 dstM，即 valM 要写入的寄存器。

这张图的右边两列给出 OPq 和 mrmovq 指令的计算，用来说明正在被计算的值。要把这些计算映射到硬件中，就要实现这样的控制逻辑：它在各硬件单元之间传送数据，并使这些单元如下工作，使得对每一种不同类型的指令都执

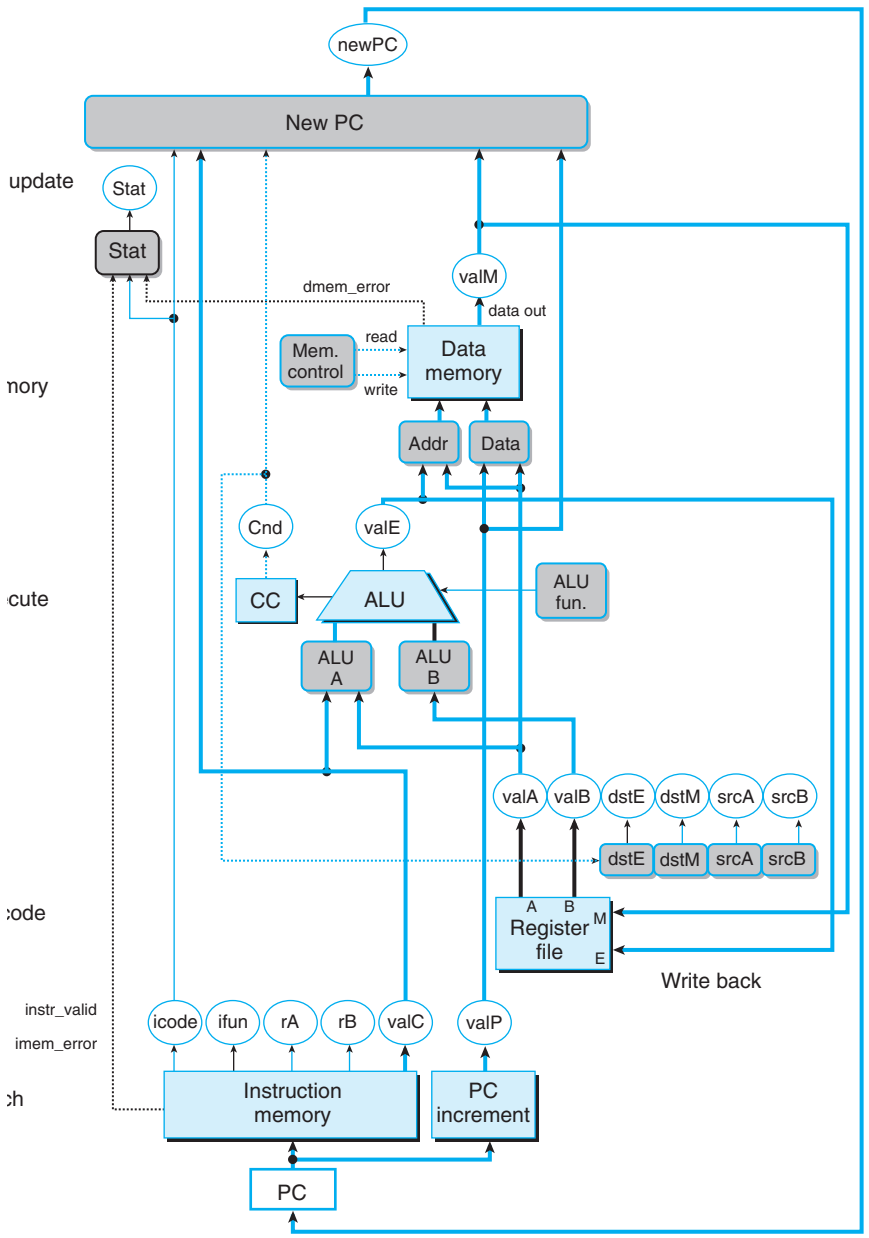


图 4.23 顺序实现 SEQ 的硬件结构。部分控制信号以及寄存器和控制字连接没有画出。

图 4.24 划分顺序实现中的各个计算步骤。第二列标明在 SEQ 各阶段中要计算的值或要执行的操作。以 OPq 和 mrmovq 的计算为例。

阶段	计算	OPq rA, rB	mrmovq D(rB), rA
取指	icode, ifun	icode : ifun $\leftarrow M_1[PC]$	icode : ifun $\leftarrow M_1[PC]$
	rA, rB	rA : rB $\leftarrow M_1[PC+1]$	rA : rB $\leftarrow M_1[PC+1]$
	valC		valC $\leftarrow M_8[PC+2]$
	valP	valP $\leftarrow PC+2$	valP $\leftarrow PC+10$
译码	valA, srcA	valA $\leftarrow R[rA]$	
	valB, srcB	valB $\leftarrow R[rB]$	valB $\leftarrow R[rB]$
执行	valE	valE $\leftarrow$ valB OP valA	valE $\leftarrow$ valB + valC
	条件码	设置 CC	
访存	读/写		valM $\leftarrow M_8[valE]$
写回	E 端口,dstE	$R[rB] \leftarrow$ valE	
	M 端口,dstM		$R[rA] \leftarrow$ valM
更新 PC	PC	PC $\leftarrow$ valP	PC $\leftarrow$ valP

行规定操作。这就是控制逻辑块的用途——在图 4.23 中它们被画成灰色圆角方框。我们的任务是逐一走过各个阶段,为这些块做出详细的设计。

4.10 SEQ 的时序

在介绍图 4.18 至图 4.21 的表时,我们说过应当把它们当作用一种程序设计记法写出的程序来读,其中的赋值自上而下顺序执行。另一方面,图 4.23 的硬件结构却以一种截然不同的方式运作:由单个时钟变换触发一次穿过组合逻辑的流动,来执行一条完整的指令。下面我们看看硬件是如何实现这些表中所列行为的。

我们的 SEQ 实现由组合逻辑和两类存储设备组成:时钟寄存器(程序计数器和条件码寄存器)与随机访问存储器(寄存器堆、指令存储器和数据存储器)。组合逻辑不需要任何定序或控制——只要输入一变,值就会穿过一个逻辑门网络传播开来。如前所述,我们还假设从随机访问存储器中读出与组合逻辑非常相似:输出的字根据地址输入产生。对较小的存储器(例如寄存器堆)来说,这是一个合理的假设;对更大的电路,可以用专门的时钟电路来模拟这种效果。由于我们的指令存储器只用来读指令,因此可以把这个单元当作组合逻辑对待。

这样一来,剩下需要对其定序加以显式控制的硬件单元只有 4 个:程序计数器、条件码寄存器、数据存储器和寄存器堆。它们都由一个单一的时钟信号控制,该信号触发向寄存器装入新值以及向随机访问存储器写入值。程序计数器在每个时钟周期都要装入一个新的指令地址。条件码寄存器只在执行整数

运算指令时装入。数据存储器只在执行 rmmovq、pushq 或 call 指令时才写入。寄存器堆的两个写端口允许每个周期更新两个程序寄存器，但我们可以用特殊的寄存器 ID 0xF 作为端口地址，表示这个端口不应执行写操作。

对寄存器和存储器的这种时钟控制，就是控制处理器内各项活动定序所需的全部手段。虽然所有状态更新实际上都是同时发生的，而且只在时钟上升、开始下一个周期时发生，但我们的硬件达到了与顺序执行图 4.18 至图 4.21 中表中所示的赋值一样的效果。这种等价性之所以成立，是因为 Y86-64 指令集的性质，也因为我们把计算组织得使设计遵循下面这条原理：

原理：无读回

处理器从不需要为了完成一条指令的处理而读回这条指令更新的状态。

这条原理对我们的实现能否成功至关重要。举例来说，假设我们实现 pushq 指令的方式是：先把%rsp 减 8，然后用更新后的%rsp 值作为一次写操作的地址。这种做法会违背上面陈述的原理，因为为了执行访存操作，它要求从寄存器堆中读出更新后的栈指针。我们的实现（图 4.20）与此相反：它把减 8 之后的栈指针值生成为信号 valE，然后把这个信号既用作寄存器写入的数据，又用作存储器写入的地址。于是，当时钟上升、下一个时钟周期开始时，寄存器写入和存储器写入可以同时进行。

再举一个体现这条原理的例子。可以看到，有些指令（整数运算指令）设置条件码，有些指令（条件传送和跳转指令）读取条件码，但没有指令既要设置条件码又要读取条件码。尽管条件码要到时钟上升、下一个时钟周期开始时才会被设置，但在任何指令试图读取它们之前，它们就已经被更新了。

图 4.25 展示了 SEQ 硬件将如何处理下面这段代码序列中的第 3 行和第 4 行的指令，这里用汇编代码书写，左侧列出指令地址：

```
1  0x000:  irmovq $0x100,%rbx      # %rbx <-- 0x100
2  0x00a:  irmovq $0x200,%rdx      # %rdx <-- 0x200
3  0x014:  addq %rdx,%rbx          # %rbx <-- 0x300 CC <-- 000
4  0x016:  je dest                 # 分支不发生
5  0x01f:  rmmovq %rbx,0(%rdx)     # M[0x200] <-- 0x300
6  0x029: dest: halt
```

从 1 号到 4 号标注的每个小图都展示了四个状态元件，加上组合逻辑以及状态元件之间的连接。我们把组合逻辑画成环绕条件码寄存器的样子，因为一部分组合逻辑（例如 ALU）生成条件码寄存器的输入，而另一些部分（例如分支计算和 PC 选择逻辑）则把条件码寄存器作为输入。寄存器堆和数据存储器都画成了读写分开的连接，因为读操作像穿过组合逻辑那样穿过这些单元，而写操作由时钟控制。

图 4.25 中的颜色编码表明电路信号与正在执行的不同指令之间的关联。我们假设处理开始时条件码为 100，条件码按 ZF、SF 和 OF 的顺序列出。在时钟周期 3 开始时（点 1），状态元件保存着第二条 `irmovq` 指令（清单第 2 行）更新完的状态，图中用浅灰色表示。组合逻辑显示为白色，表明它还没有来得及对改变后的状态做出反应。这个时钟周期从把地址 `0x014` 装入程序计数器开始。这使 `addq` 指令（清单第 3 行）被取出并处理，图中用蓝色表示。值流过组合逻辑，包括随机访问存储器的读出。到周期结束时（点 2），组合逻辑已经为条件码生成了新值（000），为程序寄存器`%rbx` 生成了一个更新，还为程序计数器生成了新值（`0x016`）。此时，组合逻辑已按照 `addq` 指令（蓝色）更新，但状态中保存的仍是第二条 `irmovq` 指令设置的值（浅灰色）。

当时钟上升、周期 4 开始时（点 3），对程序计数器、寄存器堆和条件码寄存器的更新发生了，因此我们把它们显示为蓝色，而组合逻辑尚未对这些变化做出反应，所以显示为白色。在这个周期里，`je` 指令（清单第 4 行，深灰色）被取出并执行。由于条件码 ZF 为 0，分支不发生。到周期结束时（点 4），程序计数器得到了新值 `0x01f`。组合逻辑已按照 `je` 指令（深灰色）更新，但状态中保存的仍是 `addq` 指令设置的值（蓝色），直到下一个周期开始。

正如此例所示，用时钟控制状态元件的更新，再加上值在组合逻辑中的传播，就足以控制我们的 SEQ 实现中对每条指令执行的计算。时钟每次从低电平转变到高电平，处理器就开始执行一条新的指令。

before any instruction attempts to read them.

Figure 4.25 shows how the SEQ hardware would process the instructions at lines 3 and 4 in the following code sequence, shown in assembly code with the instruction addresses listed on the left:

图 4.25

4.11 SEQ 各阶段的实现

在本节中，我们为实现 SEQ 所需的控制逻辑块设计 HCL 描述。SEQ 的完整 HCL 描述见网络边栏 `arch:hcl`。我们在这里展示一些示例块，其余的留作练习。我们建议你完成这些练习，以此检验自己是否理解了各块与不同指令的计算需求之间的关系。

SEQ 的 HCL 描述中我们没有在此给出的部分，是对各种整数信号和布尔信号的定义，它们可用作 HCL 运算的实参。其中包括各个硬件信号的名称，以及对应不同指令码、功能码、寄存器名、ALU 运算和状态码的常量值。只有控制逻辑中必须显式引用的那些才会出现。我们使用的常量记录在图 4.26 中。按照惯例，常量值用大写名称。

图 4.26　HCL 描述中使用的常量值。这些值表示指令、功能码、寄存器 ID、ALU 运算和状态码的编码。

名称	值(十六进制)	含义
IHALT	0	halt 指令的码
INOP	1	nop 指令的码
IRRMOVQ	2	rrmovq 指令的码
IIRMOVQ	3	irmovq 指令的码
IRMMOVQ	4	rmmovq 指令的码
IMRMOVQ	5	mrmovq 指令的码
IOPL	6	整数运算指令的码
IJXX	7	跳转指令的码
ICALL	8	call 指令的码
IRET	9	ret 指令的码
IPUSHQ	A	pushq 指令的码
IPOPQ	B	popq 指令的码
FNONE	0	默认功能码
RESP	4	%rsp 的寄存器 ID
RNONE	F	表示不访问寄存器堆
ALUADD	0	加法运算的功能码
SAOK	1	正常操作的状态码
SADR	2	地址异常的状态码
SINS	3	非法指令异常的状态码
SHLT	4	halt 的状态码

除了图 4.18 至图 4.21 中给出的指令，我们还加上对 nop 和 halt 指令的处理。nop 指令只是流过各个阶段，除了把 PC 加 1 之外几乎没有别的处理。halt 指令将处理器状态设置为 HLT，使它停止运作。

取指阶段

如图 4.27 所示，取指阶段包括指令存储器硬件单元。这个单元每次从存储器读出 10 个字节，用 PC 作为第一个字节(字节 0)的地址。这个字节被解释为指令字节，并由标为“Split”的单元拆分成两个 4 位的量。标为“icode”和“ifun”的控制逻辑块随后计算指令码和功能码：它们要么等于从存储器读出的值，要么在指令地址无效(由信号 imem_error 指示)时，等于与 nop 指令对应的值。根据 icode 的值，可以计算三个 1 位信号(显示为虚线)：

instr_valid。　这个字节对应的是合法的 Y86-64 指令吗？这个信号用来检测非法指令。

need_regids。　这条指令包含寄存器说明符字节吗？

need_valC。　这条指令包含常数字吗？

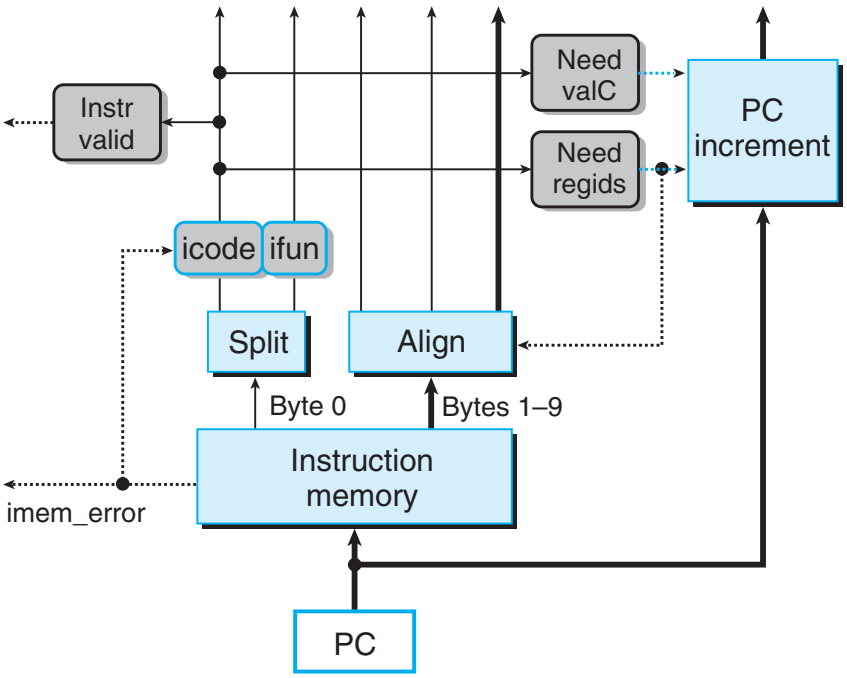


图 4.27 SEQ 的取指阶段。以 PC 为起始地址，从指令存储器中读出六个字节，并根据这些字节生成各种指令字段。PC 增量块计算信号 valP。

信号 instr_valid 和 imem_error（指令地址越界时产生）用来在访存阶段生成状态码。

作为一个例子，need_regids 的 HCL 描述只是判断 icode 的值是不是带有寄存器说明符字节的那几类指令之一：

```
bool need_regids =
        icode in { IRRMOVQ, IOPQ, IPUSHQ, IPOPQ,
                   IIRMOVQ, IRMMOVQ, IMRMOVQ };
```

练习 4.19 请为 SEQ 实现中的信号 need_valC 编写 HCL 代码。

如图 4.27 所示，从指令存储器读出的其余 9 个字节编码了寄存器说明符字节与常数字的某种组合。这些字节由标为“Align”的硬件单元处理成寄存器域和常数字。当算出的信号 need_regids 为 1 时，字节 1 被拆分成寄存器说明符 rA 和 rB。如果 need_regids 为 0，两个寄存器说明符都被设为 0xF（RNONE），表示这条指令没有指明任何寄存器。还要回忆（图 4.2），对任何只带一个寄存器操作数的指令，寄存器说明符字节的另一个域都是 0xF（RNONE）。因此可以认定，信号 rA 和 rB 要么编码了我们要访问的寄存器，要么表明不需要访问寄存器。标为“Align”的单元还生成常数字 valC。根据信号 need_regids 的值，它要么是字节 1–8，要么是字节 2–9。

PC 增量器硬件单元根据 PC 的当前值以及 need_regids 和 need_valC 两个信号生成信号 valP。对 PC 值 p、need_regids 值 r 和 need_valC 值 i，增量器生成的值为 $p+1+r+8i$。

译码阶段和写回阶段

图 4.28 详细展示了实现 SEQ 的译码阶段和写回阶段的逻辑。这两个阶段合在一起介绍，是因为它们都要访问寄存器堆。

Decode and Write-Back Stages

Figure 4.28 provides a detailed view of logic that implements both the decode and write-back stages in SEQ. These two stages are combined because they both access the register file.

The register file has four ports. It supports up to two simultaneous reads (on

图 4.28

寄存器堆有 4 个端口。它支持最多两个同时进行的读（端口 A 和 B）和两个同时进行的写（端口 E 和 M）。每个端口既有地址连接又有数据连接，其中地址连接是一个寄存器 ID，数据连接是一捆 64 根导线，用作寄存器堆的输出字（对读端口）或输入字（对写端口）。两个读端口的地址输入是 srcA 和 srcB，两个写端口的地址输入是 dstE 和 dstM。任一地址端口上的特殊标识 0xF（RNONE）表示不访问任何寄存器。

图 4.28 底部的四个块根据指令码 icode、寄存器说明符 rA 和 rB，可能还有执行阶段算出的条件信号 Cnd，为寄存器堆生成四个不同的寄存器 ID。寄存器 ID srcA 表明应当读哪个寄存器来生成 valA。想要的值取决于指令类型，如图 4.18 至图 4.21 中译码阶段的第一行所示。把所有这些条目合并成一个计算，得到 srcA 的如下 HCL 描述（回忆 RESP 是%rsp 的寄存器 ID）：

```
word srcA = [
        icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ  } : rA;
        icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # 不需要寄存器
];
```

练习 4.20 寄存器信号 srcB 表明应当读哪个寄存器来生成信号 valB。想要的值显示为图 4.18 至图 4.21 中译码阶段的第二步。请为 srcB 编写 HCL 代码。

寄存器 ID dstE 表明写端口 E 的目的寄存器，计算出的值 valE 存放在这里。这显示为图 4.18 至图 4.21 中写回阶段的第一步。如果暂时不考虑条件传送指令，那么我们可以把各种不同指令的目的寄存器合并起来，得到 dstE 的如下 HCL 描述：

```
# 警告：条件传送没有在这里正确实现
```

```
word dstE = [
        icode in { IRRMOVQ } : rB;
        icode in { IIRMOVQ, IOPQ} : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE; # 不写任何寄存器
];
```

当我们考察执行阶段时,再回来讨论这个信号以及如何实现条件传送。

练习 4.21 寄存器 ID dstM 表明写端口 M 的目的寄存器,从存储器读出的值 valM 存放在这里。这显示为图 4.18 至图 4.21 中写回阶段的第二步。请为 dstM 编写 HCL 代码。

练习 4.22 只有 popq 指令会同时使用寄存器堆的两个写端口。对指令 popq %rsp 而言,E 和 M 两个写端口将使用同一个地址,但数据不同。为了处理这个冲突,必须在两个写端口之间建立一种优先次序(priority),使得当两者要在同一个周期里写同一个寄存器时,只有来自较高优先级端口的写操作生效。要实现练习 4.8 所确定的想要的行为,两个端口中哪一个应当被赋予优先权?

执行阶段

执行阶段包括算术/逻辑单元(ALU)。这个单元依据 alufun 信号的设置,对输入 aluA 和 aluB 执行加、减、与或异或运算。这些数据信号和控制信号由三个控制块生成,如图 4.29 所示。ALU 的输出成为信号 valE。

在图 4.18 至图 4.21 中,每条指令的 ALU 计算显示为执行阶段的第一步。操作数把 aluB 列在 aluA 之前,以确保 subq 指令从 valB 中减去 valA。可以看到,aluA 的值可以是 valA、valC,也可以是 −8 或 +8,取决于指令类型。因此,我们可以把生成 aluA 的控制块的行为表述如下:

```
word aluA = [
        icode in { IRRMOVQ, IOPQ } : valA;
        icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ } : valC;
        icode in { ICALL, IPUSHQ } : -8;
        icode in { IRET, IPOPQ } : 8;
        # 其他指令不需要 ALU
];
```

练习 4.23 根据图 4.18 至图 4.21 中执行阶段第一步的第一个操作数,为 SEQ 中的信号 aluB 编写一段 HCL 描述。

看看执行阶段中 ALU 执行的运算,可以发现它大多被用作加法器。不过对于 OPq 指令,我们希望它使用指令 ifun 域中编码的运算。因此,ALU 控制的 HCL 描述可以写成如下:

```
word alufun = [
```

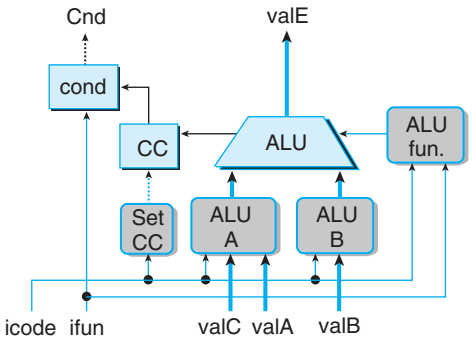


Figure 4.29
SEQ execute stage. The ALU either performs the operation for an integer operation instruction or acts as an adder. The condition code registers are set according to the ALU value. The condition code values are tested to determine whether a branch should be taken.

图 4.29

```
        icode == IOPQ : ifun;
        1 : ALUADD;
];
```

执行阶段还包括条件码寄存器。我们的 ALU 每次运算都会生成条件码所依据的三个信号——零、符号和溢出。不过，我们只想在执行 OPq 指令时设置条件码。因此我们生成一个信号 set_cc，用来控制条件码寄存器是否应当更新：

```
bool set_cc = icode in { IOPQ };
```

标为“cond”的硬件单元把条件码与功能码结合起来，判定应当进行条件分支还是条件数据传送（图 4.3）。它生成信号 Cnd，这个信号既用于条件传送时对 dstE 的设置，也用于条件分支的下一 PC 逻辑。对其他指令，Cnd 信号可能被设为 1 或 0，取决于该指令的功能码和条件码的设置，但它会被控制逻辑忽略。我们省略这个单元的详细设计。

练习 4.24 条件传送指令（缩写 cmovXX）的指令码是 IRRMOVQ。如图 4.28 所示，只要利用执行阶段生成的信号 Cnd，就可以实现这些指令。请修改 dstE 的 HCL 代码来实现它们。

访存阶段

访存阶段的任务是读出或写入程序数据。如图 4.30 所示，两个控制块生成访存地址和存储器输入数据（对写操作）的值。另外两个块生成控制信号，指明要执行读操作还是写操作。执行读操作时，数据存储器生成值 valM。

每种指令类型想要的访存操作显示在图 4.18 至图 4.21 的访存阶段中。可以看到，存储器读和写的地址总是 valE 或 valA。我们可以用 HCL 把这个块描述如下：

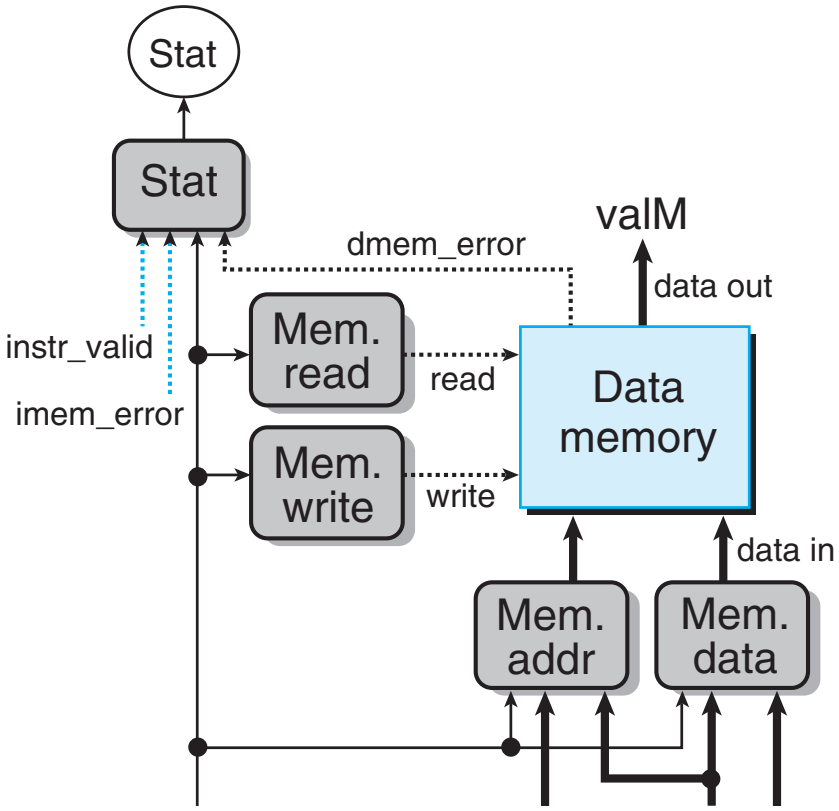


图 4.30 SEQ 的访存阶段。数据存储器可以写入或读出存储器的值。从存储器读出的值形成信号 valM。

```
word mem_addr = [
        icode in { IRMMOVQ, IPUSHQ, ICALL, IMRMOVQ } : valE;
        icode in { IPOPQ, IRET } : valA;
        # 其他指令不需要地址
];
```

练习 4.25 看看图 4.18 至图 4.21 中不同指令的访存操作，可以发现存储器写的数据总是 valA 或 valP。请为 SEQ 中的信号 mem_data 编写 HCL 代码。

我们只想为从存储器读数据的那些指令设置控制信号 mem_read，如下面的 HCL 代码所表达的：

```
bool mem_read = icode in { IMRMOVQ, IPOPQ, IRET };
```

练习 4.26 我们只想为向存储器写数据的那些指令设置控制信号 mem_write。请为 SEQ 中的信号 mem_write 编写 HCL 代码。

访存阶段的最后一个功能，是根据取指阶段生成的 icode、imem_error 和 instr_valid 的值，以及数据存储器生成的信号 dmem_error 的值，计算指令执行产生的状态码 Stat。

练习 4.27　请为 Stat 编写 HCL 代码，生成四个状态码 SAOK、SADR、SINS 和 SHLT（见图 4.26）。

更新 PC 阶段

SEQ 的最后一个阶段生成程序计数器的新值（见图 4.31）。如图 4.18 至图 4.21 的最后一步所示，新的 PC 将是 valC、valM 或 valP，取决于指令类型以及是否应当发生分支。这个选择可以用 HCL 描述如下：

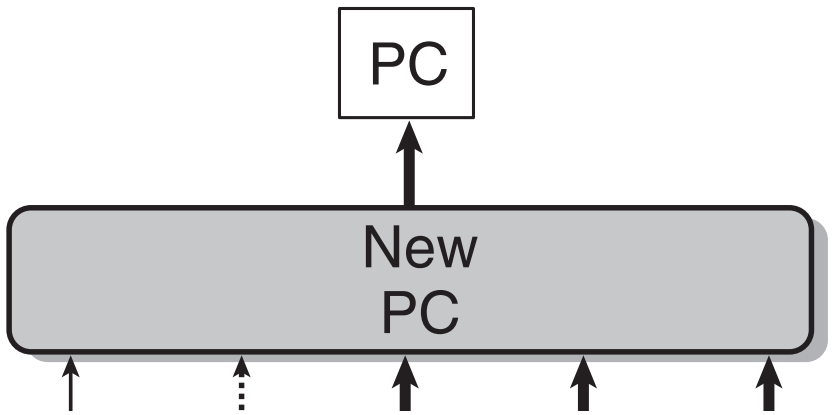


图 4.31　SEQ 的 PC 更新阶段。根据指令代码和分支标志，从信号 valC、valM 和 valP 中选出 PC 的下一个值。

```
word new_pc = [
        # call。使用指令中的常量
        icode == ICALL : valC;
        # 分支发生。使用指令中的常量
        icode == IJXX && Cnd : valC;
        # ret 指令执行完毕。使用从栈中读出的值
        icode == IRET : valM;
        # 默认:使用增量后的 PC
        1 : valP;
];
```

SEQ 概览

至此，我们已经完整地走过了 Y86-64 处理器的一个设计。我们看到，把执行各条不同指令所需的步骤组织成一个统一的流程，就能用少量不同的硬件单元实现整个处理器，并且只用一个时钟来控制计算的定序。于是，控制逻辑要做的是在这些单元之间路由信号，并根据指令类型和分支条件生成正确的控制信号。

SEQ 唯一的问题在于它太慢。时钟必须运行得足够慢，使信号能够在一个周期之内穿过所有阶段。以一条 ret 指令的处理为例。从时钟周期开始时已更新的程序计数器出发，必须先从指令存储器中读出指令，再从寄存器堆中读

出栈指针，ALU 将栈指针加 8，然后从存储器中读出返回地址，以确定程序计数器的下一个值。所有这些都必须在时钟周期结束之前完成。

这种实现方式对我们硬件单元的利用并不好，因为每个单元在总时钟周期里只有一小段时间是活跃的。我们将会看到，引入流水线可以获得好得多的性能。

4.12 流水线的通用原理

在着手设计流水线化的 Y86-64 处理器之前，我们先考虑一下流水线系统的一些一般性质和原理。凡是排过自助餐厅取餐的队，或者用自动洗车机洗过车的人，对这样的系统都不会陌生。在流水线化的系统中，要执行的任务被划分成一系列离散的阶段。在自助餐厅，这指的是供应沙拉、主菜、甜点和饮料；在洗车房，这指的是喷水和皂液、刷洗、打蜡和烘干。我们不让一位顾客从头到尾走完全部流程、下一位顾客才能开始，而是允许多位顾客同时在这个系统中行进。在传统的自助餐厅取餐线上，顾客在流水线中保持相同的顺序，走完所有阶段，即使有些菜品他们并不想要。对洗车房来说，当前面一辆车从喷淋阶段移到刷洗阶段时，就允许一辆新车进入喷淋阶段。一般说来，各辆车必须以相同的速度在系统中移动，以免一辆车撞上另一辆。

流水线化的一个关键特点是它能提高系统的吞吐量（throughput，即单位时间服务的顾客数），但也会略微增加延迟（latency，即服务单个顾客所需的时间）。例如，自助餐厅里只要一份甜点的顾客，在未流水线化的系统中可以很快通过，只在甜点台前停一下；而在流水线化的系统中，试图直奔甜点台的顾客很可能招致其他顾客的怒火。

4.13 计算流水线

把注意力转到计算流水线上，“顾客”就是指令，各个阶段执行指令处理的一部分。图 4.32(a) 展示了一个简单的非流水线化硬件系统的例子。它由执行某种计算的逻辑和一个保存这次计算结果的寄存器组成。时钟信号按固定的时间间隔控制寄存器的装入。这种系统的一个例子是光盘（CD）播放机中的译码器。输入的信号是从 CD 表面读出的位，逻辑把这些位译码，生成音频信号。图中的计算块实现为组合逻辑，即信号会穿过一系列逻辑门，经过一段时间延迟后，输出成为输入的某种函数。

在当代逻辑设计中，电路延迟以皮秒（picosecond，缩写“ps”）为单位计量，即 10^{-12} 秒。在这个例子中，我们假设组合逻辑需要 300 ps，而寄存器的装入需要 20 ps。图 4.32 展示了时序图的一种形式，称为流水线图（pipeline diagram）。

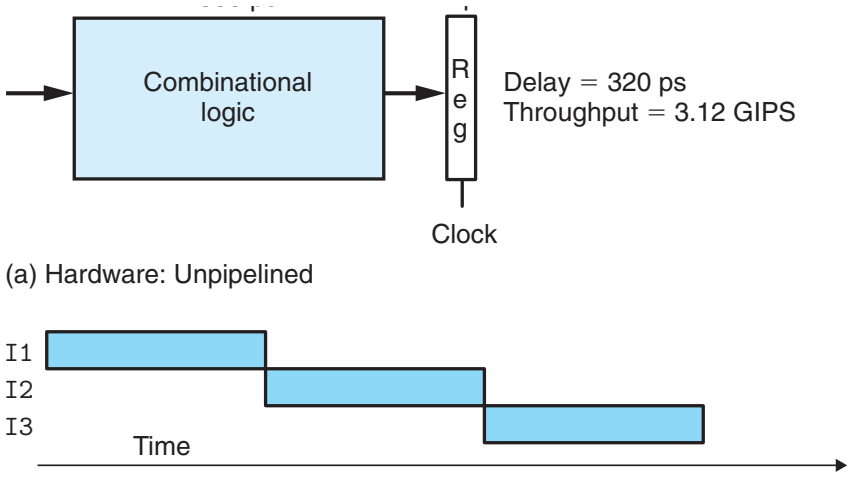


图 4.32 非流水线的计算硬件。在每个 320 ps 的周期中，系统用 300 ps 计算一个组合逻辑函数，用 20 ps 将结果保存到输出寄存器中。

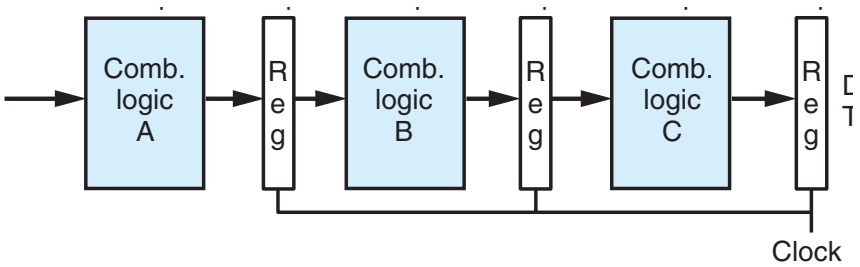
在这种图中，时间从左向右流动。一系列指令（这里记为 I1、I2 和 I3）自上而下写出。实心矩形指明这些指令被执行的时段。在这种实现中，必须完成一条指令才能开始下一条，因此各个方框在垂直方向上互不重叠。下面的公式给出了我们能够运行这个系统的最大速率：

$$\text{吞吐量} = \frac{1\text{ 条指令}}{(20+300)\text{ 皮秒}} \cdot \frac{1{,}000\text{ 皮秒}}{1\text{ 纳秒}} \approx 3.12\text{ GIPS}$$

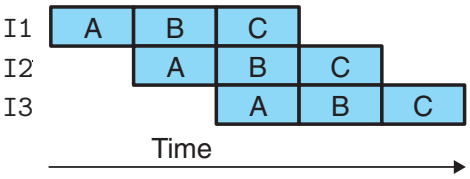
我们用每秒十亿条指令（giga-instructions per second，缩写 GIPS）作为吞吐量的单位。从头到尾执行一条指令所需的总时间称为延迟（latency）。在这个系统中，延迟是 320 ps，即吞吐量的倒数。

假设我们可以把系统执行的计算划分成三个阶段 A、B 和 C，每个阶段需要 100 ps，如图 4.33 所示。于是我们可以在各阶段之间放置流水线寄存器（pipeline register），使每条指令以三步流过系统，从头到尾需要三个完整的时钟周期。如图 4.33 的流水线图所示，只要 I1 从阶段 A 移到阶段 B，就可以让 I2 进入阶段 A，依此类推。在稳态（steady state）下，所有三个阶段都是活跃的，每个时钟周期都有一条指令离开这个系统，同时有一条新指令进入这个系统。在流水线图中我们可以看到这一点：第三个时钟周期里，I1 处于阶段 C，I2 处于阶段 B，I3 处于阶段 A。

在这样的系统中，我们可以让时钟每 100+20 = 120 皮秒变动一次，得到约 8.33 GIPS 的吞吐量。由于处理一条指令需要 3 个时钟周期，这条流水线的延迟是 3 × 120 = 360 皮秒。我们以增加一些硬件、延迟略有增加（360/320 = 1.12）

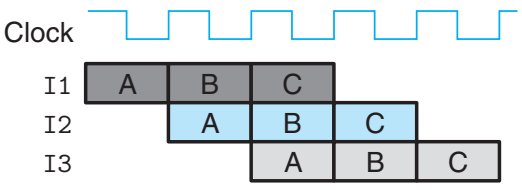


(a) Hardware: Three-stage pipeline



(b) Pipeline diagram

Figure 4.33 **Three-stage pipelined computation hardware** into stages A, B, and C. On each 120 ps cycle, each instructio stage.



ıe
dge of
·ols the
ctions

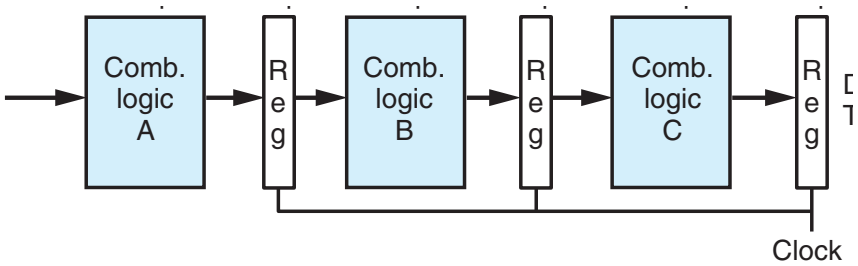
图 4.33　三级流水线计算硬件。计算被划分为阶段 A、B 和 C。在每个 120 ps 的周期中，每条指令向前推进一个阶段。

为代价，把系统的吞吐量提高了 $8.33/3.12 = 2.67$ 倍。延迟的增加，源于新添的流水线寄存器带来的时间开销。

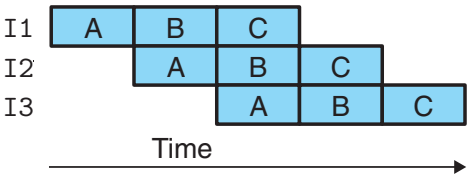
4.14 流水线操作的详细考察

为了更好地理解流水线的工作方式，我们来细致考察流水线计算的时间安排和操作。图 4.34 给出了我们已经见过的三级流水线（图 4.33）的流水线图。指令在流水线阶段之间的传送由时钟信号控制，如图中流水线图上方所示。每 120 皮秒，该信号从 0 上升到 1，触发下一组流水线阶段计算。

图 4.35 追踪了时刻 240 与 360 之间的电路活动，此时指令 I1（深灰色显示）正在穿过阶段 C，I2（蓝色显示）正在穿过阶段 B，I3（浅灰色显示）正在穿过阶段



(a) Hardware: Three-stage pipeline



(b) Pipeline diagram

Figure 4.33 Three-stage pipelined computation hardware
into stages A, B, and C. On each 120 ps cycle, each instructio
stage.

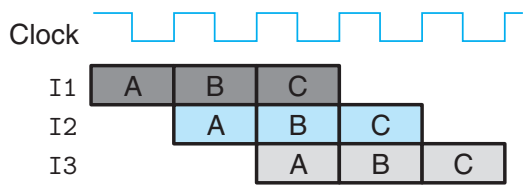
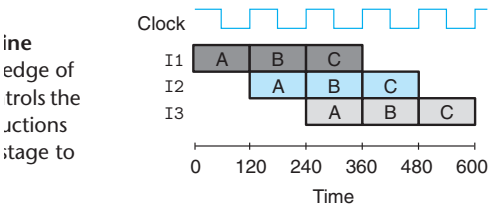


图 4.34 三级流水线时序。时钟信号的上升沿控制指令从一个流水线阶段向下一个阶段的移动。

A。就在时刻 240 的时钟上升之前(点 1),阶段 A 为指令 I2 算出的值已经到达第一个流水线寄存器的输入,但该寄存器的状态和输出仍保持为阶段 A 为指令 I1 算出的值。阶段 B 为指令 I1 算出的值已经到达第二个流水线寄存器的输入。随着时钟上升,这些输入被装入流水线寄存器,成为寄存器的输出(点 2)。同时,阶段 A 的输入被设置好,以开始对指令 I3 的计算。接下来,信号经过各阶段相应的组合逻辑传播(点 3)。正如点 3 处图中弯曲的波前所暗示的,信号在电路各个不同部分中的传播速度可以不同。在时刻 360 之前,各条指令的结果值已经到达流水线寄存器的输入(点 4)。当时钟在时刻 360 上升时,每条指令都已前进了一个流水线阶段。



this system, we could cycle the clocks every $100 + 20 = 120$ picoseconds, giving a throughput of around 8.33 GIPS. Since processing a single instruction requires 3 clock cycles, the latency of this pipeline is $3 \times 120 = 360$ ps. We have increased the throughput of the system by a factor of $8.33/3.12 = 2.67$ at the expense of some added hardware and a slight increase in the latency ($360/320 = 1.12$). The increased latency is due to the time overhead of the added pipeline registers.

4.4.2 A Detailed Look at Pipeline Operation

To better understand how pipelining works, let us look in some detail at the timing and operation of pipeline computations. Figure 4.34 shows the pipeline diagram for the three-stage pipeline we have already looked at (Figure 4.33). The transfer of the instructions between pipeline stages is controlled by a clock signal, as shown above the pipeline diagram. Every 120 ps, this signal rises from 0 to 1, initiating the next set of pipeline stage evaluations.

Figure 4.35 traces the circuit activity between times 240 and 360, as instruction `I1` (shown in dark gray) propagates through stage C, `I2` (shown in blue)

图 4.35

从流水线操作的这一细致视角可以看出，放慢时钟并不会改变流水线的行为。信号会传播到流水线寄存器的输入，但在时钟上升之前，寄存器状态不会发生任何变化。反过来，如果时钟跑得太快，就可能带来灾难性后果。值还来不及在组合逻辑中传播完，当时钟上升时寄存器的输入尚未有效。

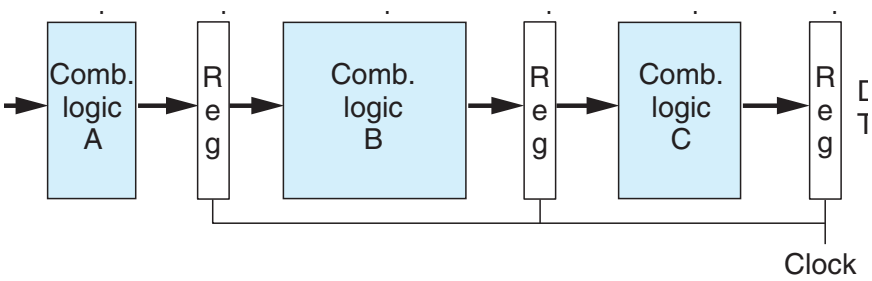
与我们对 SEQ 处理器时序的讨论（4.3.3 节）一样，这里也看到：在组合逻辑的各部分之间放置由时钟控制的寄存器，这种简单的机制就足以控制流水线中指令的流动。随着时钟反复上升与下降，不同的指令在流水线各阶段之间流动，互不干扰。

4.15 流水线的局限

图 4.33 的例子给出了一个理想的流水线系统：我们可以把计算划分成三个相互独立的阶段，每个阶段需要的时间只为原逻辑的三分之一。可惜，常常会出现其他因素削弱流水线化的效果。

非均匀划分

图 4.36 给出的系统像前面一样把计算划分成三个阶段，但各阶段的延迟从 50 到 150 皮秒不等。所有阶段延迟之和仍然是 300 皮秒。然而，时钟的运行速率要受最慢阶段延迟的限制。如图中的流水线图所示，阶段 A 每个时钟周期都会空闲（图中显示为白色方框）100 皮秒，而阶段 C 每个时钟周期空闲 50 皮秒，只有阶段 B 持续工作。我们必须把时钟周期设为 150 + 20 = 170 皮秒，得到 5.88 GIPS 的吞吐量。此外，由于时钟频率变慢，延迟会上升到 510 皮秒。



Hardware: Three-stage pipeline, nonuniform stage delays

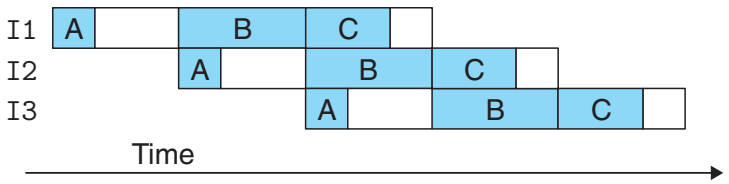


图 4.36　流水线因阶段延迟不均匀而受到的限制。系统吞吐量受最慢阶段速度的限制。

把对整个系统的计算进行划分、使各阶段延迟均匀，对硬件设计者来说可能是一项重大挑战。处理器中的某些硬件单元（例如算术/逻辑单元和存储器）往往无法再分解成延迟更短的多个单元，这就很难造出一组均衡的阶段。在设计我们的流水线化 Y86-64 处理器时，我们不会考虑这么细的程度，但认识到时序优化在实际系统设计中的重要性，这一点很关键。

练习 4.28　假设我们分析图 4.32 的组合逻辑，发现它可以划分成由块 A 到 F 组成的序列，各块的延迟分别为 80、30、60、50、70 和 10 皮秒，如下图所示：

Decode and Write-Back Stages

Figure 4.28 provides a detailed view of logic that implements both the decode and write-back stages in SEQ. These two stages are combined because they both access the register file.

The register file has four ports. It supports up to two simultaneous reads (on

图 4.28

我们在这些块两两之间插入流水线寄存器，就能得到这一设计的流水线化版本。根据插入流水线寄存器的位置不同，会得到流水线深度（即阶段数）与最大吞吐量之间的不同组合。假设一个流水线寄存器的延迟为 20 皮秒。

A. 插入单个寄存器就得到一个两级流水线。应该把寄存器插在哪里才能使吞吐量最大？吞吐量和延迟各是多少？

B. 应该把两个寄存器插在哪里，才能使三级流水线的吞吐量最大？吞吐量和延迟各是多少？

C. 应该把三个寄存器插在哪里，才能使四级流水线的吞吐量最大？吞吐量和延迟各是多少？

D. 至少需要多少个阶段，才能得到具有最大可达吞吐量的设计？描述该设计及其吞吐量和延迟。

深流水线的收益递减

图 4.37 揭示了流水线化的另一个局限。这个例子中，我们将计算划分成六个阶段，每个阶段需要 50 皮秒，在每对阶段之间插入一个流水线寄存器，就得到一个六级流水线。该系统的最小时钟周期是 $50 + 20 = 70$ 皮秒，吞吐量为 14.29 GIPS。于是，阶段数翻了一倍，性能却只提高 $14.29/8.33 = 1.71$ 倍。尽管每个计算块所需的时间已缩短为原来的二分之一，由于穿过流水线寄存器的延迟，吞吐量并没有翻倍。这一延迟成了流水线吞吐量的限制因素。在我们的新设计中，它占总时钟周期的 28.6%。

现代处理器采用非常深的流水线（15 个或更多阶段），力图把处理器的时钟频率提到最大。处理器体系结构设计师把指令的执行划分成大量非常简单的步骤，使每个阶段的延迟都很小。电路设计者精心设计流水线寄存器，把它们的延迟降到最低。芯片设计者还必须精心设计时钟分配网络，确保时钟在整个芯片上精确地同时变化。所有这些因素加在一起，构成了设计高速微处理器的挑战。

练习 4.29 假设我们可以把图 4.32 的系统划分成任意数目 k 个流水线阶段，每个阶段的延迟为 $300/k$，每个流水线寄存器的延迟为 20 皮秒。

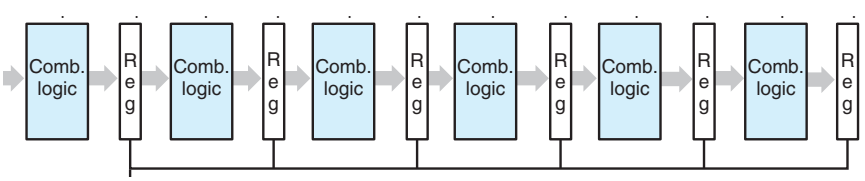


图 4.37　流水线因开销而受到的限制。随着组合逻辑被划分成更短的块，寄存器更新造成的延迟成为限制因素。

A. 作为 k 的函数，系统的延迟和吞吐量各是多少？

B. 吞吐量的最终上限是多少?

4.16 带有反馈的系统的流水线化

到目前为止,我们考虑的都只是这样的系统:流经流水线的对象——无论是汽车、人还是指令——彼此完全独立。但是,对于执行 x86-64 或 Y86-64 这类机器程序的系统来说,相邻指令之间可能存在相关性。例如,考虑下面的 Y86-64 指令序列:

```
1  irmovq $50, %rax
2  addq   %rax, %rbx
3  mrmovq 100(%rbx), %rdx
```

在这个三条指令的序列中,每相邻的指令对之间都存在数据相关性,如圈出的寄存器名和它们之间的箭头所示。`irmovq` 指令(第 1 行)把结果存入`%rax`,这个值接下来必须由 `addq` 指令(第 2 行)读取;而这条指令把结果存入`%rbx`,它接下来又必须由 `mrmovq` 指令(第 3 行)读取。

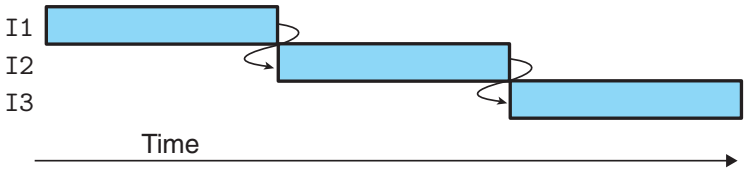
另一类顺序相关性源于指令的控制流。考虑下面的 Y86-64 指令序列:

```
1  loop:
2      subq %rdx,%rbx
3      jne targ
4      irmovq $10,%rdx
5      jmp loop
6  targ:
7  halt
```

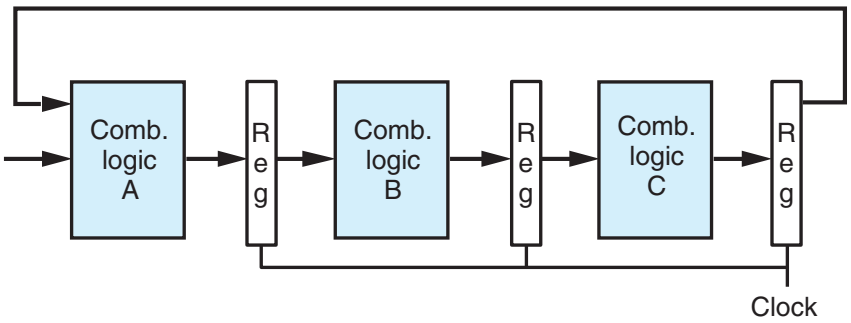
`jne` 指令(第 3 行)造成一条控制相关性,因为条件测试的结果决定了下一条要执行的指令是 `irmovq` 指令(第 4 行)还是 `halt` 指令(第 7 行)。在 `SEQ` 的设计中,这些相关性由图 4.22 右侧的反馈路径处理。这一反馈把更新过的寄存器值送回寄存器堆,并把新的 PC 值送回 PC 寄存器。

图 4.38 展示了把流水线化引入含反馈路径的系统所带来的危险。在原系统(图 4.38(a))中,每条指令的结果被反馈给下一条指令。流水线图(图 4.38(b))说明了这一点:I1 的结果成为 I2 的输入,以此类推。如果我们试图以最直接的方式把它转换成三级流水线(图 4.38(c)),就会改变系统的行为。如图 4.38(c)所示,I1 的结果变成了 I4 的输入。我们试图通过流水线化加速系统,却改变了系统的行为。

向 Y86-64 处理器引入流水线化时,必须正确处理反馈效应。显然,像图 4.38 的例子中那样改变系统行为是不可接受的。我们必须以某种方式处理指令之间的数据相关性和控制相关性,使最终的行为与 ISA 定义的模型一致。



(b) Pipeline diagram



(c) Hardware: Three-stage pipeline with feedback

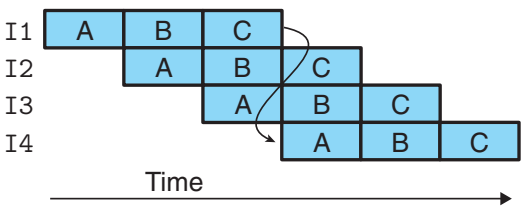


图 4.38 流水线因逻辑依赖而受到的限制。从带反馈的非流水线系统 (a) 改为流水线系统 (c) 时，其计算行为发生了改变，从两个流水线图(b 和 d)可以看出。

4.17 Y86-64 的流水线实现

我们终于可以迎接本章的主要任务了——设计一个流水线化的 Y86-64 处理器。我们首先对顺序处理器 SEQ 作一个小改动，把 PC 的计算移入取指阶段。然后在各阶段之间加入流水线寄存器。最初的努力并不能正确处理各种数据相关性和控制相关性。不过，只需稍作修改，就能实现目标——一个高效实现了 Y86-64 ISA 的流水线化处理器。

4.18 SEQ+：重新排列计算阶段

作为通向流水线设计的过渡步骤，我们略微调整 SEQ 中五个阶段的顺序，让 PC 更新阶段出现在时钟周期的开头，而不是结尾。这种变换只需对整体硬

件结构作极小的改动，而且它与流水线各阶段内部活动的时序配合得更好。我们把这个修改后的设计称为 SEQ+。

要让 PC 更新阶段的逻辑在时钟周期开头就处于活动状态，可以让它计算当前指令的 PC 值。图 4.39 展示了 SEQ 与 SEQ+ 在 PC 计算上的区别。采用 SEQ（图 4.39(a)）时，PC 计算发生在时钟周期的末尾，根据当前时钟周期内算出的各信号的值为 PC 寄存器计算新值。采用 SEQ+（图 4.39(b)）时，我们创建状态寄存器来保存一条指令执行期间算出的信号。于是，随着新时钟周期开始，这些值流经完全相同的逻辑，为现在这条指令计算 PC。我们把寄存器标记为 pIcode、pCnd 等，表示在任一时钟周期中，它们保存的都是上一个周期生成的控制信号。

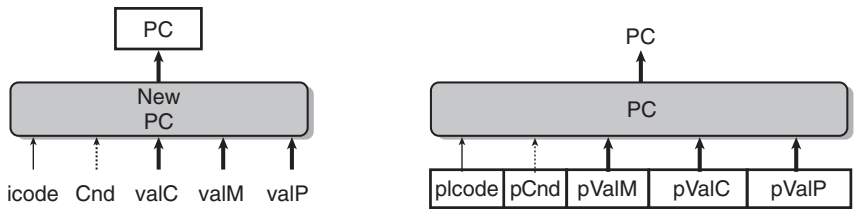


图 4.39　将 PC 计算的时序前移。在 SEQ+ 中，把当前状态下程序计数器的值作为指令执行的第一步来计算。

边栏　SEQ+ 中 PC 在哪里？

SEQ+ 有一个奇特之处：没有存放程序计数器的硬件寄存器，PC 是根据前一条指令保存下来的状态信息动态计算出来的。这只是一个小例子，说明只要处理器能正确执行任意的机器语言程序，它的实现方式就可以与 ISA 所隐含的概念模型不同。我们不必按程序员可见状态所指定的形式对状态编码，只要处理器能为程序员可见状态的任何部分（例如程序计数器）生成正确的值即可。这个原则在构造流水线设计时还会得到更大程度的运用。乱序处理技术（见 5.7 节）把这个思想推到极致：指令的执行顺序可以与它们在机器级程序中出现的顺序完全不同。

图 4.40 给出了 SEQ+ 硬件更详细的视图。可以看到，它包含的硬件单元和控制块与 SEQ（图 4.23）完全相同，只是把 PC 逻辑从顶部移到了底部——从前它在时钟周期末尾处于活跃状态，现在在周期的开头。

从 SEQ 到 SEQ+ 的这些状态元素的搬移，是电路重定时（circuit retiming）[68] 这一通用变换的一个例子。重定时在不改变系统逻辑行为的前提下改变其状态表示，常常用于平衡流水线系统各阶段之间的延迟。

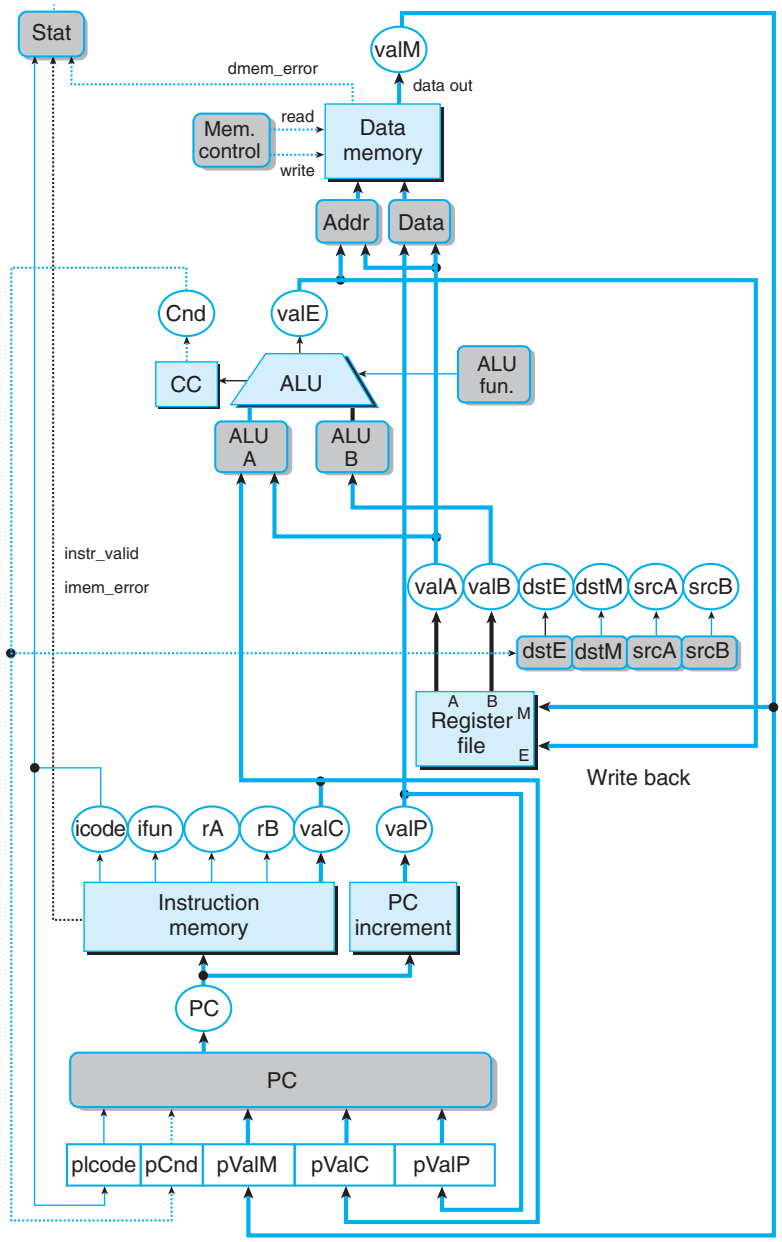


图 4.40 SEQ+ 的硬件结构。把 PC 计算从时钟周期末尾移到开头，使它更适合流水线化。

4.19 插入流水线寄存器

向流水线化 Y86-64 处理器迈出第一步时，我们在 SEQ+ 的各阶段之间插入流水线寄存器，并稍微重新排列信号，得到 PIPE- 处理器。名称中的“-”表示这个处理器的性能比我们最终的设计略差。PIPE- 的结构见图 4.41。图中流水线寄存器画成蓝色方框，每个方框内是若干用白色小框表示的域。正如多个域所显示的，每个流水线寄存器保存多个字节和字。与两个顺序处理器（图 4.23 和图 4.40）硬件结构中圆角框内的标签不同，这里的白色小框代表实际的硬件部件。

可以看到，PIPE- 使用的硬件单元与顺序设计 SEQ（图 4.40）几乎相同，只是用流水线寄存器隔开了各个阶段。两个系统中信号之间的差别将在 4.5.3 节讨论。

流水线寄存器标记如下：

F 保存程序计数器的预测值，稍后讨论。

D 位于取指阶段和译码阶段之间，保存最近取出的指令的信息，供译码阶段处理。

E 位于译码阶段和执行阶段之间，保存最近译码的指令以及从寄存器堆读出的值，供执行阶段处理。

M 位于执行阶段和访存阶段之间，保存最近执行的指令的结果，供访存阶段处理。它还保存分支条件和分支目标的信息，供条件跳转指令使用。

W 位于访存阶段与反馈路径之间；反馈路径把已经算出的结果送往寄存器堆写入，并在 ret 指令完成时把返回地址送往 PC 选择逻辑。

图 4.42 展示了下面的代码序列如何流经我们的五级流水线，注释中标出指令 I1 到 I5，以便引用：

```
1  irmovq   $1,%rax    # I1
2  irmovq   $2,%rbx    # I2
3  irmovq   $3,%rcx    # I3
4  irmovq   $4,%rdx    # I4
5  halt                # I5
```

图的右侧给出了这个指令序列的流水线图。与 4.4 节中简单流水线计算单元的流水线图一样，图中反映了每条指令在流水线各阶段中的推进情况，时间从左到右递增。顶部标出的数字确定了各个阶段的发生周期。例如，周期 1 取出指令 I1，随后它流过流水线各阶段，其结果在周期 5 结束之后写入寄存器堆。指令 I2 在周期 2 取出，其结果在周期 6 结束之后写回，依此类推。图下方给出了周期 5 的流水线详细视图，此时每个流水线阶段中各有一条指令。

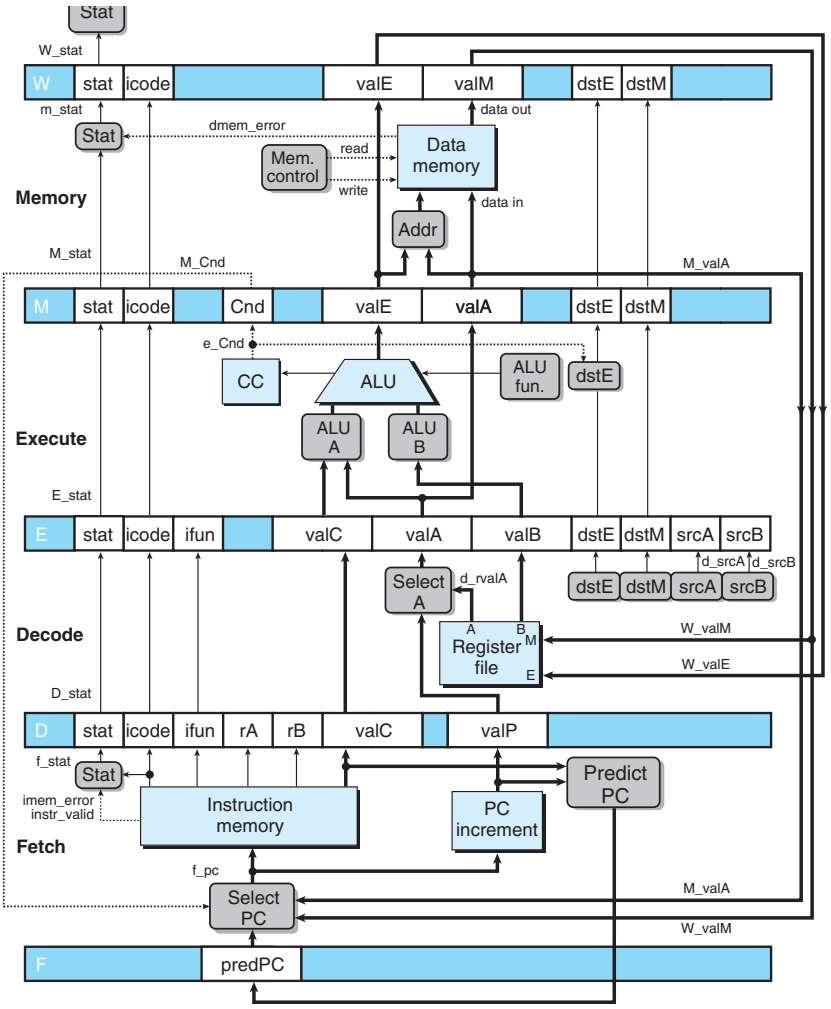


Figure 4.41 **Hardware structure of PIPE–, an initial pipelined implementation.** By inserting pipeline registers into SEQ+ (Figure 4.40), we create a five-stage pipeline. There are several shortcomings of this version that we will deal with shortly.

图 4.41

从图 4.42 还可以解释我们画处理器时让指令自下而上流动的惯例。周期 5 的局部放大视图显示的流水线阶段是取指阶段在底部、写回阶段在顶部，与流水线硬件图（图 4.41）一致。观察流水线各阶段中指令的顺序，可以看到它们与在程序清单中的顺序相同。由于正常的程序流是从清单顶部向底部进行的，我们让流水线自下而上流动，以保持这种次序。使用本书配套的模拟器时，这一惯例特别有用。

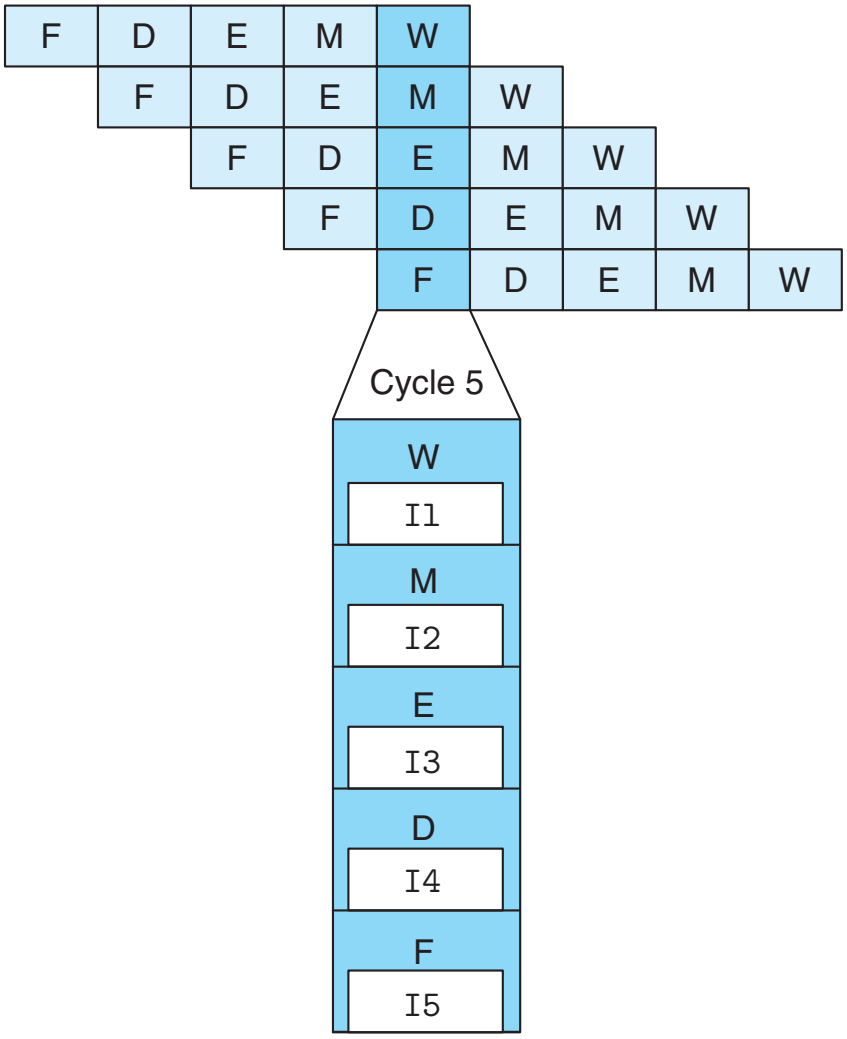


图 4.42 指令流过流水线的例子。

4.20 重新排列和标号信号

顺序实现 SEQ 和 SEQ+ 一次只处理一条指令，因此像 valC、srcA、valE 这样的信号只有唯一的值。在流水线设计中，流经系统的指令各不相同，这些值会有多个版本同时存在。例如，在 PIPE- 的详细结构中，有四个标记为“Stat”的白色小框，保存四条不同指令的状态码（见图 4.41）。我们必须格外小心，确保使用信号的正确版本，否则可能出现严重错误，例如把为一条指令算出的结果存到另一条指令指定的目的寄存器中。我们采用的命名方案是：保存在流水线寄存器中的信号，在信号名前面加上大写的流水线寄存器名作前缀，就能唯一标识。例如，四个状态码分别叫 D_stat、E_stat、M_stat 和 W_stat。我们还

需要引用一些刚刚在某个阶段中算出的信号,这类信号在信号名前面加上阶段名的首字母(小写)作前缀。仍以状态码为例,取指阶段和访存阶段中都有标记为“Stat”的控制逻辑块,这些块的输出相应地命名为 f_stat 和 m_stat。整个处理器的实际状态 Stat 则由写回阶段中的一个块根据流水线寄存器 W 中的状态值计算得出。

> **边栏 信号 M_stat 与 m_stat 有什么区别?**
> 按照我们的命名体系,大写前缀“D”“E”“M”“W”指流水线寄存器,因此 M_stat 指流水线寄存器 M 的状态码域。小写前缀“f”“d”“e”“m”“w”指流水线阶段,因此 m_stat 指访存阶段中由控制逻辑块生成的状态信号。
> 理解这一命名惯例,对理解流水线处理器的工作方式至关重要。

SEQ+ 和 PIPE- 的译码阶段都会生成信号 dstE 和 dstM,指明值 valE 和 valM 的目的寄存器。在 SEQ+ 中,可以把这些信号直接连到寄存器堆写端口的地址输入上。而在 PIPE- 中,这些信号随流水线穿过执行和访存阶段,直到到达写回阶段(见各阶段更详细的视图)才送往寄存器堆。这样做的目的是确保写端口的地址输入和数据输入来自同一条指令。否则,写回阶段所写的将是为写回阶段指令算出的值,用的却是译码阶段指令的寄存器标识符。一般原则是:关于某条特定指令的全部信息,都要保存在单个流水线阶段之内。

PIPE- 中有一个部件在 SEQ+ 中没有完全相同的形式,就是译码阶段中标记为“Select A”的块。可以看到,这个块为流水线寄存器 E 生成值 valA:要么选择流水线寄存器 D 中的 valP,要么选择从寄存器堆 A 端口读出的值。设置这个块的目的是减少必须向前传送到流水线寄存器 E 和 M 的状态数量。在所有指令中,只有 call 需要在访存阶段用到 valP,只有跳转指令需要在执行阶段用到 valP(当跳转不发生时),而这些指令都不需要从寄存器堆读出的值。因此,可以把这两个信号合并成单个信号 valA 沿流水线传递,从而减少流水线寄存器保存的状态。这也省去了 SEQ(图 4.23)和 SEQ+(图 4.40)中服务类似目的的“Data”块。在硬件设计中,通常先仔细弄清各个信号的用途,再来合并信号,以减少寄存器状态和连线数量。

如图 4.41 所示,我们的流水线寄存器中有一个保存状态码 stat 的域,它在取指阶段初步算出,并可能在访存阶段被修改。在讲完正常执行的实现之后,我们再来讨论如何实现对异常事件的处理(见 4.5.6 节)。这里只需说明,最系统的做法是让每条指令在流经流水线时都带着一个状态码,如图中所示。

4.21 预测下一个 PC

在 PIPE- 的设计中，我们已经采取了一些措施来正确处理控制相关性。流水线设计的目标是每个时钟周期发射一条新指令，即在任一时钟周期中，都有一条新指令进入执行阶段并最终完成。实现这一目标就能达到每周期一条指令的吞吐量。为此，我们必须在取出当前指令之后立即确定下一条指令的位置。遗憾的是，如果取出的指令是条件分支，要等几个周期之后、指令穿过执行阶段，我们才能知道这条分支是否应该发生转移。类似地，如果取出的指令是 ret，要等指令穿过访存阶段之后才能确定返回位置。

> **边栏 其他的分支预测策略**
> 我们的设计采用“总是发生转移”的分支预测策略。研究表明这一策略的成功率约为 60% [44, 122]。反过来，“从不发生转移”（NT）策略的成功率约为 40%。还有一种稍微复杂些的策略，称为后向发生转移、前向不发生转移（backward taken, forward not taken，BTFNT）：预测指向更低地址的分支会发生转移，指向更高地址的则不发生转移。这一策略的成功率约为 65%。它的改进来自这样的事实：循环靠后向分支闭合，而循环一般会执行多次；前向分支用于条件操作，发生转移的可能性较小。在家庭作业 4.55 和 4.56 中，你可以修改 Y86-64 流水线处理器，实现 NT 和 BTFNT 分支预测策略。
> 正如 3.6.6 节所见，分支预测错误会显著降低程序的性能，这促使我们在可能时用条件数据传送来替代条件控制传送。

除条件跳转指令和 ret 之外，我们都可以根据取指阶段计算的信息确定下一条指令的地址。对 call 和 jmp（无条件跳转），这个地址是 valC，即指令中的常数字；对其他所有指令，它是 valP，即下一条指令的地址。因此，大多数情况下，通过预测 PC 的下一个值就能实现每时钟周期发射一条指令的目标。对大多数指令类型，这一预测完全可靠。对条件跳转，我们可以预测跳转发生，此时 PC 新值为 valC；也可以预测跳转不发生，此时 PC 新值为 valP。无论哪种情况，都必须设法处理预测错误的情形——那时我们已经取出并部分执行了错误的指令。我们将在 4.5.8 节回到这个问题。

这种先猜分支方向、再按照猜想立即开始取指的技术称为分支预测。几乎所有处理器都以某种形式运用它。关于预测分支是否发生转移的有效策略，人们已经做过大量实验 [46, 2.3 节]。有些系统为此投入大量硬件。我们的设计采用简单策略：总是预测条件分支会发生转移，因此预测 PC 的新值为 valC。

还剩下 ret 指令产生的 PC 新值需要预测。与条件跳转不同，这里可能出现的结果几乎没有上限——返回地址就是栈顶的那一个字。

在我们的设计中，不尝试预测返回地址的值，而是干脆停止处理后续的所有指令，直到 ret 指令穿过写回阶段。这一部分实现将在 4.5.8 节回过头来讨论。

> **边栏　用栈预测返回地址**
> 对大多数程序来说，返回地址是很容易预测的，因为过程调用与返回总是成对出现。大多数情况下，一个过程被调用后就返回到调用指令后面的那条指令。高性能处理器利用这一性质，在取指单元中设置一个硬件栈，保存过程调用指令生成的返回地址。每当执行一条过程调用指令，其返回地址就被压入该栈。当取出一条返回指令时，就把栈顶值弹出，作为预测的返回地址。与分支预测一样，必须提供预测错误时的恢复机制，因为调用与返回并不总是匹配。总体来说，这种预测非常可靠。这个硬件栈不属于程序员可见状态。

图 4.41 底部画出的 PIPE- 取指阶段，既要预测 PC 的下一个值，又要为取指选择实际的 PC。可以看到，标记为“Predict PC”的块可以选择 valP（由 PC 增值器算出）或 valC（来自取出的指令）。这个值装入流水线寄存器 F，作为程序计数器的预测值。标记为“Select PC”的块与 SEQ+ PC 选择阶段（图 4.40）中标记为“PC”的块类似。它从三个值中选择一个作为指令存储器的地址：预测的 PC、到达流水线寄存器 M 的“不发生转移”分支指令的 valP 值（保存在 M_valA 中），或者当 ret 指令到达流水线寄存器 W 时的返回地址值（保存在 W_valM 中）。

4.22 流水线冒险

PIPE- 是构造流水线化 Y86-64 处理器的一个良好开端。不过，回想 4.4.4 节的讨论：把流水线化引入带反馈的系统之后，一旦相邻指令之间存在相关性，就会出问题。我们必须解决这个问题，才能完成设计。这些相关性有两种形式：(1) 数据相关性，即一条指令算出的结果被后面的指令用作数据；(2) 控制相关性，即一条指令决定下一条指令的位置，例如执行跳转、调用或返回时。当这类相关性可能使流水线算出错误结果时，就称为冒险（hazard）。与相关性一样，冒险也分为数据冒险和控制冒险。我们先讨论数据冒险，再考虑控制冒险。

图 4.43 展示了 PIPE- 处理器对我们称之为 prog1 的指令序列的处理。在这个例子以及随后的几个例子中，假设所有程序寄存器的初值都是 0。这段代码把值 10 和 3 分别装入程序寄存器%rdx 和%rax，执行三条 nop 指令，然后把寄存器%rdx 的值加到%rax 上。我们关注的是两条 irmovq 指令与 addq 指令之间的数据相关性可能引发的数据冒险。图的右侧给出了这一指令序列的流水线图，周期 6 和周期 7 的流水线阶段在图中以高亮显示。其下是周期 6 写回

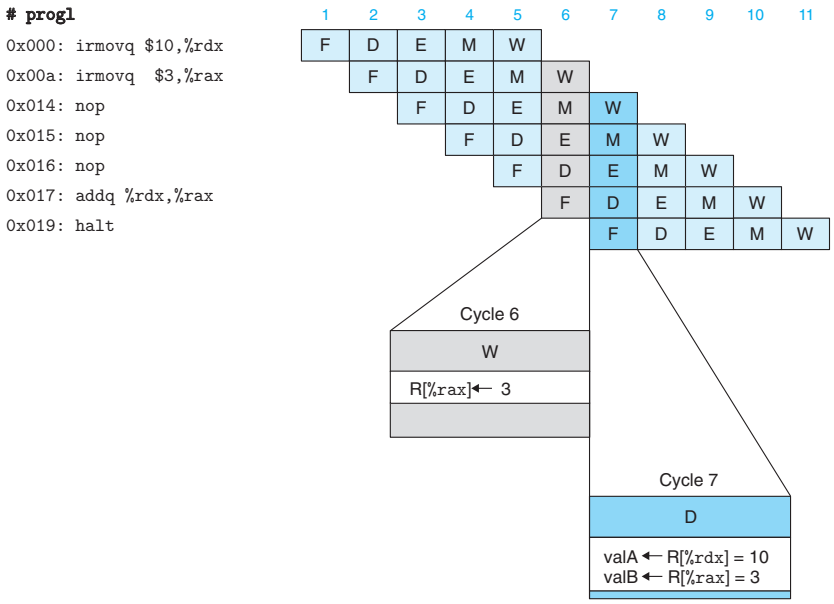


Figure 4.43 Pipelined execution of `prog1` without special pipeline control. In cycle 6, the second `irmovq` writes its result to program register `%rax`. The `addq` instruction reads its source operands in cycle 7, so it gets correct values for both `%rdx` and `%rax`.

图 4.43

活动和周期 7 译码活动的局部放大视图。进入周期 7 之后，两条 `irmovq` 指令都已穿过写回阶段，因此寄存器堆保存着更新后的`%rdx` 和`%rax` 值。`addq` 指令在周期 7 穿过译码阶段时，就会读到其源操作数的正确值。在本例中，两条 `irmovq` 指令与 `addq` 指令之间的数据相关性没有造成数据冒险。

我们看到，`prog1` 能流经流水线并得到正确结果，因为三条 `nop` 指令在具有数据相关性的指令之间制造了一段时间延迟。下面看去掉这些 `nop` 指令时会发生什么。图 4.44 展示了名为 `prog2` 的程序的流水线流程：它在为寄存器`%rdx` 和`%rax` 生成值的两条 `irmovq` 指令与以这两个寄存器为操作数的 `addq` 指令之间，夹着两条 `nop` 指令。此例中，关键时刻发生在周期 6，`addq` 指令此时从寄存器堆读操作数。图底部给出了该周期中流水线活动的局部放大视图。第一条 `irmovq` 指令已穿过写回阶段，因此程序寄存器`%rdx` 已在寄存器堆中更新。第二条 `irmovq` 指令在该周期正处在写回阶段，所以对程序寄存器`%rax` 的写要等到周期 7 开始、时钟上升时才发生。结果，寄存器`%rax` 会读到错误值 0（记住，我们假设所有寄存器的初值都是 0），因为这个寄存器等待写入的值还没有写进去。显然，我们必须改造流水线，才能正确处理这种冒险。

图 4.45 展示了在 `irmovq` 指令与 `addq` 指令之间只留一条 `nop` 指令时的情况，得到程序 `prog3`。这时必须在周期 5 `addq` 指令穿过译码阶段时考察流

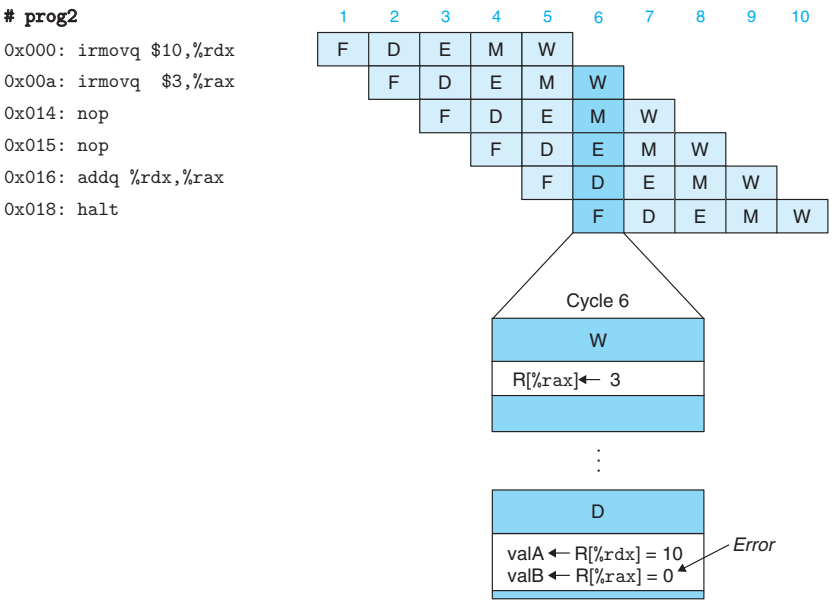


Figure 4.44 Pipelined execution of `prog2` without special pipeline control. The write to program register `%rax` does not occur until the start of cycle 7, and so the `addq` instruction gets the incorrect value for this register in the decode stage.

图 4.44

水线的行为。遗憾的是，寄存器`%rdx` 的等待写入操作还在写回阶段，而`%rax` 的等待写入操作还在访存阶段。因此，`addq` 指令的两个操作数都会取到错误的值。

图 4.46 展示了去掉 `irmovq` 指令与 `addq` 指令之间全部 `nop` 指令时的情形，得到程序 `prog4`。这时必须在周期 4 `addq` 指令穿过译码阶段时考察流水线的行为。遗憾的是，寄存器`%rdx` 的等待写入操作还在访存阶段，而`%rax` 的新值刚刚在执行阶段算出。因此，`addq` 指令的两个操作数都会取到错误的值。

这些例子说明：当一条指令的某个操作数被它前面三条指令中的任何一条更新时，这条指令就可能出现数据冒险。产生这些冒险的原因是，我们的流水线处理器在译码阶段从寄存器堆读指令的操作数，却要等到三个周期之后（指令穿过写回阶段时）才把该指令的结果写入寄存器堆。

用暂停避免数据冒险

一种非常通用的避免冒险的技术是暂停（stalling）：处理器把一条或多条指令留在流水线中不动，直到冒险条件不再成立。我们的处理器可以这样避免数据冒险：把一条指令留在译码阶段，直到生成其源操作数的那些指令全部穿过写回阶段。该机制的细节将在 4.5.8 节讨论，它只需要对流水线控制逻辑作些简单的增强。暂停的效果画在图 4.47（`prog2`）和图 4.48（`prog4`）中。（讨论略

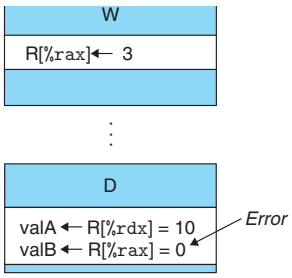


Figure 4.44 Pipelined execution of `prog2` without special pipeline control. The write to program register `%rax` does not occur until the start of cycle 7, and so the `addq` instruction gets the incorrect value for this register in the decode stage.

dependencies. Let us see what happens as these `nop` instructions are removed. Figure 4.44 illustrates the pipeline flow of a program, named `prog2`, containing two `nop` instructions between the two `irmovq` instructions generating values for registers `%rdx` and `%rax` and the `addq` instruction having these two registers as operands. In this case, the crucial step occurs in cycle 6, when the `addq` instruction reads its operands from the register file. An expanded view of the pipeline activities during this cycle is shown at the bottom of the figure. The first `irmovq` instruction has passed through the write-back stage, and so program register `%rdx` has been updated in the register file. The second `irmovq` instruction is in the write-back stage during this cycle, and so the write to program register `%rax` only occurs at the start of cycle 7 as the clock rises. As a result, the incorrect value zero would be read for register `%rax` (recall that we assume all registers are initially zero), since the pending write for this register has not yet occurred. Clearly, we will have to adapt our pipeline to handle this hazard properly.

Figure 4.45 shows what happens when we have only one `nop` instruction between the `irmovq` instructions and the `addq` instruction, yielding a program `prog3`. Now we must examine the behavior of the pipeline during cycle 5 as the `addq` instruction passes through the decode stage. Unfortunately, the pending

图 4.45

去了 `prog3`,因为它的行为与另外两个例子类似。)当 `addq` 指令位于译码阶段时,流水线控制逻辑检测到执行、访存或写回阶段中至少有一条指令要更新寄存器`%rdx` 或寄存器`%rax`。控制逻辑不让 `addq` 指令带着错误的结果穿过该阶段,而是让它暂停,把它留在译码阶段,多停留 1 个周期(对 `prog2`)或 3 个周期(对 `prog4`)。对这三个程序来说,`addq` 指令最终都在周期 7 得到两个源操作数的正确值,然后沿流水线继续前进。

把 `addq` 指令留在译码阶段的同时,还必须把取指阶段中跟在它后面的 `halt` 指令也留下来。办法是让程序计数器保持不变,这样 `halt` 指令就会被反复取出,直到暂停结束。

暂停意味着把一组指令留在它们所在的阶段,同时让其他指令继续流过流水线。那么,本该处理 `addq` 指令的那些阶段又该怎么办呢? 办法是:每当把一

```
0x014: nop
0x015: addq %rdx,%rax
0x017: halt
```

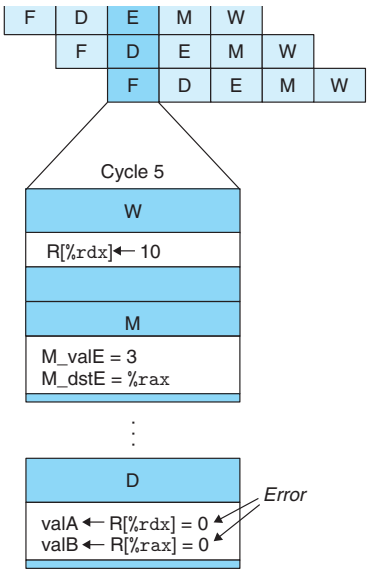


Figure 4.45 Pipelined execution of `prog3` without special pipeline control. In cycle 5, the `addq` instruction reads its source operands from the register file. The pending write to register `%rdx` is still in the write-back stage, and the pending write to register `%rax` is still in the memory stage. Both operands valA and valB get incorrect values.

write to register `%rdx` is still in the write-back stage, and the pending write to `%rax` is still in the memory stage. Therefore, the `addq` instruction would get the incorrect values for both operands.

Figure 4.46 shows what happens when we remove all of the `nop` instructions between the `irmovq` instructions and the `addq` instruction, yielding a program `prog4`. Now we must examine the behavior of the pipeline during cycle 4 as the `addq` instruction passes through the decode stage. Unfortunately, the pending write to register `%rdx` is still in the memory stage, and the new value for `%rax` is just being computed in the execute stage. Therefore, the `addq` instruction would get the incorrect values for both operands.

These examples illustrate that a data hazard can arise for an instruction

图 4.46

条指令留在译码阶段，就向执行阶段注入一个气泡。气泡就像一条动态生成的 nop 指令——它不会对寄存器、存储器、条件码或程序状态产生任何改变。

边栏 枚举数据冒险的类别

当一条指令更新了程序状态中将被后面指令读取的部分时，就可能发生冒险。对 Y86-64 来说，程序状态包括程序寄存器、程序计数器、存储器、条件码寄存器和状态寄存器。我们逐一考察所提设计中这些状态形式可能产生的冒险。

程序寄存器。这就是我们已经识别出的那些冒险。产生的原因是寄存器堆在一个阶段读、在另一个阶段写，不同指令之间就可能发生非预期的相互作用。

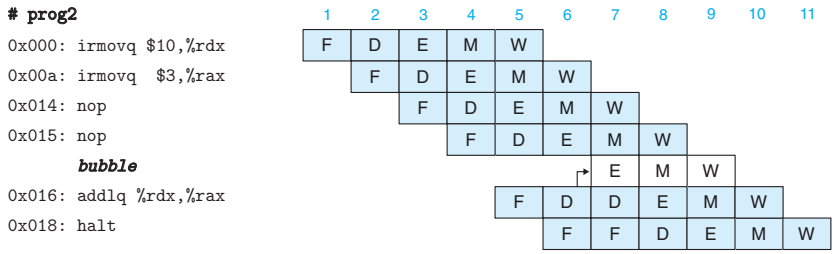


Figure 4.47 Pipelined execution of `prog2` using stalls. After decoding the `addq` instruction in cycle 6, the stall control logic detects a data hazard due to the pending write to register `%rax` in the write-back stage. It injects a bubble into the execute stage and repeats the decoding of the `addq` instruction in cycle 7. In effect, the machine has dynamically inserted a `nop` instruction, giving a flow similar to that shown for `prog1` (Figure 4.43).

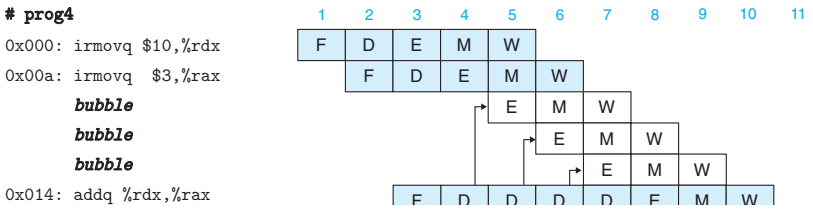


图 4.47

程序计数器。更新与读取程序计数器之间的冲突引发控制冒险。如果取指阶段逻辑在取出下一条指令之前就正确预测了程序计数器的新值，就不会有冒险。预测错误的分支和 `ret` 指令需要特殊处理，如 4.5.5 节所述。

存储器。数据存储器的写和读都发生在访存阶段。当读存储器的指令到达该阶段时，前面所有写存储器的指令早已完成。另一方面，在访存阶段写数据的指令可能与取指阶段读取指令之间发生干扰，因为指令存储器和数据存储器引用同一个地址空间。这只会在包含自修改代码（self-modifying code）的程序中发生，即指令写入稍后被取指的存储器区域。有些系统用复杂的机制来检测和避免这类冒险，另一些系统则直接规定程序不得使用自修改代码。为简单起见，我们假设程序不会修改自身，因此执行期间无需根据对数据存储器的更新去更新指令存储器。

条件码寄存器。它们由执行阶段中的整数运算写入，由执行阶段中的条件传送和访存阶段中的条件跳转读取。当条件传送或条件跳转到达执行阶段时，前面所有整数运算早已完成该阶段。不会产生冒险。

状态寄存器。程序状态会随着指令流过流水线而受到影响。让流水线中每条指令都携有一个状态码的机制，能够使处理器在异常发生时有序停止，见 4.5.6 节。

这一分析表明，我们只需处理寄存器数据冒险、控制冒险，以及保证异常得到正确处理。在设计一个复杂系统时，这种系统性的分析很重要。它能识别出实现系统的潜在困难，还能指导生成用于检验系统正确性的测试程序。

在图 4.47 和图 4.48 的流水线图中，这些气泡显示为白色方框。图中 `addq` 指令的“D”方框与某条流水线气泡的“E”方框之间的箭头表示：有一个气泡被

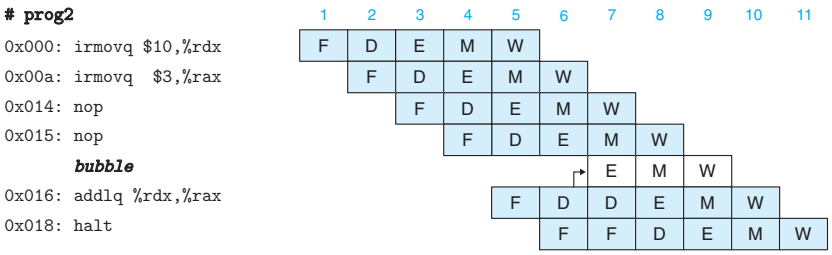


Figure 4.47 Pipelined execution of `prog2` using stalls. After decoding the `addq` instruction in cycle 6, the stall control logic detects a data hazard due to the pending write to register `%rax` in the write-back stage. It injects a bubble into the execute stage and repeats the decoding of the `addq` instruction in cycle 7. In effect, the machine has dynamically inserted a `nop` instruction, giving a flow similar to that shown for `prog1` (Figure 4.43).

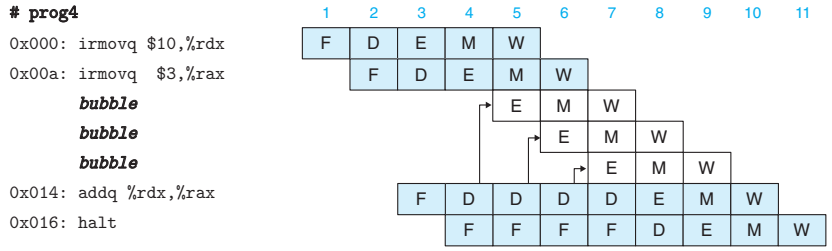


Figure 4.48 Pipelined execution of `prog4` using stalls. After decoding the `addq` instruction in cycle 4, the stall control logic detects data hazards for both source registers. It injects a bubble into the execute stage and repeats the decoding of the `addq` instruction on cycle 5. It again detects hazards for both source registers, injects a bubble into the execute stage, and repeats the decoding of the `addq` instruction on cycle 6. Still, it detects a hazard for source register `%rax`, injects a bubble into the execute stage, and repeats the decoding of the `addq` instruction on cycle 7. In effect, the machine has dynamically inserted three `nop` instructions, giving a flow similar to that shown for `prog1` (Figure 4.43).

图 4.48

注入执行阶段，代替本应从译码阶段进入执行阶段的 `addq` 指令。让流水线暂停和注入气泡的详细机制见 4.5.8 节。

用暂停处理数据冒险时，我们实际上是在动态生成与 `prog1`（图 4.43）相同的流水线流程来执行 `prog2` 和 `prog4`。为 `prog2` 注入一个、为 `prog4` 注入三个气泡，与在第二条 `irmovq` 指令和 `addq` 指令之间放三条 `nop` 指令效果相同。这种机制实现起来相当容易（见家庭作业 4.53），但性能并不好。一条指令更新某个寄存器、紧随其后的指令使用同一寄存器的情形非常多，这会使流水线暂停多达三个周期，显著降低整体吞吐量。

用转发避免数据冒险

我们的 PIPE- 设计在译码阶段从寄存器堆读源操作数，而此时写回阶段可能还有一条针对某个源寄存器的等待写入操作。与其暂停等待写操作完成，不如直接把即将写入的值作为源操作数送进流水线寄存器 E。图 4.49 以 prog2 周期 6 的流水线图局部放大视图展示了这一策略。译码阶段逻辑检测到寄存器%rax 是操作数 valB 的源寄存器，而且写端口 E 上还存在一条针对%rax 的等待写入操作。它可以直接用供给端口 E 的数据字（信号 W_valE）作为操作数 valB 的值，从而避免暂停。这种把结果值直接从一个流水线阶段传回更早阶段的技术通常称为数据转发（data forwarding，或简称 forwarding，有时也叫旁路（bypassing））。它使 prog2 的指令无需任何暂停就能流过流水线。数据转发需要在基本硬件结构上增加额外的数据连接和控制逻辑。

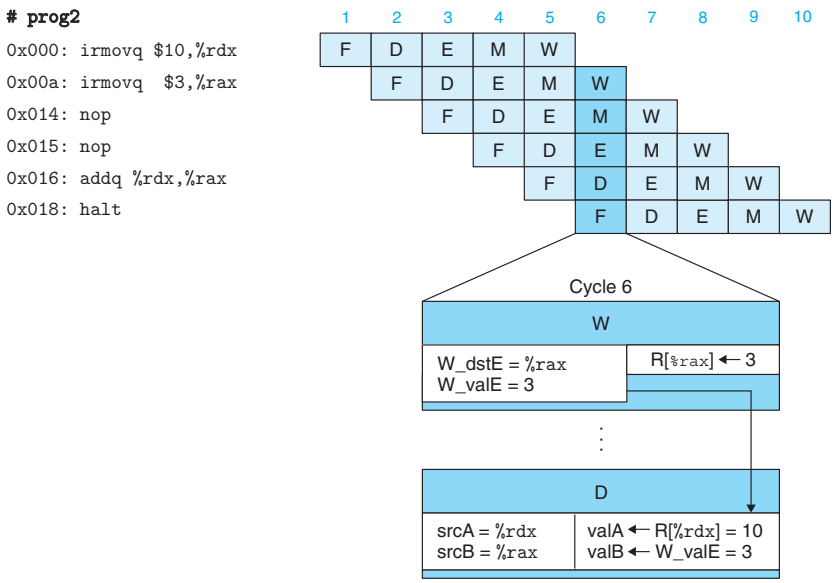


Figure 4.49 **Pipelined execution of** prog2 **using forwarding.** In cycle 6, the decode-stage logic detects the presence of a pending write to register %rax in the write-back stage. It uses this value for source operand valB rather than the value read from the register file.

图 4.49

如图 4.50 所示，当访存阶段也存在对某个寄存器的等待写入操作时，同样可以使用数据转发，从而让 prog3 无需暂停。周期 5 中，译码阶段逻辑检测到写回阶段端口 E 上的寄存器%rdx 的等待写入操作，同时还有一条通往端口 E 的寄存器%rax 的等待写入操作——它还在访存阶段。与其暂停等待写操作发生，不如把写回阶段中的值（信号 W_valE）用作操作数 valA，把访存阶段中的值（信号 M_valE）用作操作数 valB。

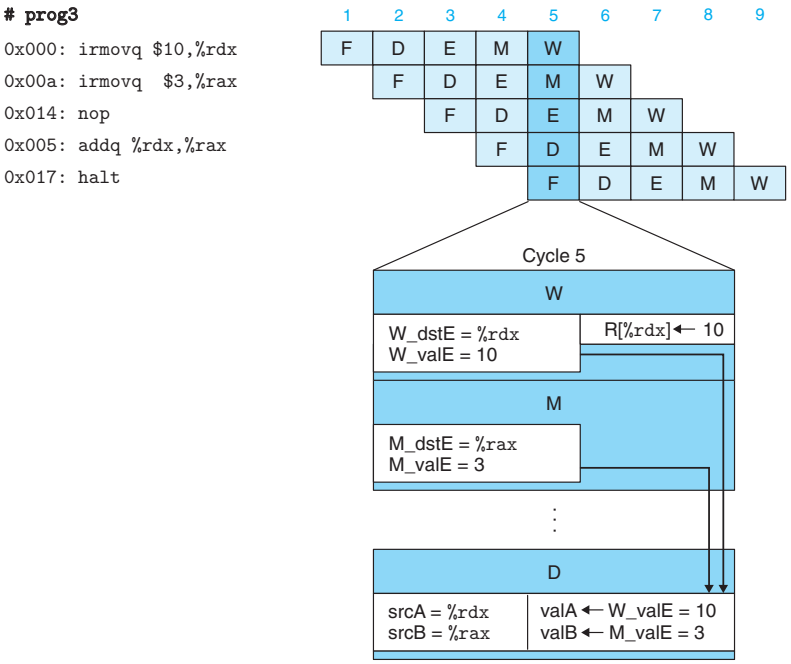


Figure 4.50 **Pipelined execution of** `prog3` **using forwarding.** In cycle 5, the decode-stage logic detects a pending write to register `%rdx` in the write-back stage and to register `%rax` in the memory stage. It uses these as the values for valA and valB rather than the values read from the register file.

图 4.50

为了最大限度地发挥数据前递的作用，我们还可以把刚计算出来的值从执行阶段传递给译码阶段，这样就不必为了处理程序 `prog4` 而停顿，如图 4.51 所示。在第 4 个周期，译码阶段逻辑检测到访存阶段中存在一个对寄存器`%rdx`的待完成写操作，还检测到执行阶段中 ALU 正在计算的值随后会写入寄存器`%rax`。于是，它可以用访存阶段中的值（信号 `M_valE`）作为操作数 `valA`，也可以用 ALU 的输出（信号 `e_valE`）作为操作数 `valB`。注意，使用 ALU 的输出不会引入任何时序问题。译码阶段只需要在时钟周期结束之前生成信号 `valA` 和 `valB`，这样当时钟上升、开始下一个周期时，流水线寄存器 E 才能装入译码阶段的结果。ALU 的输出在这之前就已有效。

程序 `prog2` 至 `prog4` 中展示的前递用法，都是针对由 ALU 生成、准备写入端口 E 的值。对于从存储器中读出、准备写入端口 M 的值，同样可以使用前递。在访存阶段，我们可以前递刚从数据存储器中读出的值（信号 `m_valM`）；在写回阶段，我们可以前递对端口 M 的待完成写操作（信号 `W_valM`）。这样一来，总共有五个不同的前递来源（`e_valE`、`m_valM`、`M_valE`、`W_valM` 和 `W_valE`）和两个不同的前递目的地（`valA` 和 `valB`）。

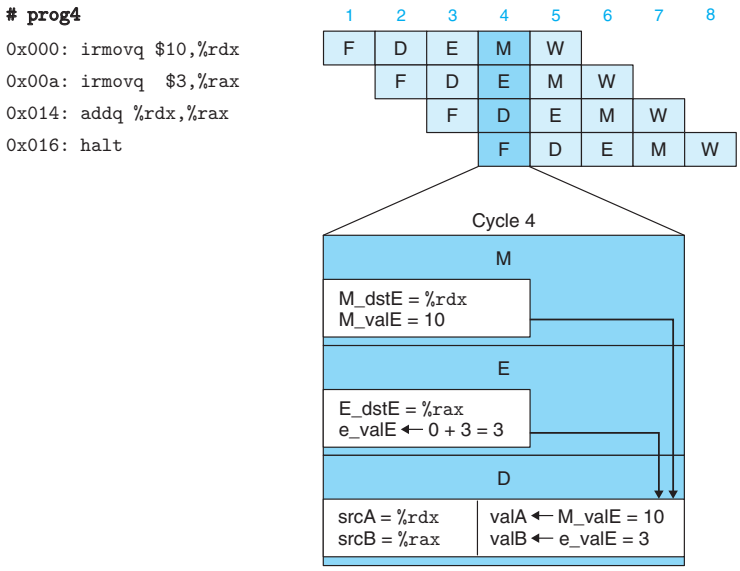


Figure 4.51 Pipelined execution of `prog4` using forwarding. In cycle 4, the decode-stage logic detects a pending write to register `%rdx` in the memory stage. It also detects that a new value is being computed for register `%rax` in the execute stage. It uses these as the values for valA and valB rather than the values read from the register file.

图 4.51

图 4.49 至图 4.51 的展开图还表明，译码阶段逻辑如何决定使用从寄存器堆读出的值，还是使用一个前递值。每个将要写回寄存器堆的值都带有自己的目的寄存器标识符。该逻辑可以把这些标识符与源寄存器标识符 `srcA` 和 `srcB` 相比较，从而发现需要前递的情形。可能出现多个目的寄存器标识符与同一个源标识符相匹配的情况。在这种情况下，我们必须给不同的前递来源定出优先级。在讨论前递逻辑的详细设计时会处理这个问题。

图 4.52 给出了 PIPE 的结构，它是 `PIPE–` 的扩展版本，能够通过前递来处理数据冒险。与 `PIPE–` 的结构（图 4.41）相比可以看到，来自五个前递来源的值被送回译码阶段中标记为“Sel+Fwd A”和“Fwd B”的两个块。标记为“Sel+Fwd A”的块把 `PIPE–` 中“Select A”块的职责与前递逻辑合并在了一起，使送往流水线寄存器 E 的 `valA` 可以是递增后的程序计数器值 `valP`、寄存器堆 A 端口读出的值，或者是某个前递值。标记为“Fwd B”的块实现针对源操作数 `valB` 的前递逻辑。

加载/使用数据冒险

有一类数据冒险不能只靠前递来处理，因为对存储器的读操作发生在流水线中较晚的阶段。图 4.53 给出了一个加载/使用（load/use）冒险的例子：一条指令（地址 `0x028` 处的 `mrmovq`）从存储器读出一个值写入寄存器`%rax`，而紧接着

This will be discussed when we look at the detailed design of the forwarding logic.

Figure 4.52 shows the structure of PIPE, an extension of PIPE− that can handle data hazards by forwarding. Comparing this to the structure of PIPE− (Figure 4.41), we can see that the values from the five forwarding sources are fed back to the two blocks labeled "Sel+Fwd A" and "Fwd B" in the decode stage. The block labeled "Sel+Fwd A" combines the role of the block labeled "Select A" in PIPE− with the forwarding logic. It allows valA for pipeline register E to be either the incremented program counter valP, the value read from the A port of the register file, or one of the forwarded values. The block labeled "Fwd B" implements the forwarding logic for source operand valB.

图 4.52

的下一条指令(地址 0x032 处的 addq)需要把这个值用作源操作数。图的下部给出了第 7 个和第 8 个周期的展开视图,其中假设所有程序寄存器的初值都是 0。addq 指令在第 7 个周期就需要这个寄存器的值,但 mrmovq 指令要到第 8 个周期才生成这个值。要想把值从 mrmovq "前递"给 addq,前递逻辑就得让这个值在时间上倒流!这显然不可能,因此必须寻找某种别的机制来处理这种形式的数据冒险。(对寄存器%rbx 的数据冒险可以靠前递解决:该值由地址 0x01e 处的 irmovq 指令生成,由地址 0x032 处的 addq 指令使用。)

```
M_dstE = %rbx
M_valE = 10
M
M_dstM = %rax
m_valM      M[128] = 3
M
F
0x000: irmovq $128,%rdx
# prog5
# prog5
0x00a: irmovq  $3,%rcx
0x014: rmmovq %rcx, 0(%rdx)
0x01e: irmovq $10,%rbx
0x028: mrmovq 0(%rdx),%rax # Load %rax
0x032: addq %ebx,%eax # Use %rax
0x034: halt
D
E
M
W
F
D
E
M
W
F
D
E
M
W
F
```

```
D
E
M
W
F
D
E
M
W
F
D
E
M
W
F
D
E
M
W
1
2
3
4
5
6
7
8
9
10
11
D
valA      M_valE = 10
valB      R[%rax] = 0
Cycle 7
Cycle 8
Error
. . .
Figure 4.53
Example of load/use data hazard. The addq instruction requires the
 ↪  value
of register %rax during the decode stage in cycle 7. The preceding
        mrmovq reads a new
value for this register during the memory stage in cycle 8,
      which is too late for the addq
instruction.
```

正如图 4.54 所示的，我们可以用停顿和前递的组合来避免加载/使用数据冒险。这需要对控制逻辑做些修改，但可以复用现有的旁路通路。当 `mrmovq` 指令经过执行阶段时，流水线控制逻辑发现译码阶段中的指令（`addq`）需要它从存储器读出的结果，于是把译码阶段中的这条指令停顿一个周期，并向执行阶段注入一个气泡。正如第 8 个周期的展开视图所展示的，从存储器读出的值随后便可以从访存阶段前递给译码阶段中的 `addq` 指令；寄存器`%rbx` 的值也同

时从写回阶段前递给访存阶段。如图中从第 7 个周期标着“D”的方框指向第 8 个周期标着“E”的方框的那支箭头所示，注入的气泡取代了正常情况下会继续在流水线中流动的 addq 指令。

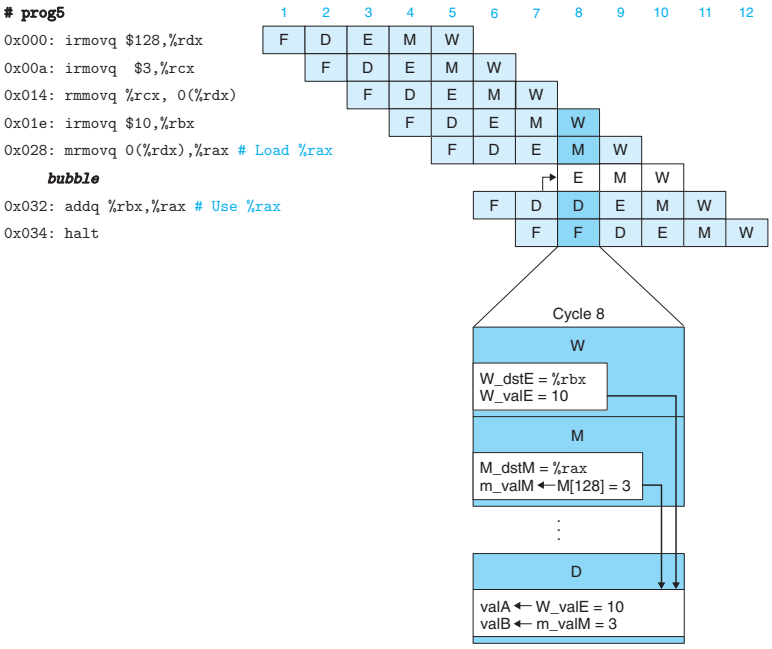


Figure 4.54 **Handling a load/use hazard by stalling.** By stalling the addq instruction for one cycle in the decode stage, the value for valB can be forwarded from the mrmovq instruction in the memory stage to the addq instruction in the decode stage.

图 4.54

这种用停顿来处理加载/使用冒险的方法称为加载互锁(load interlock)。加载互锁再配上前递，就足以处理数据冒险的所有可能形式。由于只有加载互锁会降低流水线的吞吐量，我们几乎可以达到每个时钟周期发出一条新指令的吞吐量目标。

避免控制冒险

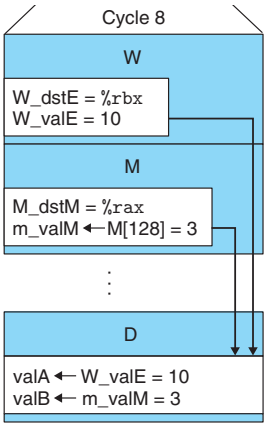
当处理器无法根据取指阶段中的当前指令可靠地确定下一条指令的地址时，就会产生控制冒险。正如 4.5.4 节讨论的，在我们的流水线处理器中，控制冒险只会发生在 ret 指令和跳转指令上；而且跳转指令也只有在条件跳转的方向被预测错了的时候才会造成麻烦。本节先从较高的层次介绍这类冒险的处理方法，详细实现将在 4.5.8 节中结合对流水线控制的一般性讨论给出。

对于 ret 指令，考虑下面的示例程序。程序以汇编代码的形式给出，左侧列出了各条指令的地址，便于对照：

```
0x000:    irmovq stack,%rsp        #  初始化栈指针
```

```
0x00a:    call proc                 #   过程调用
0x013:    irmovq $10,%rdx           #   返回点
0x01d:    halt
0x020: .pos 0x20
0x020: proc:                        # proc:
0x020:    ret                       #   立即返回
0x021:    rrmovq %rdx,%rbx          #   不执行
0x030: .pos 0x30
0x030: stack:                       # stack: 栈指针
```

图 4.55 展示了我们希望流水线如何处理这条 `ret` 指令。与前面的流水线图一样，该图也把时间增长的方向画成向右。与之前不同的是，指令并没有按照它们在程序中出现的顺序排列，因为这个程序的控制流不是让指令按线性顺序执行的。借助指令地址来辨认程序中的各条指令是很有用的。



lling a load/use hazard by stalling. By stalling the `addq` instruction for one cycle in the value for valB can be forwarded from the `mrmovq` instruction in the memory stage to the the decode stage.

```
0x000:    irmovq stack,%rsp  #   Initialize stack pointer
0x00a:    call proc          #   Procedure call
0x013:    irmovq $10,%rdx    #   Return point
0x01d:    halt
0x020: .pos 0x20
0x020: proc:                 # proc:
0x020:    ret                #   Return immediately
0x021:    rrmovq %rdx,%rbx   #   Not executed
0x030: .pos 0x30
0x030: stack:                # stack: Stack pointer
```

Figure 4.55 shows how we want the pipeline to process the `ret` instruction. As with our earlier pipeline diagrams, this figure shows the pipeline activity with

图 4.55

从这个图可以看出，ret 指令在第 3 个周期被取出，然后沿着流水线向下推进，到第 7 个周期到达写回阶段。在它经过译码、执行和访存阶段的那段时间里，流水线做不了任何有用的工作，此时要做的反而是向流水线注入三个气泡。一旦 ret 指令到达写回阶段，PC 选择逻辑就会把程序计数器设为返回地址，取指阶段于是取回返回点（地址 0x013）处的 irmovq 指令。

要处理预测错误的分支，考虑下面的程序。程序同样以汇编代码的形式给出，左侧列出指令地址，便于对照：

```
0x000:    xorq %rax,%rax
0x002:    jne target                # 不跳转
0x00b:    irmovq $1, %rax           # 贯穿
0x015:    halt
0x016: target:
0x016:    irmovq $2, %rdx           # 跳转目标
0x020:    irmovq $3, %rbx           # 目标 +1
0x02a:    halt
```

图 4.56 给出了这些指令的处理过程。与前一样，指令按它们进入流水线的顺序排列，而不是按它们在程序中出现的顺序排列。由于跳转指令被预测为会发生跳转，第 3 个周期会把位于跳转目标处的指令取出来，第 4 个周期会把该指令的下一条指令取出来。等到第 4 个周期分支逻辑检测出这条跳转不该发生时，已经有两条本不该继续执行的指令被取出来了。所幸的是，这两条指令都还没有改变程序员可见的状态；指令对状态的改变只可能发生在

Figure 4.56 shows how these instructions are processed. As before, the instructions are listed in the order they enter the pipeline, rather than the order they occur in the program. Since the jump instruction is predicted as being taken, the instruction at the jump target will be fetched in cycle 3, and the instruction following this one will be fetched in cycle 4. By the time the branch logic detects that the jump should not be taken during cycle 4, two instructions have been fetched that should not continue being executed. Fortunately, neither of these instructions has caused a change in the programmer-visible state. That can only occur when an instruction

图 4.56

这种改变只可能发生在指令到达执行阶段的时候——正是在那里，指令可以改变条件码。此时，流水线只需要在下一个周期向译码阶段和执行阶段注入气泡、同时取出跳转指令之后的那条指令，就可以取消这两条被错误取出的指令（这种操作有时称为指令报废（instruction squashing））。于是这两条指令就从流水线中消失，不会对程序员可见状态产生任何影响。唯一的代价是浪费了两个时钟周期的指令处理能力。

关于控制冒险的讨论表明，这类冒险可以靠仔细考虑流水线控制逻辑来处理。当特殊情况出现时，停顿和注入气泡之类的技术能够动态调整流水线的流

动。正如 4.5.8 节将要讨论的,对基本的时钟寄存器设计做一个简单的扩展,我们就能让流水线控制逻辑在停顿各级流水线的同时向流水线寄存器注入气泡。

4.23 异常处理

正如第 8 章将要讨论的,处理器中的许多种活动都可能引发异常控制流,使程序正常的执行链条被打破。异常可能由内部产生,来自正在执行的程序;也可能由外部产生,来自某个外界信号。我们的指令集体系结构包含三种不同的内部生成异常,分别由如下情况引起:(1) 执行 `halt` 指令;(2) 指令与功能码的组合无效;(3) 试图访问一个无效地址——无论是取指令地址,还是数据读写地址。更完整的处理器设计还要处理外部异常,例如处理器接到信号表明网络接口收到了一个新分组,或者用户单击了鼠标按钮。正确地处理异常是任何微处理器设计中最具挑战性的方面之一。异常可能在不可预测的时刻发生,它要求在流经处理器流水线的指令流中制造一次干净的断点。我们对这三种内部异常的处理,只是正确检测与处理异常的真实复杂性的一瞥。

我们把导致异常的指令称为异常指令。当指令地址无效时,并不存在真正的异常指令,但设想在无效地址上有一条“虚拟指令”是有用的。在我们这个简化版的 ISA 模型中,希望处理器一遇到异常就停下来,并设置适当的状态码,如 4.1.1 节图 4.5 所列。从效果上看,异常指令之前的所有指令都应完成,而其后继指令对程序员可见状态不得有任何影响。在更完整的设计中,处理器会转而调用异常处理程序,一个属于操作系统的过程;不过,实现这部分异常处理超出了我们的讨论范围。

在流水线系统中,异常处理牵涉到几个微妙之处。第一,多条指令可能同时触发异常。例如,在一个流水线周期内,可能同时取指阶段中有 `halt` 指令、存储器报告访存阶段中的指令发生数据地址越界。我们必须决定处理器应该向操作系统报告哪一个异常。基本规则是把最高的优先级给予流水线中走得最远的指令触发的异常。在上述例子中,就是访存阶段中的指令试图访问的越界地址。就机器级程序而言,访存阶段中的指令看起来已经执行,而取指阶段中的指令还没有,因此只应向操作系统报告这一个异常。

第二处微妙之处出现在这种情况:一条指令刚被取出并开始执行,触发了一个异常,但随后却因分支预测错误而被取消。下面是一个这样的程序,以其目标代码的形式给出:

```
0x000: 6300                 |    xorq %rax,%rax
0x002: 741600000000000000   |    jne target              # 不跳转
0x00b: 30f00100000000000000 |    irmovq $1, %rax         # 贯穿
0x015: 00                   |    halt
0x016:                      | target:
```

```
0x016: ff                   |    .byte 0xFF              # 无效指令代码
```

在这个程序中,流水线会预测分支应该跳转,于是会取回地址 0x016 处的字节,并试图把它当作指令来用(这个字节是用汇编代码中的.byte 伪指令生成的)。译码阶段因此会检测到一个无效指令异常。随后流水线会发现该分支并不应该跳转,所以地址 0x016 处的指令根本不应该被取出来。流水线控制逻辑会取消这条指令,但我们要避免让这个异常被触发。

第三处微妙之处源于这样一点:流水线处理器在不同的阶段更新系统状态的不同部分。某条导致异常的指令后面的指令,可能在这条指令完成之前就改变了状态的一部分。例如,考虑下面的代码序列,其中假设用户程序不允许访问 64 位地址范围的高端地址:

```
1  irmovq $1,%rax
2  xorq %rsp,%rsp       # 把栈指针置为 0,条件码置为 100
3  pushq %rax           # 试图写入 0xfffffffffffffff8
4  addq %rax,%rax       # (不应执行)会把条件码置为 000
```

pushq 指令引发了一个地址异常,因为栈指针减 1 后绕回到了 0xfffffffffffffff8。这个异常在访存阶段被检测到。而在同一个周期内,addq 指令正位于执行阶段,它将把条件码置成新的值。这就违背了我们的要求:异常指令之后的所有指令都不应当对系统状态产生任何影响。

一般而言,把异常处理逻辑并入流水线结构,我们就既能正确地在各个不同的异常之间做出选择,也能避免对因分支预测错误而被取出的指令触发异常。这正是我们在每个流水线寄存器(图 4.41 和图 4.52)中都放入一个状态码 stat 的动机。当指令在处理的某个阶段产生异常时,该指令的状态域会被置成表明异常性质的值。异常状态随这条指令在流水线中的其余信息一同传播,直到它到达写回阶段。此时,流水线控制逻辑检测到异常的发生,并停止执行。

为了避免异常指令之后的指令对程序员可见状态产生任何更新,流水线控制逻辑必须在访存阶段或写回阶段中的指令已导致异常时,禁止对条件码寄存器和数据存储器的任何更新。在上面的示例程序中,控制逻辑会发现访存阶段中的 pushq 已导致异常,于是执行阶段中 addq 指令对条件码寄存器的更新就被禁止了。

我们来看看这种处理异常的方法是怎样应对上述几个微妙之处的。当流水线的一个或多个阶段发生异常时,相关信息只是被存入流水线寄存器的状态域。在异常指令到达最后一级流水线阶段之前,这个事件对流水线中指令的流动没有任何影响,唯一的附带效果是禁止流水线中较晚的指令对程序员可见状态(条件码寄存器和存储器)做任何更新。由于指令到达写回阶段的顺序与它们在非流水线处理器中的执行顺序相同,可以保证第一条遇到异常的指令会最

先到达写回阶段,这时程序的执行可以停止,流水线寄存器 W 中的状态码可以直接记录为程序状态。如果某条指令被取出后又被取消,那么与该指令相关的异常状态信息也会一同被取消。异常指令后面的任何指令都无法改变程序员可见状态。把异常状态与指令的其他所有信息一道携带通过流水线,这一简单的规则为处理异常提供了一种简单而可靠的机制。

4.24 PIPE 各阶段的实现

至此,我们建立了 PIPE——我们那个带前递的流水线化 Y86-64 处理器——的整体结构。它所使用的硬件单元与之前的顺序设计相同,在此基础上增加了流水线寄存器、一些重新构造的逻辑块和额外的流水线控制逻辑。本节讨论各个逻辑块的设计,流水线控制逻辑的设计留到下一节。有许多逻辑块与 SEQ 和 SEQ+ 中对应的块完全相同,只是我们必须从流水线寄存器(前缀为流水线寄存器名,用大写字母)或各阶段内的计算结果(前缀为阶段名第一个字母,用小写字母)中挑选出适当版本的信号。

以生成 `srcA` 信号的逻辑为例,比较 SEQ 中的 HCL 代码与 PIPE 中对应的代码:

```
# 来自 SEQ 的代码
word srcA = [
        icode in { IRRMOVQ, IRMMOVQ, IOPQ, IPUSHQ                } : rA;
        icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # 不需要寄存器
];

# 来自 PIPE 的代码
word d_srcA = [
        D_icode in { IRRMOVQ, IRMMOVQ, IOPQ,
      IPUSHQ                } : D_rA;
        D_icode in { IPOPQ, IRET } : RRSP;
        1 : RNONE; # 不需要寄存器
];
```

这两段代码的唯一差别在于给 PIPE 信号加的前缀:给源信号加上前缀 `D_`,表示信号来自流水线寄存器 D;给结果信号加上前缀 `d_`,表示它在译码阶段生成。为避免重复,本书不再列出那些与 SEQ 中对应块的差别仅仅在于名字前缀的逻辑块的 HCL 代码。作为参考,PIPE 的完整 HCL 代码见网络边栏 arch:hcl。

PC 选择和取指阶段

图 4.57 给出了 PIPE 取指阶段逻辑的详细视图。正如前面讨论过的,这一级还必须为程序计数器选择当前值,并预测下一个 PC 值。从存储器读出指令

和从中抽取各指令域所用的硬件单元，与我们在 SEQ 中讨论过的相同（见 4.3.4 节中的取指阶段）。

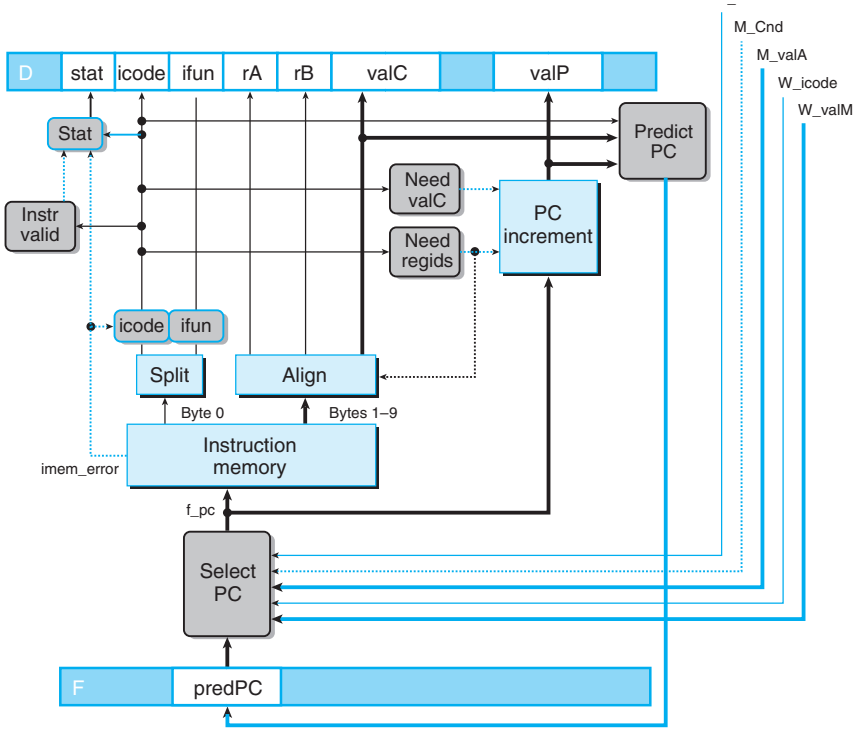


图 4.57　PIPE 的 PC 选择和取指逻辑。在一个周期的时限内，处理器只能预测下一条指令的地址。

PC 选择逻辑在三个程序计数器来源之间做出选择。当预测错误的分支进入访存阶段时，从流水线寄存器 M 中读出该指令的 valP 值（它表明下一条指令的地址，信号 M_valA）。当 ret 指令进入写回阶段时，从流水线寄存器 W 中读出返回地址（信号 W_valM）。其余所有情况都使用保存在流水线寄存器 F 中的 PC 预测值（信号 F_predPC）：

```
word f_pc = [
        # 预测错误的分支。取递增后的 PC
        M_icode == IJXX && !M_Cnd : M_valA;
        # RET 指令的完成
        W_icode == IRET : W_valM;
        # 默认:使用 PC 的预测值
        1 : F_predPC;
];
```

当取出的指令是 call 或跳转指令时，PC 预测逻辑为它选择 valC；否则选择 valP：

```
word f_predPC = [
```

```
        f_icode in { IJXX, ICALL } : f_valC;
        1 : f_valP;
];
```

标记为“Instr valid”“Need regids”和“Need valC”的逻辑块与 SEQ 中的相同，只是其源信号换成了相应命名的信号。

与 SEQ 不同的是，我们必须把指令状态的计算拆成两部分。在取指阶段，可以测试是否有因指令地址越界而造成的存储器错误，并检测出非法指令或 `halt` 指令。检测无效的数据地址必须推迟到访存阶段。

练习 4.30 写出信号 `f_stat` 的 HCL 代码，给出取出指令的暂定状态。

译码和写回阶段

图 4.58 给出了 PIPE 译码和写回逻辑的详细视图。标记为 dstE、dstM、srcA 和 srcB 的那些块与 SEQ 实现中相应的块非常相似。注意，送到写端口的寄存器标识符来自写回阶段（信号 `W_dstE` 和 `W_dstM`），而不是来自译码阶段。这是因为我们希望写操作发生在由写回阶段中的那些指令指定的目的寄存器上。

Decode and Write-Back Stages

Figure 4.58 gives a detailed view of the decode and write-back logic for PIPE. The blocks labeled dstE, dstM, srcA, and srcB are very similar to their counterparts in the implementation of SEQ. Observe that the register IDs supplied to the write ports come from the write-back stage (signals W_dstE and W_dstM), rather than from the decode stage. This is because we want the writes to occur to the destination registers specified by the instruction in the write-back stage.

图 4.58

练习 4.31 译码阶段中标记为“dstE”的块根据流水线寄存器 D 中取出指令的域，生成寄存器堆 E 端口的寄存器标识符，在 PIPE 的 HCL 描述中这个信号称为 `d_dstE`。以 SEQ 的 `dstE` 信号的 HCL 描述为基础，写出这一信号的 HCL 代码（见 4.3.4 节中 SEQ 的译码阶段）。暂时不必操心实现条件传送的逻辑。

这一级的复杂性主要与前递逻辑有关。正如前面提到的，标记为“Sel+Fwd A”的块起两个作用。它把 `valP` 信号并入 `valA` 信号，供后面的流水级使用，从而减少流水线寄存器中状态的数量。它还实现针对源操作数 `valA` 的前递逻辑。

把 `valA` 和 `valP` 合并的做法利用了这样一个事实：只有 `call` 和跳转指令才会在后面的阶段中用到 `valP` 的值，而这两条指令并不需要从寄存器堆 A 端口读出的值。这一选择由本级的 `icode` 信号控制。当信号 `D_icode` 与 `call` 或者 `jXX` 的指令码匹配时，该块就选择 `D_valP` 作为输出。

正如 4.5.5 节说明的，有五个不同的前递来源，每个来源都带有一个数据字和一个目的寄存器标识符：

数据字	寄存器标识符	来源说明
e_valE	e_dstE	ALU 输出
m_valM	M_dstM	存储器输出
M_valE	M_dstE	访存阶段中写入端口 E 的待完成写
W_valM	W_dstM	写回阶段中写入端口 M 的待完成写
W_valE	W_dstE	写回阶段中写入端口 E 的待完成写

如果这些前递条件都不成立，该块就应当选择 d_rvalA，即从寄存器 A 端口读出的值，作为其输出。

综合以上所有内容，我们得到送往流水线寄存器 E 的 valA 新值的如下 HCL 描述：

```
word d_valA = [
        D_icode in { ICALL, IJXX } : D_valP; # 使用递增后的 PC
        d_srcA == e_dstE : e_valE;   # 从执行阶段前递 valE
        d_srcA == M_dstM : m_valM;   # 从访存阶段前递 valM
        d_srcA == M_dstE : M_valE;   # 从访存阶段前递 valE
        d_srcA == W_dstM : W_valM;   # 从写回阶段前递 valM
        d_srcA == W_dstE : W_valE;   # 从写回阶段前递 valE
        1 : d_rvalA; # 使用从寄存器堆读出的值
];
```

上面 HCL 代码中给五个前递来源规定的优先级非常重要。这个优先级是由 HCL 代码中依次测试五个目的寄存器标识符的顺序决定的。如果选用了这里所示的顺序之外的任何顺序，流水线对某些程序就会出错。图 4.59 给出了一个程序，它需要在执行阶段和访存阶段的前递来源之间正确设定优先级。在这个程序中，前两条指令都写寄存器%rdx，第三条指令却把这个寄存器用作源操作数。当 rrmovq 指令在第 4 个周期到达译码阶段时，前递逻辑必须在两个都写往其源寄存器的值之间做出选择。究竟该选哪个？要确定优先级，我们必须考察机器级程序逐条执行指令时的行为：第一条 irmovq 会把寄存器%rdx 置为 10，第二条把它置为 3，然后 rrmovq 指令从%rdx 读到的是 3。为了模仿这一行为，我们的流水线实现应当始终把靠前流水线阶段中的前递来源排在最高的优先级上，因为它保存的是程序序列中最新的、设置该寄存器的指令。因此，上面 HCL 代码中的逻辑先测试执行阶段的前递来源，然后测试访存阶段的，最后测试写回阶段的。访存阶段或写回阶段内部两个前递来源之间的优先级问题只对指令 popq %rsp 有影响，因为只有这条指令才可能对同一个寄存器同时执行两次写。

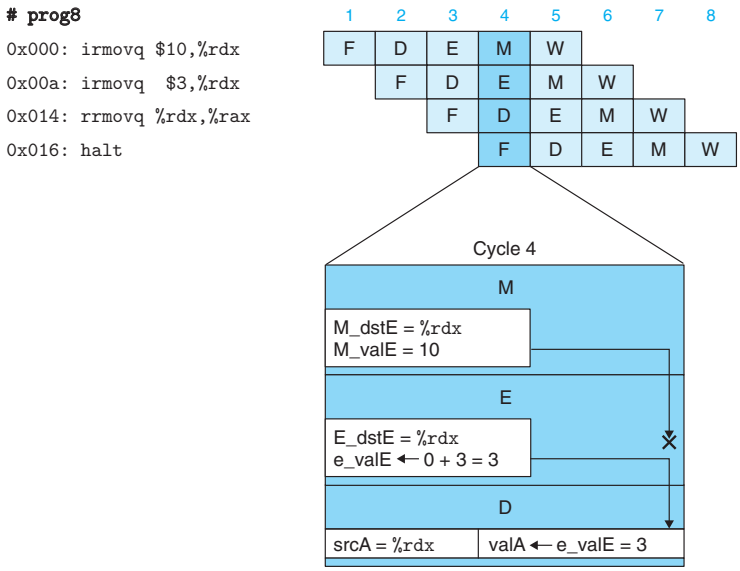


Figure 4.59 **Demonstration of forwarding priority.** In cycle 4, values for %rdx are available from both the execute and memory stages. The forwarding logic should choose the one in the execute stage, since it represents the most recently generated value for this register.

图 4.59

练习 4.32 假设把 d_valA 的 HCL 代码中第三种和第四种情况(来自访存阶段的两个前递来源)的顺序颠倒过来。描述在下面这个程序里 rrmovq 指令(第 5 行)由此产生的运行行为:

```
1  irmovq $5, %rdx
2  irmovq $0x100,%rsp
3  rmmovq %rdx,0(%rsp)
4  popq %rsp
5  rrmovq %rsp,%rax
```

练习 4.33 假设把 d_valA 的 HCL 代码中第五种和第六种情况(来自写回阶段的两个前递来源)的顺序颠倒过来。写一个会被错误执行的 Y86-64 程序,描述错误是如何发生的,以及对程序行为的影响。

练习 4.34 写出信号 d_valB 的 HCL 代码,给出送往流水线寄存器 E 的源操作数 valB 的值。

写回阶段还剩一小部分。如图 4.52 所示,处理器的整体状态 Stat 由一个块根据流水线寄存器 W 中的状态值计算而来。回忆 4.1.1 节的内容,这个码要么指明正常运行(AOK),要么指明三种异常情形之一。由于流水线寄存器 W 保存着最近刚完成的那条指令的状态,把它当作处理器整体状态的指示是最自然

的。唯一需要考虑的特殊情形是写回阶段中出现气泡的情况。这属于正常运行，所以这时我们也希望状态码为 AOK：

```
word Stat = [
        W_stat == SBUB : SAOK;
        1 : W_stat;
];
```

执行阶段

图 4.60 给出了 PIPE 的执行阶段逻辑。这里的硬件单元和逻辑块与 SEQ 中完全相同，只是信号名做了相应的改动。可以看到，信号 e_valE 和 e_dstE 被引向译码阶段，作为前递来源之一。一个不同之处在于，判定是否更新条件码的逻辑块“Set CC”增加了信号 m_stat 和 W_stat 作为输入，用来检测这样的情形：一条导致异常的指令正位于更靠后的流水线阶段，条件码的任何更新都应当被禁止。设计的这一方面将在 4.5.8 节讨论。

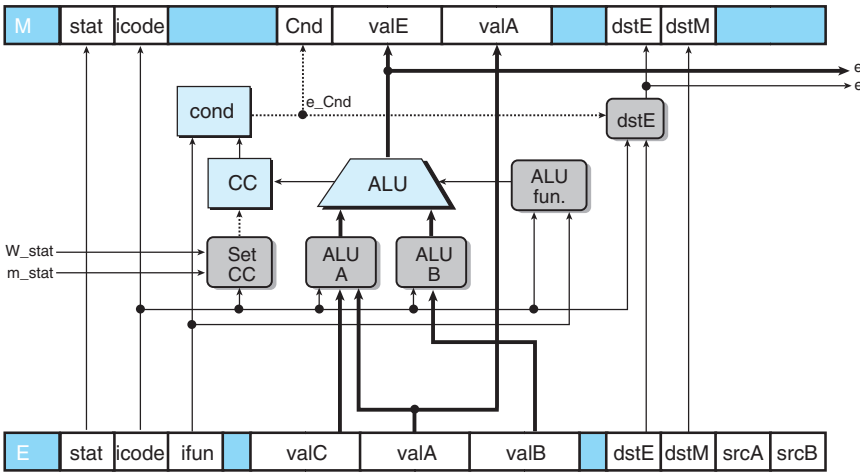


图 4.60 PIPE 的执行阶段逻辑。设计的这一部分与 SEQ 实现中的逻辑非常相似。

练习 4.35 d_valA 的 HCL 代码中的第二种情况用信号 e_dstE 来判断是否选择 ALU 输出 e_valE 作为前递来源。假设改用流水线寄存器 E 中的目的寄存器标识符，也就是信号 E_dstE，来做这个选择。写出一个会出现错误结果的 Y86-64 程序来验证这样修改之后的前递逻辑。

访存阶段

图 4.61 给出了 PIPE 的访存阶段逻辑。与 SEQ 的访存阶段（图 4.30）比较可以看到，正如前面所指出的，SEQ 中标记为“Mem. data”的块在 PIPE 中没有出现。这个块的作用是在数据来源 valP（call 指令用）和 valA 之间做出选择，现在这个选择已改由译码阶段中标记为“Sel+Fwd A”的块来完成。这一级的其

他大多数块都与 SEQ 中相应的块相同，只是信号名做了相应改动。从图 4.61 中还能看到，流水线寄存器 M 和 W 中的许多值被送到电路的其他部分，作为前递逻辑和流水线控制逻辑的组成部分。

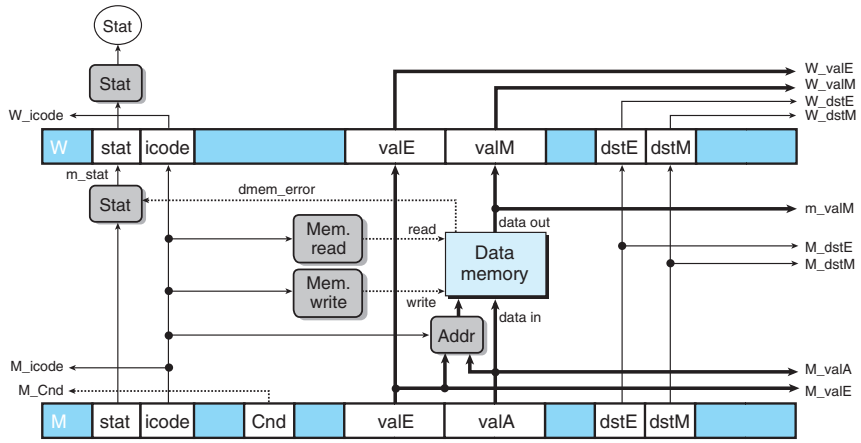


图 4.61　PIPE 的访存阶段逻辑。来自流水线寄存器 M 和 W 的许多信号被向下传送到更早的阶段，以提供写回结果、指令地址和转发的结果。

练习 4.36　在这一级中，我们可以通过检测数据存储器地址无效的情形，完成状态码 Stat 的计算。写出信号 m_stat 的 HCL 代码。

4.25 流水线控制逻辑

现在可以创建流水线控制逻辑，来完成 PIPE 的设计了。数据前递和分支预测这类机制不足以应付的下述四个控制情形，必须由这个逻辑来处理：

- **加载/使用冒险**。在一条从存储器读值的指令和一条使用该值的指令之间，流水线必须停顿一个周期。
- **处理 ret**。流水线必须停顿，直到 ret 指令到达写回阶段。
- **预测错误的分支**。等分支逻辑检测到一条跳转本不该发生时，跳转目标处的几条指令已经进入了流水线。这些指令必须被取消，取指应改为从跳转之后的指令开始。
- **异常**。当一条指令导致异常时，我们要禁止后面的指令对程序员可见状态的更新，并在异常指令到达写回阶段时停止执行。

我们将逐个给出这些情形期望的处理动作，然后设计出能同时应对全部这些情形的控制逻辑。

特殊控制情形期望的处理方式

对于加载/使用冒险，我们在 4.5.5 节已经描述了期望的流水线操作，并以图 4.54 的例子加以说明。只有 mrmovq 和 popq 指令会从存储器中读取数据。当 (1) 这两条指令之一位于执行阶段，且 (2) 译码阶段中有一条需要其目的寄存器的指令时，我们要把译码阶段中的后一条指令固定在原地，并在下一个周期向执行阶段注入一个气泡。此后，前递逻辑会解决这个数据冒险。流水线可以让流水线寄存器 D 保持固定状态，从而把译码阶段中的指令固定在原地；同时它还应让流水线寄存器 F 保持固定状态，这样下一条指令才会被再次取出。概括起来，实现这种流水线流动需要三件事：检测到冒险条件；让流水线寄存器 F 和 D 保持固定；向执行阶段注入一个气泡。

关于 ret 指令的处理，我们在 4.5.5 节描述了期望的流水线操作。流水线应当停顿三个周期，直到 ret 指令经过访存阶段时读出返回地址。

对下面这段程序的处理，曾在图 4.55 中用一幅简化的流水线图加以说明：

```
0x000:    irmovq stack,%rsp        #   初始化栈指针
0x00a:    call proc                #   过程调用
0x013:    irmovq $10,%rdx          #   返回点
0x01d:    halt
0x020: .pos 0x20
0x020: proc:                       # proc:
0x020:    ret                      #   立即返回
0x021:    rrmovq %rdx,%rbx         #   不执行
0x030: .pos 0x30
0x030: stack:                      # stack: 栈指针
```

图 4.62 给出了对这个示例程序中 ret 指令处理的详细视图。这里的关键观察是：通往流水线取指阶段根本没有地方可以注入气泡。在每个周期里，取指阶段都会从指令存储器中读出某条指令。看一下 4.5.7 节中实现 PC 预测逻辑的 HCL 代码可以发现，对 ret 指令，PC 的新值被预测为 valP，即其下一条指令的地址。在我们这个示例程序里，就是 0x021，ret 后面那条 rrmovq 的地址。对这个例子来说，这个预测是错的，在大多数情况下它也都是错的，但我们的设计并不打算把返回地址预测对。在三个时钟周期里，取指阶段都在停顿，于是那条 rrmovq 被取出来，然后在译码阶段被气泡取代。这一过程在图 4.62 中表现为三次取指，并有一条箭头指向穿过其余流水线阶段的气泡。最后，irmovq 指令在第 7 个周期被取出来。把图 4.62 与图 4.55 相比较可以看到，我们的实现达到了预期的效果，只是取指阶段要连续三个周期取出一条不正确的指令，显得有些古怪。

当分支预测错误时，我们在 4.5.5 节描述了期望的流水线操作，并以图 4.56 加以说明。预测错误会在跳转指令到达执行阶段时被检测到。控制逻辑随后会

Figure 4.62 provides a detailed view of the processing of the `ret` instruction for the example program. The key observation here is that there is no way to inject a bubble into the fetch stage of our pipeline. On every cycle, the fetch stage reads *some* instruction from the instruction memory. Looking at the HCL code for implementing the PC prediction logic in Section 4.5.7, we can see that for the `ret` instruction, the new value of the PC is predicted to be valP, the address of the following instruction. In our example program, this would be `0x021`, the address of the `rrmovq` instruction following the `ret`. This prediction is not correct for this example, nor would it be for most cases, but we are not attempting to predict return addresses correctly in our design. For three clock cycles, the fetch stage stalls, causing the `rrmovq` instruction to be fetched but then replaced by a bubble in the decode stage. This process is illustrated in Figure 4.62 by the three fetches, with an arrow leading down to the bubbles passing through the remaining pipeline stages. Finally, the `irmovq` instruction is fetched on cycle 7. Comparing Figure 4.62 with

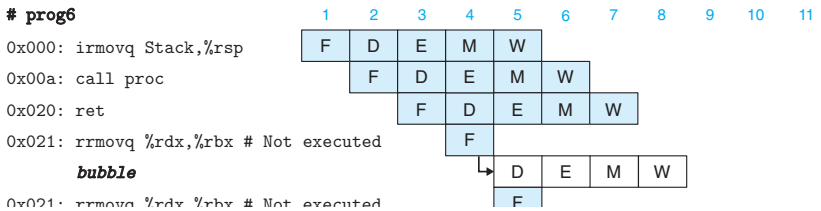


图 4.62

在下一个周期向译码阶段和执行阶段注入气泡，从而取消两条被错误取出的指令。同一个周期里，流水线会把正确的指令读进取指阶段。

对于导致异常的指令，我们必须让流水线实现与期望的 ISA 行为一致：其前面的指令全部完成，其后面的指令对程序状态不产生任何影响。要实现这些效果之所以复杂，是因为 (1) 异常是在程序执行的两个不同阶段（取指和访存）检测到的，而 (2) 程序状态是在三个不同阶段（执行、访存和写回）更新的。

我们的各级阶段设计在每个流水线寄存器里都带有一个状态码 `stat`，用来在指令流经各流水线阶段的同时跟踪它的状态。当异常发生时，我们把该信息记录为该指令状态的一部分，然后照常取出、译码和执行后续指令，仿佛什么都没发生。当异常指令到达访存阶段时，我们采取如下步骤，防止后面的指令修改程序员可见状态：(1) 禁止执行阶段中的指令设置条件码；(2) 向访存阶段注入气泡，以切断对数据存储器的任何写；(3) 当写回阶段中有异常指令时停顿写回阶段，从而让流水线停下来。

图 4.63 的流水线图展示了当一条导致异常的指令后面跟着一条会改变条件码的指令时，我们的流水线控制是如何处理的。在第 6 个周期，`pushq` 指令到达访存阶段并产生了存储器错误。同一个周期里，执行阶段中的 `addq` 指令为条件码生成了新值。当访存阶段或写回阶段中有异常指令时，我们通过检视 `m_stat` 和 `W_stat` 然后把信号 `set_cc` 置为 0，禁止条件码的设置。在图 4.63

的例子里还能看到“向访存阶段注入气泡”与“停顿写回阶段中的异常指令”的组合——pushq 指令一直停顿在写回阶段，而它后面的指令没有一条能越过执行阶段。

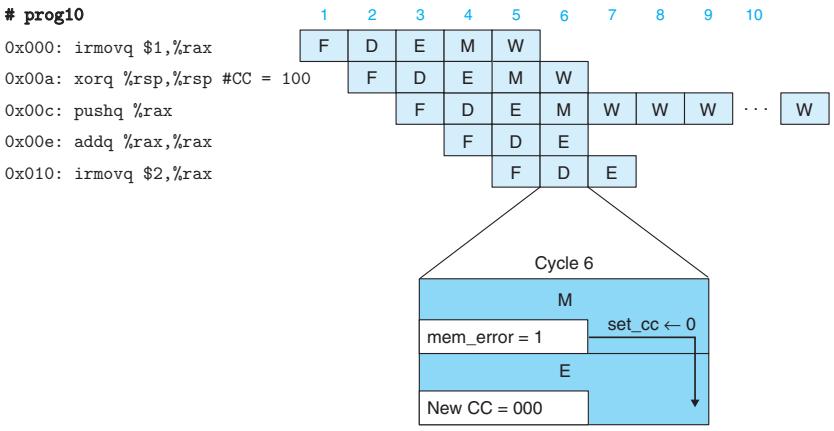


Figure 4.63 **Processing invalid memory reference exception.** On cycle 6, the invalid memory reference by the pushq instruction causes the updating of the condition codes to be disabled. The pipeline starts injecting bubbles into the memory stage and stalling the excepting instruction in the write-back stage.

图 4.63

通过把状态信号逐级传递、控制条件码的设置、并控制各流水线阶段，我们为异常实现了期望的行为：异常指令之前的所有指令都完成，后面的指令对程序员可见状态不产生任何影响。

检测特殊控制条件

图 4.64 汇总了需要特殊流水线控制的各种条件，给出了三类特殊情形出现时信号所要满足的布尔式。

图 4.64　流水线控制逻辑的检测条件。四类不同的条件要求通过停顿流水线或取消部分执行的指令来改变流水线的流动。

条件	触发
处理 ret	IRET ∈ {D_icode, E_icode, M_icode}
加载/使用冒险	E_icode ∈ {IMRMOVQ, IPOPQ} && E_dstM ∈ {d_srcA, d_srcB}
预测错误的分支	E_icode = IJXX && !e_Cnd
异常	m_stat ∈ {SADR, SINS, SHLT} || W_stat ∈ {SADR, SINS, SHLT}

这些表达式由一些简单的组合逻辑块实现，它们必须在时钟周期结束之前给出结果，以便在时钟上升、开始下一个周期时控制流水线寄存器的动作。在一个时钟周期内，流水线寄存器 D、E 和 M 分别保存着位于译码、执行和访存

级中的指令的状态。随着时钟周期接近尾声，信号 d_srcA 和 d_srcB 会被置为译码阶段中指令的源操作数的寄存器标识符。检测 ret 指令在流水线中的通过，只需检查译码、执行和访存阶段中指令的指令码。检测加载/使用冒险，只需检查执行阶段中的指令的类型（mrmovq 或 popq），并把它的目的寄存器与译码阶段中指令的源寄存器相比较。流水线控制逻辑应当在跳转指令位于执行阶段时检测出分支预测错误，这样等指令进入访存阶段时，就已经为从预测错误中恢复而准备好条件了。当跳转指令位于执行阶段时，信号 e_Cnd 表明这个跳转是否应当发生。检测异常指令的方法是检视访存阶段和写回阶段中指令的状态值。对访存阶段，用的是本阶段内部算出的信号 m_stat，而不是流水线寄存器中的 M_stat。这个内部信号把出现数据存储器地址错误的可能也纳入了进来。

流水线控制机制

图 4.65 给出了一些低层机制，流水线控制逻辑借助它们可以把流水线寄存器中的指令固定在原地，或者向流水线注入一个气泡。这些机制对 4.2.5 节描述的基本时钟寄存器做了小幅扩展。假设每个流水线寄存器都有两个控制输入 stall 和 bubble。这两个信号的取值决定了时钟上升时流水线寄存器如何更新。正常运行时（图 4.65(a)），这两个输入都置为 0，寄存器加载其输入作为新状态。当 stall 信号置为 1 时（图 4.65(b)），状态更新被禁用，寄存器保持它先前的状态。这样就可以把一条指令固定在某个流水线阶段里。当 bubble 信号置为 1 时（图 4.65(c)），寄存器的状态被置成某个固定的复位配置，等价于一条 nop 指令的状态。流水线寄存器的复位配置中 1 与 0 的具体取值模式，取决于该寄存器包含的域的集合。例如，要向流水线寄存器 D 注入一个气泡，我们希望它的 icode 域被置为常量 INOP（图 4.26）；要向流水线寄存器 E 注入一个气泡，我们希望它的 icode 域被置为 INOP，而 dstE、dstM、srcA 和 srcB 域被置为常量 RNONE。确定复位配置是硬件设计者在设计流水线寄存器时的任务之一，这里不讨论其细节。把 bubble 和 stall 两个信号同时置为 1 被视为错误。

图 4.66 流水线控制逻辑的动作。这些不同的条件要求通过停顿流水线或取消部分执行的指令来改变流水线的流动。

条件	流水线寄存器				
	F	D	E	M	W
处理 ret	停顿	注泡	正常	正常	正常
加载/使用冒险	停顿	停顿	注泡	正常	正常
预测错误的分支	正常	注泡	注泡	正常	正常

Figure 4.65 shows low-level mechanisms that allow the pipeline control logic to hold back an instruction in a pipeline register or to inject a bubble into the pipeline. These mechanisms involve small extensions to the basic clocked register described

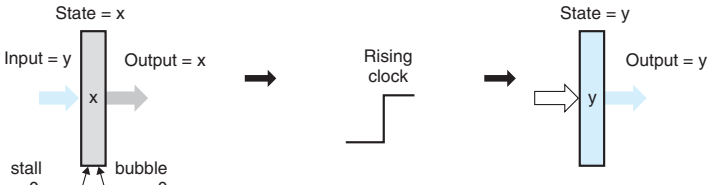


图 4.65

图 4.66 中的表给出了三类特殊条件下不同的流水线阶段各自应采取的动作。每个都是流水线寄存器的正常运行、停顿和注泡操作的某种组合。就时序而言，流水线寄存器的 `stall` 和 `bubble` 控制信号由若干逻辑块生成，这些逻辑块必须在时钟上升、开始下一个周期时使各个流水线寄存器加载、停顿或者注泡。有了对流水线寄存器设计的这点小扩展，我们就可以用组合逻辑、时钟寄存器和随机访问存储器这些基本构件，实现包含全部控制的完整流水线。

控制条件的组合

到目前为止，在讨论这些特殊流水线控制条件时，我们都假设任一时钟周期内至多出现一个特殊情形。系统设计中一个常见的错误就是没有处理多个特殊条件同时出现的情况。这里来分析一下存在的可能性。无须担心与程序异常有关的各种组合，因为我们在设计异常处理机制时已经把流水线中的其他指令考虑进去了。图 4.67 图示了引起其他三个特殊控制条件的流水线状态。这些图给出译码、执行和访存阶段的方框，阴影框表示条件成立时必须满足的具体约束。加载/使用冒险要求执行阶段中的指令从存储器读值放入寄存器，且译码阶段中的指令把这个寄存器作为源操作数。预测错误的分支要求执行阶段中的指令是一条跳转指令。`ret` 有三种可能的情形——这条指令可能位于译码、执行或访存阶段。随着 `ret` 指令在流水线中向前推进，靠前的流水线阶段会出现气泡。

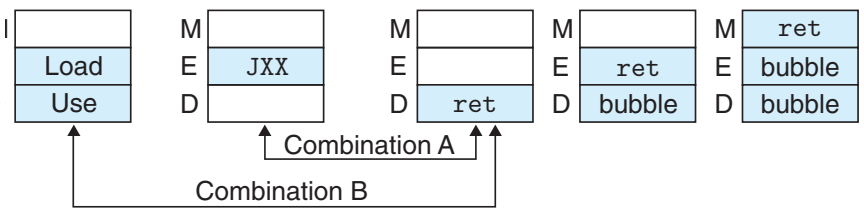


图 4.67　特殊控制条件下的流水线状态。所示的两对状态可能同时出现。

从这些图可以看出，大多数控制条件是互相排斥的。例如，加载/使用冒险和预测错误的分支不可能同时出现，因为前者要求执行阶段中的指令是加载指令（mrmovq 或 popq），而后者要求是跳转指令。类似地，第二种和第三种 ret 组合也不可能同时与加载/使用冒险或预测错误的分支出现。只有图中用箭头标出的两个组合可以同时出现。

组合 A 的情形是：执行阶段有一条不跳转的跳转指令，译码阶段有一条 ret 指令。要构造出这个组合，必须把 ret 放在一个不发生跳转的分支的目标位置。流水线控制逻辑应当检测出分支被预测错了，从而取消 ret 指令。

练习 4.37　写一个 Y86-64 汇编语言程序，用它构造组合 A 的情形，并判断控制逻辑对它的处理是否正确。

把组合 A 的条件所对应的控制动作（图 4.66）合并起来（假设注泡或停顿都要先于正常运行），得到如下流水线控制动作：

条件	流水线寄存器				
	F	D	E	M	W
处理 ret	停顿	注泡	正常	正常	正常
预测错误的分支	正常	注泡	注泡	正常	正常
组合	停顿	注泡	注泡	正常	正常

也就是说，它会被当作预测错误的分支来处理，只是在取指阶段多了一个停顿。幸运的是，在下一个周期，PC 选择逻辑会选择跳转之后的指令地址，而不是预测的程序计数器，因此流水线寄存器 F 中发生了什么无关紧要。我们可以断定，流水线能正确处理这一组合。

组合 B 的情形是一个加载/使用冒险，此时加载指令设置寄存器%rsp，而 ret 指令随后把这个寄存器用作源操作数，因为它必须从栈中弹出返回地址。流水线控制逻辑应当让 ret 指令固定在译码阶段。

练习 4.38　写一个 Y86-64 汇编语言程序，用它构造组合 B 的情形；如果流水线运行正确，这个程序应当以一条 halt 指令结束。

把组合 B 的条件所对应的控制动作（图 4.66）合并起来，得到如下流水线控制动作：

条件	流水线寄存器				
	F	D	E	M	W
处理 ret	停顿	注泡	正常	正常	正常
加载/使用冒险	停顿	停顿	注泡	正常	正常
组合	停顿	注泡 + 停顿	注泡	正常	正常
期望	停顿	停顿	注泡	正常	正常

如果这两套动作都被触发了，控制逻辑既会为避免加载/使用冒险而停顿 `ret` 指令，又会因为这条 `ret` 而向译码阶段注入气泡。显然我们不希望流水线同时执行这两套动作，而是希望它只执行为加载/使用冒险指定的动作；那些处理 `ret` 的动作要推迟一个周期再执行。

这一分析说明组合 B 需要特殊的处理。实际上，我们最初版本的 PIPE 控制逻辑实现并没有正确处理这个组合。虽然该设计通过了大量的模拟测试，但是正是这里给出的分析才揭示出它带有一个微妙的 bug。当运行一个有组合 B 的程序时，控制逻辑会把流水线寄存器 D 的 `bubble` 和 `stall` 两个信号都置为 1。这个例子说明了系统性分析的重要性，仅靠正常运行各类程序几乎不可能发现这个 bug；若不被发现，流水线将不能忠实地实现 ISA 的行为。

控制逻辑的实现

图 4.68 给出了流水线控制逻辑的整体结构。在来自流水线寄存器和各流水线阶段的信号的基础上，该逻辑生成各流水线寄存器的 `stall` 和 `bubble` 控制信号，并决定条件码寄存器是否应当更新。把图 4.64 的检测条件与图 4.66 的动作结合起来，我们就能写出各个流水线控制信号的 HCL 描述。

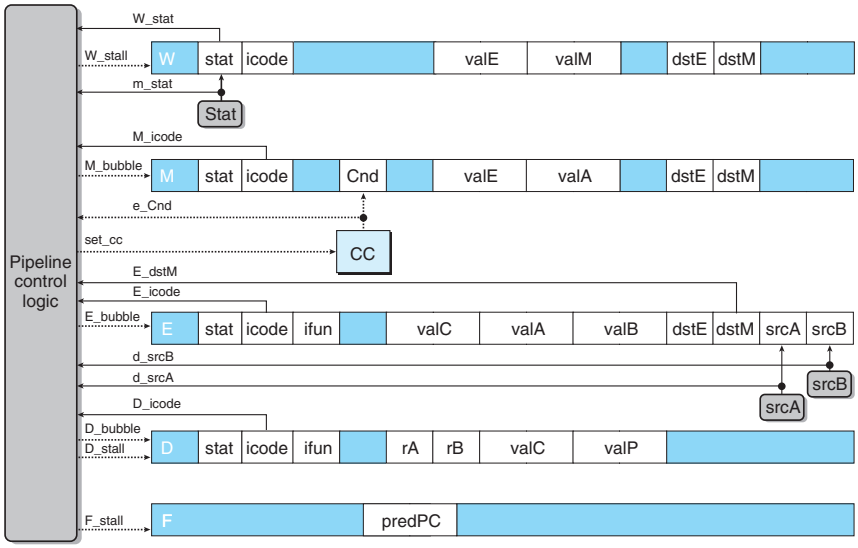


图 4.68　PIPE 的流水线控制逻辑。该逻辑改变指令流过流水线的正常流动，以处理特殊条件，例如过程返回、预测错误的分支、load/use 冒险和程序异常。

对加载/使用冒险或者 `ret` 指令，流水线寄存器 F 都必须停顿：

```
bool F_stall =
        # 加载/使用冒险的条件
        E_icode in { IMRMOVQ, IPOPQ } &&
         E_dstM in { d_srcA, d_srcB } ||
```

```
        # 在取指阶段停顿,让 ret 通过流水线
        IRET in { D_icode, E_icode, M_icode };
```

练习 4.39 写出 PIPE 实现中信号 D_stall 的 HCL 代码。

对预测错误的分支或者 ret 指令,流水线寄存器 D 应当置为注泡。不过,正如前一节的分析所表明的,当加载/使用冒险与 ret 指令同时出现时,它不应注入气泡:

```
bool D_bubble =
        # 预测错误的分支
        (E_icode == IJXX && !e_Cnd) ||
        # 在取指阶段停顿,让 ret 通过流水线
        # 但是排除加载/使用冒险的条件
        !(E_icode in { IMRMOVQ, IPOPQ } && E_dstM in { d_srcA,
      d_srcB }) &&
           IRET in { D_icode, E_icode, M_icode };
```

练习 4.40 写出 PIPE 实现中信号 E_bubble 的 HCL 代码。

练习 4.41 写出 PIPE 实现中信号 set_cc 的 HCL 代码。这种更新应当只对 OPq 类指令发生,并且要考虑程序异常的影响。

练习 4.42 写出 PIPE 实现中信号 M_bubble 和 W_stall 的 HCL 代码。后一个信号需要对图 4.64 中列出的异常条件加以修改。

至此,所有特殊的流水线控制信号值都覆盖了。在 PIPE 的完整 HCL 代码里,所有其余的流水线控制信号都置为 0。

4.26 性能分析

可以看到,所有需要流水线控制逻辑采取特殊动作的条件,都会让流水线达不到每个时钟周期发出一条新指令的目标。气泡被注入时会产生未被利用的流水线周期,所以可以通过测量注入流水线的气泡的频率来度量这种效率损失。一条 ret 指令产生三个气泡,一个加载/使用冒险产生一个,一次分支预测错误产生两个。我们可以通过估计 PIPE 执行每条指令平均需要的时钟周期数来量化这些惩罚项的总体影响,这个量称为每条指令的周期数(cycles per instruction,CPI)。它是流水线平均吞吐量的倒数,只是时间以时钟周期而不是皮秒计时。它是衡量设计体系结构效率的一个有用指标。

> **边栏 设计的测试**
> 我们已看到,即使对一个简单的微处理器,把 bug 引入设计的方式也不胜枚举。引入流水线之后,位于不同流水线级的指令之间存在许多微妙的交互。我们已经看到,许多设计上的难题都涉及不寻常的指令(例如向栈指针弹出)或不寻常的指令组合(例如一条不跳转的跳转后面跟着 ret)。我们还看到,异常处理给可能的流水线行

为增添了一个全新的维度。那么，怎样才能确信我们的设计是正确的呢？对硬件制造商来说，这是一个头等重要的问题，因为他们不能简单地报告一个错误，然后让用户通过因特网下载补丁代码。哪怕一个很简单的逻辑设计错误也可能带来严重的后果，尤其是随着微处理器越来越多地被用来操控关乎我们生命健康的系统，例如汽车防抱死制动系统、心脏起搏器和飞机控制系统。

让一个设计运行若干“典型”程序并加以模拟，不是一种足以测试系统的方法。彻底的测试要求设计出能够系统地生成大量测试的方法，让它尽可能多地执行各种不同的指令和指令组合。在创建 Y86-64 处理器设计的过程中，我们还编制了若干测试脚本，每个脚本生成许多不同的测试，运行处理器的模拟，并把得出的寄存器和存储器值与我们的 `yis` 指令集模拟器所给出的值加以比较。这些脚本的简述如下：

optest. 对不同的 Y86-64 指令、不同的源寄存器和目的寄存器，运行 49 个测试。

jtest. 对不同的跳转和调用指令、分支发生与不发生的不同组合，运行 64 个测试。

cmtest. 对不同的条件传送指令、不同的控制条件组合，运行 28 个测试。

htest. 对各种数据冒险可能性、源指令与目的指令的不同组合，以及指令对之间 `nop` 指令数量的不同取值，运行 600 个测试。

ctest. 基于 4.5.8 节中所做的类似分析，测试 22 种不同的控制组合。

etest. 测试 12 种组合：一条指令导致了异常，而其后的指令可能改变程序员可见状态。

这种测试方法的关键思想是：尽可能系统地生成测试，构造出可能引起流水线错误的各种条件。

如果暂时忽略异常对性能的影响（按定义，异常只会很少出现），可以把 CPI 设想成是在某个基准程序上运行处理器并观察执行阶段的动作。每个周期里，执行阶段要么 (1) 处理一条指令，使它继续流经后面的各级流水线直至完成；要么 (2) 处理一个因上述三种特殊情形之一而被注入的气泡。如果该阶段一共处理了 C_i 条指令和 C_b 个气泡，那么处理器执行这 C_i 条指令大约需要 $C_i + C_b$ 个时钟周期。说“大约”，是因为我们忽略了让指令开始流入流水线所需要的那几个周期。于是可以按下面的式子计算这个基准程序的 CPI：

$$\text{CPI} = \frac{C_i + C_b}{C_i} = 1.0 + \frac{C_b}{C_i}$$

也就是说，CPI 等于 1.0 加上惩罚项 C_b/C_i，后者表示平均每执行一条指令注入的气泡数。由于只有三类不同的指令类型会造成气泡被注入，我们可以把这个惩罚项拆分成三个部分：

边栏　设计的形式化验证

即使一个设计通过了大量测试，我们也不能确信它对所有可能的程序都能正确运行。可以测试的程序数量大得难以想象，哪怕只考虑由短代码段构成的测试。不过，

较新的形式化验证（formal verification）方法给我们带来了这样的希望：我们可以拥有严格考察系统所有可能行为、并判断是否存在设计错误的工具。

我们曾给 Y86-64 处理器的一个早期版本做形式化验证 [13]。我们搭建了一个框架，用它比较流水线设计 PIPE 与非流水线版本 SEQ 的行为，从而能对任意的机器级程序证明：两个处理器对程序员可见状态的影响完全相同。当然，我们的验证器并不能真的运行所有可能的程序——那样的程序有无穷多个——它采用的是归纳证明的一种形式，逐个周期地证明两个处理器之间的一致性。做这一分析要用符号方法来推理硬件：我们假定所有的程序值都是任意整数，并把 ALU 抽象成一种“黑箱”，让它对输入计算某个未指定的函数；我们唯一的假设是 SEQ 与 PIPE 的 ALU 计算的是同一个函数。

我们用控制逻辑的 HCL 描述来生成符号处理器模型中的控制逻辑，因此能够捕获 HCL 代码中的任何 bug。能够证明 SEQ 与 PIPE 等价，并不能保证它们中任何一个都忠实地实现了指令集体系结构，但它能发现任何由流水线设计错误导致的 bug，而这正是设计错误的主要来源。

在实验中，我们验证的不仅是与本节介绍的相近的 PIPE 版本，还有作为家庭作业给出的几个变体：增加新的指令、修改硬件能力，或者采用不同的分支预测策略。有趣的是，我们在所有设计中只发现了一个 bug——它出现在 4.5.8 节描述的控制组合 B 上，针对的是问题 4.58 所描述变体的解。这暴露了测试体系的一个弱点，促使我们向 `ctest` 测试脚本补充了若干测试用例。

形式化验证仍处于发展的早期阶段。这些工具往往难以使用，也没有能力处理大规模的设计。我们能够验证自己的处理器，部分原因是它们相对简单。即便如此，这项工作还是花了几周的时间和多次运行这些工具的功夫，而每次运行最多需要 8 小时的计算机时间。这是一个活跃的研究领域——有些工具已经商用，有些在 Intel、AMD 和 IBM 这样的公司里使用着。

网络边栏　ARCH:VLOG——流水线化 Y86-64 处理器的 Verilog 实现

我们已提到，现代逻辑设计要把硬件设计写成硬件描述语言中的文本表示。随后通过仿真和多种形式化验证工具来测试设计。对设计建立了信心之后，就能用逻辑综合工具把设计翻译成实际的逻辑电路。

我们已经用 Verilog 硬件描述语言为 Y86-64 处理器设计建立了模型。这些设计把实现处理器基本构件的模块与直接从 HCL 描述生成的控制逻辑组合在一起。其中一些设计已经能够被综合，把逻辑电路的描述下载到现场可编程门阵列（FPGA）硬件上，并在实际的程序上运行处理器。

$$\text{CPI} = 1.0 + l_p + m_p + r_p$$

式中，l_p（load penalty，加载惩罚）是加载/使用冒险停顿期间气泡被注入的平均频率，m_p（mispredicted branch penalty，预测错误分支惩罚）是取消因分支预测错误而取出的指令时气泡被注入的平均频率，r_p（return penalty，返回惩罚）是

为 ret 指令停顿期间气泡被注入的平均频率。每个惩罚项都表示因那个原因而注入的气泡总数(也就是 C_b 的一部分)除以执行的指令总数(C_i)。

要估计各个惩罚项,需要知道相关指令(加载、条件分支和返回)出现的频繁程度,并且对每种指令要知道该特定条件出现的频繁程度。为 CPI 的计算选取下面这组频率(它们与文献 [44] 和 [46] 报告的测量值相当):

- 加载指令(mrmovq 和 popq)占所有已执行指令的 25%。其中 20% 引起加载/使用冒险。
- 条件分支占所有已执行指令的 20%。其中 60% 发生跳转,40% 不发生跳转。
- 返回指令占所有已执行指令的 2%。

于是可以用三个频率的乘积来估计每个惩罚项:指令类型的频率、条件出现的频率,以及该条件出现时注入的气泡数:

起因	名称	指令频率	条件频率	气泡数	乘积
加载/使用	l_p	0.25	0.20	1	0.05
预测错误	m_p	0.20	0.40	2	0.16
返回	r_p	0.02	1.00	3	0.06
总惩罚					0.27

三项惩罚之和为 0.27,因此 CPI 为 1.27。

我们的目标是设计一条每个周期能发出一条指令的流水线,即 CPI 达到 1.0。这一目标没有完全达到,但整体性能仍然相当不错。还可以看出,要进一步降低 CPI,努力的方向应当集中在预测错误的分支上。在 0.27 的总惩罚中,预测错误分支占了 0.16,因为条件分支很常见,我们的预测策略屡屡失手,而且每次预测错误都要取消两条指令。

练习 4.43 假设采用一种成功率为 65% 的分支预测策略,例如 4.5.4 节描述的 "向后跳转预测发生、向前跳转预测不发生"(backward taken, forward not taken,BTFNT)。假设其他所有频率都不受影响,这对 CPI 会产生什么影响?

练习 4.44 分析一下你在练习 4.5 和练习 4.6 中所写的程序里,使用条件数据传送与使用条件控制传送两种方案相对性能如何。假设用这些程序来计算一个很长数组的元素绝对值之和,这样整体性能主要由内循环需要的周期数决定。假设我们的跳转指令被预测为发生跳转,且数组中大约 50% 的值是正数。

A. 平均来说,两个程序的内循环各执行多少条指令?

B. 平均来说,两个程序的内循环各注入多少个气泡?

C. 两个程序平均每个数组元素需要多少个时钟周期?

4.27 尚未完成的工作

我们已经建立了 PIPE 流水线微处理器的结构,设计了各个控制逻辑块,并实现了流水线控制逻辑,使正常流水线流动不足以应付的特殊情形能得到处理。尽管如此,PIPE 还缺少在实际微处理器设计中必备的几个关键特性。我们着重指出其中的几个,并讨论补上它们需要什么。

多周期指令

Y86-64 指令集中的所有指令都只涉及数值相加这类简单操作,这些操作可以在执行阶段的一个时钟周期内处理完毕。在一个更完备的指令集中,还需要实现需要更复杂操作(例如整数乘、除法和浮点运算)的指令。

除法与浮点数运算。在中等性能的处理器(如 PIPE)中,这些操作的典型执行时间从浮点加法的 3 ~ 4 个周期一直到整数除法的 64 个周期。要实现这些指令,我们既需要用于执行这些计算的额外硬件,也需要一种把对它们的处理与流水线的其余部分协调起来的机制。

实现多周期指令的一个简单做法,是直接扩充执行级逻辑的能力,为它加上整数和浮点算术单元。指令在执行级停留所需数目的时钟周期,这会使取指级和译码级停顿。这种方法实现起来简单,但得到的性能并不好。

用一些独立于主流水线运转的特殊硬件功能单元来处理较复杂的运算,可以获得更好的性能。通常会有一个功能单元负责整数乘法和除法,另一个负责浮点运算。当一条指令进入译码级时,就可以把它发射给专用单元。当该单元执行运算时,流水线继续处理其他指令。浮点单元通常自身也是流水线化的,因此多个操作能同时在主流水线和各个单元中并发执行。

不同单元的操作必须同步,才能避免不正确的行为。例如,如果由不同单元处理的操作之间存在数据依赖,控制逻辑可能需要停顿系统的一部分,直到系统中另一部分处理的某个操作的结果完成。通常会用各种不同形式的转发把结果从系统的一部分传送到其他部分,就像我们在 PIPE 的各流水级之间看到的那样。整体设计会比我们用 PIPE 看到的更复杂,但仍然可以运用停顿、转发和流水线控制的相同技术,使整体行为与顺序的 ISA 模型一致。

4.27.1 与存储器系统的接口

在讲述 PIPE 时,我们假定取指单元和数据存储器都能在一个时钟周期内读或写任意一个存储位置。我们还忽略了自修改代码可能引起的冒险——这种代码里,某条指令会向后面的指令从中取出的存储区域写入内容。此外,我

们是按存储位置的虚拟地址来引用它们的，而在实际进行读或写操作之前，这些虚拟地址必须先被翻译成物理地址。显然，要在单个时钟周期内完成所有这些处理是不现实的。更糟的是，正在访问的存储值可能位于磁盘上，要花数百万个时钟周期才能读入处理器存储器。

正如第 6 章和第 9 章将要讨论的，处理器的存储器系统把多种硬件存储器与操作系统软件结合起来管理虚拟内存系统。存储器系统组织成一个层次结构，由更快但更小的存储器保存正在被访问的那部分存储器内容，

由更慢但更大的存储器作为后备。在离处理器最近的层次上，高速缓存提供对被引用最频繁的存储位置的快速访问。典型的处理器有两个第一级高速缓存——一个用于读取指令，一个用于读和写数据。另一种类型的高速缓存存储器称为转换后备缓冲区（translation look-aside buffer，TLB），提供从虚拟地址到物理地址的快速转换。把 TLB 与高速缓存结合起来使用，大多数时候确实可以在单个时钟周期内读取指令以及读、写数据。因此，我们对处理器存储器访问的简化看法实际上相当合理。

虽然高速缓存保存着被引用最频繁的存储位置，但仍会出现高速缓存不命中的情况——某次访问指向的位置不在高速缓存中。最好的情况下，缺失的数据可以从更高层的高速缓存或者处理器的主存中取回，需要 3 ～ 20 个时钟周期。与此同时，流水线只是停顿，把指令保持在取指级或访存级，直到高速缓存能完成这个读或写操作。就我们的流水线设计而言，这可以通过给流水线控制逻辑增加更多停顿条件来实现。高速缓存不命中以及随之而来的与流水线的同步完全由硬件处理，把所需时间降到少数几个时钟周期。

有些情况下，被引用的存储位置实际上保存在磁盘或非易失性存储器中。此时硬件会发出缺页异常。像其他异常一样，这会使处理器调用操作系统的异常处理代码。这段代码随后会安排一次从磁盘到主存的传送。传送完成后，操作系统返回原来的程序，引发缺页的那条指令被重新执行。这一次存储器访问将会成功，尽管可能造成一次高速缓存不命中。由硬件调用一个操作系统例程、该例程运行完再把控制交还给硬件，这样硬件和系统软件就能协作处理缺页。由于访问磁盘可能需要数百万个时钟周期，OS 缺页处理程序所执行的几千个周期对性能影响很小。

从处理器的角度看，处理短时间高速缓存不命中的停顿与处理长时间缺页的异常处理相结合，就能消除存储器层次结构给存储器访问时间带来的任何不可预测性。

4.28 小结

我们已经看到,ISA 在处理器的行为——就指令集合及其编码而言——与处理器如何实现之间提供了一个抽象层。ISA 为程序执行提供了一个非常顺序化的视图:每条指令都先执行完毕,下一条指令才开始执行。

边栏 最先进的微处理器设计

一个五级流水线,就像我们用 PIPE 处理器展示的那样,在 20 世纪 80 年代中期代表了处理器设计的最先进水平。Patterson 在伯克利的研究小组开发的 RISC 原型处理器,为 Sun Microsystems 公司 1987 年开发的第一款 SPARC 处理器奠定了基础。Hennessy 在斯坦福的研究小组开发的处理器则由 MIPS Technologies(Hennessy 参与创立的一家公司)在 1986 年实现商业化。这两款处理器都使用五级流水线。Intel 公司的 i486 处理器同样使用五级流水线,只是在各级之间采用了不同的职责划分:它有两个译码级,还有一个执行与访存合并的级 [27]。

这些流水线设计的吞吐量至多达到每个时钟周期一条指令。4.5.9 节描述的 CPI(cycles per instruction,每条指令的周期数)度量永远不会小于 1.0。每个流水级一次只能处理一条指令。较新的处理器支持超标量操作,也就是说,它们能通过并行地取指、译码和执行多条指令,取得小于 1.0 的 CPI。随着超标量处理器越来越普遍,公认的性能度量已经从 CPI 转变为它的倒数——每个周期执行的指令条数的平均值,即 IPC。对超标量处理器,它可能会超过 1.0。最先进的设计使用一种称为乱序执行(out-of-order execution)的技术来并行执行多条指令,其顺序可能与程序中的顺序完全不同,同时保持由顺序 ISA 模型所隐含的整体行为。这种执行形式将在第 5 章讨论程序优化时介绍。

不过,流水线处理器并非只是历史上的产物。市面上大多数处理器都用在嵌入式系统中,控制汽车功能、消费产品和其他设备,处理器并不直接呈现在系统用户面前。在这些应用中,像本章探索的这类流水线处理器的简单性,使它的成本和功耗需求都低于性能更高的型号。

最近,随着多核处理器流行起来,有人主张与其在单块芯片上集成数量较少但更复杂的处理器,不如集成许多简单的处理器,从而获得更大的总体计算能力。这种策略有时被称为“众核”(many-core)处理器 [10]。

我们通过从 x86-64 指令出发,大幅简化数据类型、寻址方式和指令编码,定义了 Y86-64 指令集。得到的 ISA 兼有 RISC 和 CISC 指令集的属性。然后,我们把执行不同指令所需的处理组织成五个阶段,每个阶段的操作随所执行的指令而变化。据此,我们构造了 SEQ 处理器:指令流经全部五个阶段,每个时钟周期完整地执行一条指令。

流水线化通过让不同流水级并发工作来改进系统的吞吐量性能。任一时刻,多个操作正由不同的流水级处理。在引入这种并发性时,我们必须小心,确保它提供与程序的顺序执行相同的程序级行为。我们是通过重新组织 SEQ 的各部分得到 SEQ+、再加上流水线寄存器创建 PIPE- 流水线来引入流水线化的。

网络边栏 ARCH:HCL Y86-64 处理器的 HCL 描述
在本章中,我们查看了几个简单逻辑设计的部分 HCL 代码,以及 Y86-64 处理器 SEQ 和 PIPE 的控制逻辑的部分 HCL 代码。作为参考,我们提供了 HCL 语言的文档和这两个处理器控制逻辑的完整 HCL 描述。每个描述只需要 5 ～ 7 页的 HCL 代码,把它们完整地研究一遍是值得的。

我们通过增加转发逻辑来加快把一条指令的结果传送给另一条指令,从而提高了流水线的性能。有几种特殊情况需要额外的流水线控制逻辑来停顿或取消某些流水级。

我们的设计包括了处理异常的基本机制:我们确保只有截至引发异常的那条指令为止的指令会影响程序员可见的状态。实现完整的异常处理会困难得多。在流水线化和并行化程度更高的系统中,正确处理异常还会更加复杂。

在本章中,我们学到了关于处理器设计的几点重要经验:

- 管理复杂性是头等大事。我们要以最低的成本对硬件资源做最优的利用,以获得最高性能。为此,我们创建了一个非常简单而统一的框架来处理所有不同的指令类型。有了这个框架,我们就可以让处理不同指令类型的逻辑共享各硬件单元。
- 我们不需要直接实现 ISA。直接实现 ISA 意味着一种非常顺序化的设计。为了取得更高的性能,我们要利用硬件同时执行许多运算的能力。这导致了流水线设计的使用。通过仔细的设计和分析,我们可以处理各种流水线冒险,从而使运行一个程序的整体效果与 ISA 模型的结果完全一致。
- 硬件设计人员必须一丝不苟。芯片一旦制造出来,几乎不可能改正任何错误。第一次就把设计做对非常重要。这意味着要认真分析各种不同的指令类型和组合,甚至包括那些看似无意义的组合,比如弹出到栈指针。设计必须用系统化的模拟测试程序进行彻底的测试。在开发 PIPE 的控制逻辑时,我们的设计中就曾有一个微妙的缺陷,直到对控制组合做了仔细而系统的分析之后才被发现。

4.28.1 Y86-64 模拟器

本章的实验材料包括 SEQ 和 PIPE 处理器的模拟器。每个模拟器都有两个版本:

- GUI (graphic user interface, 图形用户界面) 版本以图形窗口显示存储器、程序代码和处理器状态。这提供了一种直观查看指令流经处理器的方式。控制面板还支持交互式地复位、单步执行或运行模拟器。

- 文本版运行同一个模拟器,但它只通过向终端打印信息来显示内容。这个版本对调试没有多大帮助,不过它支持对处理器的自动化测试。

模拟器的控制逻辑是这样生成的:把各逻辑块的 HCL 声明翻译成 C 代码,然后把这些代码与模拟器的其余代码一起编译和链接。有了这样的组合,你就可以用模拟器来测试原始设计的各种变体。还提供了测试脚本,它们能充分检验各条指令和各种冒险可能。

4.29 参考文献说明

对逻辑设计有进一步学习兴趣的读者, Katz 和 Borriello 的逻辑设计教材 [58] 是标准的入门读物, 强调硬件描述语言的使用。Hennessy 和 Patterson 的计算机体系结构教材 [46] 对处理器设计作了大量讲述, 既包括我们在这里呈现的这类简单流水线, 也包括能并行执行更多条指令的高级处理器。Shriver 和 Smith[101] 对 AMD 制造的与 Intel 兼容的 x86-64 处理器给出了非常详尽的介绍。

4.30 家庭作业

家庭作业 4.45 ▲ 在 3.4.2 节中, x86-64 的 `pushq` 指令被描述成先把栈指针递减, 然后把寄存器存储到栈指针位置。因此, 如果我们有一条形如 `pushq REG`(REG 为某个寄存器)的指令,它将等价于下面的指令序列:

```
subq $8,%rsp              # 递减栈指针
 movq REG,(%rsp)           # 把 REG 存到栈上
```

A. 根据练习 4.7 中所做的分析,这个指令序列正确地描述了指令 `pushq %rsp` 的行为吗? 请解释。

B. 你要如何改写这个指令序列,使它既正确描述 REG 就是`%rsp` 的情形,也正确描述其他任何寄存器的情形?

家庭作业 4.46 ▲ 在 3.4.2 节中,x86-64 的 `popq` 指令被描述成先把栈顶的内容复制到目的寄存器,然后再递增栈指针。因此,如果我们有一条形如 `popq REG` 的指令,它将等价于下面的指令序列:

```
movq (%rsp),REG           # 从栈中读出 REG
 addq $8,%rsp              # 递增栈指针
```

A. 根据练习 4.8 中所做的分析, 这个指令序列正确地描述了指令 `popq %rsp` 的行为吗? 请解释。

B. 你要如何改写这个指令序列,使它既正确描述 REG 就是`%rsp` 的情形,也正确描述其他任何寄存器的情形?

家庭作业 4.47 ▲▲▲ 你的任务是要编写一个执行冒泡排序的 Y86-64 程序。作为参考，下面这个 C 函数用数组引用的方式实现了冒泡排序：

```
1  /* 冒泡排序:数组版 */
2  void bubble_a(long *data, long count) {
3      long i, last;
4      for (last = count-1; last > 0; last--) {
5          for (i = 0; i < last; i++)
6              if (data[i+1] < data[i]) {
7                  /* 交换相邻元素 */
8                  long t = data[i+1];
9                  data[i+1] = data[i];
10                 data[i] = t;
11             }
12     }
13 }
```

A. 编写并测试一个用指针而不是用数组下标来引用数组元素的 C 版本。

B. 编写并测试一个由该函数和测试代码组成的 Y86-64 程序。你会发现，让你的实现模仿编译你的 C 代码生成的 x86-64 代码会很有效。虽然指针比较通常是用无符号算术完成的，但在这道练习中你可以使用有符号算术。

家庭作业 4.48 ▲▲ 修改你在家庭作业 4.47 中写的代码，使冒泡排序函数中的测试和交换（第 6 ~ 11 行）不使用任何跳转，并且至多用三条条件传送指令。

家庭作业 4.49 ▲▲▲ 修改你在家庭作业 4.47 中写的代码，使冒泡排序函数中的测试和交换(第 6 ~ 11 行)不使用任何跳转，只用一条条件传送指令。

家庭作业 4.50 ▲▲▲ 在 3.6.8 节中我们看到，实现 `switch` 语句的一个常用方法是创建一组代码块，然后用跳转表来索引这些代码块。考虑下面图 4.69 中函数 `switchv` 的 C 代码及其相关的测试代码：

```
#include <stdio.h>
/* Example use of switch statement */
long switchv(long idx) {
long result = 0;
switch(idx) {
case 0:
result = 0xaaa;
break;
case 2:
case 5:
result = 0xbbb;
break;
case 3:
result = 0xccc;
break;
default:
result = 0xddd;
```

```
}
return result;
}
/* Testing Code */
#define CNT 8
#define MINVAL -1
int main() {
long vals[CNT];
long i;
for (i = 0; i < CNT; i++) {
vals[i] = switchv(i + MINVAL);
printf("idx = %ld, val = 0x%lx\n", i + MINVAL, vals[i]);
}
return 0;
}
Figure 4.69
Switch statements can be translated into Y86-64 code. This requires
implementation of a jump table.
```

请用跳转表以 Y86-64 实现 `switchv`。虽然 Y86-64 指令集没有包含间接跳转指令,但你可以先把计算出的地址压入栈中,再执行 `ret` 指令,从而获得同样的效果。请实现类似于 C 代码中所示那样的测试代码,来证明你的 `switchv` 实现既处理了显式列出的情形,也处理了触发 default 的情形。

家庭作业 4.51 ▲ 练习 4.3 引进了把立即数加到寄存器的 `iaddq` 指令。请描述实现这条指令时要执行的计算。可以参照 `irmovq` 和 `OPq`(图 4.18)的计算作为范例。

家庭作业 4.52 ▲▲ 文件 `seq-full.hcl` 包含 SEQ 的 HCL 描述,以及十六进制值为 `C`(`iaddq` 的指令代码)的常数 `IIADDQ` 的声明。请修改各控制逻辑块的 HCL 描述,以实现练习 4.3 和家庭作业 4.51 所描述的 `iaddq` 指令。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.53 ▲▲▲ 假设我们想创建一个低成本的流水线处理器,基于我们为 `PIPE-`(图 4.41)设计的结构,不带任何旁路。这种设计将通过停顿来处理所有数据依赖———一直停顿到产生所需值的指令通过写回级为止。

文件 `pipe-stall.hcl` 包含 PIPE 的 HCL 代码的一个修改版本,其中旁路逻辑已经被禁用。也就是说,信号 `e_valA` 和 `e_valB` 只是如下声明:

```
## 不要修改下面的代码。
## 不做转发。valA 要么是 valP,要么是从寄存器堆读出的值
word d_valA = [
        D_icode in { ICALL, IJXX } : D_valP;   # 使用递增过的 PC
        1 : d_rvalA;                           # 使用从寄存器堆读出的值
];
## 不做转发。valB 为从寄存器堆读出的值
word d_valB = d_rvalB;
```

请修改这个文件末尾的流水线控制逻辑,使它能正确处理所有可能的控制冒险和数据冒险。作为设计工作的一部分,你应该像我们在设计 PIPE 的流水线控制逻辑时那样,分析各种控制情形的组合。你会发现很多不同的组合都可能出现,因为要求流水线停顿的条件多得多。务必确保你的控制逻辑正确处理每一种组合。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.54 ▲▲ 文件 pipe-full.hcl 包含 PIPE 的 HCL 描述的一份副本,以及常数 IIADDQ 的声明。请修改这个文件,以实现练习 4.3 和家庭作业 4.51 所描述的 iaddq 指令。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.55 ▲▲▲ 文件 pipe-nt.hcl 包含 PIPE 的 HCL 代码的一份副本,外加一处对常数 J_YES(值为 0,是无条件跳转指令的函数码)的声明。请修改分支预测逻辑,使它预测条件跳转不发生,同时继续预测无条件跳转和 call 发生跳转。你需要设计一种方法,把 $valC$(即跳转目标地址)送入流水线寄存器 M,以便从预测错误的分支中恢复。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.56 ▲▲▲ 文件 pipe-btfnt.hcl 包含 PIPE 的 HCL 代码的一份副本,外加一处对常数 J_YES(值为 0,是无条件跳转指令的函数码)的声明。请修改分支预测逻辑,使它预测条件跳转在 $valC < valP$ 时发生(向后跳转),在 $valC \geq valP$ 时不发生(向前跳转)。(Y86-64 不支持无符号算术,所以你应该用一次有符号比较来实现这个测试。)继续预测无条件跳转和 call 发生跳转。你需要设计一种方法,把 $valC$ 和 $valP$ 两个值都送入流水线寄存器 M,以便从预测错误的分支中恢复。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.57 ▲▲▲ 在我们的 PIPE 设计中,只要有一条指令执行加载、把一个值从存储器读入寄存器,而下一条指令把这个寄存器作为源操作数,我们就会产生一个停顿。当源操作数要在执行级使用时,这种停顿是避免冒险的唯一办法。但对于第二条指令把源操作数存储到存储器的情形,比如 rmmovq 或 pushq 指令,这种停顿是不必要的。考虑下面这些代码示例:

```
1  mrmovq 0(%rcx),%rdx     # 加载 1
2  pushq %rdx              # 存储 1
3  nop
4  popq %rdx               # 加载 2
5  rmmovq %rax,0(%rdx)     # 存储 2
```

在第 1 行和第 2 行中, mrmovq 指令把一个值从存储器读入%rdx, 随后 pushq 指令把这个值压入栈。我们为 PIPE 作出的设计会让 pushq 指令停顿,

以避免加载/使用冒险。不过要注意,直到到达访存级时 `pushq` 才需要`%rdx` 的值。我们可以增加一条额外的旁路通路,如图 4.70 所示,把存储器的输出(信号 `m_valM`)转发给流水线寄存器 M 的 valA 字段。在下一个时钟周期中,这个被转发的值随后就可以写入存储器。这种技术称为加载转发。

要注意,上面代码序列中的第二个例子(第 4 行和第 5 行)不能利用加载转发。`popq` 指令加载的值会

被下一条指令用作地址计算的一部分,而且这个值要在执行级而不是访存级被用到。

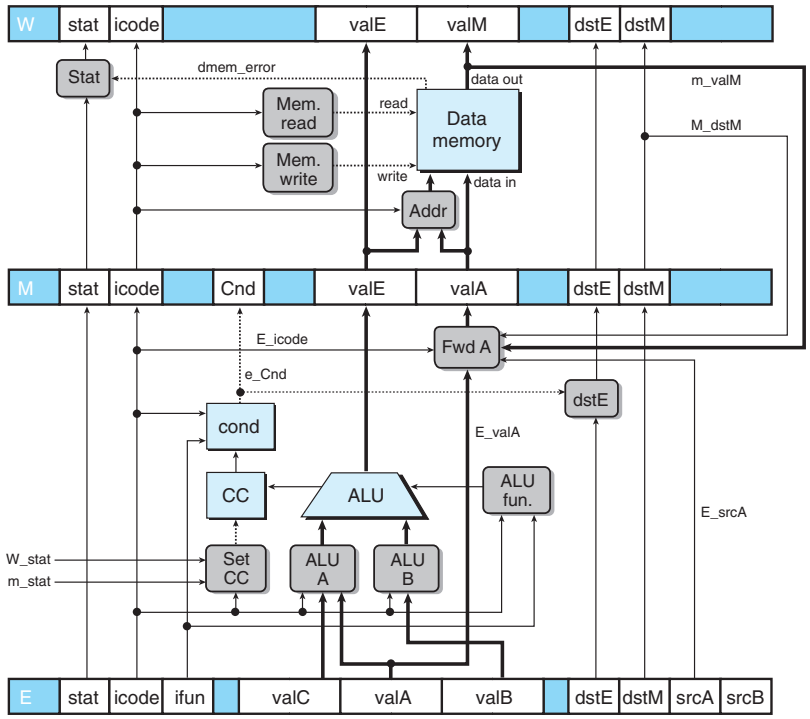


Figure 4.70 Execute and memory stages capable of load forwarding. By adding a bypass path from the memory output to the source of valA in pipeline register M, we can use forwarding rather than stalling for one form of load/use hazard. This is the subject of Problem 4.57.

图 4.70

A. 写出一个逻辑公式,描述加载/使用冒险的检测条件,类似图 4.64 所给的那个,但它不会在可以用加载转发的情形造成停顿。

B. 文件 `pipe-lf.hcl` 包含 `PIPE` 控制逻辑的一个修改版本。它包含信号 `e_valA` 的一个定义,用于实现图 4.70 中标着“Fwd A”的块。它还把流水线控制逻辑中加载/使用冒险的条件设置为零,因此该流水线控制逻辑不会检测

任何形式的加载/使用冒险。请修改这个 HCL 描述，实现加载转发。关于如何为你的解答生成模拟器以及如何测试它，请参见实验材料中的说明。

家庭作业 4.58 ▲▲▲ 我们的流水线设计有点不切实际：我们的寄存器堆有两个写端口，但只有 popq 指令要求同时写两个端口。所有其他指令因此都可以只用单个写端口，用它来写 valE 和 valM。下图展示了写回逻辑的一个修改版本，其中我们把两个写回寄存器 ID（W_dstE 和 W_dstM）合并成单个信号 w_dstE，把两个写回值（W_valE 和 W_valM）合并成单个信号 w_valE：

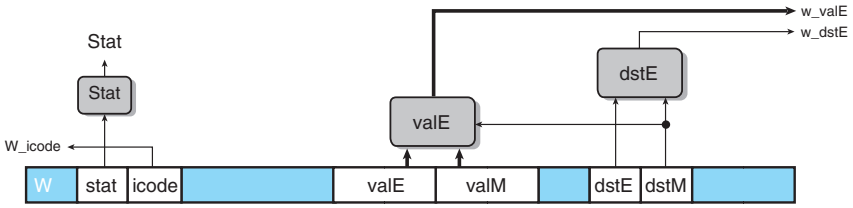


图 4.58-port （示意图）

执行合并的逻辑的 HCL 代码如下：

```
## 设置 E 口的寄存器 ID
word w_dstE = [
        ## 从 valM 写
        W_dstM != RNONE : W_dstM;
        1: W_dstE;
];

## 设置 E 口的值
word w_valE = [
        W_dstM != RNONE : W_valM;
        1: W_valE;
];
```

这些多路选择器的控制由 dstE 决定——当它指示存在某个寄存器时，就选择端口 E 的值；否则选择端口 M 的值。

随后在模拟器模型中，我们可以禁用寄存器端口 M，HCL 代码如下：

```
## 禁用寄存器端口 M
## 设置 M 口寄存器 ID
word w_dstM = RNONE;
## 设置 M 口值
word w_valM = 0;
```

这样，难点就变成设计一种处理 popq 的方法。一种做法是用控制逻辑动态地处理指令 popq rA，使它具有与下面两条指令组成的序列相同的效果：

```
iaddq $8, %rsp
 mrmovq -8(%rsp), rA
```

(iaddq 指令的说明见练习 4.3。)要注意这两条指令的先后顺序,以保证 popq %rsp 能正确工作。实现方法是让译码级的逻辑按处理上面所列 iaddq 的方式来处理 popq,差别只是它预测下一 PC 等于当前 PC。在下一个周期中,这条 popq 指令会被重新取指,但指令代码被转换成一个特殊值 IPOP2。这个值被当作一个特殊指令来处理,它的行为与上面所列的 mrmovq 指令相同。

文件 pipe-1w.hcl 包含上述修改过的写端口逻辑。它包含一个十六进制值为 E 的常数 IPOP2 的声明,还包含信号 f_icode 的定义,该信号为流水线寄存器 D 生成 icode 字段。可以修改这个定义,使 popq 指令第二次被取指时插入指令代码 IPOP2。这个 HCL 文件还包含信号 f_pc 的声明,它是在取指级由标着"Select PC"的块(图 4.57)生成的程序计数值。

请按我们描述的方式修改这个文件中的控制逻辑来处理 popq 指令。关于如何为你的解答生成模拟器以及如何测试它,请参见实验材料中的说明。

家庭作业 4.59 ▲▲ 比较冒泡排序的三个版本(家庭作业 4.47、4.48 和 4.49)的性能。请解释为什么一个版本比另一个性能更好。

4.31 练习答案

答案 4.1 手工编码指令相当费事,但它能使你更牢固地理解汇编代码由汇编器转换成字节序列这个概念。下面的输出来自我们的 Y86-64 汇编器,每一行给出一个地址,以及从那个地址开始的字节序列:

```
1   0x100:                      | .pos 0x100           # 从地址 0x100
       开始放代码
2   0x100: 30f30f00000000000000 |     irmovq $15,%rbx
3   0x10a: 2031                 |     rrmovq %rbx,%rcx
4   0x10c:                      | loop:
5   0x10c: 4013fdffffffffffffff |     rmmovq %rcx,-3(%rbx)
6   0x116: 6031                 |     addq   %rbx,%rcx
7   0x118: 700c01000000000000   |     jmp loop
```

这种编码有几个特征值得注意:

- 十进制 15(第 2 行)的十六进制表示是 0x000000000000000f。把这些字节按反向顺序写出,就得到 0f 00 00 00 00 00 00 00。
- 十进制 −3(第 5 行)的十六进制表示是 0xfffffffffffffffd。把这些字节按反向顺序写出,就得到 fd ff ff ff ff ff ff ff。
- 这段代码从地址 0x100 开始。第一条指令需要 10 字节,第二条需要 2 字节。因此,循环的目标是 0x0000010c。把这些字节按反向顺序写出,就得到 0c 01 00 00 00 00 00 00。

答案 4.2 手工解码一个字节序列,能帮助你理解处理器所面对的任务:它必须读取字节序列,并确定要执行哪些指令。下面我们给出用来生成这些字节序列中每一条的汇编代码。在汇编代码的左边,可以看到每条指令的地址和字节序列。

A. 一些用立即数和地址位移的操作:

```
0x100: 30f3fcffffffffffff |           irmovq $-4,%rbx
0x10a: 4063000800000000000 |           rmmovq %rsi,0x800(%rbx)
0x114: 00                  |           halt
```

B. 包含一次函数调用的代码:

```
0x200: a06f                |     pushq %rsi
0x202: 800c02000000000000  |     call proc
0x20b: 00                  |     halt
0x20c:                     | proc:
0x20c: 30f30a00000000000000 |     irmovq $10,%rbx
0x216: 90                  |     ret
```

C. 包含非法指令说明符字节 `0xf0` 的代码:

```
0x300: 5054070000000000000 |     mrmovq 7(%rsp),%rbp
0x30a: 10                  |     nop
0x30b: f0                  | .byte 0xf0    # 无效的指令代码
0x30c: b01f                |     popq %rcx
```

D. 包含一个跳转操作的代码:

```
0x400:                     | loop:
0x400: 6113                |     subq %rcx, %rbx
0x402: 730004000000000000  |     je loop
0x40b: 00                  |     halt
```

E. `pushq` 指令的第二个字节无效的代码:

```
0x500: 6362                |      xorq %rsi,%rdx
0x502: a0                  |      .byte 0xa0   # pushq 指令的代码
0x503: f0                  |      .byte 0xf0   # 无效的寄存器说明符字节
```

答案 4.3 使用 `iaddq` 指令,我们可以把 sum 函数改写为

```
# long sum(long *start, long count)
# start 在 %rdi 中,count 在 %rsi 中
sum:
        xorq %rax,%rax          # sum = 0
        andq %rsi,%rsi          # 设置条件码
        jmp    test
loop:
        mrmovq (%rdi),%r10      # 取得 *start
```

```
        addq %r10,%rax          # 加到 sum 上
        iaddq $8,%rdi           # start++
        iaddq $-1,%rsi          # count--
test:
        jne    loop             # 为 0 时停止
        ret
```

答案 4.4 gcc 在 x86-64 机器上运行时,为 `rproduct` 生成如下代码:

```
long rproduct(long *start, long count)
start 在 %rdi 中,count 在 %rsi 中
rproduct:
  movl     $1, %eax
  testq    %rsi, %rsi
  jle      .L9
  pushq    %rbx
  movq     (%rdi), %rbx
  subq     $1, %rsi
  addq     $8, %rdi
  call     rproduct
  imulq    %rbx, %rax
  popq     %rbx
.L9:
  rep; ret
```

这段代码可以很容易地改编成 Y86-64 代码:

```
# long rproduct(long *start, long count)
# start 在 %rdi 中,count 在 %rsi 中
rproduct:
        xorq   %rax,%rax        # 把返回值设为 1
        andq   %rsi,%rsi        # 设置条件码
        je     return           # 若 count <= 0,返回 1
        pushq  %rbx             # 保存被调用者保存的寄存器
        mrmovq (%rdi),%rbx      # 取得 *start
        irmovq $-1,%r10
        addq   %r10,%rsi        # count--
        irmovq $8,%r10
        addq   %r10,%rdi        # start++
        call   rproduct
        imulq  %rbx,%rax        # 把 *start 乘入乘积
        popq   %rbx             # 恢复被调用者保存的寄存器
return:
        ret
```

答案 4.5 这道题给你一个动手编写汇编代码的机会。

```
1  # long absSum(long *start, long count)
2  # start 在 %rdi 中,count 在 %rsi 中
3  absSum:
4          irmovq $8,%r8           # 常数 8
5          irmovq $1,%r9           # 常数 1
6          xorq %rax,%rax          # sum = 0
```

```
 7          andq %rsi,%rsi          # 设置条件码
 8          jmp test
 9  loop:
10          mrmovq (%rdi),%r10      # x = *start
11          xorq %r11,%r11          # 常数 0
12          subq %r10,%r11          # -x
13          jle pos                 # 若 -x <= 0,跳过
14          rrmovq %r11,%r10        # x = -x
15  pos:
16          addq %r10,%rax          # 加到 sum 上
17          addq %r8,%rdi           # start++
18          subq %r9,%rsi           # count--
19  test:
20          jne     loop            # 为 0 时停止
21          ret
```

答案 4.6 这道题给你一个动手用条件传送编写汇编代码的机会。我们只给出循环部分的代码,其余部分与练习 4.5 相同:

```
 9  loop:
10              mrmovq (%rdi),%r10          # x = *start
11              xorq %r11,%r11              # 常数 0
12              subq %r10,%r11              # -x
13              cmovg %r11,%r10             # 若 -x > 0,则 x = -x
14              addq %r10,%rax              # 加到 sum 上
15              addq %r8,%rdi               # start++
16              subq %r9,%rsi               # count--
17  test:
18              jne    loop                 # 为 0 时停止
```

答案 4.7 虽然很难想象这条指令有什么实际用处,但在设计一个系统时,规格说明中没有歧义这一点很重要。我们要为这条指令的行为确定一个合理的约定,并确保我们的每种实现都遵守这个约定。

这个测试中的 `subq` 指令把`%rsp` 的初始值与压入栈中的值进行比较。相减结果为 0 这一事实说明被压入的是`%rsp` 的旧值。

答案 4.8 想象为什么会有人想把值弹出给栈指针就更难了。不过,我们还是应该确定一个约定并坚持用它。这个代码序列把 `0xabcd` 压入栈,弹出给`%rsp`,并返回弹出的值。既然结果等于 `0xabcd`,我们可以推断出 `popq %rsp` 会把栈指针设置为从存储器读出的值。因此它等价于指令 `mrmovq (%rsp), %rsp`。

答案 4.9 异或函数要求这两个位的取值相反:

```
bool xor = (!a && b) || (a && !b);
```

一般来说,信号 `eq` 和 `xor` 互为补。也就是说,其中一个为 1 时,另一个必为 0。

答案 4.10 这些异或电路的输出将是各位相等值的补。利用德摩根定律(见网络边栏 data:bool),我们可以用 `or` 运算和 `not` 运算来实现 `and` 运算,得到的电路如图 4.71 所示。

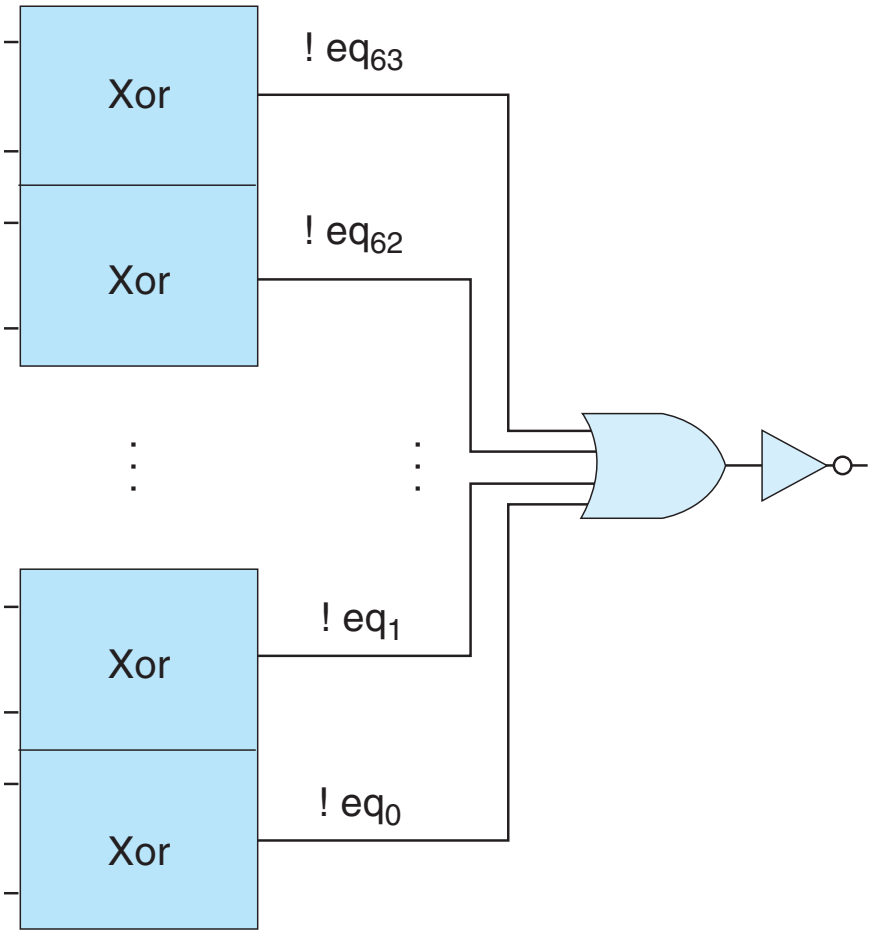


图 4.71 练习 4.10 的答案。

答案 4.11 可以看到,case 表达式的第二部分可以写成

```
        B <= C : B;
```

由于第一行会检测出 A 是三个元素中最小元素的情形,第二行只需要确定 B 和 C 中哪一个是最小元素。

答案 4.12 这个设计是求三个输入最小值的那个设计的一个变体:

```
word Med3 = [
        A <= B && B <= C : B;
        C <= B && B <= A : B;
        B <= A && A <= C : A;
```

```
        C <= A && A <= B : A;
        1                : C;
];
```

答案 4.13 这样的练习能使流水级的计算变得更加具体。从目标代码可以看出,这条指令位于地址 0x016。它有 10 字节,头两个字节是 0x30 和 0xf4,后 8 个字节是 0x0000000000000080(十进制 128)按反字节顺序的版本。

	通用	具体
阶段	`irmovq V, rB`	`irmovq $128,%rsp`
取指	$icode : ifun \leftarrow \mathrm{M}_1[PC]$	$icode : ifun \leftarrow \mathrm{M}_1[\texttt{0x016}] = 3 : 0$
	$rA : rB \leftarrow \mathrm{M}_1[PC + 1]$	$rA : rB \leftarrow \mathrm{M}_1[\texttt{0x017}] = f : 4$
	$valC \leftarrow \mathrm{M}_8[PC + 2]$	$valC \leftarrow \mathrm{M}_8[\texttt{0x018}] = 128$
	$valP \leftarrow PC + 10$	$valP \leftarrow \texttt{0x016} + 10 = \texttt{0x020}$
译码		
执行	$valE \leftarrow 0 + valC$	$valE \leftarrow 0 + 128 = 128$
访存		
写回	$\mathrm{R}[rB] \leftarrow valE$	$\mathrm{R}[\texttt{\%rsp}] \leftarrow valE = 128$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow valP = \texttt{0x020}$

这条指令把寄存器%rsp 设置为 128,并把 PC 增加 10。

答案 4.14 可以看出,这条指令位于地址 0x02c,有两个字节,值为 0xb0 和 0x00f。寄存器%rsp 在第 6 行被 pushq 指令设置为 120,该指令也把 9 存到了这个存储位置。

	通用	具体
阶段	`popq rA`	`popq %rax`
取指	$icode : ifun \leftarrow \mathrm{M}_1[PC]$	$icode : ifun \leftarrow \mathrm{M}_1[\texttt{0x02c}] = b : 0$
	$rA : rB \leftarrow \mathrm{M}_1[PC + 1]$	$rA : rB \leftarrow \mathrm{M}_1[\texttt{0x02d}] = 0 : f$
	$valP \leftarrow PC + 2$	$valP \leftarrow \texttt{0x02c} + 2 = \texttt{0x02e}$
译码	$valA \leftarrow \mathrm{R}[\texttt{\%rsp}]$	$valA \leftarrow \mathrm{R}[\texttt{\%rsp}] = 120$
	$valB \leftarrow \mathrm{R}[\texttt{\%rsp}]$	$valB \leftarrow \mathrm{R}[\texttt{\%rsp}] = 120$
执行	$valE \leftarrow valB + 8$	$valE \leftarrow 120 + 8 = 128$
访存	$valM \leftarrow \mathrm{M}_8[valA]$	$valM \leftarrow \mathrm{M}_8[120] = 9$
写回	$\mathrm{R}[\texttt{\%rsp}] \leftarrow valE$	$\mathrm{R}[\texttt{\%rsp}] \leftarrow 128$
	$\mathrm{R}[rA] \leftarrow valM$	$\mathrm{R}[\texttt{\%rax}] \leftarrow 9$
更新 PC	$PC \leftarrow valP$	$PC \leftarrow \texttt{0x02e}$

这条指令把%rax 设置为 9,把%rsp 设置为 128,并把 PC 增加 2。

答案 4.15 按图 4.20 列出的步骤来追踪,其中 rA 等于%rsp,我们可以看到在访存级这条指令会把 valA,即栈指针原来的值,存储到存储器中,与我们对 x86-64 发现的结果一样。

答案 4.16 按图 4.20 列出的步骤来追踪，其中 rA 等于`%rsp`，我们可以看到两个写回操作都会更新`%rsp`。由于写 $valM$ 的那个操作发生在最后，这条指令的净效果是把从存储器读出的值写到`%rsp`，与我们对 x86-64 看到的一样。

答案 4.17 对寄存器到寄存器的传送只做极小的改动就能实现条件传送。我们只需让写回步骤以条件测试的结果为条件：

阶段	`cmovXX rA, rB`
取指	$icode : ifun \leftarrow \mathrm{M}_1[PC]$
	$rA : rB \leftarrow \mathrm{M}_1[PC + 1]$
	$valP \leftarrow PC + 2$
译码	$valA \leftarrow \mathrm{R}[rA]$
执行	$valE \leftarrow 0 + valA$
	$Cnd \leftarrow \mathrm{Cond}(\mathrm{CC}, ifun)$
访存	
写回	若Cnd，则$\mathrm{R}[rB] \leftarrow valE$
更新 PC	$PC \leftarrow valP$

答案 4.18 可以看出，这条指令位于地址`0x037`，长 9 字节。第一个字节的值为`0x80`，而后 8 个字节是`0x0000000000000041`（`call`的目标）按反字节顺序的版本。栈指针在第 7 行被`popq`指令设置为 128。

	通用	具体
阶段	`call Dest`	`call 0x041`
取指	$icode : ifun \leftarrow \mathrm{M}_1[PC]$	$icode : ifun \leftarrow \mathrm{M}_1[\texttt{0x037}] = 8 : 0$
	$valC \leftarrow \mathrm{M}_8[PC + 1]$	$valC \leftarrow \mathrm{M}_8[\texttt{0x038}] = \texttt{0x041}$
	$valP \leftarrow PC + 9$	$valP \leftarrow \texttt{0x037} + 9 = \texttt{0x040}$
译码		
	$valB \leftarrow \mathrm{R}[\texttt{\%rsp}]$	$valB \leftarrow \mathrm{R}[\texttt{\%rsp}] = 128$
执行	$valE \leftarrow valB + (-8)$	$valE \leftarrow 128 + (-8) = 120$
访存	$\mathrm{M}_8[valE] \leftarrow valP$	$\mathrm{M}_8[120] \leftarrow \texttt{0x040}$
写回	$\mathrm{R}[\texttt{\%rsp}] \leftarrow valE$	$\mathrm{R}[\texttt{\%rsp}] \leftarrow 120$
更新 PC	$PC \leftarrow valC$	$PC \leftarrow \texttt{0x041}$

这条指令的效果是把`%rsp`设置为 120，把`0x040`（返回地址）存到这个存储地址，并把 PC 设置为`0x041`（`call`的目标）。

答案 4.19 这道题和其他练习题的所有 HCL 代码都很直白，但你自己试着生成这些代码，会有助于思考各条指令以及它们是如何被处理的。对于这道题，我们只需查看 Y86-64 指令集（图 4.2），确定哪些指令有常数字段。

```
bool need_valC =
        icode in { IIRMOVQ, IRMMOVQ, IMRMOVQ, IJXX, ICALL };
```

答案 4.20 这段代码与`srcA`的代码类似。

```
word srcB = [
```

```
        icode in { IOPQ, IRMMOVQ, IMRMOVQ } : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # 不需要寄存器
];
```

答案 4.21 这段代码与 dstE 的代码类似。

```
word dstM = [
        icode in { IMRMOVQ, IPOPQ } : rA;
        1 : RNONE;  # 不写任何寄存器
];
```

答案 4.22 正如我们在练习 4.16 中发现的那样，我们希望经 M 口的写优先于经 E 口的写，这样才能把从存储器读出的值存入%rsp。

答案 4.23 这段代码与 aluA 的代码类似。

```
word aluB = [
        icode in { IRMMOVQ, IMRMOVQ, IOPQ, ICALL,
                    IPUSHQ, IRET, IPOPQ } : valB;
        icode in { IRRMOVQ, IIRMOVQ } : 0;
        # 其他指令不需要 ALU
];
```

答案 4.24 实现条件传送出奇地简单：当条件不成立时，我们把目的寄存器设置为 RNONE，从而禁止写寄存器堆。

```
word dstE = [
        icode in { IRRMOVQ } && Cnd : rB;
        icode in { IIRMOVQ, IOPQ} : rB;
        icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # 不写任何寄存器
];
```

答案 4.25 这段代码与 mem_addr 的代码类似。

```
word mem_data = [
        # 来自寄存器的值
        icode in { IRMMOVQ, IPUSHQ } : valA;
        # 返回 PC
        icode == ICALL : valP;
        # 默认:不写任何内容
];
```

答案 4.26 这段代码与 mem_read 的代码类似。

```
bool mem_write = icode in { IRMMOVQ, IPUSHQ, ICALL };
```

答案 4.27 计算 Stat 字段需要从多个流水级收集状态信息：

```
## 确定指令状态
word Stat = [
        imem_error || dmem_error : SADR;
        !instr_valid : SINS;
        icode == IHALT : SHLT;
        1 : SAOK;
];
```

答案 4.28 这道题是一道有趣的练习，要在多个划分方案之间找到最优的平衡。它提供了许多计算流水线吞吐量和延迟的机会。

A. 对于一个两级流水线，最好的划分是让块 A、B、C 在第一级，块 D、E、F 在第二级。第一级的延迟是 170 ps，所以总周期时间就是 $170 + 20 = 190$ ps。于是我们得到吞吐量 5.26 GIPS 和延迟 380 ps。

B. 对于一个三级流水线，我们应该让块 A 和 B 在第一级，块 C 和 D 在第二级，块 E 和 F 在第三级。前两级的延迟是 110 ps，所以总周期时间是 130 ps，吞吐量为 7.69 GIPS。延迟是 390 ps。

C. 对于一个四级流水线，我们应该让块 A 在第一级，块 B 和 C 在第二级，块 D 在第三级，块 E 和 F 在第四级。第二级需要 90 ps，所以总周期时间是 110 ps，吞吐量为 9.09 GIPS。延迟是 440 ps。

D. 最优设计将是一个五级流水线，每个块各占一级，例外是第五级包含块 E 和 F。周期时间是 $80 + 20 = 100$ ps，吞吐量约为 10.00 GIPS，延迟为 500 ps。增加更多的流水级不会有帮助，因为流水线的运行不可能快过每 100 ps 一个周期。

答案 4.29 每个流水级中的组合逻辑需要 $300/k$ ps，而每个流水线寄存器需要 20 ps。

A. 总延迟将是 $300 + 20k$ ps，而吞吐量（以 GIPS 计）将是

$$\frac{1000}{\frac{300}{k} + 20} = \frac{1000k}{300 + 20k}$$

B. 当 k 趋于无穷大时，吞吐量变成 $1000/20 = 50$ GIPS。当然，延迟也会趋于无穷大。

这道练习量化了深度流水线化收益递减的问题。当我们试图把逻辑细分到许多级时，流水线寄存器的延迟就成了限制因素。

答案 4.30 这段代码与 SEQ 相应的代码非常相似，只是此时我们还不能确定数据存储器是否会为这条指令产生错误信号。

```
# 确定取出的指令的状态
word f_stat = [
        imem_error: SADR;
        !instr_valid : SINS;
```

```
        f_icode == IHALT : SHLT;
        1 : SAOK;
];
```

答案 4.31 这段代码只是把 SEQ 相应代码中的信号名加上前缀 d_ 和 D_。

```
word d_dstE = [
        D_icode in { IRRMOVQ, IIRMOVQ, IOPQ} : D_rB;
        D_icode in { IPUSHQ, IPOPQ, ICALL, IRET } : RRSP;
        1 : RNONE;  # 不写任何寄存器
];
```

答案 4.32 rrmovq 指令(第 5 行)会由于 popq 指令(第 4 行)造成的加载/使用冒险而停顿一个周期。当 popq 进入译码级时,它正位于访存级,这使 M_dstE 和 M_dstM 都等于%rsp。如果把这两个情形颠倒过来,那么写回中来自 M_valE 的写会获得优先权,把递增过的栈指针作为参数

传给 rrmovq 指令。这与练习 4.8 中确定的处理 popq %rsp 的约定不一致。

答案 4.33 这道题让你体验处理器设计中一项重要工作——为新的处理器设计测试程序。一般来说,我们应该有一些测试程序,它们会遍历所有不同的冒险可能性,并且在某个依赖没有被正确处理时产生不正确的结果。

对于这个例子,我们可以用练习 4.32 中所示程序的一个稍加修改的版本:

```
1  irmovq $5, %rdx
2  irmovq $0x100,%rsp
3  rmmovq %rdx,0(%rsp)
4  popq %rsp
5  nop
6  nop
7  rrmovq %rsp,%rax
```

两条 nop 指令会使得当 rrmovq 指令位于译码级时,popq 指令位于写回级。如果写回级的两个转发源被赋予错误的优先级,那么寄存器%rax 就会被设置为递增过的程序计数器,而不是从存储器读出的值。

答案 4.34 这个逻辑只需要检查五个转发源:

```
word d_valB = [
        d_srcB == e_dstE : e_valE;   # 由执行级转发 valE
        d_srcB == M_dstM : m_valM;   # 由访存级转发 valM
        d_srcB == M_dstE : M_valE;   # 由访存级转发 valE
        d_srcB == W_dstM : W_valM;   # 由写回级转发 valM
        d_srcB == W_dstE : W_valE;   # 由写回级转发 valE
        1 : d_rvalB;  # 使用从寄存器堆读出的值
];
```

答案 4.35 这个改动不能处理条件传送不满足条件、因而把 dstE 值设置为 RNONE 的情形。此时,即使条件传送没有发生,结果值也可能被转发给下一条指令。

```
1  irmovq $0x123,%rax
2  irmovq $0x321,%rdx
3  xorq %rcx,%rcx          # CC = 100
4  cmovne %rax,%rdx        # 未传送
5  addq %rdx,%rdx          # 应当为 0x642
6  halt
```

这段代码把寄存器%rdx 初始化为 0x321。条件数据传送没有发生，因此最后的 addq 指令应当把%rdx 中的值翻倍，变成 0x642。然而在改过的设计中，条件传送的源值 0x321 被转发进 ALU 输入 valA，而输入 valB 正确地得到操作数值 0x123。这两个输入相加产生结果 0x444。

答案 4.36 这段代码完成了这条指令状态码的计算。

```
## 更新状态
word m_stat = [
        dmem_error : SADR;
        1 : M_stat;
];
```

答案 4.37 下面的测试程序用来建立控制组合 A（图 4.67），并检测是否出了什么问题：

```
1  # 生成一个不发生跳转的分支与 ret 的组合的代码
2          irmovq Stack, %rsp
3          irmovq rtnp,%rax
4          pushq %rax       # 设置返回指针
5          xorq %rax,%rax   # 设置 Z 条件码
6          jne target       # 没有跳转(组合的第一部分)
7          irmovq $1,%rax   # 应当执行这一条
8          halt
9  target: ret              # 组合的第二部分
10         irmovq $2,%rbx   # 不应当执行这一条
11         halt
12 rtnp:   irmovq $3,%rdx   # 不应当执行这一条
13         halt
14 .pos 0x40
15 Stack:
```

这个程序的设计思路是：一旦出了问题（比如 ret 指令真的被执行了），程序就会执行多出来的某一条 irmovq 指令，然后 halt。因此，流水线中的一个错误就会导致某个寄存器被错误地更新。这段代码说明了编写测试程序所需的谨慎：它必须建立一个潜在的出错条件，然后检测是否真的发生了错误。

答案 4.38 下面的测试程序用来建立控制组合 B（图 4.67）。模拟器会检测到对某个流水线寄存器的冒泡和停顿控制信号同时为零的情形，所以我们的测试程序只需要建立起那个组合，它就会被检测到。最大的挑战是让程序在被正确处理时能有合理的动作。

```
 1  # 测试一条修改 %esp 的指令之后再跟 ret 的指令
 2  irmovq mem,%rbx
 3  mrmovq 0(%rbx),%rsp  # 把 %rsp 设置为指向返回点
 4  ret                  # 返回到返回点
 5  halt                 #
 6  rtnpt: irmovq $5,%rsi  # 返回点
 7  halt
 8  .pos 0x40
 9  mem:    .quad stack  # 保存所需的栈指针
10  .pos 0x50
11  stack: .quad rtnpt   # 栈顶:保存返回点
```

这个程序用到了存储器中两个初始化的字。第一个字(mem)保存第二个字的地址(stack,即所需的栈指针)。第二个字保存 ret 指令所需的返回点的地址。程序把栈指针装入%rsp,然后执行 ret 指令。

答案 4.39 从图 4.66 可以看出,对于加载/使用冒险,流水线寄存器 D 必须停顿:

```
bool D_stall =
        # 加载/使用冒险的条件
        E_icode in { IMRMOVQ, IPOPQ } &&
         E_dstM in { d_srcA, d_srcB };
```

答案 4.40 从图 4.66 可以看出,对于加载/使用冒险或预测错误的分支,流水线寄存器 E 必须置为冒泡:

```
bool E_bubble =
        # 预测错误的分支
        (E_icode == IJXX && !e_Cnd) ||
        # 加载/使用冒险的条件
        E_icode in { IMRMOVQ, IPOPQ } &&
         E_dstM in { d_srcA, d_srcB};
```

答案 4.41 这个控制需要检查正在执行的指令的代码,并检查流水线中靠后的异常。

```
## 是否应当更新条件码?
bool set_cc = E_icode == IOPQ &&
       # 只在正常运行期间才发生状态变化
       !m_stat in { SADR, SINS, SHLT } && !W_stat in { SADR, SINS, SHLT
        };
```

答案 4.42 在下一个周期向访存级插入冒泡,需要检查当前周期中访存级或写回级是否出现异常。

```
# 一旦异常通过访存级,就开始插入冒泡
bool M_bubble = m_stat in { SADR, SINS, SHLT } || W_stat in { SADR,
        SINS,
      SHLT };
```

对于停顿写回级，我们只检查这一级中指令的状态。如果我们还在访存级有引发异常的指令时停顿，那么这条指令就无法进入写回级了。

```
bool W_stall = W_stat in { SADR, SINS, SHLT };
```

答案 4.43 我们的预测错误频率将是 0.35，给出 $m_p = 0.20 \times 0.35 \times 2 = 0.14$，于是总体 CPI 为 1.25。这看上去是个相当微小的收益，但如果实现新分支预测策略的代价不太高，那么还是值得的。

答案 4.44 这种把焦点放在内循环上的简化分析，是估计程序性能的一种有效方法。只要数组足够大，花在代码其他部分上的时间就可以忽略不计。

A. 使用条件跳转的代码的内循环有 11 条指令，当数组元素为零或为负时全部执行，而当数组元素为正时执行其中的 10 条。平均是 10.5。使用条件传送的代码的内循环有 10 条指令，每次全部执行。

B. 关闭循环的跳转会被正确预测，除了循环终止的时候。对一个很长的数组，这一次预测错误对性能的影响可以忽略不计。基于跳转的代码中气泡的另一个来源是条件跳转，取决于数组元素是否为正。这会造成两个气泡，但它只在 50% 的时间里发生，所以平均是 1.0。条件传送的代码中没有气泡。

C. 我们的条件跳转代码平均每个数组元素需要 $10.5 + 1.0 = 11.5$ 个周期（最好情形 11 个周期，最坏情形 12 个周期），而我们的条件传送代码在所有情形都需要 10.0 个周期。

我们的流水线的分支预测错误代价只有两个周期——远低于性能更高的处理器的深层流水线中的代价。因此，使用条件传送对程序性能没有太大影响。

5

优化程序性能

编写程序时的首要目标，必须是让它在一切可能的条件下都能正确工作。一个运行得很快却给出错误结果的程序毫无用处。程序员必须写出清晰、简洁的代码：不仅是为了自己能读懂，也是为了在代码评审和日后需要修改时，别人能读懂并理解这些代码。

另一方面，也有许多场合需要把提高程序的运行速度作为一个重要考虑。如果程序必须实时地处理视频帧或网络数据包，运行缓慢的程序就无法提供所需的功能。当一个计算任务繁重到需要数天乃至数周才能完成时，把它加快哪怕 20% 也会带来显著的影响。本章将探讨如何通过若干不同类型的程序优化来使程序运行得更快。

编写一个高效的程序需要几类活动。首先，我们必须选择合适的一组算法和数据结构。其次，我们必须把源代码写成编译器能有效优化、从而生成高效可执行代码的形式。做好这第二步，关键在于理解优化编译器的能力和局限性。程序写法上看似微小的改动，就可能使编译器优化它的效果大不一样。有些程序设计语言比别的语言更容易优化。C 的某些特性（比如能够进行指针运算和强制类型转换）会给编译器优化带来挑战。程序员通常可以用便于编译器生成高效代码的方式来编写程序。第三种对付要求特别高的计算的技术，是把任务分解成可以在多个核与多个处理器的某种组合上并行计算的部分。性能提升的这个方面我们推迟到第 12 章再讨论。即使利用了并行性，每个并行线程也必须以最高性能执行，因此本章的材料在任何情况下都是适用的。

在着手程序开发和优化时，我们必须考虑代码将怎样使用，以及影响它的关键因素。总的来说，程序员必须在程序易于实现和维护与程序运行得快之间做出权衡。在算法层面上，一个简单的插入排序几分钟就能编好，而一个高度高效的排序程序可能要花一天甚至更多时间来实现和优化。在编码层面上，许多低层优化往往会降低代码的可读性和模块化程度，使程序更容易产生 bug，也更难修改或扩展。对于在性能攸关的环境中反复执行的代码，深入的优化也

许是合适的。一个挑战是：在经历了大量变换之后，仍要在某种程度上保持代码的优雅和可读。

我们描述若干改善代码性能的技术。理想情况下，编译器能够接收我们编写的任意代码，并生成具有指定行为的最有效的机器级程序。现代编译器采用复杂的分析与优化形式，而且还在不断改进。不过，即使是最好的编译器，也可能被优化障碍（optimization blocker）所阻挠——它指的是程序行为中那些严重依赖执行环境的方面。程序员必须通过编写便于优化的代码来辅助编译器。

优化程序的第一步是消除不必要的工作，让代码以尽可能高的效率完成它的既定任务。这包括消除不必要的函数调用、条件测试和存储器引用。这些优化并不依赖于目标机器的任何特定性质。

要使程序性能达到最大，程序员和编译器都需要一个目标机器的模型，指明指令是如何处理的，以及各类操作的时序特性。例如，编译器必须知道时序信息，才能决定该用乘法指令，还是用移位与加法的某种组合。现代计算机用复杂的技术处理机器级程序：并行地执行许多指令，并且可能采用与程序中指令顺序不同的顺序。程序员必须理解这些处理器的工作方式，才能把自己的程序调校到最快。我们基于 Intel 和 AMD 处理器近来的设计，给出这类机器的一个高层模型。我们还设计了一种图形化的数据流表示法，用来可视化处理器对指令的执行，借助它可以预测程序性能。

在理解了处理器的运转方式之后，我们可以迈出程序优化的第二步：利用处理器提供指令级并行的能力，同时执行多条指令。我们将介绍几种程序变换，它们减少计算的不同部分之间的数据依赖，提高这些部分所能执行的并行程度。

本章最后讨论与优化大型程序相关的问题。我们描述代码剖析器（profiler）的用法——这类工具用来测量程序不同部分的性能。这种分析有助于找出代码中的低效率之处，并确定应当把优化精力集中在程序的哪些部分。

在这份讲述中，代码优化看上去像是一个简单的线性过程：按特定顺序对代码施加一系列变换。实际上，任务远非如此直截了当，需要相当多的试错实验。尤其在接近后面的优化阶段时更是如此：表面上很小的改动就可能引起性能的重大变化，而一些很有前景的技术却被证明无效。正如我们将要在例子中看到的那样，要确切解释为什么某段特定的代码序列会得到某个特定的执行时间是很困难的。性能可能依赖于处理器设计的许多细节特征，而对这些特征，我们拥有的文档和理解都相对匮乏。这也是要尝试多种不同技术的变体与组合的又一个原因。

研究程序的汇编代码表示，是理解编译器及其生成的代码将如何运行的最有效手段之一。一个好的策略是先仔细研究内循环的代码，找出降低性能的属

性，例如过多的存储器引用和寄存器的不良使用。从汇编代码出发，还可以预测哪些操作将被并行执行，以及它们能多好地利用处理器资源。正如我们将要看到的，通常可以通过识别关键路径（critical path）——在循环反复执行过程中形成的数据依赖链——来确定执行一个循环所需的时间（或至少是时间下界）。然后可以回过头来修改源代码，引导编译器选择更有效的实现。

包括 gcc 在内的大多数主流编译器都在持续更新和改进，其优化能力尤其是如此。一种有用的策略是只对程序做必需的改写，使编译器随后能够生成高效代码。这样，我们就不至于像被迫在能力很弱的编译器下工作时那样，在代码的可读性、模块化和可移植性上做出太多妥协。同样，迭代地修改代码，并通过测量和检查生成的汇编代码来分析它的性能，也很有帮助。

对新手程序员来说，不断修改源代码以便“诱导”编译器生成高效代码，似乎有些奇怪，但许多高性能程序确实就是这样写出来的。与直接用汇编语言写代码相比，这种间接方法的优势在于：得到的代码仍然能在其他机器上运行，虽然性能可能不是最高。

5.1 优化编译器的能力和局限性

现代编译器采用复杂的算法来确定程序中计算了哪些值、这些值是怎样使用的，然后利用机会来简化表达式，把一次计算的结果用在若干不同位置，并减少某个给定计算必须执行的次数。包括 gcc 在内的大多数编译器都给用户提供了一定的控制权，让他们指定应用哪些优化。正如第 3 章讨论过的，最简单的控制方法是指定优化级别。例如，用命令行选项 `-Og` 调用 gcc，就是指定它应用一组基本优化。

用选项 `-O1` 或更高的优化级别（如 `-O2`、`-O3`）调用 gcc，会使它应用更广泛的优化。这些优化可以进一步提高程序性能，但可能扩大程序的规模，也可能使程序更难用标准调试工具调试。在我们的讲述中，主要考虑用优化级别 `-O1` 编译的代码，尽管 `-O2` 已经成为大多数使用 gcc 的软件项目公认的标准。我们有意把优化级别限制得低一些，为的是展示用不同方式编写 C 函数会如何影响编译器生成代码的效率。我们会看到，只用选项 `-O1` 编译的 C 代码，其性能可以远远胜过用最高优化级别编译的更为朴素的版本。

编译器必须只对程序应用安全的优化。所谓安全，指的是对于程序可能遇到的一切情形，经优化的程序的行为都与未优化的版本完全一样，其许诺范围以 C 语言标准提供的保证为限。把编译器限制为只做安全的优化，消除了一些产生不希望出现的运行时行为的来源，但这也意味着程序员必须更加用心地编

写程序，使编译器随后能够把它变换成高效的机器级代码。要体会判定哪些变换安全、哪些不安全有多难，考虑下面这两个过程：

```
1  void twiddle1(long *xp, long *yp)
2  {
3      *xp += *yp;
4      *xp += *yp;
5  }
6
7  void twiddle2(long *xp, long *yp)
8  {
9      *xp += 2* *yp;
10 }
```

乍一看，这两个过程似乎行为相同。它们都是把存储在指针 yp 所指位置上的值的两倍，加到指针 xp 所指的位置上。但从另一方面看，函数 twiddle2 更高效：它只需要 3 次存储器引用（读 *xp、读 *yp、写 *xp），而 twiddle1 需要 6 次（两次读 *xp、两次读 *yp、两次写 *xp）。因此，假如让编译器编译过程 twiddle1，人们可能认为它会基于 twiddle2 所执行的计算生成更高效的代码。

不过，考虑一下 xp 与 yp 相等的情形。这时函数 twiddle1 将执行下面的计算：

```
3  *xp += *xp;    /* 把 xp 处的值加倍 */
4  *xp += *xp;    /* 把 xp 处的值加倍 */
```

结果是 xp 处的值变为原来的 4 倍。而函数 twiddle2 将执行下面的计算：

```
9  *xp += 2* *xp;    /* 把 xp 处的值增至三倍 */
```

结果是 xp 处的值变为原来的 3 倍。编译器对 twiddle1 会以何种方式被调用一无所知，因此必须假定参数 xp 和 yp 有可能相等，所以它不可能生成 twiddle2 风格的代码作为 twiddle1 的优化版本。

两个指针可能指向同一个存储器位置的情形，称为存储器别名（memory aliasing）。在只做安全优化的前提下，编译器必须假定不同的指针可能会互为别名。再举一例，对于一个有指针变量 p 和 q 的程序，考虑下面的代码序列：

```
x = 1000; y = 3000;
*q = y;    /* 3000 */
*p = x;    /* 1000 */
t1 = *q;   /* 1000 或 3000 */
```

t1 得到的值取决于指针 p 和 q 是否互为别名——如果不是，它等于 3,000；如果是，它等于 1,000。这引出了主要的优化障碍之一：程序中那些会严重限制编译器生成优化代码机会的方面。如果编译器不能确定两个指针是否会互为别名，它就必须假定两种情况都有可能，从而限制了可能的优化范围。

练习 5.1　下面这道题说明存储器别名会怎样导致意料之外的程序行为。考虑下面这个交换两个值的过程：

```
1  /* 把 xp 处的值 x 与 yp 处的值 y 交换 */
2  void swap(long *xp, long *yp)
3  {
4      *xp = *xp + *yp;    /* x+y */
5      *yp = *xp - *yp;    /* x+y-y = x */
6      *xp = *xp - *yp;    /* x+y-x = y */
7  }
```

如果调用该过程时 xp 等于 yp，会产生什么效果？

造成第二个优化障碍的原因是函数调用。作为一个例子，考虑下面两个过程：

```
1  long f();
2
3  long func1() {
4    return f() + f() + f() + f();
5  }
6
7  long func2() {
8    return 4*f();
9  }
```

第一眼看去，两者似乎计算出同样的结果，但 func2 只调用 f 一次，而 func1 调用了四次。如果给定 func1 作源，很自然会生成 func2 风格的代码。

不过，考虑下面这个 f 的代码：

```
1  long counter = 0;
2
3  long f() {
4    return counter++;
5  }
```

这个函数有一个副作用——它修改了全局程序状态的某一部分。改变它被调用的次数，程序行为就会改变。具体地说，假设全局变量 counter 从 0 开始计数，调用 func1 会返回 $0+1+2+3=6$，而调用 func2 会返回 $4\cdot 0=0$。

边栏　通过内联替换优化函数调用

涉及函数调用的代码可以通过一种称为内联替换（inline substitution，简称“inlining”，内联）的过程来优化：把函数调用替换成函数体本身的代码。例如，我们可以把函数 f 的四个实例代入 func1，展开它的代码。

```
1  /* 在 func1 中内联展开 f 的结果 */
2  long func1in() {
3      long t = counter++;    /* +0 */
4      t += counter++;        /* +1 */
5      t += counter++;        /* +2 */
```

```
6      t += counter++;        /* +3 */
7      return t;
8  }
```

这种变换既减少了函数调用的开销，又为进一步优化展开后的代码创造了条件。例如，编译器可以把 `func1in` 中对全局变量 `counter` 的多处更新合并起来，生成该函数的优化版本：

```
1  /* 内联代码的优化 */
2  long func1opt() {
3      long t = 4 * counter + 6;
4      counter += 4;
5      return t;
6  }
```

对于函数 f 的这个特定定义，这段代码忠实地再现了 func1 的行为。gcc 的新近版本会尝试这种形式的优化：既可以在命令行选项 `-finline` 的指示下进行，也可以在优化级别 `-O1` 及以上时自动进行。遗憾的是，gcc 只对定义在同一个文件之内的函数尝试内联。这就意味着，在常见情形——一组库函数定义在一个文件里，却被其他文件里的函数调用——内联不会得到应用。

有时最好阻止编译器进行内联替换。一种情形是程序要用符号调试器来求值，例如 3.10.2 节描述的 `gdb`。如果某个函数调用已通过内联替换被优化掉了，那么试图跟踪该调用或为它设置断点的操作都会失败。另一种情形是如 5.14.1 节讨论的，通过剖析来评估程序的性能——被内联替换删除调用的那些函数不会被正确地剖析。

大多数编译器并不试图去判断一个函数是否没有副作用、因而是否可以成为 func2 这类优化的候选。编译器宁可假定最坏情形，让函数调用保持原样。在各家编译器中，gcc 的优化能力被认为足够好但非出类拔萃：它做基本的优化，但不做那些更“激进”的编译器所做的大刀阔斧的变换。因此，使用 gcc 的程序员必须投入更多心思，按便于编译器生成高效代码的方式编写程序。

5.2 表示程序性能

我们引入每元素的周期数（cycles per element，缩写为 CPE）这一度量，用它来表示程序性能，指导我们改进代码。CPE 度量帮助我们细致地了解迭代程序的循环性能。它适用于执行重复计算的程序，比如处理图像中的像素或计算矩阵乘积的各种元素。

处理器各种活动的时序由一个时钟控制，时钟提供某个频率的规则信号，频率通常以千兆赫兹（GHz，每秒十亿个周期）表示。例如，产品说明书把一个系统描述为“4 GHz”处理器，就是指处理器时钟以每秒 4.0×10^9 个周期运行。

每个时钟周期所需的时间是时钟频率的倒数，通常用纳秒（1 纳秒为 10^{-9} 秒）或皮秒（1 皮秒为 10^{-12} 秒）表示。例如，4 GHz 时钟的周期既可以表示成 0.25 纳秒，也可以表示成 250 皮秒。从程序员的角度看，以时钟周期而不是纳秒或皮秒来表达度量更有意义：这样度量表达的是执行了多少条指令，而不是时钟跑得有多快。

许多过程都包含一个对一组元素进行迭代的循环。例如，图 5.1 中的函数 `psum1` 和 `psum2` 都计算长度为 n 的向量的前缀和（prefix sum）。对于向量 $\vec{a} = \langle a_0, a_1, \ldots, a_{n-1} \rangle$，前缀和 $\vec{p} = \langle p_0, p_1, \ldots, p_{n-1} \rangle$ 定义为

$$\begin{aligned} p_0 &= a_0 \\ p_i &= p_{i-1} + a_i, \qquad 1 \leq i < n \end{aligned} \tag{5.1}$$

函数 psum1 每次迭代计算结果向量的一个元素；函数 psum2 采用一种称为循环展开（loop unrolling）的技术，每次迭代计算两个元素。本章稍后将探讨循环展开的好处。（关于分析和优化前缀和计算的更多内容，请见练习 5.11、5.12 和 5.19。）

```
1  /* 计算向量 a 的前缀和 */
2  void psum1(float a[], float p[], long n)
3  {
4      long i;
5      p[0] = a[0];
6      for (i = 1; i < n; i++)
7          p[i] = p[i-1] + a[i];
8  }
9
10 void psum2(float a[], float p[], long n)
11 {
12     long i;
13     p[0] = a[0];
14     for (i = 1; i < n-1; i+=2) {
15         float mid_val = p[i-1] + a[i];
16         p[i]   = mid_val;
17         p[i+1] = mid_val + a[i+1];
18     }
19     /* 若 n 为偶数,补算剩下的那个元素 */
20     if (i < n)
21         p[i] = p[i-1] + a[i];
22 }
```

图 5.1 前缀和函数。这两个函数为我们表达程序性能提供了例子。

这类过程所需的时间可以刻画为一个常数加上一个与被处理元素数量成正比的因子。例如，图 5.2 给出了当 n 在一定范围内变化时，这两个函数所需的时钟周期数。用最小二乘拟合（least squares fit）可以得出，psum1 和 psum2 的运行时间（以时钟周期计）分别可以近似为 $368 + 9.0n$ 和 $368 + 6.0n$。这些方程

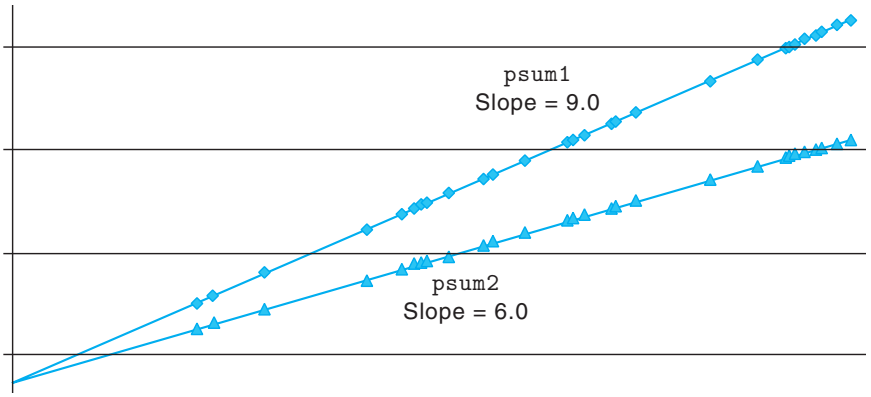


图 5.2 前缀和函数的性能。直线的斜率表示每个元素的时钟周期数(CPE)。

表明,计时代码以及过程的启动、循环的建立和完成带来 368 个周期的开销,此外还有每元素 6.0 或 9.0 周期的线性因子。当 n 较大(大于 200)时,运行时间将由线性因子主导。我们把这些项的系数称为每元素的周期数。我们之所以更愿意度量每元素的周期数而不是每迭代的周期数,是因为像循环展开这样的技术可以用更少的迭代次数完成计算,而我们最终关心的是过程对给定向量长度运行得多快。我们的优化努力集中在把计算的 CPE 降到最小。按这一度量,CPE 为 6.0 的 psum2 优于 CPE 为 9.0 的 psum1。

> **边栏 什么是最小二乘拟合?**
> 对一组数据点 $(x_1, y_1), \ldots, (x_n, y_n)$,我们通常想画出一条最能逼近这些数据所代表的 X–Y 趋势的直线。最小二乘拟合寻找的是一条形如 $y = mx + b$ 的直线,它使下面的误差度量最小:$E(m, b) = \sum_{i=1}^{n}(mx_i + b - y_i)^2$ 分别求 $E(m, b)$ 对 m 和 b 的导数并令其为 0,就可以推导出计算 m 和 b 的算法。

练习 5.2 本章稍后我们将从一个函数出发,生成许多保持函数行为不变、但性能特征各不相同的变体。对其中三个变体,我们发现运行时间(以时钟周期计)可以分别近似为:

```
版本 1:60 + 35n
版本 2:136 + 4n
版本 3:157 + 1.25n
```

对哪些 n 值,每个版本是三者中最快的?记住 n 永远是整数。

5.3 程序示例

为了演示如何把一个抽象程序系统地变换成更高效的代码,我们用一个贯穿始终的例子,它基于图 5.3 所示的向量数据结构。向量由两个存储器块表示:头部和数据数组。头部是一个结构,声明如下:

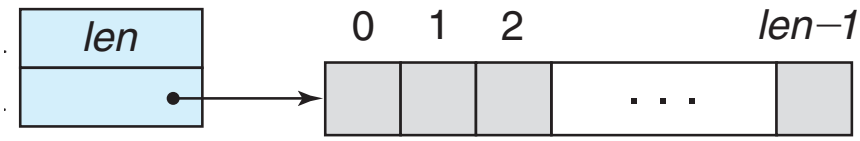


图 5.3 向量抽象数据类型。向量由头部信息加上一个指定长度的数组来表示。

code/opt/vec.h

```
1  /* 建立向量的抽象数据类型 */
2  typedef struct {
3      long len;
4      data_t *data;
5  } vec_rec, *vec_ptr;
```

声明使用 data_t 来指定基本元素的数据类型。在评估中,我们分别针对整数(C 的 int 和 long)和浮点数(C 的 float 和 double)数据测量了代码性能。做法是针对不同的类型声明分别编译并运行程序,比如对数据类型 long 如下:

```
typedef long data_t;
```

数据数组块用来存放向量元素,它是一个由 len 个类型为 data_t 的对象组成的数组。

图 5.4 给出了一些基本过程,用于生成向量、访问向量元素和确定向量的长度。需要注意的一个重要特性是:向量访问例程 get_vec_element 对每次向量引用都要做边界检查(bounds checking)。这段代码与包括 Java 在内的许多其他语言中使用的数组表示类似。边界检查减少了程序出错的机会,但也会使程序运行变慢。

code/opt/vec.c

```
1  /* 创建指定长度的向量 */
2  vec_ptr new_vec(long len)
3  {
4      /* 分配头部结构 */
5      vec_ptr result = (vec_ptr) malloc(sizeof(vec_rec));
6      data_t *data = NULL;
7      if (!result)
8          return NULL;    /* 无法分配存储空间 */
9      result->len = len;
10     /* 分配数组 */
11     if (len > 0) {
12         data = (data_t *)calloc(len, sizeof(data_t));
13         if (!data) {
14             free((void *) result);
15             return NULL;    /* 无法分配存储空间 */
16         }
17     }
```

```
18     /* data 要么为 NULL,要么是已分配的数组 */
19     result->data = data;
20     return result;
21 }
22
23 /*
24  * 取回向量元素并存储到 dest。
25  * 返回 0(越界)或 1(成功)
26  */
27 int get_vec_element(vec_ptr v, long index, data_t *dest)
28 {
29     if (index < 0 || index >= v->len)
30         return 0;
31     *dest = v->data[index];
32     return 1;
33 }
34
35 /* 返回向量长度 */
36 long vec_length(vec_ptr v)
37 {
38     return v->len;
39 }
```

图 5.4 向量抽象数据类型的实现。在实际程序中,数据类型 data_t 被声明为 int、long、float 或 double。

作为一个优化的例子,考虑图 5.5 给出的代码,它按某种运算把向量中所有元素合并成一个值。通过使用编译时常数 IDENT 和 OP 的不同定义,可以把这段代码重新编译成对数据执行不同的运算。具体地说,使用声明

```
1  /* 完全使用数据抽象的实现 */
2  void combine1(vec_ptr v, data_t *dest)
3  {
4      long i;
5
6      *dest = IDENT;
7      for (i = 0; i < vec_length(v); i++) {
8          data_t val;
9          get_vec_element(v, i, &val);
10         *dest = *dest OP val;
11     }
12 }
```

图 5.5 合并运算的初始实现。通过不同的单位元素 IDENT 与合并运算 OP 的声明组合,可以对不同的运算测量这个例程。

```
#define IDENT 0
#define OP  +
```

它对向量中的元素求和。使用声明

```
#define IDENT 1
#define OP  *
```

它计算向量各元素的乘积。

在讲述中，我们将对这段代码进行一系列变换，写出合并函数的不同版本。为了衡量进展，我们在一台装配 Intel Core i7 Haswell 处理器的机器上测量了各函数的 CPE 性能，我们称之为参照机（reference machine）。该处理器的部分特征已在 3.1 节给出。这些测量刻画的是程序在这台特定机器上运行的性能，因此不保证在其他机器与编译器的组合上有可比的性能。不过，我们把结果与许多不同的编译器/处理器组合做了比较，发现它们与这里给出的结果大体一致。

随着我们对代码施加一组变换，会发现有些变换带来的性能提升微乎其微，而另一些则有戏剧性的效果。决定采用哪些变换组合，确实是编写快速代码这门“黑魔法”的一部分。有些组合确实无效，不能带来可测的收益；另一些组合则很重要，因为它们能为编译器创造进一步优化的机会。以我们的经验，最好的方法是实验与分析相结合：反复尝试不同的方法，进行测量，并考察汇编代码表示，找出潜在的性能瓶颈。

作为起点，下表给出了 combine1 在参照机上运行时的 CPE 测量值，覆盖不同的运算（加法或乘法）与数据类型（long 整数和双精度浮点数）的组合。我们对许多不同程序所做的实验表明，对 32 位和 64 位整数的运算性能完全相同（涉及除法的代码除外）。类似地，我们发现对单精度和双精度浮点数据进行运算的程序性能也完全相同。因此，我们的表中只分别给出整数数据和浮点数据的结果。

函数	页	方法	整数 +	整数 *	浮点 +	浮点 *
`combine1`	543	抽象实现（未优化）	22.68	20.02	19.98	20.18
`combine1`	543	抽象实现（`-O1`）	10.12	10.12	10.17	11.14

可以看到，我们得到的测量不完全精确：整数和的 CPE 更可能是 23.00，而不是 22.68；整数积更可能是 20.0，而不是 20.02。我们不去“修饰”数字让它们显得漂亮，而是如实给出实际得到的测量值。要可靠地测量某段代码序列所需的确切时钟周期数，会有许多因素造成干扰。看这些数字时，最好在心里把它们向上或向下舍入百分之几个周期。

未优化的代码是 C 代码到机器代码的直接翻译，往往带有明显的低效率。只要给编译器加上命令行选项 `-O1`，就能启用一组基本优化。可以看到，仅此一步就显著提高了程序性能——提升超过 2 倍，而程序员无须付出任何努力。总的来说，养成启用一定级别优化的习惯是有好处的。（用优化级别 `-Og` 也得

到了相似的性能结果。)在随后的测量中,我们生成和测量程序时都用优化级别 -O1 和 -O2。

5.4 消除循环的低效率

注意,图 5.5 中的过程 combine1 把函数 vec_length 的调用作为 for 循环的测试条件。回忆一下 3.6.7 节关于把包含循环的代码翻译成机器级程序的讨论,测试条件在循环的每次迭代中都必须求值。但是向量的长度在循环推进时并不变化,因此我们只需计算一次向量长度,然后在测试条件中使用这个值。

图 5.6 给出了一个改写版本,称为 combine2。它在开头调用 vec_length,把结果赋给局部变量 length。这项变换对某些数据类型和运算的总体性能有明显影响,对另一些则影响很小甚至没有影响。无论如何,这项变换都是必需的:它消除了那些在我们尝试进一步优化时会成为瓶颈的低效率。

函数	页	方法	整数 +	整数 *	浮点 +	浮点 *
combine1	543	抽象实现(-O1)	10.12	10.12	10.17	11.14
combine2	545	把 vec_length 调用移出循环	7.02	9.03	9.02	11.03

```
1  /* 把对 vec_length 的调用移出循环 */
2  void combine2(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6
7      *dest = IDENT;
8      for (i = 0; i < length; i++) {
9          data_t val;
10         get_vec_element(v, i, &val);
11         *dest = *dest OP val;
12     }
13 }
```

图 5.6 提高循环测试的效率。把对 vec_length 的调用移出循环测试,就不必在每次迭代时都执行它了。

这项优化是一类称为代码移动(code motion)的优化的一个实例。这类优化识别的是这样一种计算:虽然被执行多次(例如在循环内),但计算的结果不会变化,因此可以把它移到代码中在此之前求值次数较少的部分。在这个例子中,我们把对 vec_length 的调用从循环内移到了循环之前。

优化编译器会试图进行代码移动。遗憾的是,正如前面讨论过的,它们对改变过程调用位置或调用次数的变换通常非常谨慎。它们不能可靠地检测一个函数是否有副作用,因此只能假定它可能有。例如,假如 vec_length 有某

种副作用，那么 combine1 和 combine2 的行为就可能不同。要改进代码，程序员常常必须显式地进行代码移动来帮助编译器。

作为 combine1 中所见的循环低效率的一个极端例子，考虑图 5.7 所示的过程 lower1。这个过程的样式仿照了几个学生在网络编程项目作业中提交的例程。它的目的是把字符串中的所有大写字母转换成小写。过程一步步遍历字符串，把每个大写字符转换成小写。大小写转换的做法，是把范围从‘A’到‘Z’的字符平移到范围从‘a’到‘z’上。

库函数 strlen 是在 lower1 的循环测试中被调用的。虽然 strlen 通常是用特殊的 x86 字符串处理指令实现的，但它的整体执行过程与图 5.7 中同时给出的简单版本类似。因为 C 中的字符串是以空字符结尾的字符序列，strlen 只有逐步遍历整个序列、直至遇到一个空字符，才能确定字符串的长度。对一个长度为 n 的字符串，strlen 的执行时间正比于 n。由于在 lower1 的 n 次迭代中每次都要调用 strlen，lower1 的总体运行时间与字符串长度呈平方关系，正比于 n^2。

```
1  /* 将字符串转换成小写:慢 */
2  void lower1(char *s)
3  {
4      long i;
5
6      for (i = 0; i < strlen(s); i++)
7          if (s[i] >= 'A' && s[i] <= 'Z')
8              s[i] -= ('A' - 'a');
9  }
10
11 /* 将字符串转换成小写:更快 */
12 void lower2(char *s)
13 {
14     long i;
15     long len = strlen(s);
16
17     for (i = 0; i < len; i++)
18         if (s[i] >= 'A' && s[i] <= 'Z')
19             s[i] -= ('A' - 'a');
20 }
21
22 /* 库函数 strlen 的样例实现 */
23 /* 计算字符串长度 */
24 size_t strlen(const char *s)
25 {
26     long length = 0;
27     while (*s != '\0') {
28         s++;
29         length++;
30     }
31     return length;
32 }
```

图 5.7 小写字母转换例程。这两个过程的性能有天壤之别。

图 5.8 中对不同长度字符串实测这两个函数的结果（使用库版本的 strlen）证实了这一分析。lower1 的运行时间随字符串长度的增大而急剧上升（图 5.8(a)）。图 5.8(b) 给出了七种不同长度的运行时间（不同于图中画出的那些种），每种长度都是 2 的幂。可以看到，对 lower1 而言，字符串长度每翻一番，运行时间就变为 4 倍。这是运行时间为平方级的明确标志。对长度为 1,048,576 的字符串，lower1 需要超过 17 分钟的 CPU 时间。

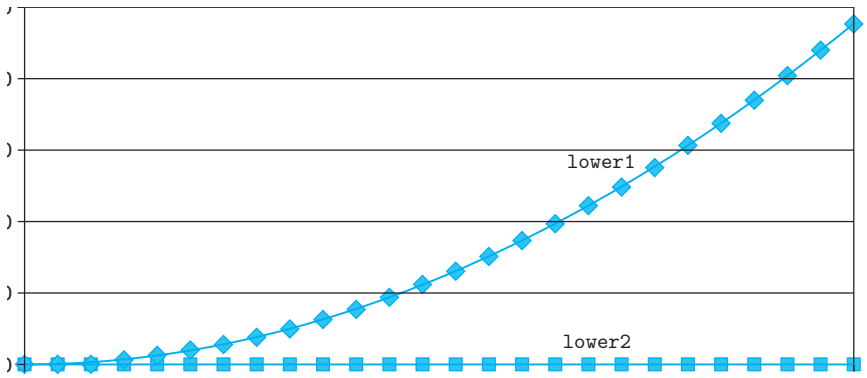


图 5.8 小写转换例程的性能比较。原始代码 `lower1` 因循环结构低效而具有二次运行时间，修改后的代码 `lower2` 具有线性运行时间。

图 5.7 中的函数 lower2 与 lower1 相同，只是我们把对 strlen 的调用移到了循环之外。性能有了戏剧性的改善：对长度为 1,048,576 的字符串，该函数只需要 2.0 毫秒——比 lower1 快 500,000 倍以上。字符串长度每翻一番，运行时间也翻一番——这是运行时间为线性的明确标志。对更长的字符串，运行时间的改善还会更大。

在理想世界中，编译器会认识到循环测试中对 strlen 的每次调用都会返回相同的结果，因此可以把调用移出循环。这将要求非常复杂的分析：strlen 在检查字符串的元素，而这些值会随着 lower1 的推进而改变。编译器将不得不探测到：虽然字符串内的字符在变化，但没有任何字符从非零变成零，或反向变化。这样的分析即使采用内联，也远超最复杂的编译器的能力，所以程序员必须自己完成这类变换。

这个例子说明了编写程序中的一个常见问题：一段看似无关紧要的代码却隐藏着渐进式低效率（asymptotic inefficiency）。人们不会料到一个小写转换例程会成为程序性能的限制因素。通常，程序是在小数据集上测试和分析的，对这些数据而言 lower1 的性能已经足够。但是当程序最终部署时，这个过程完全可能被应用于超过一百万个字符的字符串。这块无害的代码于是突然变成了重大的性能瓶颈。与之相反，lower2 的性能对任意长度的字符串都是可用的。

重大的编程项目里，这类问题的故事比比皆是。称职的程序员的一项职责，就是绝不要引入这种渐进式低效率。

练习 5.3 考虑下面这些函数：

```
long min(long x, long y) { return x < y ? x : y; }
long max(long x, long y) { return x < y ? y : x; }
void incr(long *xp, long v) { *xp += v; }
long square(long x) { return x*x; }
```

下面的三个代码片段调用了这些函数：

```
A.
for (i = min(x, y); i < max(x, y); incr(&i, 1))
    t += square(i);
```

```
B.
for (i = max(x, y) - 1; i >= min(x, y); incr(&i, -1))
    t += square(i);
```

```
C.
long low = min(x, y);
long high = max(x, y);

for (i = low; i < high; incr(&i, 1))
    t += square(i);
```

假设 x 等于 10，y 等于 100。请填写下面的表格，说明在代码片段 A–C 中这四个函数各被调用了多少次：

代码	min	max	incr	square
A.				
B.				
C.				

5.5 减少过程调用

正如我们所见，过程调用会有开销，还会阻碍大多数形式的程序优化。从 combine2 的代码（图 5.6）可以看出，每次循环迭代都要调用 `get_vec_element` 来获取下一个向量元素。这个函数在每次向量引用时都要对照循环边界检查下标 i，这显然是低效率的来源。对待任意数组访问，边界检查可能是有用的特性，但只要对 combine2 的代码做一个简单的分析就能证明所有引用都是有效的。

code/opt/vec.c

```
1  data_t *get_vec_start(vec_ptr v)
2  {
3      return v->data;
4  }
```

code/opt/vec.c

```
1  /* 直接访问向量数据 */
2  void combine3(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7
8      *dest = IDENT;
9      for (i = 0; i < length; i++) {
10         *dest = *dest OP data[i];
11     }
12 }
```

图 5.9 消除循环内的函数调用。得到的代码没有带来性能提升，但它为后面的优化创造了条件。

我们反其道而行之：假设给抽象数据类型增加一个函数 `get_vec_start`，它返回数据数组的起始地址，如图 5.9 所示。这样就可以写出图中 combine3 这样的过程，使内循环中没有函数调用。它不通过函数调用来获取每个向量元素，而是直接访问数组。讲究纯粹的人可能会说，这项变换严重损害了程序的模块性。原则上确实如此：向量抽象数据类型的用户按理根本不该需要知道向量内容是作为数组存放的，而不是作为链表之类的其他数据结构存放的。务实的程序员则会说，这项变换是走向高性能结果的必要一步。

函数	页	方法	整数 +	整数 *	浮点 +	浮点 *
`combine2`	545	把 `vec_length` 调用移出循环	7.02	9.03	9.02	11.03
`combine3`	549	直接访问数据	7.17	9.02	9.02	11.03

令人意外的是，这里看不出性能有明显的改进。实际上，整数和的性能还略有下降。显然，内循环的其他运算形成了瓶颈，对性能的限制超过了调用 `get_vec_element`。我们稍后会回到这个函数(5.11.2 节)，那时会看到为什么 combine2 反复进行的边界检查并没有带来性能损失。眼下，可以把这项变换看作一连串变换步骤中的一步，这些步骤最终会带来大幅的性能改进。

5.6 消除不必要的存储器引用

combine3 的代码把合并运算计算出的值累积在指针 dest 所指示的位置上。这一点可以通过考察编译生成的代码的内循环汇编代码看出来。这里给出数据类型为 double、合并运算为乘法时生成的 x86-64 代码：

```
combine3 的内循环。
data_t = double,OP = *
dest 在 %rbx,data+i 在 %rdx,data+length 在 %rax

1  .L17:                              loop:
2    vmovsd (%rbx), %xmm0               从 dest 读出乘积
3    vmulsd (%rdx), %xmm0, %xmm0        把乘积乘以 data[i]
4    vmovsd %xmm0, (%rbx)               把乘积存入 dest
5    addq $8, %rdx                      data+i 加 1
6    cmpq %rax, %rdx                    与 data+length 比较
7    jne .L17                           若不相等,转 loop
```

从这段循环代码可以看到，与指针 dest 对应的地址保存在寄存器%rbx 中。编译器还对代码做了变换，让寄存器%rdx 保存指向第 i 个数据元素的指针，注解中记作 data+i。这个指针每次迭代加 8。循环终止由把这个指针与保存在寄存器%rax 中的指针比较来判定。可以看到，累积值每次迭代都要从存储器读出并写回，这样的读写是浪费的：每次迭代开始时从 dest 读出的值，就应该是上一次迭代结束时写回的那个值。

我们可以像图 5.10 中的 combine4 那样改写代码，消除这种对存储器的多余读写。我们引入一个临时变量 acc，在循环中用它累积计算出的值，循环结束之后才把结果存入 dest。正如下面的汇编代码所表明的，编译器现在可以使用寄存器%xmm0 来保存累积值。与 combine3 的循环相比，每迭代的存储器操作从两次读、一次写减少到了一次读。

```
combine4 的内循环。
data_t = double,OP = *
acc 在 %xmm0,data+i 在 %rdx,data+length 在 %rax

1  .L25:                              loop:
2    vmulsd (%rdx), %xmm0, %xmm0        把 acc 乘以 data[i]
3    addq $8, %rdx                      data+i 加 1
4    cmpq %rax, %rdx                    与 data+length 比较
5    jne .L25                           若不相等,转 loop
```

程序性能有了显著提高，如下表所示：

```
1  /* 把结果累积到局部变量中 */
2  void combine4(vec_ptr v, data_t *dest)
3  {
4      long i;
```

```
5       long length = vec_length(v);
6       data_t *data = get_vec_start(v);
7       data_t acc = IDENT;
8
9       for (i = 0; i < length; i++) {
10          acc = acc OP data[i];
11      }
12      *dest = acc;
13  }
```

图 5.10 在临时变量中累积结果。把累积值保持在局部变量 acc（“accumulator（累加器）”的缩写）中，就不必在每个循环迭代中从存储器读出它、再把更新后的值写回去了。

函数	页	方法	整数 +	整数 *	浮点 +	浮点 *
combine3	549	直接访问数据	7.17	9.02	9.02	11.03
combine4	551	在临时变量中累积	1.27	3.01	3.01	5.01

所有这些测得的时间都取得了 2.2 ～ 5.7 倍的改进，其中整数相加的情形降到每元素只需 1.27 个时钟周期。

再一次，人们可能又会认为，编译器应当能够自动把图 5.9 所示的 combine3 代码变换成像图 5.10 中 combine4 那样在寄存器中累积数值。然而事实上，由于存储器别名，这两个函数可能有不同的行为。例如，考虑整数数据、运算为乘法、单位元素为 1 的情形。设 $v = [2, 3, 5]$ 是一个由三个元素组成的向量，考虑下面两个函数调用：

```
combine3(v, get_vec_start(v) + 2);
combine4(v, get_vec_start(v) + 2);
```

也就是说，我们在向量最后一个元素与存放结果的目的位置之间制造了一个别名。这两个函数随后的执行情况如下：

函数	初始	循环前	$i = 0$	$i = 1$	$i = 2$	最终
combine3	[2, 3, 5]	[2, 3, 1]	[2, 3, 2]	[2, 3, 6]	[2, 3, 36]	[2, 3, 36]
combine4	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 5]	[2, 3, 30]

如前所述，combine3 是在目的位置累积结果，而这个位置此时也就是向量的最后一个元素。因此这个位置的值先被置为 1，然后是 $2 \cdot 1 = 2$，再是 $3 \cdot 2 = 6$。在最后一次迭代中，这个值又乘以它自身，得到最终值 36。对 combine4 来说，向量保持不变，直到最后才把最后一个元素置为计算出的结果 $1 \cdot 2 \cdot 3 \cdot 5 = 30$。

当然，我们这个展示 combine3 与 combine4 差别的例子是刻意构造的。有人可能会争辩说，combine4 的行为更接近函数描述的本意。遗憾的是，编译器

无法就“函数可能在什么条件下使用、程序员的意图可能是什么”做出判断。所以，在给定 combine3 的情况下，保守的做法是继续对存储器进行读写，尽管这样效率较低。

练习 5.4 我们用 gcc 的命令行选项 `-O2` 编译 combine3 时，得到的代码 CPE 性能明显好于用 `-O1` 编译：

函数	页	方法	整数 +	整数 *	浮点 +	浮点 *
`combine3`	549	用 `-O1` 编译	7.17	9.02	9.02	11.03
`combine3`	549	用 `-O2` 编译	1.60	3.01	3.01	5.01
`combine4`	551	在临时变量中累积	1.27	3.01	3.01	5.01

除了整数相加的情形之外，我们的性能都达到了与 combine4 相当的水平，而整数相加也有显著改善。考察编译器生成的汇编代码，我们发现内循环的一个有趣的变体：

```
combine3 的内循环。
data_t = double,OP = *。
用 -O2 编译
dest 在 %rbx,data+i 在 %rdx,data+length 在 %rax
累积的乘积在 %xmm0

1  .L22:                              loop:
2    vmulsd (%rdx), %xmm0, %xmm0        把乘积乘以 data[i]
3    addq $8, %rdx                      data+i 加 1
4    cmpq %rax, %rdx                    与 data+length 比较
5    vmovsd %xmm0, (%rbx)               把乘积存入 dest
6    jne .L22                           若不相等,转 loop
```

我们可以把它与用优化级别 1 生成的版本比较一下：

```
combine3 的内循环。
data_t = double,OP = *。
用 -O1 编译
dest 在 %rbx,data+i 在 %rdx,data+length 在 %rax

1  .L17:                              loop:
2    vmovsd (%rbx), %xmm0               从 dest 读出乘积
3    vmulsd (%rdx), %xmm0, %xmm0        把乘积乘以 data[i]
4    vmovsd %xmm0, (%rbx)               把乘积存入 dest
5    addq $8, %rdx                      data+i 加 1
6    cmpq %rax, %rdx                    与 data+length 比较
7    jne .L17                           若不相等,转 loop
```

我们看到，除了指令顺序的调整之外，唯一的区别是：优化程度较高的版本没有包含执行从 dest 所示位置读出的那条 `vmovsd`（第 2 行）。

A. 这两个循环中寄存器%xmm0 的作用有什么不同？

B. 这个优化程度较高的版本会忠实地实现 combine3 的 C 代码吗？包括当 dest 与向量数据之间存在存储器别名时？

C. 要么解释为什么这项优化保持了想要的行为，要么举出一个例子，说明它会得到与优化程度较低的代码不同的结果。

完成这最后一项变换后达到了这样的境地：计算每个元素只需 1.25 ～ 5 个周期。与最初启用优化时的 9 ～ 11 个周期相比，这是相当大的改进。现在我们想知道，到底哪些因素在制约代码的性能，以及怎样才能把性能再往上推。

5.7 理解现代处理器

到目前为止，我们应用的优化都没有依赖目标机器的任何特性，只是减少了过程调用的开销，并消除了几个给优化编译器带来困难的关键“优化障碍”。要把性能继续推向更高，我们必须考虑利用处理器微体系结构（microarchitecture）的优化——所谓微体系结构，就是处理器执行指令所依据的底层系统设计。要把性能榨取得一分不剩，需要对程序做细致分析，并针对目标处理器编写和生成代码。虽然如此，还是有一些基本优化可以用在一大类处理器上，带来整体性能的提升。我们在这里报告的详细性能结果可能不适用于其他机器，但其运转与优化的普遍原理适用于各种各样的机器。

要理解改进性能的方法，需要对现代处理器的微体系结构有基本的了解。由于一枚芯片上可以集成的晶体管数量巨大，现代微处理器采用了以最大化程序性能为目标的复杂硬件。其结果是，这些处理器的实际运转方式与观察机器级程序所得的印象大不相同。在代码层面上看，指令似乎是一条接一条执行的：每条指令从寄存器或存储器取值、执行一个操作，再把结果存回某个寄存器或存储器位置。而在实际的处理器中，许多指令是同时求值的，这种现象称为指令级并行（instruction-level parallelism）。在某些设计中，“正在执行中”的指令可以有 100 条或者更多。处理器采用精心设计的机制来保证这种并行执行的行为，精确符合机器级程序所要求的顺序语义模型。这是现代微处理器的一项了不起的成就：它们采用复杂而奇特的微体系结构让多条指令并行执行，同时又呈现出一个简单的顺序指令执行的操作视图。

虽然现代微处理器的详细设计远远超出本书的范围，但只要对它们的工作原理有个大致概念，就足以理解它们是怎样实现指令级并行的。我们会发现，有两个不同的下界刻画了程序的最大性能。延迟界（latency bound）对应于一系列操作必须严格按顺序执行的情形——因为一个操作的结果在下一个操作开始前就是必需的。当代码中的数据依赖限制了处理器利用指令级并行的能力

时,这个界就会限制程序性能。吞吐量界(throughput bound)刻画的是处理器功能单元的原始计算能力,它构成了程序性能的终极上限。

5.7.1 总体操作

图 5.11 给出了现代微处理器的一个非常简化的视图。我们假想的处理器设计在结构上大致基于 Intel 最近的处理器。这些处理器在业界被称为超标量(superscalar)处理器,意思是它们每个时钟周期可以执行多个操作,并且是乱序(out-of-order)执行的——指令执行的顺序不必与它们在机器级程序中的顺序一致。整体设计有两个主要部分:指令控制单元(ICU),负责从存储器读取指令序列,并根据这些指令生成对程序数据执行的一组基本运算;以及执行单元(EU),随后执行这些运算。与我们在第 4 章研究的简单按序流水线相比,乱序处理器需要的硬件规模更大、复杂度也高得多,但它们在实现更高程度的指令级并行方面更胜一筹。

Figure 5.11 shows a very simplified view of a modern microprocessor. Our hypothetical processor design is based loosely on the structure of recent Intel processors. These processors are described in the industry as being *superscalar*, which means they can perform multiple operations on every clock cycle and *out of order*, meaning that the order in which instructions execute need not correspond to their ordering in the machine-level program. The overall design has two main parts: the *instruction control unit* (ICU), which is responsible for reading a sequence of instructions from memory and generating from these a set of primitive operations to perform on program data, and the *execution unit* (EU), which then executes these operations. Compared to the simple *in-order* pipeline we studied in Chapter 4, out-of-order processors require far greater and more complex hardware, but they are better at achieving higher degrees of instruction-level parallelism.

The ICU reads the instructions from an *instruction cache*—a special high-

图 5.11

ICU 从指令高速缓存(instruction cache)读取指令——指令高速缓存是保存最近访问过的指令的特殊高速缓存。在一般情况下,ICU 取出的指令比当前正在执行的指令超前很多,以便有足够的时间来对它们译码并把运算送往 EU。不过,这里有一个问题:当程序遇到一条分支指令[1]时,程序接下来可能走向两个方向。分支可以被选中(taken),控制转移到分支目标;也可以不被选中(not taken),控制转到指令序列中的下一条指令。现代处理器采用一种称为分支预测(branch prediction)的技术:它们猜测一个分支是否会被选中,并且预测分支的目标地址。利用一种称为推测执行(speculative execution)的技术,处理器开始取出并译码分支预测目的地的指令,甚至在这些预测是否正确已经确定下来

[1]本书中,“分支”一词特指条件跳转指令。其他能够把控制转移到多个目的地的指令(如过程返回和间接跳转)也会给处理器带来类似的挑战。

之前就开始执行其中的运算。如果处理器后来确定预测有误,它就把状态复位到分支点上的状态,开始沿另一个方向取指和执行指令。图中标记为“取指控制”的块采用了分支预测来决定要取出哪些指令。

指令译码逻辑接收实际的程序指令,把它们转换成一组基本运算(primitive operations),有时也称为微操作(micro-operations)。每个运算完成某种简单的计算任务,比如将两个数相加、从存储器读出数据或把数据写入存储器。对于指令复杂的机器(如 x86 处理器),一条指令可以被译码成多个运算。指令如何译码成运算序列因机器而异,并且这些信息被视为高度保密的。幸好,我们无须知道某台机器实现的底层细节,也能优化我们的程序。

在典型的 x86 实现中,一条只对寄存器操作的指令,比如

```
addq %rax,%rdx
```

会被转换成单个运算。另一方面,一条涉及一个或多个存储器引用的指令,比如

```
addq %rax,8(%rdx)
```

会产生多个运算,把存储器引用与算术运算分开。这条指令会被译码成三个运算:一个把值从存储器加载到处理器中,一个把加载的值加到寄存器%eax的值上,一个把结果存回存储器。译码把指令拆分开,为的是让一组专用硬件单元得以分工合作;随后这些单元就能并行地执行多条指令的不同部分。

EU 从取指单元接收运算,通常每个时钟周期都能接收若干个。这些运算被分派给一组实际执行运算的功能单元。这些功能单元是特制的,用来处理不同类型的运算。

读存储器和写存储器由加载单元和存储单元来实现。加载单元处理把存储器中的数据读入处理器的运算。这个单元有一个加法器用于地址计算。类似地,存储单元处理把处理器中的数据写出到存储器的运算,它也有一个用于地址计算的加法器。如图所示,加载单元和存储单元通过一个数据高速缓存(data cache)访问存储器——这是一种保存最近访问的数据值的高速缓存。

在推测执行下,运算虽然会被求值,但在处理器能够确定这些指令确实应该被执行之前,最终结果不会存入程序寄存器或数据存储器。分支运算也被送到 EU,但不是为了确定分支该往哪里去,而是为了判定它们是否被预测正确。如果预测不正确,EU 就丢弃已经算出的越过分支点的结果,并通知分支单元预测有误、正确的分支目的地是什么。此时分支单元开始从新的位置取指。正如 3.6.6 节中所见,预测错误会对性能造成很大的代价,因为要让新指令取来、译码并送到功能单元,还需要一段时间。

图 5.11 表明,不同的功能单元是被设计来执行不同运算的。其中标注为执行“算术运算”的那些单元通常被特化,用以执行整数和浮点运算的不同组合。而随着一枚芯片上能够集成的晶体管数量

微处理器芯片上可以集成的晶体管数量随时间不断增长,后续各代微处理器都增加了功能单元的总数、每个单元可以执行的操作组合以及每个单元自身的性能。算术单元被有意设计成能执行多种不同的运算,因为不同程序所需的运算差别很大。例如,有的程序可能涉及大量整数运算,而有的程序则需要大量浮点运算。如果一个功能单元被专门化到只能执行整数运算,而另一个只能执行浮点运算,那么这些程序都无法从多个功能单元中获得全部好处。

例如,我们的参考机 Intel Core i7 Haswell 共有 8 个功能单元,编号为 0 ~ 7。下面是每个单元功能的部分列表:

0. 整数运算、浮点乘法、整数除法与浮点除法、分支
1. 整数运算、浮点加法、整数乘法、浮点乘法
2. 加载、地址计算
3. 加载、地址计算
4. 存储
5. 整数运算
6. 整数运算、分支
7. 存储地址计算

在上面的列表中,“整数运算”指的是加法、位级运算和移位这样的基本运算。乘法和除法需要更专门的资源。可以看到,一次存储运算需要两个功能单元——一个计算要存储的地址,一个真正执行数据的存储。我们将在 5.12 节讨论存储(和加载)操作的机制。

可以看到,这组功能单元有潜力同时执行多个同类运算:其中有 4 个单元能够执行整数运算,2 个能够执行加载运算,2 个能够执行浮点乘法。我们稍后会看到这些资源对程序所能达到的最高性能的影响。

在 ICU 内部,退役单元(retirement unit)记录着正在进行的处理,并确保它遵守机器级程序的顺序语义。我们的图把包含整数寄存器、浮点寄存器以及新近的 SSE 和 AVX 寄存器的寄存器堆画成退役单元的一部分,因为这些寄存器的更新由这个单元控制。一条指令被译码时,有关它的信息被放进一个先进先出的队列。这些信息在队列中一直保留到出现如下两种结果之一:第一,一旦该指令的各运算都已完成,并且通向该指令的各个分支点都被确认预测正确,这条指令就可以退役,同时完成对程序寄存器的更新;另一方面,如果通向该指

令的某个分支点预测错了，这条指令就会被清除，已经算出来的任何结果都被丢弃。这样，错误的预测就不会改变程序状态。

边栏　乱序处理的历史

乱序处理最早是在 1964 年的 Control Data Corporation 6600 处理器中实现的。指令由 10 个不同的功能单元处理，每个单元都可以独立运行。在那个年代，这台时钟频率为 10 MHz 的机器堪称科学计算领域的高端机器。

IBM 在 1966 年首次用 IBM 360/91 处理器实现了乱序处理，但只用于执行浮点指令。此后大约 25 年里，乱序处理一直被视为一项稀奇的技术，只出现在力求最高性能的机器上，直到 IBM 在 1990 年的 RS/6000 工作站产品线中重新采用它。这一设计成为 IBM/Motorola PowerPC 产品线的基础，其中 1993 年问世的 601 型成为第一款使用乱序处理的单片微处理器。Intel 则在 1995 年的 PentiumPro 中引入乱序处理，其底层微体系结构与我们的参考机相近。

正如我们所描述的，对程序寄存器的更新只发生在指令退役之时，而且这只有等到处理器确定通向该指令的所有分支都预测正确之后才会进行。为了让结果能尽快从一条指令传递给另一条，这类信息中的大部分都在各执行单元之间交换，如图中标注“运算结果”（Operation results）的部分所示。如图中箭头所示，各执行单元可以把结果直接发给彼此。这是 4.5.5 节中我们在简单处理器设计中引入的数据转发技术的一个更精细的版本。

控制操作数在执行单元之间传递的最常用机制称为寄存器重命名（register renaming）。当一条更新寄存器 r 的指令被译码时，会生成一个标记 t，为该运算的结果赋予一个唯一的标识符。表中加入一项 (r, t)，用来维护程序寄存器 r 与将要更新该寄存器的运算的标记 t 之间的关联。当后续一条以寄存器 r 作为源操作数的指令被译码时，送往执行单元的运算会带着 t，作为该操作数值的来源。当某个执行单元完成第一个运算时，它会生成结果 (v, t)，表示标记为 t 的运算产生了值 v。此时，所有等待 t 作为源的操作都会改用 v 作为源值，这是数据转发的一种形式。通过这一机制，值可以直接从一个运算转发给另一个，而不必写回寄存器堆再读出来，使第二个运算在第一个完成时就能立刻开始。重命名表中只保存那些有等待完成的写操作的寄存器的表项。当译码出的指令需要寄存器 r，而这个寄存器没有任何关联的标记时，就直接从寄存器堆中取操作数。借助寄存器重命名，即使寄存器要等到处理器确认分支结果之后才会更新，整段运算序列也都可以推测执行。

5.7.2 功能单元的性能

图 5.12 记录了我们的参考机 Intel Core i7 Haswell 上部分算术运算的性能，这些数据来自实际测量，并参考了 Intel 的文献资料 [49]。对于其他处理器，这

图 5.12　参考机各操作的延迟、发射时间和容量特性。延迟(Latency)表示执行实际运算所需的总时钟周期数,发射时间(Issue time)表示两次相互独立的运算之间最少的周期数。容量(Capacity)表示可以同时发射多少个这样的运算。除法的时间取决于数据值。

	整数			浮点数		
操作	延迟	发射	容量	延迟	发射	容量
加法	1	1	4	3	1	1
乘法	3	1	1	5	1	2
除法	3–30	3–30	1	3–15	3–15	1

些时间特性也是典型的。每个操作都用三个特性来刻画:延迟(latency),即完成该运算所需的总时间;发射时间(issue time),即同一类型的两次相互独立的运算之间最少的时钟周期数;容量(capacity),指能够执行该运算的功能单元的数目。

从整数运算到浮点运算,延迟在增大。加法和乘法的发射时间都是 1,这意味着处理器在每一个时钟周期都能开始一个新的这类运算。这样短的发射时间是靠流水线化实现的。流水线化的功能单元由一组流水级实现,每个流水级完成运算的一部分。例如,典型的浮点加法器包含三个流水级(因而延迟是三个周期):一个处理阶码值,一个做小数部分相加,一个对结果舍入。算术运算可以紧密衔接地通过各流水级,而不必等一个运算完成之后下一个才开始。只有当存在前后相继、逻辑上相互独立的运算要执行时,这一能力才能派上用场。发射时间为 1 个周期的功能单元称为全流水线化的(fully pipelined):它们每个时钟周期都能开始一个新运算。容量大于 1 的运算则来自多个功能单元的能力,如前面在描述参考机时已经看到的。

除法器(用于整数除法和浮点除法,也用于浮点平方根)没有流水线化——它的发射时间等于它的延迟。这意味着除法器必须完整地完成一次除法才能开始新的一次。除法的延迟和发射时间给出的都是一个范围,因为有些被除数与除数的组合比其他组合需要更多的步骤。长延迟和长发射时间使除法成为一种代价相对高昂的运算。

表达发射时间的一种更常用的方式,是给出该单元的最大吞吐量(throughput),其定义为发射时间的倒数。全流水线化的功能单元的最大吞吐量是每时钟周期 1 个运算,而发射时间更长的单元最大吞吐量也更低。拥有多个功能单元还能进一步提高吞吐量。对一个容量为 C、发射时间为 I 的运算,处理器潜在的吞吐量可达 C/I 个运算每时钟周期。例如,我们的参考机执行浮点乘法的速率可达每时钟周期 2 个。我们会看到如何利用这一能力来提升程序性能。

电路设计者可以制造出性能特性各不相同的功能单元。实现短延迟或者流水线化需要更多硬件,对于乘法和浮点运算这类更复杂的函数尤其如此。由

于微处理器芯片上留给这些单元的空间有限，CPU 设计者必须在功能单元的数量和每个单元的性能之间仔细权衡，以取得最优的整体性能。他们评估许多不同的基准测试程序，把最多的资源投给最关键的运算。如图 5.12 所示，整数乘法及浮点乘法和加法在 Core i7 Haswell 处理器的设计中被认为是重要的运算，尽管要达到图中所示那样低的延迟和高的流水线化程度需要相当数量的硬件。另一方面，除法相对少见，而且要实现短延迟或完全流水线化都比较困难。

这些算术运算的延迟、发射时间和容量会影响我们的合并函数（combining function）的性能。可以把这种影响表示为 CPE 值上的两个基本界限：

	整数		浮点数	
界限	+	*	+	*
延迟界限	1.00	3.00	3.00	5.00
吞吐量界限	0.50	1.00	1.00	0.50

延迟界限给出的，是任何必须严格按顺序执行合并运算的函数的 CPE 最小值。吞吐量界限根据功能单元产生结果的最大速率给出 CPE 的一个下界。例如，整数乘法器只有一个，而且发射时间是 1 个时钟周期，所以处理器不可能维持每时钟周期超过 1 次乘法的速率。另一方面，整数加法有 4 个功能单元可以执行，处理器潜在的速率可达每周期 4 个运算。不幸的是，从存储器读取元素的需要带来了另一个吞吐量界限。两个加载单元把处理器的读取速率限制为每时钟周期至多 2 个数据值，因而吞吐量界限为 0.50。我们将用不同版本的合并函数来演示延迟界限和吞吐量界限的影响。

5.7.3 处理器操作的抽象模型

作为分析机器级程序在现代处理器上执行性能的工具，我们将使用程序的数据流（data-flow）表示。这是一种图形记号，展示不同运算之间的数据依赖如何约束各项运算执行的先后。这些约束进而会在图中形成关键路径，为执行一组机器指令所需的时钟周期数给出一个下界。

在进入技术细节之前，先看一下到目前为止我们最快的代码——函数 `combine4` 的 CPE 测量值是有启发意义的：

			整数		浮点数	
函数	页	方法	+	*	+	*
`combine4`	551	累积到临时变量	1.27	3.01	3.01	5.01
延迟界限			1.00	3.00	3.00	5.00
吞吐量界限			0.50	1.00	1.00	0.50

可以看到，除了整数加法之外，这些测量值都与处理器的延迟界限吻合。这不是巧合——它说明这些函数的性能由正在执行的求和或求积运算的延迟决定。对 n 个元素求积或求和大约需要 $L \cdot n + K$ 个时钟周期，其中 L 是合并

运算的延迟，K 表示调用函数以及启动和结束循环的开销。因此 CPE 就等于延迟界限 L。

从机器级代码到数据流图

我们使用的数据流表示是非正式的，只用来直观地体会程序中的数据依赖如何决定性能。我们以 `combine4`（图 5.10）为例来介绍这种数据流记号。我们只关注循环执行的计算，因为对大向量来说，它是决定性能的主要因素。考虑数据类型为 double、合并运算为乘法的情形。其他数据类型与运算的组合会得到类似的代码。这个循环编译出的代码由四条指令组成，其中`%rdx` 保存指向数组 data 第 i 个元素的指针，`%rax` 保存指向数组末尾的指针，`%xmm0` 保存累积值 acc。

```
combine4 的内循环。data_t = double, OP = *
acc 在 %xmm0,data+i 在 %rdx,data+length 在 %rax
 1  .L25:                                  loop:
 2    vmulsd (%rdx), %xmm0, %xmm0           把 acc 乘上 data[i]
 3    addq      $8, %rdx                    递增 data+i
 4    cmpq      %rax, %rdx                  与 data+length 比较
 5    jne       .L25                        若 !=,跳转到 loop
```

```
%rax
%rdx
%xmm0
vmulsd (%rdx), %xmm0, %xmm0
addq $8,%rdx
cmpq %rax,%rdx
jne loop
%rax
%rdx
%xmm0
load
mul
add
cmp
jne
Figure 5.13
Graphical representation of inner-loop code for combine4. Instructions
are dynamically translated into one or two operations,
      each of which receives values
from other operations or from registers and produces values for other
        operations and for
registers. We show the target of the final instruction as the label
        loop. It jumps to the
first instruction shown.
```

如图 5.13 所示，在我们假想的处理器设计中，这四条指令被指令译码器展开成五个运算：最初的乘法指令被展开成一个从存储器读取源操作数的 load 运算，和一个执行乘法的 mul 运算。

作为生成程序数据流图表示的准备工作，图 5.13 左侧的方框和连线显示了这些寄存器如何被各运算使用和更新：上方一排方框表示循环开始时的寄存器值，下方一排方框表示循环结束时的值。例如，`%rax` 只被 cmp 运算用作源值，因此它在循环结束时的值与循环开始时相同。而`%rdx` 在循环内既被使用又被更新：它的初始值被 load 和 add 运算使用，新值由 add 运算生成，随后被 cmp 运算使用。`%xmm0` 同样在循环内被 mul 运算更新，该运算首先会使用它的初始值作为源值。

图 5.13 中有一些运算产生的值并不对应任何寄存器。我们把它们表示为右侧运算之间的弧线。load 运算从存储器读出一个值并直接传给 mul 运算。由于这两个运算源自同一条 `vmulsd` 指令的译码，它们之间传递的中间值没有对应的寄存器。cmp 运算更新条件码，然后由 jne 运算测试。

对于一个形成循环的代码段，我们可以把被访问的寄存器分成四类：

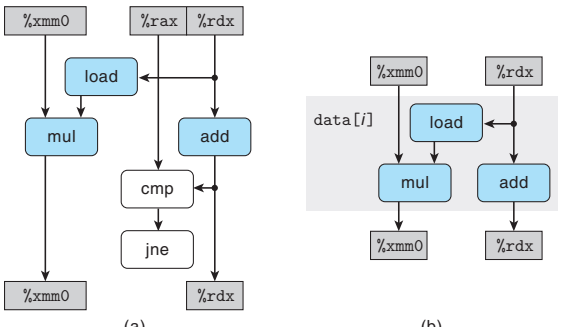


Figure 5.14
Abstracting `combine4` operations as a data-flow graph. We rearrange the operators of Figure 5.13 to more clearly show the data dependencies (a), and then further show only those operations that use values from one iteration to produce new values for the next (b).

图 5.14

- **只读**　这些寄存器用作源值——或作为数据，或用来计算内存地址——但在循环内不被修改。`combine4` 循环中唯一的只读寄存器是`%rax`。
- **只写**　这些寄存器用作数据传送运算的目的地。这个循环中没有这样的寄存器。
- **局部**　这些寄存器在循环内被更新和使用，但从一个迭代到下一个迭代之间没有依赖。条件码寄存器就是这个循环中局部寄存器的例子：它们被 cmp 运算更新、被 jne 运算使用，不过这一依赖完全包含在单个迭代之内。
- **循环**　这些寄存器既用作循环的源值，又用作循环的目的地，一个迭代中生成的值会用在另一个迭代中。可以看到，`%rdx` 和`%xmm0` 是 `combine4` 的循环寄存器，分别对应程序值 data+i 和 acc。

正如我们将看到的，循环寄存器之间的运算链决定了限制性能的数据依赖。

图 5.14 是对图 5.13 图形表示的进一步提炼，目标是只显示那些会影响程序执行时间的运算和数据依赖。在图 5.14(a) 中，我们重新排列了各个运算，以便更清楚地显示数据从顶部的源寄存器（既包括只读寄存器也包括循环寄存器）流向底部的目的寄存器（既包括只写寄存器也包括循环寄存器）。

在图 5.14(a) 中，我们还把不属于循环寄存器之间任何依赖链的运算涂成白色。对于这个例子，比较（cmp）和分支（jne）运算并不直接影响程序中的数据流。我们假设指令控制单元预测该分支会被执行，因而程序会继续循环。比较和分支运算的作用是测试分支条件，并在分支不执行时通知 ICU。我们假定这一检查足够快，不会拖慢处理器。

在图 5.14(b) 中，我们删去了在左图中被涂成白色的运算，只保留循环寄存器。剩下的是一个抽象模板，显示一次循环迭代在循环寄存器之间形成的数据依赖。从这幅图中可以看到两条从一个迭代通向下一个迭代的数据依赖。一侧是程序值 acc 的相继值之间的依赖，acc 保存在`%xmm0` 中：循环把旧值乘以一个（由 load 运算产生的）数据元素，为 acc 算出新值。另一侧是指向第 i 个数据元素的指针的相继值之间的依赖：每次迭代中，旧值既被用作 load 运算的地址，也被 add 运算递增以算出新值。

图 5.15 给出了 `combine4` 内循环 n 次迭代的数据流表示。这幅图是把图 5.14(b) 所示的模板原样复制 n 次得到的。可以看到程序有两条数据依赖链，

分别对应于用 mul 和 add 运算更新程序值 acc 和 data+i。由于浮点乘法的延迟是 5 个周期，而整数加法的延迟是 1 个周期，可以看到左边的链会形成一条关键路径，执行它需要 $5n$ 个周期；右边的链只需要 n 个周期就能执行，所以不会限制程序的性能。

图 5.15 说明了为什么 `combine4` 在执行浮点乘法时得到了等于延迟界限 5 个周期的 CPE。执行这个函数时，浮点乘法器成了限制性的资源。循环中所需的其他运算——操纵和测试指针值 data+i、从存储器读取数据——与乘法并行进行。每当算出一个相继的 acc 值，它就被反馈回来用于计算下一个值，不过这要到 5 个周期之后才会发生。

其他数据类型与运算的组合，其数据流与图 5.15 所示完全相同，只是左边那条数据依赖链由不同的数据运算构成。在运算延迟 L 大于 1 的所有情形中，测得的 CPE 就是 L，说明这条链构成了限制性能的关键路径。

其他性能因素

不过对整数加法，我们测得 `combine4` 的 CPE 是 1.27，小于图 5.15 中沿左右任一侧的依赖链预测出的 CPE 1.00 所对应的速度。这说明了这样一条原理：数据流表示中的关键路径给出的只是程序所需周期数的下界。其他因素也可

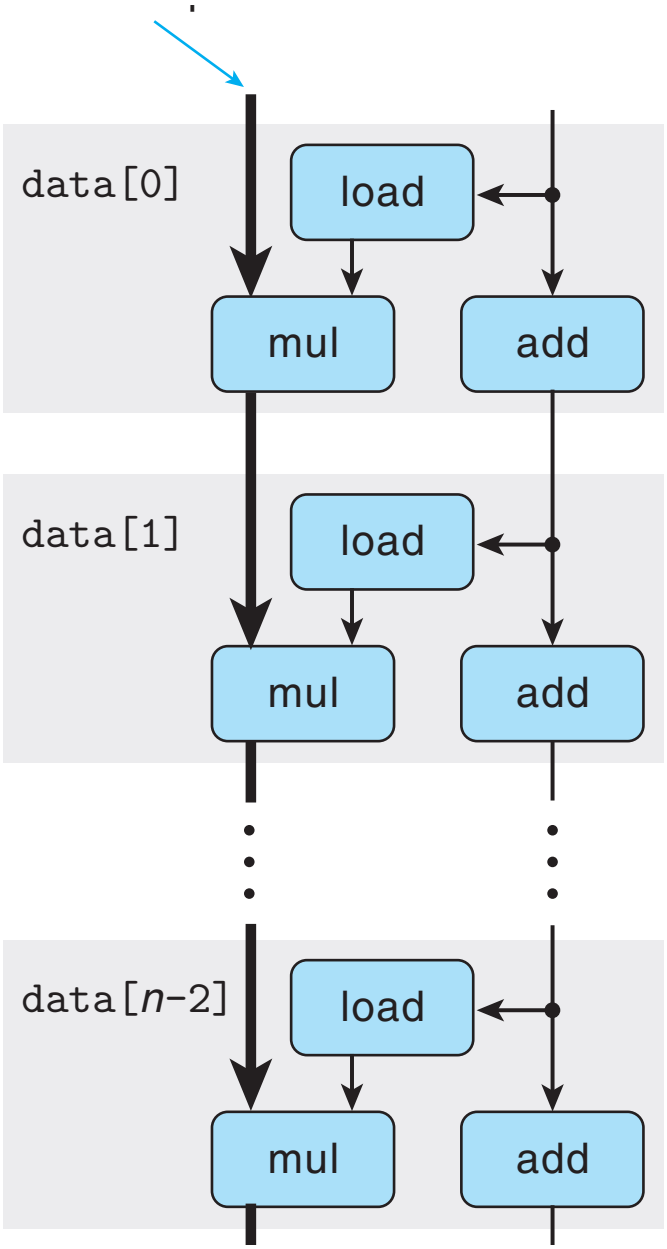


图 5.15 combine4 的内层循环迭代 n 次进行计算的数据流表示。乘法运算的序列形成一条限制程序性能的关键路径。

能限制性能，包括可用功能单元的总数，以及在任何一步中能够在功能单元之间传递的数据值的数目。以整数加法作为合并运算时，数据运算本身足够快，使得其余的运算无法足够快地供给数据。要确切解释程序为什么每个元素需要 1.27 个周期，需要比公开资料更详尽地了解硬件设计。

总结一下我们对 `combine4` 的性能分析：程序操作的抽象数据流表示显示，`combine4` 有一条长度为 $L \cdot n$ 的关键路径，它由程序值 acc 的相继更新造成，这条路径把 CPE 限制到至少为 L。这确实就是除整数加法之外所有情形下我们测得的 CPE；整数加法的测量值是 1.27，而我们按关键路径长度预期的 CPE 是 1.00。

延迟界限看起来像是限制合并运算执行速度的根本界限。我们接下来的任务是重新组织运算来增强指令级并行。我们希望这样变换程序，使唯一的限制变成吞吐量界限，从而得到低于或接近 1.00 的 CPE。

练习 5.5 假设我们想写一个函数来求多项式的值。n 次多项式定义为由一组系数 $a_0, a_1, a_2, \ldots, a_n$ 组成。对于值 x，我们通过计算下式来求多项式的值：

$$a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n \tag{5.2}$$

下面的函数实现了这一计算，它的参数是系数数组 a、值 x 和多项式的阶数 degree（即公式 5.2 中的 n）。这个函数在单个循环内同时计算公式中相继的各项以及 x 的相继次幂：

```
1  double poly(double a[], double x, long degree)
2  {
3      long i;
4      double result = a[0];
5      double xpwr = x; /* 等于循环开始时的 x^i */
6      for (i = 1; i <= degree; i++) {
7          result += a[i] * xpwr;
8          xpwr = x * xpwr;
9      }
10     return result;
11 }
```

A. 对于阶数 n，这段代码执行了多少次加法和多少次乘法？

B. 在我们的参考机上，算术运算的延迟如图 5.12 所示，我们测得这个函数的 CPE 是 5.00。请依据实现第 7 ~ 8 行的运算在迭代之间形成的数据依赖，解释这个 CPE 是如何产生的。

练习 5.6 我们继续探索求多项式值的方法，正如练习 5.5 中那样。可以应用以英国数学家 William G. Horner（1786 ~ 1837）的姓氏命名的霍纳方法

(Horner's method)来减少求多项式值所需的乘法次数。其思想是反复从式中提出 x 的幂,得到下面的求值方式:

$$a_0 + x(a_1 + x(a_2 + ... + x(a_{n-1} + xa_n)\,...)) \tag{5.3}$$

采用霍纳方法,我们可以用下面的代码来实现多项式求值:

```
1  /* 应用霍纳方法 */
2  double polyh(double a[], double x, long degree)
3  {
4      long i;
5      double result = a[degree];
6      for (i = degree-1; i >= 0; i--)
7          result = a[i] + x*result;
8      return result;
9  }
```

A. 对于阶数 n,这段代码执行了多少次加法和多少次乘法?

B. 在我们的参考机上,算术运算的延迟如图 5.12 所示,我们测得这个函数的 CPE 是 8.00。请依据实现第 7 行的运算在迭代之间形成的数据依赖,解释这个 CPE 是如何产生的。

C. 尽管练习 5.5 中给出的函数需要执行更多的运算,请解释它为什么能运行得更快。

5.8 循环展开

循环展开是一种程序变换,它通过增加每次迭代所做的元素计算来减少循环的迭代次数。我们在函数 `psum2`(图 5.1)中见过一个这样的例子:每次迭代计算前缀和的两个元素,从而使所需的总迭代次数减半。循环展开可以从两个方面改进性能。其一,它减少了那些不直接对程序结果做出贡献的运算的数量,比如循环索引和条件分支。其二,它暴露出进一步变换代码的机会,可以减少整体计算的关键路径中的运算数量。本节中,我们只考察不带任何其他变换的简单循环展开。

图 5.16 给出了合并代码的一个版本,它使用我们将称为"2×1 循环展开"的方法。第一个循环每次步进两个数组元素。也就是说,每次迭代中循环索引 i 增加 2,并且在一次迭代内把合并运算施加于数组元素 i 和 $i+1$。

一般说来,向量长度不会是 2 的倍数。我们的代码要对任意向量长度都能正确工作。我们用两种方式来满足这一要求。第一,保证第一个循环不会越出数组边界。对长度为 n 的向量,我们把循环上界设为 $n-1$。这样就保证了循环只在循环索引 i 满足 $i < n-1$ 时才执行,因而最大数组下标 $i+1$ 会满足 $i+1 < (n-1)+1 = n$。

我们可以把这个思想推广到以任意因子 k 展开循环,得到 $k \times 1$ 循环展开。做法是:把上界设为 $n-k+1$,循环体内把合并运算施加于元素 i 到 $i+k-1$。每次迭代中循环索引 i 增加 k。这样最大数组下标 $i+k-1$ 就会小于 n。我们用第二个循环每次一个地处理向量最后剩下的几个元素;这个循环的循环体会执行 0 到 $k-1$ 次。对于 $k=2$,我们本来可以用一个简单的条件语句

```
1
/* 2 x 1 loop unrolling */
2
void combine5(vec_ptr v, data_t *dest)
3
{
4
long i;
5
long length = vec_length(v);
6
long limit = length-1;
7
data_t *data = get_vec_start(v);
8
data_t acc = IDENT;
9
10
/* Combine 2 elements at a time */
11
for (i = 0; i < limit; i+=2) {
12
acc = (acc OP data[i]) OP data[i+1];
13
}
14
15
/* Finish any remaining elements */
16
for (; i < length; i++) {
17
acc = acc OP data[i];
18
}
19
*dest = acc;
20
}
Figure 5.16
Applying 2 × 1 loop unrolling. This transformation can reduce the
↪  effect
of loop overhead.
```

来选择性地增加最后一次迭代,就像我们在函数 psum2(图 5.1)中所做的那样。对 $k>2$ 收尾的各种情形,用循环来表示更好,所以我们对 $k=2$ 也采用

这一编程惯例。我们把这种变换称为“$k \times 1$ 循环展开”，因为我们以因子 k 展开，但只用单个变量 acc 来累积各个值。

练习 5.7　修改 `combine5` 的代码，把循环以因子 $k = 5$ 展开。

当我们测量展开因子 $k = 2$（`combine5`）和 $k = 3$ 的展开代码的性能时，得到如下结果：

			整数		浮点数	
函数	页	方法	+	*	+	*
`combine4`	551	未展开	1.27	3.01	3.01	5.01
`combine5`	568	2×1 展开	1.01	3.01	3.01	5.01
		3×1 展开	1.01	3.01	3.01	5.01
延迟界限			1.00	3.00	3.00	5.00
吞吐量界限			0.50	1.00	1.00	0.50

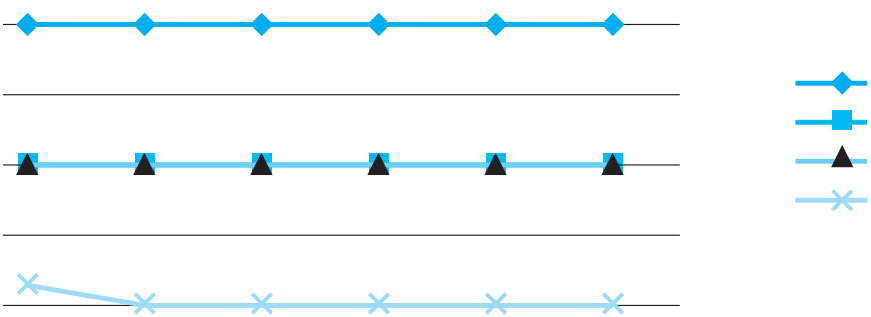


图 5.17　不同 k×1 循环展开程度下的 CPE 性能。只有整数加法能从这种变换中获益。

可以看到整数加法的 CPE 得到了改善，达到了 1.00 的延迟界限。这一结果可以归功于减少循环开销运算的收益。通过减少开销运算相对于计算向量和所需的加法次数的数量，我们可以达到这样一个程度：整数加法 1 个周期的延迟成为限制性能的因素。另一方面，其他情形都没有改善——它们已经处在各自的延迟界限上了。图 5.17 给出了循环最多展开到因子 10 时的 CPE 测量结果。可以看到，展开 2 和 3 我们观察到的趋势仍在继续——没有任何一个低于各自的延迟界限。

要理解为什么 $k \times 1$ 展开无法把性能提高到延迟界限之上，我们来看一下 $k = 2$ 时 `combine5` 内循环的机器级代码。当类型 data_t 为 double 且运算为乘法时，会生成下面的代码：

```
combine5 的内循环。data_t = double, OP = *
i 在 %rdx,data 在 %rax,limit 在 %rbx,acc 在 %xmm0
 1  .L35:                                  loop:
 2    vmulsd (%rax,%rdx,8), %xmm0, %xmm0    把 acc 乘上 data[i]
 3    vmulsd 8(%rax,%rdx,8), %xmm0, %xmm0   把 acc 乘上 data[i+1]
 4    addq      $2, %rdx                    i 增加 2
 5    cmpq      %rdx, %rbp                  与 limit:i 比较
```

```
6    jg        .L35                          若 >,跳转到 loop
```

可以看到，与为 combine4 生成的基于指针的代码相比，gcc 对 C 代码中的数组引用做了更直接的翻译[2]。循环索引 i 保存在寄存器%rdx 中，data 的地址保存在寄存器%rax 中。如前所述，累积值 acc 保存在向量寄存器%xmm0 中。循环展开产生两条 vmulsd 指令———一条把 data[i] 合并到 acc 上，

```
%rax
%rbp
%rdx
%xmm0
vmulsd (%rax,%rdx,8), %xmm0, %xmm0
vmulsd 8(%rax,%rdx,8), %xmm0, %xmm0
addq $2,%rdx
cmpq %rdx,%rbp
jg loop
%rax
%rbp
%rdx
%xmm0
load
mul
load
mul
add
cmp
jg
Figure 5.18
Graphical representation of inner-loop code for combine5. Each
iteration has two vmulsd instructions,
      each of which is translated into a load and a
mul operation.
```

另一条把 data[i+1] 合并到 acc 上。图 5.18 给出了这段代码的图形表示。每条 vmulsd 指令都被翻译成两个运算：一个从存储器加载一个数组元素，另一个把这个值乘上累积值。可以看到，寄存器%xmm0 在循环的每次执行中被读写两次。我们可以按照图 5.19(a) 所展示的过程对这幅图进行重排、简化和抽象，得到图 5.19(b) 中所示的模板。然后把该模板复制 $n/2$ 次，以展示对长度为 n 的向量的计算，从而得到

图 5.20 所示的数据流表示。可以看到，这幅图中仍然有一条由 n 个 mul 运算组成的关键路径——迭代的次数减半了，但每次迭代中仍有两个依次执行的乘法运算。既然关键路径是不带循环展开的代码性能的限制因素，那么对 $k \times 1$ 循环展开它依然如此。

[2]gcc 优化器会为函数生成多个变体，然后从中选择它预测能带来最好性能和最小代码量的一个。因此，源代码的微小改动就可能产生差别很大的机器代码。我们发现，在参考机上运行的程序中，选择基于指针还是基于数组的代码对性能没有影响。

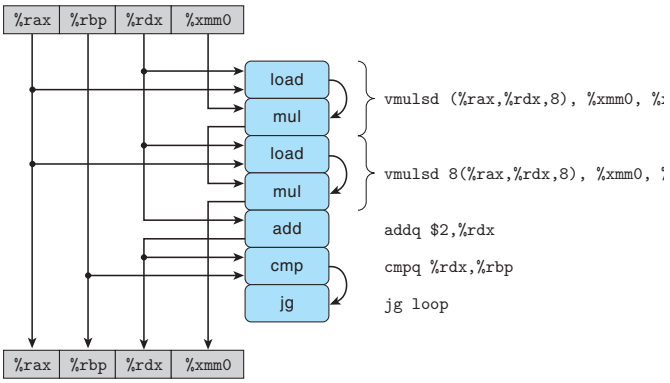


Figure 5.18 **Graphical representation of inner-loop code for** `combin` iteration has two `vmulsd` instructions, each of which is translated into a l mul operation.

Figure 5.19
Abstracting `combine5` **operations as a data-flow graph.** We rearrange, simplify, and abstract the representation of Figure 5.18 to show the data dependencies between successive iterations (a). We see that each iteration must perform two multiplications in sequence (b).

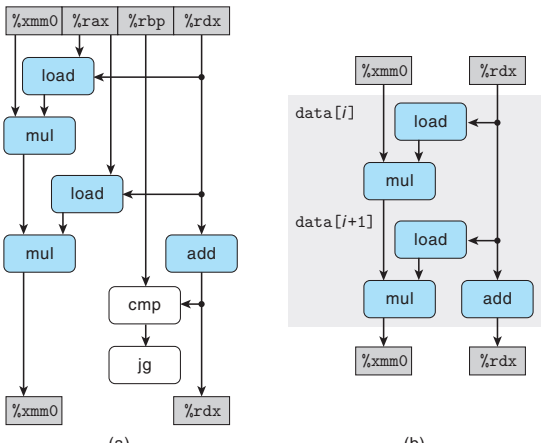


图 5.19

边栏　让编译器展开循环
编译器很容易就能执行循环展开。许多编译器把这一工作作为其优化组合的一部分。当以 3 级或更高的优化级别调用时，`gcc` 会执行某些形式的循环展开。

5.9 增强并行性

到这一步，我们的函数已经触及算术单元延迟所施加的界限。不过正如我们已经指出的，执行加法和乘法的功能单元都是全流水线化的，就是说它们每个时钟周期都能开始新的运算，而且有些运算可以由多个功能单元执行。硬件有能力以高得多的速率执行乘法和加法，但我们的代码无法利用这一能力，即使循环展开也不行，因为我们把值累积在单个变量 acc 中。在前一次计算完成之前，我们无法为 acc 算出新值。即使为 acc 计算新值的功能单元每个时钟周

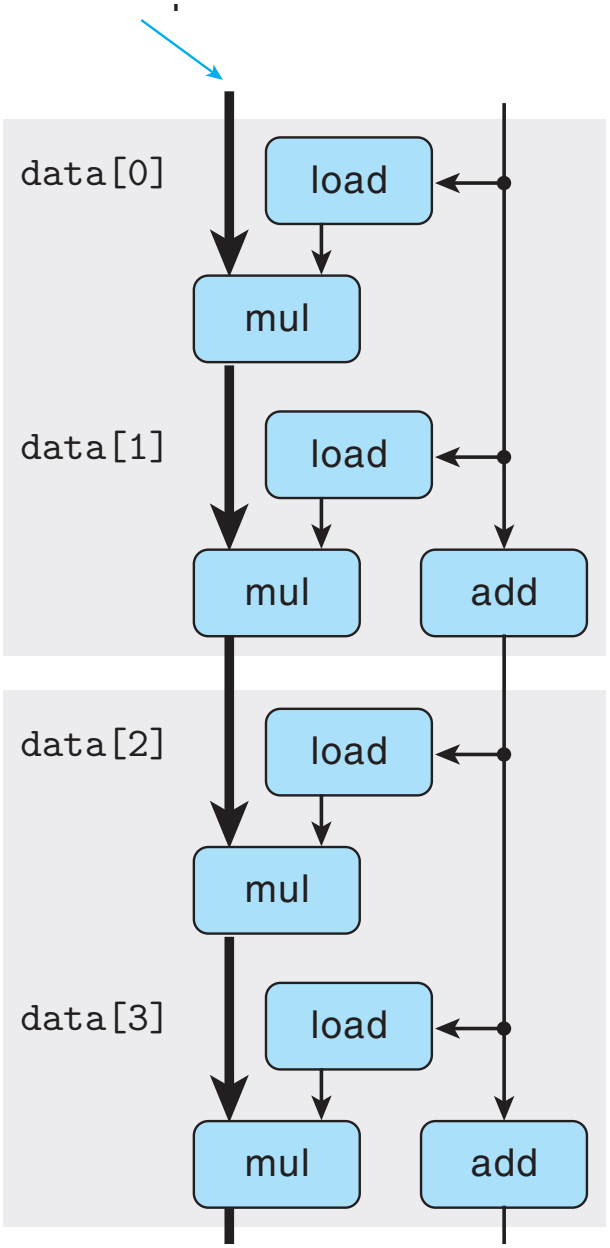


图 5.20 combine5 在长度为 n 的向量上操作的数据流表示。虽然循环已按因子 2 展开，但关键路径上仍有 n 次 mul 运算。

期都能开始一个新运算，实际上它每 L 个周期才开始一个，其中 L 是合并运算的延迟。现在我们来研究一些打破这种顺序依赖的方法，以获得好于延迟界限的性能。

5.9.1 多个累加器

对于既满足结合律又满足交换律的合并运算，比如整数加法和乘法，我们可以通过把合并运算的集合拆分成两个或更多部分、最后再把结果合并起来的办法来提升性能。例如，设 P_n 表示元素 $a_0, a_1, \ldots, a_{n-1}$ 的乘积：

$$P_n = \prod_{i=0}^{n-1} a_i$$

假设 n 是偶数，我们还可以把它写成 $P_n = P_{\text{En}} \times P_{\text{On}}$，其中 P_{En} 是下标为偶数的元素的乘积，P_{On} 是下标为奇数的元素的乘积：

$$P_{\text{En}} = \prod_{i=0}^{n/2-1} a_{2i}$$

$$P_{\text{On}} = \prod_{i=0}^{n/2-1} a_{2i+1}$$

图 5.21 给出的代码就采用了这一方法。它同时使用了两路循环展开——每次迭代合并更多元素，以及两路并行——把下标为偶数的元素累积在变量 acc0 中，下标为奇数的元素累积在变量 acc1 中。因此我们把它称为“2 × 2 循环展开”。和以前一样，我们还增设了一个第二循环，用在向量长度不是 2 的倍数的情形下累积剩余的数组元素。最后我们把合并运算施加于 acc0 和 acc1 得到最终结果。

把单纯的循环展开与循环展开加两路并行作比较，我们得到如下性能：

```
1
/* 2 x 2 loop unrolling */
2
void combine6(vec_ptr v, data_t *dest)
3
{
4
long i;
5
long length = vec_length(v);
6
long limit = length-1;
7
data_t *data = get_vec_start(v);
8
```

```
data_t acc0 = IDENT;
9
data_t acc1 = IDENT;
10
11
/* Combine 2 elements at a time */
12
for (i = 0; i < limit; i+=2) {
13
acc0 = acc0 OP data[i];
14
acc1 = acc1 OP data[i+1];
15
}
16
17
/* Finish any remaining elements */
18
for (; i < length; i++) {
19
acc0 = acc0 OP data[i];
20
}
21
*dest = acc0 OP acc1;
22
}
Figure 5.21
Applying 2 × 2 loop unrolling. By maintaining multiple accumulators,
this approach can make better use of the multiple functional units and
        their pipelining
capabilities.
```

函数	页	方法	整数 +	整数 *	浮点数 +	浮点数 *
combine4	551	累积到临时变量	1.27	3.01	3.01	5.01
combine5	568	2 × 1 展开	1.01	3.01	3.01	5.01
combine6	573	2 × 2 展开	0.81	1.51	1.51	2.51
延迟界限			1.00	3.00	3.00	5.00
吞吐量界限			0.50	1.00	1.00	0.50

可以看到,所有情形的性能都得到了改善:整数乘积、浮点加法和浮点乘法大约改善 2 倍,整数加法也有一定改善。最显著的是,我们突破了延迟界限造成的屏障。处理器不再需要等前一个求和或求积运算完成后才开始下一个。

要理解 combine6 的性能,我们先来看图 5.22 给出的代码和运算序列。可以按照图 5.23 所示的过程,推导出一个显示

```
%rax
%rbp
%rdx
%xmm0
```

```
vmulsd (%rax,%rdx,8), %xmm0, %xmm0
vmulsd 8(%rax,%rdx,8), %xmm1, %xmm1
addq $2,%rdx
cmpq %rdx,%rbp
jg loop
%rax
%rbp
%rdx
%xmm0
%xmm1
%xmm1
load
mul
load
mul
add
cmp
jg
Figure 5.22
Graphical representation of inner-loop code for combine6. Each
        iteration has two vmulsd
instructions, each of which is translated into a load and a mul
        operation.
```

迭代间数据依赖的模板。与 `combine5` 一样，内循环包含两条 `vmulsd` 指令，但它们被翻译成的 mul 运算读写的是不同的寄存器，彼此之间没有数据依赖（图 5.23(b)）。然后我们把该模板复制 $n/2$ 次（图 5.24），以建模该函数在长度为 n 的向量上的执行。可以看到，我们现在有两条关键路径：一条对应于计算下标为偶数的元素的乘积（程序值 acc0），

另一条对应于下标为奇数的元素（程序值 acc1）。每条关键路径只包含 $n/2$ 个运算，因此得到的 CPE 大约为 $5.00/2 = 2.50$。类似的分析解释了为什么对不同数据类型与合并运算的组合、延迟为 L 的运算，我们观测到的 CPE 大约为 $L/2$。从操作的角度看，这些程序是在利用功能单元的能力把它们的利用率提高到 2 倍。唯一的例外是整数加法。我们已经把 CPE 降到了 1.0 以下，但要达到理论极限 0.50，仍然有太多循环开销。

我们可以把多累加器变换推广为以因子 k 展开循环，并行地累积 k 个值，得到 $k \times k$ 循环展开。

图 5.25 展示了把这一变换应用于 k 直到 10 的值的效果。可以看到，对于足够大的 k，程序可以

几乎达到所有情形的吞吐量界限。整数加法在 $k = 7$ 时达到 CPE 0.54，接近由两个加载单元造成的 0.50 的吞吐量界限。整数乘法和浮点加法在 $k \geq 3$ 时达到 CPE 1.01，接近由它们的功能单元设定的 1.00 的吞吐量界限。浮点乘法在 $k \geq 10$ 时达到 CPE 0.51，接近由两个浮点乘法器和两个加载单元设定的

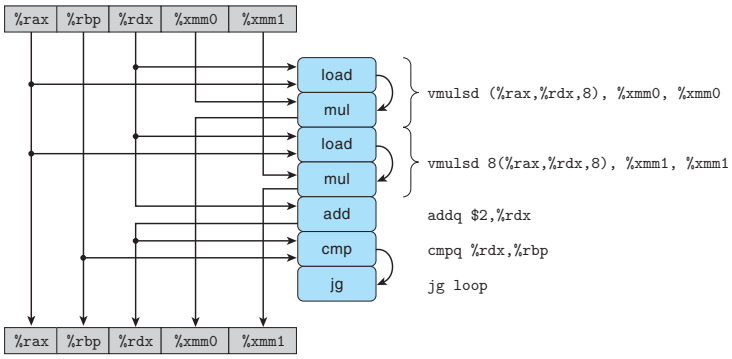


Figure 5.22 **Graphical representation of inner-loop code for** `combine6`. Each iteration has two `vmulsd` instructions, each of which is translated into a load and a mul operation.

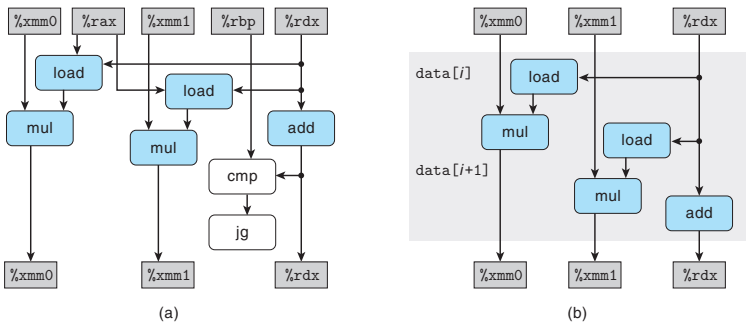


Figure 5.23 **Abstracting** `combine6` **operations as a data-flow graph.** We rearrange, simplify, and abstract the representation of Figure 5.22 to show the data dependencies between successive iterations (a). We see that there is no dependency between the two mul operations (b).

图 5.23

0.50 的吞吐量界限。值得指出的是，我们的代码用浮点乘法能达到的吞吐量几乎是浮点加法的两倍，尽管乘法是更复杂的运算。

一般说来，只有当一个程序能够让所有能够执行某个运算的功能单元的流水线都保持充满，它才能达到该运算的吞吐量界限。对一个延迟为 L、容量为 C 的运算，这要求展开因子 $k \geq C \cdot L$。例如，浮点乘法有 $C = 2$、$L = 5$，需要 $k \geq 10$ 的展开因子。浮点加法有 $C = 1$、$L = 3$，在 $k \geq 3$ 时达到最大吞吐量。

在执行 $k \times k$ 展开变换时，我们必须考虑它是否保持了原函数的功能。我们在第 2 章已经看到，即使发生溢出，补码算术也满足交换律和结合律。因此，对整数数据类型，`combine6` 计算出的结果在所有可能的条件下都与 `combine5` 相同。这样，优化编译器就有可能把 `combine4` 的代码先通过循环展开变成 `combine5` 的两路展开变体，再通过引入并行性变成 `combine6` 的类似形式。有些编译器确实会做这一变换或类似的变换，以改进整数数据的性能。

另一方面，浮点乘法和加法不满足结合律。因此，由于舍入或溢出，`combine5` 和 `combine6` 可能产生不同的结果。设想一个乘积计算，其中所

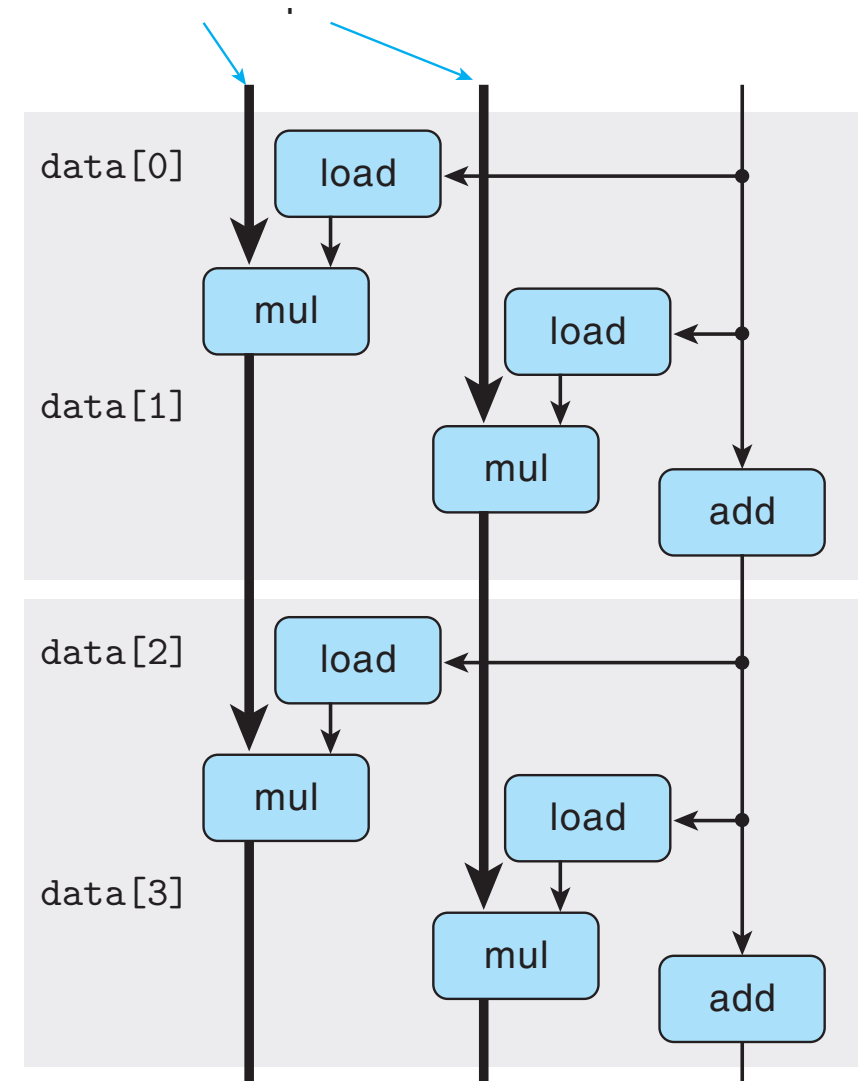


图 5.24 combine6 在长度为 n 的向量上操作的数据流表示。现在有两条关键路径，每条包含 n/2 次运算。

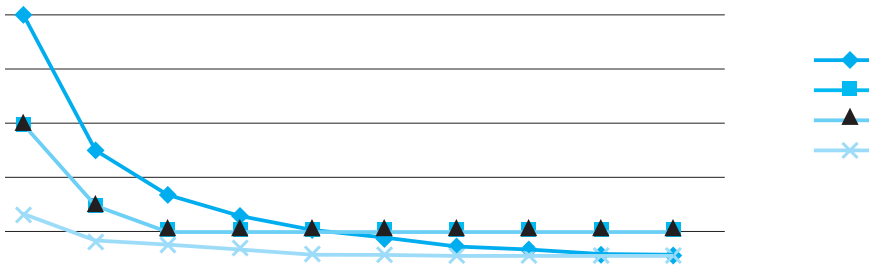


图 5.25 k×k 循环展开的 CPE 性能。所有的 CPE 都因这种变换而改善，达到或接近其吞吐量界限。

有下标为偶数的元素都是绝对值非常大的数，而下标为奇数的元素都非常接近0.0。在这种情形下，乘积 P_{En} 可能溢出，或者 P_{On} 可能下溢，尽管按正常顺序计算乘积 P_n 照常能够完成。不过在大多数实际应用中，这样的模式不太可能发生。物理现象大多是连续的，数值数据往往相当平滑、表现良好。即使出现不连续，它们一般也不至于造成周期性的模式，从而引发前面描述的那种情形。严格按顺序把各元素相乘，其精度从根本上优于把两组元素分别独立相乘、再把两个乘积相乘，这种情况的可能性不大。对大多数应用来说，取得 2× 的性能提升所带来的好处，超过了在特殊数据模式下得到不同结果的风险。不过，程序开发者应当与潜在用户确认，是否存在某些条件会使修改后的算法不可接受。大多数编译器不会对浮点代码尝试这类变换，因为对引入这种会改变程序行为的变换的风险，无论多小，它们都没有办法判断。

5.9.2 重关联变换

现在我们来探索另一种打破顺序依赖、把性能提高到延迟界限之上的方法。我们看到，`combine5` 的 $k \times 1$ 循环展开并没有改变合并向量元素形成其和或乘积所执行的运算集合。然而，只要对代码做很小的改动，我们就能从根本上改变执行合并的方式，并大幅提高程序性能。

图 5.26 给出的函数 `combine7` 与 `combine5` 的展开代码（图 5.16）之间的唯一区别，在于内循环中合并各元素的方式。在 `combine5` 中，合并由下面的语句执行：

```
12  acc = (acc OP data[i]) OP data[i+1];
```

而在 `combine7` 中，则由下面的语句执行：

```
12  acc = acc OP (data[i] OP data[i+1]);
```

两处差异只是两个括号的位置。我们把它称为重关联变换（reassociation transformation），因为括号改变了向量元素与累积值 acc 合并的先后，从而得到了我们称为“$2 \times 1a$”的一种循环展开形式。

在未经训练的眼看来，这两条语句基本上是一致的，但当我们测量 CPE 时，得到了一个令人惊讶的结果：

			整数		浮点数	
函数	页	方法	+	*	+	*
`combine4`	551	累积到临时变量	1.27	3.01	3.01	5.01
`combine5`	568	2×1 展开	1.01	3.01	3.01	5.01
`combine6`	573	2×2 展开	0.81	1.51	1.51	2.51
`combine7`	578	$2 \times 1a$ 展开	1.01	1.51	1.51	2.51
延迟界限			1.00	3.00	3.00	5.00
吞吐量界限			0.50	1.00	1.00	0.50

```
1
/* 2 x 1a loop unrolling */
2
void combine7(vec_ptr v, data_t *dest)
3
{
4
long i;
5
long length = vec_length(v);
6
long limit = length-1;
7
data_t *data = get_vec_start(v);
8
data_t acc = IDENT;
9
10
/* Combine 2 elements at a time */
11
for (i = 0; i < limit; i+=2) {
12
acc = acc OP (data[i] OP data[i+1]);
13
}
14
15
/* Finish any remaining elements */
16
for (; i < length; i++) {
17
acc = acc OP data[i];
18
}
19
*dest = acc;
20
}
Figure 5.26
Applying 2 × 1a unrolling. By reassociating the arithmetic, this
        approach
increases the number of operations that can be performed in parallel.
```

整数加法的情形与 $k \times 1$ 展开(`combine5`)的性能相同,而其他三个情形与使用并行累加器的版本(`combine6`)的性能相同,相对于 $k \times 1$ 展开性能提高了一倍。这几种情形都突破了延迟界限造成的屏障。

图 5.27 说明了 `combine7` 的内循环代码(合并运算为乘法、数据类型为 double 的情形)如何被译码成运算以及由此产生的数据依赖。可以看到,由 `vmovsd` 指令和第一条 `vmulsd` 指令产生的 load 运算从存储器加载向量元素 i 和 $i+1$,而第一条 mul 运算把它们相乘。然后第二条 mul 运算把这个结果乘上累积值 acc。图 5.28(a) 展示了我们如何重排、细化和抽象图 5.27 的运算,得到

一个表示一次迭代的数据依赖的模板(图 5.28(b))。与 combine5 和 combine7 的模板一样,我们有两个 load 运算和两个 mul 运算,但只有其中一个 mul 运算构成循环寄存器之间的数据依赖链。接下来,我们把这个模板复制 $n/2$ 次,以显示对 n 个向量元素做乘法的计算(图 5.29),可以看到关键路径上只有 $n/2$ 个运算。每次迭代中的第一个乘法不必等待上一个迭代累积的值就可以执行。这样,最小可能的 CPE 就降低到大约原来的 1/2。

```
%rax
%rbp
%rdx
%xmm0
vmulsd 8(%rax,%rdx,8), %xmm0, %xmm0
vmulsd %xmm0, %xmm1, %xmm1
vmovsd (%rax,%rdx,8), %xmm0
addq $2,%rdx
cmpq %rdx,%rbp
jg loop
%rax
%rbp
%rdx
%xmm0
%xmm1
%xmm1
load
load
mul
mul
add
cmp
jg
Figure 5.27
Graphical representation of inner-loop code for combine7. Each
iteration gets decoded into similar operations as for combine5 or
        combine6, but with
different data dependencies.
```

图 5.30 展示了把重关联变换应用于我们称为 $k \times 1a$ 循环展开、直到 $k = 10$ 的值的效果。可以看到,这一变换取得的性能结果类似于 $k \times k$ 展开中维护 k 个单独累加器所能达到的效果。所有情形都接近功能单元施加的吞吐量界限。

在执行重关联变换时,我们又一次改变了向量元素合到一起的顺序。对整数加法和乘法,这些运算满足结合律的事实意味着这种重排不会对结果产生影响。对浮点数的情形,我们必须再一次评估这种重关联是否可能显著影响

结果。我们会说,对大多数应用来说,这一差别无关紧要。

总而言之,重关联变换可以减少计算中关键路径上的运算数量,通过更好地利用多个功能单元及其流水线能力取得更好的性能。大多数编译器不会尝试对浮点运算做任何重关联,因为这些运算不保证满足结合律。当前版本的

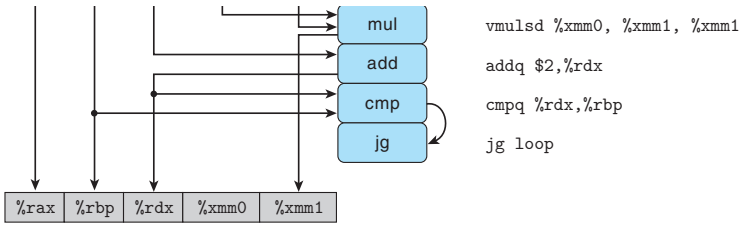


Figure 5.27 **Graphical representation of inner-loop code for** `combine7`. Each iteration gets decoded into similar operations as for `combine5` or `combine6`, but with different data dependencies.

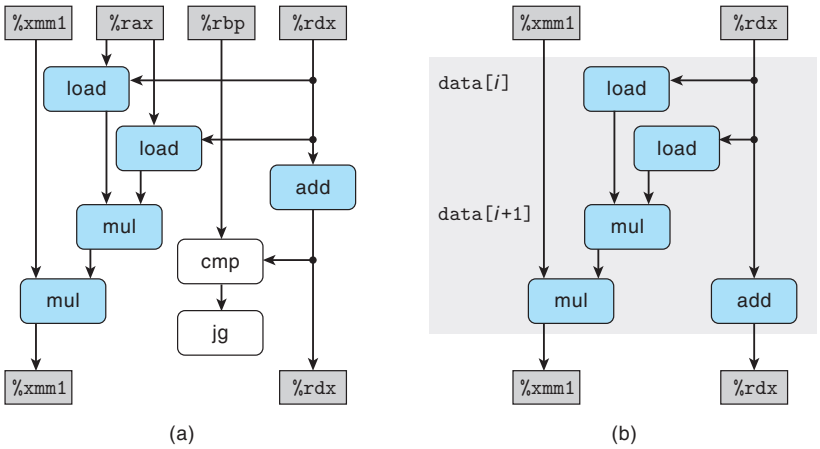


Figure 5.28 **Abstracting** `combine7` **operations as a data-flow graph.** We rearrange, simplify, and abstract the representation of Figure 5.27 to show the data dependencies between successive iterations. The upper mul operation multiplies two 2-vector elements with each other, while the lower one multiplies the result by loop variable `acc`.

图 5.28

gcc 确实会对整数运算做重关联，但效果不一定好。总的来说，我们发现展开循环并并行地累积多个值是取得程序性能改进的更可靠途径。

练习 5.8 考虑下面这个函数，它计算一个由 n 个 double 精度数组成的数组的乘积。我们已经把循环以因子 3 做了展开：

```
double aprod(double a[], long n)
{
    long i;
    double x, y, z;
    double r = 1;
    for (i = 0; i < n-2; i+= 3) {
        x = a[i]; y = a[i+1]; z = a[i+2];
        r = r * x * y * z; /* 乘积计算 */
    }
    for (; i < n; i++)
        r *= a[i];
    return r;
```

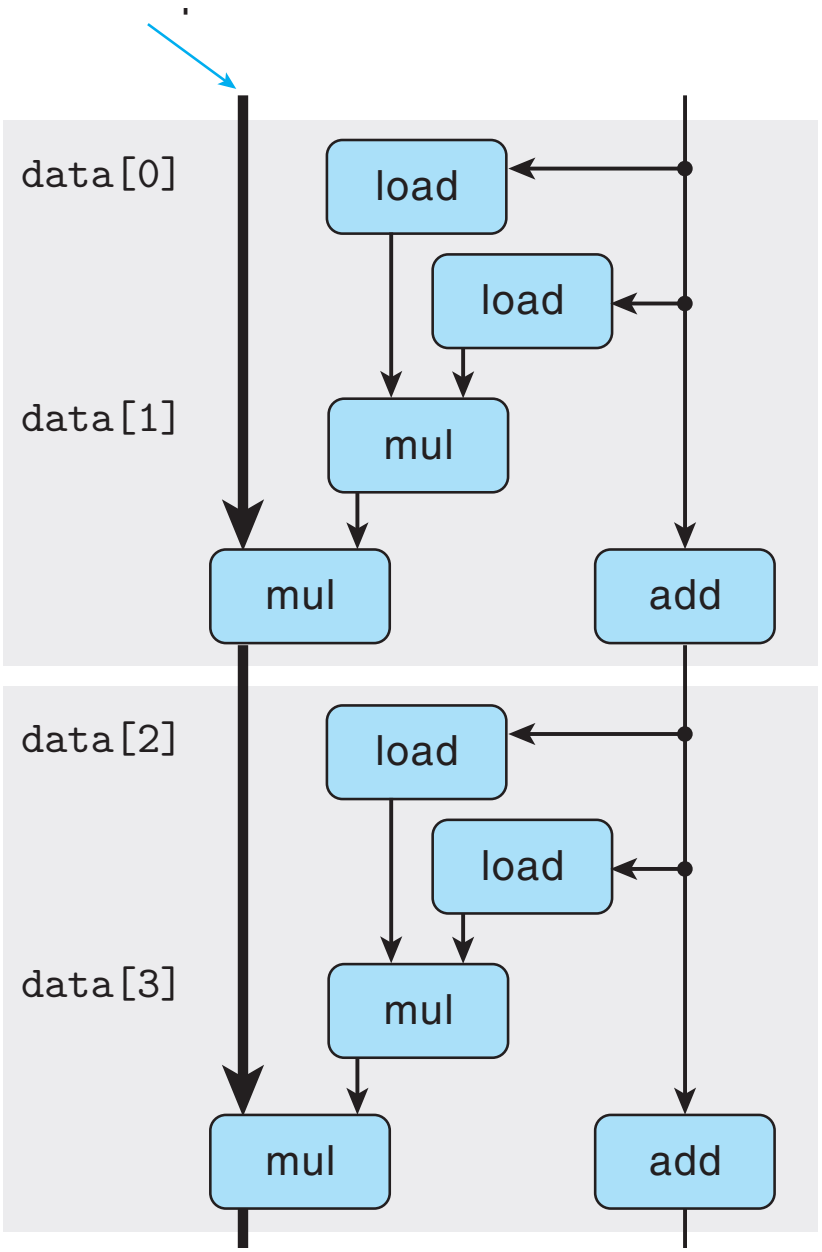


图 5.29 combine7 在长度为 n 的向量上操作的数据流表示。我们只有一条关键路径，但它只包含 n/2 次运算。

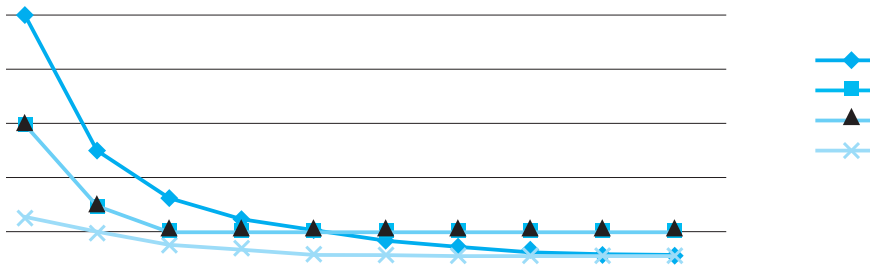


图 5.30 k×1a 循环展开的 CPE 性能。所有的 CPE 都因这种变换而改善，接近其吞吐量界限。

```
}
```

对于标注着“乘积计算”的那一行，我们可以用括号为这一计算创建五种不同的结合方式，如下：

```
r = ((r * x) * y) * z;   /* A1 */
r = (r * (x * y)) * z;   /* A2 */
r = r * ((x * y) * z);   /* A3 */
r = r * (x * (y * z));   /* A4 */
r = (r * x) * (y * z);   /* A5 */
```

假设我们在一台浮点乘法延迟为 5 个时钟周期的机器上运行这些函数。请确定由乘法的数据依赖给 CPE 设定的下界。(提示：画出每次迭代中 r 的计算方式的数据流表示会很有帮助。)

网络边栏 OPT:SIMD 用向量指令实现更高程度的并行

如 3.1 节所述，Intel 在 1999 年引入了 SSE 指令，SSE 是“流式 SIMD 扩展”(streaming SIMD extensions) 的缩写，而 SIMD (读作 “sim-dee”) 又是 “单指令多数据”(single instruction, multiple data) 的缩写。SSE 能力经历了多代演化，较新的版本称为高级向量扩展 (advanced vector extensions，AVX)。SIMD 执行模型涉及在单条指令内对整个数据向量进行操作。这些向量保存在`%ymm0` ~ `%ymm15` 这样一组特殊的向量寄存器中。当前的 AVX 向量寄存器长 32 字节，因此每个可以存放 8 个 32 位数或 4 个 64 位数，这些数既可以是整数，也可以是浮点数。AVX 指令随即就可以对这些寄存器执行向量运算，例如并行地对 8 组或 4 组值做加法或乘法。例如，如果 YMM 寄存器`%ymm0` 包含 8 个单精度浮点数，我们记作 $a_0, \ldots, a_7$，而`%rcx` 包含 8 个紧邻的单精度浮点数的内存地址，我们记作 $b_0, \ldots, b_7$，那么指令 `vmulps (%rcs), %ymm0, %ymm1` 会从存储器读出这 8 个值，并行执行 8 个乘法——对 $0 \le i \le 7$ 计算 $a_i \leftarrow a_i \cdot b_i$，并把得到的 8 个乘积存入向量寄存器`%ymm1`。我们看到一条指令能够对多个数据值引发计算，因此称它为“SIMD”。

gcc 支持 C 语言的扩展，让程序员能够以向量运算的形式表达程序，从而可以编译成 AVX 的向量指令(以及基于较早 SSE 指令的代码)。这种编码风格比直接用汇编语言写代码更可取，因为 gcc 还能为其他处理器上的向量指令生成代码。

把 gcc 扩展、循环展开和多个累加器结合起来，我们可以为合并函数取得如下性能：

	整数				浮点数			
	int		long		int		long	
方法	+	*	+	*	+	*	+	*
标量 10 × 10	0.54	1.01	0.55	1.00	1.01	0.51	1.01	0.52
标量吞吐量界限	0.50	0.50	1.00	1.00	1.00	1.00	0.50	0.50
向量 8 × 8	0.05	0.24	0.13	1.51	0.12	0.08	0.25	0.16
向量吞吐量界限	0.06	0.12	0.12	–	0.12	0.06	0.25	0.12

在这张表中，第一组数字对应的是按 `combine6` 风格编写的常规标量代码——展开因子为 10，同时维护 10 个累加器。第二组数字对应的是用 `gcc` 能编译成 AVX 向量代码的形式编写的代码。除了使用向量运算之外，这一版本还把主循环展开 8 倍，维护 8 个独立的向量累加器。我们分别给出 32 位数和 64 位数的两组结果，因为向量指令在前一种情形有 8 路并行，在后一种情形只有 4 路并行。

可以看到，向量代码在四个 32 位的情形中差不多取得了 8 倍的改进，在四个 64 位的情形中有三个取得了 4 倍的改进。只有 long 整数乘法代码在我们尝试用向量代码表达时表现不佳。AVX 指令集没有包含对 64 位整数做并行乘法的指令，所以 `gcc` 不能为这一情形生成向量代码。使用向量指令为合并运算带来了一个新的吞吐量界限。与标量界限相比，32 位运算的这个界限低 8 倍，64 位运算低 4 倍。我们的代码在若干数据类型与运算的组合上接近达到这些界限。

5.10 合并代码优化的结果小结

我们把向量的加法与乘法例程的性能推到极限，这些努力显然得到了回报。下面小结我们用标量代码、不利用 AVX 向量指令提供的向量并行所得到的结果：

			整数		浮点数	
函数	页	方法	+	*	+	*
`combine1`	543	抽象代码，-O1	10.12	10.12	10.17	11.14
`combine6`	573	2 × 2 展开	0.81	1.51	1.51	2.51
		10 × 10 展开	0.55	1.00	1.01	0.52
延迟界限			1.00	3.00	3.00	5.00
吞吐量界限			0.50	1.00	1.00	0.50

综合运用多种优化之后，我们已经能把 CPE 提高到接近 0.50 和 1.00 的吞吐量界限，唯一的限制来自功能单元的能力。相对于最初的代码，这相当于 10 ∼ 20 倍的改进。这一切都是用普通的 C 代码和一个标准编译器做到的。改写代码以利用较新的 SIMD 指令，还能带来接近 4 倍或 8 倍的额外性能收益。以单精度乘法为例，CPE 从最初的 11.14 降到 0.06，整体性能收益超过 180 倍。

这个例子说明,现代处理器拥有相当可观的计算能力,但要发挥出这份能力,我们可能需要用非常程式化的方式来编写程序。

5.11 一些限制因素

我们已经看到,程序的数据流图表示中的关键路径指示出程序执行所需时间的一个基本下界。也就是说,如果程序中存在某条数据依赖链,链上所有延迟之和等于 T,那么该程序至少需要 T 个周期才能执行完。

我们还看到,功能单元的吞吐量界限同样会给程序的执行时间带来一个下界。也就是说,假设一个程序总共需要对某种运算执行 N 次计算,微处理器有 C 个能执行这种运算的功能单元,这些单元的发射时间为 I,那么该程序至少需要 $N \cdot I/C$ 个周期才能执行完。

本节考虑在实际机器上限制程序性能的其他一些因素。

5.11.1 寄存器溢出

循环并行性收益的大小,受限于用汇编代码表达计算的能力。如果程序的并行度 P 超过了可用寄存器的数量,编译器就会转而采用寄存器溢出,把一些临时值放进存储器,通常是在运行时栈上分配空间。举个例子,下面的测量把 `combine6` 的多累积变量方案扩展到 $k = 10$ 和 $k = 20$ 的情形,比较其结果:

			整数		浮点数	
函数	页码	方法	+	*	+	*
`combine6`	573					
		10×10 展开	0.55	1.00	1.01	0.52
		20×20 展开	0.83	1.03	1.02	0.68
吞吐量界限			0.50	1.00	1.00	0.50

可以看到,随着展开程度加大,没有一个 CPE 得到改进,有的反而更差。现代 x86-64 处理器有 16 个整数寄存器,还能以 16 个 YMM 寄存器来存放浮点数据。一旦循环变量的个数超过了可用寄存器的数量,程序就必须把其中一些分配到栈上。

举个例子,下面这段代码展示了 10×10 展开的代码在内循环中是如何更新累积变量 `acc0` 的:

```
10 × 10 展开代码中对累积变量 acc0 的更新
  vmulsd  (%rdx), %xmm0, %xmm0        acc0 *= data[i]
```

可以看到,累积变量保存在寄存器`%xmm0` 中,因此程序只需从存储器读出 `data[i]`,与这个寄存器相乘即可。

20×20 展开的代码中与之对应的部分,形式则大不相同:

```
20 × 20 展开代码中对累积变量 acc0 的更新
  vmovsd   40(%rsp), %xmm0
  vmulsd   (%rdx), %xmm0, %xmm0
  vmovsd   %xmm0, 40(%rsp)
```

累积变量作为栈上的局部变量保存,距栈指针的偏移量为 40。程序必须从存储器读出它的值和 data[i] 的值,把两者相乘,再把结果存回存储器。

一旦编译器不得不采用寄存器溢出,维护多个累积变量的优势多半就会丧失。幸运的是,x86-64 的寄存器足够多,大多数循环在出现这种情况之前就已经达到了吞吐量限制。

5.11.2 分支预测和预测错误处罚

3.6.6 节曾用实验证明:当分支预测逻辑没有正确预测出一条条件分支是否会发生时,这条条件分支可能带来很大的预测错误处罚。现在我们已经了解了处理器的工作方式,可以弄明白这种处罚是从哪里来的了。

现代处理器远远超前于当前执行的指令,从存储器读出新指令并对它们译码,以确定要对什么操作数执行什么操作。只要指令按简单的顺序排列,这种指令流水线化就能很好地工作。一旦遇到分支,处理器就必须猜测分支会往哪个方向走。对条件跳转来说,这意味着预测分支是否会发生;对间接跳转(正如我们在按跳转向量表项给出的地址进行跳转的代码中所见)或过程返回这类指令来说,这意味着预测目标地址。这里的讨论集中在条件分支上。

在采用推测执行的处理器中,处理器会开始执行预测出的分支目标处的指令。它这样做的方式避免了在确定实际结果之前修改任何实际的寄存器或存储器位置。如果预测正确,处理器就可以

通过把这些推测执行的指令的结果写入寄存器或存储器来“提交”。如果预测不正确,处理器必须丢弃所有推测执行的结果,并从正确的位置重新开始取指过程。这样做会招致预测错误处罚,因为在生成有效结果之前必须重新填满指令流水线。

我们在 3.6.6 节看到,较新的 x86 处理器版本,包括所有能执行 x86-64 程序的处理器,都有条件传送指令。gcc 在编译条件语句和条件表达式时,可以生成使用这些指令的代码,而不是更传统的基于条件控制转移的实现方式。用条件传送来翻译的核心思想是:把条件表达式或条件语句两条分支上的值都计算出来,然后用条件传送指令选出想要的那个值。我们在 4.5.7 节看到,条件传送指令可以作为对普通指令流水线化处理的一部分来实现。这样就不需要猜测条件是否成立,因而也没有猜错的处罚。

那么，C 程序员怎样才能确保分支预测错误处罚不至于拖累程序的效率呢？考虑到我们在参考机上测得 19 个周期的预测错误处罚，这个问题事关重大。它没有简单的答案，但下面几条一般性原则是适用的。

不要为可预测的分支过分担心

我们已经看到，一次预测错误的分支可能造成很大的影响，但这并不意味着程序中的所有分支都会拖慢程序。实际上，现代处理器中的分支预测逻辑非常善于辨识各类分支指令的规律性模式和长期趋势。例如，我们的合并例程中结束循环的那条分支通常会被预测为发生，因而只有在最后一次迭代时才会招致预测错误处罚。

再举一个例子：把函数 `get_vec_element` 移出 `combine2` 的内循环而得到 `combine3` 时，我们观察到如下结果：

			整数		浮点数	
函数	页码	方法	+	*	+	*
`combine2`	545	移动 `vec_length`	7.02	9.03	9.02	11.03
`combine3`	549	直接数据访问	7.17	9.02	9.02	11.03

尽管这次变换在每次迭代中消除了两个检查向量下标是否越界的条件测试，CPE 也没有改善。对这个函数来说，这些检查总是成功，因而它们是高度可预测的。

要测量边界检查对性能的影响，可以考虑下面这段合并代码：我们用对 `get_vec_element` 的代码做内联替换后得到的结果来访问数据元素，以此修改了 `combine4` 的内循环。我们将把这个新版本

称为 `combine4b`。这段代码执行边界检查，并且还通过向量数据结构来引用向量元素。

```
1  /* 在循环中包含边界检查 */
2  void combine4b(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t acc = IDENT;
7
8      for (i = 0; i < length; i++) {
9          if (i >= 0 && i < v->len) {
10             acc = acc OP v->data[i];
11         }
12     }
13     *dest = acc;
14 }
```

然后我们就可以直接比较带边界检查与不带边界检查的两个函数的 CPE：

			整数		浮点数	
函数	页码	方法	+	*	+	*
combine4	551	无边界检查	1.27	3.01	3.01	5.01
combine4b	551	边界检查	2.02	3.01	3.01	5.01

带边界检查的版本在整数加法的情形下稍慢一些,但在其他三种情形下取得了相同的性能。这三种情形的性能本来就受限于各自合并操作的延迟。执行边界检查所需的额外计算可以与合并操作并行进行。处理器能够预测这些分支的结果,因此所有这些求值对程序执行中构成关键路径的那些指令的读取和处理都没有多大影响。

编写适合用条件传送来实现的代码

分支预测只在有规律的模式下才可靠。程序中的许多测试是完全不可预测的,取决于数据的任意特征,例如一个数是负还是正。对这些测试,分支预测逻辑会表现得很差。对于本质上不可预测的情形,如果编译器能够生成用条件数据传送而不是条件控制转移的代码,程序性能就能获得大幅提升。这一点不能由 C 程序员直接控制,但在表达条件行为时,有些写法可以比其他写法更直接地翻译成条件传送。

我们发现,gcc 能为按更“函数式”的风格编写的代码生成条件传送——在这种风格中,我们用条件运算来计算

出结果值,然后再用这些值更新程序状态;与此相对的是更“命令式”的风格,即用条件来选择性地更新程序状态。

这两种风格都没有严格的规则可言,所以我们用一个例子来说明。假设给定两个整数数组 `a` 和 `b`,在每个位置 i 上,我们想把 `a[i]` 设置为 `a[i]` 和 `b[i]` 中的较小值,把 `b[i]` 设置为较大值。

实现这个函数的一种命令式风格是:在每个位置 i 上进行检查,若两个元素次序颠倒就交换它们:

```
1  /* 调整两个数组的排列,使得对每个 i,都有 b[i] >= a[i] */
2  void minmax1(long a[], long b[], long n) {
3      long i;
4      for (i = 0; i < n; i++) {
5          if (a[i] > b[i]) {
6              long t = a[i];
7              a[i] = b[i];
8              b[i] = t;
9          }
10     }
11 }
```

我们对这个函数的测量结果表明,随机数据的 CPE 约为 13.5,可预测数据的 CPE 为 2.5 ~ 3.5,这表明预测错误处罚约为 20 个周期。

实现这个函数的函数式风格是:在每个位置 i 上计算出最小值和最大值,然后把这些值分别赋给 `a[i]` 和 `b[i]`:

```
1  /* 调整两个数组的排列,使得对每个 i,都有 b[i] >= a[i] */
2  void minmax2(long a[], long b[], long n) {
3      long i;
4      for (i = 0; i < n; i++) {
5          long min = a[i] < b[i] ? a[i] : b[i];
6          long max = a[i] < b[i] ? b[i] : a[i];
7          a[i] = min;
8          b[i] = max;
9      }
10 }
```

我们对这个函数的测量结果表明,无论数据是任意的还是可预测的,CPE 都约为 4.0。(我们还检查了生成的汇编代码,以确认它确实使用了条件传送。)

正如 3.6.6 节讨论的,不是所有条件行为都能用条件数据传送来实现,因此难免出现程序员无法避免的情形:不得不写出某些代码,它们产生的条件分支会让处理器的分支预测表现得很差。不过正如我们所展示的,程序员只需一点巧思,有时就能使代码更容易被翻译成条件数据传送。这需要一些实验

,即编写函数的不同版本,然后检查生成的汇编代码并测量性能。

练习 5.9 归并排序合并步骤的传统实现需要三个循环 [98]:

```
1  void merge(long src1[], long src2[], long dest[], long n) {
2      long i1 = 0;
3      long i2 = 0;
4      long id = 0;
5      while (i1 < n && i2 < n) {
6          if (src1[i1] < src2[i2])
7              dest[id++] = src1[i1++];
8          else
9              dest[id++] = src2[i2++];
10     }
11     while (i1 < n)
12         dest[id++] = src1[i1++];
13     while (i2 < n)
14         dest[id++] = src2[i2++];
15 }
```

把变量 `i1` 和 `i2` 与 `n` 进行比较所产生的分支有良好的预测表现——唯一的预测错误发生在它们第一次变成假的时候。而 `src1[i1]` 与 `src2[i2]` 两个值之间的比较(第 6 行)对典型的数据来说是高度不可预测的。这个比较控制着一条条件分支,在随机数据上运行时,CPE(元素个数为 $2n$)约为 15.0。

改写这段代码,使第一个循环(第 6 ~ 9 行)中条件语句的效果可以用一次条件传送来实现。

5.12 理解存储器性能

到目前为止我们编写的全部代码和运行的测试，所访问的存储量都比较小。例如，合并例程是在长度不足 1000 个元素的向量上测量的，所需数据不超过 8000 字节。所有现代处理器都包含一个或多个高速缓存存储器，以便快速访问这样小量的存储数据。本节将进一步考察涉及加载（load，把数据从存储器读入寄存器）与存储（store，把寄存器的数据写到存储器）操作的程序的性能，且只考虑所有数据都保存在高速缓存中的情形。在第 6 章中，我们会更详细地讨论高速缓存如何工作、它们的性能特性，以及如何编写能最好地利用高速缓存的代码。

如图 5.11 所示，现代处理器有专门执行加载和存储操作的功能单元，这些单元带有内部缓冲区，用来保存尚未完成的存储器操作请求的集合。例如，我们的参考机有两个加载单元，每个最多能保存 72 个待处理的读请求；有一个存储单元，其存储缓冲区最多能保存 42 个写请求。这些单元每个都能在一个时钟周期内发起 1 个操作。

5.12.1 加载性能

包含加载操作的程序的性能取决于加载单元的流水线化能力和延迟。在用参考机做合并操作的实验中，我们看到，除使用 SIMD 操作的情形外，对数据类型与合并操作的任何组合，CPE 都从未低于 0.50。限制我们这些示例的 CPE 的一个因素是，它们每计算一个元素都要从存储器读出一个值。有两个加载单元，每个最多能在一个时钟周期内发起 1 次加载操作，CPE 就不可能低于 0.50。对于每计算一个元素必须加载 k 个值的应用来说，CPE 永远不可能低于 $k/2$（例如参见练习 5.15）。

在我们至此的示例中，还没有看到加载操作的延迟带来的任何性能影响。这些示例中加载操作的地址只取决于循环下标 `i`，因此加载操作没有成为限制性能的关键路径的一部分。

要确定一台机器上加载操作的延迟，可以构造一种计算，让它由一串加载操作组成，其中一次加载的结果决定下一次加载的地址。举个例子，考虑图 5.31 中计算链表长度的函数 `list_len`。在这个函数的循环中，变量 `ls` 的每个新值都取决于指针引用 `ls->next` 读出的值。我们的测量表明，函数 `list_len`

```
1
typedef struct ELE {
2
struct ELE *next;
3
long data;
```

```
4
} list_ele, *list_ptr;
5
6
long list_len(list_ptr ls) {
7
long len = 0;
8
while (ls) {
9
len++;
10
ls = ls->next;
11
}
12
return len;
13
}
Figure 5.31
Linked list function. Its performance is limited by the latency of the
        load
operation.
```

的 CPE 是 4.00,我们认为这个值直接反映了加载操作的延迟。要看出这一点,考虑这个循环的汇编代码:

```
list_len 的内循环
ls 在 %rdi 中,len 在 %rax 中
 1  .L3:                                  loop:
 2    addq    $1, %rax                    递增 len
 3    movq    (%rdi), %rdi                ls = ls->next
 4    testq   %rdi, %rdi                  测试 ls
 5    jne     .L3                         若非空,转到 loop
```

第 3 行的 `movq` 指令构成了这个循环的关键瓶颈。寄存器`%rdi` 的每个新值都依赖于以`%rdi` 中的值为地址的一次加载操作的结果。这样, 某一次迭代的加载操作要等上一次迭代的加载完成之后才能开始。这个函数 4.00 的 CPE 由加载操作的延迟决定。事实上, 这一测量值与参考机 L1 高速缓存 4 个周期的标称访问时间相符,见 6.4 节的讨论。

5.12.2 存储性能

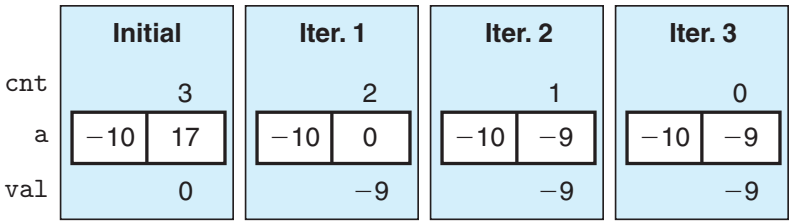
在我们至此示例中分析的函数里, 访问存储器用的几乎都是加载操作, 即从某个存储位置读到寄存器。与之对偶的是存储操作,它把一个寄存器的值写到存储器。存储操作的性能, 尤其是它与加载操作的相互作用, 涉及若干微妙的问题。

与加载操作一样,在大多数情况下存储操作也能以完全流水线化的模式工作,每个周期开始一个新的存储操作。例如,考虑图 5.32 所示的函数,它把长度

为 n 的数组 dest 的各元素清零。我们的测量表明 CPE 为 1.0。这已经是在只有单个存储功能单元的机器上所能达到的最好结果。

```
1
/* Set elements of array to 0 */
2
void clear_array(long *dest, long n) {
3
long i;
4
for (i = 0; i < n; i++)
5
dest[i] = 0;
6
}
Figure 5.32
Function to set array elements to 0. This code achieves a CPE of 1.0.
```

与我们至此考虑过的其他操作不同,存储操作不影响任何寄存器的值。因此就其本质而言,一连串存储操作不可能创建数据依赖。只有加载操作会受到存储操作结果的影响,因为只有加载能把存储操作写入的存储器值读回来。图 5.33 所示的函数



Example B: `write_read(&a[0],&a[0],3)`

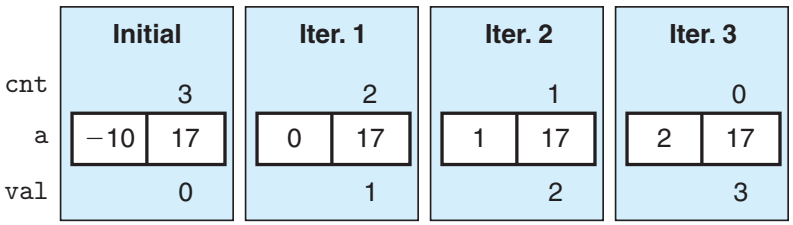


图 5.33 写和读内存位置的代码及其示例执行。该函数展示了当参数 src 和 dest 相等时存储与加载之间的相互作用。

write_read 展示了加载与存储之间可能存在的相互作用。该图还给出了这个函数的两次示例执行:调用时针对的是二元素数组 a,a 的初始内容为 −10 和 17,参数 cnt 等于 3。这两次执行展示了加载和存储操作的一些微妙之处。

在图 5.33 的示例 A 中,参数 src 是指向数组元素 a[0] 的指针,而 dest 是指向数组元素 a[1] 的指针。在这种情形下,经指针引用 *src 进行的每次加

载得到的都是值 −10。因而，两次迭代之后，两个数组元素将分别固定为 −10 和 −9。从 src 读取的结果不受写到 dest 的影响。对更多次迭代测量这个例子，得到的 CPE 为 1.3。

在图 5.33 的示例 B 中，参数 src 和 dest 都是指向数组元素 a[0] 的指针。在这种情形下，经指针引用 *src 进行的每次加载得到的，都是上一次执行指针引用 *dest 时存储的值。

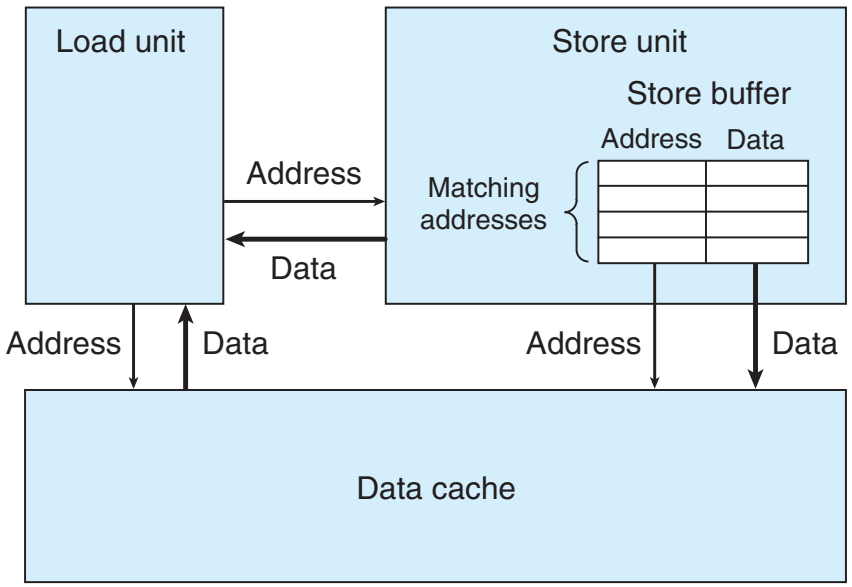


图 5.34 加载单元和存储单元的细节。存储单元维护一个待写缓冲区。加载单元必须将它的地址与存储单元中的地址进行比较，以检测写/读依赖。

结果，一系列递增的值会被存入这个位置。一般而言，如果调用函数 write_read 时参数 src 和 dest 指向同一个存储位置，参数 cnt 的值为某个 $n > 0$，那么总效果就是把该位置设置为 $n - 1$。这个例子展示了一种现象，我们称之为写/读依赖（write/read dependency）：一次存储器读取的结果取决于最近的一次存储器写入。我们的性能测量表明，示例 B 的 CPE 为 7.3。写/读依赖使处理过程慢了大约 6 个时钟周期。

要弄明白处理器如何区分这两种情形、以及为什么其中的一个比另一个慢，我们必须更细致地考察加载和存储执行单元，如图 5.34 所示。存储单元包含一个存储缓冲区，其中存放着已发射给存储单元但尚未完成的存储操作的地址和数据，这里的完成意味着更新数据高速缓存。提供这个缓冲区，是为了让一连串存储操作得以执行，而不必等每一个操作都更新完高速缓存。发生加载操作时，它必须检查存储缓冲区中的各项是否有地址匹配。如果找到匹配（即

正被写的字节与正在被读的字节具有相同地址),加载操作就把相应的数据项取出,作为加载操作的结果。

gcc 为 write_read 的内循环生成如下代码:

```
write_read 的内循环
src 在 %rdi 中,dst 在 %rsi 中,val 在 %rax 中
.L3:                                    loop:
  movq    %rax, (%rsi)          把 val 写到 dst
  movq    (%rdi), %rax          t = *src
  addq    $1, %rax              val = t+1
  subq    $1, %rdx              cnt--
  jne     .L3                   若非 0,转到 loop
```

```
Figure 5.35
Graphical representation
of inner-loop code
for write_read. The
first movl instruction is
decoded into separate
operations to compute the
store address and to store
the data to memory.
%rax
%rdi
%rsi
%rdx
movq %rax,(%rsi)
movq (%rdi),%rax
addq $1,%rax
subq $1,%rdx
jne loop
%rax
%rdi
%rsi
%rdx
s_addr
s_data
load
add
sub
jne
```

图 5.35 给出了这个循环代码的数据流表示。指令 movq %rax,(%rsi) 被译码成两个操作:s_addr 指令为存储操作计算地址,在存储缓冲区中创建一个表项,并设置该表项的地址字段。s_data 操作为该表项设置数据字段。我们将看到,这两项计算各自独立进行这一点,对程序性能可能很重要。参考机为这些操作安排单独的功能单元,原因正在于此。

除了因寄存器的写入和读取造成的操作间数据依赖外,运算符右侧的弧线表示这些操作的一组隐式依赖。特别要指出,s_addr 操作的地址计算显然必

须先于 s_data 操作。此外,译码指令 movq (%rdi),%rax 产生的加载操作必须检查所有待处理存储操作的地址,这就创建了它与 s_addr 操作之间的数据依赖。图中在 s_data 操作与加载操作之间画出一条虚线弧。这个依赖是有条件的:如果两个地址相同,加载操作必须等到 s_data 把结果送入存储缓冲区;如果两个地址不同,两个操作可以独立进行。

图 5.36 展示了 write_read 内循环中诸操作之间的数据依赖。在图 5.36(a) 中,我们把这些操作重新摆了一下位置,使依赖关系看得更清楚。为了引起对涉及加载和存储操作的三个依赖的特别注意,我们给它们分别加了编号。标着“1”的弧线表示存储数据之前必须先计算出存储地址这一要求。标着“2”的弧线表示加载操作需要把它的地址与任何待处理存储操作的地址进行比较。最后,标着“3”的虚线弧线表示加载地址与存储地址相同时出现的条件数据依赖。

图 5.36(b) 展示了去掉那些对数据从一个迭代流向下一个迭代没有直接影响的操作之后的情况。数据流图只剩下两条依赖链:左边这条,是数据被存储、加载并递增(仅在地址相同的情形);右边这条,是递减变量 cnt。

```
Figure 5.36
Abstracting the
operations for write_
read. We first rearrange the
operators of Figure 5.35(a)
and then show only those
operations that use values
from one iteration to
produce new values for the
next (b).
%rax
%rdi
%rsi
%rdx
%rax
%rdx
s_addr
1
2
3
s_data
load
(a)
(b)
add
sub
jne
%rax
%rdx
%rax
%rdx
s_data
load
```

```
add
sub
```

现在我们可以理解函数 write_read 的性能特性了。图 5.37 展示了它的内循环多次迭代所形成的数据依赖。对图 5.33 中示例 A 的情形,源地址与目的地址不同,加载和存储操作可以独立进行,因此唯一的关键路径由变量 cnt 的递减形成,给出的 CPE 下界为 1.0。对示例 B 的情形,源地址与目的地址相同,s_data 指令与加载指令之间的数据依赖导致形成一条关键路径,涉及数据的存储、加载和递增。我们发现,这三个操作依次执行总共需要约 7 个时钟周期。

正如这两个例子所表明的,存储器操作的实现涉及许多微妙之处。对寄存器操作而言,处理器在把指令译码成操作时,就能确定哪些指令会影响哪些指令。而对存储器操作而言,处理器要等到加载地址和存储地址被计算出来之后,才能预测哪些操作会影响哪些操作。高效地处理存储器操作对许多程序的性能至关重要。存储器子系统利用了多种优化,例如在多个操作可以独立进行时利用潜在的并行性。

练习 5.10 作为带有潜在加载-存储相互作用的代码的另一个例子,考虑下面这个把一个数组的内容复制到另一个数组的函数:

```
1  void copy_array(long *src, long *dest, long n)
2  {
3      long i;
4      for (i = 0; i < n; i++)
5          dest[i] = src[i];
6  }
```

假设 a 是长度为 1000 的数组,初始化为每个元素 a[i] 等于 i。

A. 调用 copy_array(a+1,a,999) 会产生什么效果?

B. 调用 copy_array(a,a+1,999) 会产生什么效果?

C. 我们的性能测量表明,A 部分的调用 CPE 为 1.2(循环以系数 4 展开时降为 1.0),而 B 部分的调用 CPE 为 5.0。你认为这一性能差异应归因于什么因素?

D. 对于调用 copy_array(a,a,999),你预期性能会是怎样的?

练习 5.11 我们看到,对前缀和函数 psum1(图 5.1)的测量结果表明,在一台机器上,该函数要执行的基本操作——浮点加法——只有 3 个时钟周期的延迟,但 CPE 却为 9.00。我们来试着理解为什么这个函数的表现这么差。

下面是这个函数内循环的汇编代码:

```
psum1 的内循环
a 在 %rdi 中,i 在 %rax 中,cnt 在 %rdx 中
 1  .L5:                                    loop:
```

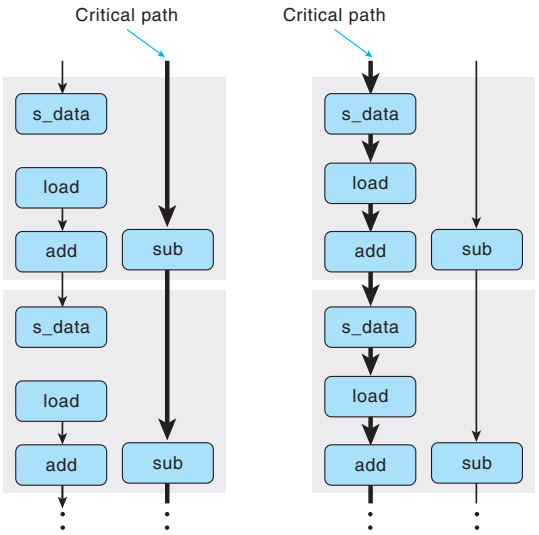


Figure 5.37
Data-flow representation of function `write_read`. When the two addresses do not match, the only critical path is formed by the decrementing of `cnt` (Example A). When they do match, the chain of data being stored, loaded, and incremented forms the critical path (Example B).

图 5.37

```
2    vmovss   -4(%rsi,%rax,4), %xmm0        取 p[i-1]
3    vaddss   (%rdi,%rax,4), %xmm0, %xmm0   加上 a[i]
4    vmovss   %xmm0, (%rsi,%rax,4)          存到 p[i]
5    addq     $1, %rax                      递增 i
6    cmpq     %rdx, %rax                    比较 i:cnt
7    jne      .L5                           若不相等,转到 loop
```

请做一次类似对 `combine3`（图 5.14）和 `write_read`（图 5.36）所做的分析：画出这个循环所创建的数据依赖，并由此画出计算进行过程中形成的关键路径。解释为什么 CPE 这么高。

练习 5.12　改写 `psum1`（图 5.1）的代码，使它不需要反复从存储器读取 `p[i]` 的值。不必使用循环展开。我们测得得到的代码 CPE 为 3.00，受限于浮点加法的延迟。

5.13 现实世界中的情况：性能改进技术

虽然我们只考虑了有限的一组应用，但我们能够就如何编写高效代码总结出一些重要的经验。我们已经介绍了优化程序性能的几种基本策略：

- **高层设计。** 针对手头的问题选择合适的算法和数据结构。要特别警惕避免那些会带来渐近低劣性能的算法或编码技术。
- **基本编码原则。** 避免优化阻碍因素，使编译器能够生成高效的代码。
- **消除过多的函数调用。** 只要可能，就把计算移出循环。为了获得更高的效率，可以考虑有选择地对程序的模块性做出折中。

- **消除不必要的存储器引用**。 引入临时变量来存放中间结果。只有在把最终值计算出来之后,才把它存入数组或全局变量。
- **低级优化**。 使代码结构能够充分发挥硬件的能力。
- **展开循环**。 以减少开销,并为进一步的优化创造条件。
- **设法提高指令级并行**。 可用的技术包括多累积变量和重新结合。
- **用函数式风格改写条件运算**。 使编译器能够通过条件数据传送的方式来编译这些运算。

给读者的最后一点忠告:在为了提高效率而改写程序时,要警惕不要引入错误。引入新变量、修改循环边界、让代码整体上更复杂时,都很容易出错。一种有用的方法是使用检查代码(checking code)来测试一个函数的各个优化版本,确保这个过程中没有引入缺陷。检查代码对一个函数的新版本应用一系列测试,确保它们能得到与原版本相同的结果。对高度优化的代码,测试用例的集合必须更加全面,因为需要考虑的情形更多。例如,使用循环展开的检查代码需要针对许多不同的循环边界进行测试,确保它能处理结尾处所需的各种单步迭代次数。

5.14 识别并消除性能瓶颈

到目前为止,我们只考虑了小程序的优化。对这类程序来说,程序中限制性能的位置通常是显而易见的,应当把优化精力集中在那里。而在处理大型程序时,就连确定把优化精力集中在何处都很困难。本节介绍如何使用剖析器(code profiler)这样一类分析工具,它们在程序执行过程中收集程序的性能数据。我们还会讨论代码优化的一些一般原则,包括 1.9.1 节引入的阿姆达尔定律的含义。

5.14.1 程序剖析

程序剖析(profiling)是指运行程序的一个版本,其中加入了插桩代码(instrumentation code),用来确定程序的各个部分各需要多少时间。它对于找出程序中值得投入优化精力的部分非常有用。剖析的一个优点是,可以一边在真实的基准数据上运行实际的程序,一边进行剖析。

Unix 系统提供一个剖析程序 `gprof`。这个程序生成两种形式的信息。第一,它确定程序中的每个函数各花费了多少 CPU 时间。第二,它统计每个函数被调用的次数,并按执行调用的函数来分类。这两种信息都相当有用。计时信息让我们能够了解各个函数在决定总体运行时间方面的相对重要性,调用信息则让我们能够理解程序的动态行为。

使用 gprof 进行剖析需要三个步骤。以 C 程序 prog.c 为例,它运行时带有命令行参数 file.txt:

- **第 1 步。** 程序必须为剖析而编译和链接。使用 gcc(和其他 C 编译器)时,只需在命令行上加上运行时标志 -pg。必须确保编译器不会试图通过内联替换进行任何优化,否则对函数的调用可能无法被准确地统计。我们使用优化标志 -Og,保证函数调用会被正确地跟踪。

```
linux> gcc -Og -pg prog.c -o prog
```

- **第 2 步。** 程序照常运行:

```
linux> ./prog file.txt
```

它的运行会稍慢一些(大约慢一倍),除此之外唯一的区别是会生成一个文件 gmon.out。

- **第 3 步。** 调用 gprof 分析 gmon.out 中的数据:

```
linux> gprof prog
```

剖析报告的第一部分列出执行各个函数所花费的时间,按降序排列。举个例子,下面的清单显示了某个程序中最耗时的三个函数对应的这部分报告:

```
 %    cumulative      self                 self        total
time    seconds      seconds    calls        s/call      s/call
 ↪  name
97.58     203.66      203.66        1        203.66      203.66
       sort_words
 2.32     208.50        4.85   965027          0.00        0.00
       find_ele_rec
 0.14     208.81        0.30 12511031          0.00        0.00
       Strlen
```

每一行代表对某个函数的全部调用所花费的时间。第一列给出该函数占总时间的百分比。第二列显示到本行(含本行)为止各个函数累计花费的时间。第三列显示花费在这个函数上的时间,第四列显示它被调用的次数(不计递归调用)。在我们的例子中,函数 sort_words 只被调用了一次,但这一次调用就花掉了 203.66 秒;而函数 find_ele_rec 被调用了 965,027 次(不包括递归调用),总共花掉 4.85 秒。函数 Strlen 通过调用库函数 strlen 来计算字符串的长度。库函数调用通常不会显示在 gprof 的结果中,它们的时间通常被计入调用它们的函数。通过创建一个“包装函数”Strlen,我们能够可靠地跟踪对 strlen 的调用,结果显示它被调用了 12,511,031 次,但总共只需要 0.30 秒。

剖析报告的第二部分显示各个函数的调用历史。下面是递归函数 find_ele_rec 的调用历史:

```
                158655725             find_ele_rec [5]
 4.85      0.10    965027/965027         insert_string [4]
[5]         2.4      4.85      0.10    965027+158655725 find_ele_rec
     [5]
                0.08      0.01    363039/363039      save_string
                ↪  [8]
                0.00      0.01    363039/363039      new_ele [12]
                158655725             find_ele_rec [5]
```

这段历史既显示了调用 `find_ele_rec` 的函数，也显示了它调用的函数。前两行显示对这个函数的调用：它自己递归调用了 158,655,725 次，函数 `insert_string`（它本身也被调用了 965,027 次）调用了 965,027 次。而 `find_ele_rec` 依次又调用了另外两个函数 `save_string` 和 `new_ele`，每个函数总共被它调用了 363,039 次。

从这些调用数据出发，我们常常可以推断出关于程序行为的有用信息。例如，函数 `find_ele_rec` 是一个递归过程，它沿着哈希桶的链表扫描，寻找某个特定的字符串。对这个函数来说，把递归调用的次数与顶层调用的次数做比较，能够给出关于沿这些链表进行的遍历的长度的统计信息。已知两者的比为 164.4:1，我们可以推断出程序每次平均扫描大约 164 个元素。

`gprof` 的一些特性值得注意：

- **计时的精度不高。** 它基于一种简单的间隔计数方案：编译后的程序为每个函数维护一个计数器，记录执行该函数所花费的时间。操作系统使程序以某个固定的时间间隔 δ 被中断。δ 的典型取值在 1.0 ~ 10.0 毫秒之间。当发生中断时，它会确定程序正在执行哪个函数，并把该函数的计数器加上 δ。当然，可能出现这个函数刚开始执行、很快就会完成的情况，但它却被记上了自上次中断以来的全部执行代价。而某些其他函数可能在两次中断之间运行，因而完全没有被计入任何时间。在较长的运行中，这一方案的效果相当不错。从统计上讲，每个函数都应当按照执行它所占用时间的相对比例被计时。但是对于运行时间不到约 1 秒的程序，这些数字只能被看成粗略的估计。
- **调用信息相当可靠。** 这里假设没有执行内联替代。编译后的程序为调用者和被调用者的每种组合各维护一个计数器，每当发生一次过程调用，就递增相应的计数器。
- **默认不显示库函数的计时。** 这些时间反而被并入调用它们的函数的计时中。

5.14.2 用剖析器指导优化

为了示范如何用剖析器指导程序优化，我们创建了一个包含若干不同任务和数据结构的应用程序。这个应用程序分析文本文档的 n 元词组（n-gram）统计信息，n 元词组就是文档中连续出现的 n 个词构成的一个序列。对 $n = 1$，我们统计单个词的信息；$n = 2$ 统计相邻词对，以此类推。对一个给定的 n 值，我们的程序读入一个文本文件，创建一个由不重复的 n 元词组及其出现次数组成的表，然后按出现次数对 n 元词组降序排序。

作为基准，我们在一个由莎士比亚全部作品组成的文件上运行了这个程序，共有 965,028 个词，其中 23,706 个是不重复的。我们发现，当 $n = 1$ 时，即便编写得很糟糕的分析程序也能在 1 秒内轻松处理完整个文件，因此我们取 $n = 2$ 来增加挑战性。$n = 2$ 时的 n 元词组称为二元词组（bigram，读音为“bye-grams”）。我们查明莎士比亚的著作包含 363,039 个不重复的二元词组。最常见的是“I am”，出现 1,892 次。他最著名的二元词组“to be”出现 1,020 次。足有 266,018 个二元词组只出现过一次。

我们的程序由以下几个部分组成。我们创建了多个版本，先为各个部分使用简单的算法，然后再用更成熟的算法替换它们：

1. 从文件中读入每个词，并转换成小写。我们的最初版本使用函数 `lower1`（图 5.7），我们知道它由于反复调用 `strlen` 而具有二次复杂度的运行时间。
2. 对字符串应用一个哈希函数，产生一个 0 到 $s-1$ 之间的数，这里 s 是哈希表的桶数。我们最初的函数只是简单地把各字符的 ASCII 代码求和，再对 s 取模。
3. 每个哈希桶组织成一个链表。程序沿着这个链表扫描，寻找匹配的表项。如果找到，就把这个 n 元词组的频度加 1。否则就创建一个新的表项。我们最初的版本用递归方式执行这一操作，把新元素插到链表的末尾。
4. 表生成之后，我们按频度对所有元素排序。我们最初的版本使用插入排序。

图 5.38 给出了我们的 n 元词组频度分析程序六个版本的剖析结果。对每个版本，我们把时间分成如下几类：

- **Sort**。 按频度对 n 元词组排序。
- **List**。 扫描链表寻找匹配的 n 元词组，必要时插入一个新元素。
- **Lower**。 把字符串转换成小写。
- **Strlen**。 计算字符串长度。

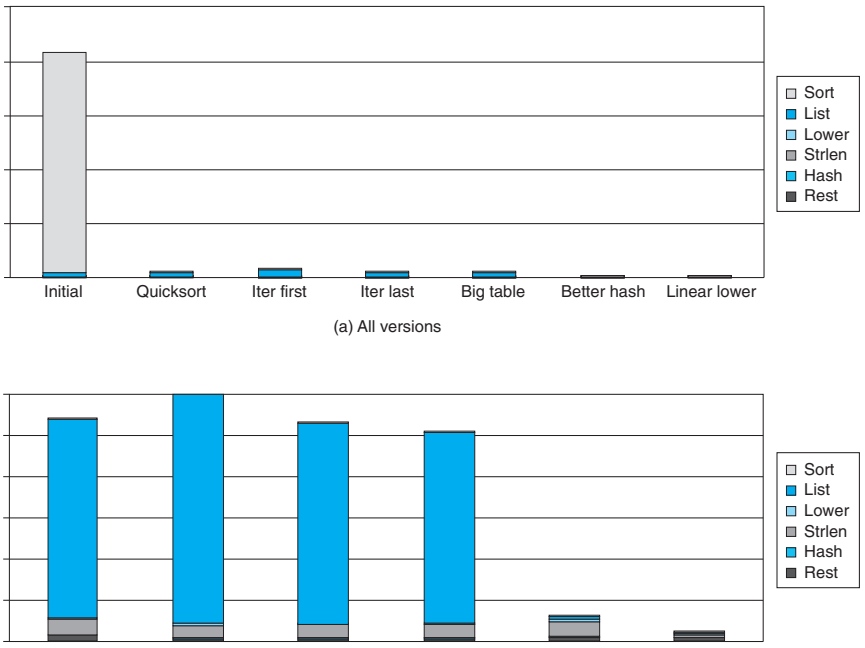


图 5.38 二元组(bigram)频率计数程序不同版本的性能剖析结果。时间按程序中不同的主要操作来划分。

- **Hash**。 计算哈希函数。
- **Rest**。 所有其他函数的总和。

如图中 (a) 部分所示,我们的最初版本需要 3.5 分钟,大部分时间花在排序上。这一结果并不奇怪,因为插入排序具有二次复杂度的运行时间,而程序要对 363,039 个值排序。

在下一个版本中,我们使用库函数 `qsort` 进行排序,它以快速排序算法 [98] 为基础,期望运行时间为 $O(n \log n)$。这个版本在图中标记为“Quicksort”。更高效的排序算法使排序所花费的时间变得可以忽略不计,整体运行时间降到 5.4 秒左右。图 (b) 部分在一个能把这些时间看得更清楚的标度上显示了其余版本的时间。

改进排序之后,我们发现链表扫描成了瓶颈。考虑到低效来自函数的递归结构,我们用一个迭代版本替换了它,在图中标记为“Iter first”。出人意料的是,运行时间反而增加到 7.5 秒左右。仔细研究之后,我们发现这两个链表操作函数之间有一个微妙的差别:递归版本把新元素插到链表的末尾,而迭代版本把它们插到链表的开头。为了获得最高的性能,我们希望出现频度最高的 n 元词组位于链表的开头附近,这样函数就能快速定位常见情形。假设 n 元词组在文档中均匀分布,我们可以预期一个高频词组的首次出现要早于一个更低频词组的首次出现。把新的 n 元词组插到末尾,第一个函数会让各个 n 元词组大致按

频度降序排列，而第二个函数则恰恰相反。因此，我们又创建了一个链表操作函数，它使用迭代，但把新元素插到链表的末尾。在这个标记为“Iter last”的版本中，时间降到 5.3 秒左右，略优于递归版本。这些测量结果证明了在优化过程中对程序进行实际实验的重要性。我们最初假定把递归代码改成迭代代码会提高其性能，没有考虑加到链表末尾与加到开头之间的差别。

接下来我们考虑哈希表的结构。最初的版本只有 1,021 个桶（通常，桶数取一个质数，以提高哈希函数把关键字均匀分布到各桶中的能力）。对一个有 363,039 个表项的表来说，这意味着平均负载为 $363039/1021 = 355.6$。这就解释了为什么这么多时间花在链表操作上——每次搜索都要测试相当多的候选 n 元词组。这也解释了为什么性能对链表的顺序如此敏感。然后我们把桶数增加到 199,999，把平均负载降到 1.8。但奇怪的是，整体运行时间只降到 5.1 秒，差别只有 0.2 秒。

进一步考察可以看到，更大的表只带来很小的性能提升，原因是哈希函数选得不好。简单地把字符串的字符代码求和，产生的取值范围并不宽。特别是一个字母的最大代码值为 122，因此一个 n 字符的字符串产生的和最多是 $122n$。我们文档中最长的二元词组 “honorificabilitudinitatibus thou” 的和只有 3,371，所以哈希表中的大部分桶都不会被用到。此外，像加法这样的可交换哈希函数，无法区分字符串中字符的不同排列。例如，“rat” 和 “tar” 会产生相同的和。

我们换成了使用移位和异或运算的哈希函数。在这个标记为“Better hash”的版本中，时间降到 0.6 秒。更系统的方法是更仔细地研究关键字在各桶之间的分布，确认它接近哈希函数输出为均匀分布时的期望值。

最后，我们把运行时间降到了这样的程度：大部分时间花在 `strlen` 上，而大多数对 `strlen` 的调用发生在小写转换的过程中。我们已经看到函数 `lower1` 具有二次复杂度的性能，对长字符串尤其如此。这个文档中的词足够短，避开了二次性能带来的灾难性后果；最长的二元词组也不过 32 个字符。尽管如此，切换到 `lower2`（图中标记为“Linear lower”）仍然带来了明显的改进，整体时间降到 0.2 秒左右。

通过这个练习，我们展示了代码剖析能够帮助把一个简单应用程序所需的时间从 3.5 分钟降到 0.2 秒，获得大约 1,000 倍的性能提升。剖析器帮助我们集中关注程序中最耗时的部分，还提供关于过程调用结构的有用信息。我们代码中的一些瓶颈，比如使用二次复杂度的排序例程，是可以预先估到的；而另一些，比如把元素追加到链表的开头还是末尾，只有经过仔细的分析才会显现出来。

可以看到，剖析是工具箱中的一个有用工具，但不应该是唯一的工具。计时测量并不完美，对较短（小于 1 秒）的运行时间尤其如此。更重要的是，结果只适用于测试所用到的特定数据。例如，如果我们让最初的函数处理由数量较少但更长的字符串组成的数据，我们会发现小写转换例程才是主要的性能瓶颈。更糟的是，如果我们只剖析短词的文档，就可能永远发现不了 `lower1` 二次性能这样的隐藏瓶颈。一般而言，假设我们在有代表性的数据上运行程序，剖析能帮助我们针对典型情形进行优化，但我们也应该确保程序对所有可能的情形都有说得过去的性能。这主要意味着要避免采用会带来渐近低劣性能的算法（比如插入排序）和编程习惯（比如 `lower1`）。

1.9.1 节描述的阿姆达尔定律，为有针对性的优化所能带来的性能提升提供了一些额外的洞见。对我们的 n 元词组代码来说，把插入排序换成快速排序之后，总执行时间从 209.0 秒降到 5.4 秒。最初版本 209.0 秒中的 203.7 秒花在插入排序上，给出 $\alpha = 0.974$，即受提速影响的那部分时间。使用快速排序后，排序时间变得微不足道，预测提速比为 $209/\alpha = 39.0$，接近实测的 38.5。我们能获得很大的提速，是因为排序在总体执行时间中占了非常大的比例。但是一个瓶颈被消除之后，新的瓶颈又会出现，因此要获得进一步的提速，就必须把注意力集中到程序的其他部分。

5.15 小结

大多数关于代码优化的论述讲的都是编译器如何生成高效代码，但应用程序程序员自己其实可以做很多事情来帮助编译器完成任务。没有一个编译器能把低劣的算法或数据结构换成好的，所以程序设计的这几个方面应当始终是程序员首要考虑的问题。我们还看到，存储器别名引用和过程调用这类优化阻碍因素，会严重限制编译器进行充分优化的能力。同样，程序员必须承担消除这些阻碍因素的主要责任。应当把它们视为良好编程习惯的一部分，因为它们能消除不必要的工作。

超出基本水平的性能调优需要对处理器的微体系结构有一定的理解，也就是描述处理器实现其指令集体系结构的基本机制。对乱序处理器来说，仅仅了解功能单元的操作、能力、延迟和发射时间，就能建立起预测程序性能的基准。

我们研究了一系列技术——包括循环展开、创建多个累积变量和重新结合——这些技术能够利用现代处理器提供的指令级并行。随着优化工作不断深入，研究生成的汇编代码并理解机器如何执行计算就变得重要起来。找出由程序中的数据依赖确定的关键路径会带来很大收益，尤其是循环各次迭代之间的

关键路径。我们还可以根据必须完成的操作的个数以及执行这些操作的单元的个数和发射时间,为某个计算算出吞吐量界限。

涉及条件分支或与存储器系统复杂交互的程序,比我们最初考虑的简单循环程序更难分析和优化。基本策略是设法让分支更容易预测,或者使它们能够用条件数据传送实现。我们还必须留意存储操作与加载操作之间的相互作用。把值保存在局部变量中、使它们能够存放到寄存器里,通常会有帮助。

处理大型程序时,把优化精力集中在最耗时的部分就变得非常重要。代码剖析器及相关工具能帮助我们系统地评估和改善程序性能。我们介绍了`gprof`,这是一个标准的 Unix 剖析工具。还有一些更精巧的剖析器可用,例如 Intel 的 `vtune` 程序开发系统,以及 Linux 系统上普遍提供的 `valgrind`。这些工具能把执行时间的分解做到过程级以下,估计程序中每个基本块(basic block)的性能。(基本块是一条指令序列,中间没有控制转移,因此整个块总是被完整地执行。)

5.16 参考文献说明

我们的重点是从程序员的角度描述代码优化,展示如何编写能让编译器更容易生成高效代码的程序。Chellappa、Franchetti 和 Püschel [19] 的一篇长文采用了类似的方法,但对处理器的特性讨论得更详细。

许多文献从编译器的角度描述代码优化,提出让编译器生成更高效代码的方法。Muchnick 的书被公认为最全面的 [80]。Wadleigh 与 Crawford 的软件优化著作 [115] 涵盖了我们介绍的部分内容,同时也描述了在并行机器上获得高性能的过程。Mahlke 等人 [75] 的一篇早期论文描述了为把程序映射到并行机器的编译器开发的一些技术,如何被改造为利用现代处理器的指令级并行。这篇论文涵盖了我们介绍的代码变换,包括循环展开、多累积变量(他们称为"累积变量扩展",accumulator variable expansion)以及重新结合(他们称为"树高削减",tree height reduction)。

我们对乱序处理器工作原理的介绍相当简略而抽象。关于一般原则的更完整的描述,可以在高级计算机体系结构的教科书中找到,例如 Hennessy 与 Patterson 的著作 [46,第 2 ~ 3 章]。Shen 与 Lipasti 的著作 [100] 对现代处理器设计做了深入的论述。

5.17 家庭作业

家庭作业 5.13 ▲▲ 假设我们要编写一个计算两个向量 u 与 v 的内积的过程。这个函数的一个抽象版本,对不同类型的整数和浮点数据,在 x86-64 上

的 CPE 为 14 ~ 18。通过进行与我们把抽象程序 combine1 改造成更高效的 combine4 时相同类型的变换,我们得到下面的代码:

```
 1  /* 求内积。在临时变量中累积 */
 2  void inner4(vec_ptr u, vec_ptr v, data_t *dest)
 3  {
 4      long i;
 5      long length = vec_length(u);
 6      data_t *udata = get_vec_start(u);
 7      data_t *vdata = get_vec_start(v);
 8      data_t sum = (data_t) 0;
 9
10      for (i = 0; i < length; i++) {
11          sum = sum + udata[i] * vdata[i];
12      }
13      *dest = sum;
14  }
```

我们的测量结果表明,这个函数在整数数据上 CPE 为 1.50,在浮点数据上 CPE 为 3.00。对数据类型 double,其内循环的 x86-64 汇编代码如下:

```
inner4 的内循环。data_t = double,OP = *
udata 在 %rbp 中,vdata 在 %rax 中,sum 在 %xmm0 中
i 在 %rcx 中,limit 在 %rbx 中
 1  .L15:                                       loop:
 2    vmovsd  0(%rbp,%rcx,8), %xmm1             取 udata[i]
 3    vmulsd  (%rax,%rcx,8), %xmm1, %xmm1       乘以 vdata[i]
 4    vaddsd  %xmm1, %xmm0, %xmm0               加到 sum 上
 5    addq    $1, %rcx                          递增 i
 6    cmpq    %rbx, %rcx                        比较 i:limit
 7    jne     .L15                              如果 !=,跳转到 loop
```

假设功能单元具有图 5.12 中列出的特性。

A. 仿照图 5.13 和图 5.14 的风格,画出这个指令序列会被译码成哪些操作,并说明这些操作之间的数据依赖如何形成一条关键路径。

B. 对数据类型 double,这条关键路径给 CPE 确定的下界是多少?

C. 假设整数代码也有类似的指令序列,那么对整数数据而言,关键路径给 CPE 确定的下界是多少?

D. 解释为什么尽管乘法运算需要 5 个时钟周期,各个浮点版本的 CPE 却可以达到 3.00。

家庭作业 5.14▲ 编写家庭作业 5.13 中所描述的内积过程的一个使用 6×1 循环展开的版本。对 x86-64,我们测得展开版本在整数数据上的 CPE 为 1.07,但两种浮点数据仍然都是 3.01。

A. 解释为什么在 Intel Core i7 Haswell 处理器上运行的内积过程的任何(标量)版本,其 CPE 都不可能低于 1.00。

B. 解释为什么浮点数据的性能没有因为循环展开而提高。

家庭作业 5.15 ▲ 编写家庭作业 5.13 中所描述的内积过程的一个使用 6×6 循环展开的版本。我们对这个函数在 x86-64 上的测量给出：整数数据的 CPE 为 1.06，浮点数据的 CPE 为 1.01。

是什么因素把性能限制在 CPE 1.00?

家庭作业 5.16 ▲ 编写家庭作业 5.13 中所描述的内积过程的一个使用 $6 \times 1a$ 循环展开的版本，以获得更高的并行性。我们对该函数的测量给出：整数数据的 CPE 为 1.10，浮点数据的 CPE 为 1.05。

家庭作业 5.17 ▲▲ 库函数 `memset` 具有如下原型：

```
void *memset(void *s, int c, size_t n);
```

这个函数用 c 的最低位字节的副本填充从 s 开始的存储器区域中的 n 个字节。例如，给 c 传入 0，就可以用它把一段存储器区域清零，当然其他值也是可以的。

下面是 `memset` 的一个直接实现：

```
1  /* memset 的基本实现 */
2  void *basic_memset(void *s, int c, size_t n)
3  {
4      size_t cnt = 0;
5      unsigned char *schar = s;
6      while (cnt < n) {
7          *schar++ = (unsigned char) c;
8          cnt++;
9      }
10     return s;
11 }
```

请用一个 `unsigned long` 类型数据的字打包 c 的 8 份副本，然后以字级写的方式遍历这个存储器区域，实现该函数的一个更高效率的版本。你还会发现，再做额外的循环展开可能也有帮助。在我们的参考机器上，我们把 CPE 从直接实现的 1.00 降到了 0.127。也就是说，这个程序每个时钟周期可以写 8 个字节。下面是一些额外的指导原则。为了保证可移植性，令 K 表示你运行本程序的机器上 `sizeof(unsigned long)` 的值。

- **不允许调用任何库函数**。
- **代码应当能处理 n 的任意取值**。 包括 n 不是 K 的倍数的情形。可以采用类似于我们用循环展开处理结尾几次迭代的方式来做到这一点。
- **代码应当能编译并在任何机器上正确运行，而不依赖于 K 的取值**。 为了做到这一点，请利用 `sizeof` 运算。

- **在某些机器上，非对齐的写可能比对齐的写慢得多。（在某些非 x86 的机器上，甚至会引发段错误。）** 请在编写代码时先进行字节级写，直到目的地址变成 K 的倍数，然后进行字级写，最后（如有必要）再以字节级写结束。
- **要警惕 cnt 小到使某些循环的上界变成负数的情形。** 对涉及 `sizeof` 运算符的表达式，测试可能是用无符号算术执行的。（见 2.2.8 节和家庭作业 2.72。）

家庭作业 5.18 ▲▲▲ 在练习 5.5 和练习 5.6 中，我们讨论过多项式求值任务，既做过直接求值，也做过用霍纳方法求值。试着用我们研究过的各种优化技术，写出这个函数的更快版本，包括循环展开、并行累积和重新结合。你会发现，把霍纳方法和直接求值与这些优化技术混合起来，有许多种不同的组合方式。

理想情况下，你应当能够把 CPE 做到接近你的机器的吞吐量界限。我们最好的版本在我们的参考机器上达到了 CPE 1.07。

家庭作业 5.19 ▲▲▲ 在练习 5.12 中，我们把前缀和计算的 CPE 降到了 3.00，受限于这台机器上浮点加法的延迟。简单循环展开无助于事。

把循环展开和重新结合结合起来，写出一个前缀和代码，使它的 CPE 低于你的机器上浮点加法的延迟。这实际上要求增加执行的加法次数。例如，我们的 2 路展开版本每次迭代需要 3 次加法，而我们的 4 路展开版本需要 5 次。我们最好的实现在我们的参考机器上达到了 CPE 1.67。

确定你的机器的吞吐量界限和延迟界限如何限制了前缀和运算所能达到的最小 CPE。

5.18 练习题答案

答案 5.1 这道题演示了存储器别名引用带来的某些微妙影响。

如下面带注释的代码所示，其效果是把 xp 所指的值设置为 0：

```
4       *xp = *xp + *xp;   /* 2x */
5       *xp = *xp - *xp;   /* 2x-2x = 0 */
6       *xp = *xp - *xp;   /* 0-0 = 0 */
```

这个例子说明，我们对程序行为的直觉常常是错误的。我们自然会想到 xp 与 yp 不同的情况，却忽略了它们可能相等。程序缺陷常常产生于程序员没有预料到的条件。

答案 5.2 这道题考察 CPE 与绝对性能之间的关系。用初等代数就能求解。我们得到：$n \leq 2$ 时版本 1 最快；$3 \leq n \leq 7$ 时版本 2 最快；$n \geq 8$ 时版本 3 最快。

答案 5.3 这是一道简单的练习，但重要的是要认识到，一个 for 循环的四条语句——初始化、测试、更新和循环体——被执行的次数各不相同。

代码	min	max	incr	square
A.	1	91	90	90
B.	91	1	90	90
C.	1	1	90	90

答案 5.4 这段汇编代码演示了 `gcc` 检测到的一个聪明的优化机会。值得仔细研究这段代码，以便更好地理解代码优化的微妙之处。

A. 在优化程度较低的代码中，寄存器`%xmm0` 只是简单地用作临时值，每次循环迭代中既要设置又要使用。而在优化程度较高的代码中，它以更接近 `combine4` 中变量 `acc` 的方式使用，累积向量各元素的乘积。与 `combine4` 不同的是，位置 $dest$ 会在每次迭代中被第二条 `vmovsd` 指令更新。

可以看到，这个优化版本的操作方式与下面的 C 代码非常相似：

```
1  /* 确保 dest 在每次迭代中都被更新 */
2  void combine3w(vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(v);
6      data_t *data = get_vec_start(v);
7      data_t acc = IDENT;
8
9      /* 初始化,以防 length <= 0 */
10     *dest = acc;
11
12     for (i = 0; i < length; i++) {
13         acc = acc OP data[i];
14         *dest = acc;
15     }
16 }
```

B. `combine3` 的两个版本具有完全相同的功能，即使出现存储器别名引用也是如此。

C. 可以在不改变程序行为的前提下完成这一变换，因为除第一次迭代外，每次迭代开始时从 $dest$ 读出的值，与上一次迭代结束时写入这个寄存器的值相同。因此，合并指令可以直接使用循环开始时`%xmm0` 中已有的值。

答案 5.5 多项式求值是解决许多问题的一种核心方法。例如，数学库中常用这些相关函数的多项式来逼近三角函数。

A. 该函数执行 $2n$ 次乘法和 n 次加法。

B. 可以看到，这里限制性能的计算是对表达式 `xpwr = x * xpwr` 的反复求值。这需要一次浮点乘法（5 个时钟周期），而且一次迭代的计算必须等上

一次迭代的计算完成之后才能开始。在相邻迭代之间,对 result 的更新只需要一次浮点加法(3 个时钟周期)。

答案 5.6 这道题表明,缩减计算所用运算的数目未必能提高其性能。

A. 该函数执行 n 次乘法和 n 次加法,乘法次数只是原函数 poly 的一半。

B. 可以看到, 这里限制性能的计算是对表达式 result = a[i] + x*result 的反复求值。以上一次迭代结束后 result 的值为起点, 必须先乘以 x(5 个时钟周期),再加上 a[i](3 个周期),才能得到本次迭代的值。因此每次迭代强加一个 8 个周期的最小延迟,恰好就是我们测得的 CPE。

C. 尽管函数 poly 的每次迭代需要两次乘法而不是一次,但每次迭代沿关键路径只出现一次乘法。

答案 5.7 下面的代码直接遵循我们给出的按因子 k 展开循环的规则:

```
1  void unroll5(vec_ptr v, data_t *dest)
2  {
3      long i;
4      long length = vec_length(v);
5      long limit = length-4;
6      data_t *data = get_vec_start(v);
7      data_t acc = IDENT;
8
9      /* 每次合并 5 个元素 */
10     for (i = 0; i < limit; i+=5) {
11         acc = acc OP data[i]    OP data[i+1];
12         acc = acc OP data[i+2] OP data[i+3];
13         acc = acc OP data[i+4];
14     }
15
16     /* 处理剩余的元素 */
17     for (; i < length; i++) {
18         acc = acc OP data[i];
19     }
20     *dest = acc;
21 }
```

答案 5.8 这道题展示了程序中的微小改动如何产生巨大的性能差异,特别是在一台采用乱序执行的机器上。图 5.39 给出了该函数单次迭代中三次乘法运算的示意。在这个图中,以蓝色方框表示的操作位于关键路径上——为了给循环变量 r 算出新值,它们必须按顺序依次计算。以浅色方框表示的操作可以与关键路径上的操作并行计算。沿着关键路径有 P 个操作的循环,每次迭代最少需要 $5P$ 个时钟周期,并计算三个元素的乘积,给出 CPE 的下界 $5P/3$。这意味着 A1 的下界为 5.00,A2 和 A5 为 3.33,A3 和 A4 为 1.67。我们在一台 Intel Core i7 Haswell 处理器上运行这些函数,发现它能够达到这些 CPE 值。

答案 5.9 这个例子再次说明,编码风格上的一个细微改动,就能让编译器更容易发现使用条件传送的机会:

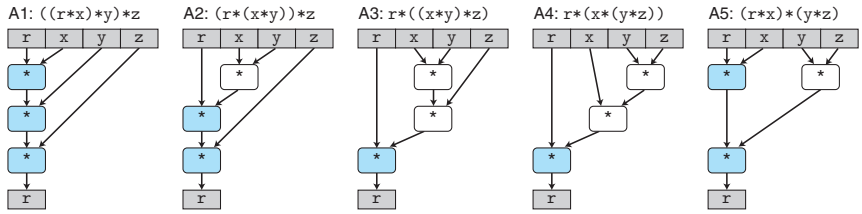


图 5.39 练习 5.8 中各情形下乘法运算之间的数据依赖。图中以蓝色方框表示的运算构成各次迭代的关键路径。

```
while (i1 < n && i2 < n) {
    long v1 = src1[i1];
    long v2 = src2[i2];
    long take1 = v1 < v2;
    dest[id++] = take1 ? v1 : v2;
    i1 += take1;
    i2 += (1-take1);
}
```

我们测得这个版本的 CPE 约为 12.0,比原来的 15.0 略有改进。

答案 5.10 这道题要求你分析程序中潜在的加载-存储相互作用。

A. 它会把每个元素 `a[i]` 设置为 $i+1$,其中 $0 \le i \le 998$。

B. 它会把每个元素 `a[i]` 设置为 0,其中 $1 \le i \le 999$。

C. 在第二种情形中,一次迭代的加载要依赖前一次迭代存储的结果。因此,相邻的迭代之间存在一个写/读依赖。

D. 它给出的 CPE 为 1.2,与示例 A 的情形相同,因为各次存储与其后的加载之间没有依赖关系。

答案 5.11 可以看到,这个函数在相邻迭代之间存在一个写/读依赖——一次迭代中的目的值 `p[i]` 与下一次迭代中的源值 `p[i-1]` 相同。因此,每次迭代都会形成一条由一次存储(来自前一次迭代)、一次加载和一次浮点加法组成的关键路径。CPE 测量值 9.0 与我们在 `write_read` 存在数据依赖时测得的 7.3 相一致,因为 `write_read` 涉及整数加法(1 个时钟周期的延迟),而 `psum1` 涉及浮点加法(3 个时钟周期的延迟)。

答案 5.12 下面是这个函数的一个修改版本:

```
 1  void psum1a(float a[], float p[], long n)
 2  {
 3      long i;
 4      /* last_val 保存 p[i-1];val 保存 p[i] */
 5      float last_val, val;
 6      last_val = p[0] = a[0];
 7      for (i = 1; i < n; i++) {
 8          val = last_val + a[i];
 9          p[i] = val;
10          last_val = val;
```

```
11      }
12  }
```

我们引入一个局部变量 last_val。在第 i 次迭代的开始,它保存 p[i-1] 的值。然后我们把 val 计算成 p[i] 的值,并让它成为 last_val 的新值。

这个版本编译成如下汇编代码:

```
psum1a 的内循环
a 在 %rdi 中,i 在 %rax 中,cnt 在 %rdx 中,last_val 在 %xmm0 中
 1  .L16:                                   loop:
 2    vaddss  (%rdi,%rax,4), %xmm0,
      %xmm0   last_val = val = last_val + a[i]
 3    vmovss  %xmm0, (%rsi,%rax,4)          把 val 存入 p[i]
 4    addq    $1, %rax                      递增 i
 5    cmpq    %rdx, %rax                    比较 i:cnt
 6    jne     .L16                          如果 !=,跳转到 loop
```

这段代码把 last_val 保持在寄存器%xmm0 中, 从而不必从存储器读取 p[i-1],消除了 psum1 中看到的写/读依赖。

6

存储器层次结构

在对系统的研究中，到本章为止，我们一直依赖一个简单的计算机系统模型：一个执行指令的 CPU，加上一个为 CPU 保存指令与数据的存储器系统。在这个简单模型中，存储器系统是一个由字节组成的线性数组，CPU 能够用常量的时间访问每个存储器位置。虽然这个模型在一定范围内行之有效，但它并没有反映出现代系统实际的工作方式。

实际上，存储器系统是一个由容量、成本和访问时间各不相同的存储设备组成的层次结构。CPU 寄存器保存着最经常使用的数据。靠近 CPU 的小而快速的高速缓存存储器，作为存放在相对较慢的主存中的一部分数据和指令的暂存区（staging area）。主存又暂存存放在较大较慢的磁盘上的数据，而这些磁盘通常又充当通过网络相连的其他机器上的磁盘或磁带中数据的暂存区。

存储器层次结构之所以行得通，是因为编写良好的程序倾向于更频繁地访问某个特定层次上的存储，较少访问下面一层较低层次上的存储。因此，下面一层的存储可以更慢一些，从而也更大、每位更便宜。总的效果就是一个大的存储池：它的价格相当于层次结构底部附近的廉价存储，而向程序提供数据的速率相当于层次结构顶部附近的快速存储。

作为程序员，你需要理解存储器层次结构，因为它对你应用程序的性能有重要影响。程序需要的数据如果存放在 CPU 寄存器里，在指令执行期间花 0 个周期就能访问到；如果存放在高速缓存里，需要 4 到 75 个周期；如果存放在主存里，需要几百个周期；而如果存放在磁盘上，就要几千万个周期！

由此得出计算机系统中的一个基本而持久的思想：如果你理解了系统是如何在存储器层次结构中上下搬运数据的，那么你就可以编写应用程序，让你的数据项存放在层次结构中较高的位置，在那里 CPU 能够更快地访问它们。

这一思想的核心是计算机程序的一种基本属性，称为局部性（locality）。具有良好局部性的程序倾向于反复访问相同的数据项，或者倾向于访问相邻的数据项。与局部性差的程序相比，具有良好局部性的程序倾向于从存储器层次结构的较高层次访问更多的数据项，因而运行得更快。例如，在我们的 Core i7 系

统上，执行相同数量算术运算但局部性程度不同的矩阵乘法内核，运行时间可以相差近 40 倍！

本章中，我们将考察基本的存储技术——SRAM 存储器、DRAM 存储器、ROM 存储器，以及旋转磁盘和固态硬盘——并描述它们是如何被组织成层次结构的。我们特别关注那些充当 CPU 与主存之间暂存区的高速缓存存储器，因为它们对应用程序性能的影响最大。我们将展示如何分析 C 程序的局部性，还会介绍改进程序局部性的一些技术。你还将学到一种刻画存储器层次结构在特定机器上性能的有趣方法：把读访问时间表示为局部性的函数，画成一座“存储器山”。

6.1 存储技术

计算机技术获得的很大成功源自存储技术的巨大进步。早期的计算机只有几千字节的随机访问存储器。最早的 IBM PC 甚至连硬盘都没有。1982 年 IBM PC-XT 的推出改变了这一点，它配备了一块 10 MB 的磁盘。到 2015 年，典型机器的磁盘存储量是那时的 300,000 倍，而且存储量每两年左右就翻一倍。

6.1.1 随机访问存储器

随机访问存储器（random access memory，RAM）分两大类——静态的和动态的。静态 RAM（static RAM，SRAM）比动态 RAM（dynamic RAM，DRAM）更快，也贵得多。SRAM 用来实现高速缓存存储器，既可以放在 CPU 芯片内，也可以放在其外。DRAM 用来实现主存，以及图形系统的帧缓冲区（frame buffer）。典型地，一台台式系统将拥有不超过几十 MB 的 SRAM，但是有几百乃至上千 MB 的 DRAM。

静态 RAM

SRAM 把每个位存储在一个双稳态（bistable）存储单元里。每个单元用一个六晶体管电路实现。这个电路的属性是，它可以无限期地保持在两种不同的电压配置（或状态）之一。任何其他的状态都是不稳定的——从那里出发，电路会迅速地向两个稳定状态之一移动。这样的存储单元类似图 6.1 所画的倒摆。

当摆完全向左倾斜或者完全向右倾斜时，它是稳定的。从任何其他位置出发，摆都会倒向一边或者另一边。原则上，摆也可以无限期地保持垂直平衡，但这个状态是亚稳的（metastable）——最小的扰动都会使它开始倾倒，一旦倒下，它就再也不会回到垂直位置。

由于具有双稳性，只要保持供电，SRAM 存储单元就会永远保留它的值。即使电噪声之类的扰动干扰了电压，在扰动消除时，电路也会恢复到稳定值。

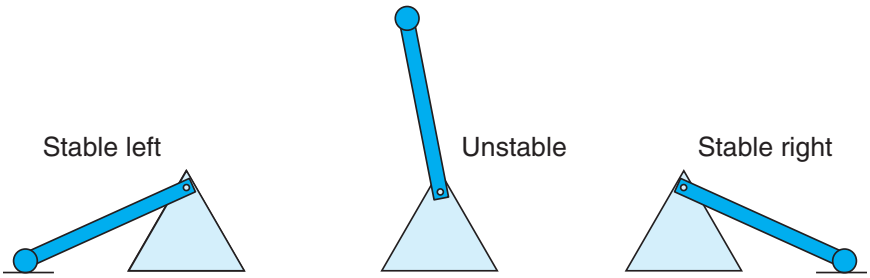


图 6.1 倒立摆。像 SRAM 单元一样，摆只有两种稳定的构型，也就是状态。

图 6.2 DRAM 与 SRAM 存储器的特性

	每位晶体管数	相对访问时间	是否持续	是否敏感	相对成本	应用
SRAM	6	1×	是	否	1,000×	高速缓存存储器
DRAM	1	10×	否	是	1×	主存、帧缓冲区

动态 RAM

DRAM 把每个位存储为电容器上的电荷。这个电容器非常小——典型值大约 30 飞法(femtofarad)，也就是 30×10^{-15} 法拉。不过要记得，法拉是一个很大的度量单位。DRAM 存储可以做得非常密集——每个单元由一个电容器和一个访问晶体管组成。但是与 SRAM 不同，DRAM 存储单元对任何扰动都非常敏感。电容器电压一旦被扰动，它就永远不会恢复了。暴露在光线下会引起电容器电压的变化。实际上，数码相机和摄像机中的传感器本质上就是 DRAM 单元的阵列。

各种漏电流的来源导致 DRAM 单元在大约 10 到 100 毫秒内失去电荷。幸运的是，对于时钟周期以纳秒来度量的计算机来说，这个保持时间是相当长的。存储器系统必须通过读出再重写的方式，周期性地刷新每一位。有的系统还使用纠错码(error-correcting code)，机器字用几个额外的位来编码(例如，一个 64 位的字可能用 72 位来编码)，这样电路就能检测并纠正一个字内任何单个的错误位。

图 6.2 总结了 SRAM 和 DRAM 存储器的特性。只要保持供电，SRAM 就是持续的。与 DRAM 不同，不需要对它刷新。SRAM 的访问速度比 DRAM 快。SRAM 对光和电噪声之类的扰动不敏感。付出的代价是 SRAM 单元使用的晶体管比 DRAM 单元多，因而密度更小、更贵、功耗更大。

常规 DRAM

DRAM 芯片中的单元(位)被划分成 d 个超级单元(supercell)，每个超级单元由 w 个 DRAM 单元组成。一个 $d \times w$ 的 DRAM 总共存储 dw 位信息。超

级单元被组织成一个有 r 行、c 列的矩形阵列，$rc = d$。每个超级单元都有形如 (i, j) 的地址，i 表示行，j 表示列。

例如，图 6.3 展示了一个 16×8 DRAM 芯片的组织，它有 $d = 16$ 个超级单元，每个超级单元 $w = 8$ 位，$r = 4$ 行，$c = 4$ 列。带阴影的方框表示地址为 $(2, 1)$ 的超级单元。信息经过称为引脚的外部连接器流出和流入芯片。每个引脚携带 1 位信号。图 6.3 给出了其中的两组引脚：8 根数据引脚，可以传出或传入一根芯片的 1 字节数据；还有 2 根 addr 引脚，携带 2 位的行和列超级单元地址。携带控制信息的其他引脚没有画出来。

边栏　关于术语的一点说明

对于 DRAM 阵列单元，存储器界从未确定过一个标准名称。计算机体系结构专家倾向于称之为“单元（cell）”，这就引起了与 DRAM 存储单元重名的争歧。电路设计专家倾向于称之为“字（word）”，这会与主存中字的原有含义相混淆。为避免混淆，我们采用了一个无歧义的术语“超级单元（supercell）”。

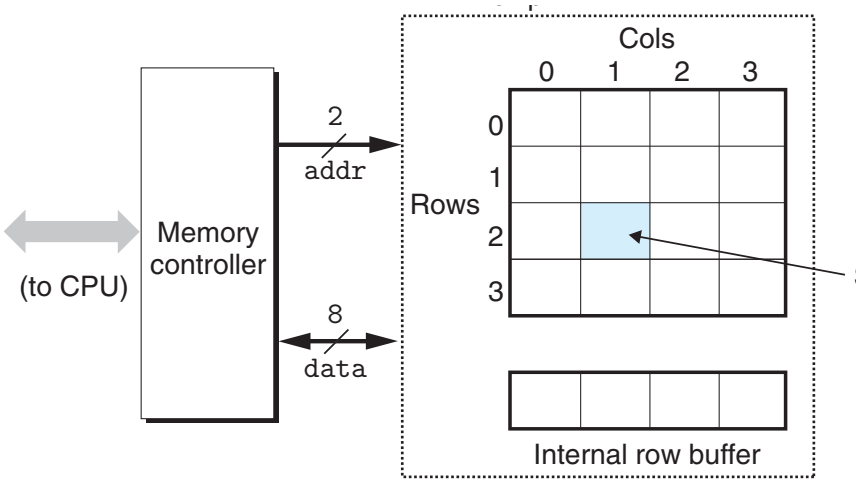


图 6.3　128 位 16×8 DRAM 芯片的高级视图。

每个 DRAM 芯片都连接到一些称为存储控制器（memory controller）的电路，它能一次传入或传出 w 位的数据。为读取超级单元 (i, j) 的内容，存储控制器向 DRAM 发送行地址 i，然后发送列地址 j。DRAM 会把超级单元 (i, j) 的内容送回给控制器作为响应。行地址 i 叫做行访问选通（row access strobe，RAS）请求，列地址 j 叫做列访问选通（column access strobe，CAS）请求。注意，RAS 和 CAS 请求共享相同的 DRAM 地址引脚。

例如，为了从图 6.3 的 16×8 DRAM 中读取超级单元 $(2, 1)$，存储控制器发送行地址 2，如图 6.4(a) 所示。DRAM 会把第 2 行的整体内容拷贝进一个内部

行缓冲区。接下来，如图 6.4(b) 所示，存储控制器发送列地址 1。DRAM 会从行缓冲区中拷贝出超级单元 (2, 1) 的 8 位，并发送给存储控制器。

电路设计者把 DRAM 组织成二维阵列而不是线性阵列的目的之一是减少芯片上的地址引脚数。例如，假设例子中的那个 128 位 DRAM 被组织成一个地址从 0 到 15、含 16 个超级单元的线性阵列，那么芯片将需要 4 根地址引脚而不是 2 根。二维阵列组织的不利之处是地址必须分两步发送，这增加了访问时间。

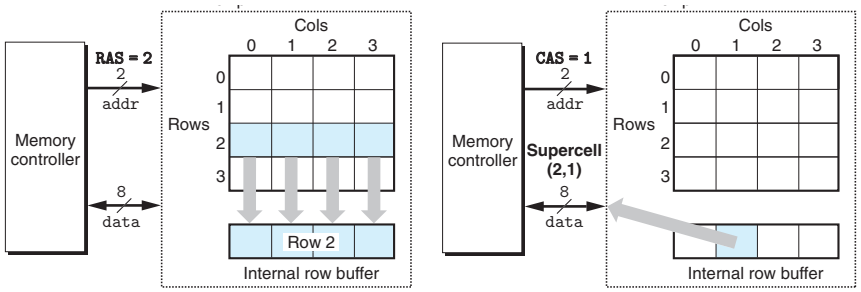


图 6.4　读 DRAM 超级单元(supercell)的内容。

存储器模块

DRAM 芯片被封装在存储器模块（memory module）里，插在主系统板（主板）上的扩展槽。Core i7 系统使用 240 引脚的双内联存储器模块（DIMM），它以 64 位大小的数据为单位与存储控制器往来传输。

图 6.5 展示了存储器模块的基本思路。示例模块使用 8 个 64 Mbit 的 8M×8 DRAM 芯片（编号为 0 ~ 7），总共存储 64 MB。每个超级单元存储主存的 1 字节；主存中位于字节地址 A 的每个 64 位字由对应超级单元地址为 (i, j) 的 8 个超级单元表示。在图 6.5 的示例中，DRAM 0 存储第一个（低序）字节，DRAM 1 存储下一个字节，以此类推。

为了取出存储器地址 A 处的字，存储控制器把 A 转换成一个超级单元地址 (i, j) 发送给存储器模块。模块再把 i 和 j 广播给每个 DRAM。作为响应，每个 DRAM 输出它的 (i, j) 超级单元的 8 位内容。模块中的电路收集这些输出，把它们组合成一个 64 位字，然后返回给存储控制器。

主存可以通过把多个存储器模块连接到存储控制器上来扩大容量。这时，当控制器接收到一个地址 A 时，控制器选出包含 A 的模块 k，把 A 转换成 (i, j) 的形式，并把 (i, j) 发送给模块 k。

练习 6.1　在下面的题中，令 r 表示 DRAM 阵列的行数，c 表示列数，b_r 表示行寻址需要的位数，b_c 表示列寻址需要的位数。对下表中的每个 DRAM，确定使 $\max(b_r, b_c)$ 最小的、2 的幂次大小的阵列尺寸。

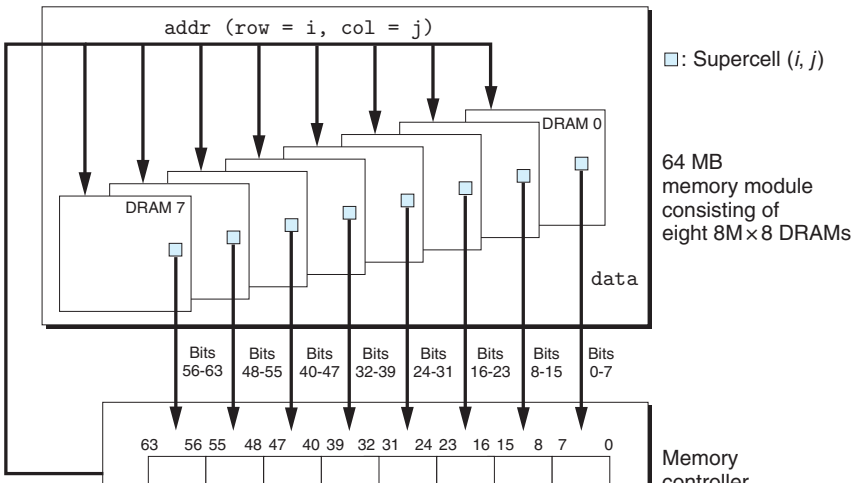


图 6.5 读存储器模块的内容。

组织	r	c	b_r	b_c	$\max(b_r, b_c)$
16×1					
16×4					
128×8					
512×4					
$1{,}024 \times 4$					

增强型 DRAM

市场上有许多种 DRAM 存储器，随着制造商想赶上不断提升的处理器速度，新的种类还会定期上市。每一种都建立在常规的 DRAM 单元之上，配上一些优化，用来提高对基本 DRAM 单元的访问速度。

- **快速页模式 DRAM（fast page mode DRAM，FPM DRAM）**。常规 DRAM 会把一整行超级单元拷贝进内部行缓冲区，用掉其中一个，然后丢弃其余的。FPM DRAM 对此有所改进，允许对同一行的连续访问直接从行缓冲区获得服务。比如，要从常规 DRAM 的第 i 行读出四个超级单元，存储控制器必须发送四个 RAS/CAS 请求，即使每次的行地址 i 都相同。而要从 FPM DRAM 的同一行读出超级单元，存储控制器发送一个初次 RAS/CAS 请求，后面跟三个 CAS 请求。初始的 RAS/CAS 请求把行 i 拷贝进行缓冲区，并返回由首个 CAS 寻址的超级单元。其余三个超级单元由行缓冲区直接提供服务，因而比初始的超级单元返回得更快。
- **扩展数据输出 DRAM（extended data out DRAM，EDO DRAM）**。一种增强形式的 FPM DRAM，它允许相邻的各个 CAS 信号在时间上靠得更近。

- **同步 DRAM（synchronous DRAM，SDRAM）**。常规的、FPM 和 EDO DRAM 都是异步的，这里异步的含义是，它们用一组显式的控制信号与存储控制器通信。SDRAM 用驱动存储控制器的同一个外部时钟信号的上升沿替代了其中许多控制信号。总的效果是，SDRAM 输出其超级单元内容的速率比异步的同类产品更快。
- **双倍数据速率同步 DRAM（double data-rate synchronous DRAM，DDR SDRAM）**。DDR SDRAM 是对 SDRAM 的增强，它通过使用时钟的两个边沿作为控制信号，把 DRAM 的速度翻了一倍。不同类型的 DDR SDRAM 由一个小预取缓冲区（prefetch buffer）的大小来区分，预取缓冲区能够增大有效带宽：DDR（2 位）、DDR2（4 位）、DDR3（8 位）。
- **视频 RAM（video RAM，VRAM）**。用在图形系统的帧缓冲区中。VRAM 的思路与 FPM DRAM 相似。两个主要差别是：(1) VRAM 的输出是通过顺序移出内部缓冲区的整体内容来产生的；(2) VRAM 允许对存储器并发地读和写。这样，系统就可以一边用帧缓冲区中的像素绘制屏幕（读），一边并发地为下一次更新写入新值（写）。

非易失性存储器

DRAM 和 SRAM 都是易失的（volatile），即一旦供电电压被切断，它们就会丢失信息。非易失性存储器（nonvolatile memory）则相反，即使在断供电源时，它们仍然保留信息。非易失性存储器有很多种。由于历史原因，虽然有些类型的 ROM 既可读也可写，它们仍被统称为只读存储器（read-only memory，ROM）。ROM 的分类依据是可重新编程（写入）的次数和重新编程的方式。

> **边栏　DRAM 技术的历史**
> 1995 年以前，大多数 PC 都是用 FPM DRAM 构造的。1996 年到 1999 年，EDO DRAM 占据了市场，同时 FPM DRAM 几乎消失了。SDRAM 最早出现在 1995 年的高端系统中，到 2002 年大多数 PC 都是用 SDRAM 和 DDR SDRAM 构造的。到 2010 年，大多数服务器和桌面系统都是用 DDR3 SDRAM 构造的。实际上，Intel Core i7 也只支持 DDR3 SDRAM。

可编程 ROM（programmable ROM，PROM）恰好只能被编程一次。PROM 在每个存储单元中都包含了一种熔丝，可以用大电流一次性地把它烧断，从而把“1”写成“0”。

可擦可编程 ROM（erasable programmable ROM，EPROM）有一个透明的石英窗口，允许光线到达存储单元。通过窗口照射紫外线可以把 EPROM 单元清除成全 0。EPROM 的编程由一种特殊的设备完成，它把 1 写入 EPROM 中的位单元。EPROM 能够被擦除并重新编程大约 1,000 次。电可擦可编程 PROM

(electrically erasable PROM, EEPROM)与 EPROM 相似,但它不需要单独的编程设备,因而能够在印刷电路卡上就地重新编程。EEPROM 能够被重新编程大约 10^5 次后才会磨损。

闪存(flash memory)是一类基于 EEPROM 的非易失性存储器,它已经成为一类重要的存储技术。闪存无处不在:它为大量的电子设备(包括数码相机、手机、音乐播放器,还有笔记本电脑、台式机和服务器系统)提供快速而耐用的非易失性存储器。在 6.1.3 节中,我们将详细讲述一种新式的基于闪存的称为固态硬盘(solid state disk, SSD)的磁盘。它比传统的旋转磁盘更快、更结实耐用,也更省电。

存储在 ROM 设备中的程序通常被称为固件(firmware)。当计算机系统加电后,它会运行保存在 ROM 中的固件。有些系统在固件中提供一组基本的输入和输出函数——例如 PC 的 BIOS(基本输入/输出系统)例程。一些复杂的设备(例如图形卡和磁盘控制器)也依靠固件来翻译来自 CPU 的 I/O(输入/输出)请求。

访问主存

数据在处理器与 DRAM 主存之间来回流动,流经的都是称为总线(bus)的共享电气通路。每一次 CPU 与存储器之间的数据传输都由一系列步骤完成,称为总线事务(bus transaction)。读事务将数据从主存传送到 CPU;写事务将数据从 CPU 传送到主存。

总线是一组并行导线,它能携带地址信号、数据信号和控制信号。根据具体的总线设计不同,数据信号和地址信号可以共享同一组导线,也可以使用分开的导线组。还有,超过两个设备也可以共享同一根总线。控制导线携带同步事务的信号,并且识别当前正在执行的事务的类型。比如,这个事务是针对主存,还是针对磁盘控制器这样的其他 I/O 设备?这个事务是读还是写?总线上的信息是地址还是数据项?

图 6.6 显示了一个示例计算机系统的配置。主要部件是 CPU 芯片、一个称为 I/O 桥的芯片集(其中包含存储控制器),以及构成主存的 DRAM 存储器模块。这些部件由一对总线连接:用于连接 CPU 与 I/O 桥的系统总线,和用于连接 I/O 桥与主存的内存总线。I/O 桥把系统总线上的电信号翻译成内存总线上的电信号。我们将会看到,I/O 桥也把系统总线和内存总线连接到一根由磁盘和图形卡这样的 I/O 设备共享的 I/O 总线上。不过,目前我们只关注内存总线。

边栏 关于总线设计的说明

总线设计是计算机系统中一个复杂且快速变化的方面。不同的厂商开发不同的总线体系结构，作为区分其产品的一种方式。例如，有些 Intel 系统使用称为北桥（northbridge）和南桥（southbridge）的芯片组，分别把 CPU 连接到内存和 I/O 设备。在较老的 Pentium 和 Core 2 系统中，前端总线（front side bus，FSB）把 CPU 连接到北桥。AMD 的系统用 HyperTransport 互连替代 FSB，而较新的 Intel Core i7 系统使用 QuickPath 互连。这些不同总线架构的细节超出了本书的范围。因此，我们将统一使用图 6.6 中的高层总线结构作为贯穿全书的运行示例。它是一个简单但有用的抽象，让我们能够说得具体，抓住主要思想，而不必与任何专有设计的细节绑得太紧。

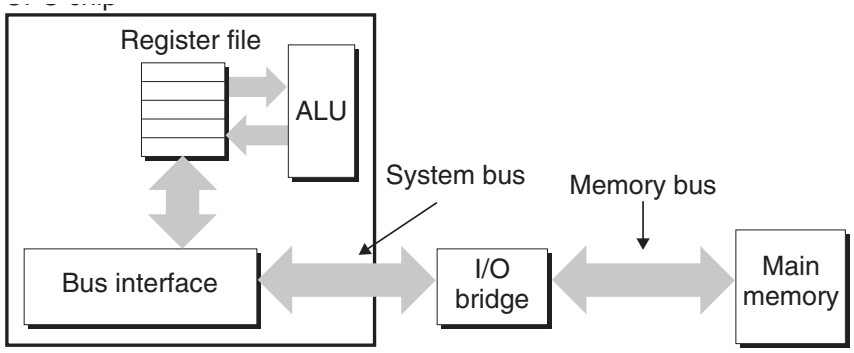


图 6.6 连接 CPU 和主存的示例总线结构。

考虑一下 CPU 执行诸如下面这样的加载操作时会发生什么

```
movq A,%rax
```

该指令把地址 A 中内容加载进寄存器%rax 中。CPU 芯片上称为总线接口（bus interface）的电路发起一次总线上的读事务。读事务由三个步骤组成。首先，CPU 将地址 A 放到系统总线上；I/O 桥将信号传递到内存总线（图 6.7(a)）。接下来，主存感应到内存总线上的地址信号，从内存总线读出地址，从 DRAM 取出数据，并把数据写到内存总线上。I/O 桥把内存总线信号转换成系统总线信号，并通过系统总线传递出去（图 6.7(b)）。最后，CPU 感应到系统总线上的数据，从总线读出数据，并把数据拷贝到寄存器%rax 中（图 6.7(c)）。

反过来，当 CPU 执行一个像下面这样的存储操作时

```
movq %rax,A
```

该指令把寄存器%rax 的内容写到地址 A 处。CPU 会发起一次写事务。同样，这里也有三个基本步骤。首先，CPU 将地址放到系统总线上；主存储器从内存总线上读取地址并等待数据到达（图 6.8(a)）。接下来，CPU 把%rax 中的数据拷贝到系统总线上（图 6.8(b)）。最后，主存从内存总线读出数据，并将这些位存放到 DRAM 中（图 6.8(c)）。

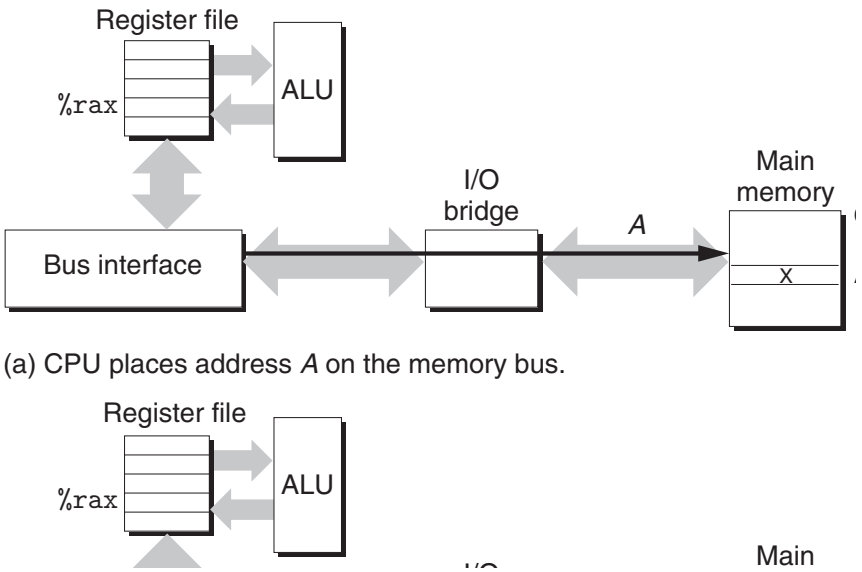


图 6.7 加载操作 `movq A,%rax` 的存储器读事务。

6.1.2 磁盘存储

磁盘是主力存储设备，它保存着大量的数据，数量级在几百 GB 到几千 GB，而基于 RAM 的存储器只能保存几百到几千 MB。但是，从磁盘上读取信息要花毫秒量级的时间，比 DRAM 慢十万倍，比 SRAM 慢一百万倍。

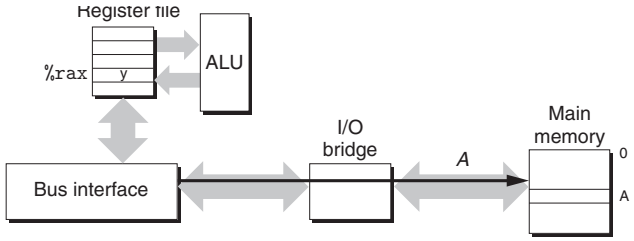
磁盘几何结构

磁盘是用盘片（platter）构造出来的。每个盘片有两个面（或者说是两个表面），表面覆盖着磁性记录材料。位于盘片中心的一个旋转主轴（spindle）按固定的转速旋转盘片，转速典型值在每分钟 5,400 到 15,000 转（RPM）之间。一个磁盘通常包含一个或多个这样的盘片，封装在一个封闭容器里。

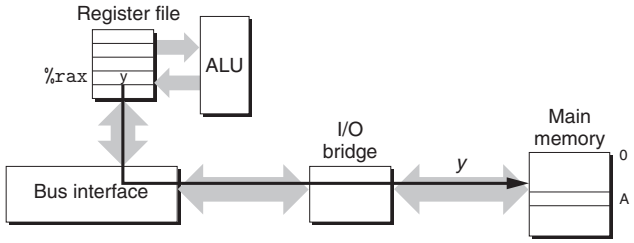
图 6.9(a) 展示了一个典型磁盘的几何结构。每个表面由一组称为磁道（track）的同心圆环组成。每个磁道被划分成若干个扇区。每个扇区内包含相同数量的数据位（通常是 512 字节数据位），这些数据位编码进该扇区的磁记录材料中。扇区之间由不存储任何数据位的间隙隔开。间隙中保存着用于标识扇区的格式化位。

一个磁盘由一个或多个叠放在一起的盘片组成，并被封装在一个密封的包装里，如图 6.9(b) 所示。整个组装体通常被称为磁盘驱动器，尽管我们通常就把它简称为磁盘。我们有时也称磁盘为旋转磁盘，以区别于基于闪存的没有运动部件的固态硬盘。

磁盘驱动器的多盘片几何结构是用柱面来描述的。一个柱面是所有表面上与主轴中心距离相等的那些磁道的集合。比如，若一个驱动器有三个盘片和



(a) CPU places address *A* on the memory bus. Main memory reads it and waits for the data word.



(b) CPU places data word *y* on the bus.

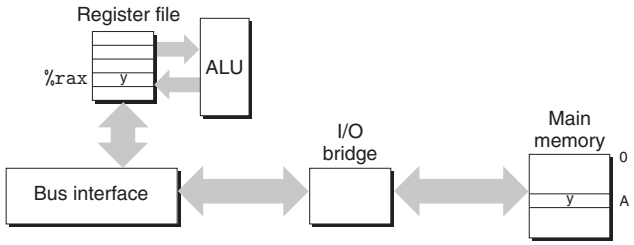


图 6.8 存储操作 `movq %rax,A` 的存储器写事务。

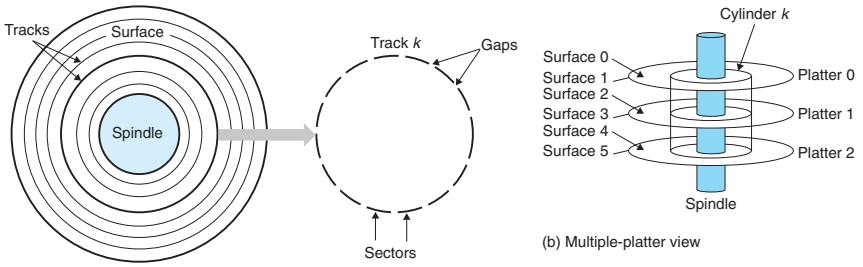


图 6.9 磁盘几何结构。

六个表面，并且每个表面上的磁道都是按统一的方式编号的，那么柱面 k 就是六个表面上第 k 号磁道的集合。

磁盘容量

一个磁盘所能记录的最大位数称为它的最大容量，或者简称容量。磁盘容量由以下几个技术因素决定：

- **记录密度**（recording density，位/英寸）。1 英寸磁道弧段中可以挤进的位数。

- **磁道密度**(track density,道/英寸)。从盘片中心出发沿半径方向 1 英寸线段中可以挤进的磁道数。
- **面密度**(areal density,位/平方英寸)。记录密度和磁道密度的乘积。

磁盘制造商们不知疲倦地提高面密度(并因此提高容量),这大约每两年就翻一倍。最初的磁盘是在面密度还相对较低的年代设计的,它们把每条磁道划分成相同数量的扇区,这个数量由最内圈磁道上能记录的扇区数决定。为了保持每条磁道固定的扇区数,外圈磁道上的扇区排布得稀疏一些。这是面密度还比较低时的一个合理做法。然而,随着面密度的提高,扇区之间不存数据位的间隙变得大得让人不可接受。于是,现代的大容量磁盘使用一种称为多区记录(multiple zone recording)的技术,把柱面集合划分成不相交的子集,称为记录区(recording zone)。每个区由一组连续的柱面组成。一个区中每个柱面上的每条磁道都有相同数量的扇区,这一数量由该区最内圈磁道能挤进的扇区数决定。

> **边栏 一个 GB 有多大?**
> 不幸的是,千(kilo,K)、兆(mega,M)、千兆(giga,G)和太(tera,T)这样的前缀的含义取决于上下文。对于与 DRAM 和 SRAM 容量相关的度量,通常 $K = 2^{10}$、$M = 2^{20}$、$G = 2^{30}$、$T = 2^{40}$。对于与磁盘和网络之类 I/O 设备容量相关的度量,通常 $K = 10^3$、$M = 10^6$、$G = 10^9$、$T = 10^{12}$。速率与吞吐量一般也使用这些前缀值。
> 幸运的是,对于我们所依赖的粗略估算来说,任一种假设在实际中都挺好。例如,2^{30} 与 10^9 的相对差异并不大:$(2^{30} - 10^9)/10^9 \approx 7\%$。类似地,$(2^{40} - 10^{12})/10^{12} \approx 10\%$。

磁盘的容量由下面这个公式给出:

$$\begin{aligned}\text{容量} = &\frac{\text{每扇区字节数}}{\text{扇区}} \times \frac{\text{每磁道平均扇区数}}{\text{磁道}} \times \frac{\text{每面磁道数}}{\text{面}} \\ &\times \frac{\text{每盘片面数}}{\text{盘片}} \times \frac{\text{每磁盘盘片数}}{\text{磁盘}}\end{aligned}$$

例如,假设我们有一个磁盘:五个盘片,每扇区 512 字节,每面 20,000 条磁道,每条磁道平均 300 个扇区。那么这个磁盘的容量是

$$\begin{aligned}\text{容量} &= \frac{512\,\text{字节}}{\text{扇区}} \times \frac{300\,\text{扇区}}{\text{磁道}} \times \frac{20{,}000\,\text{磁道}}{\text{面}} \times \frac{2\,\text{面}}{\text{盘片}} \times \frac{5\,\text{盘片}}{\text{磁盘}} \\ &= 30{,}720{,}000{,}000\ \text{字节} \\ &= 30.72\ \text{GB}\end{aligned}$$

注意,制造商用吉字节(gigabyte,GB)或太字节(terabyte,TB)来表达磁盘容量,其中 $1\,\text{GB} = 10^9$ 字节、$1\,\text{TB} = 10^{12}$ 字节。

练习 6.2 一个磁盘有 3 个盘片、15,000 个柱面、每条磁道平均 500 个扇区、每扇区 1,024 字节，它的容量是多少？

磁盘操作

如图 6.10(a) 所示，磁盘用一个连接到传动臂 (actuator arm) 末端的读/写头来读写磁性表面上的位。沿着传动臂的径向前后移动传动臂，驱动器就能把读/写头定位在盘面上的任意一个磁道上。这种机械运动称为寻道(seek)。一旦读/写头定位到目标磁道上方，随着磁道上的每个位在磁头下方经过，磁头就可以感知这个位的值(读这个位)，或者改变这个位的值(写这个位)。有多个盘片的磁盘为每个表面配一个单独的读/写头，如图 6.10(b) 所示。这些磁头垂直排列、统一移动。在任何时刻，所有的磁头都位于同一个柱面上。

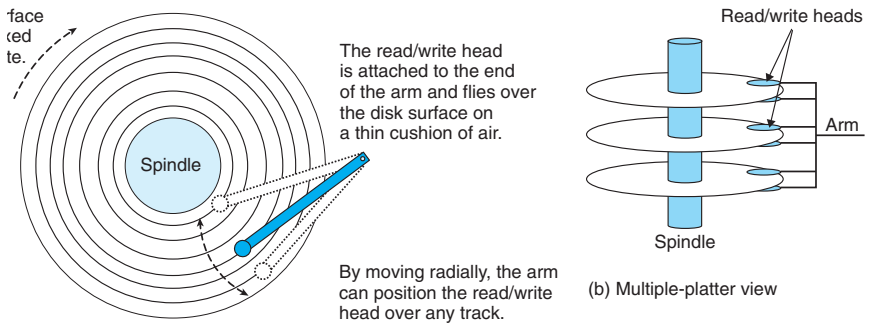


图 6.10 磁盘动力学特性。

传动臂末端的读/写头在盘面上方大约 0.1 微米的高度、以大约 80 千米/小时的速度“飞行”(字面意义上的飞行)，悬停在一层薄薄的空气垫上。这就像把一座摩天大楼放倒横过来，在离地面 2.5 厘米(约 1 英寸)的高度上环绕地球飞行，而且每绕地球一圈只需要 8 秒钟！这样的精度之下，盘面上的一小粒灰尘就好比一块巨石。如果读/写头撞上这样一块巨石，它就会停止飞行并撞上盘面(这就是所谓的头碰撞)。正因为如此，磁盘总是被封在完全不透气的封装里。

磁盘以扇区大小的块为单位读写数据。访问一个扇区要有三个主要部分：寻道时间(seek time)、旋转延迟(rotational latency)和传送时间(transfer time)：

- **寻道时间。** 为了读出某个目标扇区的内容，传动臂首先把读/写头定位到包含目标扇区的磁道上。移动传动臂所花的时间叫做寻道时间。寻道时间 T_{seek} 取决于磁头之前的位置和传动臂在盘面上移动的速度。现代驱动器的平均寻道时间 $T_{\text{avg seek}}$ 是通过对几千次到随机扇区的寻道取平均值来测量的，其典型值在 3 到 9 毫秒的量级。单次寻道的最大时间 $T_{\text{max seek}}$ 可以高达 20 毫秒。

- **旋转延迟**。 一旦读/写头定位到了磁道上方，驱动器就要等待目标扇区的第一个位转到磁头下方。这一步的耗时既取决于读/写头到达目标磁道时盘面所处的位置，也取决于磁盘的转速。最坏的情况是，磁头恰好错过目标扇区，只能等磁盘转完一整圈。因此，最大旋转延迟（以秒为单位）为

$$T_{\text{max rotation}} = \frac{1}{\text{RPM}} \times \frac{60\,\text{秒}}{1\,\text{分}}$$

平均旋转延迟 $T_{\text{avg rotation}}$ 就是 $T_{\text{max rotation}}$ 的一半。

- **传送时间**。 当目标扇区的第一个位转到磁头下方时，驱动器就开始读或写这个扇区的内容。一个扇区的传送时间取决于转速和每条磁道的扇区数。因此，我们可以粗略地估计出一个扇区的平均传送时间（以秒为单位）为

$$T_{\text{avg transfer}} = \frac{1}{\text{RPM}} \times \frac{1}{(\text{每磁道平均扇区数})} \times \frac{60\,\text{秒}}{1\,\text{分}}$$

我们可以把访问磁盘扇区内容的平均时间估计成平均寻道时间、平均旋转延迟和平均传送时间之和。例如，考虑一个具有如下参数的磁盘：

参数	值
旋转速率	7,200 RPM
$T_{\text{avg seek}}$	9 ms
每磁道平均扇区数	400

对于这个磁盘，平均旋转延迟（以 ms 计）是

$$\begin{aligned} T_{\text{avg rotation}} &= \tfrac{1}{2} \times T_{\text{max rotation}} \\ &= \tfrac{1}{2} \times (60\,\text{秒}/7{,}200\,\text{RPM}) \times 1{,}000\,\text{ms/秒} \\ &\approx 4\,\text{ms} \end{aligned}$$

平均传送时间是

$$\begin{aligned} T_{\text{avg transfer}} &= 60/7{,}200\,\text{RPM} \times 1/400\,\text{扇区/磁道} \times 1{,}000\,\text{ms/秒} \\ &\approx 0.02\,\text{ms} \end{aligned}$$

把这些加在一起，访问的总时间估计为

$$\begin{aligned} T_{\text{access}} &= T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}} \\ &= 9\,\text{ms} + 4\,\text{ms} + 0.02\,\text{ms} \\ &= 13.02\,\text{ms} \end{aligned}$$

这个例子说明了一些要点：

- 访问一个磁盘扇区中 512 字节的时间由寻道时间和旋转延迟主导。访问扇区中第一个字节要花很长时间，而剩余的字节读起来几乎不必花时间。
- 由于寻道时间和旋转延迟大致相当，把寻道时间加倍是一个估计磁盘访问时间的简单而合理的方法。
- 存放到 SRAM 中的 64 位字的访问时间大约是 4 纳秒，DRAM 大约是 60 纳秒。这样，从内存读一个扇区大小 512 字节的块，SRAM 大约 256 纳秒，DRAM 大约 4,000 纳秒。磁盘访问时间大约 10 毫秒，大约是 SRAM 的 40,000 倍、DRAM 的 2,500 倍。

练习 6.3 估计在下面的磁盘上访问一个扇区的平均时间（以 ms 计）：

参数	值
旋转速率	12,000 RPM
$T_{\text{avg seek}}$	5 ms
每磁道平均扇区数	300

逻辑磁盘块

我们已经看到了，现代磁盘具有复杂的几何结构，由多个表面以及这些表面上的不同记录区组成。为了在操作系统面前隐藏这种复杂性，现代磁盘给出了一个比较简单的视图：把它们自己的几何结构表示成 B 个扇区大小的逻辑块（logical block）的序列，编号 0、1、...、$B-1$。磁盘封装中的一个称为磁盘控制器的小硬件/固件设备，维护着逻辑块号和真正的（物理）磁盘扇区之间的映射。

当操作系统想执行一个 I/O 操作时，比如把磁盘扇区读进主存，它就会向磁盘控制器发送一条命令，请求读一个特定的逻辑块号。控制器上的固件执行一次快速的表查找，把逻辑块号翻译成唯一标识相应物理扇区的一个三元组（表面，磁道，扇区）。控制器上的硬件解释这个三元组，把磁头移到合适的柱面，等到扇区在磁头下方经过，把磁头感知到的那些位收集到控制器上一个小存储缓冲区中，并把它们拷贝到主存。

练习 6.4 假设有一个由 512 字节逻辑块组成的 1 MB 文件，存放在一个具有如下特性的磁盘驱动器上：

> **边栏 格式化后的磁盘容量**
> 磁盘在存数据之前，必须先由磁盘控制器完成格式化。这包括把扇区间隙填入标识扇区的信息；找出有表面缺陷的柱面并让它们失效；并且在每个区里预留一组备用柱面，以备这个区中有一个或多个柱面在磁盘使用期之内失效时调用。因为存在这些备用柱面，磁盘厂商给出的格式化容量要小于最大容量。

参数	值
旋转速率	13,000 RPM
$T_{\text{avg seek}}$	6 ms
每磁道平均扇区数	5,000
面数	4
扇区大小	512 字节

对于下面的每种情况，假设程序依次读取文件的逻辑块，一个接一个，并把磁头定位到第一个块上所需的时间是 $T_{\text{avg seek}} + T_{\text{avg rotation}}$。

A. 最佳情况。估计在逻辑块到磁盘扇区的最佳可能映射（即顺序映射）下读取该文件所需的最短时间（以 ms 计）。

B. 随机情况。估计在块被随机映射到磁盘扇区的情况下读取该文件所需的时间（以 ms 计）。

连接 I/O 设备

诸如图形卡、监视器、鼠标、键盘和磁盘这样的输入/输出（I/O）设备，用一根 I/O 总线连接到 CPU 和主存。不同于 CPU 专用的系统总线和内存总线，I/O 总线被设计成与底层 CPU 无关。图 6.11 给出了一个连接 CPU、主存和 I/O 设备的代表性 I/O 总线结构。

虽然 I/O 总线要比系统总线和内存总线慢，但是它能容纳各种各样的第三方 I/O 设备。比如，图 6.11 所示的总线上就接着三种不同类型的设备。

- **通用串行总线（USB）控制器**。 它是连接到 USB 总线上的设备的一个通道。USB 总线是一种非常流行的标准，用来连接各种外围 I/O 设备，包括键盘、鼠标、调制解调器、数码相机、游戏手柄、打印机、外部磁盘驱动器和固态硬盘。USB 3.0 总线的带宽上限是 625 MB/s；USB 3.1 总线的带宽上限是 1,250 MB/s。
- **图形卡（显卡）**。 图形卡（graphics card，或者说 graphics adapter）包含负责在显示监视器上为 CPU 绘出像素的硬件和软件逻辑。

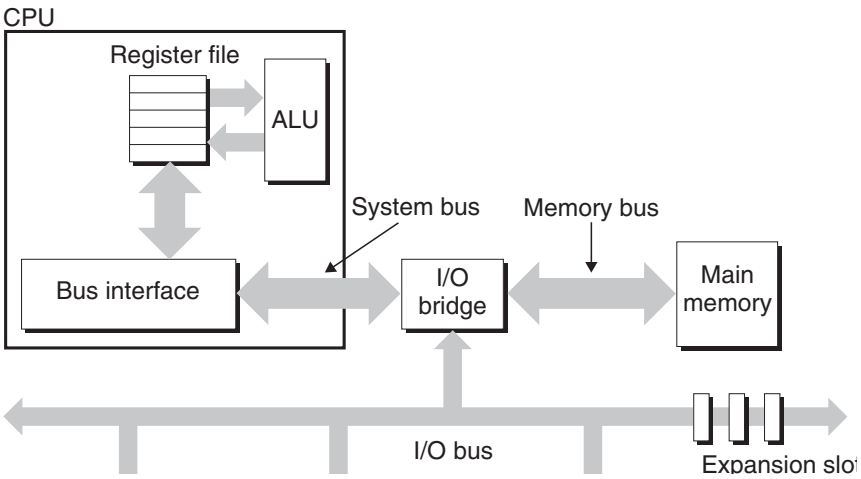


图 6.11 连接 CPU、主存和 I/O 设备的示例总线结构。

- **主机总线适配器。** 一个主机总线适配器使用由特定主机总线接口定义的通信协议，把一个或多个磁盘连接到 I/O 总线。对磁盘来说，两种最流行的这类接口是 SCSI（读作“scuzzy”）和 SATA（读作“sat-uh”）。SCSI 磁盘通常比 SATA 驱动器更快也更贵。一个 SCSI 主机总线适配器（常被称作 SCSI 控制器）可以支持多个磁盘驱动器，SATA 适配器则只能支持一个。

其他设备（例如网络适配器）可以通过把适配器插入主板上的空闲扩展槽连接到 I/O 总线，扩展槽提供与总线的直接电气连接。

访问磁盘

对 I/O 设备如何工作以及如何对它们进行编程，详细的讨论超出了本书的范围，不过我们可以给你一个大致的概念。例如，图 6.12 概括了 CPU 读磁盘时发生的步骤。

> **边栏 I/O 总线设计的进展**
> 图 6.11 中的 I/O 总线是一个让我们把话说得具体、又不必与任何特定系统的细节绑得太紧的简单抽象。它基于外围组件互连（PCI）总线，这种总线到 2010 年左右一直很流行。在 PCI 模型中，系统中的所有设备都共享总线，同一时刻只有一个设备能够访问总线上的导线。在这些现代系统中，共享的 PCI 总线已经被 PCI express（PCIe）总线替代，后者是一组用开关连接的高速串行点对点线路，与第 11 章讲到的交换式以太网类似。PCIe 总线的最大吞吐率是 16 GB/s，相比之下最大吞吐率 533 MB/s 的 PCI 总线要慢一个数量级。除了测得的 I/O 性能不同，各种总线设计间的差异对应用程序是不可见的，所以我们全书都使用简单的共享总线抽象。

CPU 用一种称为内存映射 I/O（memory-mapped I/O）的技术向 I/O 设备发出命令（图 6.12(a)）。在带内存映射 I/O 的系统中，地址空间中会保留一段地址，

用于与 I/O 设备通信。每个这样的地址都称为一个 I/O 端口。设备连到总线上时，它就（或被映射）与一个或多个端口相关联。

举一个简单的例子，假设磁盘控制器被映射到端口 0xa0。为了发起一次磁盘读，CPU 可以执行三条向地址 0xa0 存储的指令。第一条发送一个命令字，通知磁盘发起一次读，并附带其他参数，例如读完成后是否中断 CPU（中断将在 8.1 节讨论）。第二条指明应该读的逻辑块号。第三条指明磁盘扇区的内容应当保存到哪个主存地址。

发出请求之后，在磁盘执行读操作时，CPU 通常会继续做其他工作。回想一下，一个时钟周期 1 纳秒的 1 GHz 处理器，在磁盘读的 16 毫秒里原则上可以执行 1,600 万条指令。只是干等传输完成，那会极其浪费。

磁盘控制器收到 CPU 发出的读命令之后，它把逻辑块号翻译成扇区地址，读出扇区内容，并把内容直接送入主存，不需要 CPU 的任何干预（图 6.12(b)）。这种一个设备不牵涉 CPU、自己发起读或写总线事务的过程称为直接内存访问（direct memory access，DMA）。这次数据传输被称作一次 DMA 传送（DMA transfer）。

在 DMA 传送结束且这些扇区的内容安全地保存在主存之后，磁盘控制器向 CPU 发出一个中断信号来告知 CPU（图 6.12(c)）。基本思想是，一次中断向 CPU 芯片的一个外部引脚发出信号。这使得 CPU 停止当前的工作，并跳转到操作系统的一个例程。例程记录下 I/O 已完成，然后把控制返回到 CPU 被中断时所处的位置。

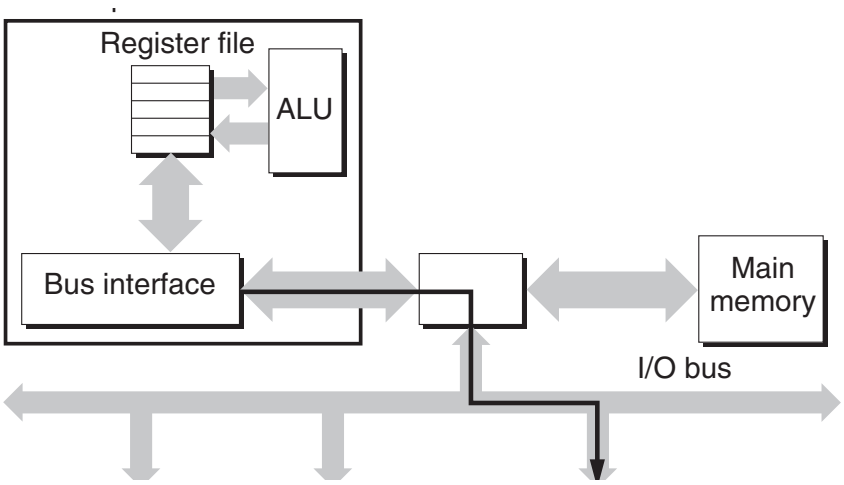


图 6.12 读一个磁盘扇区。

边栏 一款商用磁盘驱动器的特性

磁盘厂商在他们的网站上公布许多有用的较高层次的技术信息。例如，希捷（Seagate）公司的网站上有关于他们流行的驱动器之一 Barracuda 7400 的如下信息（当然还有更多其他信息！）。（Seagate.com）

几何特性	值	几何特性	值
表面直径	3.5 英寸	旋转速率	7,200 RPM
格式化容量	3 TB	平均旋转延迟	4.16 ms
盘片	3	平均寻道时间	8.5 ms
面	6	磁道间寻道时间	1.0 ms
逻辑块	5,860,533,168	平均传送速率	156 MB/s
逻辑块大小	512 字节	最大持续传送速率	210 MB/s

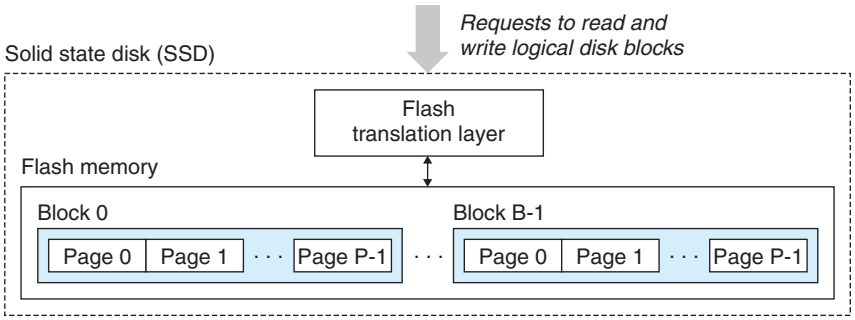


图 6.13 固态硬盘（SSD）。

6.1.3 固态硬盘

固态硬盘（solid state disk，SSD）是一种基于闪存（6.1.1 节）的存储技术，在某些情形下是传统旋转磁盘的一种颇具吸引力的替代品。图 6.13 给出了它的基本思想。一个 SSD 封装插在 I/O 总线的标准磁盘插槽上（一般是 USB 或 SATA），行为与其他任何磁盘一样，处理来自 CPU 的对逻辑磁盘块的读写请求。一个 SSD 封装由一个或多个闪存芯片和一个闪存翻译层（flash translation layer）组成。前者替代了传统旋转磁盘中的机械驱动器；后者是一个硬件/固件设备，扮演与磁盘控制器同样的角色，把对逻辑块的请求翻译成对底层物理设备的访问。

图 6.14 给出了一块典型 SSD 的性能特性。注意，从 SSD 读数据比向 SSD 写数据快。随机读写之间这样的性能差异，是由底层闪存芯片的一个基本属性造成的。如图 6.13 所示，闪存芯片由 B 个块（block）组成的序列，每个块包括 P

个页(page)。典型地,页的大小在 512 字节到 4 KB 之间,块的大小在 32 到 128 个页之间,整个块的范围是 16

图 6.14 某商用固态硬盘的性能特性。资料来源:Intel SSD 730 产品规格书 [53]。IOPS 表示每秒 I/O 操作次数。吞吐量数字基于 4 KB 数据块的读和写。(Intel SSD 730 产品规格书。Intel 公司。52.)

读取		写入	
顺序读吞吐量	550 MB/s	顺序写吞吐量	470 MB/s
随机读吞吐量(IOPS)	89,000	随机写吞吐量(IOPS)	74,000
随机读吞吐量(MB/s)	365 MB/s	随机写吞吐量(MB/s)	303 MB/s
平均顺序读访问时间	50 μs	平均顺序写访问时间	60 μs

KB 到 512 KB。数据的读出和写入以页为单位。必须先把页所属的整个块擦除,才能写入该页(通常,这意味着把块中的所有位都置为 1)。不过,块一旦被擦除,块内的每一页就可以再写入一次,无需再次擦除。块在大约 100,000 次重复写入之后会磨损。块一旦磨损,就不能再使用了。

随机写慢有两个原因。第一,擦除一个块需要相当长的时间,在 1 ms 的量级上,比访问一页所需的时间高出一个数量级还多。第二,如果一次写操作试图修改页 p,而页 p 中已经存有数据(即并非全为 1),那么同一块内所有含有有用数据的页都必须先复制到一个新的(已擦除的)块中,然后才能写页 p。制造商在闪存转换层中加入了复杂的逻辑,试图分摊擦除块的高昂代价,并尽量减少写操作时的内部复制,但随机写不太可能达到与读一样的性能。

固态硬盘相对旋转磁盘有诸多优点:它们由半导体存储器构成,没有移动部件,因此随机访问时间比旋转磁盘快得多,功耗更低,也更坚固。不过它们也有一些缺点。第一,由于闪存块在反复写入后会磨损,固态硬盘本身也有磨损的可能。闪存转换层中的磨损均衡逻辑试图把擦除操作均匀地分散到所有块上,从而延长每个块的寿命。实际上,磨损均衡逻辑做得非常好,固态硬盘要过很多年才会磨损(见练习 6.5)。第二,固态硬盘每字节的成本大约是旋转磁盘的 30 倍,因此典型的存储容量明显小于旋转磁盘。不过,随着固态硬盘日益普及,其价格正在迅速下降,与旋转磁盘的差距也在不断缩小。

固态硬盘已经完全取代旋转磁盘,成为便携式音乐设备的主流,作为笔记本电脑的磁盘替代品也很流行,甚至已经开始出现在台式机和服务器中。旋转磁盘还会继续存在下去,但固态硬盘显然已成为一种重要的替代品。

练习 6.5 正如我们所见,固态硬盘的一个潜在缺点是底层的闪存会磨损。例如,对于图 6.14 中的 SSD,Intel 保证在驱动器磨损之前可以承受约 128 PB (128×10^{15} 字节)的写入。给定这一假设,估算下述几种工作负载下这块 SSD 的使用寿命(以年计):

A. 最坏情况的顺序写：SSD 以 470 MB/s 的速率（该设备的平均顺序写吞吐量）不停地被写入。

B. 最坏情况的随机写：SSD 以 303 MB/s 的速率（该设备的平均随机写吞吐量）不停地被写入。

C. 平均情况：SSD 以每天 20 GB 的速率（一些计算机制造商在移动计算机工作负载模拟中假设的日平均写速率）被写入。

6.1.4 存储技术趋势

回顾我们对存储技术的讨论，有几个重要的概念值得记住。

不同的存储技术在价格与性能上的折中不同。SRAM 比 DRAM 稍快，DRAM 比磁盘快得多。另一方面，快速的存储总是比慢速的存储贵。SRAM 每字节的成本高于 DRAM，DRAM 的成本又远高于磁盘。固态硬盘的价位则介于 DRAM 与旋转磁盘之间。

不同存储技术的价格与性能属性发生变化的速率也截然不同。图 6.15 总结了自 1985 年（第一代 PC 刚刚问世后不久）以来存储技术的价格与性能属性。这些数字取自历年行业杂志和万维网。虽然收集的方式并不正式，但这些数字揭示了一些有趣的趋势。

自 1985 年以来，SRAM 技术的成本与性能以大致相同的速率得到了改进：访问时间和每兆字节成本都降到原来的约 1/100（图 6.15(a)）。而 DRAM 和磁盘的趋势要剧烈得多，也更分化：DRAM 每兆字节的成本降到原来的 1/44,000（超过四个数量级！），访问时间却只降到原来的 1/10（图 6.15(b)）。磁盘技术遵循与 DRAM 相同的趋势，而且更为剧烈：自 1980 年以来，磁盘存储每兆字节的成本暴跌到原来的 1/3,000,000 以下（超过六个数量级！），访问时间的改善却慢得多，只降到原来的 1/25（图 6.15(c)）。这些惊人的长期趋势凸显了存储器与磁盘技术的一个基本事实：提高密度（从而降低成本）要比缩短访问时间容易得多。

DRAM 和磁盘的性能落后于 CPU 的性能。如图 6.15(d) 所示，1985—2010 年间，CPU 周期时间降到原来的 1/500。如果考察有效周期时间——我们把它定义为单个 CPU（处理器）的周期时间除以其处理器核数——那么 1985—2010 年间的改善更大，达到了 1/2,000。

CPU 性能曲线在 2003 年前后出现的转折，反映了多核处理器的引入（见边栏“当时钟周期停滞不前：多核处理器的出现”）。转折之后，单个核的周期时间实际上先略有上升，然后才开始再次下降，不过下降的速率比以前慢了。

SRAM 的性能虽然滞后，但大致跟上了变化。然而，DRAM 和磁盘性能与 CPU 性能之间的差距实际上还在扩大。在 2003 年前后多核处理器出现之前，

图 6.15 存储技术与处理技术趋势。2010 年前后的 Core i7 采用 Nehalem 处理器核；2015 年的 Core i7 采用 Haswell 核。

指标	1985	1990	1995	2000	2005	2010	2015	2015:1985
美元/MB	2,900	320	256	100	75	60	25	116
访问时间(ns)	150	35	15	3	2	1.5	1.3	115

(a) SRAM 趋势

指标	1985	1990	1995	2000	2005	2010	2015	2015:1985
美元/MB	880	100	30	1	0.1	0.06	0.02	44,000
访问时间(ns)	200	100	70	60	50	40	20	10
典型容量(MB)	0.256	4	16	64	2,000	8,000	16,000	62,500

(b) DRAM 趋势

指标	1985	1990	1995	2000	2005	2010	2015	2015:1985
美元/GB	100,000	8,000	300	10	5	0.3	0.03	3,333,333
最小寻道时间(ms)	75	28	10	8	5	3	3	25
典型容量(GB)	0.01	0.16	1	20	160	1,500	3,000	300,000

(c) 旋转磁盘趋势

指标	1985	1990	1995	2000	2003	2005	2010	201
Intel CPU	80286	80386	Pent.	P-III	Pent. 4	Core 2	Core i7 (n)	Core i7 (
时钟频率(MHz)	6	20	150	600	3,300	2,000	2,500	3,00
周期时间(ns)	166	50	6	1.6	0.3	0.5	0.4	0.3
核数	1	1	1	1	1	2	4	
有效周期时间(ns)	166	50	6	1.6	0.30	0.25	0.10	0.0

(d) CPU 趋势

这一性能差距是延迟造成的：DRAM 和磁盘访问时间的缩短比单个处理器的周期时间慢。引入多核之后，这一性能差距越来越多地体现在吞吐量上：多个处理器核并行地向 DRAM 和磁盘发出请求。

图 6.16 非常清晰地展示了上述种种趋势，它把图 6.15 中的访问时间和周期时间以半对数刻度画了出来。

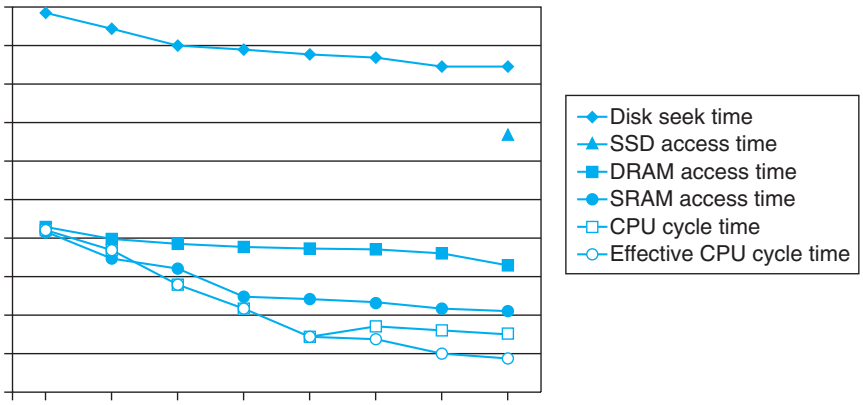


图 6.16 磁盘、DRAM 和 CPU 速度之间的差距。

正如 6.4 节将要讲到的，现代计算机大量使用基于 SRAM 的高速缓存，试图弥合处理器与存储器之间的差距。这种方法行得通，是因为应用程序具有一种称为局部性(locality)的基本属性，这正是我们接下来要讨论的内容。

练习 6.6 利用图 6.15(c) 中 2005—2015 年的数据，估计在哪一年你可以用 200 美元买到 1 PB(10^{15} 字节)的旋转磁盘存储。假设美元为实际币值(不计通货膨胀)。

6.2 局部性

编写良好的计算机程序往往表现出良好的局部性。也就是说，它们倾向于引用的数据项，要么临近最近引用过的其他数据项，要么自身刚刚被引用过。这种倾向称为局部性原理(principle of locality)，是一个影响深远的概念，对硬件和软件系统的设计与性能都有着巨大的影响。

局部性通常被认为有两种不同的形式：时间局部性(temporal locality)和空间局部性(spatial locality)。在具有良好时间局部性的程序中，被引用过一次的存储位置很可能在不远的将来再被引用多次。在具有良好空间局部性的程序中，如果某个存储位置被引用了一次，那么程序很可能在不远的将来引用它附近的一个存储位置。

程序员应当理解局部性原理，因为一般而言，局部性好的程序比局部性差的程序运行得更快。现代计算机系统的各个层次——从硬件、操作系统到应用程序——的设计都利用了局部性。在硬件层面，局部性原理使计算机设计者能够通过引入小而快的存储器来加速对主存的访问，这类存储器称为高速缓存(cache)，其中保存着最近引用的指令和数据项块。在操作系统层面，局部性原理允许系统把主存用作最近引用的虚拟地址空间块的高速缓存。类似地，操作系统用主存来缓存磁盘文件系统中最近用过的磁盘块。局部性原理在应用程

序的设计中也起着关键作用。例如，Web 浏览器把最近引用的文档缓存在本地磁盘上，利用了时间局部性；大容量的 Web 服务器把最近请求的文档保存在前端磁盘高速缓存中，不需要服务器进行任何干预，就能满足对这些文档的请求。

边栏　当时钟周期停滞不前：多核处理器的出现

计算机的历史上有若干标志性事件，它们给整个行业乃至世界带来了深刻的变化。有趣的是，这些转折点大约每十年出现一次：20 世纪 50 年代 Fortran 的开发，60 年代初 IBM 360 的推出，70 年代初因特网（当时叫做 ARPANET）的诞生，80 年代初 IBM PC 的推出，以及 90 年代初万维网的创建。

最近的一次这样的事件发生在 21 世纪初。计算机制造商一头撞上了所谓的功耗墙（power wall），发现不能像以往那样快速地提高 CPU 时钟频率了，否则芯片的功耗会过高。解决的办法是从性能入手：用多个较小的处理器核取代单个大处理器，每个核都是一个完整的处理器，可以独立运行程序，也能与其他核并行地运行程序。这种多核方法之所以行得通，部分原因在于处理器消耗的功率与 $P = fCV^2$ 成正比，其中 f 是时钟频率，C 是电容，V 是电压。电容 C 大致与面积成正比，因此只要所有核的总面积保持不变，多核消耗的功率就可以保持不变。只要工艺特征尺寸继续按摩尔定律的指数速率缩小，每个处理器中的核数以及由此获得的有效性能就会继续增加。

从今以后，计算机会变得更快，不是因为时钟频率在提高，而是因为每个处理器中的核数在增加，也因为体系结构上的创新使运行在这些核上的程序更有效率。这一趋势在图 6.16 中看得十分清楚：CPU 周期时间在 2003 年到达最低点，随后实际上开始上升，然后走平，又以比先前更慢的速率开始再次下降。不过，由于多核处理器的出现（2004 年出现双核，2007 年出现四核），有效周期时间仍以接近原先的速率持续下降。

(a)

```
1  int sumvec(int v[N])
2  {
3      int i, sum = 0;
4
5      for (i = 0; i < N; i++)
6          sum += v[i];
7      return sum;
8  }
```

(b)

地址	0	4	8	12	16	20	24	28
内容	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
访问顺序	1	2	3	4	5	6	7	8

图 6.17 (a) 局部性良好的函数。(b) 向量 $v(N = 8)$的引用模式。注意向量元素的访问次序与它们在存储器中的存放次序相同。

6.2.1 对程序数据引用的局部性

考虑图 6.17(a) 中对向量元素求和的简单函数。这个函数有良好的局部性吗？要回答这个问题，我们逐一考察每个变量的引用模式。在这个例子中，sum 变量在每次循环迭代中都被引用一次，因此对 sum 有良好的时间局部性。另一方面，由于 sum 是标量，对 sum 没有空间局部性。

如图 6.17(b) 所示，向量 v 的元素是按它们在存储器中的存放次序一个接一个地被顺序读出的（为方便起见，我们假设数组从地址 0 开始）。因此，就变量 v 而言，这个函数有良好的空间局部性，而时间局部性较差，因为每个向量元素都只被访问一次。既然该函数对循环体内的每个变量要么有良好的空间局部性，要么有良好的时间局部性，我们可以得出结论：sumvec 函数具有良好的局部性。

像 sumvec 这样依次访问向量每个元素的函数，被称为具有步长（stride）为 1 的引用模式（stride-1 reference pattern，步长以元素大小为单位）。我们有时也把步长为 1 的引用模式称为顺序引用模式（sequential reference pattern）。在连续向量中每隔 k 个元素访问一个，称为步长为 k 的引用模式（stride-k reference pattern）。步长为 1 的引用模式是程序中空间局部性常见而重要的来源。一般而言，步长越大，空间局部性越差。

步长对引用多维数组的程序来说同样是个重要问题。例如，考虑图 6.18(a) 中对二维数组元素求和的 sumarrayrows 函数。

双层嵌套循环按行优先（row-major）次序读取数组的元素。也就是说，内层循环先读取第一行的元素，然后是第二行的元素，依此类推。sumarrayrows 函数具有良好的空间局部性，因为它按数组存放时相同的行优先次序访问数组（图 6.18(b)）。结果是一个很好的步长为 1 的引用模式，具有极佳的空间局部性。

(a)

```
1  int sumarrayrows(int a[M][N])
```

```
2  {
3      int i, j, sum = 0;
4
5      for (i = 0; i < M; i++)
6          for (j = 0; j < N; j++)
7              sum += a[i][j];
8      return sum;
9  }
```

(b)

地址	0	4	8	12	16	20
内容	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
访问顺序	1	2	3	4	5	6

图 6.18 (a) 局部性良好的另一个函数。(b) 数组 $a(M=2,N=3)$的引用模式。空间局部性良好,因为对数组的访问次序与它在存储器中存放的行优先次序相同。

(a)

```
1  int sumarraycols(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (j = 0; j < N; j++)
6          for (i = 0; i < M; i++)
7              sum += a[i][j];
8      return sum;
9  }
```

(b)

地址	0	4	8	12	16	20
内容	a_{00}	a_{01}	a_{02}	a_{10}	a_{11}	a_{12}
访问顺序	1	3	5	2	4	6

图 6.19 (a) 空间局部性差的函数。(b) 数组 $a(M=2,N=3)$ 的引用模式。该函数以步长为 N 的引用模式扫描存储器,因而空间局部性很差。

对程序中看似微不足道的改动,就可能对它的局部性造成很大影响。例如,图 6.19(a) 中的 `sumarraycols` 函数计算的结果与图 6.18(a) 中 `sumarrayrows` 函数相同,唯一的区别是我们把 `i` 循环和 `j` 循环互换了。交换循环对它的局部性有什么影响?

`sumarraycols` 函数的空间局部性很差,因为它按列而不是按行扫描数组。由于 C 语言数组在存储器中是按行存放的,就得到了一个步长为 N 的引用模式,如图 6.19(b) 所示。

6.2.2 取指令的局部性

由于程序指令存储在存储器中，必须由 CPU 取出（读出），我们也可以评价一个程序在取指令方面的局部性。例如，图 6.17 中 for 循环体内的指令按存储器顺序依次执行，因此循环具有良好的空间局部性。又因为循环体会被执行多次，它也具有良好的时间局部性。

代码有别于程序数据的一个重要属性是它在运行时很少被修改。程序执行时，CPU 从存储器中读取指令，很少改写或修改这些指令。

6.2.3 局部性小结

在本节中，我们介绍了局部性的基本思想，并归纳出了几条定性地评价程序局部性的简单规则：

- 反复引用相同变量的程序具有良好的时间局部性。
- 对于具有步长为 k 的引用模式的程序，步长越小，空间局部性越好。步长为 1 的引用模式具有良好的空间局部性；以大步长在存储器中跳来跳去的程序空间局部性很差。
- 循环对取指令而言具有良好的时间局部性和空间局部性。循环体越小、循环迭代次数越多，局部性越好。

在本章后面，学习了高速缓存存储器及其工作原理之后，我们会演示如何用命中与不命中来量化局部性的概念。为什么局部性好的程序通常比局部性差的程序运行得更快，届时你也会明白原因。即便如此，学会如何扫一眼源代码，就对程序中的局部性有一个高层次的感觉，仍然是程序员应当掌握的一项有用而重要的技能。

练习 6.7 交换下面函数中循环的次序，使它以步长为 1 的引用模式扫描三维数组 `a`。

```
1  int productarray3d(int a[N][N][N])
2  {
3      int i, j, k, product = 1;
4
5      for (i = N-1; i >= 0; i--) {
6          for (j = N-1; j >= 0; j--) {
7              for (k = N-1; k >= 0; k--) {
8                  product *= a[j][k][i];
9              }
10         }
11     }
12     return product;
13 }
```

(a) 结构体数组

```
#define N 1000

typedef struct {
    int vel[3];
    int acc[3];
} point;

point p[N];
```

(b) 函数 clear1

```
void clear1(point *p, int n)
{
    int i, j;

    for (i = 0; i < n; i++) {
        for (j = 0; j < 3; j++)
            p[i].vel[j] = 0;
        for (j = 0; j < 3; j++)
            p[i].acc[j] = 0;
    }
}
```

(c) 函数 clear2

```
void clear2(point *p, int n)
{
    int i, j;

    for (i = 0; i < n; i++) {
        for (j = 0; j < 3; j++) {
            p[i].vel[j] = 0;
            p[i].acc[j] = 0;
        }
    }
}
```

(d) 函数 clear3

```
void clear3(point *p, int n)
{
    int i, j;

    for (j = 0; j < 3; j++) {
        for (i = 0; i < n; i++)
            p[i].vel[j] = 0;
        for (i = 0; i < n; i++)
            p[i].acc[j] = 0;
    }
}
```

图 6.20 练习 6.8 的代码示例。

练习 6.8 图 6.20 中的三个函数执行同样的操作，但空间局部性的程度各不相同。请按空间局部性对这些函数排序，并说明你是如何得出这个排序的。

6.3 存储器层次结构

6.1 节和 6.2 节描述了存储技术与计算机软件的一些基本而持久的属性：

- **存储技术**。不同存储技术的访问时间差别很大。越快的存储技术每字节的成本越高，容量也越小。CPU 与主存速度之间的差距在不断扩大。
- **计算机软件**。编写良好的程序往往表现出良好的局部性。

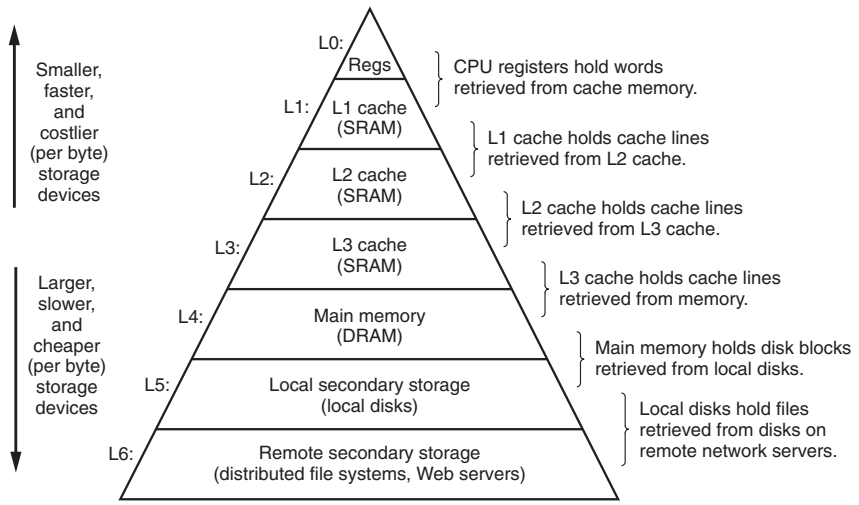


图 6.21 存储器层次结构。

计算领域有一个皆大欢喜的巧合：硬件和软件的这些基本属性完美地互补。它们的互补性启发了一种组织存储器系统的方法，称为存储器层次结构（memory hierarchy），这一方法被所有现代计算机系统采用。图 6.21 展示了一个典型的存储器层次结构。

一般而言，从高层向低层走，存储设备变得更慢、更便宜，也更大。最高层（L0）是少量的快速 CPU 寄存器，CPU 可以在单个时钟周期内访问它们。接下来是一个或多个小型到中型的基于 SRAM 的高速缓存存储器，访问它们需要几个 CPU 时钟周期。然后是大的基于 DRAM 的主存，访问它需要几十到几百个时钟周期。接下来是慢速但容量巨大的本地磁盘。最后，有些系统甚至还包括附加的一层磁盘，它们位于远程服务器上，可以通过网络访问。例如，像 Andrew 文件系统（AFS）或网络文件系统（NFS）这样的分布式文件系统，允许程序访问存储在远程联网服务器上的文件。类似地，万维网允许程序访问存放在世界上任何地方的 Web 服务器上的远程文件。

6.3.1 存储器层次结构中的高速缓存

一般来说，高速缓存（cache）是一种小而快的存储设备，充当存放于更大、更慢设备中的数据对象的暂存区（staging area）。使用高速缓存的过程称为缓存（caching）。

存储器层次结构的核心思想是，对每一级 k，第 k 层上更快、更小的存储设备都充当第 $k+1$ 层上更大、更慢存储设备的高速缓存。换句话说，层次结构中的每一层都缓存来自下一较低层的数据对象。例如，本地磁盘充当通过网络从远程磁盘获取的文件（如 Web 页）的高速缓存；主存充当本地磁盘上数据的高速缓存；依此类推，直到最小的高速缓存——CPU 寄存器组。

> **边栏　其他存储器层次结构**
> 我们前面展示了存储器层次结构的一个例子，但其他组合也是可能的，而且实际上很常见。例如，包括 Google 数据中心在内的许多站点把本地磁盘备份到存档磁带上。在这些站点中，有些由操作员根据需要人工把磁带安装到磁带机上；另一些则由磁带机器人自动完成这项工作。无论哪种情况，磁带的集合都代表存储器层次结构中低于本地磁盘的一层，同样的通用原理依然适用。磁带每字节的成本低于磁盘，这使得站点能够保存本地磁盘的多个快照。代价是磁带的访问时间比磁盘长。再举一个例子，固态硬盘在存储器层次结构中扮演着越来越重要的角色，填补了 DRAM 与旋转磁盘之间的鸿沟。

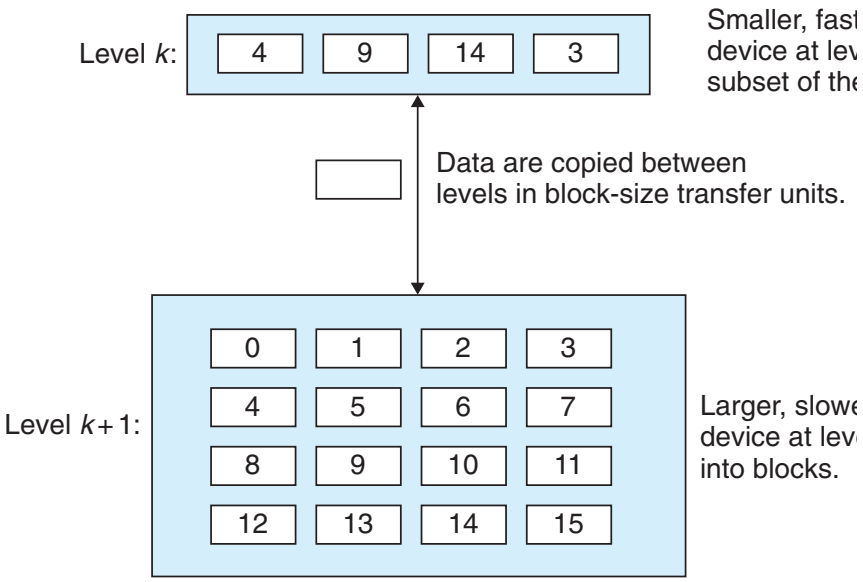


图 6.22　存储器层次结构中缓存的基本原理。

图 6.22 展示了存储器层次结构中缓存的一般概念。第 $k+1$ 层的存储被划分成称为块（block）的连续数据对象片段。每个块都有一个唯一的地址或名称，

以便与其他块相区别。块可以是固定大小的(通常情况),也可以是可变大小的(例如存储在 Web 服务器上的远程 HTML 文件)。例如,图 6.22 中的第 $k+1$ 层存储被划分成 16 个固定大小的块,编号为 0 到 15。

类似地,第 k 层的存储也被划分成一组较小的块,其大小与第 $k+1$ 层的块相同。在任一时刻,第 k 层的高速缓存都包含第 $k+1$ 层某个块子集的副本。例如,在图 6.22 中,第 k 层的高速缓存有容纳 4 个块的空间,当前包含块 4、9、14 和 3 的副本。

数据总是以块大小的传输单位在第 k 层与第 $k+1$ 层之间来回复制。应当认识到,虽然层次结构中任何一对相邻层之间的块大小是固定的,但其他层对可以有不同的块大小。例如,在图 6.21 中,L1 与 L0 之间的传输通常使用字大小的块;L2 与 L1(以及 L3 与 L2、L4 与 L3)之间的传输通常使用几十字节的块;而 L5 与 L4 之间的传输使用几百或几千字节的块。一般而言,层次结构中位置越低的设备(离 CPU 越远)访问时间越长,因此往往使用更大的块,以分摊这些较长的访问时间。

高速缓存命中

当程序需要第 $k+1$ 层的某个特定数据对象 d 时,它会先在第 k 层当前存放的某个块中寻找 d。如果 d 恰好缓存在第 k 层,这就是我们所说的高速缓存命中(cache hit)。程序直接从第 k 层读取 d,而根据存储器层次结构的特性,这比从第 $k+1$ 层读取 d 更快。例如,具有良好时间局部性的程序可能从块 14 中读取一个数据对象,得到第 k 层的一次命中。

高速缓存不命中

反过来,如果数据对象 d 没有缓存在第 k 层,这就是所说的高速缓存不命中(cache miss)。发生不命中时,第 k 层的高速缓存会从第 $k+1$ 层的高速缓存中取出包含 d 的块,如果第 k 层的高速缓存已经满了,还可能覆盖原有的某个块。

覆盖现存块的这个过程称为对该块的替换(replacing)或驱逐(evicting)。被驱逐的块有时被称为牺牲块(victim block)。替换哪个块由高速缓存的替换策略(replacement policy)决定。例如,采用随机替换策略的高速缓存会随机选择一个牺牲块;采用最少使用(least recently used,LRU)替换策略的高速缓存会选择最久未被访问的块。

第 k 层的高速缓存从第 $k+1$ 层取出块之后,程序可以像以前一样从第 k 层读取 d。例如,在图 6.22 中,从块 12 读取一个数据对象会导致第 k 层高速

缓存不命中，因为块 12 目前没有存放在第 k 层的高速缓存中。一旦块 12 从第 $k+1$ 层复制到第 k 层，它就会保存在那里，等待以后的访问。

不命中的种类

区分不同种类的不命中有时是有帮助的。如果第 k 层的高速缓存是空的，那么对任何数据对象的任何访问都会不命中。空的高速缓存有时称为冷缓存（cold cache），这种不命中称为强制不命中（compulsory miss）或冷不命中（cold miss）。冷不命中之所以重要，是因为它们往往是暂时性的，在高速缓存被反复的存储器访问“预热”之后，稳态下可能不会再出现。

一旦发生不命中，第 k 层的高速缓存就必须实现某种放置策略（placement policy），决定把从第 $k+1$ 层取回的块放到什么位置。最灵活的放置策略是允许把第 $k+1$ 层的任何块存放到第 k 层的任何块中。对于存储器层次结构高层（靠近 CPU）中用硬件实现、速度至上的一些高速缓存来说，这个策略通常因实现代价太高而不可行，因为随机放置的块难以定位。

因此，硬件高速缓存通常实现一种较简单的放置策略，把第 $k+1$ 层的某个块限制在第 k 层块的一个小子集（有时甚至是单个块）上。例如，在图 6.22 中，我们可以规定第 $k+1$ 层的块 i 必须放到第 k 层的块 $(i \bmod 4)$ 中。这样，第 $k+1$ 层的块 0、4、8 和 12 会映射到第 k 层的块 0；块 1、5、9 和 13 会映射到块 1；依此类推。注意，图 6.22 中的示例高速缓存采用的就是这种策略。

这种限制性的放置策略会导致一类称为冲突不命中（conflict miss）的不命中：高速缓存的容量足以容纳被引用的数据对象，但由于这些对象映射到同一个高速缓存块，缓存会不断不命中。例如，在图 6.22 中，如果程序先请求块 0，然后请求块 8，然后又是块 0，再是块 8，依此类推，那么对这两个块的每次引用都会在第 k 层的高速缓存中不命中，尽管这个缓存总共能容纳 4 个块。

程序常常按一系列阶段（phase）运行（例如循环），每个阶段访问一组基本固定的缓存块。例如，嵌套循环可能反复访问同一数组的元素。这组块称为该阶段的工作集（working set）。当工作集的大小超过高速缓存的大小时，缓存就会出现所谓容量不命中（capacity miss）。换句话说，缓存太小，应付不了这个特定的工作集。

高速缓存的管理

正如我们所指出的，存储器层次结构的实质是每层存储设备都是下一较低层的高速缓存。每一层都有某种形式的逻辑来管理这个缓存。也就是说，必须有某种东西把缓存存储划分成块、在不同层之间传输块、判定何时发生命中与不命中，并随后进行处理。管理缓存的逻辑可以是硬件、软件，或者两者的结合。

例如，编译器管理着寄存器堆，即高速缓存层次结构的最高层。它决定何时在不命中时发出加载指令，并决定把数据存到哪个寄存器中。L1、L2 和 L3 层的高速缓存完全由内建于缓存中的硬件逻辑来管理。在具有虚拟内存的系统中，DRAM 主存充当磁盘上数据块的高速缓存，由操作系统软件和 CPU 上的地址转换硬件共同管理。对于配有 AFS 这类分布式文件系统的机器，本地磁盘充当高速缓存，由运行在本地机上的 AFS 客户端进程管理。在大多数情况下，高速缓存自动运作，不需要程序进行任何特殊的或显式的操作。

图 6.23 高速缓存在现代计算机系统中的普遍性。缩写：TLB：**转换旁视缓冲区**（translation lookaside buffer）；MMU：**存储器管理单元**（memory management unit）；OS：**操作系统**；NFS：**网络文件系统**。

类型	缓存的内容	缓存在哪里	延迟（时钟周期）	由谁管理
CPU 寄存器	4 字节或 8 字节的字	片上 CPU 寄存器	0	编译器
TLB	地址转换	片上 TLB	0	硬件 MMU
L1 高速缓存	64 字节的块	片上 L1 高速缓存	4	硬件
L2 高速缓存	64 字节的块	片上 L2 高速缓存	10	硬件
L3 高速缓存	64 字节的块	片上 L3 高速缓存	50	硬件
虚拟内存	4 KB 的页	主存	200	硬件 + 操作系统
缓冲区高速缓存	文件的部分内容	主存	200	操作系统
磁盘高速缓存	磁盘扇区	磁盘控制器	100,000	控制器固件
网络高速缓存	文件的部分内容	本地磁盘	10,000,000	NFS 客户端
浏览器高速缓存	Web 页	本地磁盘	10,000,000	Web 浏览器
Web 高速缓存	Web 页	远程服务器磁盘	1,000,000,000	Web 代理服务器

6.3.2 存储器层次结构概念小结

概括起来，基于缓存的存储器层次结构之所以有效，是因为较慢的存储比较快的存储便宜，也因为程序往往表现出局部性：

- **利用时间局部性**。由于时间局部性，同一数据对象很可能被重复使用多次。数据对象在第一次不命中时被复制进缓存之后，就可以预计这个对象随后还会有一系列命中。由于缓存比下一较低层的存储快，为这些后续命中服务的速度要远远快于最初那次不命中。
- **利用空间局部性**。块中通常包含多个数据对象。由于空间局部性，可以预计不命中时复制一个块的开销，会被随后对该块内其他对象的引用所分摊。

现代系统中到处都在使用缓存。从图 6.23 可以看到，CPU 芯片、操作系统、分布式文件系统以及万维网上都在使用缓存。它们由硬件和软件的各种组合

来构成和管理。注意，图 6.23 中还有许多我们尚未讲到的术语和缩写，在这里列出它们，只是为了展示缓存有多么常见。

6.4 高速缓存存储器

早期计算机系统的存储器层次结构只有三层：CPU 寄存器、主存和磁盘存储。然而，由于 CPU 与主存之间的差距不断加大，系统设计者不得不在 CPU 寄存器堆与主存之间插入一个小型 SRAM 高速缓存存储器，称为 L1 高速缓存（L1 cache，一级高速缓存），如图 6.24 所示。访问 L1 高速缓存的速度几乎和寄存器一样快，通常大约需要 4 个时钟周期。

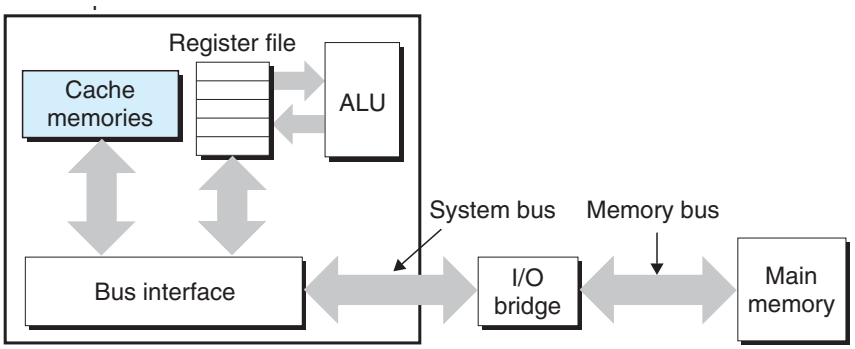


图 6.24 用于高速缓存存储器的典型总线结构。

由于 CPU 与主存之间的性能差距还在继续扩大，系统设计者的应对方法是在 L1 高速缓存与主存之间再插入一个更大的 L2 高速缓存，访问它大约需要 10 个时钟周期。许多现代系统还包括一个更大的 L3 高速缓存，它位于存储器层次结构中 L2 高速缓存与主存之间，访问它大约需要 50 个周期。这些做法虽然花样繁多，但一般原理是相同的。为了便于下一节的讨论，我们假设一个简单的存储器层次结构：CPU 与主存之间只有一个 L1 高速缓存。

6.4.1 通用高速缓存存储器组织结构

考虑这样一个计算机系统：每个存储器地址有 m 位，共构成 $M = 2^m$ 个不同的地址。如图 6.25(a) 所示，这种机器的高速缓存被组织成一个包含 $S = 2^s$ 个组（set）的数组。每个组包含 E 个高速缓存行（cache line）。每行由一个 $B = 2^b$ 字节的数据块、一个指示该行是否含有有效信息的有效位（valid bit），以及 $t = m - (b + s)$ 个标记位（tag bit，即当前块的存储器地址位的一个子集，用来唯一标识该高速缓存行中存放的块）组成。

一般来说，一个高速缓存的组织结构可以用元组 (S, E, B, m) 来刻画。高速缓存的大小（或容量）C 按所有块的总大小计，不计标记位和有效位。于是，$C = S \times E \times B$。

当一条加载指令要求 CPU 从主存的地址 A 读出一个字时，它把地址 A 发送给高速缓存。如果高速缓存中保存着地址 A 处那个字的副本，它就立即把这个字发回 CPU。那么，高速缓存如何知道自己是否包含地址 A 处字的副本呢？高速缓存的组织方式是：只需检查地址的位就能找到被请求的字，这类似于一个带有极其简单的哈希函数的哈希表。它的工作过程如下：

参数 S 和 B 把 m 个地址位划分成图 6.25(b) 所示的三个域。A 中的 s 个组索引位构成一个索引，指向

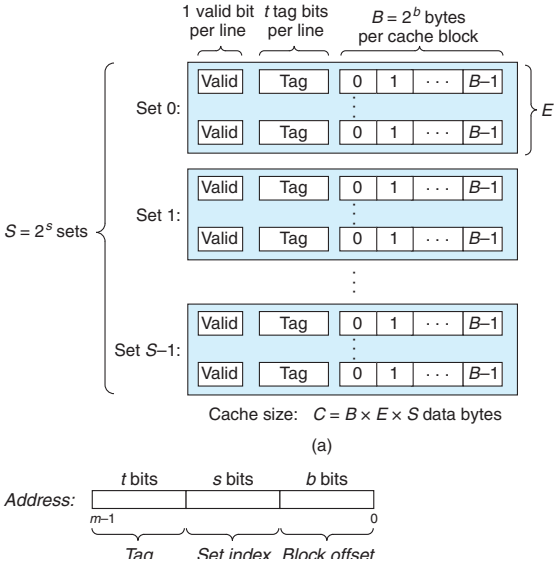


Figure 6.25
General organization of cache (S, E, B, m).
(a) A cache is an array of sets. Each set contains one or more lines. Each line contains a valid bit, some tag bits, and a block of data. (b) The cache organization induces a partition of the m address bits into t tag bits, s set index bits, and b block offset bits.

图 6.25

由 S 个组组成的数组。第一个组是组 0，第二个组是组 1，以此类推。把这些组下标位解释为一个无符号整数时，它告诉我们这个字必须存放在哪个组中。一旦知道了这个字必须包含在哪个组里，A 中的 t 个标记位就会告诉我们（如果有的话）该组中的哪一行包含这个字。当且仅当该行的有效位置位，而且行中的标记位与地址 A 中的标记位相匹配时，组中的这一行才包含这个字。一旦定位了由组下标所标识的组中由标记所标识的行，那么 b 个块偏移位就给出了这个字在 B 字节数据块中的偏移。

你可能已经注意到，描述高速缓存要用到大量的符号。图 6.26 对这些符号做了归纳，以供参考。

练习 6.9　下面这张表给出了若干不同高速缓存的参数。对每个高速缓存，确定高速缓存的组数(S)、标记位数(t)、组下标位数(s)和块偏移位数(b)。

高速缓存	m	C	B	E	S	t	s	b
1.	32	1,024	4	1				
2.	32	1,024	8	4				
3.	32	1,024	32	32				

图 6.26　高速缓存参数小结。

参数	说明
基本参数	
$S = 2^s$	组数
E	每组中的行数
$B = 2^b$	块大小(字节)
$m = \log_2 M$	物理(主存)地址的位数
派生量	
$M = 2^m$	不同存储器地址的最大数目
$s = \log_2 S$	组下标位数
$b = \log_2 B$	块偏移位数
$t = m - (s + b)$	标记位数
$C = B \times E \times S$	高速缓存大小(字节)，不包括有效位和标记位等开销

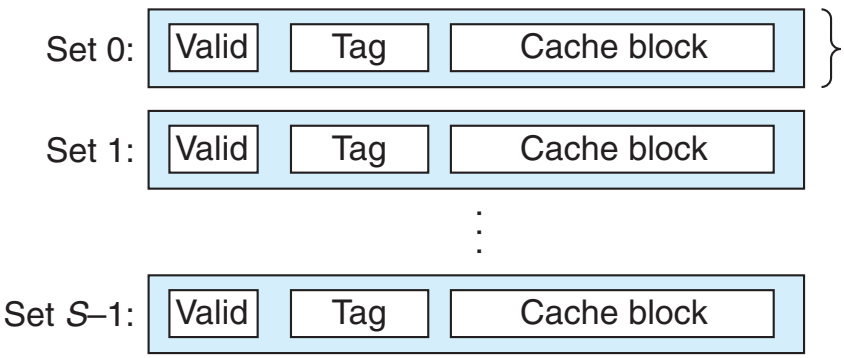


图 6.27　直接映射高速缓存(E = 1)。每组恰好有一行。

6.4.2 直接映射高速缓存

根据 E，即每组中高速缓存行的数目，高速缓存划分为不同的类别。每组恰好只有一行（$E = 1$）的高速缓存称为直接映射高速缓存（direct-mapped cache，见图 6.27）。直接映射高速缓存最容易实现，也最容易理解，所以我们用它来阐明高速缓存工作原理的一些一般性概念。

假设我们有一个由 CPU、寄存器堆、L1 高速缓存和主存组成的系统。当 CPU 执行一条读存储器字 w 的指令时，它会向 L1 高速缓存请求这个字。如果 L1 高速缓存缓存了 w 的副本，那么就是一次 L1 高速缓存命中，高速缓存很快提取出 w 并把它返回给 CPU。否则，就是一次不命中，当 L1 高速缓存向主存请求包含 w 的块的副本时，CPU 必须等待。当被请求的块最终从存储器到达时，L1 高速缓存把这个块存储在它的某个高速缓存行里，从已存储的块中提取出字 w，然后把字返回给 CPU。高速缓存判断一次请求是命中还是不命中并提取出所请求的字，这一过程由三个步骤组成：(1) 组选择，(2) 行匹配，(3) 字提取。

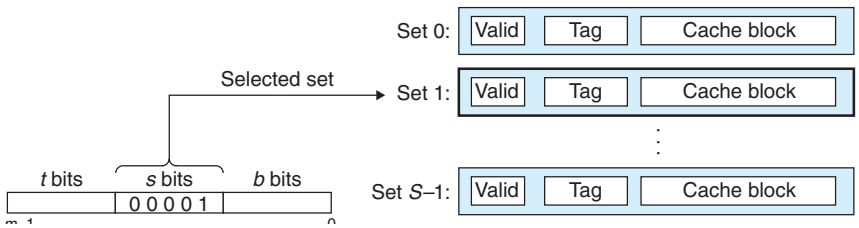


图 6.28　直接映射高速缓存中的组选择。

直接映射高速缓存中的组选择

在这个步骤中，高速缓存从 w 的地址中间抽取 s 个组下标位。这些位被解释为一个无符号整数，对应一个组号。换句话说，如果我们把高速缓存看成由 S 个组组成的一个一维数组，那么这些组下标位就是这个数组的一个索引。图 6.28 展示了直接映射高速缓存中组选择的工作方式。在这个示例中，组下标位 00001_2 被解释为一个整数索引，它选择组 1。

直接映射高速缓存中的行匹配

既然在上一步中我们选出某个组 i，下一步就是判断 w 的副本是否存放在组 i 所包含的某个高速缓存行中。在直接映射高速缓存中，这一步既容易又快捷，因为每组恰好只有一行。当且仅当有效位置位，而高速缓存行中的标记与 w 地址中的标记相匹配时，这一行才包含 w 的副本。

图 6.29 展示了直接映射高速缓存中行匹配的工作方式。在这个示例中，被选中的组里恰好有一行高速缓存行。该行的有效位置位了，所以我们知道标记

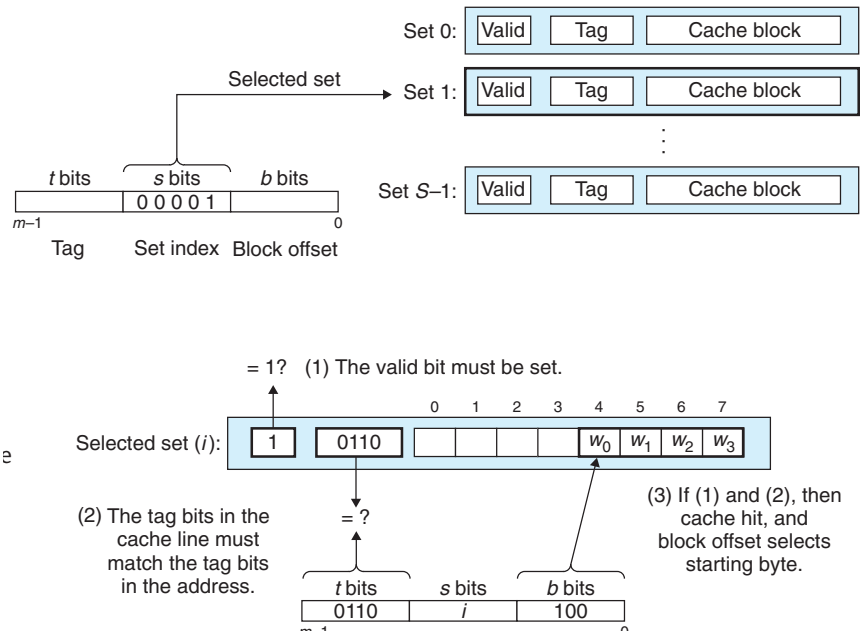


图 6.29 直接映射高速缓存中的行匹配和字选择。在高速缓存块内，w0 表示字 w 的低序字节，w1 表示下一个字节，依此类推。

和块中的位是有意义的。因为高速缓存行中的标记位与地址中的标记位相匹配，所以我们想要的字的副本确实存储在这一行中。换句话说，这是一次命中。另一方面，如果有效位没有置位，或者标记不匹配，那么就是一次不命中。

直接映射高速缓存中的字提取

一旦命中，我们就知道 w 在块中的某个地方。最后一步确定想要的字在块中的起始位置。如图 6.29 所示，块偏移位给出了想要的字在块中的第一个字节的偏移。与把高速缓存看成行的数组一样，我们也可以把一个块看成字节的数组，而字节偏移就是这个数组的一个索引。在示例中，块偏移位 100_2 表明 w 的副本从块中的字节 4 开始。(我们假设每个字长为 4 字节。)

直接映射高速缓存中不命中时的行替换

如果高速缓存不命中，那么它需要从存储器层次结构的下一层取出被请求的块，并把新块存储在由组下标位指示的组的某个高速缓存行中。一般而言，如果该组中有效的高速缓存行已经占满，那么就必须驱逐其中一行。对于每组恰好只有一行的直接映射高速缓存来说，替换策略很简单：当前这一行由新取回的行来替换。

综合起来：一个运转中的直接映射高速缓存

高速缓存用来选择组和识别行的机制极其简单。它们也必须如此，因为硬件必须在几个纳秒内完成这些操作。不过，以这种方式摆弄这些位对我们人类来说容易引起混淆。一个具体的例子将有助于澄清整个过程。假设我们有一个由下式描述的直接映射高速缓存：

$$(S, E, B, m) = (4, 1, 2, 4)$$

换句话说，这个高速缓存有 4 个组，每组 1 行，每块 2 字节，地址有 4 位。我们还假设每个字是一个单字节。当然，这些假设完全不合实际，但有助于让这个例子保持简单。

初次学习高速缓存时，把整个地址空间逐一枚举出来并对各部分位进行划分是很有启发性的，就跟我们在图 6.30 中对 4 位例子的做法一样。关于这个枚举出来的地址空间，有几点值得注意：

- 标记位与下标位的连接唯一地标识了存储器中的每个块。例如，块 0 由地址 0 和 1 组成，块 1 由地址 2 和 3 组成，块 2 由地址 4 和 5 组成，以此类推。
- 由于有 8 个存储器块却只有 4 个高速缓存组，多个块会映射到同一个高速缓存组（也就是说，它们的组下标相同）。例如，块 0 和块 4 都映射到组 0，块 1 和块 5 都映射到组 1，以此类推。
- 映射到同一个高速缓存组的块由标记唯一地标识。例如，块 0 的标记位为 0，块 4 的标记位为 1；块 1 的标记位为 0，块 5 的标记位为 1，以此类推。

下面我们来模拟高速缓存的动作，考察 CPU 执行一系列读操作时的情形。记住，对本例我们假设 CPU 读的是 1 字节的字。虽然这种手工模拟枯燥乏味，你可能很想跳过它，但根据我们的经验，学生只有亲手做过几次这样的模拟，才能真正理解高速缓存是如何工作的。

初始时，高速缓存是空的（也就是说，每个有效位都是 0）：

组	有效	标记	`block[0]`	`block[1]`
0	0			
1	0			
2	0			
3	0			

表中每一行代表一个高速缓存行。第一列指示该行所属的组，但请记住，这一列只是为方便而设，并不是高速缓存的实际组成部分。接下来的 4 列表示

图 6.30 示例直接映射高速缓存的 4 位地址空间。

地址(十进制)	标记位($t=1$)	下标位($s=2$)	偏移位($b=1$)	块号(十进制)
0	0	00	0	0
1	0	00	1	0
2	0	01	0	1
3	0	01	1	1
4	0	10	0	2
5	0	10	1	2
6	0	11	0	3
7	0	11	1	3
8	1	00	0	4
9	1	00	1	4
10	1	01	0	5
11	1	01	1	5
12	1	10	0	6
13	1	10	1	6
14	1	11	0	7
15	1	11	1	7

每个高速缓存行中的实际位。现在,我们来看看 CPU 执行一系列读操作时会发生什么:

1. 读地址 0 处的字。因为组 0 的有效位是 0,这是一次不命中。高速缓存从存储器(或更低层的某级)取出块 0,并把它存储到组 0 中。然后,高速缓存从新取来的高速缓存行的 `block[0]` 中返回 m[0](存储器位置 0 的内容)。

组	有效	标记	`block[0]`	`block[1]`
0	1	0	m[0]	m[1]
1	0			
2	0			
3	0			

2. 读地址 1 处的字。这是一次命中。高速缓存立刻从高速缓存行的 `block[1]` 返回 m[1]。高速缓存的状态没有变化。

3. 读地址 13 处的字。因为组 2 中的高速缓存行是无效的,这是一次不命中。高速缓存把块 6 装载到组 2 中,并从新高速缓存行的 `block[1]` 返回 m[13]。

组	有效	标记	block[0]	block[1]
0	1	0	m[0]	m[1]
1	0			
2	1	1	m[12]	m[13]
3	0			

4. 读地址 8 处的字。这是一次不命中。组 0 中的行确实有效，但标记不匹配。高速缓存把块 4 装载到组 0（替换了在读地址 0 时存入的那一行），并从新高速缓存行的 `block[0]` 返回 m[8]。

组	有效	标记	block[0]	block[1]
0	1	1	m[8]	m[9]
1	0			
2	1	1	m[12]	m[13]
3	0			

5. 读地址 0 处的字。这是又一次不命中，原因的很不幸，我们在上一次引用地址 8 时刚替换掉了块 0。这种不命中——高速缓存中明明有大量空间，我们却在映射到同一个组的块之间来回交替引用——是冲突不命中（conflict miss）的一个例子。

组	有效	标记	block[0]	block[1]
0	1	0	m[0]	m[1]
1	0			
2	1	1	m[12]	m[13]
3	0			

直接映射高速缓存中的冲突不命中

冲突不命中在真实程序中很常见，会造成令人费解的性能问题。直接映射高速缓存中的冲突不命中通常发生在程序访问大小为 2 的幂的数组时。例如，考虑下面这个计算两个向量点积的函数：

```
1  float dotprod(float x[8], float y[8])
2  {
3      float sum = 0.0;
4      int i;
5
```

```
6      for (i = 0; i < 8; i++)
7          sum += x[i] * y[i];
8      return sum;
9  }
```

这个函数对于 x 和 y 有很好的空间局部性,所以我们可能预期它能获得相当数量的高速缓存命中。遗憾的是,情况并不总是如此。

假设 float 类型占 4 字节,x 被加载到从地址 0 开始的 32 字节连续存储区中,而 y 紧随 x,从地址 32 开始。为简单起见,假设一个块为 16 字节(足够存放 4 个 float 值),高速缓存由两个组组成,总大小为 32 字节。我们还假设变量 sum 实际存储在 CPU 的一个寄存器里,因此不需要存储器引用。基于这些假设,每个 x[i] 和 y[i] 都会映射到同一个高速缓存组:

元素	地址	组下标	元素	地址	组下标
x[0]	0	0	y[0]	32	0
x[1]	4	0	y[1]	36	0
x[2]	8	0	y[2]	40	0
x[3]	12	0	y[3]	44	0
x[4]	16	1	y[4]	48	1
x[5]	20	1	y[5]	52	1
x[6]	24	1	y[6]	56	1
x[7]	28	1	y[7]	60	1

在运行时,循环的第一次迭代引用 x[0],这是一次不命中,导致包含 x[0] ~ x[3] 的块被装载到组 0。下一次引用的是 y[0],这又是一次不命中,导致包含 y[0] ~ y[3] 的块被复制到组 0,覆盖了上一次引用复制进去的 x 的那些值。在下一次迭代中,对 x[1] 的引用不命中,又把 x[0] ~ x[3] 的块装载回组 0,覆盖了 y[0] ~ y[3] 的块。于是我们就遇到了冲突不命中;实际上,对 x 和 y 的每一次后续引用都会导致冲突不命中,我们在 x 和 y 的块之间来回颠簸。抖动(thrashing)一词描述的正是高速缓存反复装载和驱逐同一组高速缓存块的任何情形。

> **边栏 为什么用中间的位做下标?**
> 你也许感到奇怪,高速缓存为什么用中间的位来做组下标,而不是用最高位。用中间的位更好,这是有充分理由的。图 6.31 展示了原因。如果用最高位做下标,那么一些连续的存储器块就会映射到同一个高速缓存组。例如在这个图中,前 4 个块映射到第一个高速缓存组,接着 4 个块映射到第二个组,以此类推。如果一个程序有

很好的空间局部性，按顺序扫描一个数组的元素，那么任一时刻高速缓存只能保存这个数组中一个块大小的片段。这是对高速缓存的一种低效使用。与之相反，用中间的位做下标时，相邻的块总是映射到不同的高速缓存组。这种情况下，高速缓存可以保存数组的一个大小为 C 的完整片段，其中 C 是高速缓存的大小。

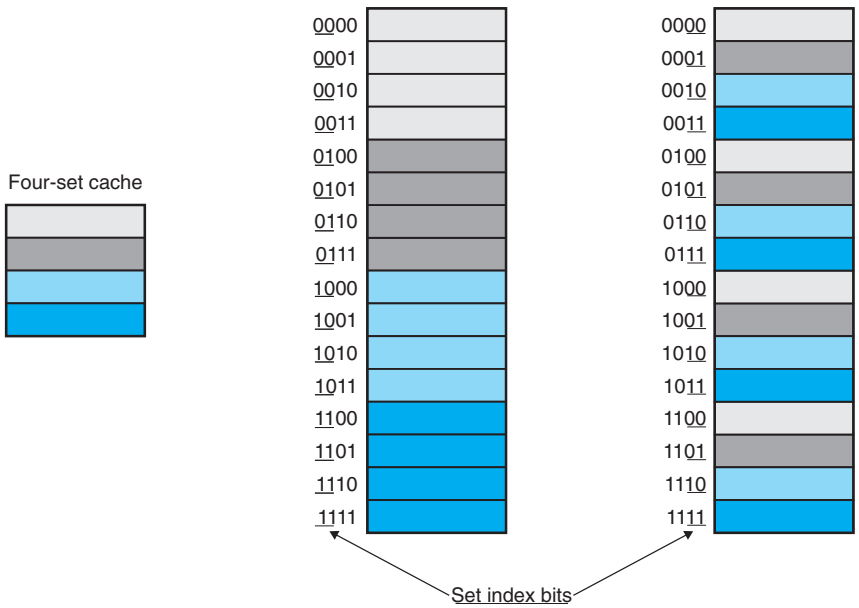


图 6.31 高速缓存为什么用地址中间的位来索引。

结论是：即使程序有很好的空间局部性，高速缓存中也有空间可以容纳 `x[i]` 和 `y[i]` 的块，每次引用还是会出现冲突不命中，因为这些块映射到了同一个高速缓存组。这种抖动造成 2 ~ 3 倍的减速也并不罕见。另外请注意，虽然我们的例子极其简单，但对更大、更接近实际的直接映射高速缓存而言，这个问题是真实存在的。

幸运的是，抖动问题一旦弄清原因，对程序员来说就很容易修复。一个简单的办法是在每个数组的末尾加 B 字节的填充。

例如，不把 `x` 定义为 `float x[8]`，而是定义为 `float x[12]`。假设 `y` 在存储器中紧跟 `x`，我们就得到如下数组元素到组的映射：

元素	地址	组下标	元素	地址	组下标
x[0]	0	0	y[0]	48	1
x[1]	4	0	y[1]	52	1
x[2]	8	0	y[2]	56	1
x[3]	12	0	y[3]	60	1
x[4]	16	1	y[4]	64	0
x[5]	20	1	y[5]	68	0
x[6]	24	1	y[6]	72	0
x[7]	28	1	y[7]	76	0

在 x 的末尾加填充之后，x[i] 和 y[i] 现在映射到不同的组，这就消除了引起抖动的冲突不命中。

练习 6.10 在前面的 dotprod 例子中，一旦给数组 x 做了填充，对 x 和 y 的全部引用中有多少比例会命中？

练习 6.11 想象一个用地址的最高 s 位作组下标的假想高速缓存。对这样的高速缓存，存储器块的连续片段会被映射到同一个高速缓存组。

A. 这些连续的数组片段中，每个片段包含多少个块？

B. 考虑下面的代码，它运行在一台高速缓存形式为 $(S, E, B, m) = (512, 1, 32, 32)$ 的机器上：

```
int array[4096];

for (i = 0; i < 4096; i++)
    sum += array[i];
```

在任一时刻，高速缓存中最多能存放多少个数组块？

6.4.3 组相联高速缓存

直接映射高速缓存中冲突不命中的问题，源于每组恰好只有一行（或者说 $E = 1$）这一约束。组相联高速缓存（set associative cache）放宽了这一约束，使得每组能保存多于一行的高速缓存行。$1 < E < C/B$ 的高速缓存通常称为 E 路组相联高速缓存。我们将在下一节讨论 $E = C/B$ 的特殊情况。图 6.32 展示了一个二路组相联高速缓存的组织结构。

组相联高速缓存中的组选择

组选择与直接映射高速缓存中完全相同，由组下标位标识组。图 6.33 归纳了这一原理。

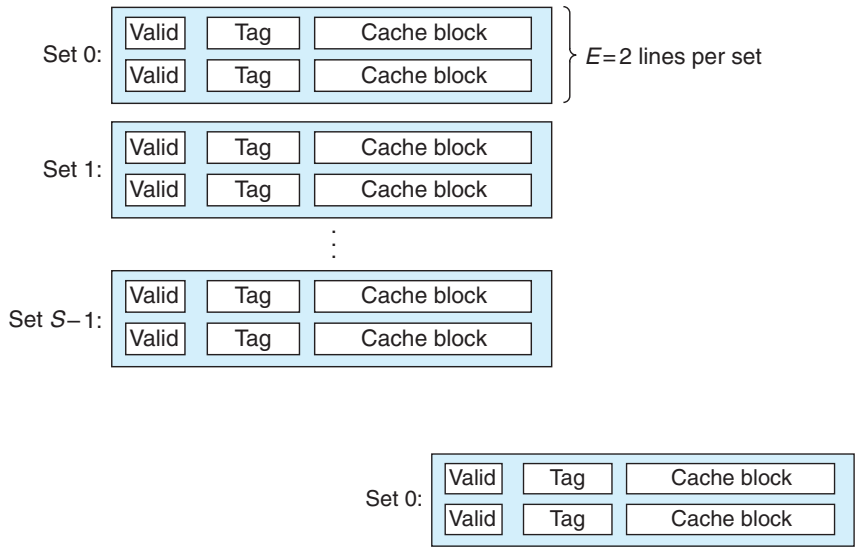
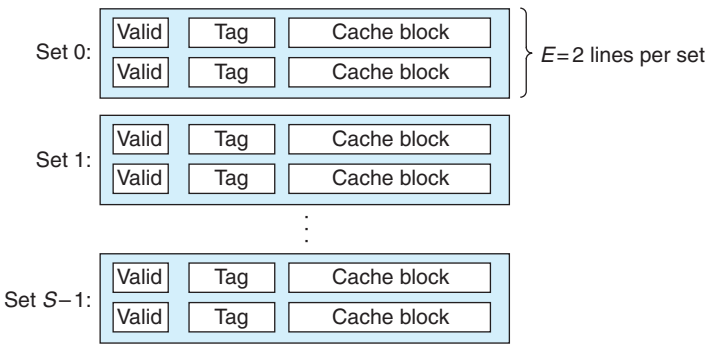


图 6.32 组相联高速缓存(1 < E < C/B)。在组相联高速缓存中,每组包含多于一行。该示例具体展示了一个两路组相联高速缓存。



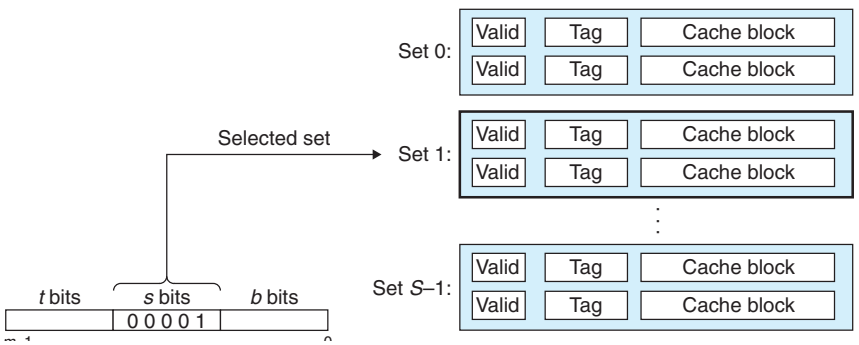


图 6.33 组相联高速缓存中的组选择。

组相联高速缓存中的行匹配与字提取

组相联高速缓存中的行匹配比直接映射高速缓存中的更复杂，因为它必须检查多行的标记和有效位，才能确定请求的字是否在组中。传统的存储器是一个值的数组，它以一个地址为输入，返回存储在那个地址上的值。相联存储器（associative memory）则是一个（键，值）对的数组，它以键为输入，返回某个与输入键相匹配的（键，值）对中的值。因此，我们可以把组相联高速缓存中的每个组看成一个小相联存储器，其中键是标记位与有效位的连接，值则是块的内容。

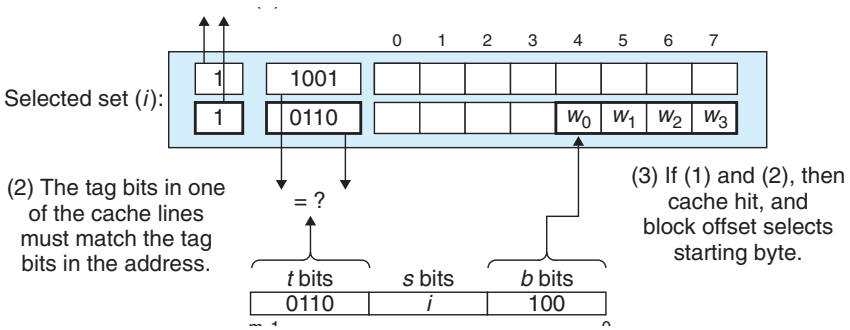


图 6.34 组相联高速缓存中的行匹配和字选择。

图 6.34 展示了相联高速缓存中行匹配的基本思想。这里的一个重要概念是：组中的任何一行都可以包含映射到该组的任何存储器块。因此，高速缓存必须搜索组中的每一行，寻找一个有效行，其标记与地址中的标记相匹配。如果高速缓存找到了这样一行，那么就是一次命中，块偏移从块中选择出想要的字，和从前一样。

组相联高速缓存中不命中时的行替换

如果 CPU 请求的字不存储在组内的任何一行中，那么就是一次不命中，高速缓存必须从存储器取出包含这个字的块。不过，一旦取回了块，应该替换哪一行呢？当然，如果组中有一行是空的，那么它就是理想的候选。但如果组中没有空行，我们就必须选择非空行中的一行，并指望 CPU 不会很快引用被替换的那一行。

程序员在自己的代码中想利用高速缓存替换策略的知识非常困难，所以我们不在这里深入讲述。最简单的替换策略是随机选择一行来替换。其他一些更复杂的策略利用局部性原理，力图使被替换的行在近期被引用的概率最小。例如，最不经常使用（least frequently used，LFU）策略会替换过去某个时间窗口内被引用次数最少的那一行；最近最少使用（least recently used，LRU）策略会替换最后访问时间距今最久的那一行。所有这些策略都需要额外的时间和硬件。

不过,在存储器层次结构中沿着离 CPU 越来越远的方向往下走时,一次不命中的开销会越来越高,用好的替换策略把不命中减到最少也就越值得。

6.4.4 全相联高速缓存

全相联高速缓存(fully associative cache)由单一的组构成(即 $E = C/B$),这个组包含所有的高速缓存行。图 6.35 展示了它的基本组织结构。

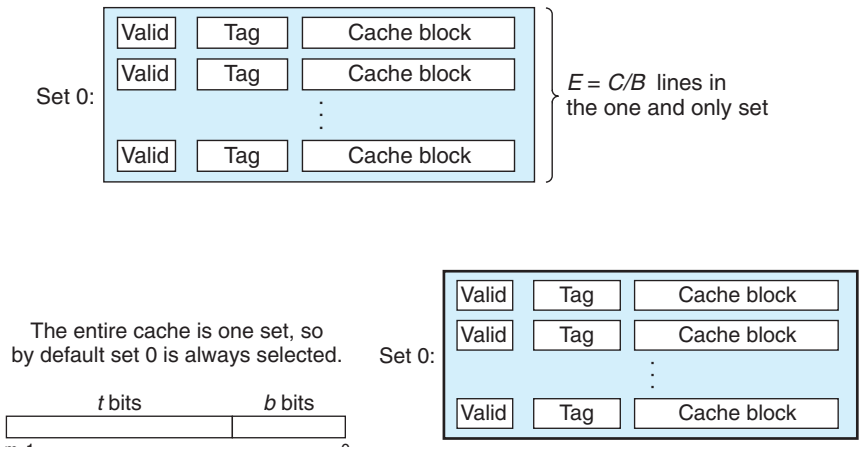


图 6.35　全相联高速缓存(E = C/B)。在全相联高速缓存中,一个单独的组包含所有的行。

全相联高速缓存中的组选择

全相联高速缓存中的组选择非常简单,因为只有一个组,如图 6.36 所示。注意,地址中没有组下标位,地址只被划分成一个标记和一个块偏移。

全相联高速缓存中的行匹配与字提取

全相联高速缓存中的行匹配和字提取与组相联高速缓存中相同,如图 6.37 所示。区别主要是规模不同。

因为高速缓存电路必须并行搜索许多相匹配的标记,要构建一个又大又快的相联高速缓存非常困难和昂贵。因此,全相联高速缓存只适合用于小的高速缓存,比如虚拟内存系统中缓存页表项的转换后备缓冲区(TLB)(9.6.2 节)。

练习 6.12　下面几道题将帮助你巩固对高速缓存工作原理的理解。假设满足下列条件:

- 存储器按字节编址。
- 存储器访问针对的是 1 字节的字(而不是 4 字节的字)。
- 地址宽度为 13 位。

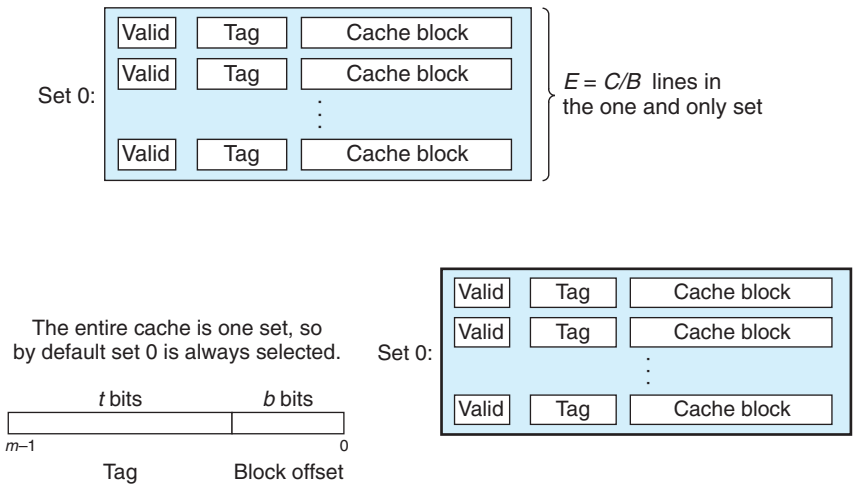


图 6.36　全相联高速缓存中的组选择。注意没有组索引位。

- 高速缓存是二路组相联的($E = 2$),块大小为 4 字节($B = 4$),有 8 个组($S = 8$)。

高速缓存的内容如下所示,所有数字均按十六进制记法给出。

组下标	标记	有效	字节 0	字节 1	字节 2	字节 3	标记	有效	字节 0	字节 1	字节 2	字节 3
0	09	1	86	30	3F	10	00	0	–	–	–	–
1	45	1	60	4F	E0	23	38	1	00	BC	0B	37
2	EB	0	–	–	–	–	0B	0	–	–	–	–
3	06	0	–	–	–	–	32	1	12	08	7B	AD
4	C7	1	06	78	07	C5	05	1	40	67	C2	3B
5	71	1	0B	DE	18	4B	6E	0	–	–	–	–
6	91	1	A0	B7	26	2D	F0	0	–	–	–	–
7	46	0	–	–	–	–	DE	1	12	C0	88	37

下面的图给出一个地址的格式(每个小格 1 位)。请在图上标注,指出用于确定下列内容所使用的字段:

- **CO.**　高速缓存块偏移
- **CI.**　高速缓存组下标
- **CT.**　高速缓存标记

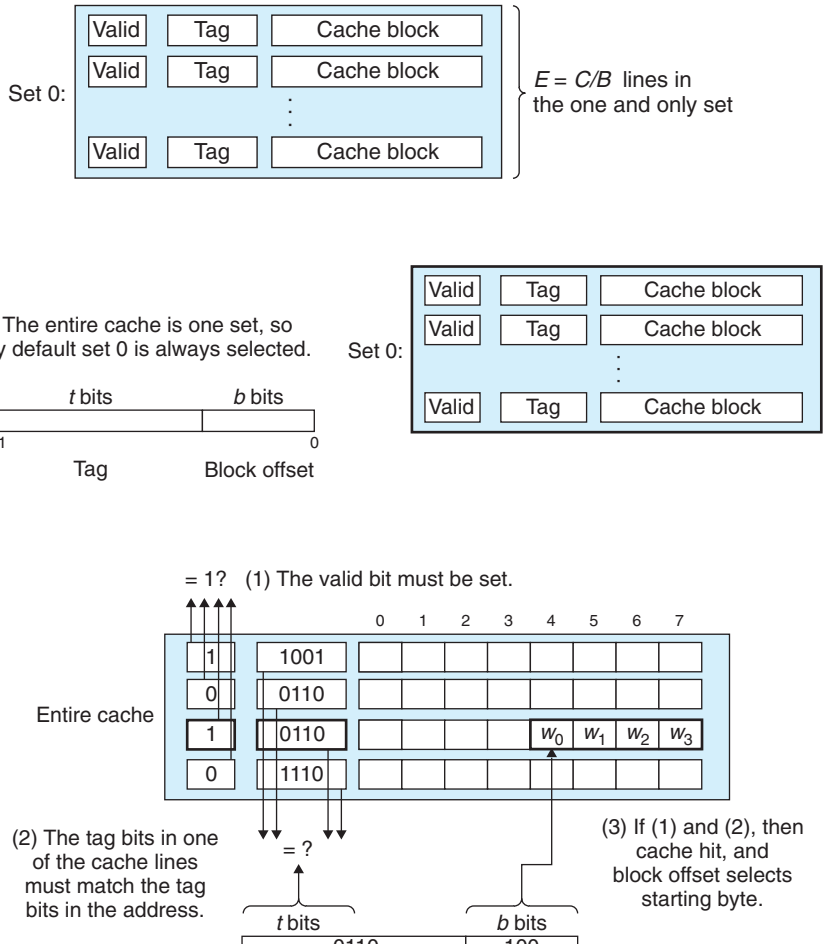


图 6.37 全相联高速缓存中的行匹配和字选择。

12 11 10 9 8 7 6 5 4 3 2 1 0

练习 6.13 假设运行在练习 6.12 那台机器上的一个程序访问地址 `0x0D53` 处的 1 字节字。指出所访问的高速缓存项，以及以十六进制表示的返回的高速缓存字节的值。指出是否发生一次高速缓存不命中。如果发生不命中，在“返回的高速缓存字节”一栏中填入“—”。

A. 地址格式(每个小格 1 位)：

12 11 10 9 8 7 6 5 4 3 2 1 0

B. 存储器访问：

参数	值
高速缓存块偏移(CO)	0x
高速缓存组下标(CI)	0x
高速缓存标记(CT)	0x
高速缓存命中?(Y/N)	
返回的高速缓存字节	0x

练习 6.14 对存储器地址 0x0CB4 重做练习 6.13。

A. 地址格式(每个小格 1 位):

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 存储器访问:

参数	值
高速缓存块偏移(CO)	0x
高速缓存组下标(CI)	0x
高速缓存标记(CT)	0x
高速缓存命中?(Y/N)	
返回的高速缓存字节	0x

练习 6.15 对存储器地址 0x0A31 重做练习 6.13。

A. 地址格式(每个小格 1 位):

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 存储器访问:

参数	值
高速缓存块偏移(CO)	0x
高速缓存组下标(CI)	0x
高速缓存标记(CT)	0x
高速缓存命中?(Y/N)	
返回的高速缓存字节	0x

练习 6.16　对于练习 6.12 中的高速缓存,列出所有会在组 3 中命中的十六进制存储器地址。

6.4.5 有关写的问题

我们已经看到,就读操作而言,高速缓存的工作直截了当。首先,在高速缓存中查找想要的字 w 的副本。如果命中,立刻返回 w。如果不命中,就从存储器层次结构的下一层取出包含 w 的块,把块存储在某个高速缓存行中(可能会驱逐一行有效行),然后返回 w。

写操作的情形要稍微复杂一些。假设我们要写一个已经缓存了副本的字 w(一次写命中)。在高速缓存更新了自己保存的 w 副本之后,对于更新层次结构下一层里 w 的副本,它该怎么处理呢? 最简单的方法称为直写(write-through),即立即把 w 的高速缓存块写到下一层。直写虽然简单,缺点却是每次写都会引发一次总线流量。另一种方法称为写回(write-back),它把更新尽可能地往后拖延,只有当替换算法把更新过的块从高速缓存驱逐出去时,才把块写到下一层。由于局部性,写回可以显著减少总线流量,缺点是增加了复杂性:高速缓存必须为每个高速缓存行维护一个额外的脏位(dirty bit),指示这个高速缓存块是否已被修改。

另一个问题是如何处理写不命中。一种方法称为写分配(write-allocate):先把相应的块从下一层加载进高速缓存,然后更新这个高速缓存块。写分配试图利用写的空间局部性,缺点是每次不命中都会导致一次从下一层到高速缓存的块传送。另一种替代方法称为非写分配(no-write-allocate):绕开高速缓存,把字直接写到下一层。直写的高速缓存通常是非写分配的;写回的高速缓存通常是写分配的。

针对写操作优化高速缓存是一个微妙而困难的问题,我们在这里只是稍有触及。细节因系统而异,而且常常是专有的,文档记录得也不完善。对于想编写对高速缓存相当友好的程序的程序员,我们建议采用一个假定高速缓存为写回、写分配的思考模型(mental model)。提出这条建议有几个理由:作为一条规

则，存储器层次结构中更低层的高速缓存更可能采用写回而不是直写，因为传送时间更长。例如，虚拟内存系统（用主存作为存放在磁盘上的块的缓存）就只用写回。但随着逻辑密度的提高，写回复杂性增加的烦恼正在变小，我们看到现代系统在所有层次上都使用写回高速缓存。所以这个假设符合当前趋势。假定写回、写分配的另一个理由是它处理写的方式与读是对称的，也即写回写分配置力于利用局部性。这样，我们就可以在高层面上把程序开发得具有良好的空间局部性和时间局部性，而不必针对某个特定的存储器系统去优化。

6.4.6 一个真实的高速缓存层次结构的解剖

到目前为止，我们一直假设高速缓存只保存程序数据。但实际上，高速缓存既保存数据，也保存指令。只保存指令的高速缓存称为 i-cache。只保存程序数据的高速缓存称为 d-cache。既保存指令又保存数据的高速缓存称为统一高速缓存（unified cache）。现代处理器包含独立的 i-cache 和 d-cache。这样做有若干理由：有两个独立的高速缓存，处理器可以同时读取一个指令字和一个数据字。i-cache 通常是只读的，因而更简单。两个高速缓存通常针对不同的访问模式做了优化，可以有不同的块大小、相联度和容量。此外，独立的高速缓存确保数据访问不会与指令访问产生冲突不命中，反之亦然，代价是容量不命中可能增加。

图 6.38 展示了 Intel Core i7 处理器的高速缓存层次结构。每颗 CPU 芯片有 4 个核。每个核有自己的私有 L1 i-cache、L1 d-cache 和 L2 统一高速缓存。所有的核共享一个片上 L3 统一高速缓存。这个层次结构有一个有趣的特性：所有的 SRAM 高速缓存存储器都封装在 CPU 芯片内。

图 6.39 归纳了 Core i7 高速缓存的基本特性。

6.4.7 高速缓存参数的性能影响

高速缓存的性能用若干指标来评价：

- **不命中率（miss rate）**。在一个程序（或程序的某一部分）的执行过程中，不命中的存储器访问所占的比例。计算方法是 # 不命中次数 / # 访问次数。
- **命中率（hit rate）**。命中的存储器访问所占的比例。计算方法是 1− 不命中率。
- **命中时间（hit time）**。把一个字从高速缓存传送给 CPU 所需的时间，包括组选择的时间、行确认的时间和字选择的时间。对 L1 高速缓存而言，命中时间是几个时钟周期的量级。

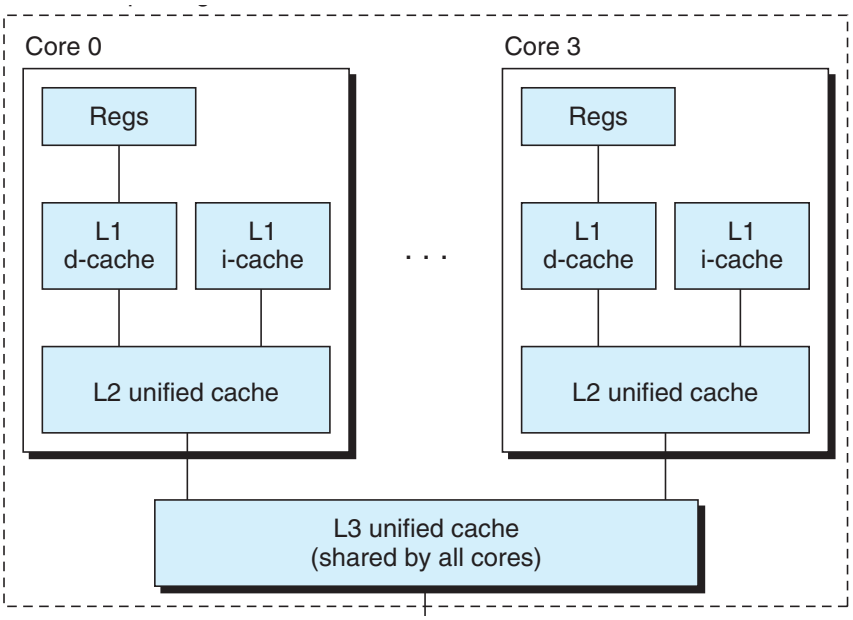


图 6.38 Intel Core i7 的高速缓存层次结构。

图 6.39 Intel Core i7 高速缓存层次结构的特性。

高速缓存类型	访问时间(周期)	高速缓存大小(C)	相联度(E)	块大小(B)	组数(S)
L1 i-cache	4	32 KB	8	64 B	64
L1 d-cache	4	32 KB	8	64 B	64
L2 统一高速缓存	10	256 KB	8	64 B	512
L3 统一高速缓存	40–75	8 MB	16	64 B	8,192

- **不命中罚时(miss penalty)**。由于一次不命中而需要的任何额外时间。从 L2 取数处理 L1 不命中的罚时约为 10 个周期;从 L3 取数约为 50 个周期;从主存取数约为 200 个周期。

优化高速缓存存储器的成本与性能折中是一件微妙的事情,需要在现实的基准测试程序上做大量的仿真,因而超出了本书范围。不过,还是可以识别出一些定性的折中。

高速缓存大小的影响

一方面,较大的高速缓存倾向于提高命中率。另一方面,让大容量的存储器跑得更快,总是更难以做到。结果是,较大的高速缓存倾向于增加命中时间。这就解释了为什么 L1 高速缓存比 L2 高速缓存小,而 L2 高速缓存又比 L3 高速缓存小。

块大小的影响

较大的块有利有弊。一方面，较大的块可以利用程序中可能存在的空间局部性来帮助提高命中率。不过，在高速缓存大小给定的情况下，较大的块意味着高速缓存行数更少，这会损害程序中时间局部性多于空间局部性情形的命中率。较大的块还会对不命中罚时造成负面影响，因为块越大传送时间越长。Core i7 这类现代系统的折中办法，是采用 64 字节的高速缓存块。

相联度的影响

这里要考虑的问题是选择参数 E（每组中高速缓存行的数目）所带来的影响。提高相联度（也就是取较大的 E 值）的优点，是降低了高速缓存因冲突不命中而发生抖动的脆弱性。不过，提高相联度要付出不小的代价。高相联度实现起来造价高，而且难以做快。它需要每行有更多的标记位、每行额外的 LRU 状态位，还有更多的控制逻辑。提高相联度会增加命中时间，因为复杂度增加了；它也会增加不命中罚时，因为选择替换行的复杂度增加了。

相联度的选择最终归结为命中时间与不命中罚时之间的折中。传统上，追求高时钟速率的高性能系统会为 L1 高速缓存选择较低的相联度（这里的不命中罚时只有几个周期），而对罚时更高的较低层次选择较高的相联度。例如，Intel Core i7 系统中，L1 和 L2 高速缓存为 8 路相联，L3 高速缓存为 16 路。

写策略的影响

直写高速缓存实现起来更简单，而且可以使用一个独立于高速缓存工作的写缓冲区来更新存储器。此外，读不命中也比较便宜，因为它们不会触发存储器写操作。另外，写回高速缓存造成的传送更少，这给执行 DMA 的 I/O 设备留出更多到存储器的带宽。而且，当我们在层次结构中向下移动、传送时间变得更大时，减少传送次数就变得越来越重要。一般而言，处于层次结构中越低位置的高速缓存，越可能使用写回而不是直写。

6.5 编写高速缓存友好的代码

6.2 节用定性的说法介绍了局部性的概念，讲了什么样的局部性算好局部性。现在我们理解了高速缓存的工作原理，可以讲得更精确。局部性更好的程序倾向于具有更低的不命中率，而不命中率更低的程序倾向于比不命中率更高的程序运行得更快。因此，优秀的程序员应当时刻尽力去编写具有良好局部性的、对高速缓存友好的代码。下面是我们用来保证代码对高速缓存友好的基本做法。

边栏 高速缓存行、组和块：有什么区别？

高速缓存行、组和块之间的区别很容易混淆。我们来重温这些概念，确保它们是清楚的：

- 块是一种固定大小的信息包，在高速缓存和主存（或更低层的高速缓存）之间来回传送。
- 行是高速缓存中的容器，它存储一个块，另外还存有有效位和标记位之类的其他信息。
- 组是一行或多行的集合。直接映射高速缓存中的组只有一行；组相联和全相联高速缓存中的组包含多行。

在直接映射高速缓存中，组和行确实等价。但在相联高速缓存中，组与行是很不同的两个概念，这两个术语不能互换使用。

由于一行总是恰好存储一个块，“行”和“块”这两个术语常常被互换使用。例如，系统专业人士通常说高速缓存的“行大小”，而他们实际指的是块大小。这种用法非常常见，只要理解了块和行的区别，就不会造成混淆。

1. 让常见情况走得快。程序往往把大部分时间花在少数几个核心函数上，这些函数又往往把大部分时间花在少数几个循环上。所以要专注于核心函数的内层循环，忽略其余部分。
2. 把每个内层循环中的高速缓存不命中次数降到最少。在其他条件（例如加载和存储的总数）相同的情况下，不命中率更好的循环运行得更快。

为了看看这个方法在实践中是如何运用的，考虑 6.2 节的 `sumvec` 函数：

```
1  int sumvec(int v[N])
2  {
3      int i, sum = 0;
4
5      for (i = 0; i < N; i++)
6          sum += v[i];
7      return sum;
8  }
```

这个函数对高速缓存友好吗？首先，注意到循环体内对局部变量 i 和 `sum` 的时间局部性很好。实际上，由于它们是局部变量，任何像样的优化编译器都会把它们缓存在寄存器堆中——存储器层次结构的最高层。现在考虑对向量 v 的步长为 1 的引用。一般而言，如果高速缓存的块大小为 B 字节，那么步长为 k（这里 k 以字为单位）的引用模式每次循环迭代平均产生 $\min(1, (\text{字大小} \times k)/B)$ 次不命中。在 $k = 1$ 时这个值最小，所以对 v 的步长为 1 的引用确实是高速缓存友好的。例如，假设 v 是块对齐的，字为 4 字节，高速缓存块为 4 个字，且高速缓存初始为空（冷缓存，cold cache）。那么，无论高速缓存的组织结构如何，对 v 的引用都会产生如下的命中与不命中模式：

v[i]	$i=0$	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$
访问顺序,[h] 命中,[m] 不命中	1 [m]	2 [h]	3 [h]	4 [h]	5 [m]	6 [h]	7 [h]	8 [h]

在这个例子中,对 v[0] 的引用不命中,包含 v[0] ~ v[3] 的相应块从存储器装载到高速缓存。于是接下来三次引用都是命中。对 v[4] 的引用引发又一次不命中,因为一个新的块被装载进高速缓存,接下来三次引用又是命中,以此类推。一般而言,四次引用中有三次命中,这是在冷缓存的情况下所能达到的最好结果。

总之,这个简单的 sumvec 例子阐明了编写高速缓存友好代码的两个要点:

- 对局部变量的重复引用是好的,因为编译器可以把它们缓存在寄存器堆中(时间局部性)。
- 步长为 1 的引用模式是好的,因为存储器层次结构所有层次上的高速缓存都以连续块的方式存储数据(空间局部性)。

空间局部性在处理多维数组的程序中尤其重要。例如,考虑 6.2 节的 sumarrayrows 函数,它按行优先的次序对一个二维数组求和:

```
1  int sumarrayrows(int a[M][N])
2  {
3      int i, j, sum = 0;
4
5      for (i = 0; i < M; i++)
6          for (j = 0; j < N; j++)
7              sum += a[i][j];
8      return sum;
9  }
```

由于 C 语言按行优先次序存储数组,该函数的内层循环具有和 sumvec 一样理想的步长 1 访问模式。例如,假设我们对高速缓存作与 sumvec 相同的假设,那么对数组 a 的引用会产生如下的命中与不命中模式:

a[i][j]	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1 [m]	2 [h]	3 [h]	4 [h]	5 [m]	6 [h]	7 [h]	8 [h]
$i=1$	9 [m]	10 [h]	11 [h]	12 [h]	13 [m]	14 [h]	15 [h]	16 [h]
$i=2$	17 [m]	18 [h]	19 [h]	20 [h]	21 [m]	22 [h]	23 [h]	24 [h]
$i=3$	25 [m]	26 [h]	27 [h]	28 [h]	29 [m]	30 [h]	31 [h]	32 [h]

但请考虑,如果我们对循环的次序做调换,即进行一个看似无关痛痒的改变,会发生什么:

```
1  int sumarraycols(int a[M][N])
```

```
2  {
3      int i, j, sum = 0;
4
5      for (j = 0; j < N; j++)
6          for (i = 0; i < M; i++)
7              sum += a[i][j];
8      return sum;
9  }
```

这里我们改为按列而不是按行逐个扫描数组。如果运气好,整个数组能放进高速缓存,那么我们还可以享受同样的 1/4 不命中率。然而,如果数组大于高速缓存(这是更可能的情形),那么对 `a[i][j]` 的每一次访问都会不命中!

`a[i][j]`	$j=0$	$j=1$	$j=2$	$j=3$	$j=4$	$j=5$	$j=6$	$j=7$
$i=0$	1 [m]	5 [m]	9 [m]	13 [m]	17 [m]	21 [m]	25 [m]	29 [m]
$i=1$	2 [m]	6 [m]	10 [m]	14 [m]	18 [m]	22 [m]	26 [m]	30 [m]
$i=2$	3 [m]	7 [m]	11 [m]	15 [m]	19 [m]	23 [m]	27 [m]	31 [m]
$i=3$	4 [m]	8 [m]	12 [m]	16 [m]	20 [m]	24 [m]	28 [m]	32 [m]

更高的不命中率会对运行时间产生显著影响。例如,在我们的台式机上,对较大的数组,`sumarrayrows` 的运行速度是 `sumarraycols` 的 25 倍。总之,程序员应该了解自己程序中的局部性,并尽力编写能利用局部性的程序。

练习 6.17 转置一个矩阵的行和列是信号处理和科学计算中的一个重要问题。从局部性的角度看这个题目也很有意思,因为它的引用模式既是按行的又是按列的。例如,考虑下面的转置例程:

```
1  typedef int array[2][2];
2
3  void transpose1(array dst, array src)
4  {
5      int i, j;
6
7      for (i = 0; i < 2; i++) {
8          for (j = 0; j < 2; j++) {
9              dst[j][i] = src[i][j];
10             }
11         }
12 }
```

假设这段代码运行在一台具有下列特性的机器上:

- `sizeof(int)` $= 4$。
- `src` 数组从地址 0 开始,`dst` 数组从地址 16(十进制)开始。
- 有一个唯一的 L1 数据高速缓存,它是直接映射、直写、写分配的,块大小为 8 字节。

- 高速缓存的数据总大小为 16 字节,且初始为空。
- 对 `src` 和 `dst` 数组的访问分别是读不命中和写不命中的唯一来源。

A. 对每一行 row 和每一列 col，指出对 `src[row][col]` 和 `dst[row][col]` 的访问是命中(h)还是不命中(m)。例如,读 `src[0][0]` 是一次不命中,写 `dst[0][0]` 也是一次不命中。

	dst 数组		src 数组	
行	列 0	列 1	列 0	列 1
行 0	m		m	
行 1				

B. 对一个数据字节数为 32 的高速缓存重做这道题。

练习 6.18 近来热门游戏 SimAquarium 的核心是一个紧凑循环，它计算 512 株水藻的平均位置。你正在一台机器上评估它的高速缓存性能,该机器有一个 2048 字节的直接映射数据高速缓存,块大小为 32 字节($B = 32$)。已知如下定义:

```
1  struct algae_position {
2      int x;
3      int y;
4  };
5
6  struct algae_position grid[32][32];
7  int total_x = 0, total_y = 0;
8  int i, j;
```

你还应当假定下列条件成立:

- `sizeof(int)` $= 4$。
- `grid` 从存储器地址 0 开始。
- 高速缓存初始为空。
- 仅有的存储器访问是对数组 `grid` 元素的访问。变量 `i`、`j`、`total_x` 和 `total_y` 存放在寄存器中。

对下面的代码求其高速缓存性能:

```
1  for (i = 31; i >= 0; i--) {
2      for (j = 31; j >= 0; j--) {
3          total_x += grid[i][j].x;
4      }
5  }
6
7  for (i = 31; i >= 0; i--) {
```

```
8       for (j = 31; j >= 0; j--) {
9           total_y += grid[i][j].y;
10      }
11  }
```

A. 读操作的总数是多少？

B. 在缓存中会不命中的读操作总数是多少？

C. 不命中率是多少？

练习 6.19 在练习题 6.18 的假定条件下，确定下面代码的高速缓存性能：

```
1  for (i = 31; i >= 0; i--){
2       for (j = 31; j >= 0; j--) {
3           total_x += grid[j][i].x;
4           total_y += grid[j][i].y;
5       }
6  }
```

A. 读操作的总数是多少？

B. 在缓存中会命中的读操作总数是多少？

C. 命中率是多少？

D. 如果缓存的大小扩大一倍，命中率会是多少？

练习 6.20 在练习题 6.18 的假定条件下，确定下面代码的高速缓存性能：

```
1  for (i = 31; i >= 0; i--){
2       for (j = 31; j >= 0; j--) {
3           total_x += grid[i][j].x;
4           total_y += grid[i][j].y;
5       }
6  }
```

A. 读操作的总数是多少？

B. 在缓存中会命中的读操作总数是多少？

C. 命中率是多少？

D. 如果缓存的大小扩大一倍，命中率会是多少？

6.6 综合：高速缓存对程序性能的影响

这一节通过研究高速缓存对程序在真实机器上运行时性能的影响，来结束我们对存储器层次结构的讨论。

6.6.1 存储器山

程序从存储器系统读出数据的速率称为读吞吐量（read throughput），有时也称读带宽（read bandwidth）。如果程序在 s 秒内读出 n 字节，那么这段时间里的读吞吐量就是 n/s，通常以兆字节每秒（MB/s）为单位。

如果我们写一个程序，让它在一个紧凑的程序循环中发出一串读请求，测得的读吞吐量便能让我们了解存储器系统对这一特定读序列的性能。图 6.40 给出的一对函数用来测量某个特定读序列的读吞吐量。

`test` 函数以步长 stride 扫描数组的前 elems 个元素，从而生成读序列。为了增加内循环中可用的并行性，它采用了 4×4 循环展开（见 5.9 节）。`run` 函数是一个包装（wrapper）函数，它调用 `test` 函数并返回测得的读吞吐量。第 37 行对 `test` 函数的调用会预热缓存。第 38 行的 `fcyc2` 函数以 elems 和 stride 为参数调用 `test` 函数，并以 CPU 周期为单位估计 `test` 函数的运行时间。注意，`run` 函数的 size 参数以字节为单位，而与它对应的 `test` 函数的 elems 参数以数组元素为单位。还要注意，第 39 行把 MB/s 按 10^6 字节/秒来计算，而不是 2^{20} 字节/秒。

`run` 函数的 size 和 stride 参数让我们能够控制所产生的读序列的时间局部性和空间局部性的程度。size 值越小，工作集就越小，因而时间局部性越好。stride 值越小，空间局部性就越好。如果我们用不同的 size 值和 stride 值反复调用 `run` 函数，就能得到一个有趣的二维函数：以时间局部性和空间局部性为自变量的读吞吐量。这个函数称为存储器山（memory mountain）[112]。

每台计算机都有它自己独有的存储器山，表征它存储器系统的能力。例如，图 6.41 给出了一台 Intel Core i7 Haswell 系统的存储器山。在本例中，size 的范围从 16 KB 到 128 MB，stride 的范围从 1 到 12 个元素，每个元素是一个 8 字节的 long int（长整型）。

```
code/mem/mountain/mountain.c
1
long data[MAXELEMS];
/* The global array we'll be traversing */
2
3
/* test - Iterate over first "elems" elements of array "data" with
4
*
stride of "stride", using 4 x 4 loop unrolling.
5
*/
6
int test(int elems, int stride)
7
{
8
long i, sx2 = stride*2, sx3 = stride*3, sx4 = stride*4;
9
long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
10
long length = elems;
11
```

```
long limit = length - sx4;
12
13
/* Combine 4 elements at a time */
14
for (i = 0; i < limit; i += sx4) {
15
acc0 = acc0 + data[i];
16
acc1 = acc1 + data[i+stride];
17
acc2 = acc2 + data[i+sx2];
18
acc3 = acc3 + data[i+sx3];
19
}
20
21
/* Finish any remaining elements */
22
for (; i < length; i++) {
23
acc0 = acc0 + data[i];
24
}
25
return ((acc0 + acc1) + (acc2 + acc3));
26
}
27
28
/* run - Run test(elems, stride) and return read throughput (MB/s).
29
*
"size" is in bytes, "stride" is in array elements, and Mhz is
30
*
CPU clock frequency in Mhz.
31
*/
32
double run(int size, int stride, double Mhz)
33
{
34
double cycles;
35
int elems = size / sizeof(double);
36
37
test(elems, stride);
/* Warm up the cache */
38
cycles = fcyc2(test, elems, stride, 0);
/* Call test(elems,stride) */
```

```
39
return (size / stride) / (cycles / Mhz); /* Convert cycles to MB/s */
40
}
code/mem/mountain/mountain.c
```

图 6.40　测量并计算读吞吐量的函数。对一个特定的计算机，用不同的 size（对应于时间局部性）和 stride（对应于空间局部性）值调用 `run` 函数，就能生成一个存储器山。

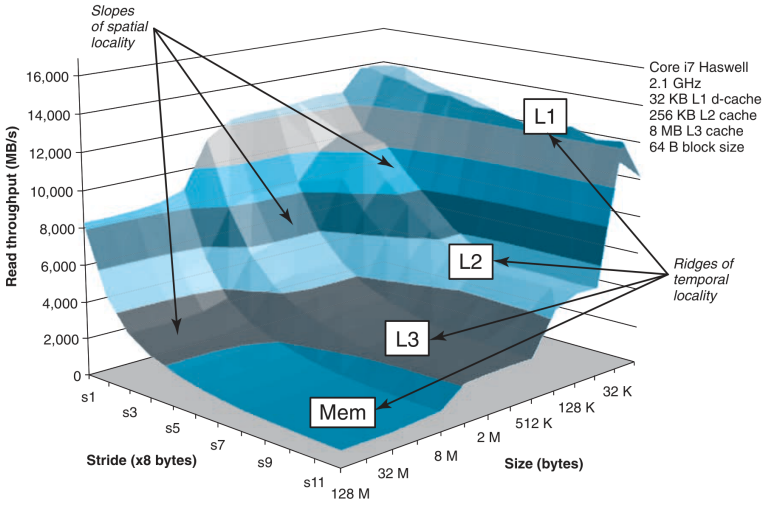


Figure 6.41 **A memory mountain.** Shows read throughput as a function of temporal and spatial locality.

The geography of the Core i7 mountain reveals a rich structure. Perpendicular to the `size` axis are four *ridges* that correspond to the regions of temporal locality where the working set fits entirely in the L1 cache, L2 cache, L3 cache, and main memory, respectively. Notice that there is more than an order of magnitude difference between the highest peak of the L1 ridge, where the CPU reads at a rate of over 14 GB/s, and the lowest point of the main memory ridge, where the CPU reads at a rate of 900 MB/s.

On each of the L2, L3, and main memory ridges, there is a slope of spatial locality that falls downhill as the stride increases and spatial locality decreases. Notice that even when the working set is too large to fit in any of the caches, the highest point on the main memory ridge is a factor of 8 higher than its lowest point. So even when a program has poor temporal locality, spatial locality can still come to the rescue and make a significant difference.

There is a particularly interesting flat ridge line that extends perpendicular to the stride axis for a stride of 1, where the read throughput is a relatively flat 12 GB/s, even though the working set exceeds the capacities of L1 and L2. This is apparently due to a hardware *prefetching* mechanism in the Core i7 memory system that automatically identifies sequential stride-1 reference patterns and attempts to fetch those blocks into the cache before they are accessed. While the

图 6.41
quad 存储器山的几何形态。

Core i7 存储器山的地理形态揭示出一种丰富的结构。垂直于 size 轴的方向上有四道山脊，分别对应于工作集能完整放入 L1 缓存、L2 缓存、L3 缓存和主存的时间局部性区域。注意，L1 山脊的最高峰（CPU 以超过 14 GB/s 的速率

读数据）与主存山脊的最低点（CPU 以 900 MB/s 的速率读数据）之间相差超过一个数量级。

在 L2、L3 和主存每道山脊上，都有一道空间局部性的斜坡，随着步长增大、空间局部性减弱而向下倾斜。注意，即使工作集大到放不进任何缓存，主存山脊的最高点也比最低点高 8 倍。所以，即使程序的时间局部性很差，空间局部性仍然能救场，带来显著的差别。

有一道特别有趣的平坦山脊线：在步长为 1 时它垂直于步长轴延伸，读吞吐量保持在大约 12 GB/s 左右的平缓水平，尽管此时工作集已经超出了 L1 和 L2 的容量。这显然要归功于 Core i7 存储器系统中的硬件预取机制，它会自动识别顺序的步长为 1 的访问模式，并尝试在这些块被访问之前把它们预取到缓存中。虽然这种预取算法的具体细节没有公开，但从存储器山可以清楚地看出，该算法对小步长的效果最好——这又多了一个在你的代码中偏向顺序的步长为 1 的访问的理由。

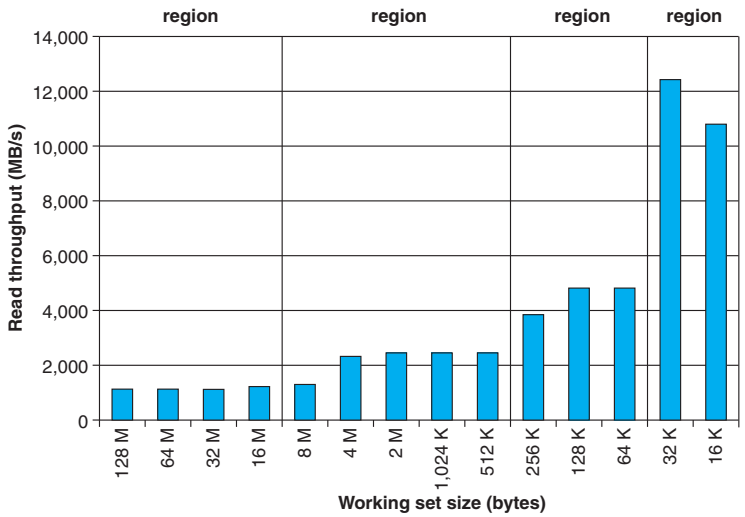


Figure 6.42 **Ridges of temporal locality in the memory mountain.** The graph shows a slice through Figure 6.41 with `stride` = 8.

图 6.42

如果我们沿存储器山切一刀，固定步长不变（如图 6.42 所示），就能看出缓存大小和时间局部性对性能的影响。对于大小不超过 32 KB 的情况，工作集可以完整装进 L1 数据缓存，因此读请求直接由 L1 服务，吞吐量约为 12 GB/s。对于大小不超过 256 KB 的情况，工作集可以完整装进统一的 L2 缓存；对于大小不超过 8 MB 的情况，工作集可以完整装进统一的 L3 缓存。更大的工作集主要由主存服务。

有意思的是，在 L2 缓存区域和 L3 缓存区域最左边缘处读吞吐量的下陷（工作集大小为 256 KB 和 8 MB，恰好分别等于这两个缓存的容量）。这些下陷出现的确切原因还不清楚。要弄明白，唯一可靠的办法是进行详细的缓存仿真，不过这些下降很可能是由与其他代码和数据行的冲突造成的。

沿反方向切存储器山——固定工作集大小不变——可以让我们洞察空间局部性对读吞吐量的影响。例如，图 6.43 给出了工作集大小固定为 4 MB 时的切面。这个切面是沿图 6.41 中的 L3 山脊切出来的，这里工作集可以完整放进 L3 缓存，但是对 L2 缓存来说太大了。

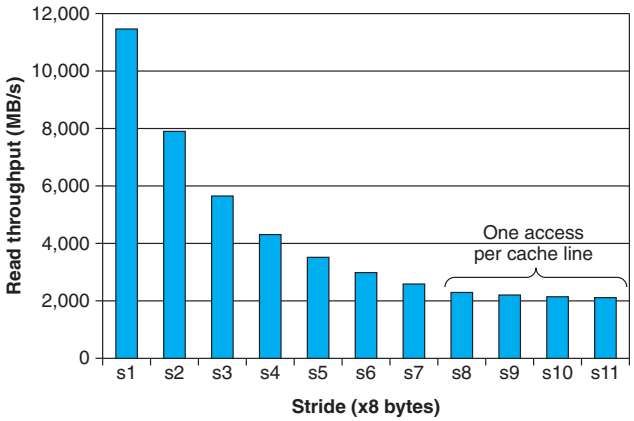


Figure 6.43 **A slope of spatial locality.** The graph shows a slice through Figure 6.41 with `size` = 4 MB.

图 6.43

注意，随着步长从 1 个字增加到 8 个字，读吞吐量会稳定下降。在存储器山的这一区域，位于 L2 上的一次读不命中导致一个块从 L3 传送到 L2。随后是在 L2 中对该块的若干次命中，其次数取决于步长。随着步长的增大，L2 不命中与 L2 命中之比增大。由于不命中比命中慢，读吞吐量随之下降。一旦步长达到 8 个 8 字节的字——在这台系统上恰好等于 64 字节的块大小——每次读请求都会在 L2 中不命中，必须从 L3 服务。因此，步长至少为 8 时，读吞吐量是一个常数速率，由缓存块从 L3 传送到 L2 的速率决定。

对存储器山讨论做个总结：存储器系统的性能不能只用一个数字来描述。它是一座由时间局部性和空间局部性组成的大山，高度的变化能超过一个数量级。有智慧的程序员会努力组织自己的程序，让它在山峰上运行而不是在山谷中。目标是利用时间局部性，使得频繁使用的字从 L1 缓存中取出；利用空间局部性，使得尽可能多的字在同一条 L1 缓存行上被访问。

练习 6.21　用图 6.41 中的存储器山估算从 L1 数据缓存读出一个 16 字节的字所需要的时间（以 CPU 周期为单位）。

6.6.2 重新排列循环以提高空间局部性

考虑两个 $n \times n$ 矩阵相乘的问题：$C = AB$。例如，当 $n = 2$ 时

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

其中

$$\begin{aligned} c_{11} &= a_{11}b_{11} + a_{12}b_{21} \\ c_{12} &= a_{11}b_{12} + a_{12}b_{22} \\ c_{21} &= a_{21}b_{11} + a_{22}b_{21} \\ c_{22} &= a_{21}b_{12} + a_{22}b_{22} \end{aligned}$$

矩阵乘法函数通常用三个嵌套的循环实现，分别由索引 i、j 和 k 标识。如果重排这些循环并做一些其他次要的代码改动，我们就能得到图 6.44 所示的六个功能上等价的矩阵乘法版本。每个版本都由它的循环顺序唯一标识。

在较高的抽象层面上看，六个版本非常相似。如果加法满足结合律，那么每个版本计算出一个相同的结果。[1]每个版本执行 $O(n^3)$ 次总操作，加法和乘法的次数相同。A 和 B 的 n^2 个元素每个被读取 n 次。C 的 n^2 个元素每个要累加 n 个值。然而，如果我们分析最内层循环迭代的行为，就会发现各版本在要访问的次数（load/store 次数）和局部性方面有差别。为了做这个分析，假设如下：

- 每个数组都为 $n \times n$ 的 double 数组，且 sizeof(double) $= 8$。
- 只有单一一个缓存，块大小为 32 字节（$B = 32$）。
- 数组大小 n 大到矩阵的一行都放不进 L1 缓存。
- 编译器把局部变量保存在寄存器中，因而循环内对局部变量的引用不需要任何装载或存储指令。

图 6.45 汇总了内循环分析的结果。注意，六个版本组成三对等价类，我们用在最内层循环中被访问的那对矩阵来命名。例如，版本 `ijk` 和 `jik` 属于 AB

[1]在第 2 章中我们讲过，浮点数加法满足交换律，但是一般不满足结合律。实际中，如果矩阵不把极大的数值同极小的数值混在一起（当矩阵存放物理量时通常正是如此），那么结合律的假设是合理的。

类，因为它们的内循环引用的是数组 A 和 B（不引用 C）。对每个类，我们统计了每次内循环迭代中装入（读）和存储（写）的次数，每次循环迭代中对 A、B、C 的引用中会在缓存中不命中的次数，以及每次迭代的缓存不命中总数。

AB 类（图 6.44(a) 和 (b)）的内循环以步长 1 扫描数组 A 的一行。由于每个缓存块容纳四个 8 字节字，A 的不命中率是每次迭代 0.25 次不命中。另一方面，内循环以步长 n 扫描 B 的一列。由于 n 很大，每次对数组 B 的访问都会不命中，所以每次迭代总共 1.25 次不命中。

AC 类（图 6.44(c) 和 (d)）的内循环有一些问题。每次迭代执行两次装入和一次存储（AB 类例行程序则是两次装入、不存储）。此外，内循环以步长 n 扫描 A 和 C 的列。其结果是每次装入都不命中，总共每次迭代 2 次不命中。注意，交换循环会降低相对于 AB 类例行程序的空间局部性。

(a) ijk 版本
code/mem/matmult/mm.c

```
1  for (i = 0; i < n; i++)
2      for (j = 0; j < n; j++) {
3          sum = 0.0;
4          for (k = 0; k < n; k++)
5              sum += A[i][k]*B[k][j];
6          C[i][j] += sum;
7      }
```

code/mem/matmult/mm.c

(b) jik 版本
code/mem/matmult/mm.c

```
1  for (j = 0; j < n; j++)
2      for (i = 0; i < n; i++) {
3          sum = 0.0;
4          for (k = 0; k < n; k++)
5              sum += A[i][k]*B[k][j];
6          C[i][j] += sum;
7      }
```

code/mem/matmult/mm.c

(c) jki 版本
code/mem/matmult/mm.c

```
1  for (j = 0; j < n; j++)
2      for (k = 0; k < n; k++) {
3          r = B[k][j];
4          for (i = 0; i < n; i++)
5              C[i][j] += A[i][k]*r;
6      }
```

code/mem/matmult/mm.c

(d) kji 版本
code/mem/matmult/mm.c

```
1  for (k = 0; k < n; k++)
2      for (j = 0; j < n; j++) {
3          r = B[k][j];
4          for (i = 0; i < n; i++)
5              C[i][j] += A[i][k]*r;
6      }
```
code/mem/matmult/mm.c

(e) kij 版本

```
code/mem/matmult/mm.c
1  for (k = 0; k < n; k++)
2      for (i = 0; i < n; i++) {
3          r = A[i][k];
4          for (j = 0; j < n; j++)
5              C[i][j] += r*B[k][j];
6      }
```
code/mem/matmult/mm.c

(f) ikj 版本

```
code/mem/matmult/mm.c
1  for (i = 0; i < n; i++)
2      for (k = 0; k < n; k++) {
3          r = A[i][k];
4          for (j = 0; j < n; j++)
5              C[i][j] += r*B[k][j];
6      }
```
code/mem/matmult/mm.c

图 6.44　矩阵乘法的六个版本。每个版本都由其循环顺序唯一标识。

图 6.45　矩阵乘法内循环的分析。这六个版本分成三个等价类，用在最内层循环中被访问的那对数组来命名。

矩阵乘法版本(等价类)	装入(每次)	存储(每次)	A 不命中	B 不命中	C 不命中	总不命中
ijk & jik(AB)	2	0	0.25	1.00	0.00	1.25
jki & kji(AC)	2	1	1.00	0.00	1.00	2.00
kij & ikj(BC)	2	1	0.00	0.25	0.25	0.50

BC 类(图 6.44(e) 和 (f))表现出一个有趣的折中：它要执行两次装入和一次存储，比 AB 类例行程序多一次访存操作。另一方面，由于内循环以步长 1 的方式按行扫描 B 和 C，每个数组的不命中率只有每次迭代 0.25 次，总共每次迭代 0.50 次不命中。

图 6.46 总结了矩阵乘法的不同版本在一个 Core i7 系统上的性能。图中画出的是每次内循环实测的 CPU 周期数随数组大小 n 变化的曲线。

关于这张图，有几个值得注意的有趣要点：

- **对于大的 n，最快的版本几乎比最慢的版本快 40 倍，即便每个版本执行相同次数的浮点算术运算。**

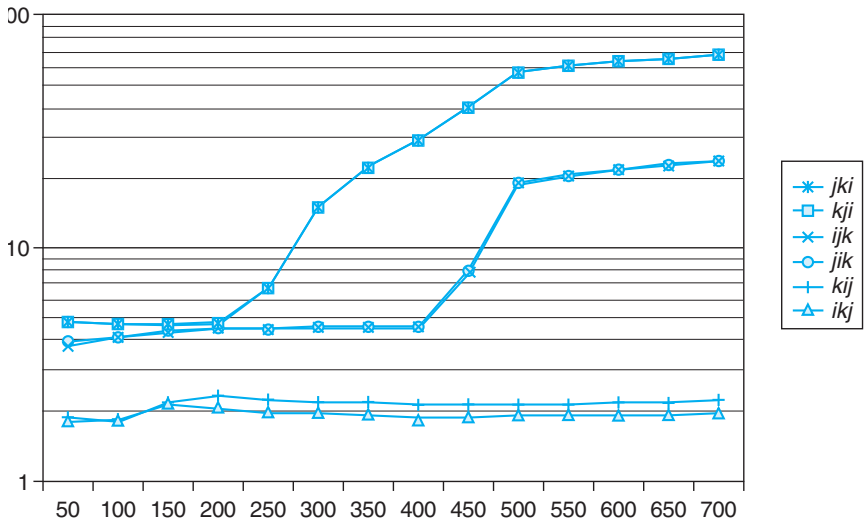


图 6.46　Core i7 矩阵乘法的性能。

- 访存次数和每次迭代不命中次数都相同的版本对，实测性能几乎相同。
- 在存储器行为上最差的两个版本（以每次迭代的访问数和不命中数来衡量）明显比其他四个版本慢，后者的不命中更少、访问更少或者两者都少。
- 就预测性能而言，不命中率比访存总次数更有用。例如，BC 类例行程序每次迭代有 0.5 次不命中，性能远好于 AB 类例行程序（每次迭代 1.25 次不命中），尽管 BC 类例行程序在内循环中执行的访存多得多（两次装入和一次存储，而 AB 类只有两次装入）。
- 对于大的 n，最快的版本对（`kij` 和 `ikj`）的性能是恒定不变的。虽然数组远大于任何 SRAM 缓存存储器，但预取硬件足够聪明，能够识别步长为 1 的访问模式，又足够快，能跟得上紧凑的内循环中的访存。这是设计这套存储器系统的 Intel 工程师的一项惊人成就，它进一步激励程序员去开发具有良好空间局部性的程序。

网络边栏　MEM:BLOCKING　用阻塞来增加时间局部性

有一种叫作阻塞（blocking）的有趣技术可以提高内循环的时间局部性。阻塞的基本思想是把程序中的数据结构组织成大的分块，称为块（block）。（在这里，“块”指的是应用级的数据分块，而不是缓存块。）程序的组织方式是：把一个分块装入 L1 缓存，在这个分块上做完所有需要的读写，然后丢弃这个分块，装入下一个分块，如此反复。

与改善空间局部性的简单循环变换不同，阻塞会让代码更难读懂。因此它最适合优化编译器或频繁执行的库例程。由于 Core i7 有复杂的预取硬件，阻塞并不能改善矩

> 阵乘法在这台机器上的性能。尽管如此，这项技术仍然值得一学、值得理解，因为它是一个一般的概念，能在没有预取机制的系统上带来很大的性能提升。

6.6.3 在程序中利用局部性

我们已经看到，存储器系统被组织为存储器设备的层次结构，较高的层次上是较小、更快的设备，较低的层次上是较大、更慢的设备。由于有了这种层次结构，程序访问存储器的一个存储位置的有效速率不能用单个数字来描述。相反，它是程序局部性的一个变动剧烈的函数（我们称之为存储器山），能相差几个数量级。局部性好的程序主要从快速的缓存存储器读取数据。局部性差的程序主要从相对慢一些的 DRAM 主存读取数据。

理解存储器层次结构本质的程序员可以利用这种理解来写出更高效的程序，而不必关心对特定的存储器系统的组织。特别地，我们推荐下列技术：

- 把注意力集中在内循环上，大部分计算和访存发生在那里。
- 通过按存储器中存储的顺序、以步长 1 顺序读取数据对象，尽量使程序的空间局部性最大化。
- 通过在把一个数据对象从存储器中读出之后尽量频繁地使用它，尽量使程序的时间局部性最大化。

6.7 小结

基本的存储技术包括随机访问存储器（RAM）、非易失性存储器（ROM）和磁盘。RAM 有两种基本形式：静态 RAM（SRAM）更快，也更贵，用做高速缓存存储器；动态 RAM（DRAM）较慢，也较便宜，用做主存和图形帧缓冲区。ROM 即使掉电也能保持其信息，用做存储固件的介质。旋转磁盘是机械式的非易失性存储设备，能够以每字节的低成本持有大量的数据，但是访问时间远长于 DRAM。基于非易失性闪存的固态硬盘（SSD）对某些应用而言正在成为旋转磁盘越来越有吸引力的替代品。

一般来说，较快的存储技术在每比特的成本上也更贵，容量也较小。这些技术的价格和性能特性正以极其不同的速率变化着。特别地，DRAM 和磁盘的访问时间比 CPU 周期时间大得多。系统通过把存储器组织成存储器件的层次结构来弥合这些差距：最高的层次上是较小、更快的器件，较低的层次上是较大、更慢的器件。因为编写良好的程序具有良好的局部性，其大多数数据可以由较高的层次来访问，其效果就是存储器系统以较高层次的速率运行，而以较低层次的成本和容量运行。

程序员通过编写具有良好空间局部性和时间局部性的程序，能够显著地提高程序的运行时间。利用基于 SRAM 的高速缓存器尤为重要。主要从缓存存储器读取数据的程序比主要从存储器读取数据的程序运行得快得多。

6.8 参考文献说明

存储器和磁盘技术变化很快。根据我们的经验，最好的技术信息来源是制造商维护的网页。Micron、Toshiba 和 Samsung 这样的公司在它们的网页上提供了大量关于存储器设备的最新技术信息。Seagate 和 Western Digital 的网页也提供类似的有用的磁盘信息。

讲述电路和逻辑设计的教科书详细介绍了存储器技术 [58, 89]。IEEE Spectrum 发表了一系列关于 DRAM 的综述文章 [55]。国际计算机体系结构会议（ISCA）和高性能计算机体系结构（HPCA）是表征 DRAM 存储器性能的常见论坛 [28, 29, 18]。

Wilkes 撰写了第一篇关于缓存存储器的论文 [117]。Smith 撰写了一篇经典的综述 [104]。Przybylski 撰写了一本关于缓存设计的权威专著 [86]。Hennessy 和 Patterson 全面地讨论了缓存设计问题 [46]。Levinthal 编写了一本平装的 Intel Core i7 性能手册 [70]。

Stricker 在 [112] 中提出把存储器山作为对存储器系统的一种全面刻画，并在后来对该工作的介绍中非正式地使用了“存储器山”这个术语。编译器研究人员通过自动执行我们在 6.6 节中讨论过的那类手工代码变换来提高局部性 [22, 32, 66, 72, 79, 87, 119]。Carter 及其同事提出了一种缓存感知（cache-aware）的存储器控制器 [17]。其他研究者开发了缓存无关（cache-oblivious）的算法，设计目标是在完全不显式了解底层缓存存储器结构的情况下也能运行良好 [30, 38, 39, 9]。

关于建造和使用磁盘存储的文献有很多。许多存储研究者寻找把单个磁盘聚合为更大、更健壮、更安全的存储池的方法 [20, 40, 41, 83, 121]。另一些研究者寻找用缓存和局部性来改善磁盘访问性能的方法 [12, 21]。Exokernel 这类系统提供了用户级对磁盘和存储器资源的更多控制 [57]。Andrew 文件系统 [78] 和 Coda [94] 这类系统把存储器层次结构扩展到计算机网络和移动笔记本电脑。Schindler 和 Ganger 开发了一个有趣的工具，能自动测出 SCSI 磁盘驱动器的几何结构和性能 [95]。研究者还研究了建造和使用基于闪存的固态硬盘的技术 [8, 81]。

6.9 家庭作业

家庭作业 6.22 ▲▲ 假设想你设计一块每磁道位数恒定的旋转磁盘。已知每磁道位数由最里面一条磁道的周长决定，而你可以假设这条磁道的周长就等于中心孔的周长。这样，如果加大磁盘中心的孔，每磁道位数会增多，但磁道总数会减少。如果用 r 表示盘片半径，$x < r$ 表示孔的半径，那么 x 取什么值能使磁盘的容量最大？

家庭作业 6.23 ▲ 估算访问下列磁盘上一个扇区所需的时间（以 ms 为单位）：

家庭作业 6.23 磁盘参数

参数	值
转速	12,000 RPM
$T_{\text{avg seek}}$	3 ms
每磁道扇区数	500

家庭作业 6.24 ▲▲ 假设有一个 2 MB 的文件由 512 字节的逻辑块组成，存储在一个具有下列特性的磁盘驱动器上：

家庭作业 6.24 磁盘驱动器参数

参数	值
转速	18,000 RPM
$T_{\text{avg seek}}$	8 ms
每磁道扇区数	2,000
盘面数	4
扇区大小	512 字节

对下面的每种情况，假设一个程序按顺序依次读取文件的逻辑块，并且把磁头定位到第一个块上的时间等价于 $T_{\text{avg seek}}$ 与 $T_{\text{avg rotation}}$ 之和。则：

A. 最好情况：估算在逻辑块被映射到磁盘扇区的最佳情况下（即顺序读取）读取文件所需的最短时间（以 ms 为单位）。

B. 随机情况：估算当各块被随机映射到磁盘扇区时读取文件所需的时间（以 ms 为单位）。

家庭作业 6.25 ▲ 下表给出许多不同高速缓存的参数。对每一个缓存，请填写表中缺失的字段。回想一下：m 是物理地址位数，C 是缓存大小（数据字节数），B 是块大小（以字节为单位），E 是相联度，S 是缓存组数，t 是标记位数，s 是组索引位数，b 是块偏移位数。

家庭作业 6.25 高速缓存参数

缓存	m	C	B	E	S	t	s	b
1.	32	1,024	4	4				
2.	32	1,024	4	256				
3.	32	1,024	8	1				
4.	32	1,024	8	128				
5.	32	1,024	32	1				
6.	32	1,024	32	4				

家庭作业 6.26 ▲ 下表给出许多不同高速缓存的参数。你的任务是填写表中缺失的字段。回想一下：m 是物理地址位数，C 是缓存大小（数据字节数），B 是块大小（以字节为单位），E 是相联度，S 是缓存组数，t 是标记位数，s 是组索引位数，b 是块偏移位数。

家庭作业 6.26 高速缓存参数

缓存	m	C	B	E	S	t	s	b
1.	32		8	1		21	8	3
2.	32	2,048			128	23	7	2
3.	32	1,024	2	8	64			1
4.	32	1,024		2	16	23	4	

家庭作业 6.27 ▲ 考虑练习题 6.12 中的缓存：**A.** 列出所有靠组 1 命中的十六进制存储器地址。**B.** 列出所有靠组 6 命中的十六进制存储器地址。

家庭作业 6.28 ▲▲ 考虑练习题 6.12 中的缓存：**A.** 列出所有靠组 2 命中的十六进制存储器地址。**B.** 列出所有靠组 4 命中的十六进制存储器地址。**C.** 列出所有靠组 5 命中的十六进制存储器地址。**D.** 列出所有靠组 7 命中的十六进制存储器地址。

家庭作业 6.29 ▲▲ 假设有这样一个系统，它具有如下特性：

- 存储器按字节寻址。
- 存储器访问是针对 1 字节的字（而不是 4 字节的字）。
- 地址的宽度为 12 位。
- 缓存是 2 路组相联的（$E = 2$），缓存块大小为 4 字节（$B = 4$），共有 4 个组（$S = 4$）。

缓存的内容如下，其中所有的地址、标记和值都以十六进制表示：

家庭作业 6.29 2 路组相联缓存的内容

组索引	标记	有效位	字节 0	字节 1	字节 2	字节 3
0	00	1	40	41	42	43
	83	1	FE	97	CC	D0
1	00	1	44	45	46	47
	83	0	—	—	—	—
2	00	1	48	49	4A	4B
	40	0	—	—	—	—
3	FF	1	9A	C0	03	FF
	00	0	—	—	—	—

A. 下面的图以十六进制（每格 1 位）的形式显示地址的格式。请通过标注该图来说明确定下面各项需要使用哪些字段：**CO.** 缓存块偏移 **CI.** 缓存组索引 **CT.** 缓存标记（见下面的表格）

家庭作业 6.29 地址格式（每格 1 位）

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 对于下面列出的每个存储器访问，当严格按照顺序执行时，指出它是一个缓存命中还是不命中。如果可能的话，给出读出的值：

家庭作业 6.29 存储器访问

操作	地址	命中?	读出的值(或未知的)
读	0x834		
写	0x836		
读	0xFFD		

家庭作业 6.30 ▲ 假设有这样一个系统,它具有如下特性:

- 存储器按字节寻址。
- 存储器访问是针对 1 字节的字(而不是 4 字节的字)。
- 地址的宽度为 13 位。
- 缓存是 4 路组相联的($E = 4$),缓存块大小为 4 字节($B = 4$),共有 8 个组($S = 8$)。

考虑下面的缓存状态。其中所有的地址、标记和值均以十六进制给出。索引(Index)列包含每组四个缓存行中该组的组索引;标记(Tag)列包含每个缓存行的标记值;V 列包含每个缓存行的有效位;字节 0 ~ 字节 3 列包含每个缓存行中的数据,最左边为字节 0,从左往右编号。

家庭作业 6.30 4 路组相联缓存

组索引	标记	V	字节 0 ~ 3	标记	V	字节 0 ~ 3	标记	V	字节 0 ~ 3	标记	V	字节 0 ~ 3
0	F0	1	ED 32 0A A2	8A	1	BF 80 1D FC	14	1	EF 09 86 2A	BC	0	25 44 6F 1A
1	BC	0	03 3E CD 38	A0	0	16 7B ED 5A	BC	1	8E 4C DF 18	E4	1	FB B7 12 02
2	BC	1	54 9E 1E FA	B6	1	DC 81 B2 14	00	0	B6 1F 7B 44	74	0	10 F5 B8 2E
3	BE	0	2F 7E 3D A8	C0	1	27 95 A4 74	C4	0	07 11 6B D8	BC	0	C7 B7 AF C2
4	7E	1	32 21 1C 2C	8A	1	22 C2 DC 34	BC	1	BA DD 37 D8	DC	0	E7 A2 39 BA
5	98	0	A9 76 2B EE	54	0	BC 91 D5 92	98	1	80 BA 9B F6	BC	1	48 16 81 0A
6	38	0	5D 4D F7 DA	BC	1	69 C2 8C 74	8A	1	A8 CE 7F DA	38	1	FA 93 EB 48
7	8A	1	04 2A 32 6A	9E	0	B1 86 56 0E	CC	1	96 30 47 F2	BC	1	F8 1D 42 30

A. 这个缓存的大小 C 是多少字节?

B. 下面的框显示了地址的格式(每格 1 位)。请通过标注该框来说明确定下面各项需要使用哪些字段:**CO.** 缓存块偏移 **CI.** 缓存组索引 **CT.** 缓存标记(见下面的表格)

家庭作业 6.31 ▲▲ 假设一个程序在家庭作业 6.30 的缓存上引用地址 0x071A 处的 1 字节字。请指出被访问的缓存项以及以十六进制形式返回的缓存字节的值。请指出是否会发生缓存不命中。如果发生了缓存不命中,则对于“返回的缓存字节”请填入“—”。提示:注意那些有效位!

A. 地址格式(每格 1 位):

B. 存储器访问:

家庭作业 6.30 地址格式(每格 1 位)

12	11	10	9	8	7	6	5	4	3	2	1	0

家庭作业 6.31 地址格式(每格 1 位)

12	11	10	9	8	7	6	5	4	3	2	1	0

家庭作业 6.31 存储器访问参数

参数	值
块偏移(CO)	0x
组索引(CI)	0x
缓存标记(CT)	0x
缓存命中?(Y/N)	
返回的缓存字节	0x

家庭作业 6.32 ▲▲ 对存储器地址 0x16E8 重复家庭作业 6.31。

A. 地址格式(每格 1 位):

家庭作业 6.32 地址格式(每格 1 位)

12	11	10	9	8	7	6	5	4	3	2	1	0

B. 存储器访问:

家庭作业 6.33 ▲▲ 对于家庭作业 6.30 中的缓存,列出会在组 2 中命中的 8 个存储器地址的十六进制表示。

家庭作业 6.34 ▲▲ 考虑下面的矩阵转置例程:

```
1  typedef int array[4][4];
2
3  void transpose2(array dst, array src)
```

家庭作业 6.32 存储器访问参数

参数	值
缓存偏移(CO)	0x
缓存索引(CI)	0x
缓存标记(CT)	0x
缓存命中?(Y/N)	
返回的缓存字节	0x

```
4  {
5      int i, j;
6
7      for (i = 0; i < 4; i++) {
8          for (j = 0; j < 4; j++) {
9              dst[j][i] = src[i][j];
10         }
11     }
12 }
```

假设这段代码运行在一台具有下列特性的机器上:

- sizeof(int) = 4。
- 数组 src 从地址 0 开始,数组 dst 从地址 64(十进制)开始。
- 只有一个 L1 数据缓存,直接映射,写通,写分配,块大小为 16 字节。
- 缓存的全部数据大小为 32 字节,且缓存初始时为空。
- 对数组 src 和 dst 的访问分别是读不命中和写不命中的唯一来源。

A. 对每个 row(行)和 col(列),指出对 src[row][col] 的访问和对 dst[row][col] 的访问是命中(记作 h)还是不命中(记作 m)。例如:读 src[0][0] 是不命中,写 dst[0][0] 也是不命中。

家庭作业 6.34 dst 数组与 src 数组的访问模式

dst 数组					src 数组				
	列 0	列 1	列 2	列 3		列 0	列 1	列 2	列 3
行 0	m				行 0	m			
行 1					行 1				
行 2					行 2				
行 3					行 3				

家庭作业 6.35 ▲▲ 把上述问题重做一遍,这次的缓存总数据大小为 128 字节。

家庭作业 6.35 dst 数组与 src 数组的访问模式

dst 数组					src 数组				
	列 0	列 1	列 2	列 3		列 0	列 1	列 2	列 3
行 0					行 0				
行 1					行 1				
行 2					行 2				
行 3					行 3				

家庭作业 6.36 ▲▲ 本题测试你预测 C 代码缓存行为的能力。给你下面的代码以供分析：

```
1  int x[2][128];
2  int i;
3  int sum = 0;
4
5  for (i = 0; i < 128; i++) {
6      sum += x[0][i] * x[1][i];
7  }
```

假设在下列条件下执行这段代码：

- sizeof(int) = 4。
- 数组 x 从存储器地址 0x0 开始，以行主序存储。
- 在下面每种情况中，缓存初始时都是空的。
- cache 的唯一存储器访问是对数组 x 元素的访问；所有其他变量都设置在寄存器中。

有了这些假设，估计下面各种情况的不命中率：

A. 情况 1：假设缓存大小为 512 字节，直接映射，缓存块大小为 16 字节。不命中率是多少？

B. 情况 2：如果缓存大小放大一倍（为 1,024 字节），不命中率是多少？

C. 情况 3：现在假设缓存大小为 512 字节，采用 LRU 替换策略的 2 路组相联，缓存块大小为 16 字节。此时缓存不命中率是多少？

D. 对情况 3，加大缓存大小有助于降低不命中率吗？为什么？

E. 对情况 3，加大块大小有助于降低不命中率吗？为什么？

家庭作业 6.37 ▲▲ 这是另一个测试你分析 C 代码缓存能力的问题。假设在下面条件下执行图 6.47 中的三个求和函数：

- sizeof(int) = 4。

- 计算机(机器)有一个 4 KB 的直接映射缓存,块大小为 16 字节。
- 在两个内循环之间,代码使用的唯一存储器访问方式是针对数组数据的访问。下标值和值 sum 保存在寄存器中。
- 数组 a 从存储器地址 0x08000000 开始存储。

请在表中填写两个用于比较的情况 $N = 64$ 和 $N = 60$ 的大致缓存不命中率:

家庭作业 6.37 缓存不命中示例

函数	$N = 64$	$N = 60$
sumA		
sumB		
sumC		

```
1  typedef int array_t[N][N];
2
3  int sumA(array_t a)
4  {
5      int i, j;
6      int sum = 0;
7      for (i = 0; i < N; i++)
8          for (j = 0; j < N; j++) {
9              sum += a[i][j];
10         }
11     return sum;
12 }
13
14 int sumB(array_t a)
15 {
16     int i, j;
17     int sum = 0;
18     for (j = 0; j < N; j++)
19         for (i = 0; i < N; i++) {
20             sum += a[i][j];
21         }
22     return sum;
23 }
24
25 int sumC(array_t a)
26 {
27     int i, j;
28     int sum = 0;
29     for (j = 0; j < N; j+=2)
30         for (i = 0; i < N; i+=2) {
31             sum += (a[i][j] + a[i+1][j]
32                     + a[i][j+1] + a[i+1][j+1]);
33         }
34     return sum;
35 }
```

图 6.47 在家庭作业 6.37 中被引用的函数。

家庭作业 6.38 ▲ 3M 公司通过在白色纸片上印出黄色方块生产报事贴，现在需要你测试所用算法的效率。作为打印过程的一部分，它需要为正方形中的每个点设置 CMYK（青色、洋红、黄色和黑色）的值。3M 聘请你判断下列算法在一台具有 1,024 字节直接映射数据缓存（块大小为 16 字节）的机器上的效率。提供给你下面的定义以供参考。

```
1  struct point_color {
2      int c;
3      int m;
4      int y;
5      int k;
6  };
7
8  struct point_color square[16][16];
9  int i, j;
```

假设条件如下：

- sizeof(int) $= 4$。
- 数组 square 从存储器地址 0 开始。
- 缓存初始时是空的。
- 缓存访问的唯一存储器访问是对数组 square 元素的访问。变量 i 和 j 保存在寄存器中。

确定下面代码的高速缓存性能：

```
1  for (i = 15; i >= 0; i--){
2      for (j = 15; j >= 0; j--) {
3          square[i][j].c = 0;
4          square[i][j].m = 0;
5          square[i][j].y = 1;
6          square[i][j].k = 0;
7      }
8  }
```

A. 写的总次数是多少？

B. 在缓存中会命中的写操作总数是多少？

C. 命中率是多少？

家庭作业 6.39 ▲ 根据家庭作业 6.38 中的假设，确定下面代码的高速缓存性能：

```
1  for (i = 15; i >= 0; i--){
2      for (j = 15; j >= 0; j--) {
3          square[j][i].c = 0;
4          square[j][i].m = 0;
```

```
5          square[j][i].y = 1;
6          square[j][i].k = 0;
7      }
8  }
```

A. 写的总次数是多少?

B. 在缓存中会命中的写操作总数是多少?

C. 命中率是多少?

家庭作业 6.40 ▲ 根据家庭作业 6.38 中的假设,确定下面代码的高速缓存性能:

```
1  for (i = 15; i >= 0; i--) {
2      for (j = 15; j >= 0; j--) {
3          square[i][j].y = 1;
4      }
5  }
6  for (i = 15; i >= 0; i--) {
7      for (j = 15; j >= 0; j--) {
8          square[i][j].c = 0;
9          square[i][j].m = 0;
10         square[i][j].k = 0;
11     }
12 }
```

A. 写的总次数是多少?

B. 在缓存中会命中的写操作总数是多少?

C. 命中率是多少?

家庭作业 6.41 ▲▲ 假设你正在编写一个全新的三维游戏,希望它让你名利双收。目前你正在写一个函数,用来在绘制下一帧图像之前清空屏幕缓冲区。你使用的屏幕为 640×480 的像素数组,而你现在用的机器上有一个 32 KB 直接映射的缓存,行大小为 8 字节。你使用的 C 结构定义如下:

```
1  struct pixel {
2      char r;
3      char g;
4      char b;
5      char a;
6  };
7
8  struct pixel buffer[480][640];
9  int i, j;
10 char *cptr;
11 int *iptr;
```

- sizeof(char) $= 1$ 且 sizeof(int) $= 4$。
- 数组 buffer 从存储器地址 0 开始。

- 该缓存初始时是空的。
- 对数组 buffer 元素的访问是唯一的存储器访问。变量 i、j、cptr 和 iptr 保存在寄存器中。

下面代码中的写操作有多大比例会命中缓存?

```
1  for (j = 639; j >= 0; j--) {
2      for (i = 479; i >= 0; i--){
3          buffer[i][j].r = 0;
4          buffer[i][j].g = 0;
5          buffer[i][j].b = 0;
6          buffer[i][j].a = 0;
7      }
8  }
```

家庭作业 6.42 ▲▲ 根据家庭作业 6.41 中的假设, 下面代码中的写操作有多大比例会命中缓存?

```
1  char *cptr = (char *) buffer;
2  for (; cptr < (((char *) buffer) + 640 * 480 * 4); cptr++)
3      *cptr = 0;
```

家庭作业 6.43 ▲▲ 根据家庭作业 6.41 中的假设, 下面代码中的写操作有多大比例会命中缓存?

```
1  int *iptr = (int *)buffer;
2  for (; iptr < ((int *)buffer + 640*480); iptr++)
3      *iptr = 0;
```

家庭作业 6.44 ▲▲▲ 请从 CS:APP 站点下载 mountain 程序, 并在你最熟悉的 PC/Linux 系统上运行。根据运行结果估算出你的系统的缓存大小。

家庭作业 6.45 ▲▲▲▲ 在这个作业中, 你要把第 5 章和第 6 章学到的概念应用到对一个访存密集型程序的优化上。考虑一个拷贝并转置 int 类型的 $N \times N$ 矩阵元素的过程。也就是说, 对源矩阵 S 和目标矩阵 D, 要把每个元素 $s_{i,j}$ 拷贝到 $d_{j,i}$。这段代码可以用一个简单的循环写成:

```
1  void transpose(int *dst, int *src, int dim)
2  {
3      int i, j;
4
5      for (i = 0; i < dim; i++)
6          for (j = 0; j < dim; j++)
7              dst[j*dim + i] = src[i*dim + j];
8  }
```

其中, 过程的参数是指向目标(dst)矩阵和源(src)矩阵的指针, 以及矩阵的大小 N(dim)。你的任务是设计出一个尽可能快的转置例程。

家庭作业 6.46 ▲▲▲▲ 这个作业是家庭作业 6.45 的一个有趣变种。考虑把有向图 g 转换成其无向对应图 g' 的问题。图 g' 中从顶点 u 到顶点 v 有一条边，当且仅当原图 g 中从 u 到 v 或从 v 到 u 有一条边。图 g 由它的邻接矩阵 G 表示如下：如果 N 是 g 的顶点数，则 G 是一个 $N \times N$ 矩阵，其元素全为 0 或 1。假设 g 的顶点命名为 $v_0, v_1, v_2, \ldots, v_{N-1}$，那么当存在一条从 v_i 到 v_j 的边时，$G[i][j]$ 为 1，否则为 0。注意，邻接矩阵对角线上的元素总是 1，并且无向图的邻接矩阵是对称的。这段代码可以用一个简单的循环写成：

```
1  void col_convert(int *G, int dim) {
2      int i, j;
3
4      for (i = 0; i < dim; i++)
5          for (j = 0; j < dim; j++)
6              G[j*dim + i] = G[j*dim + i] || G[i*dim + j];
7  }
```

你的任务是设计一个尽可能快的转换例程。和前面一样，你需要运用第 5 章和第 6 章学到的概念来构造一个好的解法。

6.10 练习答案

答案 6.1 这里的目标是选择地址位数中的高位（高位），选择折衷比例 $\max(r, c)/\min(r, c)$ 中较小的那个。换句话说，阵列的形状越接近方形，所需的地址位数就越少。

答案 6.1 用于地址分解的组织方式

组织方式	r	c	b_r	b_c	$\max(b_r, b_c)$
16×1	4	4	2	2	2
16×4	4	4	2	2	2
128×8	16	8	4	3	4
512×4	32	16	5	4	5
1,024×4	32	32	5	5	5

答案 6.2 这个小练习的要点是确认你理解柱面和磁道这两者的关系。理清之后，剩下的就是一步步算下去：

$$\begin{aligned}\text{磁盘容量} &= \frac{1{,}024\ \text{字节}}{\text{扇区}} \times \frac{500\ \text{扇区}}{\text{磁道}} \times \frac{15{,}000\ \text{磁道}}{\text{面}} \times \frac{2\ \text{面}}{\text{盘片}} \times \frac{3\ \text{盘片}}{\text{磁盘}} \\ &= 46{,}080{,}000{,}000\ \text{字节} \\ &= 46.08\ \text{GB}\end{aligned}$$

答案 6.3 本题解答就是直接我们已有的磁盘访问时间公式的一个应用。平均旋转延迟(以 ms 为单位)为:

$$\begin{aligned} T_{\text{avg rotation}} &= 1/2 \times T_{\text{max rotation}} \\ &= 1/2 \times (60 \text{ secs}/12{,}000 \text{ RPM}) \times 1{,}000 \text{ ms/sec} \\ &\approx 2.5 \text{ ms} \end{aligned}$$

平均的传送时间为:

$$\begin{aligned} T_{\text{avg transfer}} &= (60 \text{ secs}/12{,}000 \text{ RPM}) \times 1/300 \text{ 扇区/磁道} \times 1{,}000 \text{ ms/sec} \\ &\approx 0.016 \text{ ms} \end{aligned}$$

汇总起来,估算的总访问时间就为:

$$\begin{aligned} T_{\text{access}} &= T_{\text{avg seek}} + T_{\text{avg rotation}} + T_{\text{avg transfer}} \\ &= 5 \text{ ms} + 2.5 \text{ ms} + 0.016 \text{ ms} \\ &\approx 7.516 \text{ ms} \end{aligned}$$

答案 6.4 这是检验你对影响磁盘性能的各因素的理解的好机会。首先需要确定文件和磁盘的一些基本性质。文件由 10,000 个 512 字节的逻辑块组成。对这块磁盘而言, $T_{\text{avg seek}} = 6$ ms, $T_{\text{max rotation}} = 4.61$ ms, $T_{\text{avg rotation}} = 2.30$ ms。**A.** 最好情况: 在最优情况下, 各块被映射到同一柱面上的连续扇区, 可以一个接一个地读出而无需移动磁头。一旦磁头定位到第一个扇区上方, 读完 10,000 个块需要磁盘整整转两圈(每圈 5,000 个扇区)。所以读文件的总时间为 $T_{\text{avg seek}} + T_{\text{avg rotation}} + 2 \times T_{\text{max rotation}} = 6 + 2.30 + 9.22 = 17.52$ ms。**B.** 随机情况: 在这种情况下, 各块被随机映射到扇区, 读 10,000 个块中的每一块都需要 $T_{\text{avg seek}} + T_{\text{avg rotation}}$ ms,所以读文件的总时间为 $(T_{\text{avg seek}} + T_{\text{avg rotation}}) \times 10{,}000 =$ 83,000 ms(整整 83 秒!)。现在你该明白为什么对磁盘驱动器做碎片整理往往是好主意了!

答案 6.5 这是一个简单的问题,它给出关于固态硬盘可行性的有趣洞见。回想一下,对磁盘而言 1 PB $= 10^9$ MB。于是下面这个简单的单位换算就给出各种情况下的预测年限:

A. 最坏情况的顺序写(520 MB/s):

$$(10^9 \times 128) \times (1/520) \times (1/(86{,}400 \times 365)) \approx 7 \text{ 年}$$

B. 最坏情况的随机写(205 MB/s):

$$(10^9 \times 128) \times (1/205) \times (1/(86{,}400 \times 365)) \approx 19 \text{ 年}$$

C. 平均情况(每天 50 GB):

$$(10^9 \times 128) \times (1/50{,}000) \times (1/365) \approx 6{,}912 \text{ 年}$$

所以即使 SSD 连续工作,它也应该至少能用 7 年,这比大多数计算机的预期寿命还要长。

答案 6.6 在 2005 年到 2015 年这 10 年间,旋转磁盘的单位价格下降了 166 倍,这意味着价格大约每 18 个月就减半。假设这一趋势继续下去,一套 1 PB 的存储系统在 2015 年的价格约为 30,000 美元,大约经过 8 次这样的减半之后就会降到 200 美元以下。由于这种减半每 18 个月发生一次,可以预计到 2027 年前后,1 PB 的存储就能以 200 美元左右的价格买到。

答案 6.7 要创造出一个步长为 1 的引用模式,唯一的方法是把磁性存储按反转(内层)的顺序排列,变址顺序上右端下标变化最快。

```
1  int productarray3d(int a[N][N][N])
2  {
3      int i, j, k, product = 1;
4
5      for (j = N-1; j >= 0; j--) {
6          for (k = N-1; k >= 0; k--) {
7              for (i = N-1; i >= 0; i--) {
8                  product *= a[j][k][i];
9              }
10         }
11     }
12     return product;
13 }
```

这是一个非常重要的思想。确保你理解为什么这个特定的循环重排列出的访问模式步长为 1。

答案 6.8 解决这个问题的要领是想象出数组在存储器里如何布局,然后分析引用模式。`clear1` 函数采用顺序访问的引用模式访问数组,它显然是最好的。`clear2` 函数对结构体中的元素进行顺序访问,plant 具体顺序借助存储器的调度 0、12、4、16、8、20。因此 `clear2` 的空间局部性不如 `clear1`。`clear3` 函数也非顺序地访问结构体里的成员,且它并不按顺序访问所有结构体,因此其局部性显著地比 `clear2` 和 `clear1` 差。

答案 6.9 此解答应用图 6.26 中讲解的缓存参数的契约，非常了解了。它并非十分简单，但是相当接近于：缓存如何把地址位划分为多个域，以及为什么它们是如此划分的，是理解缓存工作关键。然后，基本的缓存划分原则是：

答案 6.9 高速缓存参数

缓存	m	C	B	E	S	t	s	b
1.	32	1,024	4	1	256	22	8	2
2.	32	1,024	8	4	32	24	5	3
3.	32	1,024	32	32	1	27	0	5

答案 6.10 这里的填充(padding)消除了冲突不命中。因此，四分之三的引用是命中。

答案 6.11 有时候，了解为什么一个设计是坏主意，有助于理解为什么其他方案是好的。这里，我们做的坏设计是用高位而不是中间为成分索引缓存。**A.** 有了高位索引之后，每连续的数组分块由 2^t 个缓存框成，其中 t 是标记位数。因此，数组的前 2^t 个完整性分块会映射到组 0，下一批 2^t 个分块会映射到组 1，依此类推。**B.** 对于一个 $(S, E, B, m) = (512, 1, 32, 32)$ 的直接映射缓存，缓存容量是 512 个 32 字节的缓存框，$t = 18$ 位标记。不过这是一个新的方法：数组的前 2^{18} 个分块会映射到组 0，下一批 2^{18} 个分块映射到组 1。由于数组只包含 $(4{,}096 \times 4)/32 = 512$ 个分块，数组中的所有分块都会映射到组 0。结果，即使数组一次只能装下数组的一个分块，也无法装下数组的整体大小。显然，用高位来索引缓存并不是空间的高效方式。

答案 6.12 低 2 位是块偏移(CO)，其后 3 位是组索引(CI)，剩下的作为标记(CT)：

答案 6.12 地址划分(每格 1 位)

CT 12	CT 11	CT 10	CT 9	CT 8	CT 7	CT 6	CT 5	CI 4	CI 3	CI 2	CO 1	CO 0

答案 6.13 地址 `0x0D53`：**A.** 地址格式(每格 1 位)：

答案 6.13 地址划分(每格 1 位)

CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO
0	1	1	0	1	0	1	0	1	0	0	1	1
12	11	10	9	8	7	6	5	4	3	2	1	0

B. 存储器访问:

答案 6.13 存储器访问参数

参数	值
缓存块偏移(CO)	0x3
缓存组索引(CI)	0x4
缓存标记(CT)	0x6A
缓存命中?(Y/N)	N
返回的缓存字节	—

答案 6.14 地址 `0x0CB4`:**A.** 地址格式(每格 1 位):

答案 6.14 地址划分(每格 1 位)

CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO
0	1	1	0	0	1	0	1	1	0	1	0	0
12	11	10	9	8	7	6	5	4	3	2	1	0

B. 存储器访问:

答案 6.14 存储器访问参数

参数	值
缓存块偏移(CO)	0x0
缓存组索引(CI)	0x5
缓存标记(CT)	0x65
缓存命中?(Y/N)	N
返回的缓存字节	—

答案 6.15 地址 `0x0A31`:**A.** 地址格式(每格 1 位):

B. 存储器访问:

答案 6.16 这个问题算是练习题 6.12 ~ 6.15 的一种反向版本:要求从缓存的内容出发反推出会在某个特定组中命中的地址。本例中,组 3 包含一条标记

答案 6.15　地址划分(每格 1 位)

CT	CT	CT	CT	CT	CT	CT	CT	CI	CI	CI	CO	CO
0	1	0	1	0	0	0	1	1	0	0	0	1
12	11	10	9	8	7	6	5	4	3	2	1	0

答案 6.15　存储器访问参数

参数	值
缓存块偏移(CO)	0x1
缓存组索引(CI)	0x4
缓存标记(CT)	0x51
缓存命中?(Y/N)	N
返回的缓存字节	—

为 0x32 的有效行。由于该组中只有一条有效行,将有四个地址命中。这些地址的二进制形式为 0 0110 0100 11xx。于是,命中的四个十六进制地址是

```
0x064C, 0x064D, 0x064E, 0x064F
```

A. 解答这个问题的关键是画出图 6.48 那样的图。注意:每条缓存行恰好容纳数组的一整行,缓存又恰好能装下一个数组,并且对所有 i, src 的第 i 行和 dst 的第 i 行都映射到同一条缓存行。因为缓存太小,装不下两个数组,对一个数组的引用会不断把另一个数组的有用行挤出去。例如,写 dst[0][0] 会把读 src[0][0] 时装入的那条行挤出去,于是接下来读 src[0][1] 时就不命中了。

答案 6.17　dst 数组与 src 数组的访问模式

dst 数组			src 数组		
	列 0	列 1		列 0	列 1
行 0	m	m	行 0	m	m
行 1	m	m	行 1	m	h

B. 当缓存为 32 字节时,它就装得下两个数组了。于是唯一的不命中是最初的冷不命中。

答案 6.17　dst 数组与 src 数组的访问模式(缓存大小 32 字节)

dst 数组			src 数组		
	列 0	列 1		列 0	列 1
行 0	m	h	行 0	m	h
行 1	m	h	行 1	m	h

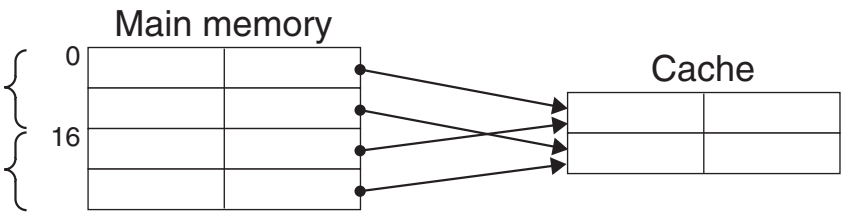


图 6.48 练习 6.17 答案所用的图。

答案 6.18 每个 32 字节的缓存行容纳两个相邻的 `algae_position` 结构。每个循环按存储器顺序访问这些结构,每次读一个整型元素。所以每个循环的模式都是不命中、命中、不命中、命中,如此继续。注意,对这个问题,我们本可以不必枚举读和不命中的总数就能预测出不命中率。**A.** 读访问的总次数是多少? 2,048 次读。**B.** 在缓存中会不命中的读访问总次数是多少? 1,024 次不命中。**C.** 不命中率是多少? $1024/2048 = 50\%$。

答案 6.19 解决这个问题的关键在于注意到缓存只能装下数组的一半。于是在扫描数组后半部分时的按列访问会把扫描前半部分时装入的行挤出去。例如, 读 grid[8][0] 的第一个元素会挤掉读 grid[0][0] 中的元素时装入的那条行, 而那条行里还含有 grid[0][1]。于是在开始扫描下一列时, 对 grid[0][1] 第一个元素的引用就会不命中。**A.** 读访问的总次数是多少? 2,048 次读。**B.** 在缓存中会命中的读访问总次数是多少? 1,024 次命中。**C.** 命中率是多少? $1024/2048 = 50\%$。**D.** 如果缓存的大小扩大一倍,命中率会是多少? 如果缓存大小加倍,它就能装下整个 grid 数组。唯一的不命中是最初的冷不命中,命中率为 $3/4 = 75\%$。

答案 6.20 这个循环有很好的步长为 1 的引用模式,所以唯一的不命中是最初的冷不命中。**A.** 读访问的总次数是多少? 2,048 次读。**B.** 在缓存中会命中的读访问总次数是多少? 1,536 次命中。**C.** 命中率是多少? $1536/2048 = 75\%$。**D.** 如果缓存的大小扩大一倍,命中率会是多少? 无论把这个缓存增大多少,都不会改变不命中率,因为冷不命中是不可避免的。

答案 6.21 用大步长从 L1 读取时的持续吞吐量约为 12,000 MB/s,时钟频率为 2,100 MHz, 每次读访问以 16 字节的字为单位。于是由这张图可以估算出:在这台机器上从 L1 访问一个字大约需要 $2{,}100/12{,}000 \times 16 = 2.8 \approx 3.0$ 个周期,比 L1 名义的 4 周期延迟大约快 1.25 倍。这是由 4×4 展开循环的并行性造成的,它允许多个装载同时处于未决状态。

第二部分

在系统上运行程序

我们对计算机系统的探索继续，进一步考察系统软件如何建立并运行应用程序。链接器把程序的各个部分合并成一个文件，读者可以把它加载到存储器中并在处理器上执行。现代操作系统与硬件协作，为每个程序提供一种假象，认为它独占地使用处理器和主存，而实际上任何时候都有很多程序运行在系统上。

在本书的第一部分中，你对程序与硬件的交互建立起了一个很好的理解。第二部分将竭力扩展你对系统的认识，使你理解程序与操作系统之间的交互。你将学到如何用操作系统提供的服务构建系统级程序，例如 Unix shell 程序和动态存储器分配包。

7

链接

链接（linking）是将各种各样的代码和数据部分收集起来并组合成一个单一文件的过程，这个文件可以被加载（复制）到内存并执行。链接可以在编译时执行，也就是在源代码被翻译成机器码时；可以在加载时执行，也就是在程序被加载器加载到内存并执行时；甚至可以在运行时由应用程序自身执行。早期的计算机系统上，链接是手工执行的。现代系统上，链接由称为链接器（linker）的程序自动执行。

链接器在软件开发中扮演着至关重要的角色，因为它使分离编译成为可能。我们可以不再把一个大型应用程序组织成一个庞大的源文件，而是把它分解成更小、更好管理的模块，这些模块可以独立地修改和编译。当我们改变其中一个模块时，只需简单地重新编译它，然后重新链接应用程序，而不必重新编译其他文件。

链接通常由链接器悄悄地完成，对于那些在入门程序设计课中编写小程序的学生来说，它并不是一个值得关注的问题。那么，为什么还要费心学习链接呢？

- **理解链接器将帮助你构建大型程序**。构建大型程序的程序员经常会遇到由缺失的模块、缺失的库或不兼容的库版本引起的链接器错误。除非你理解链接器如何解析引用、什么是库、以及链接器如何使用库来解析引用，否则这类错误会令人困惑和沮丧。
- **理解链接器将帮助你避免危险的编程错误**。Linux 链接器解析符号引用时做的决定可能悄悄影响你程序的正确性。在默认情况下，定义了多个全局变量（这样的做法并不正确）的程序可以毫无警告地通过链接器。得到的程序可能出现令人困惑的运行时行为，而且极难调试。我们将向你展示这种情况是如何发生的，以及如何避免它。

- **理解链接将帮助你理解语言作用域规则是如何实现的**。例如,全局变量和局部变量有什么区别?给变量或函数定义加上 `static` 属性到底是什么意思?
- **理解链接将帮助你理解其他重要的系统概念**。链接器产生的可执行目标文件在加载和运行程序、虚拟内存、分页和存储器映射这样的重要系统功能中扮演着关键角色。
- **理解链接使你能够利用共享库**。多年以来,链接被认为相当直截了当、乏味。然而,随着共享库和动态链接在现代操作系统中日益重要,链接已成为一个复杂的过程,它为通晓内情的程序员提供了强大的能力。例如,许多软件产品使用共享库在运行时升级成品二进制程序。许多 Web 服务器也依靠共享库的动态链接来提供动态内容。

```
(a) main.c

1  int sum(int *a, int n);
2
3  int array[2] = {1, 2};
4
5  int main()
6  {
7      int val = sum(array, 2);
8      return val;
9  }
```

```
(b) sum.c

1  int sum(int *a, int n)
2  {
3      int i, s = 0;
4
5      for (i = 0; i < n; i++) {
6          s += a[i];
7      }
8      return s;
9  }
```

图 7.1 示例程序 1。示例程序由两个源文件 `main.c` 和 `sum.c` 组成。`main` 函数初始化一个 `int` 数组,然后调用 `sum` 函数对数组元素求和。

本章全面讨论链接的各个方面,从传统的静态链接,到加载时共享库的动态链接,再到运行时共享库的动态链接。我们将用真实的例子来描述基本机制,并指出链接问题会在哪些情况下影响程序的性能和正确性。为了让讨论具体而易于理解,我们以运行 Linux、使用标准 ELF-64(以下简称 ELF)目标文件格式的 x86-64 系统为背景展开讨论。不过要认识到,链接的基本概念是通用的,

与操作系统、指令集体系结构或目标文件格式无关。细节可能有所不同，但概念是一样的。

7.1 编译器驱动程序

考虑图 7.1 中的 C 程序。它将作为本章一直使用的简单示例，帮助我们阐明链接器工作的几个要点。

大多数编译系统会按需替用户调用语言预处理器、编译器、汇编器和链接器，这样的程序称为编译器驱动程序（compiler driver）。例如，要用 GNU 编译系统构建这个示例程序，我们可以输入下面的 shell 命令来调用 `gcc` 驱动程序：

```
linux> gcc -Og -o prog main.c sum.c
```

图 7.2 概括了驱动程序把示例程序从 ASCII 源文件翻译成可执行目标文件时的全部活动。（如果你想亲眼看看这些步骤，可以加上 `-v` 选项运行 `gcc`。）驱动程序首先运行 C 预处理器（cpp）[1]，把 C 源文件 `main.c` 翻译成 ASCII 中间文件 `main.i`：

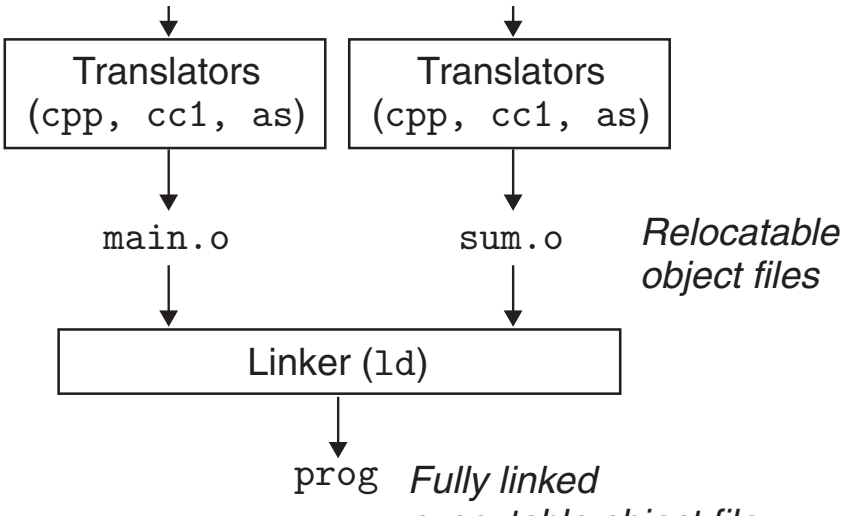


图 7.2 静态链接。链接器将可重定位目标文件组合到一起，形成一个可执行目标文件 `prog`。

```
cpp [other arguments] main.c /tmp/main.i
```

接下来，驱动程序运行 C 编译器（cc1），把 `main.i` 翻译成 ASCII 汇编语言文件 `main.s`：

[1] 在某些版本的 `gcc` 中，预处理器已集成到编译器驱动程序里。

```
cc1 /tmp/main.i -Og [other arguments] -o /tmp/main.s
```

然后,驱动程序运行汇编器(as),把 main.s 翻译成二进制可重定位目标文件 main.o:

```
as [other arguments] -o /tmp/main.o /tmp/main.s
```

驱动程序经过同样的过程生成 sum.o。最后, 它运行链接器程序 ld, 把 main.o 和 sum.o 以及必要的系统目标文件组合起来,创建二进制可执行目标文件 prog:

```
ld -o prog [system object files and args] /tmp/main.o /tmp/sum.o
```

要运行可执行文件 prog,我们在 Linux shell 的命令行上输入它的名字:

```
linux> ./prog
```

shell 调用操作系统中一个称为加载器 (loader) 的函数, 它把可执行文件 prog 中的代码和数据复制到内存,然后将控制转移到程序的开头。

7.2 静态链接

静态链接器 (static linker) 如 Linux 的 ld 程序那样, 以一组可重定位目标文件和命令行参数作为输入, 生成一个可以加载和运行的、完全链接的可执行目标文件作为输出。输入的可重定位目标文件由各种不同的代码和数据节(section)组成,每节都是连续的字节序列。指令在一个节中,已初始化的全局变量在另一个节中,未初始化的变量又在另一个节中。

要构建可执行文件,链接器必须完成两个主要任务:

第 1 步符号解析(symbol resolution)。目标文件定义并引用符号,每个符号对应于一个函数、一个全局变量或一个静态变量(即用 static 属性声明的任何 C 变量)。符号解析的目的是把每个符号引用与恰好一个符号定义关联起来。

第 2 步重定位(relocation)。编译器和汇编器生成的代码和数据节从地址 0 开始。链接器通过把每个符号定义与一个存储器位置关联起来,然后修改所有对这些符号的引用,使它们指向这个存储器位置,从而重定位这些节。链接器全靠汇编器产生的被称为重定位条目 (relocation entry) 的详细指令, 盲目地执行这些重定位。

后面几节将更详细地描述这些任务。阅读时,请记住关于链接器的一些基本事实:目标文件不过是字节块的集合。其中一些块包含程序代码,一些包含程序数据,还有一些包含引导链接器和加载器的数据结构。链接器把这些块拼

接在一起，确定拼接后各块的运行时地址，并修改代码和数据块中的各个位置。链接器对目标机器的了解非常有限。生成目标文件的编译器和汇编器已经完成了大部分工作。

7.3 目标文件

目标文件有三种形式：

- **可重定位目标文件（relocatable object file）**。包含二进制的代码和数据，其形式可以在编译时与其他可重定位目标文件合并起来，创建可执行目标文件。
- **可执行目标文件（executable object file）**。包含二进制的代码和数据，其形式可以被直接复制到内存并执行。
- **共享目标文件（shared object file）**。一种特殊类型的可重定位目标文件，可以在加载时或运行时加载到内存并动态链接。

编译器和汇编器生成可重定位目标文件（包括共享目标文件）。链接器生成可执行目标文件。严格来说，一个目标模块（object module）就是一个字节序列，而目标文件（object file）就是一个以文件形式存放在磁盘上的目标模块。不过，下面我们将互换地使用这些术语。

目标文件是按照特定的目标文件格式来组织的，不同系统各不相同。贝尔实验室的第一个 Unix 系统使用 a.out 格式。（直到今天，可执行文件仍然被称为 a.out 文件。）Windows 使用可移植可执行（Portable Executable，PE）格式。Mac OS-X 使用 Mach-O 格式。现代 x86-64 Linux 和 Unix 系统使用可执行和可链接格式（Executable and Linkable Format，ELF）。尽管我们的讨论集中在 ELF 上，但基本概念是相似的，与具体格式无关。

7.4 可重定位目标文件

图 7.3 展示了一个典型的 ELF 可重定位目标文件的格式。ELF 头（ELF header）以一个 16 字节的序列开始，这个序列描述了生成该文件的系统的字的大小和字节顺序。ELF 头剩下的部分包含帮助链接器解析和解释目标文件的信息，包括 ELF 头的大小、目标文件的类型（比如可重定位、可执行或共享）、机器类型（比如 x86-64）、节头表（section header table）的文件偏移，以及节头表中条目的大小和数量。各节的位置和大小由节头表描述，节头表为目标文件中的每个节提供一个固定大小的条目。

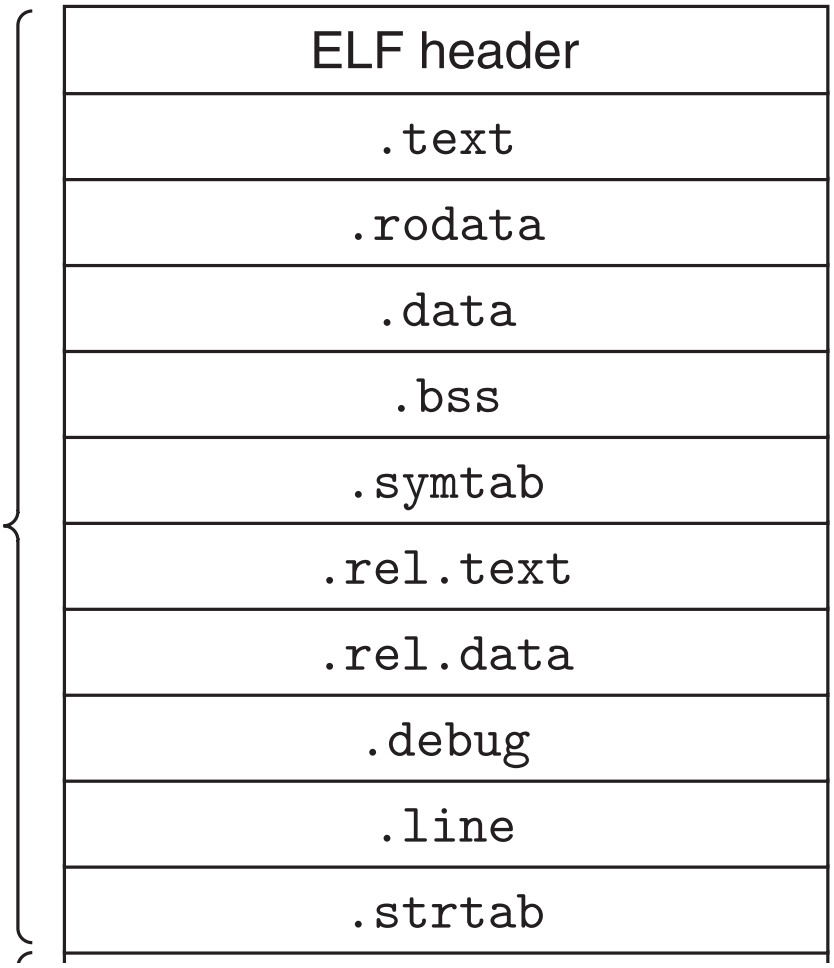


图 7.3 典型的 ELF 可重定位目标文件。

夹在 ELF 头和节头表之间的正是节本身。一个典型的 ELF 可重定位目标文件包含下面几个节：

- `.text` 已编译程序的机器代码。
- `.rodata` 只读数据，比如 `printf` 语句中的格式串和 `switch` 语句的跳转表。
- `.data` 已初始化的全局变量和静态 C 变量。局部 C 变量在运行时保存在栈中，既不出现在`.data` 节中，也不出现在`.bss` 节中。
- `.bss` 未初始化的全局变量和静态 C 变量，以及所有被初始化为 0 的全局变量或静态变量。这个节在目标文件中不占据实际空间，它只是一个占位符。目标文件格式之所以区分已初始化和未初始化的变量，是出于空间

效率的考虑：未初始化的变量不必在目标文件中占用任何实际的磁盘空间。运行时，这些变量会在内存中被分配初值为零的存储空间。

边栏　为什么未初始化的数据称为.bss？
用.bss 这个术语来表示未初始化数据是普遍做法。它最初是 IBM 704 汇编语言（1957 年前后）中“block started by symbol（由符号开始的块）”指令的缩写，这个缩写一直沿用至今。有一个简单的方法可以记住.data 节和.bss 节的区别：把“bss”看成“Better Save Space!（更好地节省空间！）”的缩写。

- `.symtab`　符号表，存放程序中定义和引用的函数与全局变量的信息。有些程序员错误地认为必须用 `-g` 选项来编译程序才能获得符号表信息。实际上，每个可重定位目标文件在`.symtab` 中都有一张符号表（除非程序员特意用 `strip` 命令去掉了它）。不过，和编译器中的符号表不同，`.symtab` 符号表不包含局部变量的条目。
- `.rel.text`　`.text` 节中位置的列表，当链接器把这个目标文件与其他文件组合时，会修改这些位置。一般而言，任何调用外部函数或者引用全局变量的指令都需要修改。另一方面，调用本地函数的指令不需要修改。注意，可执行目标文件不需要重定位信息，因此通常不包含它，除非用户显式地要求链接器包含这些信息。
- `.rel.data`　被模块引用或定义的所有全局变量的重定位信息。一般而言，任何初始值是全局变量地址或外部定义函数地址的已初始化全局变量都需要修改。
- `.debug`　调试符号表，其条目是程序中定义的局部变量和类型定义、程序中定义和引用的全局变量，以及原始的 C 源文件。只有在用 `-g` 选项调用编译器驱动程序时才会得到这张表。
- `.line`　原始 C 源程序中的行号和`.text` 节中机器指令之间的映射。只有在用 `-g` 选项调用编译器驱动程序时才会得到这张表。
- `.strtab`　字符串表，其内容包括`.symtab` 和`.debug` 节中的符号表，以及节头部中的节名字。字符串表就是以 null 结尾的字符串的序列。

7.5 符号和符号表

每个可重定位目标模块 m 都有一个符号表，它包含 m 定义和引用的符号的信息。在链接器的上下文中，有三种不同的符号：

- 由模块 m 定义并能被其他模块引用的全局符号。全局链接器符号对应于非静态的 C 函数和全局变量。

- 由模块 m 引用、由其他模块定义的全局符号。这样的符号称为外部符号（external），对应于定义在其他模块中的非静态 C 函数和全局变量。
- 只被模块 m 定义和引用的本地符号。它们对应于带 `static` 属性的 C 函数和全局变量的定义。这些符号在模块 m 中任何位置都可见，但是不能被其他模块引用。

认识到本地链接器符号和本地程序变量不同很重要。.symtab 中的符号表不含本地非静态程序变量的任何符号。这些变量在运行时由栈管理，链接器不关心它们。

有趣的是，定义中带 C `static` 属性的本地过程变量不在栈中管理。相反，编译器在.data 或.bss 中为每个定义分配空间，并在符号表中创建一个有唯一名字的本地链接器符号。例如，假设同一个模块中的两个函数各定义了一个静态局部变量 x：

```
1  int f()
2  {
3      static int x = 0;
4      return x;
5  }
6
7  int g()
8  {
9      static int x = 1;
10    return x;
11 }
```

在这种情况下，编译器向汇编器输出一对名字不同的本地链接器符号。例如，它可能用 x.1 表示函数 f 中的定义，而用 x.2 表示函数 g 中的定义。

符号表是由汇编器构造的，使用编译器输出到汇编语言.s 文件中的符号。ELF 符号表包含在.symtab 节中。它包含一个条目的数组。图 7.4 展示了每个条目的格式。

name 是指向符号的以 null 结尾的字符串名字的字节偏移量（位于字符串表中）。value 是符号的地址。对于可重定位的模块来说，value 是距定义目标的节的起始位置的偏移。对于可执行目标文件来说，value 是一个绝对运行时地址。size 是目标的大小（以字节计）。type 通常要么是数据，要么是函数。符号表还可以包含各个节的条目

> **C 初学者须知　用 static 隐藏变量名和函数名**
> C 程序员使用 `static` 属性把变量和函数声明隐藏在模块内，就像你在 Java 和 C++ 中使用 public 和 private 声明一样。在 C 中，源文件扮演模块的角色。任何用 `static` 属性声明的全局变量或函数都对该模块是私有的。类似地，任何不带 `static` 属性

> 声明的全局变量或函数都是公有的，可以被任何其他模块访问。尽可能用 static 属性保护你的变量和函数是很好的编程习惯。

```
1  typedef struct {
2      int   name;      /* 字符串表偏移 */
3      char type:4,     /* 函数或数据(4 位) */
4           binding:4; /* 局部或全局(4 位) */
5      char reserved;   /* 未使用 */
6      short section;   /* 节头表索引 */
7      long value;      /* 节偏移或绝对地址 */
8      long size;       /* 目标大小(字节) */
9  } Elf64_Symbol;
```

图 7.4 ELF 符号表条目。type 字段和 binding 字段各占 4 位。

以及原始源文件的路径名。所以这些目标也有不同的类型。binding 字段指明这个符号是本地的还是全局的。

每个符号都被分配到目标文件的某个节，由 section 字段表示，它是一个指向节头表的索引。有三个特殊的伪节（pseudosection），它们在节头表中没有条目：ABS 代表不该被重定位的符号。UNDEF 代表未定义的符号，也就是在本目标模块中引用、但是在别处定义的符号。COMMON 代表还未分配位置的未初始化数据目标。对于 COMMON 符号，value 字段给出对齐要求，size 给出最小的大小。要注意，这些伪节只存在于可重定位目标文件中，在可执行目标文件中不存在。

COMMON 和.bss 的区别很微妙。现代版本的 gcc 使用下面的惯例把可重定位目标文件中的符号分配到 COMMON 和.bss：

- `COMMON` 未初始化的全局变量
- `.bss` 未初始化的静态变量，以及初始化为 0 的全局变量或静态变量

这个看似随意的差别源于链接器执行符号解析的方式，我们将在 7.6 节中进行解释。

GNU 的 readelf 程序是查看目标文件内容的一个十分方便的工具。例如，下面是示例程序（图 7.1）的可重定位目标文件 main.o 的最后三个符号表条目。没有显示的头八个条目是链接器内部使用的本地符号。

```
Num:    Value              Size Type      Bind   Vis         Ndx Name
  8: 0000000000000000     24 FUNC    GLOBAL DEFAULT       1 main
  9: 0000000000000000      8 OBJECT  GLOBAL DEFAULT       3 array
 10: 0000000000000000      0 NOTYPE  GLOBAL DEFAULT     UND sum
```

在这个例子中，我们看到一个关于全局符号 main 定义的条目，这是一个 24 字节的函数，位于.text 节中偏移量（即 value）为零处。后面的是全局符号 array

的定义,这是一个 8 字节的目标,位于.data 节中偏移量为零处。最后一个条目来自对外部符号 sum 的引用。readelf 用一个整数索引来标识每个节。Ndx=1 表示.text 节,Ndx=3 表示.data 节。

练习 7.1 本题针对图 7.5 中的 m.o 和 swap.o 模块。对 swap.o 中定义或引用的每个符号,请指出它是否会在模块 swap.o 的.symtab 节中有符号表条目。如果有,请指出定义该符号的模块(swap.o 还是 m.o)、符号类型(local、global 还是 extern),以及它被分配到该模块中的哪个节(.text、.data、.bss 还是 COMMON)。

(a) m.c

```
1  void swap();
2
3  int buf[2] = {1, 2};
4
5  int main()
6  {
7      swap();
8      return 0;
9  }
```

(b) swap.c

```
1  extern int buf[];
2
3  int *bufp0 = &buf[0];
4  int *bufp1;
5
6  void swap()
7  {
8      int temp;
9
10     bufp1 = &buf[1];
11     temp = *bufp0;
12     *bufp0 = *bufp1;
13     *bufp1 = temp;
14 }
```

图 7.5 练习 7.1 的示例程序。

符号	.symtab 条目?	符号类型	定义所在的模块	节
buf				
bufp0				
bufp1				
swap				
temp				

7.6 符号解析

链接器解析符号引用的方法,是把每个引用与它输入的可重定位目标文件的符号表中的一个确定的符号定义关联起来。对那些和引用定义在同一个模块中的本地符号的解析,符号解析是简单明了的。编译器保证每个模块中每个本地符号只有一个定义。编译器还保证静态局部变量(它们也会得到本地链接器符号)拥有唯一的名字。

不过,对全局符号的引用解析要棘手得多。当编译器遇到一个不在当前模块中定义的符号(变量或函数名)时,它会假设该符号是在某个其他模块中定义的,生成一个链接器符号表条目,并把它留给链接器处理。如果链接器在它的任何输入模块中都找不到被引用符号的定义,它就会打印一条(通常很难懂的)错误信息并终止。例如,如果我们在一台 Linux 机器上试图编译和链接下面的源文件,

```
1  void foo(void);
2
3  int main() {
4      foo();
5      return 0;
6  }
```

那么编译可以毫无障碍地进行,但是当链接器无法解析对 foo 的引用时,它会终止:

```
linux> gcc -Wall -Og -o linkerror linkerror.c
/tmp/ccSz5uti.o: In function ‘main’:
/tmp/ccSz5uti.o(.text+0x7): undefined reference to ‘foo’
```

对全局符号的解析之所以棘手,还因为多个目标模块可能定义名字相同的全局符号。在这种情况下,链接器要么标出一个错误,要么以某种方式选择一个定义并抛弃其他定义。Linux 系统采取的方法涉及编译器、汇编器和链接器之间的协作,可能会给粗心的程序员带来一些莫名其妙的 bug。

边栏 C++ 和 Java 中链接器符号的修饰(mangling)

C++ 和 Java 都支持重载的方法,这些方法在源代码中名字相同,参数列表却不同。那么链接器是如何区分这些不同的重载函数的呢?C++ 和 Java 中重载函数能够工作,是因为编译器将每个唯一的方法和参数列表组合编码成一个唯一的名字给链接器使用。这种编码过程称为修饰(mangling),其逆过程称为解修饰(demangling)。

可喜的是,C++ 和 Java 使用了兼容的修饰方案。被修饰的类名由名字中字符的整数个数加上原始名字构成。例如,类 Foo 被编码为 3Foo。一个方法被编码为原始方法名,后面跟上 __,再加上被修饰的类名,然后是每个参数的单字母编码。例如,

> Foo::bar(int, long) 被编码为 bar__3Fooil。修饰全局变量名和模板名时，会使用类似的方法。

7.6.1 链接器如何解析重名符号

链接器的输入是一组可重定位目标模块。每个模块都定义一组符号，有些是本地的（只对定义它的模块可见），有些是全局的（对其他模块也可见）。如果多个模块定义名字相同的全局符号会发生什么？下面是 Linux 编译系统采用的方法。

在编译时，编译器向汇编器输出一个全局符号，该符号要么是强（strong）的，要么是弱（weak）的，汇编器把信息隐式地编码在可重定位目标文件的符号表里。函数和已初始化的全局变量是强符号。未初始化的全局变量是弱符号。

关于强符号和弱符号的概念，Linux 链接器用下面的规则来处理重名符号：

- **规则 1** 不允许多个名字相同的强符号。
- **规则 2** 如果一个强符号和多个弱符号名字相同，那么选择强符号。
- **规则 3** 如果多个弱符号名字相同，那么任意选择一个弱符号。

例如，假设我们试图编译和链接下面两个 C 模块：

```
1  /* foo1.c */
2  int main()
3  {
4      return 0;
5  }
```

```
1  /* bar1.c */
2  int main()
3  {
4      return 0;
5  }
```

这种情况下，链接器将产生一条错误信息，因为强符号 main 被定义了多次（规则 1）：

```
linux> gcc foo1.c bar1.c
/tmp/ccq2Uxnd.o: In function ‘main’:
bar1.c:(.text+0x0): multiple definition of ‘main’
```

类似地，对于下面的模块，链接器也会产生一条错误信息，因为强符号 x 被定义了两次（规则 1）：

```
1  /* foo2.c */
2  int x = 15213;
```

```
3
4  int main()
5  {
6      return 0;
7  }
```

```
1  /* bar2.c */
2  int x = 15213;
3
4  void f()
5  {
6  }
```

不过，如果 x 在一个模块中未被初始化，那么链接器将平静地选择另一个模块中定义的强符号（规则 2）：

```
1  /* foo3.c */
2  #include <stdio.h>
3  void f(void);
4
5  int x = 15213;
6
7  int main()
8  {
9      f();
10     printf("x = %d\n", x);
11     return 0;
12 }
```

```
1  /* bar3.c */
2  int x;
3
4  void f()
5  {
6      x = 15212;
7  }
```

运行时，函数 f 将 x 的值从 15213 改为 15212，这对写 `main` 函数的人来说可能是个不受欢迎的意外！注意，链接器通常不会给出关于检测到 x 多重定义的任何提示：

```
linux> gcc -o foobar3 foo3.c bar3.c
linux> ./foobar3
x = 15212
```

如果 x 有两个弱定义，同样的事情也会发生（规则 3）：

```
1  /* foo4.c */
2  #include <stdio.h>
3  void f(void);
4
5  int x;
6
7  int main()
8  {
9      x = 15213;
10     f();
11     printf("x = %d\n", x);
12     return 0;
13 }
```

```
1  /* bar4.c */
2  int x;
3
4  void f()
5  {
6      x = 15212;
7  }
```

规则 2 和规则 3 的使用会引入一些非常阴险的运行时 bug，粗心的程序员很难察觉，尤其是当重复的符号定义具有不同类型时。考虑下面的例子，一个模块中 x 不小心被定义成了 `int`，而在另一个模块中被定义成了 `double`：

```
1  /* foo5.c */
2  #include <stdio.h>
3  void f(void);
4
5  int y = 15212;
6  int x = 15213;
7
8  int main()
9  {
10     f();
```

```
11     printf("x = 0x%x  y = 0x%x  \n",
12            x, y);
13     return 0;
14 }
```

```
1  /* bar5.c */
2  double x;
3
4  void f()
5  {
6      x = -0.0;
7  }
```

在 x86-64/Linux 机器上, double 是 8 字节, int 是 4 字节。在我们的系统上, x 的地址是 0x601020, y 的地址是 0x601024。因此, bar5.c 第 6 行的赋值 $x = -0.0$ 将用负零的双精度浮点表示覆盖 x 和 y(foo5.c 第 5 和第 6 行)的内存位置!

```
linux> gcc -Wall -Og -o foobar5 foo5.c bar5.c
/usr/bin/ld: Warning: alignment 4 of symbol ‘x’ in /tmp/cclUFK5g.o
is smaller than 8 in /tmp/ccbTLcb9.o
linux> ./foobar5
x = 0x0  y = 0x80000000
```

这是一个细微而可怕的 bug, 尤其是因为它只引发链接器的一个警告, 还因为它通常发生在程序执行中离出错很远之后的某个时刻。在一个有几百个模块的大型系统中, 这种 bug 极难修复, 尤其是因为许多程序员并不了解链接器的工作方式,而且他们经常忽视编译器的警告。有疑问时,可以带上诸如 gcc 的 -fno-common 这样的标志来调用链接器,它会在遇到多重定义的全局符号时触发一个错误。或者使用 -Werror 选项,它把所有警告都变成错误。

在 7.5 节中, 我们看到编译器用一个看似随意的惯例把符号分配到 COMMON 和.bss。实际上,这个惯例是因为在某些情况下链接器允许多个模块定义名字相同的全局符号。当编译器翻译某个模块时遇到一个弱全局符号,比如 x, 它不知道其他模块是否也定义了 x, 如果是, 它也无法预测链接器可能选择 x 多个实例中的哪一个。所以编译器通过把 x 分配到 COMMON 以及把决定权推迟给链接器来解决这个问题。另一方面,如果 x 被初始化为 0,那么它就是一个强符号(因此根据规则 2 必须唯一),编译器可以放心地把它分配到.bss。类似地, 静态符号按其构造就是唯一的, 所以编译器可以放心地把它们分配到.data 或.bss。

在本题中,令 REF(x.i) → DEF(x.k) 表示链接器将把模块 i 中对符号 x 的任意引用与模块 k 中 x 的定义关联起来。对下面的每个例子,用这种记号指出链接器会如何解析每个模块中对多重定义符号的引用。如果出现链接时错误(规则 1),写“error”。如果链接器任意选择其中一个定义(规则 3),写“unknown”。

```
/* Module 1 */    /* Module 2 */
int main()        int main;
{                 int p2()
}                 {
                  }

(a) REF(main.1) → DEF(        .        )
(b) REF(main.2) → DEF(        .        )
```

```
/* Module 1 */    /* Module 2 */
```

```
void main()       int main = 1;
{                 int p2()
}                 {
                  }

(a) REF(main.1) → DEF(        .        )
(b) REF(main.2) → DEF(        .        )
```

```
/* Module 1 */    /* Module 2 */
int x;            double x = 1.0;
void main()       int p2()
{                 {
}                 }

(a) REF(x.1) → DEF(           .        )
(b) REF(x.2) → DEF(           .        )
```

7.6.2 与静态库链接

到目前为止，我们一直假设链接器读取一组可重定位目标文件，并把它们链接起来形成一个输出的可执行文件。实际上，所有的编译系统都提供一种机制，把相关的目标模块打包成一个称为静态库（static library）的单一文件，它可以用作链接器的输入。构建输出的可执行文件时，链接器只复制库里被应用程序引用的目标模块。

为什么系统要支持库的概念？考虑 ISO C99，它定义了一整套标准 I/O、字符串操作和整数数学函数，比如 atoi、printf、scanf、strcpy 和 rand。它们对每个 C 程序都是可用的，放在 libc.a 库中。ISO C99 还在 libm.a 库中定义了一整套浮点数学函数，比如 sin、cos 和 sqrt。

考虑一下编译器开发者在没有静态库好处的情况下可以采用哪些不同做法来向用户提供这些函数。一种做法是让编译器识别对标准函数的调用，并直接生成相应的代码。Pascal 采用的就是这种做法，它提供了一小组标准函数，但这对 C 来说不可行，因为 C 标准定义大量的标准函数。这会给编译器增加显著的复杂性，而且每次添加、删除或修改一个函数都需要一个新版本的编译器。不过，对应用编程的人来说，这种做法将十分方便，因为标准函数总是可用的。

另一种做法是把所有标准 C 函数都放在一个可重定位目标模块中，比如 libc.o，应用编程的人把它的实现链接到它们的可执行文件中：

```
linux> gcc main.c /usr/lib/libc.o
```

这种做法的优点是，它将标准函数的实现与编译器的实现解耦，而且对程序员来说仍然相当方便。不过，一个很大的缺点是，系统上每个可执行文件现在都会包含标准函数集合的一个完整副本，这会极端浪费磁盘空间。（在我们的

系统上，libc.a 大约 5 MB，而 libm.a 大约 2 MB。）更糟的是，每个正在运行的程序现在都会把这些函数的副本放在自己的内存中，这极端浪费内存。另一个大缺点是，任何对任何标准函数的修改，无论多么小，都要求库开发者重新编译整个源文件，这是一个耗时的操作，会使标准函数的开发和维护复杂化。

我们可以通过为每个标准函数创建一个独立的可重定位文件，并把它们存放在一个众所周知的目录中来解决这些问题中的一部分。不过，这种方法要求应用程序员显式地将相关的目标模块链接到他们的可执行文件中，这个过程容易出错而且耗时：

```
linux> gcc main.c /usr/lib/printf.o /usr/lib/scanf.o ...
```

静态库（static library）概念的提出就是为了消除这些不同方法的缺点。相关的函数可以被编译为独立的目标模块，然后打包成一个单独的静态库文件。应用程序可以用在命令行上指定单个文件名的方式来使用库中定义的任何函数。比如，一个使用 C 标准库和数学库中函数的程序可以用如下形式的命令来编译和链接：

```
linux> gcc main.c /usr/lib/libm.a /usr/lib/libc.a
```

```
(a) addvec.o

1  int addcnt = 0;
2
3  void addvec(int *x, int *y,
4              int *z, int n)
5  {
6      int i;
7
8      addcnt++;
9
10     for (i = 0; i < n; i++)
11         z[i] = x[i] + y[i];
12 }
```

```
(b) multvec.o

1  int multcnt = 0;
2
3  void multvec(int *x, int *y,
4               int *z, int n)
5  {
6      int i;
7
8      multcnt++;
9
10     for (i = 0; i < n; i++)
11         z[i] = x[i] * y[i];
12 }
```

图 7.6 libvector 库中的成员目标文件。

链接时，链接器只复制程序引用的目标模块，这会减小可执行文件在磁盘和内存中的大小。另一方面，应用编程的人只需要包含少数几个库文件的名字。（实际上，C 编译器驱动程序总是把 libc.a 传给链接器，所以前面提到的对 libc.a 的引用是不必要的。）

在 Linux 系统中，静态库以一种称为存档（archive）的特殊文件格式存放在磁盘上。存档是一组连接起来的可重定位目标文件，有一个头部用来描述每个成员目标文件的大小和位置。存档文件名以.a 后缀标明。

为了使关于库的讨论更具体，考虑图 7.6 中的一对向量例程。每个例程定义在自己的目标模块中，对一对输入向量执行一个向量操作，并把结果放到一个输出向量中。作为副作用，每个例程通过递增一个全局变量来记录它被调用的次数。（当我们解释 7.12 节中与位置无关代码的思想时，这会很有用。）

要创建这些函数的静态库，我们会像下面这样使用 ar 工具：

```
linux> gcc -c addvec.c multvec.c
linux> ar rcs libvector.a addvec.o multvec.o
```

要使用这个库，我们可以写一个应用程序，比如图 7.7 中的 main2.c，它调用了 addvec 库例程。include（头）文件 vector.h 定义了 libvector.a 中例程的函数原型。

要构建可执行文件，我们会编译并链接输入文件 main2.o 和 libvector.a：

```
linux> gcc -c main2.c
linux> gcc -static -o prog2c main2.o ./libvector.a
```

```
1  #include <stdio.h>
2  #include "vector.h"
3
4  int x[2] = {1, 2};
5  int y[2] = {3, 4};
6  int z[2];
7
8  int main()
9  {
10     addvec(x, y, z, 2);
11     printf("z = [%d %d]\n", z[0], z[1]);
12     return 0;
13 }
```

图 7.7 示例程序 2。该程序调用 libvector 库中的一个函数。

或者等价地：

```
linux> gcc -c main2.c
linux> gcc -static -o prog2c main2.o -L. -lvector
```

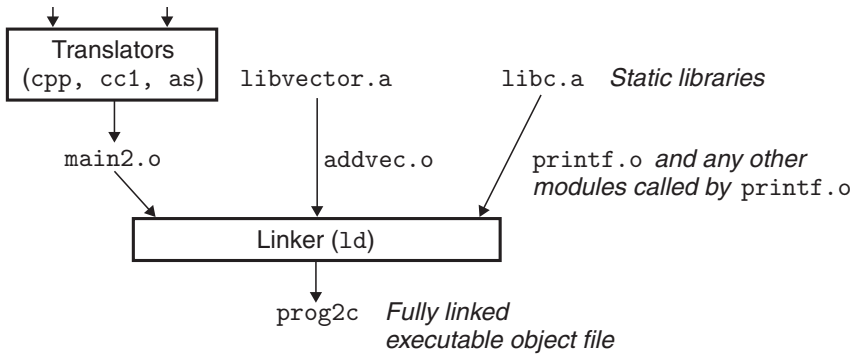


图 7.8 与静态库链接。

图 7.8 概括了链接器的活动。`-static` 参数告诉编译器驱动程序，链接器应该构建一个完全链接的可执行目标文件，它可以被加载到内存中并运行，在加载时无需再进行任何链接。`-lvector` 参数是 libvector.a 的简写，`-L.` 参数告诉链接器在当前目录中查找 libvector.a。

当链接器运行时，它判定 addvec.o 定义的 addvec 符号被 main2.o 引用，所以它就把 addvec.o 复制到可执行文件中。

因为程序不引用 multvec.o 定义的任何符号，所以链接器不会把这个模块复制到可执行文件。链接器还从 libc.a 复制 printf.o 模块，以及来自 C 运行时系统的其他一些模块。

7.6.3 链接器如何使用静态库解析引用

静态库虽然有用，但会让程序员感到困惑，因为 Linux 链接器以特殊的方式使用静态库来解析外部引用。在符号解析阶段，链接器按编译器驱动程序命令行上它们出现的同样顺序从左到右扫描可重定位目标文件和存档。（驱动程序自动把命令行上所有的.c 文件翻译成.o 文件。）在这个过程中，链接器维护一个可重定位目标文件的集合 E，这些文件会被合并起来形成可执行文件；一个未解析符号的集合 U（也就是那些被引用但尚未定义的符号）；以及一个 D 集合，包含所有前一个输入文件中定义的符号。一开始，E、U 和 D 都是空的。

- 对于命令行上的每个输入文件 f，链接器会判断 f 是一个目标文件还是一个存档。如果 f 是目标文件，链接器把 f 添加到 E，更新 U 和 D 以反映 f 中的符号定义和引用，并继续处理下一个输入文件。
- 如果 f 是一个存档，链接器会尝试将 U 中未解析的符号与存档成员定义的符号相匹配。如果某个存档成员 m 定义了一个符号能解析 U 中的一个引用，那么就将 m 添加到 E，并且链接器更新 U 和 D 以反映 m 中的符号定义和引用。这个过程会迭代存档中的目标文件，直至达到一个不动点，

U 和 D 不再变化。此时，任何不在 E 中的成员目标文件都直接被丢弃，链接器继续处理下一个输入文件。

- 如果链接器完成扫描命令行上的输入文件后 U 非空，它就打印一条错误信息并终止。否则，它合并并重定位 E 中的目标文件，从而构建输出的可执行文件。

不幸的是，这种算法可能导致令人困惑的链接时错误，因为命令行上库和目标文件的顺序是有影响的。如果定义一个符号的库出现在命令行上引用该符号的目标文件之前，那么引用就无法解析，链接就会失败。例如，考虑下面的情况：

```
linux> gcc -static ./libvector.a main2.c
/tmp/cc9XH6Rp.o: In function ‘main’:
/tmp/cc9XH6Rp.o(.text+0x18): undefined reference to ‘addvec’
```

发生了什么？在处理 libvector.a 时，U 是空的，所以没有把 libvector.a 中的成员目标文件添加到 E。因此，对 addvec 的引用从来没有被解析过，链接器发出一条错误信息并终止。

对库的一般准则是：把库放在命令行的最后。如果不同库的成员之间是独立的，即没有成员引用另一个成员定义的符号，那么这些库在命令行上可以按任意顺序放在任何位置。另一方面，如果这些库不是独立的，那么必须对它们排序，使得对于每个被存档成员外部引用的符号 s，在命令行上必须至少有一个 s 的定义跟在 s 的引用之后。例如，假设 foo.c 调用 libx.a 和 libz.a 中的函数，而这两个库又调用 liby.a 中的函数，那么命令行上 libx.a 和 libz.a 必须在 liby.a 之前：

```
linux> gcc foo.c libx.a libz.a liby.a
```

如果有必要满足依赖关系的要求，库可以在命令行上重复。例如，假设 foo.c 调用一个 libx.a 中的函数，而这个函数调用 liby.a 中的函数，而 liby.a 中的这个函数又调用了 libx.a 中的函数。那么 libx.a 必须在命令行上重复出现：

```
linux> gcc foo.c libx.a liby.a libx.a
```

我们也可以把 libx.a 和 liby.a 合并到一个单一存档中。

令 a 和 b 表示当前目录中的目标模块或静态库，令 $a \to b$ 表示 a 依赖 b，即 b 定义了 a 引用的一个符号。对下面每种情况，给出能让静态链接器解析所有符号引用的最小命令行（也就是说，目标文件和库参数的个数最少）：

- **A.** `p.o` $\to$ `libx.a`
- **B.** `p.o` $\to$ `libx.a` $\to$ `liby.a`
- **C.** `p.o` $\to$ `libx.a` $\to$ `liby.a`，且 `liby.a` $\to$ `libx.a` $\to$ `p.o`

7.7 重定位

一旦链接器完成了符号解析这一步，它就把代码中的每个符号引用和确定的一个符号定义（也就是它的输入目标模块之一中的一个符号表条目）关联起来了。此时，链接器知道它的输入目标模块中的代码节和数据节的确切大小。现在它已准备好开始重定位步骤了，在这个步骤中，它合并输入模块并为每个符号分配运行时地址。重定位由两步组成：

1. 重定位节和符号定义。在这一步，链接器把所有相同类型的节合并成同一类型的新聚合节。例如，来自输入模块的.data 节都被合并成一个节，这个节会成为输出的可执行目标文件的.data 节。链接器把运行时内存地址分配给新的聚合节，分配给输入模块定义的每个节，以及分配给输入模块定义的每个符号。当这一步完成时，程序中的每条指令和每个全局变量都有唯一的运行时内存地址。

2. 重定位节中的符号引用。在这一步，链接器修改代码节和数据节中每个符号引用，使得它们指向正确的运行时地址。为了执行这一步，链接器依赖于可重定位目标模块中称为重定位条目（relocation entry）的数据结构，我们接下来会描述它。

7.7.1 重定位条目

汇编器在生成一个目标模块时并不知道代码和数据最终在内存中的位置。它也不知道模块引用的任何外部定义函数或全局变量的位置。所以，无论何时汇编器遇到对最终位置未知的目标的引用，它都会生成一个重定位条目，告诉链接器在将目标文件合并成可执行文件时如何修改这个引用。代码的重定位条目放在.rel.text 中。数据的重定位条目放在.rel.data 中。

图 7.9 展示了 ELF 重定位条目的格式。offset 是需要被修改的引用的节偏移。symbol 标记被修改引用应该指向的符号。type 告诉链接器如何修改新的引用。addend 是一个有符号常数，一些重定位类型用它对被修改引用的值作偏置。

ELF 定义了 32 种不同的重定位类型，许多都相当深奥。我们只关心其中两种最基本的重定位类型：

R_X86_64_PC32。重定位一个使用 32 位 PC 相对地址的引用。回忆一下 3.6.3 节，PC 相对地址是一个相对当前程序计数器（PC）运行时值的偏移。当 CPU 执行一条用 PC 相对寻址的指令时，它形成的有效地址（例如 call 指令的目标）是把指令中编入

```
1  typedef struct {
2      long offset;    /* 待重定位引用的偏移 */
```

```
3      long type:32,   /* 重定位类型 */
4           symbol:32; /* 符号表索引 */
5      long addend;    /* 重定位表达式中的常数部分 */
6  } Elf64_Rela;
```

图 7.9 ELF 重定位条目。每个条目标记一个必须被重定位的引用，并指定如何计算被修改的引用。

的 32 位值与 PC 的当前运行时值相加，而 PC 的当前运行时值总是内存中下一条指令的地址。

R_X86_64_32。重定位一个使用 32 位绝对地址的引用。在绝对寻址下，CPU 把指令中编入的 32 位值直接用作有效地址，不再做任何修改。

这两种重定位类型支持 x86-64 小代码模型（small code model），它假定可执行目标文件中代码和数据的总大小小于 2 GB，因此可以在运行时用 32 位 PC 相对地址来访问。小代码模型是 `gcc` 的默认设置。大于 2 GB 的程序可以用 `-mcmodel=medium`（中等代码模型）和 `-mcmodel=large`（大代码模型）标志来编译，但我们不讨论这两个模型。

7.7.2 重定位符号引用

图 7.10 给出了链接器重定位算法的伪代码。第 1 行和第 2 行对每个节 s 以及与每个节相关联的每个重定位条目 r 进行迭代。为具体起见，假设每个节 s 是一个字节数组，而每个重定位条目 r 是图 7.9 中定义的 `Elf64_Rela` 类型的结构。另外，假设算法运行时，链接器已经为每个节（记作 `ADDR(s)`）和每个符号（记作 `ADDR(r.symbol)`）选定了运行时地址。第 3 行计算需要被重定位的 4 字节引用在 s 数组中的地址。如果这个引用使用 PC 相对寻址，那么由第 5–9 行重定位它。如果引用使用绝对寻址，那么由第 11–13 行重定位它。

```
1  foreach section s {
2      foreach relocation entry r {
3          refptr = s + r.offset;          /* 指向待重定位引用的指针 */
4
5          /* 重定位 PC 相对引用 */
6          if (r.type == R_X86_64_PC32) {
7              refaddr = ADDR(s) + r.offset;   /* 引用的运行时地址 */
8              *refptr = (unsigned) (ADDR(r.symbol) + r.addend -
       refaddr);
9          }
10
11         /* 重定位绝对引用 */
12         if (r.type == R_X86_64_32)
13             *refptr = (unsigned) (ADDR(r.symbol) + r.addend);
14     }
15 }
```

图 7.10 重定位算法。

```
code/link/main-relo.d

1     0000000000000000 <main>:
2        0:   48 83 ec 08             sub    $0x8,%rsp
3        4:   be 02 00 00 00          mov    $0x2,%esi
4        9:   bf 00 00 00 00          mov    $0x0,%edi
        %edi = &array
5                             a: R_X86_64_32 array
        重定位条目
6        e:    e8 00 00 00 00                  callq 13 <main+0x13>
        sum()
7                                    f: R_X86_64_PC32 sum-0x4
        重定位条目
8       13:    48 83 c4 08                     add    $0x8,%rsp
9       17:    c3                              retq
```

图 7.11 main.o 的代码与重定位条目。原始 C 代码见图 7.1。

下面来看看链接器如何运用这个算法，重定位图 7.1 中示例程序里的引用。图 7.11 给出了 GNU objdump 工具(`objdump -dx main.o`)生成的反汇编代码。

main 函数引用了两个全局符号 array 和 sum。对每个引用，汇编器都生成了一个重定位条目，显示在下一行中。[2]重定位条目告诉链接器：对 sum 的引用应当被重定位为 32 位 PC 相对地址，而对 array 的引用应当被重定位为 32 位绝对地址。接下来两个小节将详细介绍链接器如何重定位这些引用。

重定位 PC 相对引用

在图 7.11 的第 6 行中，main 函数调用了 sum 函数，sum 定义在模块 sum.o 中。call 指令从节偏移 0xe 处开始，由单字节操作码 0xe8 后跟一个用于存放 32 位 PC 相对引用的占位符构成，其目标是 sum。

与之相应的重定位条目 r 由四个字段组成：

```
r.offset = 0xf
r.symbol = sum
  r.type = R_X86_64_PC32
r.addend = -4
```

[2]回忆一下，重定位条目和指令实际上存储在目标文件的不同节中。objdump 工具为了方便起见把两者放在一起显示。

这些字段告诉链接器：修改从偏移 0xf 开始的 32 位 PC 相对引用，使得它在运行时指向 sum 例程。现在，假设链接器已经确定了

$$\texttt{ADDR(s)} = \texttt{ADDR(.text)} = \texttt{0x4004d0}$$

以及

$$\texttt{ADDR(r.symbol)} = \texttt{ADDR(sum)} = \texttt{0x4004e8}$$

使用图 7.10 中的算法，链接器首先计算引用的运行时地址（第 7 行）：

$$\begin{aligned}\texttt{refaddr} &= \texttt{ADDR(s)} + \texttt{r.offset} \\ &= \texttt{0x4004d0} + \texttt{0xf} \\ &= \texttt{0x4004df}\end{aligned}$$

然后它更新该引用，使得它在运行时指向 sum 例程（第 8 行）：

$$\begin{aligned}\texttt{*refptr} &= \texttt{(unsigned) (ADDR(r.symbol) + r.addend - refaddr)} \\ &= \texttt{(unsigned) (0x4004e8 + (-4) - 0x4004df)} \\ &= \texttt{(unsigned) (0x5)}\end{aligned}$$

在最终的可执行目标文件中，call 指令呈如下重定位后的形式：

```
4004de:    e8 05 00 00 00              callq   4004e8 <sum>
      sum()
```

在运行时，call 指令将位于地址 0x4004de 处。当 CPU 执行 call 指令时，PC 的值为 0x4004e3，即紧随 call 指令之后那条指令的地址。为了执行 call 指令，CPU 执行以下步骤：

- 把 PC 压入栈
- $\texttt{PC} \leftarrow \texttt{PC} + \texttt{0x5} = \texttt{0x4004e3} + \texttt{0x5} = \texttt{0x4004e8}$

于是，接下来要执行的就是 sum 例程的第一条指令，这当然正是我们想要的！

重定位绝对引用

重定位绝对引用的过程很简单。例如，在图 7.11 的第 4 行中，mov 指令把 array 的地址（一个 32 位立即数）复制到寄存器%edi 中。mov 指令从节偏移 0x9 处开始，由单字节操作码 0xbf 后跟一个用于存放指向 array 的 32 位绝对引用的占位符构成。

与之相应的重定位条目 r 由四个字段组成:

$$\begin{aligned} \texttt{r.offset} &= \texttt{0xa} \\ \texttt{r.symbol} &= \texttt{array} \\ \texttt{r.type} &= \texttt{R_X86_64_32} \\ \texttt{r.addend} &= \texttt{0} \end{aligned}$$

这些字段告诉链接器: 修改从偏移 0xa 开始的绝对引用, 使得它在运行时指向 array 的第一个字节。现在, 假设链接器已经确定了

```
(a) 重定位后的 .text 节
 1    00000000004004d0 <main>:
 2      4004d0: 48 83 ec 08                    sub     $0x8,%rsp
 3      4004d4: be 02 00 00 00                 mov     $0x2,%esi
 4      4004d9: bf 18 10 60 00                 mov     $0x601018,%edi
        %edi = &array
 5      4004de: e8 05 00 00 00                 callq   4004e8 <sum>
        sum()
 6      4004e3: 48 83 c4 08                    add     $0x8,%rsp
 7      4004e7: c3                             retq

 8    00000000004004e8 <sum>:
 9      4004e8: b8 00 00 00 00                 mov    $0x0,%eax
10      4004ed: ba 00 00 00 00                 mov    $0x0,%edx
11      4004f2: eb 09                          jmp    4004fd
 ↪  <sum+0x15>
12      4004f4: 48 63 ca                       movslq %edx,%rcx
13      4004f7: 03 04 8f                       add
        (%rdi,%rcx,4),%eax
14      4004fa: 83 c2 01                       add    $0x1,%edx
15      4004fd: 39 f2                          cmp    %esi,%edx
16      4004ff: 7c f3                          jl     4004f4 <sum+0xc>
17      400501: f3 c3                          repz retq

(b) 重定位后的 .data 节
 1    0000000000601018 <array>:
 2      601018: 01 00 00 00 02 00 00 00
```

图 7.12 可执行文件 prog 重定位后的.text 节与.data 节。原始 C 代码见图 7.1。

$$\texttt{ADDR(r.symbol)} = \texttt{ADDR(array)} = \texttt{0x601018}$$

链接器用图 7.10 中算法的第 13 行来更新该引用:

$$\begin{aligned} \texttt{*refptr} &= \texttt{(unsigned) (ADDR(r.symbol) + r.addend)} \\ &= \texttt{(unsigned) (0x601018 + 0)} \\ &= \texttt{(unsigned) (0x601018)} \end{aligned}$$

在最终的可执行目标文件中,该引用呈如下重定位后的形式:

```
4004d9:   bf 18 10 60 00              mov     $0x601018,%edi
      %edi = &array
```

把所有内容合到一起，图 7.12 展示了最终可执行目标文件中重定位后的.text 节与.data 节。在加载时,加载器可以把这些节中的字节直接复制到内存中,并执行其中的指令,而不需要再做任何进一步的修改。

练习 7.4 本题考察图 7.12(a) 中重定位后的程序。

A. 第 5 行中重定位后的 sum 引用的十六进制地址是多少?

B. 第 5 行中重定位后的 sum 引用的十六进制值是多少?

练习 7.5 考察目标文件 m.o(图 7.5)中对函数 swap 的调用。

```
  9:    e8 00 00 00 00                  callq   e <main+0xe>
      swap()
```

它带如下的重定位条目:

$$\begin{aligned} \texttt{r.offset} &= \texttt{0xa} \\ \texttt{r.symbol} &= \texttt{swap} \\ \texttt{r.type} &= \texttt{R_X86_64_PC32} \\ \texttt{r.addend} &= \texttt{-4} \end{aligned}$$

现在假设链接器把 m.o 中的.text 重定位到地址 0x4004d0,把 swap 重定位到地址 0x4004e8。那么 callq 指令中重定位后的 swap 引用的值是多少?

7.8 可执行目标文件

我们已经看到,链接器是如何把多个目标文件合并成单个可执行目标文件的。我们的示例 C 程序最初是一组 ASCII 文本文件,现在已经变成一个单独的二进制文件,其中包含把程序加载到内存中并运行它所需的全部信息。图 7.13 概括了典型的 ELF 可执行文件中各类信息。

```
code/link/prog-exe.d
```

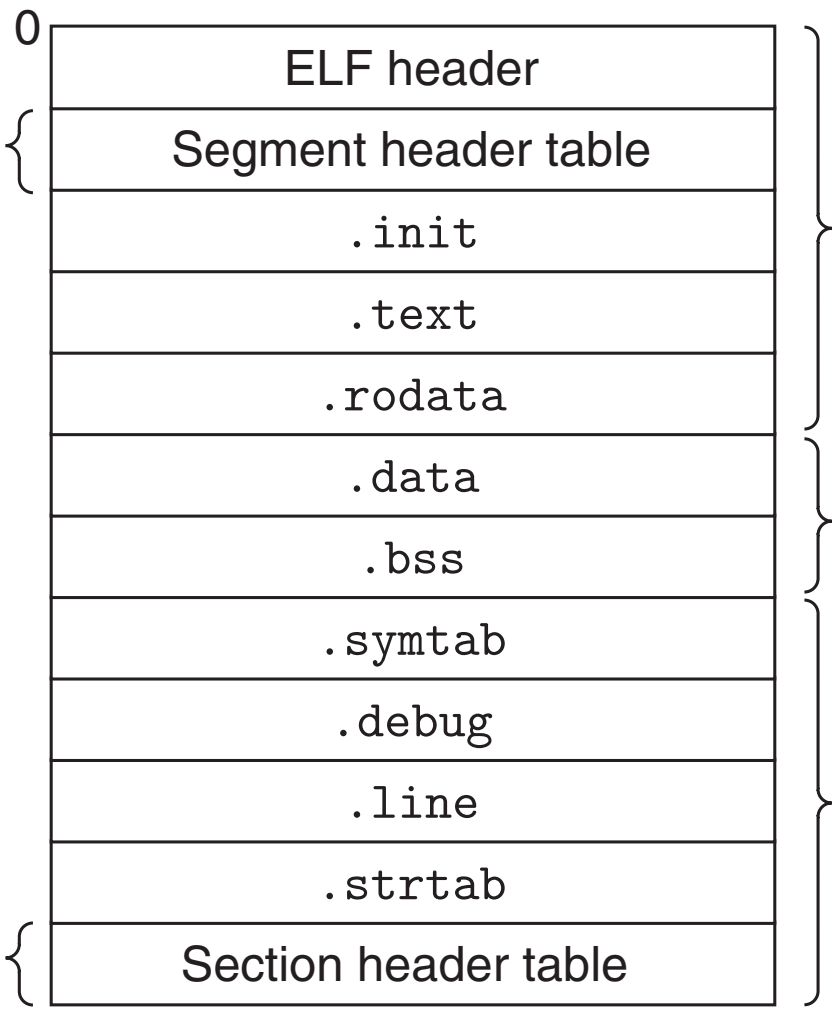


图 7.13 典型的 ELF 可执行目标文件。

```
    只读代码段
1     LOAD off    0x0000000000000000 vaddr 0x0000000000400000 paddr
        0x0000000000400000 align 2**21
2           filesz 0x000000000000069c memsz 0x000000000000069c flags r-x

    可读写数据段
3     LOAD off    0x00000000000000df8 vaddr 0x0000000000600df8 paddr
        0x0000000000600df8 align 2**21
4           filesz 0x0000000000000228 memsz 0x0000000000000230 flags rw-
```

图 7.14 示例可执行文件 prog 的程序头部表。off：目标文件中的偏移；vaddr/paddr：内存地址；align：对齐要求；filesz：目标文件中的段大小；memsz：内存中的段大小；flags：运行时权限。

可执行目标文件的格式与可重定位目标文件的格式相似。ELF 头描述文件的总体格式，还包括程序的入口点（entry point），即程序运行时要执行的第一条指令的地址。.text、.rodata 和.data 节与可重定位目标文件中的那些节类似，只是这些节已经被重定位到它们最终的运行时内存地址。.init 节定义了一个名为 `_init` 的小函数，程序的初始化代码会调用它。由于可执行文件已经被完整地链接（重定位），它不再需要.rel 节。

ELF 可执行文件的设计目标就是便于加载到内存，可执行文件中连续的块被映射到连续的内存段。这一映射关系由程序头部表（program header table）描述。图 7.14 给出了 objdump 显示的示例可执行文件 prog 的程序头部表的一部分。

从程序头部表中可以看到，两个内存段将用可执行目标文件的内容来初始化。第 1 行和第 2 行告诉我们：第一个段（代码段）具有读/执行权限，从内存地址 0x400000 处开始，在内存中的总大小为 0x69c 字节，由可执行目标文件的前 0x69c 字节初始化，其中包括 ELF 头、程序头部表以及.init、.text 和.rodata 节。

第 3 行和第 4 行告诉我们：第二个段（数据段）具有读/写权限，从内存地址 0x600df8 处开始，在内存中的总大小为 0x230 字节，由目标文件中从偏移 0xdf8 处开始的.data 节中的 0x228 字节初始化。该段中剩下的 8 字节对应于.bss 中的数据，它们将在运行时被初始化为零。

对于任意一个段 s，链接器必须选择一个起始地址 vaddr，使得

$$\texttt{vaddr} \bmod \texttt{align} = \texttt{off} \bmod \texttt{align}$$

其中，off 是该段的第一个节在目标文件中的偏移，而 align 是程序头部指定的对齐要求（$2^{21} = \texttt{0x200000}$）。例如，对于图 7.14 中的数据段，

$$\texttt{vaddr} \bmod \texttt{align} = \texttt{0x600df8} \bmod \texttt{0x200000} = \texttt{0xdf8}$$

$$\texttt{off} \bmod \texttt{align} = \texttt{0xdf8} \bmod \texttt{0x200000} = \texttt{0xdf8}$$

这个对齐要求是一项优化，它使目标文件中的段在程序运行时能够被高效地传送到内存。其原因有一点微妙，它与虚拟内存被组织成较大的、连续的 2 的幂字节块的方式有关。第 9 章会完整地讲述虚拟内存。

7.9 加载可执行目标文件

要运行一个可执行目标文件 prog，可以在 Linux shell 的命令行上键入它的名字：

```
linux> ./prog
```

由于 prog 不是某个 shell 内置命令的名字，shell 会假定 prog 是一个可执行目标文件，并通过调用某段常驻内存的操作系统代码（称为加载器）来替我们运行它。任何 Linux 程序都可以通过调用 execve 函数来调用加载器，我们将在 8.4.6 节详细描述 execve。加载器把可执行目标文件中的代码和数据从磁盘复制到内存中，然后跳转到程序的第一条指令（即入口点）运行该程序。把程序复制到内存并运行它的这一过程称为加载。

每个正在运行的 Linux 程序都有一幅与图 7.15 相似的运行时内存映像。在 Linux x86-64 系统中，代码段从地址 0x400000 处开始，后面是数据段。运行时堆紧跟在数据段之后，并通过调用 malloc 库向上生长。（我们将在 9.9 节详细描述 malloc 和堆。）堆之后是为共享模块保留的一个区域。用户栈从最大的合法用户地址（$2^{48}-1$）起向下生长，朝向更小的内存地址。栈上方、从地址 2^{48} 开始的区域保留给内核中的代码和数据，而内核是操作系统中常驻内存的部分。

为简单起见，我们把堆、数据和代码段画成彼此相邻的样子，并把栈顶放在最大的合法用户地址处。实际情况中，由于.data 节的对齐要求（7.8 节），代码段和数据段之间存在间隙。此外，链接器在为栈、共享库和堆段分配运行时地址时，使用了地址空间布局随机化（address-space layout randomization，简称 ASLR，见 3.10.4 节）。虽然这些区域的位置在每次运行程序时都会变化，但它们的相对位置是相同的。

当加载器运行时，它会创建一个与图 7.15 所示相似的内存映像。它以程序头部表为指引，把可执行目标文件的分块复制到代码段和数据段中。接下来，加载器跳转到程序的入口点，也就是 `_start` 函数的地址。`_start` 函数定义在系统目标文件 crt1.o 中，对所有 C 程序都相同。它调用系统启动函数 `__libc_start_main`，这个函数定义在 libc.so 中。它初始化执行环境，调用用户层的 main 函数，处理其返回值，并且在必要时把控制权交还给内核。

7.10 动态链接共享库

我们在 7.6.2 节研究的静态库解决了把大量相关函数集合提供给应用程序使用的许多问题。不过，静态库也有一些明显的缺点。静态库与所有软件一样，需要定期维护和更新。如果应用程序的程序员想使用某个库的最新版本，他们必须以某种方式了解到库已经发生变化，然后显式地将他们的程序与更新后的库重新链接。

另一个问题是，几乎每个 C 程序都会使用 printf 和 scanf 之类的标准 I/O 函数。在运行时，这些函数的代码会重复地出现在每个运行进程的代码段中。在一个运行着数百个进程的典型系统上，这可能是对稀缺存储器资源的严重浪

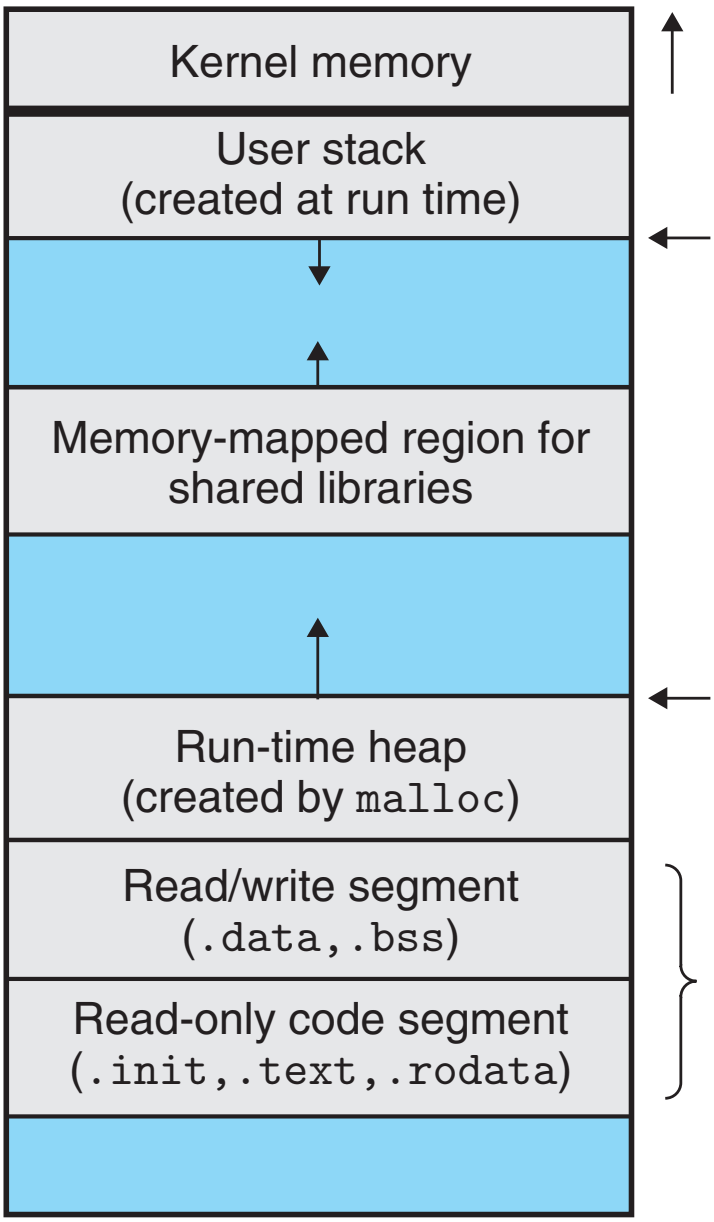


图 7.15 Linux x86-64 运行时内存映像。因段对齐要求和地址空间布局随机化(ASLR)造成的间隙没有画出。图中比例并非实际比例。

费。(存储器有一个有趣的特性:无论系统中一共有多少存储,它总是稀缺资源。磁盘空间和厨房垃圾桶都同样如此。)

> **边栏 加载器究竟是如何工作的?**
> 我们对加载过程的描述在概念上是正确的,但有意不去完全贴合实际。要理解加载过程究竟是如何工作的,你必须理解进程、虚拟内存和内存映射这些概念,而这几个概念我们还没有讨论。当我们在后文第 8 章和第 9 章遇到这些概念时,会回过头来重新审视加载,并逐步为你揭开谜底。
> 对于心急的读者,这里先预览一下加载实际工作的方式:Linux 系统中的每个程序都在一个进程的上下文中运行,并拥有自己的虚拟地址空间。shell 运行程序时,父 shell 进程会 fork 出一个作为父进程副本的子进程,子进程通过 execve 系统调用调用加载器。加载器删除子进程已有的虚拟内存段,并创建一组新的代码、数据、堆和栈段。新的栈和堆段被初始化为零。新的代码段和数据段则把虚拟地址空间中的页映射到可执行文件中页大小的分块,从而被初始化为可执行文件的内容。最后,加载器跳转到 `_start` 地址,它最终会调用应用程序的 main 例程。除了一些头部信息之外,加载过程中并不发生从磁盘到内存的数据复制。复制会被推迟到 CPU 引用一个已映射的虚拟页时,那时操作系统会用它的分页机制自动把这个页从磁盘传送到内存。

共享库(shared library)是解决静态库这些缺点的一项现代化产物。共享库是一种目标模块,在运行时或加载时可以加载到任意的内存地址,并和内存中的某个程序链接起来。这个过程称为动态链接(dynamic linking),由一个叫作动态链接器(dynamic linker)的程序来执行。共享库也被称为共享目标文件(shared object),在 Linux 系统中用.so 后缀来表示。微软的操作系统大量使用共享库,把它们称为 DLL(动态链接库)。

共享库以两种不同的方式“共享”。首先,在任何给定的文件系统中,对于一个特定的库来说,只存在一个.so 文件。这个.so 文件中的代码和数据被所有引用该库的可执行目标文件共享,而不像静态库的内容那样,被复制并嵌入引用它们的可执行文件中。其次,内存中共享库的.text 节的一个副本可以被不同的运行进程共享。我们在第 9 章学习虚拟内存时会更详细地讨论这一点。

图 7.16 概括了图 7.7 中示例程序的动态链接过程。为了构建包含图 7.6 中示例向量例程的共享库 libvector.so,我们用一些特殊的指令参数调用编译器驱动程序,把它们传给编译器和链接器:

```
linux> gcc -shared -fpic -o libvector.so addvec.c multvec.c
```

`-fpic` 选项指示编译器生成位置无关代码(下一节详述)。`-shared` 选项指示链接器创建一个共享目标文件。一旦创建了库,我们就可以把它与图 7.7 中的示例程序链接:

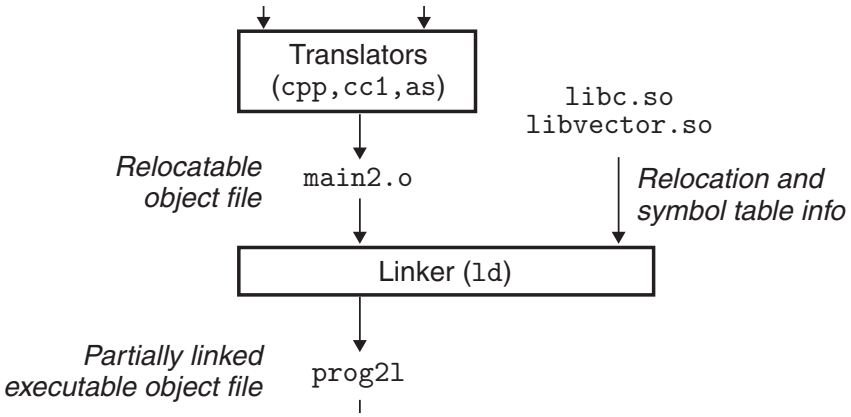


图 7.16 与共享库进行动态链接。

```
linux> gcc -o prog2l main2.c ./libvector.so
```

这样就创建了一个可执行目标文件 prog2l，其形式可以在运行时与 libvector.so 链接。基本思想是：在创建可执行文件时静态地完成一部分链接，然后在程序被加载时动态地完成余下的链接过程。值得注意的是，此时 libvector.so 的代码节和数据节并没有任何一节被真正复制到可执行文件 prog2l 中。反过来，链接器只复制了一些重定位和符号表信息，它们允许对 libvector.so 中代码和数据的引用在加载时被解析。

当加载器加载并运行可执行文件 prog2l 时，它按照 7.9 节讨论的技术把部分链接的可执行文件 prog2l 加载进来。接着，它注意到 prog2l 包含一个.interp 节，这个节包含动态链接器的路径名，而动态链接器本身就是一个共享目标文件（例如 Linux 系统中的 ld-linux.so）。加载器并不像通常那样把控制传递给应用程序，而是加载并运行这个动态链接器。然后，动态链接器通过执行下面的重定位来完成链接任务：

- 把 `libc.so` 的代码和数据重定位到某个内存段中
- 把 `libvector.so` 的代码和数据重定位到另一个内存段中
- 重定位 `prog2l` 中所有对 `libc.so` 和 `libvector.so` 所定义的符号的引用

最后，动态链接器把控制传递给应用程序。从这一刻起，共享库的位置就固定了，并且在程序的执行过程中不会再改变。

7.11 从应用程序中加载和链接共享库

到目前为止，我们讨论的都是这样的场景：动态链接器在应用程序被加载时（它开始执行之前）加载并链接共享库。不过，应用程序也有可能在运行时请

求动态链接器加载并链接任意的共享库,而无须在编译时把应用程序与这些库链接起来。

动态链接是一种强大而实用的技术。下面是它在现实世界中的一些应用例子:

- **软件发行**。微软 Windows 应用程序的开发者经常使用共享库来分发软件更新。他们生成共享库的一个新副本,用户随后可以下载它,用它替换现有的版本。下次运行应用程序时,它就会自动链接并加载这个新的共享库。
- **构建高性能 Web 服务器**。许多 Web 服务器会产生动态内容,如个性化网页、账户余额和横幅广告。早期的 Web 服务器使用 fork 和 execve 创建一个子进程并在其上下文中运行一段“CGI 程序”来生成动态内容。

  现代的高性能 Web 服务器可以用一种更高效、更精巧的基于动态链接的方法来生成动态内容:把每个生成动态内容的函数都打包到一个共享库中。当一个来自 Web 浏览器的请求到达时,服务器动态地加载并链接合适的函数,然后直接调用它,而不像以前那样用 fork 和 execve 在子进程的上下文中运行函数。函数保留在服务器地址空间的缓存中,因此后续的请求只需要付出一次简单函数调用的代价。这对繁忙站点的吞吐量有显著影响。而且,已有函数可以在运行时被更新,新函数也可以被加入,而无须停止服务器。

Linux 系统给动态链接器提供了一个简单的接口,它允许应用程序在运行时加载和链接共享库。

```
#include <dlfcn.h>
void *dlopen(const char *filename, int flag);
返回:成功则为指向句柄的指针,出错则为 NULL
```

`dlopen` 函数加载并链接共享库 filename。filename 的外部符号使用先前用 `RTLD_GLOBAL` 标志打开的库来解析。如果当前可执行文件是用 `-rdynamic` 标志编译的,那么它的全局符号也可用于符号解析。flag 参数必须包含 `RTLD_NOW`(它告诉链接器立即解析对外部符号的引用)或 `RTLD_LAZY` 标志(它指示链接器把符号解析推迟到库中的代码被执行时)二者之一。这两个值中的任何一个都可以与 `RTLD_GLOBAL` 标志求或。

```
#include <dlfcn.h>
void *dlsym(void *handle, char *symbol);
返回:成功则为指向符号的指针,出错则为 NULL
```

`dlsym` 函数以一个先前打开的共享库的句柄和一个符号名为输入,如果符号存在,则返回该符号的地址,否则返回 NULL。

```
#include <dlfcn.h>
int dlclose (void *handle);
返回:成功则为 0,出错则为 -1
```

dlclose 函数在没有其他共享库仍在使用某个共享库时卸载它。

```
#include <dlfcn.h>
const char *dlerror(void);
返回:如果此前对 dlopen、dlsym 或 dlclose 的调用失败,则为错误消息;
     如果此前调用成功,则为 NULL
```

dlerror 函数返回一个描述最近一次错误的字符串，该错误由此前调用 dlopen、dlsym 或 dlclose 引起;如果没有发生错误,则返回 NULL。

图 7.17 展示了如何使用这个接口在运行时动态链接我们的 libvector.so 共享库,然后调用其中的 addvec 例程。编译这个程序时,我们用下面的方式调用 gcc:

```
linux> gcc -rdynamic -o prog2r dll.c -ldl
```

```
code/link/dll.c
 1  #include <stdio.h>
 2  #include <stdlib.h>
 3  #include <dlfcn.h>
 4
 5  int x[2] = {1, 2};
 6  int y[2] = {3, 4};
 7  int z[2];
 8
 9  int main()
10  {
11      void *handle;
12      void (*addvec)(int *, int *, int *, int);
13      char *error;
14
15      /* 动态加载包含 addvec() 的共享库 */
16      handle = dlopen("./libvector.so", RTLD_LAZY);
17      if (!handle) {
18          fprintf(stderr, "%s\n", dlerror());
19          exit(1);
20      }
21
22      /* 获取刚加载的 addvec() 函数的指针 */
23      addvec = dlsym(handle, "addvec");
24      if ((error = dlerror()) != NULL) {
25          fprintf(stderr, "%s\n", error);
26          exit(1);
27      }
28
29      /* 现在可以像调用任何其他函数一样调用 addvec() 了 */
30      addvec(x, y, z, 2);
```

```
31      printf("z = [%d %d]\n", z[0], z[1]);
32
33      /* 卸载共享库 */
34      if (dlclose(handle) < 0) {
35          fprintf(stderr, "%s\n", dlerror());
36          exit(1);
37      }
38      return 0;
39  }
```

图 7.17 示例程序 3。在运行时动态加载并链接共享库 libvector.so。

边栏 共享库与 Java 本地接口

Java 定义了一种称为 Java 本地接口(Java Native Interface, JNI)的标准调用约定,它允许在 Java 程序中调用“本地”的 C 和 C++ 函数。JNI 的基本思想是把本地的 C 函数(比如 foo)编译成共享库(比如 foo.so)。当运行中的 Java 程序试图调用函数 foo 时,Java 解释器会使用 dlopen 接口(或类似的接口)来动态链接并加载 foo.so,然后调用 foo。

7.12 位置无关代码(PIC)

共享库的一个主要目的,是让多个正在运行的进程在内存中共享同一份库代码,从而节省宝贵的存储器资源。那么,多个进程怎样才能共享程序的一个单一副本呢?一种做法是先验地为每个共享库分配地址空间中一块专属区域,然后要求加载器总是把共享库加载到那个地址。这种方法虽然直接,却会造成一些严重的问题。它会低效地使用地址空间,因为即使某个进程不使用该库,这部分空间也已经被分配出去了。它还难以管理:我们必须确保各块区域互不重叠;每次修改一个库时,都必须确保它仍然放得进分配给它的区域;如果放不下,就必须为它寻找一块新区域;而如果新建了一个库,也必须为它找到地方。长此以往,面对系统中存在的成百上千个库和库的各个版本,很难防止地址空间碎片化成许多闲置却又无法使用的小空洞。更糟的是,库在各系统内存中的布局会互不相同,从而带来更多管理上的麻烦。

为了避免这些问题,现代系统把共享模块的代码段编译成无需链接器修改就可以加载到内存中任何位置的形式。采用这种方式,共享模块代码段的一个单一副本可以被无限多个进程共享。(当然,每个进程仍然会得到自己独立的可读写数据段副本。)

无须任何重定位就可以加载的代码称为位置无关代码 (position-independent code, PIC)。用户通过 `gcc` 的 `-fpic` 选项让 GNU 编译系统生成 PIC 代码。共享库必须始终用该选项编译。

在 x86-64 系统上，对同一可执行目标模块中符号的引用不需要特殊处理就已经是 PIC 的。这些引用可以按 PC 相对寻址方式编译，并在静态链接器构建目标文件时完成重定位。不过，对由共享模块定义的外部过程和全局变量的引用则需要一些特殊的技术，接下来我们就来介绍。

PIC 数据引用

编译器通过利用下面这个有趣的事实来为全局变量生成 PIC 引用：无论我们把目标模块(包括共享目标模块)加载到内存中的哪个位置，数据段总是与代码段保持相同的距离。因此，代码段中任何指令与数据段中任何变量之间的距离都是一个运行时常量，它与代码段和数据段的绝对内存位置无关。

想为全局变量生成 PIC 引用的编译器利用这个事实，在数据段开头创建一个叫作全局偏移表(global offset table，GOT)的表。GOT 包含目标模块所引用的每个全局数据对象(过程或全局变量)的一个 8 字节条目。编译器还为 GOT 中的每个条目生成一条重定位记录。在加载时，动态链接器重定位每个 GOT 条目，使其包含该对象的绝对地址。每个引用全局对象的目标模块都拥有自己的 GOT。

图 7.18 给出了我们的示例共享模块 libvector.so 的 GOT。addvec 例程通过 GOT[3] 间接地加载全局变量 addcnt 的地址，然后递增内存中的 addcnt。这里的关键是，对 GOT[3] 的 PC 相对引用中的偏移是一个运行时常量。

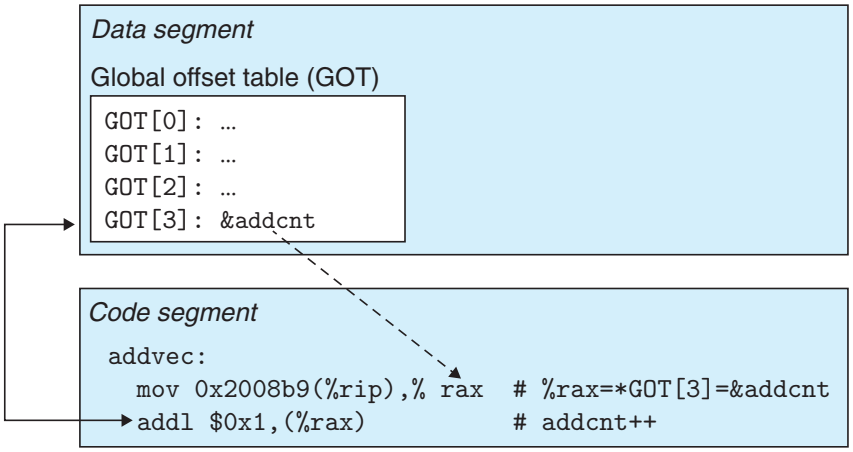


图 7.18 用 GOT 引用一个全局变量。libvector.so 中的 addvec 例程通过 libvector.so 的 GOT 间接引用 addcnt。

由于 addcnt 是由 libvector.so 模块定义的，编译器本可以生成一个直接的 PC 相对引用，利用代码段和数据段之间距离恒定的特性，并加入一条供链接器在构建共享模块时解析的重定位。但如果 addcnt 是由另一个共享模块定义的，

那么就必须通过 GOT 进行间接访问。此例中,编译器为所有引用选择了最通用的解决方案,即 GOT。

PIC 函数调用

假设一个程序调用了由共享库定义的函数。编译器没有办法预测这个函数的运行时地址,因为定义它的共享模块在运行时可以被加载到任何位置。正常的方法是为这个引用生成一条重定位记录,由动态链接器在程序加载时解析。然而,这种方法不是 PIC 的,因为它要求链接器修改调用模块的代码段。GNU 编译系统用一种有趣的技术来解决这个问题,这种技术称为惰性绑定(lazy binding),它把每个过程地址的绑定推迟到该过程第一次被调用时。

惰性绑定的动机是:一个典型的应用程序只会调用像 libc.so 这样的共享库所导出的成百上千个函数中的少数几个。通过把函数地址的解析推迟到函数真正被调用时,动态链接器可以在加载时避免成百上千次不必要的重定位。第一次调用该函数时会有不小的运行时开销,但此后的每次调用只花费一条指令和一次用于间接引用的内存访问。

惰性绑定是用两个数据结构——GOT 和过程链接表(procedure linkage table,PLT)——之间紧凑却又略显复杂的交互来实现的。如果一个目标模块调用了任何由共享库定义的函数,那么它就拥有自己的 GOT 和 PLT。GOT 是数据段的一部分,PLT 是代码段的一部分。

图 7.19 展示了 PLT 和 GOT 如何配合来在运行时解析一个函数的地址。首先,让我们分别考察这两个表的内容。

过程链接表。PLT 是一个由 16 字节的代码条目组成的数组。PLT[0] 是一个特殊的条目,它会跳转到动态链接器。可执行文件调用的每个共享库函数都有自己的 PLT 条目。每条这样的条目负责调用一个特定的函数。PLT[1](此处未显示)调用系统启动函数 `__libc_start_main`,它初始化执行环境、调用 main 函数并处理其返回值。从 PLT[2] 开始的条目调用用户代码调用的函数。在我们的示例中,PLT[2] 调用 addvec,PLT[3](未显示)调用 printf。

全局偏移表。如我们所见,GOT 是一个由 8 字节的地址条目组成的数组。当与 PLT 一起使用时,GOT[0] 和 GOT[1] 包含动态链接器解析函数地址时使用的信息。GOT[2] 是动态链接器在 ld-linux.so 模块中的入口点。其余的每个条目都对应一个被调用的函数,其地址需要在运行时解析。每一条都有一个匹配的 PLT 条目。例如,GOT[4] 和 PLT[2] 对应 addvec。最初,每个 GOT 条目都指向其对应 PLT 条目中的第二条指令。

图 7.19(a) 展示了 GOT 和 PLT 如何配合,在函数 addvec 第一次被调用时以惰性方式解析它的运行时地址:

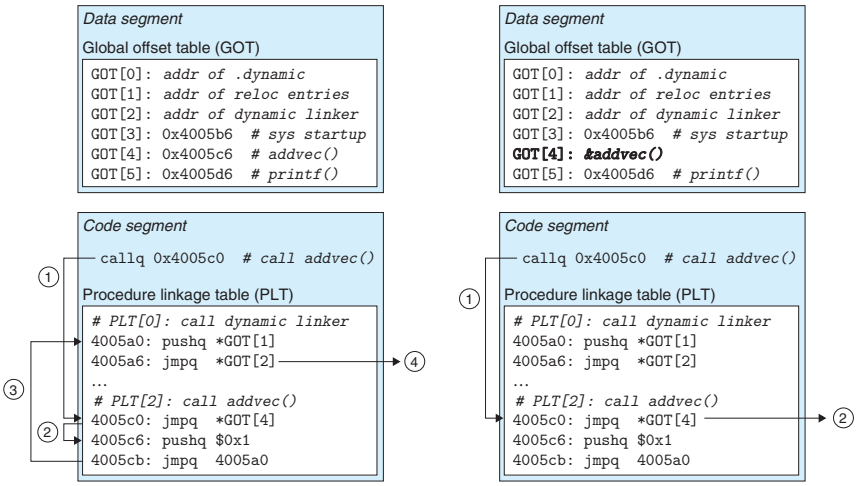


图 7.19 用 PLT 和 GOT 调用外部函数。动态链接器在 addvec 第一次被调用时解析出它的地址。

- **步骤 1**。程序不是直接调用 addvec，而是调用进入 PLT[2]，它是 addvec 的 PLT 条目。
- **步骤 2**。PLT 的第一条指令通过 GOT[4] 执行一次间接跳转。由于每个 GOT 条目最初都指向相应 PLT 条目中的第二条指令，因此这次间接跳转只是简单地把控制转回 PLT[2] 中的下一条指令。
- **步骤 3**。在把 addvec 的一个 ID(0x1)压入栈之后，PLT[2] 跳转到 PLT[0]。
- **步骤 4**。PLT[0] 通过 GOT[1] 间接地为动态链接器压入一个参数，然后通过 GOT[2] 间接地跳转到动态链接器中。动态链接器使用这两个栈条目来确定 addvec 的运行时位置，用这个地址重写 GOT[4]，并把控制传递给 addvec。

图 7.19(b) 展示了 addvec 后续调用的控制流程：

- **步骤 1**。和以前一样，控制传递到 PLT[2]。
- **步骤 2**。不过这一次，通过 GOT[4] 的间接跳转把控制直接传递给了 addvec。

7.13 库打桩机制

Linux 链接器支持一项称为库打桩机制（library interpositioning）的强大技术，它允许你截获对共享库函数的调用，转而执行你自己的代码。使用打桩机制，你可以记录某个库函数被调用的次数，对它的输入和输出值进行验证和跟踪，甚至可以把它替换成一个完全不同的实现。

其基本思想是：给定某个要对其打桩的目标函数，你创建一个原型与目标函数完全相同的包装函数（wrapper function）。然后，利用某种特定的打桩机制，你骗过系统，使其调用包装函数而不是目标函数。包装函数通常先执行自己的逻辑，然后调用目标函数，并把它的返回值传回给调用者。

打桩可以在程序被加载和执行时的编译期、链接期或运行期发生。为了探讨这些不同的机制，我们将以图 7.20(a) 中的示例程序作为贯穿全节的例子。它调用 C 标准库（libc.so）中的 malloc 和 free 函数。对 malloc 的调用从堆中分配一个 32 字节的块，并返回指向该块的指针。对 free 的调用把该块交还给堆，供后续对 malloc 的调用使用。我们的目标是使用打桩来跟踪程序运行时对 malloc 和 free 的调用。

7.13.1 编译时打桩

图 7.20 展示了如何在编译时使用 C 预处理器进行打桩。mymalloc.c（图 7.20(c)）中的每个包装函数调用目标函数，打印一条跟踪记录，然后返回。本地 malloc.h 头文件（图 7.20(b)）指示预处理器，把对目标函数的每个调用都替换为对其包装函数的调用。下面是编译并链接该程序的方法：

```
linux> gcc -DCOMPILETIME -c mymalloc.c
linux> gcc -I. -o intc int.c mymalloc.o
```

打桩之所以发生，是因为 `-I.` 参数告诉 C 预处理器在常规的系统目录之前，先在当前目录中寻找 malloc.h。注意，mymalloc.c 中的包装函数是用标准 malloc.h 头文件编译的。

```
(a) 示例程序 int.c
code/link/interpose/int.c
 1  #include <stdio.h>
 2  #include <malloc.h>
 3
 4  int main()
 5  {
 6      int *p = malloc(32);
 7      free(p);
 8      return(0);
 9  }

(b) 本地的 malloc.h 文件
code/link/interpose/malloc.h
 1  #define malloc(size) mymalloc(size)
 2  #define free(ptr) myfree(ptr)
 3
 4  void *mymalloc(size_t size);
 5  void myfree(void *ptr);

(c) mymalloc.c 中的包装函数
```

```
code/link/interpose/mymalloc.c
 1  #ifdef COMPILETIME
 2  #include <stdio.h>
 3  #include <malloc.h>
 4
 5  /* malloc 包装函数 */
 6  void *mymalloc(size_t size)
 7  {
 8      void *ptr = malloc(size);
 9      printf("malloc(%d)=%p\n",
10             (int)size, ptr);
11      return ptr;
12  }
13
14  /* free 包装函数 */
15  void myfree(void *ptr)
16  {
17      free(ptr);
18      printf("free(%p)\n", ptr);
19  }
20  #endif
```

图 7.20 用 C 预处理器进行的编译时打桩。

运行程序产生下面的踪迹:

```
linux> ./intc
malloc(32)=0x9ee010
free(0x9ee010)
```

7.13.2 链接时打桩

Linux 静态链接器支持用 `--wrap f` 选项进行链接时打桩。这个选项告诉链接器:把对符号 f 的引用解析为 `__wrap_f`(前缀是两个下划线),并把对符号 `__real_f`(前缀是两个下划线)的引用解析为 f。图 7.21 给出了我们示例程序的包装函数。

下面是把源文件编译成可重定位目标文件的方法:

```
linux> gcc -DLINKTIME -c mymalloc.c
linux> gcc -c int.c
```

```
code/link/interpose/mymalloc.c
 1  #ifdef LINKTIME
 2  #include <stdio.h>
 3
 4  void *__real_malloc(size_t size);
 5  void __real_free(void *ptr);
 6
 7  /* malloc 包装函数 */
 8  void *__wrap_malloc(size_t size)
 9  {
```

```
10      void *ptr = __real_malloc(size); /* 调用 libc malloc */
11      printf("malloc(%d) = %p\n", (int)size, ptr);
12      return ptr;
13  }
14
15  /* free 包装函数 */
16  void __wrap_free(void *ptr)
17  {
18      __real_free(ptr); /* 调用 libc free */
19      printf("free(%p)\n", ptr);
20  }
21  #endif
```

图 7.21 用 `--wrap` 标志进行的链接时打桩。

下面是把目标文件链接成一个可执行文件的方法：

```
linux> gcc -Wl,--wrap,malloc -Wl,--wrap,free -o intl int.o mymalloc.o
```

`-Wl,option` 标志把 option 传递给链接器。option 中的每个逗号都会被替换成一个空格。因此 `-Wl,--wrap,malloc` 会把 `--wrap malloc` 传给链接器，`-Wl,--wrap,free` 同理。

运行程序产生下面的踪迹：

```
linux> ./intl
malloc(32) = 0x18cf010
free(0x18cf010)
```

7.13.3 运行时打桩

编译时打桩需要访问程序的源文件。链接时打桩需要访问程序的可重定位目标文件。不过，还有一种在运行时打桩的机制，它只要求访问可执行目标文件。这个奇妙的机制所基于的是动态链接器的 `LD_PRELOAD` 环境变量。

如果 `LD_PRELOAD` 环境变量被设置成一个共享库路径名的列表（以空格或冒号分隔），那么当你加载并执行程序时，动态链接器（ld-linux.so）在解析未定义引用时，会先搜索 `LD_PRELOAD` 库，然后才搜索其他任何共享库。利用这个机制，你可以在加载并执行任何可执行程序时，对任何共享库（包括 libc.so）中的任何函数进行打桩。

图 7.22 给出了 malloc 和 free 的包装函数。在每个包装函数中，对 dlsym 的调用返回指向目标 libc 函数的指针。接着，包装函数调用目标函数，打印一条跟踪记录，然后返回。

下面是构建包含包装函数的共享库的方法：

```
linux> gcc -DRUNTIME -shared -fpic -o mymalloc.so mymalloc.c -ldl
```

下面是编译主程序的方法：

```
linux> gcc -o intr int.c
```

下面是从 bash shell 运行程序的方法。[3]

```
linux> LD_PRELOAD="./mymalloc.so" ./intr
malloc(32) = 0x1bf7010
free(0x1bf7010)
```

下面是从 csh 或 tcsh shell 运行它的方法：

```
linux> (setenv LD_PRELOAD "./mymalloc.so"; ./intr; unsetenv LD_PRELOAD)
malloc(32) = 0x2157010
free(0x2157010)
```

注意，你可以使用 LD_PRELOAD 对任何可执行程序的库调用进行打桩！

```
linux> LD_PRELOAD="./mymalloc.so" /usr/bin/uptime
malloc(568) = 0x21bb010
free(0x21bb010)
malloc(15) = 0x21bb010
malloc(568) = 0x21bb030
malloc(2255) = 0x21bb270
free(0x21bb030)
malloc(20) = 0x21bb030
malloc(20) = 0x21bb050
malloc(20) = 0x21bb070
malloc(20) = 0x21bb090
malloc(20) = 0x21bb0b0
malloc(384) = 0x21bb0d0
20:47:36 up 85 days, 6:04, 1 user, load average: 0.10, 0.04, 0.05
```

```
code/link/interpose/mymalloc.c
 1  #ifdef RUNTIME
 2  #define _GNU_SOURCE
 3  #include <stdio.h>
 4  #include <stdlib.h>
 5  #include <dlfcn.h>
 6
 7  /* malloc 包装函数 */
 8  void *malloc(size_t size)
 9  {
10      void *(*mallocp)(size_t size);
11      char *error;
12
13      mallocp = dlsym(RTLD_NEXT, "malloc"); /* 获取 libc malloc 的地址
 ↪  */
14      if ((error = dlerror()) != NULL) {
15          fputs(error, stderr);
16          exit(1);
```

[3]如果你不知道自己运行的是哪个 shell，请在命令行上键入 printenv SHELL。

```
17      }
18      char *ptr = mallocp(size); /* 调用 libc malloc */
19      printf("malloc(%d) = %p\n", (int)size, ptr);
20      return ptr;
21  }
22
23  /* free 包装函数 */
24  void free(void *ptr)
25  {
26      void (*freep)(void *) = NULL;
27      char *error;
28
29      if (!ptr)
30          return;
31
32      freep = dlsym(RTLD_NEXT, "free"); /* 获取 libc free 的地址 */
33      if ((error = dlerror()) != NULL) {
34          fputs(error, stderr);
35          exit(1);
36      }
37      freep(ptr); /* 调用 libc free */
38      printf("free(%p)\n", ptr);
39  }
40  #endif
```

图 7.22 用 LD_PRELOAD 进行的运行时打桩。

7.14 处理目标文件的工具

Linux 系统上有许多可用的工具，可以帮助你理解目标文件并对其进行操作。特别值得一提的是，GNU binutils 软件包格外有用，它可以运行在每种 Linux 平台上。

- `ar`。创建静态库，以及插入、删除、列出和提取成员。
- `strings`。列出一个目标文件中所有可打印的字符串。
- `strip`。从一个目标文件中删除符号表信息。
- `nm`。列出一个目标文件的符号表中定义的符号。
- `size`。列出一个目标文件中各节的名字和大小。
- `readelf`。显示一个目标文件的完整结构，包括 ELF 头中编码的所有信息。它涵盖了 `size` 与 `nm` 的功能。
- `objdump`。所有二进制工具之母。可以显示一个目标文件中的所有信息。它最有用的功能是反汇编.text 节中的二进制指令。

Linux 系统也提供了 ldd 程序来操作共享库：

- `ldd`：列出一个可执行文件在运行时需要的共享库。

7.15 小结

链接可以由静态链接器在编译时完成，也可以由动态链接器在加载时和运行时完成。链接器处理的是称为目标文件的二进制文件，它们有三种不同的形式：可重定位的、可执行的，以及共享的。可重定位的目标文件由静态链接器组合成一个可执行目标文件，它可以被加载到内存中执行。共享目标文件（共享库）在运行时由动态链接器链接和加载，或者隐式地在调用程序被加载并开始执行时进行，或者按需地在程序调用 dlopen 库的函数时进行。

链接器的两个主要任务是符号解析：把目标文件中的每个全局符号都绑定到一个唯一的定义上；以及重定位：确定每个符号最终的内存地址，并修改对这些对象的引用。

静态链接器由编译器驱动程序（如 gcc）调用。它们把多个可重定位目标文件组合成一个可执行目标文件。多个目标文件可以定义同一个符号，而链接器为静默解析这些多重定义所使用的规则，可能会在用户程序中引入微妙的 bug。

多个目标文件可以被合并到一个静态库中。链接器使用库来解析其他目标模块中的符号引用。许多链接器用来解析符号引用的从左到右顺序扫描，也是另一个造成令人困惑的链接期错误的来源。

加载器把可执行文件的内容映射到内存中，并运行程序。链接器还可以生成部分链接的可执行目标文件，其中包含一些到共享库中定义的例程和数据的未解析引用。在加载时，加载器把部分链接的可执行文件映射到内存中，然后调用动态链接器，由它通过加载共享库并重定位程序中的引用来完成链接任务。

以位置无关代码方式编译的共享库可以被加载到任何地方，并由多个进程在运行时共享。应用程序也可以在运行时使用动态链接器来加载、链接和访问共享库中的函数和数据。

7.16 参考文献说明

在计算机系统的文献中，链接一直缺乏完善的文档。由于链接处于编译器、计算机体系结构和操作系统的交叉点上，理解它需要具备代码生成、机器语言编程、程序创建和虚拟内存的知识。它不能很好地归入任何一个常规的计算机系统专业方向，因而这些领域的经典教材也没有对它给出充分的覆盖。不过，Levine 的专著为该主题提供了一部很好的通用参考文献 [69]。ELF 和 DWARF（一个对.debug 和.line 节内容的规范）最初的 IA32 规范在 [54] 中有描述。ELF 文件格式的 x86-64 扩展在 [36] 中有描述。x86-64 应用二进制接口（application

binary interface，ABI）描述了编译、链接和运行 x86-64 程序的约定，包括重定位和位置无关代码的规则 [77]。

7.17 家庭作业

家庭作业 7.6 ▲ 本题考察图 7.5 中的 m.o 模块以及下面这个会对自己被调用的次数进行计数的 swap.c 函数版本：

```
1  extern int buf[];
2
3  int *bufp0 = &buf[0];
4  static int *bufp1;
5
6  static void incr()
7  {
8      static int count=0;
9
10     count++;
11 }
12
13 void swap()
14 {
15     int temp;
16
17     incr();
18     bufp1 = &buf[1];
19     temp = *bufp0;
20     *bufp0 = *bufp1;
21     *bufp1 = temp;
22 }
```

对 swap.o 中定义并引用的每一个符号，指出它是否会出现在模块 swap.o 的.symtab 节中而成为一个符号表条目。如果有，请指出定义该符号的模块（swap.o 还是 m.o）、符号类型（局部、全局还是 extern），以及该模块中它所占据的节（.text、.data 还是.bss）。

符号	swap.o 的.symtab 条目？	符号类型	定义所在的模块	节
buf				
bufp0				
bufp1				
swap				
temp				
incr				
count				

家庭作业 7.7 ▲ 在不改变任何变量名的前提下，修改 7.6.1 节中的 bar5.c，使得 foo5.c 打印出 x 和 y 的正确值（即整数 15213 和 15212 的十六进制表示）。

家庭作业 7.8 ▲ 本题中，用 REF$(x.i) \rightarrow$ DEF$(x.k)$ 表示链接器会把模块 i 中对符号 x 的任意一个引用与模块 k 中 x 的定义关联起来。对下面的每个例子，用这种记号指出链接器会如何解析这些模块中对多重定义符号的引用。如果会出现链接时错误（规则 1），就写“error”。如果链接器任意选择了其中一个定义（规则 3），就写“unknown”。

```
/* 模块 1 */                    /* 模块 2 */
int main()                      static int main=1;
{                               int p2()
}                               {
                                }
```

(a) REF(main.1) $\rightarrow$ DEF(____ . ____)，**(b)** REF(main.2) $\rightarrow$ DEF(____ . ____)

```
/* 模块 1 */                    /* 模块 2 */
int x;                          double x;
void main()                     int p2()
{                               {
}                               }
```

(a) REF$(x.1) \rightarrow$ DEF(____ . ____)，**(b)** REF$(x.2) \rightarrow$ DEF(____ . ____)

```
/* 模块 1 */                    /* 模块 2 */
int x=1;                        double x=1.0;
void main()                     int p2()
{                               {
}                               }
```

(a) REF$(x.1) \rightarrow$ DEF(____ . ____)，**(b)** REF$(x.2) \rightarrow$ DEF(____ . ____)

家庭作业 7.9 ▲ 考虑下面这个由两个目标模块组成的程序：

```
1  /* foo6.c */
2  void p2(void);
3
4  int main()
5  {
6      p2();
7      return 0;
8  }
```

```
1  /* bar6.c */
2  #include <stdio.h>
3
4  char main;
5
6  void p2()
7  {
8      printf("0x%x\n", main);
9  }
```

当这个程序在一个 x86-64 Linux 系统上编译并执行时，它会打印字符串 `0x48\n` 并正常终止，尽管函数 p2 从未初始化变量 main。你能解释这是为什么吗？

家庭作业 7.10 ▲▲ 设 a 和 b 表示当前目录中的目标模块或静态库，并用 $a \to b$ 表示 a 依赖于 b，即 b 定义了一个 a 所引用的符号。对下面每种情况，给出能让静态链接器解析所有符号引用的最短命令行（即目标文件和库参数数目最少的命令行）：

- **A.** p.o → libx.a → p.o
- **B.** p.o → libx.a → liby.a 与 liby.a → libx.a
- **C.** p.o → libx.a → liby.a → libz.a 与 liby.a → libx.a → libz.a

家庭作业 7.11 ▲▲ 图 7.14 中的程序头部表明数据段在内存中占据 0x230 字节。然而，这些字节中只有最开始的 0x228 字节来自可执行文件的节。是什么造成了这个差别？

家庭作业 7.12 ▲▲ 考察目标文件 m.o（家庭作业 7.6）中对函数 swap 的调用。

```
9:    e8 00 00 00 00               callq   e <main+0xe>
    swap()
```

它带如下的重定位条目：

```
r.offset = 0xa
r.symbol = swap
  r.type = R_X86_64_PC32
r.addend = -4
```

A. 假设链接器把 m.o 中的.text 重定位到地址 0x4004e0，把 swap 重定位到地址 0x4004f8。那么 callq 指令中重定位后的 swap 引用的值是多少？

B. 假设链接器把 m.o 中的.text 重定位到地址 0x4004d0，把 swap 重定位到地址 0x400500。那么 callq 指令中重定位后的 swap 引用的值是多少？

家庭作业 7.13 ▲▲ 完成下面的任务会帮助你更加熟悉操作目标文件的各种工具：

- **A.** 你的系统中的 libc.a 和 libm.a 版本里各包含多少个目标文件？
- **B.** `gcc -Og` 产生的可执行代码与 `gcc -Og -g` 产生的有不同吗？
- **C.** 你的系统上的 gcc 驱动程序使用哪些共享库？

7.18 练习答案

答案 7.1 本题的目的在于帮助你理解链接器符号与 C 变量和函数之间的关系。注意,C 局部变量 temp 没有符号表条目。

符号	.symtab 条目?	符号类型	定义所在的模块	节
buf	是	extern	m.o	.data
bufp0	是	全局	swap.o	.data
bufp1	是	全局	swap.o	COMMON
swap	是	全局	swap.o	.text
temp	否	–	–	–

答案 7.2 这是一个简单的练习,检查你对 Unix 链接器解析在多个模块中定义的全局符号所使用的规则的理解。理解这些规则能帮助你规避一些难缠的编程 bug。

A. 链接器选择模块 1 中定义的强符号而不是模块 2 中定义的弱符号(规则 2):

(a) REF(main.1) $\rightarrow$ DEF(main.1) **(b)** REF(main.2) $\rightarrow$ DEF(main.1)

B. 这是一个错误,因为每个模块都定义了一个强符号 main(规则 1)。

C. 链接器选择模块 2 中定义的强符号而不是模块 1 中定义的弱符号(规则 2):

(a) REF(x.1) $\rightarrow$ DEF(x.2) **(b)** REF(x.2) $\rightarrow$ DEF(x.2)

答案 7.3 在命令行上把静态库放在错误的位置,是造成令许多程序员困扰的链接器错误的一个常见原因。不过,一旦理解了链接器如何使用静态库来解析引用,这个问题就非常直截了当。这个小练习检查你对这一思想的理解:

```
A.  linux> gcc p.o libx.a
B.  linux> gcc p.o libx.a liby.a
C.  linux> gcc p.o libx.a liby.a libx.a
```

答案 7.4 本题考察图 7.12(a) 中的反汇编清单。这里的目的是给你一些阅读反汇编清单的练习,并检查你对 PC 相对寻址的理解。

A. 第 5 行中重定位后的引用的十六进制地址是 0x4004df。

B. 第 5 行中重定位后的引用的十六进制值是 0x5。记住,反汇编清单显示的是引用按小端字节顺序存放的值。

答案 7.5 本题检验你对链接器如何重定位 PC 相对引用的理解。题目已经给出了

$$\texttt{ADDR(s)} = \texttt{ADDR(.text)} = \texttt{0x4004d0}$$

以及

$$\texttt{ADDR(r.symbol)} = \texttt{ADDR(swap)} = \texttt{0x4004e8}$$

使用图 7.10 中的算法,链接器首先计算该引用的运行时地址:

$$\begin{aligned}\texttt{refaddr} &= \texttt{ADDR(s)} + \texttt{r.offset}\\ &= \texttt{0x4004d0} + \texttt{0xa}\\ &= \texttt{0x4004da}\end{aligned}$$

然后它更新该引用:

$$\begin{aligned}\texttt{*refptr} &= \texttt{(unsigned) (ADDR(r.symbol) + r.addend - refaddr)}\\ &= \texttt{(unsigned) (0x4004e8 + (-4) - 0x4004da)}\\ &= \texttt{(unsigned) (0xa)}\end{aligned}$$

于是,在最终生成的可执行目标文件中,对 swap 的 PC 相对引用的值为 0xa:

```
4004d9:    e8 0a 00 00 00              callq   4004e8 <swap>
```

8

异常控制流

从给处理器上电到关机的整个过程中，程序计数器取过一串这样的值：

$$a_0, a_1, \ldots, a_{n-1}$$

其中每个 a_k 都是与之对应的那条指令 I_k 的地址。从 a_k 到 a_{k+1} 的每一次过渡称为一次控制转移（control transfer）。这样的一系列控制转移就称为处理器的控制流（flow of control，或称 control flow）。

最简单的一类控制流是“平滑”的序列，I_k 与 I_{k+1} 在存储器中相邻。平滑流中通常的突变由跳转、调用和返回这些我们熟悉的程序指令造成，此时 I_{k+1} 与 I_k 不相邻。这些指令是必要的机制，允许程序对由程序变量所表示的内部程序状态的变化做出反应。

但是，系统还必须能够对系统状态中另一些变化做出反应，这些变化既不被内部程序变量所捕捉，也未必与该程序的执行有关。例如，硬件定时器会周期性地超时，必须加以处理；分组到达网络适配器，必须存入存储器；程序向磁盘请求数据，然后休眠，直到数据准备就绪的通知到来；创建了子进程的父进程必须在子进程终止时得到通知。

现代系统应对这些情况的方法是让控制流发生突变。一般说来，我们把这些突变统称为异常控制流（exceptional control flow，ECF）。ECF 存在于计算机系统的每个层次上。例如，在硬件层，硬件检测到的事件会触发控制流突然转到异常处理程序；在操作系统层，内核通过上下文切换把控制从一个用户进程转移到另一个用户进程；在应用层，一个进程可以向另一个进程发送信号，使控制突然转到接收进程的信号处理程序；单个程序可以对错误做出反应：绕开通常的栈规则，进行非本地跳转，跳到其他函数中的任意位置。

作为程序员，理解 ECF 对你十分重要，原因有以下几点：

- 理解 ECF 有助于你理解重要的系统概念。ECF 是操作系统用来实现 I/O、进程和虚拟内存的基本机制。在真正理解这些重要的概念之前，你需要先理解 ECF。
- 理解 ECF 有助于你理解应用程序如何与操作系统交互。应用程序用一种叫作陷阱（`trap`）或系统调用（`system call`）的 ECF 机制向操作系统请求服务。例如，向磁盘写数据、从网络读数据、创建新进程、终止当前进程，全都是靠应用程序调用系统调用完成的。理解基本的系统调用机制，有助于你理解这些服务是如何提供给应用程序的。
- 理解 ECF 有助于你编写有趣的新应用程序。操作系统为应用程序提供了强大的 ECF 机制，可以创建新进程、等待进程终止，把系统中的异常事件通知其他进程，还可以检测并响应这些事件。理解这些 ECF 机制之后，你就能利用它们来编写 Unix shell 和 Web 服务器这样有趣的程序。
- 理解 ECF 有助于你理解并发。ECF 是计算机系统中实现并发的基本机制。下面这些都是并发起作用的例子：异常处理程序中断应用程序的执行；执行在时间上相互重叠的进程和线程；信号处理程序中断应用程序的执行。理解 ECF 是理解并发的重要一步，我们将在第 12 章更详细地研究它。
- 理解 ECF 有助于你理解软件异常是如何工作的。C++ 和 Java 这样的语言通过 `try`、`catch` 和 `throw` 语句提供软件异常机制。软件异常让程序可以对出错条件做出响应，进行非本地跳转（也就是违反通常的调用/返回栈规则的跳转）。非本地跳转是应用层 ECF 的一种形式，在 C 中由 `setjmp` 和 `longjmp` 函数提供。理解这些底层函数，有助于你理解更高级别的软件异常是如何实现的。

到目前为止，在学习系统的过程中，你已经了解了应用程序如何与硬件交互。本章至关重要，因为从本章开始，你将学习应用程序如何与操作系统交互。有趣的是，这些交互全都围绕 ECF 展开。我们将描述存在于计算机系统各个层次的各种形式的 ECF。我们从异常开始，它位于硬件和操作系统的交界处；我们还将讨论系统调用，它们是为应用程序提供操作系统入口点的异常；然后，我们上升到更高一个抽象层次，描述进程和信号，它们位于应用程序和操作系统的交界处；最后，我们讨论非本地跳转，它是 ECF 的一种应用层形式。

8.1 异常

异常（exception）是异常控制流的一种形式，一部分由硬件实现，一部分由操作系统实现。因为有一部分由硬件实现，所以其中的细节因系统而异。不过，基本思想对每一个系统来说都是相同的。本节目的在于让你对异常和异常处

理有一个一般性的了解，并帮助你揭开现代计算机系统中这个常令人困惑的方面的神秘面纱。

一个异常就是响应处理器状态中的某些变化而发生的控制流中的突变。图 8.1 示意了基本思想。

在图中，处理器正在执行某条当前指令 I_{curr}，此时处理器状态发生了显著的变化。状态被编码在处理器内的各种位和信号中。状态的变化称为事件（event）。

> **边栏　硬件异常与软件异常**
> C++ 和 Java 程序员会注意到，“exception” 一词也用来描述 C++ 和 Java 以 `catch`、`throw` 和 `try` 语句的形式提供的那种应用层 ECF 机制。如果我们想做到完全清晰，也许应该区分“硬件异常”和“软件异常”，不过这通常没有必要，因为其含义从上下文中一看便知。

Figure 8.1
Anatomy of an exception.
A change in the processor's state (an event) triggers an abrupt control transfer (an exception) from the application program to an exception handler. After it finishes processing, the handler either returns control to the interrupted program or aborts.

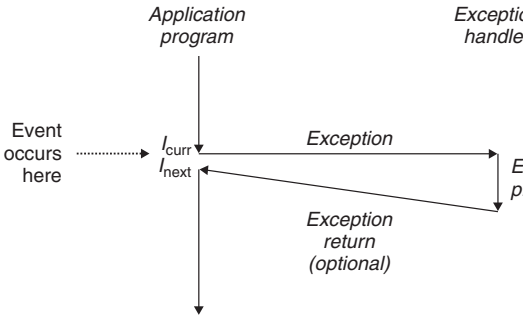


图 8.1

事件可能与当前指令的执行直接相关。例如，发生了虚拟内存缺页（page fault），发生了算术溢出，或者一条指令试图除以零。另一方面，事件也可能与当前指令的执行无关。例如，系统定时器超时了，或者一个 I/O 请求完成了。

不管怎样，当处理器检测到事件发生时，它会通过一张叫作异常表（exception table）的跳转表，进行一次间接过程调用（即这个异常），转到操作系统的一个子程序，即异常处理程序（exception handler），后者专门用来处理这类特殊的事件。异常处理程序处理完毕后，根据引起异常的事件类型，会发生以下三种情况之一：

1. 处理程序把控制返回给当前指令 I_{curr}，即事件发生时正在执行的那条指令。
2. 处理程序把控制返回给 I_{next}，即如果没有发生异常，接下来会被执行的那条指令。

3. 处理程序中止被中断的程序。

8.1.2 节会进一步讨论这些可能性。

8.1.1 异常处理

异常并不容易理解，因为对异常的处理涉及硬件和软件的紧密协作。很容易搞混究竟由哪个部件来执行哪项任务。我们来更详细地看看硬件和软件是如何分工的。

系统为每种可能类型的异常分配了一个唯一的非负整数异常号（exception number）。其中一些号码由处理器的设计者确定，另一些号码由操作系统内核（操作系统中常驻存储器的部分）的设计者确定。前者的例子包括被零除、缺页、存储器访问违例、断点和算术溢出；后者的例子包括系统调用和来自外部 I/O 设备的信号。

在系统启动时（计算机复位或上电的时候），操作系统分配并初始化一张叫作异常表（exception table）的跳转表，使表项 k 包含异常 k 的处理程序代码的地址。图 8.2 展示了异常表的格式。

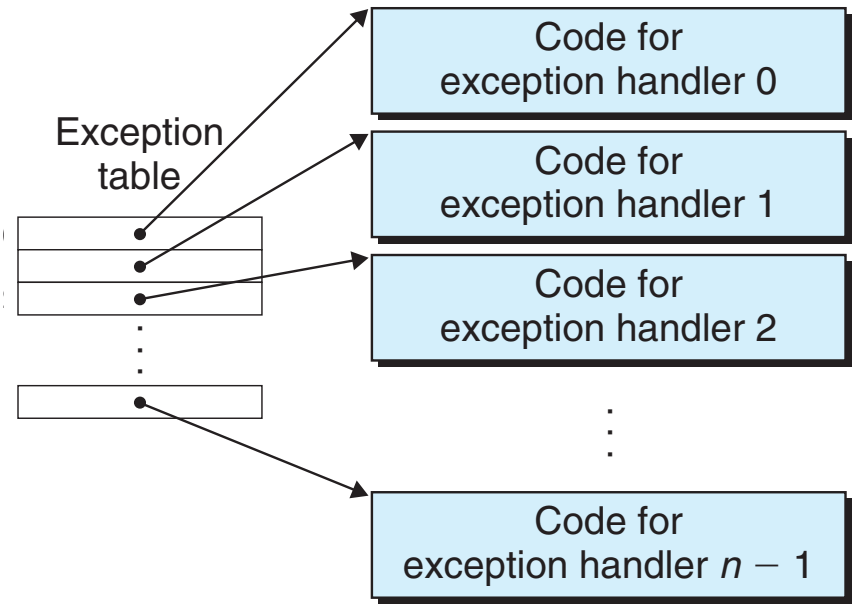


图 8.2　异常表。异常表是一张跳转表，其中第 k 个表项包含异常 k 的处理程序代码的地址。

在运行时（系统执行某个程序的时候），处理器检测到事件发生，并确定相应的异常号 k。随后，处理器通过异常表的表项 k 进行一次间接过程调用，转到相应的处理程序，从而触发异常。图 8.3 展示了处理器如何利用异常表来形成合适的异常处理程序的地址。异常号是异常表的一个索引，异常表的起始地

址存放在一个叫作异常表基址寄存器(exception table base register)的特殊 CPU 寄存器中。

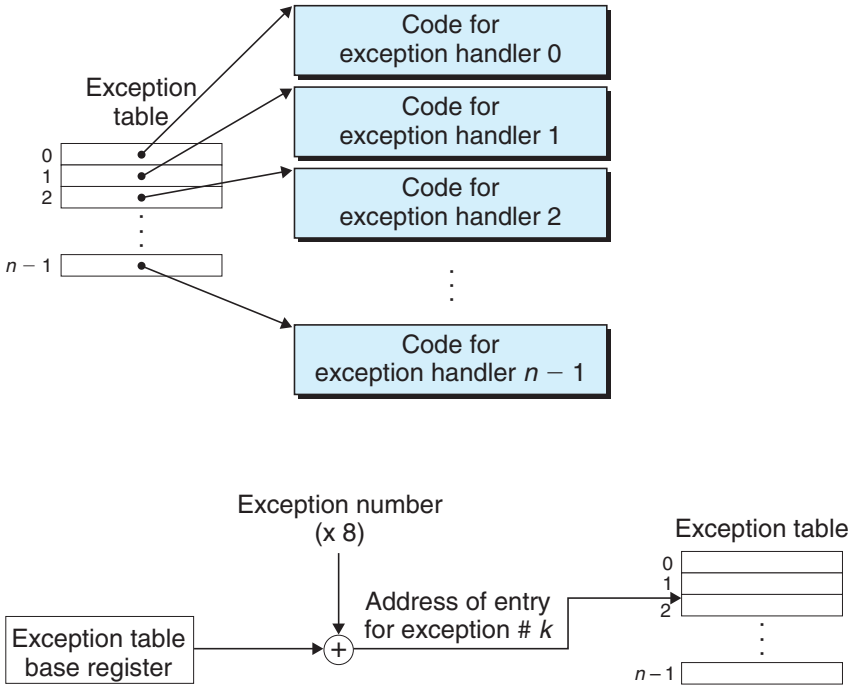


图 8.3 生成异常处理程序的地址。异常号是异常表的一个索引。

异常与过程调用有相似之处,但也有一些重要的差别:

- 与过程调用一样,处理器在转到处理程序之前,也把返回地址压入栈中。不过,根据异常类别的不同,返回地址要么是当前指令(事件发生时正在执行的那条指令),要么是下一条指令(如果没有事件发生,在当前指令之后下一步要执行的那条指令)。
- 处理器还把一些额外的处理器状态压入栈中,在处理程序返回时,重新启动被中断的程序将需要这些状态。例如,x86-64 系统会把包含当前条件码(condition code)在内的其他内容一起构成的 EFLAGS 寄存器压入栈中。
- 当控制从用户程序转到内核时,所有这些项都被压入内核的栈,而不是压入用户的栈。
- 异常处理程序运行在内核模式(8.2.4 节)下,这意味着它们对系统中所有资源都有完全的访问权限。

硬件触发异常之后,剩下的工作由软件完成,即由异常处理程序完成。处理程序处理完该事件后,可以有选择地执行一条特殊的“中断返回”指令回到被中断的程序。这条指令把适当的状态弹回到处理器的控制和数据寄存器中,如

果异常中断的是一个用户程序，就把状态恢复为用户模式（8.2.4 节），然后把控制返回给被中断的程序。

8.1.2 异常的类别

异常可以分为四类：中断（interrupt）、陷阱（trap）、错误（fault）和中止（abort）。图 8.4 中的表概括了这些类别的属性。

中断

中断是处理器外部的 I/O 设备发出的信号引起的，它是异步发生的。硬件中断之所以说它是异步的，是因为它不是由任何一条具体指令的执行引起的。硬件中断的异常处理程序通常称为中断处理程序（interrupt handler）。

图 8.5 概括了对一个中断的处理过程。网络适配器、磁盘控制器和定时器芯片这一类 I/O 设备在处理器芯片的引脚上发出信号，并把标识引起中断的那个设备的异常号放上系统总线，以此触发中断。

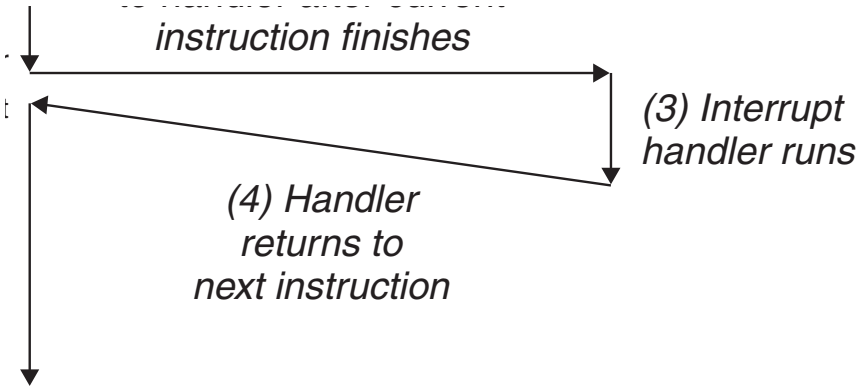


图 8.5 中断处理。中断处理程序将控制返回给应用程序控制流中的下一条指令。

图 8.4 异常的类别。异步异常由处理器外部 I/O 设备中的事件引起；同步异常是执行一条指令的直接结果。

类别	起因	异步/同步	返回行为
中断	来自 I/O 设备的信号	异步	总是返回到下一条指令
陷阱	有意的异常	同步	总是返回到下一条指令
错误	潜在可恢复的错误	同步	可能返回到当前指令
中止	不可恢复的错误	同步	从不返回

在当前指令执行完毕之后，处理器注意到中断引脚已经变为高电平，从系统总线上读出异常号，然后调用相应的中断处理程序。处理程序返回时，它把

控制返回给下一条指令(即,如果没有发生中断,在控制流中会接着当前指令执行的那条指令)。实际效果是,程序继续执行,就好像中断从来没有发生过一样。

其余几类异常(陷阱、错误和中止)是同步发生的,由执行当前指令而引起。我们把这条指令称为引发指令(faulting instruction)。

陷阱和系统调用

陷阱是有意产生的异常,由执行一条指令而引起。与中断处理程序一样,陷阱处理程序也将控制返回给下一条指令。陷阱最重要的用途是在用户程序和内核之间提供一条类似过程调用的接口,称为系统调用。

用户程序常常需要向内核请求服务,比如读取文件(`read`)、创建新进程(`fork`)、加载新程序(`execve`)和终止当前进程(`exit`)。为了允许对这些内核服务进行受控访问,处理器提供了一条特殊的 `syscall` n 指令,用户程序想要请求服务 n 时就可以执行这条指令。执行这条 `syscall` 指令会引发一个到异常处理程序的陷阱,该处理程序对参数进行译码,然后调用相应的内核例程。图 8.6 概括了对一个系统调用的处理过程。

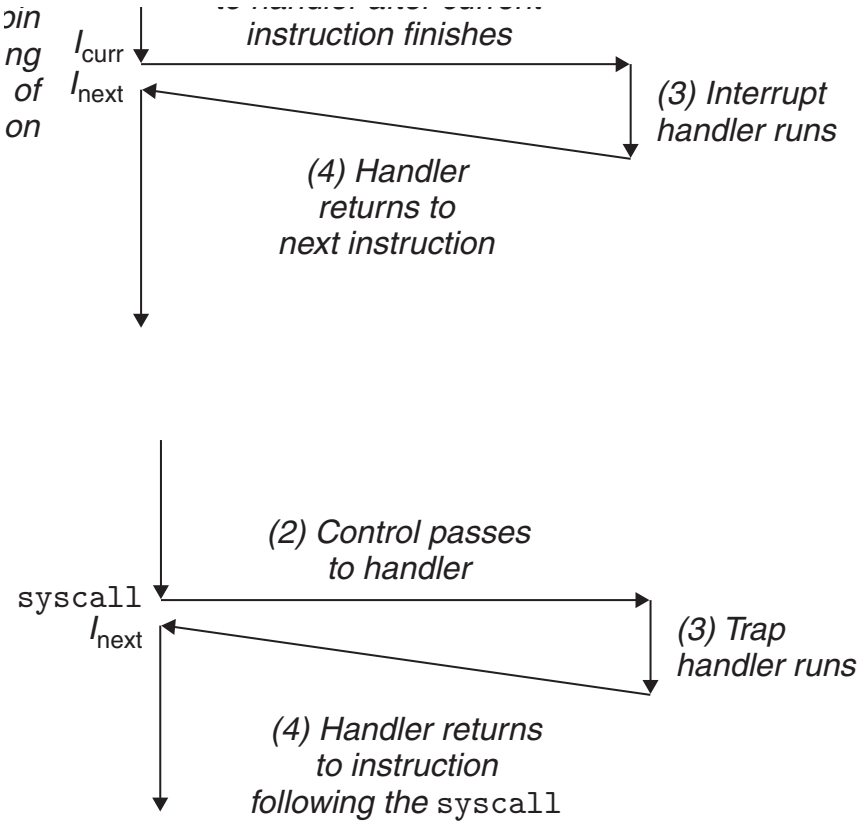


图 8.6 陷阱处理。陷阱处理程序将控制返回给应用程序控制流中的下一条指令。

从程序员的视角看，系统调用与普通的函数调用完全一样。但是两者的实现差别很大。普通函数运行在用户模式中，它们能执行的指令类型受到限制，而且它们与调用自己的函数访问同一个栈。系统调用则运行在内核模式中，允许执行特权指令，并访问内核中定义的栈。8.2.4 节会更详细地讨论用户模式和内核模式。

错误

错误是由那些处理程序可能能够纠正的错误情况造成的。当错误发生时，处理器把控制转移给错误处理程序。如果处理程序能够改正这个错误条件，它就把控制返回给引发指令，从而重新执行它；否则，处理程序返回到内核中的一个中止例行程序，由它来终止引起这个错误的应用程序。图 8.7 概括了对一个错误的处理过程。

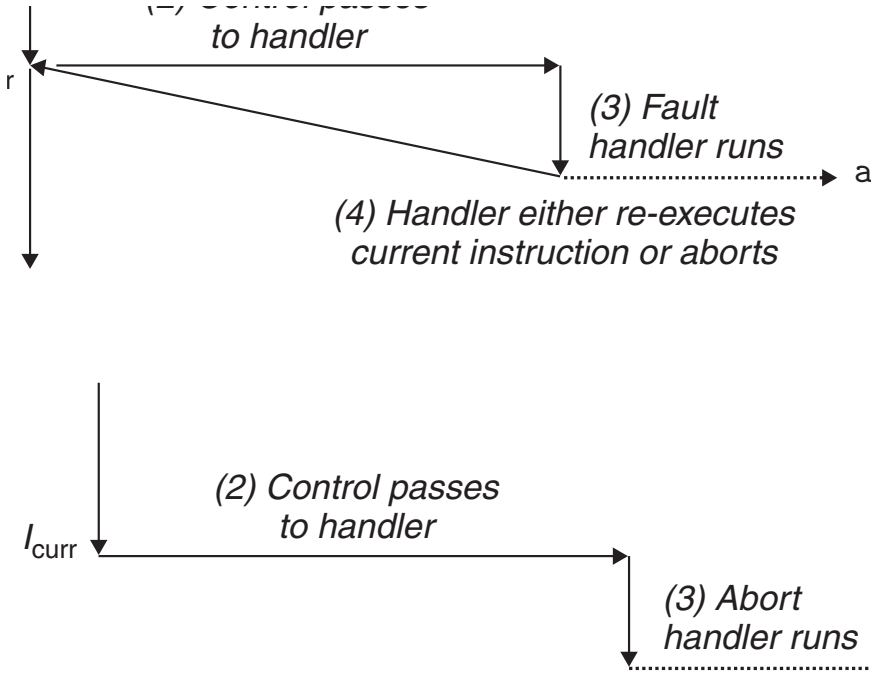


图 8.7 故障处理。根据故障能否被修复，故障处理程序要么重新执行引起故障的指令，要么终止(abort)。

错误的一个经典例子是缺页异常。当一条指令引用的虚拟地址对应的页不在存储器中、必须从磁盘取出时，就会发生缺页。正如我们在第 9 章将会看到的，页是虚拟内存的一个连续块(通常为 4 KB)。缺页处理程序从磁盘装入相应的页，然后把控制返回给引起这个错误的指令。当这条指令再次执行时，相应的页已经驻留在存储器中，这条指令也就能顺利地执行完而不出错。

中止

中止是由不可恢复的致命错误造成的，通常是 DRAM 或 SRAM 位被损坏时发生的硬件错误，比如奇偶校验错误。中止处理程序从不把控制返回给应用程序。如图 8.8 所示，处理程序把控制返回给一个中止例行程序，由它来终止应用程序。

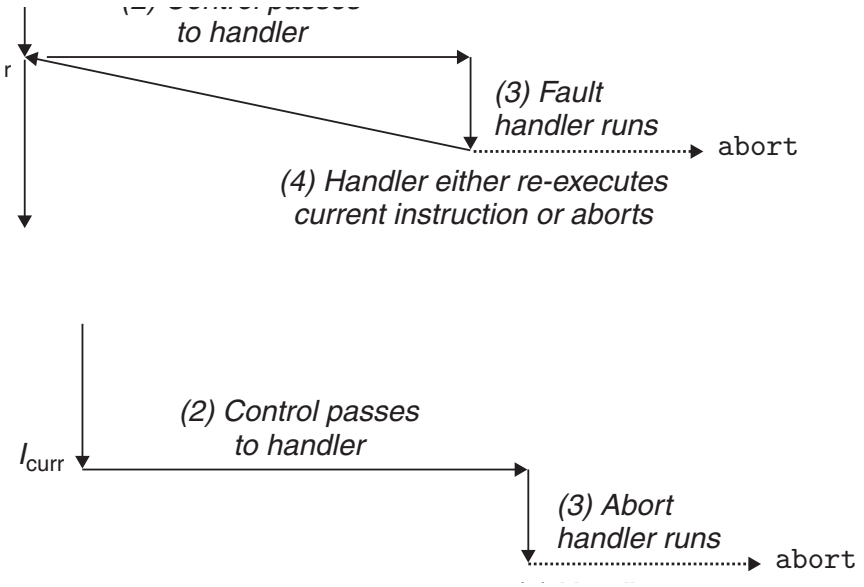


图 8.8 终止处理。终止处理程序将控制传给一个内核终止例程，该例程终止应用程序。

图 8.9 x86-64 系统中的异常示例。

异常号	描述	异常类别
0	除法错误	错误
13	一般保护错误	错误
14	缺页	错误
18	机器检查	中止
32–255	操作系统定义的异常	中断或陷阱

8.1.3 Linux/x86-64 系统中的异常

为了让内容更加具体，我们来看看为 x86-64 系统定义的一些异常。异常类型有多达 256 种 [50]。0 到 31 之间的号码对应于由 Intel 架构师定义的异常，因此对任何 x86-64 系统来说都是相同的。32 到 255 之间的号码对应于由操作系统定义的中断和陷阱。图 8.9 给出了几个例子。

Linux/x86-64 的错误与中止

除法错误。除法错误(异常 0)发生在应用程序试图除以零的时候,或者除法指令的结果对目标操作数来说太大的时候。Unix 不会试图从除法错误中恢复,而是选择中止程序。Linux shell 通常把除法错误报告为“Floating exceptions”(浮点异常)。

一般保护错误。臭名昭著的一般保护错误(异常 13)会因为许多原因发生,通常是因为程序引用了一个未定义的虚拟内存区域,或者程序试图写入一个只读的代码段。Linux 不会试图从这种错误中恢复。Linux shell 通常把一般保护错误报告为“Segmentation faults”(段错误)。

缺页。缺页(异常 14)是会重新执行引发指令的那类异常的一个例子。处理程序把磁盘上相应的虚拟内存页映射到物理内存的一页中,然后重新执行引发指令。我们将在第 9 章详细了解缺页是如何工作的。

机器检查。机器检查(异常 18)是执行引发指令的过程中检测到致命的硬件错误而导致的。机器检查处理程序从不把控制返回给应用程序。

Linux/x86-64 的系统调用

Linux 提供了几百个系统调用,供应用程序向内核请求服务时使用,比如读取文件、写文件和创建新进程。图 8.10 列出了一些常用的 Linux 系统调用。每个系统调用都有一个唯一的整数号,对应于内核中一张跳转表里的偏移量。(注意,这张跳转表与异常表不是同一张表。)

图 8.10 Linux x86-64 系统中一些常用系统调用的示例。

编号	名称	描述	编号	名称	描述
0	read	读文件	33	pause	挂起进程直到信号到达
1	write	写文件	37	alarm	安排传送告警信号
2	open	打开文件	39	getpid	获得进程 ID
3	close	关闭文件	57	fork	创建进程
4	stat	获得文件的有关信息	59	execve	执行一个程序
9	mmap	把内存页映射到文件	60	_exit	终止进程
12	brk	重置堆顶	61	wait4	等待某进程终止
32	dup2	复制文件描述符	62	kill	向进程发送信号

C 程序可以调用 `syscall` 函数直接调用任意一个系统调用。不过在实际应用中,这很少有必要。C 标准库为大多数系统调用提供了一组方便的包装函数(wrapper)。包装函数把参数打包起来,用适当的系统调用指令陷入内核,再把系统调用的返回状态传回给调用程序。本书自始至终把系统调用和它们关联的包装函数不加区分地统称为系统级函数(system-level function)。

在 x86-64 系统上，系统调用是通过一条叫作 `syscall` 的陷阱指令提供的。研究程序可以如何使用这条指令来直接调用 Linux 系统调用是相当有意思的。Linux 系统调用的所有参数都是通过通用寄存器而不是栈传递的。按照约定，寄存器`%rax` 中包含系统调用号，最多可以有六个参数存放在`%rdi`、`%rsi`、`%rdx`、`%r10`、`%r8` 和`%r9` 中。第一个参数在`%rdi` 中，第二个参数在`%rsi` 中，依此类推。从系统调用返回时，寄存器`%rcx` 和`%r11` 会被破坏，`%rax` 中包含返回值。一个介于 −4 095 与 −1 之间的负返回值表示出现了错误，对应于取负的 `errno`。

例如，考虑下面这个大家熟悉的 hello 程序的版本，它用 `write` 系统级函数(10.4 节)代替了 `printf`：

```
1  int main()
2  {
3      write(1, "hello, world\n", 13);
4      _exit(0);
5  }
```

`write` 的第一个参数把输出发送到 `stdout`。第二个参数是要写的字节序列，第三个参数给出了要写的字节数。

边栏　关于术语的说明

各类异常的术语因系统而异。处理器 ISA 规范经常区分异步的“中断”(interrupt)和同步的“异常”(exception)，却又不给出一个可以同时指代这两个非常相近的概念的统称。为了避免总是念叨“异常和中断”“异常或中断”，我们用“异常”一词作为总称，只在合适的时候才区分异步异常(中断)和同步异常(陷阱、错误和中止)。正如我们已经注意到的，对所有系统来说基本思想都是一样的，但是你应该知道，有些厂商的手册只用“异常”一词指那些由同步事件引起的控制流变化。

```
1    .section .data
2    string:
3      .ascii "hello, world\n"
4    string_end:
5      .equ len, string_end - string
6    .section .text
7    .globl main
8    main:
    首先,调用 write(1, "hello, world\n", 13)
9    movq $1, %rax           write 是 1 号系统调用
10   movq $1, %rdi           参数 1:stdout 的描述符为 1
11   movq $string, %rsi      参数 2:hello world 字符串
12   movq $len, %rdx         参数 3:字符串长度
13   syscall                 发出系统调用
    接下来,调用 _exit(0)
14   movq $60, %rax          _exit 是 60 号系统调用
15   movq $0, %rdi           参数 1:退出状态为 0
16   syscall                 发出系统调用
```

图 8.11　直接用 Linux 系统调用实现 hello 程序。

图 8.11 给出了 hello 程序的一个汇编语言版本，它用 `syscall` 指令直接调用 `write` 和 `exit` 系统调用。第 9 ~ 13 行调用 `write` 函数：第 9 行先把 `write` 系统调用的号存入`%rax`，第 10 ~ 12 行设置好参数列表，然后第 13 行用 `syscall` 指令发出系统调用。类似地，第 14 ~ 16 行调用 `_exit` 系统调用。

8.2 进程

异常是让操作系统内核得以提供进程这一概念的基本构件，而进程是计算机科学中最深刻、最成功的思想之一。

当我们把程序放到现代系统上运行时，会产生一种假象，好像我们的程序是系统中当前唯一运行的程序。我们的程序看起来好像独占地使用处理器和存储器。处理器好像是不间断地、一条接一条地执行我们程序中的指令。最后，我们程序的代码和数据看起来好像是系统存储器中唯一的对象。这些假象都是由进程这个概念提供给我们的。

进程的经典定义是“一个执行中的程序的实例”。系统中的每个程序都运行在某个进程的上下文（context）中。上下文由程序正确运行所需的状态组成。这些状态包括存放在存储器中的程序的代码和数据、它的栈、通用寄存器的内容、它的程序计数器、环境变量以及打开文件描述符的集合。

用户每次以 shell 的方式输入一个可执行目标文件的名字来运行程序时，shell 会创建一个新进程，然后在这个新进程的上下文中运行这个可执行目标文件。应用程序也能创建新进程，并且在新进程的上下文中运行自己的代码或其他应用程序。

操作系统如何实现进程的细节超出了我们的讨论范围。我们着重于进程提供给应用程序的关键抽象：

- 一个独立的逻辑控制流，让我们的程序产生独占地使用处理器的假象。
- 一个私有的地址空间，让我们的程序产生独占地使用存储器系统的假象。

我们来更仔细地看看这些抽象。

8.2.1 逻辑控制流

即使系统中通常有许多其他的程序在并发运行，进程还是让每个程序产生了自己独占处理器的假象。假如我们用调试器单步执行自己的程序，我们会看到一系列的程序计数器（PC）值，这些值完全对应于程序的可执行目标文件中的指令，或者对应于运行时动态链接进程序的那些共享对象中的指令。这一列 PC 值称为逻辑控制流（logical control flow），或简称逻辑流。

考虑一个运行着三个进程的系统，如图 8.12 所示。处理器唯一的物理控制流被划分成三个逻辑流，每个进程一个。图中，三个逻辑流的执行是相互交错的。在这个例子中，进程 A 运行了一段时间，然后是 B，B 运行到完成。接着 C 运行了一段时间，然后是 A，A 运行到完成。最后，C 得以运行到完成。

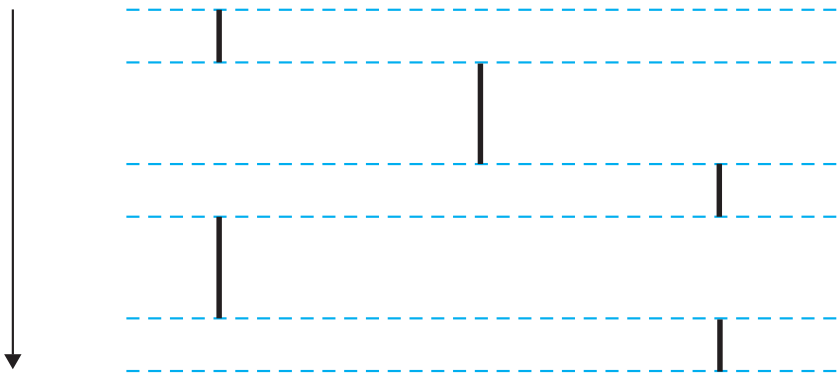


图 8.12 逻辑控制流。进程为每个程序提供一种假象，仿佛它在独占地使用处理器。每条竖线表示某个进程的逻辑控制流的一部分。

图 8.12 的关键点在于进程轮流使用处理器。每个进程执行它的流的一部分，然后被抢占（preempted，暂时挂起），让其他进程轮流执行。对运行在这些进程上下文之一的程序来说，它看起来好像独占了处理器。唯一的反证是，如果我们精确测量每条指令的执行时间，就会发现 CPU 好像在周期性地停顿于程序某些指令的执行之间。不过，每次停顿后，处理器又会继续执行我们的程序，而程序的存储器单元和寄存器的内容没有任何变化。

8.2.2 并发流

逻辑流在计算机系统中有很多不同的形式。异常处理程序、进程、信号处理程序、线程和 Java 进程都是逻辑流的例子。

一个逻辑流的执行在时间上与另一个流重叠，就称为并发流（concurrent flow），这两个流就称为并发运行（run concurrently）。更精确地说，流 X 与流 Y 相互并发，当且仅当 X 在 Y 开始之后、结束之前开始，或者 Y 在 X 开始之后、结束之前开始。例如，在图 8.12 中，进程 A 和 B 是并发运行的，A 和 C 也是如此。而 B 和 C 不是并发运行的，因为 B 的最后一条指令在 C 的第一条指令之前就执行完了。

多个流并发执行的一般现象称为并发（concurrency）。一个进程跟其他进程轮流执行的概念也称为多任务（multitasking）。进程执行它的流的一部分的每个时间段称为时间片（time slice）。因此，多任务也叫时间分片（time slicing）。例如，在图 8.12 中，进程 A 的流由两个时间片组成。

注意,并发流思想与流运行在多少个处理器核上或多少台计算机上是无关的。如果两个流在时间上重叠,那么它们就是并发的,即使它们在同一个处理器上运行。不过,我们有时会发现标识出并发流的一个真子集,也就是并行流(parallel flow),对我们有帮助。如果两个流并发地运行在不同的处理器核或者不同的计算机上,我们就说它们是并行流(parallel flows),它们在并行地运行,具有并行执行(parallel execution)。

练习 8.1 (答案见本章练习答案)考虑三个具有如下开始时间和结束时间的进程:

进程	开始时间	结束时间
A	1	3
B	2	5
C	4	6

对每一个进程对,指出它们是否并发运行(Y)或者不并发(N):

进程对	并发?
AB	
AC	
BC	

8.2.3 私有地址空间

进程为每个程序提供了它独占系统的地址空间这一假象。在一台 n 位地址的机器上,地址空间就是 2^n 个可能地址的集合,即 0、1、...、$2^n - 1$。进程给每个程序提供一个它自己的私有地址空间(private address space)。这个空间之所以是私有的,是因为一般来说,与该空间中某个特定地址相关联的存储器字节不能被任何其他进程读或写。

尽管与每个私有地址空间相关联的存储器内容一般来说是不同的,但每个这样的空间都有相同的一般结构。例如,图 8.13 给出了 x86-64 Linux 进程的地址空间结构。

地址空间的底部留给用户程序,包含通常的代码段、数据段、堆和栈段。代码段总是从地址 `0x400000` 开始。地址空间的顶部留给内核(操作系统中常驻存储器的部分)。地址空间的这个部分包含内核代表进程执行指令(例如,应用程序执行系统调用时)时所使用的代码、数据和栈。

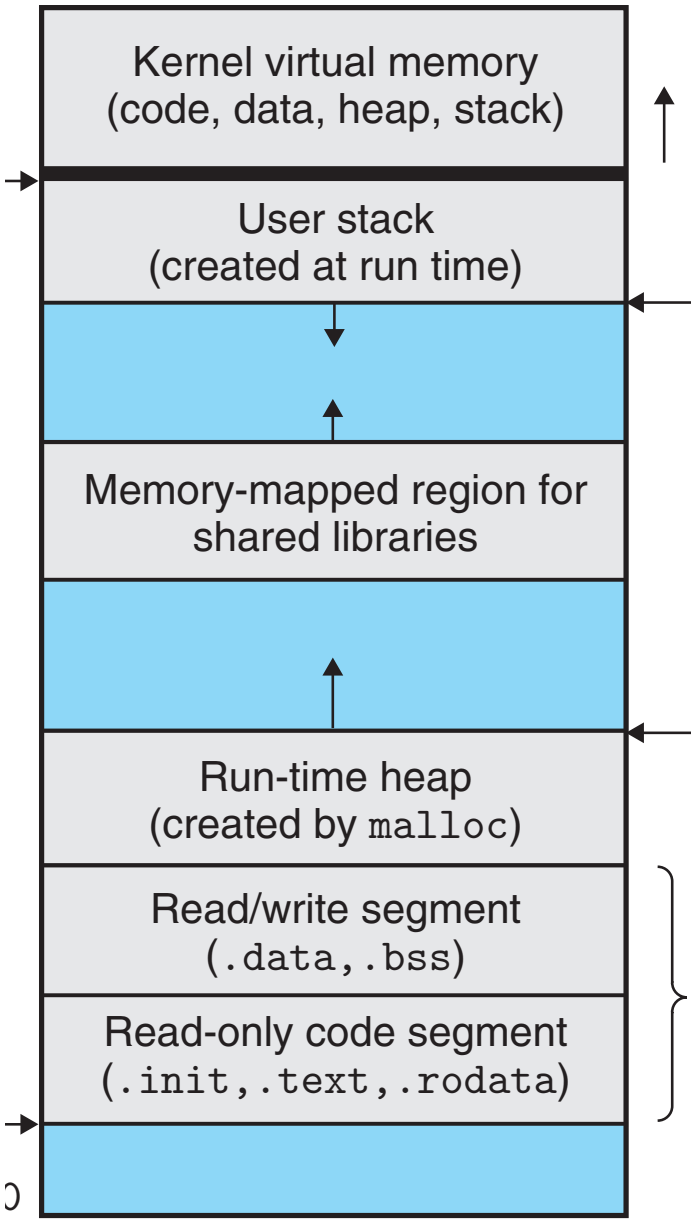


图 8.13 进程的地址空间。

8.2.4 用户模式和内核模式

为了让操作系统内核提供一个无懈可击的进程抽象，处理器必须提供一种机制，限制应用程序可以执行的指令，以及它可以访问的地址空间部分。

处理器通常用某个控制寄存器中的一个模式位（mode bit）来提供这种能力，该位描述了进程当前享有的特权。当设置了模式位时，进程就运行在内核模式（kernel mode）中（有时称为超级用户模式）。运行在内核模式中的进程可以执行指令集中的任何指令，并且可以访问系统中的任何存储器位置。

当没有设置模式位时，进程运行在用户模式（user mode）中。用户模式中的进程不允许执行特权指令（privileged instruction），比如停止处理器、改变模式位，或者发起一次 I/O 操作。也不允许用户模式中的进程直接引用地址空间中内核区的代码或数据。任何这样的企图都会导致致命的保护错误（protection fault）。用户程序必须通过系统调用接口间接地访问内核的代码和数据。

运行应用程序代码的进程最初在用户模式中。进程从用户模式变为内核模式的唯一途径是通过异常，比如中断、错误，或者陷入式的系统调用。当异常发生，控制传递到异常处理程序时，处理器将模式从用户模式变到内核模式。处理程序运行在内核模式中；当它返回到应用程序代码时，处理器再将模式从内核模式变回用户模式。

Linux 提供了一种聪明的机制，叫作 /proc 文件系统，它允许用户模式的进程访问内核数据结构的内容。/proc 文件系统把许多内核数据结构的内容导出为一组分层组织的、用户程序可以读取的文本文件。例如，你可以用 /proc 文件系统查出一般性的系统属性，比如 CPU 类型（`/proc/cpuinfo`），或者某个特定进程使用的存储器段（`/proc/process-id/maps`）。Linux 内核的 2.6 版本引入了 /sys 文件系统，它导出了关于系统总线和设备的额外的低层信息。

8.2.5 上下文切换

操作系统内核用一种更高层次的异常控制流形式来实现多任务，这种形式称为上下文切换（context switch）。上下文切换机制建立在 8.1 节讨论过的较低层次的异常机制之上。

内核为每个进程维护一个上下文。上下文就是内核重新启动一个被抢占的进程所需的状态。它由一些对象的值组成，这些对象包括通用寄存器、浮点寄存器、程序计数器、用户栈、状态寄存器、内核栈以及各种内核数据结构，比如刻画地址空间的页表、包含当前进程信息的进程表，以及包含进程已打开文件的信息的文件表。

在进程执行的某些时刻，内核可以决定抢占当前进程，并重新启动一个先前被抢占的进程。这种决定称为调度（scheduling），由内核中称为调度器

(scheduler)的代码处理。当内核选择一个新的进程运行时,我们就说内核调度了这个进程。在内核调度一个新进程运行后,它就抢占当前进程,并用一种称为上下文切换的机制将控制传递给新的进程。上下文切换会(1)保存当前进程的上下文,(2)恢复某个先前被抢占的进程保存的上下文,(3)将控制传递给这个新恢复的进程。

当内核代表用户执行系统调用时,就可能发生上下文切换。如果系统调用因为等待某个事件发生而阻塞,那么内核可以让当前进程休眠,并切换到另一个进程。例如,如果一个 `read` 系统调用需要访问磁盘,内核可以选择执行一次上下文切换,运行另一个进程,而不是等待数据从磁盘中到来。另一个例子是睡眠系统调用 `sleep`,它是让调用进程休眠的显式请求。一般说来,即便系统调用没有阻塞,内核也可以决定执行一次上下文切换,而不是把控制返回给调用进程。

上下文切换也可能由中断造成。例如,所有系统都有产生周期性定时器中断的机制,通常每 1 ms 或 10 ms 一次。每当定时器中断发生时,内核就可能判定当前进程已经运行得足够久了,转而切换到一个新的进程。

图 8.14 给出了一对进程 A 和 B 之间上下文切换的例子。在这个例子中,开始时进程 A 运行在用户模式中,直到它通过执行 `read` 系统调用陷入内核。内核中的陷阱处理程序向磁盘控制器请求一次 DMA 传输,并安排让磁盘在磁盘控制器完成把数据从磁盘传输到存储器的工作之后中断处理器。

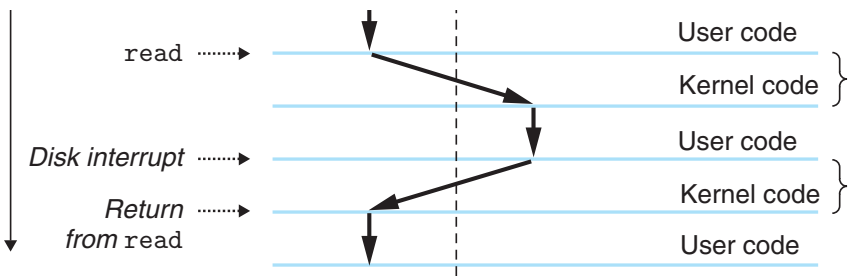


图 8.14 上下文切换的剖析。

磁盘读取数据需要相当长的时间(以几十毫秒计),所以内核不会等待并在期间无所事事,而是执行一次从进程 A 到进程 B 的上下文切换。注意,在切换之前,内核代表进程 A 以用户模式执行指令(也就是说,这里并没有一个单独的内核进程)。在切换的前半部分,内核代表进程 A 以内核模式执行指令。然后在某个时刻,它开始执行指令(仍然在内核模式中),这一次是代表进程 B。切换之后,内核代表进程 B 以用户模式执行指令。

随后进程 B 在用户模式中运行一段时间,直到磁盘发出中断,通知已经把数据从磁盘传输到了存储器。内核判定进程 B 已经运行得足够久了,就执行一

次从进程 B 到进程 A 的上下文切换，将控制返回给进程 A 中紧跟在 read 系统调用之后的那条指令。进程 A 继续运行，直到下一个异常发生，如此等等。

8.3 系统调用错误处理

Unix 系统级函数遇到错误时，通常返回 −1，并设置全局整数变量 errno 来指示出了什么错。程序员应当始终检查错误，但不幸的是，许多人会跳过错误检查，因为这会让代码膨胀、更难读。例如，调用 Linux 的 fork 函数时，我们可以像下面这样检查错误：

```
1  if ((pid = fork()) < 0) {
2      fprintf(stderr, "fork error: %s\n", strerror(errno));
3      exit(0);
4  }
```

strerror 函数为 errno 的某个特定值返回一个描述该错误的文本串。我们可以定义下面这样一个报错函数，从而在一定程度上简化这段代码：

```
1  void unix_error(char *msg) /* Unix 风格的错误 */
2  {
3      fprintf(stderr, "%s: %s\n", msg, strerror(errno));
4      exit(0);
5  }
```

有了这个函数，我们对 fork 的调用就从四行缩减到两行：

```
1  if ((pid = fork()) < 0)
2      unix_error("fork error");
```

我们还可以进一步简化代码，就是使用错误处理包装函数（error-handling wrapper）——这一方法由 Stevens[110] 开创。对一个给定的基本函数 foo，我们定义一个包装函数 Foo，它的参数完全相同，只是名称的首字母大写了。包装函数调用基本函数，检查错误，如有任何问题就终止。例如，下面是 fork 函数的错误处理包装函数：

```
1  pid_t Fork(void)
2  {
3      pid_t pid;
4
5      if ((pid = fork()) < 0)
6          unix_error("Fork error");
7      return pid;
8  }
```

有了这个包装函数，我们对 fork 的调用就缩减成紧凑的一行：

```
1  pid = Fork();
```

本书的剩余部分将一直使用错误处理包装函数。它们让我们的代码示例保持简洁，又不会给你留下可以忽略错误检查的错误印象。注意，在正文中讨论系统级函数时，我们总是用它们小写的基本名称，而不用大写的包装函数名称。

关于 Unix 错误处理以及本书通用的错误处理包装函数的讨论，见附录 A。这些包装函数定义在一个叫作 `csapp.c` 的文件中，它们原型定义在一个叫作 `csapp.h` 的头文件中。这些文件可以从 CS:APP 网站在线获得。

8.4 进程控制

Unix 提供了许多系统调用，供 C 程序对进程进行操作。本节描述其中重要的函数，并给出一些使用的例子。

8.4.1 获取进程 ID

每个进程都有一个唯一的正数（非零）进程 ID，称为 PID（process ID）。`getpid` 函数返回调用进程的 PID。`getppid` 函数返回其父进程的 PID（也就是创建调用进程的那个进程的 PID）。

```
#include <sys/types.h>
#include <unistd.h>

pid_t getpid(void);
pid_t getppid(void);

返回：调用者或其父进程的 PID
```

`getpid` 和 `getppid` 例程返回一个类型为 `pid_t` 的整数值，在 Linux 系统上，这个类型在 `types.h` 中被定义为 `int`。

8.4.2 创建和终止进程

从程序员的视角来看，我们可以认为进程处于三种状态之一：

- **运行**。 进程要么正在 CPU 上执行，要么在等待执行且终将被内核调度。
- **停止**。 进程的执行被挂起， 而且不会被调度。进程因收到 `SIGSTOP`、`SIGTSTP`、`SIGTTIN` 或者 `SIGTTOU` 信号而停止，并且保持停止状态，直到收到一个 `SIGCONT` 信号，重新变为运行。（信号是一种软件中断，8.5 节会详细描述。）
- **终止**。 进程被永久地停止。进程变成终止（terminated）有三种原因：（1）收到一个默认行为是终止进程的信号，（2）从主例程返回，（3）调用 `exit` 函数。

```
#include <stdlib.h>

void exit(int status);

此函数不会返回
```

exit 函数以 status 作为退出状态(exit status)终止进程。(设置退出状态的另一种方法是从主例程中返回一个整数值。)父进程通过调用 fork 函数创建一个新的处于运行中的子进程。

```
#include <sys/types.h>
#include <unistd.h>

pid_t fork(void);

返回:子进程中返回 0,父进程中返回子进程的 PID,出错返回 -1
```

新创建的子进程几乎但不完全与父进程相同。子进程得到一份与父进程的用户级虚拟地址空间完全相同的(但是独立分离的)副本,其中包括代码段和数据段、堆、共享库和用户栈。子进程还得到父进程所有打开文件描述符的完全相同的副本,这意味着子进程可以读写父进程调用 fork 时打开的任何文件。父进程与新创建的子进程之间最显著的区别是它们有不同的 PID。

fork 函数很有意思(理解起来也常常令人困惑),因为它被调用一次,却返回两次:一次在调用进程(父进程)中,一次在新创建的子进程中。在父进程中,fork 返回子进程的 PID;在子进程中,fork 返回 0。因为子进程的 PID 总是非零的,所以返回值提供了一种明确的方法来判断程序究竟是在父进程中还是在子进程中执行。

图 8.15 给出了一个简单的例子:父进程用 fork 创建一个子进程。当第 6 行的 fork 调用返回时,父进程和子进程中的 x 都是 1。子进程在第 8 行对它自己的 x 副本进行递增并打印。类似地,父进程在第 13 行对它自己的 x 副本进行递减并打印。

我们在 Unix 系统上运行这个程序时,会得到如下结果:

```
linux> ./fork
parent: x=0
child : x=2
```

这个简单的例子有一些微妙之处。

- **调用一次,返回两次。** fork 函数被父进程调用一次,却返回两次:一次返回给父进程,一次返回给新创建的子进程。对于只创建一个子进程的程序来说,这一点相当直白。但是含有多个 fork 实例的程序可能令人困惑,需要仔细推敲。

- **并发执行**。 父进程和子进程是并发运行的独立进程。内核可以将它们的逻辑控制流中的指令交错成任意顺序。我们在自己的系统上运行这个程序时，父进程先完成它的 `printf` 语句，接着才是子进程。不过在另一个系统上，顺序可能正好相反。一般说来，作为程序员，我们永远不能对来自不同进程的指令的交错方式做任何假设。

```
1  int main()
2  {
3      pid_t pid;
4      int x = 1;
5
6      pid = Fork();
7      if (pid == 0) { /* 子进程 */
8          printf("child : x=%d\n", ++x);
9          exit(0);
10     }
11
12     /* 父进程 */
13     printf("parent: x=%d\n", --x);
14     exit(0);
15 }
```

图 8.15 用 `fork` 创建一个新进程。

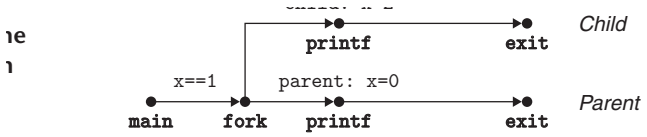
- **地址空间相同但互相分离**。 如果我们让父进程和子进程都在 `fork` 为各自返回后立刻停止执行，就会发现两个进程的地址空间是完全相同的。每个进程有同样的用户栈、同样的局部变量值、同样的堆、同样的全局变量值和同样的代码。因此，示例程序中的局部变量 x 在第 6 行 `fork` 函数返回时在父进程和子进程中都有值 1。不过，因为父进程和子进程是独立的进程，它们每方都有自己的私有地址空间。父进程或子进程对 x 的任何后续更改都是私有的，不会反映到另一个进程的存储器中。这就是为什么父进程和子进程调用各自的 `printf` 语句时，变量 x 的值不一样。

- **共享文件**。 运行这个示例程序时，我们会发现父进程和子进程都把输出打印到屏幕上。原因是子进程继承了父进程所有已打开的文件。父进程调用 `fork` 时，`stdout` 文件已经打开并指向屏幕。子进程继承这个文件，因此它的输出也指向屏幕。

初学 `fork` 函数时，画一张进程图（process graph）往往很有帮助。进程图是一种简单的前趋图（precedence graph），刻画了程序诸语句之间的偏序关系。每个顶点 a 对应于一条程序语句的执行。一条有向边 $a \to b$ 表示语句 a“先于”语句 b 发生（happens before）。边上可以标注一些信息，比如变量的当前值。对应于 `printf` 语句的顶点可以标上 `printf` 的输出。每张图都从一个对应于父进程调用 `main` 的顶点开始，这个顶点没有入边，并且恰好有一条出边。每个进程

的顶点序列都以一个对应于调用 exit 的顶点结束，这个顶点有一条入边，没有出边。

例如，图 8.16 给出了图 8.15 中的示例程序的进程图。开始时，父进程把变量 x 设置为 1。父进程调用 fork，创建一个与父进程并发运行、拥有自己私有地址空间的子进程。




```
1   int main()
2   {
3       Fork();
4       Fork();
```

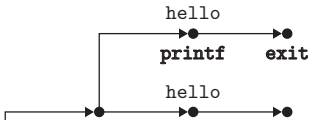


图 8.16　图 8.15 中示例程序的进程图。

对一个运行在单个处理器上的程序来说，相应进程图顶点的任何一种拓扑排序都表示程序各语句的一种可行的全序。理解拓扑排序的概念有一个简单的方法：给定进程图顶点的某个排列，把这些顶点从左到右排成一行，然后画出所有的有向边。这个排列是拓扑排序，当且仅当画出的每条边都是从左指向右的。因此，图 8.15 的示例程序中，父进程和子进程的 printf 语句可以以任一顺序发生，因为每种顺序都对应于图顶点的一个拓扑排序。

进程图在理解嵌套的 fork 调用方面特别有帮助。例如，图 8.17 给出了一个在源代码中两次调用 fork 的程序。相应的进程图帮助我们看出这个程序运行了四个进程，每个进程都调用一次 printf，而四个 printf 可以以任意顺序执行。

```
1  int main()
2  {
3      Fork();
4      Fork();
5      printf("hello\n");
6      exit(0);
7  }
```

```
fork
fork
fork
fork
printf
```

```
printf
exit
exit
hello
hello
main
main
printf
printf
exit
exit
fork
fork
printf
printf
exit
exit
hello
hello
printf
printf
exit
exit
Figure 8.17
Process graph for a nested fork.
```

练习 8.2 （答案见本章练习答案）考虑下面的程序：

```
1  int main()
2  {
3      int a = 9;
4
5      if (Fork() == 0)
6          printf("p1: a=%d\n", a--);
7      printf("p2: a=%d\n", a++);
8      exit(0);
9  }
```

A. 子进程的输出是什么？

B. 父进程的输出是什么？

8.4.3 回收子进程

当进程因为任何原因终止时，内核并不会立即把它从系统中清除。相反，进程会保持在已终止的状态中，直到被它的父进程收走（reaped）。当父进程收走到期的子进程时，内核就把子进程的退出状态传给父进程，然后抛弃这个已终止的进程；在此之后，它就不复存在了。一个终止了但还没有被收走的进程称为僵尸（zombie）进程。

当父进程终止时，内核会安排 `init` 进程成为孤儿进程的养父。`init` 进程的 PID 为 1，是在系统启动时由内核创建的，从不终止，并且是每个进程的祖

先。如果父进程终止时还没有收走它的僵尸子进程，内核就会安排 init 进程去收走它们。不过，长期运行的程序，比如 shell 或者服务器，应当始终收走它们的僵尸子进程。尽管僵尸进程不在运行，它们仍然消耗系统的存储器资源。

进程通过调用 waitpid 函数来等待它的子进程终止或者停止。

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t waitpid(pid_t pid, int *statusp, int options);

返回：成功时返回子进程的 PID；若设置了 WNOHANG 则返回 0；出错返回 -1
```

> **边栏　为什么把已终止的子进程称为僵尸？**
> 在民间传说中，僵尸是一具行走的尸体，一个半生半死的实体。僵尸进程与此相似：虽然它已经终止了，但内核仍然保存着它的一些状态，直到它被父进程收走。

waitpid 函数相当复杂。默认情况下（options 为 0），waitpid 挂起调用进程的执行，直到它的等待集（wait set）中的一个子进程终止。如果在调用 waitpid 的时候，等待集中的某个进程已经终止，那么 waitpid 会立即返回。无论哪种情况，waitpid 都会返回促使它返回的那个已终止子进程的 PID。此时，这个已终止的子进程已经被收走了，内核将它从系统中所有痕迹全部清除。

确定等待集的成员

等待集的成员由参数 pid 确定：

- 如果 pid > 0，那么等待集就是进程 ID 等于 pid 的单个子进程。
- 如果 pid $= -1$，那么等待集由父进程的所有子进程组成。

waitpid 函数还支持其他一些等待集，它们涉及 Unix 进程组，本书不做讨论。

修改默认行为

把 options 设置为 WNOHANG、WUNTRACED 和 WCONTINUED 这些常量的各种组合，可以修改默认行为：

- **WNOHANG**。 如果等待集中的任何子进程都还没有终止，就立即返回（返回值为 0）。默认行为是挂起调用进程，直到一个子进程终止；当你想在等待子进程终止的同时继续做有用工作时，这个选项很有用。
- **WUNTRACED**。 挂起调用进程的执行，直到等待集中的某个进程变成终止或者停止。返回促使返回的那个终止或停止子进程的 PID。默认行为只在子进程终止时才返回；这个选项在你同时检查终止和停止的子进程时很有用。

- **WCONTINUED**。 挂起调用进程的执行,直到等待集中的某个正在运行的进程终止,或者等到等待集中的某个已停止的进程收到 SIGCONT 信号重新启动。(信号在 8.5 节中解释。)

可以把各个选项按“或”的方式组合起来。例如:

- **WNOHANG | WUNTRACED**。 如果等待集中没有任何子进程停止或终止,就立即返回,返回值为 0;否则,返回值为已停止或已终止子进程中的某一个的 PID。

检查已收走子进程的退出状态

如果 statusp 参数非空 (non-NULL), 那么 waitpid 就会在 status 中——也就是 statusp 指向的值中——编码促使它返回的那个子进程的状态信息。头文件 wait.h 定义了若干解释 status 参数的宏:

WIFEXITED(status)。如果子进程通过调用 exit 或者一次 return 正常终止,返回真。

WEXITSTATUS(status)。返回正常终止的子进程的退出状态。这个状态只有在 WIFEXITED() 返回真时才有定义。

WIFSIGNALED(status)。如果子进程因为一个未被捕获的信号而终止,返回真。

WTERMSIG(status)。返回促使子进程终止的那个信号的编号。只有在 WIFSIGNALED() 返回真时,这个状态才有定义。

WIFSTOPPED(status)。如果促使返回的子进程当前是停止的,返回真。

WSTOPSIG(status)。返回促使子进程停止的那个信号的编号。只有在 WIFSTOPPED() 返回真时,这个状态才有定义。

WIFCONTINUED(status)。如果子进程收到 SIGCONT 信号而重新启动,返回真。

错误条件

如果调用进程没有子进程, 那么 waitpid 返回 −1, 并设置 errno 为 ECHILD。如果 waitpid 函数被一个信号中断了,那么它返回 −1,并设置 errno 为 EINTR。

练习 8.3 (答案见本章练习答案) 列出下面这个程序所有可能的输出序列:

```
1  int main()
2  {
3      if (Fork() == 0) {
4          printf("9"); fflush(stdout);
5      }
```

```
6       else {
7           printf("0"); fflush(stdout);
8           waitpid(-1, NULL, 0);
9       }
10      printf("3"); fflush(stdout);
11      printf("6"); exit(0);
12  }
```

wait 函数

wait 函数是 waitpid 的一个简化版本。

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *statusp);

返回:成功时返回子进程的 PID;出错返回 -1
```

调用 wait(&status) 等价于调用 waitpid(-1, &status, 0)。

使用 waitpid 的例子

因为 waitpid 函数有些复杂,看几个例子会有帮助。图 8.18 给出了一个程序,它用 waitpid 按任意的次序等待它的 N 个子进程全部终止。第 11 行,父进程创建了每一个子进程;第 12 行,每个子进程以一个唯一的退出状态退出。

边栏 与 Unix 函数相关的常量
WNOHANG 和 WUNTRACED 这类常量由系统头文件定义。例如,WNOHANG 和 WUNTRACED 由 wait.h 头文件(间接地)定义:

```
/* waitpid 第三个参数中的各位。 */
#define WNOHANG    1   /* 不阻塞等待。 */
#define WUNTRACED  2   /* 报告已停止子进程的状态。 */
```

为了使用这些常量,必须在你的代码中包含 wait.h 头文件:

```
#include <sys/wait.h>
```

每个 Unix 函数的手册页都列出了在代码中使用该函数时应当包含的头文件。另外,要检查 ECHILD 和 EINTR 这类返回码,必须包含 errno.h。为了简化我们的代码示例,我们包含一个单独的头文件 csapp.h,它包含了书中用到的所有函数的头文件。csapp.h 头文件可以从 CS:APP 网站在线获得。

```
1  #include "csapp.h"
2  #define N 2
3
4  int main()
5  {
6      int status, i;
```

```
7       pid_t pid;
8
9       /* 父进程创建 N 个子进程 */
10      for (i = 0; i < N; i++)
11          if ((pid = Fork()) == 0) /* 子进程 */
12              exit(100+i);
13
14      /* 父进程按任意顺序收走 N 个子进程 */
15      while ((pid = waitpid(-1, &status, 0)) > 0) {
16          if (WIFEXITED(status))
17              printf("child %d terminated normally with exit
         status=%d\n",
18                     pid, WEXITSTATUS(status));
19          else
20              printf("child %d terminated abnormally\n", pid);
21      }
22
23      /* 只有一种正常终止:必然是因为已经没有子进程了 */
24      if (errno != ECHILD)
25          unix_error("waitpid error");
26
27      exit(0);
28  }
```

图 8.18　用 waitpid 函数按任意顺序收走僵尸子进程。

在继续之前，确认你理解为什么第 12 行由每个子进程执行，而父进程不执行。

第 15 行，父进程把 waitpid 作为 while 循环的测试条件，等待它所有的子进程终止。因为第一个参数是 −1，所以对 waitpid 的调用来阻塞，直到任意一个子进程终止为止。当每个子进程终止时，对 waitpid 的调用都会以该子进程的非零 PID 返回。第 16 行检查子进程的退出状态。如果子进程正常终止——在本例中，是通过调用 exit 函数——那么父进程就取出退出状态，并把它打印到 stdout 上。

当所有的子进程都被收走后，对 waitpid 的下一次调用返回 −1，并把 errno 设置为 ECHILD。第 24 行检查 waitpid 函数是否是正常终止，如果不是就打印一条错误消息。我们在自己的 Linux 系统上运行这个程序时，得到如下输出：

```
linux> ./waitpid1
child 22966 terminated normally with exit status=100
child 22967 terminated normally with exit status=101
```

注意，这个程序收走其子进程的顺序是任意的。收走的顺序是这台特定计算机系统的一个属性。在另一个系统上，或者甚至在同一系统上再执行一次，这两个子进程被收走的顺序可能就相反了。这是不确定性（nondeterminism）行

为的一个例子,正是它使得对并发的推理变得如此困难。两种可能的结果都同样是正确的,而作为程序员,无论另一种结局看起来多么不可能,你永远都不能假设某一种结局总是会发生。唯一正确的假设是:每一种可能的结果出现的可能性都是相等的。

图 8.19 给出了一个简单的修改,它按照子进程被父进程创建的顺序收走子进程,从而消除了输出顺序上的这种不确定性。第 11 行,父进程把子进程的 PID 按顺序存放好,然后用适当的 PID 作为第一个参数调用 `waitpid`,按同样的顺序等待每一个子进程。

练习 8.4 (答案见本章练习答案)考虑下面的程序:

```
1  int main()
2  {
3      int status;
4      pid_t pid;
5
6      printf("Start\n");
7      pid = Fork();
8      printf("%d\n", !pid);
9      if (pid == 0) {
10         printf("Child\n");
11     }
12     else if ((waitpid(-1, &status, 0) > 0) &&
               (WIFEXITED(status) != 0)) {
13         printf("%d\n", WEXITSTATUS(status));
14     }
15     printf("Stop\n");
16     exit(2);
17 }
```

A. 这个程序会产生多少行输出?

B. 这些输出行的一种可能的顺序是什么?

```
1  #include "csapp.h"
2  #define N 2
3
4  int main()
5  {
6      int status, i;
7      pid_t pid[N], retpid;
8
9      /* 父进程创建 N 个子进程 */
10     for (i = 0; i < N; i++)
11         if ((pid[i] = Fork()) == 0)          /* 子进程 */
12             exit(100+i);
13
14     /* 父进程按创建的顺序回收 N 个子进程 */
15     i = 0;
16     while ((retpid = waitpid(pid[i++], &status, 0)) > 0) {
17         if (WIFEXITED(status))
```

```
18              printf("child %d terminated normally with exit
        status=%d\n",
19                      retpid, WEXITSTATUS(status));
20          else
21              printf("child %d terminated abnormally\n", retpid);
22      }
23
24      /* 唯一正常的终止情形是没有子进程可等待了 */
25      if (errno != ECHILD)
26          unix_error("waitpid error");
27
28      exit(0);
29  }
```

图 8.19　使用 waitpid 按创建的顺序回收僵死子进程。

8.4.4 让进程休眠

sleep 函数将一个进程挂起一段指定的时间。

```
#include <unistd.h>

unsigned int sleep(unsigned int secs);
        返回:剩余休眠的秒数
```

如果所要求的时间已经过去, sleep 就返回 0; 否则返回剩余还需要休眠的秒数。后一种情形是有可能发生的:如果 sleep 函数因为被一个信号中断而提前返回,就会出现这种情况。我们将在 8.5 节中详细讨论信号。

我们会发现另一个很有用的函数是 pause, 它让调用该函数的进程休眠,直到该进程接收到一个信号为止。

```
#include <unistd.h>

int pause(void);
        返回:总是 -1
```

练习 8.5　为 sleep 函数编写一个包装函数 wakeup,它具有如下接口:

```
unsigned int wakeup(unsigned int secs);
```

wakeup 函数的行为与 sleep 函数完全一样, 只是它会打印一条消息, 说明进程实际上是在何时醒来的:

```
Woke up at 4 secs.
```

8.4.5 加载并运行程序

execve 函数在当前进程的上下文中加载并运行一个新程序。

```
#include <unistd.h>

int execve(const char *filename, const char *argv[],
           const char *envp[]);
        返回:成功不返回;出错返回 -1
```

execve 函数加载并运行可执行目标文件 filename,带参数列表 argv 和环境变量列表 envp。只有在出现错误(比如找不到 filename)时,execve 才会返回到调用程序。与调用一次、返回两次的 fork 不同,execve 调用一次,而且从不返回。

参数列表由图 8.20 所示的数据结构表示。argv 变量指向一个以空值结尾的指针数组,每个指针都指向一个参数字符串。按照惯例,argv[0] 是可执行目标文件的名字。环境变量列表由一个类似的数据结构表示,如图 8.21 所示。envp 变量指向一个以空值结尾的指针数组,每个指针指向一个环境变量字符串,而每个环境变量字符串都是形如 name=value 的名值对。

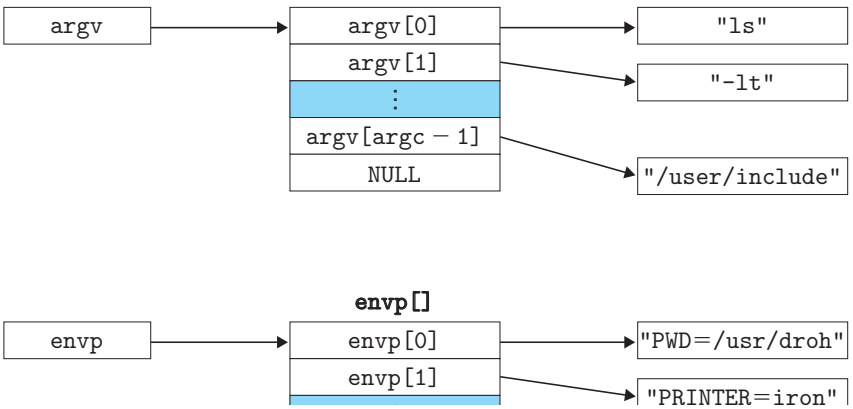


图 8.20 参数列表的组织。

execve 在加载了 filename 之后,会调用 7.9 节中描述的启动代码。启动代码设置好栈,并把控制传给新程序的 main 例程,main 具有如下形式的原型:

```
int main(int argc, char **argv, char **envp);
```

或者等价地,

```
int main(int argc, char *argv[], char *envp[]);
```

main 开始执行时,用户栈具有图 8.22 所示的组织结构。我们从栈底(最高地址)开始,一路看到栈顶(最低地址)。最先是参数字符串和环境字符串。在这之上,是一个以空值结尾的指针数组,每个指针指向栈上的一个环境变量字符串。全局变量 environ 指向这些指针中的第一个,即 envp[0]。环境数组

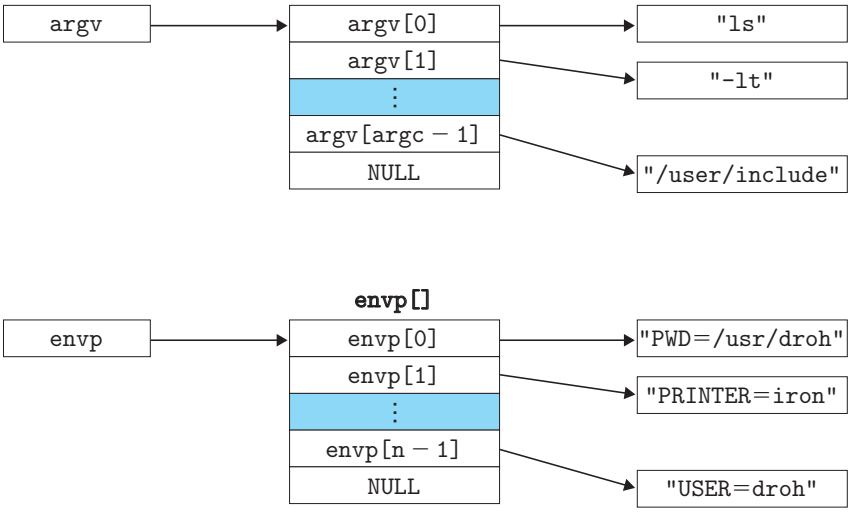


图 8.21 环境变量列表的组织。

之后是以空值结尾的 argv[] 数组，其中每个元素都指向栈上的一个参数字符串。栈的顶端是系统启动函数 libc_start_main(7.9 节)的栈帧。

函数 main 有三个参数，按照 x86-64 的栈纪律，每个参数都保存在一个寄存器中：(1) argc，给出 argv[] 数组中非空指针的个数；(2) argv，指向 argv[] 数组的第一个条目；(3) envp，指向 envp[] 数组的第一个条目。

Linux 提供了几个用于操作环境数组的函数：

```
#include <stdlib.h>

char *getenv(const char *name);
        返回：若存在则返回指向 name 的指针；若无匹配则返回 NULL
```

getenv 函数在环境数组中搜索字符串 name=value。如果找到了，就返回一个指向 value 的指针；否则返回 NULL。

```
#include <stdlib.h>

int setenv(const char *name, const char *newvalue, int overwrite);
        返回：成功返回 0；出错返回 -1

void unsetenv(const char *name);
        返回：无
```

如果环境数组中包含一个形如 name=oldvalue 的字符串，那么 unsetenv 会删除它，而 setenv 会用 newvalue 替换 oldvalue，但仅当 overwrite 非零时才替换。如果 name 不存在，那么 setenv 就把 name=newvalue 加入到数组中。

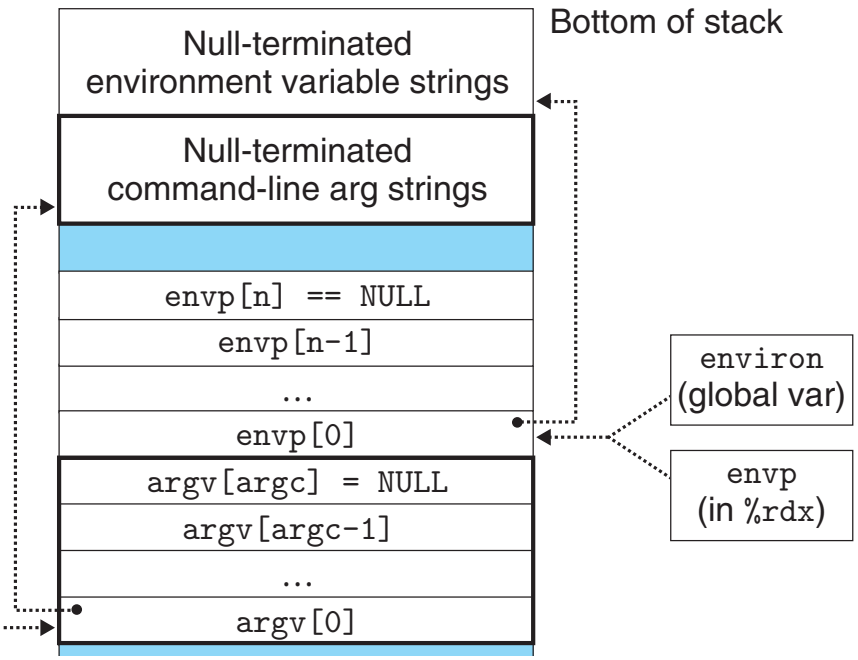


图 8.22 新程序开始时用户栈的典型组织。

练习 8.6 编写一个叫作 `myecho` 的程序，打印出它自己的命令行参数和环境变量。例如：

```
linux> ./myecho arg1 arg2
Command-line arguments:
    argv[ 0]: myecho
    argv[ 1]: arg1
    argv[ 2]: arg2
Environment variables:
    envp[ 0]: PWD=/usr0/droh/ics/code/ecf
    envp[ 1]: TERM=emacs
    .
    .
    .
    envp[25]: USER=droh
    envp[26]: SHELL=/usr/local/bin/tcsh
    envp[27]: HOME=/usr0/droh
```

8.4.6 使用 fork 和 execve 运行程序

像 Unix shell 和 Web 服务器这样的程序大量使用 `fork` 和 `execve` 函数。shell 是一种交互式的应用级程序，它代表用户运行其他程序。最初的 shell 是 `sh` 程序，随后出现了 `csh`、`tcsh`、`ksh` 和 `bash` 等变体。shell 执行一系列读/求值步骤，然后终止。读步骤从用户那里读取一条命令行。求值步骤解析这条命令行，并代表用户运行程序。

图 8.23 展示了一个简单 shell 的主例程。shell 打印一个命令行提示符，等待用户在标准输入上键入一条命令行，然后对这条命令行求值。

图 8.24 展示了对命令行求值的代码。它的第一个任务是调用 `parseline` 函数（图 8.25），该函数解析以空格分隔的命令行参数，并构造出随后会被传给 `execve` 的 `argv` 向量。第一个参数或者是一个会被立即解释的内建 shell 命令的名字，或者是一个将被加载并在新子进程的上下文中运行的可执行目标文件。

如果最后一个参数是一个‘&’字符，那么 `parseline` 返回 1，表明程序应该在后台执行（shell 不等待它完成）。否则，它返回 0，表明程序应该在前台运行（shell 等待它完成）。

> **边栏　程序与进程**
> 这正是停下来确认你是否理解了程序与进程之间区别的好时机。程序是代码和数据的集合；程序可以作为目标文件存在于磁盘上，也可以作为段存在于地址空间中。进程是程序在执行中的一个具体实例；程序总是在某个进程的上下文中运行。如果你想理解 `fork` 和 `execve` 函数，理解这个区别就很重要。`fork` 函数在一个新的子进程中运行同一个程序，这个子进程是父进程的一个复制品。`execve` 函数在当前进程的上下文中加载并运行一个新程序。它虽然会覆盖当前进程的地址空间，但并不创建一个新进程。新程序仍然具有同一个 PID，并且继承了调用 `execve` 时所有已经打开的文件描述符。

```
1  #include "csapp.h"
2  #define MAXARGS   128
3
4  /* 函数原型 */
5  void eval(char *cmdline);
6  int parseline(char *buf, char **argv);
7  int builtin_command(char **argv);
8
9  int main()
10 {
11     char cmdline[MAXLINE]; /* 命令行 */
12
13     while (1) {
14         /* 读 */
15         printf("> ");
16         Fgets(cmdline, MAXLINE, stdin);
17         if (feof(stdin))
18             exit(0);
19
20         /* 求值 */
21         eval(cmdline);
22     }
23 }
```

图 8.23　一个简单 shell 程序的主例程。

解析命令行之后，eval 函数会调用 builtin_command 函数，后者检查第一个命令行参数是不是一个内建 shell 命令。如果是，就立即解释这条命令并返回 1；否则返回 0。我们的简单 shell 只有一个内建命令，即终止 shell 的 quit 命令。真实的 shell 有许多命令，比如 pwd、jobs 和 fg。

如果 builtin_command 返回 0，那么 shell 就创建一个子进程，并在该子进程内执行所请求的程序。如果用户要求程序在后台运行，那么 shell 就返回到循环的顶端，等待下一条命令行。否则，shell 使用 waitpid 函数等待这个作业终止。作业终止后，shell 继续进行下一轮迭代。

注意，这个简单 shell 是有缺陷的，因为它不回收任何后台子进程。纠正这个缺陷需要用到信号，我们将在下一节中描述信号。

```
1  /* eval - 对一条命令行求值 */
2  void eval(char *cmdline)
3  {
4      char *argv[MAXARGS]; /* execve() 的参数列表 */
5      char buf[MAXLINE];   /* 保存修改过的命令行 */
6      int bg;              /* 作业应该在后台还是前台运行？ */
7      pid_t pid;           /* 进程 ID */
8
9      strcpy(buf, cmdline);
10     bg = parseline(buf, argv);
11     if (argv[0] == NULL)
12         return;   /* 忽略空行 */
13
14     if (!builtin_command(argv)) {
15         if ((pid = Fork()) == 0) {   /* 子进程运行用户作业 */
16             if (execve(argv[0], argv, environ) < 0) {
17                 printf("%s: Command not found.\n", argv[0]);
18                 exit(0);
19             }
20         }
21
22         /* 父进程等待前台作业终止 */
23         if (!bg) {
24             int status;
25             if (waitpid(pid, &status, 0) < 0)
26                 unix_error("waitfg: waitpid error");
27         }
28         else
29             printf("%d %s", pid, cmdline);
30     }
31     return;
32 }
33
34 /* 如果第一个参数是内建命令，就运行它并返回真 */
35 int builtin_command(char **argv)
36 {
37     if (!strcmp(argv[0], "quit")) /* quit 命令 */
```

```
38          exit(0);
39      if (!strcmp(argv[0], "&"))    /* 忽略单独的 & */
40          return 1;
41      return 0;                     /* 不是内建命令 */
42  }
```

图 8.24 `eval` 对 shell 命令行求值。

```
1   /* parseline - 解析命令行并构造 argv 数组 */
2   int parseline(char *buf, char **argv)
3   {
4       char *delim;         /* 指向第一个空格分隔符 */
5       int argc;            /* 参数个数 */
6       int bg;              /* 后台作业？ */
7
8       buf[strlen(buf)-1] = ' '; /* 把行尾的 '\n' 替换成空格 */
9       while (*buf && (*buf == ' ')) /* 忽略前导空格 */
10          buf++;
11
12      /* 构造 argv 列表 */
13      argc = 0;
14      while ((delim = strchr(buf, ' '))) {
15          argv[argc++] = buf;
16          *delim = '\0';
17          buf = delim + 1;
18          while (*buf && (*buf == ' ')) /* 忽略空格 */
19              buf++;
20      }
21      argv[argc] = NULL;
22
23      if (argc == 0)       /* 忽略空行 */
24          return 1;
25
26      /* 作业是否应该在后台运行？ */
27      if ((bg = (*argv[argc-1] == '&')) != 0)
28          argv[--argc] = NULL;
29
30      return bg;
31  }
```

图 8.25 `parseline` 解析 shell 的一行输入。

8.5 信号

到目前为止，在学习异常控制流的过程中，我们已经看到硬件和软件如何合作来提供基本的低层异常机制。我们也看到了操作系统如何利用异常来支持进程上下文切换这种形式的异常控制流。在本节中，我们将研究一种更高层的、软件形式的异常控制流，称为 Linux 信号（signal），它允许进程和内核中断其他进程。

图 8.26 Linux 信号。注:(a) 多年以前,主存是用一种称为磁芯存储器(core memory)的技术实现的。"转储核心(dumping core)"是一个历史术语,意思是把代码段和数据段的存储器映像写到磁盘上。(b) 该信号既不能被捕获,也不能被忽略。(来源:`man 7 signal`。数据来自 Linux 基金会。)

编号	名称	默认动作	相应事件
1	SIGHUP	终止	终端线路挂断
2	SIGINT	终止	来自键盘的中断
3	SIGQUIT	终止	来自键盘的退出
4	SIGILL	终止	非法指令
5	SIGTRAP	终止并转储核心[a]	跟踪陷阱
6	SIGABRT	终止并转储核心[a]	来自 abort 函数的中止信号
7	SIGBUS	终止	总线错误
8	SIGFPE	终止并转储核心[a]	浮点异常
9	SIGKILL	终止[b]	杀死程序
10	SIGUSR1	终止	用户定义信号 1
11	SIGSEGV	终止并转储核心[a]	无效的存储器引用(段违例)
12	SIGUSR2	终止	用户定义信号 2
13	SIGPIPE	终止	向一个没有读端的管道写入
14	SIGALRM	终止	来自 alarm 函数的定时器信号
15	SIGTERM	终止	软件终止信号
16	SIGSTKFLT	终止	协处理器上的栈错误
17	SIGCHLD	忽略	一个子进程已停止或已终止
18	SIGCONT	忽略	如果进程已停止,则使其继续
19	SIGSTOP	停止直到下一个 SIGCONT[b]	不来自终端的停止信号
20	SIGTSTP	停止直到下一个 SIGCONT	来自终端的停止信号
21	SIGTTIN	停止直到下一个 SIGCONT	后台进程从终端读
22	SIGTTOU	停止直到下一个 SIGCONT	后台进程向终端写
23	SIGURG	忽略	套接字上的紧急条件
24	SIGXCPU	终止	超出 CPU 时间限制
25	SIGXFSZ	终止	超出文件大小限制
26	SIGVTALRM	终止	虚拟定时器期满
27	SIGPROF	终止	性能分析定时器期满
28	SIGWINCH	忽略	窗口大小改变
29	SIGIO	终止	描述符上的 I/O 现在可以进行
30	SIGPWR	终止	电源失效

信号是一条小消息,它通知进程系统中出现了某种类型的事件。图 8.26 显示了 Linux 系统支持的 30 种不同的信号类型。

每种信号类型都对应于某种系统事件。低层的硬件异常由内核的异常处理程序处理,用户进程通常看不到这些异常。信号提供了一种机制,把这些异常的发生情况暴露给用户进程。例如,如果一个进程试图除以零,那么内核就发送给它一个 SIGFPE 信号(编号 8)。如果一个进程执行了一条非法指令,内核就发送给它一个 SIGILL 信号(编号 4)。如果一个进程进行了非法的存储器

引用，内核就发送给它一个 SIGSEGV 信号（编号 11）。其他信号对应于内核或其他用户进程中较高层的软件事件。例如，如果一个进程正在前台运行，那么你键入 Ctrl+C（即同时按下 Ctrl 键和 'c' 键）时，内核就向前台进程组（process group）中的每个进程发送一个 SIGINT 信号（编号 2）。一个进程可以通过发送一个 SIGKILL 信号（编号 9）来强行终止另一个进程。当一个子进程终止或停止时，内核就向父进程发送一个 SIGCHLD 信号（编号 17）。

8.5.1 信号术语

把信号传递给目标进程发生在两个不同的步骤中：

- **发送信号。** 内核通过更新目标进程上下文中的某个状态，向目标进程发送（递送）一个信号。信号的递送出于以下两种原因之一：(1) 内核检测到了一个系统事件，比如除零错误或一个子进程终止。(2) 一个进程调用了 kill 函数（下一节讨论），显式要求内核把信号发送给目标进程。一个进程可以给自己发送信号。
- **接收信号。** 当内核促使目标进程以某种方式对信号的递送作出反应时，目标进程就接收了信号。进程可以忽略这个信号，可以终止，也可以通过执行一个称为信号处理程序（signal handler）的用户层函数来捕获这个信号。图 8.27 展示了处理程序捕获信号的基本思想。

已经发送但尚未被接收的信号称为待处理信号（pending signal）。在任何时刻，一种特定类型的待处理信号至多只有一个。如果一个进程有一个类型为 k 的待处理信号，那么随后发送给这个进程的类型为 k 的任何信号都不会排队等待；它们只是被简单丢弃。进程可以有选择地阻塞某种信号的接收。当信号被阻塞时，它仍然可以被递送，但是所产生的待处理信号在进程解除对这种信号的阻塞之前不会被接收。

一个待处理信号至多被接收一次。对每个进程，内核维护两个集合：pending 位向量中的待处理信号集合，以及 blocked 位向量中的被阻塞信号集合[1]。每当递送一个类型为 k 的信号时，内核就设置 pending 中的第 k 位；每当接收一个类型为 k 的信号时，内核就清除 blocked 中的第 k 位。

8.5.2 发送信号

Unix 系统提供了多种把信号发送给进程的机制。所有这些机制都依赖于进程组的概念。

[1] 也称为信号掩码（signal mask）。

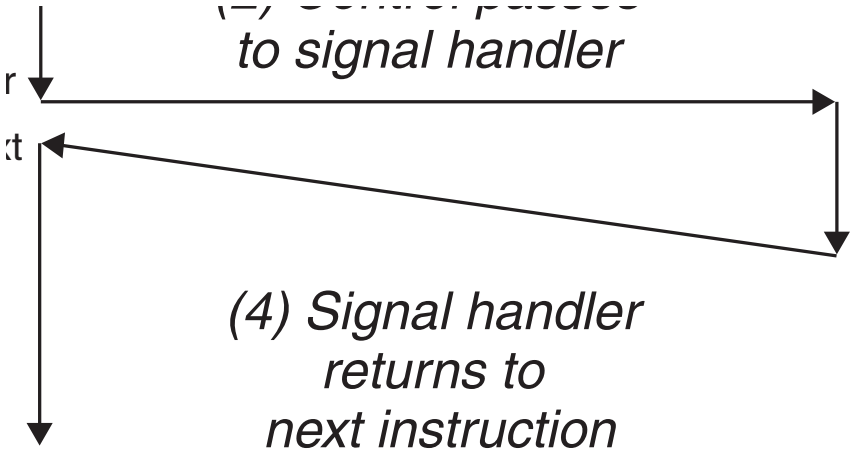


图 8.27 信号处理。收到一个信号会触发控制转移到信号处理程序。处理程序完成处理后，将控制返回给被中断的程序。

进程组

每个进程都属于恰好一个进程组，进程组由一个正整数进程组 ID（process group ID）标识。`getpgrp` 函数返回调用进程的进程组 ID。

```
#include <unistd.h>

pid_t getpgrp(void);
        返回：调用进程的进程组 ID
```

默认情况下，子进程与它的父进程属于同一个进程组。一个进程可以使用 `setpgid` 函数来改变自己或其他进程的进程组：

```
#include <unistd.h>

int setpgid(pid_t pid, pid_t pgid);
        返回：成功返回 0；出错返回 -1
```

`setpgid` 函数把进程 `pid` 的进程组改为 `pgid`。如果 `pid` 为零，就使用当前进程的 PID。如果 `pgid` 为零，就用 `pid` 所指定进程的 PID 作为进程组 ID。例如，如果进程 15213 是调用进程，那么

```
setpgid(0, 0);
```

就会创建一个新的进程组，其进程组 ID 是 15213，并且把进程 15213 加入到这个新组中。

用 /bin/kill 程序发送信号

`/bin/kill` 程序向另一个进程发送一个任意的信号。例如，命令

```
linux> /bin/kill -9 15213
```

向进程 15213 发送信号 9(SIGKILL)。一个负的 PID 会导致信号被发送到进程组 PID 中的每个进程。例如,命令

```
linux> /bin/kill -9 -15213
```

向进程组 15213 中的每个进程发送一个 SIGKILL 信号。注意,我们在这里使用完整路径 /bin/kill,是因为有些 Unix shell 有它们自己的内建 kill 命令。

从键盘发送信号

Unix shell 使用作业(job)这个抽象概念来表示求值一条命令行所产生的那些进程。在任何时刻,至多只有一个前台作业和零个或多个后台作业。例如,键入

```
linux> ls | sort
```

会创建一个前台作业,它由两个用一条 Unix 管道连接起来的进程组成:一个进程运行 ls 程序,另一个进程运行 sort 程序。shell 为每个作业创建一个单独的进程组。进程组 ID 通常取自作业中的某个父进程。例如,图 8.28 展示了一个 shell,它有一个前台作业和两个后台作业。前台作业中的父进程 PID 是 20,进程组 ID 是 20。父进程创建了两个子进程,它们也都是进程组 20 的成员。

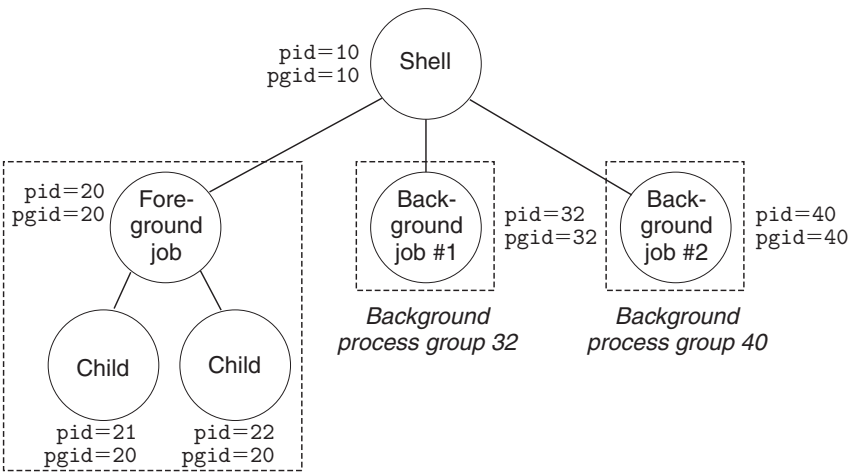


图 8.28 前台和后台进程组。

在键盘上键入 Ctrl+C 会促使内核向前台进程组中的每个进程发送一个 SIGINT 信号。默认情况下,结果是终止前台作业。类似地,键入 Ctrl+Z 会促使

内核向前台进程组中的每个进程发送一个 SIGTSTP 信号。默认情况下，结果是停止(挂起)前台作业。

用 kill 函数发送信号

进程通过调用 kill 函数向其他进程(包括它们自己)发送信号。

```
#include <sys/types.h>
#include <signal.h>

int kill(pid_t pid, int sig);
        返回:成功返回 0;出错返回 -1
```

如果 pid 大于零,那么 kill 函数向进程 pid 发送信号 sig。如果 pid 等于零,那么 kill 向调用进程所在的进程组中的每个进程发送信号 sig,包括调用进程本身。如果 pid 小于零,那么 kill 向进程组 |pid|(pid 的绝对值)中的每个进程发送信号 sig。图 8.29 给出了一个示例，父进程使用 kill 函数向它的子进程发送一个 SIGKILL 信号。

```
1  #include "csapp.h"
2
3  int main()
4  {
5      pid_t pid;
6
7      /* 子进程休眠,直到收到 SIGKILL 信号,然后死去 */
8      if ((pid = Fork()) == 0) {
9          Pause(); /* 等待一个信号到来 */
10         printf("control should never reach here!\n");
11         exit(0);
12     }
13
14     /* 父进程向子进程发送一个 SIGKILL 信号 */
15     Kill(pid, SIGKILL);
16     exit(0);
17 }
```

图 8.29　使用 kill 函数向子进程发送信号。

用 alarm 函数发送信号

进程可以通过调用 alarm 函数向自己发送 SIGALRM 信号。

```
#include <unistd.h>

unsigned int alarm(unsigned int secs);
        返回:前一个报警剩余的秒数;若无前一个报警则返回 0
```

alarm 函数安排内核在 secs 秒之后再向调用进程发送一个 SIGALRM 信号。如果 secs 为零,就不会安排任何新的报警。无论如何,对 alarm 的调用会

取消任何待处理的报警，并返回在一个待处理报警本应被递送之前剩下的秒数（假如这次对 alarm 的调用没有把它取消）；如果不存在待处理报警，就返回 0。

8.5.3 接收信号

当内核把进程 p 从内核模式切换到用户模式时（例如，从系统调用返回，或者完成一次上下文切换），它就会检查进程 p 未被阻塞的待处理信号集合（pending & ~blocked）。如果这个集合为空（通常的情况），那么内核就把控制传递给 p 的逻辑控制流中的下一条指令 I_{next}。然而，如果这个集合非空，那么内核就选择该集合中的某个信号 k（通常是编号最小的 k），并促使 p 接收信号 k。接收到信号会触发进程的某个动作。进程完成这个动作之后，控制就传回 p 的逻辑控制流中的下一条指令 I_{next}。每种信号类型都有一个预定义的默认动作（default action），它是下列动作中的某一个：

- 进程终止。
- 进程终止并转储核心。
- 进程停止（挂起），直到被一个 SIGCONT 信号重启。
- 进程忽略该信号。

图 8.26 显示了与每种信号类型相关联的默认动作。例如，接收到 SIGKILL 的默认动作是终止接收进程；而接收到 SIGCHLD 的默认动作则是忽略这个信号。进程可以使用 signal 函数来修改与某个信号相关联的默认动作。仅有的例外是 SIGSTOP 和 SIGKILL，它们的默认动作不能被改变。

```
#include <signal.h>

typedef void (*sighandler_t)(int);

sighandler_t signal(int signum, sighandler_t handler);
        返回：成功则返回指向前一个处理程序的指针；出错则返回 SIG_ERR（不设置
        ↪ errno）
```

signal 函数可以通过三种不同的方式来改变与信号 signum 相关联的动作：

- 如果 handler 是 SIG_IGN，那么类型为 signum 的信号会被忽略。
- 如果 handler 是 SIG_DFL，那么类型为 signum 的信号的动作就恢复为默认动作。
- 否则，handler 就是一个用户定义函数的地址，这个函数称为信号处理程序（signal handler），每当进程接收到一个类型为 signum 的信号时，就会调用它。通过给 signal 函数传递一个处理程序的地址来改变默认动作，

称为安装处理程序（installing the handler）。调用处理程序称为捕获信号（catching the signal）。执行处理程序称为处理信号（handling the signal）。

当进程捕获到一个类型为 k 的信号时，为信号 k 安装的处理程序会以一个整型参数被调用，该参数被设置为 k。这个参数允许同一个处理程序函数捕获不同类型的信号。

当处理程序执行它的 `return` 语句时，控制（通常）会传回到控制流中的那条指令——也就是进程接收到信号而被中断的那条指令。我们说“通常”，是因为在某些系统中，被中断的系统调用会立即带着一个错误返回。

图 8.30 给出了一个捕获 `SIGINT` 信号的程序，每当用户在键盘上键入 Ctrl+C 时，就会发送这个信号。

```
1  #include "csapp.h"
2
3  void sigint_handler(int sig) /* SIGINT 处理程序 */
4  {
5      printf("Caught SIGINT!\n");
6      exit(0);
7  }
8
9  int main()
10 {
11     /* 安装 SIGINT 处理程序 */
12     if (signal(SIGINT, sigint_handler) == SIG_ERR)
13         unix_error("signal error");
14
15     pause(); /* 等待接收一个信号 */
16
17     return 0;
18 }
```

图 8.30 一个使用信号处理程序捕获 SIGINT 信号的程序。

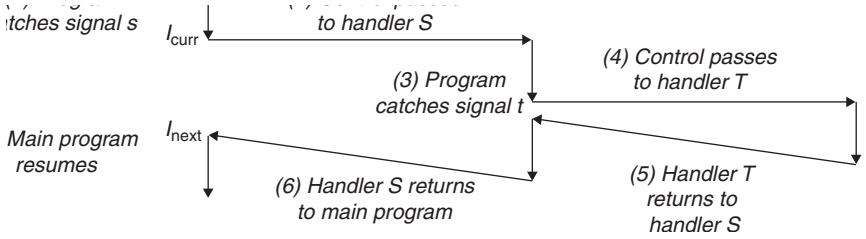


图 8.31 处理程序可以被其他处理程序中断。

`SIGINT` 的默认动作是立即终止进程。在这个例子中，我们把默认行为改为：捕获这个信号，打印一条消息，然后终止进程。

信号处理程序可能被其他信号处理程序中断，如图 8.31 所示。在这个例子中，主程序捕获信号 s，这会中断主程序，并把控制传给处理程序 S。当 S 正在

运行时,程序捕获信号 $t \neq s$,这会中断 S 并把控制传给处理程序 T。当 T 返回时,S 从它被中断的地方恢复执行。最终,S 返回,把控制交还给主程序,主程序从它离开的地方恢复执行。

练习 8.7 编写一个叫作 snooze 的程序,它取一个简单的命令行参数,用这个参数调用练习 8.5 中的 snooze 函数,然后终止。安排你的程序,使得用户可以通过在键盘上键入 Ctrl+C 来中断 snooze 函数。例如:

```
linux> ./snooze 5
CTRL+C        用户在 3 秒后按下 Ctrl+C
Slept for 3 of 5 secs.
linux>
```

8.5.4 阻塞和解除阻塞信号

Linux 提供了阻塞信号和解除阻塞信号的隐式机制和显式机制:

- **隐式阻塞机制**。 默认情况下,内核会阻塞任何处理程序当前正在处理的那种类型的待处理信号。例如,在图 8.31 中,假设程序已经捕获信号 s,并且当前正在运行处理程序 S。如果这时有另一个信号 s 发送给这个进程,那么 s 将成为待处理信号,但在处理程序 S 返回之前不会被接收。
- **显式阻塞机制**。 应用程序可以使用 sigprocmask 函数及其辅助函数,显式地阻塞和解除阻塞选定的信号。

```
#include <signal.h>

int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
        返回:成功返回 0;出错返回 -1

int sigismember(const sigset_t *set, int signum);
        返回:若是成员返回 1;若不是返回 0;出错返回 -1
```

sigprocmask 函数改变当前被阻塞信号的集合 (8.5.1 节中描述的 blocked 位向量)。具体的行为取决于 how 的值:

- SIG_BLOCK。把 set 中的信号加到 blocked 里(blocked = blocked | set)。
- SIG_UNBLOCK。从 blocked 中移除 set 中的信号(blocked = blocked & ~set)。
- SIG_SETMASK。blocked = set。

如果 oldset 不是 NULL, 那么 blocked 位向量先前的值就存放在 oldset 中。

诸如 set 这样的信号集是用下述函数来操作的：sigemptyset 把 set 初始化为空集；sigfillset 函数把每个信号都加到 set 中；sigaddset 函数把 signum 加到 set 中，sigdelset 从 set 中删除 signum；而 sigismember 在 signum 是 set 的成员时返回 1，否则返回 0。例如，图 8.32 展示了你会如何使用 sigprocmask 来暂时阻塞对 SIGINT 信号的接收。

```
1  sigset_t mask, prev_mask;
2
3  Sigemptyset(&mask);
4  Sigaddset(&mask, SIGINT);
5
6  /* 阻塞 SIGINT,并保存之前的被阻塞集合 */
7  Sigprocmask(SIG_BLOCK, &mask, &prev_mask);
     .
8  .  // 不会被 SIGINT 中断的代码区域
     .
9  /* 恢复之前的被阻塞集合,解除对 SIGINT 的阻塞 */
10 Sigprocmask(SIG_SETMASK, &prev_mask, NULL);
11
```

图 8.32　暂时阻塞一个信号使其不被接收。

8.5.5 编写信号处理程序

信号处理是 Linux 系统级编程中比较棘手的方面之一。处理程序有几个属性，使它们难以推断：(1) 处理程序与主程序并发运行，并共享同样的全局变量，因此可能干扰主程序和其他处理程序。(2) 关于信号如何以及在何种时机被接收的规则常常是反直觉的。(3) 不同的系统可能有不同的信号处理语义。

在本节中，我们来处理这些问题，并给出一些基本的指导原则，帮助你编写安全、正确、可移植的信号处理程序。

安全信号处理

信号处理程序之所以棘手，是因为它们能够与主程序以及彼此并发地运行，正如我们在图 8.31 中看到的那样。如果一个处理程序与主程序并发地访问同一个全局数据结构，那么结果可能是不可预测的，并且常常是灾难性的。

我们将在第 12 章中详细探讨并发编程。这里的目标是给出一些保守的指导原则，用来编写能够安全地并发运行的处理程序。如果你忽视这些原则，就有引入微妙并发错误的危险。有了这样的错误，你的程序在大多数时间都能正确运行。然而，一旦失败，失败的方式既不可预测，也不可重复，极难调试。有备无患！

- **G0. 保持处理程序尽可能简单。** 避免麻烦的最好办法是让处理程序尽可能小而简单。例如，处理程序可以只是设置一个全局标志并立即返回；与信

号接收有关的所有处理都由主程序执行，主程序周期性地检查（并重置）这个标志。

- **G1. 在处理程序中只调用异步信号安全的函数。** 一个函数若为异步信号安全（async-signal-safe）的——简称安全的——就具有这样的性质：可以从信号处理程序中安全地调用它。这或者是因为它是可重入的（例如，只访问局部变量；见 12.7.2 节），或者是因为它不能被信号处理程序中断。图 8.33 列出了 Linux 保证为安全的系统级函数。注意，许多流行的函数，比如 `printf`、`sprintf`、`malloc` 和 `exit`，都不在这张列表上。

从信号处理程序产生输出的唯一安全方法是使用 `write` 函数（见 10.1 节）。特别地，调用 `printf` 或 `sprintf` 是不安全的。为了绕过这个令人不快的限制，我们开发了一些安全函数，称为 Sio（Safe I/O）安全 I/O 包，你可以用它们从信号处理程序打印简单的消息。

```
#include "csapp.h"

ssize_t sio_putl(long v);
ssize_t sio_puts(char s[]);
        返回：成功则返回传输的字节数；出错则返回 -1

void sio_error(char s[]);
        返回：无
```

`sio_putl` 和 `sio_puts` 函数分别向标准输出发送一个 `long` 和一个字符串。`sio_error` 函数打印一条错误消息并终止。

图 8.34 展示了 Sio 包的实现，它用到了 `csapp.c` 中的两个私有的可重入函数。第 3 行的 `sio_strlen` 函数返回字符串 `s` 的长度。第 10 行的 `sio_ltoa` 函数基于 [61] 中的 `itoa` 函数，把 `v` 转换成它在 `s` 中以基数 `b` 表示的字符串。第 17 行的 `_exit` 函数是 `exit` 函数的一个异步信号安全的变体。

图 8.35 展示了图 8.30 中 `SIGINT` 处理程序的一个安全版本。

```
1  ssize_t sio_puts(char s[]) /* 发送字符串 */
2  {
3      return write(STDOUT_FILENO, s, sio_strlen(s));
4  }
5
6  ssize_t sio_putl(long v) /* 发送 long */
7  {
8      char s[128];
9
10     sio_ltoa(v, s, 10); /* 基于 K&R 的 itoa() */
11     return sio_puts(s);
12 }
13
14 void sio_error(char s[]) /* 发送错误消息并退出 */
```

图 8.33 异步信号安全的函数。(来源:man 7 signal。数据来自 Linux 基金会。)

_Exit	fexecve	poll	sigqueue
_exit	fork	posix_trace_event	sigset
abort	fstat	pselect	sigsuspend
accept	fstatat	raise	sleep
access	fsync	read	sockatmark
aio_error	ftruncate	readlink	socket
aio_return	futimens	readlinkat	socketpair
aio_suspend	getegid	recv	stat
alarm	geteuid	recvfrom	symlink
bind	getgid	recvmsg	symlinkat
cfgetispeed	getgroups	rename	tcdrain
cfgetospeed	getpeername	renameat	tcflow
cfsetispeed	getpgrp	rmdir	tcflush
cfsetospeed	getpid	select	tcgetattr
chdir	getppid	sem_post	tcgetpgrp
chmod	getsockname	send	tcsendbreak
chown	getsockopt	sendmsg	tcsetattr
clock_gettime	getuid	sendto	tcsetpgrp
close	kill	setgid	time
connect	link	setpgid	timer_getoverrun
creat	linkat	setsid	timer_gettime
dup	listen	setsockopt	timer_settime
dup2	lseek	setuid	times
execl	lstat	shutdown	umask
execle	mkdir	sigaction	uname
execv	mkdirat	sigaddset	unlink
execve	mkfifo	sigdelset	unlinkat
faccessat	mkfifoat	sigemptyset	utime
fchmod	mknod	sigfillset	utimensat
fchmodat	mknodat	sigismember	utimes
fchown	open	signal	wait
fchownat	openat	sigpause	waitpid
fcntl	pause	sigpending	write
fdatasync	pipe	sigprocmask	

```
15 {
16     sio_puts(s);
17     _exit(1);
18 }
```

图 8.34 用于信号处理程序的 Sio(Safe I/O)安全 I/O 包。

- **G2. 保存并恢复 errno。** 许多 Linux 异步信号安全的函数在带着错误返回时会设置 errno。在处理程序中调用这类函数,可能会干扰程序中其他依赖

errno 的部分。解决办法是：在进入处理程序时把 errno 保存在一个局部变量中，并在处理程序返回之前恢复它。注意，只有当处理程序要返回时才有必要这样做；如果处理程序通过调用 `_exit` 来终止进程，就没有必要了。

```
1  #include "csapp.h"
2
3  void sigint_handler(int sig) /* 安全的 SIGINT 处理程序 */
4  {
5      Sio_puts("Caught SIGINT!\n"); /* 安全的输出 */
6      _exit(0);                     /* 安全的退出 */
7  }
```

图 8.35　图 8.30 中 SIGINT 处理程序的安全版本。

- **G3. 通过阻塞所有信号来保护对共享全局数据结构的访问。** 如果一个处理程序与主程序或其他处理程序共享一个全局数据结构，那么你的处理程序和主程序就应该在访问（读或写）该数据结构时暂时阻塞所有信号。这条规则的理由是：从主程序访问一个数据结构 d 通常需要一串指令。如果这串指令被一个也要访问 d 的处理程序中断，那么那个处理程序就可能会发现 d 处于不一致的状态，后果不可预测。在访问 d 的同时暂时阻塞所有信号，就保证了处理程序不会中断这串指令。

- **G4. 用 volatile 声明全局变量。** 考虑这样一个处理程序和主例程，它们共享一个全局变量 g。处理程序更新 g，而 `main` 周期性地读取 g。对一个优化编译器来说，g 的值看起来在 `main` 中从不改变，因此，用缓存到寄存器中 g 的一个副本去满足对 g 的每次引用就是安全的。在这种情况下，`main` 函数就永远看不到处理程序更新的值。

  你可以用 `volatile` 类型限定符声明一个变量，来告诉编译器不要缓存它。例如：

```
volatile int g;
```

  `volatile` 限定符迫使编译器每次在代码中引用 g 时都从存储器读取 g 的值。总之，与任何共享的数据结构一样，对全局变量的每次访问都应该通过暂时阻塞信号来保护。

- **G5. 用 sig_atomic_t 声明标志。** 在一种常见的处理程序设计中，处理程序通过写一个全局标志来记录信号的接收。主程序周期性地读取该标志，对信号作出响应，然后清除该标志。对于以这种方式共享的标志，C 提供了一种整数数据类型 `sig_atomic_t`，对它的读和写可以保证是原子的（不可中断的），因为它们可以用一条指令来实现：

```
volatile sig_atomic_t flag;
```

由于这些操作不会被中断，你可以安全地读写 sig_atomic_t 变量，而不必暂时阻塞信号。注意，原子性保证只适用于单个的读和写；它并不适用于诸如 flag++ 或 flag = flag + 10 这样的更新，这些更新可能需要多条指令。

记住，我们给出的这些指导原则是保守的，也就是说，它们并不总是严格必需的。例如，如果你知道一个处理程序永远不修改 errno，那么就不需要保存并恢复 errno。或者，如果你能证明没有任何 printf 调用实例会被处理程序中断，那么在处理程序中调用 printf 就是安全的。对共享全局数据结构的访问同样如此。然而，要在一般情况下证明这样的断言是很困难的。所以我们建议你采取保守的做法，遵循这些原则：让处理程序尽可能简单，只调用安全函数，保存并恢复 errno，保护对共享数据结构的访问，并使用 volatile 和 sig_atomic_t。

正确的信号处理

信号有一个不直观的方面：待处理信号不排队等待。由于待处理位向量对每种信号类型恰好只包含一个位，所以一种特定类型的待处理信号至多只有一个。因此，如果有两个类型为 k 的信号被发送到一个目标进程，而此时信号 k 正被阻塞（因为目标进程当前正在执行信号 k 的一个处理程序），那么第二个信号就会被简单地丢弃，它不会排队等待。关键的思想是：一个待处理信号的存在仅仅表明至少有一个信号到达过。

为了看看它如何影响正确性，我们来看一个简单的应用，其性质类似于 shell 和 Web 服务器这样的真实程序。基本结构是这样的：一个父进程创建一些子进程，子进程独立运行一段时间后终止。父进程必须回收这些子进程，避免在系统中留下僵尸。但我们还希望父进程在孩子运行期间可以自由地做其他工作。于是我们决定用一个 SIGCHLD 处理程序来回收子进程，而不是显式地等待子进程终止。（回忆一下，每当有一个子进程终止或停止时，内核都会向父进程发送一个 SIGCHLD 信号。）

图 8.36 展示了我们的第一次尝试。父进程安装一个 SIGCHLD 处理程序，然后创建三个孩子。与此同时，父进程等待来自终端的一行输入，然后处理它。这个处理过程用一个无限循环来模拟。当每个子进程终止时，内核通过发送一个 SIGCHLD 信号来通知父进程。父进程捕获这个 SIGCHLD，回收一个孩子，

code/ecf/signal1.c

```
1  /* 警告:这段代码有缺陷!  */
2
3  void handler1(int sig)
4  {
5      int olderrno = errno;
6
```

```
7       if ((waitpid(-1, NULL, 0)) < 0)
8           sio_error("waitpid error");
9       Sio_puts("Handler reaped child\n");
10      Sleep(1);
11      errno = olderrno;
12  }
13
14  int main()
15  {
16      int i, n;
17      char buf[MAXBUF];
18
19      if (signal(SIGCHLD, handler1) == SIG_ERR)
20          unix_error("signal error");
21
22      /* 父进程创建子进程 */
23      for (i = 0; i < 3; i++) {
24          if (Fork() == 0) {
25              printf("Hello from child %d\n", (int)getpid());
26              exit(0);
27          }
28      }
29
30      /* 父进程等待终端输入,然后处理它 */
31      if ((n = read(STDIN_FILENO, buf, sizeof(buf))) < 0)
32          unix_error("read");
33
34      printf("Parent processing input\n");
35      while (1)
36          ;
37
38      exit(0);
39  }
```

图 8.36 `signal1` 程序。这个程序有缺陷,因为它假定信号会被排队。

做一点额外的清理工作(用 `sleep` 语句来模拟),然后返回。

图 8.36 中的 `signal1` 程序看起来相当简单直接。可是当我们在自己的 Linux 系统上运行它时,却得到了如下输出:

```
linux> ./signal1
Hello from child 14073
Hello from child 14074
Hello from child 14075
Handler reaped child
Handler reaped child
CR
Parent processing input
```

从输出中可以看到,虽然内核向父进程发送了三个 SIGCHLD 信号,但父进程只接收了其中两个,因此只收割了两个子进程。如果将父进程挂起,我们可以看到,子进程 14075 确实从未被收割,而是一直保持僵死状态(在 `ps` 命令的输出中以字符串 `<defunct>` 标示):

```
Ctrl+Z
Suspended
linux> ps t
  PID TTY          STAT   TIME COMMAND
  .
  .
  .
14072 pts/3        T      0:02 ./signal1
14075 pts/3        Z      0:00 [signal1] <defunct>
14076 pts/3        R+     0:00 ps t
```

出了什么问题？问题在于我们的代码没有考虑到信号不会排队这一事实。事件的实际经过是这样的：第一个信号被父进程接收并被处理程序捕获。当处理程序还在处理第一个信号时，第二个信号被传递下来，加入挂起信号的集合。但由于 SIGCHLD 信号已被 SIGCHLD 处理程序阻塞，所以没有接收这个信号。此后不久，当处理程序还在处理第一个信号时，第三个信号也到了。因为已经有一个挂起的 SIGCHLD，这第三个 SIGCHLD 信号就被直接丢弃了。稍后，处理程序返回之后，内核注意到有一个挂起的 SIGCHLD 信号，就强迫父进程接收它。父进程捕获这个信号并第二次执行处理程序。处理程序处理完第二个信号之后，就没有任何挂起的 SIGCHLD 信号了，以后也不会再有，因为关于第三个 SIGCHLD 的信息已经丢失。这里的关键教训是：不能用信号来对其他进程中事件的发生进行计数。

要修正这个问题，我们必须记起：挂起信号的存在只意味着，自该进程上次接收这一类型的信号以来，至少已经传递下来一个信号。因此，我们必须修改 SIGCHLD 处理程序，使它每次被调用时

code/ecf/signal2.c

```
1  void handler2(int sig)
2  {
3      int olderrno = errno;
4
5      while (waitpid(-1, NULL, 0) > 0) {
6          Sio_puts("Handler reaped child\n");
7      }
8      if (errno != ECHILD)
9          Sio_error("waitpid error");
10     Sleep(1);
11     errno = olderrno;
12 }
```

图 8.37 `signal2` 程序。图 8.36 的改进版本，正确处理了信号不排队这一事实。

都尽可能多地收割僵死的子进程。图 8.37 给出了修改后的 SIGCHLD 处理程序。

在 Linux 系统上运行 `signal2` 时，它现在能正确地收割所有僵死的子进程：

```
linux> ./signal2
Hello from child 15237
Hello from child 15238
Hello from child 15239
Handler reaped child
Handler reaped child
Handler reaped child
CR
Parent processing input
```

练习 8.8 下面这个程序的输出是什么？

code/ecf/signalprob0.c

```
1  volatile long counter = 2;
2
3  void handler1(int sig)
4  {
5      sigset_t mask, prev_mask;
6
7      Sigfillset(&mask);
8      Sigprocmask(SIG_BLOCK, &mask, &prev_mask);      /* 阻塞信号 */
9      Sio_putl(--counter);
10     Sigprocmask(SIG_SETMASK, &prev_mask, NULL);     /* 恢复信号 */
11
12     _exit(0);
13 }
14
15 int main()
16 {
17     pid_t pid;
18     sigset_t mask, prev_mask;
19
20     printf("%ld", counter);
21     fflush(stdout);
22
23     signal(SIGUSR1, handler1);
24     if ((pid = Fork()) == 0) {
25         while(1) {};
26     }
27     Kill(pid, SIGUSR1);
28     Waitpid(-1, NULL, 0);
29
30     Sigfillset(&mask);
31     Sigprocmask(SIG_BLOCK, &mask, &prev_mask); /* 阻塞信号 */
32     printf("%ld", ++counter);
33     Sigprocmask(SIG_SETMASK, &prev_mask, NULL); /* 恢复信号 */
34
35     exit(0);
36 }
```

可移植的信号处理

Unix 信号处理的另一个令人不快之处是，不同系统有着不同的信号处理语义。比如：

- `signal` 函数的语义各式各样。一些较老的 Unix 系统会在第 k 类信号被处理程序捕获之后，把信号 k 的处理动作恢复成默认动作。在这些系统上，处理程序每次运行时都必须通过调用 `signal` 显式地重新安装自己。
- 系统调用可能被中断。`read`、`wait` 和 `accept` 这类可能将进程长时间置于阻塞状态的系统调用称为慢速系统调用（slow system call）。在一些较老的 Unix 版本上，因处理程序捕获信号而被中断的慢速系统调用，在处理程序返回时并不恢复执行，而是带着出错条件立即返回给用户，并把 `errno` 置为 `EINTR`。在这些系统上，程序员必须编写代码，手动重启被中断的系统调用。

```
code/src/csapp.c

1  handler_t *Signal(int signum, handler_t *handler)
2  {
3      struct sigaction action, old_action;
4
5      action.sa_handler = handler;
6      sigemptyset(&action.sa_mask); /* 屏蔽正在被处理的这一类信号 */
7      action.sa_flags = SA_RESTART; /* 尽可能重启系统调用 */
8
9      if (sigaction(signum, &action, &old_action) < 0)
10         unix_error("Signal error");
11     return (old_action.sa_handler);
12 }
```

图 8.38 `Signal`。`sigaction` 的一个包装函数，为遵守 Posix 的系统提供可移植的信号处理。

为了解决这些问题，Posix 标准定义了 `sigaction` 函数，它允许用户在安装处理程序时清楚地指明自己想要的信号处理语义。

```
#include <signal.h>
int sigaction(int signum, struct sigaction *act,
              struct sigaction *oldact);

                                        Returns: 若成功则为 0,
                                        ↪  若出错则为 -1
```

`sigaction` 函数之所以难用，是因为它要求用户去设置一个复杂结构的各个字段。一个更干净的做法，最早由 W. Richard Stevens [110] 提出：定义一个包装函数，称为 `Signal`，由它替我们调用 `sigaction`。图 8.38 给出了 `Signal` 的定义，它以与 `signal` 函数相同的方式被调用。

Signal 包装函数安装的信号处理程序具有如下的信号处理语义：

- 只有处理程序当前正在处理的这一类型的信号才被阻塞。
- 与所有信号实现一样，信号不排队。
- 只要有可能，被中断的系统调用都自动重启。
- 信号处理程序一经安装，就保持安装状态，直到以 SIG_IGN 或 SIG_DFL 作为 handler 参数再次调用 Signal 为止。

在所有代码中都将使用 Signal 包装函数。

8.5.6 同步并发流，避免讨厌的并发错误

如何对读写同一存储位置的并发流编程，这个问题让一代又一代的计算机科学家费尽心机。一般而言，流可能出现的交错（interleaving）的数目是指令数量的指数函数。其中一些交错会产生正确的答案，另一些则不会。根本的任务在于，要以某种方式同步这些并发流，使得可行的交错集合尽可能大，而其中每个可行的交错都能产生正确的结果。

并发程序设计是一个深奥而重要的问题，我们将在第 12 章更详细地讨论。不过，你可以用本章学到的异常控制流知识来体会与并发相关的那些有趣的思想挑战。例如，考虑图 8.39 中的程序，它提炼出了典型 Unix shell 的结构。父进程用全局作业表（job list）中的表项来记录它当前的子进程，每个作业占一个表项。addjob 和 deletejob 函数分别向作业表中添加和删除表项。

父进程创建了新的子进程之后，就把这个子进程加入作业表。当父进程在 SIGCHLD 信号处理程序中收割一个已终止（僵死）的子进程时，就把它从作业表中删除。

乍一看，这段代码似乎是正确的。遗憾的是，下面的事件序列是可能发生的：

1. 父进程执行 fork 函数，内核调度新创建的子进程、而不是父进程去运行。
2. 在父进程能够再次运行之前，子进程终止并变成僵死进程，使内核向父进程传递一个 SIGCHLD 信号。
3. 稍后，当父进程重新变为可运行、但尚未开始执行时，内核注意到有挂起的 SIGCHLD，使它被父进程中运行的信号处理程序所接收。
4. 信号处理程序收割已终止的子进程并调用 deletejob，但 deletejob 什么都不做，因为父进程还没有把这个子进程加入作业表。
5. 处理程序完成之后，内核接着运行父进程，父进程从 fork 返回，并通过调用 addjob 把这个（已不存在的）子进程错误地加入作业表。

因此，在父进程主例程与信号处理控制流的某些交错方式下，deletejob 有可能在 addjob 之前被调用。结果会在作业表中留下一个不正确的表项，对应一个已经不存在、也永远不会被删除的作业。另一方面，也存在事件以正确顺序发生的交错。例如，如果内核恰好在 fork 调用返回时调度父进程而非子进程去运行，那么父进程就会在子进程终止之前正确地把它加入作业表，之后信号处理程序再把该作业从表中删除。

这是典型的同步错误的例子，称为竞争（race）。在这个例子中，竞争发生在主例程对 addjob 的调用与

code/ecf/procmask1.c

```
1  /* 警告:这段代码有缺陷!  */
2  void handler(int sig)
3  {
4      int olderrno = errno;
5      sigset_t mask_all, prev_all;
6      pid_t pid;
7
8      Sigfillset(&mask_all);
9      while ((pid = waitpid(-1, NULL, 0)) > 0) {  /* 收割一个僵死子进程
 ↪  */
10         Sigprocmask(SIG_BLOCK, &mask_all, &prev_all);
11         deletejob(pid); /* 把子进程从作业表中删除 */
12         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
13     }
14     if (errno != ECHILD)
15         Sio_error("waitpid error");
16     errno = olderrno;
17 }
18
19 int main(int argc, char **argv)
20 {
21     int pid;
22     sigset_t mask_all, prev_all;
23
24     Sigfillset(&mask_all);
25     Signal(SIGCHLD, handler);
26     initjobs(); /* 初始化作业表 */
27
28     while (1) {
29         if ((pid = Fork()) == 0) { /* 子进程 */
30             Execve("/bin/date", argv, NULL);
31         }
32         Sigprocmask(SIG_BLOCK, &mask_all, &prev_all); /* 父进程 */
33         addjob(pid); /* 把子进程加入作业表 */
34         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
35     }
36     exit(0);
37 }
```

图 8.39 带有一个微妙同步错误的 shell 程序。如果子进程在父进程还没来得及运行之前就终止,那么 `addjob` 和 `deletejob` 将以错误的顺序被调用。

处理程序对 `deletejob` 的调用之间。如果 `addjob` 在竞争中获胜,那么答案就是正确的;否则答案就是错误的。这类错误极其难以调试,因为通常不可能测试每一种交错。你可能运行代码十亿次都没出问题,但下一次测试就落在触发了竞争的那一种交错上。

图 8.40 给出了消除图 8.39 中竞争的一种方式。先在调用 `fork` 之前阻塞 SIGCHLD 信号,直到调用了 `addjob` 之后再解除阻塞,就保证了子进程一定先被加入作业表、之后才被收割。注意,子进程会继承父进程的被阻塞集合,所以必须小心,在子进程里要先解除对 SIGCHLD 信号的阻塞,然后再调用 `execve`。

8.5.7 显式地等待信号

有时,主程序需要显式地等待某个信号处理程序运行。例如,Linux shell 创建一个前台作业时,必须等待该作业终止、并被 SIGCHLD 处理程序收割之后,才能接受下一条用户命令。

图 8.41 展示了基本思路。父进程为 SIGINT 和 SIGCHLD 安装处理程序,然后进入一个无限循环。它阻塞 SIGCHLD,以避免我们在 8.5.6 节中讨论过的父进程与子进程之间的竞争。创建子进程之后,它把 pid 重置为零,解除对 SIGCHLD 的阻塞,然后在一个自旋循环(spin loop)中等待 pid 变为非零。子进程终止后,处理程序收割它,并把它的非零 PID 赋给全局变量 pid。这就终止了自旋循环,父进程在开始下一次迭代之前继续做些别的工作。

这段代码是正确的,但自旋循环浪费了处理器资源。我们可能会想在自旋循环体内插入一个 `pause` 来改善它:

```
while (!pid)        /* 有竞争!  */
    pause();
```

注意,我们仍然需要一个循环,因为 `pause` 可能会被一个或多个 SIGINT 信号的到来所中断。但是,这段代码有一个严重的竞争条件:如果 SIGCHLD 恰好在执行 `while` 测试之后、调用 `pause` 之前被收到,`pause` 将永远睡眠下去。

另一种选择是用 `sleep` 来替换 `pause`:

```
while (!pid) /* 太慢!  */
    sleep(1);
```

这段代码虽然正确,但太慢了。如果信号在 `while` 之后、`sleep` 之前被收到,程序就必须等上一段(相对)很长的时间,才能再次检查循环终止条件。用 `nanosleep` 这类分辨率更高的睡眠函数也不行,因为没有好的规则来确定睡眠间隔:设得太小,循环就太浪费;设得太大,程序就太慢。

code/ecf/procmask2.c

```
1  void handler(int sig)
2  {
3      int olderrno = errno;
4      sigset_t mask_all, prev_all;
5      pid_t pid;
6
7      Sigfillset(&mask_all);
8      while ((pid = waitpid(-1, NULL, 0)) > 0) {  /* 收割一个僵死子进程
 ↪  */
9          Sigprocmask(SIG_BLOCK, &mask_all, &prev_all);
10         deletejob(pid); /* 把子进程从作业表中删除 */
11         Sigprocmask(SIG_SETMASK, &prev_all, NULL);
12     }
13     if (errno != ECHILD)
14         Sio_error("waitpid error");
15     errno = olderrno;
16 }
17
18 int main(int argc, char **argv)
19 {
20     int pid;
21     sigset_t mask_all, mask_one, prev_one;
22
23     Sigfillset(&mask_all);
24     Sigemptyset(&mask_one);
25     Sigaddset(&mask_one, SIGCHLD);
26     Signal(SIGCHLD, handler);
27     initjobs(); /* 初始化作业表 */
28
29     while (1) {
30         Sigprocmask(SIG_BLOCK, &mask_one, &prev_one); /* 阻塞 SIGCHLD
        */
31         if ((pid = Fork()) == 0) { /* 子进程 */
32             Sigprocmask(SIG_SETMASK, &prev_one,
      NULL); /* 解除对 SIGCHLD 的阻塞 */
33             Execve("/bin/date", argv, NULL);
34         }
35         Sigprocmask(SIG_BLOCK, &mask_all, NULL); /* 父进程 */
36         addjob(pid); /* 把子进程加入作业表 */
37         Sigprocmask(SIG_SETMASK, &prev_one,
      NULL); /* 解除对 SIGCHLD 的阻塞 */
38     }
39     exit(0);
40 }
```

图 8.40 用 sigprocmask 同步进程。在这个例子中,父进程保证 addjob 在相应的 deletejob 之前执行。

code/ecf/waitforsignal.c

```
1  #include "csapp.h"
2
```

```
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* 阻塞 SIGCHLD */
27         if (Fork() == 0) /* 子进程 */
28             exit(0);
29
30         /* 父进程 */
31         pid = 0;
32         Sigprocmask(SIG_SETMASK, &prev, NULL); /* 解除对 SIGCHLD
 ↪  的阻塞 */
33
34         /* 等待收到 SIGCHLD(这是浪费的) */
35         while (!pid)
36             ;
37
38         /* 收到 SIGCHLD 之后做些别的工作 */
39         printf(".");
40     }
41     exit(0);
42 }
```

图 8.41 用自旋循环等待信号。这段代码是正确的,但自旋循环很浪费。正确的解决方案是使用 `sigsuspend`。

```
#include <signal.h>
int sigsuspend(const sigset_t *mask);

                                Returns: -1
```

`sigsuspend` 函数临时用 mask 替换当前的阻塞集合,然后挂起进程,直至收到一个信号,其行为是运行一个处理程序或终止进程。如果行为是终止,进程就终止,而不再从 `sigsuspend` 返回。如果行为是运行处理程序,则在处理

程序返回之后，sigsuspend 返回，并把阻塞集合恢复到调用 sigsuspend 之前的状态。

sigsuspend 函数等价于下述代码的一个原子(不可中断)版本：

```
1  sigprocmask(SIG_BLOCK, &mask, &prev);
2  pause();
3  sigprocmask(SIG_SETMASK, &prev, NULL);
```

原子性保证了 sigprocmask（第 1 行）和 pause（第 2 行）的调用是连接在一起完成的，不会被打断。这就消除了这样的潜在竞争：信号恰好在 sigprocmask 调用之后、pause 调用之前被收到。

图 8.42 展示了如何用 sigsuspend 替换图 8.41 中的自旋循环。每次调用 sigsuspend 之前，SIGCHLD 处于被阻塞状态。sigsuspend 临时解除对 SIGCHLD 的阻塞，然后睡眠，直到父进程捕获一个信号。在返回之前，它恢复原来的阻塞集合，SIGCHLD 再次被阻塞。如果父进程捕获到 SIGINT，那么循环测试成功，下一次迭代再调用 sigsuspend。如果父进程捕获到 SIGCHLD，那么循环测试失败，于是退出循环。此时，SIGCHLD 处于被阻塞状态，我们可以选择是否解除对它的阻塞。在一个有需要收割的后台作业的真实 shell 中，这样做可能是有用的。

sigsuspend 版本比原来的自旋循环浪费更少，避免了 pause 引入的竞争，也比 sleep 更高效。

8.6 非局部跳转

C 提供了一种用户级异常控制流的形式，称为非局部跳转(nonlocal jump)，它直接把控制从当前执行的一个函数转移到另一个正在执行的函数，而不用走正常的调用与返回的路径。非局部跳转由 setjmp 和 longjmp 函数提供。

code/ecf/sigsuspend.c

```
1  #include "csapp.h"
2
3  volatile sig_atomic_t pid;
4
5  void sigchld_handler(int s)
6  {
7      int olderrno = errno;
8      pid = Waitpid(-1, NULL, 0);
9      errno = olderrno;
10 }
11
12 void sigint_handler(int s)
13 {
14 }
```

```
15
16 int main(int argc, char **argv)
17 {
18     sigset_t mask, prev;
19
20     Signal(SIGCHLD, sigchld_handler);
21     Signal(SIGINT, sigint_handler);
22     Sigemptyset(&mask);
23     Sigaddset(&mask, SIGCHLD);
24
25     while (1) {
26         Sigprocmask(SIG_BLOCK, &mask, &prev); /* 阻塞 SIGCHLD */
27         if (Fork() == 0) /* 子进程 */
28             exit(0);
29
30         /* 等待收到 SIGCHLD */
31         pid = 0;
32         while (!pid)
33             sigsuspend(&prev);
34
35         /* 可选:解除对 SIGCHLD 的阻塞 */
36         Sigprocmask(SIG_SETMASK, &prev, NULL);
37
38         /* 收到 SIGCHLD 之后做些别的工作 */
39         printf(".");
40     }
41     exit(0);
42 }
```

图 8.42 用 sigsuspend 等待信号。

```
#include <setjmp.h>
int setjmp(jmp_buf env);
int sigsetjmp(sigjmp_buf env, int savesigs);

                    Returns: setjmp 返回时为 0,longjmp 返回时为非 0
```

setjmp 函数把当前的调用环境保存在缓冲区 env 中,供 longjmp 后续使用,并返回 0。调用环境包括程序计数器、栈指针和通用寄存器。出于一些超出本书讨论范围的微妙原因,setjmp 的返回值不应赋给一个变量:

```
rc = setjmp(env);       /* 错误!  */
```

不过,可以作为测试在 switch 或条件语句 [62] 中安全地使用它。

```
#include <setjmp.h>

void longjmp(jmp_buf env, int retval);
void siglongjmp(sigjmp_buf env, int retval);

                    Returns: 永不返回
```

longjmp 函数从 env 缓冲区中恢复调用环境，然后触发最近一次初始化了 env 的 setjmp 调用的返回。此时 setjmp 用非零的返回值 retval 返回。

setjmp 和 longjmp 的交互方式初看之下会令人困惑。setjmp 函数只被调用一次，却返回多次：一次是在最初调用 setjmp、调用环境被存入 env 缓冲区的时候；一次是在每次对应的 longjmp 调用时。另一方面，longjmp 函数只被调用一次，却从不返回。

非局部跳转的一个重要应用是，允许从一个深度嵌套的函数调用中立即返回，通常是在检测到某种错误条件之后。如果在一个嵌套函数调用的深处检测到错误条件，我们可以使用非局部跳转直接返回到一个公共的局部错误处理程序，而不必费力地展开调用栈。

图 8.43 展示了它如何工作。主例程首先调用 setjmp 保存当前的调用环境，然后调用函数 foo，而 foo 又调用函数 bar。如果 foo 或 bar 遇到一个错误，它们就通过调用 longjmp 从 setjmp 中立即返回。非零的 setjmp 返回值指示出了错误的类型，它随后可以在代码中的一个地方统一地被解码并处理。

longjmp 跳过所有中间调用的特性，可能会产生意料之外的效果。例如，如果一些数据结构是在中间的函数调用中分配的，打算在函数结束时释放它们，那么这些释放代码就会被跳过，从而造成内存泄漏。

code/ecf/setjmp.c

```
1  #include "csapp.h"
2
3  jmp_buf buf;
4
5  int error1 = 0;
6  int error2 = 1;
7
8  void foo(void), bar(void);
9
10 int main()
11 {
12     switch(setjmp(buf)) {
13     case 0:
14         foo();
15         break;
16     case 1:
17         printf("Detected an error1 condition in foo\n");
18         break;
19     case 2:
20         printf("Detected an error2 condition in foo\n");
21         break;
22     default:
23         printf("Unknown error condition in foo\n");
24     }
25     exit(0);
26 }
```

```
27
28 /* 深度嵌套的函数 foo */
29 void foo(void)
30 {
31     if (error1)
32         longjmp(buf, 1);
33     bar();
34 }
35
36 void bar(void)
37 {
38     if (error2)
39         longjmp(buf, 2);
40 }
```

图 8.43 非局部跳转的示例。这个例子展示了用非局部跳转,从深度嵌套的函数中恢复错误条件、而又无须展开整个调用栈的框架。

code/ecf/restart.c

```
1  #include "csapp.h"
2
3  sigjmp_buf buf;
4
5  void handler(int sig)
6  {
7      siglongjmp(buf, 1);
8  }
9
10 int main()
11 {
12     if (!sigsetjmp(buf, 1)) {
13         Signal(SIGINT, handler);
14         Sio_puts("starting\n");
15     }
16     else
17         Sio_puts("restarting\n");
18
19     while(1) {
20         Sleep(1);
21         Sio_puts("processing...\n");
22     }
23     exit(0); /* 控制永远不会到达这里 */
24 }
```

图 8.44 一个利用非局部跳转、在用户键入 Ctrl+C 时重启自身的程序。

非局部跳转的另一个应用,是允许控制从一个特定的信号处理程序处中跳出,转移到指定的代码位置,而不是返回给被信号中断而停止的那条指令。图 8.44 展示了一个简单的程序,说明了这个基本技术。该程序使用信号和非局部跳转,在用户键入 Ctrl+C 时执行一次软重启。`sigsetjmp` 和 `siglongjmp` 函数是 `setjmp` 和 `longjmp` 的版本,可以被信号处理程序使用。

程序首次启动时，对 sigsetjmp 函数的首次调用保存了调用环境和信号上下文（包括挂起和被阻塞的信号向量）。然后，主例程进入一个无限处理循环。当用户键入 Ctrl+C 时，内核向进程发送一个 SIGINT 信号，进程捕获这个信号。处理程序不是从信号处理程序返回（这会把控制交还给被中断的处理循环），而是执行一次非局部跳转，跳回主程序的开头。当我们在自己的系统上运行这个程序时，得到的输出如下：

> **边栏　C++ 和 Java 中的软件异常**
> C++ 和 Java 提供的异常机制，是 C 的 setjmp 和 longjmp 函数更高层次、结构更清晰的版本。可以把 try 语句内的 catch 子句看成与 setjmp 函数相当的东西。类似地，throw 语句相当于 longjmp 函数。

```
linux> ./restart
starting
processing...
processing...
Ctrl+C
restarting
processing...
Ctrl+C
restarting
processing...
```

关于这个程序，有几点值得注意。第一，为了避免竞争，我们必须在调用 sigsetjmp 之后才安装处理程序。否则，处理程序就有可能在首次调用 sigsetjmp、为 siglongjmp 建立起调用环境之前先运行的风险。第二，你可能已经注意到，sigsetjmp 和 siglongjmp 函数不在图 8.33 所列的异步信号安全函数的列表里。原因在于，一般而言 siglongjmp 可以跳转到任意的代码，所以我们必须小心，在从某条 siglongjmp 可达的任何代码中，只调用安全函数。在本例中，我们调用的是安全的 sio_puts 和 sleep 函数。不安全的 exit 函数是不可达的。

8.7 操作进程的工具

Linux 系统提供了许多有用的工具，用来监控和操作进程：

- **strace**。 打印一个正在运行的程序及其子进程所调用的每条系统调用的踪迹。对于一个好奇的学生来说，这是一个十分迷人的工具。编译你的程序时加上 -static 选项，就能得到更干净的踪迹，少掉一大堆与共享库相关的输出。
- **ps**。 列出当前系统中的进程（包括僵死进程）。

- **top**。 打印有关当前进程资源使用情况的信息。
- **pmap**。 显示一个进程的内存映射。
- **/proc**。 一个虚拟文件系统,它以用户程序能够读取的 ASCII 文本形式,导出大量内核数据结构的内容。例如,输入 `cat /proc/loadavg` 就能看到你的 Linux 系统当前的负载平均值。

8.8 小结

异常控制流(ECF)出现在计算机系统的所有层次上,是计算机系统提供并发性的基本机制。

在硬件层,异常是处理器中的事件所触发的控制流的急剧改变。控制流转入一个软件处理程序,它做一些处理,然后把控制返回给被中断的控制流。

异常有四种不同的类型:中断、故障、终止和陷阱。当定时器芯片或磁盘控制器这类外部 I/O 设备在处理器芯片上设置中断引脚时,就会产生中断(相对于任何指令而言,中断是异步发生的)。控制返回到引发异常的那条指令之后的指令。故障和终止是由执行一条指令而同步引发的。故障处理程序重启引发故障的指令,而终止处理程序从不把控制返回给被中断的控制流。最后,陷阱类似于函数调用,用来实现系统调用,而这些系统调用给应用程序提供了进入操作系统代码的受控入口点。

在操作系统层,内核用 ECF 提供了进程这一基本概念。进程向应用程序提供了两个重要的抽象:(1)逻辑控制流,它给每个程序一种它独占地使用处理器的假象;(2)私有地址空间,它给每个程序一种它独占地使用主存的假象。

在操作系统与应用程序的接口处,应用程序可以创建子进程,等待它的子进程停止或终止,运行新的程序,以及捕获来自其他进程的信号。信号处理的语义非常微妙,并且因其系统而异。不过,在遵守 Posix 的系统上,有一些机制允许程序清楚地指明所期望的信号处理语义。

最后,在应用程序层,C 程序可以用非局部跳转来绕开正常的调用/返回栈规则,直接从一个函数跳转到另一个函数。

8.9 参考文献说明

Kerrisk [62] 是 Linux 环境中编程的各个方面的必备参考书。Intel 的 ISA 规范详细讨论了 Intel 处理器上的异常和中断 [50]。操作系统教科书 [102, 106, 113] 包含关于异常、进程和信号的更多资料。W. Richard Stevens 的经典著作 [111] 是一份有价值且可读性极强的描述,讲述了应用程序如何与进程和信号

打交道。Bovet 和 Cesati [11] 对 Linux 内核给出了非常清晰的描述，包括进程和信号实现的细节。

8.10 家庭作业

家庭作业 8.9 ▲ 考虑四个具有如下开始时间和结束时间的进程：

进程	开始时间	结束时间
A	6	8
B	3	5
C	4	7
D	2	9

对每一对进程，指出它们是并发运行(Y)还是不并发运行(N)：

进程对	并发?
AB	
AC	
AD	
BC	
BD	
CD	

家庭作业 8.10 ▲ 在本章中，我们介绍了一些调用和返回行为不寻常的函数：`getenv`、`setenv`、`unsetenv` 和 `execve`。请把每个函数与下面的一种行为对应起来：

A. 只调用一次，只有出错时才返回。

B. 只调用一次，不返回任何值。

C. 只调用一次，返回一个指针或 `NULL`。

家庭作业 8.11 ▲ 这个程序会打印多少行“Example”输出？

code/ecf/global-forkprob1.c

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int i;
6
7      for (i = 3; i > 0; i--)
8          Fork();
9      printf("Example\n");
10     exit(0);
11 }
```

家庭作业 8.12 ▲ 这个程序会打印多少行“Example”输出？

code/ecf/global-forkprob4.c

```
1  #include "csapp.h"
2
3  void try()
4  {
5      Fork();
6      printf("Example\n");
7      Fork();
8      return;
9  }
10
11 int main()
12 {
13     try(); Fork();
14     printf("Example\n");
15     exit(0);
16 }
```

家庭作业 8.13 ▲ 这个程序的一种可能输出是什么?

code/ecf/global-forkprob3.c

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int a = 5;
6
7      if (Fork() != 0)
8          printf("a=%d\n", --a);
9
10     printf("a=%d\n", ++a);
11     exit(0);
12 }
```

家庭作业 8.14 ▲ 这个程序会打印多少行“Example”输出?

code/ecf/global-forkprob5.c

```
1  #include "csapp.h"
2
3  void try()
4  {
5      if (Fork() != 0) {
6          Fork();
7          printf("Example\n");
8          exit(0);
9      }
10     return;
11 }
12
13 int main()
14 {
15     try();
```

```
16     fork(); printf("Example\n");
17     exit(0);
18 }
```

家庭作业 8.15 ▲ 这个程序会打印多少行“Example”?

code/ecf/global-forkprob6.c

```
1  #include "csapp.h"
2
3  void try()
4  {
5      if (Fork() == 0) {
6          Fork(); Fork();
7          printf("Example\n");
8          return;
9      }
10     return;
11 }
12
13 int main()
14 {
15     try();
16     printf("Example\n");
17     exit(0);
18 }
```

家庭作业 8.16 ▲ 这个程序的输出是什么?

code/ecf/global-forkprob7.c

```
1  #include "csapp.h"
2  int counter = 1;
3
4  int main()
5  {
6      if (fork() == 0) {
7          counter++;
8          exit(0);
9      }
10     else {
11         Wait(NULL);
12         counter++; printf("counter = %d\n", counter);
13     }
14     exit(0);
15 }
```

家庭作业 8.17 ▲ 请枚举出练习 8.4 中程序的所有可能输出。

家庭作业 8.18 ▲▲ 考虑下面的程序:

code/ecf/forkprob2.c

```
1  #include "csapp.h"
```

```
2
3  void end(void)
4  {
5      printf("2"); fflush(stdout);
6  }
7
8  int main()
9  {
10     if (Fork() == 0)
11         atexit(end);
12     if (Fork() == 0) {
13         printf("0"); fflush(stdout);
14     }
15     else {
16         printf("1"); fflush(stdout);
17     }
18     exit(0);
19 }
```

请判断下列哪些输出是可能的。注：`atexit` 函数以一个函数指针为参数，并把它加到一个函数列表（初始为空）中，当调用 `exit` 函数时，这个列表里的函数就会被调用。

A. 112002

B. 211020

C. 102120

D. 122001

E. 100212

家庭作业 8.19 ▲▲ 如果用户输入的 n 值为 6，下面这个函数会打印多少行输出？

code/ecf/global-forkprob8.c

```
1  void foo(int n)
2  {
3      int i;
4
5      for (i = n - 1; i >= 0; i -= 2)
6          Fork();
7      printf("hello\n");
8      exit(0);
9  }
```

家庭作业 8.20 ▲▲ 请用 `execve` 编写一个名为 `myls` 的程序，它的行为与 `/bin/ls` 程序相同。你的程序应当接受相同的命令行参数，解释相同的环境变量，并产生相同的输出。

ls 程序从环境变量 COLUMNS 中获取屏幕的宽度。如果 COLUMNS 没有设置,那么 ls 就假定屏幕宽度为 80 列。这样,你就可以通过把 COLUMNS 环境变量设置为一个小于 80 的值,来检验你对环境变量的处理是否正确:

```
linux> setenv COLUMNS 40
linux> ./myls
  .
  . // 输出的宽度是 40 列
  .
linux> unsetenv COLUMNS
linux> ./myls
  .
  . // 输出的宽度现在是 80 列
  .
```

家庭作业 8.21 ▲▲ 下面这个程序可能出现的输出序列有哪些?

code/ecf/global-waitprob3.c

```
1  int main()
2  {
3      printf("p"); fflush(stdout);
4      if (fork() != 0) {
5          printf("q"); fflush(stdout);
6          return 0;
7      }
8      else {
9          printf("r"); fflush(stdout);
10         waitpid(-1, NULL, 0);
11     }
12     return 0;
13 }
```

家庭作业 8.22 ▲▲▲ 为 Unix system 函数编写你自己的版本:

```
int mysystem(char *command);
```

mysystem 函数通过调用 /bin/sh -c *command* 来执行 *command*, 然后在 *command* 完成之后返回。如果 *command* 正常退出(通过调用 exit 函数或执行一条 return 语句),那么 mysystem 就返回 *command* 的退出状态。例如,若 *command* 以调用 exit(8) 而终止, 那么 mysystem 就返回值 8。否则, 若 *command* 异常终止,mysystem 就返回 shell 返回的那个状态。

家庭作业 8.23 ▲▲ 你的一位同事正打算用信号来让父进程对子进程中发生的事件进行计数。他的想法是:每次发生一个事件时,就给父进程发送一个信号,让父进程的信号处理程序递增一个全局计数器变量;等子进程终止之后,父进程再来查看这个变量。可是,当他在他的系统上运行图 8.45 中的测试程序

时,他发现当父进程调用 `printf` 时,`counter` 的值总是 2,尽管子进程已经给父进程发送了五个信号。他百思不解,来找你求助。你能给他解释这个 bug 吗?

家庭作业 8.24 ▲▲▲ 请修改图 8.18 中的程序,使下面两个条件得到满足:

1. 每个子进程在试图写只读正文段中的一个位置之后,异常终止。
2. 父进程打印的输出与下面所示完全相同(PID 除外):

```
child 12255 terminated by signal 11: Segmentation fault
child 12254 terminated by signal 11: Segmentation fault
```

提示:请阅读 `psignal(3)` 的手册页。

code/ecf/counterprob.c

```
1  #include "csapp.h"
2
3  int counter = 0;
4
5  void handler(int sig)
6  {
7      counter++;
8      sleep(1); /* 在处理程序中做些工作 */
9      return;
10 }
11
12 int main()
13 {
14     int i;
15
16     Signal(SIGUSR2, handler);
17
18     if (Fork() == 0) { /* 子进程 */
19         for (i = 0; i < 5; i++) {
20             Kill(getppid(), SIGUSR2);
21             printf("sent SIGUSR2 to parent\n");
22         }
23         exit(0);
24     }
25
26     Wait(NULL);
27     printf("counter=%d\n", counter);
28     exit(0);
29 }
```

图 8.45 家庭作业 8.23 中引用的计数程序。

家庭作业 8.25 ▲▲▲ 请编写 `fgets` 函数的一个版本,叫作 `tfgets`,它在 5 秒之后超时。`tfgets` 函数接受与 `fgets` 相同的输入。如果用户在 5 秒内没有键入输入行,`tfgets` 就返回 `NULL`。否则,它返回一个指向输入行的指针。

家庭作业 8.26 ▲▲▲▲ 以图 8.23 中的示例为起点,编写一个支持作业控制的 shell 程序。你的 shell 应当具有如下特性:

- 用户键入的命令行由一个 *name* 和零个或多个参数组成，它们彼此之间由一个或多个空格分隔。如果 *name* 是一个内建命令，shell 就立即处理它，然后等待下一条命令行。
- 否则，shell 就假定 *name* 是一个可执行文件，把它加载进来，在一个初始子进程（作业）的上下文中运行它。这个作业的进程组 ID 与子进程的 PID 相同。
- 每个作业要么用一个进程 ID（PID）标识，要么用一个作业 ID（JID）标识，JID 是由 shell 分配的一个任意的小正整数。JID 在命令行上用前缀“%”来表示。例如，“%5”表示 JID 5，而“5”表示 PID 5。
- 如果命令行以 & 结尾，shell 就在后台运行这个作业；否则，shell 就在前台运行这个作业。
- 键入 Ctrl+C（Ctrl+Z）会使内核向你的 shell 发送一个 SIGINT（SIGTSTP）信号，shell 再把它转发给前台进程组中的每个进程。[2]
- `jobs` 内建命令列出所有后台作业。
- `bg` *job* 内建命令通过发送它一个 SIGCONT 信号来重启作业，然后在后台运行它。作业参数既可以是 PID 也可以是 JID。
- `fg` *job* 内建命令通过发送它一个 SIGCONT 信号来重启作业，然后在前台运行它。
- shell 收割它所有的僵死子进程。如果有作业因为收到一个未被捕获的信号而终止，shell 就向终端打印一条消息，包括这个作业的 PID 和对这个信号的描述。

图 8.46 展示了一个示例 shell 会话。

8.11 练习答案

答案 8.1 进程 A 与进程 B 彼此是并发的，进程 B 与进程 C 也是如此，因为它们各自的执行过程是重叠的——也就是说，一个进程在另一个进程结束之前就开始了。进程 A 与进程 C 则不是并发的，因为它们的执行不重叠：进程 A 在进程 C 开始之前就结束了。

答案 8.2 在图 8.15 的示例程序中，父进程和子进程执行的是互不交叠的指令集合。然而在这个程序里，父进程和子进程执行的指令集合是相互交叠的

[2] 注意，这是对真实 shell 工作方式的一种简化。在真实的 shell 中，内核响应 Ctrl+C（Ctrl+Z）的方式，是把 SIGINT（SIGTSTP）直接发送给终端前台进程组中的每个进程。shell 用 `tcsetpgrp` 函数来管理这个组的成员关系，用 `tcsetattr` 函数来管理终端的属性，这两者都超出了本书的范围。详见 [62]。

——这之所以可能,是因为父进程和子进程拥有完全相同的代码段。这可能是一个难以跨越的观念障碍,所以请务必弄懂这道题的答案。图 8.47 给出了进程图(process graph)。

```
linux> ./shell                                  运行你的 shell 程序
>bogus
bogus: Command not found.                       Execve 找不到可执行文件
>foo 10
Job 5035 terminated by signal: Interrupt        用户键入了 Ctrl+C
>foo 100 &
[1] 5036 foo 100 &
>foo 200 &
[2] 5037 foo 200 &
>jobs
[1] 5036 Running    foo 100 &
[2] 5037 Running    foo 200 &
>fg %1
Job [1] 5036 stopped by signal: Stopped         用户键入了 Ctrl+Z
>jobs
[1] 5036 Stopped    foo 100 &
[2] 5037 Running    foo 200 &
>bg 5035
5035: No such process
>bg 5036
[1] 5036 foo 100 &
>/bin/kill 5036
Job 5036 terminated by signal: Terminated
> fg %2                                         等待前台作业结束
>quit
linux>                                          回到 Unix shell
```

图 8.46 家庭作业 8.26 的 shell 会话示例。

```
Figure 8.47
Process graph for Practice
Problem 8.2.
fork
fork
main
main
printf
printf
exit
exit
p1: a=8
p2: a=9
Child
Parent
a==9
p2: a=10
printf
printf
printf
```

```
printf
exit
exit
```

A. 这里的关键在于子进程执行了两条 `printf` 语句。`fork` 返回之后，子进程执行第 6 行的 `printf`。然后它从 `if` 语句中跳出，执行第 7 行的 `printf`。子进程产生的输出如下：

```
p1: a=8
p2: a=9
```

B. 父进程只执行第 7 行的那条 `printf`：

```
p1: a=10
```

```
Figure 8.48
Process graph for Practice
Problem 8.3.
fork
fork
main
main
printf
printf
waitpid
waitpid
9
3
0
3
printf
printf
6
printf
printf
exit
exit
printf
printf
printf
printf
6
printf
printf
exit
exit
```

```
Figure 8.49
Process graph for Practice
Problem 8.4.
```

```
fork
fork
main
main
printf
printf
printf
printf
waitpid
waitpid
0
Stop
1
Start
2
printf
printf
Stop
printf
printf
exit
exit
printf
printf
printf
printf
Child
printf
printf
exit(2)
exit(2)
```

答案 8.3 我们知道，序列 936036、903636 和 093636 是可能的，因为它们对应于该进程图（图 8.48）的某个拓扑排序（topological sort）。但是，像 036936 和 360369 这样的序列不对应任何拓扑排序，因此是不可行的。

答案 8.4

A. 只要数一数进程图（图 8.49）中 `printf` 顶点的个数，就可以确定输出的行数。在这个例子里，这样的顶点有 7 个，因此程序将打印 7 行输出。

B. 对应于该图任何一个拓扑排序的输出序列都是可能的。例如：Start、0、1、Child、Stop、2、Stop 是可能的。

答案 8.5

code/ecf/global-snooze.c

```
1  unsigned int wakeup(unsigned int secs) {
2      unsigned int rc = sleep(secs);
3
4      printf("Woke up at %d secs.\n", secs-rc+1);
5      return rc;
6  }
```

答案 8.6

code/ecf/myecho.c

```
1  #include "csapp.h"
2
3  int main(int argc, char *argv[], char *envp[])
4  {
5      int i;
6
7      printf("Command-line arguments:\n");
8      for (i=0; argv[i] != NULL; i++)
9          printf("    argv[%2d]: %s\n", i, argv[i]);
10
11     printf("\n");
12     printf("Environment variables:\n");
13     for (i=0; envp[i] != NULL; i++)
14         printf("    envp[%2d]: %s\n", i, envp[i]);
15
16     exit(0);
17 }
```

答案 8.7 无论何时，只要睡眠中的进程收到一个没有被忽略的信号，sleep 函数就会提前返回。但既然收到 SIGINT 时的默认行为是终止进程（图 8.26），我们就必须安装一个 SIGINT 处理程序，好让 sleep 函数能够返回。处理程序只是捕获这个信号并返回；控制于是交还给 sleep 函数，后者立即返回。

code/ecf/snooze.c

```
1  #include "csapp.h"
2
3  /* SIGINT 处理程序 */
4  void handler(int sig)
5  {
6      return; /* 捕获信号并返回 */
7  }
8
9  unsigned int snooze(unsigned int secs) {
10     unsigned int rc = sleep(secs);
11
12     printf("Slept for %d of %d secs.\n", secs-rc, secs);
13     return rc;
14 }
15
16 int main(int argc, char **argv) {
17
18     if (argc != 2) {
19         fprintf(stderr, "usage: %s <secs>\n", argv[0]);
20         exit(0);
21     }
22
23     if (signal(SIGINT, handler) == SIG_ERR) /* 安装 SIGINT */
24         unix_error("signal error\n"); /* 处理程序 */
```

```
25     (void)snooze(atoi(argv[1]));
26     exit(0);
27 }
```

答案 8.8 这个程序会打印出字符串 213 ——卡内基梅隆大学 CS:APP 课程名称的缩写。在这个示例中，父进程先打印“2”，然后 fork 出一个子进程，子进程在一个无限循环中自旋。接着，父进程向子进程发送一个信号，并等待其终止。子进程捕获信号（这将中断无限循环），把计数器从初始值 2 开始递减，打印“1”，然后终止。父进程收割子进程之后，把计数器从初始值 2 开始递增，打印“3”，然后终止。

9

虚拟内存

一个系统中的各个进程都与其他进程共享 CPU 和主存。不过，共享主存会带来一些特殊的挑战。随着对 CPU 需求的增加，进程会以较为平缓的方式变慢；但是，如果有太多进程需要太多的内存，那么其中一些进程就根本没有办法运行。当一个程序没有空间可用时，它就只能自认倒霉。内存也容易遭到破坏。如果某个进程无意中写到了另一个进程使用的内存，那个进程就可能以一种与程序逻辑毫无关系、令人莫名其妙的方式失败。

为了更高效地管理内存并且少犯错误，现代系统提供了一种对主存的抽象，称为**虚拟内存**（**virtual memory**，**VM**）。虚拟内存是硬件异常、硬件地址翻译、主存、磁盘文件和内核软件的一次精妙协作，它为每个进程提供了一个大的、一致的、私有的地址空间。凭借这一个简洁的机制，虚拟内存提供了三项重要的能力：(1) 它把主存当作存储在磁盘上的地址空间的高速缓存来使用，在主存中只保留活动的区域，并根据需要在磁盘和内存之间来回传送数据，从而高效地使用主存。(2) 它为每个进程提供一致的地址空间，从而简化了内存管理。(3) 它保护每个进程的地址空间不被其他进程破坏。

虚拟内存是计算机系统中一个伟大的思想。它之所以能够成功，一个主要原因是它无声地、自动地工作，不需要应用程序员做任何干预。既然虚拟内存一直在幕后运转得这么好，程序员为什么还需要了解它呢？原因有好几个。

- **虚拟内存是核心的**。虚拟内存贯穿计算机系统的各个层面，在硬件异常、汇编器、链接器、加载器、共享对象、文件和进程的设计中都扮演着关键角色。理解了虚拟内存，你会更好地理解系统整体是如何工作的。
- **虚拟内存很强大**。虚拟内存赋予应用程序强大的能力：创建和销毁内存块、把内存块映射到磁盘文件的部分内容，与其他进程共享内存。例如，你知道吗？通过读写内存位置，你就能读取或修改磁盘文件的内容；不需要任何显式的复制，你就能把一个文件的内容加载到内存。理解了虚拟内存，你就能在自己的应用程序中驾驭这些强大的能力。

- **虚拟内存也是危险的**。应用程序每次引用变量、解引用指针，或者调用 `malloc` 这样的动态内存分配包，都在与虚拟内存打交道。如果虚拟内存使用不当，应用程序会遭受疑难深藏的、与内存相关的错误。例如，一个带有坏指针的程序可能立即崩溃，抛出“段错误”（segmentation fault）或“保护错误”（protection fault）；也可能先无声地运行几个小时再崩溃；最可怕的是，可能带着错误的结果一路运行到结束。理解虚拟内存，以及像 `malloc` 这样管理它的分配包，可以帮助你避开这些错误。

本章从两个角度来看虚拟内存。本章的前半部分讲述虚拟内存是如何工作的，后半部分讲述应用程序如何使用和管理虚拟内存。有一点避无可避：VM 是复杂的，章内的论述在有些地方也反映了这种复杂性。好消息是，只要把细节钻研清楚，你就能手工模拟一个小系统的虚拟内存机制，虚拟内存这个概念也就永远不再神秘了。

本章的后半部分建立在这一理解基础之上，展示如何在你的程序中使用和管理虚拟内存。你将学到如何通过显式的内存映射以及调用动态内存分配包（例如 `malloc` 包）来管理虚拟内存，还将了解到 C 程序中大量常见的与内存相关的错误，以及避开这些错误的方法。

9.1 物理和虚拟寻址

计算机系统的主存组织成一个由 M 个连续的、大小为一个字节的单元组成的数组。每个字节都有一个唯一的**物理地址（physical address，PA）**。第一个字节的地址为 0，下一个字节的地址为 1，再下一个为 2，依此类推。给定这种简单的组织方式，CPU 访问内存最自然的方式就是使用物理地址，我们把这种方式称为**物理寻址（physical addressing）**。图 9.1 展示了物理寻址的一个例子，其背景是一条读取从物理地址 4 开始的一个 4 字节字的加载指令。当 CPU 执行这条加载指令时，它会生成一个有效的物理地址，并通过内存总线把它传递给主存。主存取出从物理地址 4 开始的 4 字节字，并将它返回给 CPU，CPU 把它存储到一个寄存器中。

早期的 PC 使用物理寻址，数字信号处理器、嵌入式微控制器以及 Cray 超级计算机等系统至今仍然采用这种方式。不过，现代处理器使用一种称为**虚拟寻址（virtual addressing）**的寻址方式，如图 9.2 所示。

使用虚拟寻址时，CPU 通过生成一个**虚拟地址（virtual address，VA）**来访问主存，在把这个地址发送给主存之前，先把它转换成适当的物理地址。把虚拟地址转换成物理地址的任务称为**地址翻译（address translation）**。与异常处理一样，地址翻译也需要 CPU 硬件与操作系统之间的紧密协作。CPU 芯片上

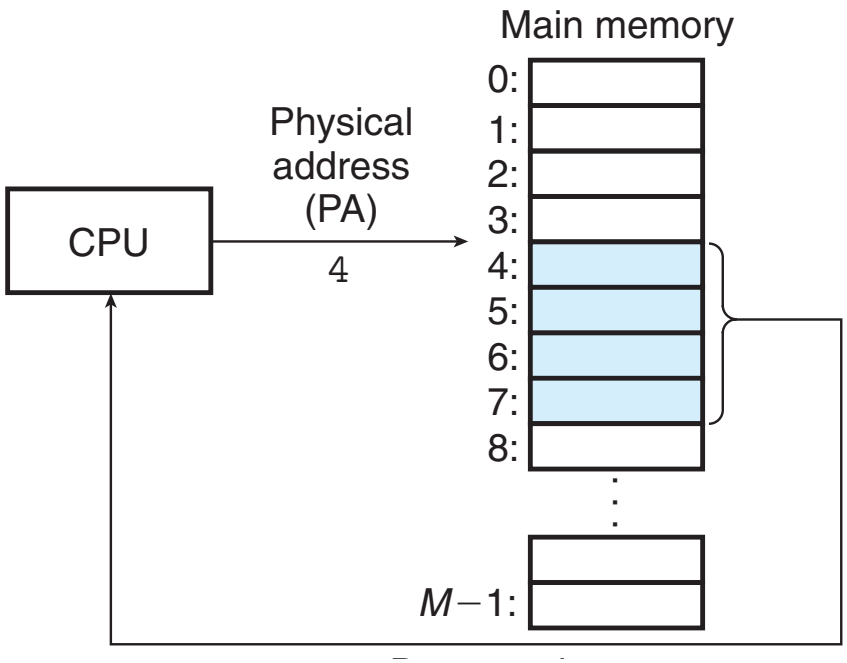


图 9.1 使用物理寻址的系统。

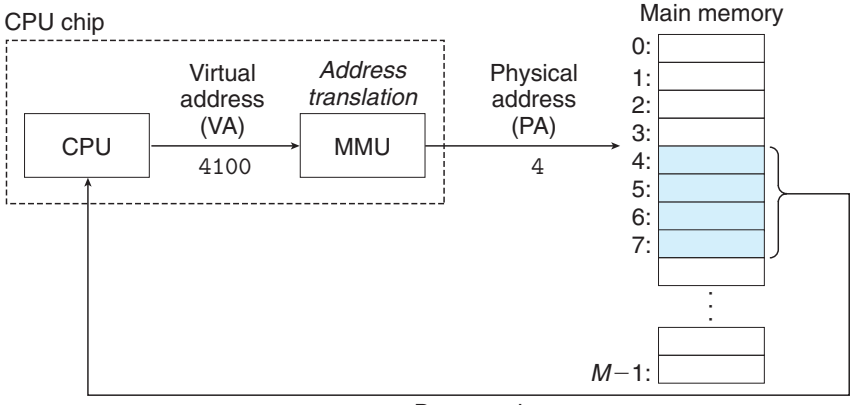


图 9.2 使用虚拟寻址的系统。

有一个专门的称为**内存管理单元**（**memory management unit**，**MMU**）的硬件，它利用存放在主存中的查询表来动态翻译虚拟地址，该表的内容由操作系统管理。

9.2 地址空间

地址空间是一个非负整数地址的有序集合

$$\{0, 1, 2, ...\}$$

如果地址空间中的整数是连续的，那么我们就说它是一个**线性地址空间（linear address space）**。为了简化讨论，我们始终假设地址空间是线性的。在支持虚拟内存的系统中，CPU 从一个含有 $N = 2^n$ 个地址的地址空间中生成虚拟地址，这个地址空间称为**虚拟地址空间（virtual address space）**：

$$\{0, 1, 2, \ldots, N-1\}$$

地址空间的大小由表示最大地址所需要的位数来刻画。例如，一个含有 $N = 2^n$ 个地址的虚拟地址空间称为一个 n 位地址空间。现代系统通常支持 32 位或 64 位虚拟地址空间。

系统还有一个物理地址空间，它对应于系统中物理内存的 M 个字节：

$$\{0, 1, 2, \ldots, M-1\}$$

M 不要求是 2 的幂，但为了简化讨论，我们假设 $M = 2^m$。

地址空间的概念很重要，因为它清楚地区分了数据对象（字节）和它们的属性（地址）。认识到这一区别后，我们就可以进一步推广这个概念，允许每个数据对象有多个独立的地址，每个地址选自不同的地址空间。这正是虚拟内存的基本思想。主存中的每个字节都有一个选自虚拟地址空间的虚拟地址，和一个选自物理地址空间的物理地址。

练习 9.1　补全下表：填入缺失的表项，并把每个问号替换成相应的整数。使用以下单位：K $= 2^{10}$（千）、M $= 2^{20}$（兆）、G $= 2^{30}$（吉）、T $= 2^{40}$（太）、P $= 2^{50}$（拍）或 E $= 2^{60}$（艾）。

练习 9.1 的表

虚拟地址位数 n	虚拟地址数 N	可能的最大虚拟地址
4		
	$2^? = 16\,\mathrm{K}$	
		$2^{24} - 1 = ?\,\mathrm{M} - 1$
	$2^? = 64\,\mathrm{T}$	
54		

9.3 虚拟内存作为缓存的工具

从概念上讲，虚拟内存被组织成一个存储在磁盘上的、包含 N 个连续字节的数组。每个字节都有一个唯一的虚拟地址作为数组的索引。磁盘上数组的内容被缓存到主存中。与存储器层次结构中的任何其他高速缓存一样，磁盘（下

层)上的数据被划分成若干块,这些块充当磁盘与主存(上层)之间的传输单元。VM 系统的做法是,把虚拟内存划分成称为**虚拟页**(**virtual page**,**VP**)的固定大小的块,每个虚拟页的大小为 $P = 2^p$ 字节。类似地,物理内存被划分成**物理页**(**physical page**,**PP**),大小也是 P 字节。(物理页也被称为**页帧**(**page frame**)。)

在任一时刻,虚拟页的集合划分成三个互不相交的子集:

- **未分配的**。VM 系统还没有分配(或创建)的页。未分配的块没有与之相关的任何数据,在磁盘上不占用任何空间。
- **缓存的**。当前已缓存到物理内存中的已分配页。
- **未缓存的**。没有缓存到物理内存中的已分配页。

图 9.3 中的例子展示了一个有 8 个虚拟页的小型虚拟内存。虚拟页 0 和 3 还没有被分配,因此还不存在于磁盘上。虚拟页 1、4 和 6 已缓存到物理内存中。页 2、5 和 7 已被分配,但当前没有缓存到物理内存中。

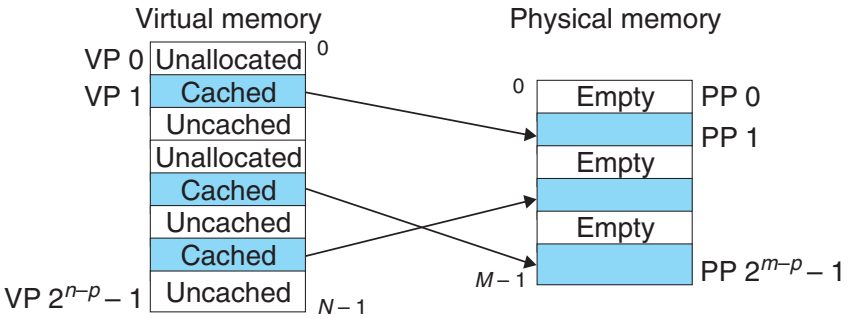


图 9.3　VM 系统如何将主存用作缓存。

9.3.1 DRAM 高速缓存的组织

为了帮助我们区分存储器层次结构中不同的高速缓存,我们用术语**SRAM 高速缓存**表示 CPU 与主存之间的 L1、L2 和 L3 高速缓存存储器,而用术语**DRAM 高速缓存**表示在主存中缓存虚拟页的 VM 系统的高速缓存。

DRAM 高速缓存在存储器层次结构中的位置对它的组织方式有很大影响。回想一下,DRAM 比 SRAM 慢至少 10 倍,而磁盘比 DRAM 慢大约 100,000 倍。因此,与 SRAM 缓存中的不命中相比,DRAM 缓存中的不命中是非常昂贵的,因为 DRAM 缓存的不命中由磁盘提供服务,而 SRAM 缓存的不命中通常由基于 DRAM 的主存提供服务。另外,从磁盘扇区读出第一个字节的时间,比读出该扇区中相继字节的时间,大约要慢 100,000 倍。一句话,DRAM 缓存的组织完全由不命中的巨大代价以及访问第一个字节的高昂开销所决定。

由于不命中的巨大代价，也由于访问第一个字节的高昂开销，虚拟内存页往往比较大——典型大小为 4 KB 到 2 MB。同样由于巨大的不命中代价，DRAM 缓存是**全相联**（**fully associative**）的；也就是说，任何虚拟页都可以放置在任何物理页中。不命中时的替换策略也变得更为重要，因为替换错一个虚拟页的代价非常高。因此，与硬件对 SRAM 缓存的做法相比，操作系统对 DRAM 缓存使用了精密得多的替换算法（这些替换算法超出了我们此处讨论的范围）。最后，由于磁盘的访问时间很长，DRAM 缓存总是使用**写回**（**write-back**），而不是**直写**（**write-through**）。

9.3.2 页表

与任何其他的高速缓存一样，VM 系统必须有某种方法来确定一个虚拟页是否已缓存到 DRAM 中的某个地方。如果是，系统必须确定它被缓存在哪个物理页中。如果出现不命中，系统必须确定虚拟页在磁盘上存放的位置，在物理内存中选择一个牺牲页，并把虚拟页从磁盘复制到 DRAM 中，替换掉那个牺牲页。

这些能力是通过操作系统软件、MMU 中的地址翻译硬件，以及存放在物理内存中的一种称为**页表**（**page table**）的数据结构的组合提供的，页表把虚拟页映射到物理页。地址翻译硬件每次把虚拟地址转换成物理地址时都要读取页表。操作系统负责维护页表的内容，并在磁盘和 DRAM 之间来回传送页。

图 9.4 展示了页表的基本组织。**页表**（**page table**）是一个由**页表项**（**page table entry**，**PTE**）组成的数组。虚拟地址空间中的每个页在页表中都有一个固定偏移处的 PTE。为了便于我们的讨论，我们假设每个 PTE 由一个**有效位**（**valid bit**）和一个 n 位地址字段组成。有效位表明虚拟页当前是否已缓存到 DRAM 中。如果设置了有效位，地址字段表示 DRAM 中缓存的物理页的起始地址。如果没有设置有效位，那么空地址表明虚拟页还未被分配；否则，地址字段指向虚拟页在磁盘上的起始位置。

图 9.4 中的例子展示了一个有 8 个虚拟页和 4 个物理页的系统的页表。4 个虚拟页（VP 1、VP 2、VP 4 和 VP 7）当前已缓存到 DRAM 中。两个页（VP 0 和 VP 5）还没有被分配，而其余两个（VP 3 和 VP 6）已被分配，但当前没有被缓存。关于图 9.4 要注意的一点是：因为 DRAM 缓存是全相联的，所以任何物理页都可能包含任何虚拟页。

练习 9.2　对于下面给出的虚拟地址大小（n）和页大小（P）的组合，确定所需**页表项**（**PTE**）的数目：

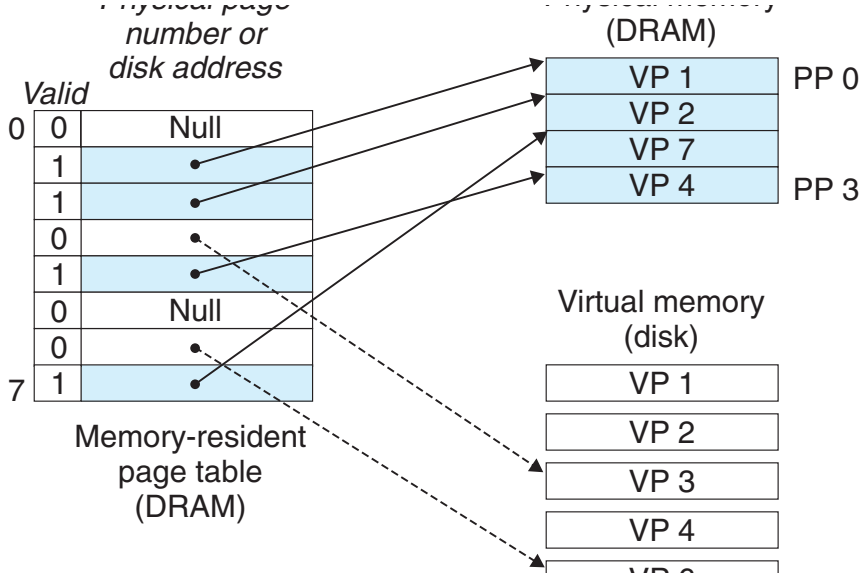


图 9.4　页表。

练习 9.2 的表

虚拟地址位数 n	页大小 $P = 2^p$	页表项数
12		
	1 K	
16	16 K	
24	2 M	
36	1 G	

9.3.3 页命中

考虑当 CPU 读取一个虚拟内存中的字(这个字包含在虚拟页 VP 2 中,而 VP 2 已缓存到 DRAM 中,见图 9.5)时会发生什么。利用我们将在 9.6 节详细描述的技术,地址翻译硬件以虚拟地址为一个索引,定位 PTE 2,并从内存中读出它。因为设置了有效位,地址翻译硬件知道 VP 2 已缓存到内存中。于是它使用 PTE 中的物理内存地址(此地址指向该页在 PP 1 中被缓存的起始处),构造出该字的物理地址。

9.3.4 缺页

在虚拟内存的行话中,DRAM 缓存的不命中称为**缺页**(**page fault**)。图 9.6 展示了缺页发生之前的示例页表的状态,CPU 引用了 VP 3 中的一个字,而 VP 3 没有缓存到 DRAM 中。地址翻译硬件从内存中读取 PTE 3,从有效位推断出 VP 3 未缓存,然后触发一个缺页异常。缺页异常会引起内核中的一个**缺页异常处理程序**(**page fault exception handler**),它会选择一个**牺牲页**(**victim page**)

——在本例中也就是存放在 PP 3 中的 VP 4。如果 VP 4 被修改过，那么内核就将它复制回磁盘。无论哪种情况，内核都会更新 VP 4 的页表项，以反映 VP 4 不再缓存到主存中这一事实。

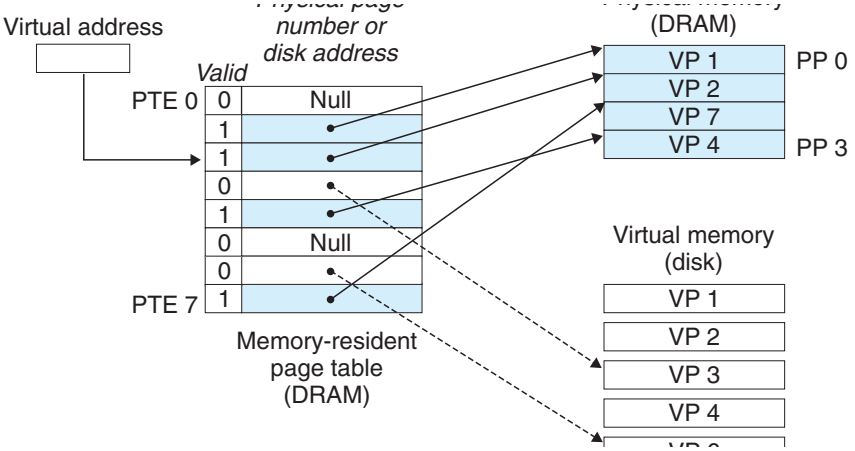


图 9.5　VM 页命中。对 VP 2 中某个字的引用是一次命中。

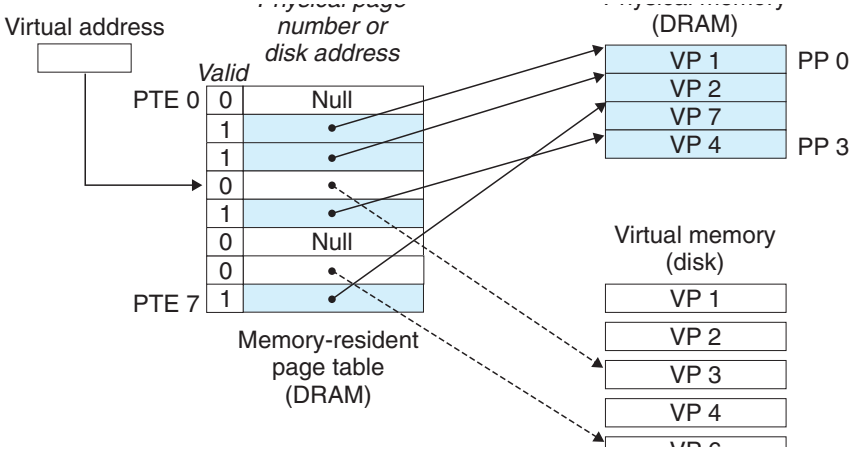


图 9.6　VM 缺页（之前）。对 VP 3 中某个字的引用是一次不命中，会触发一次缺页。

接下来，内核把 VP 3 从磁盘复制到内存中的 PP 3 上，更新 PTE 3，然后返回。当处理程序返回时，它会重启引发缺页的那条指令，该指令把缺页的虚拟地址重新发送给地址翻译硬件。不过这次，VP 3 已经缓存到主存中，页命中也由地址翻译硬件按正常的方式处理。图 9.7 展示了缺页之后示例页表的状态。

虚拟内存诞生于 20 世纪 60 年代初，远在由不断扩大 CPU-内存差距催生 SRAM 缓存之前。因此，即使许多思想都很相似，虚拟内存系统使用的术语却与 SRAM 缓存的不同：在虚拟内存的行话中，块被称为**页（page）**。在磁盘和内存之间来回传送页的活动被称为**交换（swapping）**或**分页（paging）**。页从磁盘**换**

VM page fault (before). The reference to a word in VP 3 is a miss and triggers a page fault.

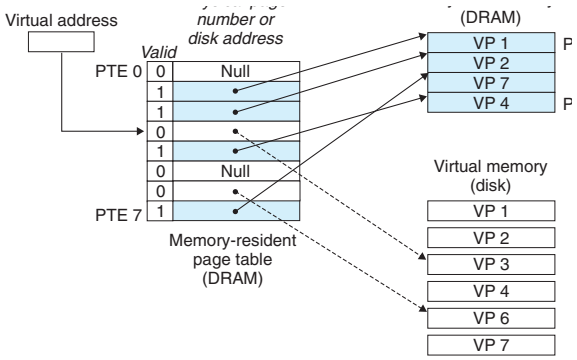


Figure 9.7
VM page fault (after). The page fault handler selects VP 4 as the victim and replaces it with a copy of VP 3 from disk. After the page fault handler restarts the faulting instruction, it will read the word from memory normally, without generating an exception.

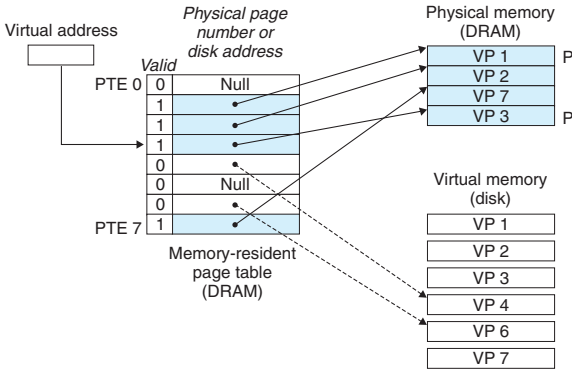


图 9.7

入(**paged in**,**swap in**)DRAM,或者从 DRAM **换出**(**paged out**,**swap out**)到磁盘上。不命中发生时等到最后一刻才换页的策略称为**按需调页**(**demand paging**)。也可以尝试其他方法,比如预测不命中并在页被实际引用之前先把页换入。不过,所有现代的系统都使用按需调页。

9.3.5 分配页面

图 9.8 展示了当操作系统分配一个新的虚拟内存页时,它对我们示例页表的影响——例如,作为调用 `malloc` 的结果。在本例中,VP 5 的分配方式是:在磁盘上创建空间并更新 PTE 5,使之指向磁盘上新创建的这个页。

9.3.6 局部性再次发挥威力

当我们中的许多人学习虚拟内存时,第一印象通常是它一定非常低效。考虑到巨大的不命中代价,我们担心换页会破坏程序的性能。而在实践中,虚拟内存工作得很好,其主要原因是我们的老朋友——局部性。

虽然程序在一个完整的运行过程中所引用的不同的页的总数可能超过物理内存的大小,但是局部性原理保证:在任一时刻,这些页都趋向于在一个较小

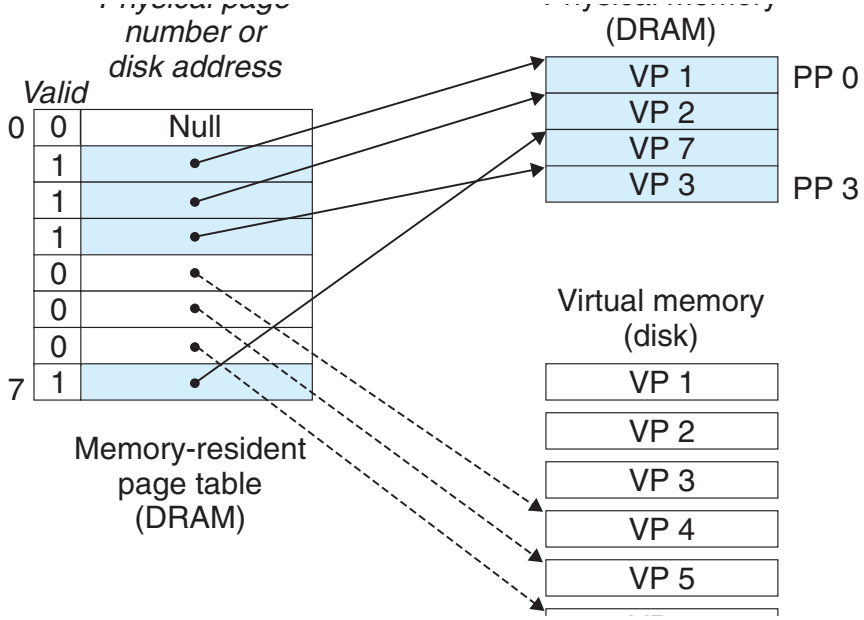


图 9.8　分配一个新的虚拟页。内核在磁盘上分配 VP 5，并将 PTE 5 指向这个新位置。

的活跃页集合上工作，这个集合称为**工作集**（**working set**）或**驻留集**（**resident set**）。在把工作集换入内存的初始开销之后，对工作集的后续引用都会命中，而不会带来额外的磁盘流量。

只要我们的程序有非常好的时间局部性，虚拟内存系统就能工作得很好。但是，当然并不是所有的程序都有很好的时间局部性。如果工作集的大小超过了物理内存的大小，那么程序就会陷入一种称为**抖动**（**thrashing**）的不幸处境，这时页会不断地在内存和磁盘之间来回换入换出。虽然虚拟内存通常是很高效的，但如果一个程序的性能降低到爬行的地步，明智的程序员就会考虑发生抖动的可能性。

> **边栏　计算缺页次数**
> 你可以使用 Linux 的 `getrusage` 函数来监控缺页的次数（还有大量的其他信息）。

9.4 虚拟内存作为内存管理的工具

在上一节中我们看到，虚拟内存提供了一种使用 DRAM 来缓存页的机制，这些页来自一个通常更大的虚拟地址空间。有趣的是，一些早期系统（例如 DEC PDP-11/70）支持的虚拟地址空间比可用的物理内存还要小。不过，虚拟内存仍然是一个有用的机制，因为它极大地简化了内存管理，并为内存保护提供了一种自然的方法。

到目前为止，我们都假设用一个单一的页表把一个单一的虚拟地址空间映射到物理地址空间。实际上，操作系统为每个进程提供了一个单独的页表，因而也就是一个单独的虚拟地址空间。图 9.9 展示了基本思想。在这个例子中，进程 i 的页表把 VP 1 映射到 PP 2，把 VP 2 映射到 PP 7。类似地，进程 j 的页表把 VP 1 映射到 PP 7，把 VP 2 映射到 PP 10。注意，多个虚拟页可以映射到同一个共享的物理页。

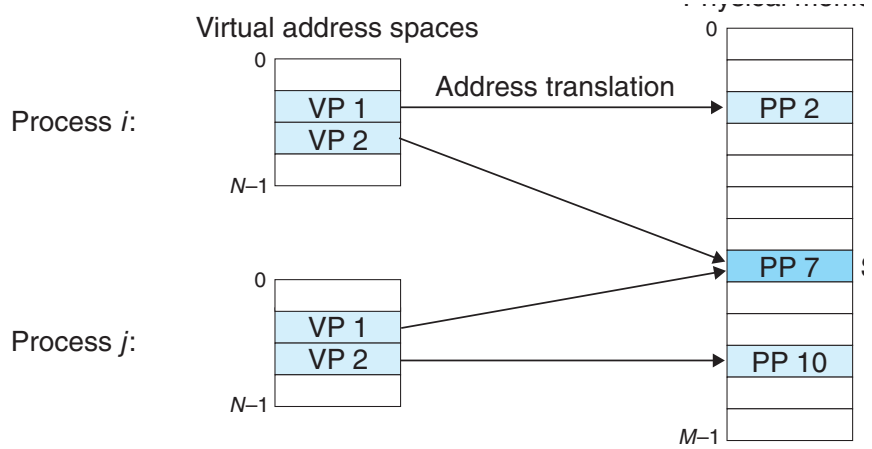


图 9.9　VM 如何为进程提供独立的地址空间。操作系统为系统中的每个进程维护一个单独的页表。

按需调页和分离的虚拟地址空间的结合，对系统中内存使用和管理的方式有深远的影响。特别地，虚拟内存简化了链接和加载、代码和数据的共享，以及向应用程序分配内存。

- **简化链接**。独立的地址空间允许每个进程的内存映像都使用相同的基本格式，而不必考虑代码和数据实际存放在物理内存中的位置。例如，正如我们在图 8.13 中所看到的，一个给定的 Linux 系统上的每个进程都有相似的内存格式。对于 64 位地址空间，代码段总是从虚拟地址 `0x400000` 开始，数据段跟随在代码段后面（中间隔着合适的对齐空隙），栈占据用户进程地址空间的最高部分，并且向下增长。这样的统一性极大简化了链接器的设计和实现，允许链接器产生完全链接的可执行文件，这些文件与代码和数据在物理内存中的最终位置无关。
- **简化加载**。虚拟内存也使把可执行文件和共享对象文件加载到内存的工作变得容易。要把目标文件的 `.text` 和 `.data` 段加载到一个新创建的进程中，Linux 加载器为代码段和数据段分配虚拟页，把这些页标记为无效的（即，未缓存的），并把它们的页表项指向目标文件中相应的位置。有趣的一点是，加载器实际上从不把任何数据从磁盘复制到内存。数据是自动

地、按需被换入(paged in)的:CPU 取一条指令,或者一条正在执行的指令引用某个内存位置,虚拟内存系统就会在第一次引用某个页时进行这样的处理。

把一组连续的虚拟页映射到任意一个文件中的任意位置,这个概念称为**内存映射(memory mapping)**。Linux 提供了一个称为 `mmap` 的系统调用,它允许应用程序做自己的内存映射。我们将在 9.8 节更详细地讨论应用级的内存映射。

- **简化共享**。独立的地址空间为操作系统提供了一致的机制,用来管理用户进程和操作系统本身之间的共享。一般而言,每个进程都有自己的私有代码、数据、堆和栈区域,这些区域不与任何其他进程共享。在这种情况下,操作系统创建页表,把相应的虚拟页映射到不相交的物理页。

  不过,在某些情况下,进程之间共享代码和数据是值得的。例如,每个进程都必须调用相同的操作系统内核代码,而且每个 C 程序都会调用标准 C 库中的例程,例如 `printf`。与其让每个进程都单独包含内核和标准 C 库的副本,操作系统不如安排多个进程共享这份代码的一个副本,做法是把不同进程中的相应的虚拟页映射到相同的物理页上,正如我们在图 9.9 中所看到的。

- **简化内存分配**。虚拟内存提供了一种向用户进程分配额外内存的简单机制。当运行在用户进程中的程序请求额外的堆空间时(例如,作为调用 `malloc` 的结果),操作系统会分配适当数目(比如 k 个)的连续虚拟内存页,并把它们映射到位于物理内存中任意位置的 k 个任意物理页上。因为页表的工作方式,操作系统不需要定位 k 个连续的物理内存页,这些页可以随机地散布在物理内存中。

9.5 虚拟内存作为内存保护的工具

任何现代计算机系统都必须提供一种手段,使操作系统能够控制对内存系统的访问。不应该允许用户进程修改它的只读代码段。也不应该允许它读取或修改内核中的任何代码和数据结构。它不应该允许读或写其他进程的私有内存,也不允许修改和其他进程共享的虚拟页,除非所有相关的进程都明确允许这种访问(这必须通过显式地调用进程间通信的系统调用来实现)。

正如我们所见到的,提供分离的虚拟地址空间使隔离不同进程的私有内存变得容易。但地址翻译机制可以以一种自然的方式加以扩展,提供更精细的访问控制。因为地址翻译硬件在 CPU 每次生成地址时都要读取一个 PTE,所以

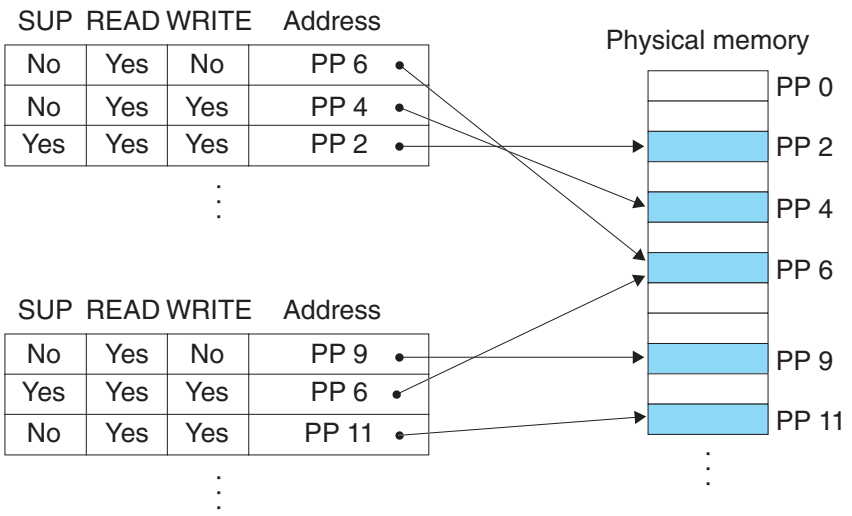


图 9.10 用 VM 提供页级内存保护。

通过给 PTE 添加一些额外的许可位来控制对虚拟页内容的访问就很简单。图 9.10 展示了其基本思想。

在本例中，我们给每个 PTE 添加了三个许可位。**SUP 位**表示进程是否必须运行在内核（管理，supervisor）态下才能访问该页。运行在内核态的进程可以访问任何页，而运行在用户态的进程只允许访问那些 SUP 位为 0 的页。**READ** 和 **WRITE** 位控制对页的读和写访问。例如，如果进程 i 运行在用户态，那么它有权限读 VP 0，有权限读或写 VP 1，但不允许访问 VP 2。

如果一条指令违反了这些许可权限，那么 CPU 就会触发一个**一般保护错误（general protection fault）**，该错误把控制传递给内核中的一个异常处理程序，它向引发错误的进程发送 `SIGSEGV` 信号。Linux shell 通常把这种异常报告为段错误（segmentation fault）。

9.6 地址翻译

本节介绍地址翻译的基础。我们的目的是让你认识到硬件在支持虚拟内存方面所起的作用，并给出足够的细节，使你能手工推演出一些具体的例子。不过，请记住，我们省略了一些对硬件设计者来说很重要但超出本书范围的细节。为了便于参考，图 9.11 概括了本节从头到尾使用的符号。

从形式上说，地址翻译是一个 N 元素的**虚拟地址空间（VAS）**和 M 元素的**物理地址空间（PAS）**之间的映射，

$$\text{MAP} : \text{VAS} \rightarrow \text{PAS} \cup \varnothing$$

图 9.11 地址翻译符号一览

基本参数	
$N = 2^n$	虚拟地址空间中的地址数
$M = 2^m$	物理地址空间中的地址数
$P = 2^p$	页大小(字节)
虚拟地址(VA)的组成	
VPO	虚拟页偏移量(字节)
VPN	虚拟页号
TLBI	TLB 索引
TLBT	TLB 标记
物理地址(PA)的组成	
PPO	物理页偏移量(字节)
PPN	物理页号
CO	高速缓存块内的字节偏移量
CI	高速缓存索引
CT	高速缓存标记

其中

$$\text{MAP}(A) = \begin{cases} A' & \text{若虚拟地址}A\text{ 的数据存在于 PAS 中的物理地址}A'\text{ 处} \\ \varnothing & \text{若虚拟地址}A\text{ 的数据不在物理内存中} \end{cases}$$

图 9.12 展示了 MMU 如何利用页表来执行这种映射。CPU 中的一个控制寄存器,**页表基寄存器(page table base register,PTBR)**指向当前页表。n 位的虚拟地址有两个组成部分:一个 p 位的**虚拟页偏移量(virtual page offset, VPO)**和一个 $(n-p)$ 位的**虚拟页号(virtual page number, VPN)**。MMU 利用 VPN 来选择相应的 PTE。例如,VPN 0 选择 PTE 0,VPN 1 选择 PTE 1,以此类推。对应的物理地址就是页表项中的**物理页号(physical page number, PPN)**与虚拟地址中的 VPO 串联而成的。注意,因为物理页和虚拟页的大小都为 P 字节,所以**物理页偏移量(physical page offset,PPO)**与 VPO 是一样的。

图 9.13(a) 展示了发生页命中时 CPU 硬件所经过的步骤。

步骤 1 处理器产生一个虚拟地址,并把它发送给 MMU。

步骤 2 MMU 产生 PTE 地址,并向高速缓存/主存请求它。

步骤 3 高速缓存/主存把 PTE 返回给 MMU。

步骤 4 MMU 构造出物理地址,并把它发送给高速缓存/主存。

步骤 5 高速缓存/主存把请求的数据字返回给处理器。

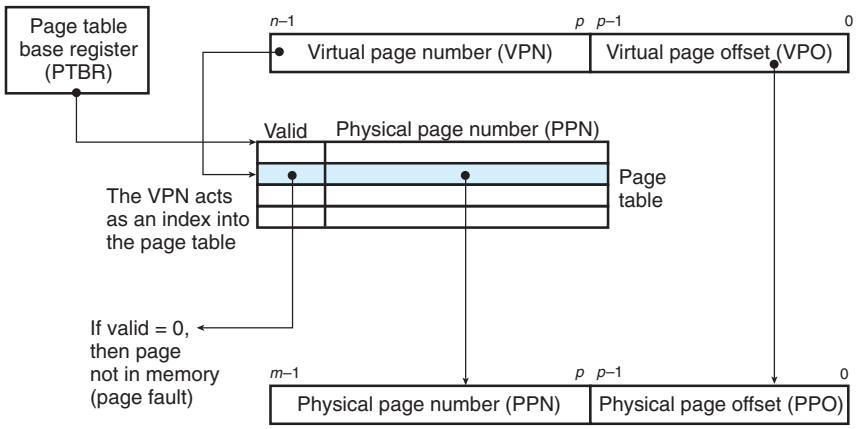


图 9.12 使用页表的地址翻译。

与完全由硬件处理的页命中不同，处理缺页需要硬件和操作系统内核相互协作（图 9.13(b)）。

步骤 1 到 3 与图 9.13(a) 中的步骤 1 到 3 相同。

步骤 4 PTE 中的有效位是 0，因此 MMU 触发一个异常，该异常把 CPU 的控制转到操作系统内核中的一个缺页异常处理程序。

步骤 5 缺页处理程序确定物理内存中的一个牺牲页，如果那个页已被修改过，就把它换出到磁盘。

步骤 6 缺页处理程序换入新页，并更新内存中的 PTE。

步骤 7 缺页处理程序返回到原来的进程，导致引发缺页的那条指令被重启。CPU 把引起缺页的虚拟地址重新发送给 MMU。因为虚拟页现在已缓存到物理内存中，所以会有一次命中，在 MMU 执行了图 9.13(a) 中的步骤之后，主存把所请求的字返回给处理器。

练习 9.3 给定一个 64 位虚拟地址空间和一个 32 位物理地址，对于下面的页大小（P），确定 VPN、VPO、PPN 和 PPO 的位数：

练习 9.3 的表

页大小 P	VPN 位数	VPO 位数	PPN 位数	PPO 位数
1 KB				
2 KB				
4 KB				
16 KB				

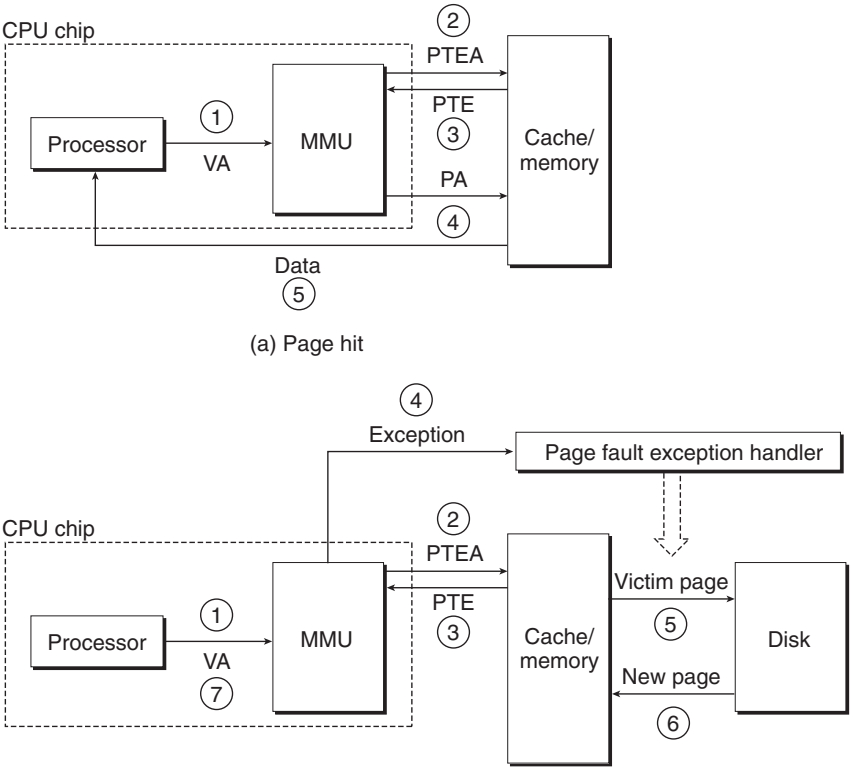


图 9.13 页面命中和缺页的操作视图。VA：虚拟地址；PTEA：页表项地址；PTE：页表项；PA：物理地址。

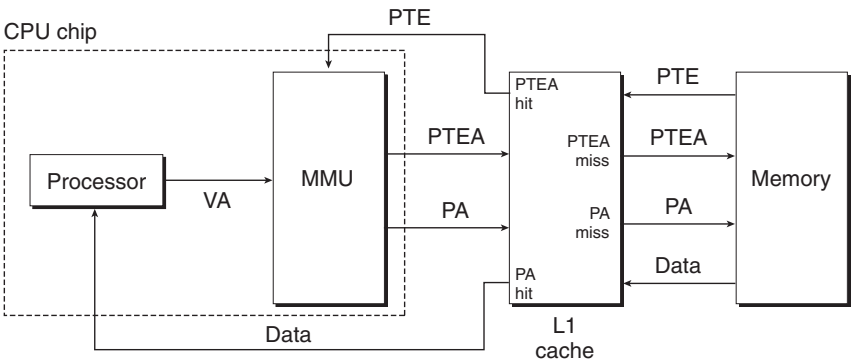


图 9.14 将 VM 与使用物理寻址的高速缓存整合起来。VA：虚拟地址；PTEA：页表项地址；PTE：页表项；PA：物理地址。

9.6.1 结合高速缓存与虚拟内存

在任何既使用了虚拟内存又使用了 SRAM 高速缓存的系统中，都有一个是否使用虚拟地址或物理地址来访问 SRAM 高速缓存的问题。虽然对这个折中问题的详细讨论超出了本书的范围，但大多数系统选择了物理寻址。对于物理寻址，多个进程的块同时放在缓存中，以及共享来自同一虚拟页的块都很容易。而且，因为访问权限的检查作为地址翻译过程的一部分，所以高速缓存不必处理保护问题。

图 9.14 展示了物理寻址的高速缓存如何与虚拟内存结合。主要的思想是在查找高速缓存之前先进行地址翻译。注意页表项可以像其他数据字一样被缓存。

9.6.2 使用 TLB 加速地址翻译

正如我们所看到的，每当 CPU 生成虚拟地址时，MMU 都必须参考一个 PTE，从而将虚拟地址翻译成物理地址。最糟糕的情况下，它需要从内存中获取额外的页表项，代价可能需要几十到几百个周期。如果 PTE 恰好被缓存到 L1 中，那么开销就会下降到几个周期。不过，许多系统试图消除这个开销，方法是在 MMU 中包括一个关于 PTE 的小的高速缓存，称为**转换后备缓冲区**（**translation lookaside buffer**，**TLB**）。

TLB 是一个小的、虚拟寻址的高速缓存，其中每一行都保存一个由单个 PTE 组成的块。TLB 通常具有高度的相联度。如图 9.15 所示，用于组选择和行匹配的索引和标记字段是从虚拟地址中的虚拟页号中提取出来的。如果 TLB 有 $T = 2^t$ 个组，那么 **TLB 索引**（**TLBI**）是由 VPN 中 t 个最低有效位组成的，而 **TLB 标记**（**TLBT**）是由 VPN 中剩余的位组成的。

图 9.16(a) 展示了发生 TLB 命中时（通常的情况）所涉及的步骤。这里的关键点在于所有地址翻译步骤都在片内的 MMU 中执行，所以速度很快。

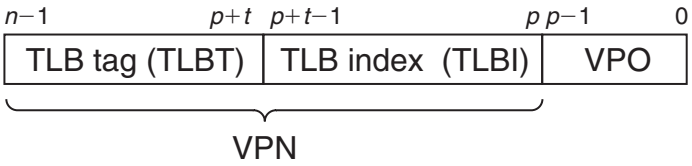
步骤 1 CPU 产生一个虚拟地址。

步骤 2 和 3 MMU 从 TLB 中取出相应的 PTE。

步骤 4 MMU 把虚拟地址翻译成一个物理地址，并把它发送给高速缓存/主存。

步骤 5 高速缓存/主存把所请求的数据字返回给 CPU。

当发生 TLB 不命中时，MMU 必须从 L1 缓存中取出相应的 PTE，如图 9.16(b) 所示。新取出的 PTE 被存放到 TLB 中，有可能覆盖一个已经存在的条目。



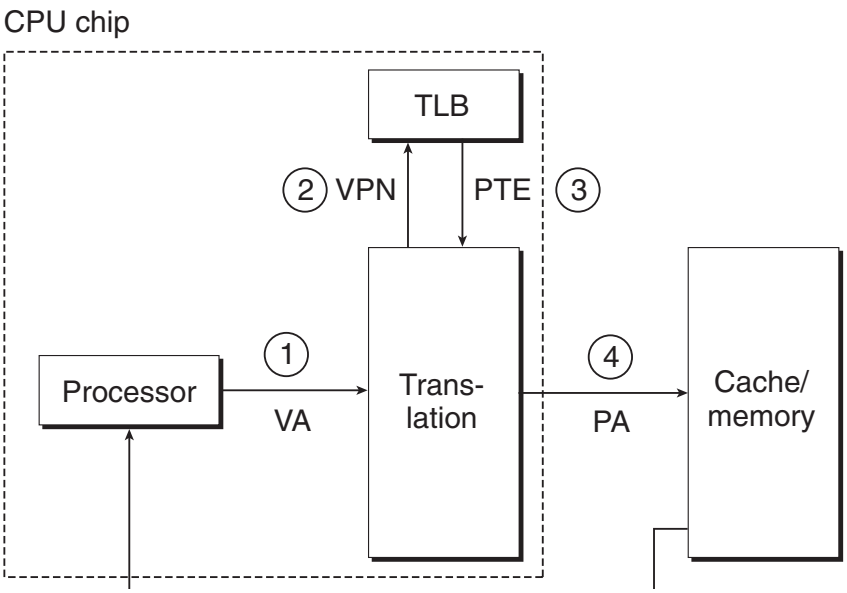


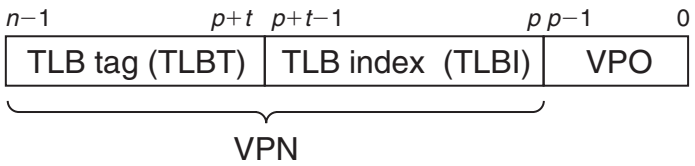
图 9.15 用于访问 TLB 的虚拟地址的组成部分。

9.6.3 多级页表

到目前为止,我们都假设系统使用一个单一的页表来做地址翻译。但是如果我们有一个 32 位地址空间、4 KB 大小的页和一个 4 字节的 PTE,那么即使应用引用了虚拟地址空间中很小的一部分,我们还是会需要一个 4 MB 的页表一直驻留在内存中。对于 64 位地址空间的系统来说,问题会更复杂。

压缩页表的常用方法是使用一个页表层次结构。用一个具体的例子来说明这个思想是最容易理解的。考虑一个 32 位虚拟地址空间,划分成 4 KB 大小的页,每个页表项是 4 字节。同时假设在当前时刻,虚拟地址空间具有如下形式:内存最前面 2K 个页已经分配给代码和数据,接下来的 6K 个页未分配,再接下来的 1,023 个页也未分配,再下一个页分配给用户栈。图 9.17 展示了如何为这个虚拟地址空间构造一个两级的页表结构。

第 1 级页表中的每个 PTE 负责映射虚拟地址空间中的一个 4 MB 的块,每个块由 1,024 个连续的页组成。例如,PTE 0 映射第一个块,PTE 1 映射下一个



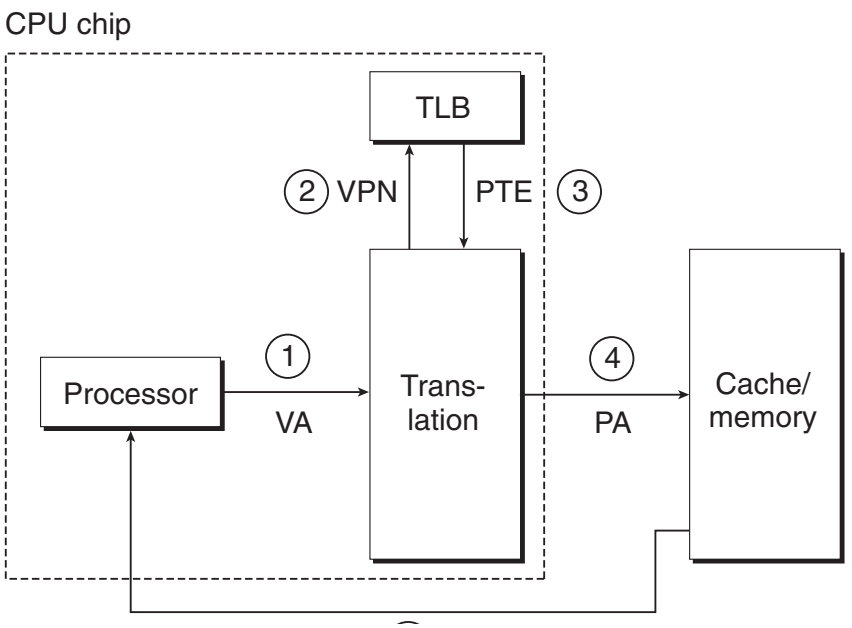


图 9.16 TLB 命中和不命中的操作视图。

块，依此类推。给定一个 4 GB 的地址空间，1,024 个 PTE 就已足够覆盖整个地址空间。

如果块 i 中的每个页都未被分配，那么第 1 级中的 PTE i 就是空的。例如，在图 9.17 中，块 2~7 都是未分配的。但是，如果块 i 中至少有一个页被分配了，那么第 1 级中的 PTE i 就指向第 2 级的页表的基址。例如，在图 9.17 中，块 0、1 和 8 的全部或部分都被分配了，因此它们的第 1 级 PTE 指向第 2 级的页表。

第 2 级页表中的每个 PTE 负责映射虚拟内存中的一个 4 KB 大小的页，这与我们观察单级页表时一样。注意，采用 4 字节的 PTE 时，每个第 1 级和第 2 级页表都是 4 KB 大小，正好等于一个页的大小。

这种方案从两个方面减少了内存的使用。第一，如果第 1 级页表中的一个 PTE 是空的，那么相应的第 2 级页表就根本没有必要存在。这起到了巨大的潜在节省作用，因为对一个典型程序来说，大多数 4 GB 的虚拟地址空间都是未分配的。第二，只有第 1 级的表需要一直驻留在主存中。VM 系统可以在需要

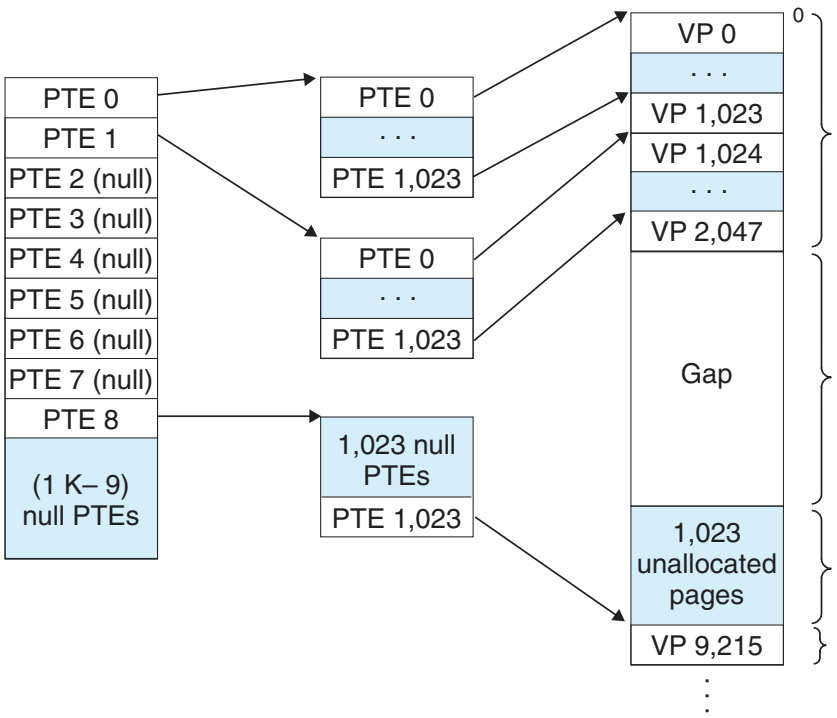


Figure 9.17 A two-level page table hierarchy. Notice that top to bottom.

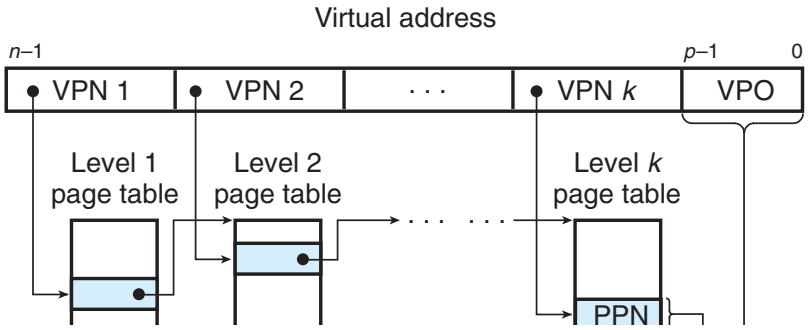


图 9.17 两级页表层次结构。注意地址从上往下增大。

时创建第 2 级页表，并把它们换入或换出内存，这就减轻了主存的压力。只有那些最频繁使用的第 2 级页表才需要缓存到主存中。图 9.18 概括了采用 k 级页表层次结构的地址翻译。虚拟地址被划分成 k 个 VPN 和一个 VPO。每个 VPN $i(1 \leq i \leq k)$都是一个指向第 i 级页表的索引。第 j 级页表中的每个 PTE $(1 \leq j \leq k-1)$ 都指向第 $j+1$ 级页表中的一个基址。第 k 级页表中的每个 PTE 或者包含某个物理页的 PPN，或者包含某个磁盘块的地址。为了构造出物理地址，MMU 必须访问 k 个 PTE 才能确定 PPN。与单级页表层次结构一样，PPO 与 VPO 相同。

访问 k 个 PTE 看上去很昂贵，也很不经济。然而，TLB 能在这里发挥作用，它把来自不同层级页表中的 PTE 缓存起来。在实际使用中，采用多级页表进行地址翻译并不比采用单级页表进行地址翻译明显慢。

9.6.4 综合应用：端到端的地址翻译

在这一节中，我们通过一个带有 TLB 和 L1 d-cache 的小系统上的端到端地址翻译例子，把所有内容综合到一起。为了使内容可控，我们做以下假设：

- 存储器按字节编址。
- 对存储器的访问都是针对 1 字节（而不是 4 字节）的字进行的。
- 虚拟地址是 14 位宽的（$n = 14$）。
- 物理地址是 12 位宽的（$m = 12$）。
- 页大小是 64 字节（$P = 64$）。
- TLB 是 4 路组相联的，总共有 16 个条目。
- L1 d-cache 是物理寻址、直接映射的高速缓存，行大小为 4 字节，总共有 16 个组。

图 9.19 展示了虚拟地址和物理地址的格式。因为每个页为 $2^6 = 64$ 字节，所以虚拟地址和物理地址的低 6 位分别作为 VPO 和 PPO。虚拟地址的高 8 位作为 VPN。物理地址的高 6 位作为 PPN。

图 9.20 展示了我们的小型内存系统的一个快照，包括 TLB（图 9.20(a)）、页表的一部分（图 9.20(b)）和 L1 高速缓存（图 9.20(c)）。在 TLB 和缓存的图示上方，我们也展示了当硬件访问这些设备时，它是如何划分虚拟地址和物理地址的位段的。

TLB。TLB 用 VPN 中的各位作为虚拟地址来寻址。因为 TLB 有 4 个组，所以 VPN 的 2 个最低有效位作为组索引（TLBI）。其余的 6 个最高有效位作为标记（TLBT），用来区分可能映射到同一个 TLB 组中的各个 VPN。

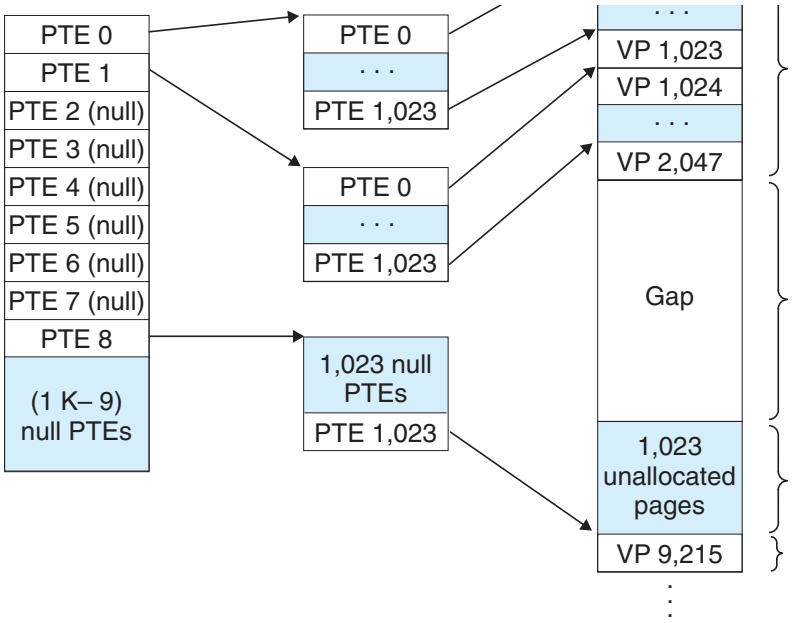


Figure 9.17 **A two-level page table hierarchy.** Notice that top to bottom.

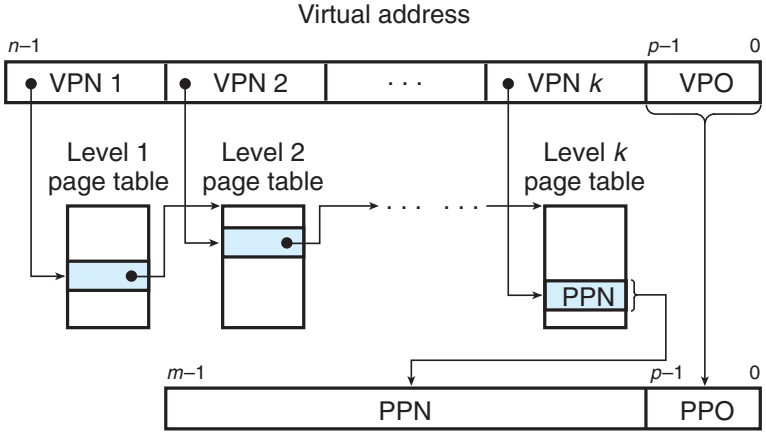


图 9.18 使用 k 级页表的地址翻译。

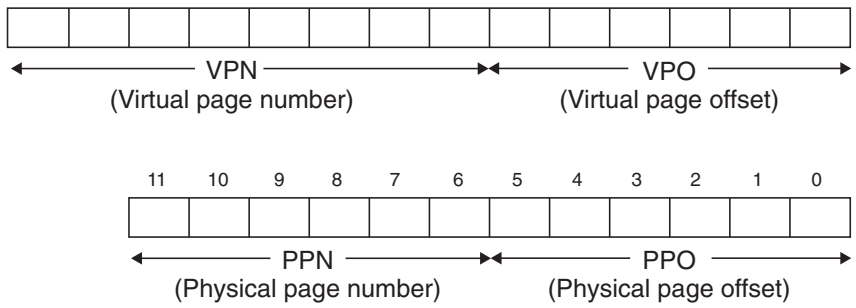


图 9.19　小内存系统的寻址。假设 14 位虚拟地址(n = 14)、12 位物理地址(m = 12)和 64 字节的页(P = 64)。

页表。页表是一个单级设计,共有 $2^8 = 256$ 个页表项(PTE)。不过我们只对其中前 16 个感兴趣。为方便起见,我们用索引它的 VPN 来标记每个 PTE;但请记住,这些 VPN 不是页表的一部分,也不存储在内存中。另外,请注意,每个无效 PTE 的 PPN 都用一根短划线来表示,以强调无论那里碰巧存储着什么样的位值,这些值都是没有意义的。

高速缓存。直接映射的高速缓存由物理地址中的字段来寻址。因为每个块是 4 字节,所以物理地址的低 2 位作为块偏移(CO)。因为有 16 个组,接下来的 4 位作为组索引(CI)。其余 6 位作为标记(CT)。

在这个初始设置的基础上,现在来看看当 CPU 执行一条加载指令、读取地址 `0x03d4` 处的字节时会发生什么。(回忆一下,我们假想的 CPU 读的是 1 字节的字,而不是 4 字节的字。)开始这种手工模拟时,把虚拟地址中的各位写下来,确定我们要用到的各个字段,并求出它们的十六进制值,这对我们是很有帮助的。硬件在对地址进行译码时也做着类似的工作。

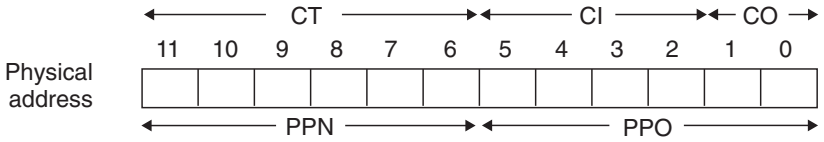
	TLBT						TLBI							
	0x03						0x03							
位位置	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VA = 0x03d4	0	0	0	0	1	1	1	1	0	1	0	1	0	0
	VPN								VPO					
	0x0f								0x14					

首先,MMU 从虚拟地址中取出 VPN(`0x0F`),并向 TLB 查询,看它是否已经缓存了来自此前某次内存引用的 PTE `0x0F` 的一个副本。TLB 从 VPN 中提取出 TLB 索引(`0x03`)和 TLB 标记(`0x3`),在 exttt0x3 组的第二个条目上命中一个有效匹配,于是把缓存的 PPN(`0x0D`)返回给 MMU。

00	28	1
01	—	0
02	33	1
03	02	1
04	—	0
05	16	1
06	—	0
07	—	0

08	13	1
09	17	1
0A	09	1
0B	—	0
0C	—	0
0D	2D	1
0E	11	1
0F	0D	1

(b) Page table: Only the first 16 PTEs are shown



Idx	Tag	Valid	Blk 0	Blk 1	Blk 2	Blk 3
0	19	1	99	11	23	11
1	15	0	—	—	—	—
2	1B	1	00	02	04	08
3	36	0	—	—	—	—
4	32	1	43	6D	8F	09
5	0D	1	36	72	F0	1D
6	31	0	—	—	—	—
7	16	1	11	C2	DF	03
8	24	1	3A	00	51	89
9	2D	0	—	—	—	—
A	2D	1	93	15	DA	3B
B	0B	0	—	—	—	—
C	12	0	—	—	—	—
D	16	1	04	96	34	15
E	13	1	83	77	1B	D3
F	14	0	—	—	—	—

图 9.20 小内存系统的 TLB、页表和高速缓存。TLB、页表和高速缓存中所有的值都以十六进制表示。

如果 TLB 不命中,那么 MMU 就需要从主存中取出 PTE。不过在这个例子中,我们运气很好,发生的是 TLB 命中。MMU 现在已经拥有构造物理地址所需的全部信息了。它把来自 PTE 的 PPN(0x0D)与来自虚拟地址的 VPO(0x14)串联起来,就形成了物理地址(0x354)。

接下来,MMU 把物理地址发送给高速缓存,高速缓存从物理地址中提取出缓存偏移 CO(0x0)、缓存组索引 CI(0x5)和缓存标记 CT(0x0D)。

	CT						CI				CO	
	0x0d						0x05				0x0	
位位置	11	10	9	8	7	6	5	4	3	2	1	0
PA = 0x354	0	0	1	1	0	1	0	1	0	1	0	0
	PPN						PPO					
	0x0d						0x14					

由于 0x5 组中的标记与 CT 相匹配,高速缓存检测到一次命中,读出偏移量 CO 处的数据字节(0x36),并把它返回给 MMU,MMU 再把它传回给 CPU。

翻译过程还可能经由其他路径。例如,如果 TLB 不命中,那么 MMU 就必须从页表中的一个 PTE 取出 PPN。如果得到的 PTE 是无效的,那么就会发生一次缺页,内核必须把相应的页换入内存,然后重新运行这条加载指令。还有一种可能:PTE 是有效的,但是所需的内存块在高速缓存中不命中。

练习 9.4　说明 9.6.4 节中的示例内存系统是如何把一个虚拟地址翻译成一个物理地址并访问高速缓存的。对于给定的虚拟地址,指出所访问的 TLB 条目、物理地址以及返回的高速缓存字节值。指出 TLB 是否不命中、是否发生缺页以及高速缓存是否不命中。如果发生一次高速缓存不命中,就在“返回的高速缓存字节”一栏填“—”。如果发生缺页,就在“PPN”一栏填“—”,并将 C 和 D 两部分留空。

虚拟地址:0x03d7

A.　虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B.　地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中?(Y/N)	
缺页?(Y/N)	
PPN	

C. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

D. 物理内存引用

参数	值
字节偏移	
缓存索引	
缓存标记	
缓存命中?(Y/N)	
返回的缓存字节	

9.7 案例研究：Intel Core i7/Linux 内存系统

我们用一个真实系统的案例研究来结束对虚拟内存机制的讨论：一个运行 Linux 的 Intel Core i7。虽然底层的 Haswell 微体系结构允许完整的 64 位虚拟和物理地址空间，但是目前的 Core i7 实现（以及可以预见的一段时间内的实现）支持 48 位（256 TB）的虚拟地址空间和 52 位（4 PB）的物理地址空间，还带有一个支持 32 位（4 GB）虚拟和物理地址空间的兼容模式。

图 9.21 给出了 Core i7 内存系统的要点。处理器封装（芯片）包括四个核、一个由所有核共享的大的 L3 高速缓存，以及一个 DDR3 内存控制器。每个核包含一个 TLB 层次结构、一个由数据高速缓存和指令高速缓存构成的层次结构，以及一组基于 QuickPath 技术的快速点对点链路，用于与其他核和外部 I/O 桥直接通信。TLB 是虚拟寻址的，并且是 4 路组相联的。L1、L2 和 L3 高速缓存是物理寻址的，块大小为 64 字节。L1 和 L2 是 8 路组相联的，L3 是 16 路组相联的。页的大小可以在启动时配置为 4 KB 或 4 MB。Linux 使用 4 KB 的页。

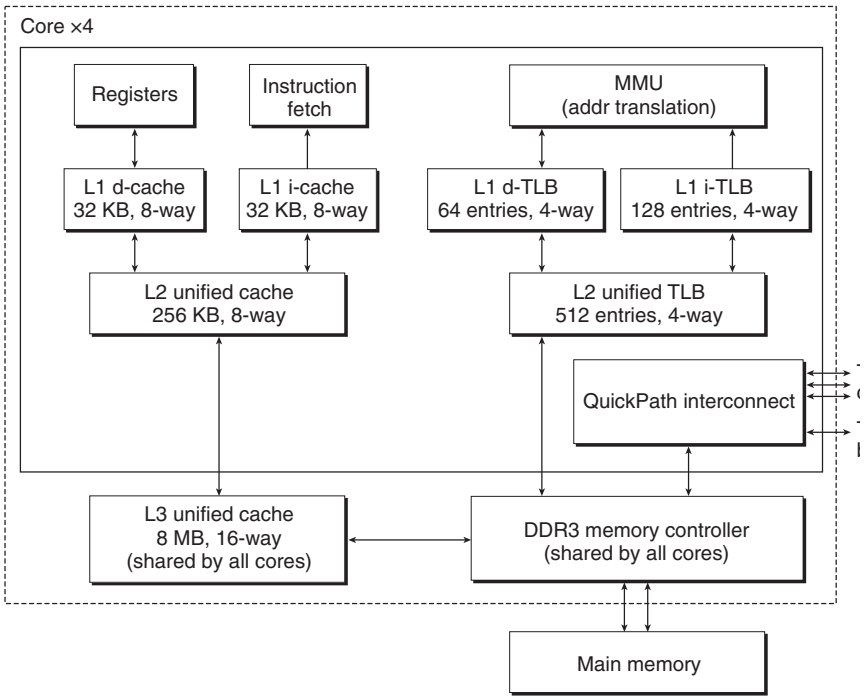


图 9.21　Core i7 的存储器系统。

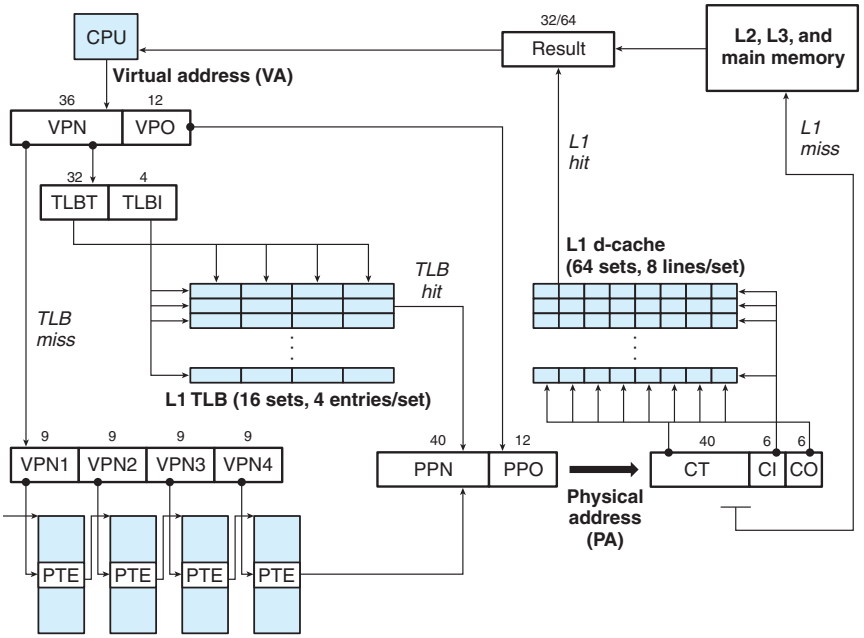


图 9.22　Core i7 地址翻译小结。为简单起见，指令高速缓存(i-cache)、i-TLB 和 L2 统一 TLB 没有画出。

9.7.1 Core i7 地址翻译

图 9.22 概括了整个 Core i7 地址翻译过程，从 CPU 产生虚拟地址开始，直到来自内存的数据字到达为止。Core i7 使用一个四级页表层次结构。每个进程都有它自己私有的页表层次结构。当一个 Linux 进程在运行时，与已分配页相关联的那些页表都是驻留在内存中的，虽然 Core i7 体系结构允许这些页表被换入和换出。extttCR3 控制寄存器包含第 1 级（L1）页表开始处的物理地址。extttCR3 的值是每个进程上下文中的一部分，并且在每次上下文切换时都会被恢复。

XD	Unused	Page table physical base addr	Unused	G	PS		A	CD	WT	U/S	R/W	P=1

Available for OS (page table location on disk)	P=0

图 9.23　1 级、2 级和 3 级页表项的格式。每个表项引用一个 4 KB 的子页表。

图 9.23 展示了第 1 级、第 2 级或第 3 级页表中一个条目的格式。当 $P = 1$ 时（在 Linux 中总是这种情况），地址字段包含一个 40 位的物理页号（PPN），它指向相应的页表的开始处。注意，这就对页表强加了一个 4 KB 的对齐要求。

图 9.24 展示了第 4 级页表中一个条目的格式。当 $P = 1$ 时，地址字段包含一个 40 位的 PPN，它指向物理内存中某个页的基址。同样，这就对物理页强加了一个 4 KB 的对齐要求。

PTE 有三个许可位控制对页的访问。R/W 位决定页的内容是读/写的还是只读的。U/S 位决定是否可以在用户模式下访问该页，它保护操作系统内核中的代码和数据不被用户程序访问。XD（execute disable，禁止执行）位是在 64 位系统中引入的，可以用来禁止从单独的内存页中取指令。这是一个重要的新特性，它使操作系统内核能够通过把执行限制在只读的代码段中，来降低缓冲区溢出攻击的风险。

当 MMU 翻译每个虚拟地址时，它还会更新另外两个位，内核的缺页处理程序可以利用它们。每次访问一个页时，MMU 都会设置 A 位，即**引用位**（**reference bit**）。内核可以用引用位来实现它的页替换算法。每次写一个页时，MMU 都会设置 D 位，即**脏位**（**dirty bit**）。被修改过的页有时称为**脏页**（**dirty page**）。脏位告诉内核，在它复制进一个替换页之前，是否必须先把牺牲页写回。内核可以调用一条特殊的内核模式指令来清除引用位或脏位。

图 9.25 展示了 Core i7 MMU 如何利用四级页表把一个虚拟地址翻译成一个物理地址。36 位的 VPN 被划分成四个 9 位的块，每个块都作为进入某个页表的偏移量。extttCR3 寄存器包含 L1 页表的物理地址。VPN 1 提供到一个 L1

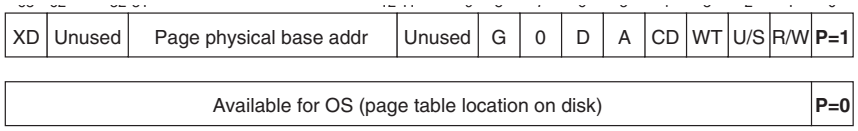
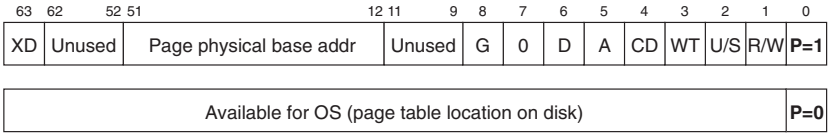


图 9.24　4 级页表项的格式。每个表项引用一个 4 KB 的子页。

PTE 的偏移量，该 PTE 包含 L2 页表的基址。VPN 2 提供到一个 L2 PTE 的偏移量，依此类推。



Field	Description
P	Child page present in physical memory (1) or not (0).
R/W	Read-only or read/write access permission for child page.
U/S	User or supervisor mode (kernel mode) access permission for child page.
WT	Write-through or write-back cache policy for the child page.
CD	Cache disabled or enabled.
A	Reference bit (set by MMU on reads and writes, cleared by software).
D	Dirty bit (set by MMU on writes, cleared by software).
G	Global page (don't evict from TLB on task switch).
Base addr	40 most significant bits of physical base address of child page.
XD	Disable or enable instruction fetches from the child page.

Figure 9.24 **Format of level 4 page table entries.** Each entry references a 4 KB child page.

write back a victim page before it copies in a replacement page. The kernel can call a special kernel-mode instruction to clear the reference or dirty bits.

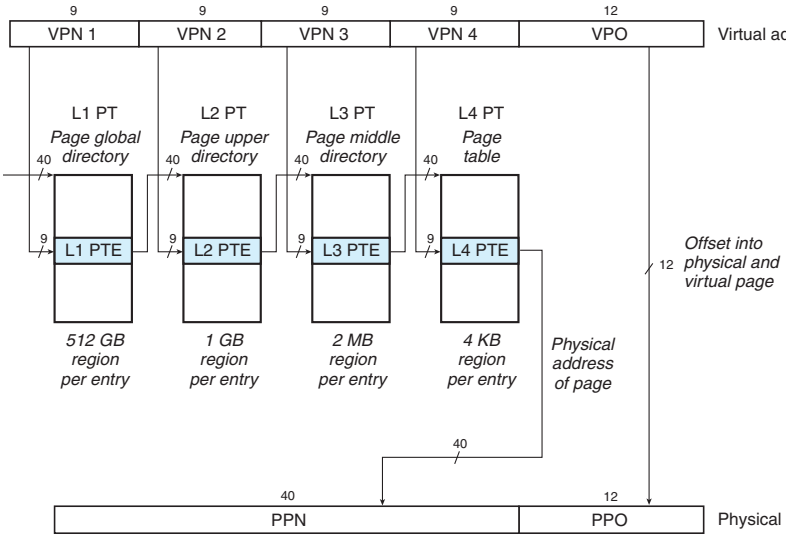
Figure 9.25 shows how the Core i7 MMU uses the four levels of page tables to translate a virtual address to a physical address. The 36-bit VPN is partitioned into four 9-bit chunks, each of which is used as an offset into a page table. The CR3 register contains the physical address of the L1 page table. VPN 1 provides an offset to an L1 PTE, which contains the base address of the L2 page table. VPN 2 provides an offset to an L2 PTE, and so on.

图 9.25

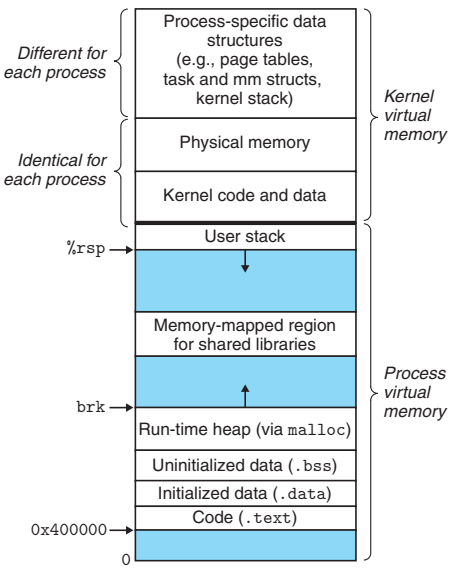
9.7.2 Linux 虚拟内存系统

一个虚拟内存系统要求硬件和内核之间紧密协作。细节因版本而异，完整的描述超出了本书的范围。尽管如此，本节目的还是描述 Linux 虚拟内存系统中足够多的内容，使你能够了解一个真实的操作系统是如何组织虚拟内存、如何处理缺页的。

Linux 为每个进程维护一个形式如图 9.26 所示的单独的虚拟地址空间。这幅图我们已经见过很多次了，它有大家熟悉的代码、数据、堆、共享库和栈段。既然我们已经理解了地址翻译，就可以对位于用户栈之上的内核虚拟内存补充一些细节了。



.25 Core i7 page table translation. PT: page table; PTE: page table entry; VPN: virtual pa ual page offset; PPN: physical page number; PPO: physical page offset. The Linux names page tables are also shown.



.26
ual memory of a ocess.

图 9.26　Linux 进程的虚拟内存。

内核虚拟内存包含内核中的代码和数据结构。内核虚拟内存的一些区域被映射到一些物理页上，这些物理页

边栏　优化地址翻译

在讨论地址翻译时，我们描述了一个顺序的两步过程：MMU（1）先把虚拟地址翻译成物理地址，然后（2）把物理地址传给 L1 高速缓存。然而，实际的硬件实现使用了一个巧妙的技巧，允许这两个步骤部分重叠，从而加速对 L1 高速缓存的访问。例如，在使用 4 KB 页的 Core i7 上，一个虚拟地址有 12 位的 VPO，而这些位与相应物理地址中的 12 位 PPO 是相同的。由于 8 路组相联的物理寻址 L1 高速缓存有 64 个组、64 字节的缓存块，每个物理地址就有 $6(\log_2 64)$ 个缓存偏移位和 $6(\log_2 64)$ 个索引位。这 12 位恰好放进一个虚拟地址的 12 位 VPO 中，这绝非偶然！当 CPU 需要翻译一个虚拟地址时，它把 VPN 发送给 MMU，把 VPO 发送给 L1 高速缓存。当 MMU 正在向 TLB 请求一个页表项时，L1 高速缓存则正忙着用 VPO 位找到相应的组，并读出该组中的 8 个标记和相应的数据字。当 MMU 从 TLB 得到 PPN 时，高速缓存已经准备好尝试把 PPN 与这 8 个标记之一进行匹配了。

被所有进程共享。例如，每个进程都共享内核的代码和全局数据结构。有趣的是，Linux 还把一组连续的虚拟页（其大小等于系统中 DRAM 的总量）映射到相应的一组连续的物理页上。这就为内核提供了一种访问任意特定物理内存位置的方便方式——例如，当它需要访问页表，或者对映射到特定物理内存位置的设备执行内存映射 I/O 操作时。

内核虚拟内存的其他区域包含的数据因进程而异。例子包括页表、当内核在进程的上下文中执行代码时它所使用的栈，以及各种跟踪当前虚拟地址空间组织情况的数据结构。

Linux 虚拟内存区域

Linux 把虚拟内存组织成若干**区域（area）**的集合（区域也称为段，segment）。**区域**是一块连续的已存在（已分配）的虚拟内存，其中的页以某种方式相互关联。例如，代码段、数据段、堆、共享库段和用户栈都是不同的区域。每个已存在的虚拟页都包含在某个区域中，而不属于任何区域的虚拟页不存在，不能被进程引用。区域这个概念很重要，因为它允许虚拟地址空间中有间隙。内核不记录那些不存在的虚拟页，这样的页也不占用内存、磁盘或内核自身的任何额外资源。

图 9.27 突出显示了记录一个进程中虚拟内存区域的那些内核数据结构。内核为系统中的每个进程维护一个单独的任务结构（task structure，源代码中的 `task_struct`）。任务结构中的元素或者是内核运行该进程所需的全部信息（例如，PID、指向用户栈的指针、可执行目标文件的名字以及程序计数器），或者是指向这些信息的指针。

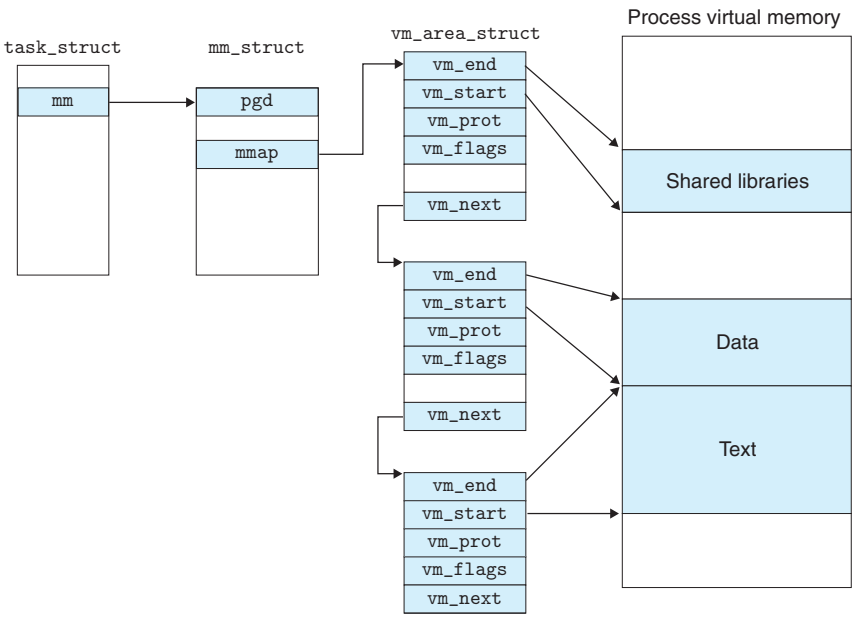


图 9.27　Linux 如何组织虚拟内存。

任务结构中的一个条目指向一个 `mm_struct`，它刻画虚拟内存的当前状态。我们感兴趣的两个字段是 `pgd`，它指向第 1 级页表（页全局目录）的基址；以及 `mmap`，它指向一个 `vm_area_structs`（区域结构）的链表，其中每个结构刻画当前虚拟地址空间的一个区域。当内核运行这个进程时，它把 `pgd` 存放在 extttCR3 控制寄存器中。

为了我们的目的，一个特定区域的区域结构包含下列字段：

- `vm_start`。指向这个区域的开始处。
- `vm_end`。指向这个区域的结束处。
- `vm_prot`。描述这个区域中包含的所有页的读/写许可权限。
- `vm_flags`。描述（除了别的以外）这个区域中的页是与其它进程共享的，还是私有于这个进程的。
- `vm_next`。指向链表中的下一个区域结构。

Linux 缺页异常处理

假设 MMU 在试图翻译某个虚拟地址 A 时触发了一次缺页。这个异常导致控制转移到内核的缺页处理程序，缺页处理程序随后执行下列步骤：

1. 虚拟地址 A 是否合法？换句话说，A 是否位于由某个区域结构定义的区域之内？为了回答这个问题，缺页处理程序搜索区域结构链表，把 A 与每个区域结构中的 `vm_start` 和 `vm_end` 进行比较。如果指令不合法，那么

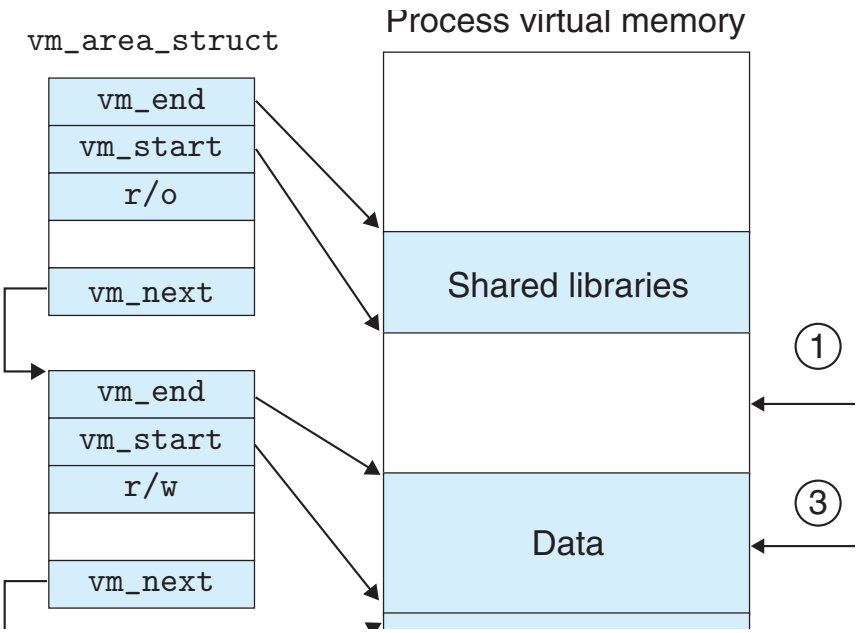


图 9.28　Linux 的缺页处理。

缺页处理程序就触发一个段错误，终止这个进程。这种情况在图 9.28 中标号为“1”。

因为一个进程可以创建任意多个新的虚拟内存区域（使用下一节中描述的 `mmap` 函数），对区域结构链表的顺序搜索可能代价很高。所以在实践中，Linux 利用一些我们还没有展示的字段，在这个链表上叠加一棵树，并在树上执行搜索。

2. 试图进行的内存访问是否合法？换句话说，进程是否有权限读、写或执行这个区域中的页？例如，缺页是否由一条试图写代码段中某个只读页的存储指令造成的？缺页是否由一个运行在用户模式下的进程试图从内核虚拟内存中读取一个字造成的？如果试图进行的访问不合法，那么缺页处理程序就触发一个保护异常，终止这个进程。这种情况在图 9.28 中标号为“2”。

3. 到这里，内核知道缺页是由对合法虚拟地址进行的合法操作造成的。它这样来处理这个缺页：选择一个牺牲页，如果牺牲页是脏的就把它换出，换入新页，并更新页表。当缺页处理程序返回时，CPU 重启引发缺页的那条指令，这条指令把 A 再次发送给 MMU。这一次，MMU 不会再引发缺页，而是正常翻译 A。

9.8 内存映射

Linux 通过把一个虚拟内存区域与磁盘上的一个对象关联起来，来初始化这个虚拟内存区域的内容，这个过程称为**内存映射**（**memory mapping**）。区域可以被映射到以下两种类型的对象之一：

1. **Linux 文件系统中的普通文件**。可以把一个区域映射到一个普通磁盘文件的连续部分，比如一个可执行目标文件。文件部分被划分成大小为一页的若干块，每块包含一个虚拟页的初始内容。因为按需调页，这些虚拟页没有一个被真正换入物理内存，直到 CPU 第一次触及该页（也就是说，发出一个落在该页地址空间区域内的虚拟地址）为止。如果区域比文件部分大，那么就用零来填充这个区域。
2. **匿名文件**。一个区域也可以被映射到一个匿名文件（anonymous file）上，匿名文件由内核创建，包含全二进制零。CPU 第一次触及这种区域中的一个虚拟页时，内核就在物理内存中找一个合适的牺牲页，如果牺牲页是脏的就把它换出，用二进制零覆盖牺牲页，并更新页表，将这个页标记为驻留的。注意，实际上没有数据在磁盘和内存之间传送。由于这个原因，映射到匿名文件的区域中的页有时称为**按需零页**（**demand-zero page**）。

无论哪种情况，一旦一个虚拟页被初始化了，它就会在一个由内核维护的特殊**交换文件**（**swap file**）与内存之间来回换入换出。交换文件也称为**交换空间**（**swap space**）或**交换区**（**swap area**）。需要认识到的一个要点是：在任何时刻，交换空间都限制了当前运行进程能够分配的虚拟页的总量。

9.8.1 再谈共享对象

内存映射的思想源于一个巧妙的洞察：如果虚拟内存系统可以被整合到传统的文件系统中，那么它就能提供一种简单而高效的方式，把程序和数据加载到内存中。

正如我们已经看到的，进程抽象为每个进程提供了它自己的私有虚拟地址空间，可以防止其他进程错误地写或读这个地址空间。不过，有很多进程都有相同的只读代码区域。例如，运行 Linux shell 程序 `bash` 的每个进程都有相同的代码区域。此外，很多程序需要访问只读运行时库代码的完全相同的副本。例如，每个 C 程序都需要标准 C 库中的诸如 `printf` 这样的函数。让每个进程都在物理内存中保留这些常用代码的重复副本会极其浪费。幸运的是，内存映射为我们提供了一种干净的机制，用来控制多个进程如何共享对象。

一个对象可以被作为**共享对象**（**shared object**）或者**私有对象**（**private object**）映射到虚拟内存的一个区域中。如果一个进程把一个共享对象映射到

它的虚拟地址空间的一个区域中，那么这个进程对该区域所做的任何写操作，对于已经把该共享对象映射到它们虚拟内存中的任何其他进程都是可见的。而且，这些变化还会反映到磁盘上原始的对象中。

另一方面，对于映射到一个私有对象上的区域所做的更改，对其他进程不可见，进程对这个区域所做的任何写操作都不会反映到磁盘上的对象中。一个映射了共享对象的虚拟内存区域通常称为**共享区域**（**shared area**）。类似地，映射了私有对象的区域称为**私有区域**（**private area**）。

假设进程 1 把一个共享对象映射到它的虚拟内存的一个区域中，如图 9.29(a) 所示。再假设进程 2 把同一个共享对

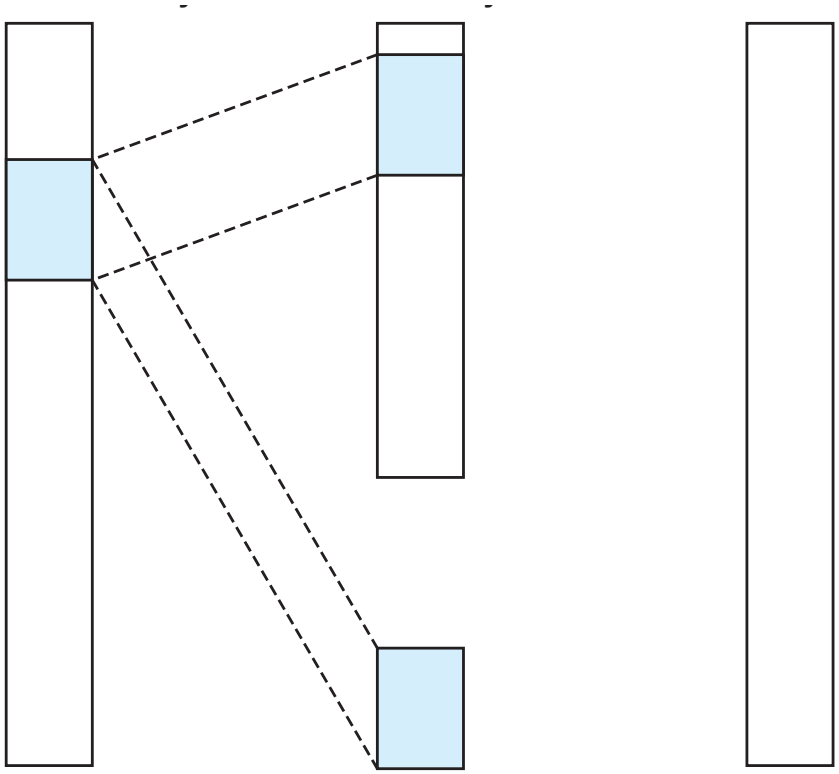


图 9.29　一个共享对象。(a) 进程 1 映射该共享对象之后。(b) 进程 2 映射同一共享对象之后。（注意，物理页不一定连续。）

象映射到它的地址空间中（不一定在与进程 1 相同的虚拟地址处），如图 9.29(b) 所示。

因为每个对象都有唯一的文件名，内核可以迅速判定进程 1 已经映射了这个对象，并可以让进程 2 的页表项指向合适的物理页。关键点在于，即使这个对象被映射到了多个共享区域中，它在物理内存中也只需要存储一个副本。为了方便起见，我们把物理页画成了连续的，当然一般来说并不是这样。

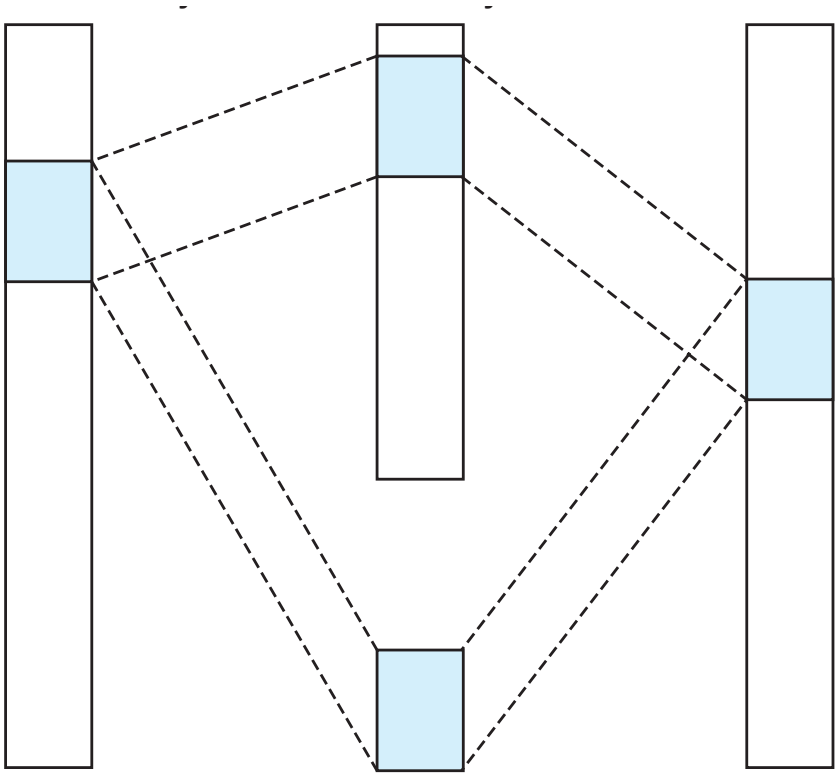


图 9.30 一个私有的写时复制对象。(a) 两个进程都映射了该私有写时复制对象之后。(b) 进程 2 写私有区域中的一个页之后。

私有对象用一种称为**写时复制**（**copy-on-write**）的巧妙技术映射到虚拟内存中。一个私有对象的开始与一个共享对象完全一样，物理内存中只保存该私有对象的一个副本。例如，图 9.30(a) 展示了一种情形：两个进程把一个私有对象映射到它们虚拟内存的不同区域中，但是共享该对象在物理内存中的同一个副本。对每个映射私有对象的进程来说，相应私有区域的页表项都被标记为只读，区域结构被标记为私有写时复制。只要两个进程都不试图写各自的私有区域，它们就继续共享该对象在物理内存中的唯一副本。但是，只要一个进程试图写私有区域中的某个页，这个写操作就会触发一个保护错误。

当缺页处理程序注意到这个保护异常是由该进程试图写某个私有写时复制区域中的一个页导致的时候，它会在物理内存中创建这个页的一个新副本，更新页表项指向新的副本，然后恢复对这个页的写许可权限，如图 9.30(b) 所示。当缺页处理程序返回时，CPU 重新执行写操作，这次写在新创建的页上正常进行。

通过把对私有对象中的页的复制推迟到尽可能晚的时刻，写时复制使稀缺的物理内存得到了最有效的利用。

9.8.2 再谈 fork 函数

既然我们已经理解了虚拟内存和内存映射，就可以清楚地理解 fork 函数是如何创建一个带有自己独立虚拟地址空间的新进程的了。

当前进程调用 fork 函数时，内核为新进程创建各种数据结构，并分配给它一个唯一的 PID。为了给新进程创建虚拟内存，内核创建了当前进程的 mm_struct、区域结构和页表的完全相同的副本。它把两个进程中的每个页都标记为只读，并把两个进程中的每个区域结构都标记为私有写时复制。

当 fork 在新进程中返回时，新进程现在拥有一个与 fork 被调用时存在的虚拟内存完全相同的副本。当两个进程中的任何一个之后进行写操作时，写时复制机制就会创建新页，从而为每个进程保留私有地址空间的抽象。

9.8.3 再谈 execve 函数

虚拟内存和内存映射也在把程序加载到内存的过程中扮演关键角色。既然我们已经理解了这些概念，就可以理解 execve 函数实际上是如何加载并运行程序的。假设当前进程中运行的程序进行下面的调用：

```
execve("a.out", NULL, NULL);
```

正如你在第 8 章中学到的那样，execve 函数在当前进程中加载并运行包含在可执行目标文件 a.out 中的程序，用 a.out 程序有效地替换了当前程序。加载并运行 a.out 需要以下步骤：

1. **删除已存在的用户区域**。删除当前进程的虚拟地址的用户部分中已存在的区域结构。
2. **映射私有区域**。为新程序的代码、数据、bss 和栈区域创建新的区域结构。所有这些新区域都是私有写时复制的。代码区域和数据区域被映射到 a.out 文件的.text 和.data 节。bss 区域是按需零页的，被映射到一个匿名文件，匿名文件的大小包含在 a.out 中。栈和堆区域也是按需零页的，初始长度为零。图 9.31 概括了私有区域的不同映射方式。
3. **映射共享区域**。如果 a.out 程序是与共享对象（比如标准 C 库 libc.so）链接的，那么这些对象就被动态链接到这个程序中，然后再被映射到用户虚拟地址空间中的共享区域中。
4. **设置程序计数器（PC）**。execve 所做的最后一件事是把当前进程上下文中的程序计数器设置为指向代码区域中的入口点。

下一次调度这个进程时，它将从入口点开始执行。Linux 会按照需要换入代码和数据的页。

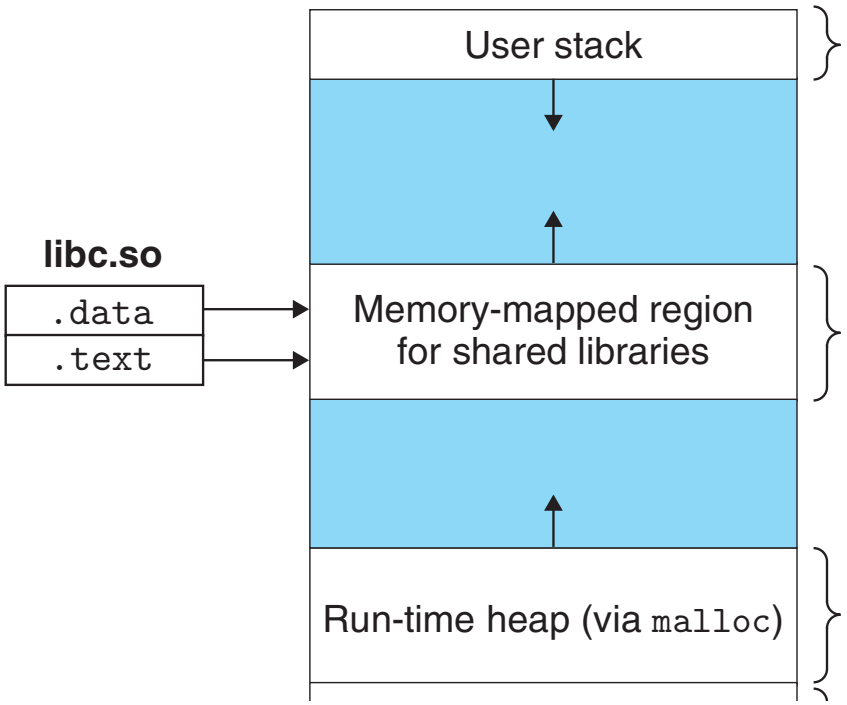


图 9.31 加载器如何映射用户地址空间中的区域。

9.8.4 用 mmap 函数进行用户级内存映射

Linux 进程可以使用 mmap 函数创建新的虚拟内存区域，并把对象映射到这些区域中。

```
#include <unistd.h>
#include <sys/mman.h>

void    *mmap(void *start, size_t length, int prot, int flags,
              int fd, off_t offset);

                                                Returns:
        若成功则为指向被映射区域的指针,若出错则为 MAP_FAILED(-1)
```

mmap 函数要求内核创建一个新的虚拟内存区域，最好是从地址 start 处开始的一个区域，并把由文件描述符 fd 指定的对象的一个连续块映射到这个新区域。这个对象的连续块大小为 length 字节，从距文件开头 offset 字节的地方开始。start 地址仅仅是一个提示，通常被指定为 NULL。为了我们的目的，我们总是假设 start 地址为 NULL。图 9.32 描述了这些参数的含义。

prot 参数包含描述新映射的虚拟内存区域的访问许可权限的位（也就是相应区域结构中的 vm_prot 位）。

- PROT_EXEC：区域中的页由 CPU 可以执行的指令组成。

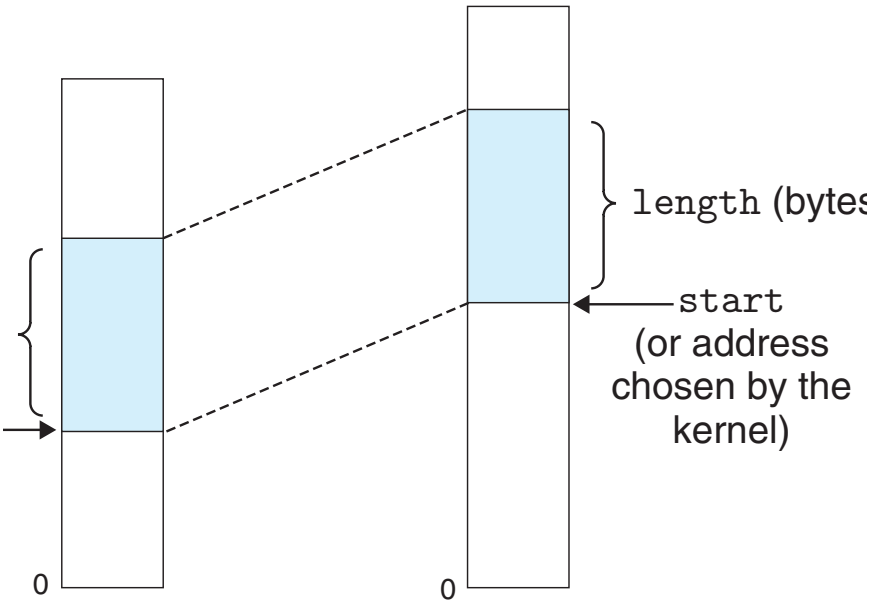


图 9.32 mmap 参数的直观解释。

- PROT_READ:区域中的页可以读。
- PROT_WRITE:区域中的页可以写。
- PROT_NONE:区域中的页不能访问。

flags 参数由描述被映射对象类型的位组成。如果设置了 MAP_ANON 标志位，那么后备存储就是一个匿名对象，相应的虚拟页就是按需零页的。MAP_PRIVATE 表示一个私有写时复制对象，MAP_SHARED 表示一个共享对象。例如，

```
bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE|MAP_ANON, 0, 0);
```

它要求内核创建一个新的只读的、私有的、按需零页的虚拟内存区域，其中有 size 个字节。如果调用成功，那么 bufp 中就包含新区域的地址。

munmap 函数删除虚拟内存的区域：

```
#include <unistd.h>
#include <sys/mman.h>

int munmap(void *start, size_t length);

                                                Returns: 若成功则为 0,
                                                ↪  若出错则为 -1
```

munmap 函数删除从虚拟地址 start 开始、其后的 length 个字节所组成的区域。以后对已删除区域的引用会导致段错误。

练习 9.5 编写一个 C 程序 `mmapcopy.c`，用 `mmap` 把磁盘上一个任意大小的文件复制到标准输出。输入文件的名字应当作为命令行参数传递。

9.9 动态内存分配

虽然确实可以使用低层次的 `mmap` 和 `munmap` 函数来创建和删除虚拟内存区域，但是 C 程序员通常发现，当需要在运行时获得额外的虚拟内存时，使用动态内存分配器会更方便，可移植性也更好。

动态内存分配器维护着一个进程的虚拟内存的一个区域，称为**堆（heap）**（图 9.33）。细节因系统而异。不过在不失一般性的前提下，我们假设堆是一个按需零页的内存区域，它紧接在未初始化数据区域之后开始，并向上（朝更高地址的方向）增长。对于每个进程，内核维护一个变量 `brk`（读作“break”），它指向堆的顶部。

分配器把堆当作一组大小不同的块来维护。每个块都是虚拟内存中一个连续的块，或者被分配，或者空闲。分配块显式地保留为供应用程序使用。空闲块可以拿来分配。空闲块保持空闲，直到它被应用程序显式分配为止。分配块保持已分配的状态，直到它被释放为止，这种释放要么由应用程序显式地进行，要么隐式地由内存分配器完成。

分配器有两种基本风格。两种风格都要求应用程序显式地分配块。它们的不同之处在于由哪个实体负责释放已分配的块。

- **显式分配器（explicit allocator）**要求应用程序显式地释放任何已分配的块。例如，C 标准库提供一个称为 `malloc` 包的显式分配器。C 程序通过调用 `malloc` 函数来分配一个块，通过调用 `free` 函数来释放一个块。C++ 中的 `new` 和 `delete` 调用与之相当。
- **隐式分配器（implicit allocator）**与之相反，它要求分配器检测已分配的块何时不再被程序使用，然后释放这个块。隐式分配器也称为**垃圾收集器（garbage collector）**，自动释放未使用的已分配块的过程称为**垃圾收集（garbage collection）**。例如，Lisp、ML 和 Java 这样的高级语言就依靠垃圾收集来释放已分配的块。

本节其余部分讨论显式分配器的设计和实现。我们将在 9.10 节讨论隐式分配器。为了具体起见，我们的讨论集中在管理堆内存的分配器上。不过你应该明白，内存分配是一个普遍的概念，会出现在各种上下文中。例如，那些需要对图进行密集操作的应用程序经常使用标准分配器获取一大块虚拟内存，然后用一个应用程序专用的分配器来管理那一大块内部的内存，随着图的节点被创建和销毁来分配和释放它们。

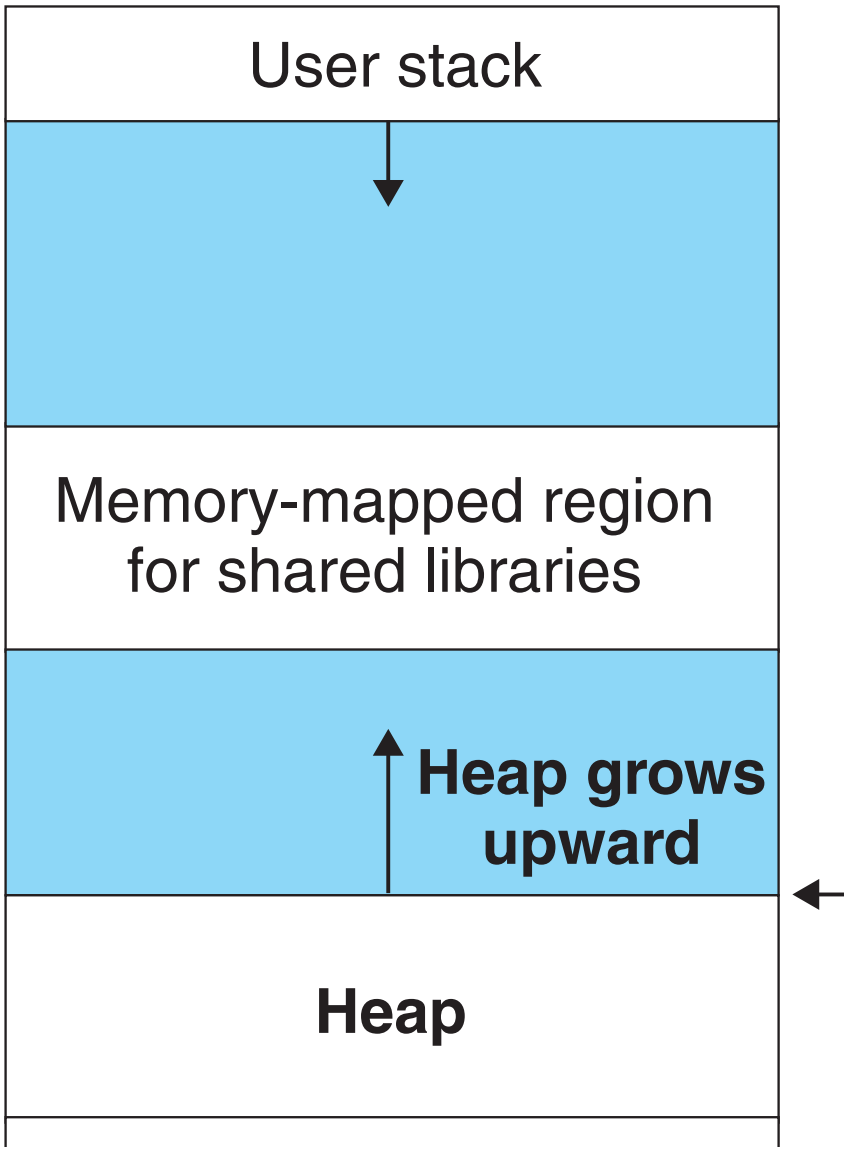


图 9.33 堆。

9.9.1 malloc 和 free 函数

C 标准库提供了一个称为 malloc 包的显式分配器。程序通过调用 malloc 函数从堆中分配块。

```
#include <stdlib.h>

void *malloc(size_t size);

                                    Returns: 若成功则为指向已分配块
                                    ↪ 的指针,若出错则为
```

```
        NULL
```

边栏 一个字有多大？
回想一下第 3 章中关于机器码的讨论，Intel 把 4 字节的对象称为双字。然而，在整个这一节中，我们都假设字是 4 字节的对象，双字是 8 字节的对象，这与传统的术语是一致的。

malloc 函数返回一个指针，指向至少为 size 个字节的内存块，这个内存块经过了适当对齐，可以对齐到块中可能包含的任何类型的数据对象。在实际中，对齐取决于代码是被编译到 32 位模式（gcc -m32）下运行，还是 64 位模式（默认）下运行。在 32 位模式中，malloc 返回的块的地址总是 8 的倍数。在 64 位模式中，地址总是 16 的倍数。

如果 malloc 遇到问题（例如，程序请求的内存块比可用的虚拟内存还要大），那么它返回 NULL，并设置 errno。malloc 不初始化它返回的内存。想要得到已初始化的内存的应用程序可以使用 calloc，它是 malloc 函数的一个薄封装，会把分配的内存初始化为零。想要改变一个以前分配的块的大小的应用程序可以使用 realloc 函数。

诸如 malloc 这样的动态内存分配器可以用 mmap 和 munmap 函数显式地分配或释放堆内存，也可以使用 sbrk 函数：

```
#include <unistd.h>
void *sbrk(intptr_t incr);

                                        Returns: 若成功则为旧的 brk 指针，
                                        ↪  若出错则为 -1
```

sbrk 函数通过把 incr 加到内核的 brk 指针上来增长或收缩堆。如果成功，它就返回 brk 的旧值，否则就返回 -1，并把 errno 设置为 ENOMEM。如果 incr 为零，那么 sbrk 就返回 brk 的当前值。用负的 incr 调用 sbrk 是合法的，不过有点棘手，因为返回值（brk 的旧值）指向距新堆顶 abs(incr) 字节处。

程序通过调用 free 函数来释放已分配的堆块。

```
#include <stdlib.h>
void free(void *ptr);

                                        Returns: 无返回值
```

ptr 参数必须指向一个从 malloc、calloc 或 realloc 得到的已分配块的开始处。如果不是这样，那么 free 的行为就是未定义的。更糟的是，由于它什么都不返回，free 就不会向应用程序给出任何指示，告诉它有什么地方出错了。正如我们在 9.11 节中将要看到的，这可能产生一些令人费解的运行时错误。

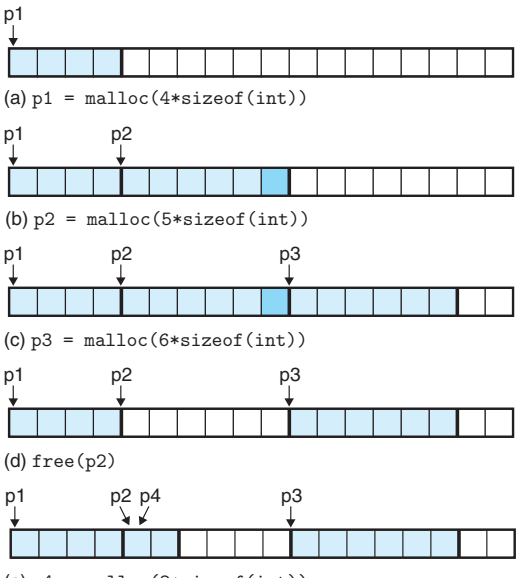


Figure 9.34
Allocating and freeing blocks with `malloc` and `free`. Each square corresponds to a word. Each heavy rectangle corresponds to a block. Allocated blocks are shaded. Padded regions of allocated blocks are shaded with a darker blue. Free blocks are unshaded. Heap addresses increase from left to right.

图 9.34

图 9.34 展示了 malloc 和 free 的一个实现是如何为一个 C 程序管理一个(非常)小的 16 字的堆的。每个方框代表一个 4 字节的字。加粗线的矩形对应于已分配的块(有阴影)和空闲的块(无阴影)。初始时,堆是一个由 16 个字组成的、双字对齐的单个空闲块。[1]

- **图 9.34(a)**。程序请求一个四字块。malloc 的响应是从空闲块的前端切出一个四字块,并返回一个指向该块的第一个字的指针。
- **图 9.34(b)**。程序请求一个五字块。malloc 的响应是从空闲块的前端分配出一个六字块。在这个例子中,malloc 在块中多填充了一个字,以保持空闲块在双字边界上对齐。
- **图 9.34(c)**。程序请求一个六字块,malloc 的响应是从空闲块中切出一个六字块。
- **图 9.34(d)**。程序释放图 9.34(b) 中分配的那个六字块。注意,在调用 free 返回之后,指针 p2 仍然指向那个已释放的块。应用程序有责任在用一次新的 malloc 调用重新初始化 p2 之前,不再使用 p2。
- **图 9.34(e)**。程序请求一个两字块。在此情况下,malloc 分配上一步中被释放的那个块的一部分,并返回一个指向这个新块的指针。

[1]在本节中,我们假设分配器返回的块都对齐到 8 字节的双字边界。

9.9.2 为什么需要动态内存分配

程序使用动态内存分配的最重要原因是,在程序实际运行之前,往往不知道某些数据结构的大小。例如,假设要求我们写一个 C 程序,从 `stdin` 读入一个由 n 个 ASCII 整数组成的列表,每行一个整数,放进一个 C 数组中。输入包括整数 n,后面跟着要读入并存储到数组中的 n 个整数。最简单的办法就是用某个硬编码的最大数组大小静态地定义数组:

```
1  #include "csapp.h"
2  #define MAXN 15213
3
4  int array[MAXN];
5
6  int main()
7  {
8      int i, n;
9
10     scanf("%d", &n);
11     if (n > MAXN)
12         app_error("Input file too big");
13     for (i = 0; i < n; i++)
14         scanf("%d", &array[i]);
15     exit(0);
16 }
```

像这样用硬编码的大小来分配数组常常是个坏主意。`MAXN` 的值是任意设定的,与机器上实际可用的虚拟内存总量没有任何关系。另外,如果这个程序的用户想读取的文件比 `MAXN` 大,唯一的补救办法就是用更大的 `MAXN` 值重新编译程序。虽然对这一个简单的例子来说不是问题,但是对于一个有上百万行代码和大量用户的大型软件产品来说,硬编码数组边界可能成为维护的噩梦。

更好的办法是在运行时、当 n 的值已知之后再动态地分配数组。在这种方式下,数组的最大大小只受可用的虚拟内存量的限制。

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int *array, i, n;
6
7      scanf("%d", &n);
8      array = (int *)Malloc(n * sizeof(int));
9      for (i = 0; i < n; i++)
10         scanf("%d", &array[i]);
11     free(array);
12     exit(0);
13 }
```

动态内存分配是一种有用且重要的编程技术。然而，要想正确而高效地使用分配器，程序员需要理解它们是如何工作的。我们将在 9.11 节中讨论一些由于分配器的不当使用而造成的令人毛骨悚然的错误。

9.9.3 分配器的要求和目标

显式分配器必须在一些相当严格的约束下工作：

- **处理任意请求顺序**。应用程序可以做出任意的分配和释放请求序列，只要满足约束：每个释放请求必须对应于一个当前已分配的块，这个块由前面的分配请求得到。因此，分配器不能对分配和释放请求的排序做任何假设。比如，分配器不能假设所有的分配请求都伴随有相配的释放请求，或者相配的分配请求和释放请求是嵌套的。
- **立即响应请求**。分配器必须立即响应分配请求。因此，不允许分配器通过重新排列或者缓冲请求来提升性能。
- **只使用堆**。为了使分配器是可扩展的，分配器使用的任何非标量数据结构都必须保存在堆自身中。
- **对齐块（对齐要求）**。分配器必须对齐块，使得它们可以容纳任何类型的数据对象。
- **不修改已分配的块**。分配器只能操作或改变空闲块。特别是，一旦一个块被分配了，就不允许修改或者移动它们。因此，对已分配块进行压缩之类的技术是不允许的。

在这些约束条件下，分配器的作者试图同时达成两个常常相互冲突的性能目标：最大化吞吐量（throughput）和内存利用率。

目标 1：最大化吞吐量。给定某个由 n 个分配和释放请求组成的序列

$$R_0,\ R_1,\ \ldots,\ R_k,\ \ldots,\ R_{n-1}$$

我们希望最大化分配器的吞吐量。吞吐量定义为分配器每单位时间内完成的请求数。例如，如果一个分配器在 1 秒内完成了 500 个分配请求和 500 个释放请求，那么它的吞吐量就是每秒 1000 次操作。一般而言，我们可以通过最小化满足分配请求和释放请求所需的平均时间来最大化吞吐量。我们会看到，要开发出性能相当好的分配器并不太难：在这种分配器中，一个分配请求的最坏运行时间与空闲块的数目成线性关系，而一个释放请求的运行时间是常数。

目标 2：最大化内存利用率。不成熟的程序员常常错误地认为虚拟内存是一种取之不尽的资源。实际上，一个系统中所有进程分配的虚拟内存总量都要受磁盘交换空间大小的限制。优秀的程序员知道，虚拟内存是一种必须高效使

用的有限资源。对动态内存分配器而言尤其如此，它有可能被要求分配和释放很大的内存块。

描述一个分配器使用堆的效率好坏，可以用多种方式。根据我们的经验，最有用的度量是峰值利用率(peak utilization)。和前面一样，给定某个由 n 个分配和释放请求组成的序列

$$R_0,\ R_1,\ \ldots,\ R_k,\ \ldots,\ R_{n-1}$$

如果应用程序请求一个 p 字节的块，那么产生的已分配块的有效载荷(payload)就是 p 字节。在请求 R_k 完成之后，令聚合载荷 P_k 表示当前已分配块的有效载荷之和，令 H_k 表示堆当前(单调不减)的大小。

那么，前 $k+1$ 个请求上的峰值利用率记作 U_k，由下式给出：

$$U_k = \frac{\max_{i \le k} P_i}{H_k}$$

分配器的目标就是使整个请求序列上的峰值利用率 U_{n-1} 达到最大。我们将会看到，最大化吞吐量和最大化利用率之间存在张力。特别地，很容易写出一个以牺牲堆利用率为代价来最大化吞吐量的分配器。在任何分配器的设计中，有趣的挑战之一就是在这两个目标之间找到恰当的平衡。

> **边栏　放宽单调性假设**
> 我们可以放宽 U_k 定义中的单调不减假设，允许堆向上和向下两个方向生长，此时令 H_k 表示前 $k+1$ 个请求的高水位(high-water mark)。

9.9.4 碎片

堆利用率不佳的一个主要原因是一种称为碎片（fragmentation）的现象：尽管有未被使用的内存，却不能满足分配请求。碎片有两种形式：内部碎片(internal fragmentation)和外部碎片(external fragmentation)。

当一个已分配块比有效载荷大时，就发生了内部碎片。造成这种情况的原因可能有好几种。例如，分配器的实现可能对已分配块强加了一个比某些所请求的有效载荷都大的最小块大小。再比如，正如我们在图 9.34(b) 中看到的，分配器可能会为了满足对齐约束而增大块的大小。

内部碎片很容易量化。它就是所有已分配块的大小与它们的有效载荷之差的总和。因此，任一时刻内部碎片的数量只取决于以前请求的模式和分配器的实现。

当空闲内存聚合起来足以满足某个分配请求，但是却没有一个单独的空闲块足够大时，就发生了外部碎片。例如，如果图 9.34(e) 中的请求不是 2 个字而是 8 个字，那么这个请求将无法被满足，除非向内核请求额外的虚拟内存，尽管堆中还剩下 8 个空闲字。问题之所以出现，是因为这 8 个字分散在两个空闲块中。

外部碎片比内部碎片难以量化得多，因为它不仅取决于以前请求的模式和分配器的实现，还取决于未来请求的模式。例如，假设经过 k 个请求之后，所有空闲块的大小都恰好是 4 个字。这个堆是否受到外部碎片的困扰？答案取决于未来请求的模式。如果将来所有的分配请求针对的都是小于或等于 4 个字的块，那么就不存在外部碎片。反过来，如果有一个或多个请求要求大于 4 个字的块，那么这个堆就确实存在外部碎片。

既然外部碎片难以量化、也无法预测，分配器通常就会采用一些启发式策略，力求保持数量较少但较大的空闲块，而不是数量较多但较小的空闲块。

9.9.5 实现问题

可以想象得到的最简单的分配器会把堆组织成一个大的字节数组，以及一个最初指向数组第一个字节的指针 `p`。要分配 `size` 个字节，`malloc` 会把 `p` 的当前值保存在栈上，把 `p` 增加 `size`，并把 `p` 的旧值返回给调用者。`free` 只需返回给调用者，什么也不做。

这个幼稚的分配器是设计空间中的一个极端点。由于每个 `malloc` 和 `free` 都只执行寥寥数条指令，吞吐量会极好。然而，因为分配器从不重用任何块，内存利用率会极差。要在吞吐量和利用率之间取得更好平衡的实用分配器必须考虑下面这些问题：

- **空闲块组织**。如何记录空闲块的情况？
- **放置**。如何选择一个合适的空闲块来放置新分配的块？
- **分割**。把一个新分配的块放进某个空闲块之后，如何处理这个空闲块的剩余部分？
- **合并**。刚被释放的块该如何处理？

本节余下的部分将更详细地考察这些问题。由于放置、分割和合并的基本技术适用于许多不同的空闲块组织方式，我们将在一种称为隐式空闲链表（implicit free list）的简单空闲块组织结构下介绍这些技术。

9.9.6 隐式空闲链表

任何实际的分配器都需要某种数据结构，使它能够辨认块的边界，并区分已分配块和空闲块。大多数分配器把这些信息嵌在块本身之中。图 9.35 展示了一种简单的方法。

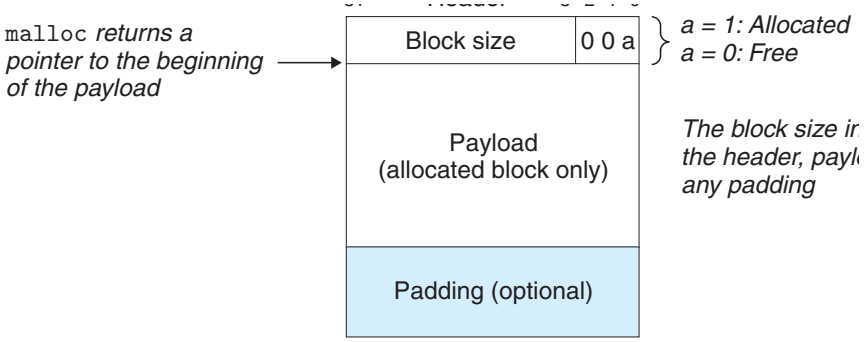


图 9.35 简单堆块的格式。

在这种情况下，一个块由一个字的头部、有效载荷以及可能的一些填充组成。头部既编码块的大小（包括头部和所有填充），也编码这个块是已分配块还是空闲块。如果我们强加双字对齐约束，那么块大小就总是 8 的倍数，块大小的低 3 位也总是零。这样，我们只需要存储块大小的高 29 位，剩下的 3 位可以用来编码其他信息。在本例中，我们用这些位中最低有效的那一位（即已分配位）来指示这个块是已分配块还是空闲块。例如，假设我们有一个块大小为 24（`0x18`）字节的已分配块，那么它的头部就是

$$\texttt{0x00000018} \mid \texttt{0x1} = \texttt{0x00000019}$$

类似地，一个块大小为 40（`0x28`）字节的空闲块的头部是

$$\texttt{0x00000028} \mid \texttt{0x0} = \texttt{0x00000028}$$

头部之后就是应用程序调用 `malloc` 时所请求的有效载荷。在有效载荷之后是一段没有使用的填充，大小可以是任意的。使用填充的理由有好几个。例如，填充可能是分配器为对抗外部碎片而采取的策略的一部分。或者填充也许是满足对齐要求所必需的。

给定图 9.35 中的块格式，我们可以把堆组织成连续的已分配块和空闲块序列，如图 9.36 所示。

我们把这种组织方式称为隐式空闲链表，因为空闲块是靠头部中的大小字段隐式地链接起来的。分配器通过遍历堆中的所有块，就能间接遍历所有的空

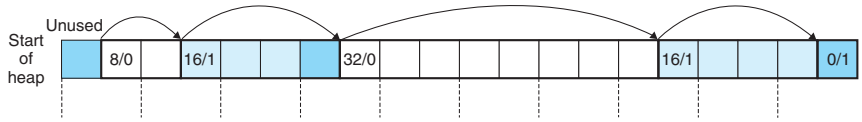


图 9.36 用隐式空闲链表组织堆。已分配块加阴影，空闲块不加阴影。头部用(大小(字节)/已分配位)来标注。

闲块。注意，我们需要某种特殊标记的结束块——在本例中，就是一个已分配位被置位、大小为零的终止头部。(正如我们在 9.9.12 节将要看到的，把已分配位设为 1 可以简化空闲块的合并。)

隐式空闲链表的优点是简单。一个显著的缺点是，任何需要搜索空闲链表的操作(比如放置已分配块)，其代价都与堆中已分配块和空闲块的总数成线性关系。

还要着重认识到：系统的对齐要求和分配器对块格式的选择，会给分配器强加一个最小块大小，任何已分配块或空闲块都不能比它更小。例如，假设有双字对齐要求，那么每个块的大小都必须是两个字(8 字节)的倍数。因此，图 9.35 的块格式所导致的最小块大小是两个字的块：一个字用于头部，另一个字用来满足对齐要求。即使应用程序只请求一个字节，分配器也仍然会创建一个两字的块。

练习 9.6 确定下面这串 `malloc` 请求所产生的块大小和头部值。假设：(1) 分配器保持双字对齐，并使用图 9.35 中块格式的隐式空闲链表；(2) 块大小向上舍入到 8 字节的最近倍数。

练习 9.6 的表

请求	块大小(十进制，字节)	块头(十六进制)
`malloc(2)`		
`malloc(9)`		
`malloc(15)`		
`malloc(20)`		

9.9.7 放置已分配的块

当应用程序请求一个 k 字节的块时，分配器会搜索空闲链表，寻找一个足以容纳这个请求块的空闲块。分配器执行这次搜索的方式由放置策略(placement policy)决定。常见的策略有首次适配(first fit)、下次适配(next fit)和最佳适配(best fit)。

首次适配从链表开头开始搜索空闲链表，选择第一个放得下的空闲块。下次适配与首次适配类似，但是它不是在每次搜索时都从链表开头开始，而是从上一次搜索结束的地方开始。最佳适配检查每一个空闲块，选择其中放得下的、大小最小的空闲块。

首次适配的一个优点是它倾向于把较大的空闲块留在链表的末端。缺点是它倾向于在链表前端留下一些小块的 201c 碎片 201d（splinters），这会增加对较大块的搜索时间。下次适配是 Donald Knuth 首先提出的，用来替代首次适配，其想法是：如果我们上一次在某个空闲块中找到了合适的块，那么下一次很有可能在这个块的剩余部分找到合适的块。下次适配的运行速度可以比首次适配快得多，尤其是当链表前端散落着许多小碎片的时候。不过，一些研究表明，下次适配的内存利用率比首次适配要差。研究发现，最佳适配的内存利用率一般比首次适配和下次适配都好。然而，在使用诸如隐式空闲链表这样简单的空闲链表组织时，最佳适配的缺点是它需要对堆进行全面搜索。稍后我们将会看到一些更复杂的分离空闲链表组织，它们无须对堆进行全面搜索，就能逼近最佳适配策略。

9.9.8 分割空闲块

一旦分配器找到了一个放得下的空闲块，它还必须做另一个策略决策：在这个空闲块中分配多少。一种选择是把整个空闲块都用上。这样做虽然简单、快速，但主要缺点是会引入内部碎片。如果放置策略倾向于找到放得比较合适的空闲块，那么容忍一些额外的内部碎片也许是可以接受的。

然而，如果找不到合适的空闲块，那么分配器通常会选择把这个空闲块分割成两个部分。第一部分变成已分配块，剩下的部分变成一个新的空闲块。图 9.37 展示了分配器如何分割图 9.36 中那个 8 字的空闲块，以满足应用程序对 3 字堆内存的请求。

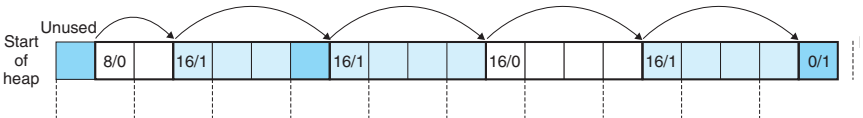


图 9.37 分割一个空闲块以满足一个三字的分配请求。已分配块加阴影，空闲块不加阴影。头部用（大小（字节）/已分配位）来标注。

9.9.9 获取额外的堆内存

如果分配器无法为请求的块找到一个放得下的空闲块，该如何应对？一种选择是通过合并（coalescing，下一节讨论）内存中物理相邻的空闲块，尝试创造

出一些较大的空闲块。然而，如果这样做还不能产生一个足够大的块，或者空闲块已经处于合并后的最大状态，那么分配器就会通过调用 `sbrk` 函数向内核请求额外的堆内存。分配器把这些额外的内存转化成一个大的空闲块，将这个块插到空闲链表中，然后把请求的块放置在这个新的空闲块里。

9.9.10 合并空闲块

当分配器释放一个已分配块时，可能会有其他空闲块与这个新释放的块相邻。这样的相邻空闲块会引起一种称为假性碎片（false fragmentation）的现象：大量可用的空闲内存被切割成许多小的、不可用的空闲块。例如，图 9.38 展示了释放图 9.37 中那个已分配块之后的结果。结果是两个相邻的空闲块，每个的有效载荷都是 3 个字。这样一来，接下来一个对 4 字有效载荷的请求就会失败，尽管这两个空闲块的聚合大小足以满足这个请求。

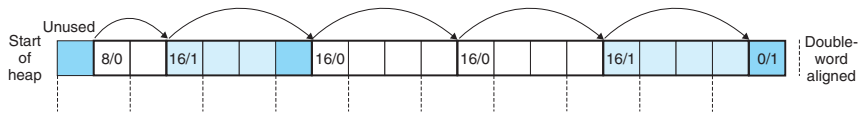


图 9.38 一个假碎片现象的例子。已分配块加阴影，空闲块不加阴影。头部用（大小（字节）/已分配位）来标注。

为了对抗假性碎片，任何实际的分配器都必须合并相邻的空闲块，这个过程称为合并（coalescing）。这就引出了另一个重要的策略决策：什么时候执行合并？分配器可以选择立即合并：在每次释放一个块时合并所有相邻的块。也可以选择推迟合并：等到某个较晚的时机再合并空闲块。例如，分配器可以一直推迟合并，直到某个分配请求失败，然后扫描整个堆，合并所有的空闲块。

立即合并简单直接，能够在常数时间内完成，但是对于某些请求模式，它会造成一种形式的颠簸：一个块刚被合并就很快又被分割开来，如此反复。例如，在图 9.38 中，反复地分配和释放一个 3 字的块就会引入大量不必要的分割和合并。在讨论分配器时，我们将假设采用立即合并，但你应该知道，快速的分配器通常会选择某种形式的推迟合并。

9.9.11 带边界标记的合并

分配器如何实现合并？我们把想要释放的块称为当前块。合并（内存中）下一个空闲块既直接又高效。当前块的头部指向下一个块的头部，检查这个头部就能判断下一个块是否空闲。如果是空闲的，那么只需把它的大小加到当前头部的大小上，就能在常数时间内合并这两个块。

但是我们如何合并前一个块呢？给定一个由带头部的块组成的隐式空闲链表，唯一的办法就是搜索整个链表，记住前一个块的位置，直到到达当前块。对于隐式空闲链表来说，这意味着每次调用 `free` 都需要花费与堆的大小成线性关系的时间。即使用更复杂的空闲链表组织，搜索时间也不是常数。

Knuth 设计了一种聪明而通用的技术，称为边界标记（boundary tag），支持在常数时间内合并前一个块。其思想如图 9.39 所示：在每个块的末尾添加一个脚部（即边界标记），脚部是头部的一个副本。如果每个块都带有这样的脚部，那么分配器通过查看前一个块的脚部（它总在距当前块起始处一个字的地方）就能确定前一个块的起始位置和状态。

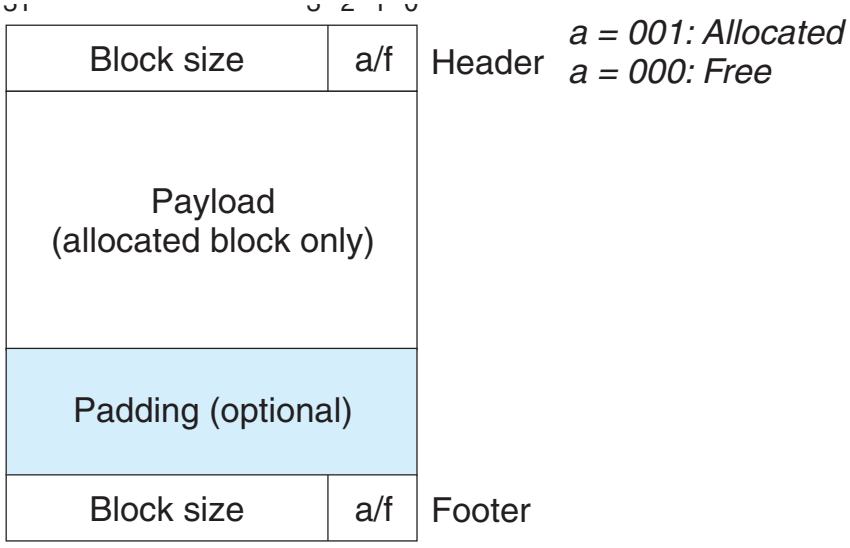


图 9.39 使用边界标记的堆块格式。

考虑分配器释放当前块时可能出现的全部情形：

1. 前一个块和下一个块都已分配。
2. 前一个块已分配，下一个块空闲。
3. 前一个块空闲，下一个块已分配。
4. 前一个块和下一个块都空闲。

图 9.40 展示了如何合并这四种情形中的每一种。

在情形 1 中，两个相邻的块都已分配，因此不能合并。所以只需把当前块的状态从已分配改为空闲。在情形 2 中，当前块与下一个块合并。当前块的头部和下一个块的脚部都会更新为当前块和下一个块的大小的总和。在情形 3 中，前一个块与当前块合并。前一个块的头部和当前块的脚部都会更新为这两个块的大小的总和。在情形 4 中，所有三个块都合并成一个空闲块，其中前一

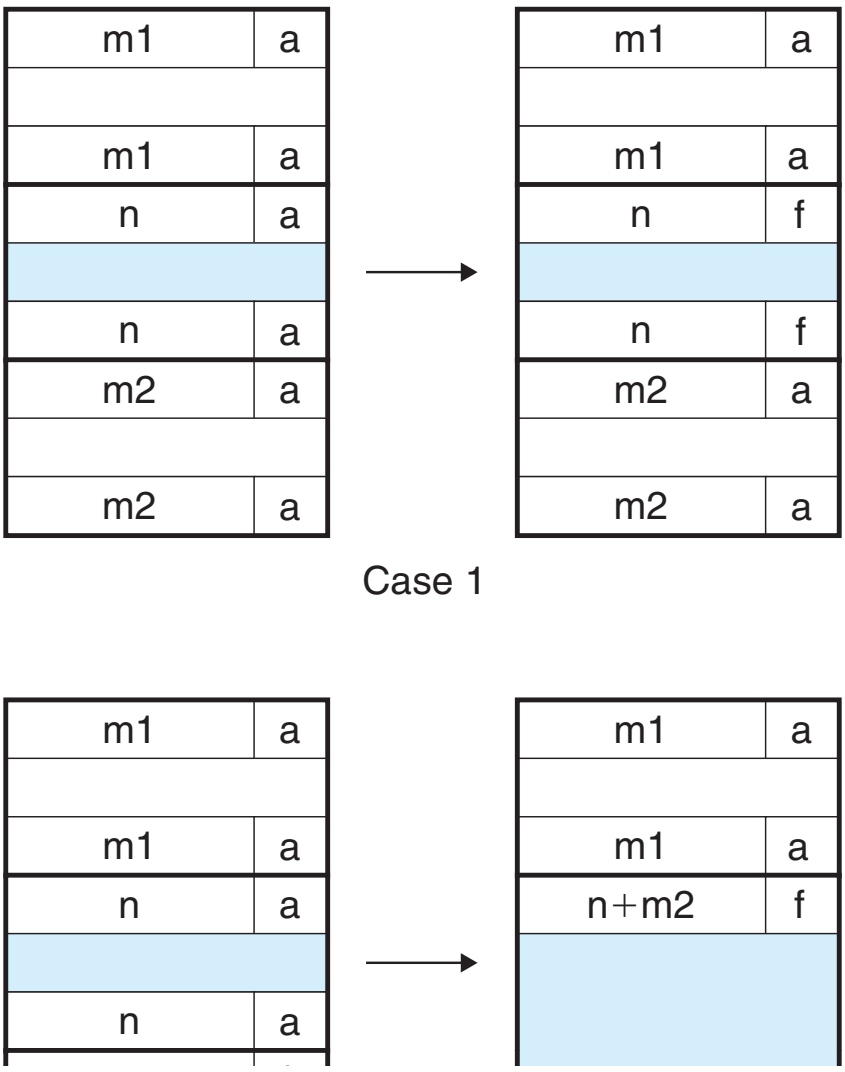


图 9.40 用边界标记进行合并。情况 1:前一块和后一块都已分配。情况 2:前一块已分配,后一块空闲。情况 3:前一块空闲,后一块已分配。情况 4:前一块和后一块都空闲。

个块的头部和下一个块的脚部都会更新为这三个块的大小的总和。在每种情形中,合并都是在常数时间内完成的。

边界标记是一个简单而优雅的思想,可以推广到许多不同类型的分配器和空闲链表组织中。不过,它也有一个潜在的缺点。如果应用程序要操纵许多小块,那么要求每个块既包含头部又包含脚部就可能带来显著的内存开销。例如,如果一个图形应用程序通过反复调用 `malloc` 和 `free` 来动态地创建和销毁图节点,而每个图节点只需要几个字的内存,那么头部和脚部就会占去每个已分配块的一半。

令人庆幸的是，有一种对边界标记的巧妙优化，可以免除已分配块的脚部。回忆一下，当我们试图把当前块与内存中的前一个块和下一个块合并时，只有在前一个块是空闲的时候，才需要用到它脚部中的大小字段。如果我们把前一个块的已分配/空闲位存放在当前块某个多余的低序位中，那么已分配块就不需要脚部了，我们可以把腾出来的这块空间用作有效载荷。不过要注意，空闲块仍然需要脚部。

练习 9.7 确定下面这些关于对齐要求和块格式的组合各自的最小块大小。假设：采用隐式空闲链表，不允许零大小的有效载荷，头部和脚部存储在 4 字节的字中。

练习 9.7 的表

对齐	已分配块	空闲块	最小块大小(字节)
单字	头部和脚部	头部和脚部	
单字	头部，但无脚部	头部和脚部	
双字	头部和脚部	头部和脚部	
双字	头部，但无脚部	头部和脚部	

9.9.12 综合：实现一个简单的分配器

构建一个分配器是一项颇具挑战性的任务。设计空间很大：在块格式和空闲链表格式，以及放置、分割和合并策略方面，都存在大量可供选择的方案。另一个挑战在于，你常常被迫在类型系统那个安全而熟悉的范围之外编程，只能依赖容易出错的指针强制类型转换和指针运算，而这正是底层系统编程的典型做法。

分配器需要的代码量虽然不算巨大，但它们十分微妙，而且容不得任何差错。熟悉 C++ 或 Java 这类高级语言的学生，初次接触到这种编程风格时，往往会撞上一堵概念上的墙。为了帮助你跨过这道坎，我们将一步步实现一个简单的分配器：它基于隐式空闲链表，采用基于边界标记的立即合并。最大块大小为 $2^{32} = 4\,\text{GB}$。这份代码是 64 位可靠的，可以在 32 位（`gcc -m32`）或 64 位（`gcc -m64`）进程中不加修改地运行。

分配器的总体设计

我们的分配器使用图 9.41 中 `memlib.c` 包所提供的一种内存系统模型。这个模型的用意是允许我们运行自己的分配器，而不去干扰现有的系统级 `malloc` 包。

```
code/vm/malloc/memlib.c
```

```
1  /* 私有全局变量 */
2  static char *mem_heap;     /* 指向堆的第一个字节 */
3  static char *mem_brk;      /* 指向堆最后一个字节加 1 的位置 */
4  static char *mem_max_addr; /* 最大合法堆地址加 1 */
5
6  /*
7   * mem_init - 初始化内存系统模型
8   */
9  void mem_init(void)
10 {
11     mem_heap = (char *)Malloc(MAX_HEAP);
12     mem_brk = (char *)mem_heap;
13     mem_max_addr = (char *)(mem_heap + MAX_HEAP);
14 }
15
16 /*
17  * mem_sbrk - sbrk 函数的简单模型。把堆扩展 incr 个字节,
18  *     并返回新区域的起始地址。在这个模型中,堆不能收缩。
19  */
20 void *mem_sbrk(int incr)
21 {
22     char *old_brk = mem_brk;
23
24     if ( (incr < 0) || ((mem_brk + incr) > mem_max_addr)) {
25         errno = ENOMEM;
26         fprintf(stderr, "ERROR: mem_sbrk failed. Ran out of
        memory...\n");
27         return (void *)-1;
28     }
29     mem_brk += incr;
30     return (void *)old_brk;
31 }
```

图 9.41 memlib.c:内存系统模型。

mem_init 函数把可供堆使用的虚拟内存建模为一个很大的、双字对齐的字节数组。mem_heap 和 mem_brk 之间的字节集中表示已分配的虚拟内存,而 mem_brk 之后的字节表示未分配的虚拟内存。分配器通过调用 mem_sbrk 函数来请求额外的堆内存。mem_sbrk 函数与系统的 sbrk 函数具有相同的接口和语义,只是它会拒绝收缩堆的请求。

分配器本身包含在一个源文件(mm.c)中,用户可以把这个文件编译并链接到自己的应用程序中。分配器向应用程序导出三个函数:

```
1  extern int mm_init(void);
2  extern void *mm_malloc (size_t size);
3  extern void mm_free (void *ptr);
```

mm_init 函数初始化分配器,成功时返回 0,否则返回 −1。mm_malloc 和 mm_free 函数与系统中的对应函数具有相同的接口和语义。分配器使用图

9.39 中所示的块格式。最小块大小是 16 字节。空闲链表被组织成隐式空闲链表,其不变形态如图 9.42 所示。

第一个字是一个不使用的、对齐到双字边界的填充字。填充之后是一个特殊的序言块 (prologue block), 它是一个 8 字节的已分配块, 只由一个头部和一个脚部组成。序言块是在初始化时创建的, 从不释放。序言块之后是零个或多个由调用 `malloc` 或 `free` 创建的普通块。堆总是以一个特殊的结尾块(epilogue block)结束,这个块的大小为零,只由一个头部组成。序言块和结尾块都是用来消除合并过程中边界条件的技巧。分配器只用一个私有(static)全局变量(`heap_listp`),它总是指向序言块。(作为一种次要的优化,我们也可以让它指向下一个块,而不是序言块。)

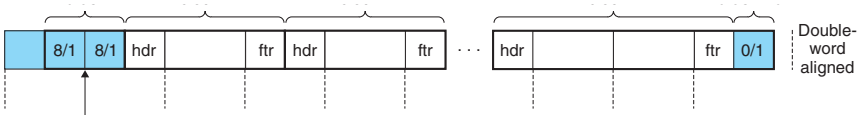


图 9.42 隐式空闲链表的不变式形式。

操作空闲链表的基本常数和宏

图 9.43 展示了我们在整个分配器代码中都会用到的一些基本常数和宏。第 2–4 行定义了一些基本的大小常数:字的大小(`WSIZE`)和双字的大小(`DSIZE`),以及初始空闲块的大小和扩展堆时使用的默认扩展量(`CHUNKSIZE`)。

code/vm/malloc/mm.c

```
1  /* 基本常数和宏 */
2  #define WSIZE       4       /* 字及头部/脚部的大小(字节数) */
3  #define DSIZE       8       /* 双字大小(字节数) */
4  #define CHUNKSIZE (1<<12) /* 按这个数量扩展堆(字节数) */
5
6  #define MAX(x, y) ((x) > (y)? (x) : (y))
7
8  /* 将一个大小和已分配位打包进一个字 */
9  #define PACK(size, alloc) ((size) | (alloc))
10
11 /* 在地址 p 处读写一个字 */
12 #define GET(p)       (*(unsigned int *)(p))
13 #define PUT(p, val) (*(unsigned int *)(p) = (val))
14
15 /* 从地址 p 处读取大小和已分配的域 */
16 #define GET_SIZE(p)  (GET(p) & ~0x7)
17 #define GET_ALLOC(p) (GET(p) & 0x1)
18
19 /* 给定块指针 bp,计算其头部和脚部的地址 */
20 #define HDRP(bp)       ((char *)(bp) - WSIZE)
21 #define FTRP(bp)       ((char *)(bp) + GET_SIZE(HDRP(bp)) - DSIZE)
22
```

```
23 /* 给定块指针 bp,计算下一个块和前一个块的地址 */
24 #define NEXT_BLKP(bp) ((char *)(bp) + GET_SIZE(((char *)(bp) -
        WSIZE)))
25 #define PREV_BLKP(bp) ((char *)(bp) - GET_SIZE(((char *)(bp) -
        DSIZE)))
```

图 9.43 操作空闲链表的基本常数和宏。

操作空闲链表中的头部和脚部可能相当麻烦,因为它需要大量使用强制类型转换和指针运算。因此,我们发现定义一小组用于访问和遍历空闲链表的宏是很有帮助的(第 9–25 行)。PACK 宏(第 9 行)把一个大小和一个已分配位组合起来,返回一个可以存入头部或脚部的值。

GET 宏(第 12 行)读取并返回参数 p 所引用的那个字。这里的强制类型转换是关键的。参数 p 通常是一个 (void *) 指针,不能对它直接解引用。类似地,PUT 宏(第 13 行)把 val 存储在参数 p 所指向的那个字里。

GET_SIZE 和 GET_ALLOC 宏(第 16–17 行)分别返回地址 p 处的头部或脚部中的大小和已分配位。其余的宏操作的是块指针(记作 bp),块指针指向有效载荷的第一个字节。给定块指针 bp,HDRP 宏和 FTRP 宏(第 20–21 行)分别返回指向块头部的指针和指向块脚部的指针。NEXT_BLKP 宏和 PREV_BLKP 宏(第 24–25 行)分别返回下一个块和前一个块的块指针。

这些宏可以用多种方式组合起来以操纵空闲链表。例如,给定一个指向当前块的指针 bp,我们就可以用下面这行代码来确定内存中下一个块的大小:

```
size_t size = GET_SIZE(HDRP(NEXT_BLKP(bp)));
```

创建初始空闲链表

在调用 mm_malloc 或 mm_free 之前,应用程序必须调用 mm_init 函数来初始化堆(图 9.44)。

mm_init 函数从内存系统获得 4 个字并将其初始化,从而创建出空的空闲链表(第 4–10 行)。然后它调用 extend_heap 函数(图 9.45),这个函数按 CHUNKSIZE 字节的大小扩展堆,并创建出初始空闲块。此时,分配器已初始化完毕,随时可以接受来自应用程序的分配和释放请求了。

code/vm/malloc/mm.c

```
1  int mm_init(void)
2  {
3      /* 创建初始的空堆 */
4      if ((heap_listp = mem_sbrk(4*WSIZE)) == (void *)-1)
5          return -1;
6      PUT(heap_listp, 0);                          /* 对齐填充 */
7      PUT(heap_listp + (1*WSIZE), PACK(DSIZE, 1)); /* 序言头部 */
8      PUT(heap_listp + (2*WSIZE), PACK(DSIZE, 1)); /* 序言脚部 */
```

```
9       PUT(heap_listp + (3*WSIZE), PACK(0, 1));     /* 结尾头部 */
10      heap_listp += (2*WSIZE);
11
12      /* 用一个 CHUNKSIZE 字节的空闲块扩展这个空堆 */
13      if (extend_heap(CHUNKSIZE/WSIZE) == NULL)
14          return -1;
15      return 0;
16  }
```

图 9.44 `mm_init` 创建一个带初始空闲块的堆。

```
code/vm/malloc/mm.c

1   static void *extend_heap(size_t words)
2   {
3       char *bp;
4       size_t size;
5
6       /* 分配偶数个字以保持对齐 */
7       size = (words % 2) ? (words+1) * WSIZE : words * WSIZE;
8       if ((long)(bp = mem_sbrk(size)) == -1)
9           return NULL;
10
11      /* 初始化空闲块的头部/脚部和结尾头部 */
12      PUT(HDRP(bp), PACK(size, 0));         /* 空闲块头部 */
13      PUT(FTRP(bp), PACK(size, 0));         /* 空闲块脚部 */
14      PUT(HDRP(NEXT_BLKP(bp)), PACK(0, 1)); /* 新的结尾头部 */
15
16      /* 如果前一个块是空闲的就进行合并 */
17      return coalesce(bp);
18  }
```

图 9.45 `extend_heap` 函数用一个新空闲块扩展堆。

`extend_heap` 函数会在两种不同的情形下被调用：(1) 堆被初始化之后；(2) `mm_malloc` 找不到一个合适的空闲块的时候。为了保持对齐，`extend_heap` 函数会把所请求的大小向上舍入到 2 字（8 字节）的最近倍数，然后向内存系统请求额外的堆空间（第 7–9 行）。

`extend_heap` 函数余下的部分（第 12–17 行）有点微妙。堆从一个双字对齐的边界开始，而且每一次对 `extend_heap` 的调用都返回一个大小为双字整数倍的块。因此，每一次对 `mem_sbrk` 的调用返回的都是紧跟在结尾块的头部后面的、双字对齐的一段内存。这个头部就成了新空闲块的头部（第 12 行），这一段内存的最后一个字就成了新的结尾块头部（第 14 行）。最后，在很可能出现的、前面的堆是以一个空闲块结束的情况下，我们就调用 `coalesce` 函数来合并这两个空闲块，并返回合并后的块的块指针（第 17 行）。

释放与合并块

应用程序通过调用 mm_free 函数(图 9.46)来释放先前已分配的块。这个函数会释放所请求的块(bp),然后用 9.9.11 节所描述的边界标记合并技术合并相邻的空闲块。coalesce 辅助函数中的代码是图 9.40 所述四种情形的一个直截了当的实现。其中有一个方面有些微妙。我们所选择的空闲链表格式——其序言块和结尾块总是被标记为已分配——允许我们忽略请求块 bp 位于堆开头或堆末尾这些潜在的麻烦边界条件。如果没有这些特殊的块,代码就会更杂乱、更容易出错、也更慢,因为每次释放请求我们都得去检查这些不常见的边界条件。

分配块

应用程序通过调用 mm_malloc 函数(图 9.47)来请求一块 size 字节的内存。在排除伪请求之后,分配器必须调整所请求的块大小,为头部和脚部留出空间,并满足双字对齐的要求。第 12–13 行强制执行 16 字节的最小块大小:8 字节用来满足对齐要求,另外 8 字节用来支付头部和脚部的开销。对于超过 8 字节的请求(第 15 行),一般规则是加上开销字节,然后向上舍入到 8 字节的最近倍数。

一旦分配器调整好了所请求的大小,它就搜索空闲链表,寻找一个合适的空闲块(第 18 行)。如果有放得下的块,分配器就放置请求的块,并且可选地分割出多余的部分(第 19 行),然后返回这个新分配块的地址。

如果分配器找不到放得下的块,它就扩展堆,创建一个新的空闲块(第 24–26 行),把请求的块放进这个新的空闲块里,可选地分割这个块(第 27 行),然后返回一个指向新分配块的指针。

code/vm/malloc/mm.c

```
1  void mm_free(void *bp)
2  {
3      size_t size = GET_SIZE(HDRP(bp));
4
5      PUT(HDRP(bp), PACK(size, 0));
6      PUT(FTRP(bp), PACK(size, 0));
7      coalesce(bp);
8  }
9
10 static void *coalesce(void *bp)
11 {
12     size_t prev_alloc = GET_ALLOC(FTRP(PREV_BLKP(bp)));
13     size_t next_alloc = GET_ALLOC(HDRP(NEXT_BLKP(bp)));
14     size_t size = GET_SIZE(HDRP(bp));
15
16     if (prev_alloc && next_alloc) {                 /* 情形 1 */
17         return bp;
```

```
18     }
19
20     else if (prev_alloc && !next_alloc) {      /* 情形 2 */
21         size += GET_SIZE(HDRP(NEXT_BLKP(bp)));
22         PUT(HDRP(bp), PACK(size, 0));
23         PUT(FTRP(bp), PACK(size,0));
24     }
25
26     else if (!prev_alloc && next_alloc) {      /* 情形 3 */
27         size += GET_SIZE(HDRP(PREV_BLKP(bp)));
28         PUT(FTRP(bp), PACK(size, 0));
29         PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
30         bp = PREV_BLKP(bp);
31     }
32
33     else {                                      /* 情形 4 */
34         size += GET_SIZE(HDRP(PREV_BLKP(bp))) +
35             GET_SIZE(FTRP(NEXT_BLKP(bp)));
36         PUT(HDRP(PREV_BLKP(bp)), PACK(size, 0));
37         PUT(FTRP(NEXT_BLKP(bp)), PACK(size, 0));
38         bp = PREV_BLKP(bp);
39     }
40     return bp;
41 }
```

图 9.46 mm_free 函数释放一个块，并采用基于边界标记的合并技术，在常数时间内把它与任何相邻的空闲块合并起来。

code/vm/malloc/mm.c

```
1  void *mm_malloc(size_t size)
2  {
3      size_t asize;      /* 调整后的块大小 */
4      size_t extendsize; /* 如果没有合适的块,用于扩展堆的数量 */
5      char *bp;
6
7      /* 忽略伪请求 */
8      if (size == 0)
9          return NULL;
10
11     /* 调整块大小,把开销和对齐要求包括进来 */
12     if (size <= DSIZE)
13         asize = 2*DSIZE;
14     else
15         asize = DSIZE * ((size + (DSIZE) + (DSIZE-1)) / DSIZE);
16
17     /* 搜索空闲链表,寻找一个合适的块 */
18     if ((bp = find_fit(asize)) != NULL) {
19         place(bp, asize);
20         return bp;
21     }
22
23     /* 没有找到合适的块。获取更多内存,并放置这个块 */
24     extendsize = MAX(asize,CHUNKSIZE);
```

```
25     if ((bp = extend_heap(extendsize/WSIZE)) == NULL)
26         return NULL;
27     place(bp, asize);
28     return bp;
29 }
```

图 9.47 mm_malloc 函数从空闲链表中分配一个块。

练习 9.8 为 9.9.12 节中描述的简单分配器实现一个 find_fit 函数。

```
static void *find_fit(size_t asize)
```

你的解答应该对隐式空闲链表执行一次首次适配搜索。

练习 9.9 为示例分配器实现一个 place 函数。

```
static void place(void *bp, size_t asize)
```

你的解答应该把请求的块放在空闲块的起始位置，只有当剩余部分的大小大于或等于最小块大小时才进行分割。

9.9.13 显式空闲链表

隐式空闲链表为我们提供了一种简单方法，可以用来介绍分配器的一些基本概念。然而，因为块的分配时间与堆块的总数成线性关系，隐式空闲链表并不适合通用的分配器（不过，对于一种堆块数目事先已知且很少的专用分配器来说，它也许就够了）。

一个更好的方法是把空闲块组织成某种显式的数据结构。由于根据定义空闲块的身体不是程序所需要的，实现这种数据结构的指针就都可以存放在空闲块的身体内。例如，可以在每个空闲块中包含一个前驱（pred）指针和一个后继（succ）指针，把堆组织成一个双向空闲链表，如图 9.48 所示。

用双向链表而不是隐式空闲链表，使得首次适配的分配时间从与块总数成线性关系降低到与空闲块数成线性关系。然而，释放一个块的时间既可以是线性的，也可以是常数，这取决于我们为空闲链表中块的排序所选择的策略。

一种方法是以后进先出（LIFO）的顺序来维护这个链表，将新释放的块插入到链表的起始处。使用 LIFO 排序和首次适配的放置策略时，分配器会先检查最近使用过的块。在这种情形下，释放一个块可以在常数时间内完成。如果使用了边界标记，那么合并也可以在常数时间内完成。

另一种方法是以地址顺序维护链表，即链表中每个块的地址都小于其后继块的地址。在这种情形下，释放一个块需要一次线性时间的搜索来定位合适的前驱。其代价是，与 LIFO 顺序的首次适配相比，按地址顺序的首次适配享有更好的内存利用率，接近最佳适配的利用率。

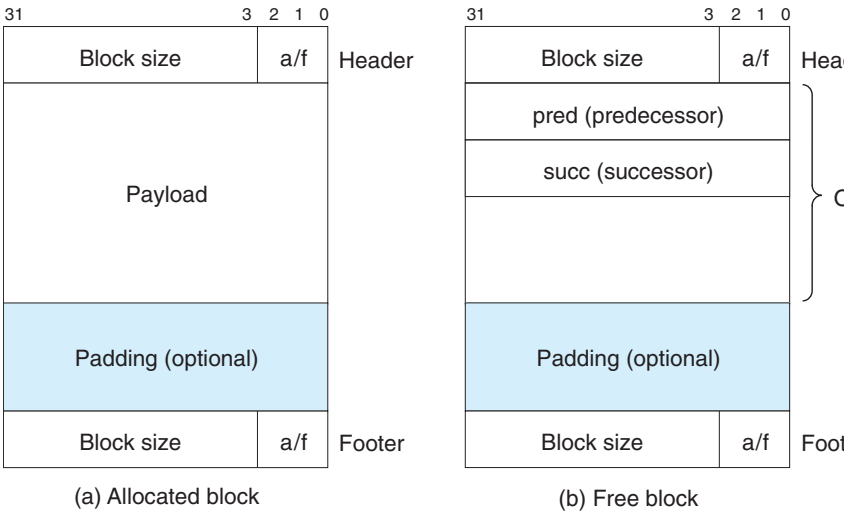


Figure 9.48 Format of heap blocks that use doubly linked free lists.

图 9.48

显式链表的总体缺点是空闲块必须足够大，以便包含所有必需的指针，以及头部（可能还有脚部）。这会导致更大的最小块大小，并加大内部碎片的可能性。

9.9.14 分离的空闲链表

正如我们已经看到的，使用单一空闲块链表的分配器需要花费与空闲块数成线性关系的时间来分配一个块。一种流行的减少分配时间的方法（一般称为分离存储（segregated storage））是维护多个空闲链表，其中每个链表持有的块大小大致相同。一般思想是把所有可能的块大小集合划分为等价类，称为大小类（size class）。定义大小类的方式有很多种。例如，我们可以按 2 的幂来划分块大小：

$$\{1\}, \{2\}, \{3, 4\}, \{5–8\}, \ldots, \{1{,}025–2{,}048\}, \{2{,}049–4{,}096\}, \{4{,}097–\infty\}$$

或者，我们可以让小块各自作为独立的大小类，而按 2 的幂来划分大块：

$$\{1\}, \{2\}, \{3\}, \ldots, \{1{,}023\}, \{1{,}024\}, \{1{,}025–2{,}048\}, \{2{,}049–4{,}096\}, \{4{,}097–\infty\}$$

分配器维护一个空闲链表数组，每个大小类有一个空闲链表，按大小递增排序。当分配器需要一个大小为 n 的块时，它就搜索相应的空闲链表。如果找不到放得下的块，它就搜索下一个链表，以此类推。

动态存储分配的文献描述了分离存储的几十种变体，它们在如何定义大小类、何时执行合并、何时向操作系统请求额外的堆内存、是否允许分割等方面各不相同。为了让你了解可能有哪些做法，我们将描述两种基本的方法：简单分离存储和分离适配。

简单分离存储

在简单分离存储中，每个大小类的空闲链表包含相同大小的块，每个块的大小都等于该大小类中最大元素的大小。例如，如果某个大小类定义为{17–32}，那么这个类的空闲链表就完全由大小为 32 的块组成。

要分配某个给定大小的块，我们就检查相应的空闲链表。如果链表非空，我们就直接把其中的第一个块整个分配出去。空闲块永远不会被分割以满足分配请求。如果链表为空，分配器就向操作系统请求一块固定大小的额外内存（通常是页大小的倍数），把这块内存切成大小相同的块，并将这些块链接起来形成新的空闲链表。要释放一个块时，分配器只需把这个块插到相应空闲链表的前端。

这个简单的方案有很多优点。分配块和释放块都是快速的常数时间操作。此外，同一大小的块组成一个组块、不分割、也不用合并，这几点的结合意味着每个块本身的内存开销都非常小。因为同一个组块（chunk）中只有相同大小的块，所以一个已分配块的大小可以从它的地址推断出来。因为没有合并，所以已分配块的头部不需要已分配/空闲位。这样一来，已分配块不需要头部，而且因为没有合并，它们也不需要任何脚部。由于分配和释放操作都是在空闲链表的前端插入和删除块，所以这个链表只需要单向链接，而不必双向链接。归根结底，任何块中唯一要求的字段就是每个空闲块中一个字的 `succ` 指针，因此最小块大小只有一个字。

一个显著的缺点是，简单分离存储容易受到内部碎片和外部碎片的影响。空闲块从不分割，所以内部碎片是可能发生的。更糟糕的是，某些引用模式会因为空闲块从不合并而导致极端的外部碎片（练习 9.10）。

练习 9.10 描述一种会导致基于简单分离存储的分配器产生严重外部碎片的引用模式。

分离适配

使用这种方法时，分配器维护一个空闲链表的数组。每个空闲链表与一个大小类相关联，并被组织成某种显式链表或隐式链表。每个链表包含的块可能有不同的大小，但其大小都属于这个大小类。分离适配分配器有很多变体。这里我们描述一种简单的版本。

要分配一个块，我们先确定该请求的大小类，然后在相应的空闲链表中做首次适配搜索，寻找一个放得下的块。如果找到了一个，我们就（可选地）分割它，并把剩下的碎片插入相应的空闲链表。如果找不到放得下的块，我们就搜索下一个更大大小类的空闲链表。如此重复，直到找到一个放得下的块。如果所有空闲链表都没有放得下的块，那么我们就向操作系统请求额外的堆内存，从这些新的堆内存中分配该块，并把剩余部分放到相应的大小类中。要释放一个块时，我们先合并它，然后把结果放到相应的空闲链表中。

分离适配方法是生产级分配器（例如 C 标准库中提供的 GNU `malloc` 包）所青睐的一种选择，因为它既快又高效地利用内存。搜索时间得以减少，因为搜索只限于堆的特定部分，而不是整个堆。内存利用率也能得到改善，这缘于一个有趣的事实：对分离空闲链表做一次简单的首次适配搜索，其结果十分近似于对整个堆做的一次最佳适配搜索。

伙伴系统

伙伴系统（buddy system）是分离适配的一种特殊情形，其中每个大小类都是 2 的幂。基本思想是：给定一个 2^m 字的堆，我们为每个块大小 $2^k (0 \leq k \leq m)$ 维护一个单独的空闲链表。被请求的块大小向上舍入到 2 的最近的幂。最初，有一个大小为 2^m 字的空闲块。

要分配一个大小为 2^k 的块，我们找到第一个大小为 2^j 的可用块，$k \leq j \leq m$。如果 $j = k$，那么我们就完成了。否则，我们就递归地把这个块对半分割，直到 $j = k$。在执行这种分割时，剩下的每一半（称为伙伴）都被放到相应的空闲链表上。要释放一个大小为 2^k 的块时，我们不断地与空闲的伙伴合并。一旦遇到一个已分配的伙伴，我们就停止合并。

关于伙伴系统有一个关键的事实：给定一个块的地址和大小，很容易计算出它伙伴的地址。例如，一个大小为 32 字节、地址为下式的块

```
xxx...x00000
```

它的伙伴在如下地址处

```
xxx...x10000
```

换句话说，一个块及其伙伴的地址恰好在一位上不同。

伙伴系统分配器的主要优点是它的快速搜索和合并。主要缺点是块大小必须是 2 的幂这一要求，可能导致大量的内部碎片。因此，伙伴系统分配器并

不适合通用的负载。不过，对于某些特定应用的负载，其中块大小已知是 2 的幂，伙伴系统分配器有一定的吸引力。

9.10 垃圾收集

对于诸如 C `malloc` 包这样的显式分配器，应用程序通过调用 `malloc` 和 `free` 来分配和释放堆块。释放任何不再需要的已分配块是应用程序的责任。

未能释放已分配的块是一种常见的编程错误。例如，考虑下面这个 C 函数，它在处理过程中分配了一块临时存储空间：

```
1  void garbage()
2  {
3      int *p = (int *)Malloc(15213);
4
5      return; /* 数组 p 在此时已经成了垃圾 */
6  }
```

既然程序不再需要 `p`，本应在 `garbage` 返回之前释放它。可惜，程序员忘记释放这个块了。它在程序的整个生命周期内都保持已分配状态，白白占据着本可用来满足后续分配请求的堆空间。

垃圾收集器（garbage collector）是一种动态存储分配器，它会自动释放程序不再需要的已分配块。这些块称为垃圾（garbage），201c 垃圾收集器 201d 由此得名。自动回收堆存储的过程称为垃圾收集（garbage collection）。在一个支持垃圾收集的系统中，应用程序显式地分配堆块，但从不显式地释放它们。在 C 程序的情形中，应用程序调用 `malloc` 却从不调用 `free`。垃圾收集器会周期性地识别出垃圾块，并恰当地调用 `free` 把这些块放回空闲链表。

垃圾收集可以追溯到 20 世纪 60 年代初期 John McCarthy 在 MIT 开发的 Lisp 系统。它是 Java、ML、Perl 和 Mathematica 这类现代语言系统的一个重要组成部分，并且至今仍是一个活跃而重要的研究领域。相关文献描述了数量惊人的垃圾收集方法。我们将把讨论限定在 McCarthy 最初的 `Mark&Sweep`（标记与清除）算法上，这个算法之所以有趣，是因为它可以构建在现有的 `malloc` 包之上，为 C 和 C++ 程序提供垃圾收集。

9.10.1 垃圾收集器基础

垃圾收集器把内存看成一张有向可达图，其形态如图 9.49 所示。图的节点被分成一组根节点和一组堆节点。每个堆节点对应于堆中的一个已分配块。有向边 $p \to q$ 表示块 p 中的某个位置指向块 q 中的某个位置。根节点对应于不在堆中、却包含指向堆内指针的位置。这些位置可以是寄存器、栈中的变量，也可以是虚拟内存读写数据区域中的全局变量。

如果存在一条从任何根节点出发、到达节点 p 的有向路径，我们就说节点 p 是可达的（reachable）。任一时刻那些不可达的节点，都对应于应用程序再也无法使用的垃圾。垃圾收集器的职责是维护可达图的某种表示，并周期性地回收不可达的节点，即释放它们并把它们送回空闲链表。

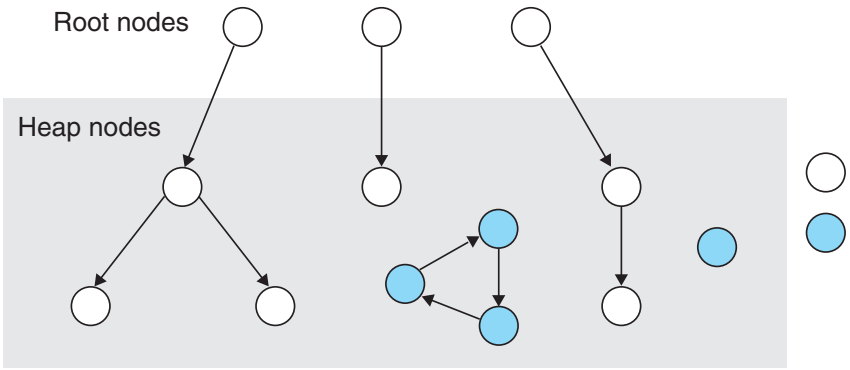


图 9.49 垃圾收集器把内存视为一个有向图。

ML 和 Java 这样的语言对应用程序创建和使用指针的方式施加了严格的控制，这类语言的垃圾收集器可以维护可达性图的精确表示，因而能够回收全部垃圾。然而，C 和 C++ 这样的语言的收集器一般无法维护可达性图的精确表示。这样的收集器称为保守（conservative）的垃圾收集器。它们之所以保守，其含义在于：每个可达的块都被正确地识别为可达，而某些不可达的节点可能会被错误地识别为可达。

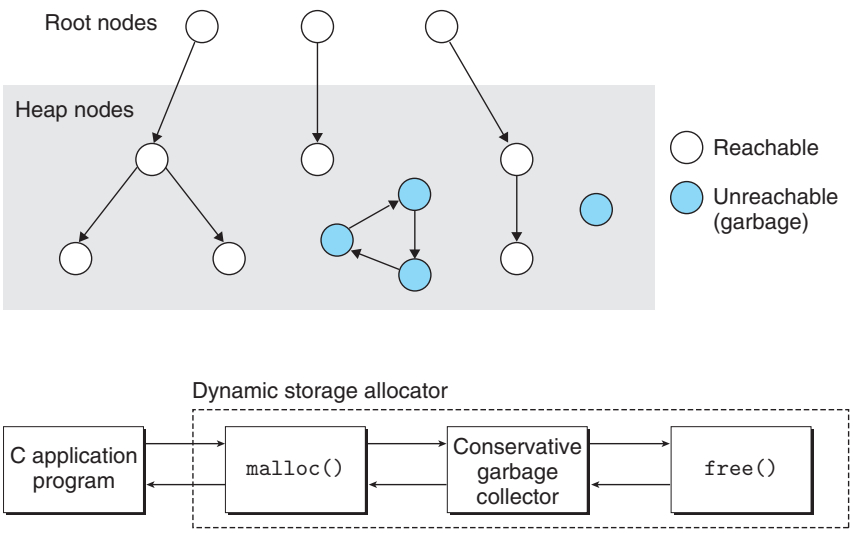


图 9.50 将一个保守的垃圾收集器与一个 C 的 `malloc` 包集成起来。

收集器可以按需提供服务，也可以作为与应用程序并行的独立线程运行，持续地更新可达性图并回收垃圾。例如，考虑如何把一个用于 C 程序的保守收集器并入现有的 malloc 包，如图 9.50 所示。

应用程序无论何时需要堆空间，都以通常的方式调用 malloc。如果 malloc 找不到一个适合的空闲块，那么它就调用垃圾收集器，希望能把一些垃圾回收到空闲链表中。收集器识别出垃圾块，并通过调用 free 函数把它们送回堆中。这里的关键想法是，收集器代替应用程序调用 free。对收集器的调用返回之后，malloc 会再次尝试寻找一个适合的空闲块。如果还是找不到，那么它可以向操作系统请求更多的内存。最终，malloc 返回一个指向请求块的指针（如果成功），或者返回 NULL 指针（如果不成功）。

9.10.2 Mark&Sweep 垃圾收集器

Mark&Sweep 垃圾收集器由标记（mark）阶段和后继的清扫（sweep）阶段组成：标记阶段标记根节点所有可达且已分配的后代，清扫阶段释放每个未标记的已分配块。通常，用块头部中一个富余的低位来指示块是否已被标记。

```
(a) mark 函数

void mark(ptr p) {
    if ((b = isPtr(p)) == NULL)
      return;
    if (blockMarked(b))
      return;
    markBlock(b);
    len = length(b);
    for (i=0; i < len; i++)
      mark(b[i]);
    return;
}

(b) sweep 函数

void sweep(ptr b, ptr end) {
    while (b < end) {
      if (blockMarked(b))
          unmarkBlock(b);
      else if (blockAllocated(b))
          free(b);
      b = nextBlock(b);
    }
    return;
}
```

图 9.51 mark 函数和 sweep 函数的伪代码。

我们对 Mark&Sweep 的描述将假设存在下列函数，其中 ptr 定义为 typedef void *ptr：

- `ptr isPtr(ptr p)`。如果 `p` 指向某个已分配块中的某个字,返回一个指向该块开头的指针 `b`;否则返回 `NULL`。
- `int blockMarked(ptr b)`。如果块 `b` 已经被标记,返回 true。
- `int blockAllocated(ptr b)`。如果块 `b` 已分配,返回 true。
- `void markBlock(ptr b)`。标记块 `b`。
- `int length(ptr b)`。返回块 `b` 以字为单位的长度(不计头部)。
- `void unmarkBlock(ptr b)`。把块 `b` 的状态从已标记改为未标记。
- `ptr nextBlock(ptr b)`。返回块 `b` 在堆中的后继。

标记阶段对每个根节点都调用一次图 9.51(a) 所示的 `mark` 函数。如果 `p` 并不指向一个已分配且未标记的堆块,`mark` 函数就立即返回。否则,它标记这个块,并对块中的每个字递归地调用自身。`mark` 函数的每次调用都会标记某个根节点的所有未标记且可达的后代。在标记阶段结束时,任何没有被标记的已分配块都保证是不可达的,因而是在清扫阶段中可以回收的垃圾。

清扫阶段是对图 9.51(b) 所示的 `sweep` 函数的一次调用。`sweep` 函数遍历堆中的每个块,释放它所遇到的每个未标记的已分配块(也就是垃圾)。

图 9.52 给出了 Mark&Sweep 在一个小堆上的图形化解释。块的边界用粗线表示。每个方块对应内存中的一个字。每个块都有一个单字的头部,它要么已标记,要么未标记。

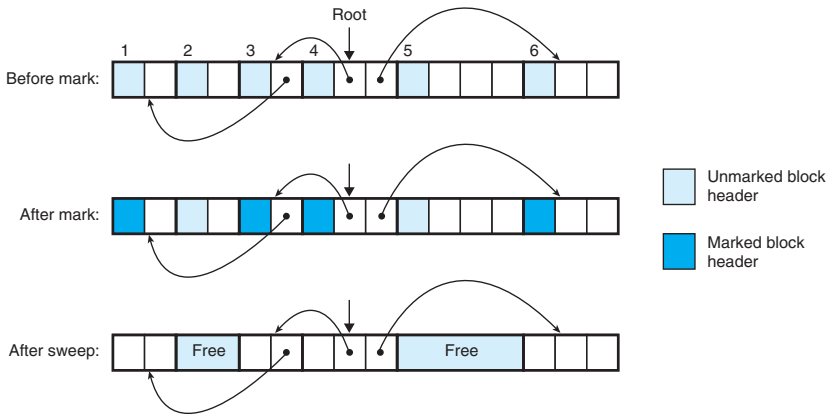


图 9.52 标记和清除(Mark&Sweep)的例子。注意,本例中的箭头表示内存引用,而不是空闲链表指针。

开始时，图 9.52 中的堆由六个已分配块组成，每个块都未被标记。块 3 中有一个指向块 1 的指针。块 4 中有指向块 3 和块 6 的指针。根指向块 4。标记阶段之后，块 1、3、4 和 6 被标记，因为它们从根是可达的。块 2 和 5 未被标记，因为它们不可达。清扫阶段之后，那两个不可达的块被回收到空闲链表中。

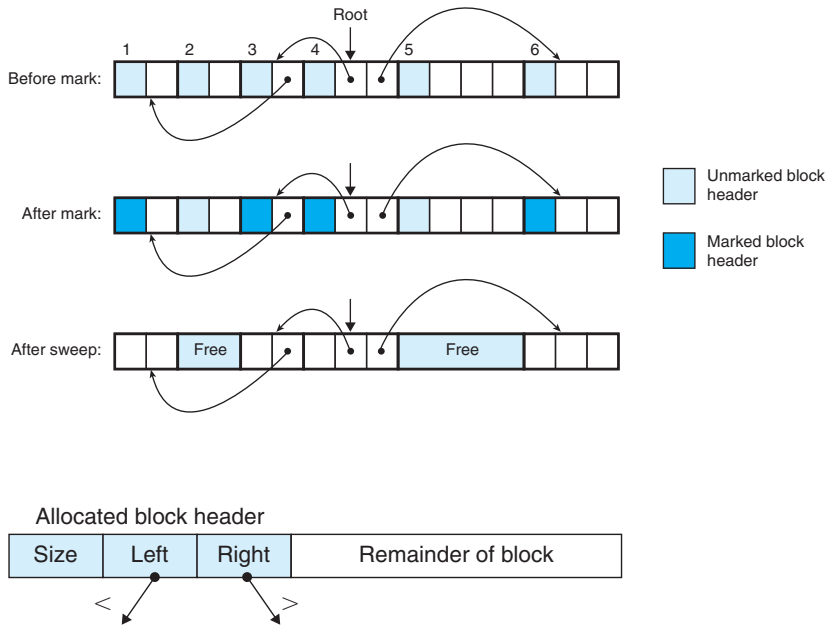


图 9.53 已分配块平衡树中的左指针和右指针。

9.10.3 C 程序的保守 Mark&Sweep

对 C 程序的垃圾收集而言，Mark&Sweep 是一种合适的办法，因为它原地工作，不移动任何块。不过，C 语言给 `isPtr` 函数的实现带来了一些有趣的挑战。

首先，C 语言不为内存位置附加任何类型信息。因此，`isPtr` 没有显而易见的办法判断它的输入参数 `p` 是不是指针。其次，即使我们能够知道 `p` 是一个指针，`isPtr` 也没有显而易见的办法判断 `p` 是否指向某个已分配块载荷中的某个位置。

解决后一个问题的一种办法是，把已分配块的集合维护成一棵平衡二叉树，这棵树保持这样一个不变式：左子树中的所有块都位于较小的地址处，而右子树中的所有块都位于较大的地址处。如图 9.53 所示，这要求每个已分配块的头部中有两个附加字段（`left` 和 `right`），每个字段都指向某个已分配块的头

部。`isPtr(ptr p)` 函数利用这棵树对已分配块进行二分查找。在每一步中，它都依靠块头部中的 `size` 字段来判断 `p` 是否落在这个块的范围内。

平衡树方法在如下意义上是正确的：它保证把所有从根出发可达的节点都标记出来。这是一个必要的保证，因为应用程序的用户肯定不希望自己的已分配块被过早地归还给空闲链表。但它在如下意义上又是保守的：它可能把实际上不可达的块也标记为可达，因此可能无法释放某些垃圾。虽然这并不影响应用程序的正确性，却会造成不必要的外部碎片。

C 程序的 Mark&Sweep 收集器必须保守，根本原因在于 C 语言不为内存位置附加类型信息。因此，像 `int` 或 `float` 这样的标量可以伪装成指针。例如，假设某个可达的已分配块的载荷中有一个 `int`，其值碰巧对应于另一个已分配块 b 的载荷中的某个地址。收集器无从推断这个数据真的是一个 `int` 而不是指针。于是，分配器只能保守地把块 b 标记为可达的，虽然实际上它可能并非如此。

9.11 C 程序中常见的与内存有关的错误

对 C 程序员来说，管理和使用虚拟内存是一件困难且容易出错的工作。与内存有关的错误属于最令人害怕的一类，因为距离错误的源头，无论时间上还是空间上，它们都常常在很远的地方才显现出来。把错误的数据写到错误的位置，你的程序可能还会运行几个小时，才最终在程序中相距遥远的地方失败。作为虚拟内存讨论的收尾，我们来看看一些常见的与内存有关的错误。

9.11.1 解引用坏指针

正如我们在 9.7.2 节中学到的，进程的虚拟地址空间中存在大片空洞，它们没有映射到任何有意义的数据。如果我们试图解引用一个指向这些空洞的指针，操作系统就会以段异常终止我们的程序。此外，虚拟内存的某些区域是只读的。试图向这样的区域写入，程序就会因保护异常而终止。

解引用坏指针的一个常见例子是经典的 `scanf` 错误。假设我们想用 `scanf` 从 stdin 读入一个整数，存放到一个变量中。正确的做法是向 `scanf` 传递一个格式串和这个变量的地址：

```
scanf("%d", &val)
```

然而，新学的 C 程序员（有经验的也一样！）很容易把 val 的内容而不是它的地址传进去：

```
scanf("%d", val)
```

在这种情况下，scanf 会把 val 的内容解释为一个地址，并试图向那个位置写入一个字。最好的情况，程序立即因异常而终止。最坏的情况，val 的内容对应虚拟内存中某个有效的读/写区域，于是我们覆写了内存，通常会在很久之后带来灾难性的、令人摸不着头脑的后果。

9.11.2 读取未初始化的内存

虽然 bss 中的存储位置（例如未初始化的 C 全局变量）总是被加载器初始化为零，但堆内存并不是这样。一个常见的错误是以为堆内存已被初始化为零：

```
1  /* 返回 y = Ax */
2  int *matvec(int **A, int *x, int n)
3  {
4      int i, j;
5
6      int *y = (int *)Malloc(n * sizeof(int));
7
8      for (i = 0; i < n; i++)
9          for (j = 0; j < n; j++)
10             y[i] += A[i][j] * x[j];
11     return y;
12 }
```

在这个例子中，程序员错误地假定向量 y 已经被初始化为零。正确的实现应当显式地把 y[i] 置零，或者使用 calloc。

9.11.3 允许栈缓冲区溢出

正如我们在 3.10.3 节中所见，如果一个程序不检查输入串的大小就向栈上的目标缓冲区写入，那么它就有缓冲区溢出错误。例如，下面的函数就有缓冲区溢出错误，因为 gets 函数会向缓冲区复制任意长度的字符串。要修正这个问题，我们需要使用 fgets 函数，它限制了输入串的大小。

```
1  void bufoverflow()
2  {
3      char buf[64];
4
5      gets(buf); /* 这里就是栈缓冲区溢出错误 */
6      return;
7  }
```

9.11.4 假定指针与它们所指向的对象大小相同

一个常见的错误是假定指向对象的指针与它所指向的对象大小相同：

```
1  /* 创建一个 n×m 的数组 */
2  int **makeArray1(int n, int m)
3  {
```

```
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int));
6
7      for (i = 0; i < n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }
```

这里的意图是创建一个由 n 个指针组成的数组，每个指针都指向一个由 m 个 `int` 组成的数组。然而，由于程序员在第 5 行写的是 `sizeof(int)` 而不是 `sizeof(int *)`，代码实际上创建的是一个 `int` 数组。

在 `int` 和指向 `int` 的指针大小相同的机器上，这段代码会运行得很好。但如果我们在一台像 Core i7 这样指针比 `int` 大的机器上运行这段代码，那么第 7 ~ 8 行的循环就会写到 A 数组末尾之外。由于这些字中的一个很可能是已分配块的边界标签脚部，我们也许要到程序中很靠后的地方释放这个块时才会发现错误，而那时分配器中的合并代码会莫名其妙地彻底失败。这是“远距作用”式错误的一个阴险的例子，而远距作用正是与内存有关的程序错误的典型特征。

9.11.5 造成差一错误

差一错误（off-by-one error）是覆写错误的另一个常见来源：

```
1  /* 创建一个 n×m 的数组 */
2  int **makeArray2(int n, int m)
3  {
4      int i;
5      int **A = (int **)Malloc(n * sizeof(int *));
6
7      for (i = 0; i <= n; i++)
8          A[i] = (int *)Malloc(m * sizeof(int));
9      return A;
10 }
```

这是上一节中那个程序的另一个版本。这里我们在第 5 行创建了一个包含 n 个元素的指针数组，接着却在第 7 行和第 8 行中试图初始化它的 $n+1$ 个元素，在此过程中覆写了紧跟在 A 数组后面的内存。

9.11.6 引用指针而不是它所指向的对象

如果对 C 运算符的优先级和结合性不加小心，我们就会误去操作指针，而不是它所指向的对象。例如，考虑下面这个函数，它的目的是从含有 `*size` 个元素的二叉堆中删除第一项，然后对其余的 `*size - 1` 个元素重新堆化：

```
1  int *binheapDelete(int **binheap, int *size)
```

```
2  {
3      int *packet = binheap[0];
4
5      binheap[0] = binheap[*size - 1];
6      *size--; /* 这里应该是 (*size)-- */
7      heapify(binheap, *size, 0);
8      return(packet);
9  }
```

在第 6 行，本意是把 size 指针所指向的整数值减一。然而，由于一元 `--` 和 * 运算符具有相同的优先级，并且从右向左结合，第 6 行的代码实际上是把指针本身减了一，而不是把指针所指向的整数值减一。运气好的话，程序会立即崩溃。但更可能的情况是，要等到程序在执行的后期给出一个错误答案时，才会让我们挠头不解。这里的教训是：只要对优先级和结合性心存疑问，就使用括号。例如在第 6 行中，我们本应该用表达式 `(*size)--` 来清楚地表明自己的意图。

9.11.7 误解指针运算

另一个常见的错误是忘记了：对指针的算术运算是以指针所指向对象的大小为单位的，而不一定是字节。例如，下面这个函数的本意是扫描一个 `int` 数组，返回指向 val 第一次出现处的指针：

```
1  int *search(int *p, int val)
2  {
3      while (*p && *p != val)
4          p += sizeof(int); /* 应该是 p++ */
5      return p;
6  }
```

然而，由于第 4 行每次循环都把指针增加 4（一个整数的字节数），这个函数错误地只扫描了数组中每第 4 个整数。

9.11.8 引用不存在的变量

不理解栈规则的天真的 C 程序员有时会引用已经失效的局部变量，如下面这个例子所示：

```
1  int *stackref ()
2  {
3      int val;
4
5      return &val;
6  }
```

这个函数返回一个指向栈上局部变量的指针（比如说 p），然后弹出自己的栈帧。虽然 p 仍然指向一个有效的内存地址，但它不再指向有效的变量。当程

序在后面调用其他函数时，这块内存会被重用作为它们的栈帧。此后，如果程序向 *p 赋某个值，实际上修改的可能是另一个函数栈帧中的条目，其后果可能是灾难性的，而且令人迷惑。

9.11.9 引用已释放堆块中的数据

一个类似的错误是引用已经释放的堆块中的数据。考虑下面的例子，它在第 6 行分配整数数组 x，在第 10 行过早地释放块 x，然后又在第 14 行引用了它：

```
1  int *heapref(int n, int m)
2  {
3      int i;
4      int *x, *y;
5
6      x = (int *)Malloc(n * sizeof(int));
7
8      .
9      .    // 对 malloc 和 free 的其他调用在这里
10     .
11     free(x);
12
13     y = (int *)Malloc(m * sizeof(int));
14     for (i = 0; i < m; i++)
15         y[i] = x[i]++; /* 哎呀! x[i] 是空闲块中的一个字 */
16
17     return y;
18 }
```

取决于第 6 行到第 10 行之间 malloc 和 free 调用的模式，当程序在第 14 行引用 x[i] 时，数组 x 可能已经成为其他某个已分配堆块的一部分，而且可能已被覆写。与许多与内存有关的错误一样，这个错误只有在程序的后期才会显露出来，那时我们会发现 y 中的值已被破坏。

9.11.10 引入内存泄漏

内存泄漏是缓慢而无声的杀手，当程序员忘记释放已分配的块、无意中在堆里制造了垃圾时，就会出现内存泄漏。例如，下面的函数分配了一个堆块 x，然后未释放它就返回了：

```
1  void leak(int n)
2  {
3      int *x = (int *)Malloc(n * sizeof(int));
4
5      return;    /* 此时 x 已经成为垃圾 */
6  }
```

如果 leak 被频繁调用，堆就会逐渐被垃圾填满，最坏的情况是耗尽整个虚拟地址空间。对守护进程和服务器这类程序来说，内存泄漏尤其严重，因为这类程序就定义而言永远不会终止。

9.12 小结

虚拟内存是对主存的抽象。支持虚拟内存的处理器使用一种称为虚拟寻址的间接形式来引用主存。处理器产生一个虚拟地址，在把它发送给主存之前，先把它翻译成物理地址。从虚拟地址空间到物理地址空间的地址翻译，需要硬件与软件的密切合作。专用硬件使用页表来翻译虚拟地址，而页表的内容由操作系统供给。

虚拟内存提供三项重要的能力。第一，它自动地把虚拟地址空间中最近使用的内容由磁盘缓存到主存中。虚拟内存缓存中的块称为页。对磁盘上某个页的引用会触发一次缺页，把控制转移给操作系统中的一个缺页处理程序。缺页处理程序把页从磁盘复制到主存缓存中，如有必要，还把被驱逐的页写回。第二，虚拟内存简化了内存管理，这进而简化了链接、进程间的数据共享、为进程分配内存以及程序加载。最后，虚拟内存把保护位纳入每个页表项中，从而简化了内存保护。

地址翻译过程必须与系统中任何硬件高速缓存的操作整合在一起。大多数页表项都位于 L1 缓存中，不过从 L1 访问页表项的开销通常会被一个称为 TLB 的片上页表项缓存所消除。

现代系统通过把虚拟内存的块与磁盘上文件的块关联起来对虚拟内存进行初始化，这个过程称为内存映射。内存映射为共享数据、创建新进程和加载程序提供了高效的机制。应用程序可以用 `mmap` 函数手工创建和删除虚拟地址空间中的区域。不过，大多数程序依赖一个像 `malloc` 这样的动态内存分配器，它管理虚拟地址空间中称为堆的区域里的内存。动态内存分配器是应用层程序，却带有一股系统级的味道，它们直接操纵内存，很少得到类型系统的帮助。分配器有两种风格。显式分配器要求应用程序显式地释放它们的内存块。隐式分配器（垃圾收集器）自动释放任何未使用的且不可达的块。

对 C 程序员来说，管理和使用内存是困难且容易出错的工作。常见错误的例子包括：解引用坏指针、读取未初始化的内存、允许栈缓冲区溢出、假定指针和它们所指向的对象大小相同、引用指针而不是它所指向的对象、误解指针运算、引用不存在的变量，以及引入内存泄漏。

9.13 参考文献说明

Kilburn 及其同事发表了第一篇描述虚拟内存的文献 [63]。体系结构教科书中有更多关于硬件在虚拟内存中所起作用的细节 [46]。操作系统教科书中有更多关于操作系统所起作用的信息 [102, 106, 113]。Bovet 和 Cesati [11] 详细

描述了 Linux 的虚拟内存系统。Intel 公司提供了关于 IA 处理器上 32 位和 64 位地址翻译的详细文档 [52]。

Knuth 在 1968 年写下了关于存储分配的经典之作 [64]。自那时以来，这个领域涌现出了大量的工作。Wilson、Johnstone、Neely 和 Boles 撰写过一篇出色的显式分配器综述与性能评估 [118]。本书中关于不同分配器策略的吞吐量和利用率的总体评论就转述自他们的综述。Jones 和 Lins 提供了一篇关于垃圾收集的全面综述 [56]。Kernighan 和 Ritchie [61] 展示了一个基于显式空闲链表的简单分配器的完整代码，其中每个空闲块里都有一个块大小和一个后继指针。这段代码有趣的地方在于它用联合（union）消除了大量复杂的指针运算，但代价是释放操作是线性时间的（而不是常数时间）。Doug Lea 开发了一个被广泛使用的开源 `malloc` 包，称为 `dlmalloc` [67]。

9.14 家庭作业

家庭作业 9.11 ▲ 在下面这一系列问题中，你要说明 9.6.4 节中的示例内存系统是如何把虚拟地址翻译成物理地址并访问高速缓存的。对给出的虚拟地址，指出所访问的 TLB 条目、物理地址以及返回的高速缓存字节值。指出 TLB 是否不命中、是否发生缺页、是否发生高速缓存不命中。如果发生高速缓存不命中，就在“返回的高速缓存字节”一栏填入“—”。如果发生缺页，就在“PPN”一栏填入“—”，并把 C、D 两部分留空。

虚拟地址 `0x027c`

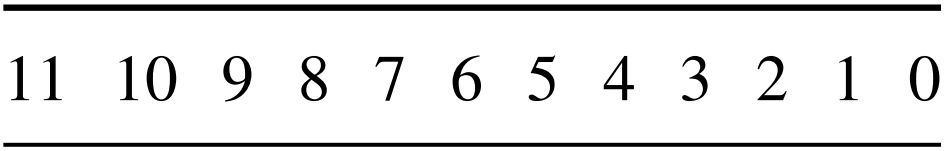
A. 虚拟地址格式

13	12	11	10	9	8	7	6	5	4	3	2	1	0

B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中?（Y/N）	
缺页?（Y/N）	
PPN	

C. 物理地址格式



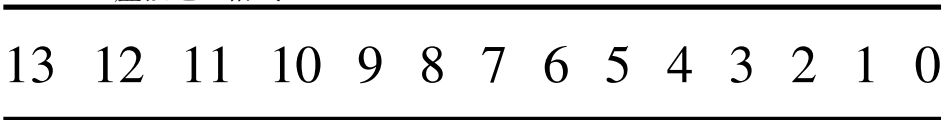
D. 物理内存访问

参数	值
字节偏移	
高速缓存索引	
高速缓存标记	
高速缓存命中?(Y/N)	
返回的高速缓存字节	

家庭作业 9.12 ▲ 对下面的地址重复家庭作业 9.11。

虚拟地址 `0x03a9`

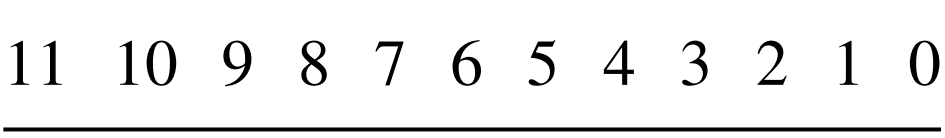
A. 虚拟地址格式



B. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中?(Y/N)	
缺页?(Y/N)	
PPN	

C. 物理地址格式



D. 物理内存访问

参数	值
字节偏移	
高速缓存索引	
高速缓存标记	
高速缓存命中?(Y/N)	
返回的高速缓存字节	

家庭作业 9.13 ▲ 对下面的地址重复家庭作业 9.11。

虚拟地址 `0x0040`

13	12	11	10	9	8	7	6	5	4	3	2	1	0

A. 地址翻译

参数	值
VPN	
TLB 索引	
TLB 标记	
TLB 命中?(Y/N)	
缺页?(Y/N)	
PPN	

B. 物理地址格式

11	10	9	8	7	6	5	4	3	2	1	0

C. 物理内存访问

参数	值
字节偏移	
高速缓存索引	
高速缓存标记	
高速缓存命中?(Y/N)	
返回的高速缓存字节	

家庭作业 9.14 ▲▲ 给定一个输入文件 `hello.txt`，它由字符串 `Hello, world!\n` 组成。编写一个 C 程序，用 `mmap` 把 `hello.txt` 的内容改为 `Jello, world!\n`。

家庭作业 9.15 ▲ 确定下面这一系列 `malloc` 请求所产生的块大小与头部值。假设:(1)分配器维持双字对齐,并使用图 9.35 中块格式的隐式空闲链表;(2)块大小向上舍入到 8 字节的最近倍数。

请求	块大小(十进制字节)	块头部(十六进制)
`malloc(4)`		
`malloc(7)`		
`malloc(19)`		
`malloc(22)`		

家庭作业 9.16 ▲ 针对下面每一种对齐要求与块格式的组合,确定最小块大小。假设:显式空闲链表,每个空闲块中有 4 字节的 `pred` 和 `succ` 指针,不允许大小为零的载荷,头部和脚部存放在 4 字节的字中。

对齐	已分配块	空闲块	最小块大小(字节)
单字	头部和脚部	头部和脚部	
单字	有头部而无脚部	头部和脚部	
双字	头部和脚部	头部和脚部	
双字	有头部而无脚部	头部和脚部	

家庭作业 9.17 ▲▲▲ 开发 9.9.12 节分配器的一个版本,执行下次适配搜索而不是首次适配搜索。

家庭作业 9.18 ▲▲▲ 9.9.12 节中的分配器要求每个块都同时具有头部和脚部,才能在常数时间内完成合并。请修改这个分配器,使空闲块需要头部和脚部,而已分配块只需要头部。

家庭作业 9.19 ▲ 下面给出与内存管理和垃圾收集有关的三组陈述。每组中只有一句是正确的。你的任务是指出哪一句是正确的。

1. (a) 在伙伴系统中,多达 50% 的空间会因内部碎片而被浪费。
   (b) 首次适配的内存分配算法比最佳适配算法慢(平均而言)。
   (c) 只有当空闲块链表按内存地址递增排序时,使用边界标签的释放才快。
   (d) 伙伴系统有内部碎片的问题,但没有外部碎片的问题。
2. (a) 在按块大小递减排序的空闲链表上使用首次适配算法,分配的性能低,但可以避免外部碎片。
   (b) 对最佳适配方法来说,空闲块链表应当按内存地址递增排序。
   (c) 最佳适配方法选择的是能装下所请求段的那个最大的空闲块。
   (d) 在按块大小递增排序的空闲链表上使用首次适配算法,等价于使用最佳适配算法。
3. Mark&Sweep 垃圾收集器称为保守的条件是:

(a) 它们只有在内存请求得不到满足时,才合并已释放的内存。

(b) 它们把所有看起来像指针的东西都当作指针。

(c) 它们只有在内存耗尽时才执行垃圾收集。

(d) 它们不释放构成循环链表的内存块。

家庭作业 9.20 ▲▲▲▲ 编写你自己的 `malloc` 和 `free` 版本,并把它们的运行时间和空间利用率与标准 C 库提供的 `malloc` 作比较。

9.15 练习答案

答案 9.1 这个练习让你切实体会一下不同地址空间的大小。曾经有一段时间,32 位地址空间看上去大得不可思议。但现在有些数据库应用和科学应用需要更大的地址空间,而且可以预期这一趋势还会继续。在你一生的某个时刻,你大概会发现自己正在抱怨个人电脑上那狭窄的 64 位地址空间!

地址位数(n)	虚拟地址数(N)	最大可能的虚拟地址
4	$2^4 = 16$	$2^4 - 1 = 15$
14	$2^{14} = 16\,\mathrm{K}$	$2^{14} - 1 = 16\,\mathrm{K} - 1$
24	$2^{24} = 16\,\mathrm{M}$	$2^{24} - 1 = 16\,\mathrm{M} - 1$
46	$2^{46} = 64\,\mathrm{T}$	$2^{46} - 1 = 64\,\mathrm{T} - 1$
54	$2^{54} = 16\,\mathrm{P}$	$2^{54} - 1 = 16\,\mathrm{P} - 1$

答案 9.2 因为每个虚拟页都是 $P = 2^p$ 字节,所以系统中可能的页总共有 $2^n/2^p = 2^{n-p}$ 个,每个页都需要一个页表项(PTE)。

n	$P = 2^p$	页表项数
12	1 K	4
16	16 K	4
24	2 M	8
36	1 G	64

答案 9.3 要完全掌握地址翻译,就必须非常熟悉这类问题。下面是第一个小题的解法:给定 $n = 64$ 个虚拟地址位和 $m = 32$ 个物理地址位。页大小 $P = 1\,\mathrm{KB}$ 意味着 VPO 和 PPO 都需要 $\log_2(1\,\mathrm{K}) = 10$ 位。(回想一下,VPO 和 PPO 是相同的。)剩下的地址位分别是 VPN 和 PPN。

P	VPN 位数	VPO 位数	PPN 位数	PPO 位数
1 KB	54	10	22	10
2 KB	53	11	21	11
4 KB	52	12	20	12
16 KB	50	14	18	14

答案 9.4 手工做几次这样的模拟,是巩固你对地址翻译的理解的好办法。你可能会发现,先写出地址的所有位,然后在 VPN、TLBI 等各个位域周围画上

方框,会很有帮助。在这个题中不存在任何类型的不命中:TLB 中有 PTE 的副本,高速缓存中有请求数据字的副本。关于命中和不命中的其他一些组合,请看练习 9.11、9.12 和 9.13。

A. `00 0011 1101 0111`

参数	值
VPN	0xf
TLB 索引	0x3
TLB 标记	0x3
TLB 命中?(Y/N)	Y
缺页?(Y/N)	N
PPN	0xd

C. `0011 0101 0111`

参数	值
字节偏移	0x3
高速缓存索引	0x5
高速缓存标记	0xd
高速缓存命中?(Y/N)	Y
返回的高速缓存字节	0x1d

答案 9.5 解决这个问题会让你很好地体会内存映射的思想。自己试一试。我们还没有讨论过 `open`、`fstat` 或 `write` 函数,所以你需要阅读它们的手册页,了解它们是如何工作的。

code/vm/mmapcopy.c

```
 1  #include "csapp.h"
 2
 3  /*
 4   * mmapcopy ——使用 mmap 把文件 fd 复制到标准输出
 5   */
 6  void mmapcopy(int fd, int size)
 7  {
 8      char *bufp; /* 指向内存映射 VM 区域的指针 */
 9
10      bufp = Mmap(NULL, size, PROT_READ, MAP_PRIVATE, fd, 0);
11      Write(1, bufp, size);
12      return;
13  }
14
15  /* mmapcopy 驱动例程 */
16  int main(int argc, char **argv)
17  {
18      struct stat stat;
19      int fd;
20
21      /* 检查必需的命令行参数 */
22      if (argc != 2) {
23          printf("usage: %s <filename>\n", argv[0]);
```

```
24          exit(0);
25      }
26
27      /* 把输入参数复制到标准输出 */
28      fd = Open(argv[1], O_RDONLY, 0);
29      fstat(fd, &stat);
30      mmapcopy(fd, stat.st_size);
31      exit(0);
32  }
```

答案 9.6 这个练习涉及一些核心思想，如对齐要求、最小块大小和头部编码。确定块大小的一般做法是：把请求的载荷加上头部的大小，向上舍入到对齐要求（这里是 8 字节）的最近倍数。例如，`malloc(2)` 请求的块大小是 $4+2=6$，向上舍入到 8。`malloc(20)` 请求的块大小是 $20+4=24$，向上舍入到 24；它已经是对齐的，无须再舍入。

请求	块大小（十进制字节）	块头部（十六进制）
`malloc(2)`	8	0x9
`malloc(9)`	16	0x11
`malloc(15)`	24	0x19
`malloc(20)`	24	0x19

答案 9.7 最小块的大小对内部碎片有显著影响。因此，理解不同分配器设计与对齐要求所对应的最小块大小是很有好处的。这里的微妙之处在于，同一个块在不同的时间点上既可以是已分配的，也可以是空闲的。所以，最小块大小是最小已分配块大小与最小空闲块大小两者中较大的那个。例如，在最后一个小题中，最小已分配块大小是一个 4 字节头部加一个 1 字节载荷，向上舍入到 8 字节。最小空闲块大小是一个 4 字节头部加一个 4 字节脚部，它已经是 8 的倍数，无须舍入。因此这个分配器的最小块大小是 8 字节。

对齐	已分配块	空闲块	最小块大小（字节）
单字	头部和脚部	头部和脚部	12
单字	有头部而无脚部	头部和脚部	8
双字	头部和脚部	头部和脚部	16
双字	有头部而无脚部	头部和脚部	8

答案 9.8 这里没有什么特别棘手的东西。不过，这道题要求你理解我们的简单隐式链表分配器的其余部分是如何工作的，以及如何操纵和遍历各个块。

code/vm/malloc/mm.c

```
1  static void *find_fit(size_t asize)
2  {
3      /* 首次适配搜索 */
4      void *bp;
5
6      for (bp = heap_listp; GET_SIZE(HDRP(bp)) > 0; bp =
```

```
        NEXT_BLKP(bp)) {
 7          if (!GET_ALLOC(HDRP(bp)) && (asize <= GET_SIZE(HDRP(bp))))
 ↪  {
 8              return bp;
 9          }
10      }
11      return NULL; /* 找不到合适的块 */
12  #endif
13  }
```

答案 9.9 这是帮助你熟悉分配器的又一个热身练习。注意，对这个分配器来说，最小块大小是 16 字节。如果分割之后剩下的块大于或等于最小块大小，那么我们就继续分割这个块（第 6 ~ 10 行）。这里唯一棘手的地方是要认识到：必须先放置新的已分配块（第 6 行和第 7 行），然后才能移到下一个块（第 8 行）。

code/vm/malloc/mm.c

```
 1  static void place(void *bp, size_t asize)
 2  {
 3      size_t csize = GET_SIZE(HDRP(bp));
 4
 5      if ((csize - asize) >= (2*DSIZE)) {
 6          PUT(HDRP(bp), PACK(asize, 1));
 7          PUT(FTRP(bp), PACK(asize, 1));
 8          bp = NEXT_BLKP(bp);
 9          PUT(HDRP(bp), PACK(csize-asize, 0));
10          PUT(FTRP(bp), PACK(csize-asize, 0));
11      }
12      else {
13          PUT(HDRP(bp), PACK(csize, 1));
14          PUT(FTRP(bp), PACK(csize, 1));
15      }
16  }
```

答案 9.10 下面是一种会导致外部碎片的模式：应用程序先对第一个大小类发起大量的分配和释放请求，接着对第二个大小类发起大量的分配和释放请求，再接着对第三个大小类发起大量的分配和释放请求，以此类推。对每个大小类来说，分配器都会制造许多永远不会被回收的内存，因为分配器不进行合并，也因为应用程序再也不会从这个大小类请求块。

第三部分

程序间的交互和通信

到目前为止，在计算机系统的学习中，我们一直都假定程序是在孤立的环境中运行的，输入和输出很少。然而在真实世界中，应用程序会使用操作系统提供的服务，与 I/O 设备通信，也与其他程序通信。

本书的这一部分将让你理解 Unix 操作系统提供的基本 I/O 服务，以及如何使用这些服务来构建像 Web 客户端和服务器那样通过因特网相互通信的应用程序。你将学到编写并发程序的技术，比如能同时服务多个客户的 Web 服务器。编写并发应用程序还能让它们在现代多核处理器上执行得更快。读到本部分结束时，你对计算机系统及其对你程序的影响已经有了成熟的理解，在成为一名高手程序员的路上也就大有可为了。

10

系统级 I/O

输入/输出(input/output,I/O)是在主存和外部设备(例如磁盘驱动器、终端和网络)之间复制数据的过程。输入操作把数据从 I/O 设备复制到主存,而输出操作把数据从内存复制到设备。

所有语言的运行时系统都提供了执行 I/O 的较高级设施。例如,ANSI C 提供了标准 I/O 库,其中包含 `printf` 和 `scanf` 这样执行带缓冲 I/O 的函数。C++ 语言用重载的 <<(“放入”)和 >>(“取出”)运算符提供了类似的功能。在 Linux 系统上,这些较高级的 I/O 函数是使用内核提供的系统级 Unix I/O 函数实现的。大多数时候,较高级的 I/O 函数工作得很好,无须直接使用 Unix I/O。既然如此,为什么还要费心去学习 Unix I/O?

- **理解 Unix I/O 将帮助你理解其他系统概念**。I/O 系统的运行离不开它,正因为如此,我们常常会在 I/O 与其他系统思想之间遇到循环依赖。例如,I/O 在进程的创建和执行中扮演着关键角色;反过来,进程的创建又在不同进程如何共享文件的过程中扮演着关键角色。因此,要真正理解 I/O,就需要理解进程;反之亦然。在讨论存储器层次结构、链接和加载、进程以及虚拟内存时,我们已经接触了 I/O 的一些方面。既然你已经对这些思想有了更好的理解,现在我们就可以把这个循环闭合起来,更深入地探究 I/O。
- **有时你别无选择,只能使用 Unix I/O**。有一些重要的情形,使用较高级的 I/O 函数要么不可能,要么不合适。例如,标准 I/O 库没有提供访问文件元数据(metadata)的方法,比如文件大小或文件创建的时间。另外,标准 I/O 库还存在一些问题,用它来做网络编程是有风险的。

本章将向你介绍 Unix I/O 和标准 I/O 的一般概念,并展示如何在你的 C 程序中可靠地使用它们。除了作为一篇总体的导言之外,本章还为我们随后对网络编程和并发的研究打下坚实的基础。

10.1 Unix I/O

一个 Linux 文件是一个 m 个字节的序列：

$$B_0, B_1, \dots, B_k, \dots, B_{m-1}$$

所有的 I/O 设备（例如网络、磁盘和终端）都被模型化为文件，而所有的输入和输出都被当作对相应文件的读和写来执行。这种把设备映射为文件的巧妙处理，使得 Linux 内核能够提供一个简单、低级的应用接口，称为 Unix I/O，它让所有的输入和输出都能以一种统一且一致的方式来执行：

- **打开文件**。应用程序通过要求内核打开相应的文件，来宣布它要访问一个 I/O 设备的意图。
  内核返回一个小的非负整数，称为描述符（descriptor），在后续对该文件的所有操作中，内核都用它来标识这个文件。内核记录有关打开文件的全部信息，而应用程序只需记住这个描述符。
  由 Linux shell 创建的每个进程一开始就有了三个打开的文件：标准输入（描述符 0）、标准输出（描述符 1）和标准错误（描述符 2）。头文件 `<unistd.h>` 定义了常量 `STDIN_FILENO`、`STDOUT_FILENO` 和 `STDERR_FILENO`，可以用它们来代替显式的描述符值。
- **改变当前文件位置**。内核为每个打开的文件维护一个文件位置 k，初始值为 0。文件位置是从文件开头算起的字节偏移（offset）。应用程序可以通过执行 `seek` 操作，显式地设置当前文件位置 k。
- **读和写文件**。读操作从文件复制 $n > 0$ 个字节到内存，从当前文件位置 k 开始，然后把 k 增加 n。给定一个大小为 m 字节的文件，当 $k \geq m$ 时执行读操作会触发一个条件，称为文件结束（end-of-file，EOF），应用程序能够检测到它。文件的末尾并没有显式的“EOF 字符”。
  类似地，写操作从内存复制 $n > 0$ 个字节到文件，从当前文件位置 k 开始，然后更新 k。
- **关闭文件**。当应用程序结束了对文件的访问时，它就要求内核关闭该文件，把这一消息通知给内核。
  内核对此的响应是，释放它在文件打开时建立的数据结构，并把描述符归还到可用描述符的池中。无论进程出于任何原因终止，内核都会关闭所有打开的文件，并释放这些文件的内存资源。

10.2 文件

每个 Linux 文件都有一个类型，它表明这个文件在系统中的角色：

- **普通文件（regular file）**包含任意的数据。应用程序常常区分文本文件（text file），即只包含 ASCII 或 Unicode 字符的普通文件，和二进制文件（binary file），即其余所有文件。对内核来说，文本文件和二进制文件没有差别。一个 Linux 文本文件由一系列文本行（text line）组成，每行是一个以换行符（newline character）结尾的字符序列。换行符与 ASCII 换行（LF）字符相同，用 `\n` 表示，数值为 0x0a。
- **目录（directory）**是一个由链接（link）组成的数组构成的文件，每个链接把一个文件名映射到一个文件，该文件也可能是另一个目录。每个目录至少包含两个条目：.（点）是指向该目录自身的链接，..（双点）是指向目录层次结构中父目录的链接（见下文）。你可以用 `mkdir` 命令创建一个目录，用 `ls` 查看它的内容，用 `rmdir` 删除它。
- **套接字（socket）**是用来与通过网络相连的另一个进程通信的文件（见 11.4 节）。

> **边栏　行末（EOL）指示符**
> 使用文本文件时有一处很麻烦：不同的系统使用不同的字符来标记一行的结束。Linux 和 Mac OS X 使用 `\n`（0xa），即 ASCII 换行（LF）字符。然而，MS Windows 和 HTTP 这样的因特网协议使用序列 `\r\n`（0xd 0xa），即 ASCII 回车（CR）字符后跟一个换行（LF）字符。如果在一个 Windows 系统上创建了一个文件 `foo.txt`，然后在 Linux 的文本编辑器中查看它，就会看到每一行的末尾都带着一个讨厌的 ^M，这是 Linux 工具显示 CR 字符的方式。只需运行下面的命令，就可以把这些不需要的 CR 字符从 `foo.txt` 中就地清除：
>
> ```
> linux> perl -pi -e "s/\r\n/\n/g" foo.txt
> ```

其他文件类型还包括命名管道（named pipe）、符号链接（symbolic link）、字符设备（character device）和块设备（block device），它们超出了本书的讨论范围。

Linux 内核把所有文件组织成一个单一的目录层次结构，其根节点是名为 /（斜杠）的根目录。系统中的每个文件都是根目录的直接或间接后代。图 10.1 展示了本书所用 Linux 系统上目录层次结构的一部分。

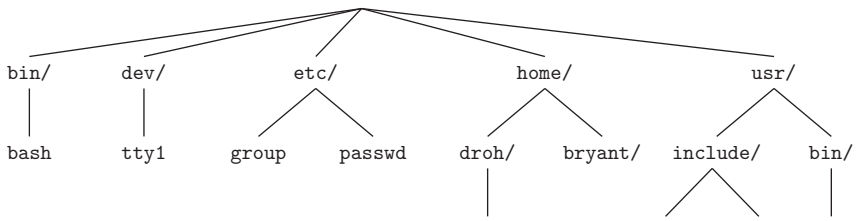


图 10.1　Linux 目录层次结构的一部分。带末尾斜杠表示一个目录。

作为其上下文的一部分，每个进程都有一个当前工作目录（current working directory），用来标识它在目录层次结构中的当前位置。你可以用 `cd` 命令改变 shell 的当前工作目录。

目录层次结构中的位置用路径名（pathname）来指定。一个路径名是一个字符串，由一个可选的斜杠开头，后跟一系列用斜杠分隔的文件名。路径名有两种形式：

- **绝对路径名（absolute pathname）** 以斜杠开始，表示从根节点出发的一条路径。例如在图 10.1 中，`hello.c` 的绝对路径名是 `/home/droh/hello.c`。
- **相对路径名（relative pathname）** 以文件名开始，表示从当前工作目录出发的一条路径。例如在图 10.1 中，如果 `/home/droh` 是当前工作目录，那么 `hello.c` 的相对路径名是 `./hello.c`。另一方面，如果 `/home/bryant` 是当前工作目录，那么相对路径名就是 `../home/droh/hello.c`。

10.3 打开和关闭文件

进程通过调用 `open` 函数来打开一个已存在的文件或创建一个新文件。

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int open(char *filename, int flags, mode_t mode);

返回：成功时返回新的文件描述符；出错返回 -1
```

`open` 函数把一个文件名转换为一个文件描述符，并返回描述符编号。返回的描述符总是进程中当前未打开的最小描述符。`flags` 参数指明进程打算以何种方式访问该文件：

- `O_RDONLY`。只读。
- `O_WRONLY`。只写。
- `O_RDWR`。读和写。

例如，下面是打开一个现有文件进行读取的方法：

```
fd = Open("foo.txt", O_RDONLY, 0);
```

`flags` 参数也可以与一个或更多个位掩码（bit mask）做按位或运算，为写操作提供附加的指示：

- `O_CREAT`。如果该文件不存在，就创建一个截断的（空的）版本。

- O_TRUNC。如果该文件已经存在,就截断它。
- O_APPEND。每次写操作之前,把文件位置设置到该文件的末尾。

图 10.2 访问权限位。定义在 sys/stat.h 中。

掩码	说明
S_IRUSR	用户(所有者)可以读这个文件
S_IWUSR	用户(所有者)可以写这个文件
S_IXUSR	用户(所有者)可以执行这个文件
S_IRGRP	所有者所在组的成员可以读这个文件
S_IWGRP	所有者所在组的成员可以写这个文件
S_IXGRP	所有者所在组的成员可以执行这个文件
S_IROTH	其他人(任何用户)可以读这个文件
S_IWOTH	其他人(任何用户)可以写这个文件
S_IXOTH	其他人(任何用户)可以执行这个文件

例如,你可能会像下面这样打开一个现有文件,目的是向其中追加一些数据:

```
fd = Open("foo.txt", O_WRONLY|O_APPEND, 0);
```

mode 参数指定新文件的访问权限位。这些位的符号名称如图 10.2 所示。

作为上下文的一部分,每个进程都有一个通过调用 umask 函数而设置的掩码 (umask)。当进程通过调用 open 函数、给定某个 mode 参数来创建一个新文件时,该文件的访问权限位被设置为 mode & ~umask。例如,假设给定了 mode 和 umask 的如下默认值:

```
#define DEF_MODE      S_IRUSR|S_IWUSR|S_IRGRP|S_IWGRP|S_IROTH|S_IWOTH
#define DEF_UMASK     S_IWGRP|S_IWOTH
```

那么下面的代码片段所创建的新文件中,文件所有者具有读和写权限,而其他所有用户都具有读权限:

```
umask(DEF_UMASK);
fd = Open("foo.txt", O_CREAT|O_TRUNC|O_WRONLY, DEF_MODE);
```

最后,进程通过调用 close 函数来关闭一个已打开的文件。

```
#include <unistd.h>

int close(int fd);

返回:成功返回 0;出错返回 -1
```

关闭一个已经关闭的描述符是一个错误。

练习 10.1 下面的程序输出是什么?

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6
7      fd1 = Open("foo.txt", O_RDONLY, 0);
8      Close(fd1);
9      fd2 = Open("baz.txt", O_RDONLY, 0);
10     printf("fd2 = %d\n", fd2);
11     exit(0);
12 }
```

10.4 读和写文件

应用程序分别通过调用 `read` 和 `write` 函数来执行输入和输出。

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t n);

返回:成功时返回读取的字节数;遇到 EOF 返回 0;出错返回 -1

ssize_t write(int fd, const void *buf, size_t n);

返回:成功时返回写入的字节数;出错返回 -1
```

`read` 函数从描述符 `fd` 的当前文件位置复制至多 n 个字节到内存位置 `buf`。返回值 -1 表示出错,返回值 0 表示 EOF。否则,返回值表示实际传送的字节数。

`write` 函数从内存位置 `buf` 复制至多 n 个字节到描述符 `fd` 的当前文件位置。图 10.3 展示了一个程序,它用 `read` 和 `write` 调用把标准输入复制到标准输出,每次 1 个字节。

应用程序可以通过调用 `lseek` 函数显式地修改当前文件位置,这超出了我们的讨论范围。

在某些情况下,`read` 和 `write` 实际传送的字节数比应用程序请求的要少。这样的短计数(short count)并不表明发生了错误。出现短计数有多种原因:

> **边栏 `ssize_t` 与 `size_t` 有什么差别?**
> 你可能已经注意到,`read` 函数有一个 `size_t` 类型的输入参数和一个 `ssize_t` 类型的返回值。那么这两个类型有什么差别呢?在 x86-64 系统上,`size_t` 被定义为 `unsigned long`,而 `ssize_t`(signed size,有符号的字节数)被定义为 `long`。`read` 函数返回有符号的字节数而不是无符号的字节数,是因为它必须在出错时返

> 回 −1。有趣的是，可能返回这唯一的 −1，使得一次读操作能读取的最大字节数减少了一半。

code/io/cpstdin.c

```
1  #include "csapp.h"
2
3  int main(void)
4  {
5      char c;
6
7      while(Read(STDIN_FILENO, &c, 1) != 0)
8          Write(STDOUT_FILENO, &c, 1);
9      exit(0);
10 }
```

图 10.3 用 `read` 和 `write` 把标准输入复制到标准输出，每次 1 个字节。

- **读取时遇到 EOF**。假设我们准备从一个文件读取，从当前文件位置起该文件只剩 20 个字节，而我们正在以 50 字节的块读取该文件。那么下一次 `read` 将返回 20 的短计数，再下一次 `read` 将通过返回 0 的短计数来通报 EOF。
- **从终端读取文本行**。如果打开的文件与一个终端（即一个键盘和显示器）相关联，那么每次调用 `read` 函数都将返回一行文本，返回的短计数等于该文本行的大小。
- **读写网络套接字**。如果打开的文件对应于一个网络套接字（见 11.4 节），那么内部缓冲区的约束和较长的网络延迟都可能导致 `read` 和 `write` 返回短计数。当你对 Linux 管道（pipe，一种超出本书讨论范围的进程间通信机制）调用 `read` 和 `write` 时，也可能出现短计数。

在实践中，你从磁盘文件读取时，除了 EOF 绝不会遇到短计数；你向磁盘文件写入时，也绝不会遇到短计数。然而，如果你要构建如 Web 服务器这样健壮（可靠）的网络应用程序，那么就必须通过反复调用 `read` 和 `write` 来应对短计数，直到所有请求的字节都已传送完毕为止。

10.5 用 Rio 包健壮地读写

在本节中，我们将开发一个称为 Rio（Robust I/O，健壮 I/O）包的 I/O 包，它为你自动处理这些短计数。对于网络程序这样容易出现短计数的应用程序，Rio 包提供了方便、健壮且高效的 I/O。Rio 提供两类不同的函数：

- **无缓冲的输入和输出函数**。这些函数在内存和文件之间直接传送数据，没有应用级缓冲。它们特别适合于在网络上读写二进制数据。
- **带缓冲的输入函数**。这些函数使你能够高效地从文件内容中读取文本行和二进制数据，其内容被缓存在一个应用级缓冲区中，类似于以 `printf` 为代表的标准 I/O 函数所提供的缓冲。与文献 [110] 中给出的带缓冲 I/O 例程不同，带缓冲的 Rio 输入函数是线程安全的（见 12.7.1 节），并且在同一个描述符上可以任意交叉使用。例如，你可以从一个描述符读取一些文本行，然后读一些二进制数据，接着再读一些文本行。

我们介绍这些 Rio 例程有两个原因。第一，在接下来两章开发的网络应用程序中，我们会用到它们。第二，通过研究这些例程的代码，你将会对一般意义上的 Unix I/O 有更深入的理解。

10.5.1 Rio 无缓冲的输入和输出函数

应用程序通过调用 `rio_readn` 和 `rio_writen` 函数，可以直接在内存和文件之间传送数据。

```
#include "csapp.h"

ssize_t rio_readn(int fd, void *usrbuf, size_t n);
ssize_t rio_writen(int fd, void *usrbuf, size_t n);

返回：成功时返回传送的字节数；遇到 EOF 返回 0（仅 rio_readn）；出错返回 -1
```

`rio_readn` 函数最多传送 n 个字节，从描述符 `fd` 的当前文件位置传到内存位置 `usrbuf`。类似地，`rio_writen` 函数从位置 `usrbuf` 传送 n 个字节到描述符 `fd`。`rio_readn` 函数只有在遇到 EOF 时才可能返回一个短计数，`rio_writen` 函数则从不返回短计数。对 `rio_readn` 和 `rio_writen` 的调用在同一个描述符上可以任意交叉进行。

图 10.4 展示了 `rio_readn` 和 `rio_writen` 的代码。注意，如果 `read` 或 `write` 函数被应用程序信号处理程序的返回所中断，每个函数都会手动重新启动它。为了尽可能保证可移植性，我们对被中断的系统调用留有余地，并在必要时重新启动它们。

10.5.2 Rio 带缓冲的输入函数

假设我们想编写一个程序，来统计一个文本文件中的行数。可以怎么做呢？一种办法是利用 `read` 函数，每次从文件向用户内存传送 1 个字节，并检查每个字节是不是换行字符。这种方法的缺点是效率不高，读取文件中的每个字节都需要陷入内核。

更好的办法是调用一个包装函数(rio_readlineb),它从一个内部读缓冲区复制文本行,并在缓冲区一旦变空的时候,自动调用 read 来重新填满缓冲区。对于既包含文本行又包含二进制数据的文件(参见 11.5.3 节描述的 HTTP 响应),我们还提供了 rio_readn 的一个带缓冲的版本,称为 rio_readnb,它从与 rio_readlineb 相同的读缓冲区传送原始字节。

```
#include "csapp.h"

void rio_readinitb(rio_t *rp, int fd);

返回:无

ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen);
ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n);

返回:成功时返回读取的字节数;遇到 EOF 返回 0;出错返回 -1
```

rio_readinitb 函数对每个打开的描述符只调用一次。它把描述符 fd 和类型为 rio_t、位于地址 rp 的读缓冲区关联起来。

rio_readlineb 函数从文件 rp 中读出下一行文本(包括结尾的换行字符),把它复制到内存位置 usrbuf,并用 NULL(零)字符终止这行文本。rio_readlineb 函数最多读 maxlen-1 个字节,为结尾的 NULL 字符留出空间。超过 maxlen-1 字节的文本行会被截断,并以一个 NULL 字符结尾。

rio_readnb 函数从文件 rp 读出至多 n 个字节到内存位置 usrbuf。对 rio_readlineb 和 rio_readnb 的调用在同一个描述符上可以任意交叉进行。但是,对这两个带缓冲函数的调用,不应当与对无缓冲的 rio_readn 函数的调用交叉进行。

在本书其余的部分里,你会遇到许多使用 Rio 函数的例子。图 10.5 展示了如何使用 Rio 函数,把一个文本文件从标准输入复制到标准输出,每次复制一行。

图 10.6 展示了一个读缓冲区的格式,以及初始化该缓冲区的 rio_readinitb 函数的代码。rio_readinitb 函数创建一个空的读缓冲区,并把一个打开的文件描述符与这个缓冲区关联起来。

code/src/csapp.c

```
1  ssize_t rio_readn(int fd, void *usrbuf, size_t n)
2  {
3      size_t nleft = n;
4      ssize_t nread;
5      char *bufp = usrbuf;
6
7      while (nleft > 0) {
```

```
8           if ((nread = read(fd, bufp, nleft)) < 0) {
9               if (errno == EINTR) /* 被信号处理程序的返回中断 */
10                  nread = 0;      /* 然后再次调用 read() */
11              else
12                  return -1;      /* errno 由 read() 设置 */
13          }
14          else if (nread == 0)
15              break;              /* EOF */
16          nleft -= nread;
17          bufp += nread;
18      }
19      return (n - nleft);         /* 返回值 >= 0 */
20  }
```

code/src/csapp.c

```
1   ssize_t rio_writen(int fd, void *usrbuf, size_t n)
2   {
3       size_t nleft = n;
4       ssize_t nwritten;
5       char *bufp = usrbuf;
6
7       while (nleft > 0) {
8           if ((nwritten = write(fd, bufp, nleft)) <= 0) {
9               if (errno == EINTR) /* 被信号处理程序的返回中断 */
10                  nwritten = 0;    /* 然后再次调用 write() */
11              else
12                  return -1;       /* errno 由 write() 设置 */
13          }
14          nleft -= nwritten;
15          bufp += nwritten;
16      }
17      return n;
18  }
```

图 10.4 `rio_readn` 和 `rio_writen` 函数。

code/io/cpfile.c

```
1   #include "csapp.h"
2
3   int main(int argc, char **argv)
4   {
5       int n;
6       rio_t rio;
7       char buf[MAXLINE];
8
9       Rio_readinitb(&rio, STDIN_FILENO);
10      while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0)
11          Rio_writen(STDOUT_FILENO, buf, n);
12  }
```

图 10.5 把一个文本文件从标准输入复制到标准输出。

code/include/csapp.h

```
1  #define RIO_BUFSIZE 8192
2  typedef struct {
3      int rio_fd;                /* 这个内部缓冲区的描述符 */
4      int rio_cnt;               /* 内部缓冲区中未读的字节数 */
5      char *rio_bufptr;          /* 内部缓冲区中下一个未读字节 */
6      char rio_buf[RIO_BUFSIZE]; /* 内部缓冲区 */
7  } rio_t;
```

code/src/csapp.c

```
1  void rio_readinitb(rio_t *rp, int fd)
2  {
3      rp->rio_fd = fd;
4      rp->rio_cnt = 0;
5      rp->rio_bufptr = rp->rio_buf;
6  }
```

图 10.6 类型为 rio_t 的读缓冲区，以及初始化它的 rio_readinitb 函数。

Rio 读例程的心脏是图 10.7 所示的 rio_read 函数。rio_read 函数是 Linux 的 read 函数的一个带缓冲版本。当以读取 n 个字节的请求调用 rio_read 时，读缓冲区中还有 rp->rio_cnt 个未读字节。如果缓冲区为空，就会通过一次 read 调用来重新填充。这个 read 调用返回一个短计数并不是错误，它只是起到部分填满读缓冲区的作用。缓冲区一旦非空，rio_read 就从读缓冲区把 n 与 rp->rio_cnt 中较小的一个数量的字节复制到用户缓冲区，并返回复制了的字节数。

code/src/csapp.c

```
1  static ssize_t rio_read(rio_t *rp, char *usrbuf, size_t n)
2  {
3      int cnt;
4
5      while (rp->rio_cnt <= 0) { /* 如果缓冲区为空则重新填充 */
6          rp->rio_cnt = read(rp->rio_fd, rp->rio_buf,
7                             sizeof(rp->rio_buf));
8          if (rp->rio_cnt < 0) {
9              if (errno != EINTR) /* 被信号处理程序的返回中断 */
10                 return -1;
11         }
12         else if (rp->rio_cnt == 0) /* EOF */
13             return 0;
14         else
15             rp->rio_bufptr = rp->rio_buf; /* 重置缓冲区指针 */
16     }
17
18     /* 把 min(n, rp->rio_cnt) 个字节从内部缓冲区复制到用户缓冲区 */
19     cnt = n;
```

```
20     if (rp->rio_cnt < n)
21         cnt = rp->rio_cnt;
22     memcpy(usrbuf, rp->rio_bufptr, cnt);
23     rp->rio_bufptr += cnt;
24     rp->rio_cnt -= cnt;
25     return cnt;
26 }
```

图 10.7 内部的 rio_read 函数。

对一个应用程序来说，rio_read 函数与 Linux 的 read 函数有着相同的语义。出错时它返回 −1，并相应地设置 errno。在 EOF 时返回 0。如果请求的字节数超过了读缓冲区中未读字节的数目，它就返回一个短计数。这两个函数的相似之处让构造各种带缓冲的读函数变得容易：只要用 rio_read 代替 read 就可以了。例如，图 10.8 中的 rio_readnb 函数与 rio_readn 结构相同，只是以 rio_read 替换了 read。类似地，图 10.8 中的 rio_readlineb 例程最多调用 maxlen-1 次 rio_read。每次调用都从读缓冲区返回 1 个字节，然后检查这个字节是不是结尾的换行符。

code/src/csapp.c

```
1  ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)
2  {
3      int n, rc;
4      char c, *bufp = usrbuf;
5
6      for (n = 1; n < maxlen; n++) {
7          if ((rc = rio_read(rp, &c, 1)) == 1) {
8              *bufp++ = c;
9              if (c == '\n') {
10                 n++;
11                 break;
12             }
13         } else if (rc == 0) {
14             if (n == 1)
15                 return 0; /* EOF,没有数据被读出 */
16             else
17                 break;    /* EOF,读出了一些数据 */
18         } else
19             return -1;    /* 出错 */
20     }
21     *bufp = 0;
22     return n-1;
23 }
```

code/src/csapp.c

```
1  ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)
2  {
3      size_t nleft = n;
4      ssize_t nread;
```

```
5       char *bufp = usrbuf;
6
7       while (nleft > 0) {
8           if ((nread = rio_read(rp, bufp, nleft)) < 0)
9               return -1;          /* errno 由 read() 设置 */
10          else if (nread == 0)
11              break;              /* EOF */
12          nleft -= nread;
13          bufp += nread;
14      }
15      return (n - nleft);         /* 返回值 >= 0 */
16  }
```

图 10.8 rio_readlineb 和 rio_readnb 函数。

边栏 Rio 包的起源

Rio 函数的灵感来自 W. Richard Stevens 在其经典网络编程著作 [110] 中描述的 readline、readn 和 writen 函数。rio_readn 和 rio_writen 函数与 Stevens 的 readn 和 writen 函数完全相同。但是,Stevens 的 readline 函数存在一些限制,这些问题在 Rio 中得到了修正。第一,由于 readline 是带缓冲的而 readn 不是,这两个函数不能在同一个描述符上一起使用。第二,由于它使用了一个静态缓冲区,Stevens 的 readline 函数不是线程安全的,这迫使 Stevens 引入一个不同的线程安全版本 readline_r。我们以 rio_readlineb 和 rio_readnb 函数修正了这两处缺陷,它们相互兼容并且是线程安全的。

10.6 读取文件元数据

应用程序可以通过调用 stat 和 fstat 函数,检索关于文件的元信息(有时称为文件的元数据)。

```
#include <unistd.h>
#include <sys/stat.h>

int stat(const char *filename, struct stat *buf);
int fstat(int fd, struct stat *buf);

返回:成功返回 0;出错返回 -1
```

stat 函数以一个文件名作为输入,填写图 10.9 所示 stat 结构的各个成员。fstat 函数与之类似,只是以文件描述符而不是文件名作为输入。在 11.5 节讨论 Web 服务器时,我们会用到 stat 结构的 st_mode 成员和 st_size 成员。其余成员超出了我们的讨论范围。

st_size 成员包含文件的大小,单位是字节。st_mode 成员同时编码了文件的权限位(图 10.2)和文件类型(见 10.2 节)。Linux 在 sys/stat.h 中定义了一系列宏谓词(macro predicate),用来从 st_mode 成员判断文件类型:

- S_ISREG(m)。这是一个普通文件吗?
- S_ISDIR(m)。这是一个目录文件吗?
- S_ISSOCK(m)。这是一个网络套接字吗?

图 10.10 展示了我们如何使用这些宏和 stat 函数来读取和解释一个文件的 st_mode 位。

```
statbuf.h(由 sys/stat.h 包含)

/* stat 和 fstat 函数返回的元数据 */
struct stat {
    dev_t         st_dev;      /* 设备 */
    ino_t         st_ino;      /* inode */
    mode_t        st_mode;     /* 保护模式和文件类型 */
    nlink_t       st_nlink;    /* 硬链接的数量 */
    uid_t         st_uid;      /* 所有者的用户 ID */
    gid_t         st_gid;      /* 所有者的组 ID */
    dev_t         st_rdev;     /* 设备类型(如果是 inode 设备) */
    off_t         st_size;     /* 总大小,以字节为单位 */
    unsigned long st_blksize;  /* 用于文件系统 I/O 的块大小 */
    unsigned long st_blocks;   /* 已分配块的数量 */
    time_t        st_atime;    /* 最后一次访问时间 */
    time_t        st_mtime;    /* 最后一次修改时间 */
    time_t        st_ctime;    /* 最后一次改变时间 */
};
```

图 10.9 stat 结构。

```
code/io/statcheck.c

1  #include "csapp.h"
2
3  int main (int argc, char **argv)
4  {
5      struct stat stat;
6      char *type, *readok;
7
8      Stat(argv[1], &stat);
9      if (S_ISREG(stat.st_mode))      /* 判断文件类型 */
10         type = "regular";
11     else if (S_ISDIR(stat.st_mode))
12         type = "directory";
13     else
14         type = "other";
15     if ((stat.st_mode & S_IRUSR))   /* 检查读权限 */
16         readok = "yes";
17     else
18         readok = "no";
19
20     printf("type: %s, read: %s\n", type, readok);
21     exit(0);
22 }
```

图 10.10 查询和操作一个文件的 st_mode 位。

10.7 读取目录内容

应用程序可以通过 readdir 函数族来读取一个目录的内容。

```
#include <sys/types.h>
#include <dirent.h>

DIR *opendir(const char *name);

返回:成功时返回指向句柄的指针;出错返回 NULL
```

opendir 函数以一个路径名为输入，返回一个指向目录流的指针。流(stream)是对一个有序条目列表的抽象,在这里就是一个目录项的列表。

```
#include <dirent.h>

struct dirent *readdir(DIR *dirp);

返回:成功时返回指向下一个目录项的指针;没有更多目录项或出错时返回 NULL
```

每次调用 readdir 都返回一个指针，指向流 dirp 中的下一个目录项;如果没有更多的目录项就返回 NULL。每个目录项都是一个具有如下形式的结构:

```
struct dirent {
    ino_t d_ino;      /* inode 编号 */
    char d_name[256]; /* 文件名 */
};
```

虽然 Linux 的某些版本还包含其他结构成员,但这两个成员是所有系统都遵循的标准成员。d_name 成员是文件名,d_ino 是文件在磁盘上的位置。

出错时，readdir 返回 NULL 并设置 errno。遗憾的是，要把出错与流结束条件区分开来,唯一的办法是在调用 readdir 之后检查 errno 是否被修改过。

```
#include <dirent.h>

int closedir(DIR *dirp);

返回:成功返回 0;出错返回 -1
```

closedir 函数关闭一个流并释放它的资源。图 10.11 展示了我们如何使用 readdir 读取一个目录的内容。

```
code/io/readdir.c

1  #include "csapp.h"
```

```
2
3  int main(int argc, char **argv)
4  {
5      DIR *streamp;
6      struct dirent *dep;
7
8      streamp = Opendir(argv[1]);
9
10     errno = 0;
11     while ((dep = readdir(streamp)) != NULL) {
12         printf("Found file: %s\n", dep->d_name);
13     }
14     if (errno != 0)
15         unix_error("readdir error");
16
17     Closedir(streamp);
18     exit(0);
19 }
```

图 10.11 读取一个目录的内容。

10.8 共享文件

Linux 文件可以用许多不同的方式来共享。除非你对内核如何表示打开的文件有清晰的认识，否则文件共享这个想法会相当令人困惑。内核用三个相关的数据结构来表示打开的文件：

- **描述符表（descriptor table）**。每个进程都有自己独立的描述符表，其条目由该进程打开的文件描述符来索引。每个打开描述符条目指向文件表中的一个条目。
- **文件表（file table）**。打开文件的集合由一个所有进程共享的文件表来表示。就本文的目的而言，每个文件表条目由当前文件位置、一个对该条目当前有多少描述符条目指向它的引用计数，以及一个指向 v-node 表中条目的指针组成。关闭一个描述符时，相应文件表条目中的引用计数会减 1。内核只有等到这个引用计数为零时才会删除该文件表条目。
- **v-node 表**。与文件表一样，v-node 表也由所有进程共享。每个条目包含 `stat` 结构中的大部分信息，包括 `st_mode` 和 `st_size` 成员。

图 10.12 给出了一个例子：描述符 1 和 4 通过互不相同的打开文件表条目引用两个不同的文件。这是典型的情况，文件没有被共享，每个描述符对应一个不同的文件。

多个描述符也可以通过不同的文件表条目引用同一个文件，如图 10.13 所示。这种情形可能发生在这样的场景中：以相同的文件名调用了两次 `open` 函

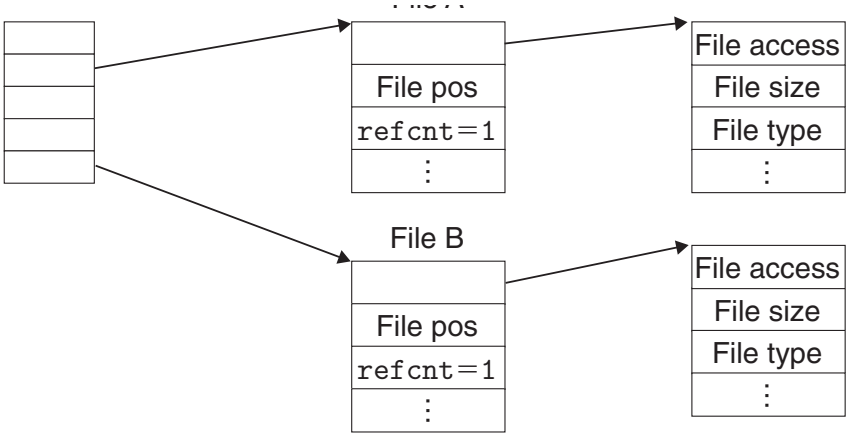


图 10.12　打开文件的典型内核数据结构。在此例中，两个描述符引用不同的文件，没有共享。

structures for open files. In this example, two descriptors reference distinct files. There is no sharing.

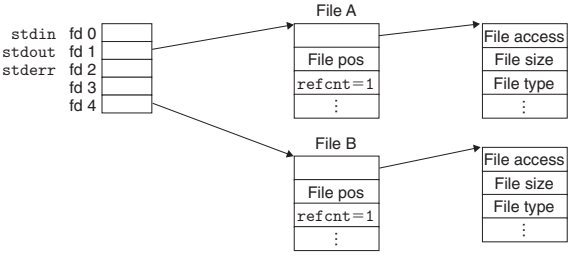


Figure 10.13
File sharing. This example shows two descriptors sharing the same disk file through two open file table entries.

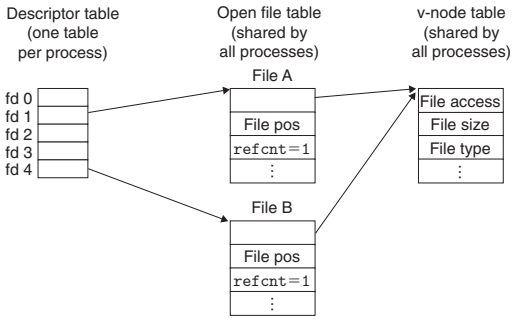


图 10.13

数。关键思想是，每个描述符都有它自己的文件位置，因此在不同描述符上的读取可以取到文件中不同位置的数据。

我们还可以由此理解父进程和子进程是如何共享文件的。假设在调用 `fork` 之前，父进程拥有图 10.12 所示的打开文件。那么调用 `fork` 之后的状态就如图 10.14 所示。

子进程获得了父进程描述符表的一个复本。父进程和子进程共享同一组打开文件表条目，因此也共享同一个文件位置。一个重要的结果是，父进程和子进程必须都关闭它们的描述符，内核才会删除相应的文件表条目。

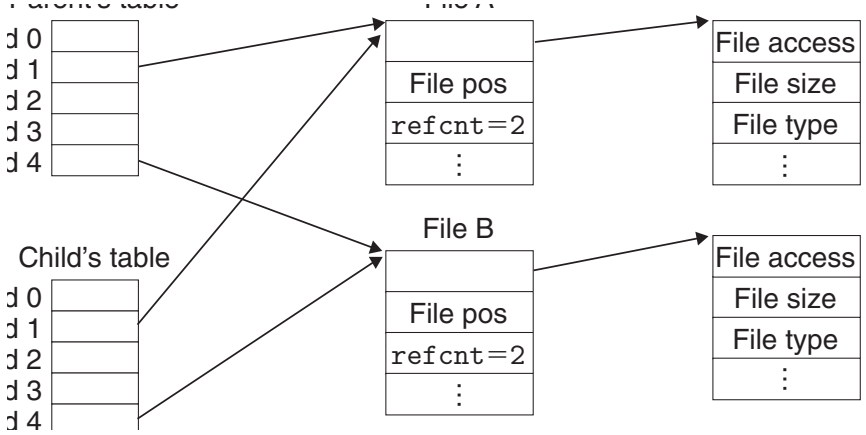


图 10.14 子进程如何继承父进程打开的文件。初始情况如图 10.12 所示。

练习 10.2 假设磁盘文件 `foobar.txt` 由六个 ASCII 字符 foobar 组成。那么，下面的程序输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6      char c;
7
8      fd1 = Open("foobar.txt", O_RDONLY, 0);
9      fd2 = Open("foobar.txt", O_RDONLY, 0);
10     Read(fd1, &c, 1);
11     Read(fd2, &c, 1);
12     printf("c = %c\n", c);
13     exit(0);
14 }
```

练习 10.3 与前面一样，假设磁盘文件 `foobar.txt` 由六个 ASCII 字符 foobar 组成。那么，下面的程序输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd;
6      char c;
7
8      fd = Open("foobar.txt", O_RDONLY, 0);
9      if (Fork() == 0) {
```

```
10         Read(fd, &c, 1);
11         exit(0);
12     }
13     Wait(NULL);
14     Read(fd, &c, 1);
15     printf("c = %c\n", c);
16     exit(0);
17 }
```

10.9 I/O 重定向

Linux shell 提供了一些 I/O 重定向操作符，允许用户使标准输入和输出与磁盘文件关联起来。例如，键入

```
linux> ls > foo.txt
```

会使 shell 加载并执行 `ls` 程序，并把标准输出重定向到磁盘文件 `foo.txt`。正如我们将在 11.5 节看到的，Web 服务器在代表客户端运行一个 CGI 程序时，也执行类似的重定向。那么，I/O 重定向是如何工作的呢？有一种方法是使用 `dup2` 函数。

```
#include <unistd.h>

int dup2(int oldfd, int newfd);

返回：成功时返回非负的描述符；出错返回 -1
```

`dup2` 函数把描述符表条目 `oldfd` 复制到描述符表条目 `newfd`，覆盖描述符表条目 `newfd` 以前的内容。如果 `newfd` 已经打开，那么 `dup2` 会先关闭 `newfd`，然后复制 `oldfd`。

假设在调用 `dup2(4,1)` 之前，我们处在图 10.12 所示的情形，其中描述符 1（标准输出）对应文件 A（比如一个终端），描述符 4 对应文件 B（比如一个磁盘文件）。A 和 B 的引用计数都等于 1。图 10.15 展示了调用 `dup2(4,1)` 之后的状态。两个描述符现在都指向文件 B；文件 A 已经被关闭，它的文件表和 v-node 表条目也被删除；文件 B 的引用计数相应增加。从这里开始，任何写到标准输出的数据都被重定向到文件 B。

练习 10.4 你会如何用 `dup2` 把标准输入重定向到描述符 5？

> **边栏 右尖符和左尖符**
> 为了避免与其他括号类运算符（比如] 和 [）混淆，我们一直把 shell 的 > 操作符称为右尖符（right hoinky），把 < 操作符称为左尖符（left hoinky）。

练习 10.5 假设磁盘文件 `foobar.txt` 由六个 ASCII 字符 foobar 组成。那么，下面的程序输出是什么？

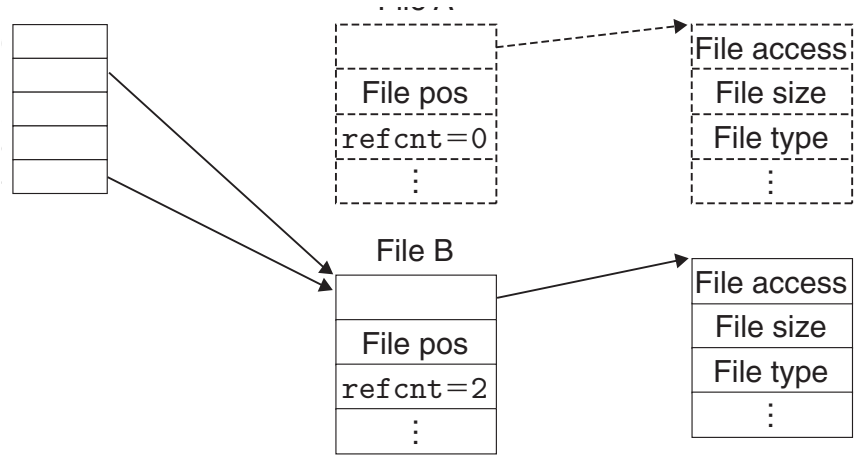


图 10.15 调用 dup2(4,1) 重定向标准输出之后的内核数据结构。初始情况如图 10.12 所示。

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6      char c;
7
8      fd1 = Open("foobar.txt", O_RDONLY, 0);
9      fd2 = Open("foobar.txt", O_RDONLY, 0);
10     Read(fd2, &c, 1);
11     Dup2(fd2, fd1);
12     Read(fd1, &c, 1);
13     printf("c = %c\n", c);
14     exit(0);
15 }
```

10.10 标准 I/O

C 语言定义了一组较高级的输入和输出函数，称为标准 I/O 库，它为程序员提供了 Unix I/O 的一个较高级替代。这个库（libc）提供了打开和关闭文件（fopen 与 fclose）、读取和写入字节（fread 与 fwrite）、读取和写入字符串（fgets 与 fputs）以及复杂格式化 I/O（scanf 与 printf）的函数。

标准 I/O 库把一个打开的文件模型化为一个流（stream）。对程序员来说，一个流是指向 FILE 类型结构体的指针。每个 ANSI C 程序都从三个已打开的流开始，它们分别对应标准输入、标准输出和标准错误：stdin、stdout 和 stderr，其声明为

```
#include <stdio.h>
extern FILE *stdin;          /* 标准输入(描述符 0) */
extern FILE *stdout;         /* 标准输出(描述符 1) */
```

```
extern FILE *stderr;          /* 标准错误(描述符 2) */
```

FILE 类型的流是对文件描述符和一个流缓冲区的抽象。流缓冲区的目的和 Rio 读缓冲区的目的相同：尽量减少开销高昂的 Linux I/O 系统调用次数。例如，假设我们有一个程序反复调用标准 I/O 的 getc 函数，其中每次调用都返回文件中的下一个字符。当 getc 第一次被调用时，库会通过一次对 read 函数的调用填满流缓冲区，然后把缓冲区中的第一个字节返回给应用程序。只要缓冲区中还有未读的字节，后续对 getc 的调用就可以直接由流缓冲区提供服务。

10.11 综合：应该使用哪些 I/O 函数？

图 10.16 总结了本章讨论的各种 I/O 包。

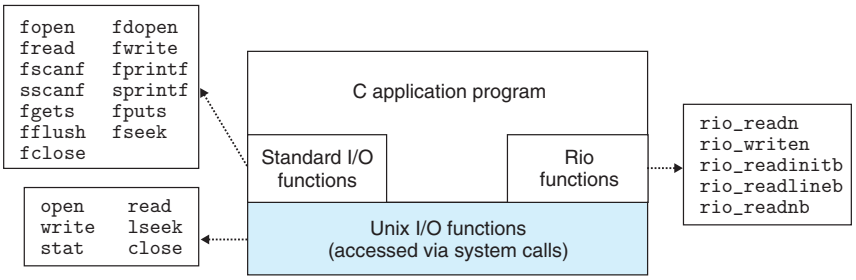


图 10.16　Unix I/O、标准 I/O 和 Rio 之间的关系。

Unix I/O 模型是在操作系统内核中实现的，应用程序可以通过 open、close、lseek、read、write 和 stat 这样的函数使用它。较高级的 Rio 函数和标准 I/O 函数是“架在”Unix I/O 函数（利用它们）之上实现的。Rio 函数是专门为本书开发的 read 和 write 的健壮包装。它们自动处理短计数，并为读取文本行提供了高效的带缓冲方式。标准 I/O 函数提供了对 Unix I/O 函数更完整的带缓冲替代，包括诸如 printf 和 scanf 这样的格式化 I/O 例程。

那么，在你的程序里应该使用这些函数中的哪一种呢？下面是一些基本的准则：

G1：只要有可能，就使用标准 I/O 函数。对于磁盘和终端设备上的 I/O，标准 I/O 函数是首选方法。许多 C 程序员在他们的整个职业生涯中都用而且只用标准 I/O，从不使用较底层的 Unix I/O 函数（可能除了 stat，在标准 I/O 库中没有与它对应的函数）。只要有可能，我们也建议你这样做。

G2：不要用 scanf 或 rio_readlineb 来读二进制文件。像 scanf 和 rio_readlineb 这样的函数是专为读取文本文件设计的。学生们常犯的一个错误是用这些函数读二进制数据，导致程序以奇怪且难以预料的方式失败。例如，二进制文件中可能散布着许多 0xa 字节，它们与文本行的结尾毫无关系。

G3：对网络套接字的 I/O 使用 Rio 函数。遗憾的是，用标准 I/O 函数对网络执行输入和输出会遇到一些令人头疼的问题。正如我们将在 11.4 节看到的，Linux 对网络的抽象是一类称为套接字（socket）的文件。像任何 Linux 文件一样，套接字由文件描述符引用，此处称为套接字描述符。应用程序进程通过读写套接字描述符与运行在其他计算机上的进程通信。

标准 I/O 流是全双工的，也就是说，程序可以在同一个流上既执行输入又执行输出。然而，流上有一些鲜有文档记载的限制，这些限制与套接字上的限制相互作用很差：

限制 1：输入函数跟在输出函数之后。如果中间没有对 `fflush`、`fseek`、`fsetpos` 或 `rewind` 函数的调用，输入函数就不能跟在输出函数之后。`fflush` 函数清空与一个流相关联的缓冲区。后三个函数用 Unix I/O 的 `lseek` 函数来重置当前的文件位置。

限制 2：输出函数跟在输入函数之后。除非输入函数遇到了文件结束，否则如果中间没有对 `fseek`、`fsetpos` 或 `rewind` 函数的调用，输出函数就不能跟在输入函数之后。

这些限制给网络应用程序带来了一个问题，因为在套接字上使用 `lseek` 函数是非法的。对流 I/O 的第一个限制可以通过养成一个纪律来绕过：每次输入操作之前都先清空缓冲区。然而，绕过第二个限制的唯一办法是在同一个打开的套接字描述符上打开两个流，一个用于读，一个用于写：

```
FILE *fpin, *fpout;

fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

但是这个方法也有问题，因为它要求应用程序对这两个流都调用 `fclose`，以释放与每个流相关联的内存资源，避免内存泄漏：

```
fclose(fpin);
fclose(fpout);
```

这两个操作都试图关闭同一个底层套接字描述符，所以第二个关闭操作会失败。这对顺序执行程序来说不是问题，但在一个线程程序中，关闭一个已经关闭的描述符会带来灾难（见 12.7.4 节）。

因此，我们建议你对网络套接字上的输入和输出不要使用标准 I/O 函数，而应当使用健壮的 Rio 函数。如果你需要格式化的输出，可以用 `sprintf` 函数在内存中格式化一个字符串，然后用 `rio_writen` 把它发送到套接字。如果你需要格式化的输入，可以用 `rio_readlineb` 读一整行文本，然后用 `sscanf` 从这行文本中提取不同的字段。

10.12 小结

Linux 以 Unix I/O 模型为基础，提供了少量系统级函数，应用程序可以用它们来打开、关闭、读和写文件，获取文件的元数据，以及执行 I/O 重定向。Linux 的读写操作会遇到短计数，应用程序必须预料到它们并正确处理。应用程序应当使用 Rio 包，而不是直接调用 Unix I/O 函数；Rio 包会自动处理短计数，方式是反复执行 `read` 和 `write` 操作，直到所有请求的数据都被传送为止。

Linux 内核用三个相关的数据结构来表示打开的文件。描述符表中的条目指向打开文件表中的条目，而打开文件表中的条目又指向 v-node 表中的条目。每个进程都有自己独有的描述符表，而所有进程共享同一个打开文件表和 v-node 表。理解这些数据结构的一般组织方式，有助于我们理解文件共享和 I/O 重定向。

标准 I/O 库是在 Unix I/O 之上实现的，它提供了一组功能强大的较高级 I/O 例程。对大多数应用程序而言，标准 I/O 比 Unix I/O 更简单，是首选。然而，由于标准 I/O 与网络文件各自都有一些彼此不兼容的限制，网络应用程序应当使用 Unix I/O，而不是标准 I/O。

10.13 参考文献说明

Kerrisk 对 Unix I/O 和 Linux 文件系统做了全面的论述 [62]。Stevens 写了 Unix I/O 最初的标准参考书 [111]。Kernighan 和 Ritchie 对标准 I/O 函数做了清晰而完整的讨论 [61]。

10.14 家庭作业

家庭作业 10.6 ▲ 下面的程序输出是什么？

```
1  #include "csapp.h"
2
3  int main()
4  {
5      int fd1, fd2;
6
7      fd1 = Open("foo.txt", O_RDONLY, 0);
8      fd2 = Open("bar.txt", O_RDONLY, 0);
9      Close(fd2);
10     fd2 = Open("baz.txt", O_RDONLY, 0);
11     printf("fd2 = %d\n", fd2);
12     exit(0);
13 }
```

家庭作业 10.7 ▲ 修改图 10.5 中的 `cpfile` 程序，使它使用 Rio 函数，每次以 `MAXBUF` 字节为单位把标准输入复制到标准输出。

家庭作业 10.8 ▲▲ 编写一个 statcheck 程序（图 10.10）的版本，称为 fstatcheck，让它从命令行接受一个描述符编号，而不是一个文件名。

家庭作业 10.9 ▲▲ 考虑对家庭作业 10.8 中 fstatcheck 程序的下述调用：

```
linux> fstatcheck 3 < foo.txt
```

你也许会以为这个调用会取出并显示文件 foo.txt 的元数据。然而，在本书作者的 Linux 系统上运行它时，却因“坏的文件描述符”（bad file descriptor）而失败。根据这个行为，填写 shell 在 fork 和 execve 调用之间必须执行的下述伪代码中的空缺：

```
if (Fork() == 0) { /* 子进程 */
    /* shell 在这里执行的是什么代码？ */
    Execve("fstatcheck", argv, envp);
}
```

家庭作业 10.10 ▲▲ 修改图 10.5 中的 cpfile 程序，使它接受一个可选的命令行参数 infile。如果给出了 infile，就把 infile 复制到标准输出；否则，像以前一样把标准输入复制到标准输出。玄机在于：你的解答必须在两种情形下都用原来的复制循环（第 9 ~ 11 行）。你只被允许插入代码，不能改动任何已有的代码。

10.15 练习答案

答案 10.1 Unix 进程从一开始就拥有分配给 stdin（描述符 0）、stdout（描述符 1）和 stderr（描述符 2）的打开描述符。open 函数总是返回最小的未打开描述符，所以第一次调用 open 会返回描述符 3。调用 close 函数释放了描述符 3。最后一次调用 open 会返回描述符 3，因此程序的输出是 fd2 = 3。

答案 10.2 描述符 fd1 和 fd2 各自拥有自己的打开文件表条目，因此每个描述符对 foobar.txt 都有自己独立的文件位置。于是，从 fd2 读出的将是 foobar.txt 的第一个字节，程序的输出是

```
c = f
```

而不是你可能一开始所想的

```
c = o
```

答案 10.3 回想一下，子进程会继承父进程的描述符表，并且所有进程共享同一个打开文件表。因此，父进程和子进程中的描述符 fd 都指向同一个打开文件表条目。当子进程读出文件第一个字节时，文件位置增加 1。于是，父进程读出的将是第二个字节，程序的输出是

```
c = o
```

答案 10.4 要想把标准输入(描述符 0)重定向到描述符 5,我们可以调用 `dup2(5,0)`,或者等价地,调用 `dup2(5,STDIN_FILENO)`。

答案 10.5 一开始你可能会以为输出会是

```
c = f
```

但由于我们把 `fd1` 重定向到了 `fd2`,真正的输出其实是

```
c = o
```

11

网络编程

网络应用无处不在。你浏览 Web、发送电子邮件、玩在线游戏的时候，用到的都是网络应用。有意思的是，所有的网络应用都基于同样的基本编程模型，具有相似的总体逻辑结构，并依赖同样的编程接口。

网络应用要用到许多你在学习系统的过程中已经见过的概念。例如，进程、信号、字节顺序、内存映射和动态存储分配都扮演着重要角色。同时也有一些新概念需要掌握。你需要理解基本的客户端-服务器编程模型，以及如何编写使用因特网所提供服务的客户端-服务器程序。在本章的最后，我们会把这些想法综合起来，开发一个小但完整的 Web 服务器，它能用文本和图形向真实的 Web 浏览器同时提供静态和动态内容。

11.1 客户端-服务器编程模型

每一个网络应用都基于客户端-服务器模型。按照这个模型，一个应用由一个服务器进程和一个或多个客户端进程组成。服务器管理某种资源，并通过操作这种资源为它的客户端提供某种服务。例如，Web 服务器管理一组磁盘文件，它代表客户端检索并执行这些文件。FTP 服务器管理一组磁盘文件，它为客户端存储与检索这些文件。类似地，电子邮件服务器管理一个假脱机文件（spool file），它为客户端读取与更新这个文件。

客户端-服务器模型中的基本操作是事务（transaction，图 11.1）。一次客户端-服务器事务由四个步骤组成：

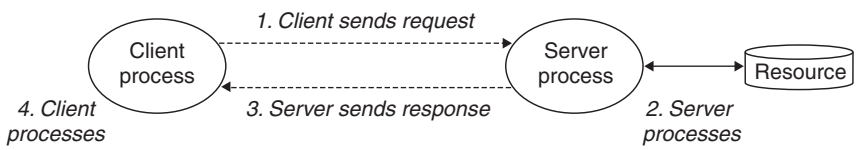


图 11.1　客户端-服务器事务。

1. 当客户端需要服务时，它通过向服务器发送一个请求来发起一次事务。例如，当 Web 浏览器需要一个文件时，它向 Web 服务器发送一个请求。
2. 服务器收到请求，对它进行解释，并以适当的方式操作它的资源。例如，当 Web 服务器收到浏览器的请求时，它读取一个磁盘文件。
3. 服务器向客户端发送响应，然后等待下一个请求。例如，Web 服务器把文件发回客户端。
4. 客户端收到响应并处理它。例如，当 Web 浏览器从服务器收到一个页面时，把它显示在屏幕上。

必须认识到，客户端和服务器是进程，而不是机器——在这种语境下，机器常常被称为主机。一台单独的主机可以并发地运行多个不同的客户端和服务器，一次客户端与服务器之间的事务可以在同一台主机上进行，也可以在不同主机之间进行。无论客户端和服务器如何映射到主机，客户端-服务器模型都是相同的。

> **边栏　客户端-服务器事务与数据库事务**
> 客户端-服务器事务不是数据库事务，也不具有数据库事务的任何属性，例如原子性。在我们的语境中，事务不过是由一个客户端和一个服务器执行的一系列步骤。

11.2 网络

客户端和服务器经常运行在不同的主机上，并利用计算机网络中的硬件和软件资源来通信。网络是复杂的系统，我们只能在这里大致触及它的表面。我们的目标是给你一个从程序员角度得到的、可用的心理模型。

对主机而言，网络不过是另一种 I/O 设备，它是数据的源和汇，如图 11.2 所示。插在 I/O 总线扩展槽中的网络适配器提供了到网络的物理接口。从网络接收到的数据经由 I/O 总线和内存总线，典型情况下通过 DMA 传送被复制到存储器中。类似地，数据也可以从存储器复制到网络。

从物理上讲，网络是一个按照地理上的临近程度组织起来的层次系统。最低层是局域网（LAN，local area network），覆盖一座建筑或一个园区。到目前为止最流行的局域网技术是以太网（Ethernet），它是 20 世纪 70 年代中期在施乐帕克研究中心（Xerox PARC）开发出来的。以太网已经证明了它有非凡的生命力，速率从 3 Mb/s 演进到了 10 Gb/s。

一个以太网段（segment）由一些导线（通常是双绞线）和一个称为集线器（hub）的小盒子组成，如图 11.3 所示。以太网段通常覆盖较小的区域，比如一个房间，或者一栋楼里的一层。每根导线都有相同的最大位带宽，典型的是 100

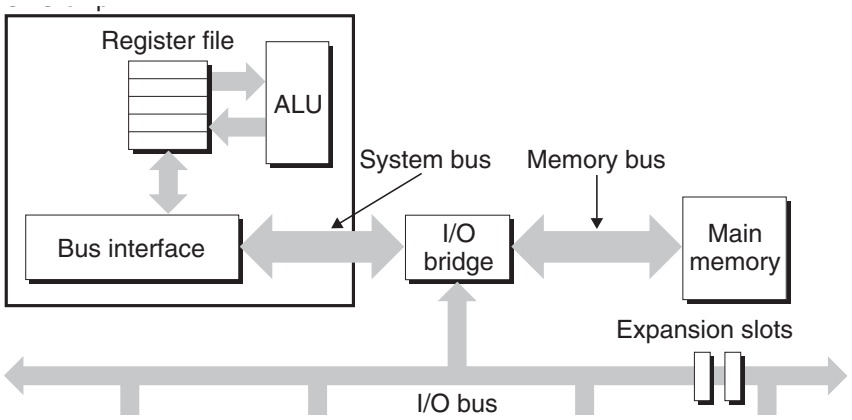


图 11.2 网络主机的硬件组成。

Mb/s 或 1 Gb/s。一端接到主机上的一块适配器,另一端接到集线器的一个端口上。集线器不加区分地把它在每个端口上收到的每一位复制到其他所有端口上。这样,每台主机都能看到每一个位。

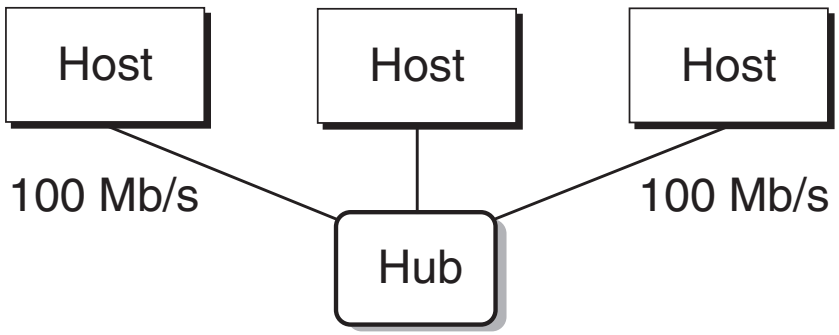


图 11.3 以太网网段。

每个以太网适配器都有一个全局唯一的 48 位地址,保存在适配器的一个非易失性存储器中。一台主机可以向网段上任意其他的主机发送一串称为帧(frame)的位块。每个帧都包含一些固定数目的头部位,用来标识帧的源和目的以及帧长度,后面跟着数据位组成的有效载荷(payload)。每台主机适配器都能看到这个帧,但只有目的主机才真正读取它。

多个以太网段可以用一组导线和称为网桥(bridge)的小盒子连接成较大的局域网,称为桥接以太网(bridged Ethernet),如图 11.4 所示。桥接以太网能覆盖整栋楼或整个园区。在一个桥接以太网中,有些导线连接网桥与网桥,另一些连接网桥与集线器。导线的带宽可以各不相同。在我们的例子中,网桥–网桥导线的带宽为 1 Gb/s,而四根集线器–网桥导线的带宽为 100 Mb/s。

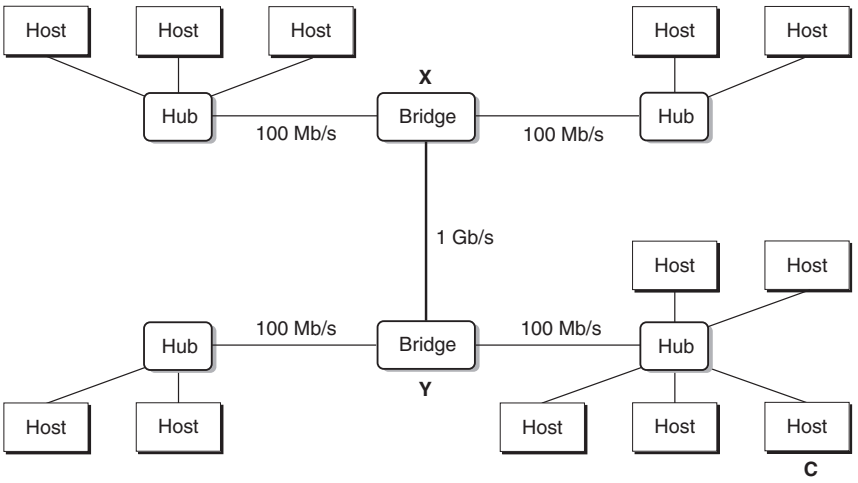


Figure 11.4 **Bridged Ethernet segments.**

Figure 11.5
Conceptual view of a



图 11.4 桥接的以太网网段。

网桥对可用导线带宽的利用率比集线器更高。网桥使用一种巧妙的分布式算法，会随时间自动学习哪些主机从哪些端口可以到达，然后只在必要的时候才把帧从一个端口选择性地复制到另一个端口。例如，如果主机 A 向同一网段上的主机 B 发送一个帧，那么当帧到达网桥 X 的输入端口时，网桥 X 会把这个帧扔掉，从而节省其他网段上的带宽。不过，如果主机 A 向位于另一个网段上的主机 C 发送帧，那么网桥 X 会把帧只复制到与网桥 Y 相连的端口上，网桥 Y 再把帧只复制到与主机 C 所在网段相连的端口上。

为了简化局域网的画法，我们把集线器和网桥以及连接它们的导线画成一条单一的水平线，如图 11.5 所示。

在层次结构中的更高一层，多个不兼容的局域网可以通过称为路由器（router）的专用计算机连接起来，构成一个互联网（internet，互联网络）。每台路由器对它所连接的每个网络都有一个适配器（端口）。路由器还可以连接高速的点对点电话连接，这些网络被称为广域网（WAN，wide area network），之所以这样称呼，是因为它们比局域网覆盖更大的地理区域。一般而言，路由器可以用来从局域网和广域网的任意集合构建互联网。例如，图 11.6 展示了一个互联网的例子，其中一对局域网和广域网由三台路由器连接起来。

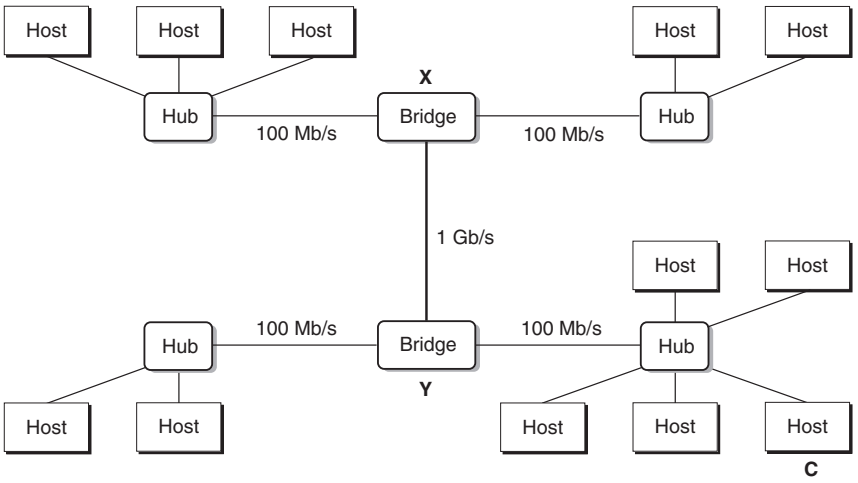


Figure 11.4 **Bridged Ethernet segments.**

Figure 11.5
Conceptual view of a



图 11.5 局域网(LAN)的概念视图。

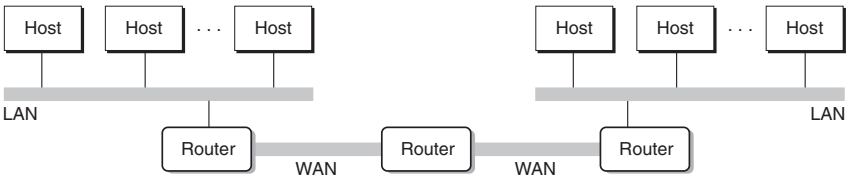


图 11.6 一个小型互联网。两个局域网和两个广域网由三台路由器连接起来。

> **边栏 Internet 与 internet**
> 我们始终用小写的 internet 表示一般概念，用大写的 Internet 表示一种特定的实现，即全球 IP 因特网。在本译本中，小写 internet 译作“互联网”，大写 Internet 译作“因特网”。

互联网的一个关键特性是，它可以由各种技术上截然不同、互不兼容的局域网和广域网组成。每台主机在物理上都与其他所有主机相连，但是某个源主机如何能跨过这么多互不兼容的网络，把数据位发送给另一个目的主机呢？

解决方案是在每台主机和路由器上运行的一层协议软件，它抹平了不同网络之间的差异。这层软件实现了一种协议，规范主机和路由器如何协作来传送数据。这种协议必须提供两种基本能力：

- **命名方案**。不同的局域网技术为主机分配地址的方式不同且互不兼容。互联网协议定义了一种统一格式的主机地址，从而抹平了这些差异。从此，每台主机都至少被分配一个互联网地址，用它来唯一地标识自己。
- **传送机制**。各种网络技术在导线上编码位、以及把这些位打包成帧的方式不同且互不兼容。互联网协议定义了一种统一的方式，把数据位捆绑成离散的、称为分组（packet）的块，从而抹平了这些差异。一个分组由头部和有效载荷组成，头部包含分组大小以及源主机和目的主机的地址，有效载荷则包含从源主机发来的数据位。

图 11.7 展示了一个例子，说明主机和路由器如何使用互联网协议在互不兼容的多个局域网上传送数据。这个例子中的互联网由一个路由器连接的两个局域网组成。运行在连接到 LAN1 上的主机 A 中的一个客户端，向运行在连接到 LAN2 上的主机 B 中的一个服务器发送一串数据字节。共有八个基本步骤：

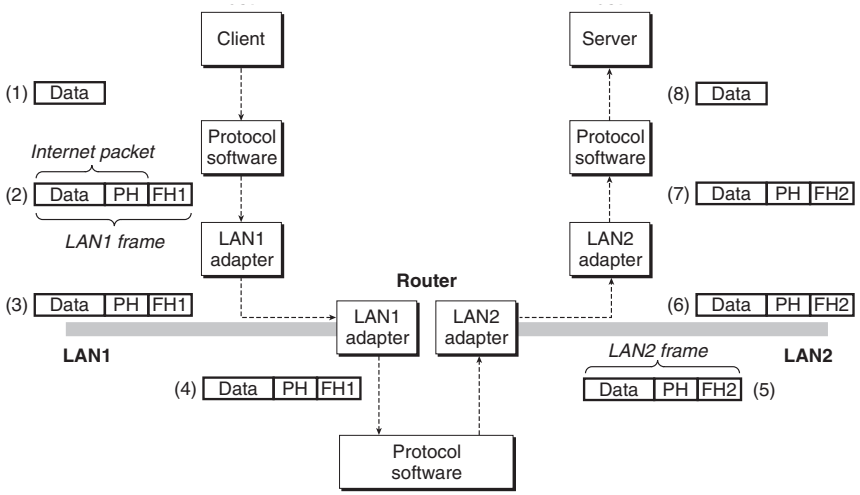


图 11.7 数据如何在一个互联网上从一台主机传到另一台主机。PH：网际分组首部；FH1：LAN1 的帧首部；FH2：LAN2 的帧首部。

1. 主机 A 上的客户端调用一个系统调用，把数据从客户端的虚拟地址空间复制到一个内核缓冲区中。
2. 主机 A 上的协议软件通过在数据前面附加一个互联网头部和一个 LAN1 帧头部，创建一个 LAN1 帧。互联网头部寻址到互联网主机 B；LAN1 帧头部寻址到路由器。然后协议软件把帧递交给适配器。注意，LAN1 帧的有效载荷是一个互联网分组，而互联网分组的有效载荷才是真正的用户数据。这种封装（encapsulation）是网际互连的基本思想之一。

3. LAN1 适配器把帧复制到网络中。
4. 当帧到达路由器时，路由器的 LAN1 适配器从导线上读取它，并递交给协议软件。
5. 路由器从互联网分组头部获取目的互联网地址，用它作为索引查询一张路由表，以决定把分组向哪里转发——在这个例子里就是 LAN2。然后，路由器剥去原来的 LAN1 帧头部，加上一个新的、寻址到主机 B 的 LAN2 帧头部，并把得到的帧递交给适配器。
6. 路由器的 LAN2 适配器把帧复制到网络中。
7. 当帧到达主机 B 时，它的适配器从导线上读取帧，并递交给协议软件。
8. 最后，主机 B 上的协议软件剥去分组头部和帧头部。当服务器调用一个读取数据的系统调用时，协议软件最终会把得到的数据复制到服务器的虚拟地址空间中。

当然，我们在上面回避了许多困难的问题。如果不同的网络有不同的最大帧长怎么办？路由器如何知道该把帧转发到哪里？网络拓扑变化时，路由器如何得知？如果丢失了一个分组怎么办？尽管如此，我们的例子抓住了互联网思想的精髓，封装就是其中的关键。

11.3 全球 IP 因特网

全球 IP 因特网是迄今为止最著名、最成功的互联网实现。自 1969 年以来，它就以这样或那样的形式存在着。虽然因特网的内部体系结构复杂而且一直在变，但自 20 世纪 80 年代初以来，客户端-服务器应用的组织方式却一直保持着惊人的稳定。图 11.8 展示了一个因特网客户端-服务器应用基本的硬件和软件组织。

每台因特网主机上都运行着实现 TCP/IP 协议（Transmission Control Protocol/Internet Protocol，传输控制协议/网际协议）的软件，几乎每个现代计算机系统都支持它。因特网客户端和服务器混合使用套接字接口函数和 Unix I/O 函数来通信。（我们将在 11.4 节描述套接字接口。）套接字函数通常实现为系统调用，它们会陷入内核，并调用 TCP/IP 中的各种内核态函数。

TCP/IP 实际上是一个协议族，其中每一个协议都贡献不同的能力。例如，IP 提供基本的命名方案和一种传送机制，把分组（称为数据报（datagram））从一台因特网主机送到任意其他主机。IP 机制是不可靠的，意思是说，如果数据报在网络中丢失或被重复，它不会做任何恢复的努力。UDP（Unreliable Datagram Protocol，不可靠数据报协议）对 IP 做了些许扩展，使得数据报能够在进程之间而不是主机之间传送。TCP 是一个建立在 IP 之上的复杂协议，为进程之间提

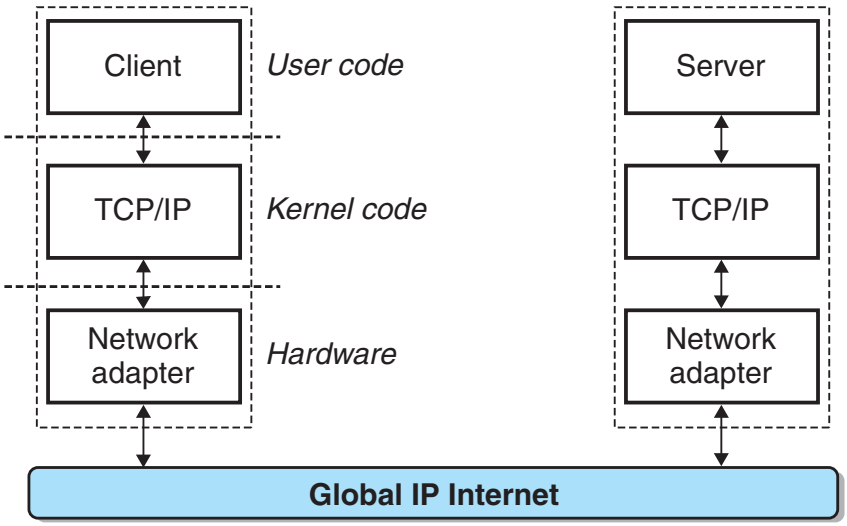


图 11.8 因特网应用的硬件和软件组成。

供可靠的全双工(双向)连接。为了简化讨论,我们把 TCP/IP 看成一个单一的、整体性的协议。我们不会讨论它的内部工作原理,而只讨论 TCP 和 IP 提供给应用程序的一些基本能力。我们不讨论 UDP。

从程序员的角度,我们可以把因特网看成是一个具有下列属性的世界范围的主机集合:

- 主机集合被映射到一组 32 位的 IP 地址。
- 这组 IP 地址被映射到一组称为因特网域名(Internet domain name)的标识符。
- 一台因特网主机上的进程可以通过连接与任何其他因特网主机上的进程通信。

边栏 IPv4 与 IPv6

原始的因特网协议使用 32 位地址,称为网际协议版本 4(IPv4)。1996 年,因特网工程任务组(IETF)提出了 IP 的一个新版本,称为网际协议版本 6(IPv6),它使用 128 位地址,本意是作为 IPv4 的后继者。然而,快 20 年过去了,截至 2015 年,绝大多数因特网流量仍然由 IPv4 网络承载。例如,只有 4% 的用户使用 IPv6 访问 Google 的服务 [42]。

由于采用率很低,本书不会详细讨论 IPv6,而只专注于 IPv4 背后的概念。当我们谈到因特网时,指的是基于 IPv4 的因特网。不过,本章稍后要教你的编写客户端和服务器的技术都基于现代接口,与任何特定协议无关。

后面几节将更详细地讨论这些基本的因特网概念。

11.3.1 IP 地址

一个 IP 地址就是一个无符号的 32 位整数。网络程序把 IP 地址存放在图 11.9 所示的 IP 地址结构(IP address structure)中。

把一个标量地址存放到结构里,是套接字接口早期实现遗留下来的一个不幸的做法。更合理的做法是为 IP 地址定义一个标量类型,但由于应用程序的安装量实在太庞大,现在已经来不及改变了。

因为因特网主机可以有不同的主机字节顺序,TCP/IP 为任意在分组头部中跨越网络传送的整数数据项(比如 IP 地址)定义了统一的网络字节顺序(network byte order,大端字节顺序)。IP 地址结构中的地址总是以(大端的)网络字节顺序存放,即使主机字节顺序是小端法。Unix 为在网络字节顺序和主机字节顺序之间进行转换提供了下列函数。

```
/* IP 地址结构 */
struct in_addr {
    uint32_t  s_addr; /* 以网络字节顺序(大端法)存放的地址 */
};
```

图 11.9 IP 地址结构。

```
#include <arpa/inet.h>
uint32_t htonl(uint32_t hostlong);
uint16_t htons(uint16_t hostshort);
        返回:网络字节顺序表示的值

uint32_t ntohl(uint32_t netlong);
uint16_t ntohs(uint16_t netshort);
        返回:主机字节顺序表示的值
```

htonl 函数把一个无符号 32 位整数从主机字节顺序转换成网络字节顺序。ntohl 函数把一个无符号 32 位整数从网络字节顺序转换成主机字节顺序。htons 和 ntohs 函数为无符号 16 位整数执行相应的转换。注意,没有针对 64 位值的等价函数。

IP 地址通常以点分十进制表示法(dotted-decimal notation)的形式呈现给人类,其中每个字节都用它的十进制值表示,字节之间用句点分隔。例如,128.2.194.242 就是地址 0x8002c2f2 的点分十进制表示。在 Linux 系统上,你可以用 hostname 命令确定自己主机的点分十进制地址:

```
linux> hostname -i
128.2.210.175
```

应用程序可以用函数 `inet_pton` 和 `inet_ntop` 在 IP 地址和点分十进制字符串之间来回转换。

```
#include <arpa/inet.h>
int inet_pton(AF_INET, const char *src, void *dst);
                返回:成功返回 1;src 不是有效的点分十进制返回 0;出错返回 -1

const char *inet_ntop(AF_INET, const void *src, char *dst,
                      socklen_t size);
                返回:成功返回指向点分十进制字符串的指针;出错返回 NULL
```

在这两个函数名中,“n”代表网络(network),“p”代表表示(presentation,即文本形式)。它们既可以操纵 32 位的 IPv4 地址(AF_INET),如这里所示,也可以操纵 128 位的 IPv6 地址(AF_INET6),后者不作介绍。

`inet_pton` 函数把点分十进制字符串(`src`)转换成网络字节顺序表示的二进制 IP 地址(`dst`)。如果 `src` 没有指向一个有效的点分十进制字符串,它就返回 0。任何其他错误都返回 -1 并设置 `errno`。类似地,`inet_ntop` 函数把网络字节顺序表示的二进制 IP 地址(`src`)转换成相应的点分十进制表示,并把结果中以 null 结尾的字符串中至多 `size` 个字节复制到 `dst`。

练习 11.1 补全下表:

点分十进制地址	十六进制地址
107.212.122.205	
64.12.149.13	
107.212.96.29	
	0x00000080
	0xFFFFFF00
	0x0A010140

练习 11.2 编写一个程序 hex2dd.c,把命令行传入的 16 位十六进制数参数转换成 16 位网络字节顺序,并打印结果。例如,

```
linux> ./hex2dd 0x400
1024
```

练习 11.3 编写一个程序 dd2hex.c,把 16 位网络字节顺序的数转换成一个 16 位十六进制数并打印结果。例如,

```
linux> ./dd2hex 1024
0x400
```

11.3.2 因特网域名

因特网客户端和服务器相互通信时使用的是 IP 地址。不过,大整数让人很难记住,所以因特网还定义了一组对人类更友好的域名,以及一种把域名集合映射到 IP 地址集合的机制。域名是一个用句点分隔的单词(字母、数字和连字符)序列,比如 whaleshark.ics.cs.cmu.edu。

域名集合构成一个层次结构，每个域名编码了它在层次结构中的位置。理解这一点最容易的方法是看一个例子。图 11.10 展示了这个域名层次结构的一部分。

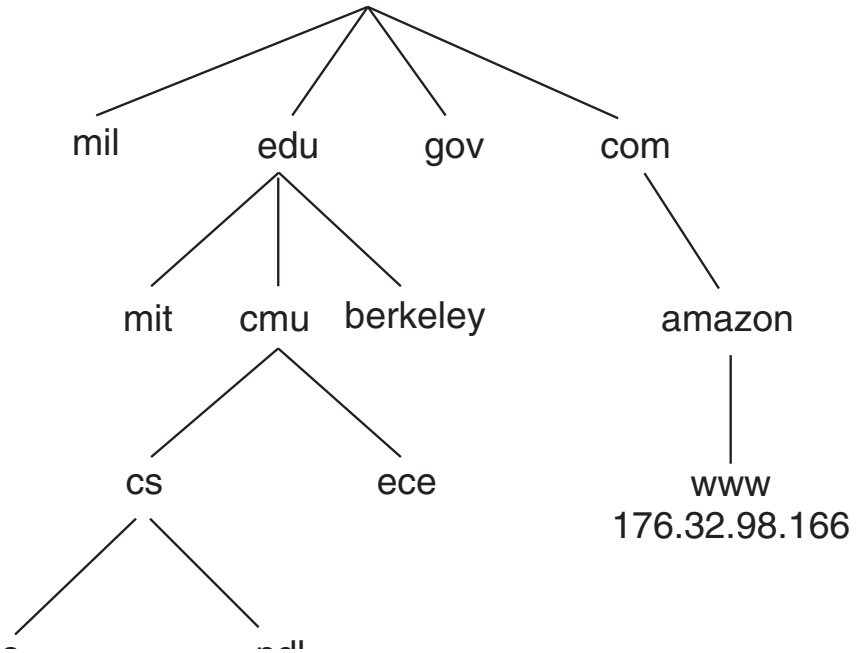


图 11.10 因特网域名层次结构的子集。

层次结构表示为一棵树。树的节点表示域名，由从该节点回溯到根的路径构成。子树被称为子域（subdomains）。层次结构的第一层是一个未命名的根节点。下一层是一组一级域名（first-level domain names），由一家称为 ICANN（Internet Corporation for Assigned Names and Numbers，互联网名称与数字地址分配机构）的非营利组织定义。常见的一级域名包括 com、edu、gov、org 和 net。

再下一层是 cmu.edu 这样的二级域名（second-level domain names），由 ICANN 的多家授权代理按先到先得的原则分配。一个组织一旦获得了一个二级域名，就可以随意在自己的子域中创建任何其他的新域名，比如 cs.cmu.edu。

因特网定义了域名集合和 IP 地址集合之间的一个映射。在 1988 年之前，这个映射一直由人工维护在一个名为 HOSTS.TXT 的单一文本文件中。从那以后，这个映射就维护在一个称为 DNS（Domain Name System，域名系统）的分布式世界范围数据库中。从概念上讲，DNS 数据库由数百万个主机条目（host entry）组成，每个主机条目定义了一组域名和一组 IP 地址之间的映射。从数学意义上说，可以认为每个主机条目就是域名和 IP 地址的一个等价类。我们可

以用 Linux 的 nslookup 程序来探究 DNS 映射的一些性质，该程序显示与一个域名相关联的 IP 地址。[1]

每台因特网主机都有一个本地定义的域名 localhost，它总是映射到环回地址（loopback address）127.0.0.1：

```
linux> nslookup localhost
Address: 127.0.0.1
```

localhost 这个名字提供了一种方便且可移植的方式，用来引用运行在同一台机器上的客户端和服务器，这对调试特别有用。我们可以用 hostname 来确定本地主机的正式域名：

```
linux> hostname
whaleshark.ics.cs.cmu.edu
```

在最简单的情况下，一个域名映射到一个唯一的 IP 地址：

```
linux> nslookup whaleshark.ics.cs.cmu.edu
Address: 128.2.210.175
```

不过，在某些情况下，多个域名会映射到同一个 IP 地址：

```
linux> nslookup cs.mit.edu
Address: 18.62.1.6
linux> nslookup eecs.mit.edu
Address: 18.62.1.6
```

在最一般的情况下，多个域名会映射到同一组多个 IP 地址：

```
linux> nslookup www.twitter.com
Address: 199.16.156.6
Address: 199.16.156.70
Address: 199.16.156.102
Address: 199.16.156.230
linux> nslookup twitter.com
Address: 199.16.156.102
Address: 199.16.156.230
Address: 199.16.156.6
Address: 199.16.156.70
```

最后，我们注意到，有些合法的域名并不映射到任何 IP 地址：

```
linux> nslookup edu
*** Can't find edu: No answer
linux> nslookup ics.cs.cmu.edu
*** Can't find ics.cs.cmu.edu: No answer
```

[1] 我们对 nslookup 的输出重新做了排版，以提高可读性。

11.3.3 因特网连接

因特网客户端和服务器通过在连接上发送和接收字节流来通信。从连接一对进程的意义上说，一个连接是点对点的。从数据可以同时在两个方向上流动的意义上说，它是全双工的。它还是可靠的——除了发生某些灾难性的故障，比如那条出了名的被粗心的挖掘机操作员挖断的光缆——从源进程发送的字节流最终会按其发送的顺序被目的进程接收到。

> **边栏 因特网上有多少台主机？**
> 自 1987 年开始，Internet 系统联盟（Internet Systems Consortium）每半年进行一次因特网域名调查。该调查通过统计已分配域名的 IP 地址的数量来估计因特网主机的数量，揭示出一种惊人的趋势。1987 年，因特网约有 20000 台主机；从那时起，主机数量一直在呈指数增长。到 2015 年，因特网主机已经超过 1000000000 台！

套接字（socket）是连接的一个端点。每个套接字都有一个相应的套接字地址（socket address），它由一个因特网地址和一个 16 位整数端口组成，记作 `address:port`。[2]

客户端套接字地址中的端口是客户端发起连接请求时由内核自动分配的，称为临时端口（ephemeral port）。但是，服务器套接字地址中的端口通常是一个与该服务永久关联的知名端口（well-known port）。例如，Web 服务器通常使用端口 80，而电子邮件服务器使用端口 25。与每个绑定到知名端口上的服务相关联，有一个对应的知名服务名（well-known service name）。例如，Web 服务的知名名是 http，电子邮件的知名名是 smtp。知名名与知名端口之间的映射关系保存在一个称为 /etc/services 的文件中。

一个连接由它两个端点的套接字地址唯一标识。这对套接字地址称为套接字对（socket pair），用下面的二元组表示：

$$(\texttt{cliaddr:cliport}, \texttt{servaddr:servport})$$

其中 cliaddr 是客户端的 IP 地址，cliport 是客户端的端口，servaddr 是服务器的 IP 地址，servport 是服务器的端口。例如，图 11.11 展示了一个 Web 客户端与一个 Web 服务器之间的连接。

在这个例子中，Web 客户端的套接字地址是

```
128.2.194.242:51213
```

其中端口 51213 是由内核分配的一个临时端口。Web 服务器的套接字地址是

[2] 这些软件端口与网络交换机和路由器中的硬件端口没有任何关系。

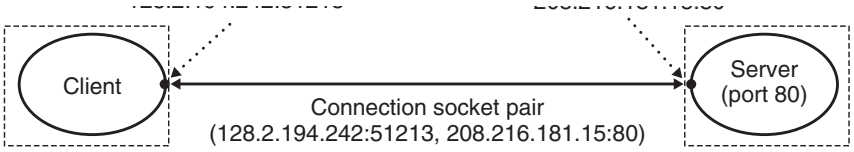


图 11.11 因特网连接的剖析。

```
208.216.181.15:80
```

其中端口 80 是与 Web 服务相关联的知名端口。给定客户端和服务器的套接字地址,客户端与服务器之间的连接就由下面这个套接字对唯一确定:

$$(\texttt{128.2.194.242:51213}, \texttt{208.216.181.15:80})$$

边栏 因特网的起源

因特网是政府、大学和产业界相互合作最成功的例子之一。促成它成功的因素很多,但我们认为有两个特别重要:美国政府持续 30 年的投入,以及满腔热忱的研究者们对麻省理工学院 Dave Clarke 所说的“粗糙的共识和能运行的代码”(rough consensus and working code)这一纲领的坚持。

因特网的种子在 1957 年播下。当时正处于冷战的高潮期,苏联发射了第一颗人造地球卫星斯普特尼克(Sputnik),令世界震惊。作为回应,美国政府创建了高级研究计划署(ARPA),它的使命是重建美国在科学与技术上的领先地位。1967 年,ARPA 的 Lawrence Roberts 发表了一个称为 ARPANET 的网络的规划。第一批 ARPANET 节点在 1969 年建立并运行起来。到 1971 年,已有 13 个 ARPANET 节点,电子邮件也作为第一个重要的网络应用出现了。

1972 年,Robert Kahn 概述了网际互连的一般原理:一个由相互连接的网络组成的集合,网络之间的通信由称为“路由器”的“黑盒子”按“尽力而为”的方式独立处理。1974 年,Kahn 和 Vinton Cerf 发表了 TCP/IP 的最初细节,到了 1982 年,它已经成为 ARPANET 的标准网际互连协议。1983 年 1 月 1 日,ARPANET 上的每一个节点都切换到了 TCP/IP,这标志着全球 IP 因特网的诞生。

1985 年,Paul Mockapetris 发明了 DNS,因特网主机超过了 1000 台。第二年,美国国家科学基金会(NSF)建成了 NSFNET 骨干网,用 56 Kb/s 的电话线连接了 13 个站点。它于 1988 年升级为 1.5 Mb/s 的 T1 链路,1991 年升级为 45 Mb/s 的 T3 链路。到 1988 年,已有 50000 多台主机。1989 年,最初的 ARPANET 正式退役。1995 年,当因特网主机接近 10000000 台时,NSF 退役了 NSFNET,代之以现代的因特网体系结构——由公共网络接入点连接起来的私有商业骨干网。

11.4 套接字接口

套接字接口(sockets interface)是一组函数,它们与 Unix I/O 函数结合起来,用来构建网络应用。它已经被实现于大多数现代系统上,包括所有的 Unix 变种以及 Windows 和 Macintosh 系统。图 11.12 从一次典型的客户端-服务器事

务的角度给出了套接字接口的概览。在我们讨论各个函数时，你应当把这幅图当作路线图来用。

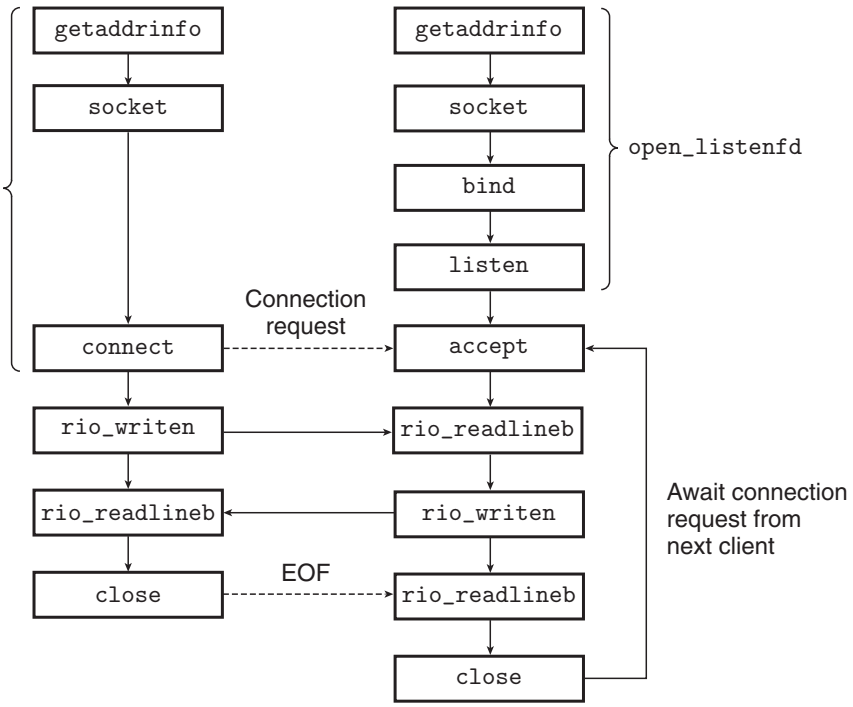


图 11.12 基于套接字接口的网络应用概览。

边栏 套接字接口的起源

最初的套接字接口是 20 世纪 80 年代初由加州大学伯克利分校的研究者们开发的。因为这个缘故，它常常被称为伯克利套接字（Berkeley sockets）。伯克利的研究者们开发的套接字接口可以与任何底层协议协同工作。第一个实现是针对 TCP/IP 的，他们把它包含在 Unix 4.2BSD 内核中，分发给了众多的大学和实验室。这是因特网历史上一个重要的事件。几乎在一夜之间，成千上万的人接触到了 TCP/IP 及其源代码。它激发了极大的热情，掀起了网络和网际互连研究中一连串狂热的新工作。

```
/* 因特网(IP)套接字地址结构 */
struct sockaddr_in {
    uint16_t        sin_family; /* 协议族(总是 AF_INET) */
    uint16_t        sin_port;   /* 以网络字节顺序存放的端口号 */
    struct in_addr  sin_addr;   /* 以网络字节顺序存放的 IP 地址 */
    unsigned char   sin_zero[8];/* 补足到 sizeof(struct sockaddr) */
};

/* 通用套接字地址结构(用于 connect、bind 和 accept) */
struct sockaddr {
    uint16_t sa_family;   /* 协议族 */
```

```
    char      sa_data[14]; /* 地址数据 */
};
```

图 11.13 套接字地址结构。

11.4.1 套接字地址结构

从 Linux 内核的角度看，套接字是通信的一个端点。从 Linux 程序的角度看，套接字是一个有相应描述符的打开的文件。

因特网套接字地址存放在大小为 16 字节、类型为 sockaddr_in 的结构中，如图 11.13 所示。对因特网应用而言，sin_family 字段是 AF_INET，sin_port 字段是一个 16 位端口号，而 sin_addr 字段包含一个 32 位 IP 地址。IP 地址和端口号总是以网络（大端）字节顺序存放。

> **边栏 `_in` 后缀是什么意思？**
> `_in` 后缀是 internet 的缩写，不是 input 的缩写。

connect、bind 和 accept 函数要求一个指向特定协议的套接字地址结构的指针。套接字接口的设计者面临的问题是，如何定义这些函数，使之能够接受任何类型的套接字地址结构。放在今天，我们会使用通用的 `void *` 指针，但这种指针当年在 C 语言里还不存在。他们的解决方案是，把套接字函数定义为要求传入一个指向通用 sockaddr 结构的指针（图 11.13），然后要求应用程序把指向特定协议的结构的指针强制转换成这个通用结构的指针。为了简化示例代码，我们遵照 Stevens 的做法，定义如下类型：

```
typedef struct sockaddr SA;
```

之后，每当需要把一个 sockaddr_in 结构强制转换成通用的 sockaddr 结构时，我们就使用这个类型。

11.4.2 socket 函数

客户端和服务器用 socket 函数来创建一个套接字描述符。

```
#include <sys/types.h>
#include <sys/socket.h>

int socket(int domain, int type, int protocol);
        返回：成功返回非负描述符，出错返回 -1
```

如果我们想让套接字成为连接的一个端点，那么可以用下面的硬编码参数来调用 socket：

```
clientfd = Socket(AF_INET, SOCK_STREAM, 0);
```

其中 AF_INET 表示我们使用 32 位的 IP 地址，而 SOCK_STREAM 表示该套接字将成为连接的一个端点。不过，最佳实践是使用 getaddrinfo 函数（11.4.7 节）来自动生成这些参数，使代码与协议无关。我们将在 11.4.8 节展示如何把 getaddrinfo 和 socket 函数一起使用。

socket 返回的 clientfd 描述符只是部分打开的，还不能用于读写。如何完成套接字的打开，取决于我们是客户端还是服务器。下一节描述当我们作为客户端时如何完成打开套接字的工作。

11.4.3 connect 函数

客户端通过调用 connect 函数来建立与一个服务器的连接。

```
#include <sys/socket.h>
int connect(int clientfd, const struct sockaddr *addr,
            socklen_t addrlen);
                                        返回：成功返回 0，出错返回 -1
```

connect 函数试图与套接字地址 addr 处的服务器建立一次因特网连接，其中 addrlen 是 sizeof(sockaddr_in)。connect 函数会阻塞，直到连接成功建立或者发生错误为止。如果成功，clientfd 描述符现已就绪、可读可写，得到的连接由下面这个套接字对刻画：

$$(x{:}y, addr.sin_addr{:}addr.sin_port)$$

其中 x 是客户端的 IP 地址，y 是唯一标识客户端主机上的该客户端进程的临时端口。与 socket 一样，最佳实践是用 getaddrinfo 为 connect 提供参数（见 11.4.8 节）。

11.4.4 bind 函数

其余的套接字函数——bind、listen 和 accept——由服务器用来建立与客户端的连接。

```
#include <sys/socket.h>
int bind(int sockfd, const struct sockaddr *addr,
         socklen_t addrlen);
                                        返回：成功返回 0，出错返回 -1
```

bind 函数要求内核把 addr 中的服务器套接字地址与套接字描述符 sockfd 关联起来。addrlen 参数是 sizeof(sockaddr_in)。与 socket 和 connect 一样，最佳实践是用 getaddrinfo 为 bind 提供参数（见 11.4.8 节）。

11.4.5 listen 函数

客户端是发起连接请求的主动实体。服务器是等待来自客户端的连接请求的被动实体。默认情况下，内核假定 socket 函数创建的描述符对应于一个主动套接字——它将存在于连接的客户端一端。服务器调用 listen 函数来告诉内核，该描述符将被服务器而不是客户端使用。

```
#include <sys/socket.h>

int listen(int sockfd, int backlog);
                                        返回：成功返回 0，出错返回 -1
```

listen 函数把 sockfd 从一个主动套接字转换成一个监听套接字，它可以接受来自客户端的连接请求。backlog 参数是一个提示，指示内核在开始拒绝请求之前，应该为多少个未处理的连接请求排队。backlog 参数的确切含义需要对 TCP/IP 的理解，超出了我们的范围。我们通常把它设置成一个较大的值，比如 1024。

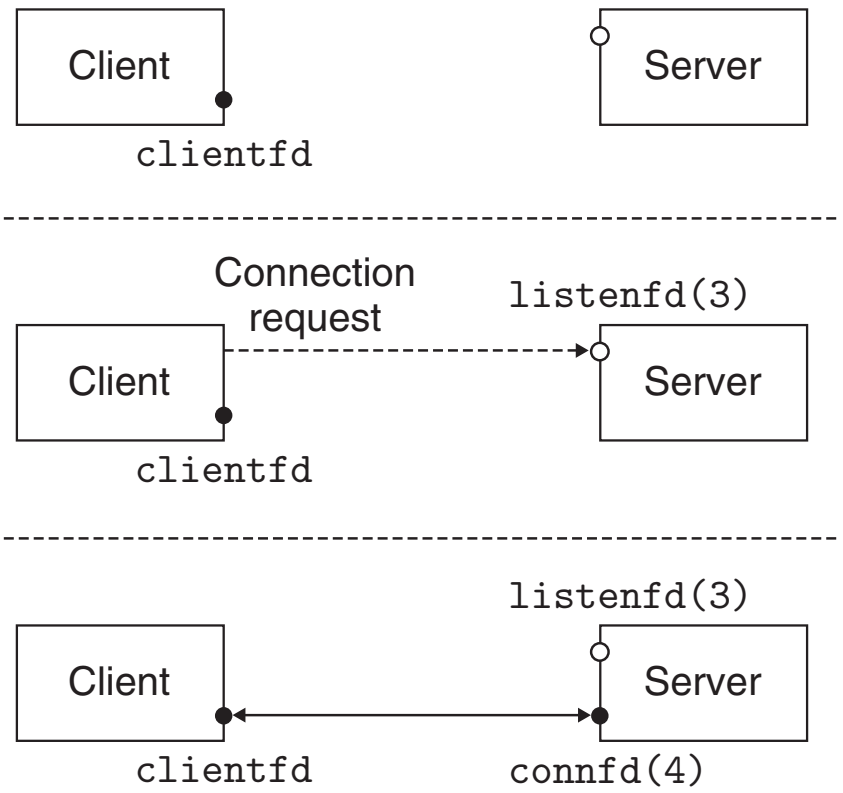


图 11.14 监听描述符和已连接描述符的角色。

11.4.6 accept 函数

服务器通过调用 accept 函数来等待来自客户端的连接请求。

```
#include <sys/socket.h>

int accept(int listenfd, struct sockaddr *addr, int *addrlen);
                返回:成功返回非负的已连接描述符,出错返回 -1
```

accept 函数等待从客户端到来的连接请求到达监听描述符 listenfd,然后在 addr 中填入客户端的套接字地址,并返回一个已连接描述符,该描述符可以使用 Unix I/O 函数来与客户端通信。

监听描述符与已连接描述符之间的区别使许多学生感到困惑。监听描述符是客户端连接请求的一个端点。它通常只创建一次,并且在服务器的生存期内一直存在。已连接描述符是已建立起来的、客户端与服务器之间连接的端点。服务器每次接受一个连接请求都要创建一个,它只存在于服务器为该客户端提供服务所需的那段时间里。

图 11.14 概括了监听描述符和已连接描述符的角色。在第 1 步中,服务器调用 accept,等待连接请求到达监听描述符,为了具体起见,我们假设它是描述符 3。回忆一下,描述符 0 ~ 2 是留给标准文件的。

图 11.14　监听描述符和已连接描述符的角色。

在第 2 步中,客户端调用 connect 函数,向 listenfd 发送一个连接请求。在第 3 步中,accept 函数打开一个新的已连接描述符 connfd(我们假设它是描述符 4),在 clientfd 与 connfd 之间建立连接,然后把 connfd 返回给应用程序。客户端也随后从 connect 返回。从这一刻起,客户端和服务器就可以各自通过读写 clientfd 和 connfd 来来回回地传递数据了。

> **边栏　为什么要区分监听描述符和已连接描述符?**
> 你可能会想,套接字接口为什么要作出监听描述符与已连接描述符之间的区别。乍一看这似乎是不必要的复杂化。然而,区分这两者却相当有用,因为它允许我们构建能够同时处理多个客户端连接的并发服务器。例如,每当监听描述符上有一个连接请求到来,我们就可以 fork 一个新进程,通过已连接描述符与客户端通信。关于并发服务器,你将在第 12 章学到更多。

11.4.7 主机和服务转换

Linux 提供了一些强大的函数,称为 getaddrinfo 和 getnameinfo,用于在二进制的套接字地址结构和主机名、主机地址、服务名以及端口号的字符串表示之间来回转换。把它们与套接字接口结合起来使用,我们就能编写出与任何特定 IP 协议版本无关的网络程序。

getaddrinfo 函数

getaddrinfo 函数把主机名、主机地址、服务名和端口号的字符串表示转换成套接字地址结构。它是已过时的 gethostbyname 和 getservbyname 函数的现代替代者。与那些函数不同，它是可重入的（reentrant，见 12.7.2 节），并且可以使用任何协议。

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

int getaddrinfo(const char *host, const char *service,
                const struct addrinfo *hints,
                struct addrinfo **result);
                        返回：成功返回 0，出错返回非零的错误码

void freeaddrinfo(struct addrinfo *result);
                        返回：无

const char *gai_strerror(int errcode);
                        返回：错误消息
```

给定 host 和 service（一个套接字地址的两个组成部分），getaddrinfo 返回一个 result，指向一个 addrinfo 结构的链表，链表中的每个 addrinfo 结构都指向一个与 host 和 service 对应的套接字地址结构（图 11.15）。

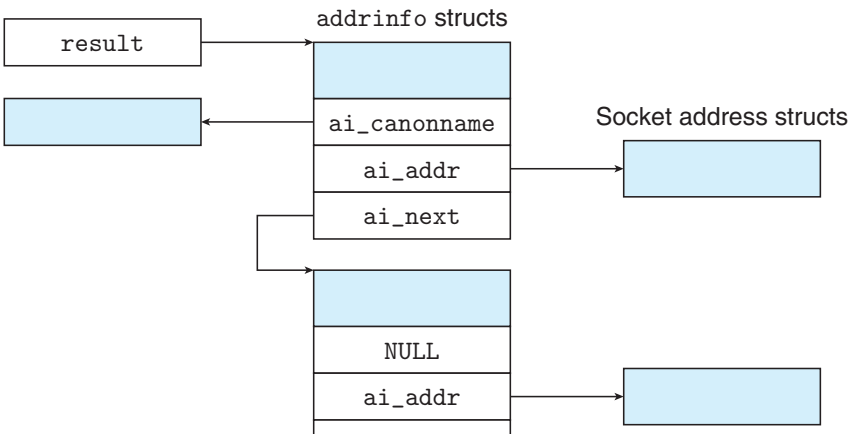


图 11.15 getaddrinfo 返回的数据结构。

客户端调用 getaddrinfo 之后，就遍历这个列表，逐个尝试其中的套接字地址，直到对 socket 和 connect 的调用成功，连接得以建立。类似地，服务器也尝试列表中的每个套接字地址，直到对 socket 和 bind 的调用成功，描述符绑定到一个有效的套接字地址为止。为避免内存泄漏，应用程序最终必须调用

freeaddrinfo 来释放这个列表。如果 getaddrinfo 返回一个非零的错误码，应用程序可以调用 gai_strerror 把错误码转换成一个消息字符串。

getaddrinfo 的 host 参数既可以是域名，也可以是数字形式的地址（比如点分十进制的 IP 地址）。service 参数既可以是服务名（比如 http），也可以是一个十进制的端口号。如果我们不打算把主机名转换成地址，可以把 host 设为 NULL。对 service 同样如此。不过，两者至少必须指定一个。

可选的 hints 参数是一个 addrinfo 结构（图 11.16），它对 getaddrinfo 返回的套接字地址列表提供了更精细的控制。作为 hints 参数传入时，只有 ai_family、ai_socktype、ai_protocol 和 ai_flags 字段可以设置。其他字段必须设为 0（或 NULL）。在实践中，我们先用 memset 把整个结构清零，然后再设置若干选定的字段：

- 默认情况下，getaddrinfo 可以同时返回 IPv4 和 IPv6 的套接字地址。把 ai_family 设置为 AF_INET 会把列表限制为 IPv4 地址。设置为 AF_INET6 则把列表限制为 IPv6 地址。

```
struct addrinfo {
    int              ai_flags;     /* hints 参数的标志 */
    int              ai_family;    /* socket 函数的第一个参数 */
    int              ai_socktype;  /* socket 函数的第二个参数 */
    int              ai_protocol;  /* socket 函数的第三个参数 */
    char            *ai_canonname; /* 规范(正式)主机名 */
    size_t           ai_addrlen;   /* ai_addr 结构的大小 */
    struct sockaddr *ai_addr;      /* 指向套接字地址结构的指针 */
    struct addrinfo *ai_next;      /* 指向链表下一项的指针 */
};
```

图 11.16　getaddrinfo 使用的 addrinfo 结构。

- 默认情况下，对于与 host 关联的每个唯一地址，getaddrinfo 函数最多可以返回三个 addrinfo 结构，每个结构的 ai_socktype 字段各不相同：一个用于连接，一个用于数据报（不介绍），还有一个用于原始套接字（不介绍）。把 ai_socktype 设置为 SOCK_STREAM 会把列表限制为每个唯一地址至多一个 addrinfo 结构，该结构的套接字地址可以作为连接的端点。这是对我们所有示例程序来说都合乎需要的行为。
- ai_flags 字段是一个进一步修改默认行为的位掩码。通过对多个值的不同组合做按位或运算来创建它。下面是我们发现有用的几个：
  - **AI_ADDRCONFIG.** 如果你在使用连接，就推荐这个标志 [34]。它要求 getaddrinfo 只有在本地主机配置了 IPv4 的情况下才返回 IPv4 地址。对 IPv6 同理。

- **AI_CANONNAME.** 默认情况下，ai_canonname 字段为 NULL。如果设置了这个标志，它会指示 getaddrinfo 把列表中第一个 addrinfo 结构的 ai_canonname 字段指向 host 的规范（正式）名字（见图 11.15）。
- **AI_NUMERICSERV.** 默认情况下，service 参数可以是服务名，也可以是端口号。这个标志强制 service 参数是一个端口号。
- **AI_PASSIVE.** 默认情况下，getaddrinfo 返回的套接字地址可供客户端在调用 connect 时作为主动套接字使用。这个标志指示它返回的套接字地址可供服务器用作监听套接字。这种情况下，host 参数应当为 NULL。得到的套接字地址结构中的地址字段将是通配地址（wildcard address），它告诉内核，该服务器将接受发往本主机任何 IP 地址的请求。这是对我们所有示例服务器来说都合乎需要的行为。

当 getaddrinfo 在输出列表中创建一个 addrinfo 结构时，它会填入除 ai_flags 之外的每个字段。ai_addr 字段指向一个套接字地址结构，ai_addrlen 字段给出这个套接字地址结构的大小，ai_next 字段指向列表中下一个 addrinfo 结构。其余字段描述该套接字地址的各种属性。

getaddrinfo 的一个精妙之处在于，addrinfo 结构中的字段是不透明的（opaque），意思是说，它们可以直接传给套接字接口中的函数，应用程序代码不需要任何进一步的处理。例如，ai_family、ai_socktype 和 ai_protocol 可以直接传给 socket。类似地，ai_addr 和 ai_addrlen 可以直接传给 connect 和 bind。这个强大的特性让我们能够编写独立于任何特定 IP 协议版本的客户端和服务器。

getnameinfo 函数

getnameinfo 函数是 getaddrinfo 的逆函数。它把一个套接字地址结构转换成相应的主机名和服务名字符串。它是已过时的 gethostbyaddr 和 getservbyport 函数的现代替代者，而且与那些函数不同，它是可重入的，且与协议无关。

```
#include <sys/socket.h>
#include <netdb.h>
int getnameinfo(const struct sockaddr *sa, socklen_t salen,
                char *host, size_t hostlen,
                char *service, size_t servlen, int flags);
                        返回：成功返回 0，出错返回非零的错误码
```

sa 参数指向一个大小为 salen 字节的套接字地址结构，host 指向一个大小为 hostlen 字节的缓冲区，service 指向一个大小为 servlen 字节的缓冲区。getnameinfo 函数把套接字地址结构 sa 转换成相应的主机名和服务名字符串，

并复制到 host 和 service 缓冲区中。如果 getnameinfo 返回一个非零的错误码，应用程序可以调用 gai_strerror 把它转换成一个字符串。

如果我们不想要主机名，可以把 host 设为 NULL，并把 hostlen 设为 0。对 service 同样如此。不过，两者必须设置其一。

flags 参数是一个修改默认行为的位掩码。通过对多个值的不同组合做按位或运算来创建它。下面是几个有用的：

- **NI_NUMERICHOST.** 默认情况下，getnameinfo 试图在 host 中返回一个域名。设置这个标志会让它改为返回一个数字形式的地址字符串。
- **NI_NUMERICSERV.** 默认情况下，getnameinfo 会查找 /etc/services，并尽可能返回服务名而不是端口号。设置这个标志会强制它跳过这次查找，直接返回端口号。

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct addrinfo *p, *listp, hints;
6      char buf[MAXLINE];
7      int rc, flags;
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <domain name>\n", argv[0]);
11         exit(0);
12     }
13
14     /* 获取 addrinfo 记录的列表 */
15     memset(&hints, 0, sizeof(struct addrinfo));
16     hints.ai_family = AF_INET;       /* 只用 IPv4 */
17     hints.ai_socktype = SOCK_STREAM; /* 只用于连接 */
18     if ((rc = getaddrinfo(argv[1], NULL, &hints, &listp)) != 0) {
19         fprintf(stderr, "getaddrinfo error: %s\n",
 ↪  gai_strerror(rc));
20         exit(1);
21     }
22
23     /* 遍历列表并显示每个 IP 地址 */
24     flags = NI_NUMERICHOST; /* 显示地址字符串,而不是域名 */
25     for (p = listp; p; p = p->ai_next) {
26         Getnameinfo(p->ai_addr, p->ai_addrlen, buf, MAXLINE, NULL,
 ↪  0,
       flags);
27         printf("%s\n", buf);
28     }
29
30     /* 清理 */
31     Freeaddrinfo(listp);
32
33     exit(0);
34 }
```

图 11.17 hostinfo 显示一个域名到其相关联的 IP 地址的映射。

图 11.17 展示了一个称为 hostinfo 的简单程序，它使用 getaddrinfo 和 getnameinfo 来显示一个域名到其相关联 IP 地址的映射。它类似于 11.3.2 节中的 nslookup 程序。

首先，我们初始化 hints 结构，使 getaddrinfo 返回我们想要的地址。在这里，我们要找的是可以用作连接端点的 32 位 IP 地址（第 16 和第 17 行）。因为我们只要求 getaddrinfo 转换域名，因此调用它时将 service 参数设为 NULL。

调用 getaddrinfo 之后，我们遍历 addrinfo 结构的列表，用 getnameinfo 把每个套接字地址都转换成一个点分十进制的地址字符串。遍历完列表之后，我们小心地调用 freeaddrinfo 把它释放掉（虽然对这个简单程序来说并非严格必要）。

运行 hostinfo 时，我们看到 twitter.com 映射到四个 IP 地址，这与我们在 11.3.2 节中用 nslookup 看到的输出一致。

```
linux> ./hostinfo twitter.com
199.16.156.102
199.16.156.230
199.16.156.6
199.16.156.70
```

练习 11.4 getaddrinfo 和 getnameinfo 函数分别涵盖了 inet_pton 和 inet_ntop 的功能，并提供了与任何特定地址格式无关的更高层次的抽象。为了让你自己体会这有多方便，请编写一个版本的 hostinfo（图 11.17），它用 inet_ntop 而不是 getnameinfo 来把每个套接字地址转换成点分十进制的地址字符串。

11.4.8 套接字接口的辅助函数

初次接触时，getaddrinfo 函数和套接字接口会显得有点吓人。我们发现，把它们用更高层级的辅助函数 open_clientfd 和 open_listenfd 包起来会很方便，客户端和服务器想要相互通信时都会用到这两个函数。

open_clientfd 函数

客户端通过调用 open_clientfd 来建立与一个服务器的连接。

```
#include "csapp.h"
int open_clientfd(char *hostname, char *port);
                                    返回：成功返回描述符，出错返回 -1
```

open_clientfd 函数与一个运行在主机 hostname 上、在端口号 port 上监听连接请求的服务器建立连接。它返回一个打开的套接字描述符，已可以用 Unix I/O 函数进行输入和输出。图 11.18 展示了 open_clientfd 的代码。

我们调用 getaddrinfo，它返回一个 addrinfo 结构的列表，每个结构都指向一个套接字地址结构，可用于连接到运行在 hostname 上并在 port 上监听的服务器。

然后我们遍历这个列表，逐个尝试其中的列表项，直到对 socket 和 connect 的调用成功。如果 connect 失败，我们会小心地在尝试下一个列表项之前关闭套接字描述符。如果 connect 成功，我们就释放列表占用的存储空间，并把套接字描述符返回给客户端，客户端就可以立即开始用 Unix I/O 与服务器通信。

```
1  int open_clientfd(char *hostname, char *port) {
2      int clientfd;
3      struct addrinfo hints, *listp, *p;
4
5      /* 获取候选服务器地址的列表 */
6      memset(&hints, 0, sizeof(struct addrinfo));
7      hints.ai_socktype = SOCK_STREAM; /* 打开一个连接 */
8      hints.ai_flags = AI_NUMERICSERV; /* ……使用数字形式的端口参数 */
9      hints.ai_flags |= AI_ADDRCONFIG; /* 推荐用于连接 */
10     Getaddrinfo(hostname, port, &hints, &listp);
11
12     /* 遍历列表,寻找我们能够成功连接的那一个 */
13     for (p = listp; p; p = p->ai_next) {
14         /* 创建一个套接字描述符 */
15         if ((clientfd = socket(p->ai_family, p->ai_socktype,
       p->ai_protocol)) < 0)
16             continue; /* 套接字失败,尝试下一个 */
17
18         /* 连接到服务器 */
19         if (connect(clientfd, p->ai_addr, p->ai_addrlen) != -1)
20             break; /* 成功 */
21         Close(clientfd); /* 连接失败,尝试另一个 */
22     }
23
24     /* 清理 */
25     Freeaddrinfo(listp);
26     if (!p) /* 所有连接都失败了 */
27         return -1;
28     else     /* 最后一次连接成功了 */
29         return clientfd;
30 }
```

图 11.18　open_clientfd：建立与一个服务器的连接的辅助函数。它是可重入的，且与协议无关。

注意，代码中的任何地方都不依赖于某个特定版本的 IP。socket 和 connect 的参数是由 getaddrinfo 为我们自动生成的，这使我们的代码整洁而可移植。

open_listenfd 函数

服务器通过调用 open_listenfd 函数来创建一个准备好接收连接请求的监听描述符。

```
#include "csapp.h"
int open_listenfd(char *port);
                                    返回:成功返回描述符,出错返回 -1
```

open_listenfd 函数返回一个在端口 port 上准备好接收连接请求的监听描述符。图 11.19 展示了 open_listenfd 的代码。

其风格与 open_clientfd 类似。我们调用 getaddrinfo,然后遍历结果列表,直到对 socket 和 bind 的调用成功。注意,在第 20 行我们使用 setsockopt 函数(此处不作描述)来配置服务器,使它可以被终止、重启并立即开始接受连接请求。默认情况下,重启后的服务器将在大约 30 秒内拒绝来自客户端的连接请求,这会严重妨碍调试。

由于我们调用 getaddrinfo 时带上了 AI_PASSIVE 标志并把 host 参数设为 NULL,每个套接字地址结构中的地址字段都被设置为通配地址,它告诉内核,该服务器将接受发往本主机任何 IP 地址的请求。

最后,我们调用 listen 函数把 listenfd 转换成一个监听描述符,并把它返回给调用者。如果 listen 失败,我们会小心地在返回之前关闭描述符,以避免内存泄漏。

```
1  int open_listenfd(char *port)
2  {
3      struct addrinfo hints, *listp, *p;
4      int listenfd, optval=1;
5
6      /* 获取候选服务器地址的列表 */
7      memset(&hints, 0, sizeof(struct addrinfo));
8      hints.ai_socktype = SOCK_STREAM;             /* 接受连接 */
9      hints.ai_flags = AI_PASSIVE | AI_ADDRCONFIG; /* ……在任何 IP
 ↪  地址上 */
10     hints.ai_flags |= AI_NUMERICSERV;            /* ……使用端口号 */
11     Getaddrinfo(NULL, port, &hints, &listp);
12
13     /* 遍历列表,寻找我们能够绑定的那一个 */
14     for (p = listp; p; p = p->ai_next) {
15         /* 创建一个套接字描述符 */
16         if ((listenfd = socket(p->ai_family, p->ai_socktype,
       p->ai_protocol)) < 0)
17             continue; /* 套接字失败,尝试下一个 */
18
19         /* 消除 bind 的“Address already in use”错误 */
20         Setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR,
21                    (const void *)&optval , sizeof(int));
22
23         /* 把描述符绑定到地址 */
24         if (bind(listenfd, p->ai_addr, p->ai_addrlen) == 0)
25             break; /* 成功 */
26         Close(listenfd); /* 绑定失败,尝试下一个 */
27     }
```

```
28
29      /* 清理 */
30      Freeaddrinfo(listp);
31      if (!p) /* 没有一个地址能工作 */
32          return -1;
33
34      /* 使之成为准备好接受连接请求的监听套接字 */
35      if (listen(listenfd, LISTENQ) < 0) {
36          Close(listenfd);
37          return -1;
38      }
39      return listenfd;
40  }
```

图 11.19 open_listenfd：打开并返回一个监听描述符的辅助函数。它是可重入的，且与协议无关。

11.4.9 回送客户端和服务器示例

学习套接字接口最好的方式是研读示例代码。图 11.20 展示了一个回送客户端的代码。与服务器建立连接之后，客户端进入一个循环，反复地从标准输入读取一行文本，把该文本行发送给服务器，从服务器读取回送的行，并把结果打印到标准输出。当 fgets 在标准输入上遇到 EOF 时，循环终止，这可能是因为用户在键盘上键入了 Ctrl+D，也可能是因为重定向输入文件中的文本行已经用尽。

```
1   #include "csapp.h"
2
3   int main(int argc, char **argv)
4   {
5       int clientfd;
6       char *host, *port, buf[MAXLINE];
7       rio_t rio;
8
9       if (argc != 3) {
10          fprintf(stderr, "usage: %s <host> <port>\n", argv[0]);
11          exit(0);
12      }
13      host = argv[1];
14      port = argv[2];
15
16      clientfd = Open_clientfd(host, port);
17      Rio_readinitb(&rio, clientfd);
18
19      while (Fgets(buf, MAXLINE, stdin) != NULL) {
20          Rio_writen(clientfd, buf, strlen(buf));
21          Rio_readlineb(&rio, buf, MAXLINE);
22          Fputs(buf, stdout);
23      }
24      Close(clientfd);
```

```
25     exit(0);
26 }
```

图 11.20　回送客户端的 main 例程。

循环终止之后，客户端关闭描述符。这将导致一个 EOF 通知被发送给服务器，当服务器从自己的 rio_readlineb 函数接收到零返回码时，它就检测到了这个通知。关闭描述符之后，客户端终止。因为客户端的内核会在进程终止时自动关闭所有打开的描述符，所以第 24 行的 close 不是必需的。不过，显式地关闭你打开过的任何描述符都是良好的编程实践。

图 11.21 展示了回送服务器的 main 例程。打开监听描述符之后，它进入一个无限循环。每次迭代都等待来自客户端的一个连接请求，打印出已连接客户端的域名和端口，然后调用为该客户端提供服务的 echo 函数。echo 例程返回之后，main 例程关闭已连接描述符。一旦客户端和服务器都关闭了各自的描述符，连接就终止了。

```
1  #include "csapp.h"
2
3  void echo(int connfd);
4
5  int main(int argc, char **argv)
6  {
7      int listenfd, connfd;
8      socklen_t clientlen;
9      struct sockaddr_storage clientaddr; /* 为任何地址准备足够空间 */
10     char client_hostname[MAXLINE], client_port[MAXLINE];
11
12     if (argc != 2) {
13         fprintf(stderr, "usage: %s <port>\n", argv[0]);
14         exit(0);
15     }
16
17     listenfd = Open_listenfd(argv[1]);
18     while (1) {
19         clientlen = sizeof(struct sockaddr_storage);
20         connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
21         Getnameinfo((SA *) &clientaddr, clientlen, client_hostname,
       MAXLINE,
22                     client_port, MAXLINE, 0);
23         printf("Connected to (%s, %s)\n", client_hostname,
       client_port);
24         echo(connfd);
25         Close(connfd);
26     }
27     exit(0);
28 }
```

图 11.21　迭代式回送服务器的 main 例程。

第 9 行的 clientaddr 变量是一个传给 accept 的套接字地址结构。在 accept 返回之前，accept 会把连接另一端的客户端的套接字地址填入 clientaddr。注意，我们把 clientaddr 声明为 struct sockaddr_storage 类型，而不是 struct sockaddr_in。根据定义，sockaddr_storage 结构大到足以容纳任何类型的套接字地址，这使代码与协议无关。

注意，我们这个简单的回送服务器每次只能处理一个客户端。像这样一次一个、按顺序处理客户端的服务器称为迭代式服务器（iterative server）。在第 12 章中，我们将学习如何构建更复杂的并发服务器，它们能够同时处理多个客户端。

最后，图 11.22 展示了 echo 例程的代码，它反复地读写文本行，直到第 10 行的 rio_readlineb 函数遇到 EOF。

```
1  #include "csapp.h"
2
3  void echo(int connfd)
4  {
5      size_t n;
6      char buf[MAXLINE];
7      rio_t rio;
8
9      Rio_readinitb(&rio, connfd);
10     while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
11         printf("server received %d bytes\n", (int)n);
12         Rio_writen(connfd, buf, n);
13     }
14 }
```

图 11.22　读入并回送文本行的 echo 函数。

边栏　连接上的 EOF 意味着什么？

EOF 的概念经常让学生困惑，尤其是在因特网连接的背景下。首先，我们要明白，并不存在一个 EOF 字符这样的东西。确切地说，EOF 是由内核检测到的一种情况（condition）。当应用程序从 read 函数得到一个为零的返回码时，它就会得知这个 EOF 情况。对磁盘文件来说，EOF 发生在当前文件位置超出了文件长度的时候。对因特网连接来说，EOF 发生在一个进程关闭了它这一端的连接的时候。连接另一端的进程在试图读取流中最后一个字节之后的数据时，就检测到了这个 EOF。

11.5 Web 服务器

到目前为止，我们都借由一个简单的回送服务器来讨论网络编程。在本节中，我们将向你展示如何使用网络编程的基本思想，来构建你自己的小而功能完备的 Web 服务器。

11.5.1 Web 基础

Web 客户端和服务器使用一种基于文本的应用层协议——HTTP (hypertext transfer protocol, 超文本传输协议)——来交互。HTTP 是一个简单的协议。Web 客户端(称为浏览器)向服务器打开一个因特网连接,请求某些内容。服务器响应对应的内容,然后关闭连接。浏览器读取内容并显示在屏幕上。

Web 服务与 FTP 这样的传统文件检索服务的区别是什么? 主要区别在于,Web 内容可以用一种称为 HTML(hypertext markup language, 超文本标记语言)的语言来写。一个 HTML 程序(页面)包含指令(标记,tag),告诉浏览器如何显示页面中的各种文本和图形对象。例如,代码

```
<b> Make me bold! </b>
```

告诉浏览器, 把 与 标记之间的文本用粗体打印出来。然而,HTML 的真正强大之处在于,一个页面可以包含指向存放在任何因特网主机的内容的指针(超链接,hyperlink)。例如,如下形式的 HTML 行

```
<a href="http://www.cmu.edu/index.html">Carnegie Mellon</a>
```

告诉浏览器把文本对象 Carnegie Mellon 高亮显示, 并创建一个指向存放在 CMU Web 服务器上、名为 index.html 的 HTML 文件的超链接。如果用户点击了这个高亮的文本对象,浏览器就会向 CMU 服务器请求相应的 HTML 文件并显示它。

> **边栏 万维网的起源**
> 万维网 (World Wide Web) 由 Tim Berners-Lee 发明, 他在瑞士一个物理学实验室 CERN 担任软件工程师。1989 年,Berners-Lee 写了一份内部备忘录,提议构建一个分布式超文本系统,用以连接“一个由带链接的笔记构成的网”。这个系统的目标是帮助 CERN 的科学家共享和管理信息。在其后的两年里,Berners-Lee 实现了第一个 Web 服务器和 Web 浏览器之后, Web 在 CERN 内部和其他一些站点中赢得了少数追随者。1993 年发生了一件转折性的事件:Marc Andreesen(他后来创立了 Netscape 和 Andreessen Horowitz)和他 NCSA 的同事们为所有三大平台——Linux、Windows 和 Macintosh——发布了一个名为 mosaic 的图形浏览器。mosaic 发布之后,人们对 Web 的兴趣呈爆发式增长,Web 站点的数量以指数速率增加。到 2015 年,全世界有超过 975,000,000 个站点。(资料来源:Netcraft Web Survey)

11.5.2 Web 内容

对 Web 客户端和服务器来说,内容(content)就是一个字节序列,并带有关联的 MIME(multipurpose internet mail extensions, 多用途网际邮件扩展)类型。图 11.23 展示了一些常见的 MIME 类型。

图 11.23 MIME 类型示例

MIME 类型	描述
`text/html`	HTML 页面
`text/plain`	无格式文本
`application/postscript`	Postscript 文档
`image/gif`	用 GIF 格式编码的二进制图像
`image/png`	用 PNG 格式编码的二进制图像
`image/jpeg`	用 JPEG 格式编码的二进制图像

Web 服务器以两种不同的方式向客户端提供内容:

- 抓取一个磁盘文件, 把它的内容返回给客户端。磁盘文件称为静态内容(static content), 把文件返回给客户端的过程称为服务静态内容 (serving static content)。
- 运行一个可执行文件,把它的输出返回给客户端。可执行文件在运行时产生的输出称为动态内容(dynamic content),运行这个程序并把输出返回给客户端的过程称为服务动态内容(serving dynamic content)。

Web 服务器返回的每一段内容都与它管理的某个文件相关联。这些文件中的每一个文件都有一个唯一的名字,称为 URL(universal resource locator,通用资源定位符)。例如,URL

```
http://www.google.com:80/index.html
```

标识因特网主机 www.google.com 上一个名为 /index.html 的 HTML 文件,该文件由一个监听端口 80 的 Web 服务器管理。端口号是可选的,默认为知名 HTTP 端口 80。可执行文件的 URL 可以在文件名后面包含程序参数。字符“?”把文件名与参数分开,每个参数用字符“&”分开。例如,URL

```
http://bluefish.ics.cs.cmu.edu:8000/cgi-bin/adder?15000&213
```

标识一个名为 /cgi-bin/adder 的可执行文件,调用它时将传入两个参数字符串: 15000 和 213。在一次事务中, 客户端和服务器使用的 URL 的部分是不同的。例如,客户端使用前缀

```
http://www.google.com:80
```

来确定需要联系哪种服务器、服务器在哪里以及它在哪个端口上监听。服务器使用后缀

```
/index.html
```

来在自己的文件系统上找到该文件，并判断请求的是静态内容还是动态内容。

关于服务器如何解释 URL 的后缀，有几点需要理解：

- 判定一个 URL 引用的是静态内容还是动态内容，没有标准的规则。每个服务器的规则针对的是它自己管理的文件。一种经典（老式）的做法是指定一组目录（例如 `cgi-bin`），要求所有可执行文件都必须放在这些目录里。
- 后缀中开头的“/”并不表示 Linux 根目录。确切地说，它表示所请求的那种内容的主目录（home directory）。例如，可以配置服务器，把所有静态内容存放在 /usr/httpd/html 目录中，而把所有动态内容存放在 /usr/httpd/cgi-bin 目录中。
- 最小的 URL 后缀是“/”字符，所有的服务器都会把它扩展成某个默认的主页，比如 /index.html。这就解释了为什么只要在浏览器中键入一个域名就能取回站点的主页。浏览器会把缺失的“/”附加到 URL 的后面传给服务器，服务器再把“/”扩展成某个默认的文件名。

11.5.3 HTTP 事务

因为 HTTP 基于在因特网连接上传送的文本行，我们可以使用 Linux 的 telnet 程序与因特网上的任何 Web 服务器进行事务。telnet 程序作为远程登录工具，已经被 ssh 大大取代，但它对于调试那些使用文本行经连接与客户端对话的服务器来说非常方便。例如，图 11.24 使用 telnet 从 AOL Web 服务器请求主页。

```
1  linux> telnet www.aol.com 80                客户端:打开到服务器的连接
2  Trying 205.188.146.23...                    telnet 向终端打印 3 行
3  Connected to aol.com.
4  Escape character is '^]'.
5  GET / HTTP/1.1                              客户端:请求行
6  Host: www.aol.com                           客户端:HTTP/1.1
 ↪  要求的头部
7                                              客户端:空行终止头部
8  HTTP/1.0 200 OK                             服务器:响应行
9  MIME-Version: 1.0                           服务器:后面是五个响应头部
10 Date: Mon, 8 Jan 2010 4:59:42 GMT
11 Server: Apache-Coyote/1.1
12 Content-Type: text/html                     服务器:响应主体应为 HTML
13 Content-Length: 42092                       服务器:响应主体有 42092
 ↪  字节
14                                             服务器:空行终止响应头部
15 <html>                                      服务器:响应主体中的第一行
 ↪  HTML
16 ...                                         服务器:还有 766 行 HTML
 ↪  未列出
```

```
17 </html>                                          服务器:
 ↪  响应主体中的最后一行 HTML
18 Connection closed by foreign host.               服务器:关闭连接
19 linux>                                           客户端:关闭连接并终止
```

图 11.24　服务静态内容的一次示例 HTTP 事务。

在第 1 行，我们从 Linux shell 运行 telnet，要求它打开一个到 AOL Web 服务器的连接。telnet 向终端打印三行输出，打开连接，然后等待我们输入文本（第 5 行）。每当输入一行文本并按下回车键，telnet 就读入这一行，附加回车和换行字符（用 C 记号表示为 `\r\n`），再把这一行发送给服务器。这与 HTTP 标准一致——HTTP 要求每个文本行都以一个回车-换行对终止。为了发起这次事务，我们输入一个 HTTP 请求（第 5 ~ 7 行）。服务器以一个 HTTP 响应（第 8 ~ 17 行）应答，然后关闭连接（第 18 行）。

HTTP 请求

一个 HTTP 请求由一个请求行（第 5 行）组成，后面跟着零个或多个请求头部（第 6 行），最后是一个终止头部列表的空文本行（第 7 行）。请求行的形式为

```
method URI version
```

HTTP 支持许多不同的方法，包括 GET、POST、OPTIONS、HEAD、PUT、DELETE 和 TRACE。我们只介绍主力方法 GET，绝大多数的 HTTP 请求用的都是它。GET 方法指示服务器生成并返回由 URI（uniform resource identifier，统一资源标识符）标识的内容。URI 是对应 URL 中包含文件名和可选参数的后缀。[3]

请求行中的版本字段指出该请求遵循的 HTTP 版本。最新的 HTTP 版本是 HTTP/1.1 [37]。HTTP/1.0 是 1996 年的一个更早的、简单得多的版本 [6]。HTTP/1.1 定义了额外的头部，为缓存与安全等高级特性提供了支持，还定义了一种机制，允许客户端和服务器在同一条持久连接上执行多次事务。在实践中，两个版本是兼容的，因为 HTTP/1.0 客户端和服务器会直接忽略它们不认识的 HTTP/1.1 头部。

总结来说，第 5 行的请求行要求服务器抓取并返回 HTML 文件 /index.html。它还告诉服务器，请求的其余部分将按 HTTP/1.1 格式发送。

请求头部为服务器提供额外的信息，比如浏览器的品牌名称，或者浏览器所能理解的 MIME 类型。请求头部的形式为

[3] 实际上，只有在浏览器请求内容时才是这样。如果是代理服务器请求内容，那么 URI 必须是完整的 URL。

```
header-name: header-data
```

就我们的目的而言，唯一需要关注的头部是 Host 头部（第 6 行），它在 HTTP/1.1 请求中是必需的，在 HTTP/1.0 请求中不是。代理缓存使用 Host 头部，代理缓存有时充当浏览器与管理所请求文件的源服务器（origin server）之间的中间人。在一个所谓的代理链（proxy chain）中，从一个客户端到一个源服务器之间可以存在多个代理。Host 头部中的数据指明源服务器的域名，使代理链中间的代理可以判断它是否可能有一份所请求内容的本地缓存副本。

继续看图 11.24 中的例子，第 7 行的空文本行（通过敲回车键产生）终止了头部，并指示服务器发送所请求的 HTML 文件。

HTTP 响应

HTTP 响应与 HTTP 请求类似。一个 HTTP 响应由一个响应行（第 8 行）组成，后面跟着零个或多个响应头部（第 9 ~ 13 行），然后是一个终止头部的空行（第 14 行），最后是响应主体（第 15 ~ 17 行）。响应行的形式为

```
version status-code status-message
```

版本字段描述该响应遵循的 HTTP 版本。状态码（status-code）是一个三位正整数，指出请求的处理结果。状态消息（status-message）给出与该错误码对应的英文短语。图 11.25 列出了一些常见的状态码和它们对应的消息。

图 11.25 一些 HTTP 状态码

状态码	状态消息	描述
200	OK	请求被处理，没有错误。
301	Moved permanently	内容已移动到 Location 头部指明的主机名。
400	Bad request	服务器无法理解该请求。
403	Forbidden	服务器缺少访问所请求文件的权限。
404	Not found	服务器找不到所请求的文件。
501	Not implemented	服务器不支持该请求方法。
505	HTTP version not supported	服务器不支持请求中的版本。

第 9 ~ 13 行的响应头部给出关于响应的额外信息。就我们的目的而言，两个最重要的头部是 Content-Type（第 12 行），它告诉客户端响应主体中内容的 MIME 类型；以及 Content-Length（第 13 行），它给出主体的大小（以字节为单位）。

终止响应头部的、第 14 行的空文本行之后是响应主体，其中包含所请求的内容。

边栏 在 HTTP POST 请求中传递参数
HTTP POST 请求的参数是在请求主体中传递的，而不是在 URI 中传递。

11.5.4 服务动态内容

如果我们停下来想一想一个服务器会如何向客户端提供动态内容，就会浮现出一些问题。例如，客户端如何把程序参数传给服务器？服务器如何把这些参数传给它创建的子进程？服务器如何把子进程生成内容时可能需要的其他信息传给子进程？子进程把输出送到哪里？这些问题由一个事实上的标准来处理，这个标准称为 CGI（common gateway interface，公共网关接口）。

客户端如何把程序参数传给服务器？

GET 请求的参数在 URI 中传递。正如我们所见，字符“?”把文件名与参数分开，每个参数用字符“&”分开。参数中不允许有空格，必须用字符串`%20`表示。其他特殊字符也存在类似的编码。

服务器如何把参数传给子进程？

当服务器收到像下面这样的请求时

```
GET /cgi-bin/adder?15000&213 HTTP/1.1
```

它调用 fork 创建一个子进程，并调用 execve 在子进程的上下文中运行 /cgi-bin/adder 程序。像 adder 这样遵守 CGI 标准规则的程序经常被称为 CGI 程序。在调用 execve 之前，子进程把 CGI 环境变量 QUERY_STRING 设置为 15000&213，adder 程序在运行时可以用 Linux 的 getenv 函数来引用它。

服务器如何把其他信息传给子进程？

CGI 定义了许多其他的环境变量，CGI 程序在运行时可以期待它们被设置。图 11.26 展示了一个子集。

图 11.26 CGI 环境变量示例

环境变量	描述
QUERY_STRING	程序参数
SERVER_PORT	父进程监听的端口
REQUEST_METHOD	GET 或 POST
REMOTE_HOST	客户端的域名
REMOTE_ADDR	客户端的点分十进制 IP 地址
CONTENT_TYPE	仅对 POST：请求主体的 MIME 类型
CONTENT_LENGTH	仅对 POST：请求主体的大小（字节）

子进程把它的输出送到哪里？

CGI 程序把它的动态内容发送到标准输出。在子进程加载并运行 CGI 程序之前，它使用 Linux 的 dup2 函数把标准输出重定向到与客户端相关联的已连接描述符。因此，CGI 程序写到标准输出的任何内容都直接送给客户端。

注意，因为父进程并不知道子进程生成的内容的类型或大小，所以子进程要负责生成 Content-type 与 Content-length 响应头部，以及终止这些头部的空行。

图 11.27 展示了一个简单的 CGI 程序，它对它的两个参数求和，并把一个包含结果的 HTML 文件返回给客户端。图 11.28 展示了由 adder 程序服务动态内容的一次 HTTP 事务。

练习 11.5 假设一个 CGI 程序需要向客户端发送动态内容。通常的做法是让 CGI 程序把它的内容发送到标准输出。请解释这些内容是如何被送到客户端的。

> **边栏 在 HTTP POST 请求中把参数传给 CGI 程序**
> 对 POST 请求，子进程还需要把标准输入重定向到已连接描述符。这样，CGI 程序就从标准输入读取请求主体中的参数。

```
1  #include "csapp.h"
2
3  int main(void) {
4      char *buf, *p;
5      char arg1[MAXLINE], arg2[MAXLINE], content[MAXLINE];
6      int n1=0, n2=0;
7
8      /* 提取两个参数 */
9      if ((buf = getenv("QUERY_STRING")) != NULL) {
10         p = strchr(buf, '&');
11         *p = '\0';
12         strcpy(arg1, buf);
13         strcpy(arg2, p+1);
14         n1 = atoi(arg1);
15         n2 = atoi(arg2);
16     }
17
18     /* 构造响应主体 */
19     sprintf(content, "QUERY_STRING=%s", buf);
20     sprintf(content, "Welcome to add.com: ");
21     sprintf(content, "%sTHE Internet addition portal.\r\n<p>",
       content);
22     sprintf(content, "%sThe answer is: %d + %d = %d\r\n<p>",
23             content, n1, n2, n1 + n2);
24     sprintf(content, "%sThanks for visiting!\r\n", content);
25
26     /* 生成 HTTP 响应 */
27     printf("Connection: close\r\n");
```

```
28     printf("Content-length: %d\r\n", (int)strlen(content));
29     printf("Content-type: text/html\r\n\r\n");
30     printf("%s", content);
31     fflush(stdout);
32
33     exit(0);
34 }
```

图 11.27 对两个整数求和的 CGI 程序。

```
1  linux> telnet kittyhawk.cmcl.cs.cmu.edu 8000   客户端:打开连接
2  Trying 128.2.194.242...
3  Connected to kittyhawk.cmcl.cs.cmu.edu.
4  Escape character is '^]'.
5  GET /cgi-bin/adder?15000&213 HTTP/1.0          客户端:请求行
6                                                 客户端:空行终止头部
7  HTTP/1.0 200 OK                                服务器:响应行
8  Server: Tiny Web Server                        服务器:标识服务器
9  Content-length: 115                            adder:响应主体应有 115
 ↪  字节
10 Content-type: text/html                        adder:响应主体应为 HTML
11                                                adder:空行终止头部
12 Welcome to add.com: THE Internet addition portal. adder:第一行 HTML
13 <p>The answer is: 15000 + 213 = 15213          adder:
 ↪  响应主体中的第二行 HTML
14 <p>Thanks for visiting!                        adder:响应主体中的第三行
 ↪  HTML
15 Connection closed by foreign host.             服务器:关闭连接
16 linux>                                         客户端:关闭连接并终止
```

图 11.28 服务动态 HTML 内容的一次 HTTP 事务。

11.6 综合:Tiny Web 服务器

我们将通过开发一个小但可运行的 Web 服务器 Tiny 来结束对网络编程的讨论。Tiny 是一个有意思的程序。它只用 250 行代码就把我们已经学到的许多思想——比如进程控制、Unix I/O、套接字接口以及 HTTP——结合在一起。虽然它缺少真正的服务器所具备的功能、健壮性和安全性,但它的能力足以向真正的 Web 浏览器提供静态和动态内容。我们鼓励你去研究它并亲自实现它。把一个真正的浏览器指向你自己的服务器,看着它显示出一个包含文本和图形的复杂 Web 页面,这是很让人激动的(甚至对作者本人也是如此!)。

Tiny 的 main 例程

图 11.29 展示了 Tiny 的 main 例程。Tiny 是一个迭代式服务器,它在命令行传入的端口上监听连接请求。在调用 open_listenfd 函数打开一个监听套接字之后,Tiny 就执行典型的无限服务器循环,反复地接受一个连接请求(第 32 行),执行一次事务(第 36 行),并关闭它这一端的连接(第 37 行)。

```
1  /*
2   * tiny.c - 一个简单的、迭代式的 HTTP/1.0 Web 服务器,它使用
3   *     GET 方法来服务静态和动态内容。
4   */
5  #include "csapp.h"
6
7  void doit(int fd);
8  void read_requesthdrs(rio_t *rp);
9  int parse_uri(char *uri, char *filename, char *cgiargs);
10 void serve_static(int fd, char *filename, int filesize);
11 void get_filetype(char *filename, char *filetype);
12 void serve_dynamic(int fd, char *filename, char *cgiargs);
13 void clienterror(int fd, char *cause, char *errnum,
14                  char *shortmsg, char *longmsg);
15
16 int main(int argc, char **argv)
17 {
18     int listenfd, connfd;
19     char hostname[MAXLINE], port[MAXLINE];
20     socklen_t clientlen;
21     struct sockaddr_storage clientaddr;
22
23     /* 检查命令行参数 */
24     if (argc != 2) {
25         fprintf(stderr, "usage: %s <port>\n", argv[0]);
26         exit(1);
27     }
28
29     listenfd = Open_listenfd(argv[1]);
30     while (1) {
31         clientlen = sizeof(clientaddr);
32         connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
33         Getnameinfo((SA *) &clientaddr, clientlen, hostname,
 ↪  MAXLINE,
34                     port, MAXLINE, 0);
35         printf("Accepted connection from (%s, %s)\n", hostname,
        port);
36         doit(connfd);
37         Close(connfd);
38     }
39 }
```

图 11.29 Tiny Web 服务器。

doit 函数

图 11.30 中的 doit 函数处理一次 HTTP 事务。首先,我们读入并解析请求行(第 11 ~ 14 行)。注意,我们使用的是图 10.8 中的 rio_readlineb 函数来读入请求行。

```
1  void doit(int fd)
2  {
3      int is_static;
```

```
4       struct stat sbuf;
5       char buf[MAXLINE], method[MAXLINE], uri[MAXLINE],
         version[MAXLINE];
6       char filename[MAXLINE], cgiargs[MAXLINE];
7       rio_t rio;
8
9       /* 读入请求行和请求头部 */
10      Rio_readinitb(&rio, fd);
11      Rio_readlineb(&rio, buf, MAXLINE);
12      printf("Request headers:\n");
13      printf("%s", buf);
14      sscanf(buf, "%s %s %s", method, uri, version);
15      if (strcasecmp(method, "GET")) {
16          clienterror(fd, method, "501", "Not implemented",
17                      "Tiny does not implement this method");
18          return;
19      }
20      read_requesthdrs(&rio);
21
22      /* 解析 GET 请求中的 URI */
23      is_static = parse_uri(uri, filename, cgiargs);
24      if (stat(filename, &sbuf) < 0) {
25          clienterror(fd, filename, "404", "Not found",
26                      "Tiny couldn't find this file");
27          return;
28      }
29
30      if (is_static) { /* 服务静态内容 */
31          if (!(S_ISREG(sbuf.st_mode)) || !(S_IRUSR & sbuf.st_mode)) {
32              clienterror(fd, filename, "403", "Forbidden",
33                          "Tiny couldn't read the file");
34              return;
35          }
36          serve_static(fd, filename, sbuf.st_size);
37      }
38      else { /* 服务动态内容 */
39          if (!(S_ISREG(sbuf.st_mode)) || !(S_IXUSR & sbuf.st_mode)) {
40              clienterror(fd, filename, "403", "Forbidden",
41                          "Tiny couldn't run the CGI program");
42              return;
43          }
44          serve_dynamic(fd, filename, cgiargs);
45      }
46  }
```

图 11.30　Tiny 的 doit 处理一次 HTTP 事务。

Tiny 只支持 GET 方法。如果客户端请求其他方法(如 POST),我们就向它发送一个错误消息,并返回到 main 例程(第 15 ~ 19 行),随后 main 例程关闭连接并等待下一个连接请求。否则,我们读入并(我们将看到)忽略所有请求头部(第 20 行)。

接下来，我们把 URI 解析成一个文件名和一个可能为空的 CGI 参数字符串，并设置一个标志，指示请求的是静态内容还是动态内容（第 23 行）。如果磁盘上不存在该文件，我们就立即向客户端发送一个错误消息并返回。

最后，如果请求的是静态内容，我们就验证该文件是一个普通文件，并且我们有读取它的权限（第 31 行）。如果验证通过，我们就向客户端提供静态内容（第 36 行）。类似地，如果请求的是动态内容，我们就验证该文件是可执行的（第 39 行），如果验证通过，我们就继续把动态内容提供给客户端（第 44 行）。

clienterror 函数

Tiny 缺少真正的服务器所具备的许多错误处理特性。不过，它确实检查了一些明显的错误并报告给客户端。图 11.31 中的 clienterror 函数向客户端发送一个 HTTP 响应，该响应在响应行中带有相应的状态码和状态消息，并在响应主体中带有一个 HTML 文件，用来向浏览器的用户解释这个错误。

```
1  void clienterror(int fd, char *cause, char *errnum,
2                   char *shortmsg, char *longmsg)
3  {
4      char buf[MAXLINE], body[MAXBUF];
5
6      /* 构造 HTTP 响应主体 */
7      sprintf(body, "<html><title>Tiny Error</title>");
8      sprintf(body, "%s<body bgcolor=""ffffff"">\r\n", body);
9      sprintf(body, "%s%s: %s\r\n", body, errnum, shortmsg);
10     sprintf(body, "%s<p>%s: %s\r\n", body, longmsg, cause);
11     sprintf(body, "%s<hr><em>The Tiny Web server</em>\r\n", body);
12
13     /* 打印 HTTP 响应 */
14     sprintf(buf, "HTTP/1.0 %s %s\r\n", errnum, shortmsg);
15     Rio_writen(fd, buf, strlen(buf));
16     sprintf(buf, "Content-type: text/html\r\n");
17     Rio_writen(fd, buf, strlen(buf));
18     sprintf(buf, "Content-length: %d\r\n\r\n", (int)strlen(body));
19     Rio_writen(fd, buf, strlen(buf));
20     Rio_writen(fd, body, strlen(body));
21 }
```

图 11.31　Tiny 的 clienterror 向客户端发送一个错误消息。

回忆一下，一个 HTML 响应应当指出主体中内容的大小和类型。因此，我们选择把 HTML 内容构造为一个单一字符串，这样我们就能方便地确定它的大小。另外，注意对于所有输出我们都使用图 10.4 中的健壮的 rio_writen 函数。

read_requesthdrs 函数

Tiny 不使用请求头部中的任何信息。它只是调用图 11.32 中的 read_requesthdrs 函数，把请求头部读入并忽略掉。注意，终止请求头部的那个空文本行由一个回车-换行对组成，我们在第 6 行对它进行检查。

```
1  void read_requesthdrs(rio_t *rp)
2  {
3      char buf[MAXLINE];
4
5      Rio_readlineb(rp, buf, MAXLINE);
6      while(strcmp(buf, "\r\n")) {
7          Rio_readlineb(rp, buf, MAXLINE);
8          printf("%s", buf);
9      }
10     return;
11 }
```

图 11.32　Tiny 的 read_requesthdrs 读入并忽略请求头部。

parse_uri 函数

Tiny 假定静态内容的主目录就是它的当前目录，而可执行文件的主目录是./cgi-bin。任何包含字符串 cgi-bin 的 URI 都被认为表示一个对动态内容的请求。默认文件名是./home.html。

图 11.33 中的 parse_uri 函数实现了这些策略。它把 URI 解析成一个文件名和一个可选的 CGI 参数字符串。如果请求的是静态内容（第 5 行），我们就清空 CGI 参数字符串（第 6 行），然后把 URI 转换成一个相对 Linux 路径名，比如./index.html（第 7 ~ 8 行）。如果 URI 以“/”字符结尾（第 9 行），我们就把默认文件名附加到路径末尾（第 10 行）。另一方面，如果请求的是动态内容（第 13 行），我们就提取出所有的 CGI 参数（第 14 ~ 20 行），并把 URI 的剩余部分转换成一个相对 Linux 文件名（第 21 ~ 22 行）。

```
1  int parse_uri(char *uri, char *filename, char *cgiargs)
2  {
3      char *ptr;
4
5      if (!strstr(uri, "cgi-bin")) { /* 静态内容 */
6          strcpy(cgiargs, "");
7          strcpy(filename, ".");
8          strcat(filename, uri);
9          if (uri[strlen(uri)-1] == '/')
10             strcat(filename, "home.html");
11         return 1;
12     }
13     else { /* 动态内容 */
14         ptr = index(uri, '?');
15         if (ptr) {
16             strcpy(cgiargs, ptr+1);
17             *ptr = '\0';
18         }
19         else
20             strcpy(cgiargs, "");
21         strcpy(filename, ".");
22         strcat(filename, uri);
```

```
23         return 0;
24     }
25 }
```

图 11.33 Tiny 的 parse_uri 解析一个 HTTP URI。

serve_static 函数

Tiny 服务五种常见的静态内容类型：HTML 文件、无格式文本文件，以及用 GIF、PNG 和 JPEG 格式编码的图像。

图 11.34 中的 serve_static 函数发送一个 HTTP 响应，该响应的主体包含一个本地文件的内容。首先，我们通过检查文件名中的后缀来确定文件类型（第 7 行），然后把响应行和响应头部发送给客户端（第 8 ~ 13 行）。注意，一个空行终止了响应头部。

接下来，我们通过把请求文件的内容复制到已连接描述符 fd 来发送响应主体。这段代码有些微妙，需要仔细研究。第 18 行打开 filename 以进行读取并获得它的描述符。第 19 行中，Linux 的 mmap 函数把被请求的文件映射到一个虚拟内存区域。回忆一下 9.8 节中关于 mmap 的讨论，mmap 调用把文件 srcfd 的前 filesize 个字节映射到一个从地址 srcp 开始的私有的只读虚拟内存区域。

```
1  void serve_static(int fd, char *filename, int filesize)
2  {
3      int srcfd;
4      char *srcp, filetype[MAXLINE], buf[MAXBUF];
5
6      /* 把响应头部发送给客户端 */
7      get_filetype(filename, filetype);
8      sprintf(buf, "HTTP/1.0 200 OK\r\n");
9      sprintf(buf, "%sServer: Tiny Web Server\r\n", buf);
10     sprintf(buf, "%sConnection: close\r\n", buf);
11     sprintf(buf, "%sContent-length: %d\r\n", buf, filesize);
12     sprintf(buf, "%sContent-type: %s\r\n\r\n", buf, filetype);
13     Rio_writen(fd, buf, strlen(buf));
14     printf("Response headers:\n");
15     printf("%s", buf);
16
17     /* 把响应主体发送给客户端 */
18     srcfd = Open(filename, O_RDONLY, 0);
19     srcp = Mmap(0, filesize, PROT_READ, MAP_PRIVATE, srcfd, 0);
20     Close(srcfd);
21     Rio_writen(fd, srcp, filesize);
22     Munmap(srcp, filesize);
23 }
24
25  /*
26   * get_filetype - 从文件名推导出文件类型
27   */
28 void get_filetype(char *filename, char *filetype)
29 {
```

```
30     if (strstr(filename, ".html"))
31         strcpy(filetype, "text/html");
32     else if (strstr(filename, ".gif"))
33         strcpy(filetype, "image/gif");
34     else if (strstr(filename, ".png"))
35         strcpy(filetype, "image/png");
36     else if (strstr(filename, ".jpg"))
37         strcpy(filetype, "image/jpeg");
38     else
39         strcpy(filetype, "text/plain");
40 }
```

图 11.34　Tiny 的 serve_static 向客户端提供静态内容。

一旦我们把文件映射到了内存中，就不再需要它的描述符，所以我们关闭这个文件（第 20 行）。不这样做会引入一个潜在致命的内存泄漏。第 21 行执行把文件实际传送给客户端的工作。rio_writen 函数把从位置 srcp 开始（它当然被映射到被请求的文件上）的 filesize 个字节复制到客户端的已连接描述符。最后，第 22 行释放被映射的虚拟内存区域。这一步很重要，它能避免一个潜在致命的内存泄漏。

serve_dynamic 函数

Tiny 通过 fork 一个子进程、然后在子进程的上下文中运行一个 CGI 程序来服务任何类型的动态内容。

图 11.35 中的 serve_dynamic 函数首先向客户端发送一个表示成功的响应行，以及一个提供信息的 Server 头部。由 CGI 程序负责发送响应的其余部分。注意，这里没有我们希望的那么健壮，因为它没有顾及 CGI 程序可能遇到错误的可能性。

在发送完响应的第一部分之后，我们 fork 一个新的子进程（第 11 行）。子进程用在请求 URI 中的 CGI 参数初始化 QUERY_STRING 环境变量（第 13 行）。注意，一个真正的服务器在这里还会设置其余的 CGI 环境变量。为简洁起见，我们省略了这一步。

```
1  void serve_dynamic(int fd, char *filename, char *cgiargs)
2  {
3      char buf[MAXLINE], *emptylist[] = { NULL };
4
5      /* 返回 HTTP 响应的第一部分 */
6      sprintf(buf, "HTTP/1.0 200 OK\r\n");
7      Rio_writen(fd, buf, strlen(buf));
8      sprintf(buf, "Server: Tiny Web Server\r\n");
9      Rio_writen(fd, buf, strlen(buf));
10
11     if (Fork() == 0) { /* 子进程 */
12         /* 真正的服务器会在这里设置所有的 CGI 变量 */
13         setenv("QUERY_STRING", cgiargs, 1);
```

```
14         Dup2(fd, STDOUT_FILENO);          /* 把 stdout 重定向到客户端
↪  */
15         Execve(filename, emptylist, environ); /* 运行 CGI 程序 */
16     }
17     Wait(NULL); /* 父进程等待并回收子进程 */
18 }
```

图 11.35　Tiny 的 serve_dynamic 向客户端提供动态内容。

接下来，子进程把子进程的标准输出重定向到已连接的文件描述符（第 14 行），然后加载并运行 CGI 程序（第 15 行）。因为 CGI 程序是在子进程的上下文中运行的，所以它能够访问调用 execve 函数之前就存在的打开文件和环境变量。因此，CGI 程序写到标准输出的所有内容都直接送往客户端进程，不需要父进程的任何干预。与此同时，父进程阻塞在 wait 调用中，等待子进程终止并回收它（第 17 行）。

边栏　处理过早关闭的连接

虽然 Web 服务器的各项基本功能相当简单，但我们并不想给你留下一种错误的印象，认为编写一个真正的 Web 服务器很容易。构建一个能长时间运行而不崩溃的健壮的 Web 服务器是一项困难的任务，它要求比我们这里学到的对 Linux 系统编程更深入的理解。例如，如果一个服务器把数据写给一个已经由客户端关闭的连接（比如，因为你单击了浏览器上的“停止”按钮），那么第一次这样的写会正常返回，但第二次写会导致传送一个 SIGPIPE 信号，它的默认行为是终止进程。如果 SIGPIPE 信号被捕获或被忽略，那么第二次写操作将返回 -1，并把 errno 设置为 EPIPE。strerr 函数和 perror 函数把 EPIPE 错误报告为“管道断开”（Broken pipe），这是一个不直观的消息，让一代又一代的学生感到困惑。底线是：健壮的服务器必须捕获这些 SIGPIPE 信号，并检查 write 函数调用是否出现 EPIPE 错误。

11.7 小结

每个网络应用都基于客户端-服务器模型。按照这个模型，一个应用由一个服务器和一个或多个客户端组成。服务器管理资源，并以某种方式操作资源，从而为它的客户端提供服务。客户端-服务器模型中的基本操作是一次客户端-服务器事务，它由客户端发出的一次请求和随后服务器发出的一次响应组成。

客户端和服务器在一个称为因特网的全球网络上通信。从程序员的角度，我们可以把因特网看成是一个具有以下属性的世界范围的主机集合：(1) 每台因特网主机都有一个唯一的 32 位名字，称为 IP 地址。(2) IP 地址的集合被映射到一组因特网域名。(3) 不同因特网主机上的进程可以通过连接相互通信。

客户端和服务器通过使用套接字接口来建立连接。套接字是连接的一个端点，以文件描述符的形式呈现给应用程序。套接字接口提供了打开和关闭套接字描述符的函数。客户端和服务器通过读写这些描述符来相互通信。

Web 服务器和它们的客户端（例如浏览器）用 HTTP 协议相互通信。浏览器向服务器请求静态内容或动态内容。对静态内容的请求是这样服务的：从服务器的磁盘抓取一个文件并把它返回给客户端。对动态内容的请求是这样服务的：在服务器上一个子进程的上下文中运行一个程序，并把它的输出返回给客户端。CGI 标准提供了一组规则，管理客户端如何把程序参数传给服务器，服务器如何把这些参数和其他信息传给子进程，以及子进程如何把输出送回客户端。一个简单而功能完备的、同时服务静态和动态内容的 Web 服务器可以用几百行 C 代码来实现。

11.8 参考文献说明

因特网的官方信息来源是一系列免费提供的、带编号的文档，称为 RFC（request for comments，请求评注）。RFC 的可检索索引在 Web 上可以访问到：

```
http://rfc-editor.org
```

RFC 通常是为因特网基础设施的开发者撰写的，因此对一般的读者来说常常过于详细。不过，就权威的信息而言，没有比这更好的来源了。HTTP/1.1 协议在 RFC 2616 中有正式记载。MIME 类型的权威列表维护在

```
http://www.iana.org/assignments/media-types
```

Kerrisk 的著作是 Linux 编程各个方面的“圣经”。它对现代网络编程提供了详细的讨论 [62]。还有一些很好的计算机网络通论教材 [65, 84, 114]。伟大的技术作家 W. Richard Stevens 编写了一系列经典著作，主题包括高级 Unix 编程 [111]、因特网协议 [109, 120, 107] 以及 Unix 网络编程 [108, 110]。认真学习 Unix 系统编程的学生会想读遍所有这些书。令人痛心的是，Stevens 于 1999 年 9 月 1 日去世。人们非常怀念他的贡献。

11.9 家庭作业

家庭作业 11.6 ▲▲

A. 修改 Tiny，让它回显每一条请求行和每一条请求头部。

B. 用你最喜欢的浏览器向 Tiny 发起一次对静态内容的请求。把 Tiny 的输出捕获到一个文件中。

C. 检查 Tiny 的输出，确定你的浏览器使用的 HTTP 版本。

D. 查阅 RFC 2616 中的 HTTP/1.1 标准，确定你的浏览器发出的 HTTP 请求中每个头部的含义。你可以从 www.rfc-editor.org/rfc.html 获得 RFC 2616。

家庭作业 11.7 ▲▲ 扩展 Tiny，使它能服务 MPG 视频文件。用真实浏览器检验你的工作。

家庭作业 11.8 ▲▲ 修改 Tiny，使它在一个 SIGCHLD 处理程序内回收 CGI 子进程，而不是显式地等待它们终止。

家庭作业 11.9 ▲▲ 修改 Tiny，使它在服务静态内容时，用 malloc、rio_readn 和 rio_writen 把被请求的文件复制到已连接描述符，而不是用 mmap 和 rio_writen。

家庭作业 11.10 ▲▲

A. 为图 11.27 中的 CGI adder 函数编写一个 HTML 表单。你的表单应当包括两个文本框，用户在其中填入要相加的两个数。你的表单应当使用 GET 方法来请求内容。

B. 用真实浏览器检验你的工作：向 Tiny 请求这个表单，把填写好的表单提交给 Tiny，然后显示由 adder 生成的动态内容。

家庭作业 11.11 ▲▲ 扩展 Tiny 以支持 HTTP HEAD 方法。用 telnet 作为 Web 客户端检验你的工作。

家庭作业 11.12 ▲▲▲ 扩展 Tiny，使它能服务由 HTTP POST 方法请求的动态内容。用你最喜欢的浏览器检验你的工作。

家庭作业 11.13 ▲▲▲ 修改 Tiny，使它能妥善处理（而不终止）当 write 函数试图向一个过早关闭的连接写数据时产生的 SIGPIPE 信号和 EPIPE 错误。

11.10 练习答案

答案 11.1

点分十进制地址	十六进制地址
107.212.122.205	0x6BD47ACD
64.12.149.13	0x400C950D
107.212.96.29	0x6BD4601D
[0.0].[0.128]	0x00000080
[255.255].[255.0]	0xFFFFFF00
[10.1].[1.64]	0x0A010140

答案 11.2

```
1  #include "csapp.h"
2
```

```
3  int main(int argc, char **argv)
4  {
5      struct in_addr inaddr; /* 以网络字节顺序存放的地址 */
6      uint16_t addr;         /* 以主机字节顺序存放的地址 */
7      char buf[MAXBUF];      /* 点分十进制字符串缓冲区 */
8
9      if (argc != 2) {
10         fprintf(stderr, "usage: %s <hex number>\n", argv[0]);
11         exit(0);
12     }
13     sscanf(argv[1], "%x", &addr);
14     inaddr.s_addr = htons(addr);
15
16     if (!inet_ntop(AF_INET, &inaddr, buf, MAXBUF))
17         unix_error("inet_ntop");
18     printf("%s\n", buf);
19
20     exit(0);
21 }
```

答案 11.3

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
5      struct in_addr inaddr; /* 以网络字节顺序存放的地址 */
6      int rc;
7
8      if (argc != 2) {
9          fprintf(stderr, "usage: %s <network byte order>\n",
 ↪  argv[0]);
10         exit(0);
11     }
12
13     rc = inet_pton(AF_INET, argv[1], &inaddr);
14     if (rc == 0)
15         app_error("inet_pton error: invalid network byte order");
16     else if (rc < 0)
17         unix_error("inet_pton error");
18
19     printf("0x%x\n", ntohs(inaddr.s_addr));
20     exit(0);
21 }
```

答案 11.4 下面是一种解法。请注意使用 inet_ntop 要难得多,它要求笨拙的强制类型转换和深层的结构引用。getnameinfo 函数要简单得多,因为它替我们做完了所有这些工作。

```
1  #include "csapp.h"
2
3  int main(int argc, char **argv)
4  {
```

```
5       struct addrinfo *p, *listp, hints;
6       struct sockaddr_in *sockp;
7       char buf[MAXLINE];
8       int rc;
9
10      if (argc != 2) {
11          fprintf(stderr, "usage: %s <domain name>\n", argv[0]);
12          exit(0);
13      }
14
15      /* 获取 addrinfo 记录的列表 */
16      memset(&hints, 0, sizeof(struct addrinfo));
17      hints.ai_family = AF_INET;       /* 只用 IPv4 */
18      hints.ai_socktype = SOCK_STREAM; /* 只用于连接 */
19      if ((rc = getaddrinfo(argv[1], NULL, &hints, &listp)) != 0) {
20          fprintf(stderr, "getaddrinfo error: %s\n",
 ↪  gai_strerror(rc));
21          exit(1);
22      }
23
24      /* 遍历列表并显示每个相关联的 IP 地址 */
25      for (p = listp; p; p = p->ai_next) {
26          sockp = (struct sockaddr_in *)p->ai_addr;
27          Inet_ntop(AF_INET, &(sockp->sin_addr), buf, MAXLINE);
28          printf("%s\n", buf);
29      }
30
31      /* 清理 */
32      Freeaddrinfo(listp);
33
34      exit(0);
35  }
```

答案 11.5 在运行 CGI 程序的进程开始执行之前，使用一个 Linux 的 dup2 函数，把标准输出重定向到与客户端相关联的已连接描述符。因此，CGI 程序写到标准输出的任何内容都直接传送给客户端。

12

并发编程

正如我们在第 8 章中了解到的那样，如果逻辑控制流在时间上重叠，那么它们就是并发的。这种一般现象出现在计算机系统的许多不同层次上。硬件异常处理程序、进程和 Linux 信号处理程序都是我们熟悉的例子。

到目前为止，我们一直把并发主要当作操作系统内核用来运行多个应用程序的一种机制。但并发并不只限于内核，它在应用程序中同样扮演着重要角色。例如，我们已经看到 Linux 信号处理程序如何允许应用程序响应异步事件，比如用户键入 Ctrl+C，或者程序访问了虚拟内存中一个未定义的区域。应用级并发在其他方面也很有用：

- **访问慢速 I/O 设备**。当应用程序在等待数据从磁盘这样的慢速 I/O 设备到达时，内核通过运行其他进程让 CPU 保持忙碌。单个应用程序也可以用类似的方式利用并发，把有用的工作与 I/O 请求重叠起来。
- **与人交互**。与计算机交互的人要求具有同时执行多个任务的能力。例如，他们可能想在打印文档的同时调整一个窗口的大小。现代窗口系统利用并发来提供这种能力：用户每请求一次操作（比如点击鼠标），就会创建一个单独的并发逻辑流来执行该操作。
- **通过推迟操作来降低延迟**。有时，应用程序可以利用并发，把某些操作推迟到并发的执行路径中，以此降低其他操作的延迟。例如，一个动态存储分配器可以把合并操作推迟到一个以较低优先级运行的并发“合并”流中，从而降低单次 `free` 操作的延迟，这个流会在空闲 CPU 周期出现时趁机吸收并利用它们。
- **为多个网络客户端服务**。我们在第 11 章研究的迭代网络服务器是不现实的，因为它们一次只能为一个客户端服务。因此，单个慢速客户端就能拒绝向所有其他客户端提供服务。对于一个可能每秒要为成百上千个客户端提供服务的真实服务器来说，让一个慢的客户端损害对其他客户端的服务是不可接受的。更好的做法是构建一个并发服务器，为每个客户端创

建一个单独的逻辑流。这使服务器能够并发地为多个客户端服务，并防止慢客户端独占服务器。

- **在多核机器上并行计算**。许多现代系统都配有包含多个 CPU 的多核处理器。被划分为多个并发流的应用程序在多核机器上通常比在单处理器机器上运行得更快，因为这些流不是被交替执行的，而是并行执行的。

使用应用级并发的应用程序称为**并发程序**（**concurrent program**）。现代操作系统提供了三种构建并发程序的基本方法，分别是：

- **进程**。这种方法中，每个逻辑控制流都是一个进程，由内核来调度和维护。因为各个进程拥有各自独立的虚拟地址空间，所以想要相互通信的流必须使用某种显式的进程间通信（interprocess communication，IPC）机制。
- **I/O 多路复用**。这种形式的并发编程让应用程序在单个进程的语境中显式地调度自己的逻辑流。逻辑流被建模为状态机，主程序随着文件描述符上数据的到达，显式地使状态机从一个状态转换到另一个状态。因为程序是单个进程，所以所有的流共享同一个地址空间。
- **线程**。线程是运行在单个进程语境中的逻辑流，由内核进行调度。你可以把线程理解为另外两种方法的混合体：像进程流一样由内核调度，又像 I/O 多路复用流一样共享同一个虚拟地址空间。

本章将研究这三种不同的并发编程技术。为了让讨论始终具体，我们自始至终使用同一个示例应用——11.4.9 节中迭代 echo 服务器的并发版本。

12.1 基于进程的并发编程

构建并发程序最简单的方法是用进程，使用 `fork`、`exec` 和 `waitpid` 这样我们熟悉的函数。例如，构建并发服务器的自然做法是：在父进程中接受客户端的连接请求，然后创建一个新的子进程为每个新客户端服务。

为了理解这可能如何工作，假设我们有两个客户端，服务器正在一个监听描述符（比如描述符 3）上等待连接请求。再假设服务器接受了来自客户端 1 的一个连接请求，返回了一个已连接描述符（比如描述符 4），如图 12.1 所示。接受这个连接请求之后，服务器 `fork` 一个子进程，这个子进程得到服务器描述符表的一个完整副本。子进程关闭它的监听描述符 3 的副本，而父进程关闭它的已连接描述符 4 的副本，因为它们不再需要这些描述符了。这样我们就得到了图 12.2 所示的情形，此时子进程正忙于为客户端服务。

由于父进程和子进程中的已连接描述符都指向同一个文件表项，所以父进程关闭它自己的已连接描述符是至关重要的。否则，已连接描述符 4 的文件表

项将永远不会被释放，由此产生的内存泄漏会逐渐耗尽可用的内存，最终导致系统崩溃。

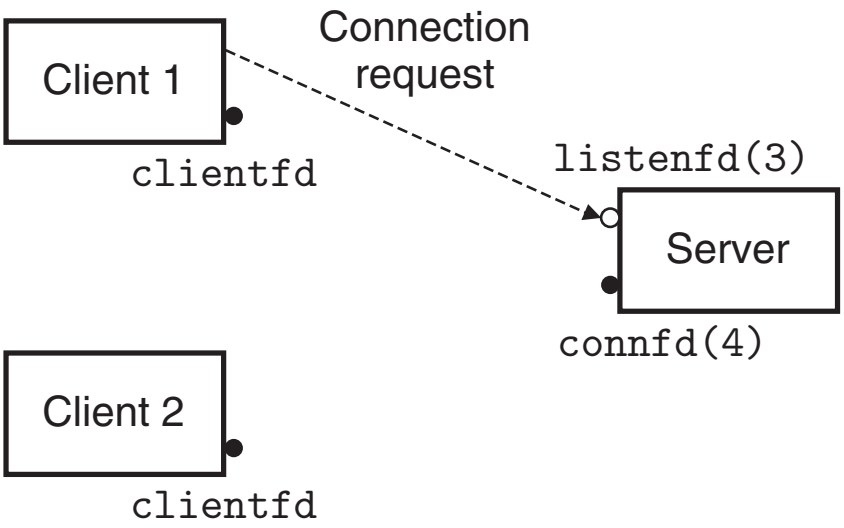


图 12.1 第 1 步：服务器接受来自客户端的连接请求。

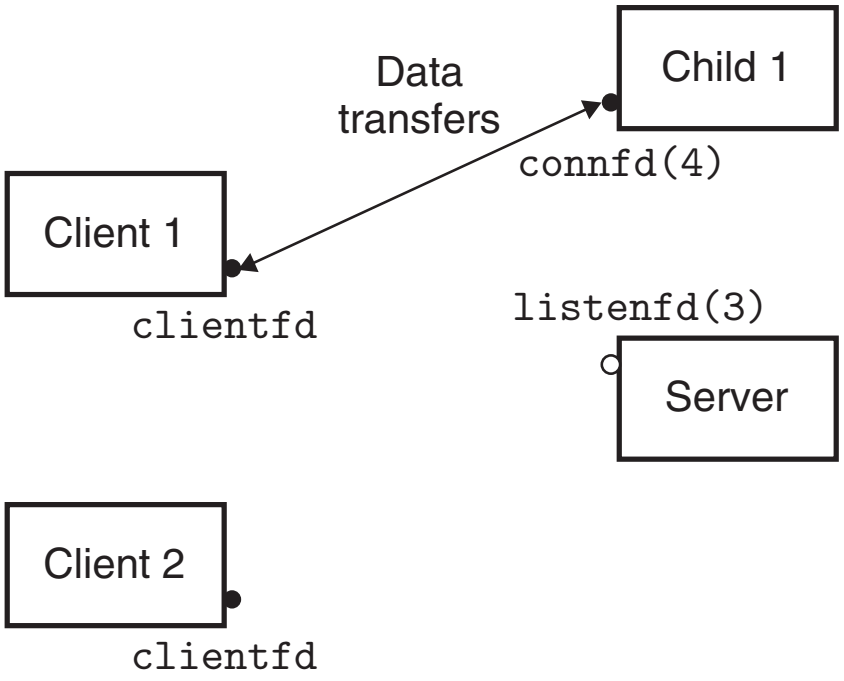


图 12.2 第 2 步：服务器派生一个子进程为客户端服务。

现在假设父进程在为客户端 1 创建了子进程之后，又接受了一个来自客户端 2 的新连接请求，返回了一个新的已连接描述符（比如描述符 5），如图 12.3

所示。接着,父进程又 fork 一个子进程,这个子进程开始使用已连接描述符 5 为它的客户端服务,如图 12.4 所示。此时,父进程正在等待下一个连接请求,而两个子进程正在并发地为它们各自的客户端服务。

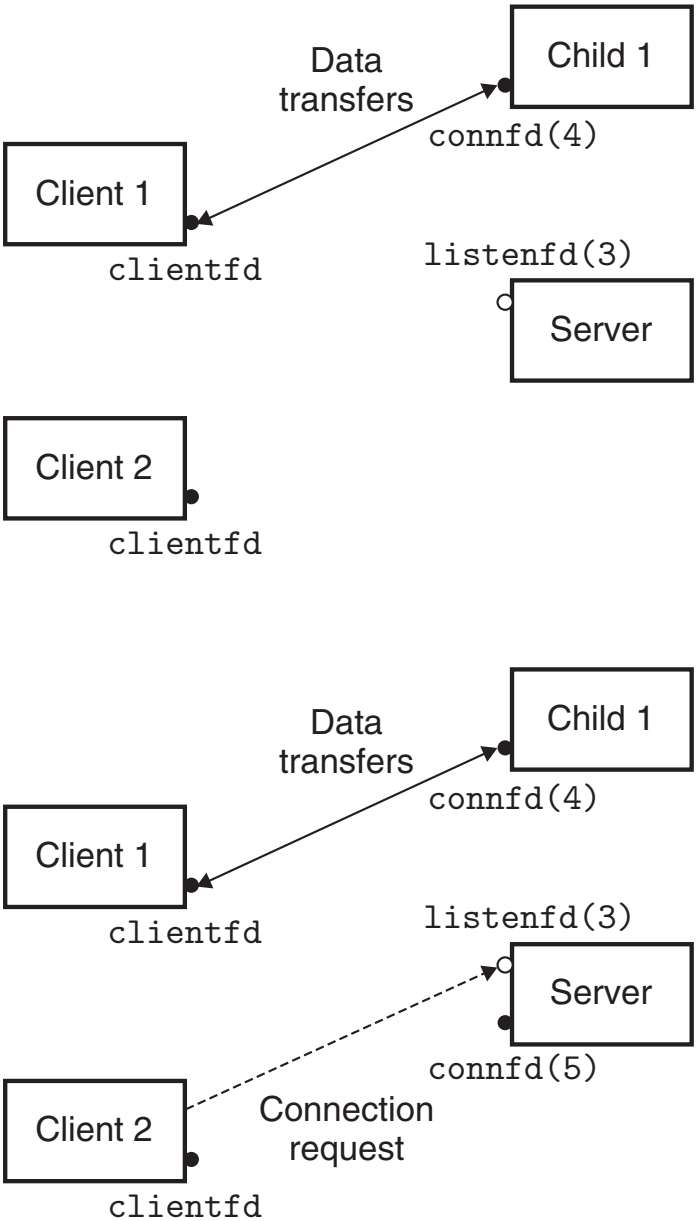


图 12.3 第 3 步:服务器接受另一个连接请求。

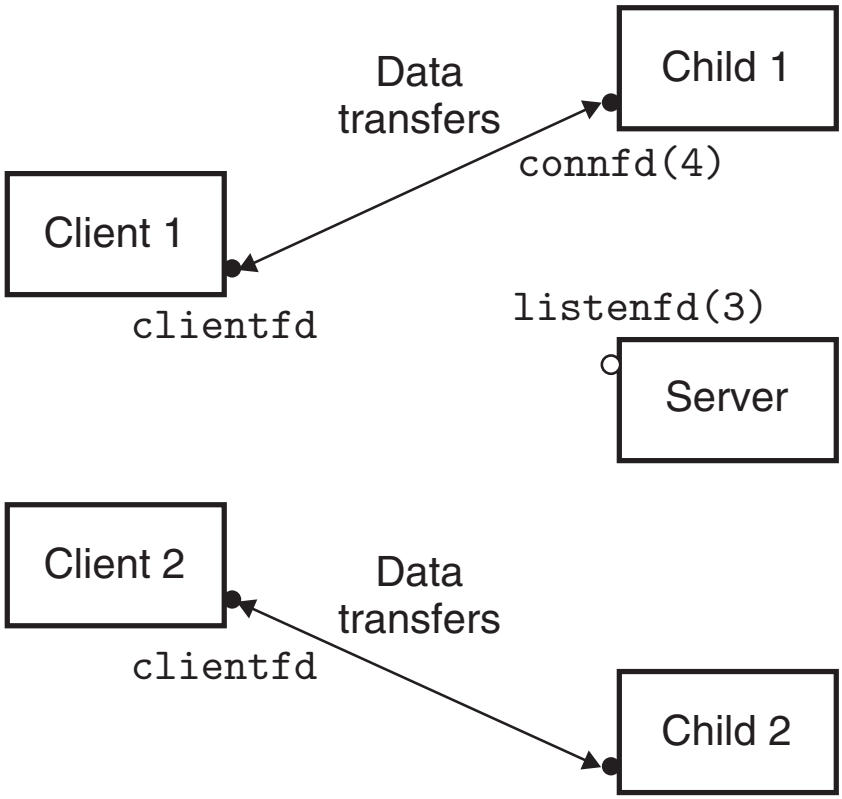


图 12.4 第 4 步:服务器再派生一个子进程为新客户端服务。

12.1.1 基于进程的并发服务器

图 12.5 给出了一个基于进程的并发 echo 服务器的代码。第 29 行调用的 echo 函数来自图 11.22。关于这个服务器,有几点值得注意:

- **第一**,服务器通常要运行很长的时间,所以我们必须包含一个 SIGCHLD 处理程序来回收僵死子进程 (第 4 ~ 9 行)。由于 SIGCHLD 处理程序在执行期间 SIGCHLD 信号是阻塞的,又由于 Linux 信号是不排队的,所以 SIGCHLD 处理程序必须准备好回收多个僵死子进程。
- **第二**,父进程和子进程必须关闭它们各自的 connfd 副本(分别为第 33 行和第 30 行)。正如我们提到过的,这对父进程尤其重要,它必须关闭自己的已连接描述符副本,以避免内存泄漏。
- **最后**,由于套接字的文件表项中有引用计数,在父进程和子进程的 connfd 副本都关闭之前,到客户端的连接不会被终止。

12.1.2 进程的优缺点

进程为父子进程之间共享状态信息提供了一个清晰的模型:文件表是共享的,而用户地址空间不共享。让进程拥有各自独立的地址空间,既是一个优点,也是一个缺点。一个进程不可能意外地覆写另一个进程的虚拟内存,这就避免了很多令人困惑的错误——这是一个明显的优点。

另一方面,分离的地址空间使进程更难共享状态信息。要共享信息,它们必须使用显式的 IPC 机制(参见“Unix IPC”边栏)。基于进程的设计的另一个缺点是它们往往比较慢,因为进程控制和 IPC 的开销都很高。

练习 12.1 图 12.5 演示了一个并发服务器,其中父进程创建一个子进程来处理每个新的连接请求。请追踪图 12.5 中相关联的文件表的引用计数器的值。

练习 12.2 假如我们删除图 12.5 中关闭已连接描述符的第 33 行,这段代码仍然是正确的——从这个意义上说,不会有内存泄漏。这是为什么?

code/conc/echoserverp.c

```
 1  #include "csapp.h"
 2  void echo(int connfd);
 3
 4  void sigchld_handler(int sig)
 5  {
 6      while (waitpid(-1, 0, WNOHANG) > 0)
 7          ;
 8      return;
 9  }
10
11  int main(int argc, char **argv)
12  {
13      int listenfd, connfd;
14      socklen_t clientlen;
15      struct sockaddr_storage clientaddr;
16
17      if (argc != 2) {
18          fprintf(stderr, "usage: %s <port>\n", argv[0]);
19          exit(0);
20      }
21
22      Signal(SIGCHLD, sigchld_handler);
23      listenfd = Open_listenfd(argv[1]);
24      while (1) {
25          clientlen = sizeof(struct sockaddr_storage);
26          connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
27          if (Fork() == 0) {
28              Close(listenfd); /* 子进程关闭它的监听套接字 */
29              echo(connfd);    /* 子进程为客户端服务 */
30              Close(connfd);   /* 子进程关闭与客户端的连接 */
31              exit(0);         /* 子进程退出 */
32          }
33          Close(connfd); /* 父进程关闭已连接套接字(重要!) */
```

```
34      }
35  }
```

图 12.5 基于进程的并发 echo 服务器。父进程 `fork` 一个子进程来处理每个新的连接请求。

> **边栏 Unix IPC**
> 你已经遇到过本书中的几个 IPC 的例子。第 8 章中的 `waitpid` 函数和信号都是原始的 IPC 机制，允许进程向运行在同一台主机上的进程发送短小的消息。第 11 章讲述的套接字接口是 IPC 的一种重要形式，允许不同主机上的进程交换任意字节流。不过，“Unix IPC” 这个术语通常是给一套大杂烩技术保留的名字，这些技术允许进程与运行在同一台主机上的其他进程通信，例子包括管道、FIFO（先进先出队列）、System V 共享内存和 System V 信号量。这些机制超出了本书的范围。Kerrisk 的书 [62] 是一本极好的参考书。

12.2 基于 I/O 多路复用的并发编程

假设你被要求编写一个 echo 服务器，它还能响应用户键入到标准输入上的交互式命令。在这种情形下，服务器必须响应两个相互独立的 I/O 事件：(1) 网络客户端发起连接请求；(2) 用户在键盘上键入一行命令。我们应该先等哪个事件呢？两种选择都不理想。如果我们正在 `accept` 中等待连接请求，就无法响应输入的命令；类似地，如果我们正在 `read` 中等待输入命令，就无法响应任何连接请求。

解决这种两难困境的一种技术称为 **I/O 多路复用（I/O multiplexing）**。基本思想是使用 `select` 函数要求内核挂起进程，只有在发生了一个或多个 I/O 事件之后，才将控制返回给应用程序，比如下面的例子：

- 当集合 $\{0, 4\}$ 中任一描述符读就绪时返回。
- 当集合 $\{1, 2, 7\}$ 中任一描述符写就绪时返回。
- 如果等待一个 I/O 事件发生已经过去了 152.13 秒，则超时。

`select` 是一个相当复杂的函数，有许多不同的使用场景。我们只讨论第一种场景：等待一组描述符读就绪。关于完整的讨论，请参见 [62, 110]。

```
#include <sys/select.h>

int select(int n, fd_set *fdset, NULL, NULL, NULL);

                                          返回：就绪描述符的个数（非零）；
                                          ↪  若出错则为 -1
```

```
FD_ZERO(fd_set *fdset);          /* 清除 fdset 中的所有位 */
```

```
FD_CLR(int fd, fd_set *fdset);   /* 清除 fdset 中的位 fd */
FD_SET(int fd, fd_set *fdset);   /* 打开 fdset 中的位 fd */
FD_ISSET(int fd, fd_set *fdset); /* fdset 中的位 fd 是打开的吗？*/
```

操作描述符集的宏

select 函数操作类型为 fd_set 的集合，这些集合称为**描述符集（descriptor set）**。从逻辑上讲，我们把一个描述符集看成是一个大小为 n 的位向量（在 2.1 节中介绍过）：

$$b_{n-1}, \ldots, b_1, b_0$$

每一位 b_k 都对应于描述符 k。当且仅当 $b_k = 1$ 时，描述符 k 是这个描述符集的一个成员。你只被允许对描述符集做三件事：(1) 分配它们；(2) 将这种类型的一个变量赋给另一个；(3) 使用 FD_ZERO、FD_SET、FD_CLR 和 FD_ISSET 宏对它们进行修改和检查。

就我们的目的而言，select 函数有两个输入：一个称为**读集（read set）**的描述符集 fdset，以及读集的基数 n（实际上是任何描述符集的最大基数）。select 函数会阻塞，直到读集中的至少一个描述符读就绪。当且仅当对该描述符读 1 字节的请求不会阻塞时，描述符 k 才是读就绪的。作为副作用，select 会修改参数 fdset 所指向的 fd_set，使之成为读集的一个称为**就绪集（ready set）**的子集，这个子集由读集中读就绪的描述符组成。函数返回值给出就绪集的基数。注意，由于这个副作用，我们必须在每次调用 select 时都更新读集。

理解 select 最好的方法是研究一个具体的例子。图 12.6 展示了我们可以如何用 select 来实现一个迭代 echo 服务器，同时还能接受用户在标准输入上键入的命令。我们先用图 11.19 中的 open_listenfd 函数打开一个监听描述符（第 16 行），再用 FD_ZERO 创建一个空的读集（第 18 行）：

	listenfd			stdin
	3	2	1	0
read_set $(\varnothing)$:	0	0	0	0

接着，在第 19 行和第 20 行中，我们把读集定义为由描述符 0（标准输入）和描述符 3（监听描述符）组成：

	listenfd			stdin
	3	2	1	0
read_set $(\{0,3\})$:	1	0	0	1

此时，我们开始典型的服务器循环。不过，我们不是通过调用 accept 函数来等待连接请求，而是调用 select 函数，它会阻塞，直到监听描述符或标准输入读就绪（第 24 行）。例如，下面是当用户按下回车键、从而使得标准输入描述符读就绪时，select 将返回的 ready_set 的值：

code/conc/select.c

```
 1  #include "csapp.h"
 2  void echo(int connfd);
 3  void command(void);
 4
 5  int main(int argc, char **argv)
 6  {
 7      int listenfd, connfd;
 8      socklen_t clientlen;
 9      struct sockaddr_storage clientaddr;
10      fd_set read_set, ready_set;
11
12      if (argc != 2) {
13          fprintf(stderr, "usage: %s <port>\n", argv[0]);
14          exit(0);
15      }
16      listenfd = Open_listenfd(argv[1]);
17
18      FD_ZERO(&read_set);              /* 清空读集 */
19      FD_SET(STDIN_FILENO, &read_set); /* 把 stdin 加进读集 */
20      FD_SET(listenfd, &read_set);     /* 把 listenfd 加进读集 */
21
22      while (1) {
23          ready_set = read_set;
24          Select(listenfd+1, &ready_set, NULL, NULL, NULL);
25          if (FD_ISSET(STDIN_FILENO, &ready_set))
26              command(); /* 从 stdin 读入一行命令 */
27          if (FD_ISSET(listenfd, &ready_set)) {
28              clientlen = sizeof(struct sockaddr_storage);
29              connfd = Accept(listenfd, (SA *)&clientaddr,
        &clientlen);
30              echo(connfd); /* 回显客户端的输入,直到 EOF */
31              Close(connfd);
32          }
33      }
34  }
35
36  void command(void) {
37      char buf[MAXLINE];
38      if (!Fgets(buf, MAXLINE, stdin))
39          exit(0); /* EOF */
40      printf("%s", buf); /* 处理输入的命令 */
41  }
```

图 12.6　使用 I/O 多路复用的迭代 echo 服务器。服务器用 `select` 等待监听描述符上的连接请求和标准输入上的命令。

	`listenfd`		`stdin`
	3	2 1	0
`ready_set` ($\{0\}$):	0	0 0	1

`select` 一旦返回，我们就用 `FD_ISSET` 宏来确定哪些描述符读就绪。如果标准输入就绪（第 25 行），我们就调用 `command` 函数，它读取、解析并响应这条命令，然后返回到主例程。如果监听描述符就绪（第 27 行），我们就调用 `accept` 来得到一个已连接描述符，然后调用图 11.22 中的 `echo` 函数，它回显来自客户端的每一行，直到客户端关闭它那一端的连接。

虽然这个程序很好地示范了 `select` 的用法，它还是有不尽如人意的地方。问题在于，一旦连接到某个客户端，它就会不断回显输入行，直到客户端关闭它那一端的连接。因此，如果你向标准输入键入一条命令，要等到服务器处理完该客户端才能得到响应。更好的做法是以更细的粒度进行多路复用，服务器循环每次迭代最多回显一行文本。

练习 12.3　在 Linux 系统中，键入 Ctrl+D 表示标准输入上遇到了 EOF。如果在图 12.6 的程序正逐行回显客户端内容时，向它键入 Ctrl+D，会发生什么？

12.2.1 基于 I/O 多路复用的并发事件驱动服务器

I/O 多路复用可以用作并发**事件驱动**（**event-driven**）程序的基础，在事件驱动程序中，各个流因某些事件的发生而向前推进。一般思路是把逻辑流建模为状态机。非正式地说，**状态机**（**state machine**）是一组状态、输入事件和转换的集合，其中转换把状态和输入事件映射为状态。每个转换都把一个（输入状态，输入事件）对映射为一个输出状态。自循环是输入状态与输出状态相同的转换。状态机通常画成有向图，其中节点表示状态，有向弧表示转换，弧上的标注表示输入事件。状态机从某个初始状态开始执行，每个输入事件都触发一次从当前状态到下一个状态的转换。

对于每个新客户端 k，基于 I/O 多路复用的并发服务器都创建一个新的状态机 s_k，并把它与已连接描述符 d_k 关联起来。如图 12.7 所示，每个状态机 s_k 有一个状态（“等待描述符 d_k 读就绪”）、一个输入事件（“描述符 d_k 读就绪”），以及一个转换（“从描述符 d_k 读一行文本”）。

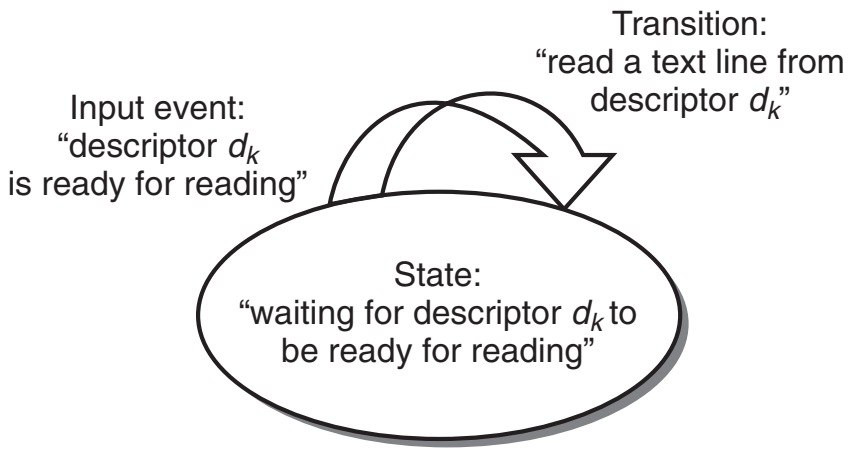


图 12.7　并发事件驱动的 echo 服务器中一个逻辑流的状态机。

服务器借助 `select` 函数提供的 I/O 多路复用，来检测输入事件的发生。每当某个已连接描述符读就绪时，服务器就执行相应状态机的转换——在这里，也就是从该描述符读出一行文本并回显。

图 12.8 给出了一个基于 I/O 多路复用的并发事件驱动服务器的完整示例代码。活动客户端的集合保存在一个**池(pool)**结构中(第 3 ~ 11 行)。在通过调用 `init_pool` 函数初始化这个池之后(第 27 行)，服务器进入一个无限循环。在循环的每次迭代中，服务器调用 `select` 函数检测两类不同的输入事件：(1) 来自新客户端的连接请求到达；(2) 已有客户端的某个已连接描述符读就绪。当连接请求到达时(第 35 行)，服务器打开连接(第 37 行)，并调用 `add_client` 函数把该客户端加入池(第 38 行)。最后，服务器调用 `check_clients` 函数，从每个读就绪的已连接描述符回显一行文本(第 42 行)。

`init_pool` 函数(图 12.9)初始化客户端池。`clientfd` 数组表示一个已连接描述符的集合，整数 −1 表示一个可用槽位。初始时，已连接描述符的集合是空的(第 5 ~ 7 行)，而监听描述符是 `select` 读集中唯一的描述符(第 10 ~ 12 行)。

`add_client` 函数（图 12.10）把一个新客户端加到活动客户端池中。在 `clientfd` 数组中找到一个空槽位之后，服务器把已连接描述符加进数组，并初始化一个相应的 Rio 读缓冲区，以便我们能够对该描述符调用 `rio_readlineb`（第 8 ~ 9 行）。然后，我们把该已连接描述符加到 `select` 读集中(第 12 行)，并更新池的一些全局属性。`maxfd` 变量(第 15 ~ 16 行)记录用于 `select` 的最大文件描述符；`maxi` 变量(第 17 ~ 18 行)记录 `clientfd` 数组的最大索引，使得 `check_clients` 函数不必搜索整个数组。

图 12.11 中的 check_clients 函数从每个读就绪的已连接描述符回显一行文本。如果我们成功地从描述符中读出了一行文本,那么我们就把这行文本回显给客户端(第 15 ~ 18 行)。注意,在第 15 行,我们还在维护一个从所有客户端收到的累计字节数。如果我们检测到了 EOF,说明客户端已经关闭了它那一端的连接,那么我们就关闭自己这一端的连接(第 23 行),并把这个描述符从池中移除(第 24 ~ 25 行)。

code/conc/echoservers.c

```
 1  #include "csapp.h"
 2
 3  typedef struct { /* 表示一个已连接描述符的池 */
 4      int maxfd;                  /* read_set 中最大的描述符 */
 5      fd_set read_set;            /* 所有活动描述符的集合 */
 6      fd_set ready_set;           /* 读就绪描述符的子集 */
 7      int nready;                 /* select 返回的读就绪描述符的个数 */
 8      int maxi;                   /* client 数组的高水位索引 */
 9      int clientfd[FD_SETSIZE];   /* 活动描述符的集合 */
10      rio_t clientrio[FD_SETSIZE]; /* 活动读缓冲区的集合 */
11  } pool;
12
13  int byte_cnt = 0; /* 记录服务器收到的总字节数 */
14
15  int main(int argc, char **argv)
16  {
17      int listenfd, connfd;
18      socklen_t clientlen;
19      struct sockaddr_storage clientaddr;
20      static pool pool;
21
22      if (argc != 2) {
23          fprintf(stderr, "usage: %s <port>\n", argv[0]);
24          exit(0);
25      }
26      listenfd = Open_listenfd(argv[1]);
27      init_pool(listenfd, &pool);
28
29      while (1) {
30          /* 等待监听/已连接描述符变成读就绪 */
31          pool.ready_set = pool.read_set;
32          pool.nready = Select(pool.maxfd+1, &pool.ready_set, NULL,
      NULL, NULL);
33
34          /* 如果监听描述符就绪,把新客户端加入池 */
35          if (FD_ISSET(listenfd, &pool.ready_set)) {
36              clientlen = sizeof(struct sockaddr_storage);
37              connfd = Accept(listenfd, (SA *)&clientaddr,
        &clientlen);
38              add_client(connfd, &pool);
39          }
40
41          /* 从每个读就绪的已连接描述符回显一行文本 */
```

```
42          check_clients(&pool);
43      }
44  }
```

图 12.8 基于 I/O 多路复用的并发 echo 服务器。服务器每次迭代都从每个读就绪的描述符回显一行文本。

code/conc/echoservers.c

```
 1  void init_pool(int listenfd, pool *p)
 2  {
 3      /* 最初,没有任何已连接描述符 */
 4      int i;
 5      p->maxi = -1;
 6      for (i=0; i< FD_SETSIZE; i++)
 7          p->clientfd[i] = -1;
 8
 9      /* 最初,listenfd 是 select 读集的唯一成员 */
10      p->maxfd = listenfd;
11      FD_ZERO(&p->read_set);
12      FD_SET(listenfd, &p->read_set);
13  }
```

图 12.9 `init_pool` 函数初始化活动客户端池。

code/conc/echoservers.c

```
 1  void add_client(int connfd, pool *p)
 2  {
 3      int i;
 4      p->nready--;
 5      for (i = 0; i < FD_SETSIZE; i++) /* 找一个可用槽位 */
 6          if (p->clientfd[i] < 0) {
 7              /* 把已连接描述符加进池 */
 8              p->clientfd[i] = connfd;
 9              Rio_readinitb(&p->clientrio[i], connfd);
10
11              /* 把该描述符加进描述符集 */
12              FD_SET(connfd, &p->read_set);
13
14              /* 更新最大描述符和池的高水位 */
15              if (connfd > p->maxfd)
16                  p->maxfd = connfd;
17              if (i > p->maxi)
18                  p->maxi = i;
19              break;
20          }
21      if (i == FD_SETSIZE) /* 没有找到空槽位 */
22          app_error("add_client error: Too many clients");
23  }
```

图 12.10 `add_client` 函数把一个新的客户端连接加进池。

code/conc/echoservers.c

```
 1  void check_clients(pool *p)
 2  {
 3      int i, connfd, n;
 4      char buf[MAXLINE];
 5      rio_t rio;
 6
 7      for (i = 0; (i <= p->maxi) && (p->nready > 0); i++) {
 8          connfd = p->clientfd[i];
 9          rio = p->clientrio[i];
10
11          /* 如果描述符就绪,就从它回显一行文本 */
12          if ((connfd > 0) && (FD_ISSET(connfd, &p->ready_set))) {
13              p->nready--;
14              if ((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
15                  byte_cnt += n;
16                  printf("Server received %d (%d total) bytes on fd
        %d\n",
17                         n, byte_cnt, connfd);
18                  Rio_writen(connfd, buf, n);
19              }
20
21              /* 检测到 EOF,把描述符从池中移除 */
22              else {
23                  Close(connfd);
24                  FD_CLR(connfd, &p->read_set);
25                  p->clientfd[i] = -1;
26              }
27          }
28      }
29  }
```

图 12.11 `check_clients` 函数为读就绪的客户端连接提供服务。

在图 12.7 的有限状态模型中,`select` 函数检测输入事件,`add_client` 函数创建一个新的逻辑流(状态机),`check_clients` 函数通过回显输入行来执行状态转换,并且在客户端发送完文本行时删除相应的状态机。

练习 12.4 在图 12.8 的服务器中,`pool.nready` 被重新初始化为调用 `select` 得到的值。这是为什么?

> **边栏 事件驱动的 Web 服务器**
> 尽管 12.2.2 节概述了这些缺点,Node.js、nginx 和 Tornado 这样的现代高性能服务器仍然使用基于 I/O 多路复用的事件驱动编程,主要原因在于它与进程和线程相比具有显著的性能优势。

12.2.2 I/O 多路复用的优缺点

图 12.8 中的服务器很好地例示了基于 I/O 多路复用的事件驱动编程的优点和缺点。一个优点是,与基于进程的设计相比,事件驱动设计给予了程序员

对程序行为更多的控制。例如,我们可以想象编写一个为某些客户端提供优先服务的事件驱动并发服务器,而这对基于进程的并发服务器来说是困难的。

另一个优点是,基于 I/O 多路复用的事件驱动服务器运行在单个进程的语境中,因此每个逻辑流都能访问该进程的整个地址空间。这使得在流之间共享数据变得容易。以单进程方式运行还有一个相关的好处:你可以像调试任何顺序程序一样调试你的并发服务器,使用诸如 gdb 这样熟悉的调试工具。最后,事件驱动设计通常比基于进程的设计高效得多,因为它们不需要进程上下文切换来调度一个新的流。

事件驱动设计的一个重要缺点是编码复杂。我们的事件驱动并发 echo 服务器所需的代码是基于进程的服务器的三倍。遗憾的是,复杂度还会随着并发粒度变细而增加。我们所说的粒度,是指每个逻辑流在每个时间片内执行的指令数。例如,在示例并发服务器中,并发粒度就是读出整行文本所需的指令数。只要某个逻辑流正忙着读一行文本,其他逻辑流就不能取得进展。这对我们的示例来说没有问题,但它使事件驱动服务器容易受到恶意客户端的攻击,这种客户端只发送半行文本,然后停下来。修改事件驱动服务器以处理不完整的文本行绝非易事,而基于进程的设计却能干净利落地自动处理这一点。事件驱动设计的另一个重要缺点是它们不能充分利用多核处理器。

12.3 基于线程的并发编程

到目前为止,我们已经看到了创建并发逻辑流的两种方法。第一种方法中,每个流使用一个单独的进程。内核自动调度每个进程,而且每个进程都有自己私有的地址空间,这使得流之间难以共享数据。第二种方法中,我们自己创建逻辑流,并用 I/O 多路复用来显式地调度这些流。因为只有一个进程,所以流们共享整个地址空间。

本节要介绍的是第三种方法——基于线程的方法,它是这两种方法的混合体。

线程(thread)是运行在进程语境中的逻辑流。到目前为止,本书中的程序都是每个进程只有一个线程。不过,现代系统也允许我们编写在一个进程中有多个线程并发运行的程序。线程由内核自动调度。每个线程都有它自己的**线程上下文(thread context)**,包括一个唯一的整数线程 ID(TID)、栈、栈指针、程序计数器、通用寄存器和条件码。在一个进程中运行的所有线程共享该进程的整个虚拟地址空间。

基于线程的逻辑流结合了基于进程的流和基于 I/O 多路复用的流的性质。像进程一样,线程由内核自动调度,并且内核通过一个整数 ID 来识别线程。像

基于 I/O 多路复用的流一样，多个线程运行在单个进程的语境中，因此它们共享该进程虚拟地址空间的全部内容，包括它的代码、数据、堆、共享库和打开的文件。

12.3.1 线程执行模型

多线程的执行模型在某些方面与多进程的执行模型相似。考虑图 12.12 的例子。每个进程都从单一线程开始，这个线程称为**主线程（main thread）**。在某个时刻，主线程创建一个**对等线程（peer thread）**，从这一时刻起，两个线程并发运行。最终，控制会经由一次上下文切换传递给对等线程，这或是因为主线程执行了一个慢系统调用（例如 `read` 或 `sleep`），或是因为它被系统的间隔定时器中断。对等线程执行一段时间，然后控制传回主线程，如此往复。

线程执行与进程执行在某些重要方面有所不同。因为线程上下文比进程上下文小得多，所以线程上下文切换比进程上下文切换快得多。另一个不同点是，线程不像进程那样组织成严格的父子层级。与某个进程相关联的线程组成一个对等线程的池，与哪个线程创建了哪些线程无关。主线程与其他线程的区别仅在于，它总是进程中第一个运行的线程。对等线程池这一概念的主要影响是，一个线程可以杀死它的任何对等线程，或者等待它的任意一个对等线程终止。而且，每个对等线程都可以读写相同的共享数据。

12.3.2 Posix 线程

Posix 线程（Pthreads）是在 C 程序中处理线程的一个标准接口。它于 1995 年被采纳，在所有 Linux 系统上都可以使用。Pthreads 定义了大约 60 个函数，允许程序创建、杀死和回收线程，与对等线程安全地共享数据，并且向对等线程通知系统状态的变化。

图 12.13 展示了一个简单的 Pthreads 程序。主线程创建一个对等线程，然后等待它终止。对等线程打印`"Hello, world!
textbackslashn"` 后终止。当主线程检测到对等线程已经终止时，它就通过调用 `exit` 来终止整个进程。这是我们看到的第一个多线程程序，所以让我们仔细剖析它。线程的代码和局部数据封装在一个线程例程（thread routine）中。正如第 2 行的原型所表明的，每个线程例程都以一个通用指针作为输入，并返回一个通用指针。如果你想给线程例程传递多个参数，那么你应该把这些参数放进一个结构中，传递一个指向该结构的指针。类似地，如果你想让线程例程返回多个参数，那么可以返回一个指向结构的指针。

code/conc/hello.c

```
1  #include "csapp.h"
```

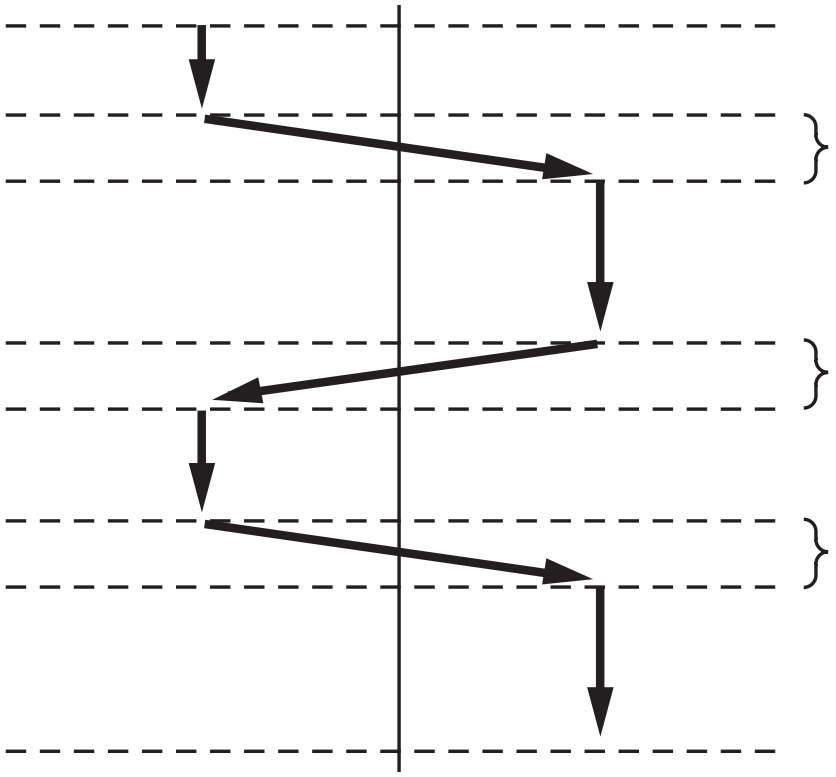


图 12.12 并发线程执行。

```
 2  void *thread(void *vargp);
 3
 4  int main()
 5  {
 6      pthread_t tid;
 7      Pthread_create(&tid, NULL, thread, NULL);
 8      Pthread_join(tid, NULL);
 9      exit(0);
10  }
11
12  void *thread(void *vargp) /* 线程例程 */
13  {
14      printf("Hello, world!\n");
15      return NULL;
16  }
```

图 12.13 `hello.c`：Pthreads 版的“Hello, world!”程序。

第 4 行是主线程代码的起始处。主线程声明了一个局部变量 `tid`，它将用来存放对等线程的线程 ID（第 6 行）。主线程通过调用 `pthread_create` 函数来创建一个新的对等线程（第 7 行）。当调用 `pthread_create` 返回时，主线程和这个新创建的对等线程就在并发运行了，而且 `tid` 中包含新线程的 ID。主线程用第 8 行的 `pthread_join` 调用来等待对等线程终止。最后，主线程调用

exit（第 9 行），它会终止当前在该进程中运行的所有线程（在这个例子中，只有主线程）。

第 12 ~ 16 行定义了对等线程的线程例程。它只是打印一个字符串，然后通过执行第 15 行的 return 语句来终止对等线程。

12.3.3 创建线程

线程通过调用 pthread_create 函数来创建其他线程。

```
#include <pthread.h>
typedef void *(func)(void *);
int pthread_create(pthread_t *tid, pthread_attr_t *attr,
                   func *f, void *arg);

                                                返回：若成功则为 0，
                                                ↪  若出错则为非 0
```

pthread_create 函数创建一个新线程，在新线程的语境中运行线程例程 f，输入参数为 arg。参数 attr 可以用来改变新创建线程的默认属性。改变这些属性超出了我们的讨论范围，在我们的示例中，我们总是以 NULL 作为 attr 参数来调用 pthread_create。

当 pthread_create 返回时，参数 tid 中包含新创建线程的 ID。新线程可以通过调用 pthread_self 函数来确定自己的线程 ID。

```
#include <pthread.h>

pthread_t pthread_self(void);

                                                返回：调用者的线程 ID
```

12.3.4 终止线程

线程以如下几种方式之一终止：

- **隐式终止。** 当顶层的线程例程返回时，线程隐式终止。
- **显式终止。** 线程通过调用 pthread_exit 函数显式终止。如果主线程调用 pthread_exit，它会等待所有其他对等线程终止，然后以返回值 thread_return 终止主线程和整个进程。

```
#include <pthread.h>

void pthread_exit(void *thread_return);

                                                永不返回
```

- **某个对等线程调用了 Linux 的 exit 函数。** exit 函数会终止进程，以及与该进程相关联的所有线程。

- **其他对等线程终止了当前线程。** 另一个对等线程以当前线程的 ID 为参数调用 `pthread_cancel` 函数,终止当前线程。

```
#include <pthread.h>

int pthread_cancel(pthread_t tid);

                                                    返回:若成功则为 0,
                                                    ↪ 若出错则为非 0
```

12.3.5 回收已终止线程的资源

线程通过调用 `pthread_join` 函数来等待其他线程终止。

```
#include <pthread.h>

int pthread_join(pthread_t tid, void **thread_return);

                                                    返回:若成功则为 0,
                                                    ↪ 若出错则为非 0
```

`pthread_join` 函数会阻塞,直到线程 `tid` 终止,把线程例程返回的通用(`void *`)指针赋值给 `thread_return` 所指向的位置,然后回收已终止线程持有的任何内存资源。

注意,与 Linux 的 `wait` 函数不同,`pthread_join` 函数只能等待某个特定的线程终止。没有任何办法指示 `pthread_join` 去等待任意一个线程终止。这会使我们的代码变得复杂,因为它强迫我们使用其他不那么直观的机制来检测线程的终止。实际上,Stevens 令人信服地论证了这是规范中的一个缺陷 [110]。

12.3.6 分离线程

在任一时刻,线程要么是**可结合(joinable)**的,要么是**分离(detached)**的。一个可结合线程能够被其他线程回收和杀死,它的内存资源(例如栈)在它被其他线程回收之前不会释放。相反,一个分离线程不能被其他线程回收或杀死,它的内存资源在它终止时由系统自动释放。

默认情况下,线程被创建为可结合的。为了避免内存泄漏,每个可结合线程要么被另一个线程显式回收,要么通过调用 `pthread_detach` 函数被分离。

```
#include <pthread.h>

int pthread_detach(pthread_t tid);

                                                    返回:若成功则为 0,
                                                    ↪ 若出错则为非 0
```

`pthread_detach` 函数分离可结合线程 `tid`。线程可以通过以 `pthread_self()` 为参数调用 `pthread_detach` 来分离自己。

虽然我们的一些示例会使用可结合线程，但在实际的程序中，使用分离线程有很好的理由。例如，一个高性能的 Web 服务器可能在每次收到来自 Web 浏览器的连接请求时，创建一个新的对等线程。既然每个连接都是由一个单独的线程独立处理的，让服务器显式地等待每个对等线程终止就是不必要的——实际上也是不希望发生的。在这种情形下，每个对等线程都应该在开始处理请求之前分离自己，这样它的内存资源就可以在它终止之后被回收。

12.3.7 初始化线程

pthread_once 函数允许你初始化与线程例程相关联的状态。

```
#include <pthread.h>

pthread_once_t once_control = PTHREAD_ONCE_INIT;
int pthread_once(pthread_once_t *once_control,
                 void (*init_routine)(void));

                                                    总是返回 0
```

once_control 变量是全局变量或静态变量，它总是被初始化为 PTHREAD_ONCE_INIT。第一次以 once_control 为参数调用 pthread_once 时，它会调用 init_routine，这是一个没有输入参数、也不返回任何值的函数。此后每次用同一个 once_control 变量调用 pthread_once 都不会做任何事情。每当你需要动态地初始化被多个线程共享的全局变量时，pthread_once 函数都很有用。我们将在 12.5.5 节看到一个例子。

12.3.8 基于线程的并发服务器

图 12.14 给出了一个基于线程的并发 echo 服务器的代码。整体结构与基于进程的设计相似：主线程反复地等待连接请求，然后创建一个对等线程来处理该请求。虽然这段代码看起来简单，却有几个一般性的、也有些微妙的问题需要我们更仔细地审视。第一个问题是，当我们调用 pthread_create 时，应该如何把已连接描述符传给对等线程。显而易见的做法是传递一个指向该描述符的指针，如下所示：

```
connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
Pthread_create(&tid, NULL, thread, &connfd);
```

然后我们让对等线程解引用该指针，并把它赋给一个局部变量，如下所示：

```
void *thread(void *vargp) {
    int connfd = *((int *)vargp);
    ...
}
```

code/conc/echoservert.c

```
 1  #include "csapp.h"
 2
 3  void echo(int connfd);
 4  void *thread(void *vargp);
 5
 6  int main(int argc, char **argv)
 7  {
 8      int listenfd, *connfdp;
 9      socklen_t clientlen;
10      struct sockaddr_storage clientaddr;
11      pthread_t tid;
12
13      if (argc != 2) {
14          fprintf(stderr, "usage: %s <port>\n", argv[0]);
15          exit(0);
16      }
17      listenfd = Open_listenfd(argv[1]);
18
19      while (1) {
20          clientlen=sizeof(struct sockaddr_storage);
21          connfdp = Malloc(sizeof(int));
22          *connfdp = Accept(listenfd, (SA *) &clientaddr,
 ↪  &clientlen);
23          Pthread_create(&tid, NULL, thread, connfdp);
24      }
25  }
26
27  /* 线程例程 */
28  void *thread(void *vargp)
29  {
30      int connfd = *((int *)vargp);
31      Pthread_detach(pthread_self());
32      Free(vargp);
33      echo(connfd);
34      Close(connfd);
35      return NULL;
36  }
```

图 12.14 基于线程的并发 echo 服务器。

然而，这样做是错误的，因为它在对等线程中的赋值语句与主线程中的 `accept` 语句之间引入了一个竞态。如果赋值语句在下一次 `accept` 之前完成，那么对等线程中的局部变量 `connfd` 就会得到正确的描述符值。但是，如果赋值语句在 `accept` 之后才完成，那么对等线程中的局部变量 `connfd` 就会得到下一个连接的描述符号。不妙的结果是，现在有两个线程在同一个描述符上执行输入和输出。为了避免这种可能造成致命后果的竞态，我们必须把 `accept` 返回的每个已连接描述符都赋给它自己的动态分配的内存块，如图 12.14 的第 21 ~ 22 行所示。我们将在 12.7.4 节回到竞态的话题。

另一个问题是线程例程中要避免内存泄漏。既然我们不显式地回收线程，就必须分离每个线程，这样它的内存资源才会在它终止时被回收(第 31 行)。此外，我们还必须小心地释放由主线程分配的那个内存块(第 32 行)。

练习 12.5 在图 12.5 所示的基于进程的服务器中，我们观察到：即使删除了第 33 行，代码也没有内存泄漏并且仍然是正确的。在图 12.14 所示的基于线程的服务器中，如果删除了第 31 行或第 32 行，有没有可能出现内存泄漏？为什么？

12.4 多线程程序中的共享变量

从程序员的角度来看，线程的一个吸引人之处是多个线程能够很容易地共享相同的程序变量。不过，这种共享可能是相当微妙的。要编写正确的多线程程序，我们必须对共享的含义以及它是如何工作的有清晰的认识。

要理解一个 C 程序中的变量是否是共享的，有一些基本的问题需要理清：(1) 线程的底层内存模型是什么样的？(2) 照此模型，变量的实例是如何映射到内存的？(3) 最后，有多少个线程引用这些实例的每一个？当且仅当有多个线程引用某个变量的实例时，这个变量才是共享的。

为了让关于共享的讨论具体些，我们用图 12.15 中的程序作为贯穿本节的示例。这个程序虽然有些刻意，但研究它仍然很有用，因为它展示了关于共享的许多微妙的要点。这个示例程序由一个主线程和两个对等线程组成，主线程给每个对等线程传递一个唯一的 ID，对等线程用这个 ID 打印一条个性化的消息，还打印线程例程被调用的总次数。

code/conc/sharing.c

```
 1  #include "csapp.h"
 2  #define N 2
 3  void *thread(void *vargp);
 4
 5  char **ptr;   /* 全局变量 */
 6
 7  int main()
 8  {
 9      int i;
10      pthread_t tid;
11      char *msgs[N] = {
12          "Hello from foo",
13          "Hello from bar"
14      };
15
16      ptr = msgs;
17      for (i = 0; i < N; i++)
18          Pthread_create(&tid, NULL, thread, (void *)i);
19      Pthread_exit(NULL);
```

```
20  }
21
22  void *thread(void *vargp)
23  {
24      int myid = (int)vargp;
25      static int cnt = 0;
26      printf("[%d]: %s (cnt=%d)\n", myid, ptr[myid], ++cnt);
27      return NULL;
28  }
```

图 12.15 展示共享的不同方面的示例程序。

12.4.1 线程内存模型

一组并发线程运行在一个进程的语境中。每个线程都有自己独立的线程上下文，包括线程 ID、栈、栈指针、程序计数器、条件码和通用寄存器的值。每个线程与其他线程共享进程上下文剩下的部分，这包括整个用户虚拟地址空间，它由只读文本（代码）、读/写数据、堆以及所有共享库代码和数据区域组成。线程还共享同一组打开的文件。

从操作的意义上说，一个线程不可能读或写另一个线程的寄存器值。另一方面，任何线程都能访问共享虚拟内存中的任意位置。如果某个线程修改了一个内存位置，那么其他每个线程只要读取那个位置，最终都会看到这一修改。因此，寄存器从不共享，而虚拟内存总是共享的。

独立的线程栈的内存模型则没有这么干净。这些栈包含在虚拟地址空间的栈区域中，通常由它们各自的线程独立访问。我们说“通常”而不说“总是”，是因为各个线程的栈并没有保护起来不被其他线程访问。所以，如果一个线程以某种方式设法获得了指向另一个线程栈的指针，那么它就能读写那个栈的任意部分。我们的示例程序在第 26 行展示了这一点：对等线程通过全局变量 `ptr` 间接地引用了主线程栈的内容。

12.4.2 将变量映射到内存

在多线程的 C 程序中，变量按照它们的**存储类别**（**storage class**）映射到虚拟内存中：

- **全局变量**。全局变量是声明在函数之外的任何变量。在运行时，虚拟内存的读/写区恰好包含每个全局变量的一个实例，任何线程都可以引用它。例如，第 5 行声明的全局变量 `ptr` 在虚拟内存的读/写区中有一个运行时实例。当某个变量只有一个实例时，我们就直接用变量名来表示这个实例——在这里就是 `ptr`。

- **局部自动变量**。局部自动变量(automatic variable)是声明在函数内部、不带 `static` 属性的变量。在运行时,每个线程的栈都包含它对任何局部自动变量的自己的实例。即使多个线程执行的线程例程相同,这一点也成立。例如,局部变量 `tid` 有一个实例,它位于主线程的栈上。我们将把这个实例记作 `tid.m`。作为另一个例子,局部变量 `myid` 有两个实例,一个位于对等线程 0 的栈上,另一个位于对等线程 1 的栈上。我们分别把这两个实例记作 `myid.p0` 和 `myid.p1`。
- **局部静态变量**。局部静态变量是声明在函数内部、带有 `static` 属性的变量。与全局变量一样,虚拟内存的读/写区恰好包含每个局部静态

声明并带有 static 属性的变量。与全局变量一样,程序中声明的每个局部静态变量,运行时在虚拟内存的可读写区域中恰好只有一个实例。例如,虽然示例程序的每个对等线程都在第 25 行声明了 `cnt`,但运行时内存的可读写区域中只有一个 `cnt` 实例,每个对等线程都会读写这个实例。

12.4.3 共享变量

当且仅当变量 v 的某个实例被多于一个线程引用时,我们称 v 是共享的(shared)。例如,示例程序中的变量 `cnt` 是共享的:它只有一个运行时实例,而这个实例被两个对等线程同时引用。反过来,`myid` 不共享,因为它的两个实例各自只被一个线程引用。不过要注意,`msgs` 这样的局部自动变量也可以是共享的。

练习 12.6 运用 12.4 节的分析方法,对图 12.15 的示例程序填写下表,每一项填"是"或"否"。第一列的记号 `v.t` 表示驻留在线程 t 的局部栈上的变量 v 的实例,其中 t 为 m(主线程)、$p0$(对等线程 0)或 $p1$(对等线程 1)。

	被谁引用		
变量实例	主线程?	对等线程 0?	对等线程 1?
`ptr`			
`cnt`			
`i.m`			
`msgs.m`			
`myid.p0`			
`myid.p1`			

B. 根据部分 A 的分析,变量 `ptr`、`cnt`、`i`、`msgs` 和 `myid` 中哪些是共享的?

12.5 用信号量同步线程

共享变量用起来方便，但也可能引入讨厌的同步错误。考虑图 12.16 的 badcnt.c 程序，它创建两个线程，每个线程都对一个名为 `cnt` 的全局共享计数变量做递增。

由于每个线程把计数器递增 `niters` 次，我们预期其最终值为 $2 \times niters$。这看起来简单而直接。然而，当我们在 Linux 系统上运行 `badcnt.c` 时，不仅答案不对，而且每次运行得到的答案还各不相同！

```
 1  /* 警告:这段代码有缺陷!  */
 2  #include "csapp.h"
 3
 4  void *thread(void *vargp);   /* 线程例程的原型 */
 5
 6  /* 全局共享变量 */
 7  volatile long cnt = 0; /* 计数器 */
 8
 9  int main(int argc, char **argv)
10  {
11      long niters;
12      pthread_t tid1, tid2;
13
14      /* 检查输入参数 */
15      if (argc != 2) {
16          printf("usage: %s <niters>\n", argv[0]);
17          exit(0);
18      }
19      niters = atoi(argv[1]);
20
21      /* 创建线程并等待它们结束 */
22      Pthread_create(&tid1, NULL, thread, &niters);
23      Pthread_create(&tid2, NULL, thread, &niters);
24      Pthread_join(tid1, NULL);
25      Pthread_join(tid2, NULL);
26
27      /* 检查结果 */
28      if (cnt != (2 * niters))
29          printf("BOOM! cnt=%ld\n", cnt);
30      else
31          printf("OK cnt=%ld\n", cnt);
32      exit(0);
33  }
34
35  /* 线程例程 */
36  void *thread(void *vargp)
37  {
38      long i, niters = *((long *)vargp);
39
40      for (i = 0; i < niters; i++)
41          cnt++;
42
43      return NULL;
```

```
44  }
```

图 12.16 badcnt.c:一个未正确同步的计数器程序。

```
linux> ./badcnt 1000000
BOOM! cnt=1445085
linux> ./badcnt 1000000
BOOM! cnt=1915220
linux> ./badcnt 1000000
BOOM! cnt=1404746
```

那么,究竟哪里出了错? 要清楚地理解这个问题,需要研究计数循环(第 40 ~ 41 行)的汇编代码,如图 12.17 所示。把线程 i 的循环代码划分为五个部分会很有帮助:

- H_i:位于循环头部的指令块。
- L_i:把共享变量 cnt 加载到累加寄存器 %rdx$_i$ 的指令,其中 %rdx$_i$ 表示线程 i 中寄存器%rdx 的值。
- U_i:更新(递增)%rdx$_i$ 的指令。
- S_i:把更新后的 %rdx$_i$ 值存回共享变量 cnt 的指令。
- T_i:位于循环尾部的指令块。

注意,头部和尾部只操作局部栈变量,而 L_i、U_i 和 S_i 操作的是共享计数变量的内容。

当 badcnt.c 的两个对等线程并发地运行在单处理器上时,机器指令按某种顺序一个接一个地完成。这样,每一次并发执行都定义了两个线程指令之间的某种全序(total ordering,即交错顺序)。遗憾的是,这些排序中有些会产生正确的结果,另一些则不会。

```
                    线程 i 的汇编代码
                       movq  (%rdi), %rcx
                       testq %rcx, %rcx
                       jle   .L2                    Hi:头部
                       movl  $0, %eax
线程 i 的 C 代码    .L3:                             Li:加载 cnt
for (i = 0; i < niters; i++)
                       movq  cnt(%rip),%rdx
                                                    Ui:更新 cnt
    cnt++;             addq  %eax
                       movq  %eax,cnt(%rip)         Si:存储 cnt
                       addq  $1, %rax
                       cmpq  %rcx, %rax             Ti:尾部
                       jne   .L3
                   .L2:
```

图 12.17 badcnt.c 计数循环(第 40 ~ 41 行)的汇编代码。

图 12.18　badcnt.c 第一次循环迭代的指令排序：(a)正确排序；(b)不正确排序

步	线程	指令	%rdx$_1$	%rdx$_2$	cnt
1	1	H1	—	—	0
2	1	L1	0	—	0
3	1	U1	1	—	0
4	1	S1	1	—	1
5	2	H2	—	—	1
6	2	L2	—	1	1
7	2	U2	—	2	1
8	2	S2	—	2	2
9	2	T2	—	2	2
10	1	T1	1	—	2

步	线程	指令	%rdx$_1$	%rdx$_2$	cnt
1	1	H1	—	—	0
2	1	L1	0	—	0
3	1	U1	1	—	0
4	2	H2	—	—	0
5	2	L2	—	0	0
6	1	S1	1	—	1
7	1	T1	1	—	1
8	2	U2	—	1	1
9	2	S2	—	1	1
10	2	T2	—	1	1

这里是关键之处：一般来说，你无法预测操作系统会不会为你的线程选择一种正确的排序。例如，图 12.18(a) 展示了一种正确指令排序的逐步执行过程。每个线程都对共享变量 cnt 做完更新之后，它在内存中的值是 2，这正是期望的结果。

另一方面，图 12.18(b) 的排序会为 cnt 产生一个不正确的值。问题出现的原因是：线程 2 在第 5 步加载 cnt，这发生在线程 1 于第 2 步加载之后、线程 1 于第 6 步存回更新值之前。这样，每个线程最终都存回一个值为 1 的递增计数器。借助于一种称为进度图(progress graph)的工具，可以澄清正确与不正确的指令排序这些概念，下一节将介绍进度图。

练习 12.7　为 badcnt.c 的下列指令排序补全表格：

步	线程	指令	%rdx$_1$	%rdx$_2$	cnt
1	1	H1	—	—	0
2	1	L1			
3	2	H2			
4	2	L2			
5	2	U2			
6	2	S2			
7	1	U1			
步	线程	指令	%rdx$_1$	%rdx$_2$	cnt
8	1	S1			
9	1	T1			
10	2	T2			

这个排序会得到正确的 cnt 值吗?

12.5.1 进度图

进度图把 n 个并发线程的执行建模为 n 维笛卡儿空间中的一条轨迹(trajectory)。每根坐标轴 k 对应线程 k 的进度。每个点 $(I_1, I_2, \ldots, I_n)$ 代表这样的状态:线程 $k(k = 1, \ldots, n)$已经完成了指令 I_k。图的原点对应所有线程都尚未完成一条指令的初始状态。

图 12.19 展示了 badcnt.c 程序第一次循环迭代的二维进度图。水平轴对应线程 1,垂直轴对应线程 2。点 (L_1, S_2) 对应线程 1 完成 L_1、线程 2 完成 S_2 的状态。

进度图把指令的执行建模为从一个状态到另一个状态的转移。转移表示为从一个点到相邻点的一条有向边。合法的转移只能向右移动(线程 1 的一条指令完成)或向上移动(线程 2 的一条指令完成)。两条指令不可能同时完成——不允许对角转移。程序绝不会倒退运行,所以向下或向左的转移同样不合法。

程序的执行历史被建模为穿过状态空间的一条轨迹。图 12.20 展示了对应于下列指令排序的轨迹:

$$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$$

对线程 i 来说,操作共享变量 cnt 内容的指令 (L_i, U_i, S_i) 构成一个相对于共享变量 cnt 的临界区(critical section),它不应与另一个线程的临界区交错。也就是说,我们在每个线程执行其临界区内的指令时,都要保证它对共享变量的访问是互斥的。这种一般现象称为互斥(mutual exclusion)。

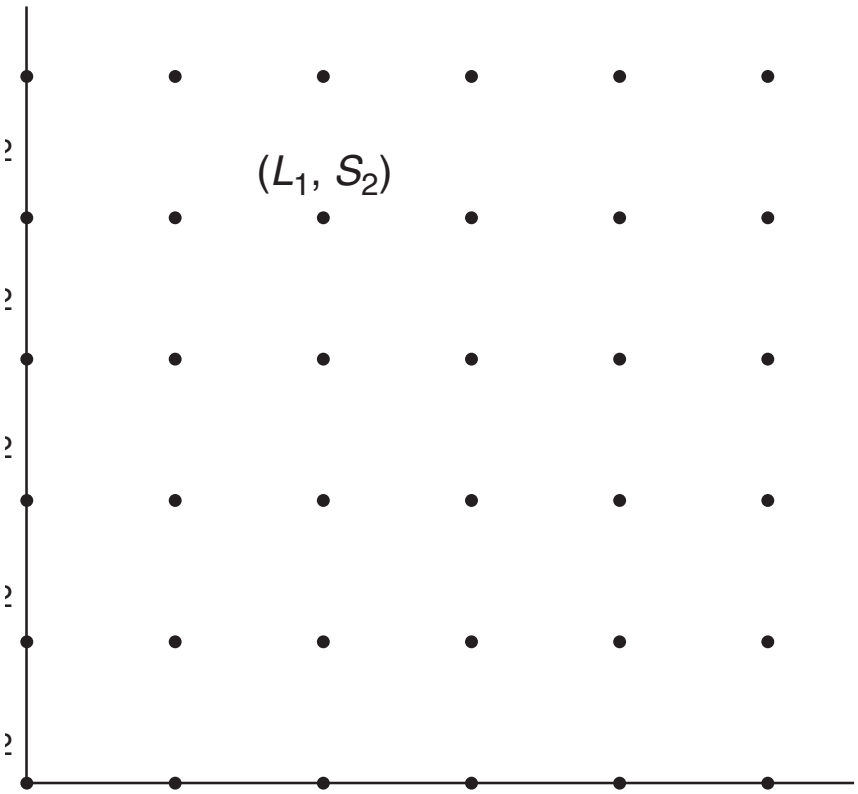


图 12.19　badcnt.c 第一次循环迭代的进度图。

在进度图上，两个临界区的相交定义了状态空间中一个称为不安全区域(unsafe region)的区域。图 12.21 展示了变量 cnt 的不安全区域。注意，不安全区域与其周界上的状态相邻，但并不把它们包括在内。例如，状态 (H_1, H_2) 和 (S_1, U_2) 与不安全区域相邻，但不属于它。绕开不安全区域的轨迹称为安全轨迹(safe trajectory)；反之，触及不安全区域任何部分的轨迹就是不安全轨迹。图 12.21 给出了在示例程序 badcnt.c 的状态空间中穿行的安全与不安全轨迹的例子。上面那条轨迹沿不安全区域的左侧和顶边绕行，因而安全；下面那条穿过不安全区域，因而不安全。

任何安全轨迹都能正确地更新共享计数器。要保证示例多线程程序正确执行——事实上，任何共享全局数据结构的并发程序都要如此——就必须以某种方式同步线程，使它们始终走在安全轨迹上。一种经典方法以信号量的思想为基础，下面就来介绍。

练习 12.8　利用图 12.21 的进度图，把下列轨迹分类为安全或不安全。

A. $H_1, L_1, U_1, S_1, H_2, L_2, U_2, S_2, T_2, T_1$

B. $H_2, L_2, H_1, L_1, U_1, S_1, T_1, U_2, S_2, T_2$

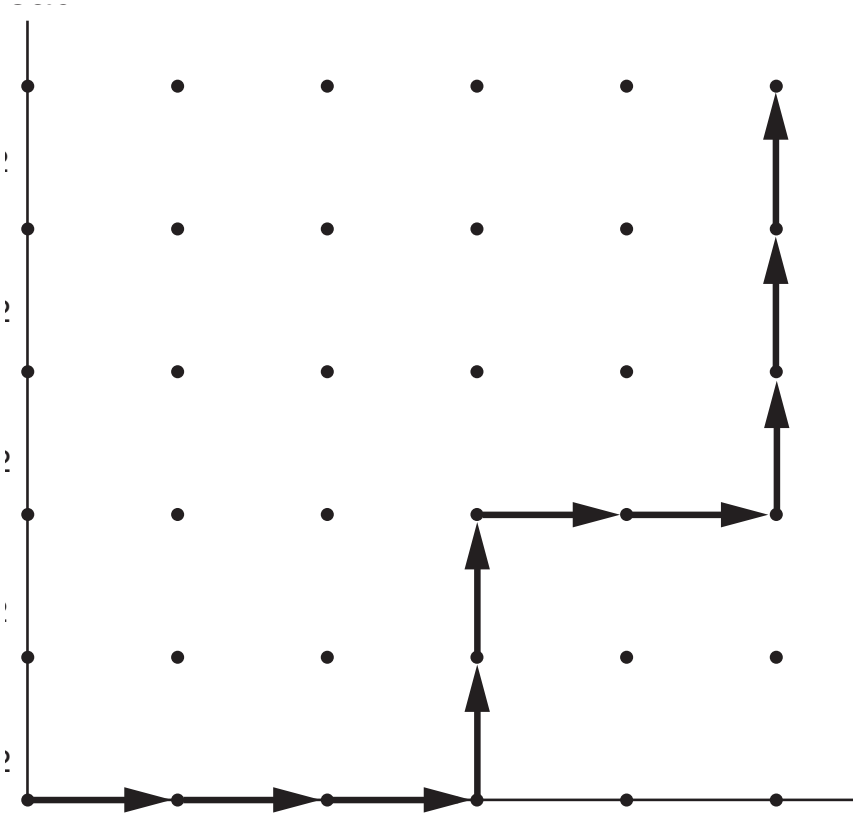


图 12.20 一个轨迹的例子。

C. $H_1, H_2, L_2, U_2, S_2, L_1, U_1, S_1, T_1, T_2$

12.5.2 信号量

并发程序设计的先驱 Edsger Dijkstra 提出了一种经典的解决方案，用来同步不同的执行线程，其基础是一种称为信号量（semaphore）的特殊变量。信号量 s 是一个取值为非负整数的全局变量，只能通过两种称为 P 和 V 的特殊操作操纵：

- $P(s)$：如果 s 非零，那么 P 操作把 s 减 1 并立即返回。如果 s 为零，那么挂起线程，直到 s 变为非零、且有一个 V 操作重启该线程。重启之后，P 操作把 s 减 1，并把控制权交还给调用者。
- $V(s)$：V 操作把 s 加 1。如果存在阻塞在某个 P 操作处等待 s 变为非零的线程，那么 V 操作恰好重启其中一个线程，该线程随后完成它的 P 操作，把 s 减 1。

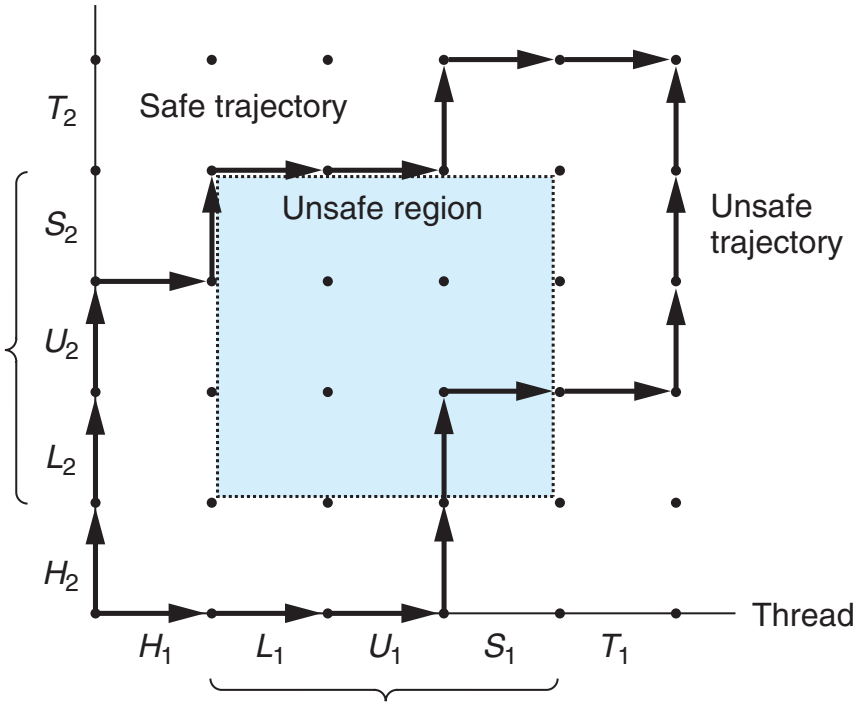


图 12.21 安全与不安全的轨迹。临界区的交集构成一个不安全区域，避开不安全区域的轨迹会正确地更新计数器变量。

> **边栏 P 和 V 两个名字的由来**
> Edsger Dijkstra（1930—2002）来自荷兰。P 和 V 两个名字来自荷兰语单词 *proberen*（测试）和 *verhogen*（递增）。

P 中的测试和减 1 操作是不可分地进行的，意思是：一旦信号量 s 变为非零，对 s 的减 1 就会不受中断地完成。V 中的加 1 操作同样不可分：它把对信号量的加载、加 1 和存储一气呵成。注意，V 的定义并没有规定等待线程被重启的顺序。唯一的要求是 V 必须恰好重启一个等待的线程。因此，当多个线程在同一个信号量处等待时，你无法预测 V 之后哪个线程会被重启。

P 和 V 的定义保证：一个正在运行的程序永远不可能进入某个正确初始化的信号量取负值的状态。这个性质称为信号量不变式（semaphore invariant），它是控制并发程序轨迹的有力工具，下一节将看到它的用法。

Posix 标准定义了多种操纵信号量的函数：

```
#include <semaphore.h>

int sem_init(sem_t *sem, 0, unsigned int value);
int sem_wait(sem_t *s);   /* P(s) */
int sem_post(sem_t *s);   /* V(s) */
        返回：成功为 0；出错为 -1
```

sem_init 函数把信号量 sem 初始化为 value。每个信号量在使用之前都必须先初始化。对我们的用途来说，中间的参数总是 0。程序分别通过调用 sem_wait 和 sem_post 函数来执行 P 和 V 操作。为简洁起见，我们更愿意改用下面两个等价的 P 和 V 包装函数：

```
#include "csapp.h"
void P(sem_t *s);       /* sem_wait 的包装函数 */
void V(sem_t *s);       /* sem_post 的包装函数 */
        返回：无
```

12.5.3 使用信号量实现互斥

信号量为确保对共享变量的互斥访问提供了便利方法。基本思想是：把一个初值为 1 的信号量 s 与

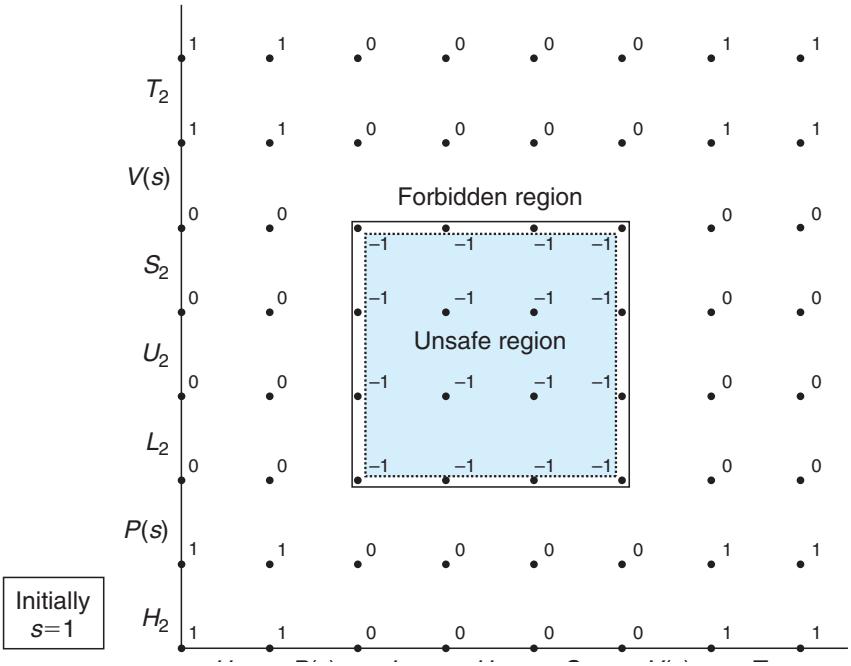


图 12.22 用信号量实现互斥。s < 0 的不可行状态定义了一个围绕不安全区域的禁区，它阻止任何可行轨迹触及不安全区域。

每个共享变量（或一组相关的共享变量）联系起来，然后用 $P(s)$ 和 $V(s)$ 操作把相应的临界区包围起来。

以这种方式用来保护共享变量的信号量称为二元信号量 （binary semaphore），因为它的值总是 0 或 1。以提供互斥为目的的二元信号量常常称为互斥锁（mutex）。对互斥锁执行 P 操作称为对它加锁（lock）；类似地，执行 V 操作称为解锁（unlock）。已经对互斥锁加锁但还没有解锁的线程称为持有

(hold) 互斥锁。用作一组可用资源计数器的信号量称为计数信号量 (counting semaphore)。

图 12.22 的进度图展示了如何用二元信号量来正确同步我们的示例计数程序。

每个状态都用该状态下信号量 s 的值标注。思想的关键在于:这一 P 和 V 操作的组合创造出一组称为禁区(forbidden region)的状态,在禁区中 $s < 0$。由于信号量不变式,任何可行轨迹都不可能包含禁区中的状态。又因为禁区完整地包围着不安全区域,任何可行轨迹都不可能触及不安全区域的任何部分。于是每条可行轨迹都是安全的,无论运行时指令以何种顺序执行,程序都能正确地对计数器做递增。

边栏 进度图的局限

进度图让我们能够直观地查看单处理器上并发程序的执行,并理解为什么需要同步。不过它们也有局限,对多处理器上的并发执行尤其如此:在多处理器上,一组 CPU/高速缓存对共享同一个主存。多处理器的行为方式是进度图解释不了的:多处理器存储器系统可能处于某种与进度图中任何轨迹都不对应的状态。但结论不变:无论你的程序运行在单处理器还是多处理器上,都要始终同步对共享变量的访问。

从操作的角度看,P 和 V 操作创造的禁区使得任何时刻都不可能有多于一个线程在被包围的临界区域中执行指令。换言之,信号量操作保证了对临界区域的互斥访问。

把这一切合在一起:要用信号量正确同步图 12.16 的示例计数程序,首先声明一个名为 `mutex` 的信号量:

```
volatile long cnt = 0;  /* 计数器 */
sem_t mutex;            /* 保护计数器的信号量 */
```

然后在主例程中把它初始化为 1:

```
Sem_init(&mutex, 0, 1);        /* mutex = 1 */
```

最后,在线程例程中用 P 和 V 操作包围对共享变量 `cnt` 的更新,保护它:

```
for (i = 0; i < niters; i++) {
    P(&mutex);
    cnt++;
    V(&mutex);
}
```

运行这个正确同步的程序,它现在每次都能产生正确的答案。

```
linux> ./goodcnt 1000000
OK cnt=2000000
linux> ./goodcnt 1000000
OK cnt=2000000
```

12.5.4 利用信号量来调度共享资源

信号量除了用来提供互斥之外，另一个重要作用是对共享资源的访问进行调度。在这种场景下，线程用信号量

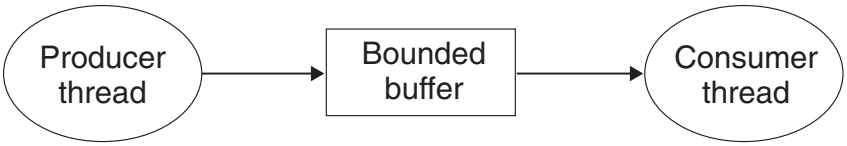


图 12.23 生产者-消费者问题。生产者生成项目并将它们插入一个有界缓冲区，消费者从缓冲区中取出项目并加以消费。

操作来通知另一个线程：程序状态中的某个条件已经变为真。两个经典而实用的例子是生产者-消费者问题和读者-写者问题。

生产者-消费者问题

生产者-消费者问题如图 12.23 所示。生产者线程和消费者线程共享一个有 n 个槽位的有界缓冲区。生产者线程反复生成新条目并插入缓冲区；消费者线程反复从缓冲区取出条目并加以消费（使用）。也可以有多个生产者和多个消费者的变体。

由于插入和取出条目都要更新共享变量，我们必须保证对缓冲区的互斥访问。但只保证互斥还不够，我们还需要对缓冲区的访问进行调度：缓冲区满了（没有空槽位）时，生产者必须等待，直到有槽位空出来；同样，缓冲区空了（没有可用的条目）时，消费者必须等待，直到有条目可用。

生产者-消费者交互在真实系统中很常见。例如在多媒体系统中，生产者编码视频帧，而消费者解码并把帧渲染到屏幕上。缓冲区的目的是减小视频流的抖动，这种抖动源于各帧编码与解码时间因数据而异的差别。缓冲区为生产者提供了一个槽位的蓄水池，为消费者提供了一个已编码帧的蓄水池。另一个常见的例子是图形用户界面的设计：生产者检测鼠标和键盘事件并把它们插入缓冲区；消费者按某种基于优先级的方式从缓冲区取出事件并绘制屏幕。

本节我们将开发一个名为 Sbuf 的简单包，用于构建生产者-消费者程序。下一节我们看看如何用它在预线程化（prethreading）技术之上构建一个有趣的并发服务器。Sbuf 操纵类型为 `sbuf_t`（图 12.24）的有界缓冲区。条目存放在一个动态分配的、容纳 n 个条目的整数数组（buf）中。下标 front 和 rear 分别记录数组中第一个和最后一个条目的位置。三个信号量同步对缓冲区的访问：信号量 mutex 提供对缓冲区的互斥访问；信号量 slots 和 items 是计数信号量，分别对空槽位的数量和可用条目的数量计数。

```
1  typedef struct {
```

```
2      int *buf;               /* 缓冲区数组 */
3      int n;                  /* 槽位的最大数目 */
4      int front;              /* buf[(front+1)%n] 是第一个条目 */
5      int rear;               /* buf[rear%n] 是最后一个条目 */
6      sem_t mutex;            /* 保护对 buf 的访问 */
7      sem_t slots;            /* 对可用槽位计数 */
8      sem_t items;            /* 对可用条目计数 */
9  } sbuf_t;
```

图 12.24 sbuf_t：Sbuf 包使用的有界缓冲区。

图 12.25 展示了 Sbuf 包的实现。sbuf_init 函数为缓冲区分配堆存储，把 front 和 rear 设置为指示空缓冲区的值，并为三个信号量赋初值。这个函数只调用一次，要在其他三个函数被调用之前调用。sbuf_deinit 函数在应用程序用完缓冲区后释放缓冲区存储。sbuf_insert 函数先等待一个可用槽位，然后对互斥锁加锁，加入条目，解锁互斥锁，并宣告有新条目可用。sbuf_remove 函数是对称的：在等待到可用条目之后，它对互斥锁加锁，从缓冲区前端取出条目，解锁互斥锁，并宣告有新槽位可用。

练习 12.9 设 p 为生产者的数量，c 为消费者的数量，n 为以条目计的缓冲区大小。对下列每种情形，指出 sbuf_insert 和 sbuf_remove 中的 mutex 信号量是否有必要。

A. $p = 1, c = 1, n > 1$

B. $p = 1, c = 1, n = 1$

C. $p > 1, c > 1, n = 1$

读者-写者问题

读者-写者问题是互斥问题的一般化。一组并发线程正在访问某个共享对象，比如主存中的数据结构或磁盘上的数据库。一些线程只读这个对象，另一些则修改它。修改对象的线程称为写者（writer），只读它的线程称为读者（reader）。写者必须对该对象有独占的访问权，而读者可以与任意多的其他读者共享它。一般情况下，并发的读者和写者的数量都不受限制。

```
1  #include "csapp.h"
2  #include "sbuf.h"
3
4  /* 创建一个有 n 个槽位的空、有界、共享 FIFO 缓冲区 */
5  void sbuf_init(sbuf_t *sp, int n)
6  {
7      sp->buf = Calloc(n, sizeof(int));
8      sp->n = n;                       /* 缓冲区最多容纳 n 个条目 */
9      sp->front = sp->rear = 0;        /* 当且仅当 front == rear
 ↪  时缓冲区为空
       */
10     Sem_init(&sp->mutex, 0, 1);      /* 用于加锁的二元信号量 */
```

```
11      Sem_init(&sp->slots, 0, n);       /* 最初,buf 有 n 个空槽位 */
12      Sem_init(&sp->items, 0, 0);       /* 最初,buf 没有数据条目 */
13  }
14
15  /* 清理缓冲区 sp */
16  void sbuf_deinit(sbuf_t *sp)
17  {
18      Free(sp->buf);
19  }
20
21  /* 将条目插入共享缓冲区 sp 的尾部 */
22  void sbuf_insert(sbuf_t *sp, int item)
23  {
24      P(&sp->slots);                          /* 等待可用槽位 */
25      P(&sp->mutex);                          /* 锁定缓冲区 */
26      sp->buf[(++sp->rear)%(sp->n)] = item;   /* 插入条目 */
27      V(&sp->mutex);                          /* 解锁缓冲区 */
28      V(&sp->items);                          /* 宣告有条目可用 */
29  }
30
31  /* 从缓冲区 sp 取出并返回第一个条目 */
32  int sbuf_remove(sbuf_t *sp)
33  {
34      int item;
35      P(&sp->items);                          /* 等待可用条目 */
36      P(&sp->mutex);                          /* 锁定缓冲区 */
37      item = sp->buf[(++sp->front)%(sp->n)]; /* 取出条目 */
38      V(&sp->mutex);                          /* 解锁缓冲区 */
39      V(&sp->slots);                          /* 宣告有槽位可用 */
40      return item;
41  }
```

图 12.25 Sbuf:一个对有界缓冲区的并发访问进行同步的包。

读者-写者交互在真实系统中也很常见。例如在联机航空订票系统中,允许不限数量的客户并发查看座位分配情况,但正在订座的客户必须独占访问数据库。另一个例子是多线程缓存 Web 代理:不限数量的线程可以从共享页面缓存读取已有页面,但任何向缓存写入新页面的线程都必须独占访问。

读者-写者问题有若干变体,每个变体对应读者与写者之间的一种优先关系。第一个读者-写者问题偏向读者:除非一个写者已经获得使用该对象的许可,否则不得让任何读者等待。也就是说,不应仅仅因为有一个写者在等待就使读者等待。第二个读者-写者问题偏向写者:一旦写者准备就绪要写,就应让它的写操作尽快执行。与第一个问题不同,在一个写者之后到达的读者必须等待,即使这个写者自己也在等待。

图 12.26 展示了第一个读者-写者问题的一个解法。与许多同步问题的解法一样,这个解法很微妙,只是简单得容易让人掉以轻心。信号量 w 控制对访问共享对象的临界区的访问。信号量 `mutex` 保护对共享变量 `readcnt` 的访问,`readcnt` 对当前在临界区中的读者数量计数。写者每次进入临界区时都锁定

互斥锁 w,每次离开时解锁。这保证了任何时刻临界区中至多有一个写者。另一方面,只有第一个进入临界区的读者锁定 w,也只有最后一个离开临界区的读者解锁它。当其他读者在场时进出临界区的读者会忽略 w。这意味着只要有一个读者持有互斥锁 w,不限数量的读者就可以不受阻碍地进入临界区。

两个读者-写者问题的任何正确解法都可能导致饥饿(starvation):一个线程无限期地阻塞,无法取得进展。例如在图 12.26 的解法中,当读者源源不断地到来时,写者可能无限期地等待。

练习 12.10 图 12.26 中第一个读者-写者问题的解法赋予了读者优先,但这种优先是弱优先:离开自己临界区的写者有可能去重启一个等待的写者,而不是一个等待的读者。请描述一个场景,说明这种弱优先如何让一群写者饿死一个读者。

12.5.5 综合:基于预线程化的并发服务器

我们已经看到信号量如何用于访问共享变量以及对共享资源的访问进行调度。为了帮助你更清楚地理解这些思想,我们把它们应用到一个基于预线程化技术的并发服务器之上。

```
/* 全局变量 */
int readcnt;    /* 初始值为 0 */
sem_t mutex, w; /* 两者初始值均为 1 */

void reader(void)
{
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* 第一个进入 */
            P(&w);
        V(&mutex);

        /* 临界区 */
        /* 执行读操作 */

        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* 最后一个离开 */
            V(&w);
        V(&mutex);
    }
}
void writer(void)
{
    while (1) {
        P(&w);

        /* 临界区 */
        /* 执行写操作 */
```

```
        V(&w);
    }
}
```

图 12.26 第一个读者-写者问题的解法。偏向读者而非写者。

在图 12.14 的并发服务器中，我们为每个新客户创建一个新线程。这种做法的缺点是：每来一个新客户，都要付出创建新线程的不可忽视的代价。基于预线程化的服务器试图用图 12.27 所示的生产者-消费者模型来减少这种开销。服务器由一个主线程和一组工作线程组成。主线程反复接受客户的连接请求，并把

边栏 其他同步机制

我们展示的线程同步方法用的是信号量，主要因为它们简单、经典，而且有干净的语义模型。但你应该知道，还存在其他同步技术。例如，Java 线程用一种称为 Java 监视器（monitor）的机制 [48] 来同步，它对信号量的互斥与调度能力提供了更高层次的抽象；实际上，监视器可以用信号量实现。再举一例，Pthreads 接口在互斥锁和条件变量（condition variable）上定义了一组同步操作。Pthreads 互斥锁用于互斥；条件变量用于调度对共享资源的访问，比如生产者-消费者程序中的有界缓冲区。

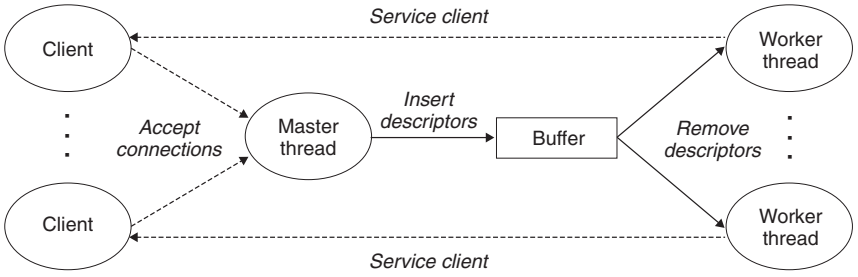


图 12.27 预线程化的并发服务器的组织结构。一组已存在的线程反复地从有界缓冲区中取出已连接描述符并加以处理。

由此得到的已连接描述符放入一个有界缓冲区。每个工作线程反复从缓冲区取出一个描述符，为客户服务，然后等待下一个描述符。

图 12.28 展示了如何用 Sbuf 包实现一个预线程化的并发 echo 服务器。初始化缓冲区 sbuf（第 24 行）之后，主线程创建工作线程集合（第 25 ~ 26 行），然后进入无限服务器循环，接受连接请求并把由此得到的已连接描述符插入 sbuf。每个工作线程的行为都非常简单：它等待直到能够从缓冲区取出一个已连接描述符（第 39 行），然后调用 echo_cnt 函数来回送客户的输入。

图 12.29 中的 echo_cnt 函数是图 11.22 的 echo 函数的一个版本，它在一个名为 byte_cnt 的全局变量中记录从所有客户接收的字节总数。这段代码值得

研究，因为它展示了一种通用技术，用于初始化那些会从线程例程中被调用的包。在我们这里，需要初始化 byte_cnt 计数器和 mutex 信号量。一种方法（Sbuf 包和 Rio 包用的就是它）是要求主线程显式调用一个初始化函数。另一种方法（这里所示的方法）使用 `pthread_once` 函数（第 19 行）来调用

```
 1  #include "csapp.h"
 2  #include "sbuf.h"
 3  #define NTHREADS 4
 4  #define SBUFSIZE 16
 5
 6  void echo_cnt(int connfd);
 7  void *thread(void *vargp);
 8
 9  sbuf_t sbuf; /* 已连接描述符的共享缓冲区 */
10
11  int main(int argc, char **argv)
12  {
13      int i, listenfd, connfd;
14      socklen_t clientlen;
15      struct sockaddr_storage clientaddr;
16      pthread_t tid;
17
18      if (argc != 2) {
19          fprintf(stderr, "usage: %s <port>\n", argv[0]);
20          exit(0);
21      }
22      listenfd = Open_listenfd(argv[1]);
23
24      sbuf_init(&sbuf, SBUFSIZE);
25      for (i = 0; i < NTHREADS; i++) /* 创建工作线程 */
26          Pthread_create(&tid, NULL, thread, NULL);
27
28      while (1) {
29          clientlen = sizeof(struct sockaddr_storage);
30          connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
31          sbuf_insert(&sbuf, connfd); /* 把 connfd 插入缓冲区 */
32      }
33  }
34
35  void *thread(void *vargp)
36  {
37      Pthread_detach(pthread_self());
38      while (1) {
39          int connfd = sbuf_remove(&sbuf); /* 从缓冲区取出 connfd */
40          echo_cnt(connfd);                /* 为客户服务 */
41          Close(connfd);
42      }
43  }
```

图 12.28 一个预线程化的并发 echo 服务器。服务器采用一方生产者、多方消费者的生产者-消费者模型。

```
1  #include "csapp.h"
```

```
 2
 3  static int byte_cnt;      /* 字节计数器 */
 4  static sem_t mutex;       /* 以及保护它的互斥锁 */
 5
 6  static void init_echo_cnt(void)
 7  {
 8      Sem_init(&mutex, 0, 1);
 9      byte_cnt = 0;
10  }
11
12  void echo_cnt(int connfd)
13  {
14      int n;
15      char buf[MAXLINE];
16      rio_t rio;
17      static pthread_once_t once = PTHREAD_ONCE_INIT;
18
19      Pthread_once(&once, init_echo_cnt);
20      Rio_readinitb(&rio, connfd);
21      while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
22          P(&mutex);
23          byte_cnt += n;
24          printf("server received %d (%d total) bytes on fd %d\n",
25                 n, byte_cnt, connfd);
26          V(&mutex);
27          Rio_writen(connfd, buf, n);
28      }
29  }
```

图 12.29 echo_cnt：统计从各客户接收的全部字节数的 echo 版本。

初始化函数，但只在第一次有线程调用 echo_cnt 函数的时候才真的调用。这种方法的优点是使包更易于使用；缺点是每次调用 echo_cnt 都要调用一次 `pthread_once`，而大多数时候这种调用什么有用的事都不做。

包初始化之后，echo_cnt 函数初始化 Rio 带缓冲的 I/O 包（第 20 行），然后回送从客户收到的每一行文本。注意，第 23 ~ 25 行对共享变量 byte_cnt 的访问由 P 和 V 操作保护。

边栏 基于线程的事件驱动程序

I/O 多路复用并不是编写事件驱动程序的唯一方式。例如，你可能已经注意到，我们刚开发的并发预线程化服务器实际上就是一个事件驱动的服务器，主线程和工作线程各带一个简单的状态机。主线程有两个状态（“等待连接请求”和“等待可用的缓冲区槽位”）、两个 I/O 事件（“连接请求到达”和“缓冲区槽位变为可用”）和两个转移（“接受连接请求”和“插入缓冲区条目”）。类似地，每个工作线程有一个状态（“等待可用的缓冲区条目”）、一个 I/O 事件（“缓冲区条目变为可用”）和一个转移（“取出缓冲区条目”）。

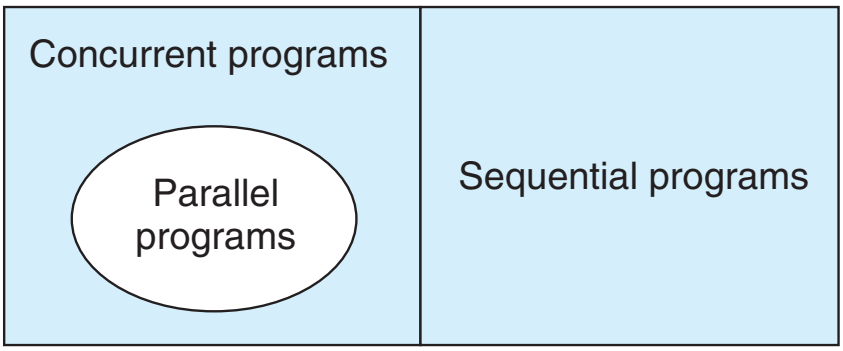


图 12.30 顺序、并发和并行程序集合之间的关系。

12.6 利用线程实现并行

到目前为止我们对并发的讨论，一直假设并发线程运行在单处理器系统上。然而，大多数现代机器都装有多核处理器。并发程序在这样的机器上往往运行得更快，因为操作系统内核会把并发线程并行地调度到多个核上，而不是在单个核上顺序执行。利用这种并行性，对繁忙的 Web 服务器、数据库服务器和大型科学计算程序等应用至关重要；在 Web 浏览器、电子表格和文字处理器等主流应用中，它也变得日益有用。

图 12.30 展示了顺序程序、并发程序和并行程序之间的集合关系。全部程序的集合可以划分为互不相交的顺序程序集合与并发程序集合。顺序程序写成单个逻辑流；并发程序写成多个并发流；并行程序是运行在多个处理器上的并发程序。因此，并行程序的集合是并发程序集合的真子集。

对并行程序的详细讨论超出了本书的范围，但研究几个简单的示例程序有助于理解并行程序设计的一些重要方面。例如，考虑怎样并行地对整数序列 $0, \ldots, n-1$ 求和。当然，对这个特定问题存在闭式解，但它仍是一个简洁易懂的范例，可以借此对并行程序提出一些有趣的论点。

把工作分配给不同线程的最直截了当的方法，是把序列划分成 t 个不相交的区域，然后把每个区域分配给 t 个不同的

线程。为简单起见，假设 n 是 t 的倍数，于是每个区域各有 n/t 个元素。下面看看多个线程并行处理各自区域的几种不同方式。

最简单、最直接的选项是让各线程把结果加到一个由互斥锁保护的全局共享变量中，图 12.31 展示了可能的实现。在第 28 ~ 33 行，主线程创建对等线程，然后等待它们终止。注意，主线程向每个对等线程传递一个小整数，作为该线程的唯一线程 ID。每个对等线程会用它的线程 ID 来确定自己该处理序列的哪一部分。向对等线程传递小的唯一线程 ID 这一思想是一种通用技术，在许多

并行应用程序中都会用到。对等线程终止之后，全局变量 gsum 中保存着最终的和。主线程再利用闭式解验证结果(第 36 ~ 37 行)。

图 12.32 展示了每个对等线程执行的函数。在第 4 行，线程从线程参数中提取线程 ID，然后用这个 ID 确定自己该处理序列的区域(第 5 ~ 6 行)。在第 9 ~ 13 行，线程在自己的那部分序列上迭代，每次迭代都更新共享的全局变量 gsum。注意，我们小心地用 P 和 V 操作配合 mutex 来保护每次更新。

当我们在一台 4 核系统上以 $n = 2^{31}$ 规模的序列运行 psum-mutex，并把运行时间(单位：秒)作为线程数的函数来测量时，得到了一个糟糕的意外结果：

	线程数				
版本	1	2	4	8	16
psum-mutex	68	432	719	552	599

这个程序以单线程顺序运行时已经极其慢，而以多线程并行运行时反而还要慢近一个数量级，并且加的核越多，性能越差。性能差的原因在于，同步操作(P 和 V)的代价相对于一次内存更新的代价来说非常昂贵。这凸显了并行程序设计的一条重要教训：同步开销很高，应尽量避免；实在无法避免，也要用尽可能多的有效计算来分摊这部分开销。

在示例程序中避免同步的一种办法，是让每个对等线程在一个不与任何其他线程共享的私有变量中计算自己的部分和，如图 12.33 所示。主线程(图中未画出)定义一个名为 psum 的全局数组，每个对等线程 i 把自己的部分和累积到 `psum[i]` 中。由于我们小心地让每个对等线程更新各不相同的内存位置，就没有必要用互斥锁保护这些更新。唯一需要的同步是主线程必须等待所有子线程结束。对等线程终止之后，主线程把 psum 向量的各元素加起来即得最终结果。

```
1  #include "csapp.h"
2  #define MAXTHREADS 32
3
4  void *sum_mutex(void *vargp); /* 线程例程 */
5
6  /* 全局共享变量 */
7  long gsum = 0;           /* 全局和 */
8  long nelems_per_thread; /* 要加总的元素个数 */
9  sem_t mutex;             /* 保护全局和的互斥锁 */
10
11 int main(int argc, char **argv)
12 {
13     long i, nelems, log_nelems, nthreads, myid[MAXTHREADS];
14     pthread_t tid[MAXTHREADS];
15
16     /* 获取输入参数 */
17     if (argc != 3) {
18         printf("Usage: %s <nthreads> <log_nelems>\n", argv[0]);
```

```
19          exit(0);
20      }
21      nthreads = atoi(argv[1]);
22      log_nelems = atoi(argv[2]);
23      nelems = (1L << log_nelems);
24      nelems_per_thread = nelems / nthreads;
25      sem_init(&mutex, 0, 1);
26
27      /* 创建对等线程并等待它们结束 */
28      for (i = 0; i < nthreads; i++) {
29          myid[i] = i;
30          Pthread_create(&tid[i], NULL, sum_mutex, &myid[i]);
31      }
32      for (i = 0; i < nthreads; i++)
33          Pthread_join(tid[i], NULL);
34
35      /* 检查最终答案 */
36      if (gsum != (nelems * (nelems-1))/2)
37          printf("Error: result=%ld\n", gsum);
38
39      exit(0);
40  }
```

图 12.31 psum-mutex 的主例程。用多个线程对序列元素求和，把结果累加到一个由互斥锁保护的全局共享变量中。

```
1  /* psum-mutex.c 的线程例程 */
2  void *sum_mutex(void *vargp)
3  {
4      long myid = *((long *)vargp);          /* 提取线程 ID */
5      long start = myid * nelems_per_thread; /* 起始元素下标 */
6      long end = start + nelems_per_thread; /* 结束元素下标 */
7      long i;
8
9      for (i = start; i < end; i++) {
10         P(&mutex);
11         gsum += i;
12         V(&mutex);
13     }
14     return NULL;
15 }
```

图 12.32 psum-mutex 的线程例程。每个对等线程把结果加到一个由互斥锁保护的全局共享变量中。

```
1  /* psum-array.c 的线程例程 */
2  void *sum_array(void *vargp)
3  {
4      long myid = *((long *)vargp);          /* 提取线程 ID */
5      long start = myid * nelems_per_thread; /* 起始元素下标 */
6      long end = start + nelems_per_thread; /* 结束元素下标 */
7      long i;
8
```

```
 9      for (i = start; i < end; i++) {
10          psum[myid] += i;
11      }
12      return NULL;
13  }
```

图 12.33 psum-array 的线程例程。每个对等线程把部分和累积到一个不与任何其他对等线程共享的私有数组元素中。

当我们在 4 核系统上运行 psum-array 时可以看到，它比 psum-mutex 快了几个数量级：

	线程数				
版本	1	2	4	8	16
psum-mutex	68.00	432.00	719.00	552.00	599.00
psum-array	7.26	3.64	1.91	1.85	1.84

在第 5 章，我们学习过如何用局部变量消除不必要的内存引用。图 12.34 展示了这一原则的一种应用：让每个对等线程把部分和累积到一个局部变量而不是全局变量中。当我们在 4 核机器上运行 psum-local 时，运行时间又下降了一个数量级：

	线程数				
版本	1	2	4	8	16
psum-mutex	68.00	432.00	719.00	552.00	599.00
psum-array	7.26	3.64	1.91	1.85	1.84
psum-local	1.06	0.54	0.28	0.29	0.30

```
 1  /* psum-local.c 的线程例程 */
 2  void *sum_local(void *vargp)
 3  {
 4      long myid = *((long *)vargp);          /* 提取线程 ID */
 5      long start = myid * nelems_per_thread; /* 起始元素下标 */
 6      long end = start + nelems_per_thread; /* 结束元素下标 */
 7      long i, sum = 0;
 8
 9      for (i = start; i < end; i++) {
10          sum += i;
11      }
12      psum[myid] = sum;
13      return NULL;
14  }
```

图 12.34 psum-local 的线程例程。每个对等线程把部分和累积到一个局部变量中。

从这个练习可以得到的一个重要教训是：编写并行程序是件棘手的事。对代码看似微小的改动，会给性能带来显著的影响。

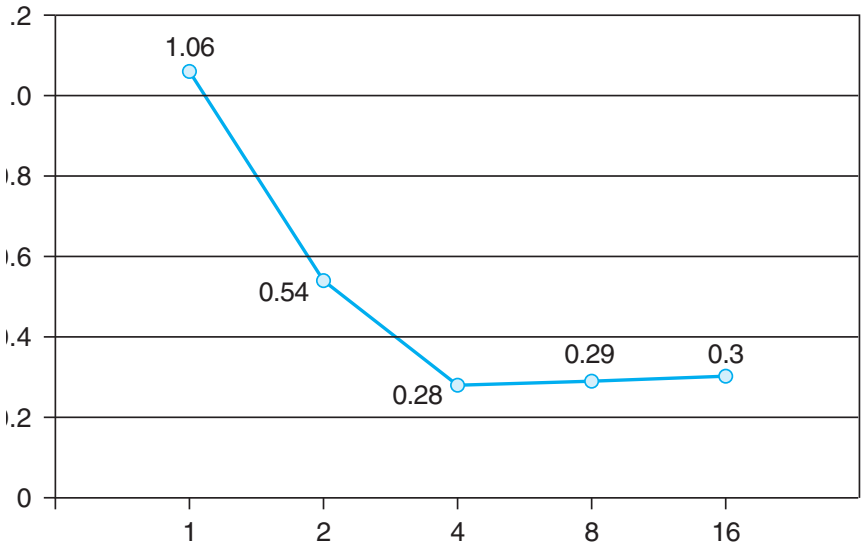


图 12.35 psum-local(图 12.34)的性能。使用四个处理器核,对一个包含 2^31 个元素的序列求和。

并行程序的性能刻画

图 12.35 给出了图 12.34 的 psum-local 程序总运行时间随线程数变化的曲线。各次运行都在具有 4 个处理器核心的系统上对一个 $n = 2^{31}$ 个元素的序列求和。可以看到,线程数一路增加到 4 为止,运行时间不断下降;此后曲线趋平,甚至开始略有上升。

在理想情况下,我们期望运行时间随核心数线性下降。也就是说,每把线程数翻倍,运行时间就减半。在四个核都忙于运行至少一个线程之前($t > 4$),情况确实如此。随着线程数继续增加,运行时间实际上会略有上升,原因是同一个核上多个线程之间上下文切换的开销。因此,并行程序常常写成每个核恰好运行一个线程的形式。

虽然绝对运行时间是衡量任何程序性能的最终标准,但仍有一些有用的相对度量,可以看出一个并行程序对潜在并行性的利用程度。并行程序的加速比(speedup)通常定义为

$$S_p = \frac{T_1}{T_p}$$

其中 p 是处理器核心数,T_k 是在 k 个核上的运行时间。这种表示法有时称为强扩展(strong scaling)。当 T_1 为

当 T_1 是程序的一个串行版本的执行时间时,S_p 称为**绝对加速比(absolute speedup)**;当 T_1 是并行程序在单核上运行时的执行时间时,S_p 就称为**相对加速比(relative speedup)**。绝对加速比比相对加速比更真实地度量了并行带来

图 12.36　图 12.35 中各执行时间的加速比与并行效率

线程(t)	1	2	4	8	16
核(p)	1	2	4	4	4
运行时间(T_p)	1.06	0.54	0.28	0.29	0.30
加速比(S_p)	1	1.9	3.8	3.7	3.5
效率(E_p)	100%	98%	95%	91%	88%

的收益。并行程序往往要付出同步开销,即使它们运行在单个处理器上也是如此,这些开销会增大分子的取值,从而人为地夸大相对加速比的数值。另一方面,测量绝对加速比又比测量相对加速比困难,因为测量绝对加速比需要两个不同的程序版本。对于复杂的并行程序,创建一个单独的串行版本可能行不通,或者是因为代码太复杂,或者是因为拿不到源代码。

另一个相关的度量称为**效率(efficiency)**,定义如下:

$$E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}$$

效率通常以百分比的形式报告,取值范围是 $(0, 100]$。效率度量的是由并行化带来的开销。与效率低的程序相比,效率高的程序把更多的时间花在做有用的工作上,而花在同步和通信上的时间更少。

图 12.36 给出了我们示例并行求和程序的各种加速比和效率度量。像这样超过 90% 的效率非常好,但可别被骗了。我们之所以能获得这样高的效率,是因为我们的问题毫不费力就能并行化。在实践中通常不是这样。几十年来,并行程序设计一直是一个活跃的研究领域。随着商用多核机器的核数每隔几年翻一番,并行程序设计仍然是一个深奥、困难而又活跃的研究领域。

对加速比还有一种解读,称为**弱可扩展性(weak scaling)**,即随着处理器数目的增加而增大问题规模问题规模,使得每个处理器上执行的工作量在处理器数目增加时保持不变。在这种定义下,加速比和效率用单位时间内完成的工作总量来表示。例如,如果能把处理器的数目翻一番,并且每小时完成的工作量也跟着翻一番,那我们就享受着线性加速比和 100% 的效率。

弱可扩展性通常比**强可扩展性(strong scaling)**更真实,因为它更准确地反映了我们想用更大的机器去做更多工作的愿望。对科学计算代码来说尤其如此:它们的规模很容易增大,而更大的问题规模直接意味着对自然更准确的预测。不过,也存在一些应用,它们的问题规模不那么容易增大,对这类应用来说,强可扩展性更合适。例如,实时信号处理应用要执行的工作量通常由产生信号的物理传感器的性质决定。改变总的工作量就意味着使用不同的物理传感器,

而这可能不可行，也没有必要。对这类应用，我们通常希望用并行性以尽可能快的速度完成固定数量的工作。

练习 12.11 （答案见本章练习答案）补全下表中这个并行程序所缺的表项。假设是强可扩展。

练习 12.11 的表

线程(t)	1	4	8
核(p)	1	4	8
运行时间(T_p)	16	8	4
加速比(S_p)			
效率(E_p)	100%		

12.7 其他并发问题

你可能已经注意到，一旦要在程序中同步对共享数据的访问，情况就变得复杂多了。到目前为止，我们研究过实现互斥以及生产者—消费者同步的技术，但这只是冰山一角。同步从本质上说就是一个难题，它会引出一些在普通串行程序中根本不会出现的问题。本节对编写并发程序时需要留意的一些问题做了概览（远不完整）。为了让讨论具体化，我们以线程为背景来展开讨论。不过请记住，对任何类型的并发流操纵共享资源的情况来说，这里讨论的都是其中的典型问题。

12.7.1 线程安全

用线程进行程序设计时，我们必须小心翼翼地去编写那些具有线程安全（thread safety）这一性质的函数。一个函数被称为是线程安全的（thread-safe），当且仅当它被多个并发线程反复调用时总能产生正确的结果。如果一个函数不是线程安全的，我们就说它是线程不安全的（thread-unsafe）。

我们可以识别出四类（并不相互排斥的）线程不安全的函数：

第 1 类：不保护共享变量的函数。我们已经遇到过这个问题：图 12.16 的线程函数对一个未加保护的全局计数器变量进行递增。把这一类线程不安全函数改造成线程安全的相对容易：用 P 和 V 这样的同步操作去保护共享变量就行。一个优点是它不需要对调用程序做任何修改；一个缺点是同步操作会拖慢这个函数。

```
code/conc/rand.c

1  unsigned next_seed = 1;
2
```

```
3  /* rand - 返回 0..32767 范围内的伪随机整数 */
4  unsigned rand(void)
5  {
6      next_seed = next_seed*1103515245 + 12543;
7      return (unsigned)(next_seed>>16) % 32768;
8  }
9
10  /* srand - 为 rand() 设置初始种子 */
11  void srand(unsigned new_seed)
12  {
13      next_seed = new_seed;
14  }
```

图 12.37 一个线程不安全的伪随机数生成器。(基于 [61])

第 2 类:跨多次调用保持状态的函数。伪随机数生成器就是这一类线程不安全函数的一个简单例子。考虑图 12.37 中的伪随机数生成器包。

rand 函数之所以线程不安全,是因为它当前这次调用的结果依赖于前一次调用的一个中间结果。在用 srand 设定初值之后,从单个线程反复地调用 rand,我们可以期待得到一个可重复的数字序列。但是,如果有多个线程在调用 rand,这个假设就不再成立了。

要把 rand 这样的函数改造成线程安全的,唯一的办法就是重写它,使它不使用任何静态数据,改为依靠调用者把状态信息作为参数传递进来。缺点是程序员现在不得不修改调用例程中的代码。在一个大型程序中,调用点可能多达数百处,做这样的修改相当费事,还容易出错。

第 3 类: 返回指向静态变量的指针的函数。有些函数, 例如 ctime 和 gethostbyname,把结果计算在一个静态变量里,然后返回指向这个变量的指针。如果我们从并发线程中调用这类函数,那么灾难就在眼前:一个线程正在使用的结果会被另一个线程悄悄地覆写掉。

code/conc/ctime-ts.c

```
1  char *ctime_ts(const time_t *timep, char *privatep)
2  {
3      char *sharedp;
4
5      P(&mutex);
6      sharedp = ctime(timep);
7      strcpy(privatep, sharedp); /* 把字符串从共享变量复制到私有变量 */
8      V(&mutex);
9      return privatep;
10  }
```

图 12.38 C 标准库函数 ctime 的线程安全包装函数。本例使用锁并复制技术来调用一个第 3 类线程不安全的函数。

对付这一类线程不安全函数有两种办法。一种选择是重写这个函数，让调用者把存放结果的那个变量的地址传进来。这样就能消除所有共享数据，但它要求程序员能够访问函数的源代码。

如果这个线程不安全函数很难修改甚至根本无法修改（例如代码非常复杂，或者没有源代码可用），那么另一个选择是使用锁并复制（lock-and-copy）技术。基本思想是给这个线程不安全函数关联一个互斥锁。在每个调用点，先锁住互斥锁，调用这个线程不安全函数，把函数返回的结果复制到一个私有的存储位置，然后再解锁互斥锁。为了尽量少改动调用方，你应该定义一个执行锁并复制操作的线程安全包装函数，然后用对这个包装函数的调用替换所有对线程不安全函数的调用。例如，图 12.38 展示了 `ctime` 的一个使用锁并复制技术的线程安全包装函数。

第 4 类：调用线程不安全函数的函数。如果函数 `f` 调用了一个线程不安全函数 `g`，那么 `f` 也线程不安全吗？要看情况。如果 `g` 是一个跨多次调用依赖状态的第 2 类函数，那么 `f` 也是线程不安全的，除了重写 `g` 之外别无他法。但是，如果 `g` 是第 1 类或第 3 类函数，那么只要用互斥锁保护好调用点和产生的所有共享数据，`f` 仍然可以是线程安全的。图 12.38 就是一个很好的例子：我们用锁并复制技术编写了一个调用了线程不安全函数的线程安全函数。

12.7.2 可重入性

有一类重要的线程安全函数，称为**可重入函数（reentrant）**，其特征是：它们被多个线程调用时不会引用任何共享数据。虽然线程安全与可重入这两个词有时被（不正确地）当作同义词使用，但两者之间存在着一个清晰的技术区别，值得保留。图 12.39 展示了可重入函数、线程安全函数和线程不安全函数这些集合之间的关系。全部函数的集合被划分成互不相交的线程安全函数集合与线程不安全函数集合，而可重入函数集合是线程安全函数集合的一个真子集。

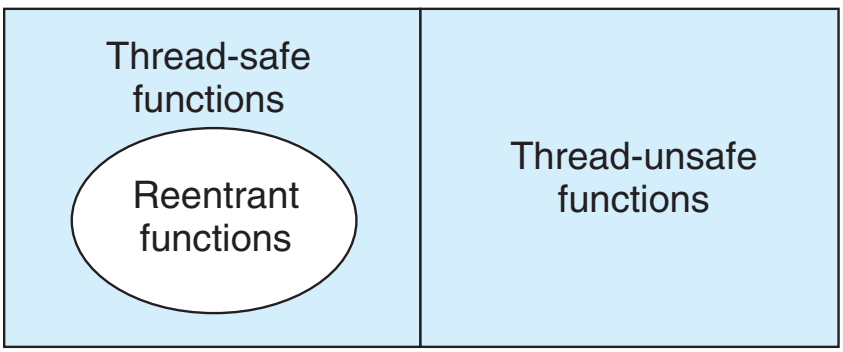


图 12.39 可重入函数、线程安全函数和非线程安全函数集合之间的关系。

可重入函数通常比不可重入的线程安全函数更高效,因为它们不需要任何同步操作。此外,把第 2 类线程不安全函数改造成线程安全函数的唯一办法,就是把它重写成可重入的。例如,图 12.40 展示了图 12.37 中 rand 函数的一个可重入版本。关键在于一点:我们用一个由调用者传入的指针替换了静态的 next 变量。

code/conc/rand-r.c

```
1  /* rand_r - 返回 0..32767 范围内的伪随机整数 */
2  int rand_r(unsigned int *nextp)
3  {
4      *nextp = *nextp * 1103515245 + 12345;
5      return (unsigned int)(*nextp / 65536) % 32768;
6  }
```

图 12.40 rand_r:图 12.37 中 rand 函数的可重入版本。

有没有可能检视一个函数的代码,然后先验地断言它是可重入的?遗憾的是,这要看情况。如果所有函数参数都是按值传递的(即没有指针),而且所有数据引用都指向局部自动栈变量(即不引用静态或全局变量),那么这个函数就是**显式可重入(explicitly reentrant)**的,意思是:无论它如何被调用,我们都可以断言它的可重入性。

但是,如果我们稍微放宽一点假设,允许这个本来显式可重入的函数的部分参数按引用传递(即允许它们传递指针),那么我们就得到一个**隐式可重入(implicitly reentrant)**函数,意思是:只有当调用线程小心地把指针指向非共享数据时,它才是可重入的。例如,图 12.40 中的 rand_r 函数就是隐式可重入的。

我们使用可重入这个词时,总是同时包括显式可重入和隐式可重入这两类函数。然而,很重要的一点是要认识到:可重入性有时既是调用者的属性,也是被调用者的属性,而不只是被调用者单独的属性。

练习 12.12 (答案见本章练习答案)解释为什么图 12.40 中的 rand_r 函数是隐式可重入的。

12.7.3 在线程化程序中使用已有的库函数

大多数 Linux 函数都是线程安全的,包括标准 C 库中定义的函数(例如 malloc、free、realloc、printf 和 scanf),只有少数例外。图 12.41 列出了一些常见的例外。(完整列表见 [110]。)strtok 是一个已废弃的函数(不建议使用),用于解析字符串。asctime、ctime 和 localtime 是一些常用的函数,用于在不同的时间和日期格式之间来回转换。gethostbyaddr、gethostbyname 和 inet_ntoa 是已经过时的网络编程函数,分别已被可重入的 getaddrinfo、getnameinfo 和 inet_ntop 函数取代(见第 11 章)。除了 rand 和 strtok 以

外,它们都属于返回一个指向静态变量的指针的第 3 类。如果在线程化程序中需要调用这些函数之一,那么对调用方改动最小的办法就是锁并复制。然而,锁并复制办法有一系列缺点。第一,额外的同步会拖慢程序。第二,有些函数返回的指针指向由结构组成的复杂结构,要把整个结构层次复制下来,就得对结构做深复制。第三,锁并复制办法对第 2 类线程不安全函数行不通,例如 `rand`,它依赖跨调用持续存在的静态状态。

图 12.41 常见的线程不安全的库函数

线程不安全的函数	不安全的类别	Linux 线程安全版本
`rand`	2	`rand_r`
`strtok`	2	`strtok_r`
`asctime`	3	`asctime_r`
`ctime`	3	`ctime_r`
`gethostbyaddr`	3	`gethostbyaddr_r`
`gethostbyname`	3	`gethostbyname_r`
`inet_ntoa`	3	(无)
`localtime`	3	`localtime_r`

因此,Linux 系统为大多数线程不安全的函数提供了可重入的版本。这些可重入版本的名字总是以 `_r` 后缀结尾。例如,`asctime` 的可重入版本称为 `asctime_r`。我们建议尽可能使用这些函数。

12.7.4 竞争

当一个程序的正确性依赖于一个线程在其控制流中到达点 x,而这个到达时刻又早于另一个线程到达其控制流中的点 y 时,程序就出现了**竞争**(**race**)。竞争的发生常常是因为程序员假设线程会沿着执行状态空间中的某条特定轨迹移动,却忘记了黄金法则:线程化程序必须对任何可行的轨迹都能正确运行。

理解竞争最容易的办法是看一个例子。考虑图 12.42 中的这个简单程序。主线程创建了四个对等线程,并把一个指向唯一整数 ID 的指针传给每个线程。每个对等线程把作为参数传入的 ID 复制到一个局部变量中(第 22 行),然后打印一条包含这个 ID 的消息。这个程序看上去再简单不过了,但是当我们在自己的系统上运行它时,却得到了下面这个错误的结果:

```
linux> ./race
Hello from thread 1
Hello from thread 3
Hello from thread 2
Hello from thread 3
```

```
code/conc/race.c
```

```
1  /* 警告:这段代码有缺陷!  */
2  #include "csapp.h"
3  #define N 4
4
5  void *thread(void *vargp);
6
7  int main()
8  {
9      pthread_t tid[N];
10      int i;
11
12      for (i = 0; i < N; i++)
13          Pthread_create(&tid[i], NULL, thread, &i);
14      for (i = 0; i < N; i++)
15          Pthread_join(tid[i], NULL);
16      exit(0);
17  }
18
19  /* 线程例程 */
20  void *thread(void *vargp)
21  {
22      int myid = *((int *)vargp);
23      printf("Hello from thread %d\n", myid);
24      return NULL;
25  }
```

图 12.42 一个带有竞争的程序。

这个问题是由每个对等线程与主线程之间的竞争引起的。你能找出这个竞争吗？情况是这样的:主线程在第 13 行创建一个对等线程时,传过去的是一个指向局部栈变量 i 的指针。于是,竞争就在第 12 行对 i 的下一次加一与第 22 行对参数的解引用和赋值之间展开。如果对等线程在第 22 行的执行先于主线程在第 12 行对 i 的加一,那么 `myid` 变量得到的就是正确的 ID。否则,它就会包含其他某个线程的 ID。可怕之处在于,我们能否得到正确的答案取决于内核如何调度线程的执行。在我们的系统上它出了错,但在别的系统上它可能正好正确运行,使程序员浑然不觉地带着一个严重的缺陷。

要消除这个竞争,我们可以为每个整数 ID 动态分配一个单独的块,并把指向这个块的指针传给线程例程,如图 12.43 所示(第 12—14 行)。请注意,线程例程必须释放这个块,以避免内存泄漏。

code/conc/norace.c

```
1  #include "csapp.h"
2  #define N 4
3
4  void *thread(void *vargp);
5
6  int main()
```

```
7  {
8      pthread_t tid[N];
9      int i, *ptr;
10
11      for (i = 0; i < N; i++) {
12          ptr = Malloc(sizeof(int));
13          *ptr = i;
14          Pthread_create(&tid[i], NULL, thread, ptr);
15      }
16      for (i = 0; i < N; i++)
17          Pthread_join(tid[i], NULL);
18      exit(0);
19  }
20
21  /* 线程例程 */
22  void *thread(void *vargp)
23  {
24      int myid = *((int *)vargp);
25      Free(vargp);
26      printf("Hello from thread %d\n", myid);
27      return NULL;
28  }
```

图 12.43 图 12.42 程序的一个没有竞争的正确版本。

当我们在自己的系统上运行这个程序时,现在得到的是正确的结果:

```
linux> ./norace
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
```

练习 12.13 （答案见本章练习答案）在图 12.43 中,我们可能很想在主线程的第 14 行之后立即释放已分配的内存块,而不是在对等线程中释放。但这是个坏主意。为什么?

练习 12.14 （答案见本章练习答案）

A. 在图 12.43 中,我们是通过为每个整数 ID 分配一个单独的块来消除竞争的。概述一种不需要调用 `malloc` 或 `free` 函数的其他做法。

B. 这种做法有哪些优点和缺点?

12.7.5 死锁

信号量会引发一种棘手的运行时错误,这种错误称为**死锁**(**deadlock**),它指的是一组线程被阻塞、等待一个永不成立的条件。进度图是理解死锁的宝贵工具。例如,图 12.44 展示了使用两个信号量实现互斥的一对线程的进度图。从这幅图我们可以提炼出关于死锁的一些重要结论:

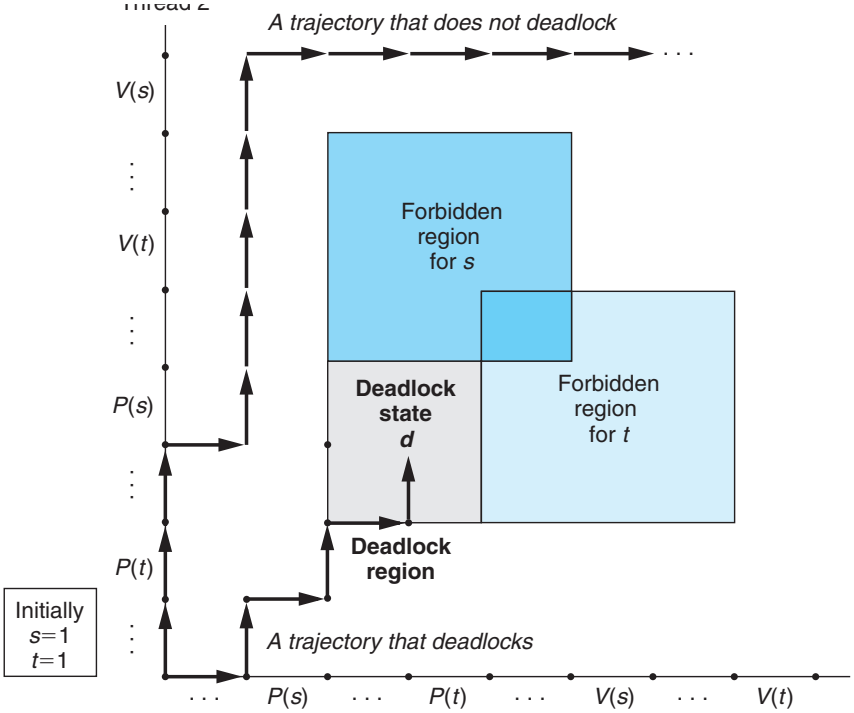


图 12.44 一个可能死锁的程序的进度图。

- 程序员把 P 和 V 操作排错了顺序，使得两个信号量的禁止区域发生了重叠。如果某条执行轨迹碰巧到达了死锁状态 d，就不可能再取得任何进展，因为相互重叠的禁止区域阻挡了每个合法方向上的推进。换句话说，此时程序死锁了，因为每个线程都在等待另一个线程去做一个永远不会发生的 V 操作。
- 相互重叠的禁止区域引出了一组称为**死锁区域（deadlock region）**的状态。如果一条执行轨迹碰巧触及了死锁区域中的某个状态，那么死锁就不可避免。执行轨迹可以进入死锁区域，但永远无法离开。
- 死锁之所以特别棘手，是因为它并不总是可以预测的。有些幸运的执行轨迹会绕开死锁区域，而另一些则会被死锁区域困住。图 12.44 为这两种情况各给出一例。对程序员来说，其后果相当可怕。同一个程序可能运行一千次都没有问题，而下一次运行时就死锁了。或者程序在一台机器上运行正常，到另一台机器上就死锁了。最糟糕的是，这个错误通常无法重现，因为不同的执行有不同的轨迹。

程序会死锁的原因有很多，一般来说，防止死锁是一个困难的问题。但是，当用二进制信号量来实现互斥时（如图 12.44 那样），你可以用下面这条简单而有效的规则来预防死锁：

互斥锁加锁顺序规则 给定所有互斥锁的一个全序，如果每个线程都按照这个顺序获取它的互斥锁，并按照相反的顺序释放它们，那么这个程序就是无死锁的。

例如，在每个线程中先锁 s、再锁 t，就能排除图 12.44 中的死锁隐患。图 12.45 展示了由此得到的进度图。

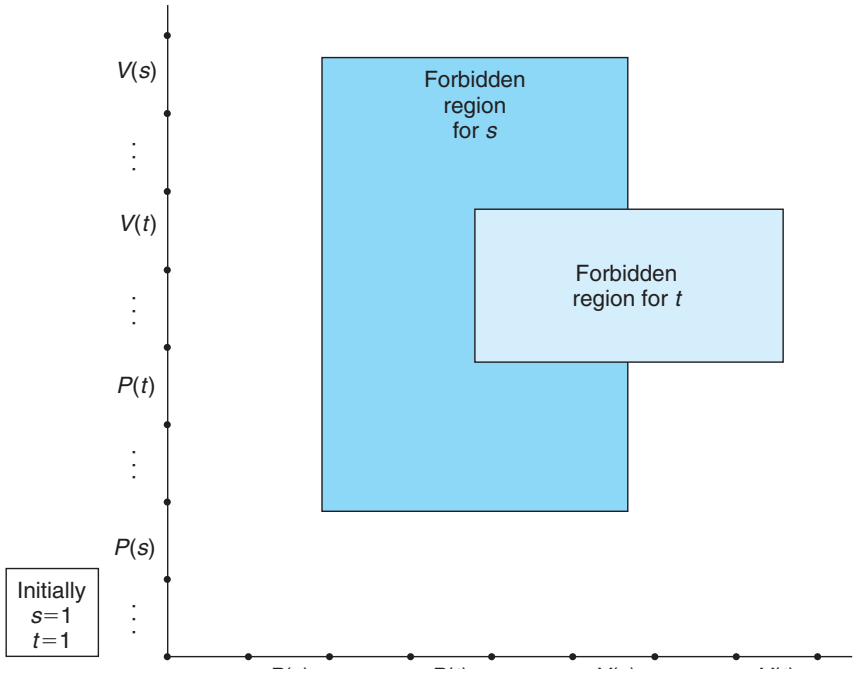


图 12.45 一个无死锁程序的进度图。

练习 12.15 （答案见本章练习答案）考虑下面这个试图用一对信号量来实现互斥的程序。

```
初始时:s = 1, t = 0。

线程 1:              线程 2:
   P(s);                P(s);
   V(s);                V(s);
   P(t);                P(t);
   V(t);                V(t);
```

A. 画出这个程序的进度图。

B. 它一定会死锁吗？

C. 如果会，对信号量的初始值做怎样的简单修改就能消除死锁的隐患？

D. 画出修正后无死锁程序的进度图。

12.8 小结

一个并发程序是由在时间上相互重叠的一组逻辑流组成的集合。在本章中，我们学习了构造并发程序的三种不同机制：进程、I/O 多路复用和线程。我们以一个并发网络服务器作为贯穿全章的示例应用。

进程由内核自动调度，而且由于每个进程都拥有相互独立的虚拟地址空间，所以要想共享数据，进程就必须使用显式的 IPC 机制。事件驱动程序创建自己的并发逻辑流，把这些流建模为状态机，并用 I/O 多路复用显式地调度这些流。因为程序运行在单个进程之内，所以在流之间共享数据快速而方便。线程是这两种方法的混合体：像基于进程的流一样，线程由内核自动调度；像基于 I/O 多路复用的流一样，线程在单个进程的上下文中运行，因此也能快速方便地共享数据。

无论采用哪种并发机制，对共享数据的并发访问进行同步都是一个困难的问题。信号量上的 P 和 V 操作就是为了帮助解决这个问题而发展起来的。信号量操作既能提供对共享数据的互斥访问，也能用来调度对资源的访问，例如生产者—消费者系统中的有界缓冲区和读—写者系统中的共享对象。一个并发预线程化的回显服务器为信号量的这些使用场景提供了一个很有说服力的例子。

并发还引入了其他一些困难的问题。由线程调用的函数必须具有称为线程安全的性质。我们识别出了四类线程不安全的函数，并给出了把它们改造为线程安全的建议。可重入函数是线程安全函数的一个真子集，它们不访问任何共享数据。可重入函数往往比不可重入的函数更高效，因为它们不需要任何同步原语。并发程序中还会出现其他一些困难的问题：竞争和死锁。当程序员对逻辑流的调度方式做出不正确假设时，就会发生竞争。当一个流在等待一个永远不会发生的事件时，就会发生死锁。

12.9 参考文献说明

信号量操作由 Dijkstra [31] 提出。进度图概念由 Coffman [23] 提出，后来由 Carson 和 Reynolds [16] 加以形式化。读—写者问题由 Courtois 等人 [25] 提出。操作系统教材更详细地描述了诸如哲学家就餐、睡眠理发师和抽烟者问题之类的经典同步问题 [102, 106, 113]。Butenhof [15] 的书全面描述了 Posix 线程接口。Birrell [7] 的论文是线程程序设计及其陷阱的很好的入门读物。Reinders [90] 的书描述了一个能简化线程程序设计与实现的 C/C++ 库。还有几本教材讨论了多核系统上并行程序设计的基本原理 [47, 71]。Pugh 指出了 Java 线程通

过内存相互作用方式中的不足,并提出了一些替代的内存模型 [88]。Gustafson 提出了弱扩展加速比模型 [43],将其作为强扩展的替代。

12.10 家庭作业

家庭作业 12.16 ▲ 编写 hello.c(图 12.13)的一个版本,创建并回收 n 个可连接(joinable)的对等线程,其中 n 是一个命令行参数。

家庭作业 12.17 ▲

A. 图 12.46 中的程序有一个缺陷。线程应当先睡眠 1 秒,然后打印一个字符串。可是,当我们在自己的系统上运行它时,却什么也没有打印出来。为什么?

B. 用两个不同的 Pthreads 函数调用之一来替换第 10 行的 `exit` 函数,就能修复这个缺陷。是哪两个?

code/conc/hellobug.c

```
1  /* 警告:这段代码有缺陷!  */
2  #include "csapp.h"
3  void *thread(void *vargp);
4
5  int main()
6  {
7      pthread_t tid;
8
9      Pthread_create(&tid, NULL, thread, NULL);
10      exit(0);
11 }
12
13 /* 线程例程 */
14 void *thread(void *vargp)
15 {
16     Sleep(1);
17     printf("Hello, world!\n");
18     return NULL;
19 }
```

图 12.46 家庭作业 12.17 用的有缺陷的程序。

家庭作业 12.18 ▲ 使用图 12.21 中的进度图,把下面的执行轨迹分别归类为安全的或不安全的。

A. H_2、L_2、U_2、H_1、L_1、S_2、U_1、S_1、T_1、T_2

B. H_2、H_1、L_1、U_1、S_1、L_2、T_1、U_2、S_2、T_2

C. H_1、L_1、H_2、L_2、U_2、S_2、U_1、S_1、T_1、T_2

家庭作业 12.19 ▲▲ 图 12.26 中第一个读—写者问题的解给读者的优先级有些弱，因为一个写者离开它的临界区时，可能会重新启动一个在等待的写者，而不是一个在等待的读者。试推导出一个给读者更强优先级的解：一个写者离开它的临界区时，只要存在等待中的读者，就一定会重新启动其中一个。

家庭作业 12.20 ▲▲▲ 考虑读—写者问题的一个更简单的变体，其中读者的数目最多为 N。试推导出一个使读者和写者享有同等优先级的解，其含义是：等待中的读者和写者被准许访问资源的机会相同。提示：你可以只用一个计数信号量和一个互斥锁解开这道题。

家庭作业 12.21 ▲▲▲▲ 试推导出第二个读—写者问题的解，这个问题偏向写者而不是读者。

家庭作业 12.22 ▲▲ 修改图 12.6 中的服务器，使服务器主循环的每次迭代至多只回显一行文本，以此来测试你对 `select` 函数的理解。

家庭作业 12.23 ▲▲ 图 12.8 中的事件驱动并发回显服务器存在缺陷，因为一个恶意客户端可以通过发送不完整的文本行，拒绝向其他客户端提供服务。试写出该服务器的一个改进版，使它能够处理这些不完整的文本行而不会阻塞。

家庭作业 12.24 ▲ Rio I/O 包（10.5 节）中的函数是线程安全的。它们同时也是可重入的吗？

家庭作业 12.25 ▲ 在图 12.28 的预线程化并发回显服务器中，每个线程都调用 `echo_cnt` 函数（图 12.29）。`echo_cnt` 是线程安全的吗？它是可重入的吗？为什么是或为什么不是？

家庭作业 12.26 ▲▲▲ 使用锁并复制技术，实现 `gethostbyname` 的一个线程安全的、不可重入的版本，称为 `gethostbyname_ts`。一个正确的解应当使用一个受互斥锁保护的 `hostent` 结构的深复制。

家庭作业 12.27 ▲▲ 一些网络编程教材建议了下面这种读写套接字的方法：在与客户端交互之前，在同一个已连接套接字描述符上打开两个标准 I/O 流，一个用于读，一个用于写：

```
FILE *fpin, *fpout;

fpin = fdopen(sockfd, "r");
fpout = fdopen(sockfd, "w");
```

在服务器结束与客户端的交互之后，按如下方式关闭这两个流：

```
fclose(fpin);
fclose(fpout);
```

然而，如果你在一个基于线程的并发服务器中试着用这一方法，就会引发一个致命的竞争条件。请解释原因。

家庭作业 12.28 ▲ 在图 12.45 中，交换两个 V 操作的顺序对程序是否会死锁有影响吗？画出四种可能情形的进度图来论证你的答案：

```
情形 1                          情形 2                          情形 3
        情形 4
线程 1        线程 2            线程 1        线程 2            线程 1
↪  线程 2
        线程 1        线程 2
 P(s)           P(s)            P(s)           P(s)             P(s)
        P(s)            P(s)           P(s)
 P(t)           P(t)            P(t)           P(t)             P(t)
        P(t)            P(t)           P(t)
 V(s)           V(s)            V(s)           V(t)             V(t)
        V(s)            V(t)           V(t)
 V(t)           V(t)            V(t)           V(s)             V(s)
        V(t)            V(s)           V(s)
```

家庭作业 12.29 ▲ 下面的程序会死锁吗？为什么？

```
初始时:a = 1, b = 1, c = 1。

线程 1:                线程 2:
   P(a);                  P(c);
   P(b);                  P(b);
   V(b);                  V(b);
   P(c);                  V(c);
   V(c);
   V(a);
```

家庭作业 12.30 ▲ 考虑下面这个会死锁的程序。

```
初始时:a = 1, b = 1, c = 1。

线程 1:           线程 2:           线程 3:
   P(a);             P(c);             P(c);
   P(b);             P(b);             V(c);
   V(b);             V(b);             P(b);
   P(c);             V(c);             P(a);
   V(c);             P(a);             V(a);
   V(a);             V(a);             V(b);
```

A. 针对每个线程，列出它同时持有的互斥锁对。

B. 如果 $a < b < c$，哪些线程违反了互斥锁加锁顺序规则？

C. 对这些线程，给出一个保证不会死锁的新加锁顺序。

家庭作业 12.31 ▲▲▲ 实现标准 I/O `fgets` 函数的一个版本，称为 `tfgets`，如果在 5 秒内没有从标准输入接收到一行输入，它就超时并返回 `NULL`。你的函数应当在一个名为 `tfgets-proc.c` 的包中实现，使用进程、信号和非本地跳转。不要使用 Linux 的 `alarm` 函数。使用图 12.47 中的驱动程序测试你的解。

```
code/conc/tfgets-main.c

1  #include "csapp.h"
2
3  char *tfgets(char *s, int size, FILE *stream);
4
5  int main()
6  {
7      char buf[MAXLINE];
8
9      if (tfgets(buf, MAXLINE, stdin) == NULL)
10         printf("BOOM!\n");
11     else
12         printf("%s", buf);
13
14     exit(0);
15 }
```

图 12.47 家庭作业 12.31—12.33 的驱动程序。

家庭作业 12.32 ▲▲▲ 实现家庭作业 12.31 中 `tfgets` 函数的一个使用 `select` 函数的版本。你的函数应当在一个名为 `tfgets-select.c` 的包中实现。使用家庭作业 12.31 的驱动程序测试你的解。你可以假设标准输入被分配到描述符 0。

家庭作业 12.33 ▲▲▲ 实现家庭作业 12.31 中 `tfgets` 函数的线程版本。你的函数应当在一个名为 `tfgets-thread.c` 的包中实现。使用家庭作业 12.31 的驱动程序测试你的解。

家庭作业 12.34 ▲▲▲ 编写 $N \times M$ 矩阵乘法内核的一个并行线程版本。将其性能同串行情形做比较。

家庭作业 12.35 ▲▲▲ 实现基于进程的 Tiny Web 服务器的一个并发版本。你的解应当为每个新的连接请求创建一个新的子进程。使用真实的 Web 浏览器测试你的解。

家庭作业 12.36 ▲▲▲ 实现基于 I/O 多路复用的 Tiny Web 服务器的一个并发版本。使用真实的 Web 浏览器测试你的解。

家庭作业 12.37 ▲▲▲ 实现基于线程的 Tiny Web 服务器的一个并发版本。你的解应当为每个新的连接请求创建一个新的线程。使用真实的 Web 浏览器测试你的解。

家庭作业 12.38 ▲▲▲▲ 实现 Tiny Web 服务器的一个并发预线程化版本。你的解应当根据当前负载动态地增大或减小线程的数量。一种策略是：当缓冲区变满时把线程数加倍，当缓冲区变空时把线程数减半。使用真实的 Web 浏览器测试你的解。

家庭作业 12.39 ▲▲▲▲ Web 代理(proxy)是一种充当 Web 服务器与浏览器之间中间人的程序。浏览器不直接联系服务器来获取网页,而是联系代理,代理再把请求转发给服务器。当服务器回应代理时,代理把回答发送给浏览器。在本实验中,你将编写一个对请求进行过滤并记录日志的简单 Web 代理。

A. 在实验的第一部分,你将把代理架设为接受请求、解析 HTTP、把请求转发给服务器并把结果返回给浏览器。你的代理应当把所有请求的 URL 记录到磁盘上的一个日志文件中,并且还应当拦截对磁盘上的过滤文件中列出的任何 URL 的请求。

B. 在实验的第二部分,你将通过为每个请求生成一个单独的线程来升级你的代理,使它得以同时处理多个打开的连接。当代理正在等待某个远程服务器对一个请求的应答以便服务于某个浏览器时,它应当继续处理另一个浏览器的未决请求。

使用真实的 Web 浏览器检验你的代理解答。

12.11 练习答案

答案 12.1 当并发服务器上的父进程开始执行时,与其关联的文件表的引用计数器从 0 增加到 1。当父进程 fork 出子进程时,引用计数器从 1 增加到 2。当父进程关闭它那份描述符副本时,引用计数器从 2 减小到 1。类似地,当子进程一端的连接关闭时,引用计数器从 1 减小到 0。

答案 12.2 无论出于任何原因,当一个进程终止时,内核都会关闭它所有打开的描述符。因此,父进程那份已连接文件描述符的副本会随父进程退出而自动地关闭。

答案 12.3 回想一下,图 11.22 的 `echo` 函数会回显来自客户端的每一行,直到客户端失去它的连接端。如果在 `echo` 函数执行期间按下 Ctrl+D,那么服务器就会把 Ctrl+D 当作 EOF,因而可能认为客户端已经关闭了它的连接端,从而可能停止向客户端回显。

答案 12.4 `pool.nready` 是一个整型变量。我们用 `select` 调用所报告的值来重新初始化 `pool.nready` 变量,以便保存 `select` 所返回的准备好描述符的数目。

答案 12.5 是的,如果从图 12.14 中删除第 31 行或第 32 行,就可能出现内存泄漏。因为这些线程不会被显式地回收,所以每个线程都必须分离,以便其内存资源在线程终止时能被回收。同样,释放由主线程分配的内存块也很重要。

答案 12.6 这里的关键思想是：栈变量是私有的，而全局变量和静态变量是共享的。像 cnt 这样的静态变量有点特别，因为它的共享范围仅限于其作用域之内的函数——在这个例子中就是线程例程。

A. 如下就是这张表：

答案 12.6 的表

变量	主线程?	对等线程 0?	对等线程 1?
ptr	是	是	是
cnt	否	是	是
i.m	是	否	否
msgs.m	是	是	是
myid.p0	否	是	否
myid.p1	否	否	是

- ptr 一个全局变量，由主线程写，由对等线程读。
- cnt 一个在内存中只有一个实例的静态变量，由两个对等线程读写。
- i.m 一个存放在主线程栈上的局部自动变量。尽管它的值被传给了对等线程，但对等线程从不引用栈上的这个变量，因此它不是共享的。
- msgs.m 存放在主线程栈上的一个局部自动变量，两个对等线程都通过 ptr 间接地引用它。
- myid.p0 和 myid.p1 一个局部自动变量的两个实例，分别驻留在对等线程 0 和对等线程 1 的栈上。

B. 变量 ptr、cnt 和 msgs 被不止一个线程引用，因此是共享的。

答案 12.7 这道题的关键是：你无法对内核调度线程时所选择的执行顺序做出任何假设。

答案 12.7 的表

步骤	线程	指令	%rdx$_1$	%rdx$_2$	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	2	H_2	—	—	0
4	2	L_2	—	0	0
5	2	U_2	—	1	0
6	2	S_2	—	1	1
7	1	U_1	1	—	1
8	1	S_1	1	—	1
9	1	T_1	1	—	1
10	2	T_2	—	1	1

变量 cnt 的最终错误值是 1。

答案 12.8 这道题是对你理解进度图中的安全轨迹和不安全轨迹的一次简单检验。像 A 和 C 这样从临界区旁边掠过的轨迹是安全的，将产生正确的结果。

A. H_1、L_1、U_1、S_1、H_2、L_2、U_2、S_2、T_2、T_1：安全

B. H_2、L_2、H_1、L_1、U_1、S_1、T_1、U_2、S_2、T_2：不安全

C. H_1、H_2、L_2、U_2、S_2、L_1、U_1、S_1、T_1、T_2：安全

答案 12.9

A. $p = 1, c = 1, n > 1$：是。互斥(mutex)信号量是必需的，因为生产者和消费者可以并发地访问缓冲区。

B. $p = 1, c = 1, n = 1$：不。在这种情况下 mutex 信号量并非必需，因为非空缓冲区等价于满缓冲区。当缓冲区中有一个数据项时，生产者被阻塞；当缓冲区为空时，消费者被阻塞。所以在任何时刻，都只有一个线程能够访问缓冲区，因此互斥不用 mutex 也能得到保证。

C. $p > 1, c > 1, n = 1$：不。依据与前一小问相同，在这种情况下同样不需要 mutex 信号量。

答案 12.10 假设有一个特定的信号量实现为每个信号量使用一个由线程组成的 LIFO 栈。当一个线程在 P 操作中阻塞在信号量上时，它的 ID 就被压入栈中。类似地，V 操作会从栈中弹出顶部的线程 ID，并重启这个线程。在这种栈实现下，一个处在临界区中的对抗性写者只需等到另一个写者也在信号量上阻塞之后，才释放信号量。在这种情形中，当两个写者你来我往地交替传递控制时，一个在等待的读者可能永远等待下去。

请注意，虽然用 FIFO 队列而不是 LIFO 栈似乎更符合直觉，但是使用这样的栈并没有错，也不违反 P 和 V 操作的语义。

答案 12.11 这道题是对你理解加速比和并行效率的一次简单的完整性检查：

答案 12.11 的表

线程(t)	1	4	8
核(p)	1	4	8
运行时间(T_p)	16	8	4
加速比(S_p)	1	2	4
效率(E_p)	100%	50%	25%

答案 12.12 `rand_r` 函数是一个隐式可重入函数,因为它是按引用传递参数的,也就是说,按引用传递参数 `*nextp`,而不是按值传递。显式可重入函数只按值传递参数,它们所有的数据引用都指向局部自动栈变量。

答案 12.13 如果我们在第 14 行调用 `pthread_create` 之后立即释放这个块,那么就会引入一个新的竞争,这一次竞争发生在主线程对 `free` 的调用和线程例程中第 24 行的赋值语句之间。

答案 12.14

A. 另一种方法是不传递指向 i 的指针,而是直接地传递整数 i:

```
for (i = 0; i < N; i++)
    Pthread_create(&tid[i], NULL, thread, (void *)i);
```

在线程例程中,我们把参数转换回 `int`,并将结果赋给 `myid`:

```
int myid = (int) vargp;
```

B. 这样做的优点是省去了对 `malloc` 和 `free` 的调用,因而减少了开销。一个显著的缺点是它假设指针至少与 `int` 一样大。虽然对于所有现代系统来说这个假设都成立,但对于传统的或未来的系统,它可能就不成立了。

答案 12.15

A. 图 12.48 给出了原程序的进度图。

B. 这个程序一定会死锁,因为任何可行的轨迹最终都会被死锁状态困住。

C. 要消除死锁隐患,应当把二进制信号量 t 初始化为 1 而不是 0。

D. 图 12.49 给出了修正后的程序的进度图。

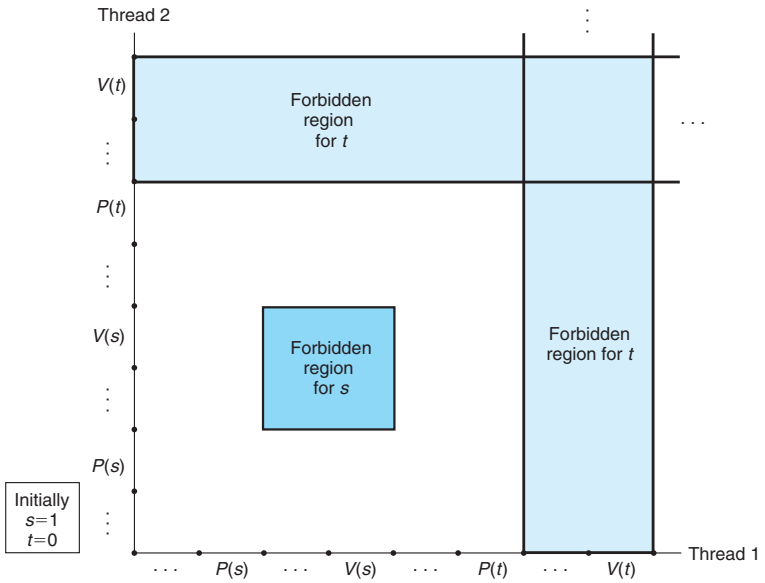


Figure 12.48 **Progress graph for a program that deadlocks.**

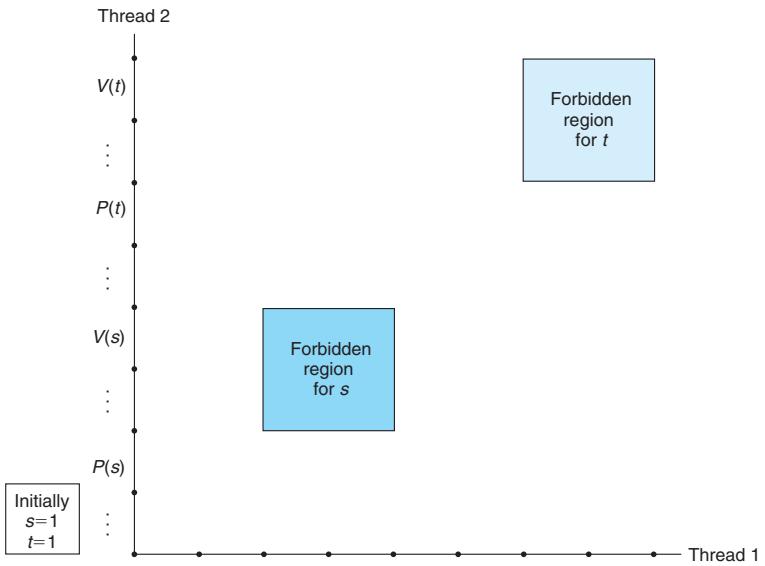


图 12.48 一个会死锁的程序的进度图。

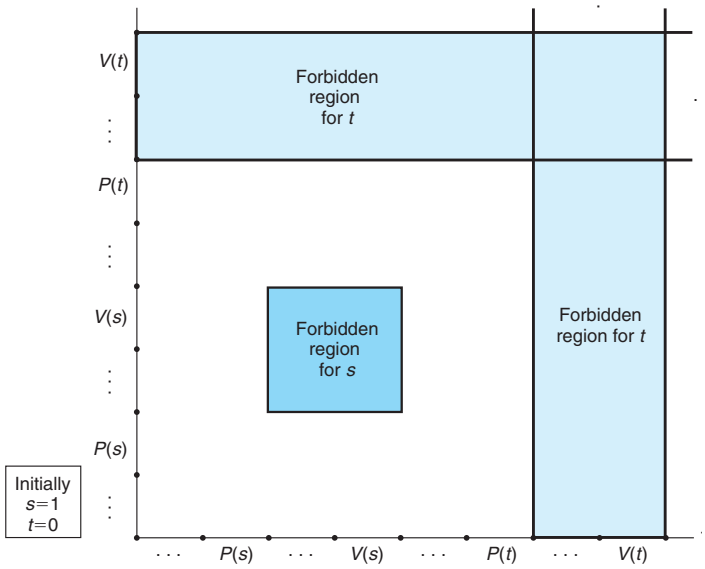


Figure 12.48 **Progress graph for a program that deadlocks.**

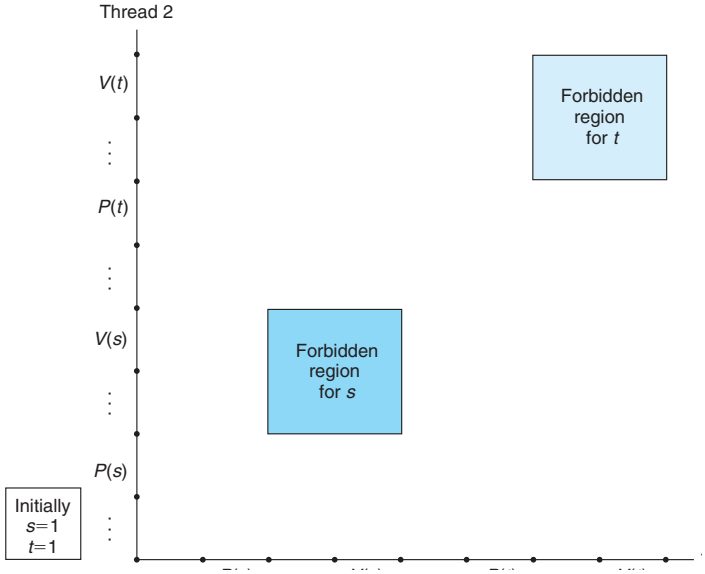


图 12.49 修正后的无死锁程序的进度图。